made with doc-o-matic

FireDAC

(c) Embarcadero Technologies 2004-2013

Embarcadero Technologies, (c) Embarcadero Technologies 2004-2013

Table of Contents

FireDAC 1

Overview 2

Getting Started 2

First Steps to use FireDAC 2

Setting up Connections 4

Very High Performance using the Array DML 8

Creating Reports with FastReport 10

Demo Databases 14

Demo Applications 16

Architecture 17

General 17

Components 20

Databases 24

Programming Tools 26

Class Hierarchy 27

Working with Connections 27

Defining Connection 27

Configuring Drivers 31

Setting Options 34

Data Type Mapping 35

Establishing Connection 37

Recovering Connection 38

Offlining Connection 40

Managing Transactions 41

Handling Errors 44

Multi Threading 46

DLL Development 49

Unicode Support 51

Working with Commands 56

Preprocessing Command Text 56

Character macro functions 60

Numeric macro functions 61

Date and time macro functions 62

System macro functions 64

Convert macro function 65

RETURNING unified support 66

Executing Command 66

Executing Stored Procedure 71
Browsing Table 73
Database Alerts 75
Fetching Rows 78
Command Batches 81
Array DML 82
Asynchronous Execution 86
Executing SQL Scripts 88
SQL Script Control Commands 92
Developing Custom Commands 95
Local SQL 96

Working with DataSets 100
Sorting Records 100
Filtering Records 102
Master-Detail Relationship 104
Finding a Record 107
Calculated and Aggregated Fields 108
Writing Expressions 110

Editing the Data 113
Changing DataSet Data 113
Caching Updates 114
Unique Identifying Fields 117
Auto-Incremental Fields 118
Update Command Generation 121
Overriding Posting Updates 123
Specifying Default Values 126

Working with Metadata 127
Querying Metadata 127
Extended Metadata 129
Metadata Structure 130
Object Names 133

Working with DBMS 135
Using SQLite with FireDAC 135
Using Oracle with FireDAC 149
Using Data Abstract with FireDAC 153

x Platform Development 158
Installing FireDAC 159
Using FireDAC 159
Known Limitations 160
UnixODBC 160

Lazarus / FPC 160
Installing on Windows 161
Installing on Linux 162
Path Variables 163

Migrating BDE applications 163
BDE name counterparts 164
BDE aliases migration 165
BDE application migration 165
Additional migration hints 168

Debugging 170
DBMS Environment Reports 170
Tracing and Monitoring 174

Deploying 177
Delpoying on Windows 177
Delpoying on Mac OS X or Linux 178

Utilities 178
ADAdministrator 178
ADDFMChanger 178
ADExecutor 180
ADExplorer 181
ADMonitor 185
Compile (tool name).bat 186
Create (DB name).bat 188

Database Connectivity 188
Common connection parameters 188
Connect to Advantage Database Server 189
Connect to Blackfish SQL Server 192
Connect to DataSnap server 193
Connect to dbExpress data source 194
Connect to IBM DB2 Server 196
Connect to Interbase or Firebird 198
Connect to Microsoft SQL Server 202
Connect to Microsoft SQL Server Compact Edition 206
Connect to Microsoft Access database 207
Connect to MySQL Server 209
Connect to ODBC data source 213
Connect to Oracle Server 215
Connect to PostgreSQL 218
Connect to SQLite database 221
Connect to Sybase SQL Anywhere 224

FAQ **227**
Installation 227
General Questions 229
TADManager and TADConnection Questions 230
TADQuery, TADStoredProc and TADUpdateSQL Questions 231
TADTable Questions 234
TADMemTable Questions 234
Fetching and Populating Questions 236
Sorting, Searching, Locating, Filtering Questions 237
Editing Questions 239
GUI Questions 242
SQL Scripts Questions 244
Metadata Questions 246
Debugging and Reporting Environment Questions 247
Integration with 3d Party Products Questions 248
Firebird and Interbase Servers Questions 249
MS SQL Server Questions 251
Oracle Server Questions 253
SQLite Database Questions 254
MySQL Server Questions 256
MS Access Questions 256

Symbol Reference **256**
uADCompClient Namespace 257
Classes 258
TADAdaptedDataSet Class 259
TADCommand Class 267
TADConnection Class 279
TADCustomCommand Class 291
TADCustomConnection Class 318
TADCustomEventAlerter Class 358
TADCustomLocalSQL Class 363
TADCustomManager Class 365
TADCustomMemTable Class 388
TADCustomQuery Class 394
TADCustomSchemaAdapter Class 400
TADCustomStoredProc Class 403
TADCustomTransaction Class 413
TADCustomUpdateObject Class 420
TADEventAlerter Class 422
TADLocalSQLDataSet Class 425
TADLocalSQLDataSets Class 428

TADManager Class 429
TADMemTable Class 436
TADMetaInfoCommand Class 452
TADMetaInfoQuery Class 461
TADQuery Class 476
TADRdbmsDataSet Class 501
TADSchemaAdapter Class 514
TADStoredProc Class 514
TADTable Class 539
TADTransaction Class 561
TADUpdateSQL Class 563
Functions 570
uADCompClient.ADManager Function 570
uADCompClient.ADSetConnectionClass Function 570
uADCompClient.ADSetManagerClass Function 571
uADCompDataSet Namespace 571
Classes 572
TADAggregate Class 572
TADAggregates Class 577
TADAutoIncField Class 579
TADBlobStream Class 582
TADDataSet Class 584
TADIndex Class 653
TADIndexes Class 659
TADMasterDataLink Class 661
TADSQLTimeIntervalField Class 665
TADWideMemoField Class 666
TADXMLField Class 667
uADCompGUIx Namespace 668
Classes 669
TADGUIxAsyncExecuteDialog Class 669
TADGUIxComponent Class 671
TADGUIxErrorDialog Class 672
TADGUIxLoginDialog Class 675
TADGUIxScriptDialog Class 680
TADGUIxWaitCursor Class 683
uADCompScript Namespace 684
Classes 685
TADScript Class 685
TADScriptCommand Class 704
TADScriptCommandRegistry Class 708
TADScriptOptions Class 709

TADSQLScript Class 720
TADSQLScripts Class 721
uADMoniRemoteClient Namespace 722
Classes 722
TADMoniRemoteClientLink Class 723
uADPhysADS Namespace 724
Classes 725
TADADSBackup Class 725
TADADSBackupRestore Class 727
TADADSRestore Class 729
TADADSService Class 731
TADADSUtility Class 733
TADPhysADSDriverLink Class 737
uADPhysASA Namespace 740
Classes 741
TADASABackup Class 741
TADASAService Class 744
TADASAValidate Class 745
TADPhysASADriverLink Class 748
uADPhysDataSnap Namespace 749
Classes 749
TADPhysDataSnapDriverLink Class 749
uADPhysDB2 Namespace 750
Classes 750
EDB2NativeException Class 750
TADPhysDB2DriverLink Class 751
uADPhysDBExp Namespace 751
Classes 752
TADPhysDBXDriverLink Class 752
uADPhysIB Namespace 752
Classes 753
TADIBBackup Class 753
TADIBNBackup Class 756
TADIBNRestore Class 759
TADIBRestore Class 761
TADIBSecurity Class 764
TADIBService Class 769
TADIBTrace Class 771
TADIBValidate Class 774
TADPhysIBDriverLink Class 777
uADPhysManager Namespace 778
Classes 778

TADPhysDriverLink Class 778
TADPhysDriverService Class 783
uADPhysMSAcc Namespace 784
Classes 785
EMSAccessNativeException Class 785
TADMSAccessService Class 786
TADPhysMSAccessDriverLink Class 791
uADPhysMSSQL Namespace 791
Classes 792
EMSSQLNativeException Class 792
TADMSSQLError Class 793
TADPhysMSSQLDriverLink Class 793
uADPhysMySQL Namespace 794
Classes 794
TADPhysMySQLDriverLink Class 794
uADPhysODBC Namespace 796
Classes 796
TADPhysODBCDriverLink Class 796
uADPhysODBCBase Namespace 797
Classes 797
TADPhysODBCBaseDriverLink Class 797
TADPhysODBCBaseService Class 799
uADPhysOracle Namespace 800
Classes 800
TADPhysOracleDriverLink Class 800
uADPhysPG Namespace 803
Classes 803
TADPhysPgDriverLink Class 803
uADPhysSQLite Namespace 804
Classes 804
TADPhysSQLiteDriverLink Class 805
TADSQLiteBackup Class 805
TADSQLiteCollation Class 811
TADSQLiteFunction Class 814
TADSQLiteRTree Class 816
TADSQLiteSecurity Class 817
TADSQLiteService Class 821
TADSQLiteValidate Class 821
uADPhysSQLiteVDataSet Namespace 825
Classes 825
TADLocalSQL Class 825
uADPhysTDBX Namespace 829

Classes 829
TADPhysTDBXDriverLink Class 829
uADPhysTDBXBase Namespace 830
Classes 830
ETDBXNativeException Class 830
uADStanError Namespace 831
Classes 831
EADDBArrayExecuteError Class 831
EADDBEngineException Class 833
EADException Class 836
TADDBError Class 837
uADStanOption Namespace 840
Classes 840
TADBottomResourceOptions Class 841
TADBottomUpdateOptions Class 843
TADCustomOptions Class 846
TADEventAlerterOptions Class 847
TADFetchOptions Class 849
TADFormatOptions Class 860
TADMapRule Class 870
TADMapRules Class 873
TADResourceOptions Class 874
TADTopResourceOptions Class 885
TADTxOptions Class 891
TADUpdateOptions Class 896
Structs, Records, Enums 905
uADStanOption.TADActionRequest Enumeration 905
uADStanOption.TADAutoFetchAll Enumeration 906

Index b

1 FireDAC



FireDAC

API Reference (V 8.0.1.3277)

created on 18.04.2013.

Embarcadero Technologies, (c) Embarcadero Technologies 2004-2013

If you are the FireDAC beginner, please read "First Steps to use FireDAC (see page 2)" and other "Getting Started (see page 2)" articles.

Symbol Reference

Symbol Reference	Description
uADCompClient (see page 257)	Contains core data access components, including TADConnection (see page 279), TADQuery (see page 476), TADStoredProc (see page 514), TADMemTable (see page 436), etc.
uADCompDataSet (see page 571)	Contains TADDataSet (see page 584) base dataset class and additional utility methods and classes.
uADCompGUIx (see page 668)	Contains most of the UI components, including TADGUIxAsyncExecuteDialog (see page 669), TADGUIxErrorDialog (see page 672), TADGUIxLoginDialog (see page 675), TADGUIxScriptDialog (see page 680) and TADGUIxWaitCursor (see page 683) classes.
uADCompScript (see page 684)	Contains TADScript (see page 685) scripting engine class and additional utility methods and classes.
uADMoniRemoteClient (see page 722)	Contains TADMoniRemoteClientLink (see page 723) class.
uADPhysADS (see page 724)	Contains Advantage Database Server driver and service components.
uADPhysASA (see page 740)	Contains Sybase SQL Anywhere driver and services components.
uADPhysDataSnap (see page 749)	Contains Multi-Tier DataSnap driver for RAD Studio XE2 Enterprise and higher.
uADPhysDB2 (see page 750)	Contains IBM DB2 driver and services components.
uADPhysDBExp (see page 751)	Contains dbExpress v 1-3 bridge driver.
uADPhysIB (see page 752)	Contains Firebird and Interbase driver and services components.
uADPhysManager (see page 778)	Contains FireDAC driver and services base classes.
uADPhysMSAcc (see page 784)	Contains Microsoft Access driver and services components.
uADPhysMSSQL (see page 791)	Contains Microsoft SQL Server driver and services components.
uADPhysMySQL (see page 794)	Contains MySQL Server driver and services components.
uADPhysODBC (see page 796)	Contains ODBC bridge driver.
uADPhysODBCBase (see page 797)	Contains FireDAC driver and services base classes for all ODBC based drivers.
uADPhysOracle (see page 800)	Contains Oracle Database driver and services components.
uADPhysPG (see page 803)	Contains PostgreSQL driver and services components.
uADPhysSQLite (see page 804)	Contains SQLite driver and services components.
uADPhysSQLiteVDataSet (see page 825)	Contains SQLite based Local SQL engine.
uADPhysTDBX (see page 829)	Contains dbExpress v 4 bridge driver.
uADPhysTDBXBase (see page 830)	Contains FireDAC driver base classes for all TDBX based drivers.

uADStanError (see page 831)	Contains error classes - EADException (see page 836), EADDBEngineException (see page 833), etc
uADStanOption (see page 840)	Contains option classes - TADFetchOptions (see page 849), TADUpdateOptions (see page 896), etc.

1.1 Overview

FireDAC is an unique set of Universal Data Access Components (see page 20) for developing cross platform database applications on Delphi, C++Builder and FreePascal. With its powerful common architecture FireDAC enables native high-speed direct access from Delphi to Interbase, SQLite, MySQL, SQL Server, Oracle, PostgreSQL, IBM DB2, SQL Anywhere, Access, Firebird, Informix and more.

Description

Based on 10 years of experience writing native drivers for the database back-ends, FireDAC was built as powerful access layer that supports all that the features needed to build real-world high-load applications. It provides a common API for accessing different database back-ends, without giving up access to unique database-specific features, or compromising on performance.

FireDAC Core Features

1.2 Getting Started

A set of articles introducing FireDAC to you.

1.2.1 First Steps to use FireDAC

This article guides you through the creation of your first application built using the Delphi edition of FireDAC.

Description

Introduction

This tutorial has three main sections:

- **Establishing the connection to the database**: how to use Delphi to create an application that will connect to a database.
- **Selecting rows from the database**: hook the data up to a grid and display it at design time.
- **Preparing the application for runtime**: describes the necessary steps to make an application run in as standalone execute (runtime).

The screen shots and instructions below relate to Delphi 2007, so there will be minor changes if you use a different Delphi release. And the Ron Grove movie:

Establishing the Connection to the Database

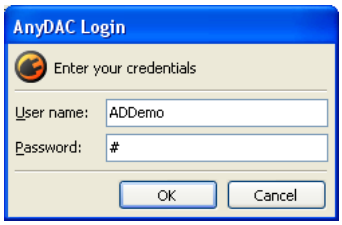
In this article, we use the Microsoft SQL Server's Northwind demo database (see page 14) and a predefined connection

definition (see page 27) - *MSSQL_Demo*. Let's start by creating a new "*VCL Forms Application Delphi for Win32*".

First, drop a TADConnection (see page 279) component onto the form selected from the "*FireDAC*" page of the Delphi Tool Palette. This component is responsible to establish and control the database connection.

Next, select *MSSQL_Demo* from the dropdown list of its ConnectionDefName (see page 283) property. This will associate the connection component with the specified connection definition. By using the predefined definitions you do not need to enter any additional parameters (e.g. the server name or the default database).

After setting the Connected (see page 282) property to True, FireDAC will display a Login Dialog:



Here you can enter your user credentials. Press the OK button to establish the connection to the DB and to create a user session on the DBMS if this DBMS supports this feature.

After the connection is successfully established, the Connected (see page 282) property will still be set to True, otherwise it will be reset to False and FireDAC will display an appropriate error message. By default, you do have three attempts to enter valid credentials. If they all fail, the login process also fails and you will get an error message.

Selecting Rows from the Database

Now drop a TADQuery (see page 476) component from the "*FireDAC*" palette page onto the form. This component is responsible for the execution of SQL commands, fetching rows from the DB and for posting changed data back to the DB.

Set its Connection (see page 503) property to *ADConnection1* to hook the query to a database connection.

Note: If a query component is dropped on a form or datamodule which already contains one or more TADConnections, FireDAC will automatically set the query's Connection property to point to the connection that was created first.

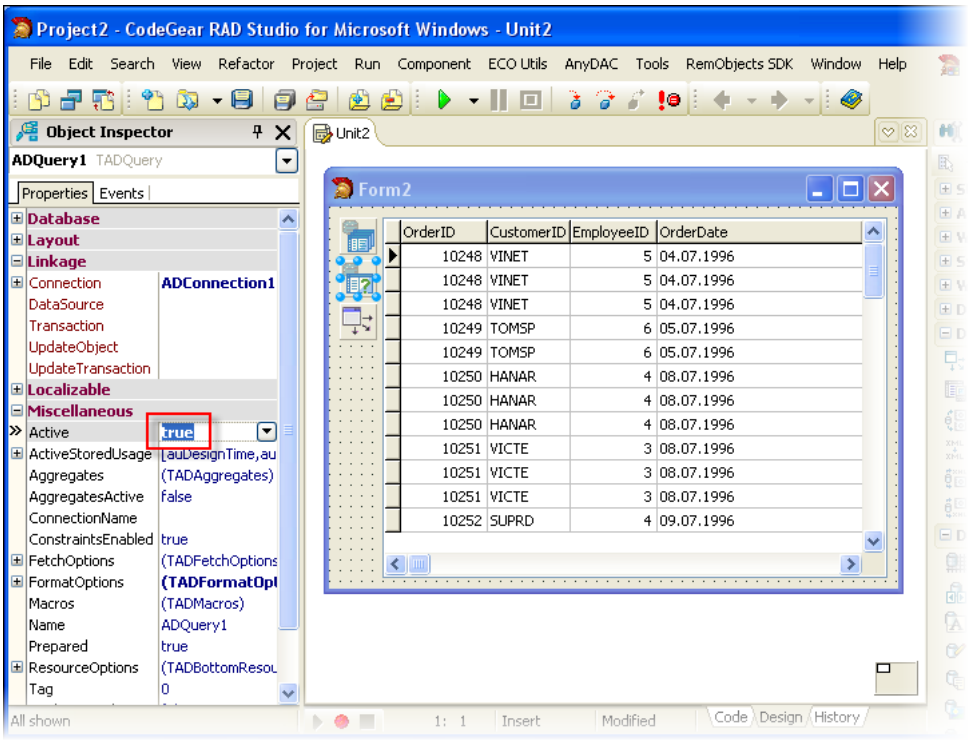
Click on its SQL (see page 499) property and enter the following SQL command into the editor window:

```
SELECT * FROM Orders
```

Press the *OK* button to close the editor. This stores the SQL command text into the TADQuery (see page 476) component's SQL (see page 499) property.

Next, drop a standard Delphi **TDataSource** component from the "*Data Access*" palette page onto your form. Set its **DataSet** property to *ADQuery1*. Now drop a **TDBGrid** control onto the form from the "*Data Controls*" page and set its **DataSource** property to *DataSource1*.

Finally, set ADQuery1's **Active** property to True. This will send the SQL command to the DBMS, which will execute the command and return a result set. This data will be displayed by the DBGrid1 control:



Preparing the Application for Runtime

To allow your application to work at runtime you will need:

- to drop the TADPhysMSSQLDriverLink (see page 793) component from the "*FireDAC Links*" palette page;
- to drop the TADGUIxWaitCursor (see page 683) component from the "*FireDAC UI*" palette page.

Now your application is ready to run. These components assure that the necessary units get linked into your application execute. For real world applications this components are normally dropped on a main data module.

Summary

This article has provided a tutorial showing how to create a simple client-server application using FireDAC for Delphi. It shows how to use the FireDAC connection and query components to establish a connection to the DB and return rows to the client without actually writing any code.

We suggest that you also read the Setting up Connections (see page 4) article for all the details how to setup the connection definitions.

For other DBMS Getting Started demo applications see FireDAC\Samples\Getting Started folder.

1.2.2 Setting up Connections

This article guides you through the process of defining an FireDAC database connection definition. In this article we will create the connection definition for the Microsoft SQL Server Northwind demo database.

Description

Introduction

This article describes:

- **What is an FireDAC Connection Definition**: use an FireDAC connection definition to specify the DBMS connection

parameters.

- **Using the ADExplorer Utility**: use FireDAC Explorer to create the connection parameter sets saved for system wide usage in a centralized storage file.
- **Using the TADConnection Design Time Editor**: use the TADConnection design time editor to set up connection parameters at design time.

What is an FireDAC Connection Definition?

The FireDAC components use the concept of connection definitions to submit all necessary connection parameters, like **Server**, **Database**, **User_Name** to the FireDAC driver level (at run- and design time). Please read the Connection Definition reference (see page 27) to learn all the details about the exact technical definition, like how to create a connection definition at runtime using Delphi code.

FireDAC offers two basic methods to specify the connection definition (see page 27) at design time:

- Create a shared and centralized persistent connection definition using the ADExplorer (see page 181). Later, this definition can be assigned to the TADConnection.ConnectionDefName (see page 283) property. For customers who know the CodeGear BDE, the ADExplorer is similar to the BDE Administrator tool.
- Create a temporary connection definition using the TADConnection (see page 279) design time editor, by filling the TADConnection.Params (see page 289) property within the Delphi design time editor. For customers who know the ADO, this editor is similar to a connection string builder.

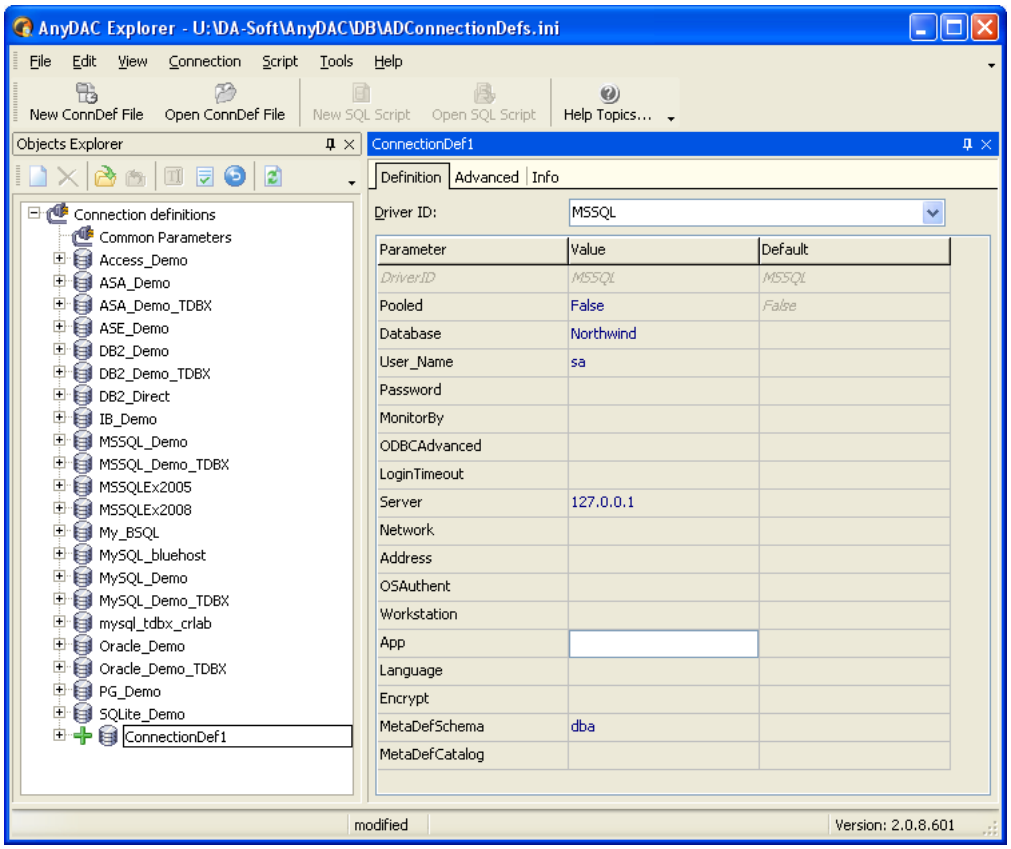
Using the ADExplorer Utility

The ADExplorer (see page 181) utility is the main tool to maintain the centralized persistent connection definitions (see page 27). Please read the ADExplorer reference (see page 181) to understand the detailed usage of this tool.

To run ADExplorer, just click the Delphi IDE menu item **Tools** -> **FireDAC** -> **Explorer**. Then click **Ctrl-N** to create a new empty connection definition. FireDAC is a multi-DBMS data access engine offering a full set of drivers for each supported DBMS. The DriverID (see page 31) parameter value specifies the driver you decide to use. After setting the **DriverID** to **MSSQL**, FireDAC displays the driver specific set of parameters (see page 202). For the Microsoft SQL Server it includes:

Parameter	Description
Server	The SQL Server server identifier. If the host only has a single default server, then this value is the host address.
Database	The name of the default database.
OSAuthent	If **Yes**, then FireDAC will use Windows authentication. If **No** (by default), then MS SQL Server authentication is used.
User_Name	The login user name, if OSAuthent=No.
Password	The login password, if OSAuthent=No.
MetaDefSchema	Default schema name. Design time code will exclude a schema name from an object name, if it is equal to MetaDefSchema.

The next screenshot shows the connection definition setup:



Press *Ctrl-A* to save the connection definition to the connection definition file.

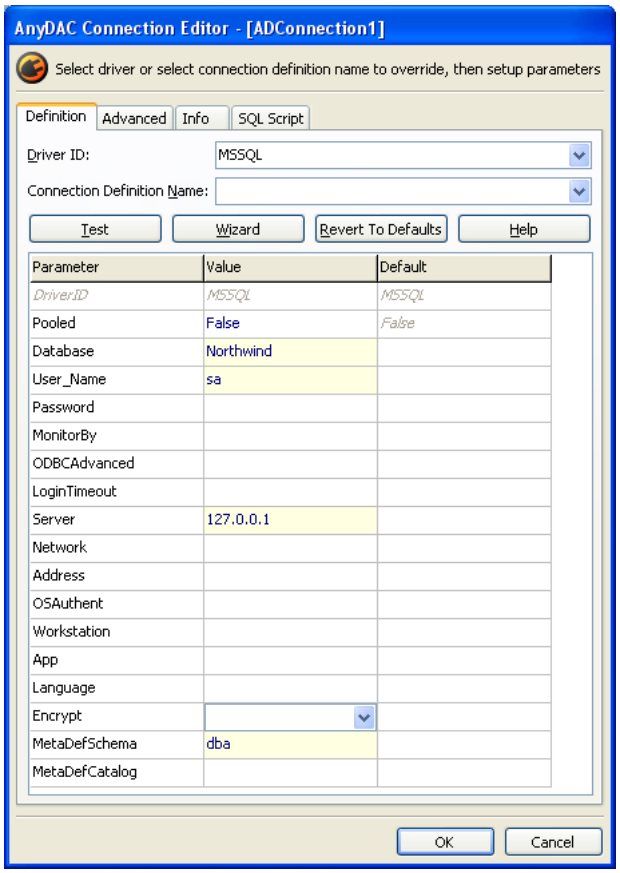
For testing a new connection definition you just click on the "+" within the tree item. The explorer will show the Login Dialog. After a successful login the tree node will expand and allows to drill down into the DB objects.

Note: If you added a new persistent connection definition using ADExplorer or ADAdministrator while the Delphi IDE is running, it will be not visible to the FireDAC design time code. To refresh the persistent connection definition list, you need to restart the Delphi IDE.

Now the connection definition is ready for usage within Delphi. Just set the the value of the TADConnection.ConnectionDefName (see page 283) property to the name of the newly created connection definition.

Using the TADConnection Design Time Editor

The TADConnection (see page 279) component design time editor is the environment to maintain temporary connection parameters. Double click any TADConnection (see page 279) component at design time. The FireDAC package will display the **Connection Editor** dialog.



This editor provides a similar functionality to the ADExplorer. Again, you start by setting:

- the **Driver ID**, if you want to create a temporary connection definition from scratch (our case);
- the **Connection Definition Name**, if you want to create a temporary connection that overrides the parameters of an existing persistent connection.

Again, you fill in the parameters as specified in the chapter above. This dialog offers the following functions:

- **Test** button - test the connection definition.
- **Wizard** button - call a DBMS specific connection definition wizard, if available.
- **Revert to default** button - reset the parameters to their default values.
- **Help** button - go to a help page with description of the current driver parameters.
- **Info** page - try to connect to a DBMS and obtain information about connection.
- **SQL Script** page - execute the SQL script commands in this connection.

After pressing the editor's **OK** button, FireDAC will load the connection parameters into the TADConnection.Params (see page 289) property and set the TADConnection.DriverName (see page 284) property to the chosen value.

Activate a Connection

After you have assigned a persistent connection definition name to the TADConnection.ConnectionDefName (see page 283) property or filled in temporary connection definition parameters into the TADConnection.Params (see page 289) property, set TADConnection.Connected (see page 282) property to True. If the parameters are specified correctly, the connection will be established.

1.2.3 Very High Performance using the Array DML

This article describes the power of the Array DML feature supported by FireDAC. This first article will lead you through an easy example that shows how to insert thousands of records per second by writing just a few lines of code.

Description

Introduction

This tutorial has three main sections:

- How to prepare your test environment.
- The main elements of the Array DML commands.
- The typical results of the Array DML test run.

FireDAC encapsulates all database server specific implementation of the Array DML commands (see page 82) and lets you use identical code for all server types. Obviously, the resulting performance will differ based on the server implementation; especially Oracle, Microsoft SQL Server and IBM DB2 have very powerful support of the Array DML and the resulting performance increase is just amazing.

Please use the sample code to get a feeling for the potential performance increase within your application and network.

Prepare your Test Environment

The following example works with the FireDAC sample database environment. For further details about the installation of this database look into FireDAC Demo Databases (see page 14). You find the demo projects in your sample directory:

- This tutorial code - <FireDAC>\Samples\Comp Layer\TADQuery\ExecSQL\AD03-ArrayDML.
- A basic example code - <FireDAC>\Samples\Comp Layer\TADQuery\ExecSQL\Batch.

How does the Array DML command work?

Imagine a "use case" where you have to INSERT, UPDATE, DELETE or run any other parametrized command N times, typically one command per single record. This means, that each set of input parameters requests to execute a SQL command and is transferred separately between the client and the server. This leads to a heavy load on the network, client and server.

Array DML allows you to transport not only one, but N-sets of data within one transfer. Have a look at the following example:

```
ADQuery1.SQL.Text:= 'insert into ADQA_Batch_test (tint, tstring) values(:f1, :f2)';
```

You can speed up your code dramatically by using Array DML commands. Such commands transfer not only one, but N sets of parameters.

```
ADQuery1.Params.ArraySize := 100;
...
for i := 0 to ADQuery1.Params.ArraySize do begin
  ADQuery1.Params[0].AsIntegers[i] := i;
  ADQuery1.Params[1].AsStrings[i] := 'Test' + IntToStr(i);
end;
ADQuery1.Execute(ADQuery1.Params.ArraySize);
```

This means the Params property of the query is no more a one- but a two-dimensional array, that allows you to store N sets of parameter values before sending them to the server.

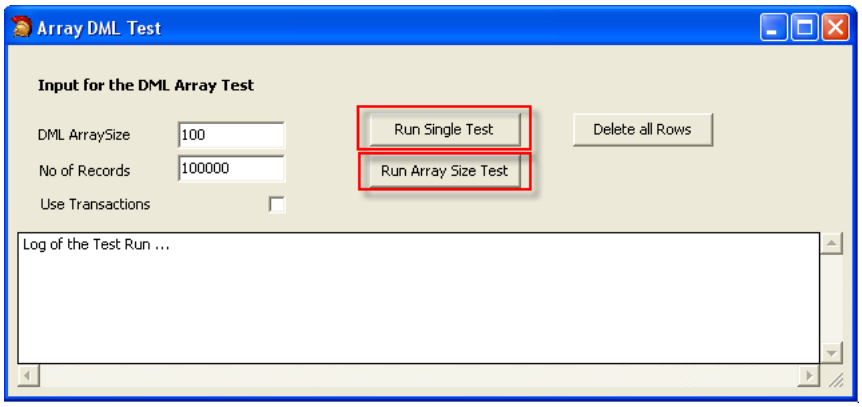
For more details, please see "Array DML (see page 82)" reference chapter.

Usage Hints

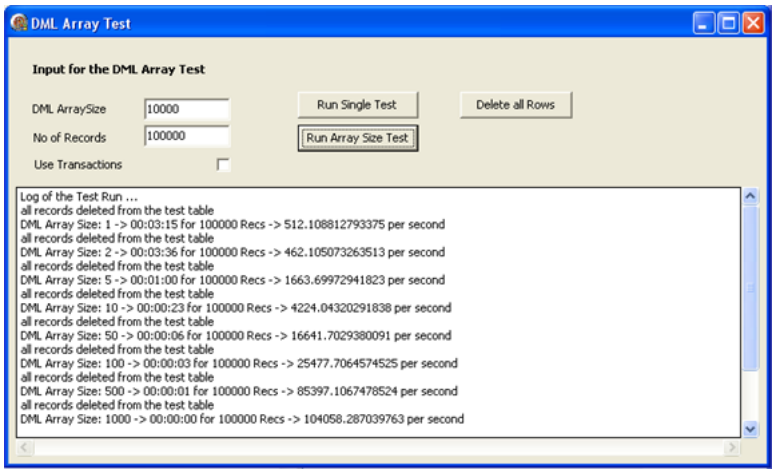
- Can be used for any SQL command that uses parameters (INSERT, UPDATE, DELETE ...).
- The error handling is supported on record level and described in a separate article.
- FireDAC unifies the Array DML for different server types (no need for you do dig into the API).

Typical Results of the Array DML Test Run

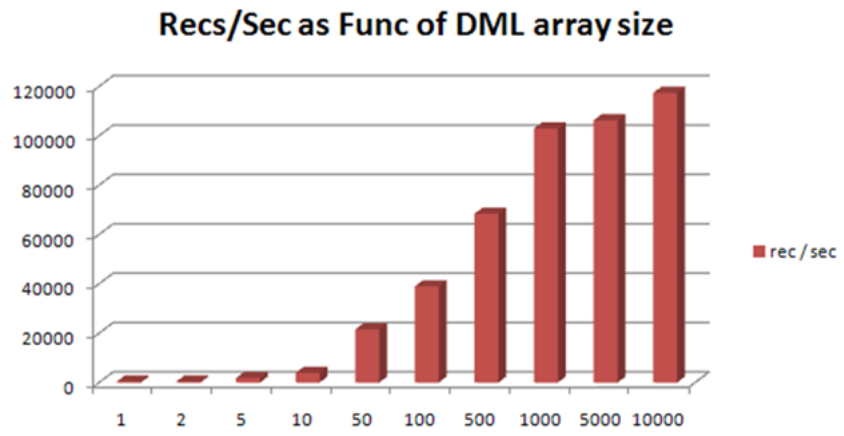
The attached test code allows you to experiment within your specific environment.



Results of the test example can differ a lot depending on host and network performance. A typical picture of a local Oracle on a rather old laptop will still show > 100'000 records per second as you can see in this screen shot:



A larger Array DML ArraySize results in a higher performance (in our case up to a factor of 2000). We expect that the performance boost in your own environment will surprise you as well.



Performance Hints

Array DML command performance is influenced by:

- The fact that they are a lot faster on slow networks as these commands create less TCP/IP packages.
- They reduce the CPU load on the client side, as most of the time the server has to work on the array command.
- The theoretical speed of > 100'000 rec/sec is not often reached as the server normally has to evaluate triggers and indexes.
- For real large batch inserts (e.g. > 1'000'000 records), you should consider to drop and recreate non primary key indexes to reach a maximum performance.

1.2.4 Creating Reports with FastReport

This article guides you through the creation of your first report built with FastReport and FireDAC.

Description

Installation

FireDAC installer is shipped with FastReport add-on's that can be found in FireDAC\AddOn\FastReportN folders. Where N corresponds to your FastReport version.

To install the add-on please:

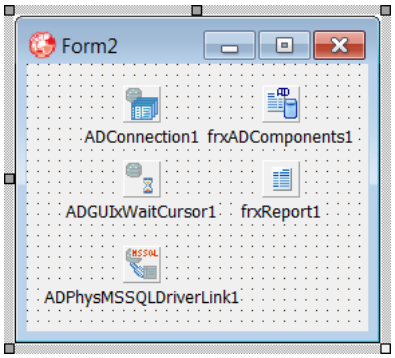
- open the project group that matches your FastReport and Delphi version;
- compile frxADXX.dpk and dclfrxADXX.dpk packages;
- right click dclfrxADXX.dpk package and select **Install**.

After installation TfrxADComponents component will appear on FastReport palette.

Before creating first report

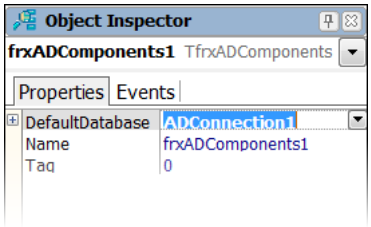
To get started with creating your first report you will need to drop the following components onto your form:

- TADConnection;
- TADGUIxWaitCursor;
- TADPhysXXXXDriverLink;
- TfrxADComponents;
- TfrxReport.



The first three components are used for connecting (see page 4) to your database. TfrxADComponents is used to link FastReport components with FireDAC components.

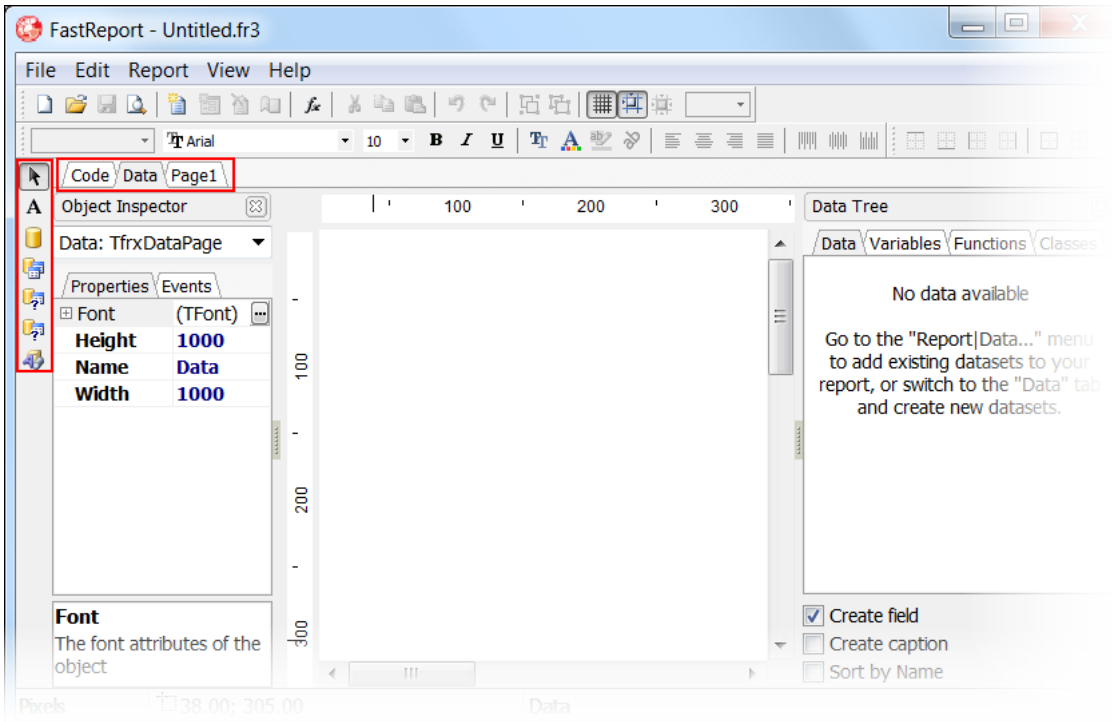
Set frxADComponents1.DefaultDatabase property to ADConnection1. This connection will be used by default for all datasets you create for the report.



Now you are ready to create the report: right click frxReport1 and choose **Edit report...**

Adding datasets to report

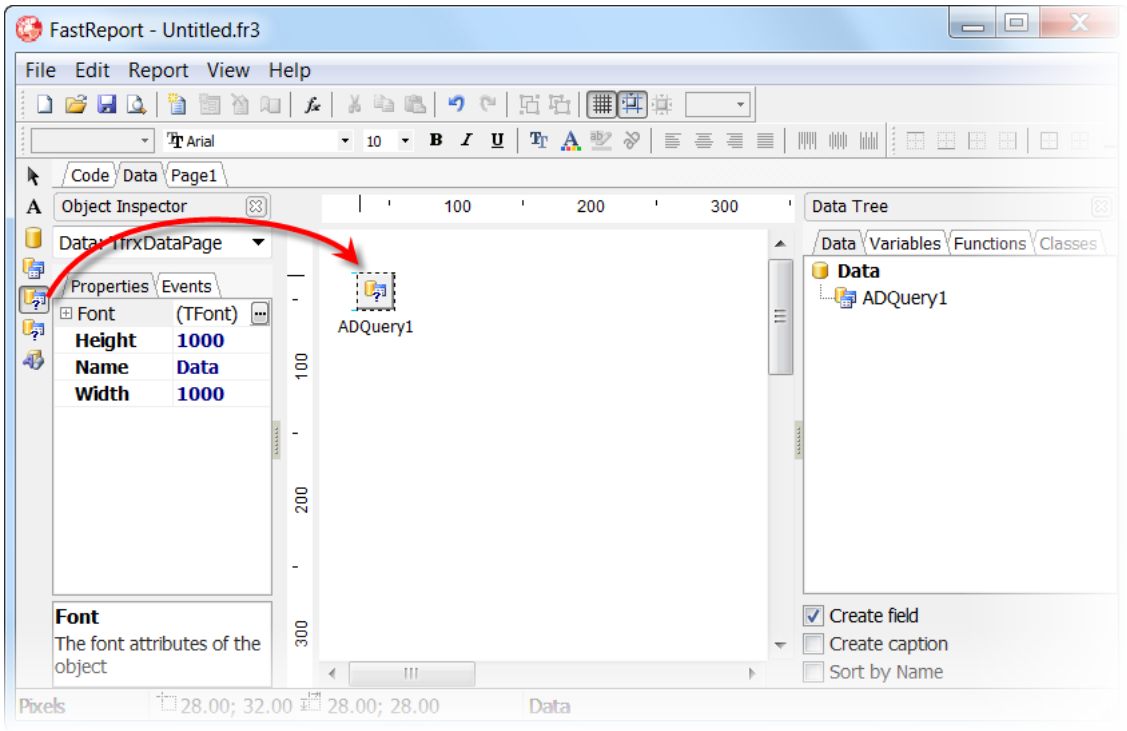
TfrxReport component editor area consists of a left-most tool palette and three pages: 'Code', 'Data' and 'Page1'.



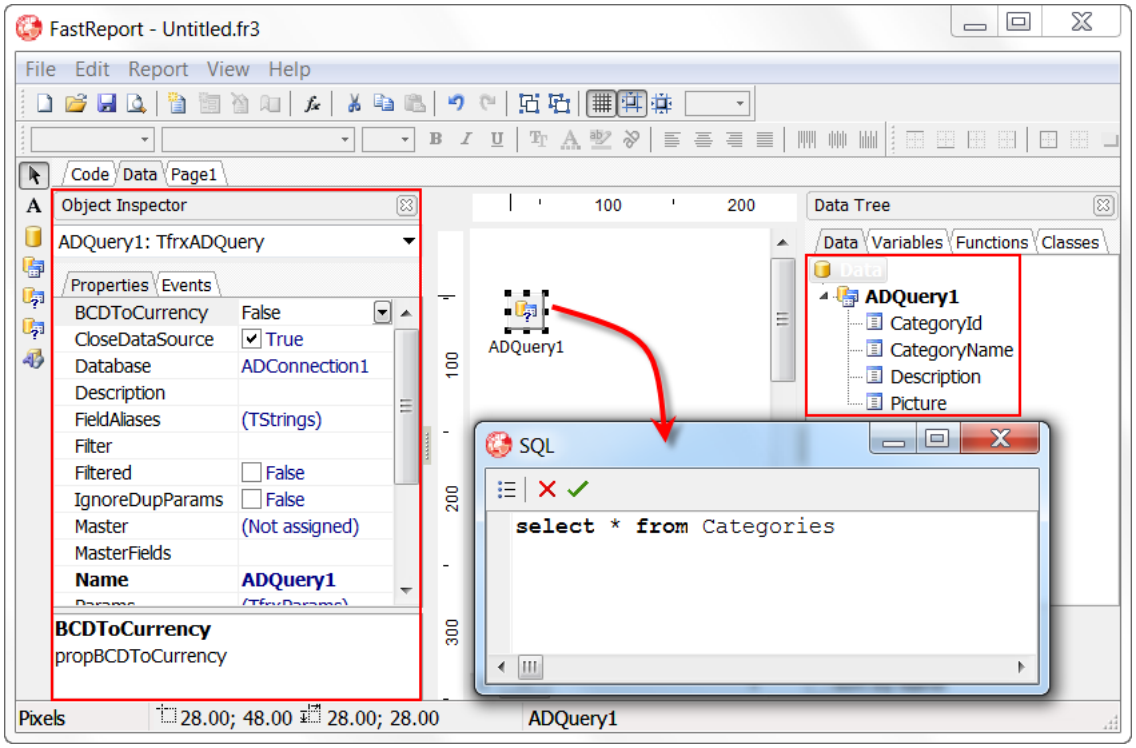
Select the 'Data' page. On the tool palette you can see the following components:

- AD Database;
- AD Table;
- AD Query;
- AD StoredProc.

All of them match the respective FireDAC components: TADConnection (see page 279), TADTable (see page 539), TADQuery (see page 476) and TADStoredProc (see page 514). To access a table in your database click **AD Query** on the palette and then click on the empty Data area.



This will add ADQuery1 component that can be configured in the Object Inspector similarly to how you would do that in Delphi IDE. Double click the component and enter the SELECT query to your table.



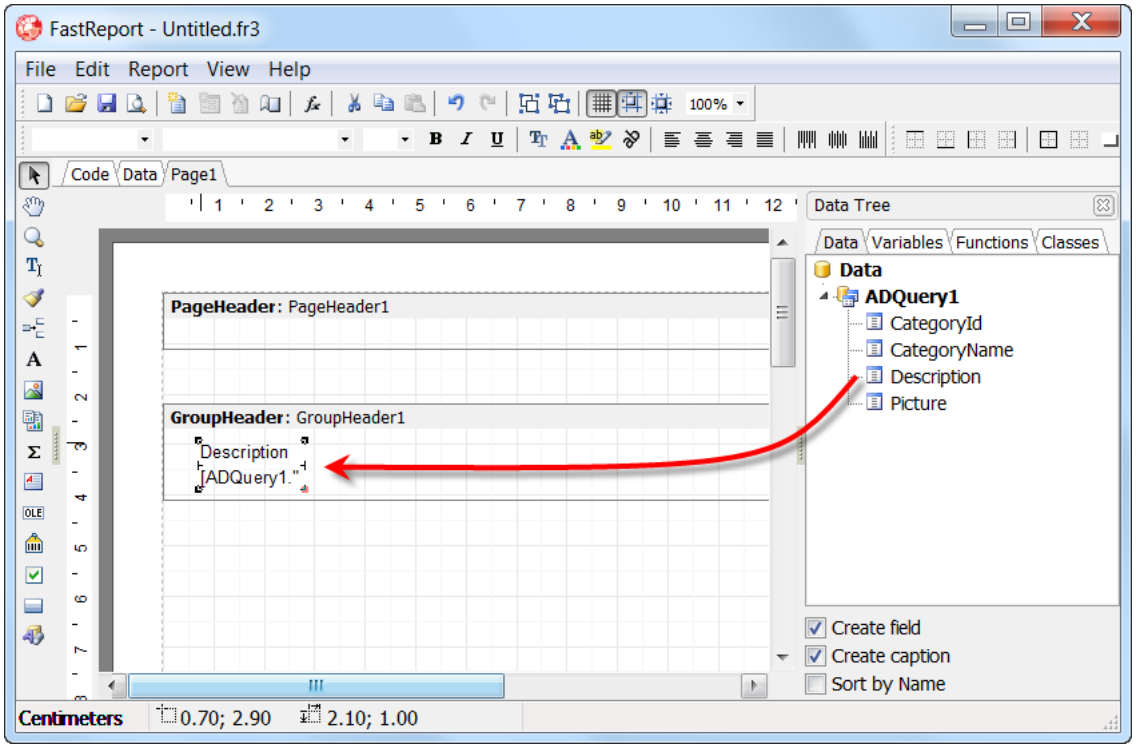
When applied the Data Tree will show the queried fields.

The same way you can configure the other datasets. In case you need to use more than one connection or connect to

different DBMS's you can drop **AD Database** components to the data area. Dataset components Database property then should be set to the appropriate ADDatabase. Also in this way you will need to drop TADPhysXXXXDriverLink to your form or data module corresponding to DBMS used.

Using datasets

Select 'Page1' of the FastReport editor. Added datasets are available in the Data Tree. Expanding the dataset nodes will allow you to drag and drop the required field to the page area to build the report.



Example

AddOn\FastReportX\Example folder contains a pre-configured sample that can be used to get started with FastReport and FireDAC.

1.2.5 Demo Databases

Describes how to install FireDAC demo databases.

Description

General

FireDAC uses the **Northwind** as a demo database. SQLite and MS Access demo databases are pre installed and does not require any additional configuration. For other DBMS you have to setup connection and create a database. The FireDAC installer contains SQL scripts, CSV data files and BAT files. The demo database may be created:

- **automatically** at the FireDAC installation;
- **manually** any time later.

Prerequisites

- The demo database files will be installed only, if "Sample Databases" on the "Select Components" page is checked in the installer.
- You should have an existing database and user account. The FireDAC installer will ask this info.
- **SQLite** - installer installs a ready to use FireDAC\DB\Data\ADDEMO.SDB database file.
- **Access database** - installer installs a ready to use FireDAC\DB\Data\ADDEMO.MDB database file.
- **SQL Server 2000** - the Northwind DB installation may be optional, because it is part of SQL Server 2000 distribution, may be downloaded separately, and may be already installed.

Automatic installation

The FireDAC installer may build a demo database in automatic mode:

- Mark required DBMS's on the "Demo Database" page of installer wizard.
- On the next pages fill in the database and user account info.
- At the end the installer creates a demo databases for marked DBMS's.

Manual installation

To build a demo database by yourself any time later, perform the following steps:

- Run ADExplorer (see page 181);
- Create a connection definition (see page 27) for a required DBMS. The connection definition must have a predefined name, depending on DBMS:

DBMS	Sample connection definition	Connection definition name
Advantage Database (see page 189)	[ADS_Demo] DriverID=ADS ServerTypes=2 Database=\\DA\ADS_DB\ADDEMO.ADD User_Name=adssys	ADS_Demo
IBM DB2 Server (see page 196)	[DB2_Demo] DriverID=DB2 Alias=addemo User_Name=db2admin	DB2_Demo
Interbase / Firebird Server (see page 198)	[IB_Demo] DriverID=IB Database=E:\Firebird\ADDEMO.FB User_Name=sysdba	IB_Demo
Microsoft Access database (see page 207)	[Access_Demo] DriverID=MSAcc Database=$(ADHOME)\DB\Data\ADDemo.mdb	Access_Demo
Microsoft SQL Server (see page 202)	[MSSQL_Demo] DriverID=MSSQL Server=127.0.0.1 Database=Northwind User_Name=sa	MSSQL_Demo

MySQL Server (see page 209)	[MySQL_Demo] DriverID=MySQL Server=127.0.0.1 Database=addemo User_Name=root	MySQL_Demo
Oracle Server (see page 215)	[Oracle_Demo] DriverID=Ora Database=ORA_920_APP User_Name=ADDemo	Oracle_Demo
PostgreSQL (see page 218)	[PG_Demo] DriverID=PG Server=127.0.0.1 Database=addemo User_Name=ad	PG_Demo
SQLite database (see page 221)	[SQLite_Demo] DriverID=SQLite Database=$(ADHOME)\DB\Data\ADDemo.sdb	SQLite_Demo
Sybase SQLAnywhere (see page 224)	[ASA_Demo] DriverID=ASA Server=addemo_asa11 Database=addemo_asa11 User_Name=dba	ASA_Demo

- Goto FireDAC\Bin folder.
- **SQL Server** - to disable Northwind DB installation, open Bin\createMSSQL.bat and replace True with False. For SQL Server 2000 replace MSSQL2005 with MSSQL.
- Run create<your DBMS>.bat (see page 188)

See Also

Database Connectivity (see page 188), Defining Connection (see page 27), ADExplorer (see page 181), Create (DB name).bat (see page 188)

1.2.6 Demo Applications

Describes how to use FireDAC demo applications.

Description

General

FireDAC has many sample applications, located in the FireDAC\Samples folder. The folder has sub-folders:

- "**Getting Started**" - basic simple demo applications, showing how to connect and execute queries;
- "**Comp Layer**" - demo applications for the FireDAC components;
- "**DBMS Specific**" - demo applications specific for some DBMS;
- "**DApt Layer**", "**DatS Layer**", "**GUIx Layer**", "**Moni Layer**", "**Phys Layer**", "**Stan Layer**" - advanced demo applications specific for low level FireDAC API's.

The more deep sub-folders are:

- for "**Comp Layer**" - demo applications for a specific component;

- for others - demo applications for a specific feature.

Prerequisites

- Most of FireDAC demo applications require the FireDAC Demo Database (see page 14) to be installed. So, first install a demo database.
- A demo application may require some minor adjustment for your Delphi version. In most cases:
  - just ignore the DFM loading warnings;
  - comment out the not found units.

See Also

ADExecutor (see page 180), ADExplorer (see page 181), Demo Databases (see page 14)

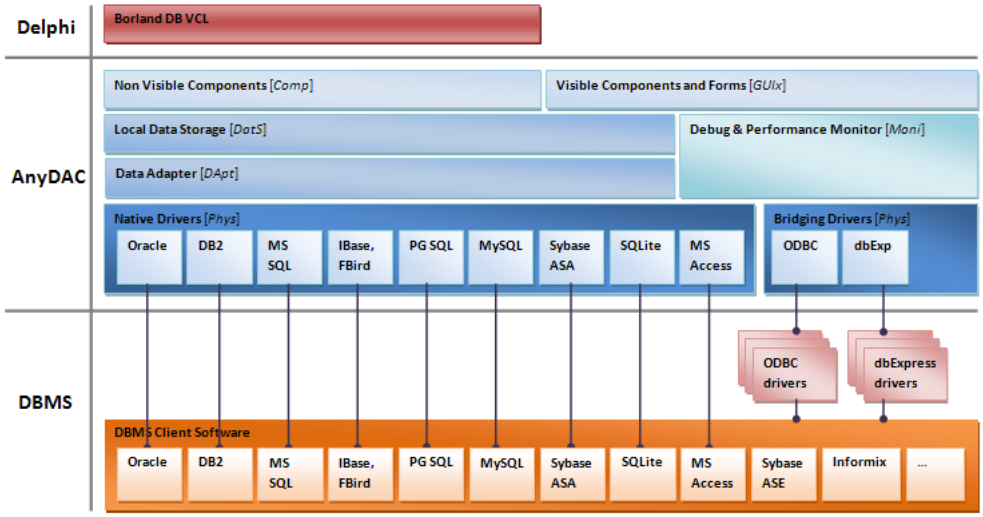
1.3 Architecture

A set of articles describing FireDAC architecture, including the components set, supported DBMS, supported tools.

1.3.1 General

FireDAC has flexible, powerful and extendable architecture.

Description



General

FireDAC has a weakly coupled multi layered architecture, where layers may provide services. A service API is defined as a COM interface, that may be requested by other layers using the interface factory.

When an interface implementation is not found, then an exception will be raised. To link the implementation into an application, the corresponding unit must be linked in. There may be alternative implementations and mandatory / optional implementations.

Example

For example, IADGUIxWaitCursor interface defines API for the mouse wait cursor. It has the three alternative implementations (providers (see page 672)):

- uADGUIxFormsWait unit contains implementation for the VCL GUI applications;
- uADGUIxFMXWait unit contains implementation for the FireMonkey GUI applications;
- uADGUIxConsoleWait unit contains implementation for console applications.

GUI or console mouse wait cursor implementation is mandatory and must always be linked into the application. Otherwise exception is raised:

```
Object factory for class {3E9B315B-F456-4175-A864-B2573C4A2201} missing.
To register it, you can drop component [TADGUIxWaitCursor] into your project
```

Note, the exception message suggests the unit to include into your project to link the standard interface implementation.

Non visible Components [Comp]

The layer represents the FireDAC public interfaces as Delphi non-visual components (see page 20), similar to other Delphi data access components. It includes components - TADConnection (see page 279) (establish connection (see page 37)), TADQuery (see page 476) (execute query (see page 66)), TADStoredProc (see page 514) (execute stored procedure (see page 71)), TADMemTable (see page 436)(in-memory dataset), TADScript (see page 685) (SQL script engine (see page 88)), etc. The main units are:

- uADCompDataSet (see page 571);
- uADCompClient (see page 257);
- uADCompScript (see page 684).

Visible Components [GUIx]

The layer provides a way to interact with the end user from an FireDAC application. It is a set of high-level components (see page 20) allowing to add the end-user dialogs for the standard database operations, like a Login or a Wait-for-operation. It includes components - TADGUIxWaitCursor (see page 683) (wait cursor), TADGUIxLoginDialog (see page 675) (login dialog), TADGUIxErrorDialog (see page 672) (error dialog), etc. Layer provides implementations for VCL / LCL, FireMonkey and console platforms. The main units are:

- uADGUIxIntf;
- uADCompGUIx (see page 668);
- uADGUIx<platform>Xxxx.

Local Data Storage [DatS]

The layer is a Local Data Storage implementation, which is analog to the ADO.NET's DataSet and its related objects (DataTable, DataRow, DataView, etc). It is an in-memory data engine, actually storing and handling all client data and meta-data. It has flexible API, allowing you to use DatS in applications. The main unit is:

- uADDatSManager.

Data Adapter [DApt]

The layer allows automation and fine-tuning of a read operation with complex result sets (master-details, nested, ADT, etc) and allows posting updates (see page 113) back to the database system. It is controlled mostly through TField and UpdateOptions properites. The main units are:

- uADDAptIntf;
- uADDAptManager.

Debug and Performance Monitor [Moni]

The layer represents FireDAC debugging capabilities (see page 174) by implementing debug monitor interfaces, which allow monitoring and tracing interactions between the FireDAC application and the DBMS. It is controlled mostly through [ADSettings] parameters of ADConnectionDefs.ini and the MonitorBy connection definition parameter. It includes components - TADMoniRemoteClientLink (see page 723) (monitoring using ADMonitor), TADMoniFlatFileClientLink (tracing into file), TADMoniCustomClientLink (custom tracing). The main units are:

- uADMoniRemoteClient (see page 722);
- uADMoniFlatFile;
- uADMoniCustom.

Drivers API [Phys]

The layer defines interfaces for physical data access. It implements them in separate packages as the drivers, where each driver package belongs to the Phys layer and implements the required interfaces using appropriate DBMS API. See Database Connectivity (see page 188) for details. The main units are:

- uADPhysIntf;
- uADPhysManager (see page 778).

By default none of the drivers are linked into the application.

Native Drivers [Phys]

The native drivers are implementing access to a DBMS using a high performance low-level API recommended by the DBMS vendor. They precisely adapt DBMS specific features to the FireDAC API. All native drivers have been tested and optimized for a DBMS. They include TADPhys<DBMS>DriverLink and service components. The main units are:

- uADPhys<DBMS>Wrapper;
- uADPhys<DBMS>Meta;
- uADPhys<DBMS>:
  - uADPhysADS (see page 724);
  - uADPhysASA (see page 740);
  - uADPhysDataSnap (see page 749);
  - uADPhysDB2 (see page 750);
  - uADPhysIB (see page 752);
  - uADPhysMSAcc (see page 784);
  - uADPhysMSSQL (see page 791);
  - uADPhysMySQL (see page 794);
  - uADPhysOracle (see page 800);
  - uADPhysPG (see page 803);
  - uADPhysSQLite (see page 804).

Bridging Drivers [Phys]

The bridging drivers are implementing generic access to a DBMS using generic data access API's - ODBC and dbExpress.

The bridging drivers are using driver supplied information regarding DBMS features, which is not covering all DBMS features interesting to FireDAC. They include TADPhysODBCDriverLink (see page 796) (ODBC driver), TADPhysTDBXDriverLink (see page 829) (dbExpress v 4 and higher driver), TADPhysDBXDriverLink (see page 752) (dbExpress v 1-3 driver). The main units are:

- uADPhysODBC (see page 796);
- uADPhysTDBX (see page 829);
- uADPhysDbExp (see page 751).

1.3.2 Components

FireDAC includes 50+ Delphi non-visual components.

Description

FireDAC Core Components

Most of the applications will use TADConnection (see page 279) and few TADQuery (see page 476).

Name	Platforms	Description
TADManager (see page 429)	All	The class is responsible to connection definitions (more (see page 27)) and connections management.
TADConnection (see page 279)	All	The class is responsible to connection establishment (more (see page 37)) with a DBMS.
TADTransaction (see page 561)	All	The class is responsible for connection transaction management (more (see page 41)).
TADCommand (see page 267)	All	The class is responsible for execution of SQL commands (more (see page 66)).
TADTableAdapter	All	The class binding TADCommand and TADMemTable.
TADSchemaAdapter (see page 514)	All	The class binding few datasets into a single in-memory storage and building a centralized change log (see page 114).
TADMemTable (see page 436)	All	The class implementing in-memory dataset.
TADQuery (see page 476)	All	The class implementing dataset, capable to execute SQL queries (more (see page 66)).
TADStoredProc (see page 514)	All	The class implementing dataset, capable to execute server side stored procedures (more (see page 71)).
TADTable (see page 539)	All	The class implementing dataset, working with single database table (more (see page 73)).
TADUpdateSQL (see page 563)	All	TADUpdateSQL applies updates on behalf of queries or stored procedures that can't post updates directly (more (see page 123)).
TADDataMove	All	The class is responsible for data movement between different data sources.
TADScript (see page 685)	All	The class implementing SQL script engine, capable to execute a series of SQL queries (more (see page 88)).
TADMetaInfoQuery (see page 461)	All	The class implementing dataset, capable to execute meta-info queries (more (see page 127)).
TADEventAlerter (see page 422)	All	The class is responsible for handling the database event notifications (more (see page 75)).
TADLocalSQL (see page 825)	All	The class implementing Local SQL engine, capable to execute SQL queries with TDataSet descendants as data sources (more (see page 96)).

FireDAC Driver Link Components

Most of the applications will use one link component per supported database.

Name	Platforms	Description
TADPhysADSDriverLink (see page 737)	Win32, Win64, Linux32, Linux64	Use the TADPhysADSDriverLink component to link the Advantage Database Server driver to an application and setup it (more (see page 189)).
TADPhysASADriverLink (see page 748)	Win32, Win64, Linux32, Linux64, MacOS	Use the TADPhysASADriverLink component to link the Sybase SQL Anywhere driver to an application and setup it (more (see page 224)).
TADPhysDataSnapDriverLink (see page 749)	Win32, Win64, Linux32, Linux64, MacOS, iOS	Use the TADPhysDataSnapDriverLink component to link the Multi-Tier DataSnap driver to an application and setup it (more (see page 193)).
TADPhysDB2DriverLink (see page 751)	Win32, Win64, Linux32, Linux64	Use the TADPhysDB2DriverLink component to link the IBM DB2 driver to an application and setup it (more (see page 196)).
TADPhysDBXDriverLink (see page 752)	Win32	Use the TADPhysDBXDriverLink component to link the dbExpress v 1-3 bridge driver to an application (more (see page 194)).
TADPhysIBDriverLink (see page 777)	Win32, Win64, Linux32, Linux64, MacOS, iOS	Use the TADPhysIBDriverLink component to link the Firebird / Interbase driver to an application and setup it (more (see page 198)).
TADPhysMSAccessDriverLink (see page 791)	Win32, Win64	Use the TADPhysMSAccessDriverLink component to link the Microsoft Access driver to an application and setup it (more (see page 207)).
TADPhysMSSQLDriverLink (see page 793)	Win32, Win64, Linux32, Linux64, MacOS	Use the TADPhysMSSQLDriverLink component to link the Microsoft SQL Server driver to an application and setup it (more (see page 202)).
TADPhysMySQLDriverLink (see page 794)	Win32, Win64, Linux32, Linux64, MacOS	Use the TADPhysMySQLDriverLink component to link the MySQL Server driver to an application and setup it (more (see page 209)).
TADPhysODBCDriverLink (see page 796)	Win32, Win64, Linux32, Linux64, MacOS	Use the TADPhysODBCDriverLink component to link the ODBC bridge driver to an application (more (see page 213)).
TADPhysOracleDriverLink (see page 800)	Win32, Win64, Linux32, Linux64, MacOS	Use the TADPhysOracleDriverLink component to link the Oracle Database driver to an application and setup it (more (see page 215)).

TADPhysPgDriverLink (see page 803)	Win32, Win64, Linux32, Linux64, MacOS	Use the TADPhysPgDriverLink component to link the PostgreSQL driver to an application and setup it (more (see page 218)).
TADPhysSQLiteDriverLink (see page 805)	Win32, Win64, Linux32, Linux64, MacOS, iOS	Use the TADPhysSQLiteDriverLink component to link the SQLite driver to an application and setup it (more 1 (see page 221), more 2 (see page 135)).
TADPhysTDBXDriverLink (see page 829)	Win32, Win64, MacOS, iOS	Use the TADPhysTDBXDriverLink component to link the dbExpress v 4 bridge driver to an application (more (see page 194)).

FireDAC Monitor Link Components

An application will use one of the monitor components, when it is needed tracing capabilities.

Name	Platforms	Description
TADMoniFlatFileClientLink	All	Use the TADMoniFlatFileClientLink component to link tracing to text file capabilities to an application and setup it (more (see page 174)).
TADMoniRemoteClientLink (see page 723)	Win32, Win64 Additional platforms with FPC.	Use the TADMoniRemoteClientLink component to link ADMonitor tracing capabilities to an application and setup it (more (see page 174)).
TADMoniCustomClientLink	All	Use the TADMoniCustomClientLink component to link custom tracing capabilities to an application and setup it (more (see page 174)).

FireDAC UI Components

Most of the applications will use TADGUIxWaitCursor (see page 683) and TADGUIxLoginDialog (see page 675).

Name	Platforms	Description
TADGUIxAsyncExecuteDialog (see page 669)	VCL, FireMonkey, LCL	The dialog showing a SQL query execution progress (more (see page 86)).
TADGUIxErrorDialog (see page 672)	VCL, FireMonkey, LCL	The dialog displaying FireDAC exceptions (more (see page 44)).
TADGUIxLoginDialog (see page 675)	VCL, FireMonkey, LCL	The dialog allows the users to enter their DB credentials (more (see page 37)).
TADGUIxScriptDialog (see page 680)	VCL, FireMonkey, LCL, Console	The dialog showing a SQL script execution progress (more (see page 88)).
TADGUIxWaitCursor (see page 683)	VCL, FireMonkey, LCL, Console	The component allowing to control the wait cursor.

FireDAC Service Components

An application will use service components to add a specific DBMS facility support.

Name	Platforms	Description
TADADSBackup (see page 725)	Win32, Win64, Linux32, Linux64	The class implementing Advantage backup database service.
TADADSRestore (see page 729)	Win32, Win64, Linux32, Linux64	The class implementing Advantage service restoring a database from a backup.
TADADSUtility (see page 733)	Win32, Win64, Linux32, Linux64	The class implementing Advantage table utilities.
TADASABackup (see page 741)	Win32	The class implementing SQL Anywhere backup database service.
TADASAValidate (see page 745)	Win32	The class implementing SQL Anywhere database validate service.
TADIBBackup (see page 753)	Win32, Win64, Linux32, Linux64, MacOS, iOS	The class implementing FB/IB backup database service.
TADIBNBackup (see page 756)	Win32, Win64, Linux32, Linux64, MacOS	The class implementing Firebird-only backup database service.
TADIBNRestore (see page 759)	Win32, Win64, Linux32, Linux64, MacOS	The class implementing Firebird-only restored database service.
TADIBRestore (see page 761)	Win32, Win64, Linux32, Linux64, MacOS, iOS	The class implementing FB/IB service restoring a database from a backup.
TADIBSecurity (see page 764)	Win32, Win64, Linux32, Linux64, MacOS, iOS	The class implementing FB/IB database security management service.
TADIBTrace (see page 771)	Win32, Win64, Linux32, Linux64, MacOS, iOS	The class implementing Firebird database trace service.
TADIBValidate (see page 774)	Win32, Win64, Linux32, Linux64, MacOS, iOS	The class implementing FB/IB database validate and repair service.
TADMSAccessService (see page 786)	Win32, Win64	The class implementing Microsoft Access database create, drop, compact and repair services
TADSQLiteBackup (see page 805)	Win32, Win64, Linux32, Linux64, MacOS, iOS	The class implementing SQLite backup, restore, copy database functionality (more (see page 135)).
TADSQLiteCollation (see page 811)	Win32, Win64, Linux32, Linux64, MacOS, iOS	The class implementing custom SQLite collation (more (see page 135)).
TADSQLiteFunction (see page 814)	Win32, Win64, Linux32, Linux64, MacOS, iOS	The class implementing custom SQLite function (more (see page 135)).
TADSQLiteSecurity (see page 817)	Win32, Win64, Linux32, Linux64, MacOS, iOS	The class allowing to manage SQLite database encryption (more (see page 135)).
TADSQLiteValidate (see page 821)	Win32, Win64, Linux32, Linux64, MacOS, iOS	The class implementing SQLite database validate service (more (see page 135)).

TADSQLiteRTree (see page 816)	Win32, Win64, Linux32, Linux64, MacOS, iOS	The class implementing custom SQLite RTree (more (see page 135)).

See Also

Class Hierarchy (see page 27)

1.3.3 Databases

FireDAC supports wide range of DBMS.

Description

Native Connections

There "Server and client version" column provides client library names for Windows only. For additional information, including other platforms support, click on the link in "Name" column.

Name	Server and client version	Platforms	Driver link component, Driver implementation unit	DriverID parameter
Advantage Database Server (see page 189)	Server and client v 8.0 and higher.	Win32, Win64, Linux32, Linux64	TADPhysADSDriverLink (see page 737), uADPhysADS (see page 724)	ADS
DataSnap server (see page 193)	DataSnap server built with RAD Studio 2007 and higher. DataSnap client built with RAD Studio XE2 Enterprise and higher.	Win32, Win64, Linux32, Linux64, MacOS, iOS	TADPhysDataSnapDriverLink (see page 749), uADPhysDataSnap (see page 749)	DataSnap
IBM DB2 Server (see page 196)	Server and client v 8.1 and higher.	Win32, Win64, Linux32, Linux64	TADPhysDB2DriverLink (see page 751), uADPhysDB2 (see page 750)	DB2
Interbase Server (see page 198)	Interbase and gds32.dll v 6 and higher.	Win32, Win64, Linux32, Linux64, MacOS, iOS	TADPhysIBDriverLink (see page 777), uADPhysIB (see page 752)	IB
Firebird Server (see page 198)	Firebird and fbclient.dll v 1.5 and higher.	Win32, Win64, Linux32, Linux64, MacOS	TADPhysIBDriverLink (see page 777), uADPhysIB (see page 752)	IB
Microsoft SQL Server (see page 202)	Microsoft SQL Server 2000, ODBC driver from MDAC 2.8. Microsoft SQL Server 2005 and higher, SQL Native Client 2005 and higher. Microsoft SQL Azure, SQL Native Client 2008 and higher. Microsoft LocalDB, SQL Native Client 2012 and higher.	Win32, Win64, Linux32, Linux64, MacOS	TADPhysMSSQLDriverLink (see page 793), uADPhysMSSQL (see page 791)	MSSQL
Microsoft Access database (see page 207)	Microsoft Access 2000 ODBC driver v 4 from MDAC 2.8 and higher. Microsoft Access 2007 ODBC driver v 12 and higher.	Win32, Win64	TADPhysMSAccessDriverLink (see page 791), uADPhysMSAcc (see page 784)	MSAcc

MySQL Server (see page 209)	Server and client 3.21 and higher.	Win32, Win64, Linux32, Linux64, MacOS	TADPhysMySQLDriverLink (see page 794), uADPhysMySQL (see page 794)	MySQL
Oracle Server (see page 215)	Server and client 8.0.3 and higher.	Win32, Win64, Linux32, Linux64, MacOS	TADPhysOracleDriverLink (see page 800), uADPhysOracle (see page 800)	Ora
PostgreSQL (see page 218)	Server and libpq.dll v 7.4 and higher.	Win32, Win64, Linux32, Linux64, MacOS	TADPhysPGDriverLink (see page 803), uADPhysPG (see page 803)	PG
SQLite database (see page 221)	sqlite3_x86.obj / sqlite3_x64.obj or sqlite3.dll v 3 and higher.	Win32,Win64, Linux32, Linux64, MacOS, iOS	TADPhysSQLiteDriverLink (see page 805), uADPhysSQLite (see page 804)	SQLite
Sybase SQL Anywhere (see page 224)	Server and client v 5.0.0 and higher.	Win32, Win64, Linux32, Linux64, MacOS	TADPhysASADriverLink (see page 748), uADPhysASA (see page 740)	ASA

Bridge Connections

Name	Server and client version	Platforms	Driver link component, Driver implementation unit	DriverID parameter
dbExpress data source (see page 194)	dbExpress drivers v 1-3 (Delphi 5 to RAD Studio 2006)	Win32	TADPhysDBXDriverLink (see page 752), uADPhysDBX (see page 829)	DBX
dbExpress data source (see page 194)	dbExpress drivers v 4 (RAD Studio 2007 and higher)	Win32, Win64, MacOS, iOS	TADPhysTDBXDriverLink (see page 829), uADPhysTDBX (see page 829)	TDBX
ODBC data source (see page 213)	ODBC drivers v 1-3	Win32, Win64, Linux32, Linux64, MacOS	TADPhysODBCDriverLink (see page 796), uADPhysODBC (see page 796)	ODBC

Additional Connections

- Blackfish SQL Server (see page 192);
- Informix Dynamic Server;
- MicroFocus Cobol;
- MS SQL Server Compact Edition (see page 206);
- Sybase Adaptive Server Enterprise;
- Unify SQLBase.

See Also

FireDAC Architecture (see page 17), Configuring Drivers (see page 31), Defining Connection (see page 27), Demo Databases (see page 14), Working with DBMS (see page 135)

1.3.4 Programming Tools

FireDAC supports wide range of programming tools.

Description

Programming Tools

Name	Platforms	Comment
Embarcadero Delphi and C++ Builder XE4	• Win32, Win64, MacOS, iOS • VCL, FireMonkey	
Embarcadero Delphi and C++ Builder XE3	• Win32, Win64, MacOS • VCL, FireMonkey	Update Pack 1is required.
Embarcadero Delphi and C++ Builder XE2	• Win32, Win64, MacOS, iOS • VCL, FireMonkey	Update Pack 4, Hot Fix 1 is required.
Embarcadero Delphi and C++ Builder XE	• Win32 • VCL	
Embarcadero Delphi and C++ Builder 2010	• Win32 • VCL	
CodeGear Delphi and C++ Builder 2009	• Win32 • VCL	
CodeGear Delphi and C++ Builder 2007	• Win32 • VCL	RAD Studio December 2007 Update required.
Borland Delphi and C++ Builder 2006	• Win32 • VCL	Update Pack 2 is required.
Borland Delphi 2005	• Win32 • VCL	Update Pack 2 is required.
Borland Delphi 7	• Win32 • VCL	
Borland C++ Builder 6	• Win32 • VCL	See Borland Delphi 6.
Borland Delphi 6	• Win32 • VCL	Update Pack 2, RTL Update Pack 2, RTL Update Pack 3 are required.
Borland C++ Builder 5	• Win32 • VCL	See Borland Delphi 5.
Borland Delphi 5	• Win32 • VCL	Update Pack 1 is required.

FreePascal 2.6.0 & Lazarus 0.9.30.4	• Win32, Win64, Linux32, Linux64, MacOS, iOS • LCL	Read the "Lazarus / FPC (see page 160)" article for details.

See Also

How to recompile FireDAC packages (see page 186)

1.3.5 Class Hierarchy

FireDAC has well designed and structured class hierarchy.

Description

1.4 Working with Connections

A set of articles describing how to setup DB drivers and manage database connections with FireDAC.

1.4.1 Defining Connection

Describes how to store and use FireDAC connection parameters and what is a connection definition. To specify connection parameters an application must use **a connection definition**. The connection definition is a set of parameters. A connection may be also **pooled**.

Description

General

A **connection definition** is a set of parameters that defines how to connect an application to a DBMS using specific FireDAC driver. It is the equivalent of a BDE alias, ADO UDL (stored OLEDB connection string) or ODBC Data Source Name (DSN). For the list of supported DBMS's and corresponding parameters see FireDAC Database Connectivity (see page 188).

FireDAC supports 3 connection definition kinds:

Type	Description	Pros	Cons
Persistent	Has an unique name, is managed by the ADManager (see page 570) and is stored in a connection definition file.	May be defined once and reused across many applications. May be pooled.	The parameters (server address, DB name, etc) are publically visible and may be changed incidentally. ADManager (see page 570) has to be be reactivated or the Delphi IDE has to be be restarted to make a newly added definition visible at design time.
Private	Has an unique name, is managed by the ADManager (see page 570) but is NOT stored in a connection definition file.	Connection definition parameters are not visible "outside" the application. May be pooled.	Application needs to create a private connection definition after each program restart and cannot share it with the other programs. Cannot be created at design time.

Temporary	Has no name, is not stored in a connection definition file and is not managed by the ADManager (see page 570).	The most simple way to create a connection definition - just fill in the TADConnection.Params (see page 334) property. May be created at design time using the TADConnection (see page 279) component editor.	Similar to private. Also cannot be referenced by name and cannot be pooled.

Connection definition file

The persistent connection definitions are stored in an external file - the **connection definition file**. This file has the standard INI text file format. It may be edited by ADExplorer (see page 181) or ADAdministrator (see page 178) utilities at first, manually or by code. By default the file is **$(ADHome)\DB\ADConnectionDefs.ini**.

Note: If you added a new persistent connection definition using ADExplorer or ADAdministrator while the Delphi IDE is running, it will not be visible to the FireDAC design time code. To refresh the persistent connection definitions list, you will need to reactivate ADManager or restart the Delphi IDE.

Sample content of this file:

```
[Oracle_Demo]
DriverID=Ora
Database=ORA_920_APP
User_Name=ADDemo
Password=a
MetaDefSchema=ADDemo
;MonitorBy=Remote

[MSSQL_Demo]
DriverID=MSSQL
Server=127.0.0.1
Database=Northwind
User_Name=sa
Password=
MetaDefSchema=dbo
MetaDefCatalog=Northwind
MonitorBy=Remote
```

An application can specify a connection definition file name in the ADManager.ConnectionDefFileName property. FireDAC searches for a connection definition file in the following places:

- If ConnectionDefFileName (see page 371) is specified:
  - search for a file name without a path, then look for it in an application EXE folder.
  - otherwise just use a specified file name.
- If ConnectionDefFileName (see page 371) is not specified:
  - look for **ADConnectionDefs.ini** in an application EXE folder.
  - If the file above is not found, look for a file specified in the registry key **HKCU\Software\da-soft\FireDAC\ConnectionDefFile**. By default it is **$(ADHome)\DB\ADConnectionDefs.ini**.

Note: At design time, FireDAC ignores the value of the ADManager.ConnectionDefFileName, and looks for a file in a Delphi Bin folder or as specified in the registry. If the file is not found, an exception is raised.

If ADManager.ConnectionDefFileAutoLoad (see page 432) is True, a connection definition file is loading automatically. Otherwise it must be loaded explicitly by calling the ADManager.LoadConnectionDefFile (see page 386) method before the first usage of the connection definitions. For example, before setting TADConnection.Connected (see page 321) to True.

Creating a persistent connection definition

A persistent connection definition may be created using ADExplorer (see page 181) or ADAdministrator (see page 178). Here, we will show you how to do that in code: **$(ADHome)\Samples\Comp Layer\TADConnection\ConnectionDefs**.

The following code snippet creates a connection definition named "MSSQL_Connection", which has all required parameters to connect to the Microsoft SQL Server running locally, using the OS authentification (SSPI):

```
uses
  uADCompClient, uADStanIntf;
var
  oDef: IADStanConnectionDef;
begin
  oDef := ADManager.ConnectionDefs.AddConnectionDef;
  oDef.Name := 'MSSQL_Connection';
  oDef.DriverID := 'MSSQL';
  oDef.Server := '127.0.0.1';
  oDef.Database := 'Northwind';
  oDef.OSAuthent := True;
  oDef.MarkPersistent;
  oDef.Apply;
  ......................
  ADConnection1.ConnectionDefName := 'MSSQL_Connection';
  ADConnection1.Connected := True;
```

The ADManager (see page 570) is a global instance of the FireDAC connection manager. Its property ConnectionDefs (see page 372): IADStanConnectionDefs is a collection of the persistent and private connection definitions. The **AddConnectionDef** method adds a new connection definition. The **MarkPersistent** method marks a connection definition as persistent. The **Apply** method saves a connection definition to a connection definition file. Without the **MarkPersistent** call, the connection definition will be private.

Creating a private connection definition

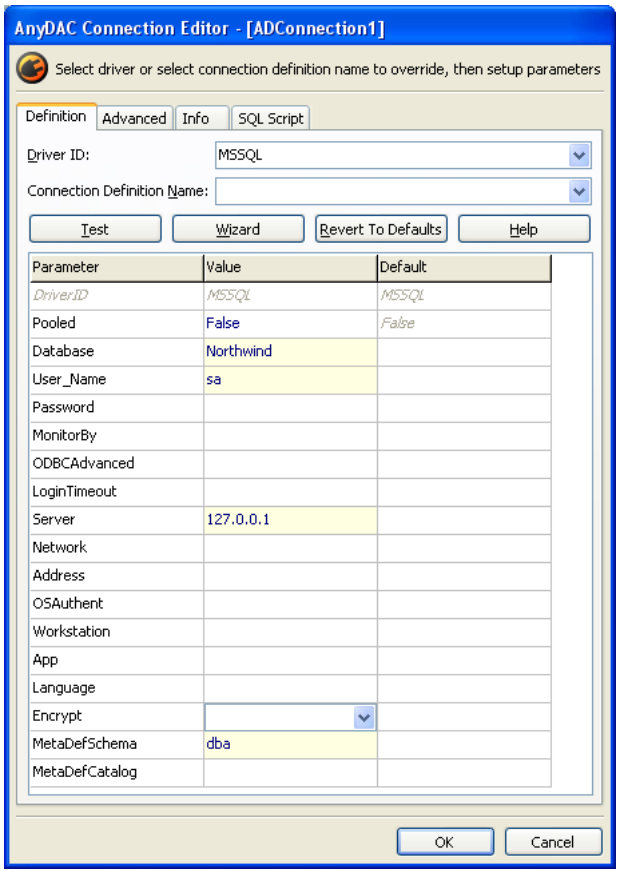
A private connection definition may be created only in code. The code is similar to the one above, but without the **MarkPersistent** call.

Also, you can use a technique similar to BDE:

```
var
  oParams: TStrings;
begin
  oParams := TStringList.Create;
  oParams.Add('Server=127.0.0.1');
  oParams.Add('Database=Northwind');
  oParams.Add('OSAuthent=Yes');
  ADManager.AddConnectionDef('MSSQL_Connection', 'MSSQL', oParams);
  ......................
  ADConnection1.ConnectionDefName := 'MSSQL_Connection';
  ADConnection1.Connected := True;
```

Creating a temporary connection definition

A temporary connection definition may be created at design-time using FireDAC Connection Editor. For that double click on a TADConnection (see page 279) to invoke the editor:



Or at run-time in code by filling the TADConnection.Params (see page 289) property. This is the simplest way to create a connection definition.

```
ADConnection1.DriverName := 'MSSQL';
ADConnection1.Params.Add('Server=127.0.0.1');
ADConnection1.Params.Add('Database=Northwind');
ADConnection1.Params.Add('User_name=sa');
ADConnection1.Connected := True;
```

Editing a connection definition

An application may need an ability to create and edit a connection definition at run-time using standard FireDAC Connection Editor dialog. To edit a temporary connection definition stored in TADConnection use the code:

```
uses
  uADGUIxFormsfConnEdit;
...
if TfrmADGUIxFormsConnEdit.Execute(ADConnection1, '') then
  ADConnection1.Connected := True;
```

To edit a connection definition represented as FireDAC connection string use the code:

```
uses
  uADGUIxFormsfConnEdit;
...
var
  sConnStr: String;
...
sConnStr := ADConnection1.ResultConnectionDef.BuildString();
if TfrmADGUIxFormsConnEdit.Execute(sConnStr, '') then begin
  ADConnection1.ResultConnectionDef.ParseString(sConnStr);
  ADConnection1.Connected := True;
end;
```

See Also

ADAdministrator utility (see page 178), ADExplorer utility (see page 181), FireDAC Database Connectivity (see page 188), Multi Threading (see page 46), Path Variables (see page 163)

1.4.2 Configuring Drivers

Describes how to configure the FireDAC DBMS drivers, including how to specify a DBMS client library. To link an FireDAC driver to an application, specify DBMS client to the driver and other optional parameters, an application may use a **TADPhysXXXDriverLink** components and/or external configuration file.

Description

General

To adjust a driver's behavior, you can use a driver configuration file or the **TADPhysXXXDriverLink** components. Most of the driver configuration file discussion also applies to the **TADPhysXXXDriverLink** components.

Driver configuration file

A driver configuration file is a standard INI text file. It may only be edited manually. Basically, the driver configuration file allows you to:

- adjust parameters of a base driver (section name = base driver ID);
- introduce a new virtual driver, which is a set of the base driver parameters saved under a new name (section name = new virtual driver ID).

FireDAC searches for the driver configuration file in the following places:

- **ADDrivers.ini** in the application EXE folder.
- If the file above is not found, it looks for the file specified in the registry key HKCU\Software\da-soft\FireDAC\DriverFile. By default, this is **$(ADHome)\DB\ADDrivers.ini**.

At design time, FireDAC will use the file in the Delphi Bin folder or as specified in the registry.

Base and Virtual Drivers

If the section name in the driver configuration file is the base driver ID (for example, "ASA" or "Ora"), the base driver uses the parameter values specified in the corresponding section.

If the section name is not an ID of any of the base drivers (for example, "Ora815" or "MySQL510_Embedded"), FireDAC registers a new virtual driver. The virtual driver uses the implementation of the base driver specified by the BaseDriverID parameter and the specified parameter values. A virtual driver ID then may be used as the value of the DriverID connection definition parameter. A virtual driver allows the application:

- to work with different versions of the DBMS client software in different connections, established in the same application run, for example to connect to Interbase and Firebird servers;
- to choose an explicit version of the DBMS client software, for example to use one of the many LIBMYSQL.DLL's installed on workstation;
- to set up drivers, which require parameters to be specified, for example MySQL Embedded server, which requires to specify arguments.

Driver Configuration Parameters

The following parameters may be specified:

Parameter	Description	Applied to Drivers
VendorHome[| Win32 | Win64 | MacOS32 | MacOS64 | UIX32 | UIX64]	The **Oracle Home** name.	Ora
	The base installation path. For example: C**:\ib\ib2007**.	IB
	The base installation path for normal server. For example: **C:\MySQL\MySQL5-1-7**. Or the path to LIBMYSQLD.DLL for embedded server. For example **C:\MyAPP\MySQL**.	MySQL
VendorLib[| Win32 | Win64 | MacOS32 | MacOS64 | UIX32 | UIX64]	The DBMS client software API DLL name. For example: libmysql510.dll.	• Ora • IB/FB • MySQL
ODBCDriver	ODBC driver name. For example: **Adaptive Server Anywhere 8.0**.	All ODBC based
ODBCAdvanced	ODBC driver additional parameters. The string consists of ODBC driver parameters separated by ?;?. Check the vendor documentation for possible values.	All ODBC based
EmbeddedArgs	MySQL embedded server arguments. The string consists of MySQL server arguments separated by ?;?. Check the vendor documentation for possible values. For example: **--datadir=./data;--language=./;--skip-innodb**.	MySQL
EmbeddedGroups	MySQL embedded server configuration file groups.	MySQL
NLSLang	Oracle NLS_LANG environment variable value.	Ora
TNSAdmin	Oracle TNS_ADMIN environment variable value.	Ora

For non-ODBC based drivers, if the VendorLib is specified, FireDAC will use the specified DLL. If the VendorHome is specified, the DLL with default name from the Bin subfolder will be used. If none is specified, a DLL with a default name from:

- primary Oracle Home for Ora driver,
- most left folder in PATH environment variable, containing the DLL, for other drivers

will be used. Additionally may be specified a suffix, designating a platform.

For ODBC based drivers, if the ODBCDriver is specified, FireDAC will use the specified driver. If it is not specified, a default driver name will be used.

Sample driver configuration file content

```
[ASA]
; ASA base driver will use specified ODBC driver
ODBCDriver=Adaptive Server Anywhere 8.0

[Ora815]
; Ora815 virtual driver will use specified Oracle Home
BaseDriverID=Ora
VendorHomeWin32=OraHome815
VendorHomeWin64=OraHome815_64

[MySQL327]
; MySQL327 virtual driver will use specified LIBMYSQL.DLL
BaseDriverID=MySQL
VendorLib=c:\LIBMYSQL327.DLL

[MySQL510_Embedded]
; MySQL510_Embedded virtual driver will use specified MySQL embedded library and arguments
```

```
BaseDriverID=MySQL
VendorLib=c:\LIBMYSQLD.DLL
EmbeddedArgs=--datadir=./data;--language=./;--skip-innodb;--skip-networking

[MSSQL_2000]
; MSSQL_2000 virtual driver will use specified ODBC driver
BaseDriverID=MSSQL
ODBCDriver=SQL SERVER
ODBCAdvanced=

[FB21]
; FB21 virtual driver will use specified Firebird client library
BaseDriverID=IB
VendorLibWin32=C:\ib\fb21\bin\fbclient.dll
VendorLibWin64=C:\ib\fb21_64\bin\fbclient.dll

[FB21_Embedded]
; FB21_Embedded virtual driver will use specified Firebird client library
BaseDriverID=IB
VendorLib=C:\ib\fb21_embed\bin\fbembed.dll
```

Configuring drivers in code

You can also configure FireDAC drivers in application code at runtime. To do that, drop the appropriate TADPhysXXXDriverLink components to your form. It has the same named properties as the parameters in the driver configuration file. The link component properties must be set before opening a first connection to a DBMS through this driver. The following code sample shows how to configure the Firebird driver at runtime:

```
interface

uses
  ..., uADPhysIB;

type
  TForm1 = class(TForm)
  ......
    ADPhysIBDriverLink1: TADPhysIBDriverLink;
    ADConnection1: TADConnection;
    procedure FormCreate(Sender: TObject);
  ......
  end;

implementation

{$R *.dfm}

procedure TForm1.FormCreate(Sender: TObject);
begin
  ADPhysIBDriverLink1.VendorLib := 'C:\ib\fb21_embed\bin\fbembed.dll';

  ADConnection1.ConnectionDefName := 'IB_Demo';
  ADConnection1.Connected := True;
end;
```

When a connection using a driver was established and an application needs to switch to the other DBMS client:

- close all connections on this driver;
- call the driver link Release (see page 780) method;
- change required link properties.

The next connection using this driver will use the new link properties. The following code sample shows how to change the Firebird driver configuration at runtime:

```
ADConnection1.Close;
ADPhysIBDriverLink1.Release;
ADPhysIBDriverLink1.VendorLib := 'C:\fbclient.dll';
ADConnection1.Open;
```

Note: Although it is possible to use link components to configure drivers at design time, we do not recommend it, because it is hard to make that a module with a link component will be loaded before any other module with the FireDAC components. So, the link component will configure the driver properly.

See Also

FireDAC Database Connectivity (see page 188), FireDAC FAQ (see page 227)

1.4.3 Setting Options

Describes why the set of options makes FireDAC a flexible database framework and how to use the options. FireDAC offers the wide range of the options organized into an hierarchical options system. Most of the options may be leaved with their default values.

Description

The FireDAC options are organized into five groups:

- FetchOptions (see page 849) - Fetch options control, how the components and Phys layer commands fetch data from a DBMS. For example, it is possible to fetch all records at once, or fetch records on demand.
- FormatOptions (see page 860) - Format options control, how DBMS data types will be mapped to the FireDAC data types and backward. For example, a programmer may setup a mapping for Oracle NUMBER (38) onto dtBCD or onto dtInt64. Read Data Type Mapping (see page 35) for more details.
- UpdateOptions (see page 896) - Update options control, how FireDAC will post updates to DBMS. For example, during an update FireDAC can update all fields in a table or only the changed ones.
- ResourceOptions (see page 874) - Resource options control, how system resources are used, dataset persistence and other. For example, an FireDAC Phys layer command can be performed asynchronously or blocked.
- TxOptions (see page 891) - Transaction options control, how transactions are performed. For example, perform them in a ReadCommitted isolation mode. Note, that TxOptions does not use option value inheritance.

Because FireDAC introduces a lot of options, setting up each dataset or command may do the programming complex and error prone. FireDAC solves this issue by introducing a parent-child option values inheritance model. The option values are propagated from a parent to a child (top-down). If a lower level has no option value assigned explicitly, a value will be taken from a higher level, where a value is assigned or from the top-most level. The FireDAC has levels:

- Manager (TADCustomManager (see page 365), IADPhysManager) level;
- Connection (TADCustomConnection (see page 318), IADPhysConnection) level;
- Command (TADCustomCommand (see page 291) / IADPhysCommand) / Dataset (TADCustomQuery (see page 394), TADStoredProc (see page 514), etc) level.

A Manager is the top-most level, a Connection is the intermediate and a Command / Dataset is the bottom level. So, by setting any particular Manager or Connection option, all Datasets will inherit its value. This is true as long as a programmer has not explicitly assigned a value to the Dataset option.

The TxxxOptions.AssignedValues (see page 851) flags controls the inheritance. If some option is changed at this level, then corresponding flag is included into AssignedValues. If flag is not included, then an option value is inherited from the more high level. Excluding flag from AssignedValues makes option inheriting its value from the more high level again.

The options may be setup for a persistent connection definition using ADExplorer (see page 181). After connection establishment, the options setup will be applied to the TADCustomConnection (see page 318) options.

See Also

Data Type Mapping (see page 35), uADStanOptions

Example 1

The mapping of data types from the DBMS types to the client types defined in FormatOptions is inherited by all Commands from their Connection.

```
with oConnection.Options.FormatOptions do begin
```

```
    OwnMapRules := True;
    MapRules.Clear;
    with MapRules.Add do begin
      PrecMax := 19;
      PrecMin := 4;
      SourceDataType := dtFmtBCD;
      TargetDataType := dtCurrency;
    end;
  end;
```

Example 2

A Data Warehouse application may setup high-speed fetching mode, using FetchOptions of the Manager level. So, all connection and all their commands will inherit these options.

```
with ADManager.FetchOptions do begin
  Items := [];
  Cache := [];
  RowsetSize := 1000;
end;
```

1.4.3.1 Data Type Mapping

FireDAC offers flexible adjustable data type mapping system, allowing to simplify migration to FireDAC or optimize data representation.

Description

General

The data type mapping allows to map:

- result set column data types returned by an FireDAC driver, to the data types preferred by application;
- command parameters data types defined by an application, to the driver supported data types.

The data type mapping is useful for:

- creation of the data type schema compatible with other data access components, when migrating application from these components to the FireDAC;
- creation of the data type schema compatible across few supported DBMS, when developing a cross-DBMS application;
- mapping the data types not supported by an application into the supported ones;
- mapping the generalized data types supported by an driver into the more specialized / convenient ones;
- optimizing the memory consumption, by specifying the more optimal data types.

Lets consider SELECT of numeric column from an Oracle table and how it may be mapped:

DDL	Driver data type	Preferred data type
NUMBER(2,0)	dtBcd, Precision=2, Scale=0	dtSByte
NUMBER(4,0)	dtBcd, Precision=4, Scale=0	dtInt16
NUMBER(8,0)	dtBcd, Precision=8, Scale=0	dtInt32
NUMBER(18,4)	dtBcd, Precision=18, Scale=4	dtCurrency

There you see, that Oracle driver returns some unified data type (dtBcd / dtFmtBCD) for all possible NUMBER(X,Y) database types. But an application may prefer to use a more specialized / convenient data type, like a dtInt32.

Defining

FireDAC applies the mapping rules at a command preparation. After the command is prepared, the rule changes will have no effect. If data type conforms to few rules, then only first one will be used. MaxStringSize (see page 866), MaxBcdPrecision (see page 865), MaxBcdScale (see page 866) properties are applied to source data type before

mapping rules.

To define the data type mapping an application must set FormatOptions.OwnMapRules (see page 866) to True and fill MapRules (see page 865) collection. Each item in collection is of TADMapRule (see page 870) class and represents a single mapping rule. In case of a result set column, each rule defines a transformation of a source data type, returned by a driver, into a target one, preferred by an application. In case of a command parameter, rule defines a transformation of a target data type, specified by an application, into a source data type, supported by a driver. All rules, excluding the name based ones work bidirectionally for both cases.

Each rule is defined by the TADMapRule properties:

Properties	Description
PrecMin (see page 871) / PrecMax (see page 871)	Defines the range of source data type numeric precision.
ScaleMin (see page 872) / ScaleMax (see page 872)	Defines the range of source data type numeric scale.
SizeMin (see page 873) / SizeMax (see page 872)	Defines the range of source data type string length.
SourceDataType (see page 873)	Source data type.
TargetDataType (see page 873)	Target data type.
NameMask (see page 871)	Column name mask.

If a precision, scale or size is not used by the rule, then its value must be -1 (default value). If a source data type conforms to some rule, then a column data type will be defined using corresponding TargetDataType (see page 873).

Example

To define mapping rules for sample above, use code:

```
with ADConnection1.FormatOptions do begin
  OwnMapRules := True;
  with MapRules.Add do begin
    ScaleMin := 0;
    ScaleMax := 0;
    PrecMin := 0;
    PrecMax := 2;
    SourceDataType := dtBcd;
    TargetDataType := dtSByte;
  end;
  with MapRules.Add do begin
    ScaleMin := 0;
    ScaleMax := 0;
    PrecMin := 3;
    PrecMax := 4;
    SourceDataType := dtBcd;
    TargetDataType := dtInt16;
  end;
  with MapRules.Add do begin
    ScaleMin := 0;
    ScaleMax := 0;
    PrecMin := 5;
    PrecMax := 8;
    SourceDataType := dtBcd;
    TargetDataType := dtInt32;
  end;
  with MapRules.Add do begin
    ScaleMin := 4;
    ScaleMax := 4;
    PrecMin := 18;
    PrecMax := 18;
    SourceDataType := dtBcd;
    TargetDataType := dtCurrency;
  end;
end;
```

1.4.4 Establishing Connection

Describes how to open and close connection to a DBMS using FireDAC. To open a connection to a database FireDAC offers the **TADConnection** component.

Description

General

After a connection definition is created (see page 27), the connection may be established to a database. In general, there are two ways:

- explicitly, by setting TADCustomConnection.Connected (see page 321) to True or calling one of the Open (see page 354) methods;
- implicitly, by performing any action requiring to talk to a DBMS. For example, by setting linked TADQuery Active property to True. Note, that ResourceOptions.AutoConnect (see page 887) must be True, otherwise an exception will be raised.

FireDAC offers few TADCustomConnection.Open (see page 353) methods additionally to the Connected property. These method allows to use an FireDAC connection string, which is a string in the form of *param=value[;...param=value]*. For example:

```
ADConnection1.Open('DriverID=SQLite;Database=c:\test.sdb;Password=12345');
```

Before the connection will be opened the BeforeConnect event will be fired. After connection establishment - AfterConnect.

Handling connection errors

If connection establishment fails, then an application may analyze failure using one of the approaches:

- using OnError (see page 332) event handler. That is more appropriate when a connection is opened implicitly;
- using **try ... except ... end** syntax. That is the best approach with explicit connection establishment. For example:

```
try
  ADConnection1.Connected := True;
except
  on E: EAbort do
    ; // user pressed Cancel button in Login dialog
  on E: EADDBEngineException do
    case E.Kind of
    ekUserPwdInvalid: ; // user name or password are incorrect
    ekUserPwdExpired: ; // user password is expired
    ekServerGone: ;     // DBMS is not accessible due to some reason
    else                // other issues
    end;
end;
```

Note, that Login dialog is automatically handling the ekUserPwdInvalid error kind, by suggesting to the user to repeat the login. To disable this handling set TADGUIxLoginDialog (see page 675).LoginRetries (see page 678) to -1. And the ekUserPwdExpired, by allowing to the user to enter the new password.

Also, if the connection recovery is setup, then ekServerGone error kind will lead to bringing a connection to an initially offlined state (see page 40). Alternatively the Ping (see page 354) method may be used to avoid ekServerGone error and make the connection active, when a DBMS is available.

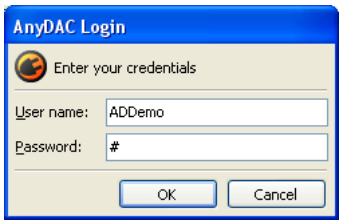
See "Handling Errors (see page 44)" for more details.

Using the Login dialog

The GUI application may use TADGUIxLoginDialog (see page 675) component to allow the end users to enter the database credentials. The login dialog may be bind by one of the ways:

- Drop the TADGUIxLoginDialog (see page 675) component to a form. No additional setup is required. This dialog will be a default Login dialog for an application.
- Drop the TADGUIxLoginDialog (see page 675) component to a form and set TADCustomConnection.LoginDialog (see page 329) to this dialog. The dialog will be used privately by this connection.

The Login dialog will be automatically invoked by the TADCustomConnection (see page 318), when LoginPrompt (see page 329) = True:



Using VisibleItems (see page 679) property you may specify which connection definition parameters to show to the end user and how to name them. The last option allows to localize the Login dialog. For example, German speaking SQL Server developers may specify:

```
with ADGUIxLoginDialog1.VisibleItems do begin
  Clear;
  Add('Server');
  Add('User_name=Benutzer');
  Add('Password=Kennwort');
  Add('OSAuthent');
end;
ADConnection1.LoginDialog := ADGUIxLoginDialog1;
ADConnection1.Connected := True;
```

When a DBMS supports password expiration, the password is expired and ChangeExpiredPassword (see page 677) is True, the dialog will ask for a new password.

Closing Connection

The connection may be closed by one of the two ways:

- explicitly, by setting TADCustomConnection.Connected (see page 321) to False.
- implicitly, when the connection object has no more active commands and datasets, and the ResourceOptions.KeepConnection (see page 889) is False.

Before connection will be closed FireDAC will finish the active transactions, if any. Use TxOptions.DisconnectAction (see page 894) to control the performed action.

Also before that the BeforeDisconnect event will be fired. After connection closing - AfterDisconnect.

See Also

Handling Errors (see page 44), Defining Connection (see page 27), Offlining Connection (see page 40), TADCustomConnection Class (see page 318), TADGUIxLoginDialog Class (see page 675)

1.4.5 Recovering Connection

Describes how to work in an unstable environment. FireDAC allows an application to recover from a connection failure.

Description

General

A database application may work within a **non stable environment**, where the network may be physically disconnected or the DBMS server may be restarted. Application need to seamlessly recover from such interruptions and continue to communicate with the DBMS.

The FireDAC automatic connection recovery allows you to detect when a connection has been lost, and to properly respond and recover from this situation. The lost is discovered only when some DB action is performed, like Open, ExecSQL (see page 397) or Ping (see page 354). Then DBMS driver raises EADDBEngineException (see page 833) with Kind (see page 835) = ekServerGone.

The most simple way to verify a connection status and/or keep the connection alive is to call TADCustomConnection (see page 318).Ping (see page 354) method. The Ping method may be used even when a connection is closed, to see when a connection may be established.

Known limitations:

DBMS	Description
Advantage Database	Not supported for a local free connection.
dbExpress bridge driver	• FireDAC may fail to detect ekServerGone; • Ping method is not supported.
Informix	Required to set DirectExecute to True.
Microsoft Access	Not supported.
MySQL	To mimimize call delays when network connection is losted, consider to adjust ReadTimeout (see page 209) and WriteTimeout connection definition parameters.
ODBC bridge driver	• FireDAC may fail to detect ekServerGone; • Ping method is not supported.
SQLite	Not supported.

Controlling connection recovery

To enable automatic connection recovery set ResourceOptions.AutoReconnect (see page 887) to True. And optionally use the TADCustomConnection event handlers:

- OnRecover (see page 333) - to respond to the connection lost event and provide the next action to FireDAC;
- OnLosted (see page 333) - fires when a connection was lost and not recovered;
- OnRestored (see page 334) - fires when a connection was lost and recovered.

The response from OnRecover (see page 333) event handler may be to re-establish the connection, go into the offline mode (see page 40), or simply close the connection. When OnRecover (see page 333) is not specified, FireDAC will try to re-establish connection. For example:

```
procedure TForm1.ADConnection1Recover(ASender: TObject;
  const AInitiator: IADStanObject; AException: Exception;
  var AAction: TADPhysConnectionRecoverAction);
var
  iRes: Integer;
begin
  iRes := MessageDlg('Connection is lost. Offline - yes, Retry - ok, Fail - Cancel',
    mtConfirmation, [mbYes, mbOK, mbCancel], 0);
  case iRes of
  mrYes:    AAction := faOfflineAbort;
  mrOk:     AAction := faRetry;
  mrCancel: AAction := faFail;
```

```
  end;
  Log('Connection is recovering');
end;
```

We strongly recommend to not close FireDAC manager from within OnRecover, OnLosted and OnRestored event handlers, as that can lead to unexpected problems.

Preparing application

When a connection is recovered, the following states are lost:

- active transactions are rolled back, and execution is continued from the failed statement;
- not yet fetched result sets are trimmed and TADDataSet.SourceEOF (see page 608) is set to True;
- database session states are lost, including Oracle package states and session variables;
- registered database alerts are unregistered.

To prepare your application to work in unstable environment and minimize failure affects, consider to setup:

- FetchOptions.Mode (see page 857) to fmAll;
- FetchOptions.RowsetSize (see page 859) to 200-500;
- FetchOptions.AutoClose (see page 851) to True (default);
- TxOptions.AutoCommit (see page 893) to True;
- ResourceOptions.AutoReconnect (see page 887) to True.

The application may consider to use Offline connection mode (see page 40) and CachedUpdates (see page 592).

See Also

TADCustomConnection Class (see page 318), Establishing Connection (see page 37), Offlining Connection (see page 40)

1.4.6 Offlining Connection

Describes how to use FireDAC offline mode allows to work with data without a persistent connection to a database.

Description

General

The FireDAC offline mode is similar to a multi-tier client, where most of time a client is disconnected from a database. The connection is active only when a client needs to exchange data with a database. That is useful when an application works in an unstable environment or requires to preserve the DBMS resources.

In offline mode a connection to a database is closed. But the datasets are opened.

Controlling offlining

To bring a connection to offline mode use one of the options:

- call the TADCustomConnection.Offline (see page 352) method;
- return AAction=amOfflineXxxx in TADCustomConnection.OnRecover (see page 333) method.

FireDAC does not allow to bring a connection to the offline mode automatically, as it does not know when the next time an application needs to contact a database. So, that must be done by the application code. Before the connection will be set to the offline mode, active datasets with not yet fetched result sets will perform an action specified by the FetchOptions.AutoFetchAll (see page 852) property.

To bring a connection to online mode use one of the following options:

- explicit call of the TADCustomConnection.Online (see page 353) method;
- implicit activation, when a connection needs to talk to a DBMS, either executing a command, either posting updates, etc. Note, that ResourceOptions.AutoConnect (see page 887) must be True, otherwise an exception will be raised.

If a dataset has ResourceOptions.PersistentFileName (see page 843) property specified, then at Open call, the connection is not required for the dataset. It will load the content from the specified file.

Note, bringing connection to offline and then to online mode is similar to recovering a broken connection in aspect that a database session state is lost. Read Recovering Connection (see page 38) article for more details.

See Also

TADCustomConnection Class (see page 318), Establishing Connection (see page 37), Recovering Connection (see page 38)

1.4.7 Managing Transactions

Describes how to manage DBMS transactions using FireDAC. To handle database transactions FireDAC offers the **TADConnection** and **TADTransaction** components.

Description

General

By default the FireDAC application is working in auto-commit mode, where a transaction is automatically started by FireDAC when it is required, and is committed on the successful command execution or rolled back on a failure. The auto-commit is controlled by the TADTxOptions (see page 891).AutoCommit (see page 893) property. The auto-commit mode is simple to use for the application, but it:

- slowdowns multiple updates to a database;
- do not allow to perform few database operation in a single transaction;
- cannot be prolonged in a time.

Alternatively the application may use the explicit transaction control. For that use the TADConnection (see page 279) methods StartTransaction (see page 358), Commit (see page 340), Rollback (see page 356). Or to use the TADTransaction (see page 561) component. Note, the TADTransaction component usage is optional in FireDAC.

FireDAC offers the Isolation (see page 894), ReadOnly (see page 895), Params (see page 895) properties allowing to control the transaction mode. They apply as to auto-commit as to explicit transactions. Not all modes may be supported by DBMS, for example read-only mode, then FireDAC will use less restrictive mode. Note, that all settings will be applied to the next transactions only.

The standard code using explicit transaction looks like:

```
ADConnection1.StartTransaction;
try
  ADQuery1.ExecSQL;
  ....
  ADQuery1.ExecSQL;
  ADConnection1.Commit;
except
  ADConnection1.Rollback;
  raise;
end;
```

The TADTransaction (see page 561) component wraps the transaction control functionality into a component. Practically it offers the same transaction functionality as TADConnection, but allows to group commands and datasets by linking them to a specific transaction object. At first that refers to the multiple active transactions support, that is Firebird / Interbase servers (see page 198) feature.

The TADTxOptions.Params (see page 895) may be used to specify DBMS specific transaction attributes. At moment only Firebird / Interbase driver supports such attributes. Each attribute must be specified on a separate line. It will correspond to isc_tpb_<attribute name> transaction parameters.

Nested transactions

Although none of the supported DBMS's supports the nested transactions, they are emulated by FireDAC using the savepoints. Means, the nested StartTransaction call will not start a new transaction and will not raise an exception, but will put a savepoint. If a DBMS does not support savepoints, then "Capability is not supported" exception will be raised.

To disable nested transaction set TADTxOptions.EnableNested (see page 894) to False. Then nesting StartTransaction call will raise an exception.

Corresponding Commit call releases a savepoint and Rollback rolls back to a savepoint. For example:

```
// start new transaction
ADConnection1.StartTransaction;
try
  ADQuery1.ExecSQL;
  ....

  // set savepoint
  ADConnection1.StartTransaction;
  try
    ADQuery1.ExecSQL;
    ....

    // release savepoint
    ADConnection1.Commit;
  except
    // rollback to savepoint
    ADConnection1.Rollback;
    raise;
  end;

  // commit transaction
  ADConnection1.Commit;

except
  // rollback transaction
  ADConnection1.Rollback;
  raise;
end;
```

Note, that nested transaction will use the settings of the most top transaction.

Continuous transactions

The CommitRetaining (see page 341) and RollbackRetaining (see page 357) methods are similar to the Commit and Rollback methods, but they do not finish the transaction. So, it remains active after these calls. The Firebird / Interbase servers are supporting this feature on the DBMS core level. For all other DBMS's this feature is emulated using the Commit / Rollback and StartTransaction calls.

Multiple active transactions

The Firebird / Interbase server supports multiple active transactions on the DBMS core level. That means, that some commands may be performed in one transaction context, others in the second transaction context, etc. To support this feature FireDAC offers TADTransaction (see page 561) component. It single instance allows to handle single transaction in each moment of the time.

The TADCustomConnection (see page 318) properties may be used to setup default transaction objects:

- Transaction (see page 337) - the default transaction object for all commands;
- UpdateTransaction (see page 338) - the default transaction for all update commands, used to post updates from the FireDAC datasets. Note, the UpdateTransaction will not be used, for example, for a UPDATE query explicitly specified for TADQuery component.

The TADCustomQuery (see page 394) and other components have similar properties:

- Transaction (see page 507) - the explicit transaction object to use to execute the SQL query;
- UpdateTransaction (see page 507) - the explicit transaction object to use to post updates from the dataset.

In general, the good practice to setup transaction objects for Firebird / Interbase application:

```
UpdateTransaction: TADTransaction;
ReadTransaction: TADTransaction;
...
// setup transaction for updating commands: read_committed, rec_version, nowait
UpdateTransaction.Connection := ADConnection1;
ADConnection1.UpdateOptions.LockWait := False;
UpdateTransaction.Options.ReadOnly := False;
UpdateTransaction.Options.Isolation := xiReadCommitted;
...
ReadTransaction.Connection := ADConnection1;
ReadTransaction.Options.ReadOnly := True;
ReadTransaction.Options.Isolation := xiReadCommitted;
...
SelectQuery.Transaction := ReadTransaction;
SelectQuery.UpdateTransaction := UpdateTransaction;
```

Note, you can use few TADTransaction's for other DBMS's. Then all TADTransaction's will share the same transaction.

Transactions and cursors

A DBMS associates an open cursor with the transaction context, where it was opened. When the transaction finishes, the DBMS may invalidate the active cursors. The exact behavior depends on the DBMS:

DBMS	Action
Microsoft Access	Invalidates a cursor on StartTransaction / Commit / Rollback.
Firebird / Interbase	Invalidates a cursor on Commit / Rollback.
IBM DB2	Invalidates a cursor on Rollback.
MySQL	Invalidates a cursor on StartTransaction / Commit / Rollback.
Oracle	<nothing>
PostgreSQL	Invalidates a cursor on Commit / Rollback.
SQLite	Invalidates a cursor on Rollback.
SQL Anywhere	<nothing>
SQL Server	Invalidates a cursor on StartTransaction / Commit / Rollback.

When FireDAC discovers a transaction control command that will lead to the cursor invalidation, FireDAC performs an actions specified by the FetchOptions.AutoFetchAll (see page 852) and releases the cursor.

Note, the Firebird / Interbase servers are invalidating the prepared statement on the transaction finish. So, the auto-commit mode may lead to performance degradation on these DBMS's.

See Also

TADCustomConnection, TADCustomTransaction, Executing Command (see page 66)

1.4.8 Handling Errors

Describes how to handle database errors with FireDAC.

Description

General

The EADDBEngineException (see page 833) class is the base class for all DBMS exceptions. A single exception object is a collection of database errors (see page 837), accessible through EADDBEngineException.Errors (see page 834)[] property and represented by the TADDBError (see page 837) class.

FireDAC combines "personalization" and unification of EADDBEngineException (see page 833) exception and TADDBError (see page 837) error classes. "Personalization" means that a driver may have its own exception and error classes, which contain information specific to the DBMS:

DBMS	Exception class	Error class
Advantage Database	uADPhysADSWrapper.EADSNativeException	uADStanError (see page 831).TADDBError (see page 837)
Sybase SQL Anywhere	uADPhysASAWrapper.EASANativeException	uADPhysODBCWrapper.TADODBCNativeError
DataSnap server	uADPhysTDBXBase (see page 830).ETDBXNativeException (see page 830)	uADStanError (see page 831).TADDBError (see page 837)
IBM DB2	uADPhysDB2 (see page 750).EDB2NativeException (see page 750)	uADPhysODBCWrapper.TADODBCNativeError
Firebird / Interbase	uADPhysIBWrapper.EIBNativeException	uADPhysIBWrapper.TADIBError
Microsoft Access	uADPhysMSAcc (see page 784).EMSAccessNativeException (see page 785)	uADPhysODBCWrapper.TADODBCNativeError
Microsoft SQL Server	uADPhysMSSQL (see page 791).EMSSQLNativeException (see page 792)	uADPhysMSSQL (see page 791).TADMSSQLError (see page 793)
MySQL	uADPhysMySQLWrapper.MySQLNativeException	uADPhysMySQLWrapper.TADMySQLError
ODBC	uADPhysODBCWrapper.EODBCNativeException	uADPhysODBCWrapper.TADODBCNativeError
Oracle	uADPhysOracleWrapper.EOCINativeException	uADPhysOracleWrapper.TOCIError
PostgreSQL	uADPhysPGWrapper.EPgNativeException	uADPhysPGWrapper.TADPgError
SQLite	uADPhysSQLiteWrapper.SQLiteNativeException	uADStanError (see page 831).TADDBError (see page 837)
dbExpress v1-3	uADPhysDBExp (see page 751).EDBXNativeException	uADStanError (see page 831).TADDBError (see page 837)
dbExpress v4	uADPhysTDBXBase (see page 830).ETDBXNativeException (see page 830)	uADStanError (see page 831).TADDBError (see page 837)

And TADDBError (see page 837) has ErrorCode (see page 838) property - the native DBMS error code.

"Unification" means that all driver exception classes are inherited from the EADDBEngineException (see page 833) - a single base class that contains driver independent information. And the Kind (see page 835) property - a DBMS independent error code. For example, code for handling a unique key violation may look like this:

```
try
  ADQuery1.ExecSQL('insert into MyTab(code, name) values (:code, :name)', [100, 'Berlin']);
except
  on E: EADDBEngineException do begin
    if E.Kind = ekUKViolated then
```

```
      ShowMessage('Please enter unique value !');
    raise;
  end;
end;
```

Error information

The error information is mainly present by the EADDBEngineException (see page 833) properties:

- Errors (see page 834) - the collection of the TADDBError (see page 837) objects.
- ErrorCount (see page 834) - the number of errors in Errors collection.
- Kind (see page 835) - the DBMS independent error kind.
- Message - the actual error message.

And by the TADDBError properties:

- ErrorCode (see page 838) - DBMS vendor specific error code.
- Kind (see page 838) - the DBMS independent error kind.
- Message (see page 839) - the error message.

To simplify the application debugging or to make exception logging more informative, the EADDBEngineException (see page 833) provides the SQL (see page 835) and Params (see page 835) properties. That is the SQL command with parameter values leaded to the exception.

Also depending on the error area and the DBMS ability to provide the advanced error information, the following properties are useful:

- When a SQL parsing error, then CommandTextOffset (see page 838) returns the offset in the SQL command text.
- When a constraint violation, a DB object alteration failure or some other cases, then the ObjName (see page 839) property returns an database object name.
- When a Array DML error, then RowIndex (see page 839) returns the array row index, to which the error belongs.

Handling exceptions

The exceptions may be processed using one of the ways:

- using try / except / end construction. This is a standard Delphi way to handle exceptions. For example:

```
ADConnection1.StartsTransaction;
try
  ADQuery1.ExecSQL;
  ADConnection1.Commit;
except
  on E: EADDBEngineException do begin
    ADConnection1.Rollback;
    // do something here
    raise;
  end;
end;
```

- setting TADQuery.OnError (see page 261) event handler;
- setting TADConnection.OnError (see page 332) event handler. These ways are good for exception logging or exception "adjusting". For example:

```
procedure TForm1.ADConnection1Error(ASender: TObject; const AInitiator: IADStanObject;
  var AException: Exception);
var
  oExc: EADDBEngineException;
begin
  if AException is EADDBEngineException then begin
    oExc := EADDBEngineException(AException);
```

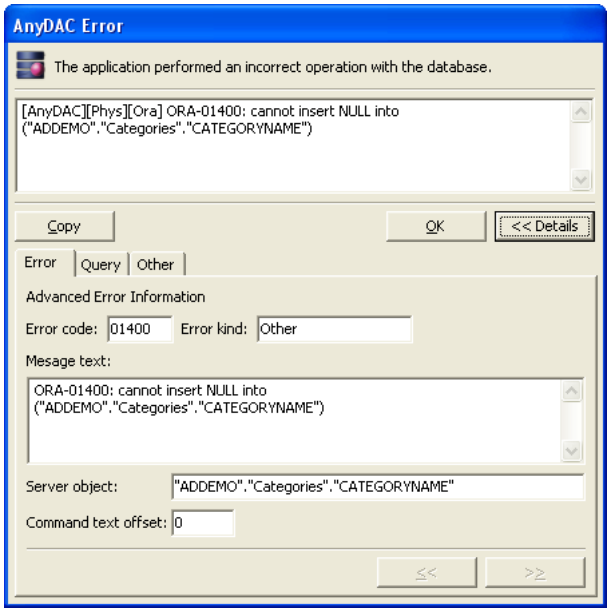
```
    if oExc.Kind = ekRecordLocked then
      oExc.Message := 'Please, try the operation later. At moment, the record is busy'
    else if (oExc.Kind = ekUKViolated) and SameText(oExc[0].ObjName, 'UniqueKey_Orders')
then
      oExc.Message := 'Please, provide the unique order information. It seems, your order
was already put';
  end;
end;

ADConnection1.OnError := ADConnection1Error;
```

- setting TADQuery.OnExecuteError (see page 262) event handler for handling Array DML specific errors;
- setting TADQuery.OnUpdateError (see page 605) event handler for handling updates posting errors;
- setting TADConnection.OnLosted (see page 333), OnRestored (see page 334), OnRecover (see page 333) event handlers for handling connection lost errors.

Using end user error dialog

With help of TADGUIxErrorDialog (see page 672) component, an end user may be notified about errors returned by the database:



To use the dialog, just drop the component somewhere on a form. The dialog will hook TApplication.OnException event handler. And will popup the dialog, when there is an unhandled FireDAC exception. The "Query" page allows to see the SQL command text produced the exception. Pressing Ctrl+C in dialog will put complete exception information into clipboard.

See Also

Recovering Connection (see page 38), Executing Command (see page 66), uADStanError Namespace (see page 831), TADCustomConnection.OnError (see page 332), TADAdaptedDataSet.OnError Event (see page 261)

1.4.9 Multi Threading

Describes how to use FireDAC in multi threading environment.

Description

General

FireDAC is thread-safe, when the following conditions are meat:

- A connection object and all associated with it objects (like TADQuery, TADTransaction, etc) in each moment of time must be used by a single thread.
- ADManager (see page 570) must be activated before threads will start by setting ADManager (see page 570).Active (see page 367) to True.

Means, after a thread opened a query, and until its processing is not finished, application cannot use this query and connection objects in an other thread. Or, after a thread started a transaction, and until it is not finished, application cannot use this transaction and connection objects in an other thread.

Practically this means an application must serialize access to a connection across all threads, and that is not a convenient technique. Breaking these rules may lead to misbehavior, AV's and errors like the SQL Server error "Connection is busy with results for another command".

The standard simplification is to create and use for each thread a dedicated connection object working with database. In this case, no additional serialization is required. For example the following code performs a DB tasks in threads:

```
type
  TDBThread = class(TThread)
  protected
    procedure Execute; override;
  end;

procedure TDBThread.Execute;
var
  oConn: TADConnection;
  oPrc: TADQuery;
begin
  FreeOnTerminate := False;
  oConn := TADConnection.Create(nil);
  oConn.ConnectionDefName := 'Oracle_Pooled'; // see next section
  oPrc := TADStoredProc.Create(nil);
  oPrc.Connection := oConn;
  try
    oConn.Connected := True;
    oPrc.StoredProcName := 'MY_LONG_RUNNING_PROC';
    oPrc.ExecProc;
  finally
    oPrc.Free;
    oConn.Free;
  end;
end;

// main application code
var
  oThread1, oThread2: TDBThread;
begin
  ADManager.Active := True;
  ...
  oThread1 := TDBThread.Create(False);
  oThread2 := TDBThread.Create(False);
  ...
  oThread1.WaitFor;
  oThread1.Free;
  oThread2.WaitFor;
  oThread2.Free;
end;
```

Note, for above case, where application runs a single SQL query in background, consider to use asynchronous query execution mode (see page 86).

Note, multi-threaded application may close connections opened in the background threads in a TADManager.BeforeShutdown (see page 369) event handler to avoid possible dead lock.

Connection Pooling

One of the expensive database interaction operations is the connection establishment (see page 37). In multi threading application, where each thread starts, establish connection, performs some short task and releases connection, the repetitive connection establishments may lead to performance degradation across all system. To avoid that the application may use the connection pooling.

The connection pooling may be enabled only for persistent or private connection definition (see page 27) by setting Pooled=True. For example, persistent definition:

```
[Oracle_Pooled]
DriverID=Ora
Database=ORA_920_APP
User_Name=ADDemo
Password=a
Pooled=True
```

Or private definition setup:

```
var
  oParams: TStrings;
begin
  oParams := TStringList.Create;
  oParams.Add('Database=ORA_920_APP');
  oParams.Add('User_Name=ADDemo');
  oParams.Add('Password=a');
  oParams.Add('Pooled=True');
  ADManager.AddConnectionDef('Oracle_Pooled', 'Ora', oParams);
  ......................
  ADConnection1.ConnectionDefName := 'Oracle_Pooled';
  ADConnection1.Connected := True;
```

None additional parameters may be specified in TADConnection.Params (see page 334) property, as all pooled connections must share the same connection parameters.

Setting TADConnection.Connected (see page 321) to True acquires a physical connection from the pool. Setting TADConnection.Connected (see page 321) to False releases the physical connection to the pool, but keeps connection opened. To close and destroy all pooled physical connections, the application may call TADManager.CloseConnectionDef (see page 377) method:

```
ADManager.CloseConnectionDef('Oracle_Pooled');
```

or close the FireDAC driver manager by calling:

```
ADManager.Close;
```

Additional connection definition parameters may be specified to setup a pool:

Parameter	Description	Example
POOL_CleanupTimeout	The timeout (msecs) when FireDAC will remove connections unused more than POOL_ExpireTimeout time. Default value is 30000 msecs (30 secs).	3600000
POOL_ExpireTimeout	The time (msecs) after which the inactive connection may be deleted from the pool and destroyed. Default value is 90000 msecs (90 secs).	600000
POOL_MaximumItems	The maximum number of connection in the pool. When application will require more connections, then exception will be riased. Default value is 50.	100

See Also

Configuring Drivers (see page 31), Defining Connection (see page 27), Establishing Connection (see page 37)

Example

For example see the FireDAC\Samples\Comp Layer\TADConnection\Pooling demo.

1.4.10 DLL Development

Describes how to use FireDAC in the dynamic loading libraries.

Description

General

FireDAC may be used in a DLL as in a normal application. But developers must be aware of two techniques, specific to DLL development.

Connection sharing between application and DLL

When a connection must be transferred from an application to the DLL's, the application should not transfer the TADCustomConnection (see page 323) object, as it may lead to AV's and other issues. This is not the FireDAC issue, it is due to the Delphi RTL / RTTI limitations. Note, a connection cannot be shared with other process, as the sharing works only inside the same address space.

To transfer a connection between an application and a DLL, application should transfer the TADCustomConnection.CliHandle (see page 323) property value to the DLL. There the handle must be assigned to the TADCustomConnection.SharedCliHandle (see page 336) property.

After the SharedCliHandle (see page 336) is assigned the DLL connection may be activated by setting Connected (see page 321) to True. Note, there is no need to setup connection definition, including DriverID. Then connection may be used as a normal database connection. Finally it may be closed by setting Connected (see page 321) to False. That will not close the physical connection, so the application connection will remain active.

The application connection does not track state changes, performed by the DLL. So, the DLL should preserve the same transaction state, as it was before DLL call. Best - do not handle transactions in a DLL, change transaction isolation level and other settings in a DLL.

Also, setting SharedCliHandle (see page 336) does not transfer any of option values from an application to a DLL connection object. The DLL connection options may be set similar to the application connection options.

For example, the DLL code:

```
procedure SomeTask(ACliHandle: LongWord); stdcall;
var
  oConn: TADConnection;
begin
  oConn := TADConnection.Create(nil);
  oQuery := TADQuery.Create(nil);
  try
    oConn.SharedCliHAndle := ACliHandle;
    oConn.Connected := True;

    oQuery.Connection := oConn;
    oQuery.ExecSQL('delete from aaa');
  finally
    oQuery.Free;
    oConn.Free;
  end;
end;

exports
  SomeTask;
```

And the application code:

```
procedure TForm1.Button1Click(Sender: TObject);
type
  TSomeTaskProc = procedure (ACliHandle: LongWord); stdcall;
```

```
var
  hDll: THandle;
  pSomeTask: TSomeTaskProc;
begin
  hDll := LoadLibrary(PChar('Project2.dll'));
  try
    @pSomeTask := GetProcAddress(FhDll, PChar('SomeTask'));
    ADConnection1.StartTransaction;
    try
      pSomeTask(ADConnection1.CliHandle);
      ADConnection1.Commit;
    except
      ADConnection1.Rollback;
      raise;
    end;
  finally
    FreeLibrary(hDll);
  end;
end;
```

Note, this code does not:

- handle DLL loading errors;
- care about FireDAC DLL unloading - see next chapter;
- keep DLL loaded for a long time and keep DLL objects for a long time.

FireDAC DLL unloading

An application may hang-up at unloading of a DLL containing FireDAC. The standard symptom for that is the application hanging on the FreeLibrary call. This is the FireDAC limitation, which has two workarounds:

1. Enable FireDAC silent mode in the DLL, so no wait cursors, no dialogs, etc will be shown by the FireDAC. For that put in your DLL DPR file:

```
uses
  uADGUIxIntf;

begin
  FADGUIxSilentMode := True;
end.
```

2. Terminate FireDAC manager before unloading of a DLL. For that the DLL should export a procedure, which will call ADTerminate procedure. For that put in your DLL DPR file:

```
uses
  uADStanFactory;

procedure Shutdown;
begin
  ADTerminate;
end;

exports
  Shutdown;
```

Then import and call the Shutdown procedure from the application before FreeLibrary call:

```
var
  Shutdown: procedure;
....
  Shutdown := GetProcAddress(hLib, 'Shutdown');
  if Assigned(Shutdown) then
    Shutdown();
  FreeLibrary(hLib);
```

Note, the above technique must be used for the ActiveX servers with FireDAC inside.

See Also

TADCustomConnection.CliHandle (see page 323), TADCustomConnection.SharedCliHandle (see page 336)

Example

For example see the FireDAC\Samples\Comp Layer\TADConnection\DLL_Sharing demo.

1.4.11 Unicode Support

Describes how FireDAC works with Unicode data and metadata. FireDAC fully support Unicode data and metadata. Although an application may need to take the additional steps to use Unicode properly.

Description

General

Most enterprise-class database applications must be able to work with character data encoded provided as Unicode. FireDAC provides seamless support for:

- different single byte client character sets and code pages, including standard ANSI;
- as well multi-byte strings such as Unicode, including UTF8, UTF16 and UCS2 encodings.

FireDAC Unicode handling depends on:

- the version of Delphi being used;
- client character set - UTF8/UTF16 or ACP compatible;
- DBMS client / driver.

FireDAC performs transparent character set conversion between Delphi application and DBMS client character set, when it is required. Note, that the complete Unicode support may be achieved only with the Unicode Delphi version usage - Delphi 2009 and higher. The following table summarizes where and what encoding may be used depending on a Delphi version:

Usage	Unicode enabled Delphi 2009 and higher	Non-Unicode enabled Delphi 2007 and less
SQL command text	Unicode	ANSI
SQL script text	Unicode	ANSI
Result set field values	Unicode / ANSI	ANSI / Unicode
Parameter values	Unicode / ANSI	ANSI / Unicode
Metadata values, including: • column names, • table names; • generator names; • index names; • etc	Unicode	ANSI
Metadata query values	ANSI	ANSI
Text data files	Unicode / ANSI	ANSI / Unicode
Trace output	Unicode / ANSI	ANSI / Unicode
Tools	Unicode	Unicode

Configuring connection definitions for Unicode

Note: It is recommended to set the DBMS client character set to Unicode, for Delphi 2009 and higher, to avoid conversion loss.

In general, to configure connection definitions for Unicode, it is required to set the DBMS client character set to Unicode. How this is done depends on the version of Delphi and the used DBMS driver:

DBMS	Parameter
Advantage Database Server	Not supported.
IBM DB2 Server	Automatically set to UTF16 in Delphi 2009 and higher. Not supported in Delphi 2007 or less.
DataSnap server	Automatically set to UTF16 in Delphi 2009 and higher. Not supported in Delphi 2007 or less.
Interbase or Firebird Server	CharacterSet=utf8
Microsoft SQL Server	Automatically set to UTF16 in Delphi 2009 and higher. Not supported in Delphi 2007 or less.
Microsoft Access Database	Automatically set to UTF16 in Delphi 2009 and higher. Not supported in Delphi 2007 or less.
MySQL Server	CharacterSet=utf8
Oracle Server	• CharacterSet=utf8; • or NLS_LANG=_.UTF8, if CharacterSet is not specified.
PostgreSQL	CharacterSet=utf8
SQLite database	Automatically set to UTF16 in Delphi 2009 and higher. Not supported in Delphi 2007 or less.
Sybase SQL Anywhere	Automatically set to UTF16 in Delphi 2009 and higher. Not supported in Delphi 2007 or less.
dbExpress bridge driver	Depends on dbExpress driver.
ODBC bridge driver	Automatically set to UTF16 in Delphi 2009 and higher. Not supported in Delphi 2007 or less.

SQL command text

Starting with Delphi 2009, TADQuery.SQL (see page 396), TADCommand.CommandText (see page 298), IADPhysCommand.CommandText, and other property values are Unicode encoded. The SQL command text preprocessor supports only UCS2 encoding, which means that surrogate pairs are not supported.

Pre Delphi 2009 versions support only ANSI encoded command text.

Before sending SQL command text to the DBMS, FireDAC converts it to:

- Active Code Page, if the client character set is SBCS (non Unicode). If the client character set is not compatible with ACP, conversion loss is possible.
- UTF8, if client character set is UTF8.
- UTF16, if client character set is UTF16.

To specify Unicode encoded SQL command text, just do the following:

```
ADQuery1.SQL.Text := 'select ''Hello world !'' where Lang = ''RU'' and Val = ''?????? ??? !''';
```

SQL script text

Starting with Delphi 2009, TADScript.SQLScripts (see page 703) collections can contain Unicode encoded SQL script texts. Other than that, SQL script text processing is similar to SQL command text processing.

Pre Delphi 2009 versions support only ANSI encoded script text. TADScript (see page 685) can load Unicode encoded SQL script files, but they will be converted into ACP (ANSI) encoding before processing.

To control SQL script file and log file encoding, use TADScript.ScriptOptions.FileEncoding (see page 715) property:

```
ADScript1.ScriptOptions.FileEncoding := enUTF16;
ADScript1.SQLScriptFileName := 'u:\builddb.sql';
ADScript1.ExecuteAll;
```

Result set fields

DBMS	Description
Advantage Database Server	• NCHAR - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • NVARCHAR - ftWideString; • NMEMO - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise.
IBM DB2 Server	• GRAPHIC - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • VARGRAPHIC - ftWideString; • LONG VARGRAPHIC, DBCLOB - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise.
DataSnap server	• ftFixedWideChar on Delphi 2006 and higher and ftWideString; • ftWideMemo on Delphi 2006 and higher and ftFmtMemo.
Interbase or Firebird Server	If CharacterSet is UTF8 or UNICODE_FSS, then: • CHAR - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • VARCHAR - ftWideString; • BLOB SUB_TYPE TEXT - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise.
Microsoft SQL Server	• NCHAR - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • NVARCHAR - ftWideString; • NTEXT, NVARCHAR(MAX) - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise;
Microsoft Access Database	• CHARACTER - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • MEMO - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise.
MySQL Server	If CharacterSet is UTF8, then: • CHAR - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • VARCHAR, TINYTEXT - ftWideString; • MEDIUMTEXT, TEXT, LONGTEXT - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise.
Oracle Server	• NCHAR - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • NVARCHAR2 - ftWideString; • NCLOB - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise. If CharacterSet is UTF8, then: • CHAR - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • VARCHAR2 - ftWideString; • LONG, CLOB - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise.

SQLite Database	If StringFormat=Choose, then: • NCHAR, NATIONAL CHAR, NATIONAL CHARACTER - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • NVARCHAR, NVARCHAR2, NATIONAL CHARACTER VARYING, NATIONAL CHAR VARYING - ftWideString; • NTEXT, WTEXT, NCLOB, NMEMO, LONG NTEXT, LONG WTEXT, NATIONAL TEXT, LONGWVARCHAR - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise. If StringFormat=Unicode, then all string column are Wide strings. If StringFormat=Ansi, then - ANSI strings.
PostgreSQL	If CharacterSet is UTF8, then: • CHAR - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • VARCHAR - ftWideString; • TEXT - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise.
Sybase SQL Anywhere	• NCHAR - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • NVARCHAR - ftWideString; • LONG NVARCHAR - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise.
dbExpress bridge driver	Depends on dbExpress driver and DBMS. The general rule: DBX1-3: • fldZSTRING, fldstUNICODE - ftWideString; • fldZSTRING, fldstUNICODE, fldstFIXED - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • fldBLOB, fldstMEMO, fldstUNICODE - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise; • fldBLOB, fldstFMTMEMO - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise. DBX4: • TDBXDataTypes.WideStringType - ftWideString; • TDBXDataTypes.WideStringType, TDBXDataTypes.FixedSubType - ftFixedWideChar; • TDBXDataTypes.BlobType, TDBXDataTypes.WideMemoSubType - ftWideMemo.
ODBC bridge driver	Depends on ODBC driver and DBMS. The general rule: • SQL_WCHAR, SQL_GRAPHIC - ftFixedWideChar on Delphi 2006 and higher, ftWideString otherwise; • SQL_WVARCHAR, SQL_VARGRAPHIC, SQL_LONGVARGRAPHIC - ftWideString; • SQL_WLONGVARCHAR, SQL_DBCLOB - ftWideMemo on Delphi 2006 and higher, ftFmtMemo otherwise.

To read/write Unicode string values programmatically use:

- TField.AsWideString or TField.Value properties;
- TField.AsString on Delphi 2009 and higher. On earlier Delphi versions reading / writing of AsString property may lead to conversion loss, since AsString returns ANSI strings there.

To read/write the Unicode field on any Delphi version:

```
ADQuery1.Edit;
ADQuery1.Fields[0].AsWideString := ADQuery1.Fields[0].AsWideString + '??????? + english';
ADQuery1.Post;
```

To read/write Unicode memo field on Delphi 2009:

```
ADQuery1.Edit;
ADQuery1.FieldsByName('memo').Assign(Memo1.Lines);
ADQuery1.Post;
```

Parameter values

The Unicode encoded parameter value is converted to a supported by DBMS Unicode character set and sent to the DBMS. This does not depend on a client character set or a Delphi version.

Note, Firebird and Interbase cannot send Unicode character data to a server if the CharacterSet parameter is not UTF8.

To read/write Unicode parameter value on any Delphi version:

```
ADQuery1.Params[0].AsWideString := '??????? + english';
// the same as following
ADQuery1.Params[0].Value := '??????? + english';
ADQuery1.Params[0].DataType := ftWideString;
```

Metadata values

Starting with Delphi 2009, Unicode encoded metadata text is supported for:

- table names;
- resultset column names;
- stored procedure names;
- stored procedure parameter names;
- SQL command parameter names;
- SQL command macro names;
- user name;
- schema, catalog names.

Pre Delphi 2009 versions support only ANSI encoded metadata text.

Note: At the moment, FireDAC does not support Unicode encoded column values in metadata result sets, returned by TADMetaInfoCommand, TADMetaInfoQuery and IADPhysMetaInfoCommand.

Text data files

Starting with Delphi 2009, TADDataMove can read/write Unicode encoded text data files without conversion loss. On prior Delphi versions TADDataMove can read/write Unicode encoded text data files, but internally handles all data as ACP encoded. So, conversion loss is possible.

To control encoding of:

- text data file, use TADDataMove.TextFileEncoding property;
- log file, use TADScript.ScriptOptions.LogFileEncoding property.

To read Unicode data file and create ACP encoded log file do:

```
ADDataMove1.TextFileName := 'u:\data.txt';
ADDataMove1.TextFileEncoding := enUTF8;
ADDataMove1.LogFileName := 'u:\log.txt';
ADDataMove1.TextFileEncoding := enANSI;
ADDataMove1.Destination := ADQuery1;
ADDataMove1.SourceKind := skText;
ADDataMove1.Execute;
```

Trace output

To enable Unicode encoding for trace file output (see page 174) (MonitorBy (see page 188)=FlatFile), set TADMoniFlatFileClientLink.FileEncoding property value to ecUTF8 or ecUTF16.

Tools

All FireDAC utilities are compiled with Unicode Delphi version and are 100% Unicode enabled.

1.5 Working with Commands

A set of articles describing how to execute SQL commands with FireDAC, exchange data and metadata.

1.5.1 Preprocessing Command Text

FireDAC offers powerful SQL command text preprocessor. Allowing to write DBMS independent SQL commands and to get more flexibility while building dynamic SQL command. The preprocessor features include substitution variables, escape sequences, escape functions and conditional escapes.

Description

General

In this text in other places a "macro" and "escape sequence" are interchangeable phrases. FireDAC supports three kinds of macro instructions:

- **Substitution variables**. They allow to put substitution parameters in a command text. This is to extend the use of parameters. For example, to parameterize a table name in FROM clause or the column names in SELECT clause, the substitution variables can be used but the parameters are of no use.
- **Escape sequences**. They allow writing DBMS independent SQL commands.
- **Conditional substitutions**. They allow expand SQL command conditionally, depending on application defined attributes or currently attached DBMS.

Setting CommandText (see page 298) or SQL (see page 396) property values will automatically fill the Macros (see page 490) collection property when ResourceOptions.MacroCreate (see page 882) is True. At Prepare call, the FireDAC command preprocessor transforms command text into a form understood by the DBMS when ResourceOptions.MacroExpand (see page 882) is True. This means, the macros are not visible to the DBMS.

The Params (see page 497) collection property is filled automatically when ResourceOptions.ParamCreate (see page 883) is True. At Prepare call FireDAC replaces the FireDAC parameter markers with the DBMS native markers when ResourceOptions.ParamExpand (see page 883) is True.

The escape sequences and conditional substitutions are processed when ResourceOptions.EscapeExpand (see page 882) is True.

SQL dialect abstraction

If the application needs to support multiple DBMS's, then it must be aware that their SQL dialects may be different. FireDAC escape sequences allow you to write SQL dialect independent SQL commands.

For example, the function to convert string to upper case is different in MySQL, Oracle and Microsoft SQL Server. But the following command will work on any DBMS:

```
SELECT {ucase(Name)} FROM MyTable
```

Note, with SQLite add the uADStanExprFuncs unit to your "uses" clause. In more complex cases, parts of the command or even the full command must be written differently. Then, the FireDAC conditional escape sequence will help:

```
{IF Oracle} SELECT * FROM OracleTab {fi}
{IF MSSQL} SELECT * FROM MSSQLTab {fi}
```

Substitution variables

A substitution variable starts with '!' or '&' symbol and is followed by the macro variable name. For example:

```
SELECT * FROM &TabName
```

The symbols have the following meaning:

• '!' - "string" substitution mode. Macro value will be substituted "as is", directly into the command text without any transformations.

• '&' – "SQL" substitution mode. Macro value will be substituted depending on the macro data type, using target DBMS syntax rules.

To use the macros use the code like this:

```
ADQuery1.SQL.Text := 'SELECT * FROM &TabName';
ADQuery1.MacroByName('TabName').AsRaw := 'Orders';
ADQuery1.Open;
```

The macros are processed when ResourceOptions.MacroCreate (see page 882) and MacroExpand (see page 882) are True.

Parameter markers

A parameter marker starts with ':' symbol and is followed by the parameter name. For example:

```
SELECT * FROM Orders WHERE OrderDate > :FROM_DATE
```

The '?' symbol is recognized an unnamed parameter marker. This is for DataSnap compatibility and should not be used in an ordinary FireDAC application.

To use the parameters read the "Using parameters" chapter at "Executing Command (see page 66)". The parameters are processed when ResourceOptions.ParamCreate (see page 883) and ParamExpand (see page 883) are True.

Escape sequences

FireDAC has 5 kinds of escape sequences:

• Allowing constant substitution.

• Allowing identifier substitution.

• Conditional substitution.

• LIKE operator escape sequence.

• Scalar functions.

The escape sequences are processed when ResourceOptions.EscapeExpand (see page 882) is True.

The constant substitution escape sequences allow writing constants in command text, independent on DBMS syntax and regional settings. Following describes escape sequences expansion to DBMS syntax:

Format	Description
{**e** <number>}	Number constant. <number> must be specified with '.' as decimal separator. For example: {e 123.7} -> 123,7 on MSAccess
{**d** <date>}	Date constant. <date> must be specified in 'yyyy-mm-dd' format. For example: {d 2004-08-30} -> TO_DATE('2004-08-30', 'yyyy-mm-dd') on Oracle.

{**t** <time>}	Time constant. <time> must be specified in 'hh24:mi:ss' format. For example: {t 14:30:00} -> CONVERT(DATETIME, '14:30:00', 114) on SQL Server
{**dt** <date & time>}	Date and time constant. <date & time> must be in the above formats.
{**l** <boolean>}	Boolean constant. <boolean> is False or True. If DBMS supports Boolean data type, then sequence expands to that type constant, otherwise to numeric values 0 or 1.
{**s** <string>}	String constant. <string> is quoted or not quoted sequence of characters. For example: {s Company '1st Coding'} -> 'Company ''1st Coding''''

The identifier substitution escape sequence allows abstracting from DBMS specific identifier quoting rules. Read the "Object Names (see page 133)" chapter for more details. The syntax is:

Format	Description
{**id** <identifier name>}	Expands to DBMS specific quoted identifier syntax. For example: {id Order Details} -> “Order Details” on Oracle.

The escape function sequence allows abstracting from a DBMS specific set of a build-in functions and their syntax. The syntax is:

Format	Description
{**fn** <function name>(<arguments>)}	The escape functions syntax and set is identical to the ODBC escape functions. In the sub topics the FireDAC escape functions are listed. For example: SELECT * FROM MyTab WHERE Year = {fn YEAR({fn NOW()}} or SELECT * FROM MyTab WHERE Year = {YEAR({NOW()}} ->\ SELECT * FROM MyTab WHERE Year = TO_NUMBER(TO_CHAR(SYSDATE, 'YYYY')) on Oracle.
{<function name>(<arguments>)}	The same as above.

Conditional substitution

Conditional substitution escape sequences allow substitute text into command, depending on either a DBMS the application is connected to, either on a macro variable value. Beside different syntaxes, these constructions handle parameters and macros in different ways. Escape sequence syntaxes are:

Format	Description
{**iif** (X1, Y1, …, XN, YN, YN+1) }	Here Xi is either: • DBMS identifier. So, if application is connected to this DBMS, then Yi text will be substituted into command. • Macro variable. If it value is not empty, then Yi text will be substituted into command. If neither of conditions is meted and YN+1 text is specified, then it will be substituted into command. Parameters and macros in either Xi and Yi will be created in Params and Macros collection.
{**if** x} y {**fi**}	Here X is either: • DBMS identifier. So, if application is connected to this DBMS, then Y text will be substituted into command. • Macro variable. If it value is not empty, then Y text will be substituted into command. Parameters and macros in Y will be created in Params and Macros collection only if X is “true”.

FireDAC DBMS identifiers are case insensitive. Following table lists them:

Identifier	DBMS
ADS	Advantage Database Server
ASA	Sybase SQL Anywhere
DB2	IBM DB2
INTRBASE	Interbase or Firebird Server
MSACCESS MSACC	Microsoft Access database
MSSQL	Microsoft SQL Server
MYSQL	MySQL Server
ORACLE ORA	Oracle Server
OTHER	Any other DBMS, not listed in the table.
POSTGRESQL PG	PostgreSQL Server
SQLITE	SQLite database

For example:

```
{iif (Oracle, TO_CHAR, MSSQL, CONVERT)}          -> 'TO_CHAR' on Oracle and 'CONVERT' on
SQL Server.
{iif (&v1, Me, &v2, You, We)}                    -> 'Me' if &v1 has nonempty value, 'you' if
&v2 has nonempty value, otherwise 'We'.
{if Oracle} TO_CHAR {fi} {if MSSQL} CONVERT {fi} -> TO_CHAR on Oracle and CONVERT on SQL
Server.
{if &v1} Me {fi} {if &v2} You {fi}               -> 'Me' if &v1 has nonempty value + 'you'
if &v2 has nonempty value.
```

Note, escape functions IF/IIF (see page 64) have the same names. To distinguish a function from a conditional substitution, use {fn IF(...)} or {fn IIF(...)} syntax.

Special character processing

To transmit special characters - '!', '&', ':', '?', '{' or '}' to the DBMS, you will need:

• to double special character. For example, '{{'.

• on MySQL precede this character by '\' character. For example, '\"'.

• for '!', '&', '{', '} set ResourceOptions.MacroCreate (see page 882), MacroExpand (see page 882) and EscapeExpand (see page 882) to False.

• for ':', '?' set ResourceOptions.ParamCreate (see page 883), ParamExpand (see page 883) to False.

Otherwise '!', '&', '{', '}' characters will be treated as a macro command. And ':', '?' as a parameter marker, excluding the following cases:

• Oracle PL/SQL assignment operator is detected by FireDAC and will not be treated as parameter marker.

• Firebird EXECUTE BLOCK parameters inside of BEGIN … END are detected by FireDAC and will not be treated as parameter marker.

• TSQL label is detected by FireDAC and will not be treated as parameter marker.

See Also

Object Names (see page 133)

1.5.1.1 Character macro functions

List of the functions working with character strings.

Description

CHARACTER

Function	Description	Local expression engine
ASCII(string_exp)	Returns the ASCII code value of the leftmost character of string_exp as an integer.	+
BIT_LENGTH(string_exp)	Returns the length in bits of the string expression.	-
CHAR(code)	Returns the character that has the ASCII code value specified by code. The value of code should be between 0 and 255; otherwise, the return value is data source–dependent.	+
CHAR_LENGTH(string_exp)	Returns the length in characters of the string expression, if the string expression is of a character data type; otherwise, returns the length in bytes of the string expression (the smallest integer not less than the number of bits divided by 8). (This function is the same as the CHARACTER_LENGTH function.)	+
CHARACTER_LENGTH(string_exp)	Returns the length in characters of the string expression, if the string expression is of a character data type; otherwise, returns the length in bytes of the string expression (the smallest integer not less than the number of bits divided by 8). (This function is the same as the CHAR_LENGTH function.)	+
CONCAT(string_exp1, string_exp2)	Returns a character string that is the result of concatenating string_exp2 to string_exp1. The resulting string is DBMS-dependent. For example, if the column represented by string_exp1 contained a NULL value, DB2 would return NULL but SQL Server would return the non-NULL string.	+
DIFFERENCE(string_exp1, string_exp2)	Returns an integer value that indicates the difference between the values returned by the SOUNDEX function for string_exp1 and string_exp2.	-
INSERT(string_exp1, start, length, string_exp2)	Returns a character string where length characters have been deleted from string_exp1, beginning at start, and where string_exp2 has been inserted into string_exp, beginning at start.	+
LCASE(string_exp) (ODBC 1.0)	Returns a string equal to that in string_exp, with all uppercase characters converted to lowercase.	+
LEFT(string_exp, count)	Returns the leftmost count characters of string_exp.	+
LENGTH(string_exp)	Returns the number of characters in string_exp, excluding trailing blanks.	+
LOCATE(string_exp1, string_exp2[, start])	Returns the starting position of the first occurrence of string_exp1 within string_exp2. The search for the first occurrence of string_exp1 begins with the first character position in string_exp2 unless the optional argument, start, is specified. If start is specified, the search begins with the character position indicated by the value of start. The first character position in string_exp2 is indicated by the value 1. If string_exp1 is not found within string_exp2, the value 0 is returned.	+
LTRIM(string_exp)	Returns the characters of string_exp, with leading blanks removed.	+
OCTET_LENGTH(string_exp)	Returns the length in bytes of the string expression. The result is the smallest integer not less than the number of bits divided by 8.	+

POSITION(character_exp, character_exp)	Returns the position of the first character expression in the second character expression. The result is an exact numeric with an implementation-defined precision and a scale of 0.	+
REPEAT(string_exp, count)	Returns a character string composed of string_exp repeated count times.	+
REPLACE(string_exp1, string_exp2, string_exp3)	Search string_exp1 for occurrences of string_exp2, and replace with string_exp3.	+
RIGHT(string_exp, count)	Returns the rightmost count characters of string_exp.	+
RTRIM(string_exp)	Returns the characters of string_exp with trailing blanks removed.	+
SOUNDEX(string_exp)	Returns a data source–dependent character string representing the sound of the words in string_exp. For example, SQL Server returns a 4-digit SOUNDEX code; Oracle returns a phonetic representation of each word.	-
SPACE(count)	Returns a character string consisting of count spaces.	+
SUBSTRING(string_exp, start, length)	Returns a character string that is derived from string_exp, beginning at the character position specified by start for length characters.	+
UCASE(string_exp)	Returns a string equal to that in string_exp, with all lowercase characters converted to uppercase.	+

1.5.1.2 Numeric macro functions

List of the functions working with numbers.

Description

NUMERIC

Function	Description	Local expression engine
ABS(numeric_exp)	Returns the absolute value of numeric_exp.	+
ACOS(float_exp)	Returns the arccosine of float_exp as an angle, expressed in radians.	+
ASIN(float_exp)	Returns the arcsine of float_exp as an angle, expressed in radians.	+
ATAN(float_exp)	Returns the arctangent of float_exp as an angle, expressed in radians.	+
ATAN2(float_exp1, float_exp2)	Returns the arctangent of the x and y coordinates, specified by float_exp1 and float_exp2, respectively, as an angle, expressed in radians.	+
CEILING(numeric_exp)	Returns the smallest integer greater than or equal to numeric_exp. The return value is of the same data type as the input parameter.	+
COS(float_exp)	Returns the cosine of float_exp, where float_exp is an angle expressed in radians.	+
COT(float_exp)	Returns the cotangent of float_exp, where float_exp is an angle expressed in radians.	+
DEGREES(numeric_exp)	Returns the number of degrees converted from numeric_exp radians.	+
EXP(float_exp)	Returns the exponential value of float_exp.	+
FLOOR(numeric_exp)	Returns the largest integer less than or equal to numeric_exp. The return value is of the same data type as the input parameter.	+
LOG(float_exp)	Returns the natural logarithm of float_exp.	+
LOG10(float_exp)	Returns the base 10 logarithm of float_exp.	+
MOD(integer_exp1, integer_exp2)	Returns the remainder (modulus) of integer_exp1 divided by integer_exp2.	+

PI( )	Returns the constant value of pi as a floating-point value.	+
POWER(numeric_exp, integer_exp)	Returns the value of numeric_exp to the power of integer_exp.	+
RADIANS(numeric_exp)	Returns the number of radians converted from numeric_exp degrees.	+
RAND([integer_exp])	Returns a random floating-point value using integer_exp as the optional seed value.	+
ROUND(numeric_exp, integer_exp)	Returns numeric_exp rounded to integer_exp places right of the decimal point. If integer_exp is negative, numeric_exp is rounded to |integer_exp| places to the left of the decimal point.	+
SIGN(numeric_exp)	Returns an indicator of the sign of numeric_exp. If numeric_exp is less than zero, –1 is returned. If numeric_exp equals zero, 0 is returned. If numeric_exp is greater than zero, 1 is returned.	+
SIN(float_exp)	Returns the sine of float_exp, where float_exp is an angle expressed in radians.	+
SQRT(float_exp)	Returns the square root of float_exp.	+
TAN(float_exp)	Returns the tangent of float_exp, where float_exp is an angle expressed in radians.	+
TRUNCATE(numeric_exp, integer_exp)	Returns numeric_exp truncated to integer_exp places right of the decimal point. If integer_exp is negative, numeric_exp is truncated to |integer_exp| places to the left of the decimal point.	+

1.5.1.3 Date and time macro functions

List of the functions working with date and time.

Description

DATE, TIME

Function	Description	Local expression engine
CURRENT_DATE( )	Returns the current date.	+
CURRENT_TIME[(time-precision)]	Returns the current local time. The time-precision argument determines the seconds precision of the returned value.	+
CURRENT_TIMESTAMP [(timestamp-precision)]	Returns the current local date and local time as a timestamp value. The timestamp-precision argument determines the seconds precision of the returned timestamp.	+
CURDATE( )	Returns the current date.	+
CURTIME( )	Returns the current local time.	+
DAYNAME(date_exp)	Returns a character string containing the data source–specific name of the day (for example, Sunday through Saturday or Sun. through Sat. for a data source that uses English, or Sonntag through Samstag for a data source that uses German) for the day portion of date_exp.	+
DAYOFMONTH(date_exp)	Returns the day of the month based on the month field in date_exp as an integer value in the range of 1–31.	+
DAYOFWEEK(date_exp)	Returns the day of the week based on the week field in date_exp as an integer value in the range of 1–7, where 1 represents Sunday.	+
DAYOFYEAR(date_exp)	Returns the day of the year based on the year field in date_exp as an integer value in the range of 1–366.	+

EXTRACT(extract-field, extract-source)	Returns the extract-field portion of the extract-source. The extract-source argument is a datetime or interval expression. The extract-field argument can be one of the following keywords: YEAR MONTH DAY HOUR MINUTE SECOND The precision of the returned value is implementation-defined. The scale is 0 unless SECOND is specified, in which case the scale is not less than the fractional seconds precision of the extract-source field.	+
HOUR(time_exp)	Returns the hour based on the hour field in time_exp as an integer value in the range of 0–23.	+
MINUTE(time_exp)	Returns the minute based on the minute field in time_exp as an integer value in the range of 0–59.	+
MONTH(date_exp)	Returns the month based on the month field in date_exp as an integer value in the range of 1–12.	+
MONTHNAME(date_exp)	Returns a character string containing the data source–specific name of the month (for example, January through December or Jan. through Dec. for a data source that uses English, or Januar through Dezember for a data source that uses German) for the month portion of date_exp.	+
NOW( )	Returns current date and time as a timestamp value.	+
QUARTER(date_exp)	Returns the quarter in date_exp as an integer value in the range of 1–4, where 1 represents January 1 through March 31.	+
SECOND(time_exp)	Returns the second based on the second field in time_exp as an integer value in the range of 0–59.	+
TIMESTAMPADD(interval, integer_exp, timestamp_exp)	Returns the timestamp calculated by adding integer_exp intervals of type interval to timestamp_exp. Valid values of interval are the following keywords: FRAC_SECOND **(*)** SECOND MINUTE HOUR DAY WEEK MONTH QUARTER YEAR where fractional seconds are expressed in billionths of a second. For example, the following SQL statement returns the name of each employee and his or her one-year anniversary date: SELECT NAME, {fn TIMESTAMPADD('YEAR', 1, HIRE_DATE)} FROM EMPLOYEES If timestamp_exp is a time value and interval specifies days, weeks, months, quarters, or years, the date portion of timestamp_exp is set to the current date before calculating the resulting timestamp. If timestamp_exp is a date value and interval specifies fractional seconds, seconds, minutes, or hours, the time portion of timestamp_exp is set to 0 before calculating the resulting timestamp.	+

TIMESTAMPDIFF(interval, timestamp_exp1, timestamp_exp2)	Returns the integer number of intervals of type interval by which timestamp_exp2 is greater than timestamp_exp1. Valid values of interval are the following keywords: FRAC_SECOND **(*)** SECOND MINUTE HOUR DAY WEEK MONTH QUARTER YEAR where fractional seconds are expressed in billionths of a second. For example, the following SQL statement returns the name of each employee and the number of years he or she has been employed: SELECT NAME, {fn TIMESTAMPDIFF('YEAR', {fn CURDATE()}, HIRE_DATE)} FROM EMPLOYEES If either timestamp expression is a time value and interval specifies days, weeks, months, quarters, or years, the date portion of that timestamp is set to the current date before calculating the difference between the timestamps. If either timestamp expression is a date value and interval specifies fractional seconds, seconds, minutes, or hours, the time portion of that timestamp is set to 0 before calculating the difference between the timestamps.	+
WEEK(date_exp)	Returns the week of the year based on the week field in date_exp as an integer value in the range of 1–53.	+
YEAR(date_exp)	Returns the year based on the year field in date_exp as an integer value. The range is data source–dependent.	+

(*) Some DBMS's fail to work with FRAC_SECOND.

1.5.1.4 System macro functions

List of the special functions.

Description

SYSTEM

Function	Description	Local expression engine
DATABASE( )	Returns the name of the database corresponding to the connection.	+
IFNULL(exp, value)	If exp is null, value is returned. If exp is not null, exp is returned. The possible data type or types of value must be compatible with the data type of exp.	+
IF(exp, value1, value2) IIF(exp, value1, value2)	If exp is True, value1 is returned, otherwise - value2. Do not mix IF/IIF escape functions with IF/IIF conditional substitutions (see page 56). To call IF/IIF escape function use {fn IF(...)} or {fn IIF(...)} syntax.	+
LIMIT([skip,] rows)	Allows to limit the result set, by skiping first <skip> records and returning to more than <rows> records. The function may be put in any place of the SQL command.	-

NEWGUID( )	Returns the new randomly generated GUID value. Is not supported as the escape function.	+
USER( )	Returns the user name in the DBMS. This can be different than the login name.	+

1.5.1.5 Convert macro function

Describes the CONVERT function.

Description

CONVERT

Function	Description	Local expression engine
`CONVERT(value_exp, data_type)`	The function returns the value specified by value_exp converted to the specified data_type, where data_type is one of the following keywords.	+

The supported keywords:

BIGINT	REAL
BINARY	SMALLINT
BIT	DATE
CHAR	TIME
DECIMAL	TIMESTAMP
DOUBLE	TINYINT
FLOAT	VARBINARY
GUID	VARCHAR
INTEGER	WCHAR
LONGVARBINARY	WLONGVARCHAR
LONGVARCHAR	WVARCHAR
NUMERIC	

The syntax for the explicit data type conversion function does not support specification of conversion format. The argument value_exp can be a column name, the result of another scalar function, or a numeric or string literal. For example:

```
{ fn CONVERT( { fn CURDATE() }, CHAR ) }
```

converts the output of the CURDATE scalar function to a character string. Because FireDAC does not mandate a data type for return values from scalar functions (because the functions are often data source–specific), applications should use the CONVERT scalar function whenever possible to force data type conversion. The following two examples illustrate the use of the CONVERT function. These examples assume the existence of a table called EMPLOYEES, with an EMPNO column of type SQL_SMALLINT and an EMPNAME column of type SQL_CHAR. If an application specifies the following SQL statement:

```
SELECT EMPNO FROM EMPLOYEES WHERE {fn CONVERT(EMPNO,CHAR)} LIKE '1%'
```

- A driver for ORACLE translates the SQL statement to:

```
SELECT EMPNO FROM EMPLOYEES WHERE to_char(EMPNO) LIKE '1%'
```

- A driver for SQL Server translates the SQL statement to:

```
SELECT EMPNO FROM EMPLOYEES WHERE convert(char,EMPNO) LIKE '1%'
```

If an application specifies the following SQL statement:

```
SELECT {fn ABS(EMPNO)}, {fn CONVERT(EMPNAME,SMALLINT)} FROM EMPLOYEES WHERE EMPNO <> 0
```

A driver for ORACLE translates the SQL statement to:

```
SELECT abs(EMPNO), to_number(EMPNAME) FROM EMPLOYEES WHERE EMPNO <> 0
```

A driver for SQL Server translates the SQL statement to:

```
SELECT abs(EMPNO), convert(smallint, EMPNAME) FROM EMPLOYEES WHERE EMPNO <> 0
```

1.5.1.6 RETURNING unified support

FireDAC supports the Firebird, Oracle and PostgreSQL **RETURNING** phrase.

Description

FireDAC offers {INTO} escape sequence to simplify and unify RETURNING handling for Firebird, Oracle and PostgreSQL:

```
ADQuery1.SQL.Text := 'insert into MyTab (f2, f3) values (:f2, :f3) returning f1 {into :f1}';
ADQuery1.Params[0].AsString := 'qwe';
ADQuery1.Params[1].AsInteger := 100;
ADQuery1.ExecSQL;
ADQuery1.Params[2].Value; // Value of a first parameter, listed in INTO
```

The parameters inside of {INTO} will be defined as ptInputOutput. FireDAC will use RETURNING {INTO} as part of the INSERT/UPDATE SQL commands, automatically generated to post updates to a database.

Note, FireDAC cannot support the Firebird INTO phrase directly, because it is supported only by PSQL, but not by DSQL, which is used by FireDAC. But you can use {INTO} escape clause. Other option with Firebird is to use Open method instead of ExecSQL:

```
ADQuery1.SQL.Text := 'insert into MyTab (f2, f3) values (:f2, :f3) returning f1';
ADQuery1.Params[0].AsString := 'qwe';
ADQuery1.Params[1].AsInteger := 100;
ADQuery1.Open;
ADQuery1.Fields[0].Value; // Value of a first field, listed in RETURNING
```

1.5.2 Executing Command

FireDAC offers TADQuery, TADCommand components and some TADConnection methods to execute SQL commands. The SQL commands are the main way to talk to SQL database.

Description

Using TADConnection

TADConnection (see page 279) offers ExecSQL (see page 343) methods. These are simple to use overloaded methods useful when:

- no result sets are returned;
- a SQL command will be executed only once, so does not need to be stored in prepared state;
- no need for advanced parameters setup;
- no need for design-time SQL command setup.

For example to execute few DDL commands:

```
ADConnection1.ExecSQL('drop table testtab');
ADConnection1.ExecSQL('create table testtab (id integer, name varchar(10))');
ADConnection1.ExecSQL('insert into testtab values (1, ''FireDAC'')');
```

To execute a parameterized query use the other overloaded method:

```
ADConnection1.ExecSQL('insert into testtab values (:id, :name)', [1, 'FireDAC']);
```

Also, TADConnection offers ExecSQLScalar (see page 344) methods different from ExecSQL methods, that they are returning a single result set value:

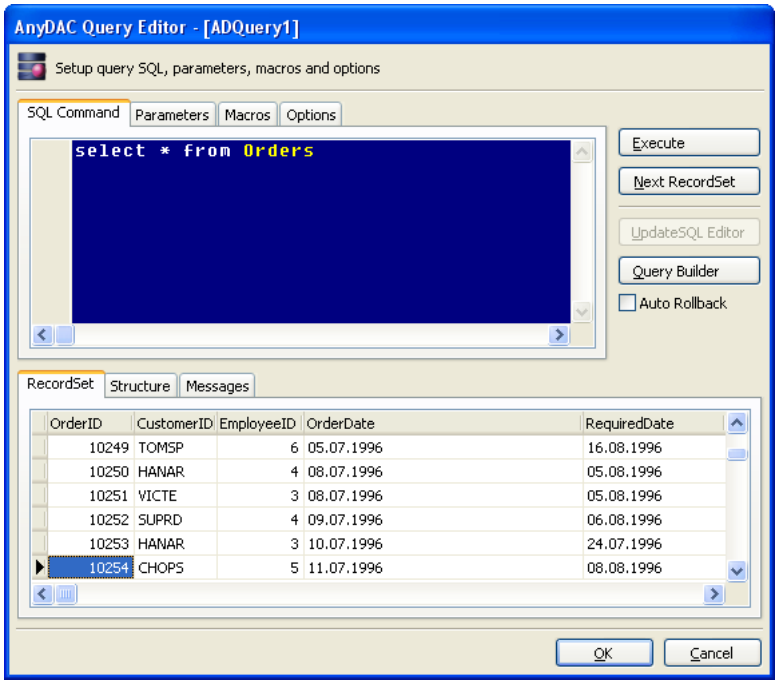
```
sName := ADConnection1.ExecSQLScalar('select name from testtab where id = :id', [1]);
```

Using TADQuery

In general, the TADQuery (see page 476) may be setup at design-time and / or at run-time.

Setting query at design time

To setup it at design-time, drop TADQuery on a form. TADQuery.Connection (see page 503) will be automatically set to point to a TADConnection on this form, if any. Then double click on TADQuery to invoke the FireDAC Query Editor:



There you can specify SQL text, optionally specify parameter and macro values and set options. Press "Query Builder" button to invoke the graphical SQL query builder tool. Note, to enable syntax highlighting check QU4 at GUI Questions (see page 242).

Press "Execute" button to execute the query. If a command returns a result set, then it will be output on "RecordSet" pane and the result set structure on the "Structure" pane. If DBMS returns any messages or warnings for a command, they will be put to "Messages" pane. The "Next RecordSet" button allows to walk through all result sets returned by command.

When you are testing DML commands, like a UPDATE / INSERT / DELETE, consider to mark "Auto Rollback" check box to automatically rollback actions performed by a command.

Press OK to store changes in the TADQuery (see page 476).

Using parameters

The parameterized query usage is one of the best practices at SQL database application development. Main benefits are:

- you may compose single SQL command and use it few times with different parameter values. DBMS will build command execution plan only once. That reduces the DBMS engine load.
- when you will execute the command next time, only parameter values will be transferred to the DBMS engine. That reduces network load.

- you will not care about correct SQL constant formats, for example - how to properly write a date constant in Microsoft Access SQL.

To put a parameter marker into the SQL text use :<name> syntax. Then assign corresponding values using Params (see page 651) collection. For example:

```
ADQuery1.SQL.Text := 'select * from tab where code = :Code';
ADQuery1.ParamByName('code').AsString := '123';
ADQuery1.Open;
```

Before query will be executed each parameter Name, DataType and ParamType must be filled. Also Position may be need to set.

When you assign SQL (see page 396) property and ResourceOptions.ParamCreate (see page 883) is True, the Params (see page 651) collection will be automatically filled in. Note, that TADParam.Name and TADParam.Position only will be set. The DataType, ParamType and other properties must be set by application. The parameters in Params collection appears in the same order as in the SQL command.

Also you can fill the Params (see page 651) by code. For that you have to turn off automatic parameters creation by setting ResourceOptions.ParamCreate (see page 883) to False. For example:

```
ADQuery1.ResourceOptions.ParamCreate := False;
ADQuery1.SQL.Text := 'select * from tab where code = :Code';
with ADQuery1.Params do begin
  Clear;
  with Add do begin
    Name := 'CODE';
    DataType := ftString;
    Size := 10;
    ParamType := ptInput;
  end;
end;
```

The parameters may be bind to the SQL command either by name, either by position. Use the Params.BindMode property to control that:

- pbByName - the parameters in Params (see page 651) collection will be bind by the Name property to the corresponding parameter markers. If SQL command has few the same named parameter markers, then only single instance will appear in Params (see page 651).
- pbByNumber - the parameters in Params (see page 651) collection will be bind by the Position property to the markers in SQL command text. If SQL command has few the same named parameter markers, then every one occurrence will appear in Params (see page 651). When Position is not set for parameters, it will be assigned automatically at command preparation using parameter indexes. The Position starts from 1.

The DataType may be specified either explicitly, either implicitly by assigning a parameter value to the Value or AsXxxx properties. Otherwise, the FormatOptions.DefaultParamDataType (see page 863) will be used.

When ParamType is not specified, the ResourceOptions.DefaultParamType (see page 881) will be used. By default all parameters are input only. If some of the parameters are output, then this must be specified explicitly.

The input parameter values must be set before query execution. That may be done using one of the ways:

- by explicitly assigning parameter values:

```
ADQuery1.ParamByName('code').AsString := 'DA1001';
ADQuery1.Open;
```

- by using one of the overloaded ExecSQL (see page 399) or Open (see page 510) methods:

```
ADQuery1.Open('', ['DA1001']);
```

To set parameter value to Null, specify parameter data type, then call Clear method:

```
with ADQuery1.ParamByName('name') do begin
  DataType := ftString;
  Clear;
end;
ADQuery1.ExecSQL;
```

The output parameter values are accessible after query is executed. Some SQL commands and DBMS may require first to process all command result sets, then output parameter values will be received. To force the output parameter values delivery use the TADAdaptedDataSet.GetResults (see page 265) method. Also, read "RETURNING unified support (see page 66)" article about output parameters in Firebird and Oracle RETURNING phrase.

Preparing the query

Before a query will be executed it must be prepared. For that FireDAC automatically performs the following actions:

- brings the connection to active (see page 37) or online (see page 40) mode;
- checks that all parameters are set correctly;
- preprocesses command text (see page 56);
- optionally applies RecsSkip (see page 859) and RecsMax (see page 858) to the command text;
- optionally starts a transaction for Firebird / Interbase DBMS, if there is no one active;
- transfers the final command text to the DBMS API. The final text may be get using TADQuery.Text (see page 397) property.

While the command is prepared the connection must be active and the query will keep server resources. To unprepare query and release all resources use Unprepare (see page 512) or Disconnect (see page 264) methods.

When you need to execute a command only once, you may set ResourceOptions.DirectExecute (see page 881) to True. For some DBMS (SQL Server, SQL Anywhere) that will speed up execution, because will avoid the costly operation of SQL command preparation.

Executing the query

To execute a query, which does not return a result set, use the ExecSQL (see page 397) methods. If a query returns a result set, then exception "[FireDAC][Phys][MSAcc]-310. Cannot execute command returning result sets" will be raised.

To execute a query, returning a result set and open this result set, use the Open (see page 509) methods. If a query returns no result sets, then exception "[FireDAC][Phys][MSAcc]-308. Cannot open / define command, which does not return result sets" will be raised.

To execute an add-hock query, use ExecuteOrOpen (see page 641) method. Note, that query may be executed asynchronously (see page 86). If the query is a command batch, then check "Command Batches (see page 81)" for details.

"Parameter data type is changed" Exception

When the parameter type was changed after the first query Prepare (see page 511) / ExecSQL (see page 397) / Open (see page 509) call, then FireDAC will raise an exception on a subsequent call:

```
[FireDAC][Phys][IB]-338. Parameter [Xxxx] data type is changed.
Query must be reprepared.
```

FireDAC at first Prepare (see page 511) / ExecSQL (see page 397) / Open (see page 509) call remembers the parameter data type. The data type may be specified either explicitly by setting TADParam.DataType property, either implicitly by assigning value using TADParam.AsXxxx or TADParam.Value properties. When before next ExecSQL / Open call an application changes parameter data type, then on the next ExecSQL / Open call "Parameter [xxxxx] data type is changed" will be raised.

The TADParam.Value property does not change parameter data type on the consequent calls. It casts the assigning value to the current data type. So, to avoid the above exception use:

- TADParam.Value property value;
- TADParam.AsXxxx property, as Xxxx was used for first time.

Getting DBMS feedback

Use the TADQuery.RowsAffected (see page 513) property to get the number of rows processed by the command. For example, the number of deleted rows by the DELETE command. Note, at MS SQL Server RowsAffected (see page 478) may be unexpectedly equal to -1 when a stored procedure or a table trigger omits SET NOCOUNT ON. And use the TADQuery.RecordCount property to get the number of fetched rows. Note, FireDAC does not provide "N rows processed" messages, if needed the application has to build it.

If the command will return an error, an exception will be raised. See the "Handling Errors (see page 44)" topic for more details. The exception may be processed using one of 3 ways:

- using try / except / end construction. For example:

```
try
  ADQuery1.ExecSQL;
except
  on E: EADDBEngineException do
    ; // do something here
end;
```

- setting TADQuery.OnError (see page 261) event handler;
- setting TADConnection.OnError (see page 332) event handler. For example:

```
procedure TForm1.ADConnection1Error(ASender: TObject; const AInitiator: IADStanObject;
  var AException: Exception);
begin
  if (AException is EADDBEngineException) and (EADDBEngineException(AException).Kind =
ekRecordLocked) then
    AException.Message := 'Please, try the operation later. At moment, the record is busy';
end;

ADConnection1.OnError := ADConnection1Error;
```

Note, in case of the Array DML (see page 82), the error handling may be more complex.

Also the command may return warnings, hints and messages, depending on a DBMS. To enable messages processing set ServerOutput (see page 890) to True. To process them use the TADConnection.Messages (see page 330) property. For example:

```
var
  i: Integer;
begin
  ADConnection1.ResourceOptions.ServerOutput := True;
  ADQuery1.ExecSQL;
  if ADConnection1.Messages <> nil then
    for i := 0 to ADConnection1.Messages.ErrorCount - 1 do
      Memo1.Lines.Add(ADConnection1.Messages[i].Message);
end;
```

Some DBMS, like SQL Server, returns messages as an additional result set. So, to process messages the application will need to process multiple result sets (see page 81). And more complex example, providing status and messages for SQL Server. As you see, we are using TADMemTable (see page 436) to store result set with rows.

```
var
  i: Integer;
begin
  ADConnection1.ResourceOptions.ServerOutput := True;
  ADQuery1.FetchOptions.AutoClose := False;
  ADQuery1.Open('select * from Region; print ''Hello''');
  ADMemTable1.Data := ADQuery1.Data;
  Memo1.Lines.Add(Format('%d rows processed', [ADMemTable1.RecordCount]));
  ADQuery1.NextRecordSet;
  if ADConnection1.Messages <> nil then
    for i := 0 to ADConnection1.Messages.ErrorCount - 1 do
      Memo1.Lines.Add(ADConnection1.Messages[i].Message);
end;
```

Query execution and transactions

By default all SQL command executions are performed in auto-commit mode (see page 893). That means, when right before command execution there was no active transaction, then it will be started. And right after command execution this implicit transaction will be finished:

- by Commit for successful execution;
- by Rollback for failure.

In case of Firebird and Interbase an implicit transaction will be started right before query preparation. Note, that there is no need to surround a single command execution into explicit transaction, just use the auto-commit mode.

Advanced

Most database applications have backend administration utilities, which must execute SQL script. These scripts are written using lines appropriate to the DBMS SQL script syntax. To execute SQL scripts (see page 88) use TADScript (see page 685) component.

Sometimes a database application may need to execute heterogeneous queries using tables from different databases or to execute SQL commands on TDataSet descendants instead of the database tables. To execute such queries use Local SQL engine (see page 96) and TADLocalSQL (see page 825) component.

See Also

Executing Stored Procedure (see page 71), Browsing Table (see page 73), Command Batches (see page 81), Asynchronous Execution (see page 86), Local SQL (see page 96), TADQuery Class (see page 476)

1.5.3 Executing Stored Procedure

FireDAC offers TADStoredProc component to execute database stored procedures, as a stored procedure may be executed directly.

Description

Using TADStoredProc

In general, the TADStoredProc (see page 514) may be setup at design-time and / or at run-time. The TADStoredProc transparently generates a SQL command to call a stored procedure basing on the TADStoredProc property values. Before executing FireDAC sends the parameter values to a DBMS, then executes a stored procedure and receives the output parameter values.

Note, FireDAC does not support parameters with the default values.

Setting stored proc at design time

To execute a stored procedure drop a TADStoredProc component on a form. TADStoredProc.Connection (see page 503) will be automatically set to point to a TADConnection on this form, if any.

Then optionally set CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404) properties or choose their values from a drop down list. After setting StoredProcName (see page 406) and when fiMeta is included into FetchOptions (see page 523).Items (see page 855), the Params (see page 534) collection will be filled in automatically.

To unify parameter names, set ResourceOptions (see page 535).UnifyParams (see page 885) to True. For example, this will exclude '@' prefix from the SQL Server stored procedure parameter names.

Setting stored proc at run time

That is similar to design time, just must be done using code:

```
ADStoredProc1.StoredProcName := 'my_proc';
ADStoredProc1.Prepare;
// now the Params collection is filled in
```

The Prepare call fills the Params collection using the mkProcArgs meta data, when fiMeta is included into FetchOptions (see page 523).Items (see page 855). Note:

- mkProcArgs querying may be time consuming;
- application cannot change parameter definitions after the Params is filled. For example, assignment to TADParam.AsXxxx properties implicitly sets the parameter data type.

To avoid above issues, exclude fiMeta from FetchOptions.Items, to avoid automatic rebuilding of Params collection at Prepare call. And semi-automatically fill the Params collection before the Prepare or ExecProc calls using the code:

```
ADStoredProc1.StoredProcName := 'my_proc';
ADStoredProc1.FetchOptions.Items := ADStoredProc1.FetchOptions.Items - [fiMeta];
ADStoredProc1.Command.FillParams(ADStoredProc1.Params);
```

Or manually:

```
ADStoredProc1.StoredProcName := 'my_proc';
ADStoredProc1.FetchOptions.Items := ADStoredProc1.FetchOptions.Items - [fiMeta];
with ADStorecProc1.Params do begin
  Clear;
  with Add do begin
    Name := 'Par1';
    ParamType := ptInput;
    DataType := ftString;
    Size := 50;
  end;
  with Add do begin
    Name := 'Par2';
    ParamType := ptOutput;
    DataType := ftInteger;
  end;
end;
```

Using packaged procedures

To use a packaged procedure, application must specify the package name. That must be done by either of ways:

- set PackageName (see page 404). The package name may be specified in [<catalog name>.][<schema name>.]<package name> format.
- set StoredProcName (see page 406). The stored procedure name may be specified in [<catalog name>.][<schema name>.]<package name>.<stored proc name> format.

To choose the overloaded procedure, application must specify the Overload (see page 404) property. For example:

```
ADStoredProc1.PackageName := 'SYS.DBMS_SQL';
ADStoredProc1.Overload := 1;
ADStoredProc1.StoredProcName := 'BIND_VARIABLE';
```

Executing the stored procedure

To execute a stored procedure, which does not return a result set, use the ExecProc (see page 410) methods. To execute a stored function, use the ExecProc (see page 410) or ExecFunc (see page 407) methods, where ExecFunc (see page 407) returns the function value. If a stored procedure returns a result set, then exception "[FireDAC][Phys][Oracl]-310. Cannot execute command returning result sets" will be raised.

Note, there are few overloaded ExecProc and ExecFunc methods, allowing to avoid TADStoredProc property usage and

specify all required information as method arguments. For example:

```
ADStoredProc1.StoredProcName := 'MY_PROC';
ADStoredProc1.Prepare;
ADStoredProc1.Params[0].Value := 100;
ADStoredProc1.Params[1].Value := 'audi';
ADStoredProc1.ExecProc;
```

Or more compact:

```
ADStoredProc1.ExecProc('MY_PROC', [100, 'audi']);
```

To execute a stored procedure, returning a result set and open this result set, use the Open (see page 509) methods. If a stored procedure returns no result sets, then exception "[FireDAC][Phys][Oracl]-308. Cannot open / define command, which does not return result sets" will be raised. If the stored procedure returns multiple result sets, then check "Command Batches (see page 81)" for details.

Note, that stored procedure may be executed asynchronously (see page 86).

Using TADQuery

The main difference of TADStoredProc from a TADQuery, is that TADStoredProc automatically generates a stored procedure call using the parameters information. The SQL code calling a stored procedure may be executed directly using any FireDAC method of the SQL command execution. For example, call an Oracle packaged proc using TADQuery:

```
with ADQuery1.SQL do begin
  Clear;
  Add('begin');
  Add('  sys.dbms_sql.bind_variable(:c, :name, :value');
  Add('end;');
end;
ADQuery1.Params[0].AsInteger := 1;
ADQuery1.Params[1].AsString := 'p1';
ADQuery1.Params[2].AsInteger := 100;
ADQuery1.ExecSQL;
```

Using TADCommand

Finally, you can use TADCommand (see page 267) to execute a stored procedure. Most of the above discussions may be applied to the TADCommand too.

See Also

Executing Command (see page 66), Command Batches (see page 81), Asynchronous Execution (see page 86), TADStoredProc Class (see page 514)

1.5.4 Browsing Table

FireDAC offers TADTable component to browse database table data, filter and sort records, and edit the table data.

Description

Using TADTable

To browse a single database table, sort and filter records, and edit the data the FireDAC offers TADTable (see page 539) component. The TADTable transparently generates a SELECT statement basing on the TADTable property values and called methods and send it to a DBMS.

The TADTable has two main operation modes:

- Live Data Window mode. Allows bi-directional navigation through large data volumes with a minimal memory usage.

- Standard mode. This mode is similar to TADQuery. TADTable generates a single SELECT command and uses the result set to walk through table records.

To open a table, the TableName (see page 559) property must be specified. Additionally may be set IndexFieldNames (see page 550) or IndexName (see page 551) properties. Note, to use IndexName property, the fiMeta must be included into FetchOptions.Items (see page 855). For example:

```
ADTable1.TableName := 'CUSTOMERS';
ADTable1.IndexFieldNames := 'CustNo';
ADTable1.Open;
```

To force TADTable to quote field names set FormatOptions.QuoteIdentifiers (see page 867) to True.

Live Data Window mode

In Live Data Window (LDV) mode FireDAC queries and keeps in memory only 2 * FetchOptions.RowsetSize (see page 859) of records - a window into the table data. When application navigates through the table data, FireDAC automatically scrolls or positions the LDW to the required position. That gives benefits:

- minimizes memory usage and allows to work with large data volumes, similar to an unidirectional dataset;
- enables bi-directional navigation, in contrast to an unidirectional dataset;
- gives always fresh data, reducing the need to refresh dataset;
- does not give a delay to fetch all result set data, required to perform sorting, record location, jumping to last record, etc.

The Filter property, range filtering, IndexFieldNames (see page 550) and IndexName (see page 551) properties, Locate (see page 634) and Lookup (see page 637) methods, key locating, setting RecNo, setting a bookmark, etc are performed by additional SELECT commands or by setting additional phrases for the main SELECT command. After changing of Filter, IndexFieldNames (see page 550) or IndexName (see page 551) the current position in dataset may change. To preserve it save a bookmark before changing and restore it after changing.

In LDV mode the Filter property value is substituted as is into WHERE phrase. To make expression compatible with the DBMS and the local expression engine (see page 110), application may use FireDAC escape sequences (see page 56). For example:

```
ADTable1.Filter := 'DateField = {d ' + FormatDateTime('yyy-mm-dd',
Trunc(MonthCalendar1.Date)) + '}';
ADTable1.Filtered := True;
```

LDV by design always applies ORDER BY phrase to the SELECT commands. The key requirement for correct LDV work are:

- a table must have unique or primary key. See Unique Identifying Fields (see page 117) for more details.
- the server side sort collation and client side sort collation must be the same. Otherwise TADTable may produce duplicated rows and raise "unique key violation" error.

Although FireDAC minimizes the number of generated and executed SQL commands in LDV mode, still it produces more heavy DB load than TADQuery. So, application developers should carefully choose when to use TADTable and LDV.

Setting LDV

The LDV is used when all the following conditions are met, otherwise the standard mode is used:

- CachedUpdates (see page 546) = False (default value);
- FetchOptions.Unidirectional (see page 860) = False (default value);
- FetchOptions.CursorKind (see page 853) in [ckAutomatic, ckDynamic] (ckAutomatic is default value);
- table has primary or unique key.

The mode may be changed only when TADTable is inactive. Consequently, for example, the CachedUpdates (see page 546) may be changed only when table is inactive. Additionally, the performance may be improved by changing FetchOptions.LiveWindowParanoic (see page 856) (False is default value) and LiveWindowFastFirst (see page 856) (False is default value).

Avoiding "unique key violation" error in LDV

To make the database sort order the same as the client side sort order, the FormatOptions.SortLocale (see page 867) and SortOptions (see page 868) may be used to adjust the client side sort order. For example, German speaking developers may setup TADTable to query the Firebird database with ISO8859_1 character set containing German language string data:

```
uses
  Windows;
...
// Set locale ID to German phone book collation
ADTable1.FormatOptions.SortLocale := MAKELCID(MAKELANGID (LANG_GERMAN, SUBLANG_GERMAN),
SORT_DEFAULT);
// Use the the punctuation and other symbols insensitive sorting
ADTable1.FormatOptions.SortOptions := [soNoSymbols];
ADTable1.IndexFieldNames := 'NAME';
ADTable1.TableName := 'CUSTOMERS';
ADTable1.Open;
```

Depending on a DBMS the following additional settings may be made:

DBMS	Settings
Firebird	May be required to include soNoSymbols into SortOptions.
MySQL	May be required to include soNoSymbols into SortOptions.
Oracle	May be required to execute ALTER SESSION SET NLS_COMP=ANSI.
SQLite	Set SortLocale to 0.

For example, Oracle developers may execute the command:

```
ADConnection1.Connected := True;
ADConnection1.ExecSQL('ALTER SESSION SET NLS_COMP=ANSI');
```

Also FormatOptions.StrsTrim (see page 869) we recommend to set to False.

Note, when a developer fails to adjust the client side sort collation, then the LDV may be turned off by setting FetchOptions.CursorKind (see page 853) to ckDefault, ckStatic or ckForwardOnly.

See Also

Object Names (see page 133), Executing Command (see page 66), TADTable (see page 539), TADFetchOptions (see page 849)

1.5.5 Database Alerts

FireDAC offers the TADEventAlerter component to listen for database alerts, send the alerts and manage them.

Description

General

The DBMS alert refers to a database notification or alert sent by a database trigger or stored procedure with intent to notify a database client about some events at database side.

An alert is identified by the name and may include additional arguments. The clients are registering with the alerts. Multiple clients may register with a single alert, and single client may register with the multiple alerts. When an alert is signaled in a database, then all registered clients will be notified about that. When application is not more interested in an alert, it unregisters with the alert.

The classic examples of the alerts are:

- a table data change. In this case an application may refresh a dataset returning this table data.

- a notification that some condition in data is met.
- a notification to other application that one application will perform some special task, like data archiving or backup.

Each DBMS implements DBMS alerts on their own. There is no standard for alerts mechanisms.

Using TADEventAlerter

FireDAC offers unified alerts API as the TADEventAlerter (see page 422) component. Some DBMS may implement few DBMS alert mechanisms. Each TADEventAlerter object may listen for few alerts by specifying their names in TADEventAlerter.Names (see page 360) property, and using single mechanism specified by TADEventAlerterOptions.Kind (see page 848) property.

FireDAC listens for alerts in a background thread using an additional private connection to the database. The additional connection is created automatically by FireDAC for each TADEventAlerter component. When the application creates multiple TADEventAlerter objects, consider to use pooled connections (see page 46) to improve performance.

To start receive the event alerts, fill in TADEventAlerter.Names (see page 360) property with required event names. Set Options (see page 361).Kind (see page 848) to the event alerter kind or leave it empty to use the default alerter. Specify OnAlert (see page 361) event handler, which will be fired when an event is occurred. And set Active (see page 359) to True or call Register (see page 362) method. To stop receive the event alerts, set Active (see page 359) to False or call Unregister (see page 362).

The OnAlert event handler may be called in the main or background thread contexts. Use Options.Synchronize property to control that. Note, application should minimize the run time of the background thread handler.

Application may set timeout for alerts by specifying Options (see page 361).Timeout (see page 849) property. When there are no alerts for specified time, then the OnTimeout (see page 361) event handler will be called.

For example, to register for "Customers" alert using "DBMS_ALERT" mechanism on Oracle database and standard Firebird mechanism, use the code:

```
ADEventAlerter1.Names.Clear;
ADEventAlerter1.Names.Add('Customers');
case ADConnection1.RDBMSKind of
  mkOracle:    ADEventAlerter1.Options.Kind := 'DBMS_ALERT';
  mkInterbase: ADEventAlerter1.Options.Kind := 'Events';
end;
ADEventAlerter1.Options.Synchronize := True;
ADEventAlerter1.Options.Timeout := 10000;
ADEventAlerter1.OnAlter := DoAlert;
ADEventAlerter1.OnTimeout := DoTimeout;
ADEventAlerter1.Active := True;
........

procedure TForm1.DoAlert(ASender: TADCustomEventAlerter;
  const AEventName: String; const AArgument: Variant);
begin
  if CompareText(AEventName, 'Customers') = 0 then
    qryCustomers.Refresh;
end;

procedure TForm1.DoTimeout(ASender: TObject);
begin
  // do something
end;
```

And the server side code for Oracle:

```
CREATE OR REPLACE TRIGGER TR_CUSTOMERS
AFTER INSERT OR UPDATE OR DELETE ON CUSTOMERS
BEGIN
  SYS.DBMS_ALERT.SIGNAL('Customers', '123');
END;
```

And for Firebird:

```
CREATE TRIGGER TR_CUSTOMERS FOR CUSTOMERS
```

```
ACTIVE AFTER INSERT OR UPDATE OR DELETE
BEGIN
  POST_EVENT 'Customers';
END;
```

DBMS alert mechanisms

As noted, every DBMS implements database alerts on her own. The alter mechanism kind is identified by TADEventAlerterOptions.Kind (see page 848) property value. If it is empty, then default mechanism will be used. For most mechanisms the client side functionality will be the same. The database side will differ.

The following table lists the DBMS and their alter mechanisms, supported by FireDAC drivers:

DBMS	Event alerter kind	Description
Advantage Database	Events (*)	Is used the standard Events (Notifications) functionality. To initiate an event, use sp_SignalEvent stored procedure. For example: sp_SignalEvent('Customers', true, 0, '123'); Note, that sp_SignalEvent may be called only from a stored procedure or a trigger. It cannot be called directly from the SQL command.
Sybase SQL Anywhere	Message (*)	Is used the MESSAGE statement functionality. To initiate an event, the specially formatted message must be sent: _AD_$$<event name>[$$<argument>]. For example: MESSAGE '_AD_$$Customers$$123'
DataSnap server	Callbacks (*)	Is used DataSnap “heavyweight” callbacks (more). The TADEventAlerter.Names content should have one of the following formats: • <channel name>. FireDAC will register a callback for the specified channel. Use TDSAdminClient.BroadcastToChannel to broadcast an event to all TADEventAlerter registered for the specified channel. • <channel name>=<callback name>. FireDAC will register a callback with the specified name for the specified channel. See above to initiate an event.
Firebird / Interbase	Events (*)	Is used the standard FB/IB mechanism for event notifications. To initiate an event, use POST_EVENT <name> statement. For example: EXECUTE BLOCK AS BEGIN POST_EVENT 'Customers'; END;

Microsoft SQL Server	Query Notification (*)	Is used the Query Update Notification service. The TADEventAlerter.Names content should have one of the following formats: • CHANGE<index>=<message>;<SELECT query>. The event will be fired when the data returned by the SELECT query will be updated and the <message> will be returned as the event name. To fire an event an UPDATE statement against the selected data must be executed. • <message>. FireDAC will create _AD_EVENTS table. The event will be fired when VALUE of the NAME=<message> row will be updated. To fire an event an UPDATE statement must be performed. Additionally parameters may be specified in the Names: • SERVICE=<name>. The name of the service to use. '?' - means to create an uniquely named service and drop it after usage. • QUEUE=<name>. The name of the message queue to use. '?' - means to create an uniquely named queue and drop it after usage. Note, to enable query notification execute the command: ALTER DATABASE <your db name> SET ENABLE_BROKER
Oracle	DBMS_ALERT (*)	Is used the DBMS_ALERT package. Before the usage a DBA must execute GRANT EXECUTE ON DBMS_ALERT TO <user or group>. To initiate an event, use DBMS_ALERT.SIGNAL call. For example: BEGIN SYS.DBMS_ALERT.SIGNAL('Customers', '123'); END;
	DBMS_PIPE	Is used the DBMS_PIPE package. Before the usage a DBA must execute GRANT EXECUTE ON DBMS_PIPE TO <user or group>. To initiate an event, use DBMS_PIPE.SEND_MESSAGE call. For example: BEGIN SYS.DBMS_PIPE.PACK_MESSAGE(123); SYS.DBMS_PIPE.SEND_MESSAGE('Customers'); END;
PostgreSQL	Notifies (*)	Is used the standard event notification mechanism. To initiate an event, use NOTIFY <name> statement. PostgreSQL 9.0 supports payload argument, use NOTIFY <name> [, <paylod>]. For example: NOTIFY Customers
SQLite	Events (*)	Is used the custom function POST_EVENT. To initiate an event, use POST_EVENT(<name>, [arg1 [,arg2 [,arg3 [,arg4]]]]). For example: SELECT POST_EVENT('Customers', 123)

Note, the asterisk marks the default event alerter kind.

See Also

TADEventAlerter (see page 422), TADEventAlerterOptions (see page 847), Multi Threading (see page 46)

Example

See FireDAC demo FireDAC\Samples\Comp Layer\TADEventAlerter\Main.

1.5.6 Fetching Rows

When a SQL command is executed it may create a cursor returning the result set. FireDAC offers wide range of features to effectively process the cursor and fetch rows into the client records cache.

Description

Handling cursors

When a SQL command is executed and it has to return rows, the DBMS creates a cursor on the DBMS server. An application uses the cursor to retrieve rows from a database. Depending on the DBMS there may be few cursor kinds. To choose the cursor kind use the FetchOptions.CursorKind (see page 853) property. By default FireDAC will choose most fast cursor kind (ckAutomatic). At first this property is meaningful for the Microsoft SQL Server.

The DBMS cursors are sensitive to the transaction context, where they were opened. For more details read the "Managing Transactions (see page 41)" chapter.

Rowset fetching

Rowset Fetching allows you to specify the number of records that will be fetched from the server in one network round trip. You can optimize this separately for each SELECT statement that you execute, thereby minimizing the number of network round trips by specifying the RowsetSize option. For example, Zeos DBO does not support Rowset Fetching, so FireDAC is a few times faster than Zeos DBO at fetching.

The row set size is controlled by the FetchOptions.RowsetSize (see page 859) property. Than higher is the number, then faster FireDAC will fetch the full result set, but the delay to fetch next row set will be higher too. Also, starting from some "high" value the performance stops to grow and even may start to degrade. The practically "high" value is 2000-3000.

Not all DBMS are supporting row set fetching. If so, FireDAC will emulate it. Even emulation speedups fetching for <= 50%.

FireDAC is fetching row sets according to the FetchOptions.Mode (see page 857) property:

- fmOnDemand - row sets will be fetched automatically, when dataset is trying to move the current position beyond the last fetched record.
- fmAll - all row sets will be fetched automatically right after executing the SQL command. This is similar to call the FetchAll (see page 624) method.
- fmManual - programmer will manually fetch row sets using the FetchNext (see page 625) or FetchAll (see page 624) methods.
- fmExactRecsMax - all rows sets will be fetched automatically right after executing the SQL command. If the number of rows will be different from FetchOptions.RecsMax (see page 858), then an exception will be raised.

When the FetchOptions.Unidirectional (see page 860) is True, then before fetching the next row set, the previous one will be discarded from memory. That allows to preserve the PC memory when fetching large result sets.

When all records are fetched, then TADDataSet.SourceEOF (see page 608) will be True, and depending on the FetchOptions.AutoClose (see page 851) the underlying command will be closed. This will not close the dataset itself. Also, read "Command Batches (see page 81)" for more details.

Application may get access to each fetched record using TADDataSet.AfterGetRecord (see page 589) event handler.

Rows paging

The FetchOptions.RecsSkip (see page 859) and RecsMax (see page 858) allows to page through the result set. After the cursor will be opened, the first RecsSkip records will be skipped, then the next up to RecxMax records will be fetched. Changing RecsSkip and RecsMax property values has no effect, when a statement is prepared. So, before to fetch the next rows page, the command must be unprepared, then executed again. For example:

```
ADQuery1.FetchOptions.RecsSkip := 0;
ADQuery1.FetchOptions.RecsMax := 100;
ADQuery1.Open;
// process rows

ADQuery1.Disconnect;
ADQuery1.FetchOptions.RecsSkip := 100;
ADQuery1.Open;
```

```
// process rows

ADQuery1.Disconnect;
ADQuery1.FetchOptions.RecsSkip := 200;
ADQuery1.Open;
// process rows
```

When RecsSkip and / or RecsMax properties are specified, FireDAC if possible will modify the original SELECT command, to apply TOP / ROWS and similar phrases.

Delayed fetching

The result set may include BLOB or / and nested datasets columns. In general, such columns slowdowns result set fetching. FireDAC allows to postpone such columns fetching until they values will be really needed. The FetchOptions.Items (see page 855) property controls that:

- when fiBlobs is excluded, then FireDAC will delay BLOB values fetching. The FetchBlobs (see page 624) method performs BLOB values fetching for the current dataset record. Alternatively, the first reading of a BLOB value will automatically call FetchBlobs, when Mode (see page 857) <> fmManual.
- when fiDetails is excluded, then FireDAC will delay nested dataset fetching. The FetchDetails (see page 625) method performs nested dataset fetching for the current record. Also, fiDetails controls the master-details handling (see page 104).

For the delayed fetching SQL command generation, read the "Update Command Generation (see page 121)" chapter.

Note, that excluding fiBlobs or fiDetails from FetchOptions.Items do not modify the SELECT list of a SQL command. If a DBMS transfers BLOB values by-value (eg, Oracle LONG, MySQL, SQL Server, SQLite), then in any case the BLOB values will be delivered to the client other the network. But they will be not stored in the client records cache. But if a DBMS transfers BLOB values by-reference (eg, Oracle CLOB / BLOB, Interbase, Firebird), then the BLOB values will be not delivered and not stored.

Refetching rows

Sometimes an application may need to append new result set to the existing one or to refetch rows or etc. For that the FetchAgain (see page 623) method may be used. Note, use the Refresh method for dataset refreshing.

General usage cases

The following table provides the common usage case and corresponding FetchOptions setup:

Case	Description
Minimal fetching time of large volume of records with limited memory usage.	CursorKind (see page 853) = ckDefault or ckForwardOnly Mode (see page 857) = fmOnDemand RowsetSize (see page 859) = 1000 Unidirectional (see page 860) = True
Minimal fetching time with delay at query opening.	CursorKind (see page 853) = ckDefault Mode (see page 857) = fmAll RowsetSize (see page 859) = 1000
Minimal delay at query opening.	CursorKind (see page 853) = ckDynamic Mode (see page 857) = fmOnDemand RowsetSize (see page 859) = 50 Exclude fiMeta from Items (see page 855).
Read-only dataset.	Exclude fiMeta from Items (see page 855). Or set UpdateOptions.RequestLive (see page 904) to False.

Batch command with multiple result sets.	AutoClose (see page 851) = False
Minimal fetching time of large volume of records with multiple large BLOB values with limited memory usage.	Exclude fiBlobs from Items (see page 855). Combine with above.

See Also

Executing Command (see page 66), Executing Stored Procedure (see page 71), Command Batches (see page 81), Update Command Generation (see page 121)

1.5.7 Command Batches

FireDAC supports SQL command batches. Command batches allowing to execute multiple SQL commands in a single step and to process multiple result sets in a sequence.

Description

General

A Command Batch is a group of SQL statements sent at one time from an application to a DBMS for execution. DBMS compiles the statements of a batch into a single execution plan. The statements in the execution plan are then executed one at a time. This minimizes network traffic and server workload. After execution, the DBMS returns to the client result sets produced by the commands.

Processing result sets

FireDAC allows you to process all result sets one by one, using the NextRecordSet (see page 265) method. To enable processing of all result sets you should set FetchOptions.AutoClose (see page 851) to False before executing a command. Otherwise right after reaching Eof at first cursor, it will be closed and other cursors will be discarded.

For example with SQL Server:

```
ADQuery1.FetchOptions.AutoClose := False;
ADQuery1.SQL.Add('select * from [Orders]');
ADQuery1.SQL.Add('select * from [Order Details]');
ADQuery1.Open; // [Orders] table rows are accessable here
ADQuery1.NextRecordSet; // [Order Details] table rows available here
```

An application may store each result set into a separate dataset, using TADMemTable (see page 436) and Data (see page 595) property. For example:

```
ADQuery1.FetchOptions.AutoClose := False;
ADQuery1.SQL.Text := 'select * from orders; select * from customers';

ADQuery1.Open;
ADQuery1.FetchAll;
// assign orders records to ADMemTable1
ADMemTable1.Data := ADQuery1.Data;

ADQuery1.NextRecordSet;
ADQuery1.FetchAll;
// assign customers records to ADMemTable2
ADMemTable2.Data := ADQuery1.Data;
```

FireDAC automatically skips empty result sets that do not have columns and rows.

DMBS and batches

A DBMS must support command batches to execute them using FireDAC. If a DBMS does not support batches, then you can use SQL scripting (see page 685). The DBMS supporting batches are:

DBMS	Notes
IBM DB2	Commands must be separated by ';'.
Firebird	Use the EXECUTE BLOCK construction.
Microsoft SQL Server	Commands may be optionally separated by ';'.
MySQL	Commands must be separated by ';'.
Oracle	Use the BEGIN END anonymous block construction.
PostgreSQL	Commands must be separated by ';'. A batch without parameters may be executed with ResourceOptions.DirectExecute (see page 881)=True. With PostgreSQL v >= 9.0 use the DO BEGIN END anonymous block construction.
SQLite	Commands must be separated by ';'.
SQL Anywhere	Commands may be optionally separated by ';'.

Also, FireDAC fully supports commands returning multiple result sets. Some examples:

- Oracle stored procedures with REF CURSOR's;
- Oracle result sets with nested cursors;
- PostgreSQL varieties of cursor returning statements.

Note, to execute multiple INSERT / UPDATE / DELETE commands, consider to use Array DML (see page 82) feature, which is far more effective for large batches.

See Also

Array DML (see page 82)

1.5.8 Array DML

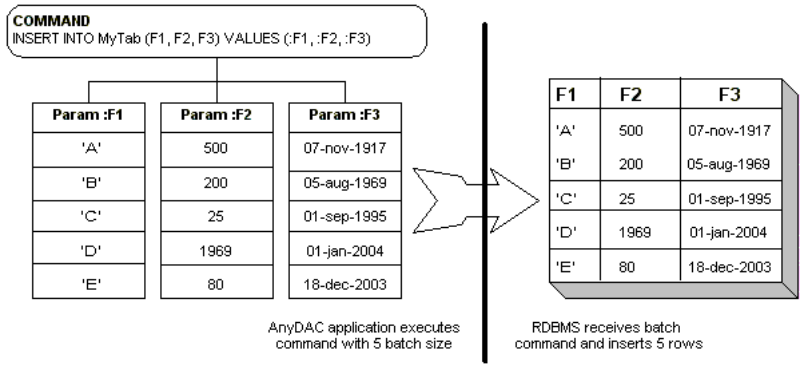
FireDAC offers high performance Array DML feature. It allows to move large volumes of data to a database using standard INSERT or UPDATE SQL commands.

Description

General

The Array DML execution technique submits a single DBMS command with an array of parameters. Each command parameter has array of values and all parameters have arrays of the same length. Then FireDAC requests the DBMS to execute a command once for each row in arrays. This technique reduces the amount of communication between DBMS and client, enables DBMS to stream command execution. That speeds up a execution in times.

Following picture shows that:



In FireDAC terms "batch command execution" and "Array DML execution" are used often as synonyms. Array DML may be used almost for all parameterized commands, including stored procedure calls. FireDAC implements Array DML using native DBMS API capabilities or emulates Array DML execution, if the DBMS API does not support it.

The following table lists DBMS and Array DML features:

DBMS	Array DML Implementation	Array DML Mode	Array DML Limit Symptoms
Advantage Database	Emulation	aeUpToFirstError	
DataSnap server	Emulation	aeUpToFirstError	
IBM DB2	Native	aeCollectAllErrors	
Interbase	Native / Emulation	aeUpToFirstError	
Firebird v < 2.1	Emulation	aeUpToFirstError	
Firebird v >= 2.1	Native (EXECUTE BLOCK)	aeOnErrorUndoAll	"Too many contexts" error
Microsoft SQL Server	Native	aeCollectAllErrors	Possible "Access violation" error
Microsoft Access database	Emulation	aeUpToFirstError	
MySQL Server	Native (INSERT with multiple VALUES)	aeOnErrorUndoAll	
Oracle Server	Native (OCI Array DML)	aeUpToFirstError	Application hangs up. Explicit limit - 65K of array items.
PostgreSQL v < 8.1	Emulation	aeUpToFirstError	
PostgreSQL v >= 8.1	Native (INSERT /MERGE with multiple VALUES)	aeOnErrorUndoAll	
SQLite database v < 3.7.11	Emulation	aeUpToFirstError	
SQLite database v >= 3.7.11	• Emulation, when Params.BindMode = pbByName • Native (INSERT with multiple VALUES), when Params.BindMode = pbByNumber	• aeUpToFirstError • aeOnErrorUndoAll	
Sybase SQL Anywhere	Native	aeUpToFirstError	

Note:

- "Array DML Mode" description see in "Error Handling" chapter.
- "Array DML Limit Symptoms" description see in "Troubleshooting" chapter.

Command execution

Before Array DML execution, the application code must setup parameter value arrays. First, setup array length by assigning value to Params.ArraySize. Assigning this property value, implicitly assigns specified array length to all parameters ArraySize property. So, Params collection must be not empty before assigning to Params.ArraySize. Second, assigns values to parameter arrays. TADParam class has a set of AsXXXs[AIndex: Integer] properties, similar to AsXXX properties. And other properties and methods accepting as first parameter array index. For example:

```
ADQuery1.SQL.Text := 'insert into MyTab values (:p1, :p2, :p3)';
// here ADQuery1.Params collection is filled by 3 parameters
ADQuery1.Params.ArraySize := 100;
for i := 0 to 100-1 do begin
  ADQuery1.Params[0].AsIntegers[i] := i;
  ADQuery1.Params[1].AsStrings[i] := 'qwe';
  ADQuery1.Params[2].Clear(i);
end;
```

TADCustomCommand (see page 291), TADQuery (see page 476) and TADStoredProc (see page 514) have the Execute (see page 622)(ATimes: Integer = 0; AOffset: Integer = 0) method. Here, ATimes defines the length of the array. AOffset is index of first item in the array. So, the command will be executed (ATimes - AOffset) times, starting from AOffset row. ATimes must be equal or less Params.ArraySize. For example:

```
ADQuery1.Execute(100, 0);
```

After Array DML execution, the property RowsAffected (see page 513) has the number of successful executions, not the total number of affected rows by all executions. For example:

```
ShowMessage(IntToStr(ADQuery1.RowsAffected));
```

Error handling

TADAdaptedDataSet, TADQuery and TADStoredProc have ability to trap errors using OnExecuteError (see page 262) event handlers. If the error handler is not assigned and an error happens, then Execute will raise an exception and RowsAffected (see page 513) will be updated.

If TADAdaptedDataSet.OnExecuteError (see page 262) event handler is assigned, it will get original exception object, current times and offset, and may return AAction value, talking what to do next. The AError.Errors (see page 834)[...] contains one or more errors. AError.Errors (see page 834)[i].RowIndex (see page 839) is a failed row index. Note, OnExecuteError will be not called for the syntax errors or when ATimes = 1.

For example:

```
procedure TForm1.ADQuery1ExecuteError(ASender: TObject; ATimes,
  AOffset: Integer; AError: EADDBEngineException; var AAction: TADErrorAction);
begin
  if AError.Errors[0].Kind = ekUKViolated then
    AAction := eaSkip
  else
    AAction := eaFail;
end;
```

The exact behavior depends on a DBMS and its corresponding Array DML mode:

Array DML Mode	Description
aeOnErrorUndoAll	Execution stops on the first error. All successfully applied array items will be undone. Then FireDAC switches to one-by-one execute mode and re-executes the full array. This is similar to aeUpToFirstError. See aeUpToFirstError below.

aeUpToFirstError	Execution stops on the first error. All successfully applied array items will be saved. DBMS returns the index of the first failed array item. RowsAffected = number of successfully applied array items. Collection of errors in AError.Errors (see page 834)[...] contains one or more errors referring to a single failed row. AError.Errors (see page 834)[i].RowIndex (see page 839) is the failed row index.
aeCollectAllErrors	All array items are executed. All successfully applied array items will be saved. DBMS returns one-by-one the index of each failed array item. RowsAffected = number of successfully applied array items. Collection of errors in AError.Errors (see page 834)[...] contains one error for each failed row. AError.Errors (see page 834)[i].RowIndex (see page 839) is a failed row index.

Note, setting ResourceOptions.ArrayDMLSize (see page 879) to 1 implicitly sets array execution mode to aeUpToFirstError. To get currently connected DBMS Array DML mode use:

```
if ADConnection1.ConnectionMetaDataIntf.ArrayExecMode = aeOnErrorUndoAll then
  ....
```

Troubleshooting

It is important to properly setup parameters, including setting property Size for the string parameters. For example, FireDAC in case of Oracle will allocate 4000 bytes for each ftString / ftWideString parameter, when Size is not explicitly specified. So, for 10,000 of values will be allocated 40 Mb buffer. If there are many parameters, then application can eat all the system memory.

Most DBMS have implicit limit for the Array DML size. It depends on the DBMS client library buffer size or the maximum allowed network packet. When a limit is reached use ResourceOptions.ArrayDMLSize (see page 879) option to transparently split large Array DML into few lesser slices.

See Also

Command Batches (see page 81)

Example 1

Array DML with IADPhysCommand:

```
var
  oCmd: IADPhysCommand;
……
  with oCmd do begin
    CommandText := 'insert into Customers (ID, Name) values (:ID, :Name)';
    // Set up parameter types
    Params[0].DataType := ftInteger;
    Params[1].DataType := ftString;
    Params[1].Size := 40;
    // Set up parameters' array size
    Params.ArraySize := 10000;
    // Set parameter values
    for i := 0 to 10000 - 1 do begin
      Params[0].AsIntegers[i] := i;
      Params[1].AsStrings[i] := 'Somebody ' + IntToStr(i);
    end;
    // Execute batch
    Execute(10000, 0);
  end;
```

Example 2

Array DML with TADQuery and error handling:

```
procedure TForm1.ADQuery1ExecuteError(ASender: TObject; ATimes,
  AOffset: Integer; AException: EADDBEngineException; var AAction: TADErrorAction);
begin
  case AException.Errors[0].Kind of
  ekPKViolated:
```

```
    begin
      // fix ID to be unique
      ADQuery.Params[0].AsIntegers[AException.Errors[0].RowIndex] :=
AException.Errors[0].RowIndex;
      AAction := eaRetry;
    end;
  ekFKViolated:
    // if Region with RegionID is not found, then just skip row
    AAction := eaSkip;
  else
    AAction := eaFail;
  end;
end;

procedure TForm1.Button1Click(ASender: TObject);
begin
  with ADQuery1 do begin
    SQL.Text := 'insert into Customers (ID, RegionID, Name, Note) values (:ID, :RegionID,
:Name, :Note)';
    // Set up parameter types
    Params[0].DataType := ftInteger;
    Params[1].DataType := ftInteger;
    Params[2].DataType := ftString;
    Params[2].Size := 40;
    Params[3].DataSize := ftMemo;
    // Set up parameters' array size
    Params.ArraySize := 10000;
    // Set parameter values
    for i := 0 to 10000 - 1 do begin
      if i mod 100 = 0 then
        // force PK violation
        Params[0].AsIntegers[i] := i - 1
      else
        Params[0].AsIntegers[i] := i;
      Params[1].AsIntegers[i] := GetRegionIdForCustomer(i);
      Params[2].AsStrings[i] := 'Somebody ' + IntToStr(i);
      Params[3].Clear(i);
    end;
    // Execute batch
    Execute(10000, 0);
  end;
end;
```

1.5.9 Asynchronous Execution

FireDAC offers multiple features for asynchronous operation execution. In addition to being ready for use in multi-threaded applications, FireDAC makes it easy for the developer to not have to worry about multi-threading in the first place by allowing you to control the timing aspects of the data access operation execution. This includes Execute, Open and Fetch operations.

Description

General

A programmer may choose between four operation modes using the ResourceOptions.CmdExecMode (see page 880) property:

Mode	Description
amBlocking	The calling thread and GUI are blocked until an action will be finished.
amNonBlocking	The calling thread is blocked until an action will be finished. The GUI is not blocked.
amCancelDialog	The calling thread and GUI are blocked until an action will be finished. FireDAC shows dialog, allowing to cancel an action.
amAsync	The calling thread and GUI are not blocked. The called method will return immediately.

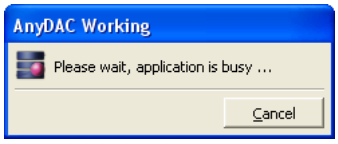
An application may check TADCommand.State (see page 306) or TADAdaptedDataSet.Command (see page 261).State (see page 306) for an operation status:

Status	Description
csInactive	A command is not prepared.
csPrepared	A command is prepared. A result set is not accessible.
csExecuting	A command execution is in progress.
csOpen	A command execution is finished. A result set is accessible and not yet fetched in full.
csFetching	A result set fetching is in progress.
csAborting	A command execution abortion is in progress.

For example:

```
ADQuery1.ResourceOptions.CmdExecMode := amAsync;
ADQuery1.Open;
while ADQuery1.Command.State = csExecuting do begin
  // do something while query is executing
end;
```

With help of TADGUIxAsyncExecuteDialog (see page 669) component, in the amCancelDialog mode a user may be notified about long running operations and have an ability to cancel operation:



To use the dialog component, drop it on a form. No additional setup is required. Read more about modes and operation notification events at ResourceOptions.CmdExecMode (see page 880) property description.

Asynchronous open and fetching

When an application needs to open a query pure asynchronously (amAsync) and the query is bind to the GUI using TDataSource, then before opening the query TDataSource must be disconnected from the query and after opening connected back. For example:

```
procedure TForm1.BeforeOpen(DataSet: TDataSet);
begin
  DataSource1.DataSet := nil;
end;

procedure TForm1.AfterOpen(DataSet: TDataSet);
begin
  DataSource1.DataSet := ADQuery1;
  ADQuery1.ResourceOptions.CmdExecMode := amBlocking;
end;

ADQuery1.BeforeOpen := ADQuery1BeforeOpen;
ADQuery1.AfterOpen := ADQuery1AfterOpen;
ADQuery1.ResourceOptions.CmdExecMode := amAsync;
ADQuery1.Open;
```

That is not required when amCancelDialog is used. Also fetching with GUI cannot be performed in amAsync mode. To perform opening a query and fetching large result set asynchronously set FetchOptions (see page 503).Mode (see page 857) = fmAll. Then both operations will be performed as a single background task. Note, that it is only valid when you need a bi-directional dataset.

To asynchronously execute Firebird query use separated transaction for this query.

Canceling long running operations

A programmer can specify the timeout for the data access operation using the ResourceOptions.CmdExecTimeout (see page 881) property. When an operation execution requires more than the specified time, then the execution will be canceled and exception will be raised. To catch a command timeout use the code:

```
try
  // set timeout to 5 seconds
  ADQuery1.ResourceOptions.CmdExecTimeout := 5000;
  ADQuery1.ExecSQL;
except
  on E: EADDBEngineException do
    if E.Kind = ekCmdAborted then
      ; // command is aborted
end;
```

Alternatively application may cancel operation execution by calling dataset or command AbortJob (see page 263) method from other thread. Or call connection AbortJob (see page 338) method to cancel all operations on this connection.

Note, that not all FireDAC drivers and DBMS are supporting execution cancellation. Also the cancellation may be performed not immediately, when A DBMS is performing some critical operations. The following table lists supported combinations:

DBMS	Notes
Advantage	
Firebird	v 2.5 or higher
IBM DB2	
Interbase	v 7.0 or higher
MySQL	v 5.0 or higher
Oracle	
PostgreSQL	
SQL Anywhere	
SQL Server	
SQLite	

Multiple asynchronous queries

Execution of the multiple asynchronous queries in parallel is not supported by FireDAC, due to the general multi-threading principle (see page 46). To workaround that consider the options:

- group few queries into a stored procedure and call the procedure asynchronously;
- use a dedicated connection for each asynchronous query running in parallel;
- launch next asynchronous query after previous one is finished. For that use the AfterOpen and AfterExecute (see page 589) event handlers;
- finally application may create own threads and execute the DB tasks in these threads (see page 46).

1.5.10 Executing SQL Scripts

Most database applications have backend administration utilities, which must execute SQL script. These scripts are written using lines appropriate to the DBMS SQL script syntax. TADScript (see page 685) is the FireDAC SQL script processor.

Description

General

A SQL script is a set of separated SQL, execution control and logging commands. The SQL scripts are useful for the backend maintenance tasks, such as backend SQL objects creation, dropping, upgrading, initial data loading and so on.

Many DBMS's allow to execute few SQL commands in a single TADQuery (see page 476).ExecSQL (see page 397) call as a batch, but with limitations. The differences between SQL script and SQL command batch are:

- script allows to use all possible SQL commands in single script. The batch may have limitation, depending on a DBMS. For example, Oracle anonymous PL/SQL block cannot contain a DDL commands.
- script may be split into few transactions. The batch must be performed in a single transaction.
- script allows to use non-SQL and custom commands. The batch may include only commands understanding by a DBMS.
- script may be split into subscripts. The batch may call stored procedures as a separated code blocks.
- script execution is fully controlled by the client. The batch execution is controlled only by the DBMS.
- script execution may be logged. The batch execution does not.
- script provides execution progress feedback. The batch execution does not.

TADScript (see page 685) has many advantages over standard utilities, like the ability to be completely integrated into the FireDAC application and the ability to extend the set of commands by the custom script commands. The TADScript (see page 685) component is aware of several industry standard SQL script syntaxes, including:

- Oracle SQL*Plus;
- Microsoft ISQL / OSQL;
- MySQL mysql.exe / mysqldump.exe;
- Firebird / Interbase ISQL.

For example, the following Firebird script creates a database, and may be executed using TADScript (see page 685):

```
SET SQL DIALECT 3;
SET NAMES UTF8;
SET CLIENTLIB 'C:\fb25\bin\fbclient.dll';
CREATE DATABASE 'E:\Test2.ib'
  USER 'sysdba' PASSWORD 'masterkey'
  PAGE_SIZE 16384
  DEFAULT CHARACTER SET NONE;

SET TERM ^ ;

CREATE PROCEDURE MY_PROC RETURNS (aParam INTEGER) AS
BEGIN
  aParam = 10;
END^
```

Executing the script

TADScript (see page 685) allows to execute a script from a file pointed by SQLScriptFileName (see page 703), if it specified. Otherwise from a script with a zero index from the SQLScripts (see page 703) collection. Note, a file with SQL script may be executed also using the ADExecutor (see page 180) utility. For example, to execute a script file:

```
with ADScript1 do begin
  SQLScriptFileName := 'c:\create.sql';
  ValidateAll;
  ExecuteAll;
end;
```

To execute a script in a memory:

```
with ADScript1 do begin
  SQLScripts.Clear;
  SQLScripts.Add;
```

```
  with SQLScripts[0].SQL do begin
    Add('INSERT INTO Brands VALUES (1, ''Audi'')');
    Add('INSERT INTO Brands VALUES (2, ''BMW'')');
  end;
  ValidateAll;
  ExecuteAll;
end;
```

Also there are few other methods, simplifying SQL script execution. You can control many other script execution aspects as from a Delphi code using ScriptOptions (see page 702), as using corresponding script control commands.

The script may also call other scripts, like a subroutines, using **@ <script>**, **@@ <script>**, **START <script>** or **INPUT <script>** commands. There <script> is either the item name from the SQLScripts collection, either the external file name. For example, the 'root' script executes 'first' and 'second' subscripts:

```
with ADScript1.SQLScripts do begin
  Clear;
  with Add do begin
    Name := 'root';
    SQL.Add('@first');  // explicitly call 'first' script
    SQL.Add('@second'); // explicitly call 'second' script
  end;
  with Add do begin
    Name := 'first';
    SQL.Add('create table t1 (...);');
    SQL.Add('create table t2 (...);');
  end;
  with Add do begin
    Name := 'second';
    SQL.Add('create procedure p1 (...);');
    SQL.Add('create procedure p2 (...);');
  end;
end;
ADScript1.ValidateAll;
ADScript1.ExecuteAll;
```

A SQL script may be executed as in full with a subscripts, using ExecuteAll (see page 691) method. As in step-by-step mode, using ExecuteStep (see page 694) method. The last method is useful for the GUI applications, allowing to execute add-hoc queries. The next command will be extracted and executed from the Position (see page 688) position in the script. To abort the script execution call the AbortJob (see page 690) method.

Separating commands

Each SQL command must be terminated by a command separator. The separator default value is a ';' and for MS SQL Server it is 'GO'. A control command does not need to be terminated by a command separator. The separator may be changed as from a Delphi code using CommandSeparator (see page 712) option, as from a SQL script using **SET CMDSEP <sep>** or **DELIMiter <sep>** commands. For example for SQL Server:

```
INSERT INTO Brands VALUES (1, 'Audi')
GO
INSERT INTO Brands VALUES (2, 'BMW')
GO
```

For Oracle:

```
INSERT INTO Brands VALUES (1, 'Audi');
INSERT INTO Brands VALUES (2, 'BMW');
```

Using a custom separator:

```
SET CMDSEP #
INSERT INTO Brands VALUES (1, 'Audi')#
INSERT INTO Brands VALUES (2, 'BMW')#
```

When a script contains a DB programming language commands or blocks, then additional consideration must be made:

DBMS	SQL commands	Description
Firebird	• CREATE FUNCTION • EXECUTE BLOCK <code block>	Must be terminated by '/', or a separator different from ';' must be set.
Oracle	• CREATE PROCEDURE / FUNCTION / PACKAGE / etc • BEGIN <code block> END	Must be terminated by '/', or a separator different from ';' must be set.
PostgreSQL	• CREATE FUNCTION • DO <code block>	No actions is required.

For example with Firebird:

```
SET CMDSEP #;
EXECUTE BLOCK ... #
SET CMDSEP ;#
INSERT INTO Brands VALUES (3, 'Mercedes');
```

Otherwise application may raise an error likes this:

```
[FireDAC][IB] Unexpected end of command line 3
```

Using parameters

A SQL script may reference to:

- parameters from the TADScript.Params (see page 701) collection. To define a parameter in script use **VARiable <name><type>=<value>** command. To define a parameter in the code add a parameter to the TADScript.Params (see page 701) before script execution.
- macros from the TADScript.Macros (see page 698) collection. To define a macro in script use **DEFine <name>=<value>** command. To define a macro in the code add a macro to the TADScript.Macros (see page 698) before script execution. To enable macro processing set MacroExpand (see page 716) to True (default) or execute **SET DEFINE ON** or **SET SCAN ON**.
- arguments using **&<argument number>** syntax. The arguments may be specified in the TADScript.Arguments (see page 696) property, as an argument to the TADScript.ExecuteFile (see page 692) or ExecuteScript (see page 693) methods, or as part of the @ / @@ commands.

For example to define and use parameters:

```
with ADScript1.SQLScripts[0].SQL do begin
  Add('VARIABLE name CHAR IN = ''aaa''');
  Add('VARIABLE id NUMBER INOUT');
  Add('INSERT INTO master (name) VALUES (:name) RETURNING id {INTO :id};');
  Add('INSERT INTO detail (fk_id, name) VALUES (:id, ''bbb'');');
end;
```

To define and reference a macro:

```
DEF tab=Brands
INSERT INTO !tab VALUES (1, 'Audi');
INSERT INTO !tab VALUES (2, 'BMW');
```

To define and use arguments:

```
ADScript1.Arguments.Add('Brands');
...
with ADScript1.SQLScripts[0].SQL do begin
  Add('INSERT INTO &1 VALUES (1, ''Audi'')');
  Add('INSERT INTO &1 VALUES (2, ''BMW'')');
end;
```

Getting feedback

To produce an execution log, you can enable spooling as from Delphi code using SpoolOutput (see page 718) and SpoolFileName (see page 718) options, as from a SQL script using **SPOol <name>** or **OUTput <name>** commands. The content of the spool output is controlled by the EchoCommands (see page 714), FeedbackCommands (see page 715), AutoPrintParams (see page 711), FeedbackScript (see page 715), IgnoreError (see page 716), Timing (see page 719), ColumnHeadings (see page 712), PageSize (see page 717), ServerOutput (see page 890) options.

You can use TADGUIxScriptDialog (see page 702) component to allow to a script execution engine to communicate with an end-user using a dialog. To interact with the end-user TADScript uses events, like the OnConsoleGet (see page 698), OnConsolePut (see page 698), OnGetText (see page 699), OnPause (see page 700), etc. And this dialog provides a standard implementation for these events. To show an execution progress, TADScript needs to know the total length of all scripts to execute. For that call the ValidateAll (see page 694) method before starting a script execution.

Resolving incompatibilities

The list of known TADScript incompatibilities with the original scripting utilities:

- Firebird ISQL works in non auto commit mode. TADScript / TADConnection by default has auto commit mode turned on. For better compatibility set ADConnection.TxOptions.AutoCommit (see page 893) to False before script execution. Or execute **SET AUTOCOMMIT OFF** script command.
- Microsoft ISQL outputs the **PRINT** command result. TADScript / TADConnection by default does not. To enable **PRINT** output set ResourceOptions.ServerOutput (see page 890) to True. Or execute **SET SERVEROUTPUT ON** script command.

See Also

TADScript (see page 685), ADExecutor (see page 180), SQL Script Control Commands (see page 92), Developing Custom Commands (see page 95)

1.5.10.1 SQL Script Control Commands

TADScript (see page 685) and ADExecutor (see page 180) utility support extended list of the SQL script execution control commands.

Description

Command	Description
(@ | @@ | START | INput) <script> [<arguments>]	Starts execution of the specified script. '@@' executes a script with a file name relative to parent script path. <arguments> is a comma-separated list of values. If not specified, then the parent arguments (see page 696) will be inherited.
ACCept <var> (NUMber|CHAR|DATE) [FORmat <fmt>] [DEFault <def>] [PROmpt '<prompt>' | NOPRompt] [HIDE]	Asks user to enter a parameter value. NUMber|CHAR|DATE - optional value type. FORmat <fmt> - Delphi format string used to parse DATE values. DEFault <def> - default value, when a user entered empty string. PROmpt '<prompt>' | NOPRompt - prompt to output to console.
CONnect <FireDAC connection string>	Connects to the specified DBMS - OpenConnection.

COPY FROM (FILE '<file>' | SQL '<sql>' [CONnect '<connection>']) TO (FILE '<file>' | TABle '<table name>' [CONnect '<connection>']) [TRUNCate | CLEar | ALWAYSinsert | APPend | UPDate | APPENDUPDate | DELete] [LOG (OFF | (SPOol|OUTput) | ([APPend] <log file name>)] [USING (FILE 'config file' | PARAMS '<definition>')]	Moves data between databases or text files. There: • FILE '<file>' - a name of a file with text data; • SQL '<sql>' - SELECT command or a DB table name; • TABle '<table name>' - name of a DB table; • CONnect '<connection>' - FireDAC connection string; • TRUNcate - truncate the destination DB table before loading the data; • CLEar - clear the destination DB table before loading the data; • ALWAYSinsert | APPend | UPDate | APPENDUPDate | DELete - the action to perform with the destination DB table; • USING PARAMS '<definition>' - there definition is a serialized TADDataMove component, as it is stored in a DFM. For example: "object TADDataMove TextFileName = 'Data.txt' LogFileName = 'Data.log' .... end" • USING FILE 'config file' - there config file is a external file, containing a serialized TADDataMove component.
DEFine [<name> | <name>=<value>]	DEFINE without name - prints all macros; DEFINE <name> - prints the specified macro; DEFINE <name>=<value> - sets the specified macro value.
DELIMiter <text>	Sets script command separator - ScriptOptions (see page 702).CommandSeparator (see page 712).
DISconnect	Disconnects from the DBMS - CloseConnection.
(EXECute | CALL) <procedure>	Executes the specified stored procedure.
EXIT	Stops script execution and commits changes.
QUIT	Stops script execution and rollback changes.
STOP	Stops script execution.
HELP	Shows help for all registered commands.
(HOst | !! | SHELL) <command>	Executes host shell command. For Windows is used call ShellExecute('open', ACommand).
PAUse <prompt>	Prints specified prompt and pauses script execution.
PROmpt <prompt>	Prints specified text.
PRInt <var 1> [..., <varN>]	Prints value of the specified parameters.
REMark <text>	Adds comment to the script.
SET ARRAY <value>	Specifies the rowset size - FetchOptions (see page 275).RowsetSize (see page 859).
SET AUTOcommit OFF|ON|<value>	Sets auto commit off, on or to commit each N commands - ScriptOptions (see page 702).CommitEachNCommands (see page 712).
SET AUTOPrint OFF|ON	Shows parameter values after command execution - ScriptOptions (see page 702).AutoPrintParams (see page 711).
SET BREAK OFF|ON	Stops script execution on error - ScriptOptions (see page 702).BreakOnError (see page 711).
SET (CMDSeparator | TERMinator) <value>	Specifies commands separators - ScriptOptions (see page 702).CommandSeparator (see page 712). TERM is for IB/FB only.
SET (DEFine | SCAN) OFF|ON	Controls macros expansion - ScriptOptions (see page 702).MacroExpand (see page 716).
SET DROPnonexistent ON|OFF	Allows to drop non-existent objects - ScriptOptions (see page 702).DropNonexistObj (see page 714).

SET ECHO OFF|(ON [SQL|ALL] [TRIM 50|<value>])	Outputs command text before it executing with optional trimming - ScriptOptions (see page 702).EchoCommands (see page 714), EchoCommandTrim (see page 714).
SET ENCoding ANSI | UTF8 | UTF16	Sets SQL script and log files encoding - ScriptOptions (see page 702).FileEncoding (see page 715).
SET (FEEDback|COUNT) 6|<value>|OFF|ON	Outputs feedback after command execution - ScriptOptions (see page 702).FeedbackCommands (see page 715).
SET HEAding OFF|ON	Outputs column names for a result set - ScriptOptions (see page 702).ColumnHeadings (see page 712).
SET LINESize 0|<value>	Sets line width - ScriptOptions (see page 702).LineSize (see page 716).
SET LONG 80|<value>	Outputs up to a specified amount of characters for a character or BLOB value - ScriptOptions (see page 702).MaxStringWidth (see page 717).
SET PAGESize 24|<value>	Sets the page length when printing a result set - ScriptOptions (see page 702).PageSize (see page 717).
SET PARAMARRAY 1|<value>	Specifies the size of the parameter array - ScriptOptions (see page 702).ParamArraySize (see page 717), Params (see page 277).ArraySize.
SET SERVEROUTPUT OFF|(ON [SIZE <value>])	Shows server output after command execution - ResourceOptions (see page 289).ServerOutput (see page 890), ServerOutputSize (see page 891).
SET (TERMout|CONsole) OFF|ON	Enables output to the console, including the command result sets, parameter values, statuses, errors, etc - ScriptOptions (see page 702).ConsoleOutput (see page 713).
SET TIMing OFF|ON	Outputs a command execution time - ScriptOptions (see page 702).Timing (see page 719).
SET TRIMout OFF|ON	Trims spaces from strings at console output - ScriptOptions (see page 702).TrimConsole (see page 719).
SET TRIMSpool ON|OFF	Trims spaces from strings at spool output - ScriptOptions (see page 702).TrimSpool (see page 719).
SET VERify OFF|ON	Outputs SQL command before execution with substituted macros - ScriptOptions (see page 702).Verify (see page 719).
(SPOol | OUTput) [OFF|[APPend] <spool name>]	without arguments - prints spooling status; OFF - turns spooling off; <spool name> - turns spooling on and redirects it to the specified item name in SQLScripts collection or to a file name.
UNDEFine <var1> [..., <varN>]	Undefines specified macros.
VARiable [<name> (NUMber|CHAR|NCHAR [(<size>)]] [TABle <size>] [IN|OUT|INOUT] [=<value>]	without arguments - prints all parameter values; <name> - defines a parameter with specified data type, size and direction; = <value> - assigns a value to the parameter.
CREATE DATABASE <db file name> [PAGE_SIZE <n>] [DEFAULT CHARACTER SET '<ch set>'] [USER '<user>'] [PASSWORD 'pwd']	Used for Firebird / Interbase connections to create database.
DROP DATABASE	Used for Firebird / Interbase connections to drop database.

See Also

Executing SQL Scripts (see page 88), ADExecutor (see page 180), TADScript (see page 685), TADScriptOptions (see page 709),

1.5.10.2 Developing Custom Commands

TADScript allows to extend the set of SQL script control execution commands.

Description

The set of the script control execution commands may be extended by the custom commands. You can find the sources of the existing commands in the uADCompScriptCommands unit and use them as a template for your own commands. The command class must be registered at the commands registry. For example, the **SPOOL** command code:

```
type
  TADSpoolScriptCommand = class(TADScriptCommand)
  private
    FMode: Integer;
    FFileName: String;
    FAppend: Boolean;
  public
    class function Help(): String; override;
    class procedure Keywords(AKwds: TStrings); override;
    function Parse(const AKwd: String): Boolean; override;
    procedure Execute(); override;
    // -1 - off, 0 - show, 1 - file
    property Mode: Integer read FMode;
    property Append: Boolean read FAppend;
    property FileName: String read FFileName;
  end;

{------------------------------------------------------------------------------}
class function TADSpoolScriptCommand.Help(): String;
begin
  Result := '(SPOol | OUTput) [OFF|[APPend] <spool name>] - turns spooling off, print it
status'#13#10 +
            '                                                    or redirect it to the name';
end;

{------------------------------------------------------------------------------}
class procedure TADSpoolScriptCommand.Keywords(AKwds: TStrings);
begin
  AKwds.Add('SPOol');
  AKwds.Add('OUTput');
end;

{------------------------------------------------------------------------------}
function TADSpoolScriptCommand.Parse(const AKwd: String): Boolean;
var
  iLastBmk: Integer;
  ucS: string;
begin
  iLastBmk := Parser.GetBookmark;
  ucS := Parser.GetUCWord;
  if ucS = 'OFF' then
    FMode := -1
  else if ucS = '' then
    FMode := 0
  else begin
    FMode := 1;
    FAppend := ADKeyWordMatch(ucS, 'APPEND', 3);
    if not FAppend then
      Parser.SetBookmark(iLastBmk);
    FFileName := Parser.GetLine();
    if FFileName = '' then
      FMode := -1;
  end;
```

```
  Result := True;
end;

{------------------------------------------------------------------------------}
procedure TADSpoolScriptCommand.Execute();
begin
  case FMode of
  -1:
    begin
      Engine.ScriptOptions.SpoolOutput := smNone;
      EngineIntf.UpdateSpool;
    end;
   0:
    if Engine.ScriptOptions.SpoolOutput <> smNone then
      EngineIntf.ConPut('currently spooling to ' + Engine.ScriptOptions.SpoolFileName,
soInfo)
    else
      EngineIntf.ConPut('not spooling currently', soInfo);
   1:
    begin
      if FAppend then
        Engine.ScriptOptions.SpoolOutput := smOnAppend
      else
        Engine.ScriptOptions.SpoolOutput := smOnReset;
      Engine.ScriptOptions.SpoolFileName := EngineIntf.ExpandString(FileName);
      EngineIntf.UpdateSpool;
    end;
  end;
end;

initialization
  ADScriptCommandRegistry().AddCommand(TADSpoolScriptCommand);
```

See Also

Executing SQL Scripts (see page 88), TADScript (see page 685), TADScriptCommand (see page 704)

1.5.11 Local SQL

FireDAC offers flexible Local SQL feature. It allows to execute SQL queries against TDataSet descendants as the data sources.

Description

General

The Local SQL feature allows to execute SQL commands (see page 66), where the TDataSet descendants are used instead of the DB tables. The Local SQL is based on the SQLite database (see page 135) and supports most of the SQLite SQL dialect. As a table name it uses name of a TDataSet (Name property value) or alternatively specified name. Each queryable dataset must be registered with TADLocalSQL (see page 825) - the Local SQL engine.

All read and write operations are performing through the TDataSet API with some extensions. The engine supports as FireDAC, as non-FireDAC datasets. Local SQL is more effective with FireDAC datasets. For non-FireDAC datasets optionally may be implemented the IADPhysLocalQueryAdapter interface as an adapter class and registered it with the Local SQL engine.

The Local SQL possible applications are:

- Heterogeneous queries. Where queryable datasets have result sets from different DB's.
- In-memory database. There TADMemTable (see page 436)'s are serving as the datasets.
- Advanced offline mode. There although the main DB is not accessible, an application still can perform the SQL queries.
- Advanced DataSnap client. There the data delivered by DataSnap driver (see page 193) to the client may be queried locally.

- Simplified migration. A developer may use the 3d party TDataSet objects in application and use FireDAC API to work with these data sources.

Configuring

FireDAC uses SQLite driver as the Local SQL engine. So first, application should have an SQLite "local" connection. That may be as a file-based read-write connection, as an in-memory connection. We recommend to use in-memory connection. To setup it:

- drop TADPhysSQLiteDriverLink (see page 805) component on a form;
- drop TADConnection (see page 279);
- set it DriverName (see page 327) to SQLite.

Then application should connect the datasets to the Local SQL engine. For that:

- drop TADLocalSQL (see page 825) component on a form;
- set it Connection (see page 364) property to a local SQLite connection object;
- verify / assign value to a dataset Name property. Required step for the datasets created dynamically in the code;
- for an FireDAC dataset set TADAdaptedDataSet.LocalSQL (see page 261) property to the TADLocalSQL (see page 825) instance;
- or for an FireDAC and non-FireDAC dataset use TADLocalSQL.DataSets (see page 364) collection to register TDataSet and optional adapter with Local SQL engine. Also DataSets collection allow to specify alternative dataset name.

Finally activate objects:

- activate local SQLite connection;
- activate Local SQL engine by setting TADLocalSQL.Active (see page 364) to True.

Note, if application is using base datasets and local datasets connected to the same SQLite connection, then the connection must be activated explicitly before any dataset on this connection will be opened / executed / prepared. Otherwise exception will be raised.

Alternatively datasets may be submitted to Local SQL engine using TADLocalSQL.OnGetDataSet (see page 365) event handler.

All associated datasets will be registered with the engine when both local connection and Local SQL engine are active. Associating a new dataset does not require to reopen something, and it will be registered automatically. When a dataset is registered, then it will be opened. A TADMemTable must have a defined structure before it will be opened. When application performs a massive datasets registration, it is recommended to deactivate Local SQL engine before and activate it after the task.

The application may setup TADLocalSQL.SchemaName (see page 829) to a schema prefix, which will be used in the SQL queries. Application may use multiple TADLocalSQL's connected to the same connection with or without schema name specified. But each TADLocalSQL.SchemaName in a single SQLite connection must be unique. A schema name allows to organize datasets into the logical groups. When a schema name is specified then dataset in a SQL command may be referenced as <dataset> or <schema>.<dataset>.

The application may optionally set TADLocalSQL.MultipleCursors (see page 828) to False to avoid dataset cloning / copying. SQLite creates a cursor to execute a SQL command referring to a dataset. TDataSet API can keep only a single position. To overcome that FireDAC clones FireDAC dataset using TADMemTable (see page 436). And copies a non-FireDAC dataset to a TADMemTable (see page 436). When programmer is sure, that a dataset will be used only a single time in each moment of the time, then MultipleCursors (see page 828) may be set to False to improve performance. We do not recommend to set MultipleCursors to False, when there are unidirectional datasets.

Querying

To perform a Local SQL query, application should use TADQuery (see page 476) / TADCommand (see page 267). And set Connection (see page 503) / ConnectionName (see page 503) to the local SQLite connection, which is set also for

the TADLocalSQL.Connection (see page 364). Now application can execute SQL queries (see page 66), the same as with non-Local SQL connections.

While a query is processing a queryable dataset is disabled by DisableControls, and enabled at end of processing. To keep a dataset enabled, set TADLocalSQL.DisableControls to False. Before a dataset will be read the TADLocalSQL.OnReadDataSet (see page 365) event handler will be called.

Local SQL engine supports SQLite SQL dialect with some limitations. The following SQL commands are not supported for the TDataSet data sources:

- ALTER TABLE ... ADD COLUMN. Instead change the dataset structure.
- DROP TABLE. When a dataset will be unlinked from Local SQL engine, then it will be dropped automatically (not freed).
- CREATE INDEX / DROP INDEX. Instead of the SQL indexes use the dataset indexes (see page 100).
- CREATE TRIGGER / DROP TRIGGER. Instead of the triggers use the dataset events.

Local SQL engine does not support datasets with multiple result sets (see page 81).

Local SQL engine supports INSERT / UPDATE / DELETE SQL commands, as transactions and savepoints. And transforms corresponding SQL commands into the TDataSet API calls.

Local SQL engine support INSERT OR REPLACE, but uses only primary key fields to find a record to replace, when a primary or unique key constraint is violated. Additionally, when only several fields are specified in INSERT OR REPLACE INTO tab (<field list>), then on updating the not specified fields will get NULL values.

Compatibility

The Local SQL engine uses TDataSet API with some extensions provided by IADPhysLocalQueryAdapter interface. FireDAC datasets implement this interface. For non-FireDAC datasets a developer optionally may create a class implementing the interface, and assign it instance to the TADLocalSQL.DataSets[..].Adapter (see page 426) property.

Also, to perform the Local SQL operations a dataset must conform to the following requirements:

Operation	Requirements
INSERT	Mandatory: • Append method; • Post method. Optionally: • PSGetKeyFields must return primary key fields.
UPDATE	Mandatory: • Edit method; • Post method; • Locate method. Optionally: • PSGetKeyFields must return primary key fields.
DELETE	Mandatory: • Delete method; • Locate method. Optionally: • PSGetKeyFields must return primary key fields.

Sorting / ORDER BY	Optionally: • afIndexFieldNames in IADPhysLocalQueryAdapter.Features, then IADPhysLocalQueryAdapter.IndexFieldNames property will be used for sorting dataset.
Filtering / WHERE	Optionally: • afRanges in IADPhysLocalQueryAdapter.Features, then IADPhysLocalQueryAdapter.SetRange method will be used to filter dataset; • afFilters in IADPhysLocalQueryAdapter.Features, then Filter property will be used too.
Savepoints	Mandatory: • afCachedUpdates, afSavePoints in IADPhysLocalQueryAdapter.Features, then IADPhysLocalQueryAdapter.Savepoint property will be used.
Transactions	Mandatory: • PSStartTransaction method; • PSEndTransaction method.

See Also

Using SQLite with FireDAC (see page 135), Executing Command (see page 66), TADLocalSQL (see page 825)

Example 1

Heterogeneous query using SQL Anywhere and Oracle tables and FireDAC TADQuery's:

```
// setup connection and query to "Orders" table in SQL Anywhere DB
ADConnection1.ConnectionDefName := 'ASA_Demo';
ADConnection1.Connected := True;
ADQuery1.Connection := ADConnection1;
ADQuery1.SQL.Text := 'select * from Orders';
// link dataset to Local SQL engine
ADQuery1.LocalSQL := ADLocalSQL1;

// setup connection and query to "Order Details" table in Oracle DB
ADConnection2.ConnectionDefName := 'Oracle_Demo';
ADConnection2.Connected := True;
ADQuery2.Connection := ADConnection2;
ADQuery2.SQL.Text := 'select * from "Order Details"';
// link dataset to Local SQL engine
ADQuery2.LocalSQL := ADLocalSQL1;

// setup SQLite in-memory connection
ADConnection3.DriverName := 'SQLite';
ADConnection3.Connected := True;
// link Local SQL to SQLite connection
ADLocalSQL1.Connection := ADConnection3;
ADLocalSQL1.Active := True;

// execute SELECT query on above datasets
ADQuery3.Connection := ADConnection3;
ADQuery3.SQL.Text := 'SELECT * FROM ADQuery1 LEFT JOIN ADQuery2 ON ADQuery1.OrderID =
ADQuery2.OrderID';
ADQuery3.Active := True;
```

Example 2

Heterogeneous query using ADO TADOQuery's and FireDAC TADQuery's:

```
// setup connection and query to "Orders" table
ADOQuery1.SQL.Text := 'select * from Orders';
// link dataset to Local SQL engine
with ADLocalSQL1.DataSets.Add do begin
  DataSet := ADOQuery1;
  Name := 'Orders';
end;

// setup connection and query to "Order Details" table
```

```
ADOQuery2.SQL.Text := 'select * from "Order Details"';
// link dataset to Local SQL engine
with ADLocalSQL1.DataSets.Add do begin
  DataSet := ADOQuery2;
  Name := 'Order Details';
end;

// setup SQLite in-memory connection
ADConnection1.DriverName := 'SQLite';
ADConnection1.Connected := True;
// link Local SQL to SQLite connection
ADLocalSQL1.Connection := ADConnection1;
ADLocalSQL1.Active := True;

// execute SELECT query on above datasets
ADQuery1.Connection := ADConnection1;
ADQuery1.SQL.Text := 'SELECT * FROM Orders o LEFT JOIN "Order Details" od ON o.OrderID =
od.OrderID';
ADQuery1.Active := True;
```

Example 3

See demos:

- Cross DBMS query - FireDAC\Samples\Comp Layer\TADLocalSQL\xDBMS

- In-memory database - FireDAC\Samples\Comp Layer\TADLocalSQL\InMemDB

1.6 Working with DataSets

A set of articles describing how to work with FireDAC dataset classes, including how to edit the dataset content.

1.6.1 Sorting Records

FireDAC offers few methods to sort the records. The dataset sorting allows to get the required records order without requering a DB.

Description

General

All FireDAC datasets are offering few approaches to locally sort the records. Before sorting the records, FireDAC will fetch all records from the result set. Internally FireDAC builds and maintains a list of sorted records. So, on large record volumes the sorting may be long, but subsequent records navigation will be the same fast as without sorting.

Standard sorting

To sort dataset by the field values application may use IndexFieldNames (see page 600) property. It supports multiple fields, case-insensitive and descending modes. Also use FormatOptions (see page 651).SortOptions (see page 868) to modify the sort order properties. For example:

```
ADQuery1.IndexFieldNames := 'ORDERID';
```

or

```
ADQuery1.IndexFieldNames := 'OrderDate:D;Price';
```

Alternatively, when a dataset has few indexes defined in the Indexes (see page 598) collection, then one of the indexes may be current. To do that use IndexName (see page 601) property. For example:

```
with ADQuery1.Indexes.Add do begin
  Name := 'By OrderDate';
  Fields := 'OrderDate';
  Active := True;
end;
ADQuery1.IndexName := 'By OrderDate';
```

Dataset views

FireDAC allows to define views of the dataset records using the Indexes (see page 598) collection. The following properties control the view:

- Sorting by fields (Fields (see page 657), CaseInsFields (see page 655), DescFields (see page 656), Options (see page 658)).
- Sorting by expression (Expression (see page 656), Options (see page 658)).
- Records filter (Filter (see page 657), FilterOptions (see page 657)).
- Records distinction flag (Distinct (see page 656)).

The view is identified by the Name (see page 658) property. It must be unique across dataset.

The view is a list of records maintained by FireDAC, when application fetches or edits records. The view is maintained when its Active (see page 655) is True and dataset IndexesActive (see page 600) is True. When view has soUnique or soPrimary in Options (see page 658) and the application needs to enforce record uniqueness, set dataset ConstraintsEnabled (see page 594) to True.

When view is current, then dataset represents the record list maintained by this view. To make view current, set it Selected (see page 659) property to True or set dataset IndexName (see page 601) property to the view name. For example:

```
with ADQuery1.Indexes.Add do begin
  Name := 'May sale dates';
  Fields := 'OrderDate';
  Filter := 'MONTH(OrderDate) = 5';
  Distinct := True;
  Active := True;
  Selected := True;
end;
```

Other options

Some of navigation methods, like the Locate (see page 634) / Lookup (see page 637) / SetRange (see page 647) are using the current sort order to optimize its operations. Note, that Filter property does not use the views to optimize filtering.

Application may use fkInternalCalc and fkAggregate fields in sorting. FireDAC does not support sorting on fields of the fkCalculated and fkLookup kinds. As a workaround the application may create fkInternalCalc field, fill it in OnCalcFields event handler by the value of the fkCalculated or fkLookup field, then sort on this fkInternalCalc field.

TADTable and sorting

The TADTable sorting is performed at server side (ORDER BY). Note, that FireDAC will automatically and transparently add primary key fields to the sorting order. That is required for proper live data window functionality.

See Also

Writing Expressions (see page 110), Filtering Records (see page 102), Finding a Record (see page 107), Calculated and Aggregated Fields (see page 108), Master-Detail Relationship (see page 104), Browsing Table (see page 73)

Example

For additional details see demos:

- FireDAC\Samples\Comp Layer\TADQuery\Indices

- FireDAC\Samples\Comp Layer\TADMemTable\Main

1.6.2 Filtering Records

FireDAC offers few methods to filter the records. The dataset filtering allows to get records subsets basing on a condition.

Description

General

All FireDAC datasets are offering few approaches to locally filter the records. After applying a filter, FireDAC will not requery records, but will filter records in local dataset cache. Internally FireDAC builds and maintains a list of filtered records. So, on large record volumes the filter activation may be long, but subsequent records navigation will be the same fast as without filtering.

Standard filtering

FireDAC datasets offer few options to filter records using a standard approach:

- Filter property allows to specify an conditional expression (see page 110) as a string. After specifying Filter value set Filtered property to True to activate the filter. For example:

```
ADQuery1.Filter := 'OrderID in (10150, 10151, 10152)';
ADQuery1.Filtered := True;
```

- OnFilterRecord event handler allows to implement filtering as a Delphi code. After specifying Filter value set Filtered property to True to activate the filter. For example:

```
ADQuery1.OnFilterRecord := Form1FilterRecord;
ADQuery1.Filtered := True;

procedure TForm1.Form1FilterRecord(DataSet: TDataSet; var Accept: Boolean);
var
  iOrderID: Integer;
begin
  iOrderID := DataSet.FieldByName('OrderID').AsInteger;
  Accept := (iOrderID = 10150) or (iOrderID = 10151) or (iOrderID = 10152);
end;
```

Filtering by a range

When dataset is sorted (see page 100) using the field list, then application may apply filtering by a field values range. This is most effective way to limit records, as it is using the dataset internal index structures.

The following methods control the filtering:

- SetRangeStart (see page 648) - brings dataset to a mode allowing to set range minimal values and erasing the previous values;
- EditRangeStart (see page 621) - brings dataset to a mode allowing to set range minimal values preserving the previous values;
- SetRangeEnd (see page 648) - brings dataset to a mode allowing to set range maximal values and erasing the previous values;
- EditRangeEnd (see page 620) - brings dataset to a mode allowing to set range maximal values preserving the previous values;
- ApplyRange (see page 611) - activates range filtering after specifying minimal and maximal values;
- SetRange (see page 647) - combines SetRangeStart, SetRangeEnd and ApplyRange into a single method;
- CancelRange (see page 613) - cancels the range filtering.

And properties:

- IsRanged (see page 632) - allows to get the current range filtering mode.
- KeyExclusive (see page 602) - get / set the inclusion of minimal and maximal values into filtered range;
- KeyFieldCount (see page 602) - get / set the number of index fields to use in range filtering.

For example:

```
ADQuery1.IndexFieldNames := 'ORDERID;ORDERDATE';
ADQuery1.SetRangeStart;
ADQuery1.KeyExclusive := False;
ADQuery1.KeyFieldCount := 1;
ADQuery1.FieldByName('OrderID').AsInteger := 10150;
ADQuery1.SetRangeEnd;
ADQuery1.KeyExclusive := False;
ADQuery1.KeyFieldCount := 1;
ADQuery1.FieldByName('OrderID').AsInteger := 10152;
ADQuery1.ApplyRange;
```

Filtering by a record status

The FilterChanges (see page 597) property allows to filter records, depending on their change status. This filtering may be used only in Cached Updates mode (see page 114). For example, to show only modified and deleted records:

```
ADQuery1.FilterChanges := [rtModified, rtDeleted];
```

To filter records, failed to process at ApplyUpdates call, use FilterChanges (see page 597) with rtHasErrors.

Other options

There are other options allowing to limit the visible records:

- Using master-detail datasets relation (see page 104);
- Combine sorting and filtering using local dataset indexes (see page 100).

Also note, FireDAC does not support filtering on fields of the fkCalculated and fkLookup kinds. But application may use fkInternalCalc and fkAggregate fields in filtering.

TADTable and filtering

TADTable (see page 539) in live data window mode uses server side filtering (WHERE) for:

- Filter property. Note, that the Filter property content is sent to a DB as is. You may use FireDAC escape sequences (see page 56) to make an expression compatible with a DBMS and with the local expression engine;
- a detail TADTable in the master-detail relationship;
- when a range is applied to a TADTable.

And client side filtering for:

- OnFilterRecord event;
- FilterChanges (see page 597) property.

See Also

Writing Expressions (see page 110), Sorting Records (see page 100), Finding a Record (see page 107), Calculated and Aggregated Fields (see page 108), Master-Detail Relationship (see page 104), Browsing Table (see page 73)

Example

See the following demos for additional details:

- FireDAC\Samples\Comp Layer\TADQuery\Filter

- FireDAC\Samples\Comp Layer\TADMemTable\MasterDetail

1.6.3 Master-Detail Relationship

FireDAC has flexible support for master-detail relationship between datasets.

Description

General

The master-detail relationship allows automatically filter a detail dataset basing on a current master dataset record. For example, the master dataset has "Order" records, and the detail dataset has "Order Line" records. So the detail dataset shows only lines for the current order.

Special setup is not required for a master dataset. FireDAC offers two base methods to setup a detail dataset in a master-detail relationship:

- Parameter based. The master dataset field values are assigned to the detail TADQuery or TADStoredProc parameters, then detail dataset query is reexecuted (see page 66).
- Range based. The master dataset field values are used to apply a range (see page 102) to the detail dataset. The detail dataset may be any FireDAC dataset with current active index (see page 100).

These methods may be combined. To choose between them consider the table:

Feature	Parameter based	Range based
The detail query returns a limited number of records.	+	
The detail records are fresh.	+	
Reduced traffic and DBMS workload on each master change.		+
The cached updates preserved on the master change.		+
Works in offline mode.		+
Supports Centralized Cached Updates with propagation		+

Also, FireDAC offers two cached updates modes (see page 114) for the datasets in master-detail relationship:

- Decentralized Cached Updates mode, when each dataset tracks the changes independently on others;
- Centralized Cached Updates mode, when few datasets in master-detail relationship share a single change log. And master dataset may propagate changes to the detail datasets in cascading fashion, including auto-incrementing field values.

Parameters based M/D

To setup the parameters based M/D relationship, perform the following steps:

1. Drop TADQuery (see page 476) (or any other FireDAC dataset) on a form. Name it **qOrders**. That will be a master dataset. Setup it - lets assign the SQL (see page 499):

```
SELECT * FROM {id Orders}
```

2. Drop TDataSource on a form. Name it **dsOrders**. Set it DataSet to **qOrders**.
3. Drop TADQuery (see page 476) on a form. Name it **qOrderDetails**. That will be a detail dataset. Setup it - lets assign the SQL (see page 499):

```
SELECT * FROM {id Order Details} WHERE OrderID = :OrderID
```

Then set MasterSource (see page 491) to **dsOrders**. The base setup is finished.

So, how it works ? FireDAC builds for **qOrderDetails** a list of pairs - **qOrders** fields and **qOrderDetails** parameters. Elements in each pair:

- when MasterFields (see page 491) is not specified, then have the same name;
- otherwise - have the same position, fields in MasterFields (see page 491) list, parameters in Params (see page 497) collection.

When the current **qOrders** record is changed, FireDAC assigns for each parameter a corresponding field value. In our case **qOrderDetails :OrderID** parameter gets the **qOrder OrderID** field value. After that the **qOrders** is reexecuted.

Note, the BeforeOpen and AfterOpen events do not fire for detail dataset. Use OnMasterSetValue (see page 604) instead.

Range based M/D

To setup the range based M/D relationship, perform the following steps:

1. Drop TADQuery (see page 476) (or any other FireDAC dataset) on a form. Name it **qOrders**. That will be a master dataset. Setup it - lets assign the SQL (see page 499):

```
SELECT * FROM {id Orders}
```

2. Drop TDataSource on a form. Name it **dsOrders**. Set it DataSet to **qOrders**.
3. Drop TADQuery (see page 476) on a form. Name it **qOrderDetails**. That will be a detail dataset. Setup it - lets assign the SQL (see page 499):

```
SELECT * FROM {id Order Details}
```

Then set MasterFields (see page 491) to **ORDERID**, IndexFieldNames (see page 489) to **ORDERID** and MasterSource (see page 491) to **dsOrders**. The base setup is finished.

So, how it works ? FireDAC builds for **qOrderDetails** a list of pairs - **qOrders** and **qOrderDetails** fields, where fields in each pair have the same position, master fields in MasterFields (see page 491), detail fields in IndexFieldNames (see page 489).

When the current **qOrders** record is changed, FireDAC applies the range to **qOrderDetails**, where details fields are equal to corresponding master fields. In our case **qOrderDetails OrderID** field is equal to **qOrder OrderID** field.

Combining methods

To combine both methods an application should use both Parameters and Range based setups and include fiDetails into FetchOptions (see page 486).Cache (see page 852). Then FireDAC at first will use range based M/D. And if a dataset is empty, then FireDAC will use parameters based M/D. The new queried records will be appended to the internal records storage.

Also the OnMasterSetValues (see page 604) event handler may be used to override M/D behavior.

Editing detail dataset

When a new record is inserted into detail dataset, the fields participating in a M/D relationship will be filled automatically by the corresponding master dataset field values. The detail dataset field list is defined:

- For parameter based M/D - DetailFields (see page 596), if specified. Otherwise - the fields with the same names as the parameters.
- For range based M/D - the index fields.

To insert a detail record, the master dataset must be in the browsing state (dsBrowse). It is impossible to have both master and detail dataset in inserting (dsInsert) or editing (dsEdit) state.

When master and details datasets are in the cached updates mode, then application may use TADSchemaAdapter (see

page 514) to enable Centralized Cached Updates (see page 114) with propagation. For that master and details datasets SchemaAdapter property must point to the same TADSchemaAdapter (see page 514) and detail dataset FetchOptions.DetailCascade (see page 854) is True.

Navigating in M/D

When an application needs to navigate in a master dataset, on each master record change the detail dataset will be refreshed. That consumes the resources and the navigation may be slow. To temporary disable M/D synchronization, application may call DisableControls / EnableControls for the master dataset:

```
qOrders.DisableControls;
try
  qOrders.First;
  while not qOrders.Eof do begin
    .....
    qOrders.Next;
  end;
finally
  qOrders.EnableControls;
end;
```

To force disabled M/D synchronization, call ApplyMaster (see page 611) method on the master dataset. To temporary disable M/D synchronization for a specific detail dataset, use DisableScroll (see page 664) / EnableScroll (see page 664) methods of the dataset MasterLink (see page 603) property:

```
qOrderDetails.MasterLink.DisableScroll;
try
  qOrders.First;
  while not qOrders.Eof do begin
    if qOrders.FieldByName('OrderID').AsInteger = 100 then begin
      qOrderDetails.ApplyMaster;
      // read qOrderDetails dataset - it is synchronized with qOrders
    end;
    qOrders.Next;
  end;
finally
  qOrderDetails.MasterLink.EnableScroll;
end;
```

The GUI applications may benefit from delayed M/D synchronization. So, when an user scrolls in a grid, a detail dataset will be refreshed not immediately, but after some delay and if there were no other navigations. To use that set FetchOptions.DetailDelay (see page 854) for a detail dataset. To temporary disable delayed M/D synchronization for a specific detail dataset and use immediate synchronization, use DisableDelayedScroll (see page 663) / EnableDelayedScroll (see page 664) methods of the MasterLink (see page 603) property.

By default a state change, a non-key field values change, refreshing of master dataset will not lead to detail dataset refreshing. This avoids extra refreshes of detail dataset. If your application expects always refresh detail dataset, then set FetchOptions.DetailOptimize (see page 855) to False.

See Also

Sorting Records (see page 100), Filtering Records (see page 102), Browsing Table (see page 73), Caching Updates (see page 114)

Example

See the following demos for additional details:

- FireDAC\Samples\Comp Layer\TADQuery\MasterDetail;
- FireDAC\Samples\Comp Layer\TADMemTable\MasterDetail.

1.6.4 Finding a Record

FireDAC offers few methods to find a specific record in the dataset.

Description

General

All FireDAC datasets are offering few approaches to find a record in a local dataset records cache. Depending on the current sorting, the record search may be optimized.

Standard locating

FireDAC datasets offer few options to locate a record or lookup a value for a specified key. These method may use the current dataset index (see page 100), if it is appropriate for the method call.

- Locate method allows to find a record for the specified key values:

```
ADQuery1.IndexFieldNames := 'ORDERID';
if ADQuery1.Locate('ORDERID', 10150, []) then
  ShowMessage('Order is found')
else
  ShowMessage('Order is not found');
```

- Lookup method allows to lookup a value for the specified key values.

Extended locating

FireDAC datasets offer few extended methods to locate a record. These method may use the current dataset index (see page 100), if it is appropriate for the method call.

- LocateEx (see page 635) method is similar to Locate method, but offers far more options, like search forward or backward, search from beginning or from current records, search using textual predicate (expression). For example:

```
if ADQuery1.LocateEx('Price >= 1000 and Price <= 2000', []) then
  ShowMessage('Order is found')
else
  ShowMessage('Order is not found');
```

- LookupEx method is similar to Lookup method, but offers similar to LocateEx options.

Locating using a filter

This approach to locate a record was introduced into FireDAC for compatibility with BDE components. It may be completely replaced with LocateEx (see page 635) method. The approach is using the methods:

- FindFirst - find first record satisfying the predicate;
- FindNext - find next record after current one satisfying the predicate;
- FindPrior - find previous record after current one satisfying the predicate;
- FindLast - find last record satisfying the predicate.

To start a search the application must specify predicate as a boolean expression in the Filter property. Note, that this method cannot be used to locate a record in a filtered dataset. Also, the filter is not activated and is used only for locating a record. For example:

```
ADQuery1.Filter := 'Price >= 1000 and Price <= 2000';
if ADQuery1.FindFirst then
  ShowMessage('Found !');
...
if ADQuery1.FindNext then
  ShowMessage('Found !');
```

Locating using an index and a key value

This approach to locate a record was introduced into FireDAC for compatibility with BDE components. But we found it helpful. The approach is using the methods:

- SetKey (see page 647) - set dataset to dsSetKey mode, allowing to specify the index field values;
- GotoKey (see page 630) - search for an exact record using the index field values, specified after SetKey method call;
- GotoNearest (see page 631) - search for an exact or next record using the index field values, specified after SetKey method call;
- FindKey (see page 625) - find an exact record using specified index field values;
- FindNearest (see page 626) - find an exact or next record using specified index field values.

Before calling these methods the current index must be set (see page 100). For example:

```
ADQuery1.IndexFieldNames := 'ORDERID';
ADQuery1.SetKey;
ADQuery1.FieldByName('ORDERID').AsInteger := 10150;
if ADQuery1.GotoKey then
  ShowMessage('Order is found')
else
  ShowMessage('Order is not found');
```

Other options

Most of the search methods are returning True, when a record is found. Alternatively application may check the dataset Found property.

Also note, FireDAC does not support locating on fields of the fkCalculated and fkLookup kinds. But application may use fkInternalCalc and fkAggregate fields in locating.

See Also

Writing Expressions (see page 110), Sorting Records (see page 100), Filtering Records (see page 102), Calculated and Aggregated Fields (see page 108), Master-Detail Relationship (see page 104), Browsing Table (see page 73)

1.6.5 Calculated and Aggregated Fields

FireDAC offers few kinds of calculated fields.

Description

General

Calculated fields are the virtual fields, whose values are not fetched / stored in database. Instead they are calculated on the client. FireDAC supports calculated fields of all TField.FieldKind types:

- **fkCalculated** - a simple calculated field. The value is calculated in TDataSet.OnCalcFields event handler.
- **fkInternalCalc** - an advanced calculated field. The value may be assigned as to regular fields and is stored in the dataset records cache. The value is calculated in TDataSet.OnCalcFields event handler or using expression specified by TField.DefaultExpression.
- **fkLookup** - a lookup fields. The value is calculated automatically, providing a value from a lookup dataset for a key value

from this dataset.

- **fkAggregate** - an aggregate calculating field. The values is calculated using expression specified by TAggregateField.Expression, which includes COUNT, SUM, MIN, MAX, AVG aggregate functions.

Only **fkInternalCalc** and **fkAggregate** fields may be used in filtering, sorting or locating. Also they will be stored with other dataset fields into persistent stream or file. The calculated field values cannot be posted to a database in automatic mode.

Standard calculated fields

fkCalculated and **fkInternalCalc** calculated field values may be assigned by the TDataSet.OnCalcFields event handler. A calculated field may be defined:

- at design-time using the dataset "Fields Editor ..." menu item;
- at run-time using the code. For example, to create a calculated field, that contains upper cased name:

```
procedure TForm1.Form1CalcFields(ADataSet: TDataSet);
begin
  ADataSet.FieldByName('UName').AsString :=
UpperCase(ADataSet.FieldByName('Name').AsString);
end;

var
  oField: TField;
  i: Integer;
...
ADQuery1.FieldDefs.Updated := False;
ADQuery1.FieldDefs.Update;
for i := 0 to ADQuery1.FieldDefs.Count - 1 do
  ADQuery1.FieldDefs[i].CreateField(Self);

oField := TStringField.Create(ADQuery1);
oField.Size := 50;
oField.FieldName := 'UName';
oField.FieldKind := fkInternalCalc; // or fkCalculated
oField.DataSet := ADQuery1;

ADQuery1.OnCalcFields := Form1CalcFields;
ADQuery1.Open;
```

Expression calculated fields

fiInternalCalc fields may be calculated automatically by an expression specified by the TField.DefaultExpression. The OnCalcFields event handler and explicit value assignment is not needed then. The expression cannot be changed when the dataset is active. For example:

```
var
  oField: TField;
  i: Integer;
...
ADQuery1.FieldDefs.Updated := False;
ADQuery1.FieldDefs.Update;
for i := 0 to ADQuery1.FieldDefs.Count - 1 do
  ADQuery1.FieldDefs[i].CreateField(Self);

oField := TStringField.Create(ADQuery1);
oField.Size := 50;
oField.FieldName := 'UName';
oField.FieldKind := fkInternalCalc;
oField.DefaultExpression := 'UPPER(Name)';
oField.DataSet := ADQuery1;

ADQuery1.Open;
```

Aggregate fields

fkAggregate aggregate fields management is similar to the expression calculated fields. FireDAC calculates aggregate fields when TADDataSet.AggregatesActive (see page 590) is True, by default it is False. The aggregate expression cannot be changed when the dataset is active. For example to create an aggregated field:

```
var
  oField: TAggregateField;
  i: Integer;
...
ADQuery1.FieldDefs.Updated := False;
ADQuery1.FieldDefs.Update;
for i := 0 to ADQuery1.FieldDefs.Count - 1 do
  ADQuery1.FieldDefs[i].CreateField(Self);

oField := TAggregateField.Create(ADQuery1);
oField.FieldName := 'Total';
oField.Expression := 'SUM((ItemPrice + ItemTaxes) * ItemCount)';
oField.DataSet := ADQuery1;

ADQuery1.AggregatesActive := True;
ADQuery1.Open;
```

An aggregated field may define grouping. Then a value is calculated for records with the same index field values, instead of all records. To define grouping set:

- TAggregateField.IndexName to a name of the index to use for grouping. By default - the current index is used.
- TAggregateField.GroupingLevel to a number of indexed fields to use for grouping. By default - 0 (no fields and no grouping).

Note, aggregated field always return Null value, when a dataset is in dsInsert state.

Aggregate values

Also, FireDAC application may use TADDataSet.Aggregates (see page 590) collection to define aggregated values. They are more light-weighted than fields and may define at any time, including when dataset is active. For example:

```
with ADQuery1.Aggregates.Add do begin
  Name := 'Total';
  Expression := 'SUM((ItemPrice + ItemTaxes) * ItemCount)';
  Active := True;
end;
ADQuery1.AggregatesActive := True;
...
Label1.Caption := VarToStr(ADQuery1.Aggregates[0].Value);
```

See Also

Sorting Records (see page 100), Filtering Records (see page 102), Writing Expressions (see page 110)

1.6.6 Writing Expressions

FireDAC offers powerful client expressions engine, used for filtering, indexing and calculated fields.

Description

General

FireDAC supports expression syntax compatible with:

- BDE data access components;
- TClientDataset;

- Oracle 8 (not for 100%);
- ODBC escape functions.

To add Oracle-like, ODBC-like and regular expression functions, you should include uADStanExprFuncs unit into any application uses clause. Otherwise you can get error, like "Column with name [NVL] is not found". The registered functions are:

- Character macro functions (see page 60);
- Numeric macro functions (see page 61);
- Date and time macro functions (see page 62);
- System macro functions (see page 64);
- Convert macro function (see page 65).

Expression engine does not support ODBC escape {fn xxx} syntax for calling of functions. But practically the full set of the escape functions is supported. To call a function just call it in the expression string, as you do in Delphi.

The dataset fields may be referenced in an expression, as the identifiers. For example:

```
ADQuery1.Filter := 'upper(name) like ''bill%''';
ADQuery1.Filtered := True;
```

Or:

```
ADQuery1.Filter := '(upper(rtrim(name)) like ''bill%'') or (upper(rtrim(name)) like
''john%'')';
ADQuery1.Filtered := True;
```

If you need to create your own function and register it with FireDAC, see uADStanExprFuncs.pas unit for details.

Note, expression engine supports FireDAC escape sequences (see page 56).

Using expression for custom purposes

An application may use FireDAC expression evaluator to perform text formula calculations. There are two basic options:

- use TADDataSet.CreateExpression (see page 617) method to perform calculations on the dataset fields. For example:

```
var
  oEval: IADStanExpressionEvaluator;
...
oEval := ADMemTable1.CreateExpresison('(sal + comm) * tax / 100');
Label1.Caption := oEval.Evaluate;
...
ADMemTable1.Next;
oEval.DataSource.Position := ADMemTable1.GetRow;
Label1.Caption := oEval.Evaluate;
...
oEval := nil;
ADMemTable1.Close;
```

- use API to extend the evaluator by custom data source. For example:

```
uses
  uADStanIntf, uADStanFactory, uADStanExpr, uADStanExprFuncs;

type
  TMyVariable = class (TInterfacedObject, IADStanExpressionDataSource)
  private
    FName: String;
    FpValue: PDouble;
  protected
    function GetVarIndex(const AName: String): Integer;
    function GetVarType(AIndex: Integer): TADDataType;
    function GetVarScope(AIndex: Integer): TADExpressionScopeKind;
    function GetVarData(AIndex: Integer): Variant;
    ...........
```

```
    public
      constructor Create(AName: String; var AValue: Double);
    end;

  { TMyVariable }

  constructor TMyVariable.Create(AName: String; var AValue: Double);
  begin
    inherited Create;
    FName := AName;
    FpValue := @AValue;
  end;

  function TMyVariable.GetVarIndex(const AName: String): Integer;
  begin
    if CompareText(AName, FName) = 0 then
      Result := 0
    else
      Result := -1;
  end;

  function TMyVariable.GetVarType(AIndex: Integer): TADDataType;
  begin
    if AIndex = 0 then
      Result := dtDouble
    else
      Result := dtUnknown;
  end;

  function TMyVariable.GetVarScope(AIndex: Integer): TADExpressionScopeKind;
  begin
    if AIndex = 0 then
      Result := ckConst
    else
      Result := ckUnknown;
  end;

  function TMyVariable.GetVarData(AIndex: Integer): Variant;
  begin
    if AIndex = 0 then
      Result := FpValue^
    else
      Result := Null;
  end;

  procedure TForm1.FormCreate(Sender: TObject);
  var
    d: Double;
    oDS: IADStanExpressionDataSource;
    oParser: IADStanExpressionParser;
    oEval: IADStanExpressionEvaluator;
  begin
    oDS := TMyVariable.Create('x', d);

    ADCreateInterface(IADStanExpressionParser, oParser);
    oEval := oParser.Prepare(oDS, '(2*x)+cos(sin(90)+10)', [], [poDefaultExpr], '');

    d := 1.2;
    ShowMessage(oEval.Evaluate);

    d := 3.4;
    ShowMessage(oEval.Evaluate);
  end;
```

Example

See FireDAC demos for more details:

- FireDAC\Samples\Comp Layer\TADQuery\Filter;
- FireDAC\Samples\DatS Layer\CalcColumnWithFuncs.

1.7 Editing the Data

A set of articles describing how to use FireDAC to edit the dataset data, post changes to a database and setup the updates posting.

1.7.1 Changing DataSet Data

FireDAC supports dataset records editing, including automatic updates posting to a database.

Description

General

FireDAC datasets, including TADQuery (see page 476), TADTable (see page 539), TADStoredProc (see page 514), TADMemTable (see page 436), are supporting inserting, editing and deleting of records, using the standard TDataSet methods. By default editing and posting updates are enabled for FireDAC datasets.

Controlling editing

A dataset editing is enabled, when UpdateOptions.ReadOnly (see page 903) is False. The particular operations are enabled:

- appending when UpdateOptions.EnableInsert (see page 900) is True:

```
ADQuery1.Append;
ADQuery1.FieldsByName('id').AsInteger := 100;
ADQuery1.FieldsByName('name').AsString := 'Audi A6';
ADQuery1.Post;
```

- editing when UpdateOptions.EnableUpdate (see page 900) is True:

```
ADQuery1.Edit;
ADQuery1.FieldsByName('name').AsString := 'Audi A6 Avant';
ADQuery1.Post;
```

- deleting when UpdateOptions.EnableDelete (see page 899) is True:

```
ADQuery1.Delete;
```

To avoid "Field value is required" at Post call, set UpdateOptions.CheckRequired (see page 898) to False (default is True).

Posting updates

To post updates to a database, FireDAC will automatically generate updating SQL commands (see page 121), when the original SQL query conforms to:

- The command must be SELECT.
- The first table in FROM clause must preserve primary key (PK).
- The PK fields must be in SELECT list.
- The SELECT must be without DISTINCT, GROUP BY, UNION, etc phrases. That is rather a logical rule, than "a must".

Also the correct metadata must be provided, including unique identifying fields (see page 117). Most simple to enable the

automatic updates posting for TADQuery is to set UpdateOptions.RequestLive (see page 904) to True (default is True). There is no automated possibility to check that a query is updatable, the programmer must evaluate that himself.

If query does not conform to above rules, or posting must be overridden by the application (see page 123), then you can use TADUpdateSQL (see page 563) component or OnUpdateRecord (see page 605) event handler.

The updates may be posted to a database immediately on Post or Delete methods call, or may be cached and posted later (see page 114). The deferred updates posting may be used together with offline connections (see page 40).

See Also

Unique Identifying Fields (see page 117), Update Command Generation (see page 121), Overriding Posting Updates (see page 123)

1.7.2 Caching Updates

FireDAC supports deferred updates posting, so called Cached Updates.

Description

General

The Cached Updates mode allows to defer posting updates to a database to a later time, than Post / Delete methods are called. That allows to post multiple updates in a single batch, optionally enclosed into a transaction.

To set a dataset to the Cached Updates mode set CachedUpdates (see page 592) property to True. Then the dataset will track all changes made after last setting to True or CancelUpdates (see page 614) / CommitUpdates (see page 615) calls. Those changes made up the change journal, where all changes are ordered by the time when a change was made. FireDAC does not track multiple versions of the same record. The last change overrides previous one and moves the record to the end of the change order.

FireDAC supports Decentralized and Centralized Cached Updates modes:

- Decentralized Cached Updates mode, when each dataset tracks the changes independently on others. This is classical and default mode.
- Centralized Cached Updates mode, when few datasets share a single change log and changes are in historical order.

Centralized cached updates

When application needs to log and apply multiple dataset changes in a chronological order, then Centralized Cached Updates may be used. For that a single TADSchemaAdapter (see page 514) instance must be assigned to few datasets SchemaAdapter (see page 506) property. There TADSchemaAdapter (see page 514) serves as a central storage for the rows and their changes for few datasets.

At first, Centralized Cached Updates is useful in master-detail relationships (see page 104), where a master dataset propagates changes to the cascading detail datasets, including auto-incrementing field values. To enable propagation for a detail dataset it FetchOptions.DetailCascade (see page 854) must be set to True. This allows:

- synchronized master-detail changes, so the changes of master and detail datasets will be recorded and applied in the chronological order. Eg, first inserted master record, then inserted corresponding detail records;
- propagation of a master identity column value to a detail dataset. Eg, when detail dataset is linked to identity column of master dataset and on applying updates the corresponding detail records must get actual master identity column value;
- cascading deletion of the detail records when a master record is deleted. Eg, when master record is deleted then all corresponding detail records will be deleted too and that will be recorded in the change log;
- cascading propagation of the master field changes to a detail dataset.

Propagation works when the following conditions are met:

- datasets are in master-detail relationship;

- range based M/D is used (see page 104);
- master dataset is an FireDAC dataset (TADQuery (see page 476), TADStoredProc (see page 514), TADMemTable (see page 436));
- detail dataset is not TADTable (see page 539) in Live Data Window Mode (see page 73).

To enable Centralized Cached Updates with propagation, perform the following steps (expected that range based M/D (see page 104) is setup and CachedUpdates (see page 592) property is True):

- Drop TADSchemaAdapter (see page 514) on a form.
- Set master dataset SchemaAdapter property to a TADSchemaAdapter (see page 514).
- Set detail dataset SchemaAdapter property to a TADSchemaAdapter (see page 514).
- Set detail dataset FetchOptions.DetailCascade (see page 854) to True.

In the same time this enables an in-memory referential constraint for a detail dataset. This is similar to a SQL command:

```
ALTER TABLE <detail> ADD CONSTRAINT
FOREIGN KEY (<detail fields>)
REFERENCES <master> (<master fields>)
ON DELETE CASCADE
ON UPDATE CASCADE
```

To apply updates the application should use TADSchemaAdapter (see page 514).ApplyUpdates (see page 402) method instead of dataset ApplyUpdates (see page 611). To reconcile errors use TADSchemaAdapter (see page 514).Reconcile (see page 402) method instead of dataset Reconcile (see page 642).

See FireDAC\Samples\Comp Layer\TADQuery\CachedUpdates\Centralized demo application for more details.

Tracking updates

When your application is working in cached updates mode, you can track the changes and optionally revert the changes for each dataset. To track changes, use the properties:

- UpdatesPending (see page 609) - returns True, if change log is not empty;
- ChangeCount (see page 593) - returns the total number of changes;
- UpdateStatus (see page 650) - returns the change kind for the current record;
- FilterChanges (see page 597) - allows to filter records by the modification kinds.

To revert existing changes, use the properties and methods:

- SavePoint (see page 607) - set/get the current change log state;
- RevertRecord (see page 644) - reverts current record to it previous (original) state;
- UndoLastChange (see page 649) - jumps to the last modified record and reverts it to it previous (original) state;
- CancelUpdates (see page 614) - reverts all records in the change log.

For example, to implement simple Undo functionality, you may create an actUndo action and attach the following event handlers:

```
procedure TForm1.actUndoUpdate(Sender: TObject);
begin
  actUndo.Enabled := ADQuery1.UpdatesPending;
end;

procedure TForm1.actUndoExecute(Sender: TObject);
begin
  ADQuery1.UndoLastChange(True);
end;
```

For other example, to implement in-memory transaction, with ability to undo group of changes, you may do:

```
ADQuery1.CachedUpdates := True;
iSavePoint := ADQuery1.SavePoint;
try
  ADQuery1.Append;
```

```
    ...
    ADQuery1.Post;
    ADQuery1.Append;
    ...
    ADQuery1.Post;
    ADQuery1.Append;
    ...
    ADQuery1.Post;
  except
    ADQuery.SavePoint := iSavePoint;
  end;
```

Note, in cached updates mode the following methods and properties are working with updates journal:

- Data (see page 595) property includes all records, even deleted, and their changes;
- Delta (see page 596) property returns deleted, inserted or updated records in updates journal;
- CopyRecord (see page 616), CopyDataSet (see page 615) methods create new changes and do not copy change journal;
- LoadFromStream (see page 633), LoadFromFile (see page 632), SaveToStream (see page 645), SaveToFile (see page 644) load / save data with updates journal.

Note, in cached updates mode some methods or property settings will raise an exception, if there are changes in updates journal. They must be saved then commited or canceled. Those are:

- Refresh;
- setting CachedUpdates (see page 592) to False.

Applying updates

To apply updates to a database use the ApplyUpdates (see page 611) method. If some record applying raised an exception, then it will be associated with the record. Note, that ApplyUpdates method:

- does not raise an exception, instead it returns the number of raised exceptions;
- does not wrap updates applying into a transaction, an application may do that itself;
- uses the same updates posting logic as the immediate updates.

After applying updates the changed records still remains in the changes log. To remove them from the changes log and mark as unmodified call the CommitUpdates (see page 615) method. For example:

```
ADQuery1.CachedUpdates := True;
ADQuery1.Append;
...
ADQuery1.Post;
ADQuery1.Append;
...
ADQuery1.Post;
ADQuery1.Append;
...
ADQuery1.Post;
ADConnection1.StartTransaction;
iErrors := ADQuery1.ApplyUpdates;
if iErrors = 0 then begin
  ADQuery1.CommitUpdates;
  ADConnection1.Commit;
end
else
  ADConnection1.Rollback;
```

Reviewing errors

If an error happens inside of ApplyUpdates call, then ApplyUpdates records the error into the internal data record structure. And continues to process updates until the errors number will be equal or greater than AMaxErrors. ApplyUpdates does not raise exceptions. To process all erroneous records after ApplyUpdates call, use either reconciling process, either filter

erroneous records.

To reconcile records assign OnReconcileError (see page 604) event handler and call the Reconcile (see page 642) method. The OnReconcileError (see page 604) event handler allows to analyze error, read / change the current record field values. On exit it should assign action, which FireDAC code should take on the current record with the error. After the Reconcile (see page 642) method call the ApplyUpdates (see page 611) may be called again to try to post again erroneous record changes.

To filter erroneous records include rtHasErrors into FilterChanges (see page 597). Then navigate through dataset and read RowError (see page 607) property to get an exception object associated with the current record. For example:

```
var
  oErr: EADException;
...
if ADQuery1.ApplyUpdate > 0 then begin
  ADQuery1.FilterChanges := [rtModified, rtInserted, rtDeleted, rtHasErrors];
  try
    ADQuery1.First;
    while not ADQuery1.Eof do begin
      oErr := ADQuery1.RowError;
      if oErr <> nil then begin
        // process exception object
        ...
      end;
      ADQuery1.Next;
    end;
  finally
    ADQuery1.FilterChanges := [rtUnmodified, rtModified, rtInserted];
  end;
end;
```

See Also

Update Command Generation (see page 121), Overriding Posting Updates (see page 123)

Example

See the demos for more details:

- FireDAC\Samples\Comp Layer\TADMemTable\CachedUpdates

- FireDAC\Samples\Comp Layer\TADQuery\CachedUpdates\OnUpdateRecord

- FireDAC\Samples\Comp Layer\TADQuery\CachedUpdates\Centralized

1.7.3 Unique Identifying Fields

FireDAC uses unique identifying columns for generating the efficient update and refresh SQL commands.

Description

General

The set of columns unique identifying each dataset record are the unique identifying columns. When a dataset is a single table SELECT, then the set is a FROM table primary key. Often an auto-incrementing field (see page 118) is a table primary key.

Usage

Unique identifying fields are used to build WHERE clause for:

- posting updates (Edit/Post) and deletes (Delete) from a dataset, when UpdateOptions.UpdateMode (see page 904) is upWhereKeyOnly or upWhereChanged;
- refreshing the current dataset record (RefreshRecord (see page 642));

- selecting deferred BLOB (FetchBlobs (see page 624)) values and nested datasets (FetchDetails (see page 625)).

Providing

The TADQuery (see page 476), TADTable (see page 539), TADMemTable (see page 436), TADCommand (see page 267) automatically retrieve the unique identifying columns (mkPrimaryKeyFields) for the main (first) table in the SELECT ... FROM ... statements, when fiMeta is included into FetchOptions (see page 651).Items (see page 855). Note:

- mkPrimaryKeyFields querying may be time consuming;
- application may need to explicitly specify unique identifying columns, when FireDAC fails to determine them correctly.

To explicitly specify columns, exclude fiMeta from FetchOptions.Items, and use one of the options:

- set UpdateOptions.KeyFields (see page 845), as a ';' separated list of the column names;
- or include pfInKey into corresponding TField.ProviderFlags property.

Note, when application is using persistent fields, then creating them the ProviderFlags will be set correctly. After that automatic retrieval will be ignored. Update ProviderFlags to adjust column list. Also, if the primary key consists of few fields then all of them must be included into persistent fields.

Row identifying columns

Alternatively a row identifying column may be included into SELECT list. When FireDAC founds such column, then it will not retrieve mkPrimaryKeyFields metadata and will use this column. The supported DBMS's:

DBMS	The row identifying column
Firebird	DB_KEY
Interbase	DB_KEY
Oracle	ROWID
PostgreSQL	OID. Note, the table must be created with OID's.
SQLite	ROWID

For example on Oracle:

```
SELECT T.*, ROWID FROM Orders T
```

See Also

Update Command Generation (see page 121), Auto-Incremental Fields (see page 118), KeyFields (see page 845), Items (see page 855), UpdateMode (see page 904), ProviderFlags

1.7.4 Auto-Incremental Fields

FireDAC supports correct appending of the new records with auto-incrementing fields and automatic refreshing of the database assigned values.

Description

General

FireDAC allows to insert a new record with an auto-incrementing column and get back a new value of this column. That works as for immediate updates, as for cached updates (see page 114). Depending on a DBMS, the auto-incrementing fields may be implemented either using a special IDENTITY (or similar) column data type, either using a generator (or a sequence) and a table trigger:

DBMS	Auto-incrementing column implementation	Recognizing
Advantage Database	AUTOINC data type	Automatic as auto-incrementing data type.
Firebird / Interbase	• Generator and BEFORE INSERT trigger • Firebird 3.0: GENERATED BY DEFAULT AS IDENTITY	Automatic and manual specifying. See below.
IBM DB2	GENERATED AS IDENTITY data type	Automatic as auto-incrementing data type.
Informix	SERIAL data type	Automatic as auto-incrementing data type.
MS Access	COUNTER data type	Automatic as auto-incrementing data type.
MS SQL Server	IDENTITY data type	Automatic as auto-incrementing data type.
MySQL	AUTO_INCREMENT data type	Automatic as auto-incrementing data type.
Oracle	Sequence and BEFORE INSERT FOR EACH ROW trigger	Manual specifying. See below.
PostgreSQL	SERIAL data type	Automatic as auto-incrementing data type.
SQL Anywhere	IDENTITY data type	Automatic as auto-incrementing data type.
SQLite	INTEGER PRIMARY KEY AUTOINCREMENT data type	Automatic as auto-incrementing data type.
Sybase Adaptive Server Enterprise	IDENTITY data type	Automatic as auto-incrementing data type.

Automatic recognition

FireDAC automatically recognizes a column of an auto-incrementing data type, and defines it as dtIntXxx, [caAutoInc, caReadOnly, caAllowNull]. This leads to TField setup:

- TField.DataType = ftAutoInc (TADAutoIncField (see page 579)) when dtInt32 or dtUInt32; otherwise one of the numeric ftXxxx data types;
- TField.Required = False;
- TField.ReadOnly = True;
- TField.ProviderFlags = [pfInWhere], or [pfInWhere, pfInKey] when the column is part of a primary key.

FireDAC automatically recognizes limited set of the Firebird auto-incrementing columns, and defines them as dtIntXxx, [caAutoInc, caAllowNull] when:

- extended metadata (see page 129) is enabled;
- a table has a BEFORE INSERT trigger;
- the trigger depends on a single column and a single generator. This column is recognizing as auto-incrementing.

Manual specifying

For Oracle, other Firebird / Interbase cases, and other columns the auto-incrementing mode may be specified by programmer, using one of the options:

- set UpdateOptions.AutoIncFields (see page 845) to the list of the auto-incrementing column names. When a column is of dtInt32 / dtUInt32 data type, then TADAutoIncField (see page 579) field will be created.
- create an TADAutoIncField (see page 579) field at design or run time. The column must be of ftAutoInc / ftInteger / ftLongWord field type. For other data types consider to use data type mapping (see page 865) to TargetDataType (see page 873) = dtInt32.

- set TField.AutoGenerateValue to arAutoInc for an auto-incrementing field. This method does not create TADAutoIncField (see page 579), works with any field type, and may require additional field properties setup, eg setting ProviderFlags, Required and ReadOnly.

All these methods at end leads to TField.AutoGenerateValue = arAutoInc.

Client auto-incrementing

By default FireDAC uses the client side auto-incrementing for the auto-incrementing columns. After calling dataset Insert / Append method, the auto-incrementing column will get -1 value. With each next method call, the value will be incremented with the -1 (negative) step. That was made to distinguish the client assigned values and the DBMS assigned values.

When UpdateOptions.RefreshMode (see page 903) <> rmManual, then after posting a new record the auto-incrementing column will get an actual positive value.

The TADAutoIncField (see page 579) properties allows to adjust the client side auto-incrementing:

- ClientAutoIncrement (see page 581) to enable or disable;
- AutoIncrementSeed (see page 580) to specify an initial value;
- AutoIncrementStep (see page 580) to specify a step.

Client sequence filling

When a DBMS supports sequences or generators (here they are synonyms), then FireDAC may fill an auto-incrementing column from a sequence on a client. This is alternative to client-side auto-incrementing.

The auto-incrementing column must have pfInUpdate in TField.ProviderFlags and one of the UpdateOptions.FetchGeneratorsPoint (see page 901) values:

- gpImmediate - a next value is fetched from a generator in TDataSet.Insert / Append method. And the value is already accessible in TDataSet.OnNewRecord event handler. When the TDataSet.Cancel method is called, then the value is lost.
- gpDeferred - a next value is fetched in TDataSet.Post method.

Set the UpdateOptions.GeneratorName (see page 901) or TADAutoIncField.GeneratorName (see page 581) to a generator name, which will be used to get next sequence value.

Universal setup

The following auto-incrementing column setup will work with any DBMS, including supporting native auto-incremental columns, and using sequences and triggers:

- UpdateOptions.FetchGeneratorsPoint (see page 901) = gpNone. The client sequence filling is disabled.
- not (pfInUpdate in TField.ProviderFlags). The column is excluded from INSERT / UPDATE clause.
- fiMeta in FetchOptions.Items (see page 855). The column, when it is possible is recognized as auto-incremental.

For Oracle, Firebird and Interbase is required to manually specify auto-incremental columns. For databases supporting native auto-incremental data types, no additional setup is required. But specifying will work in both cases.

Master-detail handling

When master and detail datasets (see page 104) are linked by an auto-incrementing master column and cached updates is enabled, then consider to use Centralized Cached Updates (see page 114) mode with propagation of changes from master to detail datasets.

See Also

Unique Identifying Fields (see page 117), Update Command Generation (see page 121), Caching Updates (see page 114), FetchGeneratorsPoint (see page 901), GeneratorName (see page 901), AutoIncFields (see page 845),

TADAutoIncField (see page 579)

1.7.5 Update Command Generation

FireDAC will automatically generate SQL commands to post changes from a dataset to a database, using the obtained meta information for the columns and update table.

Description

General

When the database application calls dataset methods:

- Insert / Edit / Post / Delete - in immediate updates mode,
- ApplyUpdates - in cached updates mode,

then updating data SQL commands will be automatically generated by FireDAC. The FireDAC SQL commands generator is aware of the database identity fields, sequences, triggers, special data types (Oracle BLOB / CLOB / BFILE), pessimistic locking commands, etc. And generates a most efficient SQL command, depending on the connected DBMS. This reduces the number of cases where a developer must use the hand made SQL commands. FireDAC does not force a developer to use TADUpdateSQL (see page 563), which may be used to override the update SQL commands.

For example, when posting a new record to an Oracle table, where ID field is filled by a trigger from a sequence and IMAGE is of BLOB type, FireDAC will generate the SQL command:

```
INSERT INTO OracleTab (NAME, DT, IMAGE)
VALUES (:NEW_NAME, :NEW_DT, EMPTY_BLOB())
RETURNING :NEW_ID, :NEW_IMAGE
```

1. FROM, INTO and UPDATE phrases

FireDAC will use the main (first) table in the SELECT ... FROM ... statement, as an updating table name. Also, this table will be used to retrieve mkPrimaryKeyFields metadata. Use UpdateOptions.UpdateTableName (see page 846) to explicitly specify the updating table. This is required, when:

- a dataset is TADStoredProc (see page 514);
- TADQuery (see page 476) has not a SELECT query;
- FireDAC fails to get the updating table name correctly from the query;
- application needs to redirect updates to a specific table.

2. WHERE phrase

The UpdateOptions.UpdateMode (see page 904) controls the WHERE clause generation for posting updates and deletions. The default value upWhereKeyOnly uses in WHERE phrase only the unique identifying columns and provides most efficient and safe way to locate the updating row. When no unique identifying columns is specified and no row identifying column is found, FireDAC will switch UpdateOptions.UpdateMode (see page 904) to upWhereAll. And some fields included into WHERE clause may lead to no rows found:

- DOUBLE, FLOAT, TIME, DATETIME fields and other floating point fields may lead to a precision lost at value comparison;
- textual fields may have an invalid encoding or extra spaces leading to unsuccessful comparision;
- other similar failures may happen.

In such cases application gets an exception:

```
[FireDAC][DApt]-400. Update command updated [0] instead of [1] record.
```

Similarly, when columns in WHERE are not unique identifying a row, more than single record may be updated. Then an exception is:

```
[FireDAC][DApt]-400. Update command updated [4] instead of [1] record.
```

To resolve these issues consider to:

- provide correct unique identifying columns;
- disable some fields WHERE usage by excluding pfInWhere from corresponding TField.ProviderFlags property;
- suppress these errors by setting UpdateOptions.CountUpdatedRecords (see page 899) to False.

Note, to avoid the above issues with a SQL Server table, which also has triggers, the triggers should have SET NOCOUNT ON specified at the beginning. If that is not possible, then set UpdateOptions.CountUpdatedRecords (see page 899) to False.

3. SET and VALUES phrases

The UpdateOptions.UpdateChangedFields (see page 904) controls the fields to include into UPDATE SET ... or INSERT VALUES ... phrases. Setting to True will include only changed fields, what helps:

- to minimize the traffic at updates posting;
- to avoid not-needed constraints validations;
- to avoid extra trigger firings;
- to minimize redo log generation.

Setting to False will include all fields, what helps:

- to reuse the same generated statement for posting all updates and minimize DBMS work to prepare a statements.

To disable updates to some columns exclude pfInUpdate from corresponding TField.ProviderFlags property.

4. RETURNING and additional SELECT's

The UpdateOptions.RefreshMode (see page 903) = rmOnDemand controls automatic refreshing of the column values, which may be changed by the DBMS after inserting or updating a record. The columns, which may require refreshing after record inserting are:

- auto-incrementing columns;
- database calculated columns;
- columns with default values;
- row identifying columns;
- timestamp columns;
- columns updated by a trigger.

The columns, which may require refreshing after record updating are:

- database calculated columns;
- timestamp columns;
- columns updated by a trigger.

Depending on a DBMS features, the additional phrases / commands will be generated, returning the refreshing column values:

- Oracle, Firebird, PostgreSQL - RETURNING (see page 66) phrase;
- DB2 - SELECT ... FROM FINAL TABLE phrase;
- SQL Server, SQL Anywhere, SQLite - SQL batch with additional SELECT command;
- otherwise - an additional SELECT command.

5. Updates command caching

For some DBMS, environment and task the UpdateChangedFields (see page 904) = False may give a better performance. And combined with few other settings as a UpdateOptions.FastUpdates (see page 900) property, it allows to get even

better performance at updates posting, by avoiding the additional queries and enabling to cache the generated update commands.

The FastUpdates = True is equivalent to:

- UpdateChangedFields (see page 904) = False;
- UpdateMode (see page 904) = upWhereKeyOnly;
- LockMode (see page 902) = lmNone;
- RefreshMode (see page 903) = rmManual;
- CheckRequired (see page 898) = False.

See Also

Editing the Data (see page 113), TADUpdateOptions (see page 896)

1.7.6 Overriding Posting Updates

FireDAC allows to selectively override the automatically generated update SQL commands by using TADUpdateSQL or completely override the posting algorithm by using OnUpdateRecord event handler.

Description

General

Although FireDAC is able to automatically generate updating SQL commands, in some cases this can not be done correctly:

Original SQL command	Generated SQL command possible issues	Possible actions
SELECT with JOIN's	Includes columns from the joined tables.	• Exclude non-base table column from update commands, by excluding pfInUpdate from TField.ProviderFlags and setting TField.ReadOnly to True; • Set UpdateOptions.UpdateNonBaseFields (see page 905) to False (default value).
SELECT with grouping operations	Updates more than one record.	• Redirect updates to a correct table by setting UpdateOptions.UpdateTableName (see page 846); • Disable updated record count checking, by setting UpdateOptions.CountUpdatedRecords (see page 899) to False; • Use custom SQL commands.
SELECT with common table expressions	Fails to get updating table name.	• Redirect updates to a correct table by setting UpdateOptions.UpdateTableName (see page 846); • Use custom SQL commands.
SELECT with aliased columns and expressions	Includes incorrect column names.	• Optionally, exclude non-base table column from update commands, by excluding pfInUpdate from TField.ProviderFlags and setting TField.ReadOnly to True. • Optionally, specify base column name, by specifying TField.Origin.
Stored procedure call	All above SELECT issues.	
Cursor expressions and variable	All above SELECT issues.	
Other non-SELECT commands	All above SELECT issues.	

Note, the above is not a must, instead check first, what really happens.

FireDAC can detect a base table and column names for an aliased or joined columns:

DBMS	Setup
Advantage Database	Is not supported.
Firebird / Interbase	Required ExtendedMetadata=True (see page 129) connection definition parameter.
IBM DB2	Automatically.
MS Access	Automatically.
MS SQL Server	Required ExtendedMetadata=True (see page 129) connection definition parameter.
MySQL	Automatically.
Oracle	Is not supported.
PostgreSQL	Required ExtendedMetadata=True (see page 129) connection definition parameter.
SQL Anywhere	Automatically.
SQLite	Automatically.

Specifying update table name

In some cases application needs to specify alternative DB table name, to which the updates will be posted. To do that set UpdateOptions.UpdateTableName (see page 846) to a required table name.

Specifying update column names and modes

In some cases application needs to exclude columns from an updating SQL command. To do that exclude pfInUpdate from TField.ProviderFlags. And set TField.ReadOnly to prohibit field value modification.

To specify alternative DB column name, set TField.Origin to a required value.

Using TADUpdateSQL

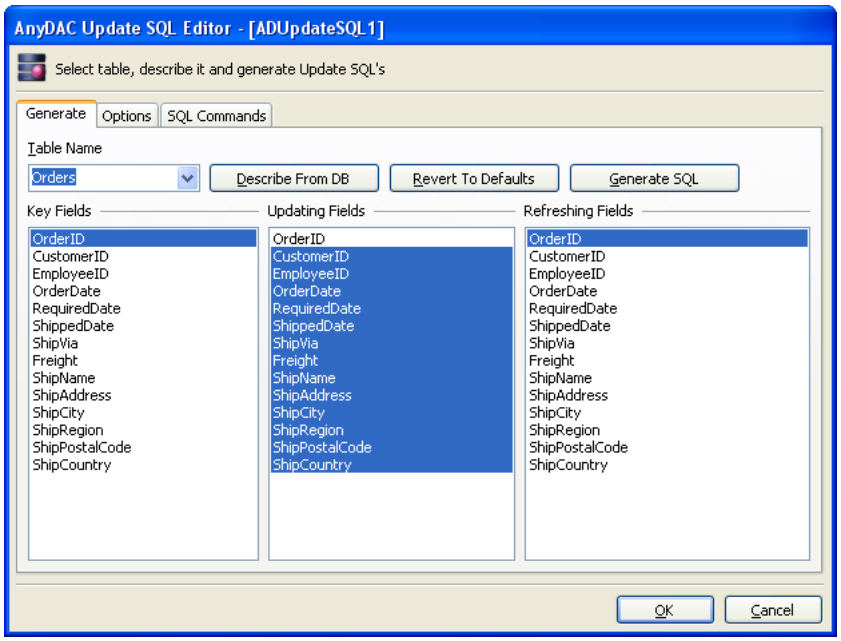
The TADUpdateSQL (see page 563) component allows to:

- selectively override the update SQL commands generated by FireDAC;
- enable updates posting, when FireDAC cannot generate the update commands, like a complex SELECT's or stored procedure calls.

In general, the TADUpdateSQL (see page 563) is a collection of SQL update commands, each of them handles specific task, like a inserting a new record into database. The TADUpdateSQL (see page 563) may be setup at design-time and / or at run-time.

Setting at design time

To setup at design-time, drop TADUpdateSQL on a form. Set dataset UpdateObject (see page 267) property to point to this component. Then double click on TADUpdateSQL to invoke the FireDAC Update SQL Editor:



The editor will automatically get the updating table name from the associated dataset and setup columns according to the dataset columns TField.ProviderFlags and TField.AutoGenerateValue.

To change setup, use:

- the "Table Name" combo to specify updating table name;
- the "Key Fields" list box to specify unique identifying columns (corresponds to pfInKey in ProviderFlags);
- the "Updating Fields" list box to specify columns to include into update (corresponds to pfInUpdate in ProviderFlags);
- the "Refreshing Fields" list box to specify columns those values must be refreshed after posting update (corresponds to AutoGenerateValue <> arNone).

The "Describe From DB" button provides setup for the specified table, by retrieving its metadata from the database. The "Revert To Defaults" button provides setup using the dataset field properties. The additional options may be specified on the "Options" page.

When setup is finished press the "Generate SQL" button to generate full set of the updating SQL's. Also, the editor may be used to manually edit update SQL commands on the "SQL Commands" page.

Press OK to store changes in the TADUpdateSQL (see page 563).

Settings at run time

To specify the TADUpdateSQL (see page 563) SQL commands at run-time, the application should use XxxxSQL properties. To reference in a SQL to a specific column value use the parameter markers:

- new column value - ':NEW_<column name>';
- old column value - ':OLD_<column name>';
- current column value - ':<column name>'.

These parameter values will be assigned automatically by FireDAC. Do not assign a value to them, because their values will be overridden. FireDAC will ignore parameters with other names.

The command parameters and command macros may be setup only at run-time. For that you have to use TADUpdateSQL.Commands (see page 564) property, which returns references to the TADCommand (see page 267) objects. For example:

```
ADUpdateSQL1.InsertSQL.Text := 'insert into &tab (id, name) values (:new_id, :new_name)';
ADUpdateSQL1.Commands[arInsert].Macros[0].AsRaw := 'Orders';
```

Using OnUpdateRecord

This event allows to completely override posting updates from a dataset. The event handler code can read the dataset fields properties:

- OldValue - returns the original field value, as it was fetched or as it is after last CommitUpdates (see page 615) / CancelUpdates (see page 614) call;
- CurValue / Value - returns the current field value.

Also, you can combine TADUpdateSQL's and OnUpdateRecord approaches, to enable semi-automatic updates posting to different tables or databases.

For details see OnUpdateRecord (see page 605) event description and the FireDAC demos:

- FireDAC\Samples\Comp Layer\TADQuery\CachedUpdates\OnUpdateRecord
- FireDAC\Samples\Comp Layer\TADUpdateSQL\Main

See Also

Update Command Generation (see page 121), TADUpdateSQL (see page 563), OnUpdateRecord (see page 605)

1.7.7 Specifying Default Values

FireDAC allows to specify default field values for new records.

Description

Using OnNewRecord event handler

The standard way in Delphi to assign the default values to the new record fields is to use OnNewRecord event handler. This event handler fires after the auto-incremental and detail record linking fields get their values and automatic default values are assigned. The event handler may override the above values or to assign initial value using some complex custom logic.

The event handler may be assigned to the dataset at any time.

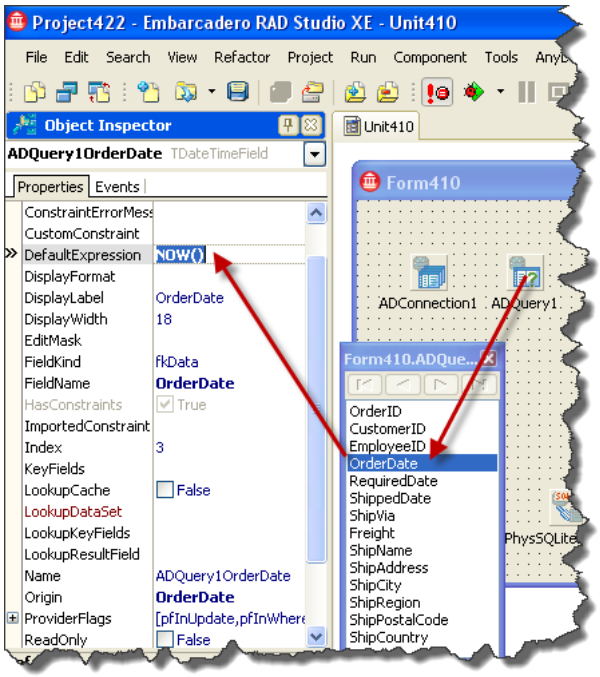
Specifying default expression value

FireDAC allows to assign an expression (see page 110) to the TField.DefaultExpression property. This expression will be evaluated one time as part of the Insert / Append method call and assigned as an initial value to the corresponding field. Later this value may be overridden by the application. Note, when a field has TField.FieldKind = fkInternalCalc, then the DefaultExpression will be used to calculate the field value.

When a value to a field DefaultExpression assigned when a dataset is closed, then it will be actualized automatically. When a dataset is opened, then call UpdateAttributes (see page 649) method to actualize changes. For example:

```
ADQuery1.Open;
...
ADQuery1.FieldByName('ObjGUID').DefaultExpression := 'NEWGUID()';
ADQuery1.UpdateAttributes;
```

FireDAC does not automatically fetch column default expression from the database dictionary. The most simple way to assign an expression is to use Fields Editor at design-time:



Refreshing default value

FireDAC automatically recognizes that a result set column has a default value for the following DBMS:

DBMS	Description
Interbase / Firebird	fiMeta is in FetchOptions.Items and ExtendedMetadata = True
MySQL	
PostgreSQL	fiMeta is in FetchOptions.Items and ExtendedMetadata = True

When FireDAC recognizes a column with a default value then TField.AutoGenerateValue is set to arDefault. If that does not happen, then AutoGenerateValue may be set by hands. If a corresponding field value was not assigned at adding a new record, then FireDAC automatically refreshes the field value after posting a new record.

See Also

Writing Expressions (see page 110), Calculated and Aggregated Fields (see page 108)

1.8 Working with Metadata

The set of articles describing how to work with database meta data in FireDAC. The metadata contains information about database structure.

1.8.1 Querying Metadata

FireDAC offers TADMetaInfoQuery components and some TADConnection methods to retrieve metadata from a database.

Description

Using TADConnection

TADConnection (see page 279) offers few GetXxxxNames simple to use methods returning the database object names list:

- GetCatalogNames (see page 346) - the catalog list;
- GetSchemaNames (see page 350) - the schema list;
- GetTableNames (see page 351) - the table and view list;
- GetFieldNames (see page 346) - the table field list;
- GetKeyFieldNames (see page 349) - the table primary key list;
- GetGeneratorNames (see page 347) - the generator / sequence list;
- GetPackageNames (see page 350) - the package list;
- GetStoredProcNames (see page 351) - the stored procedure list.

These method are useful when:

- application needs object names;
- application does not need additional info about these objects.

For example, GetTableNames (see page 351) method retrieves the table, view and synonym names from the current database and schema. To call it use the code:

```
ADConnection1.GetTableNames('Northwind', 'dbo', '', Memo1.Lines);
```

Use the ACatalogName and / or ASchemaName arguments to restrict the returned list to the specified catalog and / or schema. If they are not specified, then all "visible" objects will be returned or the objects in the current database / schema. That depends on the DBMS. Note, some DBMS (MS Access) may return error if the schema / catalog is specified and it is not supported by the DBMS.

Additionally, the application may restrict object list using AScopes argument, basing on the object scope. And the APattern argument, which is the LIKE search mask, which will be applied to object names.

The similar method you can found at TADCustomManager (see page 365) class.

See the FireDAC demo for more details: FireDAC\Samples\Comp Layer\TADConnection\GetFieldNames.

Using TADMetaInfoQuery

The TADMetaInfoQuery (see page 461) is the dataset component allowing to query and browse the metadata lists. To do that set the Connection (see page 466), MetaInfoKind (see page 471) and optionally the CatalogName (see page 466), SchemaName (see page 474), BaseObjectName (see page 465), ObjectName (see page 471) properties, and open the dataset. To get the tables list use the code:

```
ADMetaInfoQuery1.Connection := ADConnection1;
ADMetaInfoQuery1.MetaInfoKind := mkTables;
ADMetaInfoQuery1.Open;
```

For the dataset structure description read the "Metadata Structure (see page 130)" chapter. If a DBMS does not support some particular metadata kind, for example SQL Server does not support mkGenerators, then empty dataset will be returned. The metadata dataset is read-only and cannot be edited.

Metadata caching

The fetched metadata is cached by FireDAC per each connection. That is controlled by including fiMeta into FetchOptions (see page 651).Cache (see page 852).

Excluding fiMeta from FetchOptions (see page 651).Cache (see page 852) does not invalidate the metadata cache. All cached metadata remains in cache until it next usage. To refresh the metadata cache for a specified object or in full use the

TADConnection.RefreshMetadataCache (see page 355).

Current catalog and schema

To get current catalog and schema name, use TADConnection.ConnectionIntf (see page 326).CurrentCatalog and CurrentSchema. To change current catalog or schema, assign new value to these properties.

MetaCurCatalog and MetaCurSchema (see page 188) allows to override the information returned by the database session. For some databases it may be required to use these parameters to specify correct values, because API returns incorrect ones.

See Also

Metadata Structure (see page 130), Unique Identifying Fields (see page 117), TADConnection (see page 279), TADMetaInfoQuery (see page 461)

1.8.2 Extended Metadata

Depending on a DBMS the metadata may include advanced data type recognition for the columns.

Description

To enable this feature the connection definition parameter ExtendedMetadata=True must be added. And the DBMS's:

DBMS	Description
IBM DB2 (see page 196)	• Origin table and column name for each item in select list; • Columns updateability; • Columns auto-incrementing mode.
Firebird, Interbase (see page 198)	• Columns with domain %BOOL% are recognized as ftBoolean; • Columns with domain %GUID% are recognized as ftGUID; • When table has a BEFORE INSERT trigger, which depends on a single sequence and a single table column, then this column is recognized as ftAutoInc; • Columns GENERATED BY DEFAULT AS IDENTITY are recognized as ftAutoInc; • Column COMPUTED BY are recognized as TField.AutoGenerateValue = arDefault.
PostgreSQL (see page 218)	• Origin table and column name for each item in select list; • Columns optionality; • Columns updateability; • Columns with domain LO, LARGEOBJECT or BLOB are recognized as ftOraBlob (BLOB handled by the reference); • Columns with DEFAULT NEXTVAL('xxx') are recognized as ftAutoInc; • Columns with other default values are recognized as [caDefault]; • Columns with system OID are recognized as [caROWID].
SQL Server (see page 202)	• Origin table and column name for each item in select list.

Extended metadata recognition is supported only for the dataset columns and not for the mkTableFields metadata request.

Note, ExtendedMetadata=True executes additional queries. That may decrease application performance.

See Also

Overriding Posting Updates (see page 123), Auto-Incremental Fields (see page 118), Database Connectivity (see page 188)

1.8.3 Metadata Structure

The chapter describes the structures of the metadata returned by FireDAC TADMetaInfoQuery.

Description

The catalog list (mkCatalogs)

#	Column name	Data type	Description
1	RECNO	dtInt32	
2	CATALOG_NAME	dtWideString	Catalog name.

The schema list (mkSchemas)

#	Column name	Data type	Description
1	RECNO	dtInt32	
2	CATALOG_NAME	dtWideString	Catalog name.
3	SCHEMA_NAME	dtWideString	Schema name.

The table list (mkTables)

#	Column name	Data type	Description
1	RECNO	dtInt32	
2	CATALOG_NAME	dtWideString	Catalog name.
3	SCHEMA_NAME	dtWideString	Schema name.
4	TABLE_NAME	dtWideString	Table name.
5	TABLE_TYPE	dtInt32	Table type. Cast value to uADPhysIntf.TADPhysTableKind.
6	TABLE_SCOPE	dtInt32	Table scope. Cast value to uADPhysIntf.TADPhysObjectScope.

The table field list (mkTableFields)

#	Column name	Data type	Description
1	RECNO	dtInt32	
2	CATALOG_NAME	dtWideString	Catalog name.
3	SCHEMA_NAME	dtWideString	Schema name.
4	TABLE_NAME	dtWideString	Table name.
5	COLUMN_NAME	dtWideString	Column name.
6	COLUMN_POSITION	dtInt32	Column position.
7	COLUMN_DATATYPE	dtInt32	Column data type. Cast value to uADStanIntf.TADDataType.

8	COLUMN_TYPENAME	dtWideString	DBMS native column type name.
9	COLUMN_ATTRIBUTES	dtUInt32	Column attributes. Cast value to uADStanIntf.TADDataAttributes.
10	COLUMN_PRECISION	dtInt32	Numeric and date/time column precision.
11	COLUMN_SCALE	dtInt32	Numeric and date/time column scale.
12	COLUMN_LENGTH	dtInt32	Character and byte string column length.

Note, to cast COLUMN_ATTRIBUTES column value to TADDataAttributes use the code:

```
var
  i: Integer;
...
i := ADMetaInfoQuery1.FieldByName('COLUMN_ATTRIBUTES').AsInteger;
eAttrs := TADDataAttributes(Pointer(@i)^);
```

The table indexes (mkIndexes) and primary key list (mkPrimaryKey)

#	Column name	Data type	Description
1	RECNO	dtInt32	
2	CATALOG_NAME	dtWideString	Catalog name.
3	SCHEMA_NAME	dtWideString	Schema name.
4	TABLE_NAME	dtWideString	Table name.
5	INDEX_NAME	dtWideString	Index name.
6	PKEY_NAME	dtWideString	Primary key constraint name.
7	INDEX_TYPE	dtInt32	Index type. Cast value to uADPhysIntf.TADPhysIndexKind.

The table index fields (mkIndexFields) and primary key fields list (mkPrimaryKeyFields)

#	Column name	Data type	Description
1	RECNO	dtInt32	
2	CATALOG_NAME	dtWideString	Catalog name.
3	SCHEMA_NAME	dtWideString	Schema name.
4	TABLE_NAME	dtWideString	Table name.
5	INDEX_NAME	dtWideString	Index name.
6	COLUMN_NAME	dtWideString	Indexed column name.
7	COLUMN_POSITION	dtInt32	Column position in the index.
8	SORT_ORDER	dtWideString	Column sort order. 'A' - ascending. 'D' - descending.
9	FILTER	dtWideString	Optional column filter expression.

The table foreign key list (mkForeignKeys)

#	Column name	Data type	Description
1	RECNO	dtInt32	
2	CATALOG_NAME	dtWideString	Catalog name.

3	SCHEMA_NAME	dtWideString	Schema name.
4	TABLE_NAME	dtWideString	Table name.
5	FKEY_NAME	dtWideString	Foreign key constraint name.
6	PKEY_CATALOG_NAME	dtWideString	Referenced table catalog name.
7	PKEY_SCHEMA_NAME	dtWideString	Referenced table schema name.
8	PKEY_TABLE_NAME	dtWideString	Referenced table name.
9	DELETE_RULE	dtInt32	Foreign key delete rule. Cast value to uADPhysIntf.TADPhysCascadeRuleKind.
10	UPDATE_RULE	dtInt32	Foreign key update rule. Cast value to uADPhysIntf.TADPhysCascadeRuleKind.

The table foreign key fields list (mkForeignKeyFields)

#	Column name	Data type	Description
1	RECNO	dtInt32	
2	CATALOG_NAME	dtWideString	Catalog name.
3	SCHEMA_NAME	dtWideString	Schema name.
4	TABLE_NAME	dtWideString	Table name.
5	FKEY_NAME	dtWideString	Foreign key constraint name.
6	COLUMN_NAME	dtWideString	Column name.
7	PKEY_COLUMN_NAME	dtWideString	Referenced table column name.
8	COLUMN_POSITION	dtInt32	Column position in constraint.

The packages list (mkPackages)

#	Column name	Data type	Description
1	RECNO	dtInt32	
2	CATALOG_NAME	dtWideString	Catalog name.
3	SCHEMA_NAME	dtWideString	Schema name.
4	PACKAGE_NAME	dtWideString	Package name.
5	PACKAGE_SCOPE	dtInt32	Package scope. Cast value to uADPhysIntf.TADPhysObjectScope.

The stored procedures list (mkProcs)

#	Column name	Data type	Description
1	RECNO	dtInt32	
2	CATALOG_NAME	dtWideString	Catalog name.
3	SCHEMA_NAME	dtWideString	Schema name.
4	PACK_NAME	dtWideString	Optional package name.
5	PROC_NAME	dtWideString	Procedure name.
6	OVERLOAD	dtInt32	Overloaded procedure number.

7	PROC_TYPE	dtInt32	Procedure type. Cast value to uADPhysIntf.TADPhysProcedureKind.
8	PROC_SCOPE	dtInt32	Procedure scope. Cast value to uADPhysIntf.TADPhysObjectScope.
9	IN_PARAMS	dtInt32	Number of input parameters.
10	OUT_PARAMS	dtInt32	Number of output parameters.

The stored procedure arguments list (mkProcArgs)

#	Column name	Data type	Description
1	RECNO	dtInt32	
2	CATALOG_NAME	dtAnsiString	Catalog name.
3	SCHEMA_NAME	dtAnsiString	Schema name.
4	PACK_NAME	dtAnsiString	Optional package name.
5	PROC_NAME	dtAnsiString	Procedure name.
6	OVERLOAD	dtInt32	Overloaded procedure number.
7	PARAM_NAME	dtAnsiString	Parameter name.
8	PARAM_POSITION	dtInt32	Parameter position.
9	PARAM_TYPE	dtInt32	Parameter type. Cast value to DB.TParamType.
10	PARAM_DATATYPE	dtInt32	Parameter data type. Cast value to uADStanIntf.TADDataType.
11	PARAM_TYPENAME	dtAnsiString	DBMS native parameter data type name.
12	PARAM_ATTRIBUTES	dtUInt32	Parameter attributes. Cast value to uADStanIntf.TADDataAttributes.
13	PARAM_PRECISION	dtInt32	Numeric and date/time parameter precision.
14	PARAM_SCALE	dtInt32	Numeric and date/time parameter scale.
15	PARAM_LENGTH	dtInt32	Character and byte string parameter length.

The generators / sequences list (mkGenerators)

#	Column name	Data type	Description
1	RECNO	dtInt32	
2	CATALOG_NAME	dtWideString	Catalog name.
3	SCHEMA_NAME	dtWideString	Schema name.
4	GENERATOR_NAME	dtWideString	Generator / sequence name.
5	GENERATOR_SCOPE	dtInt32	Generator / sequence scope. Cast value to uADPhysIntf.TADPhysObjectScope.

See Also

Querying Metadata (see page 127)

1.8.4 Object Names

FireDAC supports quoted object names and full object names. The chapter describes how to work with object names.

Description

General

The following properties specify the database object names:

- TADStoredProc: CatalogName (see page 521), SchemaName (see page 536), PackageName (see page 404) and StoredProcName (see page 406);
- TADTable: CatalogName (see page 547), SchemaName (see page 558) and TableName (see page 559);
- TADBottomUpdateOptions: UpdateTableName (see page 846);
- TADUpdateOptions: GeneratorName (see page 901);
- TADAutoIncField: GeneratorName (see page 581);
- etc

A name may be specified:

- without quotes and FireDAC will normalize name;
- enclosed into quotes and FireDAC will use it as is.

For example on Firebird with SQL dialect 3:

DDL	Will work	Will fail
CREATE PROCEDURE Test	ADStoredProc1.StoredProcName := 'test'; ADStoredProc1.StoredProcName := 'TeSt'; ADStoredProc1.StoredProcName := '"TEST"';	ADStoredProc1.StoredProcName := '"test"';
CREATE PROCEDURE "Test"	ADStoredProc1.StoredProcName := '"Test"';	ADStoredProc1.StoredProcName := 'test'; ADStoredProc1.StoredProcName := 'TeSt'; ADStoredProc1.StoredProcName := 'Test'; ADStoredProc1.StoredProcName := '"TEST"';
CREATE PROCEDURE "Main Proc"	ADStoredProc1.StoredProcName := '"Main Proc"';	ADStoredProc1.StoredProcName := 'main proc'; ADStoredProc1.StoredProcName := 'Main Proc';

Quoted object names

FireDAC fully supports quoted object names, including different DBMS quoted characters. To write a quoted name in a SQL in a DBMS independent fashion use the {id <name>} syntax. For example:

```
SELECT * FROM {id Order Details}
```

To force TADTable to quote field names set FormatOptions.QuoteIdentifiers (see page 867) to True.

Full object names

FireDAC supports the full object names, which include the catalog and / or schema names.

When you are using the design-time editors, like a TADStoredProc.StoredProcName (see page 406) combo box, Query Builder, etc., they will return the full object name. To avoid catalog and / or schema names in the full object name, use the MetaDefCatalog and MetaDefSchema (see page 188) connection definition parameters. For example:

```
[MSSQL_Demo]
DriverID=MSSQL
...
MetaDefCatalog=Northwind
MetaDefSchema=dbo
```

When a short object name is specified to StoredProcName (see page 406), TableName (see page 559), etc, they will be

expanded into the full object names, using the current catalog and / or schema names. To override or avoid usage of current catalog and / or schema names, use the MetaCurCatalog and MetaCurSchema (see page 188) connection definition parameters. For example:

```
[MSSQL_Demo]
DriverID=MSSQL
...
MetaCurCatalog=*
MetaCurSchema=*
```

Additionally, at design-time you may force FireDAC to return object names without catalog and/or schema. For that in IDE menu item FireDAC -> Preferences ... unmark "Use full object names".

See Also

Executing Stored Procedure (see page 71), Browsing Table (see page 73), Common connection parameters (see page 188)

1.9 Working with DBMS

A set of articles describing how to work with a specific database from an FireDAC application.

1.9.1 Using SQLite with FireDAC

This article describes the aspects of developing SQLite database applications using FireDAC for Delphi. FireDAC is the leading DBMS access engine for Delphi providing powerful API to unique SQLite features. The SQLite is an embedded SQL database engine, most widely deployed in the world.

Description

1. General

This reference article has few sections:

- **Introduction to SQLite**: reviews SQLite features, missed features, possible applications, and applications not for SQLite.
- **Using SQLite Database**: explains how to create, connect to and manage SQLite database in Delphi application.
- **SQLite Encrypted Database**: the database encryption is one of the important SQLite features. The topic explains how it works and how to control it.
- **SQLite Data Types**: SQLite has an unique data type system. Without understanding of how it works, will be hard to effectively store and retrieve the data in Delphi applications.
- **SQLite SQL Commands**: main aspects of SQLite SQL dialect for the Delphi application developers.
- **SQLite Transactions, Locking and Cursors**: explains how to work with transactions in SQLite environment.
- **Extending SQLite Engine**: as an embedded DBMS, SQLite engine may be extended by the Delphi application code.
- **Advanced SQLite Techniques**: finally we want to introduce some advanced SQLite concepts, like updates logging and SQL authorization.

This article requires the knowledge of the FireDAC basics and of the main library API's. For the beginners we will suggest to start from the Getting Started (see page 2) article and look into the FireDAC\Samples\Getting Started\SQLite demo.

2. Introduction to SQLite

2.1 SQLite Database



The SQLite is an embedded SQL database engine, developed by SQLite Consortium. It is a DBMS most widely deployed in the world with rough estimate of 500M installations. You can find it on all iOS and Android mobile devices, on Mac OS and Linux desktop's. It is used by Firefox, Skype and McAfee anti-virus.

2.2 SQLite features

The source lists:

- **Transactions** are atomic, consistent, isolated, and durable (ACID) even after system crashes and power failures.
- **Zero-configuration** - no setup or administration needed.
- Implements **most of SQL92**. Supports table triggers and views.
- A complete database is stored in a **single cross-platform disk file**.
- Supports **terabyte-sized databases** and gigabyte-sized strings and blobs.
- **Faster** than popular client/server database engines for most common operations.
- **Self-contained**: no external dependencies.
- **Cross-platform**: Windows, Linux, Mac OS X, iOS and Android are supported out of the box.
- Sources are in the public domain. Use for any purpose is **free**.
- **Very powerful API**, allowing to extend engine in practically all areas.
- SQLite achieves one of the **best data access performances** among other embedded, file-server and client-server database engines used by the Delphi applications. We are aware of many successful applications with multi gigabyte databases. For example Silwood Technology Saphir is built with SQLite, Delphi and FireDAC.

2.3 Missed SQLite features

According to our experience, the developers often trying to find the following features, not presented in SQLite:

- **Stored procedures**. FireDAC offers custom functions API.
- Reach set of **build-in functions**. FireDAC pre installs about 50 standard functions.
- **Security system**, including an user and a right concepts. FireDAC offers password protected encrypted database, special callbacks to filter user actions.
- **Collations** (only ASCII and binary). FireDAC allows to define custom collations.

2.4 SQLite applications

The source lists:

- **Application File Format**. SQLite has been used with great success as the on-disk file format for desktop applications.
- **Embedded devices and applications**. Because SQLite database requires little or no administration, SQLite is a good choice for devices or services that must work unattended and without human support.
- **Websites**. SQLite usually will work great as the database engine for low to medium traffic websites (which is to say, 99.9% of all websites).
- **Replacement for ad hoc disk files**. SQLite works particularly well as a replacement for these ad hoc data files.
- **Internal or temporary databases**. For programs sifting and sorting the data in diverse ways, may be more simple to load data into in-memory SQLite database and use full scale SQL command.
- **Stand-in for an enterprise database** during demos or testing.

- **Database Pedagogy**.

2.5 Applications NOT for SQLite

The same source lists:

- **High Concurrency**. SQLite uses reader/writer locks on the entire database file. That means if any process is reading from any part of the database, all other processes are prevented from writing any other part of the database. Similarly, if any one process is writing to the database, all other processes are prevented from reading any other part of the database.
- **Client/Server Applications**. If you have many client programs accessing a common database over a network, you should consider using a client/server database engine instead of SQLite. SQLite will work over a network file system, but because of the latency associated with most network file systems, performance will not be great.
- **Very large datasets** (N tb).

3. Using SQLite Database

3.1 Connecting to SQLite database from Delphi application

To link FireDAC SQLite driver into Delphi application add TADPhysSQLiteDriverLink (see page 805) to a form or data module. To connect to SQLite database, specify SQLite driver parameters (see page 221), at least:

```
DriverID=SQLite
Database=<path to SQLite database>
```

By default, all SQLite driver settings are set for the high-performance single connection access to a database in a stable environment. The PRAGMA command allows to configure SQLite. Many FireDAC SQLite driver parameters correspond to the pragma's. Additionally **SQLiteAdvanced** allows to specify multiple pragma's separated by ';' as a single connection parameter.

The additional SQLite use cases are:

N	Application specifics	Description
1	Reading large DB.	Set **CacheSize** to a higher number of pages, which will be used to cache the DB data. The total cache size will be CacheSize * <db page size>.
2	Exclusive updating of DB.	Consider to set **JournalMode** to WAL (more).
3	Long updating transactions.	Set **CacheSize** to a higher number of pages, that will allow to run transactions with many updates without overloading the memory cache with dirty pages.
4	A few concurrent updating processes.	Set **LockingMode** to Normal to enable shared DB access. Set **Synchronous** to Normal or Full to make committed data visible to others. Set **UpdateOptions.LockWait** to True to enable waiting for locks. Increase **BusyTimeout** to raise a lock waiting time. Consider to set **JournalMode** to WAL.
5	A few concurrent updating threads.	See (4). Also set **SharedCache** to False to minimize locking conflicts.
6	A few concurrent updating transactions.	See (4) or (5). Also set **TxOptions.Isolation** to xiSnapshot or xiSerializible to avoid possible transaction deadlocks.
7	High safety.	Set **Synchronous** to Full to protect DB from the committed data losts. Also see (3). Consider to encrypt database to provide integrity.
8	High confidentiality.	Encrypt database to provide confidentiality and integrity.

9	Development time.	Set **LockingMode** to Normal to enable simultaneous use of SQLite DB in IDE and a debugged program.

3.2 Creating SQLite database from Delphi application

SQLite database will be created at a connection establishment if it does not exist. For explicit control the Delphi application may specify:

```
OpenMode=CreateUTF8 | CreateUTF16 | ReadWrite | ReadOnly
```

First two values allow the creation and differ by encoding that will be used for the new database. Also we recommend to set page_size to 4096 or more for the database with multi row tables. That may be done by specifying at creation time:

```
SQLiteAdvanced=page_size=4096
```

Consider to specify parameters using SQLiteAdvanced:

- auto_vacuum
- default_cache_size

Note, after the database file is created it has the zero size. As result the database encoding, page size and other persistent parameters are not recorded in the database. To make such parameters persistent, the application should create at least one table.

3.3 Using SQLite in-memory database in Delphi application

Next SQLite unique feature is the ability to work with the pure in-memory databases. That means, no files will be created to store database objects and all will be kept in the memory. So, more security, more performance, less requirements to the environment rights for a Delphi application.

To create and open a SQLite in-memory database use parameters:

```
DriverID=SQLite
Database=:memory:
```

Or just leave Database parameter empty:

```
DriverID=SQLite
```

One FireDAC customer had a SQLite database on a shared network resource. The database is a read/only products catalogue with many stored product attributes. To radically improve performance, the customer used TADSQLiteBackup (see page 805) to move the database in whole into the in-memory database. The sample code:

```
ADConnection1.DriverName := 'SQLite';
ADConnection1.Open;

ADSQLiteBackup1.Database := '\\srv\db\data.sdb';
ADSQLiteBackup1.DestDatabaseObj := ADConnection1.CliObj;
ADSQLiteBackup1.DestMode := smCreate;
ADSQLiteBackup1.Backup;
```

3.4 Working with Unicode and SQLite database

FireDAC supports Unicode (see page 51) Delphi's (starting with Delphi 2009) in full. For SQLite that means:

- FireDAC automatically setups SQLite database to exchange all metadata in UTF-16 encoding, when Delphi 2009 or higher is used. In Delphi 2007 or less the metadata is ANSI encoded.
- The data will be defined and exchanged as is described in the chapter "Mapping SQLite to FireDAC Data Types".

3.5 Using multiple SQLite databases in Delphi application

SQLite allows to use multiple databases in a single connection. A DB specified by the Database parameter is the main database. To attach additional databases, Delphi application should use ATTACH command. For example:

```
ADConnection1.ExecSQL('ATTACH ''c:\hr.sdb'' AS hr');
ADConnection1.ExecSQL('ATTACH ''c:\cust.sdb'' AS cust');
ADQuery1.Open('select * from "Orders" o ' +
  'left join hr."Employees" e on o.EmployeeID = e.EmployeeID ' +
  'left join cust."Customers" c on o.CustomerID = c.CustomerID');
```

Note, that FireDAC interprets a database name as a catalog name.

3.6 Manage SQLite database from Delphi application

A good Delphi (and not only) SQLite database application must be aware of the following facts:

- SQLite database may become fragmented and non-optimal after many "hard" record updates or deletes. The TADSQLiteValidate.Sweep (see page 823) method call optimizes the database. This method corresponds to the VACUUM command and PRAGMA auto_vacuum. And the example:

```
ADSQLiteValidate1.Database := 'c:\db.sdb';
ADSQLiteValidate1.Sweep;
```

- SQLite query optimizer builds a better query execution plan when it has an up-to-date database statistic. SQLite does not automatically update the statistic. The TADSQLiteValidate.Analyze (see page 823) method call collects it. This method utilizes the ANALYZE command. An application may collect statistic for the full database:

```
ADSQLiteValidate1.Database := 'c:\db.sdb';
ADSQLiteValidate1.Analyze;
```

- SQLite database may become corrupted or malformed. To verify it integrity use the TADSQLiteValidate.CheckOnly (see page 823) method. Note, to repair a broken SQLite database, Delphi application needs to restore it from a backup. CheckOnly method uses OnProgress (see page 824) event handler to notify about issues. This method performs PRAGMA integrity_check command.

```
procedure TForm1.ADSQLiteValidate1Progress(ASender: TADPhysDriverService; const AMessage:
String);
begin
  Memo1.Lines.Add(AMessage);
end;

ADSQLiteValidate1.Database := 'c:\db.sdb';
ADSQLiteValidate1.OnProgress := Form1Progress;
ADSQLiteValidate1.CheckOnly;
```

- SQLite database must be backed up regularly to preserve the data loss. The TADSQLiteBackup (see page 805) component performs the database backup copy. This is how the simplest backup code looks:

```
ADSQLiteBackup1.Database := 'c:\db.sdb';
ADSQLiteBackup1.DestDatabase := 'c:\db.backup';
ADSQLiteBackup1.DestMode := smCreate;
ADSQLiteBackup1.Backup;
```

4. SQLite Encrypted Database

4.1 Approach

One of the distinctive SQLite features is the high-speed strong database encryption. It allows to make database file content confidential and enforce integrity control on the database file.

The encrypted database format is not compatible with other similar SQLite encryption extensions. That means, you cannot use an encrypted database, encrypted with non-FireDAC libraries. If you need that, then you have to decrypt a database with original tool and encrypt it with FireDAC.

The encryption is provided through the officially supported SQLite approach - custom codec code and compilation with SQLITE_HAS_CODEC defined. All encryption routines are implemented on Delphi and embedded into sqlite3 code. As result, the encryption is correctly handled for:

- ATTACH;
- VACUUM;
- database recovery;
- database backup.

4.2 Encryption modes

Mode	Description	Usage
AES-NNN	The aes-NNN algorithms are generic compositions of AES-CTR and AES-CBC-MAC. This composition **guarantees both Confidentiality and Integrity**, meaning that only entities with access to the correct password will be able to read and modify the pages of the encrypted database. These algorithms will add a linear overhead of 32 bytes per page to your encrypted database. This algorithm will help you detect most malicious attempts to inject data into the database, but it will not prevent such attempts and it will not help you undo such modifications. It is essentially a complement to frequent backups, but it is much better than most other database encryption schemes at alerting you when you have been subject to an attack and it is time to restore your database from the backup. Please note that the aes-NNN algorithm, by itself, neither will detect deletion of entire pages at the end of the database (but it will detect deletions in the middle of the database), nor will it detect attacks that consist in reverting the database to an older version encrypted using the same password.	The AES-NNN provides the top strong Confidentiality and Integrity. But at a price of some performance reduction, noticeable across other encryption modes.
AES-CTR-NNN	The aes-ctr-NNN algorithms are AES-CTR only. It will **not detect modifications to the database**, but it will **provide Confidentiality** against passive attacks. That is, as long as the attacker does not have access to your password, and does not attempt to modify the database to see how your application reacts to the modifications, your data will remain as secret as your application allows. Needless to say, the algorithms will only protect your database file against attackers that are not able to tap into your FireDAC application using e.g. a debugger and extract the password that way. In a similar way, if you store your password in a configuration file or as a constant in the software itself, it will be trivial for any not even moderately skilled attacker to find it and compromise your security.	The AES-CTR-NNN provides the top strong Confidentiality, but not an Integrity. For that you will get a better performance.
AEC-ECB-NNN	The aes-ecb-NNN algorithms are AES-ECB only. It will **not detect modifications to the database**, and it will **not provide strong Confidentiality** against passive attacks, in contrast to AES-NNN and AES-CTR-NNN.	The AES-ECB-NNN provides comparably weak Confidentiality, and no Integrity. But it has the best performance across other encryption modes.

There NNN is the key size, which may be 128, 192 or 256 bits.

4.3 Setting up Encryption

The encryption may be controlled:

- through the connection definition parameters **Encrypt**, **NewPassword** and **Password**;
- through the TADSQLiteSecurity (see page 817) service component.

The password connection definition parameters may have a form:

```
[aes-128 | aes-192 | aes-256 | aes-ctr-128 | aes-ctr-192 | aes-ctr-256 |
 aes-ecb-128 | aes-ecb-192 | aes-ecb-256:] password
```

There "aes-XXX-NNN:" is an optional prefix, controlling the cipher algorithm to use. If it is not specified, then will be used:

- algorithm, specified by **Encrypt** parameter;
- **aes-256** if nothing is specified.

FireDAC supports encryption operations:

Operation	Using parameters	Using TADSQLiteSecurity
Open encrypted database	Password=xxxx	---
Encrypt unencrypted database	NewPassword=xxxx	ADSQLiteSecurity1.Database := '...'; ADSQLiteSecurity1.Password := 'xxxx'; ADSQLiteSecurity1.SetPassword;
Change encrypted database password	Password=xxxx NewPassword=yyyy	ADSQLiteSecurity1.Database := '...'; ADSQLiteSecurity1.Password := 'xxxx'; ADSQLiteSecurity1.ToPassword := 'yyyy'; ADSQLiteSecurity1.ChangePassword;
Decrypt encrypted database	Password=xxxx NewPassword=	ADSQLiteSecurity1.Database := '...'; ADSQLiteSecurity1.Password := 'xxxx'; ADSQLiteSecurity1.RemovePassword;
Verify encryption database status	---	ADSQLiteSecurity1.Database := '...'; ADSQLiteSecurity1.Password := 'xxxx'; ShowMessage(ADSQLiteSecurity1.CheckEncryption);

4.4 SQL Extension

The ATTACH command got an extension. The full syntax of the ATTACH now is:

```
ATTACH [DATABASE] 'filename' [AS name] [KEY 'password']
```

When KEY is omitted, then the password value will be inherited from the main database. To specify an empty password to attach an unencrypted database, use something like that:

```
ATTACH 'D:\tmp\test.db' AS tst KEY ''
```

5. SQLite Data Types

5.1 Mapping SQLite to FireDAC Data Types

SQLite has "type less" data type system. Practically that means, that you can use any identifier as a column data type name. For example "DELPHI" will work too and will correspond to the string data type. To make SQLite approach more compatible with other DBMS's and Delphi, and more comfortable for the Delphi application developers, FireDAC recognizes data type

names as described in the table:

Type name	Description
ROWID | _ROWID_ | OID	dtInt64, Attrs = [caSearchable, caAllowNull, caROWID]
BIT | BOOL | BOOLEAN | LOGICAL | YESNO	dtBoolean
TINYINT | SHORTINT | INT8 [UNSIGNED]	dtSByte / dtByte
BYTE | UINT8	dtByte
SMALLINT | INT16 [UNSIGNED]	dtInt16 / dtUInt16
WORD | UINT16 | YEAR	dtUInt16
MEDIUMINT | INTEGER | INT | INT32 [UNSIGNED]	dtInt32 / dtUInt32
LONGWORD | UINT32	dtUInt32
BIGINT | INT64 | COUNTER | AUTOINCREMENT | IDENTITY [UNSIGNED]	dtInt64 / dtUInt64
LONGLONGWORD | UINT64	dtUInt64
REAL | FLOAT | DOUBLE	dtDouble
SINGLE [PRECISION] [(P, S)]	dtSingle / dtBCD / dtFmtBCD
DECIMAL | DEC | NUMERIC | NUMBER [UNSIGNED] [(P, S)]	dtSByte / dtInt16 / dtInt32 / dtInt64 dtByte / dtUInt16 / dtUInt32 / dtUInt64 dtBCD / dtFmtBCD
MONEY | SMALLMONEY | CURRENCY | FINANCIAL [(P, S)]	dtCurrency
DATE | SMALLDATE	dtDate
DATETIME | SMALLDATETIME	dtDateTime
TIMESTAMP	dtDateTimeStamp
TIME	dtTime
CHAR | CHARACTER [(L)]	dtAnsiString, Len = L, Attrs = [caFixedLen]
VARCHAR | VARCHAR2 | TYNITEXT | CHARACTER VARYING | CHAR VARYING [(L)]	dtAnsiString, Len = L
NCHAR | NATIONAL CHAR | NATIONAL CHARACTER [(L)]	dtWideString, Len = L, Attrs = [caFixedLen]
NVARCHAR | NVARCHAR2 | NATIONAL CHAR VARYING | STRING [(L)]	dtWideString, Len = L
RAW | TYNIBLOB | VARBINARY | BINARY | BINARY VARYING [(L)]	dtByteString, Len = L
BLOB | MEDIUMBLOB | IMAGE | LONGBLOB | LONG BINARY | LONG RAW | LONGVARBINARY | GENERAL | OLEOBJECT | TINYBLOB	dtBlob
TEXT | MEDIUMTEXT | LONGTEXT | CLOB | MEMO | NOTE | LONG | LONG TEXT | LONGCHAR | LONGVARCHAR | TINYTEXT	dtMemo
NTEXT | WTEXT | NCLOB | NMEMO | LONG NTEXT | LONG WTEXT | NATIONAL TEXT | LONGWCHAR | LONGWVARCHAR | HTML	dtWideMemo
XMLDATA | XMLTYPE | XML	dtXML
GUID | UNIQUEIDENTIFIER	dtGUID
Other data types	dtWideString

Note, with SQLite the FormatOptions.StrsTrim (see page 869) works for all string data types.

5.2 Special SQLite Data Types

To add an auto incrementing column to a table, define a column as INTEGER PRIMARY KEY AUTOINCREMENT. This type is mapped to dtInt32, Attrs = [caAutoInc]. For more details about handling auto-incrementing columns read "Auto-Incremental Fields (see page 118)".

The columns with ROWID, _ROWID_ or OID type names are considered as identifying row columns. This types are mapped to dtInt64, Attrs = [caSearchable, caAllowNull, caROWID]. For more details about handling row identifying columns read "Unique Identifying Fields (see page 117)". SQLite ROWID is the fastest way to get an access to a specific row:

```
SELECT * FROM Orders WHERE ROWID = :RID
```

5.3 Adjusting FireDAC Mapping

Some SQLite driver parameters (see page 221) allow for Delphi application to adjust the data representation:

Parameter	Description
StringFormat = Choose | Unicode	When Unicode, then all dtAnsiString and dtMemo will be represented to a client as dtWideString and dtWideMemo.
GUIDFormat = String | Binary	When Binary, then dtGUID will be stored in a database as a TGUID binary value. When String, then as a string in {xxxxxxx} format. Binary requires less space in DB, String is more readable.
DateTimeFormat = String | Binary	When Binary, then dtDate, dtTime, dtDateTime will be stored in a database as a double value in Julian date format. When String, then as a character string in 'yyyy-mm-dd hh24:mi:ss' format. Binary requires less space in DB, String is more readable.

Note, that changing GUIDFormat or DateTimeFormat, when the database is not empty, may lead to errors, as FireDAC may fail to read and parse the stored values.

For an expression in a SELECT list the SQLite avoids type name information. When result set is not empty, FireDAC will use the value data types from the first record. When empty, FireDAC will describe those columns as dtWideString. To explicitly specify the column data type, append ::<type name> to the column alias:

```
SELECT count(*) as "cnt::INT" FROM mytab
```

When Delphi application requires SQLite native data type representation, then use FireDAC mapping rules. For example, map TEXT columns to dtAnsiString and INT columns to dtInt64:

```
with ADQuery1.FormatOptions do begin
  OwnMapRules := True;
  with MapRules do begin
    SourceDataType := dtMemo;
    TargetDataType := dtAnsiString;
  end;
  with MapRules do begin
    SourceDataType := dtInt32;
    TargetDataType := dtInt64;
  end;
end;
```

5.4. High-Precision Numbers

Due to SQLite type system, long numbers (>= 20 digits) will be recognized as having REAL type affinity. So, a value will be rounded to a REAL (double Delphi type) type with 15 digits precision. To resolve this issue an application should use TEXT or similar data type and a mapping rule casting dtAnsiString to dtFmtBCD or similar data type. Note, this is not a FireDAC issue and at moment cannot be resolved differently.

6. SQLite SQL Commands

6.1 SQL Dialect

Although SQLite closely follows to the ANSI SQL 92, some features and commands are not supported, and some powerful are added. You can find more info about SQLite SQL dialect at:

- SQL As Understood By SQLite.
- SQLite / ANSI SQL Commands and Features Not Supported.

6.2 SQLite SQL Command Batches

FireDAC SQLite driver supports the SQL command batches (see page 81). SQL commands must be separated by ';'. SQLite allows to mix any commands in a batch, including DDL and DML. For example:

```
with ADQuery1.SQL do begin
  SQL.Clear;
  SQL.Add('create table dbms (id integer, name varchar(20));');
  SQL.Add('insert into tab values (1, ''sqlite'');');
  SQL.Add('insert into tab values (2, ''mysql'');');
  SQL.Add('insert into tab values (3, ''firebird'');');
  SQL.Add('create table langs (id integer, name varchar(20));');
  SQL.Add('insert into tab values (1, ''delphi'');');
  SQL.Add('insert into tab values (2, ''c'');');
  SQL.Add('insert into tab values (3, ''c++'');');
  SQL.Add('select * from dbms;');
  SQL.Add('select * from langs;');
end;
ADQuery1.Open;
// process here the DBMS list
ADQuery1.NextRecordSet;
// process here the programming languages list
```

6.3 SQL script dialect

FireDAC TADScript (see page 685) does not support SQLite syntax, where script control commands are starting from '.'.

6.4 Array DML

Starting from v 3.7.11 the SQLite supports INSERT command with multiple VALUES. FireDAC uses this feature to implement Array DML (see page 82), when Params.BindMode = pbByNumber. Otherwise FireDAC will emulate Array DML. For example:

```
// here ADQuery1.Params collection is filled by 3 parameters
ADQuery1.SQL.Text := 'insert into MyTab values (:p1, :p2, :p3)';
// set "by number" parameter binding mode
ADQuery1.Params.BindMode := pbByNumber;
ADQuery1.Params.ArraySize := 100;
for i := 0 to ADQuery1.Params.ArraySize - 1 do begin
  ADQuery1.Params[0].AsIntegers[i] := i;
  ADQuery1.Params[1].AsStrings[i] := 'qwe';
  ADQuery1.Params[2].Clear(i);
end;
ADQuery1.Execute(ADQuery1.Params.ArraySize);
```

7. SQLite Transactions, Locking, Threads and Cursors

7.1 Locking and concurrent updates

Read the following original SQLite articles:

- BEGIN TRANSACTION
- SQLite Shared-Cache Mode
- Corruption Following Busy

SQLite, as a file-server DBMS, likes to lock the database tables at updates. The following settings affect the concurrent access:

- when multiple threads are updating the same database, set **SharedCache** connection parameter to False. That will help to avoid some possible deadlocks.
- when multiple processes or threads are updating the same database tables, set set **LockingMode** to Normal to enable concurrent access to the tables. Also set **Synchronous** connection parameter to Full or Normal. So, SQLite will update a database file right after the transaction is finished. And other connections will see the updates on the predictable basis.
- to avoid locking conflicts between connections, set UpdateOptions.LockWait (see page 902) to True and set **BusyTimeout** to a higher value.
- to avoid locking conflicts between long running updating transactions, set TADConnection.TxOptions.Isolation (see page 894) to xiSnapshot or xiSerializible.

7.2 Transactions and isolation modes

SQLite supports transactions (see page 41) in full, including check points. Further is a list of isolation modes supported by SQLite:

Mode	Corresponds to
xiDirtyRead	PRAGMA read_uncommitted = 1
xiReadCommitted	BEGIN TRANSACTION DEFERRED
xiRepeatableRead	The same as xiReadCommitted.
xiSnapshot	BEGIN TRANSACTION IMMEDIATE
xiSerializible	BEGIN TRANSACTION EXCLUSIVE

7.3 Transactions and DML commands

Surrounding writing commands into a transaction may radically improve SQLite performance. That is notable at large data modifications. The same is applicable to FireDAC Array DML (see page 82) feature. So, surround the data modification code into a transaction to get the best performance:

```
ADConnection1.StartTransaction;
try
  ADQuery1.SQL.Text := 'insert into tab values (:id, :name)';
  ADQuery1.Params.ArraySize := 10;
  for i := 0 to ADQuery1.Params.ArraySize - 1 do begin
    ADQuery1.Params[0].AsIntegers[i] := i;
    ADQuery1.Params[0].AsStrings[i] := 'name' + IntTostr(i);
  end;
  ADQuery1.Execute(ADQuery1.Params.ArraySize, 0);
  ADConnection1.Commit;
except
  ADConnection1.Rollback;
  raise;
end;
```

7.4 Transactions and cursors

SQLite does not allow to rollback a transaction, when there are commands with not yet fetched result sets. To workaround that, FireDAC will fetch all remaining records from a result set on a Rollback method call. See the FetchOptions.AutoFetchAll (see page 852).

8. Extending SQLite Engine

8.1 Custom Functions

SQLite does not support stored procedure or function concept, as it allows to use the host language environment to extend the engine functionality. SQLite allows to register host language functions in SQLite engine and use them in the SQL commands. FireDAC simplifies this by introducing the TADSQLiteFunction (see page 814) component.

To build a function, the developer has to set FunctionName (see page 815), ArgumentsCount (see page 815) and create the OnCalculate (see page 816) event handler. Setting Active (see page 815) to True will register custom function at SQLite engine. For example:

```
procedure TForm1.ADSQLiteFunction1Calculate(AFunc: TSQLiteFunction;
  AInputs: TSQLiteInputs; AOutput: TSQLiteOutput; var AUserData: TObject);
begin
  AOutput.AsInteger := AInputs[0].AsInteger * AInputs[1].AsInteger;
end;

ADSQLiteFunction1.DriverLink := ADPhysSQLiteDriverLink1;
ADSQLiteFunction1.FunctionName := 'XmY';
ADSQLiteFunction1.ArgumentsCount := 2;
ADSQLiteFunction1.OnCalculate := ADSQLiteFunction1Calculate;
ADSQLiteFunction1.Active := True;
```

This function usage:

```
ADQuery1.Open('select RegionID, XmY(RegionID, 10) from "Region"');
```

A function may call the FireDAC methods to query a database. To create a custom function with the default or different number of arguments you will need to specify the same FunctionName (see page 815) and different number of arguments. That will register an overloaded function inside the SQLite engine.

The above and other function samples you can find in FireDAC\Samples\DBMS Specific\SQLite\UserFunc folder.

FireDAC implements and installs to a SQLite connection about 50 functions, which are standard de-facto for many DBMS and are implemented by the FireDAC local expressions engine. Note, when you are creating SQLite connection at run-time, you should include uADStanExprFuncs unit into "uses" clause, otherwise you will get an exception:

```
[FireDAC][Phys][SQLite] ERROR: no such function: UCASE.
```

To make custom functions accessible at design-time, create a custom design-time package with a data module, drop the components on this module and setup properly. Create the module in the module unit initialization section, and destroy in the finalization section. Then install your package into Delphi IDE.

And the Ron Grove movie:

8.2 Custom Collations

SQLite stores and handles all character data either in UTF8 or UTF16, depending on the **OpenMode** connection parameter. When SQLite needs to compare or sort a character data, it must know what rules to use for that. The rules are known as a collation.

SQLite has few build-in collations. None of them produces a correct sorting for German, Cyrillic, Arabian, etc phrases. You have to use TADSQLiteCollation (see page 811) component to build your own collation. Set CollationName (see page 813), Flags (see page 813), Locale (see page 813), then set Active (see page 812) to True to register the collation with SQLite engine. For example:

```
ADSQLiteCollation1.DriverLink := ADPhysSQLiteDriverLink1;
ADSQLiteCollation1.CollationName := 'UTF16NoCase';
ADSQLiteCollation1.Flags := [sfIgnoreCase];
ADSQLiteCollation1.Active := True;
```

Above component setup with default CollationKind (see page 812)=scCompareString implements a standard Win32 Unicode collation. The application may implement custom collations using CollationKind=scCustomUTF16 or scCustomUTF8 and implementing OnCompare (see page 813) event handler. And how to use this collation:

```
SELECT * FROM "Employees" ORDER BY LastName COLLATE UTF16NoCase
```

To specify default collation for a column you can do:

```
CREATE TABLE IF NOT EXISTS test_col (f1 VARCHAR(10) COLLATE UTF16NoCase)
```

Note, that there is no ability to specify default collation for a connection, a database or a table. The above collation samples you can find in FireDAC\Samples\DBMS Specific\SQLite\UserCollation folder.

If you do not use custom collations, then by default SQLite will use a binary sorting order. For TADTable Live Data Window (see page 73) mode is important to have the same client side and database sorting orders. To enable client side binary sorting order set FormatOptions.SortLocale (see page 867) to 0.

8.3 The Database Events

FireDAC supports notification (see page 75) of an Delphi application from a SQLite database trigger about some events, like a data change. For that FireDAC uses similar to Firebird approach and registers POST_EVENT custom function. To call it from a trigger do:

```
CREATE TRIGGER update_orders UPDATE ON "Orders"
BEGIN
  SELECT POST_EVENT('Orders');
END;
```

To receive an event notification the Delphi application uses TADEventAlerter (see page 422) component. For example:

```
ADEventAlerter1.Names.Text := 'Orders';
ADEventAlerter1.Options.Synchronize := True;
ADEventAlerter1.OnAlter := DoAlert;
ADEventAlerter1.Active := True;

procedure TForm1.DoAlert(ASender: TADCustomEventAlerter;
  const AEventName: String; const AArgument: Variant);
begin
  if CompareText(AEventName, 'Orders') = 0 then
    qryOrders.Refresh;
end;
```

8.4 Custom Data Sources

Local SQL (see page 96) engine allows to use TDataSet descendants in your SQL queries. FireDAC uses SQLite Virtual Table API to implement Local SQL.

9. Advanced SQLite Techniques

9.1 Hooking Database Updates

SQLite provides an unique API allowing to monitor all updates to a database. This feature may be used, for example, to log all updates to a DB. To work with this API, a Delphi application should set OnUpdate event handler of the TSQLiteDatabase object, which is a database connection wrapping object. Hook this event after a database connection is opened. For example:

```
procedure TForm1.DoUpdate(ADB: TSQLiteDatabase; AOper: Integer; const ADatabase, ATable:
String; ARowid: sqlite3_int64);
begin
  Memo1.Lines.Add(Format('%d - %s - %s - %u', [AOper, ADatabase, ATable, ARowid]));
end;
```

```
ADConnection1.Connected := True;
TSQLiteDatabase(ADConnection1.ConnectionIntf.CliObj).OnUpdate := DoUpdate;
```

This sample you can find in FireDAC\Samples\DBMS Specific\SQLite\OnUpdate folder.

9.2 Controlling Database Access Rights

The SQLite is an embedded DBMS. That implies that it is a single user DBMS and does not need such concepts as a user, access rights, etc. Still some application may benefit from an access right control, for example:

- An application may restrict rights depending on a end-user license. Demo license - less possibilities, full license - all possibilities.
- Multi-tier data access frameworks may use their own user concept and to control data access rights using some generic approach.

And again, SQLite provides an unique feature allowing to authorize or not SQL commands. To work with this API, a Delphi application should set OnAutorize event handler of the TSQLiteDatabase object, which is a database connection wrapping object. Hook this event after a database connection is opened. For example:

```
procedure TForm1.DoAuthorize(ADB: TSQLiteDatabase; ACode: Integer; const AArg1, AArg2,
AArg3, AArg4: String; var AResult: Integer);
begin
  Memo1.Lines.Add(Format('%d - %s - %s - %s - %s', [ACode, AArg1, AArg2, AArg3, AArg4]));

  // Deny any delete operation
  if ACode = SQLITE_DELETE then
    AResult := SQLITE_DENY
  else
    AResult := SQLITE_OK;
end;

ADConnection1.Connected := True;
TSQLiteDatabase(ADConnection1.ConnectionIntf.CliObj).OnAutorize := DoAuthorize;
```

This sample you can find in FireDAC\Samples\DBMS Specific\SQLite\OnAuthorize folder.

9.3 Using SQLite low-level API

When you need to get the maximum SQLite data access performance, then you should consider to use FireDAC SQLite API wrapping classes. This is a low-level thin object oriented API, which is used by the FireDAC SQLite driver. This API is not documented and not officially supported.

The following example shows how to control transactions and fetch records using a parameterized SELECT command:

```
uses
  uADPhysSQLiteWrapper;

procedure TForm1.FormCreate(Sender: TObject);
var
  oDB: TSQLiteDatabase;
  oTran: TSQLiteStatement;
  oStmt: TSQLiteStatement;
  i: Integer;
begin
  ADConnection1.Connected := True;
  oDB := TSQLiteDatabase(ADConnection1.CliObj);

  oTran := TSQLiteStatement.Create(oDB);
  try
    // start transaction
    oTran.Prepare('BEGIN');
    oTran.Execute;

    oStmt := TSQLiteStatement.Create(oDB);
    try
```

```
      // prepare statement
      oStmt.Prepare('select * from "Orders" where OrderID > :ID1 and OrderID < :ID2');

      // add bind variables (parameters)
      for i := 1 to oStmt.ParamDefsCount do
        TSQLiteBind.Create(oStmt.Params);

      // add column variables (fields)
      for i := 1 to oStmt.ColumnDefsCount do
        TSQLiteColumn.Create(oStmt.Columns).Index := i - 1;

      // set parameter values and execute
      oStmt.Params[0].AsInteger := 11000;
      oStmt.Params[1].AsInteger := 12000;
      oStmt.Execute;

      // fetch records and read columns
      while oStmt.Fetch do
        Memo1.Lines.Add(Format('OrderID: %d, CustomerID: %s',
          [oStmt.Columns[0].AsInteger, oStmt.Columns[1].AsString]));
    finally
      oStmt.Free;
    end;

    // commit transaction
    oTran.Unprepare;
    oTran.Prepare('COMMIT');
    oTran.Execute;
  finally
    oTran.Free;
  end;
end;
```

10. Summary

This article has provided the benefits of combining SQLite and FireDAC to get premier embedded DBMS solution. The articles describes all aspects of the FireDAC SQLite driver usage.

See Also

Connect to SQLite database (see page 221), uADPhysSQLite Namespace (see page 804)

1.9.2 Using Oracle with FireDAC

This article describes the aspects of developing Oracle database applications using FireDAC for Delphi. FireDAC is the leading DBMS access engine for Delphi providing powerful API to unique Oracle features. The Oracle is an enterprise level relational database server, #1 in worldwide RDBMS software revenue share.

Description

Note. This article is a work-in-progress and is not finished !

Still we decided to include it into documentation, as it already contains helpful information.

M. Oracle Advanced Data Types

Object Extensions

XML

BFILE

BLOB / CLOB / NCLOB

PL/SQL Collections, Var Arrays

PL/SQL Tables

FireDAC supports Oracle PL/SQL tables as the parameters of PL/SQL anonymous blocks, stored procedures and functions. Note, that a PL/SQL associate table differs from VARRAY and collections. FireDAC does not support the last two.

To setup a parameter as a PL/SQL table specify TADParam.ArrayType = atPLSQLTable. Set ArraySize to a maximum table size before ExecProc (see page 410) call. When INOUT, OUT parameter ArraySize is less than the number of table elements assigned at server side, then exception will be raised:

```
[FireDAC][Phys][Ora] ORA-06513: PL/SQL: index for PL/SQL table out of range for host
language array
ORA-06512: at line 2
```

To read or write parameter values use TADParam.AsXxxs[<index>] properties. There index is zero-based. Although at server side the index is 1 based. The empty elements have NULL value and may be tested using TADParam.IsNulls[<index>].

For example the server side script:

```
CREATE OR REPLACE PACKAGE ADQA_TestPack AS
  TYPE TVC2Tbl IS TABLE OF VARCHAR2(50) INDEX BY BINARY_INTEGER;
  PROCEDURE TestPLSQLArray(ATable in out TVC2Tbl);
END ADQA_testpack;
/

CREATE OR REPLACE PACKAGE BODY ADQA_TestPack AS
  PROCEDURE TestPLSQLArray(ATable IN OUT TVC2Tbl) IS
  BEGIN
    for i in ATable.First .. ATable.Last loop
      ATable(i) := '*' || ATable(i) || '*';
    end loop;
  END;
END ADQA_testpack;
/
```

And the client side code assigning memo lines to the table parameter, executing procedure, reading the table items and filling the memo:

```
var
  i: Integer;

ADStoredProc1.PackageName := 'ADQA_TESTPACK';
ADStoredProc1.StoredProcName := 'TESTPLSQLARRAY';
ADStoredProc1.Prepare;

ADStoredProc1.Params[0].ArraySize := Memo1.Lines.Count;
for i := 0 to Memo1.Lines.Count - 1 do
  ADStoredProc1.Params[0].AsStrings[i] := Memo1.Lines[i];
Memo1.Lines.Clear;

ADStoredProc1.ExecProc;
for i := 0 to ADStoredProc1.Params[0].ArraySize - 1 do
  Memo1.Lines.Add(ADStoredProc1.Params[0].AsStrings[i]);
```

Also see FireDAC\Samples\DBMS Specific\Oracle\PLSQLAssocArray demo.

PL/SQL Records

FireDAC supports Oracle PL/SQL records as the parameters of stored procedures and functions. Note, that a PL/SQL record differs from objects. FireDAC does not support the Oracle objects.

It is possible to correctly setup such parameters only when fiMeta in TADStoredProc.FetchOptions.Items (see page 855) and ParamBindMode (see page 534) = pbByName. Then FireDAC will expand record into a flat list of corresponding Params items. Where each item has a name <parameter name>$<record field name>.

For example the server side script:

```
CREATE OR REPLACE PACKAGE ClientPack IS
  TYPE t_clnt_data IS RECORD (
       client_id numeric,
       name varchar2(10),
       act boolean
  );
  PROCEDURE CltnProc(ARec IN t_clnt_data);
END ClientPack;
/
```

And the client side code assigning parameter values:

```
ADStoredProc1.PackageName := 'MYPACK';
ADStoredProc1.StoredProcName := 'CLNTPROC';
ADStoredProc1.Prepare;
ADStoredProc1.ParamByName('AREC$CLIENT_ID').Value := 100;
ADStoredProc1.ParamByName('AREC$NAME').Value := 'Client 1';
ADStoredProc1.ParamByName('AREC$ACT').Value := True;
ADStoredProc1.ExecProc;
```

Also see FireDAC\Samples\DBMS Specific\Oracle\PLSQLRecs demo.

N. Oracle Advanced Cursors

Working with Oracle REF CURSOR

FireDAC supports Oracle REF CURSOR, returned by Oracle PL/SQL anonymous blocks, stored procedures and functions. To open first cursor call Open method, to switch to subsequent cursors use NextRecordSet (see page 265) method. After switching to next cursor the previous one is not more accessible. See Command Batches (see page 81) chapter for details.

For example:

```
CREATE PROCEDURE TestRefCrs (ACrs1 IN OUT SYS_REFCURSOR, ACrs2 IN OUT SYS_REFCURSOR) AS
BEGIN
  OPEN ACrs1 FOR SELECT * FROM "Orders";
  OPEN ACrs2 FOR SELECT * FROM "Order Details";
END;
```

Using TADStoredProc (see page 514):

```
ADStoredProc1.FetchOptions.AutoClose := False;
ADStoredProc1.StoredProcName := 'TESTREFCRS';
ADStoredProc1.Open;
// work with "Orders" table data
ADStoredProc1.NextRecordSet;
// work with "Order Details" table data
ADStoredProc1.Close;
```

Using TADQuery (see page 476):

```
ADQuery1.FetchOptions.AutoClose := False;
ADQuery1.Open('BEGIN TestRefCrs(:p1, :p2); END;');
// work with "Orders" table data
ADQuery1.NextRecordSet;
// work with "Order Details" table data
ADQuery1.Close;
```

Note, if REF CURSOR is opened using dynamic SQL command text, then before subsequent Open calls, you should call Disconnect (see page 264) method. This is because FireDAC keeps dataset prepared and expects the same cursor structure as it was on first Open call. For example:

```
CREATE PROCEDURE TestDynCrs (ASQL IN VARCHAR2, ACrs OUT SYS_REFCURSOR) AS
BEGIN
  OPEN ACrs FOR ASQL;
END;
```

Using TADQuery (see page 476):

```
ADQuery1.FetchOptions.AutoClose := False;
ADQuery1.SQL.Text := 'BEGIN TestDynCrs(:p1, :p2); END;';

ADQuery1.Params[0].AsString := 'SELECT * FROM "Orders"';
ADQuery1.Open;
// work with "Orders" table data
ADQuery1.Close;

ADQuery1.Params[0].AsString := 'SELECT * FROM "Order Details"';
ADQuery1.Disconnect;
ADQuery1.Open;
// work with "Order Details" table data
ADQuery1.Close;
```

Working with Oracle Nested Cursors

FireDAC supports CURSOR type columns in a SELECT lists. There may be multiple CURSOR's in the list. But a CURSOR nested into a CURSOR is not supported. Such columns FireDAC defines as dtRowSetRef and creates for them TDataSetField. To process their row sets, application should use TADMemTable (see page 436), and set its DataSetField property to a TDataSetField reference.

While application navigates through the main dataset, the nested datasets will be automatically open and refreshed to provide the nested cursor records for a current record of the main dataset.

For example see FireDAC\Samples\DBMS Specific\Oracle\NestedCursors demo.

Updating Cursors Data

To refresh REF CURSOR or nested cursor records, application must reexecute main query.

To edit REF CURSOR or nested cursor record, application must override updates posting (see page 123).

O. Oracle Transactions, Locking, Threads and Cursors

P. Providing Server Feedback

Oracle application backend may send feedback to an application frontend using DBMS_OUTPUT package. FireDAC allows to automatically receive the DBMS_OUTPUT content.

Optionally set TADConnection.ResourceOptions.ServerOutputSize (see page 891) to the maximum buffer size. To enable DBMS_OUTPUT set TADConnection.ResourceOptions.ServerOutput (see page 890) to True. After a SQL command execution, application may process DBMS_OUTPUT feedback using TADConnection.Messages (see page 330). Note, DBMS_OUTPUT processing affects performance, so normally it must be disabled.

For example:

```
var
  i: Integer;
...
ADConnection1.ResourceOptions.ServerOutput := True;
with ADQuery1.SQL do begin
  Clear;
  Add('begin');
  Add('  dbms_output.put_line(''Hello World !'');');
  Add('end;');
end;
ADQuery1.ExecSQL;
if ADConnection1.Messages <> nil then begin
  Memo1.Lines.Clear;
  for i := 0 to ADConnection1.Messages.ErrorCount - 1 do
    Memo1.Lines.Add(ADConnection1.Messages[i].Message);
```

```
end;
```

See Also

Connect to Oracle Server (see page 215), uADPhysOracle Namespace (see page 800)

1.9.3 Using Data Abstract with FireDAC

This article describes the power of combining the FireDAC with the RemObjects Data Abstract. FireDAC is the leading Delphi DBMS access engine and RemObjects Data Abstract is a very powerful multi-tier framework.

Description

1. General

This tutorial has two main sections:

- **Benefits of combining FireDAC and Data Abstract**: while each of the products is a state of the art solution for the specific tasks, together they allow to solve these tasks more effectively.
- **How to use the FireDAC Data Abstract driver (DAD)**: explains how to configure FireDAC DAD, how to use Direct Mode and how to migrate to FireDAC DAD from other DADs.

2. Benefits of combining

Data Abstract



The Data Abstract relies on the Data Abstract drivers to access to a specific DBMS. The Data Abstract drivers are implemented by wrapping a specialized data access framework (DAF). While this approach works just great, it has a few consequences:

- Any underlying DAF has to be be highly effective, so the combined product is also highly effective. Otherwise a DAF is only a weak link in the chain.
- To access to the multiple DBMSs effectively you will need to use the few specialized DAFs. If they are commercial products, the combined tool price is raising with the addition of an access option to each new DBMS. Also, the developers will need more time to get an expertise in all these DAFs with different APIs.
- The usage of a single unified DAF, as ADO or DbExpress (DBX), is cost effective and simple for the developers, but it is not really a high speed solution (as we will show later). Also, these DAFs hide many useful DBMS features, making combined solution less flexible.

Together



FireDAC as a Data Abstract driver provides superb integration with Data Abstract on a very low level, giving extra speed in the interaction of the two. The FireDAC rounds off the RemObjects Software offering of the Data Abstract by adding an in-house solution for the "last mile", the data access layer which previously always depended on third parties. And it does that without giving up the flexibility. The combined product benefits are:

- **Performance**:
  - The Data Abstract, FireDAC and FireDAC DAD - all three - were optimized and adjusted to each other to get maximum performance.
  - The FireDAC DAD is the fastest Data Abstract driver, implementing Direct Mode, giving 20-40% performance gain,

while the CPU load is largely reduced.

- **Features**:
  - The FireDAC DAD implements all the optional Data Abstract driver interfaces.
  - The FireDAC DAD supports many DBMSs in an effective, unified and feature rich way.
- **Price**:
  - You need to buy only a single DAF, instead multiple ones.
- **Support**. The FireDAC DAD is developed and maintained together by the Data Abstract and FireDAC teams. So, the expertise and resources of each team is fully accessible to the other team. That means:
  - Both products are getting features for better interoperability.
  - You will never hear "Sorry, this is not our issue".
  - Always up-to-date builds of the Data Abstract and FireDAC DAD.

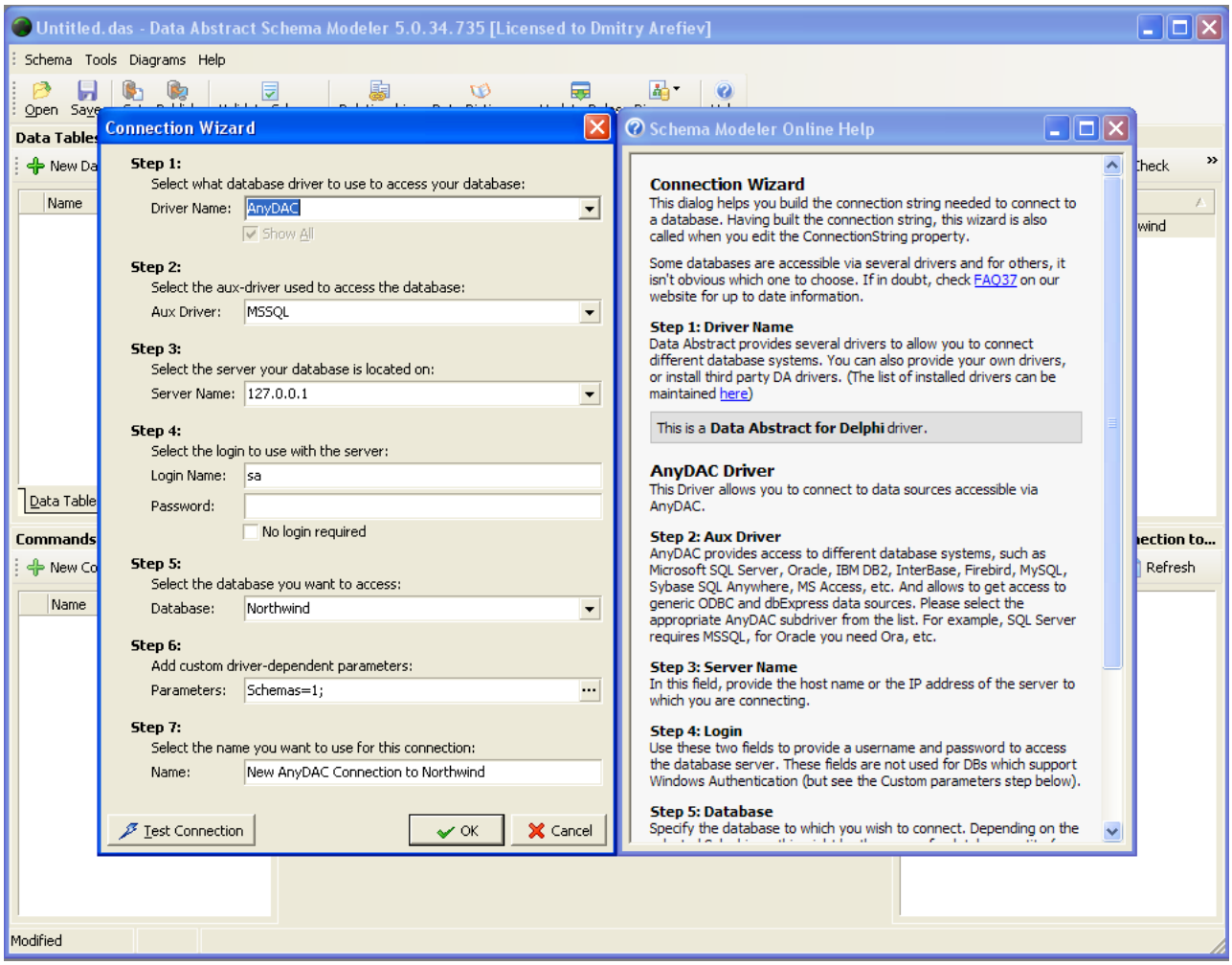
3. How to use

Configure connection

In this section we will review how to build a Data Abstract driver connection string. A connection string is a set of the DAD parameters, allowing you to establish a connection to a DBMS.

Using DASM Connection Wizard

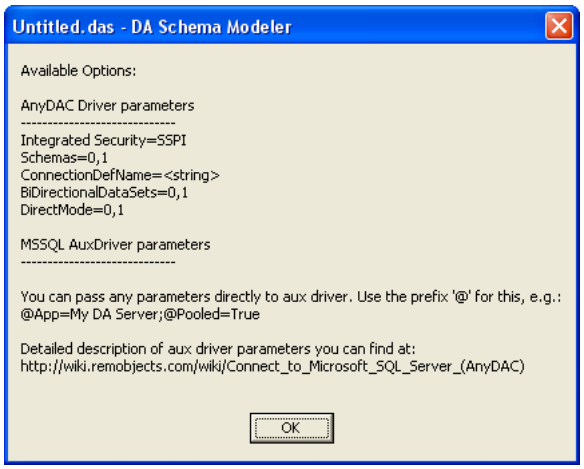
Note: Use the **DASM** -> **Schema** -> **Drivers** menu item to see the FireDAC DAD version info. This info is required to be provided to the technical support, if you contact them regarding FireDAC DAD usage.



In this part we will review step by step how to configure a connection to a Microsoft SQL Server using the FireDAC DAD. We will use the Data Abstract Schema Modeler Connection Wizard to build the connection string. So, lets run the DASM and press **New Connection** on the **Connections** pane. In general, you can follow to the instructions shown on the right of the Connection Wizard dialog:

- Step 1. Specify **FireDAC** as **Driver Name**.
- Step 2. Because FireDAC DAD provides access to the different database systems, using own drivers, you should choose the FireDAC driver ID as **Aux Driver**. In our tutorial it is **MSSQL**.
- Step 3. Specify a MS SQL Server name in **Server Name**. For example, 127.0.0.1 for a locally running default MS SQL Server instance.
- Step 4. Provide user credentials. If you use Windows Authentication, check that Custom Parameters (Step 6) has **Integrated Security=SSPI**; specified. Otherwise, delete this parameter. We use 'sa' as a **Login Name** with empty **Password**.
- Step 5. Specify a MS SQL Server database name. For example, Northwind.

Basically, at this point you should be able to connect to the DBMS. You can verify that by pressing the **Test Connection** button.



- Step 6. Here you can specify the custom driver parameters. You can press the button on the right of the **Parameters** edit box to get a help dialog, containing a list of the supported driver specific parameters. We use **Schemas=1**.
- Step 7. Now specify your connection name and press the **OK** button to save the connection.

By code

The general format for the FireDAC DAD connection string is:

```
FireDAC?AuxDriver=<FireDAC driver ID>;Server=<server>;Database=<database>;UserID=<user
name>;
Password=<password>;<FireDAC DAD specific parameters>;<FireDAC driver specific parameters>
```

So, for the above example, the connection string is:

```
FireDAC?AuxDriver=MSSQL;Server=127.0.0.1;Database=Northwind;UserID=sa;Password=;Schemas=1
```

The connection string may be specified at design time or at runtime using the TDAConnectionManager component. Set the **Default** property to **True** to use this connection as the default. To link the FireDAC DAD driver to your application, do one of the following:

- drop **TDAAnyDACDriver** to any form or data module in your application;
- include **uDAAnyDACDriver** into any uses clause in your application.

4. Custom parameters

DAD specific parameters

These parameters are used by the FireDAC DAD to control the DAD behaviour and are not visible to the FireDAC drivers. The following table lists the parameters:

Parameter	Description	Example value
ConnectionDefName	Allows to specify FireDAC connection definition name, instead of specifying FireDAC driver separate parameters in connection string.	MSSQL_Demo
BiDirectionalDataSets	FireDAC DAD dataset mode: • 0 - use unidirectional datasets (by default). This allows to speed up fetching and preserve memory usage for large result sets. • 1 - use bidirectional datasets. This allows you to walk through FireDAC DAD datasets in both directions.	1

DirectMode	FireDAC DAD mode: • 0 - standard mode, when DAD implements a IDADataset interface using TDataSet descendants. • 1 - direct mode. FireDAC implements it using high-speed low level data access interfaces. See next chapter for details.	1
Integrated Security	SSPI - set OSAuthent=Y connection definition parameter. Is used only for SQL Server connections.	SSPI
Schemas	1 - forces DAD to return object names prefixed with schema. Is used only for SQL Server connections.	1
DataTypeSchema	FIB - forces DAD to use FIB+ compatible data type mapping. Is used only for IB/FB connections.	FIB

FireDAC specific parameters

These parameter names must be prefixed by the '@' sign and are transferred directly to an FireDAC driver to establish a connection to a DBMS. Check FireDAC Database Connectivity (see page 188) for details.

Alternatively, you can specify connection definition by the **ConnectionDefName** parameter. Also to specify Format, Fetch, Update and/or Resource options:

1. Start ADExplorer and setup a connection definition.
2. At the Advanced options page set up the required options.
3. Save the connection definition.
4. In ADConnectionDefs.ini copy the saved options and append them to the connection string preceding them with "@" symbol.

For example, FireDAC connection definition with options:

```
[AnyDAC_DAD]
Database=Northwind
Server=127.0.0.1
DriverID=MSSQL
FetchOptions.AssignedValues= [evMode]
FetchOptions.Mode= fmAll
FormatOptions.AssignedValues= [fvMapRules]
FormatOptions.OwnMapRules= True
FormatOptions.MapRules= < item NameMask = 'MY_BOOL' SourceDataType = dtInt16 TargetDataType
= dtBoolean end>
```

And the extended connection string:

```
FireDAC?AuxDriver=MSSQL;Server=127.0.0.1;Database=Northwind;UserID=sa;Password=;Schemas=1;
@FetchOptions.AssignedValues=[evMode];@FetchOptions.Mode=fmAll;
@FormatOptions.AssignedValues=[fvMapRules];@FormatOptions.OwnMapRules=True;
@FormatOptions.MapRules=< item NameMask = 'MY_BOOL' SourceDataType = dtInt16 TargetDataType
= dtBoolean end>;
```

Or

```
FireDAC?AuxDriver=MSSQL;@ConnectionDefName=AnyDAC_DAD;
```

5. Direct mode

Since 5.0.31.701 version in the Data Abstract FireDAC driver was introduced new working mode so-called **Direct Mode**. The Data Abstract driver Direct Mode excludes a **TDataSet** descendants overhead and allows to use low-level high-speed DAF APIs. While the standard driver mode is the **Dataset Mode**, it uses the DAF specific **TDataSet** descendants.

In the Direct Mode the FireDAC DAD uses the Phys and DatS layers (see page 17) directly, excluding the Comp and DApt

layers overhead. That gives 20-40% performance gain. Note that:

- the same driver supports the Direct and Dataset modes;
- the driver behaviour remains almost the same in either of the modes.

Using direct mode is managed by parameter **DirectMode**, setting DirectMode=1 enables direct mode, setting DirectMode=0 disables direct mode. The Direct Mode is turned **off** by default, as it is not 100% backward compatible. You need to explicitly analyze the application and, if possible, to turn Direct Mode **on**.

The main difference between the two modes is, that in the Direct Mode the DAD does not use the **TDataSet** descendant objects. So, **IDASQLCommand.Dataset** and **TDACustomField.BindedField** are always nil. While it is not a limitation for a Data Abstract server, it may be an issue with local usage. Anyway, in DirectMode=1 you can use the **TDACustomField.BindedNativeField** instead of **TDACustomField.BindedField** and **IDASQLCommandNativeObject**.

Known limitations in DA 5.0.31.701:

- DASM doesn't support DirectMode=1 for configuring schema so it is recommended to set up your schema in DASM with DirectMode=0 and after this to switch into DirectMode=1.
- TDABinDataStreamer doesn't support DirectMode=1.

6. Migration

In this section we will review the migration of the application to FireDAC DAD from any other DAD.

In general, you should perform the following steps:

- Replace the connection string with an FireDAC compatible one.
- If your application works directly with an underlying data access framework, this code must be changed to an FireDAC compatible one.

There are possible issues:

- An FireDAC DAD data type mapping may be different from the other DADs. If you are migrating from FIB+ DAD, just use the **DataTypeSchema=FIB** connection parameter to force the FIB+ data type mapping in the FireDAC DAD. For other DADs, please contact RemObjects.
- An FireDAC DAD behavior may be different from the other DADs. For example, FireDAC offers the same range of transaction control features, just through a different API.
- The FireDAC may not contain the DBMS specific components, which other data access frameworks have, for example, IBDump from FIB+.

7. Summary

This article has provided the benefits of combining Data Abstract and FireDAC to get premier N-tier data access solutions. The articles describes all aspects of the FireDAC DAD usage.

1.10 x Platform Development

The set of articles describing how to use FireDAC for cross-platform development for Linux, Mac OS X and other environments.

1.10.1 Installing FireDAC

FireDAC requires special steps to install or deploy on Linux, Mac OS X and iOS.

Description

The following optional steps are required to install FireDAC and / or deploy FireDAC applications on Linux, Mac OS X and iOS. These steps are mandatory when you are using persistent connection definitions (see page 27) or virtual drivers (see page 31):

1. Step is required for Lazarus / FPC development and not needed for Delphi development. Copy FireDAC folder from Windows installation to Linux or Mac OS file system. We suggest to copy to the ~/FireDAC folder.
2. Create ~/.firedac folder.
3. Create there ad.conf file with the following content (see FireDAC/DB/ad.conf.linux for example):

```
[main]
ADHOME=$(HOME)/.firedac
ConnectionDefFile=$(ADHOME)/ADConnectionDefs.ini
DriverFile=$(ADHOME)/ADDrivers.ini
```

4. Use FireDAC/DB/ADConnectionDefs.linux as a starting point for your connection definition file. Copy it into ~/.firedac/ADConnectionDefs.ini, then add required connection definitions to it. The file is optional.
5. Use FireDAC/DB/ADDrivers.linux as starting point for your driver configuration file. Copy it into ~/.firedac/ADDrivers.ini, then add required driver configurations to it. The file is optional.

See Also

Lazarus / FPC (see page 160)

1.10.2 Using FireDAC

FireDAC development on Linux, Mac OS X and iOS has differences comparing to Windows.

Description

In general FireDAC application development on / for Linux, Mac OS X and iOS is similar to Windows, but has few aspects, additionally to Known Limitations (see page 160):

1. GUIx components must have Provider (see page 672) = 'FMX' to use FireMonkey implementation for FireDAC GUI dialogs. Note, switching to Provider = 'FMX', may require to delete uADGUIxFormsXxx units from the uses clauses.
2. Try to avoid to specify design-time paths for FireDAC properties, like TADPhysDriverLink.VendorLib (see page 782) or TADCustomManager.ConnectionDefFileName (see page 371). Use default values or configuration files.
3. FPC TStringField does not have published FixedChar property. So, if the code was originally developed in Delphi IDE and is planned to use it in FPC, then set TStringField.FixedChar to False for all persistent fields.
4. An application and a DBMS client dynamic library bitnesses must be the same. x86 executable will use x86 libraries. x64 executable will use x64 libraries.

See Also

Known Limitations (see page 160)

1.10.3 Known Limitations

The list of known FireDAC limitations related to cross platform development.

Description

This is the list of known FireDAC limitations comparing to using FireDAC with Delphi on Windows:

1. Supported platform are Windows x86 and x64, Linux x86 and x64, Mac OS X x86, iOS.
2. Utilities (see page 178) are provided for Windows only.
3. On Linux and Mac OS the ODBC-based drivers (SQL Server (see page 202), IBM DB2 (see page 196), SQL Anywhere (see page 224)) may require explicit ODBC driver specifying, using the TADPhysXxxxDriverLink.ODBCDriver (see page 799) property or configuration file (see page 31).
4. Microsoft Access (see page 207) driver is supported only on Windows platforms.
5. Microsoft SQL Server ODBC driver and FreeTDS ODBC driver have quite different behavior and quality.
6. SQLite (see page 221) driver supports static linking only for Win32. For other platforms the dynamic linking must be configured.
7. The ANSI <-> UCS2 conversion routines are not implemented in full for FPC / Lazarus. They supports only ASCII characters.
8. DBGrid displays SQL Server money field value with more than 4 digits after a decimal separator with FPC / Lazarus. This is due to the rounding error in FPC FloatToStrF function.
9. XML DataSet format is supported only for Delphi Win32 and Win64.

See Also

Installing FireDAC (see page 159), Using FireDAC (see page 159)

1.10.4 UnixODBC

FireDAC on the non-Windows platforms uses UnixODBC to access to the ODBC-based data sources.

Description

FireDAC on Linux, Mac OS X and other non-Windows platforms uses UnixODBC to access to Microsoft SQL Server (see page 202), IBM DB2 (see page 196), Sybase SQL Anywhere (see page 224) and other ODBC based data sources (see page 213). Often UnixODBC must be installed additionally to the OS installation. That is different from Microsoft Windows, where ODBC manager is installed as part of OS installation.

FireDAC uses **libodbc.dylib** or **.so** dynamic library to get access to ODBC API. Normally it is located in /usr/local/lib folder.

UnixODBC may be downloaded as an archive with the sources (here) (more) and saved into some folder in your home directory. To install UnixODBC on Mac OS X use the commands:

```
gunzip unixODBC*.tar.gz
tar xvf unixODBC*.tar
export CFLAGS=-m32
./configure
make
sudo make install
```

1.10.5 Lazarus / FPC

FireDAC supports Lazarus / FPC. Lazarus / FPC enables application development for Windows, Linux, Mac OS, iOS and

more.

Description

The current release of "FireDAC for Delphi" provides Lazarus / FreePascal support for the platforms:

- Windows x86 and x64;
- Linux x86 and x64;
- Mac OS X x86;
- iOS.

The current release "FireDAC for Delphi" was compiled and tested with:

- FPC 2.6.0;
- Lazarus 0.9.30.4.

Note, because FPC and Lazarus / FPC libraries (RTL, LCL) have a not stable API, which often is not backward compatible, FireDAC is tested and supports only the exact specified Lazarus / FPC version.

Note, FPC 2.6.0 may return "Internal error: 200610054" when "Generate Debugging Info for GDB" (-g) or "Display Line Numbers in Run-time Error Backtraces" (-gl) options are enabled. The workaround is to turn them off.

Installing

To install FireDAC on Windows, Linux or Mac OS X, the FireDAC must be first installed on Windows using the standard FireDAC installer. We do not provide separate installer for Lazarus / FPC. See additional articles for the platform specific instructions.

The Lazarus / FPC demos you can find at "FireDAC\Samples\FPC Specific" folder.

1.10.5.1 Installing on Windows

Describes the steps to install Lazarus, FPC and FireDAC on Windows.

Description

Installing Lazarus on Windows

Download:

- For Windows 32 bit development: lazarus-0.9.30.4-fpc-2.6.0-win32.exe
- For Windows 64 bit development: lazarus-0.9.30.4-fpc-2.6.0-win64.exe

Install Lazarus and FPC using the downloaded installer.

Installing FireDAC on Lazarus

FireDAC on Windows uses the same installation and configuration as the Delphi installation. So, you will only need to:

1. Run Lazarus IDE.
2. Open FireDAC/Packages/AnyDAC_Dcl_FPC.lpk package.
3. Press Install button.

See Also

Configuring Drivers (see page 31), Defining Connection (see page 27), Database Connectivity (see page 188)

1.10.5.2 Installing on Linux

Describes the steps to install Lazarus, FPC and FireDAC on Linux.

Description

Installing Lazarus on Linux from repository

Read the article for details.

Installing Lazarus on Linux using installer

Download for Linux 32 bit development:

- fpc-2.6.0-1.i386.deb.tar
- lazarus-0.9.30.4.i386.deb.tar

For 64 bits, please, use the corresponding 64 bit installers. Install Lazarus and FPC by hands:

1. Optionally update and upgrade your Linux installation:

```
sudo apt-get update
sudo apt-get upgrade
```

2. Optionally install Midnigth Commander, if you are not very comfortable with Linux command line:

```
sudo apt-get install mc
sudo mc
```

If you will not use MC, then run shell as root:

```
sudo bash
```

3. Create /opt/lazarus folder:

```
mkdir /opt/lazarus
cd /opt/lazarus
```

4. Extract all tar's to /opt/lazarus:

```
tar -xf <file name>
```

5. Create a local installation repository:

```
apt-get install dpkg-dev
dpkg-scanpackages . /dev/null | gzip > Packages.gz
echo "deb file:/opt/lazarus ./" >> /etc/apt/sources.list
```

6. And install Lazarus

```
apt-get install lazarus
```

Installing FireDAC in Lazarus

Install FireDAC (see page 159) on the file system, then perform the steps:

1. Run Lazarus IDE.
2. Select "Tools" -> "Configure 'Build Lazarus'". Specify -dUseCThreads in the "Options field". Press "Save settings" button.

3. Open FireDAC/Packages/AnyDAC_Dcl_FPC.lpk package'
4. Press Install button.

See Also

Configuring Drivers (see page 31), Defining Connection (see page 27), Database Connectivity (see page 188)

1.10.6 Path Variables

FireDAC allows to use a substitution variables in the paths, simplifying application setup.

Description

FireDAC allows to use a substitution variables in the paths, including:

- **Database** connection parameter, specifying a local database connection path;
- SQLScriptFileName (see page 703) property of TADScript;
- FileName property of TADMoniFlatFileClientLink;
- ConnectionDefFileName (see page 371) property of TADManager;
- DriverDefFileName (see page 373) property of TADManager.

A string value may contain substitution variables inside with format **$(name)**. There are several pre-defined variable names:

- DOC - the documents folder path;
- TEMP - the temporary files folder path;
- RUN - the application executable path

Additionally a variable may be a name of environment variable.

All variable values are without terminated slash. In a string value may be used as forward as back slashes. FireDAC will replace slashes with appropriate value for a platform.

Example

A path to an SQLite DB on iOS device:

```
$(DOC)/test.sdb
```

A path to an temporary DB:

```
$(TEMP)\data.sdb
```

1.11 Migrating BDE applications

This article guides you through a series of the steps to migrate the BDE application to the FireDAC.

Description

Introduction

This article provides a tutorial showing how to migrate a simple client-server application using BDE data access components, like TDatabase, TQuery, TTable to the FireDAC. It shows the basic principles of replacing the common components, properties and code, preserving the developers working time and avoiding the common migration pitfalls.

Overview

In general, the FireDAC components have a high compatibility level with the BDE data access components. This includes

the syntax and semantic of the properties and methods in the FireDAC and BDE. But some parts are different:

- BDE and FireDAC have components with different names. E.g. BDE - TQuery, FireDAC - TADQuery (see page 476).
- BDE and FireDAC have different alias/connection definition systems. BDE stores aliases in the binary system wide file IDAPI.CFG; the FireDAC stores the connection definitions (see page 27) in the **ADConnectionDefs.ini** file.
- Some parameters for BDE SQLLink's and FireDAC drivers are different.
- BDE and FireDAC may have the different data type mapping for the same RDBMS. FireDAC follows more close to the dbExpress data type mapping. However the FireDAC has powerful capabilities to adjust the data type mapping.
- The base API's (the BDE API and the Phys (see page 17) interface) are absolutely incompatible. If you have some code which directly uses the BDE API, it has to be be recoded.
- BDE and FireDAC have different heterogenous query syntax and capabilities. FireDAC offers Local SQL (see page 96) feature.

That is nearly all of what should be changed at migration from BDE to FireDAC. After the migration (or in parallel with it), you should consider to review your application for:

- using the extended FireDAC functionality to simplify your application;
- using the extended FireDAC functionality to extend the functionality of your application;
- using the FireDAC options to fine tune your application and speed it up.

1.11.1 BDE name counterparts

Most of BDE high-level types and components have counterparts in FireDAC.

Description

The following table lists the BDE names which have analogues in the FireDAC world:

BDE name	FireDAC name	FireDAC unit
TSession	TADManager (see page 429)	uADCompClient (see page 257)
TDatabase	TADConnection (see page 279)	uADCompClient (see page 257)
Alias*	ConnectionDef*	
Database*	Connection*	
Session	ADManager (see page 570)	uADCompClient (see page 257)
SessionName		
PrivateDir		
=====	==========	==========
TQuery	TADQuery (see page 476)	uADCompClient (see page 257)
TStoredProc	TADStoredProc (see page 514)	uADCompClient (see page 257)
TTable	TADTable (see page 539)	uADCompClient (see page 257)
TUpdateSQL	TADUpdateSQL (see page 563)	uADCompClient (see page 257)
TBatchMove	TADDataMove	uADCompDataMove
=====	==========	==========
TParam	TADParam	uADStanParam

TParams	TADParams	uADStanParam
TBlobStream	TADBlobStream (see page 582)	uADCompDataSet (see page 571)
TDBDataSet, TBDEDataSet	TADRDBMSDataSet (see page 501)	uADCompClient (see page 257)
EDBEngineError	EADDBEngineException (see page 833)	uADStanError (see page 831)

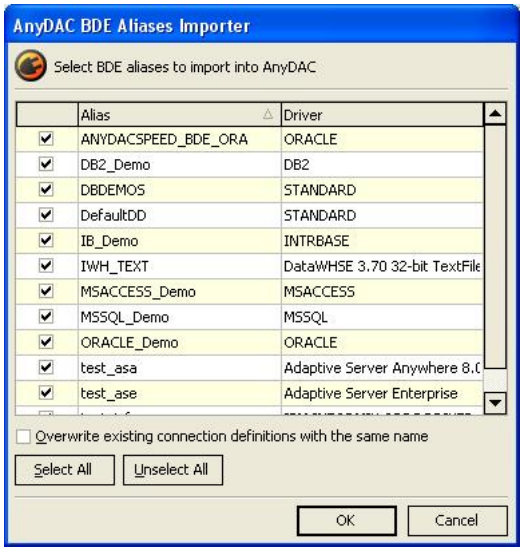
The full list you can find in <FireDAC>\Bin\BDE2FireDAC.txt file.

1.11.2 BDE aliases migration

FireDAC has the BDE aliases migration function.

Description

FireDAC Explorer (see page 181) and FireDAC Administrator (see page 178)- both have the BDE aliases migration function. Run one of the applications and choose **Connection -> Import BDE Aliases**:



This dialog shows all your BDE aliases. Select the ones you like to migrate to the FireDAC. Click **Overwrite existing connection definitions...**. The existing FireDAC alias definitions with the same name as BDE aliases will be overwritten. In case you do not like to overwrite existing aliases you will get duplicates with names like "DBDEMOS_1". After pressing OK, FireDAC imports all BDE RDBMS's aliases supported by FireDAC.

FireDAC imports a new connection definition for every existing BDE alias in FireDAC. They will be placed into the connection definitions file.

1.11.3 BDE application migration

An step-by-step example showing how to migrate BDE application to FireDAC.

Description

This example shows steps by step how the classic CodeGear demo application - MastApp - will be migrated to the FireDAC and Oracle DBMS.

Step 1

Create a new directory. For example, ADMastApp. Later we will reference to your new directory using the ADMastApp name. Then copy MastApp source files from the Delphi Demos directory to the new one created.

Step 2

Open a cmd window and change to the ADMastApp directory, or use your preferred directory manager, like FAR. Run the FireDAC transformation tool ADDFMChanger (see page 178) in order to replace the BDE terms with its FireDAC synonyms:

```
<ADHome>\Bin\ADDFMChanger.exe *.pas *.dfm -a -f <ADHome>\Bin\BDE2FireDAC.txt
```

Note, ADDFMChanger is not currently available in Embarcadero FireDAC v 7.0.1.

Step 3

Now search in all PAS's and DFM's for all properties and methods marked for removal. The simplest way to do so is to search for the **removed** string. In the MastApp the single place is the DataMod.dfm.

```
object Connection: TADConnection
  ...
  <SessionName removed> = 'Default'
  ...
end
```

Consider to remove such lines. In general, **SessionName** with different values may be a sign that your application requires multiple connections.

Step 4

Create an FireDAC connection definition (see page 27) using the FireDAC Explorer (see page 181). Then setup connection definition parameters. This is the most simple connection definition for MastApp:

```
[MASTSQL]
DriverID=Ora
DataBase=ORA_920_APP
User_Name=ADDemo
```

NOTE: This implies that you have created an Oracle user ADDemo and loaded a DBDEMOS database (Paradox or Interbase) into the Oracle ADDemo user schema. For example, you can use the BDE's DataPump for uploading, but keep in mind that you will need to adjust the identifier registers - they must be UPPER CASE in Oracle.

Step 5

Add a RDBMS specific FireDAC driver to the application. Add one of the uADPhys[RDBMS driver ID] units to your project. E.g. if you use Oracle, add uADPhysOracle (see page 800) to the project. Then add unit ADGUIxFormsWait to your project:

```
program Mastapp;

uses
  Forms,
  uADPhysOracle,
  uADGUIxFormsWait,
  MAIN in 'MAIN.PAS' {MainForm},
  ......
```

Step 6

Now open the DataMod.dfm file using a text editor. Setup the TADConnection component in the data module. Set ConnectionDefName to an Oracle connection definition. Also change the user name and password to the correct one. You will then get something like this:

```
Params.Strings = (
  'ConnectionDef=Oracle_Demo'
  'User_Name=addemo'
  'Password=a')
```

Step 7

The Interbase SQLLink and FireDAC drivers have different data type mappings, so we should adjust that to make FireDAC type mapping compatible with BDE. Otherwise, we will need to recreate all persistent fields. To setup data mapping, we should fill up the collection property TADConnection (see page 279).FormatOptions (see page 285).MapRules (see page 865). Lets do it in DataMod.DFM:

```
object Database: TADConnection
......
  FormatOptions.OwnMapRules = True
  FormatOptions.MapRules = <
    item
      PrecMax = 10
      PrecMin = 0
      ScaleMax = 0
      ScaleMin = 0
      SourceDataType = dtFmtBCD
      TargetDataType = dtInt32
    end
    item
      SourceDataType = dtFmtBCD
      TargetDataType = dtDouble
    end
    item
      SourceDataType = dtDateTimeStamp
      TargetDataType = dtDateTime
    end>
......
end
```

Additionally, set FormatOptions (see page 285).StrsTrim (see page 869) property to False, as default value for this property is incompatible with BDE.

Step 8

FireDAC does not support desktop DB's, like Paradox or Dbase. So, we have to remove all desktop DB (Paradox, Dbase) code from the application:

- method TMastData.UseLocalData in DataMod.pas.
- method TMainForm.ViewLocalClick in Main.pas
- TField.Origin property in DataMod.dfm has the format: "TABLE.DB".FieldName. We will change that to the "TABLE".FieldName, where "TABLE" is the name of the original field DB table. In most cases we can just remove "TABLE" when the query is using only a single table.

Step 9

Now we should adjust the application source code to Oracle:

- In the TMainForm.ViewRemoteClick method in Main.pas, replace string ' (Local Interbase)' with ' (Oracle)'.
- Remove the TMainForm.ViewMenuClick handler in Main.pas.

- Remove the TMastData.DataDirectory method in DataMod.pas.
- The application creates a DB connection definition on the fly if it does not exist. So, change the TMastData.UseRemoteData in the DataMod.pas to:

```
procedure TMastData.UseRemoteData;
var
  Params: TStringList;
begin
  { See if the ConnectionDef exists.  If not, add it. }
  if not ADManager.IsConnectionDef('MASTSQL') then
  begin
    Params := TStringList.create;
    try
      Params.Values['DATABASE'] := 'APPOR920_LSNR';
      Params.Values['USER NAME'] := 'MASTAPP_DB';
      ADManager.AddConnectionDef('MASTSQL', 'ORA', Params);
    finally
      Params.Free;
    end;
  end;
  SetDatabaseConnectionDef('MASTSQL');
end;
```

- We have to adjust the SQL commands used by applications. In DataMod query CustByLastInvQuery has the DESCENDING key word, but Oracle uses DESC.

1.11.4 Additional migration hints

Few additional BDE application migration considerations.

Description

Hint 1

Some properties should be removed completely, because FireDAC does not have the analogues:

- **SessionName** should be removed completely. But be careful, your application may be using multiple same named connections in different sessions.
- **PrivateDir** should be removed completely.

Hint 2

Although FireDAC has analogue to the BDE's **TTable** named TADTable (see page 539), it is a good idea to replace all TTable's with TADQuery (see page 476)'s right at the migration.

Hint 3

BDE application with persistent fields must be adjusted additionally. BDE does not provide or use persistent fields Origin and ProviderFlagsproperties, and queries DB dictionary to get the unique identifying fields. FireDAC provides and uses (see page 121) persistent fields Origin and ProviderFlags properties. When Origin is empty (as it is in BDE application), then FireDAC will use field name. But default value of ProviderFlags does not include pfInKey, and FireDAC will not query DB for PK fields. So, it will fail to get unique identifying fields (see page 117) and you must take one of the additional actions:

- recreate persistent fields;
- manually adjust ProviderFlags;
- manually specify UpdateOptions (see page 653).KeyFields (see page 845).

Hint 4

Constructions, like a:

```
Screen.Cursor := crSQLWait;
try
  ......
finally
  Screen.Cursor := crDefault;
end;
```

Should be replaced with:

```
uses
  uADStanFactory, uADGUIxIntf;
  ......
var
  oWait: IADGUIxWaitCursor;
  ......
  ADCreateInterface(IADGUIxWaitCursor, oWait);
  oWait.StartWait;
  try
    ......
  finally
    oWait.StopWait;
  end;
```

Hint 5

FireDAC does not have a BDE API analogues. So, every code directly using the BDE API should be recoded using only FireDAC API. There is no direct solution.

Hint 6

Many third party products, like a reporting or m-tier libraries, require a DAC adapter unit. In most cases it exists for BDE and does not exists for FireDAC, while we are working to add most used ones. So, either develop one yourself (taking the BDE adapter as template), or contact us.

Hint 7

EDBEngineError is the BDE specific exception class. FireDAC has an analogue - the EADDBEngineException (see page 833) class. When handling the BDE exceptions, the programmer uses the **ErrorCode** property to get an error kind. FireDAC has the property kind, which returns an enumerated value.

For example, change the following code:

```
if E is EDBEngineError then
begin
  case EDBEngineError(E).Errors[0].ErrorCode of
    DBIERR_KEYVIOL: MetaBaseDBError(SMb_DataSetInvalidPKeyValue, E);
end;
```

into:

```
if E is EADDBEngineException then
begin
  case EADDBEngineException(E).Kind of
    ekUKViolated: MetaBaseDBError(SMb_DataSetInvalidPKeyValue, E);
end;
```

Hint 8

The **TBatchMove** and TADDataMove are very different in many ways. You will need serious rework on any advanced code using the TBatchMove.

Hint 9

The TADConnection (see page 279).OnLogin (see page 287) event is incompatible with the **TDatabase.OnLogin** event parameters list. So, for example, you will need to replace the following handler:

```
procedure TMyDataModule.dbLogin(Connection: TDEConnection; LoginParams:
TStrings);
begin
  LoginParams.Values['USER NAME'] := 'me';
  LoginParams.Values['PASSWORD'] := 'pwd';
end;
```

with this one:

```
procedure TMyDataModule.dbLogin (AConnection: TADCustomConnection;
  const AConnectionDef: IADStanConnectionDef);
begin
  AConnectionDef.UserName := 'me';
  AConnectionDef.Password := 'pwd';
end;
```

Hint 10

When application uses BDE heterogeneous queries, then with FireDAC it should use Local SQL (see page 96).

1.12 Debugging

A set of articles explaining how to debug FireDAC application.

1.12.1 DBMS Environment Reports

This article guides you through different FireDAC options of DBMS environment checking and reporting. Such report kind is a must for your customer support and the developer team.

Description

General

All developers and supporters know the moment a customer calls about his application not running properly. Normally, it takes several phone calls or emails to get all the necessary details about his environment. Often the customer does not even have all this information.

In general, a DBMS application may fail due to:

- missing DBMS client libraries;
- incorrect version of the DBMS client and/or server;
- incorrect connection definition setup;
- compatibility issues like an Unicode support.

FireDAC provides the necessary methods to report all these details:

- **Using the ADExplorer or ADAdministrator**: use this tool to get the report of your persistent connection definition.
- **Using the TADConnection Design Time Editor**: use the TADConnection design time editor to get the detailed report about a connection definition.
- **by Code**: use Delphi code to integrate the reporting capabilities into your application.

Using the ADExplorer

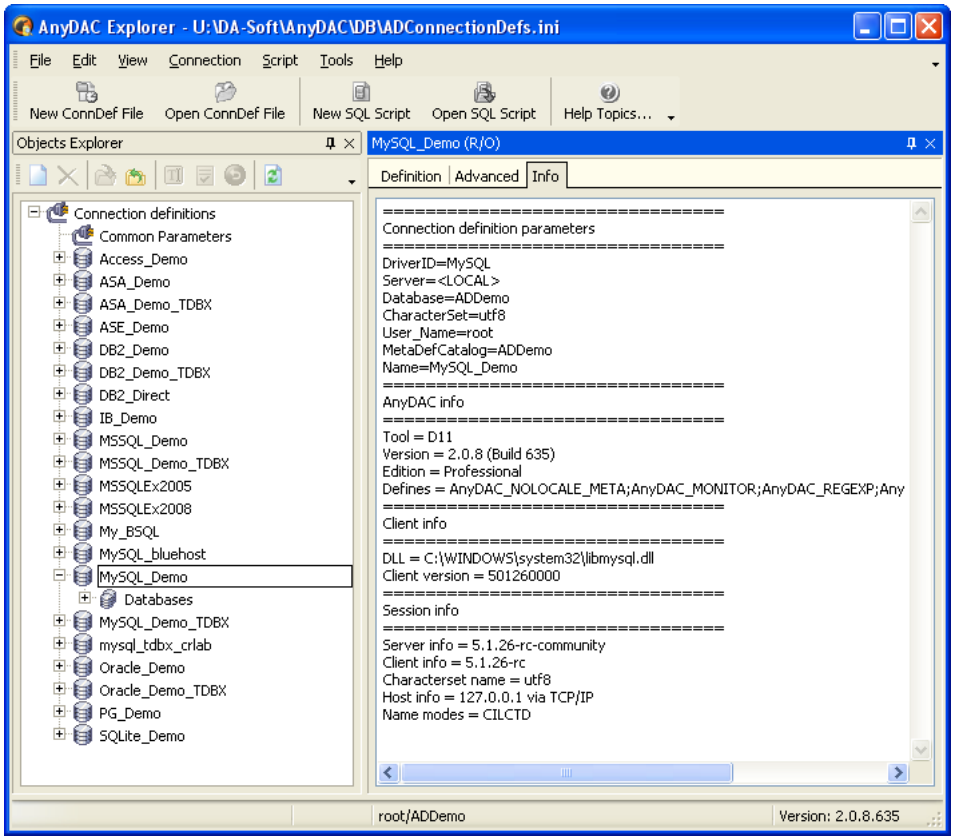
The ADExplorer utility is the main tool to maintain the centralized persistent connection definitions. Please, read the ADExplorer reference (see page 181) for further details.

To get a detailed report for your persistent connection definition, run ADExplorer. Then choose a connection definition in the tree on the left side. On the right side click on the **Info** page. There you will see detailed environment report.

The report includes the sections:

- **Connection definition parameters** - complete set of the connection definition parameters.
- **FireDAC info** - details of your FireDAC build.
- **Client info** - information about the DBMS client software, if it is properly installed and FireDAC is able to load it successfully. Otherwise - a failure error message.
- **Session info** - information about the DBMS server and the user session, if FireDAC was able to open successfully the connection. Otherwise - a failure error message.

The next screen shot shows the report for a MySQL connection definition. In this case the DBMS client software was loaded and the connection was activated:

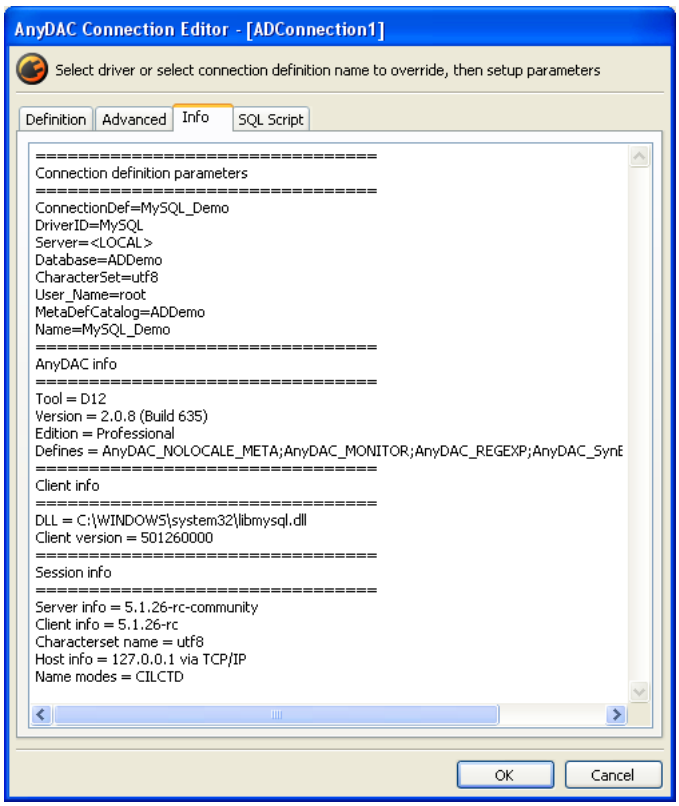


Using the TADConnection Design Time Editor

The TADConnection (see page 279) component design time editor is the environment to maintain temporary connection parameters. Double click any TADConnection (see page 279) component at design time. The FireDAC package will display the **Connection Editor** dialog.

Click on the **Info** page to get a detailed report for your connection definition. You will get a detailed environment report as described above. The only difference: if a TADConnection component is not connected, than the dialog will try to establish a temporary connection to a DBMS to fetch the server data.

The next screen shot shows the resulting report:



by Code

The FireDAC offers several possibilities to integrate the DBMS runtime information of your environment into your own project.

Call the GetInfoReport Method

You can start the DBMS report directly by calling the TADConnection.GetInfoReport (see page 347) method:

```
procedure TMainForm.Button1Click(Sender: TObject);
begin
  ADConnection1.GetInfoReport(mmInfo.Lines);
end;
```

This method is declared as:

```
procedure GetInfoReport(AList: TStrings; AItems: TADInfoReportItems);
```

The AItems flags meaning is:

- **riConnDef** - include connection definition parameters into a report.
- **riFireDAC** - include FireDAC build info into a report.

- **riClientLog** - include DBMS client loading log.
- **riClient** - include a DBMS client info into a report.
- **riSessionHints** - include an FireDAC, DBMS client and server potential incompatibilities.
- **riSession** - include a DBMS session info into a report.
- **riTryConnect** - try to connect to a DBMS, if not already connected.
- **riKeepConnected** - when method established a connection, then keep it active.
- **AList** - Target string list to append the report.

Get the FireDAC Build Number

If you want to display the FireDAC build number (e.g. in the About box) than just refer to the CONST C_AD_Version in the uADStanConst unit. Example:

```
C_AD_Version = '5.0.2 (Build 1896)';
```

Get the DBMS version

To get a DBMS client and/or server version number (e.g. to switch on or off some feature in your application), use the TADConnection component and open the connection. Then use the following code sample, showing how to access the physical metadata:

```
procedure TMainForm.Button2Click(Sender: TObject);
var
  oMetaIntf: IADPhysConnectionMetadata;
begin
  // Get client and server versions
  oMetaIntf := ADConnection1.ConnectionMetaDataIntf;
  try
    ShowMessage(Format('Client version: %.10d; Server version: %.10d',
      [oMetaIntf.ClientVersion, oMetaIntf.ServerVersion]));
  finally
    oMetaIntf:= nil; // always release the Interface before you close the connection
  end;
end;
```

You can find many predefined DBMS version numbers in **uADStanConst** unit:

- **cvOracleXXX** - for Oracle;
- **mvMySQLXXX** - for MySQL;
- **svMSSQLXXX** - for MSSQL;
- **svSQLiteXXX** - for SQLite;
- **svPGSQLXXX** - for PostgreSQL;
- **iv{IB|FB|YF}XXX** - for Interbase / Firebird / Yaffil.

The DBMS version is represented in FireDAC by the LongWord value, consisting of the 5 groups of digits, where each group has 2 digits. Although, the leftmost group may have one single digit. For example, mvMySQL032321 = 0323210000. It corresponds to 3.23.21.0.0. This allows direct comparison of a client or server version with a constant:

```
if oMetaIntf.ServerVersion >= mvMySQL050000 then begin
  // execute MySQL 5.0 or higher specific SQL
end
else begin
  // execute pre-MySQL 5.0 SQL
end;
```

See Also

Tracing and Monitoring (see page 174), TADConnection (see page 279), ADExplorer (see page 181)

Example

<FireDAC>\Samples\Comp Layer\TADConnection\InfoReport

1.12.2 Tracing and Monitoring

This article shows how to use FireDAC tracing and monitoring capabilities, which allows to see how an application is communicating with a database and is main FireDAC application debugging tool.

Description

General

FireDAC trace output is the detailed log of the communication between FireDAC application and the database software. It includes reference numbers, times, API calls, SQL sent to a database, data exchange with parameter and fields values, errors and warnings, and the DBMS environment report (see page 170).

FireDAC offers 3 methods to output the trace, which are controlled by the different components and different MonitorBy values:

Component	MonitorBy value	Description
TADMoniFlatFileClientLink	FlatFile	Outputs trace to a flat text file. When application finishes, it will show the list of produced trace files.
TADMoniRemoteClientLink (see page 723)	Remote	Outputs trace to ADMonitor (see page 185) utility. Also allows to monitor the application. ADMonitor utility must be running before activating a trace output.
TADMoniCustomClientLink	Custom	Output trace to a custom event handler. Application should use OnOutput event handler to produce custom trace output.

The Moni components are singletons referring to a single instance of a corresponding tracer implementation. All connections with enabled trace output will use the same tracer and a single output for all connections will be produced.

Controlling tracing

To enable trace output for a connection:

- Drop TADMoniXxxxClientLink component to a form.
- Set it Tracing property to True.
- Add MonitorBy (see page 188)=Xxx connection definition (see page 27) parameter.

The MonitorBy parameter associates the definition with a specific tracing method and becomes read-only after first connection is created for this definition. It should be set permanently.

TADMoniXxxxClientLink.Tracing property must be True before opening a first connection with MonitorBy=Xxx. Later to temporarily disable or enable tracing output for all connections, use TADMoniXxxxClientLink component Tracing property. Note, TADMoniXxxxClientLink should come before TADConnection in the data module or form creation order.

To initially disable trace output for a specific connection, use MonitorBy=Xxx**-**. To temporarily disable or enable tracing output for a specific connection, set ADConnection.ConnectionIntf (see page 326).Tracing property. Note, that the ConnectionIntf (see page 326) is accessible only after connection is established.

For example:

```
ADMoniFlatFileClientLink.Tracing := True;

with ADConnection1.Params do begin
  Clear;
  Add('DriverID=SQLite');
```

```
  Add('Database=c:\test.sdb');
  Add('MonitorBy=FlatFile');
end;

ADConnection1.Connected := True;
...
// disable trace output for connection
ADConnection1.ConnectionIntf.Tracing := False;
...
// enable trace output for connection
ADConnection1.ConnectionIntf.Tracing := True;
```

The trace content

To control the trace content, use TADMoniXxxxClientLink.EventKinds property. The sample trace output:

```
    1385 15:47:14.093     >> Fetch [ATable="ADQA_FK_tab", Command="SELECT AF.fk_id, AF.id,
AT1.f1, AT1.f2
FROM ADQA_FK_tab AF LEFT JOIN ADQA_tabwithpk AT1 ON AF.fk_id=AT1.f1"]
    1386 15:47:14.093          . sqlite3_column_type [stmt=$0F16E6B8, iCol=0,
Result=SQLITE_NULL]
    1387 15:47:14.093          . sqlite3_column_type [stmt=$0F16E6B8, iCol=1,
Result=SQLITE_INTEGER]
    1388 15:47:14.093          . sqlite3_column_int64 [stmt=$0F16E6B8, iCol=1, AValue^=2]
    1389 15:47:14.093          . sqlite3_column_type [stmt=$0F16E6B8, iCol=2,
Result=SQLITE_NULL]
    1390 15:47:14.093          . sqlite3_column_type [stmt=$0F16E6B8, iCol=3,
Result=SQLITE_NULL]
    1391 15:47:14.093          . sqlite3_step [stmt=$0F16E6B8]
    1392 15:47:14.093          . sqlite3_column_type [stmt=$0F16E6B8, iCol=0,
Result=SQLITE_INTEGER]
    1393 15:47:14.093          . sqlite3_column_int64 [stmt=$0F16E6B8, iCol=0, AValue^=2]
    1394 15:47:14.093          . sqlite3_column_type [stmt=$0F16E6B8, iCol=1,
Result=SQLITE_INTEGER]
    1395 15:47:14.093          . sqlite3_column_int64 [stmt=$0F16E6B8, iCol=1, AValue^=3]
    1396 15:47:14.093          . sqlite3_column_type [stmt=$0F16E6B8, iCol=2,
Result=SQLITE_INTEGER]
    1397 15:47:14.093          . sqlite3_column_int64 [stmt=$0F16E6B8, iCol=2, AValue^=2]
    1398 15:47:14.093          . sqlite3_column_type [stmt=$0F16E6B8, iCol=3,
Result=SQLITE_TEXT]
    1399 15:47:14.093          . sqlite3_column_text [stmt=$0F16E6B8, iCol=3, bytes=4]
    1400 15:47:14.093          . sqlite3_step [stmt=$0F16E6B8]
    1401 15:47:14.093          . profile [SQL="SELECT AF.fk_id, AF.id, AT1.f1, AT1.f2
FROM ADQA_FK_tab AF LEFT JOIN ADQA_tabwithpk AT1 ON AF.fk_id=AT1.f1", time=0]
    1402 15:47:14.093     << Fetch [ATable="ADQA_FK_tab", Command="SELECT AF.fk_id, AF.id,
AT1.f1, AT1.f2
FROM ADQA_FK_tab AF LEFT JOIN ADQA_tabwithpk AT1 ON AF.fk_id=AT1.f1", RowsAffected=2]
```

There:

- first number (1385-1402) is the line number, which may be used to reference the exact line;
- timestamp is the local time, when the line was generated, which may be used to measure performance;
- >> Xxxx - an FireDAC operation start. There parameters shows the command SQL text and other information;
- << Xxxx - an FireDAC operation finish;
- sqlite3_xxx - a DBMS API call (SQLite), which was performed to execute an FireDAC operation.

Flat file output

The TADMoniFlatFileClientLink component controls the flat text file output. By default the trace file name is automatically generated in the TEMP folder of your system. On termination an FireDAC application will show the list of the produced trace files.

To control output use properties:

- FileName - the trace file name. There may be used environment variables - $(name). Default - $(TEMP)\traceN.txt.

- FileEncoding - the trace file encoding. Default - ANSI.
- FileAppend - when True, then existing trace file will be appended. Otherwise will be overwritten on each new run.
- FileColumns - the columns to include into text file.

Remote output

The TADMoniRemoteClientLink (see page 723) component controls the remote trace output. Use ADMonitor (see page 185) utility to see the trace output. FireDAC uses TCP/IP as trace transport. That means that ADMonitor and application may run on different systems in a network. That may be actual for cross-platform development, where application may run on a Linux box and the ADMonitor must run on a Windows box.

To control output use properties:

- Host (see page 723) - the IP address of ADMonitor box. Default is localhost.
- Port (see page 723) - the IP port which ADMonitor is listening. Default is 8050.
- Timeout (see page 724) - the maximum number of ms allowed to establish connection to ADMonitor.

Custom output

The TADMoniCustomClientLink component may be used to produce a custom trace output. For that application should define OnOutput event handler. It is up to the application what it will do there.

Note, avoid Application.ProcessMessages call in the OnOutput event handler, as it seriously reduces performance and may lead to weird issues (due to possible FireDAC reentering, which is not supported).

Monitoring

ADMonitor utility also supports the monitoring of the FireDAC objects state. The developers may use this functionality to enable monitoring for their own objects. See FireDAC\Samples\Moni Layer\Main demo for details.

Checking the SQL command text

There are few methods to see the SQL command text, as it will be sent to a database:

- TADQuery.Text (see page 397) - returns SQL command text, as it will be sent to a database;
- EADDBEngineException.SQL (see page 835), Params (see page 835) - returns SQL command text, which leaded to an exception;
- Use above tracing capabilities to see the SQL commands.

Checking the dataset rows

The dataset rows may be dumped using one of the methods:

- DataSet.GetRow.DumpRow(True) - returns text representing current dataset row, including all field values and all field names;
- DataSet.GetRow.DumpRow() - returns text representing current dataset row, including all field values;
- DataSet.Table.Rows[i].DumpRow() - returns text representing dataset random row.

See Also

DBMS Environment Reports (see page 170)

1.13 Deploying

A set of articles explaining how to deploy a FireDAC application.

1.13.1 Delpoying on Windows

FireDAC application deployment on a x86 or x64 Windows machine includes several optional steps.

Description

To deploy FireDAC application on a Win32 or Win64 machine the following items should be considered:

1. When an application uses persistent connection definitions (see page 27), then a connection definition file must be deployed. It may be put along with application EXE or set TADCustomManager.ConnectionDefFileName (see page 371) to a file name.
2. When an application uses driver definitions (see page 31), then a driver configuration file must be deployed. It may be put along with application EXE or set TADCustomManager.DriverDefFileName (see page 373) to a file name.
3. The DBMS clients required for application connectivity (see page 188) must be obtained from the vendors and installed on the deployment machine. When an application is built as x86, then the x86 DBMS client software must be installed. When as x64, then x64 software. SQLite engine is statically linked into EXE and does not require a deployment.
4. FireDAC Administrator (see page 178) (analogue to BDE Administrator) may be deployed to give to an end-user a tool to manage FireDAC connection definitions. To deploy FireDAC Administrator one should deploy:
   - FireDAC\Bin\ADAdministrator.exe
   - FireDAC\Bin\ADExplorer.ini
5. When a FireDAC application was built with the run-time packages, then the following BPL files should be deployed. There DNnn corresponds to your Delphi version. FireDAC provides only x86 BPL's. The BPL's are located in "Package output directory", which you may find at IDE -> Tools -> Options -> Delphi options -> Library.
   1. Mandatory packages:
      - AnyDAC_ComI_DNnn.bpl
      - AnyDAC_Comp_DNnn.bpl
      - AnyDAC_GUIxForms_DNnn.bpl
      - AnyDAC_Phys_DNnn.bpl
   2. Optional packages, required when your application uses specific FireDAC drivers (see page 24):
      - AnyDAC_PhysADS_DNnn.bpl
      - AnyDAC_PhysASA_DNnn.bpl
      - AnyDAC_PhysDb2_DNnn.bpl
      - AnyDAC_PhysIB_DNnn.bpl
      - AnyDAC_PhysMSAcc_DNnn.bpl
      - AnyDAC_PhysMSSQL_DNnn.bpl
      - AnyDAC_PhysMySQL_DNnn.bpl
      - AnyDAC_PhysODBC_DNnn.bpl
      - AnyDAC_PhysOracl_DNnn.bpl
      - AnyDAC_PhysPg_DNnn.bpl
      - AnyDAC_PhysSQLite_DNnn.bpl
      - AnyDAC_PhysTDBX_DNnn.bpl

See Also

Defining Connection (see page 27), Configuring Drivers (see page 31), Database Connectivity (see page 188), ADAdministrator (see page 178)

1.13.2 Delpoying on Mac OS X or Linux

FireDAC application deployment on a Mac OS X or a Linux machine includes several optional steps.

Description

To deploy FireDAC application on a Mac OS X or a Linux machine the following items should be considered:

1. When an application uses persistent connection definitions (see page 27), then a connection definition file must be deployed. It may be put along with application executable or set TADCustomManager.ConnectionDefFileName (see page 371) to a file name. See additionally Installing FireDAC (see page 159) for Mac OS and Linux optional steps.
2. When an application uses driver definitions (see page 31), then a driver configuration file must be deployed. It may be put along with application executable or set TADCustomManager.DriverDefFileName (see page 373) to a file name. See additionally Installing FireDAC (see page 159) for Mac OS and Linux optional steps.
3. The DBMS clients required for application connectivity (see page 188) must be obtained from the vendors and installed on the deployment machine. When an application is built as x86, then the x86 DBMS client software must be installed. When as x64, then x64 software.

See Also

Defining Connection (see page 27), Configuring Drivers (see page 31), Database Connectivity (see page 188)

1.14 Utilities

The list of the FireDAC utilities, their usage details and command line parameters. FireDAC utilities are located in FireDAC\Bin folder.

1.14.1 ADAdministrator

All about FireDAC Administrator.

Description

The FireDAC Administrator is the major tool to manage the FireDAC connections. It has similar to ADExplorer (see page 181) functionality, but does not allow to browse database or work with SQL scripts. ADAdministrator may be distributed with application to give the administrators or to the users an ability to setup the database connections.

See Also

ADExecutor (see page 180), ADExplorer (see page 181), Database Connectivity (see page 188)

1.14.2 ADDFMChanger

All about automatic BDE -> FireDAC migration utility.

Description

Note, ADDFMChanger is not currently available in Embarcadero FireDAC v 7.0.1.

General

The FireDAC ADDFMChanger is a console application, aided for semi-automatic:

- conversion of a BDE application into an FireDAC application;
- migration of an FireDAC application between two FireDAC versions, with incompatible API.

The idea behind ADDFMChanger is to replace names of BDE components and properties with appropriate FireDAC names.

Note: The ADDFMChanger utility and the BDE2FireDAC.txt files are located in the Bin sub-folder of the FireDAC installation folder.

Command Line Parameters

Run the ADDFMChanger utility without any arguments. It will output the following reference text:

```
DFM-Changer 2.3
Copyright (c) 2000-2013 by DA-SOFT Technologies (www.da-soft.com)
All Rights Reserved.

Use: ADDFMChanger {<filesToProcess>} [-s] [-i] [-a] -f <RuleFile>
-s - recurse subdirectories
-i - ignore errors
-f - path to filter files
-a - migrate all occurrences; otherwise, only components will be migrated

Example: ADDFMChanger x:\myDir\*.pas x:\myDir\*.dfm -s -a -f x:\MyRules.txt
changes all pas and dfm files in myDir incl. subdirs according x:\MyRules.txt
```

Usage

In the command line you define where the files to convert are located. The sample below illustrates the migration of all PAS and DFM files in the directory "x:\myDir\":

```
ADDFMChanger.exe x:\myDir\*.pas x:\myDir\*.dfm [...]
```

You also have to define the location of the rule file. The BDE->FireDAC rule file is located in Bin\BDE2FireDAC.txt:

```
ADDFMChanger x:\myDir\*.pas x:\myDir\*.dfm -s -a -f BDE2FireDAC.txt
```

The rule file consists of a few lines; each of them looking like:

```
[old name]    -> [new name]         -> [add this unit to interface USES clause]
```

There "unit" may be a coma separated list of the units. An example of a translation rule in the rule file is shown bellow:

```
[TStoredProc] -> [TADStoredProc]     -> [uADCompClient]
```

This will replace TStoredProc with TADStoredProc and add uADCompClient unit to USES clause.

The completion code

When the utility finishes conversion successfully, it returns exit code 0. Otherwise the exit code is > 0.

See Also

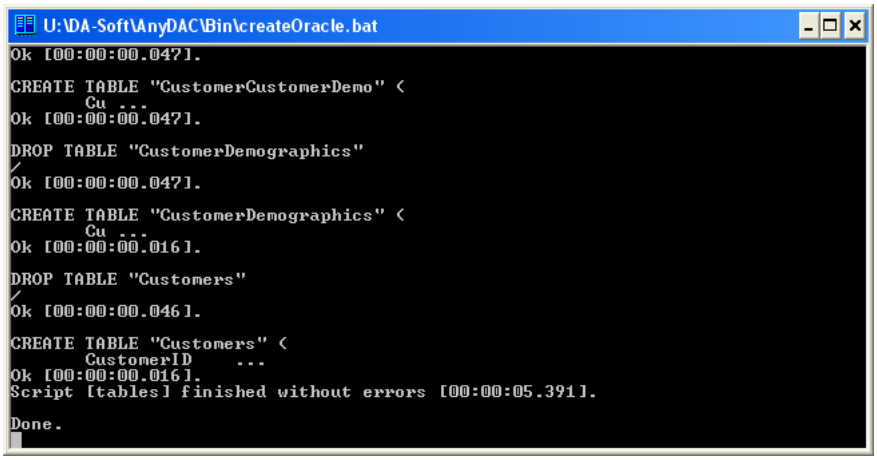
Migrating BDE applications (see page 163)

1.14.3 ADExecutor

All about FireDAC SQL Script execution utility.

Description

General



FireDAC Executor is the major SQL script console execution tool, supporting many SQL scripting dialects. The ADExecutor supports:

- standard SQL commands, like INSERT or SELECT;
- stored procedure language commands, like CREATE PROCEDURE;
- script execution control commands, like SPOOL, SET ECHO, etc., coming from Oracle SQL*Plus, Microsoft ISQL, and other script execution utilities, suplied by DBMS vendors.

FireDAC itself uses ADExecutor to build demo databases (see page 14). Users can use ADExecutor as a ready-to-use tool to deploy database objects.

Command Line Parameters

Run Executor utility with -? argument. It outputs the following reference text:

```
FireDAC Executor v 3.0.1 (Build 1337)
(c) Embarcadero Technologies 2004-2013
http://www.embarcadero.com/products/rad-studio/firedac

Use: ADExecutor [-d <name>] [-n <file name>] [-u <user>] [-w <pwd>] [-l] [-e]
             [-i] [-s] [-p <path>] [-t <path>] [{<scripts>}] [-a {<arguments>}]

-d        - connection definition name
-n        - connection definitions file name
-u        - user name
-w        - password
-l        - login prompt
-p        - path to SQL script files
-t        - path to data files
-e        - stop script execution only after a "drop non-existing object" error
-i        - stop script execution after first error
-s        - do not show messages during SQL script execution (silent)
-a        - a list of script arguments
-? or -h  - show help
```

```
Notes:
1. If scripts are not specified, ADExecutor will read standard input.

Examples:
1. ADExecutor -d Oracle_Demo -i -p x:\MyScripts s1.sql s2.sql
Executes the s1.sql and s2.sql scripts from directory x:\MyScripts, using
the Oracle_Demo connection definition, does not stop on errors.
2. echo drop table mytab | ADExecutor -d MySQL_Demo
Executes the command captured from standard input.
```

See Also

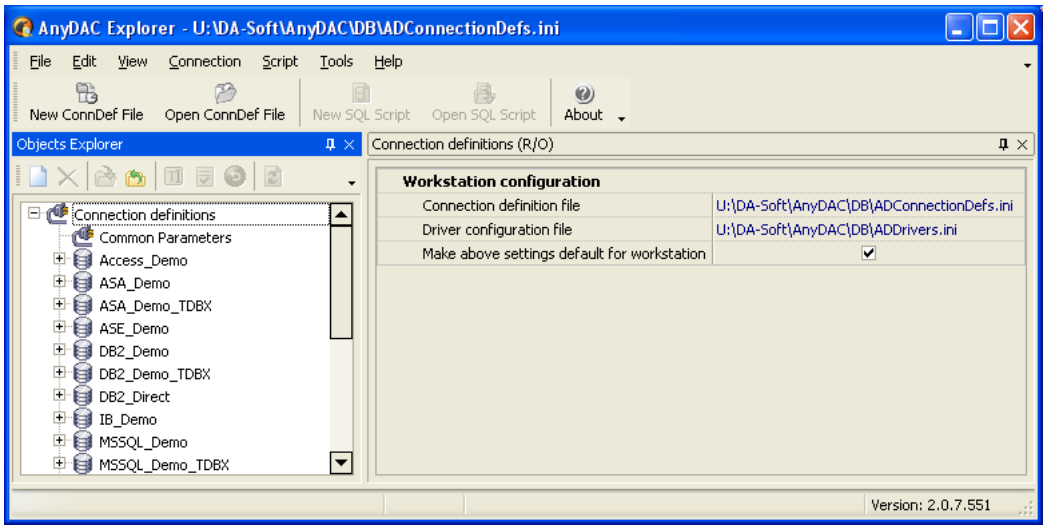
ADExplorer (see page 181), Database Connectivity (see page 188), Demo Databases (see page 14), Executing SQL Scripts (see page 88), SQL Script Control Commands (see page 92)

1.14.4 ADExplorer

All about FireDAC Explorer.

Description

General



The ADExplorer is a hierarchical database connection browser with data editing and SQL script execution capabilities. It is comparable to the "BDE Database Explorer". The ADExplorer enables you to:

- Manage FireDAC default parameters;
- Create, edit, load and save connection definition files (see page 27);
- Create, edit and delete connection definitions (see page 27);
- Browse database specific schema objects, including tables, fields, indexes, primary keys, stored procedure definitions or triggers;
- View, append and edit data in existing tables;
- Create, edit, load, save and execute SQL scripts to query, alter, test and deploy a database.

The ADExplorer utility is a standalone program and can be launched by:

- The Delphi IDE menu (Tools -> FireDAC -> Explorer);
- The FireDAC Startup menu.

Note: The ADAdministrator (see page 178) can also be used to maintain the connection definitions. It is comparable to the

"BDE Admin". ADAdministrator is actually a restricted version of the ADExplorer that does not allow to run SQL commands or to browse or edit data. The Administrator is intended to be deployed together with your applications.

Manage the Default Parameters

Select the "Connection definitions" tree item to manage FireDAC default parameters:

- Connection definition file - default name of the connection definition (see page 27) file;
- Driver configuration file - default name of the driver configuration (see page 31) file.

If the specified values are default values for your workstation, the checkbox is marked. You can edit parameter values and mark the checkbox to save the new values. They will be stored in the registry key HKCU\da-soft\FireDAC.

Manage Connection Definition Files

When you run the ADExplorer, it opens and shows the content of the default connection definitions file. Use buttons or main menu items:

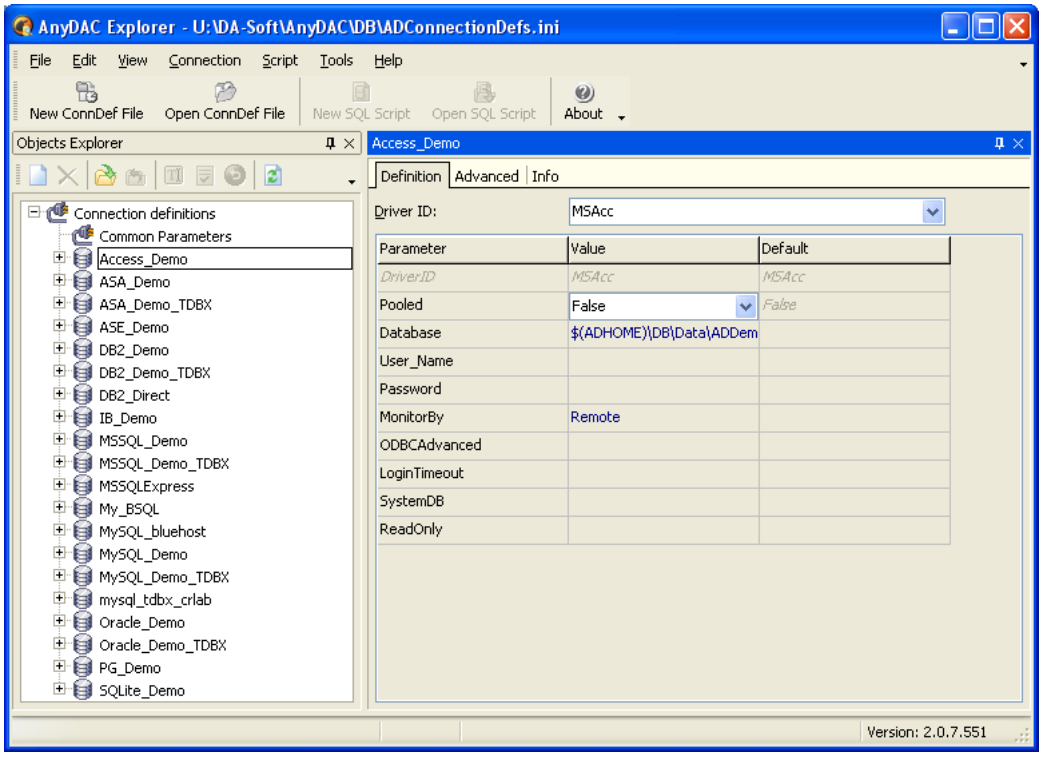
- New ConnDef File: Creates a new empty connection definitions file.
- Open ConnDef File: Opens an existing connection definitions file.

Manage the Connection Definitions

To manage connection definitions (see page 27) in the current open connection definitions file, use the toolbar buttons on the "Objects Explorer" pane:

- Connection Definition (Ctrl+N): Creates a new empty connection definition.
- Delete the current object (Ctrl+Del): Permanently deletes the currently selected connection definition.
- Open current object (or click on plus sign in tree): Activates a connection to the database using the currently selected connection definition.
- Close current object: Closes the connection to the database.
- Rename current object (Ctrl+M): Renames the currently selected connection definition.
- Save changes to current object (Ctrl+A): Permanently saves changes to the currently selected connection definition.
- Cancel changes of current object (Ctrl+Backspace): Permanently cancels changes made to the currently selected connection definition.

Edit the Connection Definitions



To edit the connection definition, choose it in the tree. On the right you will see the Connection Definition Editor. It has three panes:

- **Definition**: Allows you to edit the main connection parameters. First, select a Driver ID from the drop-down list on top. After that, the editor will become available for the selected driver parameters. For more information about the parameters see FireDAC Database Connectivity. After you finish editing, click the connection definition in the left tree again. Now the connection definition is modified, but the changes are not yet permanently stored in the connection definition file. Press Ctrl+A to store them.
- **Advanced**: Allows you to edit advanced connection options.
- **Info**: Shows all information associated with the current connection definition, including a full set of parameters and FireDAC version info. If the connection to the database is established, the page shows additional client and server information.

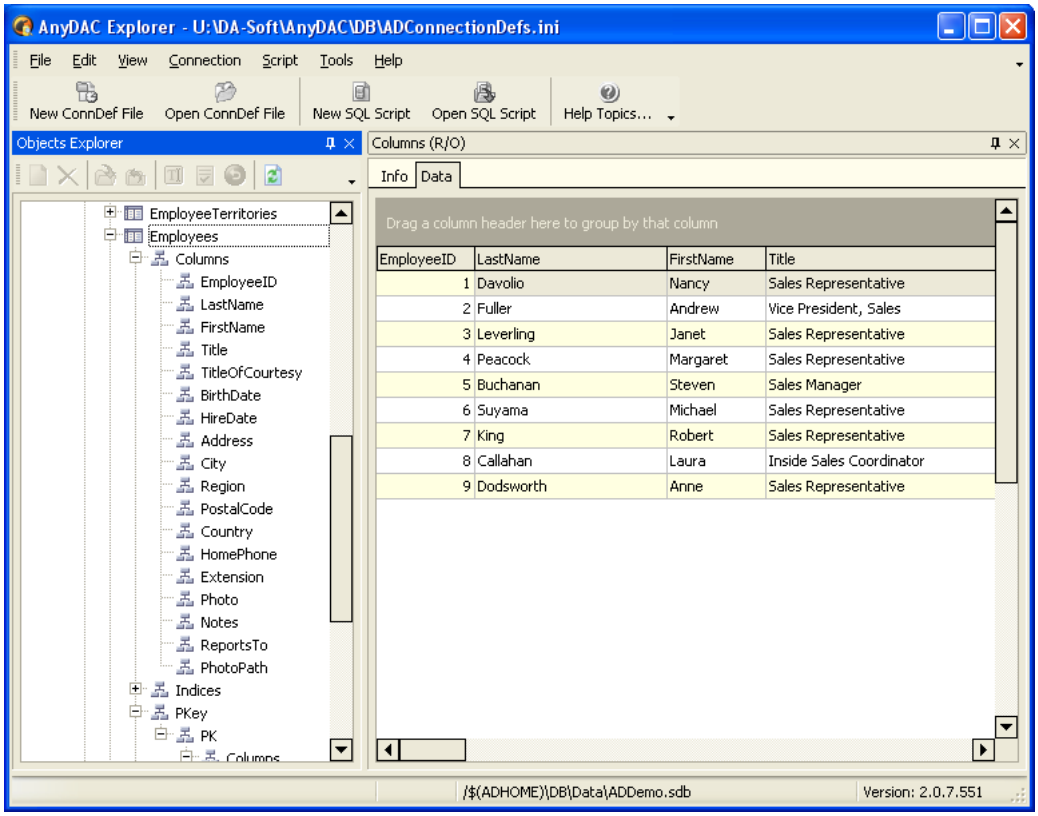
Note: Use the Info page to provide information to the FireDAC technical support team. Also, the following commands are accessible in the "Connection" menu:

- **Run Wizard** (Ctrl+W): Runs the connection definition wizard, if it is implemented for the driver. Currently only Microsoft SQL Server, Microsoft Access, IBM DB2, Sybase SQL Anywhere and ODBC Bridge drivers implement the wizard.
- **Test** (Ctrl+T): Tries to establish a connection to a DBMS without opening a connection definition node.
- **Make BDE Compatible** (Ctrl+B): Applies a data type mapping schema, compatible with BDE, to the connection definition.
- **Import BDE Aliases**: Allows you to import BDE aliases into the current connection definitions file.

And in "Help" menu:

- **Help Topics** (F1): Opens a help topic with descriptions of the DriverID driver parameters.

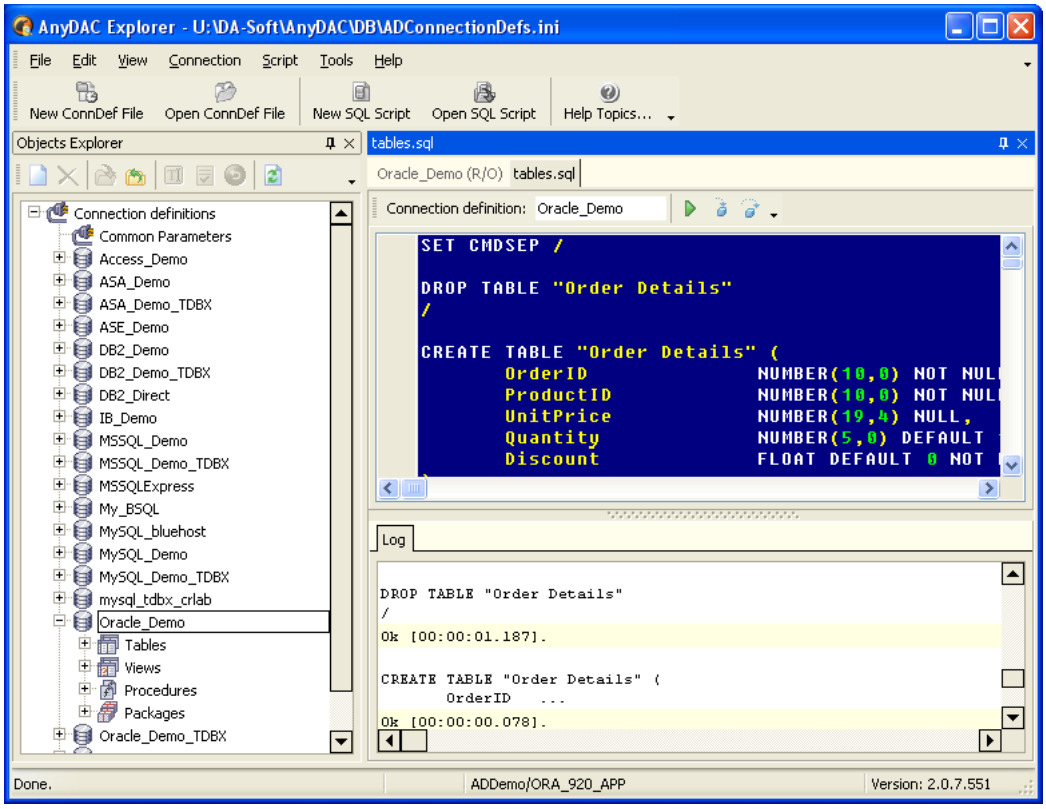
Browse the Database Contents



To browse database specific schema objects, including tables, fields, indexes, primary key, stored procedure definitions, triggers, sequences, etc., select connection definition and open it. After that, you can drill down into the database structure. The exact set of accessible objects depends on the DBMS. To see the object metadata, select it in the tree and switch to the "Info" pane on the right.

To view, append and edit data in existing tables, select table or view in the tree and switch to the "Data" pane on the right. Use **View** -> **Blob Viewer** or double click on the DB grid cell to show the content (Text, Binary data, Image, HTML, etc) of a BLOB field.

Run a Query against a Database



To query, alter, test and deploy a database you can use a SQL Script window. Select the appropriate connection definition in the tree and open it. Use buttons or main menu items:

- **New SQL Script**: Creates a new empty SQL script file.
- **Open SQL Script**: Opens an existing SQL script file.

In the SQL script window use buttons on the window toolbar or main menu items:

- **Run Script** (F9): Executes a script in full, from first to last command.
- **Run Command** (F7): Executes the next command from the current cursor position in the SQL script editor.
- **Skip Command** (Shift+F7): Skips the next command from the current cursor position in the SQL script editor.

See the ADExecutor (see page 180) description for a list of support script execution control commands, or just press F1.

See Also

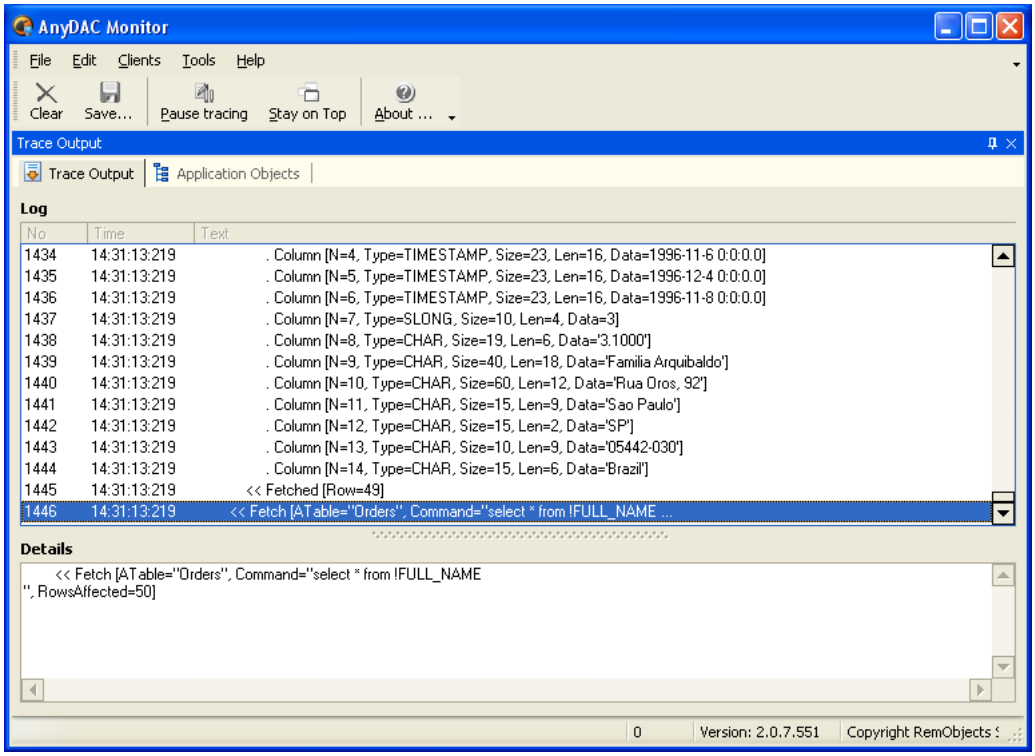
ADExecutor (see page 180), ADAdministrator (see page 178), Database Connectivity (see page 188), Defining Connection (see page 27)

1.14.5 ADMonitor

All about FireDAC interactive application monitoring utility.

Description

General



The FireDAC Monitor is a debugging tool for:

- Tracing a communication between an FireDAC application and a DBMS.
- Exploring details of commands, parameters, states, execution statistic, etc.

FireDAC remote monitoring uses TCP/IP as a transport for application output. That means, Monitor may be running on the same or on another computer than the application itself. Also, single Monitor may receive output from many FireDAC applications. To enable monitoring for particular a connection definition, set its parameter "MonitorBy" to the value "Remote".

ADMonitor is an analog to the "BDE SQL Monitor".

See Also

Tracing and Monitoring (see page 174)

1.14.6 Compile (tool name).bat

All about automatic recompiling FireDAC packages.

Description

General

The set of compile<tool name>.bat files allows you to automatically rebuild the FireDAC Win32 packages. That may be required, when you have:

- updated FireDAC source files, received from Embarcadero;
- changed FireDAC source files;

- installed IDE Service Pack, which breaks package compatibility;
- installed/removed/updated SynEdit.

Supported Tools

Tool	BAT file
Delphi 5	compileD5.bat
C++ Builder 5	compileBCB5.bat
Delphi 6	compileD6.bat
C++ Builder 6	compileBCB6.bat
Delphi 7	compileD7.bat
Delphi 2005	compileD2005.bat
Delphi 2006	compileD2006.bat
Delphi / C++ Builder 2007	compileD2007.bat
Delphi / C++ Builder 2009	compileD2009.bat
Delphi / C++ Builder 2010	compileD2010.bat
Delphi / C++ Builder XE	compileDXE.bat
Delphi / C++ Builder XE2	compileDXE2.bat
Delphi / C++ Builder XE3	compileDXE3.bat

Note: FireDAC does not support command line compiling for FPC or other than Win32 platforms. You should do that from the IDE.

Preconditions

- Close the running IDE;
- Close any running FireDAC applications built with run time packages;
- If you have SynEdit installed/updated, then check:
  - SynEdit design time package must be installed into IDE;
  - path to the SynEdit source files must be included into Library Search Path;
  - file Bin_noSynedit.txt does not exists;

Performed Actions

Each of batch files performs the following actions:

- Detect current environment and generate configuration files:
  - Bin_COMPILEENV.BAT;
  - Source\uADEnv.inc;
- Recompile all packages;
- Move DCU, OBJ to Lib\<tool name>;
- Move BPL, DCP to Package and DCP Output Path's, specified in IDE.

Note: compile<tool name>.bat does not install FireDAC design time packages (AnyDAC_Dcl_<tool name>) into the IDE. If that is required, you should do that manually.

See Also

Programming Tools (see page 26)

1.14.7 Create (DB name).bat

All about automatic FireDAC demo database creation.

Description

The set of create<db name>.bat files allows to automatically create FireDAC demo database for appropriate database. See FireDAC Demo Databases (see page 14) for more details.

See Also

ADExecutor (see page 180), ADExplorer (see page 181), Demo Databases (see page 14), Database Connectivity (see page 188)

1.15 Database Connectivity

The list of the DBMS's supported by FireDAC and corresponding connection definition parameters.

1.15.1 Common connection parameters

Describes the connection definition parameters common for all FireDAC drivers.

Description

Parameter	Description	Example value
DriverID	The FireDAC base or virtual driver ID.	MySQL
MetaDefCatalog	Specifies the default catalog for the application. The design time code will omit the catalog name in the object names, if it is equal to MetaDefCatalog. If MetaDefCatalog is '*', then catalog names will be omitted unconditionally.	Northwind
MetaDefSchema	Specifies the default schema for the application. The design time code will omit the schema name in the object names if it is equal to MetaDefSchema. If MetaDefSchema is '*', then schema names will be omitted unconditionally.	dbo
MetaCurCatalog	Specifies the current catalog for the application. If not specified, then its value will be received from the DBMS. When an application is asking for metadata and do not specify a catalog name, then FireDAC will implicitly use the current catalog. If MetaCurCatalog is '*', then catalog names will be me omitted from the metadata parameters.	*
MetaCurSchema	Specifies the current schema for the application. If not specified, then its value will be received from the DBMS. When an application is asking for metadata and do not specify a schema name, then FireDAC will implicitly use the current schema. If MetaCurSchema is '*', then schema names will be me omitted from the metadata parameters.	*

MonitorBy	Specifies the trace / monitor output (see page 174) for this connection definition. One of the following values may be specified: • **FlatFile**. The trace output will be put to a text file. To enable that, include TADMoniFlatFileClientLink into your application and set it Tracing property to True. FireDAC will show the name of a trace file after an application is finished. • **Remote**. The trace output will be sent to the ADMonitor utility. As it may be used to monitor your application. To enable that, include TADMoniRemoteClientLink into your application and set it Tracing property to True. The ADMonitor must be run before connection will be established. • **Custom**. The trace output will be send to TADMoniCustomClientLink.OnOutput event handler. To enable that, include TADMoniCustomClientLink into your application and set it Tracing property to True.	FlatFile
Pooled	Enables connection pooling (see page 46) for this connection definition. The default value is False. To use pooled connection, the connection definition must be persistent or private (see page 27).	True

See Also

Working with Metadata (see page 127), Defining Connection (see page 27), Multi Threading (see page 46), Tracing and Monitoring (see page 174)

1.15.2 Connect to Advantage Database Server

Describes how to connect to Advantage Database Server.

Description

Supported versions

The FireDAC native driver supports the Advantage Database Server v 8.0 or higher.

Windows client software

FireDAC requires the **ACE32.DLL** x86 or **ACE64.DLL** x64 client library (**Advantage Client Engine API)** for connecting to the Advantage server or to make a local connection. Ideally, the version should be equal to the server version. The full set of the v 10.0 client files:

- ace32.dll
- adsloc32.dll
- aicu32.dll
- axcws32.dll
- adscollate.adm
- adscollate.adt
- ansi.chr
- extend.chr
- icudt40l.dat
- adslocal.cfg

You can download the Advantage Client Engine API from there. The installation folder includes "Redistribute" folder, containing all required redistributable files. The files may be put into a folder:

- listed in your PATH environment variable;

- your application EXE folder;
- any other folder and specify in FireDAC\DB\ADDrivers.ini:

```
[ADS]
VendorLib=<folder>\ace32.dll
```

When the ADS client library has not been properly installed, then you will get an exception when you try to connect:

```
[FireDAC][Phys][ADS]-314. Cannot load vendor library [ACE32.dll].
The specified module could not be found.
Check it is located in one of the PATH directories or in the application EXE directory
```

Connection definition parameters

DriverID=ADS

Parameter	Description	Example value
ServerTypes	Value is a sum of values indicating the types of Advantage Servers to which connections are attempted. The values for the servers are: • ADS_REMOTE_SERVER = 2 • ADS_LOCAL_SERVER = 1 • ADS_AIS_SERVER = 4. Also the keywords may be used instead of numbers: • **Remote** • **Local** • **Internet** Few types may be concatenated by '|'. The default depends on the Server and Database values: • When they specify the local path with a driver letter, then "Local". • When a remote path, then "Remote" and "Internet".	For example, to allow the Driver to use the remote or local server, but not the Advantage Internet Server, use:ServerTypes=3 (1+2).
Protocol	Specifies the communication protocol used to connect to the Advantage Database Server: • **UDP**; • **IPX**; • **TCPIP** (default value); • **TLS.**	TCPIP
Server	The server address.	\\ADS
Port	The server port.	6262

Database	The fully qualified path to the computer where the data files exist and the default location of the data files. This fully qualified path must contain a drive letter or use UNC. "Database" may include Server and Port values. For remote server types the path must either use UNC or refer to the existing network share. For "free connections", it should be a valid path name to where the data files are located (e.g., x:\data). This path is used to automatically select all tables in the specified directory. For "database connections", it should be a valid path name including the Advantage Data Dictionary file name (e.g., x:\database\mydictionary.add). A path may include path variables (see page 163).	c:\ads\data \\ADS:6262\DB
Alias	Specifies the alias to be used when connecting to the database server. The file path associated with the specified alias will be used as the database directory. The table type associated with the alias will be set as the default table type for use with this connection. See Database Aliases and the ads.ini File for a full description of an Alias.	
CharacterSet	An optional collation language used when opening tables.	ANSI
Compress	Specifies the option for communications compression. Valid values are: • **Internet**; • **Always**; • **Never**.	Internet
TableType	Sets the type of database files to use: • **ADT** - Advantage-proprietary ADT/ADI/ADM files; • **VFP** - FoxPro-compatible DBF/CDX/FPT files; • **CDX**; • **NTX**. This setting is ignored for database connections.	ADT
TablePassword	Specifies encryption password for all tables in the connection. Applicable for free connections only.	
Locking	Specifies the locking mode to use when opening DBF tables. Valid values include: • **Proprietary** - Advantage's high-performance internal locking mode is used; • **Compatible** - DBF tables can be shared in a writeable mode with non-Advantage database applications.	Proprietary
ADSAdvanced	Allows to specify any other additional ACE API AdsConnect101 connection parameter value.	EncryptDictionary=True;DDPassword=qwe

Usage cases

- Open local ADS database in "free connection" mode:

```
DriverID=ADS
Database=c:\ads\data
```

- Open local ADS database in "free connection" mode with Visual FoxPro tables:

```
DriverID=ADS
Database=c:\ads\data
TableType=VFP
```

- Open a remote data dictionary based connection:

```
DriverID=ADS
ServerTypes=Remote
Protocol=TCPIP
Database=\\DA\ADS_DB\addemo.add
User_Name=adssys
Password=a
```

- Open local ADS database in "free connection" mode using an alias:

```
DriverID=ADS
Alias=MyConn
```

And the ads.ini content:

```
[Databases]
MyConn=c:\data;N
```

See Also

Common connection parameters (see page 188), uADPhysADS Namespace (see page 724), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.3 Connect to Blackfish SQL Server

Describes how to connect to Embarcadero Blackfish SQL Server.

Description

Blackfish SQL Server Overview

FireDAC does not provide a native Blackfish SQL Server driver, but you can use the FireDAC TDBX Bridge driver (see page 194) to connect to the Blackfish SQL Server. You should use the TDBX (DBX 4) Framework and the Blackfish SQL driver, which comes with CodeGear RadStudio 2007 or higher.

Connect using the FireDAC TDBX Bridge

To connect to the Blackfish SQL Server do the following:

- Open the file "C:\Documents and Settings\All Users\Documents\RAD Studio\dbExpress\dbxdrivers.ini". Find the [BlackfishSQL] section and modify it:
  - add "HostName=ServerName" ;
  - replace "Create=False" with "create=False".
- Create a connection definition, similar to this example. Please note that the "create" parameter must be specified using lower case characters.

```
DriverName=BlackfishSQL
DriverID=TDBX
hostname=127.0.0.1
port=2508
database=c:\addemo
create=True
user_name=sysdba
password=masterkey
```

- Drop a TADConnection component on the form.
- Set ADConnection1.ConnectionDefName to My_BSQL.

See Also

Common connection parameters (see page 188), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.4 Connect to DataSnap server

Describes how to connect to Multi-Tier DataSnap servers.

Description

Supported versions

The Multi-Tier FireDAC DataSnap driver supports:

- DataSnap servers built with RAD Studio 2007 and higher;
- DataSnap clients built with RAD Studio XE2 Enterprise and higher.

Client software

FireDAC statically links Embarcadero DataSnap dbExpress driver, so no additional files and DLLs is required. Supported is RAD Studio XE2 Enterprise and higher.

Driver linkage

To link the DataSnap driver:

- drop a TADPhysDataSnapDriverLink (see page 749) component from the "FireDAC Links" palette page;
- or include the uADPhysDataSnap (see page 749) unit into an uses clause.

Connection definition parameters

To connect to the DataSnap server, most applications will need to specify **DriverID**, **Protocol**, **Server**, **Port**, **User_Name**, **Password**.

DriverID=DataSnap

Parameter	Description	Example value
Protocol	The protocol used to connect to the DataSnap server. It may be one of the following values: • TCP/IP. Connect using TCP/IP protocol. Is default protocol. • HTTP. Connect using HTTP protocol.	TCP/IP
Server	Server address to connect to.	127.0.0.1
Port	The TCP/IP or HTTP port on which the DataSnap server is listening. The default value for TCP/IP is 211, for HTTP - 8080.	211
User_Name	DataSnap server authentication user name.	
Protocol	DataSnap server authentication user password.	

BufferKBSize	Buffer size in kilobytes to use for read and write operations. The default value is 32Kb.	64
Filters	Client filters, used to store the filter configuration. The default value is empty.	
URLPath	HTTP only. URL path used for HTTP DataSnap Service when http protocol is used. The default value is empty.	
DatasnapContext	HTTP only. Path toward the DS HTTP Service, used to compose the URL. The current convention is: http://x.com/datasnap/provider/classname/method/params. The user may change or delete datasnap word from it. The default value is 'datasnap/'.	
DSProxyHost	HTTP only. The host to proxy requests through, or empty string to not use a proxy. The default value is empty.	
DSProxyPort	HTTP only. The port on the proxy host to proxy requests through. Ignored if DSProxyHost isn't set. The default value is empty.	
DSProxyUsername	HTTP only. User name for proxy authentication. The default value is empty.	
DSProxyPassword	HTTP only. Password for proxy authentication. The default value is empty.	
DSAuthenticationScheme	HTTP only. DataSnap authentication scheme. Used in conjunction with DataSnap user name and password for authentication using the HTTP protocol. Set to "**basic**" in order to send User_Name / Password values using basic HTTP authentication, in addition to sending these values in the DBX connection string. Basic HTTP authentication may be used to pass credentials to an inter-process DataSnap HTTP tunnel server. The default value is empty.	
LoginTimeout	HTTP only. Connect timeout value. The values provides the number of milliseconds the client waits for the connection to be possible. The value provides the time out for the first server response acknowledgment rather than for the entire connect/authenticate phase. It should be used in order to avoid application freeze when it may be possible to attempt connections to older DataSnap Server versions or different application that will not acknowledge current communication protocol. The default value is empty.	
CommunicationTimeout	HTTP only. Timeout value in milliseconds for a response after the connection is established. The default value is empty.	

Usage cases

- Connect to local DataSnap server with default settings:

```
DriverID=DataSnap
```

- Connect to DataSnap server using TCP/IP protocol running on local host:

```
DriverID=DataSnap
Protocol=tcp/ip
Server=127.0.0.1
Port=211
User_Name=dsusr
Password=123
```

See Also

Common connection parameters (see page 188), uADPhysDataSnap Namespace (see page 749), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.5 Connect to dbExpress data source

Describes how to connect to dbExpress data source.

Description

Supported versions

The FireDAC DBX bridge driver supports:

- dbExpress v 1-3, accessible in Delphi versions 6 - 2005;
- dbExpress v 4 also called "DBX framework", accessible in Delphi 2007 and higher.

Windows client software

FireDAC requires that a dbExpress driver must be installed according to the topic "Deploying dbExpress Database Applications" of the Delphi/RAD Studio online help. Additionally you can find:

- the portional list of accessible 3d party drivers here;
- dbExpress v 1-3 specification here;
- dbExpress v 4 overview here.

Driver linkage

To link the dbExpress v 1-3 driver:

- drop a TADPhysDBXDriverLink (see page 752) component from the "FireDAC Links" palette page;
- or include the uADPhysDBExp (see page 751) unit into an uses clause.

To link the dbExpress v 4 driver:

- drop a TADPhysTDBXDriverLink (see page 829) component from the "FireDAC Links" palette page;
- or include the uADPhysTDBX (see page 829) unit into an uses clause.

Connection definition parameters

Connection definition parameters are specific for each driver and are taken from dbxdrivers.ini. The connection definition editor, after specifying the **DriverName** parameter, fills the connection parameter list by the parameters specific for this driver. Additionally, FireDAC DBX bridge driver supports the following parameters:

- **DriverID=DBX** for dbExpress v 1-3;
- **DriverID=TDBX** for dbExpress v 4.

Parameter	Description	Example value
MetaDefCatalog	Specifies the default catalog for the application. The design time code will omit the catalog name in object names if it is equal to MetaDefCatalog.	Northwind
MetaDefSchema	Specifies the default schema for the application. The design time code will omit the schema name in object names if it is equal to MetaDefSchema.	dbo

Usage cases

- Connect to Blackfish SQL. Note the parameters case:

```
DriverID=TDBX
DriverName=BlackfishSQL
hostname=127.0.0.1
port=2508
```

```
database=c:\addemo
create=True
user_name=sysdba
password=masterkey
```

- Connect to Oracle Database:

```
DriverID=TDBX
DriverName=Oracle
RDBMS=ORACLE
Database=ORA_920_APP
User_Name=addemo
Password=a
```

- Connect to Informix Dynamic Server:

```
DriverID=TDBX
DriverName=Informix
HostName=ol_svr_custom
Database=sysuser
User_Name=informix
Password=informix2
```

See Also

Common connection parameters (see page 188), uADPhysTDBX Namespace (see page 829), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27), Programming with dbExpress 4

1.15.6 Connect to IBM DB2 Server

Describes how to connect to IBM DB2 Server.

Description

Supported versions

The FireDAC native driver supports IBM DB2 Enterprise, Workgroup or Express editions version 8 and higher. We recommend v 8.2 at least. You can also work with other IBM database products, using FireDAC ODBC or DBX Bridge drivers.

Windows client software

FireDAC requires the "**IBM DATA SERVER DRIVER for ODBC**", "**IBM DB2 ODBC DRIVER**" or "**IBM DB2 DRIVER FOR ODBC**" x86 or x64 ODBC driver to be installed on the workstation. It may be downloaded from:

- for x86;
- for x64.

After downloading and unpacking run "<client>\bin\db2oreg1 -i" to install ODBC driver. Additionally, download and install fix packs.

When the DB2 ODBC driver has not been properly installed, you will get an exception when you try to connect:

```
[FireDAC][Phys][ODBC][Microsoft][ODBC Driver Manager] Data source name not found and no
default driver specified
```

Driver linkage

To link the driver:

- drop a TADPhysDB2DriverLink (see page 751) component from the "FireDAC Links" palette page;
- or include the uADPhysDB2 (see page 750) unit into the **uses** clause.

Connection definition parameters

There are two methods of specifying the DB2 connection attributes. If the DB2 client software has the DB2 connection aliases configured, an application should use the **Alias** parameter. If no aliases are configured, then you should use the **Protocol**, **Server**, **Port** and **Database** parameters. Also specify **User_Name** and **Password**.

DriverID=DB2

Parameter	Description	Example value
Alias	Connection alias.	MyDB2Srv
Server	Host name, if Alias is not specified.	127.0.0.1
Port	Port value, if Alias is not specified.	5000
Database	Database name, if Alias is not specified.	ADDEMO
Protocol	Protocol name, if Alias is not specified.	TCPIP
User_Name	The DB2 user name.	db2admin
Password	The DB user password. Note, the passwords with both '{' and '}' are not supported.	master
LoginTimeout	Controls the amount of time in seconds an application waits for a connection attempt to timeout while waiting to establish a connection (0 specifies an infinite wait).	30
StringFormat	Defines how to represent String values: • **Choose** - represent as ftString / ftWideString / ftMemo / ftWideMemo, depending on the declared data type name (default); • **Unicode** - always represent as ftWideString / ftWideMemo.	Unicode
ExtendedMetadata	Controls the extended describing (see page 129) of the query result sets: • **True** - FireDAC describes a result set to get all the possible column attributes - is readonly, is auto incrementing, column origin, etc. Setting this option to True, slightly slows down a dataset opening. • **False** - FireDAC uses the restricted information about the query columns (default).	True
ODBCAdvanced	Allows to specify any other additional ODBC connection parameter value. The default value is "IGNOREWARNINGS=1".	
MetaDefSchema	Default schema name. Design time code will exclude schema name from object name, if it is equal to MetaDefSchema.	db2admin

Usage cases

- Connect to DB2 using existing database alias:

```
DriverID=DB2
Alias=addemo
User_Name=db2admin
Password=mypwd
MetaDefSchema=db2admin
```

- Connect to DB using full connection information:

```
DriverID=DB2
Server=127.0.0.1
Database=addemo
Port=50000
Protocol=TCPIP
User_Name=db2admin
Password=mypwd
MetaDefSchema=db2admin
```

See Also

Common connection parameters (see page 188), uADPhysDB2 Namespace (see page 750), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.7 Connect to Interbase or Firebird

Describes how to connect to Firebird or Embarcadero Interbase Server's.

Description

Supported versions

The FireDAC native driver supports:

- Embarcadero InterBase server version 6 and higher;
- Firebird server version 1.5 and higher. Embedded edition is also supported.

FireDAC distinguishes these DBMS brands and adjusts its own behavior to the connected database. It does not officially support Jaffil.

Windows client software

FireDAC requires the following x86 or x64 client software to be installed on the workstation:

- **FBCLIENT.DLL** library to connect to the Firebird server. You can take it from a server (download) installation Bin folder.
- **FBEMBED.DLL** library to work with the database using the Firebird Embedded server (details).
- **GDS32.DLL** library to connect to the InterBase server.

Note: It is critical to use the appropriate DLL for the DBMS client library. Do not use GDS32.DLL with Firebird or FBCLIENT.DLL with Interbase.

You can put the required files into a folder:

- listed in your PATH environment variable (for example, <Windows>\SYSTEM32);
- your application EXE folder;
- any other folder and specify in FireDAC\DB\ADDrivers.ini:

```
[IB]
VendorLib=<folder>\fbclient.dll
```

When the Interbase or Firebird client library has not been properly installed, you will get an exception when you try to connect:

```
[FireDAC][Phys][IB]-314. Cannot load vendor library [gds32.dll]. The specified module could
not be found.
Check [gds32.dll], which is located in one of the PATH directories or in the application
EXE directory.
```

Linux client software

FireDAC requires:

- the **libfbclient.so** x86 or x64 client library.

To install on Linux use the commands:

```
sudo apt-get update
sudo apt-get install libfbclient2
sudo ln -s /usr/lib/libfbclient.so.2.5.0 /usr/lib/libfbclient.so
```

Mac OS X client software

FireDAC requires:

- the **libfbclient.dylib** x86 client library to connect to Firebird server. You can take it from a server (download) installation /Library/Frameworks/Firebird.framework/Libraries folder.
- the **libgds.dylib** x86 client library to connect to Interbase server. With RAD Studio XE4 use Project -> Deployment -> Add Featured Files -> Interbase Client\OSX32.
- the **libibtogo.dylib** x86 embedded Interbase server. With RAD Studio XE4 use Project -> Deployment -> Add Featured Files -> Interbase ToGo\OSX32.

iOS client software

FireDAC requires:

- For iOS Simulator - the **libibtogo.dylib** x86 embedded Interbase server. With RAD Studio XE4 use Project -> Deployment -> Add Featured Files -> Interbase ToGo\iOSSimulator.
- For iOS Device - the **libibtogo.a** embedded Interbase server as ARM static library. With RAD Studio XE4 use Project -> Deployment -> Add Featured Files -> Interbase ToGo\iOSDevice.

Note, that it is impossible to choose the linkage mode.

Driver linkage

To link the driver:

- drop a TADPhysIBDriverLink (see page 777) component from the "FireDAC Links" component palette page;
- or include the uADPhysIB (see page 752) unit into the **uses** clause.

Connection definition parameters

To connect to the Interbase or Firebird DBMS, most applications will need to specify **DriverID**, **Protocol**, **Server**, **Database**, **User_Name**, **Password** and **CharacterSet**.

We strongly recommend to explicitly set the **CharacterSet** to:

- UTF8, if your application needs to support Unicode. See Unicode Usage (FireDAC) for details.
- WIN1250 for Central Europe;
- WIN1251 for Cyrillic;
- WIN1252 for Western Europe, America;
- etc

DriverID=IB

Parameter	Description	Example value
Database	The database name to attach. The value may be: • database file path; • full database path, including server address. For an embedded database a path may include path variables (see page 163).	• C:\ib\ADDEMO_IB2007.IB • 127.0.0.1:C:\ib\ADDEMO_IB2007.IB • \\MySrv\C:\ib\ADDEMO_IB2007.IB
OSAuthent	Controls authentication mode: • **Yes -** use windows authentication; • **No** - use DBMS authentication. It is the default value. See the doc\README.trusted_authentication.txt for details.	Yes
User_Name	The user name.	sysdba
Password	The user password.	masterkey
CharacterSet	The character set to use.	WIN1252
ExtendedMetadata	Controls extended describing (see page 129) of the query result sets: • **True** - FireDAC is getting column domain names additionally to the other column attributes. If a column belongs to a domain with a name like **%BOOL%**, it will be described as a **dtBoolean**. Also, if a table has an **INSERT trigger**, which reads a single sequence and assigns it value to a single column, then this column will be described as **auto-incrementing one**. Setting this option to True will slightly slow down a dataset opening. • **False** - FireDAC uses the restricted information about the query columns. It is the default value.	False
Protocol	The protocol used to connect to the DB server. It may be one of the following values: • **Local**. Connect to a locally running server, to an embedded server or to a server using an alias. • **NetBEUI**. Connect using Microsoft NetBIOS protocol. • **SPX**. Connect using Novel SPX protocol. • **TCPIP**. Connect using TCP/IP protocol. If the Protocol parameter is specified, FireDAC will build full database paths, using the appropriate Protocol format and Server and Database parameter values.	TCPIP
Server	Server address to connect to. Server parameter value is used only if the Protocol parameter is specified. To specify a TCP/IP port number use the <host>/<port> notation.	• 127.0.0.1 • my_host/3055
InstanceName	The Interbase 2007 instance name.	srv2
SQLDialect	The SQL Dialect to use for connecting. Three is the default value.	1
RoleName	The default role name.	Admin

CreateDatabase	Controls a new database create: • **Yes** - to create a new database file, specified in the Database parameter, right after connecting to the server. • **No** - do not create. It is the default value.	Yes
DropDatabase	Controls an existing database deletion: • **Yes** - to drop the database file, specified in the Database parameter, right after disconnecting from the server. • **No** - do not drop. It is the default value.	Yes
PageSize	The page size used for the newly created database, if CreateDatabase = Yes. 1024 is the default value.	4096
IBAdvanced	The ';' separated list of additional parameters. You can find the full list of supported parameters in the uADPhysIBWrapper unit, searching for the DPBInfos constant array. You can find the description of each Code listed there in the Interbase/Firebird manuals.	

Usage cases

- Connect to a database running on a remote server via TCP/IP protocol:

```
DriverID=IB
Database=C:\ib\ADDEMO_FB21.FB
Protocol=TCPIP
Server=IBSrv
User_Name=sysdba
Password=masterkey
CharacterSet=win1252
ExtendedMetadata=True
```

- Connect to a database running on a remote server via TCP/IP protocol:

```
DriverID=IB
Database=IBSrv:C:\ib\ADDEMO_FB21.FB
User_Name=sysdba
Password=masterkey
CharacterSet=utf8
```

- Connect to a local database:

```
DriverID=IB
Database=C:\ib\ADDEMO_FB21.FB
User_Name=sysdba
Password=masterkey
CharacterSet=win1251
```

- Connect to a Firebird Embedded database:

Drop the TADPhysIBDriverLink (see page 777) component to the form and set its **VendorLib** to <your path>\fbembed.dll or use virtual driver definition.

```
DriverID=IB
Database=C:\ib\ADDEMO_FB21.FB
User_Name=sysdba
CharacterSet=utf8
```

- Connect to a Interbase server using SSL:

For details on that see Chapter 5, starting in page 5-15 of the InterBase XE Operations Guide manual.

```
DriverID=IB
Database=my.interbasehost.net/3065?ssl=true?serverPublicFile=C:\PublicCertFileOnClient\CertF
ile.pem??:C:/DB/TEST.IB
User_Name=sysdba
Password=masterkey
CharacterSet=UTF8
ExtendedMetadata=True
```

- Connect to a database file in Documents folder on iOS device or emulator:

```
DriverID=IB
User_Name=sysdba
Password=masterkey
Database=$(DOC)/ADDEMO.GDB
SQLDialect=3
CharacterSet=UTF8
ExtendedMetadata=True
```

See Also

Common connection parameters (see page 188), uADPhysIB Namespace (see page 752), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.8 Connect to Microsoft SQL Server

Describes how to connect to Microsoft SQL Server.

Description

Supported versions

The FireDAC native driver supports Microsoft SQL Server Standard and Express editions version 2000 and higher, and Microsoft SQL Azure. See Connect to Microsoft SQL Server Compact Edition (see page 206) for an explanation of how to connect to Microsoft SQL Server Compact Edition.

Windows client software

FireDAC requires one of the Microsoft SQL Server x86 or x64 ODBC drivers to be installed on the workstation:

- **SQL Server ODBC driver** as the connectivity for SQL Server 2000. Most probably, the ODBC driver is already installed on your workstation. If not, see details.
- **SQL Native Client** as the connectivity for SQL Server 2000 and 2005. We strongly recommend that you have SQL Native Client installed, if your application has to work with SQL Server 2005. See Microsoft SQL Server Native Client.
- **SQL Server Native Client NN.N** as the connectivity for SQL Server 2000, 2005, 2008, 2012 and SQL Azure. We strongly recommend that you have SQL Server Native Client NN.N installed, if your application has to work with SQL Server 2008, 2012 or SQL Azure. See Microsoft SQL Server 2008 Native Client.
- **SQL Server Native Client 11.0** as the connectivity for LocalDB.

Note: SQL Server Native Client 10.0 (SQL Server 2008) may fail to call a stored procedure, when connected to SQL Server 2000. The symptom of this issue is the error message "Incorrect Syntax near to {". In this case use ODBC driver from SQL Server 2000 or 2005 distribution.

When the SQL Server ODBC driver has not been properly installed, you will get an exception when you try to connect:

```
[FireDAC][Phys][ODBC][Microsoft][ODBC Driver Manager] Data source name not found and no
default driver specified
```

Mac OS X client software

FireDAC requires:

- the UnixODBC (see page 160) (**libodbc.dylib**) x86 ODBC driver manager library;
- the FreeTDS (**libtdsodbc.so**) x86 ODBC driver.

FreeTDS may be got from CVS into some folder in your home directory (more 1) (more 2). To install FreeTDS on Mac OS X use the commands:

```
cvs -z3 -d:pserver:anonymous@freetds.cvs.sourceforge.net:/cvsroot/freetds checkout -P
freetdsc
cd freetds
./autogen.sh
./configure --with-tdsver=8.0 --with-unixodbc=/usr/local
make
sudo make install
echo [FreeTDS] > tds.driver.template
echo Description=v0.82 with protocol v8.0 >> tds.driver.template
echo Driver=/usr/local/lib/libtdsodbc.so >> tds.driver.template
odbcinst -i -d -f tds.driver.template
```

Note, you may need additionally to install gawk utility. At this point you can configure a ODBC DSN or test FireDAC connection.

Note, FreeTDS ODBC driver is not that efficient and stable as the original Microsoft ODBC driver.

Driver linkage

To link the driver:

- drop a TADPhysMSSQLDriverLink (see page 793) component from the "FireDAC Links" palette page;
- or include the uADPhysMSSQL (see page 791) unit into the **uses** clause.

Connection definition parameters

To connect to the Microsoft SQL Server DBMS, most applications will need to specify **DriverID**, **Server**, **Database**, **OSAuthent**, **User_Name** and **Password**.

DriverID=MSSQL

Parameter	Description	Example value
Server	Name of a server running SQL Server on the network. The value must be either the name of a server on the network, or the name of a SQL Server Client Network Utility advanced server entry. When you connect to the SQL Azure, you have to prepend server name with "tcp:" prefix. Note: The alternative TPC/IP port may be specified after a server name, separated by the comma.	• 127.0.0.1\SQLEXPRESS • SrvHost, 4000 • `tcp:nasdfert6.database.windows.net`
Port	Only for Mac OS X. Specifies the port where SQL Server is listening. For Windows platform the port may be specified after a comma in the Server parameter value. Eg Server=host, port. The default port is 1433.	

Database	Name of the default database for the connection. If Database is not specified, the default database defined for the login is used.	Northwind
OSAuthent	Controls authentication mode: • **Yes** - use windows authentication. • **No** - DBMS authentication. This is default value.	No
User_Name	The SQL Server login name, if OSAuthent=No. When you connect to the SQL Azure, you have to append "@<server>" suffix to your user name.	• sa • addemo@nasdfert6
Password	The SQL Server login password, if OSAuthent=No. Note, the passwords with both '{' and '}' are not supported.	
Network	Name of a network library dynamic-link library. The name need not include the path and must not include the .dll file name extension.	dbnmpntw
Address	Network address of the server running an instance of SQL Server. Address is usually the network name of the server, but can be other names such as a pipe, or a TCP/IP port and socket address.	
MARS	Controls the MARS - multiple active result sets support in a connection: • **Yes** - MARS is enabled for a connection. This is default value. • **No** - MARS is disabled. The MARS is a feature supported by SQL 2005 and higher. It is not supported by SQL Azure. The enabled MARS may lead to fetch performance degradation. Please, read this for more details.	No
Workstation	Workstation ID. Typically, this is the network name of the computer on which the application resides (optional). If specified, this value is stored in the master.dbo.sysprocesses column hostname and is returned by sp_who and the Transact-SQL HOST_NAME function.	Bookkeeper1
Language	SQL Server language name (optional). If connecting to a SQL Server with multiple languages, Language specifies which set of messages are used for the connection.	
Encrypt	Controls the network traffic encryption: • **Yes** - network traffic will be encrypted. • **No** - no encryption is used. This is default value.	Yes
LoginTimeout	Controls the amount of time in seconds an application waits for a connection attempt to timeout while waiting to establish a connection. 0 specifies an infinite wait (default value).	30

VariantFormat	Controls SQL_VARIANT data type representation: • **String** - values will be represented as dtWideString. A string value is returned as a hex string, numbers as strings, etc. This is default value. • **Binary** - values will be represented as dtByteString with raw data presentation. A string value is returned as a string with original encoding, numbers in binary format, etc.	Binary
ExtendedMetadata	Controls extended describing (see page 129) of the query result sets: • **True** - FireDAC is getting a field origin table and column additionally to the other column attributes. Setting this option to True may slow down a dataset opening. • **False** - FireDAC uses the restricted information about the query columns. This is default value.	True
ApplicationName	Name of the application. If specified, this value is stored in the master.dbo.sysprocesses column program_name and is returned by sp_who and the Transact-SQL APP_NAME function.	AllBooks
ODBCAdvanced	Allows to specify any other additional ODBC connection parameter value.	MARS_Connection=no;Regional=yes
MetaDefCatalog	Default database name. Design time code will exclude catalog name from object name, if it is equal to MetaDefCatalog.	Northwind
MetaDefSchema	Default schema name. Design time code will exclude schema name from object name, if it is equal to MetaDefSchema.	dbo
MetaCaseIns	Controls metadata case-sensitivity: • **True** - case-insensitive metadata search will be used. • **False** - depends on the database collation. This is default value.	True

Usage cases

- Connect to local SQL Server instance, using SQL Server authentication:

```
DriverID=MSSQL
Server=127.0.0.1
Database=Northwind
User_Name=sa
MetaDefSchema=dbo
MetaDefCatalog=Northwind
```

- Connect to SQL Express 2005, using Windows authentication:

```
DriverID=MSSQL
Server=DA\SQLEXPRESS
Database=Northwind
OSAuthent=Yes
MARS=no
```

- Connect to SQL Azure. Note the "@<server>" suffix in User_Name and "tcp:" prefix in Server parameters:

```
DriverID=MSSQL
Server=tcp:nasdfert6.database.windows.net
Database=Northwind
User_Name=addemo@nasdfert6
Password=asd123zxc
Encrypt=Yes
MetaDefSchema=dbo
MetaDefCatalog=Northwind
```

- Connect to LocalDB:

```
DriverID=MSSQL
Server=(localdb)\\v11.0
Database=master
OSAuthent=Yes
```

- Connect to LocalDB and attach database file:

```
Server=(localdb)\\v11.0
DriverID=MSSQL
ODBCAdvanced=AttachDbFileName=C:\\Users\\Alex\\ADDemo.mdf
```

See Also

Common connection parameters (see page 188), uADPhysMSSQL Namespace (see page 791), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.9 Connect to Microsoft SQL Server Compact Edition

Describes how to connect to Microsoft SQL Server Compact Edition .

Description

General

Currently, FireDAC version 5.0 does not have a native Microsoft SQL Server CE driver, but you can use the FireDAC/DBX Bridge driver (see page 194) to connect to Microsoft SQL Server CE.

Note: Microsoft SQL CE is quite different from normal SQL Server's and doesn't have an ODBC driver. So it won't "just work" with the same FireDAC Microsoft SQL driver code base. New R&D will need to go into supporting Microsoft SQL CE in full.

For now though, you can work with database via the FireDAC/DBX Bridge. Working via the FireDAC/DBX Bridge, you still will be able to use all other general FireDAC features, like:

- data type mapping
- Array DML
- SQL scripting
- etc.

Connect using FireDAC/DBX Bridge

To connect to Microsoft SQL CE, do the following:

- Download and install DevArt dbExpress driver for Microsoft SQL Server.
- Create a connection definition, like this one:

```
[My_MSSQL_CE]
```

```
DriverID=TDBX
DriverName="SQLServer Compact"
Database=c:\mydb.sdf
```

- Drop TADConnection (see page 279) to the form
- Set ADConnection1.ConnectionDefName (see page 283) to My_MSSQL_CE

See Also

Common connection parameters (see page 188), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.10 Connect to Microsoft Access database

Describes how to connect to Microsoft Access database files.

Description

Supported versions

The FireDAC native driver supports the Microsoft Access 95, 97, 2000, 2003, 2007 and 2010 databases.

Windows client software

FireDAC requires one of the Microsoft Access x86 or x64 ODBC drivers to be installed on the workstation:

- "**Microsoft Access Driver (*.mdb)**" x86 ODBC driver version 3 or higher (often referred to as the Microsoft JET ODBC Driver) for 95-2003 databases. See details. You may also use one of the alternative drivers provided for non-English languages.
- "**Microsoft Access Driver (*.mdb, *.accdb)**" x86 and x64 ODBC driver version 12 or higher for 95-2010 databases. See details.

Note, to install an x86 or x64 Microsoft Access runtime on a workstation with already installed runtime with different bitness, specify "**/passive**" in the runtime installer command line. Because Delphi IDE is x86 application, you should install x86 Access version on your development workstation.

When the Microsoft Access ODBC driver has not been properly installed, you will get an exception when you try to connect:

```
[FireDAC][Phys][ODBC][Microsoft][ODBC Driver Manager] Data source name not found and no
default driver specified.
```

When trying to open a DB created with a newer Access version using an older Access driver, you will get an exception:

```
[FireDAC][Phys][ODBC][Microsoft][ODBC Microsoft Access Driver] Cannot open database
'(unknown)'.
It may not be a database that your application recognizes, or the file may be corrupt.
```

Linux and Mac OS X client software

FireDAC does not support Microsoft Access database connection on Linux, Mac OS X and iOS.

Driver linkage

To link the driver:

- drop a TADPhysMSAccessDriverLink (see page 791) component from the "FireDAC Links" component palette page;
- or include the uADPhysMSAcc (see page 784) unit into the uses clause.

Connection definition parameters

To connect to a Microsoft Access database, most applications will need to specify **DriverID** and **Database**.

Note, FireDAC supports the password protected databases, but the password length must less or equal to 14 characters. With a more long password an application will raise "Not a valid password" exception.

DriverID=MSAcc

Parameter	Description	Example value
Database	The path to the MDB file. A path may include path variables (see page 163).	c:\mydb.mdb
SystemDB	The path to the system database file. A path may include path variables (see page 163).	c:\mysystem.mdb
ReadOnly	Specify **True** to open a database in read-only mode. **False** is the default value.	True
StringFormat	Defines how to represent String values: • **Choose** - represent as ftString / ftMemo on non-Unicode Delphi (D2007 and lower) and ftWideString / ftWideMemo on Unicode Delphi (D2009 and higher) (default); • **Unicode** - always represent as ftWideString / ftWideMemo; • **ANSI** - always represent as ftString / ftMemo.	Unicode
ODBCAdvanced	Allows you to specify any other additional ODBC connection parameter value. The default value is "ExtendedAnsiSQL=1".	IMPLICITCOMMITSYNC=NO

Usage cases

- Open Microsoft Access database

```
DriverID=MSAcc
Database=c:\mydata.mdb
```

- Open the Microsoft Access database using the system database.

```
DriverID=MSAcc
Database=c:\mydata.mdb
SystemDB=c:\system.mdb
User_Name=usr
Password=pwd
```

- Open the Microsoft Access password protected database. Note, the password length must be less or equal to 14 characters.

```
DriverID=MSAcc
Database=c:\mydata.mdb
Password=pwd
```

Note: to drop/create, encrypt, compact and repair the database use TADMSAccessUtility (see page 786) component.

See Also

Common connection parameters (see page 188), uADPhysMSAcc Namespace (see page 784), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.11 Connect to MySQL Server

Describes how to connect to MySQL Server.

Description

Supported versions

The FireDAC native driver supports the MySQL Server Community, Enterprise and Embedded editions version 3.21 and higher.

Windows client software

FireDAC requires one of the following x86 or x64 libraries:

- the **LIBMYSQL.DLL** client library for connecting to MySQL server. Ideally, the version should be equal to the server version.
- the **LIBMYSQLD.DLL** embedded server library. See "MySQL Embedded server" chapter below.

You can take them from a server (details) installation Bin or Lib folders and put into a folder:

- listed in your PATH environment variable (for example, <Windows>\SYSTEM32);
- your application EXE folder;
- any other folder and specify in FireDAC\DB\ADDrivers.ini:

```
[MySQL]
VendorLib=<folder>\libmysql.dll
```

When the MySQL client library has not been properly installed, you will get an exception when you try to connect:

```
[FireDAC][Phys][MySQL]-314. Cannot load vendor library [libmysql.dll]. The specified module
could not be found.
Check [libmysql.dll], which is located in one of the PATH directories or in application EXE
directory.
```

Linux client software

FireDAC requires:

- the **libmysqlclient.so** x86 or x64 client library.

To install on Linux use the commands:

```
sudo apt-get update
sudo apt-get install libmysqlclient16
sudo ln -s /usr/lib/libmysqlclient.so.16.0.0 /usr/lib/libmysqlclient.so
```

Mac OS X client software

FireDAC requires:

- the **libmysqlclient.dylib** x86 client library.

You can download it as Connector/C for Mac OS X (here). And extract it into /usr/local folder, using the command:

```
sudo tar -C /usr/local -zxvf mysql-connector-c-6.0.2-osx10.5-x86-32bit.tar
```

iOS client software

The article (more) explains how to build **libmysqlclient.dylib** for iOS.

Note, Embarcadero Technologies not tested that and do not provide assistance with that.

Driver linkage

To link the driver:

- drop a TADPhysMySQLDriverLink (see page 794) component from the "FireDAC Links" component palette page;
- or include the uADPhysMySQL (see page 794) unit into the uses clause.

Connection definition parameters

To connect to a MySQL DBMS, most applications will need to specify **DriverID**, **Server**, **Database**, **User_Name**, **Password**, **CharacterSet**.

DriverID=MySQL

Parameter	Description	Example value
Server	The TCP/IP address or host name of the server running a MySQL server.	127.0.0.1
Port	The TCP/IP port on which the MySQL server is listening.	3306
Database	Name of the current database for the connection. If the Database is not specified, no current database is set up.	MyDB
User_Name	The MySQL user ID.	root
Password	The MySQL user password.	
CharacterSet	The default character set for the connection. The connection collation becomes the default collation of the character set. See the SET NAMES statement for details.	cp1251
Compress	Specify **True** to enable network traffic compression. By default, it is set to **False**.	
UseSSL	Specify **True** to enable SSL connection. By default, it is set to **False**. Setting **UseSSL=True** requires additionally specify connection definition parameters: • **SSL_key** - is the path name to the key file; • **SSL_cert** - is the path name to the certificate file; • **SSL_ca** - is the path name to the certificate authority file; • **SSL_capath** - is the path name to a directory that contains trusted SSL CA certificates in pem format; • **SSL_cipher** - is a list of permissible ciphers to use for SSL encryption. For additional details please check MySQL documentation.	True
LoginTimeout	Controls the amount of time (in seconds) an application waits for a connection attempt to timeout while waiting to establish a connection.	30
ReadTimeout	The timeout in seconds for attempts to read from the server. Each attempt uses this timeout value and there are retries if necessary, so the total effective timeout value is three times the option value. You can set the value so that a lost connection can be detected earlier than the TCP/IP Close_Wait_Timeout value of 10 minutes. This option works only for TCP/IP connections and, prior to MySQL 5.1.12, only for Windows. By default, it is not set.	5

WriteTimeout	The timeout in seconds for attempts to write to the server. Each attempt uses this timeout value and there are net_retry_count retries if necessary, so the total effective timeout value is net_retry_count times the option value. This option works only for TCP/IP connections and, prior to MySQL 5.1.12, only for Windows. By default, it is not set.	5
ResultMode	Controls how to fetch a result set to a client. The default value is **Store**. • **Store** - fetches all rows right after query execution and stores them on a client, making the server process ready for new requests. • **Use** - rows will be fetched on demand of a client. While not all rows from the result set are fetched, the server cannot handle new requests. It allows to reduce memory usage on very big result sets. • **Choose** - FireDAC will automatically choose a mode. If the FetchOption.Mode is one of fmAll, fmExactRecsMax, then "Store", otherwise "Use".	Use
TinyIntFormat	Controls the TINYINT(1) representation. The default value is **Boolean**. • **Boolean** - TINYINT(1) columns will be represented as dtBoolean. • **Integer** - TINYINT(1) columns will be represented as dtSByte or dtByte.	Integer
MetaDefCatalog	Default database name. Design time code will exclude the catalog name from the object name if it is equal to MetaDefCatalog. Setting the MetaDefCatalog does not change the current database in the MySQL session.	MyDB

MySQL Embedded server

Major notes:

1. All path argument values are using Unix back slashes. You can specify paths relatively to the application EXE folder, that will simplify deployment.
2. libmysqld.dll and errmsg.sys must be of the same version.
3. libmysqld.dll v 5.1.34, as probably some other versions, are broken and will not work. Update to other version.
4. The general MySQL Embedded setup may be hard. For simplified setup see examples later.
5. In case of incorrect arguments for libmysqld.dll initialization, the application may silently terminate.

To prepare your application to work with a MySQL Embedded server, you should take the following actions:

- Include the TADPhysMySQLDriverLink (see page 794) component into your application. Note, "add argument" means "add argument to TADPhysMySQLDriverLink.EmbeddedArgs (see page 795) list". Optionally you can use driver configuration file (see page 31).
- Copy **LIBMYSQLD.DLL** to the application executable folder. If this folder differs from the application executable folder, add the **--basedir=<LIBMYSQLD folder>** argument. And set TADPhysMySQLDriverLink.VendorLib (see page 782) to **<LIBMYSQLD folder>\LIBMYSQLD.DLL**.
- If only English messages are used, copy **share\english\errmsg.sys** to <LIBMYSQLD folder> and add the **--language=<LIBMYSQLD folder>** argument. If multiple language messages are used, copy appropriate folders from **share*** to <LIBMYSQLD folder>, preserving directory structure. There is no need to add the **--language** argument, because **<LIBMYSQLD folder>\share*** is the default location.
- If the used character sets are different from **ASCII**, copy **share\english\charsets** to <LIBMYSQLD folder>, preserving directory structure. There is no need to add **--character-sets-dir** argument, because **<LIBMYSQLD folder>\share\charsets** is the default location.
- If the database files are located in different folders than <LIBMYSQLD folder>, add the **--datadir=<database files folder>** argument.
- If the InnoDB engine is not used, add the **--skip-innodb** argument. Otherwise add the **--innodb_data_home_dir=<database files folder>** argument, where the value is the InnoDB data space directory.
- If your application will not connect to a remote MySQL server, add the **--skip-networking** argument.
- If your application will use external settings file, add the **--defaults-file=my.ini** argument.

- If your application will use MySQL plugins, add the **--plugin_dir=<plugin folder>** argument. Normally plugins are located in **<LIBMYSQLD folder>\lib\plugin**.
- Add appropriate arguments to tune performance.

To connect to MySQL Embedded server you should not specify Server, Host, Port, User_Name, Password connection definition parameters. If you does not specify **--skip-networking** argument, then using libmysqld.dll you can connect to the remote MySQL servers, the same as with normal libmysql.dll.

Simple setup example:

- Charsets: ASCII only
- Messages: English only
- Database: the EXE folder

```
App folder:
  <app>.EXE
  LIBMYSQLD.DLL
  errmsg.sys
  <DB files>
Arguments:
  --language=./
  --skip-innodb
  --skip-networking
```

Extended setup example:

- Charsets: multiple
- Messages: multiple
- Database: "data" subfolder

```
App folder:
  <app>.EXE
  LIBMYSQLD.DLL
  share\*
  share\charsets
  data\*
  data\<DB files>
Arguments:
  --datadir=./data
  --skip-innodb
  --skip-networking
```

Usage cases

- Connect to a locally running server, listening on the default (3306) port:

```
DriverID=MySQL
Database=addemo
User_Name=root
Password=
```

- Connect to a remote server, listening on a non-default port, using Unicode for character data:

```
DriverID=MySQL
Server=mysrv
Port=3307
Database=addemo
CharacterSet=utf8
User_Name=me
Password=123
```

- Connect to an embedded server, using Unicode for character data:

```
DriverID=MySQL
Database=addemo
CharacterSet=utf8
```

- Connect to a remote server, using a SSL connection:

```
DriverID=MySQL
Server=mysrv
Port=3307
Database=addemo
UseSSL=True
SSL_ca=ca-cert.pem
SSL_cert=client-cert.pem
SSL_key=client-key.pem
```

See Also

Common connection parameters (see page 188), uADPhysMySQL Namespace (see page 794), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.12 Connect to ODBC data source

Describes how to connect to ODBC data source.

Description

Supported versions

The FireDAC ODBC bridge driver supports ODBC Level 2, 3 drivers.

Windows client software

FireDAC requires that the x86 or x64 ODBC driver must be installed according to ODBC specification. You can find:

- the portional list of ODBC drivers here.
- the ODBC programming guide here.

Linux and Mac OS X client software

FireDAC requires that UnixODBC (see page 160) must be installed. As the required ODBC drivers.

Driver linkage

To link the driver:

- drop a TADPhysODBCDriverLink (see page 796) component from the "FireDAC Links" component palette page;
- or include the uADPhysODBC (see page 796) unit into the uses clause.

Connection definition parameters

There are two basic ways to connect to the ODBC data source:

- configure DSN using the ODBC Administrator control panel, then specify DSN name in the **DataSource** parameter;
- set the **ODBCDriver** parameter value, then call Wizard.

Note, **DataSource** and **ODBCDriver** parameters are mutually exclusive. Each ODBC driver has its own set of connection

parameters. They may be specified as the **ODBCAdvanced** parameter value.

DriverID=ODBC

Parameter	Description	Example value
ODBCDriver	The name of ODBC driver to use for connecting. If specified, other connection parameters must be specified in the ODBCAdvanced parameter.	SQL SERVER
ODBCAdvanced	Allows to specify ODBC connection parameter name=value pairs. They must be separated by ';'.	
DataSource	The name of the existing DSN to use for connecting.	MySAPDB
LoginTimeout	Controls the amount of time (in seconds) an application waits for a connection attempt to timeout while waiting to establish a connection (0 specifies an infinite wait).	30
NumericFormat	Controls the internal NUMBER binding mode. The 'Binary' is the most effective mode (default). But some ODBC drivers do not support binary mode correctly. In this case set the parameter to 'String'.	String
MetaDefCatalog	Specifies the default catalog for the application. The design time code will omit the catalog name in object names, if it is equal to MetaDefCatalog.	Northwind
MetaDefSchema	Specifies the default schema for the application. The design time code will omit the schema name in object names, if it is equal to MetaDefSchema.	dbo

Usage cases

- Connect to PostgreSQL:

```
DriverID=ODBC
User_Name=postgres
Password=marlboro
ODBCDriver=PostgreSQL ANSI
ODBCAdvanced=SERVER=localhost;PORT=5432;DATABASE=addemo
```

- Connect to Sybase Adaptive Server Enterprise:

```
DriverID=ODBC
ODBCDriver=Adaptive Server Enterprise
ODBCAdvanced=server=da;port=5000;quotedidentifier=1
Database=addemo
User_Name=sa
MetaDefCatalog=addemo
MetaDefSchema=dbo
```

- Connect to Informix Dynamic Server:

```
DriverID=ODBC
ODBCDriver=IBM INFORMIX ODBC DRIVER
User_Name=informix
Password=informix2
Database=sysuser
ODBCAdvanced=HOST=DA;SRVR=ol_svr_custom;SERV=svc_custom;PRO=olsoctcp;CLOC=en_US.CP1252;DLOC=
en_US.819
```

See Also

Common connection parameters (see page 188), uADPhysODBC Namespace (see page 796), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.13 Connect to Oracle Server

Describes how to connect to Oracle Server.

Description

Supported versions

The FireDAC native driver supports Oracle Enterprise, Standard (ex Workgroup) and Express (ex Personal) server editions version 8.0.3 and higher. Read "Using Oracle with FireDAC (see page 149)" chapter for detailed discussion of Oracle usage in FireDAC for Delphi application.

Windows client software

FireDAC requires one of the following Oracle x86 or x64 client software types to be installed on the workstation:

- "**Fat**" Oracle Client (details). It requires the standard install procedure. The driver uses the client that is installed in the primary Oracle Home, if not specified explicitly.
- "**Thin**" Oracle Instant Client (details). The driver uses the client, which is either copied into a folder in the PATH or into the application EXE folder, if not specified explicitly. See "Using Instant Client" below.

When the Oracle client software has not been properly installed, you will get an exception when trying to connect:

```
[FireDAC][Phys][Ora]-1309. OCI is not properly installed on this machine (NOE1/INIT)
```

Linux client software

FireDAC requires:

- the **libclntsh.so** x86 or x64 client library.

To install on Linux read the article.

Mac OS X client software

FireDAC requires:

- the **libclntsh.dylib** x86 client library.

You can download it as Instant Client for Mac OS X (here) (more). Then extract and copy content to /usr/local/lib folder, using the commands:

```
sudo cp * /usr/local/lib
sudo ln -s /usr/local/lib/libclntsh.dylib.10.1 /usr/local/lib/libclntsh.dylib
```

You can put tnsnames.ora and sqlnet.ora to /etc folder, using the command:

```
sudo cp *.ora /etc
```

Driver linkage

To link the driver:

- drop a TADPhysOracleDriverLink (see page 800) component from the "FireDAC Links" palette page;
- or include the uADPhysOracle (see page 800) unit into an uses clause.

Connection definition parameters

To connect to an Oracle DBMS, most applications will need to specify **DriverID**, **Database**, **User_Name** and **Password**.

DriverID=Ora

Parameter	Description	Example value
Database	The value may be one of the following: • **TNS alias name**, specifying which database to connect to. • **TNS connection descriptor**, as it is in TNSNames.ora • **Oracle connection string**, as it is in SQL*Plus. • **Oracle easy connect string**, as it is described here.	• OraSrv • (DESCRIPTION = (ADDRESS_LIST = (ADDRESS = (PROTOCOL = TCP)(HOST = OraSrv)(PORT = 1521)))(CONNECT_DATA = (SERVER = DEDICATED)(SERVICE_NAME = orcl))) • scott/tiger@OraSrv • system/manager@OraSrv as sysdba • OraSrv:1521/orcl
OSAuthent	Specify **Yes** to use OS authentification, and No to use the DBMS authentification.	No
User_Name	The Oracle user name, if OSAuthent=No	Scott
Password	The Oracle user password, if OSAuthent=No	tiger
AuthMode	The Oracle authentification mode: • **Normal** - standard user. Default value. • **SysDBA** - user with database administrator privileges. • **SysOper** - user with database operator privileges.	Normal
CharacterSet	The character set for the connection. If not specified, the **NLS_LANG** variable value will be used.	• UTF8 • cl8mswin1251
BooleanFormat	Defines how to represent Boolean values: • **Integer** - represents Boolean as Integer values, where False = 0 and True = 1. The default mode. • **String** - represents Boolean as String values, where False = 'F' and True = 'T'.	String
ApplicationName	Name of the application. If specified, this value is stored in the V$SESSION column MODULE.	AllBooks
OracleAdvanced	Additional Oracle session options. See the ALTER SESSION SET chapter "Initialization Parameters and ALTER SESSION" paragraph for details. A value format is - <option>=<value>[;...].	
NewPassword	Specifies new Oracle user password. FireDAC will connect to the DB using the old password and immediately change it to the new one.	tiger2
MetaDefSchema	Specifies the default schema for the application. The design time code will omit the schema name in object names if it is equal to MetaDefSchema.	SCOTT

Using Instant Client

To install Instant Client, download the Oracle Instant x86 or x64 client archive, unpack it and copy the files:

- oci.dll
- oraocci11.dll
- oraociei11.dll
- orasql11.dll

into your application EXE folder or into a folder in the PATH.

When you are using TNS names, put tnsnames.ora file into the same folder, or set TADPhysOracleDriverLink.TNSAdmin (see page 802) property value to a folder path with tnsnames.ora. Or use TNSAdmin driver configuration parameter.

Set TADPhysOracleDriverLink.NLSLang (see page 802) to the required value. Or use NLSLang driver configuration parameter.

Usage cases

- Connect to a database using predefined TNS name (stored in tnsnames.ora):

```
DriverID=Ora
Database=ORA_920_APP
User_Name=ADDemo
Password=a
```

- Connect to a database using host, port and instance name info:

```
DriverID=Ora
Database=(DESCRIPTION = (ADDRESS_LIST = (ADDRESS = (PROTOCOL = TCP)(HOST = OraSrv)(PORT =
1521)))
(CONNECT_DATA = (SERVER =    DEDICATED)(SERVICE_NAME = orcl)))
User_Name=ADDemo
Password=a
```

- Connect to a local database as sysdba:

```
DriverID=Ora
User_Name=sys
AuthMode=sysdba
```

- Connect to a database using TNS name and change the password:

```
DriverID=Ora
Database=ORA_920_APP
User_Name=ADDemo
Password=a
NewPassword=b
```

- Connect to a database using easy connect string:

```
DriverID=Ora
Database=OraSrv:1521/orcl
User_Name=ADDemo
Password=a
```

See Also

Using Oracle with FireDAC (see page 149), Common connection parameters (see page 188), uADPhysOracle Namespace (see page 800), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.14 Connect to PostgreSQL

Describes how to connect to PostgreSQL.

Description

Supported versions

The FireDAC native driver supports PostgreSQL Server version 7.4 and higher, because it requires a PG protocol 3.0 connection.

Windows client software

FireDAC requires the **LIBPQ.DLL** x86 or x64 client library for connecting to the PostgreSQL server. Ideally, the version should be equal to the server version. The full set of the v 9.0 client files:

- libpq.dll
- ssleay32.dll
- libeay32.dll
- libintl-8.dll
- libiconv-2.dll

You can take them from the server (details) installation Bin folder and put into a folder:

- listed in your PATH environment variable;
- your application EXE folder;
- any other folder and specify in FireDAC\DB\ADDrivers.ini:

```
[PG]
VendorLib=<folder>\libpq.dll
```

When the PostgreSQL client library has not been properly installed, then you will get an exception when you try to connect:

```
[FireDAC][Phys][PGSQL]-314. Cannot load vendor library [LIBPQ.DLL]. The specified module
could not be found.
Check [LIBPQ.DLL], which is located in one of the PATH directories or in application EXE
directory.
```

or

```
The ordinal Nnn could not be located in the dynamic link library SSLEAY32.dll.
```

Linux client software

FireDAC requires:

- the **libpq.so** x86 or x64 client library.

To install on Linux use the commands:

```
sudo apt-get update
sudo apt-get install libpq-dev
```

Mac OS X client software

FireDAC requires:

- the **libpq.dylib** x86 client library.

It comes preinstalled on Mac OS X or may be installed separately (more).

iOS client software

The article (more) explains how to build **libpq.dylib** for iOS.

Note, Embarcadero Technologies not tested that and do not provide assistance with that.

Driver linkage

To link the driver:

- drop a TADPhysPgDriverLink (see page 803) component from the "FireDAC Links" palette page;
- or include the uADPhysPG (see page 803) unit into an uses clause.

Additional setup

If an application is using {TIMESTAMPDIFF(MONTH, ...)} escape function, then we recommend to create 3 PostgreSQL functions with argument of types:

- DATE;
- TIMESTAMP;
- TIMESTAMP WITH TIME ZONE.

The function template:

```
CREATE OR REPLACE FUNCTION MONTHS_BETWEEN (timestamp with time zone, timestamp with time
zone) RETURNS integer AS
$body$
DECLARE
  mes INTEGER;
  mes1 INTEGER;
  ano INTEGER;
begin
  mes=extract(month from (age($1,$2)));
  ano=extract(year from (age($1,$2)));
  mes1=abs((ano*12) + mes);
  return mes1;
end
$body$
LANGUAGE 'plpgsql' VOLATILE CALLED ON NULL INPUT SECURITY INVOKER;
```

Connection definition parameters

To connect to a PostgreSQL DBMS, most applications will need to specify **DriverID**, **Server**, **Database**, **User_Name**, **Password**, **CharacterSet**.

DriverID=PG

Parameter	Description	Example value
Server	The TCP/IP address or host name of the server running a PostgreSQL server.	127.0.0.1
Port	The TCP/IP port on which PostgreSQL server is listening.	5432
Database	Name of the current database for the connection. If the Database is not specified, no current database is set up.	MyDB
User_Name	The PostgreSQL user ID.	postgres

Password	The PostgreSQL user password.	
CharacterSet	The default character set for the connection. See the Character Set Support chapter for details.	WIN1251
LoginTimeout	Controls the amount of time (in seconds) an application waits for a connection attempt to timeout while waiting to establish a connection.	30
ExtendedMetadata	Controls the extended describing (see page 129) of the query result sets: • **True** - FireDAC describes a result set to get all the possible column attributes - is nullable, is auto incrementing, to which domain belongs, etc. Setting this option to True, slightly slows down a dataset opening. • **False** - FireDAC uses the restricted information about the query columns (default).	False
OidAsBlob	Controls the interpretation of an OID column in a table: • **No** - an OID column is a dtUInt32 column (contains unsigned integer values); • **Yes** - an OID column is a dtHBlob column (contains Large Object values); • **Choose** - if a query selects a data from the dictionary tables or a column is not of a LO, LargeObject or BLOB domain, then an OID column is a dtUInt32 one, otherwise - dtHBlob one. The ExtendedMetadata option must be True to get a column domain (default).	Yes
UnknownFormat	Controls the handling of an unknown PostgreSQL data type: • **Error** - raise the exception "Cannot describe type" (default); • **BYTEA** - represent as a BLOB value.	BYTEA
ApplicationName	Name of the application. If specified, this value is stored in the pg_stat_activity table, application_name column.	AllBooks
PGAdvanced	Additional PostgreSQL server connection options. See the Database Connection Control Functions chapter PQconnectdb paragraph for details. A value format is - <option>=<value>[;...].	
MetaDefSchema	Default schema name. Design time code will exclude the catalog name from the object name, if it is equal to MetaDefSchema. Setting MetaDefSchema does not change the current schema search path in PostgreSQL session. A 'public' is the default value.	MySchema

Usage cases

- Connect to a server running locally, listening on the default (5432) port:

```
DriverID=PG
Database=addemo
```

- Connect to a remote server, listening on a non-default port, using Unicode for character data:

```
DriverID=PG
Server=pgsrv
Port=5433
Database=addemo
CharacterSet=utf8
MetaDefSchema=MySchema
ExtendedMetadata=True
```

See Also

Common connection parameters (see page 188), uADPhysPG Namespace (see page 803), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.15 Connect to SQLite database

Describes how to connect to SQLite database files.

Description

Supported versions

The FireDAC native driver supports SQLite database version 3.0 and higher.

Read "Using SQLite with FireDAC (see page 135)" chapter for detailed discussion of SQLite usage in FireDAC for Delphi application.

Windows client software

FireDAC supports two SQLite library linking modes:

- **Static linking**: the x86 sqlite3_x86.obj or x64 sqlite3_x64.obj client library is statically linked into application. FireDAC provides sqlite3_Xxx.obj v 3.7.15. That is default mode, no additional files or actions is required. Note, SQLite database encryption is supported only for static linking.
- **Dynamic linking**: the x86 or x64 SQLITE3.DLL client library must be available to open a SQLite database. The recommended SQLITE3.DLL versions is 3.7.7.1 or higher. That is default mode for non-Windows platforms.

You can download:

- the latest x86 DLL version from there, Chapter "Precompiled Binaries For Windows", item "This is a DLL" and put it into a folder listed in your PATH environment variable (for example, <System32> folder) or into your application EXE folder. FireDAC Installer installs SQLITE3.DLL v 3.7.15 into your windows\system32 folder.
- the x64 DLL version from there as "sqlite-netFx40-binary-x64-xxxxx.zip". Extract to a folder, copy SQLite.Interop.DLL into SQLITE3.DLL, then put it as above. FireDAC Installer installs x64 SQLITE3.DLL v 3.7.15 into your windows\system32 folder.

To choose linking mode, in FireDAC\Source\uAD.inc:

- define AnyDAC_SQLITE_STATIC for static linking;
- undefine AnyDAC_SQLITE_STATIC for dynamic linking.

When the SQLite client library has not been properly installed, you will get an exception when you try to connect:

```
[FireDAC][Phys][SQLite]-314. Cannot load vendor library [SQLITE3.DLL]. The specified module
could not be found.
Check [SQLITE3.DLL], which is located in one of the PATH directories or in the application
EXE directory.
```

Linux client software

FireDAC on Linux supports only **dynamic linking** and requires:

- the **libsqlite3.so** x86 or x64 engine.

To install on Linux use the commands:

```
sudo apt-get update
sudo apt-get install sqlite3 libsqlite3-dev
```

Mac OS X and iOS client software

FireDAC on Mac OS X and iOS supports only **dynamic linking** and requires:

- the **libsqlite3.dylib** x86 engine.

It comes pre-installed on Mac OS X and iOS. Note, the default Mac OS X and iOS libsqlite3.dylib is compiled with limited column metadata capabilities (SQLITE_ENABLE_COLUMN_METADATA is undefined). As result, FireDAC may fail to detect column auto-incremental mode and optionality.

Driver linkage

To link the driver:

- drop a TADPhysSQLiteDriverLink (see page 805) component from the "FireDAC Links" palette page;
- or include the uADPhysSQLite (see page 804) unit into an uses clause.

Connection definition parameters

To connect to a SQLite database, most applications will need to specify **DriverID** and **Database**.

DriverID=SQLite

Parameter	Description	Example value
Database	A path to a database. Use ':memory:' or empty string to create and connect to empty in-memory database. A path may include path variables (see page 163).	• c:\MyApp\db.sdb • $(temp)\db.sdb
OpenMode	A mode to open a database: • **CreateUTF8** - open a database to read or write. If the database does not exist, it will be created with the UTF8 default encoding (the default value for pre-Delphi 2009). • **CreateUTF16** - open a database to read or write. If the database does not exist, it will be created with the UTF16 default encoding (the default value for Delphi 2009 and higher). • **ReadWrite** - open a database to read or write. If the database does not exist, an exception will be raised. • **ReadOnly** - open a database to read only. If the database does not exist, an exception will be raised.	ReadOnly
Encrypt	Specified a default encryption mode for a database. The mode may be overridden by an optional password prefix. If it is not specified, then will be used mode, specified by this parameter. Otherwise will be used aes-256. Please, read this for more details.	
Password	Specifies a password for an encrypted database. The value may have a form: [**aes-128 | aes-192 | aes-256 | aes-ctr-128 | aes-ctr-192 | aes-ctr-256 | aes-ecb-128 | aes-ecb-192 | aes-ecb-256 :**] <password> There optional prefix controls the cipher algorithm to be used. The default value is empty string, which means unencrypted mode.	• aes-256:12345 • qwe12345qwe
NewPassword	Specifies a new password for a database and performs an encryption operation: • to encrypt unencrypted database, specify non-empty NewPassword and empty Password; • to decrypt encrypted database, specify empty NewPassword and non-empty Password; • to change encrypted database password, specify non-empty NewPassword and non-empty Password.	

BusyTimeout	Sets a "ms" milliseconds to sleep when a table is locked and UpdateOptions.LockWait (see page 902) is set to True. Zero means do not wait. The default value is 10000.	5000
CacheSize	Changes the maximum number of database disk pages that SQLite will hold in memory at once. Each page uses about 1.5K of memory. The default value is 10000.	10000
SharedCache	Enables or disables the SQLite shared cache feature. Please, read this for more details. The default value is True.	False
LockingMode	Sets the database connection locking-mode. The value is one of: • **Normal**. This mode gives multi-user access to database files. • **Exclusive**. This mode gives maximum performance. The default value is Exclusive, because it allows to get maximum read/write speed for single user applications.	Exclusive
Synchronous	Sets the database connection synchronization mode of in-memory cache with database files. The value is one of: • **Full**. Synchronizes at every critical moment. • **Normal**. As above, but less often. • **Off**. Gives maximum performance. This is the default value.	Off
ForeignKeys	Enables foreign key usage for the database connection, when the application is using SQLite v 3.6.19 or higher. The value is one of: • **On**. Foreign keys in a session are enabled. This is the default value. • **Off**. Foreign keys in a session are disabled.	Off
StringFormat	Defines how to represent String values: • **Choose** - represent as ftString / ftWideString / ftMemo / ftWideMemo, depending on the declared data type name (default); • **Unicode** - always represent as ftWideString / ftWideMemo; • **ANSI** - always represent as ftString / ftMemo.	Unicode
GUIDFormat	Defines how to store GUID values: • **String** - stores GUID as a character string value (default). • **Binary** - stores GUID as a binary string value.	Binary
DateTimeFormat	Defines how to store date and time values: • **String** - stores date and time as a character string value, using the YYYY-MM-DD and HH:MM:SS.XXX format (default). • **Binary** - stores date and time as a real number, which is a Julian date. • **DateTime** - stores date and time as a real number, which is a TDateTime value.	Binary
Extentions	Enables, disables or specifies the SQLite engine extensions to load: • **True** - enables extensions. • **False** - disables extensions (default). • Otherwise use a list of extensions to load in the form <library>[=<entry point][;...].	MyExt.dll;FullTS.dll
SQLiteAdvanced	Additional SQLite database connection options. See the Pragma statements supported by SQLite for details.	auto_vacuum = 1;page_size = 4096;temp_store = FILE

MetaDefCatalog	Default database name. Design time code will exclude the catalog name from the object name, if it is equal to MetaDefCatalog. Setting MetaDefCatalog does not change the current database in the SQLite session. The default value is 'MAIN'.	MyDB

Usage cases

- Connect to a local database in exclusive mode:

```
DriverID=SQLite
Database=$(ADHOME)\DB\Data\ADDemo.sdb
```

- Connect to a shared database (*it is not recommended to store SQLite databases on WinNT shared folders for multi user read-write access*):

```
DriverID=SQLite
Database=\\srv\mydb.sqlite
LockingMode=Normal
Synchronous=Normal
```

- Encrypt unencrypted database:

```
DriverID=SQLite
Database=$(ADHOME)\DB\Data\ADDemo.sdb
NewPassword=aes-256:123qwe
```

- Open encrypted database:

```
DriverID=SQLite
Database=c:\temp\test.db
Password=123qwe
```

- Connect to in-memory database:

```
DriverID=SQLite
Database=:memory:
```

- Connect to a database file in Documents folder on iOS device or emulator in exclusive mode:

```
DriverID=SQLite
Database=$(DOC)/test.sdb
```

See Also

Using SQLite with FireDAC (see page 135), Common connection parameters (see page 188), uADPhysSQLite Namespace (see page 804), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.15.16 Connect to Sybase SQL Anywhere

Describes how to connect to Sybase SQL Anywhere.

Description

Supported versions

The FireDAC native driver supports Sybase SQL Anywhere version 5 and higher.

Windows client software

FireDAC requires one of the Sybase SQL Anywhere x86 or x64 ODBC drivers to be installed on the workstation:

- "**Adaptive Server Anywhere**" ODBC driver to connect to v 5-7.
- "**SQL Anywhere N**" ODBC driver, where N is the DBMS major version number (e.g. 8, 9, 10).

The preferred driver is "SQL Anywhere N". The ODBC driver is part of Sybase SQL Anywhere SDK. You can download it from there.

When the SQL Anywhere ODBC driver has not been properly installed, you will get an exception when you try to connect:

```
[FireDAC][Phys][ODBC][Microsoft][ODBC Driver Manager] Data source name not found and no default driver specified.
```

If your application is using SQL Anywhere service components, like a TADASABackup (see page 741), then following DLL's additionally required to be installed on the workstations:

- DBTOOL<N>.DLL;
- DBLIB<N>.DLL.

There N is a version of your SQL Anywhere.

Mac OS X client software

FireDAC requires:

- the UnixODBC (see page 160) (**libodbc.dylib**) x86 ODBC driver manager library;
- the SQL Anywhere (**libdbodbcNN.so**) x86 ODBC driver.

SQL Anywhere client software for Mac OS X may be downloaded from there. After installing:

- add the following to /usr/local/etc/odbcinst.ini:

```
[SQL Anywhere]
Description=SQL Anywhere v 12
Driver=/Applications/SQLAnywhere12/System/lib32/libdbodbc12.dylib
Setup=/Applications/SQLAnywhere12/System/lib32/libdbodbc12.dylib
UsageCount=1
```

- remove libodbcinst.dylib link from /Applications/SQLAnywhere12/System/lib32 folder, as it conflicts with UnixODBC libodbcinst.dylib. This action was recommended by Sybase development team.
- run /Applications/SQLAnywhere12/System/bin32/sa_config.sh

Driver linkage

To link the driver:

- drop a TADPhysASADriverLink (see page 748) component from the "FireDAC Links" component palette page;
- include the uADPhysASA (see page 740) unit into the uses clause.

Connection definition parameters

To connect to a SQL Anywhere DBMS, most applications will need to specify **DriverID**, **Server**, **Database**, **OSAuthent**, **User_Name** and **Password**.

DriverID=ASA

Parameter	Description	Example value
Server	Specifies the name of a running database server to which you want to connect.	ASASrv
Database	Identifies a loaded database to which a connection needs to be made when connecting to a database that is already running.	
OSAuthent	Controls authentication mode: • **Yes -** an integrated login is attempted; • **No** - the DBMS login is used. This is default value.	No
User_Name	Specifies the user ID used to log in to the database, if OSAuthent=No.	dba
Password	Specifies the user password used to log in to the database, if OSAuthent=No. Note, the passwords with both '{' and '}' are not supported.	sql
DatabaseFile	Indicates which database file you want to load and connect to when starting a database that is not already running. If you want to connect to an already-running database, use the Database parameter.	
Compress	Controls network traffic compression: • **Yes** - to turn compression on for a connection; • **No** - to turn compression off. This is the default value.	Yes
Encrypt	Encrypts packets sent between the client application and the server using transport-layer security or simple encryption. The value syntax is { NONE | SIMPLE | TLS(TLS_TYPE=cipher; [FIPS={ Y | N };] TRUSTED_CERTIFICATES=public-certificate) }	• tls(tls_type=rsa;fips=n;trusted_certificates=rsaserver.crt) • simple
LoginTimeout	Controls the amount of time (in seconds) an application waits for a connection attempt to timeout while waiting to establish a connection (0 specifies an infinite wait).	30
ApplicationName	Assists administrators in identifying the origin of particular client connections from a database server. The string can be retrieved using the statement: SELECT CONNECTION_PROPERTY('AppInfo')	AllBooks
ODBCAdvanced	Allows to specify any other additional ODBC connection parameter value. The default value is "CommLinks=ShMem,TCP". Please note that, if CommLinks is specified, the server will not auto start.	AutoStart=Yes;CharSet=Windows-1251
MetaDefCatalog	Default database name. Design time code will exclude the catalog name from the object name if it is equal to MetaDefCatalog.	addemo
MetaDefSchema	Default schema name. Design time code will exclude the schema name from the object name if it is equal to MetaDefSchema.	dba

Usage cases

Auto start local server and open database file "C:\sybase\addemo_asa10.db":

```
DriverID=ASA
ODBCAdvanced=AutoStart=Yes
DatabaseFile=C:\sybase\addemo_asa10.db
User_Name=dba
Password=sql
MetaDefSchema=dba
```

- Connect to a default local server

```
DriverID=ASA
User_Name=dba
Password=sql
MetaDefSchema=dba
```

- Connect to the database ADDemo, running on server instance ASASrv:

```
DriverID=ASA
Server=ASASrv
Database=ADDemo
User_Name=dba
Password=sql
MetaDefSchema=dba
```

- Connect to the database ADDemo, running on the server instance ASASrv in another network:

```
DriverID=ASA
ODBCAdvanced=CommLinks=tcpip(host=227.12.66.1)
Server=ASASrv
Database=ADDemo
User_Name=dba
Password=sql
MetaDefSchema=dba
```

See Also

Common connection parameters (see page 188), uADPhysASA Namespace (see page 740), FAQs (see page 227), How to configure FireDAC Drivers (see page 31), How to manage FireDAC Connection Definitions (see page 27)

1.16 FAQ

The list of FireDAC related frequently asked questions.

1.16.1 Installation

The list of questions and answers related to FireDAC installation.

Description

QI1: FireDAC fails to install and gives compilation errors, referring to TDBX or DBX. That is dbExpress stuffs, which I does not need. How to completely disable dbExpress usage ?

A: Install the FireDAC packages into IDE by hands. For that do:

- install FireDAC using the installer. At installation unmark all options at "Install in IDE" page;
- go to FireDAC\Bin folder;
- run there updateEnv.bat;
- run Delphi IDE;
- choose Tools -> Options -> Delphi Options -> Library;
- verify, and add if required, the FireDAC\Source path to "Library path";
- open FireDAC\Source\uAD.inc;
- locate there line with {$DEFINE AnyDAC_DBX};
- replace the line with {.$DEFINE AnyDAC_DBX} (dot between { and $ is added);
- save the uAD.inc;
- open FireDAC\Packages\AnyDAC_DXxxx.bpg or .bdsgroup or .groupproj;
- remove the AnyDAC_PhysTDBX_DXxxx.dpk or AnyDAC_PhysDBX_DXxxx.dpk from the project group;
- save project group;
- compile all packages in the group;
- right click on AnyDAC_Dcl_DXxxx.dpk and choose "Install".

QI2: How to recompile FireDAC packages ?

A: Do the following:

- close Delphi IDE;
- with Windows Vista, 7 or newer run console in administrator mode:
  - press the "Start" button;
  - type CMD;
  - select CMD and press CTRL+SHIFT+ ENTER;
  - a User Account Control box pop up, confirm this;
- otherwise with Windows XP or when you are logged as administrator:
  - press the "Start" button;
  - type CMD;
  - press ENTER;
- go to FireDAC\Bin folder. Normally run:
  - CD "c:\Program Files\da-soft\FireDAC for Delphi\Bin"
- run there compileXxxx.bat, appropriate to your Delphi / C++ Builder version;

Finished. Now you can run IDE again.

QI3: How to install FireDAC into IDE by hands ?

A: Do the following:

- install FireDAC using the installer. At installation unmark all "Build binaries".
- go to FireDAC\Bin;
- run there updateEnv.bat;
- run Delphi IDE;
- choose Tools -> Options -> Delphi Options -> Library;
- verify, and add if required, the FireDAC\Source path to "Library path";

- open FireDAC\Packages\AnyDAC_DXxxx.bpg or .bdsgroup or .groupproj;
- compile all packages in the group;
- right click on AnyDAC_Dcl_DXxxx.dpk and choose "Install".

QI4: How to install FireDAC on Linux ?

A: Read the "Lazarus / FPC (see page 160)" article.

QI5: I got an archive with sources from the Embarcadero support. How to install it ?

A: Do the following:

- extract archive to FireDAC\Source folder. Do not remove the original folder content;
- mark FireDAC\Source\sqlite3_x86.obj and sqlite3_x64.obj as read-only. For that you can run "attrib +r sqlite3*.obj" command in FireDAC\Source folder;
- now recompile FireDAC packages. For that see QI2.

1.16.2 General Questions

The list of general questions and answers.

Description

QG1: Is there 'Getting Started Guide' (aka 'Quick Start Guide') for FireDAC?

A: First Steps to use FireDAC (see page 2)

QG2: What is the difference between mobile and full versions?

A: FireDAC Mobile version:

• does not include the library source code files;

• only iOSSimulator and iOSDevice binaries are included.

QG3: Is FireDAC thread safe?

A: Yes, it is. Read Multi Threading (see page 46) for more details.

QG4: How to handle with Exceptions?

A: Read Handling Errors (see page 44) for details. And the code sample:

```
uses
  uADPhysIBWrapper;

  try
    ...Login to Database...
  except
    { Login not correct }
    on E: Exception do
    begin
      if (E is EIBNativeException) and
         (EIBNativeException(E).Errors[0].ErrorCode = 335544472)
```

```
      then
        ShowMessage(strUserUnkown)  // your Errormessage
      else
        ShowMessage(E.Message);
    end;
  end;
```

QG5: I use Delphi 2009 with Firebird 2 with character set unicode_fss. I don't understand why FireDAC generate TWideStringField in Delphi 2009 ?

A: In Delphi 2009:

- TStringField is 1 byte ANSI string field;
- TWideStringField is multi-byte Unicode string field.

So, that is correct behavior. Also read Unicode Support (see page 51) for details.

1.16.3 TADManager and TADConnection Questions

The list of questions and answers related to TADManager and TADConnection.

Description

QC1: I have a base datamodule class with TADManager. When I am creating a descendant class, I am getting »Application must have only single ADManager« error. What is wrong ?

A: You can only have one ADManager (see page 570) in a application. Instead of explicit creating that in your classes, refer a singleton object via ADManager function.

QC2: However in my project group I have several different delphi projects (applications), each with its own DataModule and ADManager. If I happen to open more than one of these datamodules at the same time (say to copy or compare items) then I get an error saying, that you can only have one ADManager per application, and it promptly removes one of them from one of the DataModules.

A: Do you really need the TADManager (see page 429)'s ? An application does not need to explicitly create the TADManager (see page 429). It is required to set the options and the configuration files in design time.

QC3: Can this be fixed, I keep on loosing my ADManagers and having to recreate them.

A: No, this cannot be fixed. Because only single TADManager (see page 429) may be created at design or run time. If there will be few TADManager (see page 429)'s, then FireDAC will not know which one to use. Also see QC1.

QC4: Does FireDAC supports a connection pooling?

A: Yes, it has. In general, connection pooling keeps a pool of open "physical" connections, and when TADConnection.Connected (see page 282) is set to True, then FireDAC takes a "physical" connection from the pool and uses it. When application is set TADConnection.Connected (see page 282) to False, the "physical" connection is not closed, but put back into pool.

To use connection pooling in FireDAC, just add Pooled=True to you connection definition. No more special actions your application should take. Read Multi Threading (see page 46) for details.

QC5: Connection Editor seems to ignore ADPhysIBDriverLink1.VendorLib and uses hard coded driver.

A: You have to set VendorLib (see page 782) before first connection using this driver. After a connection is established, the DBMS client is loaded.

At design-time to make sure that your TADPhysXXXDriverLink (see page 778) properties are in the usage, check that TADPhysXXXDriverLink is first in the data module / form creation order. Then optionally restart Delphi IDE.

At run-time use the driver link Release (see page 780) method:

```
ADConnection1.Close;
...
ADConnectionN.Close;

ADPhysIBDriverLink.VendorLib := 'c:\fbclient.dll';
ADPhysIBDriverLink.Release;

ADConnection1.Open;
...
ADConnectionN.Open;
```

Other option is to define a virtual driver. Read the following for details Configuring Drivers (see page 31). For example:

```
[FB_Embedded]
BaseDriverID=IB
VendorLib = C:\fb\fbembed.dll
```

And use FB_Embedded driver as a DriverID at design time. At run time you can also use ADDrivers.ini or just configure TADPhysXXXDriverLink.

1.16.4 TADQuery, TADStoredProc and TADUpdateSQL Questions

The list of questions and answers related to TADQuery, TADStoredProc and TADUpdateSQL.

Description

QQ1: Can I use the TADQuery and connect it with a dataset provider and retieve the data in a Codegears' client dataset?

A: TADQuery (see page 476) is the mix of the TADMemTable (see page 436), TADTableAdapter and few TADCommand (see page 267)'s. So, the TADQuery has everythings inside to execute SQL command, send parameters data, receive and stored result sets, browse result sets, post changes back to a DB. There is no reason to use TADQuery + DSP + CDS.

You can use TADMemTable, TADTableAdapter, TADCommand directly, instead of TADQuery. It gives more flexibility. For example, synchronized cached updates accross few datasets. But it also requires more coding.

IOW, TADQuery is optimal "shortcut" for every day data application programming.

QQ2: How to force ADStoredProc to use parameters specified by hands ?

A: Exclude fiMeta from FetchOptions.Items (see page 855).

When you are creating the parameters by hands, you should exclude fiMeta from FetchOptions.Items (see page 855). When it is specified, then FireDAC will fetch the stored procedure parameter definitions from a DB and will repopulate the Params (see page 534) collection.

If you have difficulties with manual definition of parameters, then populate the Params (see page 534) collection automatically and check how the parameters are defined. Then compare that to your code.

QQ3: »[FireDAC][Phys]-308. Cannot open / define command, which does not return result sets.« and »[FireDAC][Phys]-310. Cannot execute command returning results set.« what they mean ?

A: The »[FireDAC][Phys]-308. Cannot open / define command, which does not return result sets.« exception is raised, when application is executing the Open method for a SQL command, which does not return a result set. The exception is raised after the SQL command is executed, but DBMS has not returned any result set.

The »[FireDAC][Phys]-310. Cannot execute command returning results set.« exception is raised, when application is executing the ExecSQL method for a SQL command, which is returning a result set. Does the command return a result set or not is determined by the FireDAC SQL command preprocessor. If the command is recognized as SELECT or one of it forms, then it returns a result set. Otherwise – not.

In some cases, FireDAC may fail to recognize a SQL command as returning or not a result set. And sometimes add-hok application need to execute a SQL command, not depending on how many result sets it returns. So, how to be in these case ? Here is two basic ways:

1)

```
ADQuery1.OpenOrExecute
```

It may internally raise [FireDAC][Phys]-308, but an exception will be not progated out of the OpenOrExecute (see page 641) and a SQL command will be really executed. Also, the method will return True, if the command returned a resultset.

2)

```
ADQuery1.Command.CommandKind := skInsert;
ADQuery1.ExecSQL;
```

Just say to FireDAC, that the command is really an INSERT or what else kind, which does not return a result set.

QQ4: "Out of memory" exception is raised at calling ADQuery.Execute(ADQuery.Params.ArraySize). ArraySize is about 90,000. What is wrong ?

A: 1) 90,000 is tooooo much for any DBMS, because the data will be cached few times (parameters, DBMS API buffer, network pocket buffer, etc). Also, probably, each record has a "valuable" size. Even, if that is not True, then an application may run into the DBMS API limitations. The most "dark" thing there is the high limit for the Array size. Oracle - up to $7FFF. Other DBMS's - depends on the network pocket size, etc ...

2) Split 90,000 into the chunks of the 500-5000 items, each one. See AD03-ArrayDML demo regarding that. In general, fill an array up to the chunk size, then call Execute with the chunk size, then fill again and Execute, etc.

With Firebird I use ArraySize = 1,000,000 (60 sec) without problems.

We were the lucky to find an emperical formula, allowing to determine maximum size of an array. And FireDAC automatically splits one large array into few chunks.

It is not that simple to find similar to Oracle. While it supports the size up to $7FFF, I was getting AV's or other issues with large arrays ...

Anyway, if a record size is too big, then you can run out of memory even before calling Execute. So, be carefull ...

QQ5: Is it possible to know if all ADQuery records are fetched ?

A: Check SourceEOF property.

QQ6: I want to insert a record (via plain SQL) and get back the IDENTITY / SEQUENCE. What is the most efficient, multi-database friendly way?

A: 1) The TADConnection has GetLastAutoGenValue (see page 349) method. Depending on a DBMS it will return a last auto-generated value in a session. For example, for Oracle it will be:

```
SELECT <AName>.CURRVAL FROM dual
```

For MySQL it will access to the MYSQL API to get the value without a SQL query. Also, if a DBMS does not support the sequences / generators, then AName parameter value will be just ignored.

2) There is no a common way to write a SQL command, which will insert a record and return an auto-generated value. For example, for Oracle it will be:

```
INSERT ... INTO ... RETURNING ID INTO :ID
```

For PostgreSQL that will be 2 separated commands:

```
INSERT ... INTO ..
SELECT CURRVAL(...)
```

When you are posting an insert to a DB, using TDataSet Insert / Post methods, FireDAC looks at the DBMS kind and generates appropriate efficient SQL command[s].

QQ7: How to rollback transaction after user canceled the query execution in amCancelDialog mode ?

A: Two options:

1)

```
ADTransaction.StartTransaction;
try
  ADQuery.ExecSQL;
  ADTransaction.Commit;
except
  on E: EAbort do
    // user canceled the command execution
    ADTransaction.Rollback;
end;
```

2) Create TADQuery.OnError (see page 492) event handler. When a command execution is canceled, this event handler will be called and AException parameter will meat condition:

```
EADDBEngineException(AException).Kind = ekCmdAborted
```

QQ8: My query with the '&', '!' characters inside fails to execute correctly. What is wrong ?

A: For example, the following query with default options will fail:

```
ADQuery.SQL.Text := 'select * from xy where Fieldname = ''xxx&n''';
ADQuery.open;
```

The SQL command received by a DBMS will miss '&n'. That is because '&' specifies the beginning of a macro variable. By default every variable has empty value. So, the '&n' will be replaced by the empty string. If you do not use macros, then set ResourceOptions (see page 498).MacroCreate (see page 882) and MacroExpand (see page 882) to False.

QQ9: I am getting "Parameter Xxx not found" accessing to stored procedure parameters. What is wrong ?

A: For example, consider the code:

```
ADStoredProc1.StoredProcName := 'TestProc';
ADStoredProc1.Prepare;
ADStoredProc1.ParamByName('Par').AsInteger := 100;
```

There may be different reasons for that:

- DB does not have "TestProc" stored procedure, then it must be created;
- depending on a DBMS, "TestProc" may be in invalid state, then it must be validated;
- depending on a DBMS, the stored procedure name is in mixed case, then it must be quoted (more (see page 133));
- the fiMeta is excluded from FetchOptions.Items, then it must be included or parameters must be created by hands (more (see page 71));
- stored procedure does not have "Par" parameter, the parameter name must be corrected or "Par" parameter must be added;
- depending on a DBMS, the parameter name may be prefixed with '@' or ResourceOptions.UnifyParams (see page 885) set to True.

1.16.5 TADTable Questions

The list of questions and answers related to TADTable.

Description

QT1: I am getting "[FireDAC][DatS]-15. Duplicate row found on unique index" when navigating in TADTable ? What that means ?

A: In most cases that means, that the DB sort order and the client sort order differ. That may lead to endless loops or [FireDAC][DatS]-15 error. To resolve the issue, setup the FormatOptions.SortLocale (see page 867) and SortOptions (see page 868) properties, so that the client sort order will match to the DB sort order.

QT2: When navigating through TADTable records my application is hanging. What is wrong ?

A: See QT1.

1.16.6 TADMemTable Questions

The list of questions and answers related to TADMemTable.

Description

QM1: TADMemTable vs. TClientDataSet – what is better ? I'm thinking of replacing all TClientDataSet with TADMemTable. Is a TADMemTable faster than a TClientDataSet?

A: It is MUCH faster.

2 years ago we have posted this article. There you can find "Speed" section and the screen snapshot with benchmark results. The ADClientDataSet is FireDAC v 1 in-memory table. So, the post is about FireDAC v 1, but FireDAC v 2 is faster than v 1.

As you can see TClientDataSet is extremely slow dataset ...

QM2: The nice thing with the TClientDataSet is that you can add, delete, change records and then process TClientDataSet.Delta. Is that possible with TADMemTable ?

A: The TADMemTable (see page 436) has the similar functionality. But it follows to the »Cached Updates« model. When CachedUpdates (see page 441) property value is True, then memtable will record changes in the updates journal.

To post updates to a DB you have to use ApplyUpdates (see page 611). To "merge change log" you have to call CommitUpdates (see page 615).

You can find description of these and other methods in the FireDAC help file and the examples in the FireDAC\Samples\Comp Layer\TADMemTable folder.

QM3: I use clonecursor a lot with the clientdatasets. How I can do that with TADQuery ?

A: Use the following code:

```
ADMemTable1.CloneCursor(ADQuery1, False, False);
```

QM4: How to create and populate ADMemTable ?

A: 1) Create the in-memory dataset

```
with ADMemTable1.FieldDefs do begin
  with AddFieldDef do begin
    Name := 'f1';
    DataType := ftInteger;
  end;
  with AddFieldDef do begin
    Name := 'f2';
    DataType := ftString;
    Size := 50;
  end;
end;
with ADMemTable1 do begin
  Open;
  Append;
  Fields[0].AsInteger := ...;
  Fields[1].AsString := ...;
  Post;
end;
.....
```

2) Append data to it, browse / edit it.

QM5: What is the fastest way to load a TADMemTable from a another DataSet (including structure) ?

A: The most simple way to copy the structure and the data from a TDataSet to a TADMemTable (see page 436) is to use the CopyDataSet (see page 615) method:

```
ADMemTable1.CopyDataSet(DataSet1, [coStructure, coRestart, coAppend]);
```

QM6: How to copy all records from ADQuery into ADMemTable and edit them in the ADMemTable ?

A: You can do something like that:

```
// UniDirectiona must be false if no
ADQuery1.FetchOptions.Undirectional := False;

ADQuery1.Open;
ADQuery1.FetchAll;
ADMemTable1.Data := ADQuery1.Data;
ADMemTable1.First;
while not ADMemTable1.Eof do begin
  ADMemTable1.Edit;
  .......
  ADMemTable1.Post;
  ADMemTable1.Next;
end;
```

QM7: I cannot add a persistent field of type ftGUID. I am getting error about field size, but the field size control is disabled. What is wrong ?

A: That is known issue in DB.pas unit: http://qc.embarcadero.com/wc/qcmain.aspx?d=8008. The workaround is to create a GUID field at runtime.

1.16.7 Fetching and Populating Questions

The list of questions and answers related to fetching records and populating datasets.

Description

QX1: Fast forward-only, read-only access?

A: Set ADQuery.FetchOptions (see page 486):

- CursorKind (see page 853) = ckDefault or ckForwardOnly
- Mode (see page 857) = fmOnDemand
- RowsetSize (see page 859) = 1000
- Unidirectional (see page 860) = True
- optionally exclude fiMeta from Items, if you does not need to edit dataset.

QX2: How to append multiple result sets to a ADQuery or a ADMemTable ?

A: The following approach with the ClientDataset to hold multiple result sets:

```
begin loop
  // run SqlQuery with new params
  ...
 ClientDataset.AppendData(SqlQuery.Data)
  ...
end loop
```

may be replace with the single TADQuery. Kind of that:

```
// initial open and fetch
ADQuery1.Params[0].AsInteger := 1;
ADQuery1.Open;

// reexecute command, fetch rows again and append them
// to the existing rows
ADQuery1.Command.Close;
ADQuery1.Params[0].AsInteger := 2;
ADQuery1.Command.Open;
ADQuery1.FetchAgain;
ADQuery1.FetchAll;

// reexecute again with different parameter value
ADQuery1.Command.Close;
ADQuery1.Params[0].AsInteger := 3;
ADQuery1.Command.Open;
ADQuery1.FetchAgain;
ADQuery1.FetchAll;
```

QX3: How to execute a query and append it result set to an existing dataset, without inserting these records to DB ?

A: See the TADDataSet.FetchAgain (see page 623) method description. As other option, you can execute a SQL command, which will fetch the additional records into an existing dataset:

```
ADCommand1.CommandText := 'select ... from ... where id= :id';
```

```
ADCommand1.Open;
ADCommand1.Fetch(ADQuery1.Table);
ADCommand1.Close;
```

QX4: My query returns 800 records, but RecordCount returns 50. What is wrong ?

A: The RecordCount by default shows the number of records in the dataset records cache. More about record counting modes you can read at TADFetchOptions.RecordCountMode Property (see page 858).

50 is the default rowset size (see page 859). That is the number of records FireDAC will fetch at single request. So, right after open the dataset will have <= 50 records, that is what RecordCount is showing. When you will navigate through dataset it will fetch additional rowsets and the number of records may grow.

To show the total number of records which query returns, you may do one of the following:

- perform ADQuery1.FetchAll (see page 624). So all records will be fetched and RecordCount will show their number;
- to set FetchOptions.RecordCountMode (see page 858) to cmTotal. Note, that may lead to performance degradation, as FireDAC will perform additional SELECT COUNT(*) query for each SELECT query.

1.16.8 Sorting, Searching, Locating, Filtering Questions

The list of questions and answers related to sorting, searching, locating, filtering datasets.

Description

QS1: Dataset sorts non-English strings incorrectly. Or - I am getting "[FireDAC][DatS]-2. Object [] is not found". What is wrong ?

A: To fix the issue, do the following:

- open FireDAC\Source\uAD.inc;
- find the lines:

```
{$define AnyDAC_NOLOCALE_DATA}   // define, to use binary data comparision
{$define AnyDAC_NOLOCALE_META}   // define, to use binary metadata comparison
```

- comment them out;
- save the file and recompile your application.

Note: if you are only working with English (ASCII) texts, then ensure that the above lines are uncommented. That speedups sort and locate operations considerably.

QS2: Is it necessary to create Index for ordering data?

A: You does not need to define Indexes, if you just need to order data. IndexFieldNames (see page 600) will work for you. Indexes allows to define the views, which are mix or filtering and ordering. Note, Indexes (see page 598) and IndexDefs (see page 598) are mutually exclusive for all FireDAC datasets. That means - either you should fill Indexes or IndexDefs, but not both.

QS3: Can lookup fields be used in index definitions?

A: No, you cannot. But you can use in indexes internal calculated fields. For that add persistent fields, then add fkInternalCalc field, create OnCalcFields event handlers and calculate this field in event handler. For example:

```
procedure TForm21.ADTable1CalcFields(DataSet: TDataSet);
begin
```

```
  DataSet.FieldByName('f_calc').AsString :=
    ADMemTable2.Lookup('code',
      DataSet.FieldByName('f_code').AsInteger, 'name');
end;
```

QS4: Can calculated fields be used in index definitions ?

A: Current FireDAC release does not support fields with FieldKind = fkCalculated in Locate (see page 634), Lookup (see page 637), IndexFieldNames (see page 550), etc. The workaround is to use fkInternalCalc fields.

QS5: When I call the FindNearest method at runtime I'm receiving an error message stating that there is no active index.

A: The index must be selected, not just active. For that:

- set Indexes (see page 598)[i].Selected (see page 659) := True

- or set ADTable1.IndexFieldName (see page 550) := <name of your index>

QS6: Ho to define different character collation for TADDataset indexing ?

A: In general there are 3 options:

- To assign LCID to TADMemTable.Table (see page 609).Locale. FireDAC uses CompareStringA and CompareStringW with SORT_STRINGSORT flag. The default collation is DBMS independent. It is LOCALE_USER_DEFAULT. See Win API docu for details.
- To change source code - uADDatSManager, TADDatSRow.CompareData and implement your own comparision algorythm.
- Register custom function with expression evaluator. See uADStanExpr for registration details. Then you can use this function at TADMemTable.Indexes[..].Expression. For example: Expression := 'MySort(Name)'.

In future we will implement custom collations. That will be usefull as for SQLite driver as for cases, like yours.

QS7: How to order data in such way: COL1 descending, COL2 ascending ?

A: IndexFieldNames (see page 600) := 'col_1:D;col_2';

Also, check help file, chapter "TADDataSet.IndexFieldNames property".

QS8: ADQuery.Locate raises "Function sequence error" with SQL Server. What is wrong ?

A: It looks like ADQuery was opened after an explicit transaction start, then a transaction was commited. And then you are calling ADQuery.Locate (see page 634). But not yet all records were fetched from ADQuery, but Locate imlicitly calls FetchAll (see page 624). And it gives this error on SQL Server.

To avoid it do:

- set ADQuery.FetchOption (see page 486).Mode (see page 857) = fmAll for this specific ADQuery
- call FetchAll (see page 624) for this ADQuery before calling Commit.

This is SQL Server behaviour - invalidate open cursors after Commit / Rollback.

QS9: What is the best way to implement Lookup tables ? Lookup tables seems to be big cause to why opening/loading is slow.

A: If you does not need all records on the client, then use TADQuery with SELECT ... WHERE ....

QS10: If you have any tips on how to handle lookup fields to get best performance?

A: Then set TADQuery.IndexFieldNames (see page 489) to LookupKeyFields, then FireDAC will use client index to locate record in a lookup dataset.

If the number of unique key values in not high, then set LookupCache to True.

QS11: I am failing to filter by the DateTime value. What is wrong ?

Q: I am using Delphi 2007 with FireDAC 2.0.11.895 on a Postgres database. When I try to filter on a DateTime field I can't seem to get an exact match with a value passed to it like this:

```
created_date = '8/10/2009 14:42:14'           // or even...
created_date = '8/10/2009 14:42:14.247'     // ... the exact time with milisec
```

Instead I have to end up using something like this:

```
created_date >= '8/10/2009 14:42:14' AND created_date < '8/10/2009 14:42:15' or
created_date >= '8/10/2009 14:42:14.000' AND created_date <= '8/10/2009 14:42:14.999'
```

But isn't pretty code.

A: The problem is that DBMS or DBMS API may round the time value. While a programmer expects to see the .247 as a fractional part, it may be .246 or how else. That is not FireDAC failure, but it is how a DBMS is working.

You can round the time value to the seconds and compare that value with a constant without a fractional part:

```
uses
  uADStanExprFuncs
...
  ADQuery1.Filter = 'TimeStampDiff(''second'', created_date,
    convert(''timestamp'', ''8/10/2009 14:42:14'')) = 0';
```

1.16.9 Editing Questions

The list of questions and answers related to data editing.

Description

QE1: What means "[FireDAC][DApt]-400. Update command updated [0] instead of [1] record" ?

A: This error often happens when FireDAC includes some float / double / single / date / datetime / time or some other table fields, which are the subject to the precision lost, into WHERE phrase. Then at the parameter assignments the precision lost may happen. As result of that the WHERE phrase returns no records.

FireDAC may include such fields into WHERE depending on UpdateOptions.UpdateMode (see page 904). Sometimes you can see this error with upWhereKeyOnly. Although you have specified upWhereKeyOnly, FireDAC still may use upWhereAll. Which is fallback, when there is no PK fields defined. No PK fields may be defined when:

- fiMeta is excluded from FetchOptions.Items (see page 855);
- or table in SQL does not have primary key defined;
- or UpdateOptions.KeyFields (see page 845) is empty;
- or none of TField's has pfInKey in ProviderFlags.

Other reason with some DBMS (SQL Server, PostgreSQL) is that the table has a trigger, which modifies the data. With SQL

Server put SET NOCOUNT ON at trigger beginning. With PostgreSQL set UpdateOptions.CountUpdatedRecords (see page 899) to False.

QE2: When it is necessary to use TADUpdateSQL?

A: FireDAC generates updating SQL commands automatically, when the original SQL command is a simple SELECT or a SELECT with JOIN, where is a single table preserving the primary key fields. So, the TADUpdateSQL (see page 563) usage is optional. TADUpdateSQL is required when:

- The original SQL command is not a SELECT command. Eg, stored procedure returning result sets or what else.
- The original SELECT command is not preserving primary key. Eg, join few tables, has DISTINCT or GROUP BY clauses, etc.
- An application needs non-standard update SQL command. Eg, application is posting updates using stored procedure calls.

QE3: Is it possible to use Macros inside SQLs in ADUpdateSQL?

A: Use the following code:

```
ADUpdateSQL1.Commands[arInsert].MacroByName('MacroName').Value := 'value';
```

QE4: Why in CachedUpdates mode calling ApplyUpdates more than once tries to post inserted records again ?

A: After calling ApplyUpdate (see page 611), you should call CommitUpdates (see page 615). After this call all changes will be deleted from internal cache.

QE5: How do not refresh a detail TADQuery after a master TADQuery is scrolled or posted ?

A: Two ways:

- Implement master-detail linkage on your own. This will require to add TDataSource.OnDataChange event handler. I will call this "standard" way.
- To use Centralized Cached Updates (see page 114).

QE6: How to remove few dataset records, without removing them from DB.

A: You can work directly with internal dataset data storage. It is accessible through TADDataSet (see page 584).Table (see page 609) property. For example, to delete row with index 3 do the following:

```
ADQuery1.Table.Rows[3].Free;
ADQuery1.UpdateCursorPos;
ADQuery1.Resync([]);
```

For example, to delete the current record do the following:

```
ADQuery1.UpdateCursorPos;
ADQuery1.GetRow.Free;
ADQuery1.UpdateCursorPos;
ADQuery1.Resync([]);
```

And finally you can use CachedUpdates mode. Set a dataset to the cached updates mode. Then delete a record and call CommitUpdates.

QE7: ATable.UpdateToDataset(BTable , 'mykey', [mtufEdit, mtufAppend]) – how can I do this with FireDAC?

A: Use the TADDataSet.CopyDataSet (see page 615) method with the following options:

- [coAppend] - append all records from ASource (as it was);
- [coEdit] - edit only records with existing key values;
- [coAppend, coEdit] - edit records with existing keys and append records with non-existent keys.

QE8: How to assign value to a ftGUID field ?

A: Use the following code:

```
(AMemTable.FieldByName('Field1') as TGUIDField).AsGuid := aGUID;
```

QE9: How to specify a default value for a dataset field ?

A: Assign an expression to the TField.DefaultExpression property.

QE10: Whether property TField.DefaultExpression is being supported in the analogous way as TField.CustomConstraint, and the effect of the expression is being written, as the default value for a field?

Yes, if a field is a normal resultset field. If a field is fkInternalCalc one, then the result of the DefaultExpression will be used as a field value. And it will be updated like any other calculated fields.

QE11: E.g. such an expression: {fn DAYOFMONTH({fn CURDATE()})} will be correct?

A: No, you are using FireDAC escape functions. They are supported only in the SQL commands, not in expressions. In expressions, like constraints and default value expressions, you should use functions and syntax, supported by FireDAC expression evaluator:

```
DAYOFMONTH(CURDATE())
```

Also, to use such functions, you should include uADStanExprFuncs unit into your application.

QE12: How to specify default value for a dataset Boolean field ?

A: To specify a default value for a dataset field, assign required expression to the TField.DefaultValue property.

To assign default value to a boolean field, you can use one of the following strings - F, FA, FAL, FALS, FALSE as False names. And similar for True.

QE13: Assigning TField.CustomConstraint does not work. What is wrong ?

Q: Appending constraint via:

```
ADQuery.FieldByName('FIELD_NAME').CustomConstraint := 'FIELD_NAME > 1';
ADQuery.UpdateConstraints;
ADQuery.Table.Constraints.Check(ADQuery.GetRow(), rsModified, ctAtEditEnd);
```

It's not working - exception is not raised...

A: That is OK (explanation will be later).

Q: But:

```
ADQuery.Constraints.Add.CustomConstraint := 'FIELD_NAME > 1';
ADQuery.UpdateConstraints;
ADQuery.Table.Constraints.Check(ADQuery.GetRow(), rsModified, ctAtEditEnd);
```

It's working! Why?

A: Also OK.

Q: What exactly mean ctAtEditEnd and ctAtColumnChange?

A: These enums are specifying the event, when FireDAC should check constraints:

- ctAtEditEnd - the Post is called
- ctAtColumnChange - a field value is modified

Now explanation:

```
ADQuery.FieldByName('FIELD_NAME').CustomConstraint := 'FIELD_NAME > 1';
```

This is adding a field-level constraint. They are checked at ctAtColumnChange event only.

```
ADQuery.Constraints.Add.CustomConstraint := 'FIELD_NAME > 1';
```

This is adding a record-level constraint. They are checked at ctAtEditEnd event only.

1.16.10 GUI Questions

The list of questions and answers related to the FireDAC GUI.

Description

QU1: How to show "Application is busy" dialog, while the long running query is executing ?

A: Set ADQuery.ResourceOptions (see page 652).CmdExecMode (see page 880) to amCancelDialog, drop TADGUIxAsyncExecuteDialog (see page 669) to a form, then prepare a test with a run long running query and run it.

While the query is executing, FireDAC will show a dialog with the "Wait" label and the "Cancel" button. If user will press the button, then the query execution will be canceled.

QU2: How localize the ADGUIxLoginDialog ?

A: Use the code like that:

```
with ADGUIxLoginDialog1.VisibleItems do
begin
  Clear;
  Add('User_Name=<local phrase>');
  Add('Password=<local phrase>');
end;
```

Also, you can open uADStanResStrs unit, find there a section "// Dialog captions" and translate there the items.

QU3: How to handle errors using TADGUIxErrorDialog ?

A: Just drop the TADGUIxErrorDialog (see page 672) component on your form or data module. Then for the EADDBEngineException (see page 833) unhandled exceptions the FireDAC error dialog will be shown. To achieve that FireDAC hooks the TApplication.OnException event.

QU4: I see you are using syntax highlighting in the design-time editors. How I can do that on my workstation ?

A: You need to do following:

- download appropriate for your Delphi version SynEdit archive. You can download SynEdit from da-soft site.
- extract archive to some folder;

- run Delphi IDE;
- add <synedit>\Source folder to the Library Path;
- open and install appropriate <synedit>\Packages\SynEdit_R<n>.dpk;
- open and install appropriate <synedit>\Packages\SynEdit_D<n>.dpk;
- if FireDAC is not installed, then install it. Otherwise perform the following steps:
  - close Delphi IDE;
  - remove FireDAC\Bin_noSynedit.txt
  - goto FireDAC\Bin folder;
  - run compileDXxxx.bat.

QU5: How can I turn off the SQL hourglass completely?

A: a) To disable wait cursor forever for an application use TADGUIxWaitCursor (see page 683) with Provider (see page 672) = 'Console'. The 'Console' provider contains an empty wait cursor implementation and no more wait cursor will be showed by FireDAC. If a mouse cursor is still changing, then check, that only uADGUIxConsoleWait unit is included into your application and uADGUIxFormsWait is not included. Note, that you will not have an ability to turn wait cursor on again.

b) To disable wait cursor, but to have ability later to enable it again. Use the code like that to turn off wait cursor:

```
ADWaitCursor1.ScreenCursor := gcrNone;
```

or

```
ADManager.ResourceOptions.SilentMode := True;
```

c) To disable wait cursor and FireDAC dialogs, but to have ability later to enable it again. Set the ADManager (see page 570).SilentMode (see page 375) property to True, then this will disable all wait cursors and FireDAC dialogs, including:

- error dialog
- async execution dialog
- login dialog
- script progress dialog

Setting ResourceOptions (see page 652).SilentMode (see page 884) to True, disables only wait cursors.

QU6: When I apply filter to a dataset, grid scroll bar remains unchanged. Why it is so ?

Q: I have a have grid connected to ADDataSet. When I filter records like this:

```
Grid.DataSource.Dataset.Filter := 'id_virtual_channel in (1, 2, 3)'
```

then everything is OK, but scroll bar remains unchanged like there would were all rows. It looks quite bad when for example full grid has 500 rows and filtered only 3 but scroll bar cursor stays very small and you can move it only a very litle (like from first to third row in 500).Do you have some solution?

A: Use the code like that:

```
ADQuery1.FetchOptions.RecordCountMode := cmVisible;
```

QU7: After refreshing a dataset a DBGrid with multiple selected rows looses the selection. How to preserve selection ?

A: FireDAC bookmarks are invalidated after a Refresh call. You can save PK values of the selected records before Refresh call, then reselect these record again using Locate.

1.16.11 SQL Scripts Questions

The list of questions and answers related to the SQL Script execution.

Description

QR1: Why I can't use semicolon at the end of a query? If I take this off from the query, it works.

A: The ';' in many SQL script dialects is a delimiter of the SQL commands and is not a part of the SQL language. BTW, some of the DBMSs allows ';' at end of SQL command, others - does not. IOW, just remove ';'.

QR2: Some DBMS's (PG, Oracle, FB) allow to execute only single SQL statement, but SQlite allow multiple statements to be execute at once. Why it is so ?

A: This is by design – the ExecSQL method transfers a SQL command to DBMS API as is. If a DBMS supports the "batch" queries, then it will execute the query, otherwise it will fail. TADScript (see page 685) allows to execute SQL scripts with multiple SQL commands and script control commands.

QR3: Even if I have more than one script, with ExecuteAll only first will be executed.

A: The first script is a "root" script. To execute other scripts, you should call then explicitly from "root" script. For example:

```
with ADScript1.SQLScripts do begin
  with Add do begin
    Name := 'root';
    SQL.Add('@first');  // explicitly call 'first' script
    SQL.Add('@second'); // explicitly call 'second' script
  end;
  with Add do begin
    Name := 'first';
    SQL.Add('create table t1;');
    SQL.Add('create table t2;');
  end;
  with Add do begin
    Name := 'second';
    SQL.Add('create procedure p1;');
    SQL.Add('create procedure p2;');
  end;
end;
```

ExecuteStep (see page 694) executes next script command starting from TADScript.Position (see page 688) position. ExecuteAll (see page 691) executes script in full. There is also ValidateAll (see page 694) / ValidateStep (see page 695). These methods process script, but does not execute SQL commands. At end ValidateAll (see page 694) call assigns value to TADScript.TotalJobSize (see page 690). So, next ExecuteAll (see page 691) call will correctly update TotalPct10Done (see page 690), which is 10*% of processed script commands.

QR4: If there a difference in performance between (1) execute ADConnection.ExecSQL for each SQL statement (command) and (2) execute ADScript.ExecuteAll ?

A: The SQL execution code behind each of the methods is the same. There is used IADPhysCommand.Execute method.

The TADScripts (see page 685) parser is highly optimized, knows many SQL dialect aspects, like a Oracle PL/SQL (where ';' may be in the middle of a command), and allows very soft controll.

So, if you submits command by command to the ADConnection.ExecSQL (see page 343) and there is no need to extract these commands from a script or how else, then it will be a fastest method. If you have a SQL script in a file, then ADScript.ExecuteAll (see page 691) will be the fastest method.

QR5: How I can rollback a script work, when it fails ?

A: 1) Use FireDAC transaction control:

```
ADConnection1.StartTransaction;
try
  ADScript1.ExecuteAll;
  ADConnection1.Commit;
except
  ADConnection1.Rollback;
  raise;
end;
```

2) Use PL/SQL block in case of Oracle or similar constructions for other DBMS's:

```
begin
  insert into u_btk.t_test values (1, sysdate);
  insert into u_btk.t_test values (2, sysdate);
  insert into u_btk.t_test values (1, sysdate);
  commit;
exception
  when others then
    rollback;
    raise;
end;
```

3) Use TADScript.OnError (see page 699):

```
procedure TForm1.ADScript1Error(ASender: TObject;
  const AInitiator: IADStanObject; var AException: Exception);
begin
  ADConnection1.Rollback;
end;
```

QR6: When I am executing this script on FB I get “Unexpected end of command line 3” ? How to fix this ?

EXECUTE BLOCK

AS

DECLARE VARIABLE MYVAR VARCHAR(250);

BEGIN

…

END

;

A: To execute a script with blocks you should change the command separator before a block and optionally return it back after. For example:

SET TERM #;

EXECUTE BLOCK

.....

END;

#

SET TERM ;#

1.16.12 Metadata Questions

The list of questions and answers related to the metadata retrieval.

Description

QT1: Is there any method to check if table exist inside DB?

A: There are two basic methods:

1)

```
try
  ADQuery1.Open('select * from tab where 0 = 1');
  Result := True;
except
  on E: EADDBEngineException do
    if E.Kind = ekObjNotExists then
      Result := False
    else
      raise;
end;
```

2)

```
var
  oList: TStrings;
begin
  oList := TStringList.Create;
  try
    GetTableNames('', '', ATableName, oList, [osMy, osOther, osSystem],
      [tkTable, tkTempTable, tkLocalTable]);
    Result := oList.Count > 0;
  finally
    oList.Free;
  end;
end;
```

The first one is more optimal.

QT2: For oracle, is it possible to get the package procedures using the TADMetaInfoQuery ?

A: The following code returns a list of procedures for the PACKAGE_NAME Oracle package.

```
ADMetaInfoQuery1.BaseObjectName := 'PACKAGE_NAME';
ADMetaInfoQuery1.MetaInfoKind := mkProcs;
ADMetaInfoQuery1.Open;
```

QT3: How to get index names using the TADMetaInfoQuery ?

A: The following code returns a list of indexes for the MY_TAB table.

```
ADMetaInfoQuery1.ObjectName := 'MY_TAB';
ADMetaInfoQuery1.MetaInfoKind := mkIndexes;
ADMetaInfoQuery1.Open;
```

QT4: Query Builder, design-time editors and metadata retrieval functions return object names belonging to my current schema, prefixed with schema / catalog name. How to exclude it ?

A: FireDAC has two general connection definition parameters - MetaDefCatalog and MetaDefSchema. Depending on DBMS

either or both of them are supported (see FireDAC Database Connectivity for details). If object belongs to a specified MetaDefCatalog catalog / MetaDefSchema schema, then this catalog / schema name will be excluded from an object name. So, set MetaDefCatalog / MetaDefSchema to your development catalog / schema name, if you need to deploy your application to different catalog / schema.

QT5: I am calling Oracle stored procedure using public synonym, but FireDAC always appends schema name to stored proc name. How to avoid that ?

A: FireDAC has two general connection definition parameters - MetaCurCatalog and MetaCurSchema. Depending on DBMS either or both of them are supported (see FireDAC Database Connectivity for details). Setting them to '*' will avoid usage of a corresponding name part in a full object name.

1.16.13 Debugging and Reporting Environment Questions

The list of questions and answers related to the debugging and reporting environment.

Description

QD1: How can I obtain Client / Session info like ADExplorer?

A: DBMS Environment Reports (see page 170)

QD2: So, how I can produce trace / monitor output ?

A: Tracing and Monitoring (see page 174)

QD3: How to temporarily enable / disable trace / monitor output ?

A: Use TADMoniXxxxClientLink.Tracing. It enables or disables tracing output for specific tracer kind. Also, to enable or disable trace output for specific established connection use the ADConnection.ConnectionIntf (see page 326).Tracing.

QD4: What is the meaning of EventKind's ?

A: The list:

- LiveCycle: Object creation / destruction. For example, IADPhysConnection is created.
- Error: DBMS errors.
- ConnConnect: Connection open / close.
- ConnTransact: Start / Commit / Rollback.
- ConnService: Some special events.
- CmdPrepare: IADPhysCommand.Prepare calls.
- CmdExecute: IADPhysCommand.Execute or Open calls.
- CmdDataIn: Command parameter values.
- CmdDataOut: Resultset rows.
- AdaptUpdat: All about posting updates.
- Vendor: The low level DBMS API calls.
- Component: Some high level events.

QD5: In which unit and where in the code can I view the SQL statement, I need to do some debugging ?

A: You can do that using one of the following options:

- You can use MonitorBy (see page 188) connection parameter.
- Open uADPhysManager.pas, find there TADPhysCommand.Prepare method and there is the InternalPrepare method call. Set breakpoint there and check value of the FDbCommandText field. It is the SQL command text, as it will be sent to a DBMS.

QD6: How to get the final SQL statement with all the params and macros replaced ?

A: See TADQuery.Text (see page 397). Note, that FireDAC does not substitute parameter values into the final SQL command text.

QD7: I enabled trace output and getting constant Access Violations. What is wrong ? Is it a bug ?

A: FireDAC for the ODBC based drivers uses the Microsoft ODBC driver manager tracing facilities. To produce the trace output manager internally uses C RTL functions IsBadReadPtr / IsBadWritePtr, which internally can generate a controlled AV. The exception will be not propagated outside of the manager, but IDE debugger stops on it. Just ignore them.

1.16.14 Integration with 3d Party Products Questions

The list of questions and answers related to integration with 3d party products.

Description

QP1: Is there FastReport (what else) add-on and is it supported officially?

A: Please contact FastReport.

QP2: Unidirectional query and DevExpress grid. With Query.FetchOptions.Unidirectional = True TDataset(Query) is not set as Unidirectional (TDataset(Qurey).IsUnidirectional = False). Because of that there are problems with DevExpress grid.

A: The problem is that setting TDataset(Qurey).IsUnidirectional to True will break TADDataSet functionality. At moment, I does not have a correct solution and need additional investigation. I will suggest to modify DevEx sources to overcome this issue. For that:

- open ExpressDataController\Sources\cxDBData.pas unit
- find there TcxDBDataProvider.IsUniDirectional and replace it with:

```
function TcxDBDataProvider.IsUniDirectional: Boolean;
begin
  if Assigned(DataSet) then begin
  {$IFDEF DELPHI6}
    if DataSet.IsUniDirectional then
      Result := True
    else
  {$ENDIF}
      if DataSet is TADRdbmsDataSet then
        Result := TADRdbmsDataSet(DataSet).FetchOptions.Unidirectional
      else
        Result := False;
  end
  else
    Result := False;
end;
```

1.16.15 Firebird and Interbase Servers Questions

The list of questions and answers related to Firebird and Interbase Servers.

Description

QF1: How I can set up a DLL path+name for fbclient.dll ?

A: The general approach is described in Configuring Drivers (see page 31). To configure a DLL at run time per application basis, do the following:

- drop TADPhysIBDriverLink (see page 777) component;
- set VendorLib (see page 782) property to a DLL path.

If you going to configure a DLL at design time too, then better will be to use ADDrivers.ini:

```
[IB]
VendorLib=C:\Program Files\Firebird 2.5\bin\fbclient.dll
```

QF2: Which DriverName must be used with Firebird 2.5 – IB, FB21 or ?

A: IB is the "base" driver ID. FB21 is the "virtual" driver ID. Virtual drivers are configured in the file FireDAC\DB\ADDrivers.ini. There you can see how is configured the FB21. It is the IB base driver configured to use appropriate fbclient.dll. You can create your own virtual drivers. For example:

```
[Firebird25]
BaseDriverID=IB
VendorLib=C:\Program Files\Firebird 2.5\bin\fbclient.dll
```

QF3: How to force FireDAC to recognize some field as boolean ?

A: A boolean field may be created using a domain. The domain name must contain 'BOOL' substring. And add ExtendedMetadata (see page 198)=True parameter to your connection definition. Example:

```
CREATE DOMAIN T_BOOLEAN SMALLINT;
CREATE TABLE ... (
  ...
  BOOLEAN_FIELD T_BOOLEAN,
  ...);
```

QF4: How to force FireDAC to recognize some field as GUID ?

A: A GUID field may be created using a domain. The domain name must contain 'GUID' substring. And add ExtendedMetadata (see page 198)=True parameter to your connection definition. Example:

```
CREATE DOMAIN T_GUID AS CHAR(16) CHARACTER SET OCTETS COLLATE OCTETS;
CREATE TABLE ... (
  ...
  GUID_FIELD T_GUID,
  ...);
```

QF5: Is there a possibility to drop a Firebird database by sql ?

A: It is not possible with TADQuery, because DROP DATABASE command is not a FB SQL command, but it is ISQL command. Instead specify DropDatabase=Yes (see page 198) in connection definition parameters or use TADScript with DROP DATABASE command (see page 92).

QF6: How to change a User Password in Firebird?

A: FireDAC support password changing on Firebird 2.5 (additionally to Oracle, MySQL, PostgreSQL, SQL Server and SQLite drivers) using the code:

```
ADConnection1.Connected := True;
ADConnection1.ConnectionIntf.ChangePassword('new password');
```

QF7: How to mark a database as read only ?

A: Specify in the connection definition parameters:

```
IBAdvanced=set_db_readonly=1
```

QF8: Looks like AutoCommit works with Firebird only if FetchOptions.Mode = fmAll or does not work at all. Could you please explain how AutoCommit really works ?

A: The background is - the COMMIT will invalidate all the cursors open in this transaction. FireDAC handles transactions automatically using following rules:

- if you preparing a command, and there is no active transaction, then it will be started;
- if you will explicitly call Commit method, then all open in this transaction datasets will be brought into Offline state (FetchAll + closing + unpreparing underlying command, but not a dataset itself);
- if transaction does not have more than single active cursor, then after fetching last record from this cursor it will be auto-committed (if requested by options).
- if transaction does not have any active cursors, then after executing (ExecSQL) a command, the transaction will be auto-committed (if requested by options).

There are even more nuances, but the main rules are listed above.

Automatic committing will be issued after the last record is fetched. We cannot change the behavior, so right after Open will be performed FetchAll and then Commit. As there will be no difference between fmOnDemand and fmAll.

If you need to make the changed and posted to the DB records of this dataset visible to the other sessions, while not all records are fetched from the dataset, then just use second update transaction object. So, cursor transaction will remain active and updates will be executed in the different short transactions. That will make the changes visible to the other users.

QF9: How to connect using an alias ?

A: Just set the Protocol parameter to an empty string or do not specify it at all.

QF10: What does "[FireDAC][Phys][IB]connection rejected by remote interface" mean ?

A: You are probably connecting to the Firebird server using GDS32.DLL.

QF11: What does "Assertion failure (u:\da-soft\FireDAC\source\uADPhysIBWrapper.pas, line 2052)" mean ?

A: You are probably connecting to the Interbase server using FBCLIENT.DLL.

QF12: What does "no permission for read/select access to TABLE xxxx" with FB Embedded mean ?

A: Although the security database is not used by the Embedded server, it still uses the user name to check rights. If the user name is not specified or is specified incorrectly, you get this error.

1.16.16 MS SQL Server Questions

The list of questions and answers related to MS SQL Server.

Description

QZ1: SQL Server 2005: is it possible to use the 'Shared Memory' transport with 'SQL Native Client'?

A: In general:

- check using "Client network utility", that Shared Memory protocol is enabled;
- use Server=(local) in your connection definition.

QZ2: How to get OriginTabName with SQL Server 2005 ?

A: Add ExtendedMetadata=True (see page 129) connection definition parameter.

QZ3: Calling a stored procedure, I am getting »[FireDAC][Phys][ODBC]-345. Data too large for variable [#3]. Max len = [2], actual len = [14]«. What is wrong ?

A: That happens in most cases, when a stored procedure has a VARCHAR parameter defined without specifying a size. For example:

```
PROCEDURE ANALYZETABLE @T_OWNER VARCHAR, @T_TABLE VARCHAR AS
DECLARE @FOOBAR INTEGER;
BEGIN
  /* DUMMY PROCEDURE JUST FOR COMPATIBILITY PURPOSE */
  SET @FOOBAR = 1;
END;
```

The following code will work:

```
with ADStoredProc1 do begin
  FetchOptions.Items := FetchOptions.Items - [fiMeta];
  StoredProcName := 'ANALYZETABLE';
  Params.Clear;
  Params.CreateParam(ftString, 'T_OWNER', ptInput).Size := 20;
  Params.CreateParam(ftString, 'T_TABLE', ptInput).Size := 20;
  Prepare;
  ParamByName('T_OWNER').AsString := '';
  ParamByName('T_TABLE').AsString := TableName;
  ExecProc;
end;
```

Note:

- Exclude fiMeta from FetchOptions (see page 486).Items (see page 855), so FireDAC will not fetch stored procedure parameters definition. That is required, because ODBC driver describes @T_OWNER VARCHAR as VARCHAR(1).
- Specify all parameters properties, including Size before Prepare or first ExecProc (see page 410) call.

QZ4: Calling a stored procedure, I am getting »[FireDAC][Phys][ODBC][Microsoft][SQL Server Native Client 10.0][SQL Server]Line 1: Incorrect syntax near '{'«. What is wrong ?

A: It is a known issue. It happens when:

- Connection is using SQL Server Native Client 2008 ODBC driver;
- The DBMS is SQL Server 2000;
- And a stored procedure has a BLOB input parameter.

At moment only workaround is to use SQL Server 2000 ODBC Driver or SQL Native Client 2005, when you are connecting to the Microsoft SQL Server 2000.

QZ5: Calling Array DML command, I am getting »"[FireDAC][Phys][ODBC][Microsoft][SQL Server Native Client 10.0]String data, length mismatc«. What is wrong ?

A: It is a known for me issue. It seems it is a bug in Microsoft SQL Native Client ODBC driver. For a while, we have not found a proper solution for that. The issue happens, when one of the parameters has a blob data type (ftBlob, ftMemo, etc). As a workaround set ResourceOptions.ArrayDMLSize to 1.

QZ6: Why is the application raising the "Invalid object name '#<my temp table name>'" exception?

A: The FireDAC application may raise the exception above, when it is working with the Microsoft SQL Server local temporary tables. To demonstrate this, the following code reproduces the issue:

```
ADQuery1.ExecSQL('select * into #TmpOrd from [Orders]');
ADQuery1.Open('select * from #TmpOrd');
```

To resolve the issue, set TADQuery.ResourceOptions (see page 498).DirectExecute (see page 881) to True. Also this is required, when application is extensively using:

- local temporary tables in the client SQL;
- and/or the dynamic client SQL.

As an option, consider to use global temporary tables.

QZ7: I am getting strange SQL error (8155) "No column was specified for column 1 of 'A' ". What is wrong ?

A: It seems an application sets FetchOptions.RecsMax to a value greater than zero and executes a query with expressions in SELECT list. For example:

```
SELECT MIN(MyField) FROM MyTable WHERE MyIdField > 0
```

In that case FireDAC modifies a query to:

```
SELECT TOP 10 * FROM (
  SELECT MIN(MyField) FROM MyTable WHERE MyIdField > 0
) A
```

On SQL Server this syntax fails with the above error. To resolve this issue, specify the aliases for expressions in SELECT list.

QZ8: I am failing to get Chinese characters (Big5 encoding) from a database. How to fix that ?

A: Try to add connection definition parameter:

```
ODBCAdvanced=AutoTranslate=no
```

And add the mapping rule:

```
  object ADConnection1: TADConnection
.....................
    FormatOptions.AssignedValues = [fvMapRules]
    FormatOptions.OwnMapRules = True
    FormatOptions.MapRules = <
      item
        SourceDataType = dtAnsiString
        TargetDataType = dtWideString
      end>
  end
```

Also check that you have set correct Chinese database character set and not Latin1.

QZ9: When I insert a '2011-11-13 00:00' as a datetime value I encounter the error "The conversion of a varchar data type to a datetime data type resulted in an out-of-range value". What am I doing wrong?

A: According to MSDN the used datetime format does not conform to the international standard. There are several solutions to avoid the error:

1. Using of SET DATEFORMAT option. For example, execute this to set the order of the year, month and day:

```
ADConnection1.ExecSQL('set dateformat ymd');
```

2. Using CONVERT function:

```
ADConnection1.ExecSQL('insert into Test (date_val) values (convert(datetime, ''2011-11-13 00:00'', 120)');
```

3. Changing the format of your data to international standard:

```
ADConnection1.ExecSQL('insert into Test (date_val) values(''2011-11-13T00:00:00'');
```

QZ10: Posting updates to SQL Server table, I am getting error "Update command updated [N] instead of [1] record". What may be a reason for that ?

A: In most cases with SQL Server this error happens when the table has a trigger which modifies the DB explicitly, or implicitly by calling a stored procedure which modified the DB. Then FireDAC receives not the number of records updated by UPDATE command, but the number of records modified by the trigger.

To avoid that insert SET NOCOUNT ON at trigger beginning. Or set UpdateOptions.CountUpdatedRecords (see page 899) to False.

QZ11: I declared some table columns as DATETIME2 / DATE / TIME and FireDAC returns them as WideString. Or - I am getting Type mismatch, expecting Date, actual WideString. What is wrong ?

The DATETIME2 / DATE / TIME were introduced in SQL Server 2008. "SQL Server" (SQL Server 2000) ODBC driver does not know these types and maps them to WideStrings. The SQL Server Native Client v 10 knows them and represents correctly. To resolve the issue you should install SQL Server Native Client v 10 (see page 202). It is not installed by default, but "SQL Server" ODBC driver is installed by default.

1.16.17 Oracle Server Questions

The list of questions and answers related to Oracle Server.

Description

QO1: I am looking at putting my application on a PC that already runs an Oracle client so don't want to use the registry keys. I want to make the path and TNS_Admin variables specific to my application only.

A: Just put tnsnames.ora file in the same folder, where Oracle Instant Client is located. No need to do something other, like specifying TNS_Admin or TNS_Names. If you does not need to use TNS aliases, then put empty tnsnames.ora file.

QO2: I am getting error »Cannot initialize OCI environment« in D2009 application. What is wrong ?

A: FireDAC cannot initialize OCI in the Unicode mode, that is required for D2009. Most probably, that is due to the old Oracle client version. We will suggest:

- upgrade Oracle client software at least to v 9.0;
- or stay with a non-Unicode Delphi (<= D2007).

QO3: I need to connect to an Oracle 8.0.5 server. Is that possible ?

A: Using the following "roadmap" you can choose the correct Oracle client and Delphi versions:

- FireDAC with Delphi 2009 or higher requires at least Oracle 8.1.5 client. Because only starting from Oracle 8.1.5 the client works correctly with Unicode.
- FireDAC with Delphi 2007 or less requires at least Oracle 8.0.3 client.

QO4: How to lock table with Oracle ?

A: Generic skeleton for your task may look like:

```
ADConnection1.StartTransaction;
try
  ADConnection1.ExecSQL('LOCK TABLE ...');
  .........
  ADConnection1.Commit;
except
  ADConnection1.Rollback;
  raise;
end;
```

QO5: How to get a list of Oracle services, like FireDAC is doing that in Login dialog ?

A: Use:

- TADPhysOracleDriverLink (see page 800).GetOracleHomes (see page 801) – to get a list of known Oracle homes;
- TADPhysOracleDriverLink (see page 800).GetTNSServices (see page 801) – to get a list of TNS aliases from the primary Oracle home.

QO6: Is the Oracle Objects extension supported ?

A: Not at this moment. As a workaround you can build PL/SQL wrapping procedures and functions. Using this approach, you can then work with Oracle Advanced Queuing.

1.16.18 SQLite Database Questions

The list of questions and answers related to SQLite Database.

Description

QL1: How can I define collate sequence in SQLite DB?

A: There are completed collations, functions, extensions API's and demos. Check "Demos\DBMS Specific\SQLite\UserCollation".

If you need collations / functions at design-time make design-time package (fake component) where you register user collations / functions.

QL2: SQLite exception "[FireDAC][DApt]-401. Record is locked by another user."

A: The exception is raised on UPDATE command, when another DB connection has cursor on the same table and not all

records are fetched from it. You must use FetchOptions (see page 486).Mode (see page 857)=fmAll. Even you can set this option at ADConnection level, so all linked to it datasets will inherit this option.

QL3: Data type for non-table SELECT list item is wrong ? (or) My aggregated function returns string value. What is wrong ?

A: SQLite does not return a data type for non-table SELECT list items. IOW, if an item is an expression, then FireDAC cannot get it data type. So, it just describes it as ftWideString. FireDAC has ability to specify SELECT item data type: <item> as "<alias>::<data type>". For example:

```
SELECT count(*) as "cnt::int" from ...
```

QL4: It seems that character macro are not supported with SQLite? I try with CONCAT, LEFT, SUBSTR, UCASE, ..

A: These functions are implemented using FireDAC expression evaluator. When you are creating a connection at run-time, you should include uADStanExprFuncs.pas into "uses" clause.

SQLite has a limited set of build-in functions. But it allows to write and register custom functions. FireDAC has many custom functions implemented for the expression evaluator. They implementations and registrations are in ADStanExprFuncs. SQLite driver automatically registers all known expression evaluator functions with sqlite3.dll.

QL5: Is sqlite3_progress_handler implemented with FireDAC ?

A: Yes. Use the code like that:

```
procedure Tform1.FormCreate(Sender: TObject);
begin
  ADConnection1.Open;
  with TSQLiteDatabase(ADConnection1.ConnectionIntf.CliObj) do
  begin
    ProgressNOpers := 50000;
    OnProgress := SQLiteOnProgress;
  end;
end;

procedure Tform1.SQLiteOnProgress(ADB: TSQLiteDatabase; var ACancel: Boolean);
begin
  Application.ProcessMessages;
end;
```

QL6: Our WAL files are getting huge (>1 GB) when working with our multi-threaded application. Do you have any suggestion on how to prevent WAL files from growing?

A: If WAL file keeps growing that means it's impossible to move data from the log into the database file. Probably the database is permanently used by reading threads and/or there are performed DML operations without committing a transaction. The possible solutions in this case are to use only short transactions or to create a separate thread for DML operations (in this case transactions should also be constantly committed).

If a checkpoint runs and copies all WAL data into the database file, the next writer starts writing into the start of the WAL file again. The WAL file is not usually truncated (see PRAGMA journal_size_limit if you want it to be) here. The reason being that it is faster to overwrite an existing file than it is to truncate one and then start appending to it.

So, if all goes well, SQLite should start over at the start of the WAL file after each checkpoint. Preventing the WAL file from growing indefinitely. There are two things that can go wrong:

- A reader might prevent a check-pointer from copying all data from the WAL into the database file, or
- While the checkpoint is underway, some other process may be writing to the database (appending to the WAL file).

If either of the above occurs, then the next writer will append to the WAL file, instead of writing into the start of it. If this

happens every checkpoint, then the WAL file will grow without bound.

1.16.19 MySQL Server Questions

The list of questions and answers related to MySQL Server.

Description

QY1: Is there any way to use MySQL C API function mysql_thread_id from FireDAC?

A: Yes, although a bit cryptic:

- ADConnection1.ConnectionIntf (see page 326).Driver.CliObj - returns reference to TMySQLLib;
- ADConnection1.ConnectionIntf.CliObj - returns reference to TMySQLSession, which has MySQL property;
- alltogether:

```
uses
  uADPhysMySQLCli, uADPhysMySQLWrapper;
...
  with ADConnection1.ConnectionIntf do
    TMySQLLib(Driver.CliObj).mysql_thread_id(TMySQLSession(CliObj).MySQL);
```

QY2: Update blob field with data greater then 512KB on MySQL cause "MySQL server has gone away" error.

A: Please, check: http://dev.mysql.com/doc/refman/5.0/en/gone-away.html and at first the item, which suggest to increase max_allowed_packet.

QY3: Is it possible to establish SSL connection to a MySQL server ?

A: Yes, it is possible. For that set UseSSL=True for a connection definition. Then specify SSL_key, SSL_cert, SSL_ca, SSL_capath and SSL_cipher parameters. For the parameters meaning, please, read: http://dev.mysql.com/doc/refman/5.1/en/ssl-options.html. Note, that you can also use the my.cnf configuration file.

QY4: Why application hangs on for few second at exiting ? I am using only MySQL connection.

A: If your application creates threads, this issue (AD-58) is possible with libmysql.dll v 5.x. Please, read http://bugs.mysql.com/bug.php?id=37226 for details. As workaround try to terminate all threads before terminating the application.

1.16.20 MS Access Questions

The list of questions and answers related to MS Access.

Description

QA1: I am getting error "Too few parameters. Expected N". What that means ?

A: Most probably your SELECT query references to the columns, which does not exist in a table.

1.17 Symbol Reference

1.17.1 uADCompClient Namespace

Contains core data access components, including TADConnection (see page 279), TADQuery (see page 476), TADStoredProc (see page 514), TADMemTable (see page 436), etc.

Classes

Class	Description
TADAdaptedDataSet (see page 259)	The TADAdaptedDataSet class uses data adapter to execute DBMS queries, open DBMS cursors, fetch rows and post changes back to DBMS.
TADCommand (see page 267)	The class is responsible for execution of SQL commands.
TADConnection (see page 279)	The class is responsible to connection establishment with a DBMS.
TADCustomCommand (see page 291)	The base class is responsible for execution of SQL commands.
TADCustomConnection (see page 318)	The class is responsible to connection establishment with a DBMS.
TADCustomEventAlerter (see page 358)	The class is responsible for handling the database event notifications.
TADCustomLocalSQL (see page 363)	The base class for the Local SQL engine.
TADCustomManager (see page 365)	The class is responsible to connection definitions and connections management.
TADCustomMemTable (see page 388)	The class implementing in-memory dataset.
TADCustomQuery (see page 394)	The class implementing dataset, capable to execute SQL (see page 396) queries.
TADCustomSchemaAdapter (see page 400)	The class implementing Centralized Cached Updates (see page 114) support.
TADCustomStoredProc (see page 403)	The class implementing dataset, capable to execute server side stored procedures.
TADCustomTransaction (see page 413)	The class is responsible for connection transaction management.
TADCustomUpdateObject (see page 420)	The base class for update objects used to override updates posting.
TADEventAlerter (see page 422)	The class is responsible for handling the database event notifications.
TADLocalSQLDataSet (see page 425)	TADLocalSQLDataSet represents a dataset registered with Local SQL engine.
TADLocalSQLDataSets (see page 428)	TADLocalSQLDataSets is a collection of TADLocalSQLDataSet (see page 425) objects.
TADManager (see page 429)	The class is responsible to connection definitions and connections management.
TADMemTable (see page 436)	The class implementing in-memory dataset.
TADMetaInfoCommand (see page 452)	The class is responsible for execution of meta-data commands.
TADMetaInfoQuery (see page 461)	The class implementing dataset, capable to execute meta-info queries.
TADQuery (see page 476)	The class implementing dataset, capable to execute SQL (see page 499) queries.
TADRdbmsDataSet (see page 501)	The TADRdbmsDataSet class includes everything, required by descendants to execute SQL commands.
TADSchemaAdapter (see page 514)	The class implementing Centralized Cached Updates (see page 114) support.
TADStoredProc (see page 514)	The class implementing dataset, capable to execute server side stored procedures.
TADTable (see page 539)	The class implementing dataset, working with single database table.
TADTransaction (see page 561)	The class is responsible for connection transaction management.
TADUpdateSQL (see page 563)	TADUpdateSQL applies updates on behalf of queries or stored procedures that can't post updates directly.

Functions

Function	Description
ADManager (see page 570)	Function returns reference to global TADCustomManager (see page 365) object.
ADSetConnectionClass (see page 570)	Sets the TADCustomConnection (see page 318) descendant class.
ADSetManagerClass (see page 571)	Set the TADCustomManager (see page 365) descendant class.

Description

The uADCompClient unit contains most of the core data access components:

- TADManager (see page 429) - class managing all FireDAC connections (see page 27);
- TADConnection (see page 279) - class representing FireDAC connection (see page 37) to a DBMS;
- TADTransaction (see page 561) - class controlling the DBMS transactions (see page 41);
- TADEventAlerter (see page 422) - class providing an unified interface to the DBMS events (see page 75);
- TADCommand (see page 267) - class implementing DBMS command;
- TADMemTable (see page 436) - class implementing in-memory table;
- TADTableAdapter;
- TADSchemaAdapter (see page 514) - class servicing Centralized Cached Updates (see page 114);
- TADQuery (see page 476) - class implementing query object (see page 66) with dataset interface;
- TADStoredProc (see page 514) - class implementing stored procedure object (see page 71) with dataset interface;
- TADTable (see page 539) - class implementing table object (see page 73) with dataset interface;

See Also

uADCompDataSet (see page 571), uADCompScript (see page 684)

1.17.1.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADAdaptedDataSet (see page 259)	The TADAdaptedDataSet class uses data adapter to execute DBMS queries, open DBMS cursors, fetch rows and post changes back to DBMS.
TADCommand (see page 267)	The class is responsible for execution of SQL commands.
TADConnection (see page 279)	The class is responsible to connection establishment with a DBMS.
TADCustomCommand (see page 291)	The base class is responsible for execution of SQL commands.
TADCustomConnection (see page 318)	The class is responsible to connection establishment with a DBMS.
TADCustomEventAlerter (see page 358)	The class is responsible for handling the database event notifications.
TADCustomLocalSQL (see page 363)	The base class for the Local SQL engine.
TADCustomManager (see page 365)	The class is responsible to connection definitions and connections management.
TADCustomMemTable (see page 388)	The class implementing in-memory dataset.
TADCustomQuery (see page 394)	The class implementing dataset, capable to execute SQL (see page 396) queries.
TADCustomSchemaAdapter (see page 400)	The class implementing Centralized Cached Updates (see page 114) support.
TADCustomStoredProc (see page 403)	The class implementing dataset, capable to execute server side stored procedures.
TADCustomTransaction (see page 413)	The class is responsible for connection transaction management.

TADCustomUpdateObject (see page 420)	The base class for update objects used to override updates posting.
TADEventAlerter (see page 422)	The class is responsible for handling the database event notifications.
TADLocalSQLDataSet (see page 425)	TADLocalSQLDataSet represents a dataset registered with Local SQL engine.
TADLocalSQLDataSets (see page 428)	TADLocalSQLDataSets is a collection of TADLocalSQLDataSet (see page 425) objects.
TADManager (see page 429)	The class is responsible to connection definitions and connections management.
TADMemTable (see page 436)	The class implementing in-memory dataset.
TADMetaInfoCommand (see page 452)	The class is responsible for execution of meta-data commands.
TADMetaInfoQuery (see page 461)	The class implementing dataset, capable to execute meta-info queries.
TADQuery (see page 476)	The class implementing dataset, capable to execute SQL (see page 499) queries.
TADRdbmsDataSet (see page 501)	The TADRdbmsDataSet class includes everything, required by descendants to execute SQL commands.
TADSchemaAdapter (see page 514)	The class implementing Centralized Cached Updates (see page 114) support.
TADStoredProc (see page 514)	The class implementing dataset, capable to execute server side stored procedures.
TADTable (see page 539)	The class implementing dataset, working with single database table.
TADTransaction (see page 561)	The class is responsible for connection transaction management.
TADUpdateSQL (see page 563)	TADUpdateSQL applies updates on behalf of queries or stored procedures that can't post updates directly.

1.17.1.1.1 TADAdaptedDataSet Class

The TADAdaptedDataSet class uses data adapter to execute DBMS queries, open DBMS cursors, fetch rows and post changes back to DBMS.

protected

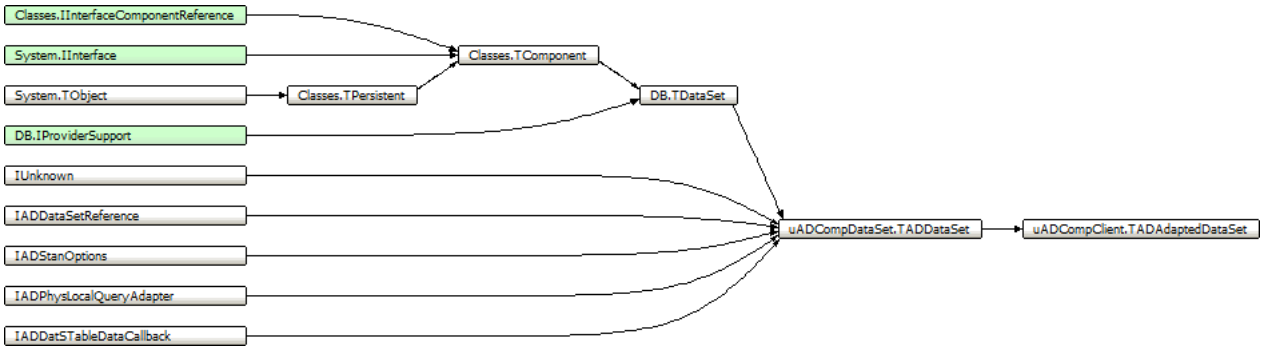
protected	Description
UpdateObject (see page 267)	Specifies the update object component used to post updates by dataset.

public

public	Description
R Adapter (see page 260)	The reference to a data adapter.
R Command (see page 261)	The reference to a command, used to execute / open queries.
W DatSManager (see page 261)	The reference to the inmemory data storage manager.
LocalSQL (see page 261)	The reference to a Local SQL engine.
OnError (see page 261)	The event is triggered when an error happens, while dataset is talking to DBMS.
OnExecuteError (see page 262)	The event is triggered when an error happens, while executing a Array DML query.
R PointedConnection (see page 263)	The actual connection object.
R ServerEditRequest (see page 263)	Used by TADDataMove to edit directly DBMS data.
AbortJob (see page 263)	Use the AbortJob method to abort current dataset operation.
V AttachTable (see page 264)	Use the AttachTable to attach the dataset to existing DatS table or view.
V Disconnect (see page 264)	Disconnects the dataset from DBMS.
GetResults (see page 265)	Forces DBMS to return the output parameter values.
NextRecordSet (see page 265)	Go to the next cursor, returned by the SQL command.
ServerAppend (see page 265)	Used by TADDataMove to edit directly DBMS data.
ServerCancel (see page 266)	Used by TADDataMove to edit directly DBMS data.

ServerDelete (see page 266)	Used by TADDataMove to edit directly DBMS data.
ServerDeleteAll (see page 266)	Deletes all records from an associated database table.
ServerEdit (see page 266)	Used by TADDataMove to edit directly DBMS data.
ServerGotoKey (see page 266)	Used by TADDataMove to edit directly DBMS data.
ServerPerform (see page 267)	Used by TADDataMove to edit directly DBMS data.
ServerSetKey (see page 267)	Used by TADDataMove to edit directly DBMS data.

Class Hierarchy



File

uADCompClient

Description

The TADAdaptedDataSet class is an intermediate class in FireDAC dataset classes hierarchy. The programmer must not use it directly.

The TADAdaptedDataSet class uses data adapter, pointed by Adapter (see page 260) property to execute DBMS queries, open DBMS cursors, fetch rows and post changes back to the DBMS. The dataset itself has no build-in capabilities to work with the DBMS.

For options description, please, look at uADStanOption (see page 840) description.

Syntax

```
TADAdaptedDataSet = class(TADDataSet);
```

See Also

TADDataSet (see page 584), uADStanOption (see page 840)

1.17.1.1.1.1 public

1.17.1.1.1.1.1 TADAdaptedDataSet.Adapter Property

The reference to a data adapter.

Description

Use this property to get/set data adapter for dataset.

The dataset will use data adapter to talk to DBMS, when he needs to execute query, fetch rows, post changes back to DBMS.

Syntax

```
property Adapter: TADCustomTableAdapter;
```

See Also

TADCustomTableAdapter

Example

See FireDAC\Samples\Comp Layer\TADMemTable (see page 436)\IncFetching_MSSQL sample for details of how to use TADMemTable (see page 436) with table adapter.

See FireDAC\Samples\Comp Layer\TADMemTable (see page 436)\MasterDetail sample for details of how to work with master-detail relation.

1.17.1.1.1.1.2 TADAdaptedDataSet.Command Property

The reference to a command, used to execute / open queries.

Description

Property returns the reference to the SELECT command of data adapter. This command is used by the data adapter to execute DBMS queries by one of the Execute (see page 622), ExecSQL (see page 397) and Open dataset methods.

Syntax

```
property Command: TADCustomCommand;
```

See Also

TADCustomCommand (see page 291)

1.17.1.1.1.1.3 TADAdaptedDataSet.DatSManager Property

The reference to the inmemory data storage manager.

Description

The property get/set the reference to the data storage manager. In general, data storage manager is a collection of data storage tables. Each dataset cursor will be represented by one data storage table.

Syntax

```
property DatSManager: TADDatSManager;
```

See Also

TADDatSManager, TADDatSTable

1.17.1.1.1.1.4 TADAdaptedDataSet.LocalSQL Property

The reference to a Local SQL engine.

Description

Use this property to get / set reference to a Local SQL engine.

When engine and Local SQL connection are active, then right after setting LocalSQL property, the dataset will be opened (if not yet) and will be registered in Local SQL engine. Otherwise dataset will be registered when engine and connection will be active. The TADMemTable (see page 436) structure must be defined before it will be opened.

The dataset will be registered using its Name property value. So, its Name property value must be not empty and unique across all datasets linked to the same Local SQL engine.

Setting this property to nil unregisters this dataset from Local SQL engine.

Syntax

```
property LocalSQL: TADCustomLocalSQL;
```

See Also

Local SQL (see page 96), TADLocalSQL (see page 825), TADMemTable (see page 436)

1.17.1.1.1.1.5 TADAdaptedDataSet.OnError Event

The event is triggered when an error happens, while dataset is talking to DBMS.

Parameters

Parameters	Description
ASender	The reference to dataset.
AInitiator	The reference to interface of object, which initially raised the exception.
AException	The exception object, which may be modified or even replaced by another exception object instance.

Description

The OnError event is triggered, when dataset executes one of the following dataset operations and error happens:

- Prepare (see page 511). The dataset is preparing DBMS command for execution.
- Open (see page 509). The dataset is executing DBMS command to open cursor.
- ExecSQL (see page 397)/ExecProc (see page 410)/Execute (see page 622). The dataset is executing DBMS command.
- `Fetch`. The dataset is fetching rows from cursor.

You can use also OnExecuteError (see page 262) event to handle Array DML errors, and OnUpdateError (see page 605) event to handle updates posting errors.

Syntax

```
property OnError: TADErrorEvent;
```

See Also

TADDataSet.Execute (see page 622), TADCustomQuery.ExecSQL (see page 397), TADCustomStoredProc.ExecProc (see page 410), OnExecuteError (see page 262), OnUpdateError (see page 605)

Example

See FireDAC\Samples\Comp Layer\TADQuery (see page 476)\ExecSQL\BatchErrorHandling demo for details.

1.17.1.1.1.6 TADAdaptedDataSet.OnExecuteError Event

The event is triggered when an error happens, while executing a Array DML query.

Parameters

Parameters	Description
ASender	The reference to dataset.
ATimes	The current Array DML size. It may be different from value of the Execute method ATimes parameter, if full operation is splitted into few more small (on Firebird, for example).
AOffset	The current Array DML offset. It may be different from value of the Execute method AOffset parameter, if there was skipped erroneous rows or full operation is splitted into few more small (on Firebird, for example).
AError	The original DBMS returned error. Check AError.Errors (see page 834)[i].RowIndex (see page 839) for the index of erroneous row in parameters array.
AAction	The action dataset should take after return from event handler.

Description

The OnExecuteError event is triggered, when the dataset failed to execute Array DML using Execute method and ATimes > 1. Using this event handler you can:

- skip erroneous row of the parameters array;
- change values in erroneous row of the parameters array;
- stop full Array DML execution.

Note, OnExecuteError will be not called for the syntax errors or when ATimes = 1.

Syntax

```
property OnExecuteError: TADExecuteErrorEvent;
```

See Also

Array DML (see page 82), EADDBEngineException (see page 833), TADErrorAction, TADDataSet.Execute (see page 622)

Example

```
procedure TForm1.ADQuery1ExecuteError(ASender: TObject; ATimes,
  AOffset: Integer; AError: EADDBEngineException; var AAction: TADErrorAction);
begin
  if AException.Errors[0].Kind = ekUKViolated then
    AAction := eaSkip
  else
    AAction := eaFail;
end;
```

1.17.1.1.1.7 TADAdaptedDataSet.PointedConnection Property

The actual connection object.

Description

The PointedConnection property returns reference to a connection object, which:

- is referenced by descendant dataset Connection (see page 503) property;
- has the name specified by descendant dataset ConnectionName (see page 503) property.

Syntax

```
property PointedConnection: TADCustomConnection;
```

See Also

TADRdbmsDataSet.Connection (see page 503), TADRdbmsDataSet.ConnectionName (see page 503), TADCustomConnection (see page 318)

1.17.1.1.1.8 TADAdaptedDataSet.ServerEditRequest Property

Used by TADDataMove to edit directly DBMS data.

Syntax

```
property ServerEditRequest: TADActionRequest;
```

1.17.1.1.1.9 TADAdaptedDataSet.AbortJob Method

Use the AbortJob method to abort current dataset operation.

Parameters

Parameters	Description
AWait: Boolean = False	If True, then caller will wait until a current operation will be terminated.

Description

The AbortJob method aborts one of the following dataset operations:

- Open (see page 509). The dataset is executing DBMS command to open cursor.
- ExecSQL (see page 397)/Execute (see page 622). The dataset is executing DBMS command.
- Fetch. The dataset is fetching rows from cursor.

After AbortJob call dataset state will be Active=False and Prepared=True for Open/ExecSQL/Execute. And Active=True for Fetch. The AbortJob method must be called from the other thread, than where a operation is performing.

Syntax

```
procedure AbortJob(AWait: Boolean = False);
```

See Also

Asynchronous Execution (see page 86), TADResourceOptions.CmdExecMode (see page 880), TADResourceOptions.CmdExecTimeout (see page 881), Disconnect (see page 264), TADCustomConnection (see page 318).AbortJob (see page 338)

Example

```
ADQuery1.ResourceOptions.CmdExecMode := amAsync;
ADQuery1.Open('select OrderID, count(*) from "Order Details" group by OrderID');
ADQuery1.AbortJob(True);
```

1.17.1.1.1.10 TADAdaptedDataSet.AttachTable Method

Use the AttachTable to attach the dataset to existing DatS table or view.

Parameters

Parameters	Description
ATable: TADDatSTable	A table to attach.
AView: TADDatSView	A view to attach.

Description

The AttachTable method attaches the dataset to existing DatS table and or view.

After attaching the application may call Open method to open and browse the dataset. This method is primary useful for TADMemTable (see page 436). At least one of the parameters must be not nil.

Syntax

```
procedure AttachTable(ATable: TADDatSTable; AView: TADDatSView); override;
```

See Also

TADMemTable (see page 436)

Example

```
ADMemTable1.AttachTable(oTab, nil) ;
ADMemTable1.Open;
```

1.17.1.1.1.11 TADAdaptedDataSet.Disconnect Method

Disconnects the dataset from DBMS.

Parameters

Parameters	Description
AAbortJob: Boolean = False	If True, then current dataset task will be aborted. The code will wait, while task abortion will be finished.

Description

The Disconnect method optionally aborts current dataset task, like a Open (see page 509), Execute (see page 622), ExecSQL (see page 397) or rows fetching. Then closes dataset and unprepares it.

Syntax

```
procedure Disconnect(AAbortJob: Boolean = False); override;
```

See Also

AbortJob (see page 263)

Example

```
ADQuery1.Disconnect(True);
// is equivalent to:
// ADQuery1.AbortJob(True);
```

```
// ADQuery1.Close;
// ADQuery1.Unprepare;
```

1.17.1.1.1.12 TADAdaptedDataSet.GetResults Method

Forces DBMS to return the output parameter values.

Description

Use GetResults to force a DBMS to return the output parameter values. This method is optional for most cases, because FireDAC will receive the output parameter values automatically. Although, when the command has few active not yet processed result sets, and you do not plan to use these result sets, but going to get output parameter values, then depending on a DBMS you may need to call first the GetResults method.

Syntax

```
procedure GetResults;
```

See Also

Executing Command (see page 66), Executing Stored Procedure (see page 71)

1.17.1.1.1.13 TADAdaptedDataSet.NextRecordSet Method

Go to the next cursor, returned by the SQL command.

Description

The NextRecordSet method closes current cursor, forwards it to the next accessible cursor and opens dataset. If there is no more accessible cursors, then after call dataset is closed. See "Command Batches (see page 81)" chapter for more details.

The Close method closes current cursor. The CloseAll (see page 309) method call discards all associated cursors. So, to get all cursors, you should set FetchOptions.AutoClose (see page 851) to False before opening dataset.

For FireDAC the Oracle and PostgreSQL REF CURSOR is a command cursors, and a NextRecordSet call will select the next REF CURSOR parameter. For SQL Server, Sybase SQL Anywhere, MySQL, etc a NextRecordSet call will select next result set, produced by batch, stored procedure or how else.

In design time, you can forward dataset to the next cursor, by right clicking component and choosing the "Next record set" popup menu item. See the "Executing Command (see page 66)" chapter for more details.

Syntax

```
procedure NextRecordSet;
```

See Also

Executing Command (see page 66), Command Batches (see page 81), TADFetchOptions.AutoClose (see page 851), TDataSet.Open, TDataSet.Close

Example 1

```
ADQuery1.FetchOptions.AutoClose := False;
ADQuery1.SQL.Text := 'select 1 as i; select ''qwe'' as s';
ADQuery1.Open;
ShowMessage(ADQuery1.Fields[0].FieldName + ' ' + ADQuery1.Fields[0].AsString); // output "i
1"
ADQuery1.NextRecordSet;
ShowMessage(ADQuery1.Fields[0].FieldName + ' ' + ADQuery1.Fields[0].AsString); // output "s
qwe"
ADQuery1.Close;
```

Example 2

See FireDAC\Samples\Comp Layer\TADStoredProc (see page 514)\Oracl_NextRecordSet demo for details of how to work with Oracle stored procedure, returning multiple REF CURSOR's.

1.17.1.1.1.14 TADAdaptedDataSet.ServerAppend Method

Used by TADDataMove to edit directly DBMS data.

Syntax

```
procedure ServerAppend;
```

1.17.1.1.1.15 TADAdaptedDataSet.ServerCancel Method

Used by TADDataMove to edit directly DBMS data.

Description

Syntax

```
procedure ServerCancel;
```

1.17.1.1.1.16 TADAdaptedDataSet.ServerDelete Method

Used by TADDataMove to edit directly DBMS data.

Syntax

```
procedure ServerDelete;
```

1.17.1.1.1.17 TADAdaptedDataSet.ServerDeleteAll Method

Deletes all records from an associated database table.

Parameters

Parameters	Description
ANoUndo: Boolean = False	Use TRUNCATE command.

Description

Use ServerDeleteAll method to delete all records from a database table, specified in UpdateOptions.UpdateTableName (see page 846), otherwise in SQL (see page 396) command or TableName (see page 559) property.

Set ANoUndo to True, to empty a table using the TRUNCATE SQL command, if DBMS supports it. Otherwise, method will issue DELETE SQL command without WHERE clause. Note, that on most DBMS's TRUNCATE command is not journaled and cannot be rolled back.

The ServerDeleteAll method may be used on inactive dataset.

Syntax

```
procedure ServerDeleteAll(ANoUndo: Boolean = False);
```

See Also

ExecSQL (see page 343)

Example

```
ADQuery1.SQL.Text := 'select * from {id MyTab}';
ADQuery1.ServerDeleteAll(True);
```

1.17.1.1.1.18 TADAdaptedDataSet.ServerEdit Method

Used by TADDataMove to edit directly DBMS data.

Syntax

```
procedure ServerEdit;
```

1.17.1.1.1.19 TADAdaptedDataSet.ServerGotoKey Method

Used by TADDataMove to edit directly DBMS data.

Syntax

```
function ServerGotoKey: Boolean;
```

1.17.1.1.1.20 TADAdaptedDataSet.ServerPerform Method

Used by TADDataMove to edit directly DBMS data.

Syntax

```
procedure ServerPerform;
```

1.17.1.1.1.21 TADAdaptedDataSet.ServerSetKey Method

Used by TADDataMove to edit directly DBMS data.

Syntax

```
procedure ServerSetKey;
```

1.17.1.1.2 protected

1.17.1.1.2.1 TADAdaptedDataSet.UpdateObject Property

Specifies the update object component used to post updates by dataset.

Description

Use UpdateObject to specify the TADUpdateSQL (see page 563) component to use in an application that must be able to post updates in a non-standard way.

The FireDAC has build in updates command generator, which using multiple information sources is able to generate SQL update commands for most of scenarios. In some cases, such as a query made against multiple tables, which may require multi-statement update, FireDAC cannot generate SQL update commands. In these cases, UpdateObject can be used to specify a TADUpdateSQL (see page 563) component that allows to specify custom SQL commands for each update posting case, like a insert / update / delete.

As alternative to update object usage, you can consider to use OnUpdateRecord (see page 605) event. It gives more options, but requires manual coding of posting updates. Also, in OnUpdateRecord (see page 605) handler you can use multiple update objects. See demo FireDAC\Samples\Comp Layer\TADUpdateSQL\Main.

Syntax

```
property UpdateObject: TADCustomUpdateObject;
```

See Also

OnUpdateRecord (see page 605), TADUpdateSQL (see page 563)

Example 1

```
ADUpdateSQL1.InsertSQL.Text := 'insert into mytab values (:new_id, :new_name)';
ADQuery1.UpdateObject := ADUpdateSQL1;
ADQuery1.Append;
ADQuery1['id'] := 100;
ADQuery1['name'] := 'qwe';
ADQuery1.Post;
```

Example 2

Demo FireDAC\Samples\Comp Layer\TADUpdateSQL (see page 563)\Main.

1.17.1.1.2 TADCommand Class

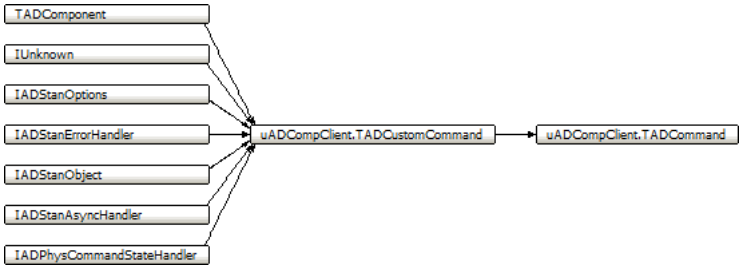
The class is responsible for execution of SQL commands.

published

published	Description
Active (see page 269)	Gets or sets command active status.
ActiveStoredUsage (see page 269)	Controls how to use Active property value saved to DFM.
AfterClose (see page 269)	The event fires after command is closed.

AfterExecute (see page 270)	The event fires after command is executed.
AfterOpen (see page 270)	The event fires after command is opened.
AfterPrepare (see page 270)	The event fires after command is prepared.
AfterUnprepare (see page 270)	The event fires after command is unprepared.
BaseObjectName (see page 271)	Identifies base object name for some of command kinds.
BeforeClose (see page 271)	The event fires before command will be closed.
BeforeExecute (see page 271)	The event fires before command will be executed.
BeforeOpen (see page 272)	The event fires before command will be opened.
BeforePrepare (see page 272)	The event fires before command will be prepared.
BeforeUnprepare (see page 272)	The event fires before command will be unprepared.
CatalogName (see page 272)	Identifies catalog name for some command kinds.
CommandKind (see page 273)	Identifies the command kind.
CommandText (see page 273)	The SQL command text to execute.
Connection (see page 274)	Specifies the connection object.
ConnectionName (see page 274)	Specifies connection name.
FetchOptions (see page 275)	The set of options to control rows fetching.
FormatOptions (see page 275)	The set of options to control data representation.
Macros (see page 275)	The collection of macro substitution variables.
OnCommandChanged (see page 276)	The event fires after CommandText is changed.
OnError (see page 276)	The event fires when an error happens, while command is communicating with DBMS.
Overload (see page 277)	Oracle stored procedure overload index.
Params (see page 277)	The collection of parameters for a SQL command.
ResourceOptions (see page 278)	The set of options to control resources usage.
SchemaName (see page 278)	Identifies schema name for some of command kinds.
Transaction (see page 279)	Specifies a transaction object.
UpdateOptions (see page 279)	The set of options to control updates posting.

Class Hierarchy



File

uADCompClient

Description

Use the TADCommand class to execute SQL command. The class does not provide TDataSet-aware access to returned result sets.

Syntax

```
TADCommand = class(TADCustomCommand);
```

See Also

Executing Command (see page 66), Executing Stored Procedure (see page 71), TADCustomCommand (see page 291), TADCustomQuery (see page 394), TADCustomStoredProc (see page 403)

1.17.1.1.2.1 published

1.17.1.1.2.1.1 TADCommand.Active Property

Gets or sets command active status.

Description

Set Active to True to execute SQL command and return cursor. After that the State (see page 306) = csOpen and you can fetch rows from current cursor using Fetch (see page 312) method. Setting Active to True, will call Open (see page 316) method.

Set Active to False to close current command cursor. After that State (see page 306) = csPrepared. Setting Active to False, will call Close (see page 308)method. To close all command cursors (if command returns multiple result sets), use CloseAll (see page 309) method.

Syntax

```
property Active: Boolean;
```

See Also

Fetch, Define, Open, Close, CloseAll, State

Example

```
ADCommand1.CommandText.Text := 'select * from "Orders"';
ADCommand1.Active := True;
oTab := ADCommand1.Define;
try
  ADCommand1.Fetch(oTab);
  ....
finally
  oTab.Free;
end;
```

1.17.1.1.2.1.2 TADCommand.ActiveStoredUsage Property

Controls how to use Active property value saved to DFM.

Description

The ActiveStoredUsage property controls how to use Active (see page 293) property value saved to DFM. Include:

- auDesignTime, to use property value at design time;
- auRunTime, to use property value at run time.

Syntax

```
property ActiveStoredUsage: TADStoredActivationUsage;
```

See Also

Active

1.17.1.1.2.1.3 TADCommand.AfterClose Event

The event fires after command is closed.

Description

The AfterClose event fires, after command is closed by calling Close (see page 308) or setting Active (see page 293) = False.

Syntax

```
property AfterClose: TNotifyEvent;
```

See Also

BeforeClose, Close, Active

1.17.1.1.2.1.4 TADCommand.AfterExecute Event

The event fires after command is executed.

Description

The AfterExecute event fires, after command is executed by calling Execute (see page 310) method.

Syntax

```
property AfterExecute: TNotifyEvent;
```

See Also

BeforeExecute, Execute

Example

```
procedure TForm1.ADCommand1AfterExecute(ASender: TObject);
begin
  if ADCommand1.RowsAffected = -1 then
    StatusBar1.SimpleText := 'Ok'
  else
    case ADCommand1.CommandKind of
    skDelete: StatusBar1.SimpleText := Format('%d rows deleted', [ADCommand1.RowsAffected]);
    skInsert: StatusBar1.SimpleText := Format('%d rows inserted',
[ADCommand1.RowsAffected]);
    skUpdate: StatusBar1.SimpleText := Format('%d rows updated', [ADCommand1.RowsAffected]);
    else      StatusBar1.SimpleText := Format('%d rows affected',
[ADCommand1.RowsAffected]);
    end;
end;
```

1.17.1.1.2.1.5 TADCommand.AfterOpen Event

The event fires after command is opened.

Description

The AfterOpen event fires, after command is opened by calling Open (see page 316) or setting Active (see page 293) = True.

Syntax

```
property AfterOpen: TNotifyEvent;
```

See Also

BeforeOpen, Open, Active

1.17.1.1.2.1.6 TADCommand.AfterPrepare Event

The event fires after command is prepared.

Description

The AfterPrepare event fires, after command is prepared by calling Prepare (see page 317) method.

Syntax

```
property AfterPrepare: TNotifyEvent;
```

See Also

BeforePrepare, Prepare

1.17.1.1.2.1.7 TADCommand.AfterUnprepare Event

The event fires after command is unprepared.

Description

The AfterUnprepare event fires, after command is unprepared by calling Unprepare (see page 318) method.

Syntax

```
property AfterUnprepare: TNotifyEvent;
```

See Also

BeforeUnprepare, Unprepare

1.17.1.1.2.1.8 TADCommand.BaseObjectName Property

Identifies base object name for some of command kinds.

Description

The BaseObjectName identifies the name of the base object. The meaning depends on CommandKind (see page 298) and MetaInfoKind (see page 457):

- If CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs], then BaseObjectName specifies the name of the package, if procedure is packaged procedure.
- If MetaInfoKind (see page 457) in [mkIndexFields, mkPrimaryKeyFields, mkForeignKeyFields], then BaseObjectName specifies the name of the table.
- If MetaInfoKind (see page 457) in [mkProcs, mkProcArgs], then BaseObjectName specifies the name of the package.

Syntax

```
property BaseObjectName: String;
```

See Also

CommandKind, TADMetaInfoCommand.MetaInfoKind (see page 457)

Example

```
ADCommand1.BaseObjectName := 'MY_PACK';
ADCommand1.CommandText.Text := 'PROC1';
// or just ADCommand1.CommandText.Text := 'MY_PACK.PROC1';
ADCommand1.CommandKind := skStoredProc;
ADCommand1.Prepare;
ADCommand1.Params[0].AsInteger := 100;
ADCommand1.Params[1].AsString := 'Codegear Delphi';
ADCommand1.Execute;
```

1.17.1.1.2.1.9 TADCommand.BeforeClose Event

The event fires before command will be closed.

Description

The BeforeClose event fires, before command will be closed by calling Close (see page 308) or setting Active (see page 293) = False.

Syntax

```
property BeforeClose: TNotifyEvent;
```

See Also

AfterClose, Close, Active

1.17.1.1.2.1.10 TADCommand.BeforeExecute Event

The event fires before command will be executed.

Description

The BeforeExecute event fires, before command will be executed by calling Execute (see page 311) method.

Syntax

```
property BeforeExecute: TNotifyEvent;
```

See Also

AfterExecute, Execute

1.17.1.1.2.1.11 TADCommand.BeforeOpen Event

The event fires before command will be opened.

Description

The BeforeOpen event fires, before command will be opened by calling Open (see page 316) or setting Active (see page 293) = True.

Syntax

```
property BeforeOpen: TNotifyEvent;
```

See Also

AfterOpen, Open, Active

1.17.1.1.2.1.12 TADCommand.BeforePrepare Event

The event fires before command will be prepared.

Description

The BeforePrepare event fires, before command will be prepared by calling Prepare (see page 317) method.

Syntax

```
property BeforePrepare: TNotifyEvent;
```

See Also

AfterPrepare, Prepare

1.17.1.1.2.1.13 TADCommand.BeforeUnprepare Event

The event fires before command will be unprepared.

Description

The BeforeUnprepare event fires, before command wille be unprepared by calling Unprepare (see page 318) method.

Syntax

```
property BeforeUnprepare: TNotifyEvent;
```

See Also

BeforePrepare, Unprepare

1.17.1.1.2.1.14 TADCommand.CatalogName Property

Identifies catalog name for some command kinds.

Description

The CatalogName identifies the name of the catalog where resides command object. The meaning depends on CommandKind (see page 298) and MetainfoKind (see page 457):

- If CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs], then CatalogName (see page 297) specifies the name of the catalog, where resides the procedure.
- If MetainfoKind (see page 457) is not mkNone, then CatalogName (see page 297) specifies the name of the catalog, where resides the describing object.

Syntax

```
property CatalogName: String;
```

See Also

CommandKind, TADMetaInfoCommand.MetaInfoKind (see page 457), SchemaName

Example 1

```
ADCommand1.CatalogName := 'MY_APP';
ADCommand1.CommandText.Text := 'PROC1';
ADCommand1.CommandKind := skStoredProc;
ADCommand1.Prepare;
ADCommand1.Params[0].AsInteger := 100;
ADCommand1.Params[1].AsString := 'Bill Gates';
ADCommand1.Execute;
```

Example 2

```
ADMetaInfoCommand1.CatalogName := 'MY_APP';
ADMetaInfoCommand1.ObjectName := 'MY_TAB';
// or on SQL Server just ADMetaInfoCommand1.ObjectName := 'MY_APP..MY_TAB';
ADMetaInfoCommand1.MetaInfoKind := mkTableFields;
ADMetaInfoCommand1.Open;
```

1.17.1.1.2.1.15 TADCommand.CommandKind Property

Identifies the command kind.

Description

The CommandKind property identifies the kind of the content of CommandText (see page 298) property. You can explicitly set it value before Prepare call, otherwise it will be automatically assigned by FireDAC after analyzing CommandText (see page 298) content. There is only single need to assign it - when you are preparing stored procedure.

Syntax

```
property CommandKind: TADPhysCommandKind;
```

See Also

CommandText

Example 1

```
ADCommand1.CommandKind := skStoredProc;
ADCommand1.CommandText.Text := 'MY_PROC';
ADCommand1.Prepare;
```

Example 2

```
ADCommand1.CommandText.Text := 'ALTER PROCEDURE MY_PROC COMPILE';
case ADCommand1.CommandKind of
skAlter: ShowMessage('Alter command');
.........
else     ShowMessage('Other command');
end;
```

1.17.1.1.2.1.16 TADCommand.CommandText Property

The SQL command text to execute.

Description

The CommandText set/get:

- SQL command text to execute
- or name of the stored procedure to execute, if CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs].

After setting CommandText, FireDAC will do for non-stored procedures:

- extract parameters names and fill Params (see page 303) collection, if ResourceOptions.ParamCreate (see page 883) = True;
- extract macro name and fill Macros (see page 301) collection, if ResourceOptions.MacroCreate (see page 882) = True;

- extract FROM table name for SELECT commands and assign it to CommandIntf (see page 298).SourceObjectName, if ResourceOptions.PreprocessCmdText (see page 878) = True;
- fill CommandKind (see page 298) property value, if it is not explicitly set.

If you are adding long command, using CommandText.Add method, then it is recommended before command modification to call CommandText.BeginUpdate and after finishing command modification to call CommandText.EndUpdate.

Later on Prepare (see page 317) call command text will be preprocessed and transformed in target DBMS command. See chapter Preprocessing Command Text (see page 56) for details about FireDAC macros and escape sequences - preprocessor instructions.

Syntax

```
property CommandText: TStrings;
```

See Also

Preprocessing Command Text (see page 56), Executing Command (see page 66), TADResourceOptions.ParamCreate (see page 883), TADResourceOptions.MacroCreate (see page 882),TADResourceOptions.PreprocessCmdText (see page 878), CommandKind, Params, Macros, ResourceOptions

Example 1

Stored procedure:

```
ADCommand1.CommandKind := skStoredProc;
ADCommand1.CommandText.Text := 'MY_PROC';
ADCommand1.Prepare;
ADCommand1.Params[0].AsString := 'qwe';
ADCommand1.Execute;
```

Example 2

General SQL command:

```
ADCommand1.CommandText.Text := 'insert into MyTab values (:p1)';
ADCommand1.Params[0].AsString := 'qwe';
ADCommand1.Execute;
```

Example 3

Adding long command:

```
with ADCommand1.CommandText do begin
  BeginUpdate;
  Add('............');
  ......
  Add('............');
  EndUpdate;
end;
```

1.17.1.1.2.1.17 TADCommand.Connection Property

Specifies the connection object.

Description

The Connection property value points to the connection object. Alternatively may be specified the ConnectionName (see page 300) property value. The Connection property value must be specified before Prepare (see page 317) call.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

ConnectionName, Prepare

1.17.1.1.2.1.18 TADCommand.ConnectionName Property

Specifies connection name.

Description

The ConnectionName property value specifies the name of the connection. It must match to the:

- name of one of connection definitions, either stored in external file or created on fly;
- ConnectionName of one of the TADConnection (see page 279) objects.

Alternatively may be specified the Connection (see page 299) property value. The ConnectionName property value must be specified before Prepare (see page 317)call. If is specified the name of one of connection definitions, then FireDAC will transparently create connection object and link it with command.

Syntax

```
property ConnectionName: String;
```

See Also

Connection, Prepare

1.17.1.1.2.1.19 TADCommand.FetchOptions Property

The set of options to control rows fetching.

Description

The FetchOptions property is the set of properties, controlling the fetching of cursor rows. These properties will inherit its values from FetchOptions of connection object.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), TADFetchOptions, Fetch

1.17.1.1.2.1.20 TADCommand.FormatOptions Property

The set of options to control data representation.

Description

The FormatOptions property is the set of properties, controlling the data type mapping, and some other aspects of data handling.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), TADFormatOptions, Fetch, Params

1.17.1.1.2.1.21 TADCommand.Macros Property

The collection of macro substitution variables.

Description

The Macros property is the collection of macro substitution variables. It is populated automatically after assigning CommandText (see page 298), if ResourceOptions (see page 304).MacroCreate (see page 882) = True. Otherwise it may be populated by hands. At Prepare (see page 317) call, macro values will be substituted into CommandText (see page 298), if ResourceOptions (see page 304).MacroExpand (see page 882) = True.

Syntax

```
property Macros: TADMacros;
```

See Also

Executing Command (see page 66), Preprocessing Command Text (see page 56), CommandText, Params, Prepare,

TADMacros, TADResourceOptions.MacroCreate (see page 882), TADResourceOptions.MacroExpand (see page 882)

Example 1

Substitute table name:

```
ADCommand1.CommandText.Text := 'select * from &Tab';
ADCommand1.Macros[0].AsIdentifier := 'Order Details';
ADCommand1.Prepare;
ShowMessage(ADCommand1.SQLText); // select * from "Order Details"
```

Example 2

Substitute WHERE condition:

```
ADCommand1.CommandText.Text := 'select * from MyTab {if !cond} where !cond {fi}';
ADCommand1.Macros[0].AsString := 'ID > 100';
ADCommand1.Prepare;
ShowMessage(ADCommand1.SQLText); // select * from MyTab where ID > 100
ADCommand1.Macros[0].Clear;
ADCommand1.Prepare;
ShowMessage(ADCommand1.SQLText); // select * from MyTab
```

Example 3

See FireDAC\Samples\Comp Layer\TADQuery\Macros demo for details.

1.17.1.1.2.1.22 TADCommand.OnCommandChanged Event

The event fires after CommandText is changed.

Description

The OnCommandChanged event fires, after CommandText (see page 298) is changed.

Syntax

```
property OnCommandChanged: TNotifyEvent;
```

See Also

CommandText

Example

```
procedure TForm1.ADCommand1CommandChanged(ASender: TObject);
begin
  if ADCommand1.FindParam('ID') <> nil then
    ADCommand1.FindParams('ID').AsInteger := 100;
end;
```

1.17.1.1.2.1.23 TADCommand.OnError Event

The event fires when an error happens, while command is communicating with DBMS.

Parameters

Parameters	Description
ASender	The reference to command.
AInitiator	The reference to interface of object, which initially raised the exception.
AException	The exception object, which may be modified or even replaced by another exception object instance.

Description

The OnError event fires, when command executes one of the following operations and an error happens:

- Prepare (see page 317). The command is preparing DBMS command text for execution.
- Open (see page 316). The command is executing DBMS command to open cursor.
- Execute (see page 311). The command is executing DBMS command text.

- Fetch (see page 312). The command is fetching rows from cursor.

If most cases AException object will of EADDBEngineException (see page 833) class. If AException is of EADDBArrayExecuteError (see page 831) class, then it is Array DML (see page 82) error handling case. You should use code like in the example.

Syntax

```
property OnError: TADErrorEvent;
```

See Also

Handling Errors (see page 44), Array DML (see page 82), Open, Execute, Prepare, Fetch, EADDBArrayExecuteError (see page 831) class, EADDBEngineException class

Example 1

Handling Array DML errors:

```
procedure TForm1.ADCommand1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if AException is EADPhysArrayExecuteError then
    with EADPhysArrayExecuteError(AException) do
      if Errors[0].Kind = ekUKViolated then
        AAction := eaSkip
      else
        AAction := eaFail
  else
    ...
end;
```

Example 2

Error substitution:

```
procedure TForm1.ADCommand1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if EADDBEngineException(AException).Errors[0].Kind = ekPKViolated then begin
    AException.Free;
    AException := Exception.Create('Please, enter unique ID value');
  end;
end;
```

1.17.1.1.2.1.24 TADCommand.Overload Property

Oracle stored procedure overload index.

Description

The Overload property specifies Oracle packaged stored procedure overload index. It has no meaning for other DBMS's.

Syntax

```
property Overload: Word;
```

See Also

CommandText, CommandKind

Example

```
ADCommand1.CommandKind := skStoredProc;
ADCommand1.CommandText.Text := 'DBMS_SQL.PARSE';
ADCommand1.Overload := 1;
```

1.17.1.1.2.1.25 TADCommand.Params Property

The collection of parameters for a SQL command.

Description

The Params property is the collection of parameters for a SQL command.

If CommandKind (see page 298) is not in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs], then Params is populated automatically after assigning CommandText (see page 298), if ResourceOptions (see page 304).ParamCreate (see page 883) = True.

If CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs], then Params is populated automatically after calling Prepare (see page 317) method, if fiMeta is in FetchOption (see page 300).Items (see page 855).

Otherwise Params may be populated by hands. At Prepare (see page 317) call, parameters will be binded to prepared SQL command. After that you cannot change parameter types, otherwise will get an exception.

Syntax

```
property Params: TADParams;
```

See Also

Executing Command (see page 66), CommandText, Macros, Prepare, TADParams, TADResourceOptions.ParamCreate, TADFetchOptions.Items

1.17.1.1.2.1.26 TADCommand.ResourceOptions Property

The set of options to control resources usage.

Description

The ResourceOptions property is the set of properties, controlling the resources usage by the command. These properties will inherit its values from ResourceOptions of connection object.

Syntax

```
property ResourceOptions: TADBottomResourceOptions;
```

See Also

Setting Options (see page 34), TADBottomResourceOptions

1.17.1.1.2.1.27 TADCommand.SchemaName Property

Identifies schema name for some of command kinds.

Description

The SchemaName identifies the name of the schema where resides command object. The meaning depends on CommandKind (see page 298) and MetainfoKind (see page 457):

- If CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs], then SchemaName specifies the name of the schema, where resides the procedure.
- If MetainfoKind (see page 457) is not mkNone, then SchemaName specifies the name of the schema, where resides the describing object.

Syntax

```
property SchemaName: String;
```

See Also

CommandKind, TADMetaInfoCommand.MetaInfoKind (see page 457), SchemaName

Example 1

```
ADCommand1.SchemaName := 'SCOTT';
ADCommand1.CommandText.Text := 'PROC1';
ADCommand1.CommandKind := skStoredProc;
ADCommand1.Prepare;
ADCommand1.Params[0].AsInteger := 100;
ADCommand1.Params[1].AsString := 'Bill Gates';
ADCommand1.Execute;
```

Example 2

```
ADMetaInfoCommand1.SchemaName := 'ADDEMO';
```

```
ADMetaInfoCommand1.ObjectName := 'MY_TAB';
// or on Oracle just ADMetaInfoCommand1.ObjectName := 'ADDEMO.MY_TAB';
ADMetaInfoCommand1.MetaInfoKind := mkTableFields;
ADMetaInfoCommand1.Open;
```

1.17.1.1.2.1.28 TADCommand.Transaction Property

Specifies a transaction object.

Description

The Transaction property value points to a transaction object. The SQL command will be prepared and executed in the context of this transaction. The Transaction property value must be specified before Prepare call. If a value is not specified, then command will use default transaction object, specified at TADCustomConnection (see page 318).Transaction (see page 337) or internal connection transaction object.

Note, at moment only Interbase / Firebird driver supports assignment of transaction to the command.

Syntax

```
property Transaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41), TADTransaction, TADCustomConnection.Transaction (see page 337)

1.17.1.1.2.1.29 TADCommand.UpdateOptions Property

The set of options to control updates posting.

Description

The UpdateOptions property is the set of properties, controlling the posting of updates. These properties will inherit its values from UpdateOptions of a connection object.

Syntax

```
property UpdateOptions: TADBottomUpdateOptions;
```

See Also

Setting Options (see page 34), TADBottomUpdateOptions

1.17.1.1.3 TADConnection Class

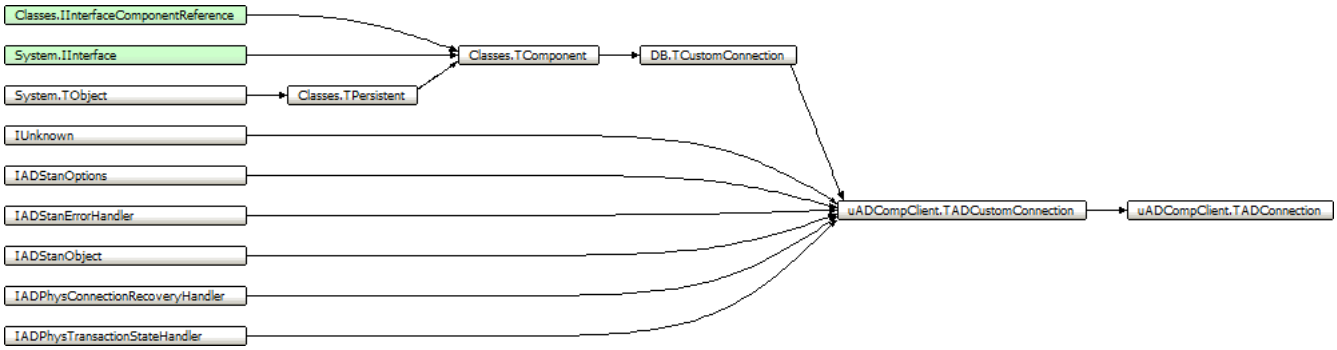
The class is responsible to connection establishment with a DBMS.

published

published	Description
AfterCommit (see page 280)	Fires after transaction is committed.
AfterRollback (see page 281)	Fires after transaction is rolled back.
AfterStartTransaction (see page 281)	Fires after transaction is started.
BeforeCommit (see page 281)	Fires before transaction will be committed.
BeforeRollback (see page 281)	Fires before transaction will be rolled back.
BeforeStartTransaction (see page 282)	Fires before transaction will be started.
Connected (see page 282)	Gets / sets connection active status.
ConnectedStoredUsage (see page 282)	Controls how to use Connected property value saved to DFM.
ConnectionDefName (see page 283)	The name of connection definition to use.
ConnectionName (see page 283)	Specifies the name of the connection to associate with this connection object.
DriverName (see page 284)	Specifies the ID of the FireDAC driver for the connection.
FetchOptions (see page 284)	The set of default options to control rows fetching.
FormatOptions (see page 285)	The set of default options to control data representation.

LoginDialog (see page 285)	Gets / sets the reference to login dialog interface.
LoginPrompt (see page 286)	Specifies whether a login dialog appears immediately before opening a new connection.
OnError (see page 286)	Fires before FireDAC will raise exception.
OnLogin (see page 287)	Fires when an application connects to a DBMS.
OnLosted (see page 288)	Fires after FireDAC was not able to restore losted connection to DBMS.
OnRecover (see page 288)	Fires after FireDAC discovered that connection to DBMS is losted.
OnRestored (see page 288)	Fires after FireDAC successfully restored losted connection to DBMS.
Params (see page 289)	Lists database connection parameters for the FireDAC connection definition associated with the connection object.
ResourceOptions (see page 289)	The set of default options to control resources usage.
Transaction (see page 289)	The reference to TADCustomTransaction object.
TxOptions (see page 290)	The set of options to control transactions.
UpdateOptions (see page 290)	The set of default options to control data updating.
UpdateTransaction (see page 290)	The reference to TADCustomTransaction object, which will be used to post changes to DB.

Class Hierarchy



File

uADCompClient

Description

Use TADConnection to establish connection to a DBMS and manage associated datasets.

Syntax

```
TADConnection = class(TADCustomConnection);
```

See Also

Working with Connections (see page 27), Managing Transactions (see page 41), TADCustomConnection (see page 318)

1.17.1.1.3.1 published

1.17.1.1.3.1.1 TADConnection.AfterCommit Event

Fires after transaction is committed.

Description

The AfterCommit event fires after the Commit method is executed and the transaction is committed. In case of a nested transaction in AfterCommit event handler the transaction is not yet finished, but nesting level is decremented.

Syntax

```
property AfterCommit: TNotifyEvent;
```

See Also

BeforeCommit

1.17.1.1.3.1.2 TADConnection.AfterRollback Event

Fires after transaction is rolled back.

Description

The AfterRollback event fires after the Rollback method is executed and the transaction is rolled back. In case of a nested transaction in AfterRollback event handler the transaction is not yet rolled back in full, but nesting level is decremented.

Syntax

```
property AfterRollback: TNotifyEvent;
```

See Also

BeforeRollback

1.17.1.1.3.1.3 TADConnection.AfterStartTransaction Event

Fires after transaction is started.

Description

The AfterStartTransaction event fires after the StartTransaction method is executed and the transaction is started. In case of a nested transaction in AfterStartTransaction event handler the transaction is not just started, but nesting level is incremented.

Syntax

```
property AfterStartTransaction: TNotifyEvent;
```

See Also

BeforeStartTransaction

1.17.1.1.3.1.4 TADConnection.BeforeCommit Event

Fires before transaction will be committed.

Description

The BeforeCommit event fires before the Commit method will be executed and the transaction will be committed.

Syntax

```
property BeforeCommit: TNotifyEvent;
```

See Also

AfterCommit

1.17.1.1.3.1.5 TADConnection.BeforeRollback Event

Fires before transaction will be rolled back.

Description

The BeforeRollback event fires before the Rollback method will be executed and the transaction will be rolled back.

Syntax

```
property BeforeRollback: TNotifyEvent;
```

See Also

AfterRollback

1.17.1.1.3.1.6 TADConnection.BeforeStartTransaction Event

Fires before transaction will be started.

Description

The BeforeStartTransaction event fires before the StartTransaction method will be executed and the transaction will be started.

Syntax

```
property BeforeStartTransaction: TNotifyEvent;
```

See Also

AfterStartTransaction

1.17.1.1.3.1.7 TADConnection.Connected Property

Gets / sets connection active status.

Description

Set Connected to True to establish a connection to a DBMS. Set Connected to False to close a connection. The default value for Connected is False.

An application can check Connected to determine the current status of a connection. If Connected is True, the connection is active; if False, and the KeepConnection (see page 889) property is also False, then the connection is inactive.

To open connection, it must be setup in one of the following ways:

1. Set ConnectionDefName (see page 325) to a name of one of a existing connection definitions and optionally override connection definition parameters in Params (see page 334). Existing connection definition may be defined either in connection definitions INI file or created on fly, for example, using TADCustomManager.AddConnectionDef (see page 376);
2. Set DriverName (see page 327) to a name of one of a registered drivers (or add DriverID into Params (see page 334)) and specify connection definition parameters in Params (see page 334).

When the connection definition is pooled (Pooled=True), then use (1) and does not override any of parameters, otherwise you will get an exception.

Setting Connected to False will try to connect to a DB. If somehow the connecting fails, then Connected will remain False and an an exception will be raised. If connecting failed due to a DB unavailability, then application may use Automatic Connection Recovery (see page 333) feature to wait and try to establish the connection.

Setting Connected to False will disconnect (close and unprepare) all associated commands and datasets. If you just need to disconnect from DBMS, but does not close objects, then use Offlined (see page 331) property.

Syntax

```
property Connected;
```

See Also

TCustomConnection.BeforeConnect, TCustomConnection.AfterConnect, TCustomConnection.BeforeDisconnect, TCustomConnection.AfterDisconnect, ConnectionDefName, DriverName, Params, Offlined, OnRecover

Example 1

```
ADConnection1.ConnectionDefName := 'ASA_Demo';
ADConnection1.Connected := True;
```

Example 2

See FireDAC\Samples\Comp Layer\TADConnection\Pooling sample for details of how to work with pooled connections.

1.17.1.1.3.1.8 TADConnection.ConnectedStoredUsage Property

Controls how to use Connected property value saved to DFM.

Description

The ConnectedStoredUsage property controls how to use Connected (see page 321) property value saved to DFM. Include:

- auDesignTime, to use property value at design time;
- auRunTime, to use property value at run time.

Syntax

```
property ConnectedStoredUsage: TADStoredActivationUsage;
```

See Also

Connected (see page 321)

1.17.1.1.3.1.9 TADConnection.ConnectionDefName Property

The name of connection definition to use.

Description

The ConnectionDefName property specifies the name of one of a existing connection definitions. It must be stored in ADConnectionDefs.ini or created dynamically.

Note, attempting to set ConnectionDefName when the Connected property is true raises an exception.

Syntax

```
property ConnectionDefName: String;
```

See Also

Defining Connection (see page 27), TADCustomManager

Example 1

```
ADConnection1.ConnectionDefName := 'ASA_Demo';
ADConnection1.Connected := True;
```

Example 2

See demo FireDAC\Samples\Comp Layer\TADConnection\ConnectionDefs for details how to create and use connection definitions.

Example 3

Specifying pooled connection:

```
var
  oList: TStringList;
  ......
  oList := TStringList.Create;
  oList.Add('Server=MyServer');
  oList.Add('Database=Northwind');
  oList.Add('User_Name=sa');
  oList.Add('Password=');
  oList.Add('Pooled=True');
  ADManager.AddConnectionDef('myconn', 'MSSQL', oList);
  ......
  ADConnection1.ConnectionDefName := 'myconn';
  ADConnection1.Connected := True;
```

1.17.1.1.3.1.10 TADConnection.ConnectionName Property

Specifies the name of the connection to associate with this connection object.

Description

Use ConnectionName to specify the name of the connection to use with a connection object. If ConnectionName is the same as an existing connection definition, then the ConnectionDefName (see page 325) and DriverName (see page 327) properties need not be set. If ConnectionName does not match an existing connection definition, then either the application must also supply a valid connection definition in the ConnectionDefName (see page 325) property in addition to the or it

must supply the DriverName (see page 327) and Params (see page 334) properties.

You can specifiy as ConnectionName for TADCustomCommand (see page 291) or TADRDBMSDataSet (see page 501) component and its descendants (TADQuery (see page 476), TADStoredProc (see page 514), TADTable (see page 539)), the ConnectionName of one of the existing TADConnection (see page 279) objects or the name of one of the existing connection definitions.

Notes

1. Attempting to set ConnectionName when the Connected property is true raises an exception.
2. At design time double-click a TConnection component to invoke the Database editor and set the ConnectionName.

Syntax

```
property ConnectionName: String;
```

See Also

ConnectionDefName, DriverName, Params, TADCustomCommand.ConnectionName (see page 300), TADRDBMSDataSet.ConnectionName (see page 503)

Example

```
ADConnection1.ConnectionName := 'TestConn';
ADConnection1.DriverName := 'Oracl';'
ADConnection1.Params.Add('Database=testdb');
....
ADQuery1.ConnectionName := 'TestConn';
ADQuery1.Open('select * from test_tab');
```

1.17.1.1.3.1.11 TADConnection.DriverName Property

Specifies the ID of the FireDAC driver for the connection.

Description

Use DriverName to specify the ID of the FireDAC driver to use for connection that do not specify an ConnectionDefName (see page 325). DriverName must be a valid FireDAC registered driver ID.

If an application sets DriverName, it must also specify connection parameters in the Params (see page 334) property. Ordinarily the alias specified in the ConnectionDefName (see page 325) property supplies connection parameters, but when DriverName is set, the ConnectionDefName (see page 325) property is automatically cleared to avoid FireDAC driver ID contention.

Attempting to set DriverName when the Connected (see page 321) property is True raises an exception.

Setting DriverName value, will add DriverID=<xxx> parameter to the Params list.

Notes

At design time double-click a TConnection component to invoke the Connection editor and set the DriverName.

Or choose DriverName value in property inspector from drop-down list.

Syntax

```
property DriverName: string;
```

See Also

Params, ConnectionDefName

Example

```
ADConnection1.DriverName := 'Oracl';'
ADConnection1.Params.Add('Database=testdb');
....
ADQuery1.Connection := ADConnection1;
ADQuery1.Open('select * from test_tab');
```

1.17.1.1.3.1.12 TADConnection.FetchOptions Property

The set of default options to control rows fetching.

Description

The FetchOptions property is the set of default properties, controlling the fetching of cursor rows. These properties will be inherited by commands and datasets, associated with this connection. Also these properties will inherit its values from FetchOptions of ADManager (see page 570) object.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), TADFetchOptions, ADManager (see page 570)

1.17.1.1.3.1.13 TADConnection.FormatOptions Property

The set of default options to control data representation.

Description

The FormatOptions property is the set of the default properties, controlling the data type mapping, and some other aspects of data handling. These properties will be inherited by commands and datasets, accossiated with this connection. Also these properties will inherit its values from FormatOptions of ADManager (see page 570) object.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), TADFormatOptions, ADManager (see page 570)

1.17.1.1.3.1.14 TADConnection.LoginDialog Property

Gets / sets the reference to login dialog interface.

Description

The LoginDialog property allows to associate the login dialog TADGUIxLoginDialog (see page 675) with this connection. The dialog instance assigned to the LoginDialog will be used only by this connection object.

If the LoginDialog property is not assigned, but the TADGUIxLoginDialog is in your application or one of the units uADGUIxXxxxLogin is linked into application, then the connection object will use the global login dialog instance.

The login dialog will popup at connecting to a DBMS only if all of the following conditions are met:

- LoginPrompt=True;
- connection definition is not pooled (Pooled=False);
- global silent flag is not set to True (ADGUIxSilent() = False);
- connection object does not use shared CLI handle (SharedCliHandle (see page 336) = nil).

Syntax

```
property LoginDialog: TADGUIxLoginDialog;
```

See Also

Establishing Connection (see page 37), TADGUIxLoginDialog, LoginPrompt, SharedCliHandle, ADGUIxSilent

Example

Using private login dialog:

```
ADGUIxLoginDialog1.Caption := 'Welcome to the Northwind DB';
ADGUIxLoginDialog1.VisibleItems.Add('User_name');
ADGUIxLoginDialog1.VisibleItems.Add('Password');
ADConnection1.LoginDialog := ADGUIxLoginDialog1;
ADConnection1.LoginPrompt := True;
ADConnection1.Connected := True;
```

1.17.1.1.3.1.15 TADConnection.LoginPrompt Property

Specifies whether a login dialog appears immediately before opening a new connection.

Description

Set LoginPrompt to True to provide login support when establishing a connection. When LoginPrompt is True, a dialog appears to prompt users for a name, password and other optional parameters. The dialog appears after the BeforeConnect event and before the AfterConnect event, unless you supply an OnLogin (see page 332) event handler. If there is an OnLogin (see page 332) event handler, that event occurs in place of the login dialog. If correct values for the user name, password and other optional parameters are not supplied in the dialog or by the OnLogin (see page 332) event handler, the connection fails. This OnLogin (see page 332) event does not fire unless LoginPrompt is set to True.

When LoginPrompt is False, the application must supply user name, password and other optional parameter values programmatically. For that application can specify User_name, Password, etc parameters in Params.

The login dialog will popup at connecting to a DBMS only if all of the following conditions are met:

- LoginPrompt=True;
- connection definition is not pooled (Pooled=False);
- global silent flag is not set to True (ADGUIxSilent() = False);
- connection object does not use shared CLI handle (SharedCliHandle (see page 336) = nil).

Syntax

```
property LoginPrompt;
```

See Also

Establishing Connection (see page 37), OnLogin, LoginDialog, Params, SharedCliHandle, ADGUIxSilent, AfterConnect

Example 1

Using login dialog:

```
ADGUIxLoginDialog1.Caption := 'Welcome to the Northwind DB';
ADGUIxLoginDialog1.VisibleItems.Add('User_name');
ADGUIxLoginDialog1.VisibleItems.Add('Password');
ADConnection1.LoginDialog := ADGUIxLoginDialog1;
ADConnection1.LoginPrompt := True;
ADConnection1.Connected := True;
```

Example 2

Connecting without login dialog:

```
procedure TForm1.ADConnection1Login(AConnection: TADCustomConnection; const AConnectionDef:
IADStanConnectionDef);
begin
  AConnectionDef.UserName := 'dba';
  AConnectionDef.Password := 'sql';
end;

procedure TForm1.Button1Click(ASender: TObject);
begin
  ADConnection1.OnLogin := ADConnection1Login;
  ADConnection1.LoginPrompt := True;
  ADConnection1.Connected := True;
end;
```

1.17.1.1.3.1.16 TADConnection.OnError Event

Fires before FireDAC will raise exception.

Parameters

Parameters	Description
ASender	This connection object.
AInitiator	The interface of the object initially discovered error.

AException	The exception object to be raised.

Description

The OnError event handler fires before FireDAC will raise exception inside of call of one of the connection or assocciated commands or datasets methods. In most cases AException will be of EADDBEngineException (see page 833) class.

The event handler can replace AException object with some other custom exception. In this case original object must be freed.

Syntax

```
property OnError: TADErrorEvent;
```

See Also

Handling Errors (see page 44), EADDBEngineException

Example

```
procedure TForm1.ADConnection1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if (AException is EADDBEngineException) and (EADDBEngineException(AException).Kind =
ekUKViolated) then begin
    AException.Free;
    AException := Exception.Create('You must enter unique document number !');
  end;
end;
```

1.17.1.1.3.1.17 TADConnection.OnLogin Event

Fires when an application connects to a DBMS.

Description

Write an OnLogin event handler to take specific actions when an application attempts to connect to a DBMS. When the LoginPrompt (see page 329) property is True, a database login is required. The login dialog and OnLogin event only occur when the LoginPrompt (see page 329)property is true.

If there is no OnLogin event handler, the current User_Name is read from the Params (see page 334) property or connection definition, and a standard Login dialog box opens. The dialog prompts for a user name, password and other optional parameters, and then uses the values entered by the user to set the User_Name, Password and other optional values in the Params property. These values are then passed to the DBMS.

Applications that provide alternative OnLogin event handlers may set the User_Name, Password and other optional values in AConnectionDef. AConnectionDef is a temporary connection definition and is freed automatically when no longer needed.

Syntax

```
property OnLogin: TADConnectionLoginEvent;
```

See Also

Establishing Connection (see page 37), LoginPrompt, Connected, IADStanConnectionDef

Example

Connecting without login dialog:

```
procedure TForm1.ADConnection1Login(AConnection: TADCustomConnection; const AConnectionDef:
IADStanConnectionDef);
begin
  AConnectionDef.UserName := 'dba';
  AConnectionDef.Password := InputBox('Login', 'Please enter DBA password', 'sql');
end;

procedure TForm1.Button1Click(ASender: TObject);
begin
  ADConnection1.OnLogin := ADConnection1Login;
  ADConnection1.LoginPrompt := True;
  ADConnection1.Connected := True;
end;
```

1.17.1.1.3.1.18 TADConnection.OnLosted Event

Fires after FireDAC was not able to restore losted connection to DBMS.

Description

The OnLosted event fires after FireDAC was not able to automatically restore connection to the DBMS, after discovering that it is losted.

Syntax

```
property OnLosted: TNotifyEvent;
```

See Also

Recovering Connection (see page 38), OnRecover, OnRestored

1.17.1.1.3.1.19 TADConnection.OnRecover Event

Fires after FireDAC discovered that connection to DBMS is losted.

Parameters

Parameters	Description
ASender	This connection object.
AInitiator	The interface of the object initially discovered error.
AException	The exception object describing losted connection issue.
AAction	The action FireDAC should take after return from event handler.

Description

The OnRecover event fires after FireDAC discovered that connection to DBMs is losted. The event handler can analyze the reason, ask user how to proceed, show some status and returns the action, FireDAC should take:

Action	Description
faDefault	If ResourceOptions (see page 335).AutoReconnect (see page 887) =True, then FireDAC will try to reconnect, otherwise will raise an exception, that connection is losted. By default ResourceOptions (see page 335).AutoReconnect (see page 887) = False.
faFail	Close connection and raise an exception, that connection is losted.
faRetry	Try to reconnect.
faCloseAbort	Close connection and abort current operation.
faOfflineAbort	Switch connection to the offline mode and abort current operation.

If event handler is not assigned, then FireDAC will take default (faDefault) action. If after 3 trials connection cannot be restored, then FireDAC will close connection and raise an exception, that connection is losted.

Syntax

```
property OnRecover: TADConnectionRecoverEvent;
```

See Also

Recovering Connection (see page 38), OnLosted, OnRestored, TADTopResourceOptions.AutoReconnect (see page 887), Connected, Offlined

1.17.1.1.3.1.20 TADConnection.OnRestored Event

Fires after FireDAC successfully restored losted connection to DBMS.

Description

The OnRestored event fires after FireDAC successfully restored connection to the DBMS, after discovering that it is losted.

Syntax

```
property OnRestored: TNotifyEvent;
```

See Also

Recovering Connection (see page 38), OnLosted, OnRecover

1.17.1.1.3.1.21 TADConnection.Params Property

Lists database connection parameters for the FireDAC connection definition associated with the connection object.

Description

Use Params to list DBMS connection definition parameters, such server name, port, database, client character set, user name and password. The possible set of parameters depends on the DBMS kind to connect to.

Params is a list of string items, each representing a different DBMS connection definition parameter. If the ConnectionDefName (see page 325) property specifies a valid FireDAC connection definition, then parameters, specified in Params, will override corresponsding connection definition parameters. If an application need to specify connection definition parameter on-fly, then an application must fill-in all required connection definition parameter into Params.

At design time double-click a TConnection component to invoke the Connection editor and set Params.

Syntax

```
property Params: TStrings;
```

See Also

Defining Connection (see page 27), ConnectionDefName, DriverName, OnLogin

Example

```
with ADConnection1.Params do begin
  Clear;
  Add('DriverID=Ora');
  Add('Database=ORA_920_APP');
  Add('User_Name=ADDemo');
  Add('Password=a');
end;
ADConnection1.Connected := True;
```

1.17.1.1.3.1.22 TADConnection.ResourceOptions Property

The set of default options to control resources usage.

Description

The ResourceOptions property is the set of the default properties, controlling the resources usage by the command. These properties will be inherited by commands and datasets, accossiated with this connection. Also these properties will inherit its values from ResourceOptions of ADManager (see page 570) object.

Syntax

```
property ResourceOptions: TADTopResourceOptions;
```

See Also

Setting Options (see page 34), TADTopResourceOptions, ADManager (see page 570)

1.17.1.1.3.1.23 TADConnection.Transaction Property

The reference to TADCustomTransaction object.

Description

The Transaction property get/set reference to the transaction object, which will be default one for the connection and all associated datasets and commands without explicitly assigned Transaction property. At moment, this behavior has the meaning only for Interbase / Firebird connection.

Syntax

```
property Transaction: TADCustomTransaction;
```

See Also

UpdateTransaction, TADCustomTransaction

1.17.1.1.3.1.24 TADConnection.TxOptions Property

The set of options to control transactions.

Description

The TxOptions property is the set of the properties, controlling the transactions in this connection.

You can change transaction properties while there is no active transactions. If you set Transaction (see page 337) property, then application should use Transaction.Options, instead of TxOptions.

Syntax

```
property TxOptions: TADTxOptions;
```

See Also

Transaction, TADTxOptions

Example

See FireDAC\Samples\Comp Layer\TADConnection\Transactions sample for details.

1.17.1.1.3.1.25 TADConnection.UpdateOptions Property

The set of default options to control data updating.

Description

The UpdateOptions property is the set of the default properties, controlling the data updating by the datasets. These properties will be inherited by datasets, associated with this connection. Also these properties will inherit its values from UpdateOptions of ADManager (see page 570) object.

Syntax

```
property UpdateOptions: TADUpdateOptions;
```

See Also

Setting Options (see page 34), TADUpdateOptions, ADManager (see page 570)

1.17.1.1.3.1.26 TADConnection.UpdateTransaction Property

The reference to TADCustomTransaction object, which will be used to post changes to DB.

Description

The UpdateTransaction property get/set reference to the transaction object, which will be default one for all associated datasets without explicitly assigned UpdateTransaction property. This transaction will be used by datasets to post changes to the DB.

If UpdateTransaction is not assigned, but Transaction (see page 337) is assigned, then Transaction (see page 337) object will be used as update transaction.

At moment, this behaviour has the meaning only for Interbase / Firebird connection.

Syntax

```
property UpdateTransaction: TADCustomTransaction;
```

See Also

Transaction, TADCustomTransaction

1.17.1.1.4 TADCustomCommand Class

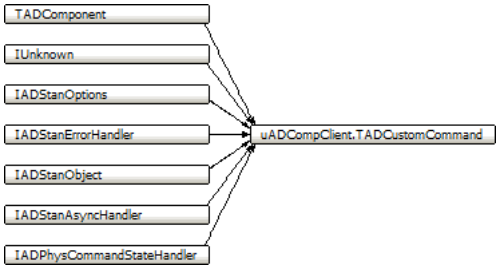
The base class is responsible for execution of SQL commands.

public

public	Description
Active (see page 293)	Gets or sets command active status.
ActiveStoredUsage (see page 293)	Controls how to use Active (see page 293) property value saved to DFM.
AfterClose (see page 293)	The event fires after command is closed.
AfterExecute (see page 294)	The event fires after command is executed.
AfterFetch (see page 294)	The event fires after fetching from command cursor.
AfterOpen (see page 294)	The event fires after command is opened.
AfterPrepare (see page 295)	The event fires after command is prepared.
AfterUnprepare (see page 295)	The event fires after command is unprepared.
BaseObjectName (see page 295)	Identifies base object name for some of command kinds.
BeforeClose (see page 296)	The event fires before command will be closed.
BeforeExecute (see page 296)	The event fires before command will be executed.
BeforeFetch (see page 296)	The event fires before fetching from command cursor.
BeforeOpen (see page 296)	The event fires before command will be opened.
BeforePrepare (see page 296)	The event fires before command will be prepared.
BeforeUnprepare (see page 297)	The event fires before command will be unprepared.
R BindedBy (see page 297)	Determines how command is connected to connection object.
CatalogName (see page 297)	Identifies catalog name for some command kinds.
R CommandIntf (see page 298)	Returns IADPhysCommand interface.
CommandKind (see page 298)	Identifies the command kind.
CommandText (see page 298)	The SQL command text to execute.
Connection (see page 299)	Specifies the connection object.
ConnectionName (see page 300)	Specifies connection name.
R DataSet (see page 300)	Reference to dataset object owning this command.
FetchOptions (see page 300)	The set of options to control rows fetching.
FixedCommandKind (see page 300)	True, if the CommandKind (see page 298) property value is set explicitly.
FormatOptions (see page 301)	The set of options to control data representation.
Macros (see page 301)	The collection of macro substitution variables.
OnCommandChanged (see page 301)	The event fires after CommandText (see page 298) is changed.
OnError (see page 302)	The event fires when an error happens, while command is communicating with DBMS.
Overload (see page 303)	Oracle stored procedure overload index.
ParamBindMode (see page 303)	Determines the order in which a component's parameters are assigned to the parameter list for SQL command on the DBMS.
Params (see page 303)	The collection of parameters for a SQL command.
Prepared (see page 304)	Gets or sets command prepare for execution status.
ResourceOptions (see page 304)	The set of options to control resources usage.
R RowsAffected (see page 304)	Returns the number of rows operated upon by the latest query execution.
SchemaName (see page 305)	Identifies schema name for some of command kinds.
R SQLText (see page 306)	The SQL command text as it will be sent to the DBMS.
R State (see page 306)	Returns the current state of the command object.
Transaction (see page 306)	Specifies a transaction object.
UpdateOptions (see page 307)	The set of options to control updates posting.

Values (see page 307)	Provides access to the values for all parameters for the command.
AbortJob (see page 307)	Use the AbortJob method to abort current command operation.
AcquireConnection (see page 308)	Acquires reference to the actual connection object.
Close (see page 308)	Closes command current cursor.
CloseAll (see page 309)	Closes all command cursors.
Define (see page 309)	Defines DatS manager for command multiple result sets.
Define (see page 309)	Define DatS table structure for command's current cursor.
Disconnect (see page 310)	Use Disconnect method to release DBMS resources, used by this command.
Execute (see page 310)	Executes SQL command.
Execute (see page 311)	Executes SQL command.
Execute (see page 311)	Executes SQL command.
Execute (see page 312)	Executes SQL command.
Fetch (see page 312)	Fetches rows from current command cursor.
FillParams (see page 313)	Fills the Params (see page 303) collection.
FindMacro (see page 314)	Returns macro by its name.
FindParam (see page 314)	Returns parameter by its name.
GetConnection (see page 314)	Returns the connection object.
MacroByName (see page 315)	Returns macro by its name.
NextRecordSet (see page 315)	Go to the next cursor, returned by the SQL command.
Open (see page 316)	Executes SQL command, returning cursor.
OpenOrExecute (see page 316)	Execute (see page 310) SQL command, optionally returning cursor.
ParamByName (see page 316)	Returns parameter by its name.
Prepare (see page 317)	Prepares SQL command for execution.
ReleaseConnection (see page 317)	Releases reference to the actual connection object.
Unprepare (see page 318)	Releases SQL command resources.

Class Hierarchy



File

uADCompClient

Description

Use the TADCustomCommand class to execute SQL command. The class does not provide TDataSet-aware access to returned resultsets.

This class hides some of properties, allowing to control visibility of properties by descendant classes. End users should use TADCommand.

Syntax

```
TADCustomCommand = class(TADComponent, IUnknown, IADStanOptions, IADStanErrorHandler,
IADStanObject, IADStanAsyncHandler, IADPhysCommandStateHandler);
```

See Also

Executing Command (see page 66), Executing Stored Procedure (see page 71), TADCommand (see page 267),

TADCustomQuery (see page 394), IADPhysCommand

1.17.1.1.4.1 public

1.17.1.1.4.1.1 TADCustomCommand.Active Property

Gets or sets command active status.

Description

Set Active to True to execute SQL command and return cursor. After that the State (see page 306) = csOpen and you can fetch rows from current cursor using Fetch (see page 312) method. Setting Active to True, will call Open (see page 316) method.

Set Active to False to close current command cursor. After that State (see page 306) = csPrepared. Setting Active to False, will call Close (see page 308)method. To close all command cursors (if command returns multiple result sets), use CloseAll (see page 309) method.

Syntax

```
property Active: Boolean;
```

See Also

Fetch (see page 312), Define (see page 309), Open (see page 316), Close (see page 308), CloseAll (see page 309), State (see page 306)

Example

```
ADCommand1.CommandText.Text := 'select * from "Orders"';
ADCommand1.Active := True;
oTab := ADCommand1.Define;
try
  ADCommand1.Fetch(oTab);
  ....
finally
  oTab.Free;
end;
```

1.17.1.1.4.1.2 TADCustomCommand.ActiveStoredUsage Property

Controls how to use Active (see page 293) property value saved to DFM.

Description

The ActiveStoredUsage property controls how to use Active (see page 293) property value saved to DFM. Include:

- auDesignTime, to use property value at design time;
- auRunTime, to use property value at run time.

Syntax

```
property ActiveStoredUsage: TADStoredActivationUsage;
```

See Also

Active (see page 293)

1.17.1.1.4.1.3 TADCustomCommand.AfterClose Event

The event fires after command is closed.

Description

The AfterClose event fires, after command is closed by calling Close (see page 308) or setting Active (see page 293) = False.

Syntax

```
property AfterClose: TNotifyEvent;
```

See Also

BeforeClose (see page 296), Close (see page 308), Active (see page 293)

1.17.1.1.4.1.4 TADCustomCommand.AfterExecute Event

The event fires after command is executed.

Description

The AfterExecute event fires, after command is executed by calling Execute (see page 310) method.

Syntax

```
property AfterExecute: TNotifyEvent;
```

See Also

BeforeExecute (see page 296), Execute (see page 310)

Example

```
procedure TForm1.ADCommand1AfterExecute(ASender: TObject);
begin
  if ADCommand1.RowsAffected = -1 then
    StatusBar1.SimpleText := 'Ok'
  else
    case ADCommand1.CommandKind of
    skDelete: StatusBar1.SimpleText := Format('%d rows deleted', [ADCommand1.RowsAffected]);
    skInsert: StatusBar1.SimpleText := Format('%d rows inserted',
[ADCommand1.RowsAffected]);
    skUpdate: StatusBar1.SimpleText := Format('%d rows updated', [ADCommand1.RowsAffected]);
    else      StatusBar1.SimpleText := Format('%d rows affected',
[ADCommand1.RowsAffected]);
    end;
end;
```

1.17.1.1.4.1.5 TADCustomCommand.AfterFetch Event

The event fires after fetching from command cursor.

Description

The AfterFetch event fires, after rows are fetched from command cursor by calling Fetch (see page 312) method.

Syntax

```
property AfterFetch: TNotifyEvent;
```

See Also

BeforeFetch (see page 296), Fetch (see page 312)

Example

```
procedure TForm1.ADCommand1AfterFetch(ASender: TObject);
begin
  StatusBar1.SimpleText := Format('%d rows fetched', [ADCommand1.RowsAffected]);
end;
```

1.17.1.1.4.1.6 TADCustomCommand.AfterOpen Event

The event fires after command is opened.

Description

The AfterOpen event fires, after command is opened by calling Open (see page 316) or setting Active (see page 293) = True.

Syntax

```
property AfterOpen: TNotifyEvent;
```

See Also

BeforeOpen (see page 296), Open (see page 316), Active (see page 293)

1.17.1.1.4.1.7 TADCustomCommand.AfterPrepare Event

The event fires after command is prepared.

Description

The AfterPrepare event fires, after command is prepared by calling Prepare (see page 317) method.

Syntax

```
property AfterPrepare: TNotifyEvent;
```

See Also

BeforePrepare (see page 296), Prepare (see page 317)

1.17.1.1.4.1.8 TADCustomCommand.AfterUnprepare Event

The event fires after command is unprepared.

Description

The AfterUnprepare event fires, after command is unprepared by calling Unprepare (see page 318) method.

Syntax

```
property AfterUnprepare: TNotifyEvent;
```

See Also

BeforeUnprepare (see page 297), Unprepare (see page 318)

1.17.1.1.4.1.9 TADCustomCommand.BaseObjectName Property

Identifies base object name for some of command kinds.

Description

The BaseObjectName identifies the name of the base object. The meaning depends on CommandKind (see page 298) and MetaInfoKind (see page 457):

- If CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs], then BaseObjectName specifies the name of the package, if procedure is packaged procedure.
- If MetaInfoKind (see page 457) in [mkIndexFields, mkPrimaryKeyFields, mkForeignKeyFields], then BaseObjectName specifies the name of the table.
- If MetaInfoKind (see page 457) in [mkProcs, mkProcArgs], then BaseObjectName specifies the name of the package.

Syntax

```
property BaseObjectName: String;
```

See Also

CommandKind (see page 298), TADMetaInfoCommand.MetaInfoKind (see page 457)

Example

```
ADCommand1.BaseObjectName := 'MY_PACK';
ADCommand1.CommandText.Text := 'PROC1';
// or just ADCommand1.CommandText.Text := 'MY_PACK.PROC1';
ADCommand1.CommandKind := skStoredProc;
ADCommand1.Prepare;
ADCommand1.Params[0].AsInteger := 100;
ADCommand1.Params[1].AsString := 'Codegear Delphi';
ADCommand1.Execute;
```

1.17.1.1.4.1.10 TADCustomCommand.BeforeClose Event

The event fires before command will be closed.

Description

The BeforeClose event fires, before command will be closed by calling Close (see page 308) or setting Active (see page 293) = False.

Syntax

```
property BeforeClose: TNotifyEvent;
```

See Also

AfterClose (see page 293), Close (see page 308), Active (see page 293)

1.17.1.1.4.1.11 TADCustomCommand.BeforeExecute Event

The event fires before command will be executed.

Description

The BeforeExecute event fires, before command will be executed by calling Execute (see page 311) method.

Syntax

```
property BeforeExecute: TNotifyEvent;
```

See Also

AfterExecute (see page 294), Execute (see page 310)

1.17.1.1.4.1.12 TADCustomCommand.BeforeFetch Event

The event fires before fetching from command cursor.

Description

The BeforeFetch event fires, before rows will be fetched from command cursor by calling Fetch (see page 312) method.

Syntax

```
property BeforeFetch: TNotifyEvent;
```

See Also

AfterFetch (see page 294), Fetch (see page 312)

1.17.1.1.4.1.13 TADCustomCommand.BeforeOpen Event

The event fires before command will be opened.

Description

The BeforeOpen event fires, before command will be opened by calling Open (see page 316) or setting Active (see page 293) = True.

Syntax

```
property BeforeOpen: TNotifyEvent;
```

See Also

AfterOpen (see page 294), Open (see page 316), Active (see page 293)

1.17.1.1.4.1.14 TADCustomCommand.BeforePrepare Event

The event fires before command will be prepared.

Description

The BeforePrepare event fires, before command will be prepared by calling Prepare (see page 317) method.

Syntax

```
property BeforePrepare: TNotifyEvent;
```

See Also

AfterPrepare (see page 295), Prepare (see page 317)

1.17.1.1.4.1.15 TADCustomCommand.BeforeUnprepare Event

The event fires before command will be unprepared.

Description

The BeforeUnprepare event fires, before command wille be unprepared by calling Unprepare (see page 318) method.

Syntax

```
property BeforeUnprepare: TNotifyEvent;
```

See Also

BeforePrepare (see page 296), Unprepare (see page 318)

1.17.1.1.4.1.16 TADCustomCommand.BindedBy Property

Determines how command is connected to connection object.

Description

The BindedBy property returns value, determining how command is connected to the connection object:

- bbObject if Connection (see page 299) property is not nil;
- bbName is ConnectionName (see page 300) property is not empty.

Syntax

```
property BindedBy: TADBindedBy;
```

See Also

Connection (see page 299), ConnectionName (see page 300)

1.17.1.1.4.1.17 TADCustomCommand.CatalogName Property

Identifies catalog name for some command kinds.

Description

The CatalogName identifies the name of the catalog where resides command object. The meaning depends on CommandKind (see page 298) and MetainfoKind (see page 457):

- If CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs], then CatalogName specifies the name of the catalog, where resides the procedure.
- If MetainfoKind (see page 457) is not mkNone, then CatalogName specifies the name of the catalog, where resides the describing object.

Syntax

```
property CatalogName: String;
```

See Also

CommandKind (see page 298), TADMetaInfoCommand.MetaInfoKind (see page 457), SchemaName (see page 305)

Example 1

```
ADCommand1.CatalogName := 'MY_APP';
ADCommand1.CommandText.Text := 'PROC1';
ADCommand1.CommandKind := skStoredProc;
ADCommand1.Prepare;
ADCommand1.Params[0].AsInteger := 100;
ADCommand1.Params[1].AsString := 'Bill Gates';
```

```
ADCommand1.Execute;
```

Example 2

```
ADMetaInfoCommand1.CatalogName := 'MY_APP';
ADMetaInfoCommand1.ObjectName := 'MY_TAB';
// or on SQL Server just ADMetaInfoCommand1.ObjectName := 'MY_APP..MY_TAB';
ADMetaInfoCommand1.MetaInfoKind := mkTableFields;
ADMetaInfoCommand1.Open;
```

1.17.1.1.4.1.18 TADCustomCommand.CommandIntf Property

Returns IADPhysCommand interface.

Description

The CommandIntf property returns the reference to the IADPhysCommand interface. It is not nil, if the command is prepared. Otherwise it is nil. It is not recommended to mix usage of TADCommand API and TADCommand.CommandIntf API.

Syntax

```
property CommandIntf: IADPhysCommand;
```

See Also

IADPhysCommand

1.17.1.1.4.1.19 TADCustomCommand.CommandKind Property

Identifies the command kind.

Description

The CommandKind property identifies the kind of the content of CommandText (see page 298) property. You can explicitly set it value before Prepare call, otherwise it will be automatically assigned by FireDAC after analyzing CommandText (see page 298) content. There is only single need to assign it - when you are preparing stored procedure.

Syntax

```
property CommandKind: TADPhysCommandKind;
```

See Also

CommandText (see page 298)

Example 1

```
ADCommand1.CommandKind := skStoredProc;
ADCommand1.CommandText.Text := 'MY_PROC';
ADCommand1.Prepare;
```

Example 2

```
ADCommand1.CommandText.Text := 'ALTER PROCEDURE MY_PROC COMPILE';
case ADCommand1.CommandKind of
skAlter: ShowMessage('Alter command');
.........
else     ShowMessage('Other command');
end;
```

1.17.1.1.4.1.20 TADCustomCommand.CommandText Property

The SQL command text to execute.

Description

The CommandText set/get:

- SQL command text to execute
- or name of the stored procedure to execute, if CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs].

After setting CommandText, FireDAC will do for non-stored procedures:

- extract parameters names and fill Params (see page 303) collection, if ResourceOptions.ParamCreate (see page 883) = True;
- extract macro name and fill Macros (see page 301) collection, if ResourceOptions.MacroCreate (see page 882) = True;
- extract FROM table name for SELECT commands and assign it to CommandIntf (see page 298).SourceObjectName, if ResourceOptions.PreprocessCmdText (see page 878) = True;
- fill CommandKind (see page 298) property value, if it is not explicitly set.

If you are adding long command, using CommandText.Add method, then it is recommended before command modification to call CommandText.BeginUpdate and after finishing command modification to call CommandText.EndUpdate.

Later on Prepare (see page 317) call command text will be preprocessed and transformed in target DBMS command. See chapter Preprocessing Command Text (see page 56) for details about FireDAC macros and escape sequences - preprocessor instructions.

Syntax

```
property CommandText: TStrings;
```

See Also

Preprocessing Command Text (see page 56), Executing Command (see page 66), TADResourceOptions.ParamCreate (see page 883), TADResourceOptions.MacroCreate (see page 882),TADResourceOptions.PreprocessCmdText (see page 878), CommandKind (see page 298), Params (see page 303), Macros (see page 301), ResourceOptions (see page 304)

Example 1

Stored procedure:

```
ADCommand1.CommandKind := skStoredProc;
ADCommand1.CommandText.Text := 'MY_PROC';
ADCommand1.Prepare;
ADCommand1.Params[0].AsString := 'qwe';
ADCommand1.Execute;
```

Example 2

General (see page 17) SQL command:

```
ADCommand1.CommandText.Text := 'insert into MyTab values (:p1)';
ADCommand1.Params[0].AsString := 'qwe';
ADCommand1.Execute;
```

Example 3

Adding long command:

```
with ADCommand1.CommandText do begin
  BeginUpdate;
  Add('............');
  ......
  Add('............');
  EndUpdate;
end;
```

1.17.1.1.4.1.21 TADCustomCommand.Connection Property

Specifies the connection object.

Description

The Connection property value points to the connection object. Alternatively may be specified the ConnectionName (see page 300) property value. The Connection property value must be specified before Prepare (see page 317) call.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

ConnectionName (see page 300), Prepare (see page 317)

1.17.1.1.4.1.22 TADCustomCommand.ConnectionName Property

Specifies connection name.

Description

The ConnectionName property value specifies the name of the connection. It must match to the:

- name of one of connection definitions, either stored in external file or created on fly;
- ConnectionName of one of the TADConnection (see page 279) objects.

Alternatively may be specified the Connection (see page 299) property value. The ConnectionName property value must be specified before Prepare (see page 317)call. If is specified the name of one of connection definitions, then FireDAC will transparently create connection object and link it with command.

Syntax

```
property ConnectionName: String;
```

See Also

Connection (see page 299), Prepare (see page 317)

1.17.1.1.4.1.23 TADCustomCommand.DataSet Property

Reference to dataset object owning this command.

Description

Use the DataSet property to get a reference to the dataset object, which is owning this command object. For example, TADQuery (see page 476).Command (see page 261).DataSet returns the reference to the TADQuery.

Syntax

```
property DataSet: TADDataSet;
```

See Also

TADAdaptedDataSet Class (see page 259)

1.17.1.1.4.1.24 TADCustomCommand.FetchOptions Property

The set of options to control rows fetching.

Description

The FetchOptions property is the set of properties, controlling the fetching of cursor rows. These properties will inherit its values from FetchOptions of connection object.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), TADFetchOptions (see page 849), Fetch (see page 312)

1.17.1.1.4.1.25 TADCustomCommand.FixedCommandKind Property

True, if the CommandKind (see page 298) property value is set explicitly.

Syntax

```
property FixedCommandKind: Boolean;
```

See Also

CommandKind (see page 298)

1.17.1.1.4.1.26 TADCustomCommand.FormatOptions Property

The set of options to control data representation.

Description

The FormatOptions property is the set of properties, controlling the data type mapping, and some other aspects of data handling.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), TADFormatOptions (see page 860), Fetch (see page 312), Params (see page 303)

1.17.1.1.4.1.27 TADCustomCommand.Macros Property

The collection of macro substitution variables.

Description

The Macros property is the collection of macro substitution variables. It is populated automatically after assigning CommandText (see page 298), if ResourceOptions (see page 304).MacroCreate (see page 882) = True. Otherwise it may be populated by hands. At Prepare (see page 317) call, macro values will be substituted into CommandText (see page 298), if ResourceOptions (see page 304).MacroExpand (see page 882) = True.

Syntax

```
property Macros: TADMacros;
```

See Also

Executing Command (see page 66), Preprocessing Command Text (see page 56), CommandText (see page 298), Params (see page 303), Prepare (see page 317), TADMacros, TADResourceOptions.MacroCreate (see page 882), TADResourceOptions.MacroExpand (see page 882)

Example 1

Substitute table name:

```
ADCommand1.CommandText.Text := 'select * from &Tab';
ADCommand1.Macros[0].AsIdentifier := 'Order Details';
ADCommand1.Prepare;
ShowMessage(ADCommand1.SQLText); // select * from "Order Details"
```

Example 2

Substitute WHERE condition:

```
ADCommand1.CommandText.Text := 'select * from MyTab {if !cond} where !cond {fi}';
ADCommand1.Macros[0].AsString := 'ID > 100';
ADCommand1.Prepare;
ShowMessage(ADCommand1.SQLText); // select * from MyTab where ID > 100
ADCommand1.Macros[0].Clear;
ADCommand1.Prepare;
ShowMessage(ADCommand1.SQLText); // select * from MyTab
```

Example 3

See FireDAC\Samples\Comp Layer\TADQuery (see page 476)\Macros demo for details.

1.17.1.1.4.1.28 TADCustomCommand.OnCommandChanged Event

The event fires after CommandText (see page 298) is changed.

Description

The OnCommandChanged event fires, after CommandText (see page 298) is changed.

Syntax

```
property OnCommandChanged: TNotifyEvent;
```

See Also

CommandText (see page 298)

Example

```
procedure TForm1.ADCommand1CommandChanged(ASender: TObject);
begin
  if ADCommand1.FindParam('ID') <> nil then
    ADCommand1.FindParams('ID').AsInteger := 100;
end;
```

1.17.1.1.4.1.29 TADCustomCommand.OnError Event

The event fires when an error happens, while command is communicating with DBMS.

Parameters

Parameters	Description
ASender	The reference to command.
AInitiator	The reference to interface of object, which initially raised the exception.
AException	The exception object, which may be modified or even replaced by another exception object instance.

Description

The OnError event fires, when command executes one of the following operations and an error happens:

- Prepare (see page 317). The command is preparing DBMS command text for execution.
- Open (see page 316). The command is executing DBMS command to open cursor.
- Execute (see page 311). The command is executing DBMS command text.
- Fetch (see page 312). The command is fetching rows from cursor.

If most cases AException object will of EADDBEngineException (see page 833) class. If AException is of EADDBArrayExecuteError (see page 831) class, then it is Array DML (see page 82) error handling case. You should use code like in the example.

Syntax

```
property OnError: TADErrorEvent;
```

See Also

Handling Errors (see page 44), Array DML (see page 82), Open (see page 316), Execute (see page 310), Prepare (see page 317), Fetch (see page 312), EADDBArrayExecuteError (see page 831) class, EADDBEngineException (see page 833) class

Example 1

Handling Array DML errors:

```
procedure TForm1.ADCommand1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if AException is EADPhysArrayExecuteError then
    with EADPhysArrayExecuteError(AException) do
      if Errors[0].Kind = ekUKViolated then
        AAction := eaSkip
      else
        AAction := eaFail
  else
    ...
end;
```

Example 2

Error substitution:

```
procedure TForm1.ADCommand1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if EADDBEngineException(AException).Errors[0].Kind = ekPKViolated then begin
    AException.Free;
    AException := Exception.Create('Please, enter unique ID value');
  end;
end;
```

1.17.1.1.4.1.30 TADCustomCommand.Overload Property

Oracle stored procedure overload index.

Description

The Overload property specifies Oracle packaged stored procedure overload index. It has no meaning for other DBMS's.

Syntax

```
property Overload: Word;
```

See Also

CommandText (see page 298), CommandKind (see page 298)

Example

```
ADCommand1.CommandKind := skStoredProc;
ADCommand1.CommandText.Text := 'DBMS_SQL.PARSE';
ADCommand1.Overload := 1;
```

1.17.1.1.4.1.31 TADCustomCommand.ParamBindMode Property

Determines the order in which a component's parameters are assigned to the parameter list for SQL command on the DBMS.

Description

Get or set ParamBindMode to determine the order in which parameters in the Params (see page 303) property are matched to the parameters used by the SQL command on the server. ParamBindMode can be one of the following:

Value	Ordering
pbByName	Parameters specified in the Params (see page 303) property are matched to identically named parameters on the server. This is the default.
pbByNumber	Parameters in Params (see page 303) are assigned one-by-one to the next available parameter on the server (for example, the first parameter in Params (see page 303) is assigned to the first parameter used by the SQL command, and so on).

Whenever possible, ParamBindMode should be pbByName. This guarantees that parameters are matched to the correct parameters used by the SQL command regardless of physical ordering in Params (see page 303). At design time, the names of known parameters appear in the Parameters editor.

Syntax

```
property ParamBindMode: TADParamBindMode;
```

See Also

Params (see page 303)

1.17.1.1.4.1.32 TADCustomCommand.Params Property

The collection of parameters for a SQL command.

Description

The Params property is the collection of parameters for a SQL command.

If CommandKind (see page 298) is not in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs], then Params is populated automatically after assigning CommandText (see page 298), if ResourceOptions (see page 304).ParamCreate (see page 883) = True.

If CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs], then Params is populated automatically after calling Prepare (see page 317) method, if fiMeta is in FetchOption (see page 300).Items (see page 855).

Otherwise Params may be populated by hands. At Prepare (see page 317) call, parameters will be binded to prepared SQL command. After that you cannot change parameter types, otherwise will get an exception.

Syntax

```
property Params: TADParams;
```

See Also

Executing Command (see page 66), CommandText (see page 298), Macros (see page 301), Prepare (see page 317), TADParams, TADResourceOptions.ParamCreate, TADFetchOptions.Items

1.17.1.1.4.1.33 TADCustomCommand.Prepared Property

Gets or sets command prepare for execution status.

Description

Set Prepared to True to prepare SQL command for execution. After that the State (see page 306) = csPrepared. Setting Prepared to True, will call Prepare (see page 317) method. It is not required to set Prepared to True for standard SQL command before execution, because first Execute (see page 310) / Open (see page 316) call will automatically prepare command. But it is required for stored procedures, if you need to automatically populate Params (see page 303) collection. After command is prepared, SQLText (see page 306) will return SQL command text as it is sent to the DBMS.

To prepare SQL command DBMS connection must be active, otherwise an exception will be raised.

After SQL command preparation application cannot change command parameter data types and sizes, otherwise on next Execute (see page 310) or Open (see page 316) call an exception will be raised. So, setup parameters before setting Prepared to True.

Set Prepared to False to unprepare SQL command and release all it resources. After that State (see page 306) = csInactive. Setting Prepared to False, will call Unprepare (see page 318) method.

Syntax

```
property Prepared: Boolean;
```

See Also

Prepare (see page 317), Unprepare (see page 318), State (see page 306), Params (see page 303)

1.17.1.1.4.1.34 TADCustomCommand.ResourceOptions Property

The set of options to control resources usage.

Description

The ResourceOptions property is the set of properties, controlling the resources usage by the command. These properties will inherit its values from ResourceOptions of connection object.

Syntax

```
property ResourceOptions: TADBottomResourceOptions;
```

See Also

Setting Options (see page 34), TADBottomResourceOptions (see page 841)

1.17.1.1.4.1.35 TADCustomCommand.RowsAffected Property

Returns the number of rows operated upon by the latest query execution.

Description

Inspect RowsAffected to determine how many rows were inserted, updated, deleted or fetched by the last command operation. If no rows were processed, RowsAffected = 0. If numbers of processed rows is not accessible, RowsAffected = –1.

At MS SQL Server RowsAffected may be unexpectedly equal to -1, if stored procedure or table trigger omits SET NOCOUNT ON. Check BOL for more details.

Syntax

```
property RowsAffected: TADCounter;
```

See Also

Execute (see page 310), Fetch (see page 312)

Example

```
procedure TForm1.ADCommand1AfterExecute(ASender: TObject);
begin
  if ADCommand1.RowsAffected = -1 then
    StatusBar1.SimpleText := 'Ok'
  else
    case ADCommand1.CommandKind of
    skDelete: StatusBar1.SimpleText := Format('%d rows deleted', [ADCommand1.RowsAffected]);
    skInsert: StatusBar1.SimpleText := Format('%d rows inserted',
[ADCommand1.RowsAffected]);
    skUpdate: StatusBar1.SimpleText := Format('%d rows updated', [ADCommand1.RowsAffected]);
    else      StatusBar1.SimpleText := Format('%d rows affected',
[ADCommand1.RowsAffected]);
    end;
end;
```

1.17.1.1.4.1.36 TADCustomCommand.SchemaName Property

Identifies schema name for some of command kinds.

Description

The SchemaName identifies the name of the schema where resides command object. The meaning depends on CommandKind (see page 298) and MetainfoKind (see page 457):

- If CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs], then SchemaName specifies the name of the schema, where resides the procedure.
- If MetainfoKind (see page 457) is not mkNone, then SchemaName specifies the name of the schema, where resides the describing object.

Syntax

```
property SchemaName: String;
```

See Also

CommandKind (see page 298), TADMetaInfoCommand.MetaInfoKind (see page 457), SchemaName

Example 1

```
ADCommand1.SchemaName := 'SCOTT';
ADCommand1.CommandText.Text := 'PROC1';
ADCommand1.CommandKind := skStoredProc;
ADCommand1.Prepare;
ADCommand1.Params[0].AsInteger := 100;
ADCommand1.Params[1].AsString := 'Bill Gates';
ADCommand1.Execute;
```

Example 2

```
ADMetaInfoCommand1.SchemaName := 'ADDEMO';
ADMetaInfoCommand1.ObjectName := 'MY_TAB';
// or on Oracle just ADMetaInfoCommand1.ObjectName := 'ADDEMO.MY_TAB';
ADMetaInfoCommand1.MetaInfoKind := mkTableFields;
ADMetaInfoCommand1.Open;
```

1.17.1.1.4.1.37 TADCustomCommand.SQLText Property

The SQL command text as it will be sent to the DBMS.

Description

The SQLText property returns SQL command text as it will be sent to the DBMS. It is accessible only after command is prepared. This text will not have any of command preprocess instructions (macros, escapes, etc), will have adjusted parameter markers and end-of-line markers.

Syntax

```
property SQLText: String;
```

See Also

CommandText (see page 298), Prepare (see page 317)

1.17.1.1.4.1.38 TADCustomCommand.State Property

Returns the current state of the command object.

Description

The property State returns the current state of the command object. It has one of the following values:

Value	Description
csInactive	The command is not prepared for execution.
csPrepared	The command is prepared for execution.
csExecuting	The command is executing.
csOpen	The command is executed and it has active cursor.
csFetching	The command is fetching rows.
csAborting	The command is aborting current operation.

csExecuting, csFetching, csAborting values are "visible" if command is executing asynchronously or State property is checked from other thread than performing operation.

Syntax

```
property State: TADPhysCommandState;
```

See Also

Prepare (see page 317), Execute (see page 310), Open (see page 316), AbortJob (see page 307), Fetch (see page 312), Close (see page 308), Unprepare (see page 318)

1.17.1.1.4.1.39 TADCustomCommand.Transaction Property

Specifies a transaction object.

Description

The Transaction property value points to a transaction object. The SQL command will be prepared and executed in the context of this transaction. The Transaction property value must be specified before Prepare call. If a value is not specified, then command will use default transaction object, specified at TADCustomConnection (see page 318).Transaction (see page 337) or internal connection transaction object.

Note, at moment only Interbase / Firebird driver supports assignment of transaction to the command.

Syntax

```
property Transaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41), TADTransaction (see page 561), TADCustomConnection.Transaction (see

page 337)

1.17.1.1.4.1.40 TADCustomCommand.UpdateOptions Property

The set of options to control updates posting.

Description

The UpdateOptions property is the set of properties, controlling the posting of updates. These properties will inherit its values from UpdateOptions of a connection object.

Syntax

```
property UpdateOptions: TADBottomUpdateOptions;
```

See Also

Setting Options (see page 34), TADBottomUpdateOptions (see page 843)

1.17.1.1.4.1.41 TADCustomCommand.Values Property

Provides access to the values for all parameters for the command.

Description

The property Values gets / sets values for parameters in a command. AParamNames is the single parameter name or ';' separated list of the parameter names to read from or write to.

Values accepts and returns a Variant, so it can handle and convert parameters of any data type. Because Values is the default property for TADCommand, the property name may be omitted when referencing this property.

Syntax

```
property Values [const AParamNames: String]: Variant;
```

See Also

Params (see page 303), ParamByName (see page 316), FindParam (see page 314), TADParam.Value

Example 1

The following statements are semantically identical and write the value from an edit box into an integer parameter:

```
ADCommand1.Values['CustNo'] := Edit1.Text;
ADCommand1['CustNo'] := Edit1.Text;
```

Example 2

The next statement reads a string value from a parameter into an edit box:

```
Edit1.Text := ADCommand1['Company'];
```

1.17.1.1.4.1.42 TADCustomCommand.AbortJob Method

Use the AbortJob method to abort current command operation.

Parameters

Parameters	Description
AWait: Boolean = False	If True, then caller will wait until a current operation will be terminated.

Description

The `AbortJob` method aborts one of the following command operations:

- Open (see page 316). The command is executing DBMS command to open cursor.
- Execute (see page 310). The command is executing DBMS command.
- Fetch (see page 312). The command is fetching rows from cursor.

After `AbortJob` call command state will be csPrepared for Open (see page 316)/Execute (see page 310) calls. And csActive for Fetch (see page 312). The AbortJob method must be called from the other thread, than where a operation is

performing.

Note, not all DBMS API's support abortion of command execution. In this case the AbortJob method will wait until the execution will be finished.

Syntax

```
procedure AbortJob(AWait: Boolean = False);
```

See Also

Asynchronous Execution (see page 86)

Example

```
ADCommand1.ResourceOptions.CmdExecMode := amAsync;
ADCommand1.Prepare('select OrderID, count(*) from "Order Details" group by OrderID');
ADCommand1.Open;
ADCommand1.AbortJob(True);
```

1.17.1.1.4.1.43 TADCustomCommand.AcquireConnection Method

Acquires reference to the actual connection object.

Returns

The connection object. Or raises exception, if Connection (see page 299) and ConnectionName (see page 300) property values are empty.

Description

The AcquireConnection method acquires connection object reference, depending on Connection (see page 299) or ConnectionName (see page 300) property values, and increment it usage count. After usage connection object must be released, by calling ReleaseConnection (see page 317) method. Command object automatically acquires connection at Prepare (see page 317) call and releases connection at Unprepare (see page 318) call.

Syntax

```
function AcquireConnection: TADCustomConnection;
```

See Also

ReleaseConnection (see page 317), Connection (see page 299), ConnectionName (see page 300)

1.17.1.1.4.1.44 TADCustomCommand.Close Method

Closes command current cursor.

Description

The Close method closes current command cursor. After that State (see page 306) = csPrepared.

To close all command cursors (if command returns multiple result sets) use CloseAll (see page 309) method. If there is only single result set, then Close is equivalent to CloseAll (see page 309). After Close call to forward command to next accessible cursor use NextRecordSet (see page 315) method.

The command object will automatically close all command cursors after fetching of last record from current cursor, if FetchOptions (see page 300).AutoClose (see page 851) = False.

Syntax

```
procedure Close;
```

See Also

Open (see page 316), Active (see page 293), CloseAll (see page 309), NextRecordSet (see page 315), BeforeClose (see page 296), AfterClose (see page 293)

Example

```
var
  oTab: TADDatSTable;
....
```

```
ADCommand1.Prepare('select * from "Orders"');
oTab := ADCommand1.Define;
try
  ADCommand1.Fetch(oTab);
finally
  ADCommand1.Close;
  oTab.Free;
end;
```

1.17.1.1.4.1.45 TADCustomCommand.CloseAll Method

Closes all command cursors.

Description

The CloseAll method closes all command cursors. After that State (see page 306) = csPrepared.

Syntax

```
procedure CloseAll;
```

See Also

Open (see page 316), Active (see page 293), Close (see page 308), NextRecordSet (see page 315), BeforeClose (see page 296), AfterClose (see page 293)

1.17.1.1.4.1.46 TADCustomCommand.Define Method (TADDatSManager, TADDatSTable, TADPhysMetaInfoMergeMode)

Defines DatS manager for command multiple result sets.

Parameters

Parameters	Description
ASchema: TADDatSManager	_nt_
ATable: TADDatSTable = nil	_nt_
AMetaInfoMergeMode: TADPhysMetaInfoMergeMode = mmReset	_nt_

Returns

..

Description

..

Syntax

```
function Define(ASchema: TADDatSManager; ATable: TADDatSTable = nil; AMetaInfoMergeMode:
TADPhysMetaInfoMergeMode = mmReset): TADDatSTable; overload;
```

See Also

..

Example

..

1.17.1.1.4.1.47 TADCustomCommand.Define Method (TADDatSTable, TADPhysMetaInfoMergeMode)

Define DatS table structure for command's current cursor.

Parameters

Parameters	Description
ATable: TADDatSTable = nil	A reference to the existing DatS table or nil.
AMetaInfoMergeMode: TADPhysMetaInfoMergeMode = mmReset	Defines how to adjust ATable DatS table structure using current result set structure.

Returns

The reference to the existing DatS table.

If ATable was nil, then returned value is new created DatS table. The calling code is responsible for freeing it after usage.

Description

The Define method defines DatS table structure for command's current cursor if command is open or for first cursor if it is closed. Method creates new DatS table, if ATable parameter value is nil. If the SQL command is SELECT and fiMeta in FetchOptions.Items, then Define will request DBMS for primary key fields for table in FROM clause. And method will fill DatS table column list.

Syntax

```
function Define(ATable: TADDatSTable = nil; AMetaInfoMergeMode: TADPhysMetaInfoMergeMode =
mmReset): TADDatSTable; overload;
```

See Also

TADDatSTable, TADDatSTable.Columns

Example

```
var
  oTab: TADDatSTable;
....
  ADCommand1.Prepare('select * from MyTab');
  oTab := ADCommand1.Define;
  ADCommand1.Open;
  ADCommand1.Fetch(oTab);
  ADCommand1.Close;

  // Some time later, after the structure of DB table MyTab have changed.
  // For example, field was removed.

  ADCommand1.Prepare('select * from MyTab');
  oTab := ADCommand1.Define(oTab, mmOverride);
  ADCommand1.Open;
  ADCommand1.Fetch(oTab);
  ADCommand1.Close;
```

1.17.1.1.4.1.48 TADCustomCommand.Disconnect Method

Use Disconnect method to release DBMS resources, used by this command.

Parameters

Parameters	Description
AAbortJob: Boolean = False	If True, then method will abort current operation.

Description

The Disconnect method optionally abort current operation and releases DBMS resources, used by this command. After the call command is in csInactive state.

Syntax

```
procedure Disconnect(AAbortJob: Boolean = False);
```

Example

```
ADCommand1.CommandText.Text := 'select * from [Order Details]';
ADCommand1.Prepare := True;
ADCommand1.Active := True;
ADCommand1.Disconnect;
// Active = False and Prepare = False
```

1.17.1.1.4.1.49 TADCustomCommand.Execute Method (Integer, Integer, Boolean)

Executes SQL command.

Parameters

Parameters	Description
ATimes: Integer = 0	A number of times to execute SQL command.
AOffset: Integer = 0	A index of first row in parameters array, to start execution.

Description

The Execute method executes SQL command specified by the CommandText (see page 298), which does not return result sets. If command will return result sets, then all they will be discarded. The command may be executed in one of the modes:

Mode	Parameter values	RowsAffected meaning
Standard or default. The SQL command will be executed single time.	ATimes <= 1	The number of rows affected by SQL command (inserted, updated, deleted, etc). This information is provided by DBMS and is not "reviewed" by FireDAC.
Array DML.	ATimes > 1	The number of successfull command executions for each of parameter array items. For detailed description see chapter Array DML (see page 82).

Syntax

```
procedure Execute(ATimes: Integer = 0; AOffset: Integer = 0; ABlocked: Boolean = False);
overload;
```

See Also

Executing Command (see page 66), RowsAffected (see page 304), CommandText (see page 298), Params (see page 303)

Example

```
ADCommand1.CommandText.Text := 'create table MyTab (f1 number, f2 varchar2(10))';
ADCommand1.Execute;
```

1.17.1.1.4.1.50 TADCustomCommand.Execute Method (String)

Executes SQL command.

Parameters

Parameters	Description
const ASQL: String	The SQL command text to execute.

Description

The Execute method executes SQL command specified by the ASQL parameter, which does not return result sets. If command will return result sets, then all they will be discarded.

Syntax

```
function Execute(const ASQL: String): LongInt; overload;
```

See Also

Executing Command (see page 66), RowsAffected (see page 304), CommandText (see page 298), Params (see page 303)

Example

```
ADCommand1.Execute('DROP TABLE MyTab');
```

1.17.1.1.4.1.51 TADCustomCommand.Execute Method (String, array of Variant)

Executes SQL command.

Parameters

Parameters	Description
const ASQL: String	The SQL command text to execute.
const AParams: array of Variant	The open array of parameter values. The values will be assigned in order of accurencies in the SQL command text.

Description

The Execute method executes parameterized SQL command specified by the ASQL parameter, which does not return result sets. If command will return result sets, then all they will be discarded.

Syntax

```
function Execute(const ASQL: String; const AParams: array of Variant): LongInt; overload;
```

See Also

Executing Command (see page 66), RowsAffected (see page 304), CommandText (see page 298), Params (see page 303)

Example

```
ADCommand1.Execute('delete from MyTab where id = : id', [100]);
```

1.17.1.1.4.1.52 TADCustomCommand.Execute Method (String, array of Variant, array of TFieldType)

Executes SQL command.

Parameters

Parameters	Description
const ASQL: String	The SQL command text to execute.
const AParams: array of Variant	The open array of parameter values. The values will be assigned in order of occurrences in the SQL command text.
const ATypes: array of TFieldType	The open array of parameter data types.

Description

The Execute method executes parameterized SQL command specified by the ASQL parameter, which does not return result sets. If command will return result sets, then all they will be discarded. The method requires to specify parameter data types.

Syntax

```
function Execute(const ASQL: String; const AParams: array of Variant; const ATypes: array
of TFieldType): LongInt; overload;
```

See Also

Executing Command (see page 66), RowsAffected (see page 304), CommandText (see page 298), Params (see page 303)

Example

```
ADCommand1.Execute('insert into MyTab (id, bval) values (:id, :bval)', [100,
StringOfChar('x', 100000)], [ftInteger, ftBLOB]);
```

1.17.1.1.4.1.53 TADCustomCommand.Fetch Method

Fetches rows from current command cursor.

Parameters

Parameters	Description
ATable: TADDatSTable	DatS table into which store fetched rows.
AAll: Boolean = True	If True, then at first Fetch call FireDAC will fetch all rows from the current cursor.

Description

The Fetch method fetches rows from current command cursor into DatS table. To call Fetch, command must be opened (in

csActive state) by Open (see page 316) call, otherwise it will raise exception.

At each call will be fetched up to FetchOptions (see page 300).RowsetSize (see page 859) rows. If FetchOptions (see page 300).AutoClose (see page 851) =True, then after fetching of last row FireDAC will automatically call CloseAll (see page 309) method.

If fiBlobs in FetchOptions (see page 300).Items (see page 855), then BLOB field values will be fetched together with other field values. If fiDetails in FetchOptions (see page 300).Items (see page 855), then nested dataset will be fetched together with other field values. While fetching, field values will be converted according FormatOptions (see page 301) and stored into DatS rows.

Syntax

```
procedure Fetch(ATable: TADDatSTable; AAll: Boolean = True; ABlocked: Boolean = False);
overload;
```

See Also

TADFetchOptions (see page 849), Close (see page 308), CloseAll (see page 309), Open (see page 316), Active (see page 293)

Example

```
var
  oTab: TADDatSTable;
...
  ADCommand1.CommandKind := skStoredProc;
  ADCommand1.CommandText.Text := 'SEL_CUST_PROC';
  oTab := ADCommand1.Define;
  try
    ADCommand1.Open;
    repeat
      oTab.Clear;
      ADCommand1.Fetch(oTab, False);
      // process next rowset
    until ADCommand1.RowsAffected = 0;
  finally
    oTab.Free;
  end;
```

1.17.1.1.4.1.54 TADCustomCommand.FillParams Method

Fills the Params (see page 303) collection.

Parameters

Parameters	Description
AParams: TADParams	The parameters collection to fill in.
const ASQL: String = ''	The SQL command from which to get parameters. If it is empty string, then will be used CommandText (see page 298).

Description

The FillParams method fills the Params (see page 303) collection, if it is not done automatically in one of the following cases:

- CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs] and fiMeta is not in FetchOptions.Items (see page 855);
- CommandKind (see page 298) is not in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs] and ResourceOptions.ParamCreate (see page 883) = False.

Syntax

```
procedure FillParams(AParams: TADParams; const ASQL: String = '');
```

See Also

CommandText (see page 298), CommandKind (see page 298), Params (see page 303), TADFetchOptions.Items (see page 855), TADResourceOptions.ParamCreate (see page 883)

1.17.1.1.4.1.55 TADCustomCommand.FindMacro Method

Returns macro by its name.

Parameters

Parameters	Description
const AValue: string	Name of a macro.

Returns

The macro with Name = AValue or nil if such macro does not exists.

Description

The FindMacro method returns a macro from Macros (see page 301) collection by its name AValue. If there is no such macro, then method will return nil.

Syntax

```
function FindMacro(const AValue: string): TADMacro;
```

See Also

Macros (see page 301), MacroByName (see page 315)

1.17.1.1.4.1.56 TADCustomCommand.FindParam Method

Returns parameter by its name.

Parameters

Parameters	Description
const AValue: string	Name of a parameter.

Returns

The parameter with Name = AValue or nil if such parameter does not exists.

Description

The FindParam method returns a parameter from Params (see page 303) collection by its name AValue. If there is no such parameter, then method will return nil.

Syntax

```
function FindParam(const AValue: string): TADParam;
```

See Also

Params (see page 303), ParamByName (see page 316), Values (see page 307)

1.17.1.1.4.1.57 TADCustomCommand.GetConnection Method

Returns the connection object.

Parameters

Parameters	Description
ACheck: Boolean	If True, and connection object is not found then method will raise the exception.

Description

Returns the connection object using either Connection (see page 299) or ConnectionName (see page 300) properties.

Syntax

```
function GetConnection(ACheck: Boolean): TADCustomConnection;
```

See Also

Connection (see page 299), ConnectionName (see page 300), AcquireConnection (see page 308), ReleaseConnection (see page 317)

1.17.1.1.4.1.58 TADCustomCommand.MacroByName Method

Returns macro by its name.

Parameters

Parameters	Description
const AValue: string	Name of a macro.

Returns

The macro with Name = AValue.

Description

The MacroByName method returns a macro from Macros (see page 301) collection by its name AValue. If there is no such macro, then exception will be raised.

Syntax

```
function MacroByName(const AValue: string): TADMacro;
```

See Also

Macros (see page 301), FindMacro (see page 314)

1.17.1.1.4.1.59 TADCustomCommand.NextRecordSet Method

Go to the next cursor, returned by the SQL command.

Description

The NextRecordSet method closes current cursor, forwards it to the next accessible cursor and opens command. If there is no more accessible cursors, then after call command is closed (State (see page 306) = csPrepared).

The Close (see page 308) method closes current cursor. The CloseAll (see page 309) method discards all assocciated cursors. So, to get all cursors, you should set FetchOptions (see page 300).AutoClose (see page 851) to False before opening command.

Oracle REF CURSOR's for FireDAC are also command cursors, and NextRecordSet call will select next PL/SQL REF CURSOR parameter.

In design time, you can forward dataset to the next cursor, by right clicking component and choosing "Next record set" popup menu item.

Syntax

```
procedure NextRecordSet;
```

See Also

TADFetchOptions.AutoClose (see page 851), Open (see page 316), Close (see page 308), CloseAll (see page 309), Fetch (see page 312)

Example

```
ADCommand1.FetchOptions.AutoClose := False;
ADCommand1.CommandText.Text := 'select 1 as i; select ''qwe'' as s';
ADCommand1.Open;
oTab := ADCommand1.Define;
try
  ADCommand1.Fetch(oTab);
  ShowMessage(oTab.Rows[0].DumpRow(True)); // output "i: 1"
finally
  oTab.Free;
end;
```

```
ADCommand1.NextRecordSet;
oTab := ADCommand1.Define;
try
  ADCommand1.Fetch(oTab);
  ShowMessage(oTab.Rows[0].DumpRow(True)); // output "s: qwe"
finally
  oTab.Free;
end;
ADQuery1.CloseAll;
```

1.17.1.1.4.1.60 TADCustomCommand.Open Method

Executes SQL command, returning cursor.

Description

Call the Open method to execute SQL command and return cursor. After that the State (see page 306) = csOpen and you can fetch rows from current cursor using Fetch (see page 312) method. The Open method will raise an exception, if command does not return cursors.

Syntax

```
procedure Open(ABlocked: Boolean = False);
```

See Also

Active (see page 293), Close (see page 308), CloseAll (see page 309), State (see page 306)

1.17.1.1.4.1.61 TADCustomCommand.OpenOrExecute Method

Execute (see page 310) SQL command, optionally returning cursor.

Returns

True, if command returned cursor.

Description

Call the OpenOrExecute method to execute SQL command, optionally returning cursor. If the SQL command returns cursor, then after call the State (see page 306)=csActive and you can fetch rows from current cursor using Fetch method. If the SQL command does not return cursors, then the State (see page 306)=csPrepared.

The OpenOrExecute method is useful, when you does not know a command kind to be executed.

Syntax

```
function OpenOrExecute(ABlocked: Boolean = False): Boolean;
```

See Also

Execute (see page 310), Open (see page 316), Active (see page 293), Close (see page 308), CloseAll (see page 309)

Example

```
if ADCommand1.OpenOrExecute then begin
  oTab := ADCommand1.Define;
  try
    ADCommand1.Fetch(oTab);
  finally
    ADCommand1.CloseAll;
    oTab.Free;
  end;
end;
```

1.17.1.1.4.1.62 TADCustomCommand.ParamByName Method

Returns parameter by its name.

Parameters

Parameters	Description
const AValue: string	Name of a parameter.

Returns

The parameter with Name = AValue.

Description

The ParamByName method returns a parameter from Params (see page 303) collection by its name AValue. If there is no such parameter, then exception will be raised.

Syntax

```
function ParamByName(const AValue: string): TADParam;
```

See Also

Params (see page 303), FindParam (see page 314), Values (see page 307)

1.17.1.1.4.1.63 TADCustomCommand.Prepare Method

Prepares SQL command for execution.

Parameters

Parameters	Description
const ACommandText: String = ''	Optional SQL command text to prepare.

Description

Call the Prepare method to prepare SQL command for execution. After that the State (see page 306) = csPrepared. It is not required to call Prepare for standard SQL command before execution, because first Execute (see page 310) / Open (see page 316) call will automatically prepare command. But it is required for stored procedures, if you need to automatically populate Params (see page 303) collection. After command is prepared, SQLText (see page 306)will return SQL command text as it is sent to the DBMS.

To prepare SQL command DBMS connection must be active, otherwise exception will be raised.

After prepare call application cannot change command parameter data types and sizes, otherwise on next Execute (see page 310) or Open (see page 316) call an exception will be raised. So, setup parameters before Prepare call.

Syntax

```
procedure Prepare(const ACommandText: String = '');
```

See Also

Prepared (see page 304), Unprepare (see page 318), State (see page 306), Params (see page 303)

Example

```
ADCommand1.Prepare('select * from MyTab');
```

1.17.1.1.4.1.64 TADCustomCommand.ReleaseConnection Method

Releases reference to the actual connection object.

Parameters

Parameters	Description
var AConnection: TADCustomConnection	The reference to the connection to release. It must be previously acquired by AcquireConnection (see page 308) call.

Description

The ReleaseConnection method releases connection object reference, and decrement it usage count. Command object automatically releases connection at Unprepare call.

Syntax

```
procedure ReleaseConnection(var AConnection: TADCustomConnection);
```

See Also

AcquireConnection (see page 308), Connection (see page 299), ConnectionName (see page 300)

1.17.1.1.4.1.65 TADCustomCommand.Unprepare Method

Releases SQL command resources.

Description

Use Unprepare call to release prepared SQL command resources. After call the State=csInactive. You can unprepare command by setting Prepared to False.

Syntax

```
procedure Unprepare;
```

See Also

Prepare (see page 317), Prepared (see page 304)

1.17.1.1.5 TADCustomConnection Class

The class is responsible to connection establishment with a DBMS.

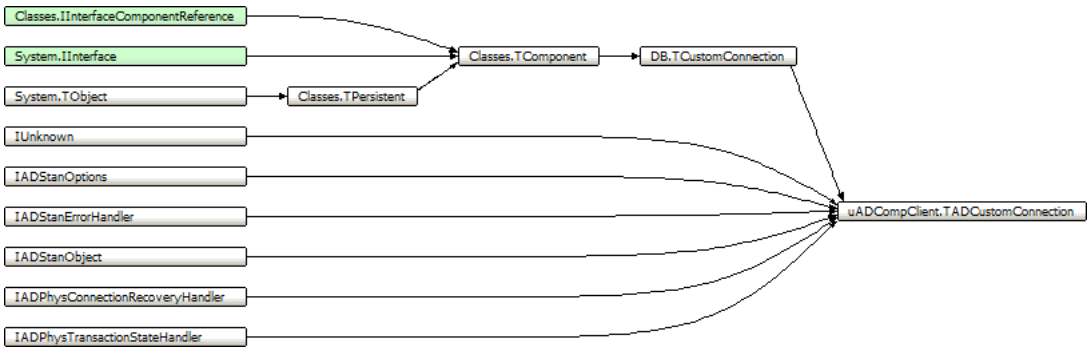
public

public	Description
AfterCommit (see page 321)	Fires after transaction is committed.
AfterRollback (see page 321)	Fires after transaction is rolled back.
Connected (see page 321)	Gets / sets connection active status.
AfterStartTransaction (see page 322)	Fires after transaction is started.
BeforeCommit (see page 322)	Fires before transaction will be committed.
BeforeRollback (see page 322)	Fires before transaction will be rolled back.
BeforeStartTransaction (see page 322)	Fires before transaction will be started.
R CliHandle (see page 323)	CLI handle for connection sharing.
R CliObj (see page 323)	Returns a CLI connection wrapping object.
R CommandCount (see page 324)	The number of assocciated TADCustomCommand (see page 291) objects.
R Commands (see page 324)	Returns assocciated TADCustomCommand (see page 291) objects.
ConnectedStoredUsage (see page 325)	Controls how to use Connected (see page 321) property value saved to DFM.
ConnectionDefName (see page 325)	The name of connection definition to use.
R ConnectionIntf (see page 326)	Returns the IADPhysConnection interface.
R ConnectionMetaDataIntf (see page 326)	Returns an IADPhysConnectionMetadata interface.
ConnectionName (see page 326)	Specifies the name of the connection to associate with this connection object.
R DataSets (see page 327)	Returns assocciated TADDataSet (see page 584) objects.
DriverName (see page 327)	Specifies the ID of the FireDAC driver for the connection.
FetchOptions (see page 328)	The set of default options to control rows fetching.
FormatOptions (see page 328)	The set of default options to control data representation.
R InTransaction (see page 328)	Indicates whether a connection transaction is in progress or not.
R IsSQLBased (see page 329)	Property returns True, if a data source supports SQL language.
R LastUsed (see page 329)	The date and time of last usage of the connection.
LoginDialog (see page 329)	Gets / sets the reference to login dialog interface.

LoginPrompt (see page 329)	Specifies whether a login dialog appears immediately before opening a new connection.
R Messages (see page 330)	Returns a list of messages returned by DBMS after recent database operation.
Offlined (see page 331)	Gets / sets connection offline status.
OnError (see page 332)	Fires before FireDAC will raise exception.
OnLogin (see page 332)	Fires when an application connects to a DBMS.
OnLosted (see page 333)	Fires after FireDAC was not able to restore losted connection to DBMS.
OnRecover (see page 333)	Fires after FireDAC discovered that connection to DBMS is losted.
OnRestored (see page 334)	Fires after FireDAC successfully restored losted connection to DBMS.
OptionsIntf (see page 334)	The reference to IADStanOptions interface.
Params (see page 334)	Lists database connection parameters for the FireDAC connection definition associated with the connection object.
R RDBMSKind (see page 335)	Returns the brand of DBMS currently connected.
R RefCount (see page 335)	Returns the number of references to this connection object.
ResourceOptions (see page 335)	The set of default options to control resources usage.
R ResultConnectionDef (see page 336)	The actual connection definition used to establish connection to a DBMS.
SharedCliHandle (see page 336)	CLI handle shared with other connection.
Temporary (see page 337)	Shows whether a connection object is created on fly (temporary one) and managed automatically or a persistent one explicitly created, managed, and freed by the application.
Transaction (see page 337)	The reference to TADCustomTransaction (see page 413) object.
TxOptions (see page 337)	The set of options to control transactions.
UpdateOptions (see page 338)	The set of default options to control data updating.
UpdateTransaction (see page 338)	The reference to TADCustomTransaction (see page 413) object, which will be used to post changes to DB.
AbortJob (see page 338)	Use the AbortJob method to abort all running dataset operations on this connection.
ApplyUpdates (see page 339)	Applies change logs of specified datasets to the DBMS.
V Assign (see page 339)	Assigns all properties of one connection to this connection.
CheckActive (see page 340)	Checks to see if the connection is active.
V CheckConnectionDef (see page 340)	Checks that connection definition is filled correctly.
CheckInactive (see page 340)	Checks to see if the connection is inactive.
CheckOnline (see page 340)	Checks to see if the connection is in online mode.
Commit (see page 340)	Permanently stores modifications to the data made in the current transaction, and optionally ends the current transactions.
CommitRetaining (see page 341)	Permanently stores modifications to the data made in the current transaction, without ending the current transaction.
DecodeObjectName (see page 342)	Splits DB object name into parts.
EncodeObjectName (see page 343)	Assembles DB object name from it parts.
ExecSQL (see page 343)	Executes a SQL command without parameters.
ExecSQL (see page 343)	Executes SQL command with parameters.
ExecSQL (see page 344)	Executes a SQL command with parameters.
ExecSQLScalar (see page 344)	Executes a SQL command without parameters and fetches a single value.
ExecSQLScalar (see page 345)	Executes a SQL command with parameters and fetches a single value.
ExecSQLScalar (see page 345)	Executes a SQL command with parameters and fetches a single value.
GetCatalogNames (see page 346)	Populates a string list with the names of catalogs in a DB.
GetFieldNames (see page 346)	Populates a string list with the names of fields in a table.
GetGeneratorNames (see page 347)	Populates a string list with the names of generators / sequences / <how else> in a database.
GetInfoReport (see page 347)	Populates a string list with a detailed report about a connection state.

GetKeyFieldNames (see page 349)	Populates a string list with the names of key fields in a table.
GetLastAutoGenValue (see page 349)	Returns last auto generated value.
GetPackageNames (see page 350)	Populates a string list with the names of the packages in a DB.
GetSchemaNames (see page 350)	Populates a string list with the names of schemas in a DB.
GetStoredProcNames (see page 351)	Populates a string list with the names of stored procedures in a DB.
GetTableNames (see page 351)	Populates a string list with the names of tables in a DB.
Offline (see page 352)	Sets connection to offline mode.
Online (see page 353)	Sets connection to online mode.
Open (see page 353)	Open connection to the database, using specified connection string.
Open (see page 354)	Open connection to the database, using specified user name and password.
Open (see page 354)	Open connection to the database, using specified connection parameters, user name and password.
Ping (see page 354)	Pings the DBMS server.
RefreshMetadataCache (see page 355)	Invalidates meta data cache.
ReleaseClients (see page 356)	Performs resource releasing operation on all associated datasets and commands.
Rollback (see page 356)	Cancels all modifications to the data made in the current transaction and optionally ends transaction.
RollbackRetaining (see page 357)	Cancels all modifications to the data made in the current transaction, without ending the current transaction.
StartTransaction (see page 358)	Starts new transaction at database.

Class Hierarchy



File

uADCompClient

Description

Use TADCustomConnection to establish connection to a DBMS and manage associated datasets.

This class hides some of properties, allowing to control visibility of properties by descendant classes. End users should use TADConnection.

Syntax

```
TADCustomConnection = class(TCustomConnection, IUnknown, IADStanOptions,
IADStanErrorHandler, IADStanObject, IADPhysConnectionRecoveryHandler,
IADPhysTransactionStateHandler);
```

See Also

Working with Connections (see page 27), Managing Transactions (see page 41), TADConnection (see page 279), IADPhysConnection

1.17.1.1.5.1 public

1.17.1.1.5.1.1 TADCustomConnection.AfterCommit Event

Fires after transaction is committed.

Description

The AfterCommit event fires after the Commit method is executed and the transaction is committed. In case of a nested transaction in AfterCommit event handler the transaction is not yet finished, but nesting level is decremented.

Syntax

```
property AfterCommit: TNotifyEvent;
```

See Also

BeforeCommit (see page 322)

1.17.1.1.5.1.2 TADCustomConnection.AfterRollback Event

Fires after transaction is rolled back.

Description

The AfterRollback event fires after the Rollback method is executed and the transaction is rolled back. In case of a nested transaction in AfterRollback event handler the transaction is not yet rolled back in full, but nesting level is decremented.

Syntax

```
property AfterRollback: TNotifyEvent;
```

See Also

BeforeRollback (see page 322)

1.17.1.1.5.1.3 TADCustomConnection.Connected Property

Gets / sets connection active status.

Description

Set Connected to True to establish a connection to a DBMS. Set Connected to False to close a connection. The default value for Connected is False.

An application can check Connected to determine the current status of a connection. If Connected is True, the connection is active; if False, and the KeepConnection (see page 889) property is also False, then the connection is inactive.

To open connection, it must be setup in one of the following ways:

1. Set ConnectionDefName (see page 325) to a name of one of a existing connection definitions and optionally override connection definition parameters in Params (see page 334). Existing connection definition may be defined either in connection definitions INI file or created on fly, for example, using TADCustomManager.AddConnectionDef (see page 376);
2. Set DriverName (see page 327) to a name of one of a registered drivers (or add DriverID into Params (see page 334)) and specify connection definition parameters in Params (see page 334).

When the connection definition is pooled (Pooled=True), then use (1) and does not override any of parameters, otherwise you will get an exception.

Setting Connected to False will try to connect to a DB. If somehow the connecting fails, then Connected will remain False and an an exception will be raised. If connecting failed due to a DB unavailability, then application may use Automatic Connection Recovery (see page 333) feature to wait and try to establish the connection.

Setting Connected to False will disconnect (close and unprepare) all associated commands and datasets. If you just need to disconnect from DBMS, but does not close objects, then use Offlined (see page 331) property.

Syntax

```
property Connected;
```

See Also

TCustomConnection.BeforeConnect, TCustomConnection.AfterConnect, TCustomConnection.BeforeDisconnect, TCustomConnection.AfterDisconnect, ConnectionDefName (see page 325), DriverName (see page 327), Params (see page 334), Offlined (see page 331), OnRecover (see page 333)

Example 1

```
ADConnection1.ConnectionDefName := 'ASA_Demo';
ADConnection1.Connected := True;
```

Example 2

See FireDAC\Samples\Comp Layer\TADConnection (see page 279)\Pooling sample for details of how to work with pooled connections.

1.17.1.1.5.1.4 TADCustomConnection.AfterStartTransaction Event

Fires after transaction is started.

Description

The AfterStartTransaction event fires after the StartTransaction method is executed and the transaction is started. In case of a nested transaction in AfterStartTransaction event handler the transaction is not just started, but nesting level is incremented.

Syntax

```
property AfterStartTransaction: TNotifyEvent;
```

See Also

BeforeStartTransaction (see page 322)

1.17.1.1.5.1.5 TADCustomConnection.BeforeCommit Event

Fires before transaction will be committed.

Description

The BeforeCommit event fires before the Commit method will be executed and the transaction will be committed.

Syntax

```
property BeforeCommit: TNotifyEvent;
```

See Also

AfterCommit (see page 321)

1.17.1.1.5.1.6 TADCustomConnection.BeforeRollback Event

Fires before transaction will be rolled back.

Description

The BeforeRollback event fires before the Rollback method will be executed and the transaction will be rolled back.

Syntax

```
property BeforeRollback: TNotifyEvent;
```

See Also

AfterRollback (see page 321)

1.17.1.1.5.1.7 TADCustomConnection.BeforeStartTransaction Event

Fires before transaction will be started.

Description

The BeforeStartTransaction event fires before the StartTransaction method will be executed and the transaction will be started.

Syntax

```
property BeforeStartTransaction: TNotifyEvent;
```

See Also

AfterStartTransaction (see page 322)

1.17.1.1.5.1.8 TADCustomConnection.CliHandle Property

CLI handle for connection sharing.

Description

The CliHandle property returns DBMS Call Level Interface connection handle. Then this value may be assigned to another TADCustomConnection object SharedCliHandle (see page 336) property. This is useful to transfer a connection from an application into a DLL. See DLL Development (see page 49).

After setting Connected (see page 321)=True for the DLL connection, both connections will share the same physical DBMS connection. This connection must be closed after all other connections, sharing the same CLI handle.

Syntax

```
property CliHandle: Pointer;
```

See Also

DLL Development (see page 49), SharedCliHandle (see page 336), Connected (see page 321)

Example 1

See demo application FireDAC\Samples\Comp Layer\TADConnection (see page 279)\DLL_Sharing.

Example 2

Application code (without checks):

```
FhDll := LoadLibrary(PChar('Project2.dll'));
@FpShowData := GetProcAddress(FhDll, PChar('ShowData'));
FpShowData(ADConnection1.CliHandle);
```

DLL code:

```
procedure ShowData(ACliHandle: LongWord);
begin
  ADConnection1.SharedCliHandle := ACliHandle;
  ADConnection1.Connected := True;
  ADQuery1.Active := True;
end;
```

1.17.1.1.5.1.9 TADCustomConnection.CliObj Property

Returns a CLI connection wrapping object.

Description

The CliObj property returns a DBMS Call Level Interface connection wrapping object. The class of the object depends on FireDAC driver. Following table lists drivers, connection wrapping classes and units. The wrapping object may be useful for performing low-level operations with the DBMS Call Level Interface.

Driver	Wrapping class and unit
Advantage	TADSConnection, uADPhysADSWrapper.pas
dbExpress (v <= 3)	ISQLConnection
Interbase, Firebird	TIBDatabase, uADPhysIBWrapper.pas

MySQL	TMySQLSession, uADPhysMySQLWrapper.pas
Microsoft SQL Server Microsoft Access IBM DB2 Sybase SQL Anywhere ODBC	TODBCConnection, uADPhysODBCWrapper.pas
Oracle	TOCIService, uADPhysOracleWrapper.pas
PostgreSQL	TPgConnection, uADPhysPgWrapper.pas
SQLite	TSQLiteDatabase, uADPhysSQLiteWrapper.pas
TDBX (v >= 4) DataSnap	TDBXConnection

Syntax

```
property CliObj: Pointer;
```

Example

```
uses
  uADPhysODBCWrapper;
....
  // returns ODBC driver DLL name for an ODBC based connection
  ShowMessage(TODBCConnection(ADConnection1.CliObj).DRIVER_NAME);
```

1.17.1.1.5.1.10 TADCustomConnection.CommandCount Property

The number of assocciated TADCustomCommand (see page 291) objects.

Description

The CommandCount property returns the number of prepared (State <> csInactive) TADCustomCommand objects, assocciated with this connection object.

Syntax

```
property CommandCount: Integer;
```

See Also

Commands (see page 324), TADCustomCommand (see page 291)

Example

```
var
  i: Integer;
...
  for i := ADConnection1.CommandCount - 1 downto 0 do
    ADConnection1.Commands[i].Disconnect(True);
```

1.17.1.1.5.1.11 TADCustomConnection.Commands Property

Returns assocciated TADCustomCommand (see page 291) objects.

Description

The Commands indexed property returns a prepared (State <> csInactive) TADCustomCommand object, assocciated with this connection object, by it index.

Syntax

```
property Commands [AIndex: Integer]: TADCustomCommand;
```

See Also

CommandCount (see page 324), TADCustomCommand (see page 291)

Example

```
var
```

```
  i: Integer;
...
  for i := ADConnection1.CommandCount - 1 downto 0 do
    ADConnection1.Commands[i].Disconnect(True);
```

1.17.1.1.5.1.12 TADCustomConnection.ConnectedStoredUsage Property

Controls how to use Connected (see page 321) property value saved to DFM.

Description

The ConnectedStoredUsage property controls how to use Connected (see page 321) property value saved to DFM. Include:

- auDesignTime, to use property value at design time;
- auRunTime, to use property value at run time.

Syntax

```
property ConnectedStoredUsage: TADStoredActivationUsage;
```

See Also

Connected (see page 321)

1.17.1.1.5.1.13 TADCustomConnection.ConnectionDefName Property

The name of connection definition to use.

Description

The ConnectionDefName property specifies the name of one of a existing connection definitions. It must be stored in ADConnectionDefs.ini or created dynamically.

Note, attempting to set ConnectionDefName when the Connected property is true raises an exception.

Syntax

```
property ConnectionDefName: String;
```

See Also

Defining Connection (see page 27), TADCustomManager (see page 365)

Example 1

```
ADConnection1.ConnectionDefName := 'ASA_Demo';
ADConnection1.Connected := True;
```

Example 2

See demo FireDAC\Samples\Comp Layer\TADConnection (see page 279)\ConnectionDefs for details how to create and use connection definitions.

Example 3

Specifying pooled connection:

```
var
  oList: TStringList;
  ......
  oList := TStringList.Create;
  oList.Add('Server=MyServer');
  oList.Add('Database=Northwind');
  oList.Add('User_Name=sa');
  oList.Add('Password=');
  oList.Add('Pooled=True');
  ADManager.AddConnectionDef('myconn', 'MSSQL', oList);
  ......
  ADConnection1.ConnectionDefName := 'myconn';
  ADConnection1.Connected := True;
```

1.17.1.1.5.1.14 TADCustomConnection.ConnectionIntf Property

Returns the IADPhysConnection interface.

Description

The ConnectionIntf property returns a reference to the IADPhysConnection interface. It is not nil, if a connection is established. Otherwise it is nil. It is not recommended to mix usage of TADCustomConnection API and TADCustomConnection.ConnectionIntf API.

Syntax

```
property ConnectionIntf: IADPhysConnection;
```

See Also

IADPhysConnection

1.17.1.1.5.1.15 TADCustomConnection.ConnectionMetaDataIntf Property

Returns an IADPhysConnectionMetadata interface.

Description

The ConnectionMetaDataIntf property returns a reference to the IADPhysConnectionMetadata interface. The IADPhysConnectionMetadata interface allows to get knowledge about a DBMS behavior.

Syntax

```
property ConnectionMetaDataIntf: IADPhysConnectionMetadata;
```

See Also

Connected (see page 321), ConnectionIntf (see page 326), IADPhysConnectionMetadata

Example

```
procedure TMainForm.Button2Click(Sender: TObject);
var
  oMetaIntf: IADPhysConnectionMetadata;
begin
  // Get client and server versions
  oMetaIntf := ADConnection1.ConnectionMetaDataIntf;
  try
    ShowMessage(Format('Client version: %.10d; Server version: %.10d',
      [oMetaIntf.ClientVersion, oMetaIntf.ServerVersion]));
  finally
    oMetaIntf:= nil; // always release the Interface before you close the connection
  end;
end;
```

1.17.1.1.5.1.16 TADCustomConnection.ConnectionName Property

Specifies the name of the connection to associate with this connection object.

Description

Use ConnectionName to specify the name of the connection to use with a connection object. If ConnectionName is the same as an existing connection definition, then the ConnectionDefName (see page 325) and DriverName (see page 327) properties need not be set. If ConnectionName does not match an existing connection definition, then either the application must also supply a valid connection definition in the ConnectionDefName (see page 325) property in addition to the or it must supply the DriverName (see page 327) and Params (see page 334) properties.

You can specifiy as ConnectionName for TADCustomCommand (see page 291) or TADRDBMSDataSet (see page 501) component and its descendants (TADQuery (see page 476), TADStoredProc (see page 514), TADTable (see page 539)), the ConnectionName of one of the existing TADConnection (see page 279) objects or the name of one of the existing connection definitions.

Notes

1. Attempting to set ConnectionName when the Connected (see page 321) property is true raises an exception.

2. At design time double-click a TConnection component to invoke the Database editor and set the ConnectionName.

Syntax

```
property ConnectionName: String;
```

See Also

ConnectionDefName (see page 325), DriverName (see page 327), Params (see page 334), TADCustomCommand.ConnectionName (see page 300), TADRDBMSDataSet.ConnectionName (see page 503)

Example

```
ADConnection1.ConnectionName := 'TestConn';
ADConnection1.DriverName := 'Oracl';'
ADConnection1.Params.Add('Database=testdb');
....
ADQuery1.ConnectionName := 'TestConn';
ADQuery1.Open('select * from test_tab');
```

1.17.1.1.5.1.17 TADCustomConnection.DataSets Property

Returns assocciated TADDataSet (see page 584) objects.

Description

The DataSets indexed property returns a prepared (State <> csInactive) TADRdbmsDataSet (see page 501) objects, assocciated with this connection object, by it index.

Syntax

```
property DataSets [AIndex: Integer]: TADDataSet;
```

See Also

DataSetCount, TADRdbmsDataSet (see page 501)

Example

```
var
  i: Integer;
...
  for i := ADConnection1.DataSetCount - 1 downto 0 do
    ADConnection1.DataSets[i].Disconnect(True);
```

1.17.1.1.5.1.18 TADCustomConnection.DriverName Property

Specifies the ID of the FireDAC driver for the connection.

Description

Use DriverName to specify the ID of the FireDAC driver to use for connection that do not specify an ConnectionDefName (see page 325). DriverName must be a valid FireDAC registered driver ID.

If an application sets DriverName, it must also specify connection parameters in the Params (see page 334) property. Ordinarily the alias specified in the ConnectionDefName (see page 325) property supplies connection parameters, but when DriverName is set, the ConnectionDefName (see page 325) property is automatically cleared to avoid FireDAC driver ID contention.

Attempting to set DriverName when the Connected (see page 321) property is True raises an exception.

Setting DriverName value, will add DriverID=<xxx> parameter to the Params list.

Notes

At design time double-click a TConnection component to invoke the Connection editor and set the DriverName.

Or choose DriverName value in property inspector from drop-down list.

Syntax

```
property DriverName: string;
```

See Also

Params (see page 334), ConnectionDefName (see page 325)

Example

```
ADConnection1.DriverName := 'Oracl';'
ADConnection1.Params.Add('Database=testdb');
....
ADQuery1.Connection := ADConnection1;
ADQuery1.Open('select * from test_tab');
```

1.17.1.1.5.1.19 TADCustomConnection.FetchOptions Property

The set of default options to control rows fetching.

Description

The FetchOptions property is the set of default properties, controlling the fetching of cursor rows. These properties will be inherited by commands and datasets, associated with this connection. Also these properties will inherit its values from FetchOptions of ADManager (see page 570) object.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), TADFetchOptions (see page 849), ADManager (see page 570)

1.17.1.1.5.1.20 TADCustomConnection.FormatOptions Property

The set of default options to control data representation.

Description

The FormatOptions property is the set of the default properties, controlling the data type mapping, and some other aspects of data handling. These properties will be inherited by commands and datasets, accossiated with this connection. Also these properties will inherit its values from FormatOptions of ADManager (see page 570) object.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), TADFormatOptions (see page 860), ADManager (see page 570)

1.17.1.1.5.1.21 TADCustomConnection.InTransaction Property

Indicates whether a connection transaction is in progress or not.

Description

Examine InTransaction at run-time to determine if a connection transaction is currently in progress. InTransaction is True if a transaction is in progress, False otherwise.

The value of InTransaction cannot be changed directly. Calling StartTransaction (see page 358) sets InTransaction to True. Calling Commit or Rollback sets InTransaction to False.

InTransaction is the shortcut for ConnectionIntf.Transaction.Active or Transaction.Active.

Syntax

```
property InTransaction: Boolean;
```

See Also

StartTransaction (see page 358), Commit (see page 340), Rollback (see page 356), TADTransaction (see page 561)

Example

See FireDAC\Samples\Comp Layer\TADConnection (see page 279)\Transactions sample for details.

1.17.1.1.5.1.22 TADCustomConnection.IsSQLBased Property

Property returns True, if a data source supports SQL language.

Description

Property returns True, if a data source supports SQL language. Actually, the property were introduced for BDE compatibility. All FireDAC drivers support SQL language, so IsSQLBased is always True.

Syntax

```
property IsSQLBased: Boolean;
```

1.17.1.1.5.1.23 TADCustomConnection.LastUsed Property

The date and time of last usage of the connection.

Description

The LastUsed property returns the date and time of the last activity on this connection.

Syntax

```
property LastUsed: TDateTime;
```

1.17.1.1.5.1.24 TADCustomConnection.LoginDialog Property

Gets / sets the reference to login dialog interface.

Description

The LoginDialog property allows to associate the login dialog TADGUIxLoginDialog (see page 675) with this connection. The dialog instance assigned to the LoginDialog will be used only by this connection object.

If the LoginDialog property is not assigned, but the TADGUIxLoginDialog is in your application or one of the units uADGUIxXxxxLogin is linked into application, then the connection object will use the global login dialog instance.

The login dialog will popup at connecting to a DBMS only if all of the following conditions are met:

- LoginPrompt=True;
- connection definition is not pooled (Pooled=False);
- global silent flag is not set to True (ADGUIxSilent() = False);
- connection object does not use shared CLI handle (SharedCliHandle (see page 336) = nil).

Syntax

```
property LoginDialog: TADGUIxLoginDialog;
```

See Also

Establishing Connection (see page 37), TADGUIxLoginDialog (see page 675), LoginPrompt (see page 329), SharedCliHandle (see page 336), ADGUIxSilent

Example

Using private login dialog:

```
ADGUIxLoginDialog1.Caption := 'Welcome to the Northwind DB';
ADGUIxLoginDialog1.VisibleItems.Add('User_name');
ADGUIxLoginDialog1.VisibleItems.Add('Password');
ADConnection1.LoginDialog := ADGUIxLoginDialog1;
ADConnection1.LoginPrompt := True;
ADConnection1.Connected := True;
```

1.17.1.1.5.1.25 TADCustomConnection.LoginPrompt Property

Specifies whether a login dialog appears immediately before opening a new connection.

Description

Set LoginPrompt to True to provide login support when establishing a connection. When LoginPrompt is True, a dialog appears to prompt users for a name, password and other optional parameters. The dialog appears after the BeforeConnect event and before the AfterConnect event, unless you supply an OnLogin (see page 332) event handler. If there is an OnLogin (see page 332) event handler, that event occurs in place of the login dialog. If correct values for the user name, password and other optional parameters are not supplied in the dialog or by the OnLogin (see page 332) event handler, the connection fails. This OnLogin (see page 332) event does not fire unless LoginPrompt is set to True.

When LoginPrompt is False, the application must supply user name, password and other optional parameter values programmatically. For that application can specify User_name, Password, etc parameters in Params.

The login dialog will popup at connecting to a DBMS only if all of the following conditions are met:

- LoginPrompt=True;
- connection definition is not pooled (Pooled=False);
- global silent flag is not set to True (ADGUIxSilent() = False);
- connection object does not use shared CLI handle (SharedCliHandle (see page 336) = nil).

Syntax

```
property LoginPrompt;
```

See Also

Establishing Connection (see page 37), OnLogin (see page 332), LoginDialog (see page 329), Params (see page 334), SharedCliHandle (see page 336), ADGUIxSilent, AfterConnect

Example 1

Using login dialog:

```
ADGUIxLoginDialog1.Caption := 'Welcome to the Northwind DB';
ADGUIxLoginDialog1.VisibleItems.Add('User_name');
ADGUIxLoginDialog1.VisibleItems.Add('Password');
ADConnection1.LoginDialog := ADGUIxLoginDialog1;
ADConnection1.LoginPrompt := True;
ADConnection1.Connected := True;
```

Example 2

Connecting without login dialog:

```
procedure TForm1.ADConnection1Login(AConnection: TADCustomConnection; const AConnectionDef:
IADStanConnectionDef);
begin
  AConnectionDef.UserName := 'dba';
  AConnectionDef.Password := 'sql';
end;

procedure TForm1.Button1Click(ASender: TObject);
begin
  ADConnection1.OnLogin := ADConnection1Login;
  ADConnection1.LoginPrompt := True;
  ADConnection1.Connected := True;
end;
```

1.17.1.1.5.1.26 TADCustomConnection.Messages Property

Returns a list of messages returned by DBMS after recent database operation.

Description

Read the Messages property to get a list of the messages, hints or warnings, returned by a DBMS after a recent DBMS call. The property is of type EADDBEngineException (see page 833). The content definition depends on a DBMS kind.

DBMS	Description
Advantage Database Server	None.

Microsoft Access Database	None.
DataSnap server	None.
IBM DB2	None.
Firebird / Interbase	None.
MySQL Server	Output of SHOW WARNINGS command.
Oracle Server	Output produced by DBMS_OUTPUT package.
PostgreSQL	Notice and warning messages.
SQLite	None.
Sybase SQL Anywhere	• ODBC warnings; • Output of MESSAGE command.
Microsoft SQL Server	• ODBC warnings; • Messages from RAISERROR with severity <= 16. For example output of EXEC sp_configure call.

Not all supported DBMS's are returning non-error messages automatically. For example, for Oracle and MySQL FireDAC need to call additional DBMS API to bring the messages to a client. That requires additional server round trips. But for SQL Server and Sybase SQL Anywhere the messages are returned always automatically. Setting ResourceOptions (see page 335).ServerOutput (see page 890) to True guarantees, that for any DBMS the messages will be returned to client automatically.

The messages will be stored in the Messages list until next DBMS call (Open, ExecSQL, StartTransaction, etc).

Syntax

```
property Messages: EADDBEngineException;
```

See Also

Executing Command (see page 66), EADDBEngineException (see page 833)

Example

```
var
  i: Integer;
begin
  ADConnection1.ResourceOptions.ServerOutput := True;
  ADQuery1.ExecSQL;
  for i := 0 to ADConnection1.ConnectionIntf.Messages.ErrorCount - 1 do
    Memo1.Lines.Add(ADConnection1.ConnectionIntf.Messages[i].Message);
end;
```

1.17.1.1.5.1.27 TADCustomConnection.Offlined Property

Gets / sets connection offline status.

Description

Set Offlined to True to switch connection and associated with it commands and datasets to the offline mode. In offline mode connection to DBMS is closed, but datasets remain open.

If dataset, associated with this connection object is active and not all records are fetched from DBMS cursor, then depending on FetchOptions (see page 328).AutoFetchAll (see page 852) value the following actions will be performed:

- afAll - the dataset will fetch all records from cursor and will go to offline mode. This is the default mode.
- afTruncate - the dataset will truncate result set and will go to offline mode.
- afDisable - exception will be raised.

If application will try to perform on a command or a dataset objects any operation requiring DBMS connection, then if ResourceOptions (see page 335).AutoConnect (see page 887) =True, the connection will automatically establish connection to the DBMS. This is the default mode. If ResourceOptions (see page 335).AutoConnect (see page 887) =

False, then connection object will raise an exception.

Set Offlined to False to establish connection with DBMS and switch to online mode.

Syntax

```
property Offlined: Boolean;
```

See Also

Offlining Connection (see page 40), TADFetchOptions.AutoFetchAll (see page 852), TADTopResourceOptions.AutoConnect (see page 887), Connected (see page 321), Offline (see page 352), Online (see page 353)

1.17.1.1.5.1.28 TADCustomConnection.OnError Event

Fires before FireDAC will raise exception.

Parameters

Parameters	Description
ASender	This connection object.
AInitiator	The interface of the object initially discovered error.
AException	The exception object to be raised.

Description

The OnError event handler fires before FireDAC will raise exception inside of call of one of the connection or assocciated commands or datasets methods. In most cases AException will be of EADDBEngineException (see page 833) class.

The event handler can replace AException object with some other custom exception. In this case original object must be freed.

Syntax

```
property OnError: TADErrorEvent;
```

See Also

Handling Errors (see page 44), EADDBEngineException (see page 833)

Example

```
procedure TForm1.ADConnection1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if (AException is EADDBEngineException) and (EADDBEngineException(AException).Kind =
ekUKViolated) then begin
    AException.Free;
    AException := Exception.Create('You must enter unique document number !');
  end;
end;
```

1.17.1.1.5.1.29 TADCustomConnection.OnLogin Event

Fires when an application connects to a DBMS.

Description

Write an OnLogin event handler to take specific actions when an application attempts to connect to a DBMS. When the LoginPrompt (see page 329) property is True, a database login is required. The login dialog and OnLogin event only occur when the LoginPrompt (see page 329)property is true.

If there is no OnLogin event handler, the current User_Name is read from the Params (see page 334) property or connection definition, and a standard Login dialog box opens. The dialog prompts for a user name, password and other optional parameters, and then uses the values entered by the user to set the User_Name, Password and other optional values in the Params property. These values are then passed to the DBMS.

Applications that provide alternative OnLogin event handlers may set the User_Name, Password and other optional values in AConnectionDef. AConnectionDef is a temporary connection definition and is freed automatically when no longer needed.

Syntax

```
property OnLogin: TADConnectionLoginEvent;
```

See Also

Establishing Connection (see page 37), LoginPrompt (see page 329), Connected (see page 321), IADStanConnectionDef

Example

Connecting without login dialog:

```
procedure TForm1.ADConnection1Login(AConnection: TADCustomConnection; const AConnectionDef:
IADStanConnectionDef);
begin
  AConnectionDef.UserName := 'dba';
  AConnectionDef.Password := InputBox('Login', 'Please enter DBA password', 'sql');
end;

procedure TForm1.Button1Click(ASender: TObject);
begin
  ADConnection1.OnLogin := ADConnection1Login;
  ADConnection1.LoginPrompt := True;
  ADConnection1.Connected := True;
end;
```

1.17.1.1.5.1.30 TADCustomConnection.OnLosted Event

Fires after FireDAC was not able to restore losted connection to DBMS.

Description

The OnLosted event fires after FireDAC was not able to automatically restore connection to the DBMS, after discovering that it is losted.

Syntax

```
property OnLosted: TNotifyEvent;
```

See Also

Recovering Connection (see page 38), OnRecover (see page 333), OnRestored (see page 334)

1.17.1.1.5.1.31 TADCustomConnection.OnRecover Event

Fires after FireDAC discovered that connection to DBMS is losted.

Parameters

Parameters	Description
ASender	This connection object.
AInitiator	The interface of the object initially discovered error.
AException	The exception object describing losted connection issue.
AAction	The action FireDAC should take after return from event handler.

Description

The OnRecover event fires after FireDAC discovered that connection to DBMs is losted. The event handler can analyze the reason, ask user how to proceed, show some status and returns the action, FireDAC should take:

Action	Description
faDefault	If ResourceOptions (see page 335).AutoReconnect (see page 887) =True, then FireDAC will try to reconnect, otherwise will raise an exception, that connection is losted. By default ResourceOptions (see page 335).AutoReconnect (see page 887) = False.
faFail	Close connection and raise an exception, that connection is losted.
faRetry	Try to reconnect.

faCloseAbort	Close connection and abort current operation.
faOfflineAbort	Switch connection to the offline mode and abort current operation.

If event handler is not assigned, then FireDAC will take default (faDefault) action. If after 3 trials connection cannot be restored, then FireDAC will close connection and raise an exception, that connection is losted.

Syntax

```
property OnRecover: TADConnectionRecoverEvent;
```

See Also

Recovering Connection (see page 38), OnLosted (see page 333), OnRestored (see page 334), TADTopResourceOptions.AutoReconnect (see page 887), Connected (see page 321), Offlined (see page 331)

1.17.1.1.5.1.32 TADCustomConnection.OnRestored Event

Fires after FireDAC successfully restored losted connection to DBMS.

Description

The OnRestored event fires after FireDAC successfully restored connection to the DBMS, after discovering that it is losted.

Syntax

```
property OnRestored: TNotifyEvent;
```

See Also

Recovering Connection (see page 38), OnLosted (see page 333), OnRecover (see page 333)

1.17.1.1.5.1.33 TADCustomConnection.OptionsIntf Property

The reference to IADStanOptions interface.

Description

The OptionsIntf returns reference to the IADStanOptions interface, implemented by this connection.

Syntax

```
property OptionsIntf: IADStanOptions;
```

See Also

IADStanOptions

1.17.1.1.5.1.34 TADCustomConnection.Params Property

Lists database connection parameters for the FireDAC connection definition associated with the connection object.

Description

Use Params to list DBMS connection definition parameters, such server name, port, database, client character set, user name and password. The possible set of parameters depends on the DBMS kind to connect to.

Params is a list of string items, each representing a different DBMS connection definition parameter. If the ConnectionDefName (see page 325) property specifies a valid FireDAC connection definition, then parameters, specified in Params, will override corresponsding connection definition parameters. If an application need to specify connection definition parameter on-fly, then an application must fill-in all required connection definition parameter into Params.

At design time double-click a TConnection component to invoke the Connection editor and set Params.

Syntax

```
property Params: TStrings;
```

See Also

Defining Connection (see page 27), ConnectionDefName (see page 325), DriverName (see page 327), OnLogin (

see page 332)

Example

```
with ADConnection1.Params do begin
  Clear;
  Add('DriverID=Ora');
  Add('Database=ORA_920_APP');
  Add('User_Name=ADDemo');
  Add('Password=a');
end;
ADConnection1.Connected := True;
```

1.17.1.1.5.1.35 TADCustomConnection.RDBMSKind Property

Returns the brand of DBMS currently connected.

Description

The RDBMSKind property returns the brand of the DBMS, to which connection object is currently connected. If connection is not active, then property returns mkUnknown.

Syntax

```
property RDBMSKind: TADRDBMSKind;
```

See Also

TADRDBMSKind, Connected (see page 321)

Example

```
case ADConnection1.RDBMSKind of
mkOracle:    s := 'select mySeq.NextVal from dual';
mkInterbase: s := 'select gen_id(myGen, 1) from rdb$database';
else         Exit;
end;
ADQuery1.Open(s);
```

1.17.1.1.5.1.36 TADCustomConnection.RefCount Property

Returns the number of references to this connection object.

Description

Read the RefCount property to see how many command, dataset and transaction objects are referencing to this connection object. Really the property value is not the exact count of objects, because some of objects may reference twice or more to the same connection object.

When no more objects are referencing to this connection object, then RefCount is equal to 0.

Syntax

```
property RefCount: Integer;
```

See Also

Temporary (see page 337), DataSets (see page 327), DataSetCount, Commands (see page 324), CommandCount (see page 324)

Example

```
if ADConnection1.RefCount = 0 then
  ADConnection1.Close;
```

1.17.1.1.5.1.37 TADCustomConnection.ResourceOptions Property

The set of default options to control resources usage.

Description

The ResourceOptions property is the set of the default properties, controlling the resources usage by the command. These properties will be inherited by commands and datasets, accossiated with this connection. Also these properties will inherit its values from ResourceOptions of ADManager (see page 570) object.

Syntax

```
property ResourceOptions: TADTopResourceOptions;
```

See Also

Setting Options (see page 34), TADTopResourceOptions (see page 885), ADManager (see page 570)

1.17.1.1.5.1.38 TADCustomConnection.ResultConnectionDef Property

The actual connection definition used to establish connection to a DBMS.

Description

The ResultConnectionDef property returns reference to the resulting connection definition used by connection object. It represents original connection definition specified by ConnectionDefName (see page 325) (if it is specified), overriden by Params (see page 334) connection parameters.

Syntax

```
property ResultConnectionDef: IADStanConnectionDef;
```

See Also

ConnectionDefName (see page 325), Params (see page 334), DriverName (see page 327), IADStanConnectionDef

1.17.1.1.5.1.39 TADCustomConnection.SharedCliHandle Property

CLI handle shared with other connection.

Description

Use SharedCliHandle, to set shared DBMS Call Level Interface connection handle, returned by other connection CliHandle property. This is useful to transfer connection from an application into a DLL. See DLL Development (see page 49). Note, a connection cannot be shared with other process, as the sharing works only inside the same address space.

After setting Connected=True for this connection, it will use the same physical DBMS connection / session as other connection. And will share the same transaction state.

After setting Connected=False, this DLL connection will not release connection resources to the DBMC client. The application connection is responsible for resources releasing. Because that the application connection must be closed after closing all of the sharing connections.

Syntax

```
property SharedCliHandle: Pointer;
```

See Also

DLL Development (see page 49), CliHandle (see page 323), Connected (see page 321)

Example 1

See demo application FireDAC\Samples\Comp Layer\TADConnection (see page 279)\DLL_Sharing.

Example 2

Application code (without checks):

```
FhDll := LoadLibrary(PChar('Project2.dll'));
@FpShowData := GetProcAddress(FhDll, PChar('ShowData'));
FpShowData(ADConnection1.CliHandle);
```

DLL code:

```
procedure ShowData(ACliHandle: LongWord);
begin
  ADConnection1.SharedCliHandle := ACliHandle;
  ADConnection1.Connected := True;
  ADQuery1.Active := True;
end;
```

1.17.1.1.5.1.40 TADCustomConnection.Temporary Property

Shows whether a connection object is created on fly (temporary one) and managed automatically or a persistent one explicitly created, managed, and freed by the application.

Description

Use Temporary property to determine if a connection component is created on fly (temporary one) and managed automatically - True, or a persistent one explicitly created, managed, and freed by the application - False.

A temporary connection object is created when a dataset / command has set ConnectionName (see page 326) to the name of the existing connection definition and needs real connection object to communicate with DBMS. That happens at Prepare / Open / Execute / ExecSQL / ExecProc calls.

A temporary connection object is freed when the dataset / command is unprepared. An application can keep temporary connection object for later reusing by setting Temporary to False. After that the application is then responsible for closing the connection when it is no longer needed.

Syntax

```
property Temporary: Boolean;
```

See Also

RefCount (see page 335)

Example

```
var
  oConn: TADconnection;
...
ADQuery1.ConnectionName := 'Access_Demo';
ADQuery1.Open;
...
oConn := ADManager.FindConnection('Access_Demo');
// here oConn.Temporary = True. The oConn will bedestroyed after disconnecting of ADQuery1
oConn.Temporary := False;
...
ADQuery1.Disconnect;
// here oConn is alive due to setting of oConn.Temporary to False
```

1.17.1.1.5.1.41 TADCustomConnection.Transaction Property

The reference to TADCustomTransaction (see page 413) object.

Description

The Transaction property get/set reference to the transaction object, which will be default one for the connection and all associated datasets and commands without explicitly assigned Transaction property. At moment, this behavior has the meaning only for Interbase / Firebird connection.

Syntax

```
property Transaction: TADCustomTransaction;
```

See Also

UpdateTransaction (see page 338), TADCustomTransaction (see page 413)

1.17.1.1.5.1.42 TADCustomConnection.TxOptions Property

The set of options to control transactions.

Description

The TxOptions property is the set of the properties, controlling the transactions in this connection.

You can change transaction properties while there is no active transactions. If you set Transaction (see page 337) property, then application should use Transaction.Options, instead of TxOptions.

Syntax

```
property TxOptions: TADTxOptions;
```

See Also

Transaction (see page 337), TADTxOptions (see page 891)

Example

See FireDAC\Samples\Comp Layer\TADConnection (see page 279)\Transactions sample for details.

1.17.1.1.5.1.43 TADCustomConnection.UpdateOptions Property

The set of default options to control data updating.

Description

The UpdateOptions property is the set of the default properties, controlling the data updating by the datasets. These properties will be inherited by datasets, associated with this connection. Also these properties will inherit its values from UpdateOptions of ADManager (see page 570) object.

Syntax

```
property UpdateOptions: TADUpdateOptions;
```

See Also

Setting Options (see page 34), TADUpdateOptions (see page 896), ADManager (see page 570)

1.17.1.1.5.1.44 TADCustomConnection.UpdateTransaction Property

The reference to TADCustomTransaction (see page 413) object, which will be used to post changes to DB.

Description

The UpdateTransaction property get/set reference to the transaction object, which will be default one for all associated datasets without explicitly assigned UpdateTransaction property. This transaction will be used by datasets to post changes to the DB.

If UpdateTransaction is not assigned, but Transaction (see page 337) is assigned, then Transaction (see page 337) object will be used as update transaction.

At moment, this behaviour has the meaning only for Interbase / Firebird connection.

Syntax

```
property UpdateTransaction: TADCustomTransaction;
```

See Also

Transaction (see page 337), TADCustomTransaction (see page 413)

1.17.1.1.5.1.45 TADCustomConnection.AbortJob Method

Use the AbortJob method to abort all running dataset operations on this connection.

Parameters

Parameters	Description
AWait: Boolean = False	If True, then caller will wait until the current operations will be terminated.

Description

The AbortJob method aborts all following dataset operations, running on this connection:

- Open (see page 509). The dataset is executing DBMS command to open cursor.
- ExecSQL (see page 397)/Execute (see page 622). The dataset is executing DBMS command.
- Fetch. The dataset is fetching rows from cursor.

Syntax

```
procedure AbortJob(AWait: Boolean = False);
```

See Also

TADAdaptedDataSet (see page 259).AbortJob (see page 263)

Example

```
ADConnection1.AbortJob(True);
ADConnection1.Connected := False;
```

1.17.1.1.5.1.46 TADCustomConnection.ApplyUpdates Method

Applies change logs of specified datasets to the DBMS.

Parameters

Parameters	Description
const ADataSets: array of TADDataSet	A open array, containing a list of datasets to post pending updates.

Description

Call ApplyUpdates to apply change logs for specified active datasets to the database. ApplyUpdates is only meaningful if the CachedUpdates (see page 592) property is True for specified datasets.

ADataSets is an open array of datasets. For each specified dataset this method calls the dataset's ApplyUpdates (see page 611) and CommitUpdates (see page 615) methods to apply that dataset's change log to database.

Note, this method is for compatibility with BDE components. To apply updates for few datasets consider to use Centralized Cached Updates (see page 114).

Syntax

```
procedure ApplyUpdates(const ADataSets: array of TADDataSet);
```

See Also

Caching Updates (see page 114), ApplyUpdates (see page 611), CommitUpdates (see page 615)

1.17.1.1.5.1.47 TADCustomConnection.Assign Method

Assigns all properties of one connection to this connection.

Parameters

Parameters	Description
Source: TPersistent	A connection to copy.

Description

Use Assign method to copy all properties and event handlers of one connection to this connection. Assign allows to create a duplicated connection.

Note, a pooled connection and copied connection are not the same. Pooled connections may be used to improve performance in multi-threaded environment.

Syntax

```
procedure Assign(Source: TPersistent); override;
```

See Also

Multi Threading (see page 46)

Example

```
oConn := TADConnection.Create(nil);
oConn.Assign(ADConnection1);
```

1.17.1.1.5.1.48 TADCustomConnection.CheckActive Method

Checks to see if the connection is active.

Description

Call CheckActive to assert that the connection to a DBMS is active. If the connection is inactive, then if ResourceOptions (see page 335).AutoConnect (see page 887)=True, then connection object will automatically connect to the DBMS, otherwise an exception is raised.

Syntax

```
procedure CheckActive;
```

See Also

TADTopResourceOptions.AutoConnect (see page 887), Connected (see page 321)

1.17.1.1.5.1.49 TADCustomConnection.CheckConnectionDef Method

Checks that connection definition is filled correctly.

Description

Checks that connection definition is filled correctly with minimal set of required parameters for current environment. The method is mostly usefull for tools or applications, creating the connection definitions of fly.

Syntax

```
procedure CheckConnectionDef; virtual;
```

1.17.1.1.5.1.50 TADCustomConnection.CheckInactive Method

Checks to see if the connection is inactive.

Description

Call CheckInactive to assert that the connection to a DBMS is inactive. If the connection is active, then if ResourceOptions (see page 335).AutoConnect (see page 887)=True, then connection object will automatically disconnect from the DBMS, otherwise an exception is raised.

Syntax

```
procedure CheckInactive;
```

See Also

TADTopResourceOptions.AutoConnect (see page 887), Connected (see page 321)

1.17.1.1.5.1.51 TADCustomConnection.CheckOnline Method

Checks to see if the connection is in online mode.

Description

Call CheckOnline to assert that the connection to a DBMS is in online mode. If the connection is offlined, then if ResourceOptions (see page 335).AutoConnect (see page 887)=True, then connection object will automatically go to the online mode, otherwise an exception is raised.

Syntax

```
procedure CheckOnline;
```

See Also

TADTopResourceOptions.AutoConnect (see page 887), Offlined (see page 331)

1.17.1.1.5.1.52 TADCustomConnection.Commit Method

Permanently stores modifications to the data made in the current transaction, and optionally ends the current transactions.

Description

Call Commit to permanently store modifications, like a INSERT's / UPDATE's / DELETE's, made in current transaction to the database.

FireDAC supports nested transactions, so the current transaction is the one started with most recent StartTransaction (see page 358) call. If the database does not support nested transactions, like most of DBMS's, then FireDAC will emulate nested transactions using savepoints.

Before calling Commit, an application may check the status of the InTransaction (see page 328) property. If an application calls Commit and there is no current transaction, an exception is raised.

A Commit call on Interbase / Firebird will close and unprepare all datasets and commands, associated with this transaction object. On some other DBMS's a call will invalidate all active result sets. For example, on MS SQL Server 2005. To force FetchAll (see page 624) on active datasets use ReleaseClients (see page 356) method.

The Commit call is the shortcut to Transaction (see page 337).Commit, if Transaction property is assigned. Otherwise Commit will operate on default connection transaction.

Syntax

```
procedure Commit;
```

See Also

Managing Transactions (see page 41), StartTransaction (see page 358), InTransaction (see page 328), Rollback (see page 356), CommitRetaining (see page 341), ReleaseClients (see page 356)

Example 1

```
procedure TForm1.DoThatButtonClick(Sender: TObject);
begin
  ADConnection1.StartTransaction;
  try
    if CustomerQuery.Locate('ID', [100]) then begin
      CustomerQuery.Edit;
      CustomerQuery.FieldByName('status').AsString := 'top';
      CustomerQuery.Post;
      ADConnection1.ExecSQL('delete from debt where CustID = 100');
      ADConnection1.Commit;
    end;
  except
    ADConnection1.Rollback;
    raise;
  end;
end;
```

Example 2

See FireDAC\Samples\Comp Layer\TADConnection (see page 279)\Transactions sample for details.

1.17.1.1.5.1.53 TADCustomConnection.CommitRetaining Method

Permanently stores modifications to the data made in the current transaction, without ending the current transaction.

Description

Call CommitRetaining to permanently store modifications, like a INSERT's / UPDATE's / DELETE's, made in current transaction to the database without finishing transaction.

FireDAC supports nested transactions, so the current transaction is the one started with most recent StartTransaction (see page 358) call. If the database does not support nested transactions, like most of DBMS's, then FireDAC will emulate nested transactions using savepoints. CommitRetaining is usefull only for main transaction, not for nested ones.

The CommitRetaining call is the shortcut to Transaction (see page 337).CommitRetaining, if Transaction property is assigned. Otherwise CommitRetaining will operate on default connection transaction.

Before calling CommitRetaining, an application may check the status of the InTransaction (see page 328) property. If an application calls CommitRetaining and there is no current transaction, an exception is raised.

CommitRetaining will use native functionality on IB/FB. On other DBMS's the method will be equal to calls of Commit (see page 340), then of StartTransaction (see page 358).

Syntax

```
procedure CommitRetaining;
```

See Also

Managing Transactions (see page 41), StartTransaction (see page 358), InTransaction (see page 328), Rollback (see page 356), Commit (see page 340)

Example

```
procedure TForm1.DoThatButtonClick(Sender: TObject);
begin
  ADConnection1.StartTransaction;
  try
    if CustomerQuery.Locate('ID', [100]) then begin
      CustomerQuery.Edit;
      CustomerQuery.FieldByName('status').AsString := 'top';
      CustomerQuery.Post;
      ADConnection1.ExecSQL('delete from debt where CustID = 100');
      ADConnection1.CommitRetaining;
    end;
  except
    ADConnection1.RollbackRetaining;
    raise;
  end;
end;
```

1.17.1.1.5.1.54 TADCustomConnection.DecodeObjectName Method

Splits DB object name into parts.

Parameters

Parameters	Description
const AFullName: String	A full DB object name to parse.
var ACatalogName: String	A catalog name specified in a object name.
var ASchemaName: String	A schema name specified in a object name.
var ABaseObjectName: String	A base object name specified in a object name, like a package name.
var AObjectName: String	A object name specified in a full object name.

Description

The DecodeObjectName method parses DB object name into parts, using the current DBMS rules.

Syntax

```
procedure DecodeObjectName(const AFullName: String; var ACatalogName: String; var
ASchemaName: String; var ABaseObjectName: String; var AObjectName: String);
```

See Also

EncodeObjectName (see page 343)

Example

On Oracle:

```
var
  sCatalog, sSchema, sBaseObj, sObj: String;
...
  ADConnection1.DecodeObjectName('ADDemo."MyPack".Proc1', sCatalog, sSchema, sBaseObj,
sObj);
  // sCatalog -> ''
  // sSchema -> ADDEMO
  // sBaseObj -> MyPack
  // sObj -> PROC1
```

1.17.1.1.5.1.55 TADCustomConnection.EncodeObjectName Method

Assembles DB object name from it parts.

Parameters

Parameters	Description
const ACatalogName: String	A catalog name part.
const ASchemaName: String	A schema name part.
const ABaseObjectName: String	A base object name part, like a package name.
const AObjectName: String	A object name.

Returns

A full object name, assembled from specified name parts.

Description

The EncodeObjectName method assembles DB object name from it parts, using the current DBMS rules.

Syntax

```
function EncodeObjectName(const ACatalogName: String; const ASchemaName: String; const
ABaseObjectName: String; const AObjectName: String): String;
```

See Also

DecodeObjectName (see page 342)

Example

On Oracle:

```
ShowMessage(ADConnection1.EncodeObjectName('', 'ADDEMO', 'MyPack', 'PROC1')); // ->
ADDEMO."MyPack".PROC1
```

1.17.1.1.5.1.56 TADCustomConnection.ExecSQL Method (String)

Executes a SQL command without parameters.

Parameters

Parameters	Description
const ASQL: String	A SQL command to execute.

Returns

The number of affected rows.

Description

The ExecSQL method executes a SQL command without parameters. All returned cursors will be discarded.

Syntax

```
function ExecSQL(const ASQL: String): LongInt; overload;
```

See Also

Executing Command (see page 66), TADQuery (see page 476)

Example

```
ADConnection1.ExecSQL('truncate table tab1');
```

1.17.1.1.5.1.57 TADCustomConnection.ExecSQL Method (String, array of Variant)

Executes SQL command with parameters.

Parameters

Parameters	Description
const ASQL: String	A SQL command to execute.

const AParams: array of Variant	An open array of parameter values.

Returns

The number of affected rows.

Description

The ExecSQL method executes a SQL command with parameters. All returned cursors will be discarded.

Syntax

```
function ExecSQL(const ASQL: String; const AParams: array of Variant): LongInt; overload;
```

See Also

Executing Command (see page 66), TADQuery (see page 476)

Example

```
ADConnection1.ExecSQL('delete from mytab where id > :p1', [1000]);
```

1.17.1.1.5.1.58 TADCustomConnection.ExecSQL Method (String, array of Variant, array of TFieldType)

Executes a SQL command with parameters.

Parameters

Parameters	Description
const ASQL: String	A SQL command to execute.
const AParams: array of Variant	An open array of parameter values.
const ATypes: array of TFieldType	An open array of parameter data types.

Returns

The number of affected rows.

Description

The ExecSQL method executes a SQL command with parameters. All returned cursors will be discarded.

The method allows to explicitly specify parameter types. If type of some parameter is not needed to be specified, then corresponding item in ATypes must be ftUnknown.

Syntax

```
function ExecSQL(const ASQL: String; const AParams: array of Variant; const ATypes: array
of TFieldType): LongInt; overload;
```

See Also

Executing Command (see page 66), TADQuery (see page 476)

Example

```
ADConnection1.ExecSQL('update mytab where id = :p1 set blobfld = :blb',
  [1000, StringOfChar('x', 100000)], [ftInteger, ftBLOB]);
```

1.17.1.1.5.1.59 TADCustomConnection.ExecSQLScalar Method (String)

Executes a SQL command without parameters and fetches a single value.

Parameters

Parameters	Description
const ASQL: String	A SQL command to execute.

Returns

The value of first column in first row of first result set.

Description

The ExecSQLScalar method executes a SQL command without parameters. The method returns a value of first column in

first row of first result set. All other cursors will be discarded.

Syntax

```
function ExecSQLScalar(const ASQL: String): Variant; overload;
```

See Also

Executing Command (see page 66), TADQuery (see page 476)

Example

```
var
  sVersion: String;
...
  sVersion := ADConnection1.ExecSQLScalar('select @@version');
```

1.17.1.1.5.1.60 TADCustomConnection.ExecSQLScalar Method (String, array of Variant)

Executes a SQL command with parameters and fetches a single value.

Parameters

Parameters	Description
const ASQL: String	A SQL command to execute.
const AParams: array of Variant	An open array of parameter values.

Returns

The value of first column in first row of first result set.

Description

The ExecSQLScalar method executes a SQL command with parameters. The method returns a value of first column in first row of first result set. All other cursors will be discarded.

Syntax

```
function ExecSQLScalar(const ASQL: String; const AParams: array of Variant): Variant;
overload;
```

See Also

Executing Command (see page 66), TADQuery (see page 476)

Example

```
var
  sName: String;
...
  sName := ADConnection1.ExecSQLScalar('select name from {id Employees} where id = :id',
[100]);
```

1.17.1.1.5.1.61 TADCustomConnection.ExecSQLScalar Method (String, array of Variant, array of TFieldType)

Executes a SQL command with parameters and fetches a single value.

Parameters

Parameters	Description
const ASQL: String	A SQL command to execute.
const AParams: array of Variant	An open array of parameter values.
const ATypes: array of TFieldType	An open array of parameter data typesd.

Returns

The value of first column in first row of first result set.

Description

The ExecSQLScalar method executes a SQL command with parameters. The method returns a value of first column in first row of first result set. All other cursors will be discarded.

The method allows to explicitly specify parameter types. If type of some parameter is not needed to be specified, then corresponding item in ATypes must be ftUnknown.

Syntax

```
function ExecSQLScalar(const ASQL: String; const AParams: array of Variant; const ATypes:
array of TFieldType): Variant; overload;
```

See Also

Executing Command (see page 66), TADQuery (see page 476)

Example

```
var
  sName: String;
...
  sName := ADConnection1.ExecSQLScalar('select name from {id Employees} where id = :id',
[100], [ftLargeInt]);
```

1.17.1.1.5.1.62 TADCustomConnection.GetCatalogNames Method

Populates a string list with the names of catalogs in a DB.

Parameters

Parameters	Description
const APattern: string	LIKE-pattern.
AList: TStrings	A list receiving catalog names.

Description

Call GetCatalogNames to retrieve a list of catalogs in a DB.

APattern is the LIKE-pattern filtering catalog names.

AList is a TStrings descendant that receives the catalog names. Any existing strings are deleted from the list before GetCatalogNames adds the names of all the catalogs in a DB.

Syntax

```
procedure GetCatalogNames(const APattern: string; AList: TStrings);
```

See Also

Querying Metadata (see page 127), TADMetaInfoQuery (see page 461)

Example

```
ADConnection1.GetCatalogNames('', ListBox1.Items);
```

1.17.1.1.5.1.63 TADCustomConnection.GetFieldNames Method

Populates a string list with the names of fields in a table.

Parameters

Parameters	Description
const ACatalogName: string	A catalog name.
const ASchemaName: string	A schema name.
const ATableName: string	A table name.
const APattern: string	A LIKE-pattern.
AList: TStrings	A list receiving field names.

Description

Call GetFieldNames to retrieve a list of fields in a table.

ACatalogName, ASchemaName, ATableName names the table whose field names you want added to the list.

APattern is the LIKE-pattern filtering field names.

AList is a TStrings descendant that receives the field names. Any existing strings are deleted from the list before GetFieldNames adds the names of all the fields in ATableName.

The field names are normalized - enclosed into quotes, if that is required, otherwise converted to default dictionary case.

Syntax

```
procedure GetFieldNames(const ACatalogName: string; const ASchemaName: string; const
ATableName: string; const APattern: string; AList: TStrings);
```

See Also

TADMetaInfoQuery (see page 461), Querying Metadata (see page 127)

Example 1

```
ADConnection1.GetFieldNames('', '', 'MYTAB', '', ListBox1.Items);
```

Example 2

See FireDAC\Samples\Comp Layer\TADConnection (see page 279)\GetFieldNames sample.

1.17.1.1.5.1.64 TADCustomConnection.GetGeneratorNames Method

Populates a string list with the names of generators / sequences / <how else> in a database.

Parameters

Parameters	Description
const ACatalogName: string	A catalog name.
const ASchemaName: string	A schema name.
const APattern: string	A LIKE-pattern.
AList: TStrings	A list receiving generator names.
AScopes: TADPhysObjectScopes = [osMy]	Generator scopes to returns.

Description

Call GetGeneratorNames to retrieve a list of generators / sequences / <how else> in a database.

ACatalogName and ASchemaName restrict generator names to the catalog and schema.

APattern is the LIKE-pattern filtering generator names.

AList is a TStrings descendant that receives the generator names. Any existing strings are deleted from the list before GetFieldNames adds the names of generators.

AScopes restricts generator names to the specified scopes.

The generator names are normalized - enclosed into quotes, if that is required, otherwise converted to default dictionary case. Specify the MetaDefSchema and/or MetaDefCatalog parameters in connection definition, to avoid these schema and/or catalog names in generator names.

Syntax

```
procedure GetGeneratorNames(const ACatalogName: string; const ASchemaName: string; const
APattern: string; AList: TStrings; AScopes: TADPhysObjectScopes = [osMy]; AFullName:
Boolean = True);
```

See Also

TADMetaInfoQuery (see page 461), Querying Metadata (see page 127)

Example

```
ADConnection1.GetGeneratorNames('', '', 'MY%', ListBox1.Items, [osMy, osSystem]);
```

1.17.1.1.5.1.65 TADCustomConnection.GetInfoReport Method

Populates a string list with a detailed report about a connection state.

Parameters

Parameters	Description
AList: TStrings	A string list, where a report is put.
AItems: TADInfoReportItems = [riConnDef .. riKeepConnected]	A report section and mode flags.

Description

Use the GetInfoReport method to get the detailed report about the connection status. The report consists from the optional sections, controlled by the AItems parameter:

Flag	Section header	Description
riConnDef	Connection definition parameters	The complete list of a connection definition parameters.
riFireDAC	FireDAC info	The detailed information about the FireDAC build,
		including version number, tool version, defines,
		Unicode mode.
riClientLog	Client info	The DBMS client loading log. When loading fails, then includes the failure error message.
riClient	-- // --	The detailed information about a DBMS client, when FireDAC successfully loaded the client. Otherwise - failure error message.
riSessionHints	Session info	The possible incompatibilities between FireDAC, DBMS client and DBMS server, when connection is active or riTryConnect parameter is True and connection successfully established.
riSession	-- // --	The detailed information about a DBMS session, when connection is active or riTryConnect parameter is True and connection successfully established. Otherwise - "not connected" or failure error message.

The connection handling mode may be specified by the AItems parameter:

Flag	Description
riTryConnect	When connection is inactive, then FireDAC will try to establish a connection.
riKeepConnected	When connection was inactive and FireDAC established it, then FireDAC on GetInfoReport exiting will keep it active. Otherwise the connection will be closed.

The report is put into AList string list. Application may use this method:

- to test a connection to a DBMS;
- to report DBMS environment;
- to report failures.

See DBMS Environment Reports (see page 170) for detailed info about this method and provided information.

Syntax

```
function GetInfoReport(AList: TStrings; AItems: TADInfoReportItems = [riConnDef ..
riKeepConnected]): TADInfoReportStatus;
```

See Also

DBMS Environment Reports (see page 170), Connected (see page 321)

Example

```
ADConnection1.GetInfoReport(mmInfo.Lines);
```

1.17.1.1.5.1.66 TADCustomConnection.GetKeyFieldNames Method

Populates a string list with the names of key fields in a table.

Parameters

Parameters	Description
const ACatalogName: string	A catalog name.
const ASchemaName: string	A schema name.
const ATableName: string	A table name.
const APattern: string	A LIKE-pattern.
AList: TStrings	A list receiving primary key field names.

Description

Call GetKeyFieldNames to retrieve a list of primary key fields in a table.

ACatalogName, ASchemaName, ATableName names the table whose primary field names you want added to the list.

APattern is the LIKE-pattern filtering field names.

AList is a TStrings descendant that receives the primary key field names. Any existing strings are deleted from the list before GetKeyFieldNames adds the names of the primary key fields in ATableName.

The field names are normalized - enclosed into quotes, if that is required, otherwise converted to default dictionary case.

Syntax

```
procedure GetKeyFieldNames(const ACatalogName: string; const ASchemaName: string; const
ATableName: string; const APattern: string; AList: TStrings);
```

See Also

TADMetaInfoQuery (see page 461), Querying Metadata (see page 127)

Example 1

```
ADConnection1.GetKeyFieldNames('NORTHWIND', 'DBO', 'MYTAB', '', ListBox1.Items);
```

Example 2

See FireDAC\Samples\Comp Layer\TADConnection (see page 279)\GetFieldNames (see page 346) sample.

1.17.1.1.5.1.67 TADCustomConnection.GetLastAutoGenValue Method

Returns last auto generated value.

Parameters

Parameters	Description
const AName: String	The name of generator or sequence or empty string.

Returns

The last auto generated value

Description

The GetLastAutoGenValue method returns last auto-generated value. The meaning and result depends on DBMS:

DBMS	Description
Oracle	AName is the name of a sequence. The method will return CurrValue, if it exists in the session.
Interbase / Firebird	AName is the name of a generator. The method will return GEN_ID(0).
MS SQL Server, MySQL, etc	The last auto generated value in the session.

Syntax

```
function GetLastAutoGenValue(const AName: String): Variant;
```

See Also

RDBMSKind (see page 335)

Example

```
ShowMessage(VarToStr(ADConnection1.GetLastAutoGenValue('MyGen')));
```

1.17.1.1.5.1.68 TADCustomConnection.GetPackageNames Method

Populates a string list with the names of the packages in a DB.

Parameters

Parameters	Description
const ACatalogName: string	A catalog name.
const ASchemaName: string	A schema name.
const APattern: string	A LIKE-pattern.
AList: TStrings	A list receiving package names.
AScopes: TADPhysObjectScopes = [osMy]	Package scopes to return.

Description

Call GetPackageNames to retrieve a list of packages in a DB.

ACatalogName, ASchemaName restrict package names to the catalog and schema.

APattern is the LIKE-pattern filtering package names.

AList is a TStrings descendant that receives the package names. Any existing strings are deleted from the list before GetPackageNames adds the names of all the packages in a DB.

AScopes restricts package names to the specified scopes.

The generator names are normalized - enclosed into quotes, if that is required, otherwise converted to default dictionary case. Specify the MetaDefSchema and/or MetaDefCatalog parameters in connection definition, to avoid these schema and/or catalog names in package names.

Syntax

```
procedure GetPackageNames(const ACatalogName: string; const ASchemaName: string; const
APattern: string; AList: TStrings; AScopes: TADPhysObjectScopes = [osMy]; AFullName:
Boolean = True);
```

See Also

TADMetaInfoQuery (see page 461), Querying Metadata (see page 127)

Example

```
ADConnection1.GetPackageNames('', 'SYS', 'DBMS%', ListBox1.Items, [osMy, osSystem]);
```

1.17.1.1.5.1.69 TADCustomConnection.GetSchemaNames Method

Populates a string list with the names of schemas in a DB.

Parameters

Parameters	Description
const ACatalogName: string	A catalog name.
const APattern: string	LIKE-pattern.
AList: TStrings	A list receiving catalog names.

Description

Call GetSchemaNames to retrieve a list of schemas in a DB.

ACatalogName restricts schema names to the specified catalog. If it is empty, then all schemas will be returned.

APattern is the LIKE-pattern filtering schema names.

AList is a TStrings descendant that receives the schema names. Any existing strings are deleted from the list before GetSchemaNames adds the names of all the schemas in a DB.

Syntax

```
procedure GetSchemaNames(const ACatalogName: string; const APattern: string; AList:
TStrings);
```

See Also

TADMetaInfoQuery (see page 461), Querying Metadata (see page 127)

Example

```
ADConnection1.GetSchemaNames('', '', ListBox1.Items);
```

1.17.1.1.5.1.70 TADCustomConnection.GetStoredProcNames Method

Populates a string list with the names of stored procedures in a DB.

Parameters

Parameters	Description
const ACatalogName: string	A catalog name.
const ASchemaName: string	A schema name.
const APackage: string	A package name.
const APattern: string	A LIKE-pattern.
AList: TStrings	A list receiving procedure names.
AScopes: TADPhysObjectScopes = [osMy]	Procedure scopes to return.

Description

Call GetStoredProcNames to retrieve a list of stored procedures and functions in a DB.

ACatalogName, ASchemaName restrict procedure names to the catalog and schema.

APackage restricts procedure names to a package.

APattern is the LIKE-pattern filtering procedure names.

AList is a TStrings descendant that receives the procedure names. Any existing strings are deleted from the list before GetStoredProcNames adds the names of all the procedures in a DB.

AScopes restricts stored procedure names to the specified scopes. Applies only to non-packaged procedures.

The stored procedure names are normalized - enclosed into quotes, if that is required, otherwise converted to default dictionary case. Specify the MetaDefSchema and/or MetaDefCatalog parameters in connection definition, to avoid these schema and/or catalog names in procedure names.

Syntax

```
procedure GetStoredProcNames(const ACatalogName: string; const ASchemaName: string; const
APackage: string; const APattern: string; AList: TStrings; AScopes: TADPhysObjectScopes =
[osMy]; AFullName: Boolean = True);
```

See Also

TADMetaInfoQuery (see page 461), Querying Metadata (see page 127)

Example

```
ADConnection1.GetStoredProcNames('', '', '', '', ListBox1.Items, [osMy, osSystem]);
```

1.17.1.1.5.1.71 TADCustomConnection.GetTableNames Method

Populates a string list with the names of tables in a DB.

Parameters

Parameters	Description
const ACatalogName: string	A catalog name.
const ASchemaName: string	A schema name.
const APattern: string	A LIKE-pattern.
AList: TStrings	A list receiving table names.
AScopes: TADPhysObjectScopes = [osMy]	Table scopes to return.
AKinds: TADPhysTableKinds = [tkSynonym, tkTable, tkView]	Object kinds to return.

Description

Call GetTableNames to retrieve a list of tables in a DB.

ACatalogName, ASchemaName restrict table names to the catalog and schema.

APattern is the LIKE-pattern filtering table names.

AList is a TStrings descendant that receives the table names. Any existing strings are deleted from the list before GetTableNames adds the names of all the tables in a DB.

AScopes restricts table names to the specified scopes.

AKinds restricts tables to the specified object kinds.

The table names are normalized - enclosed into quotes, if that is required, otherwise converted to default dictionary case. Specify the MetaDefSchema and/or MetaDefCatalog parameters in connection definition, to avoid these schema and/or catalog names in table names.

Syntax

```
procedure GetTableNames(const ACatalogName: string; const ASchemaName: string; const
APattern: string; AList: TStrings; AScopes: TADPhysObjectScopes = [osMy]; AKinds:
TADPhysTableKinds = [tkSynonym, tkTable, tkView]; AFullName: Boolean = True);
```

See Also

TADMetaInfoQuery (see page 461), Querying Metadata (see page 127)

Example 1

```
ADConnection1.GetTableNames('NORTHWIND', 'DBO', 'ADQA%', '', ListBox1.Items,
  [osMy, osSystem, osOther], [tkTable, tkView]);
```

Example 2

See FireDAC\Samples\Comp Layer\TADConnection (see page 279)\GetFieldNames (see page 346) sample.

1.17.1.1.5.1.72 TADCustomConnection.Offline Method

Sets connection to offline mode.

Description

The Offline method disconnects datasets associated with this connection from DBMS, but does not close them. Then closes the connection to the DBMS by implicitly setting Connected (see page 321) to False.

After the call the dataset continue to work as an in-memory dataset and you can perform with the dataset any client operation, which does not require a DBMS, like a browsing. When an application in offline mode will try to perform any operation, requiring to communicate with the DBMS, then following will happen:

- When ResourceOptions.AutoConnect (see page 887) is True, then the Online (see page 353) method will be called, trying to establish a connection to the DBMS;
- When ResourceOptions.AutoConnect (see page 887) is False, then an exception will be raised.

To bring connection and associated datasets to the online state, call the Online (see page 353) method.

Calling Offline method is the same as setting Offlined (see page 331) to True.

Syntax

```
procedure Offline;
```

See Also

Offlining Connection (see page 40), Online (see page 353), Offlined (see page 331), Connected (see page 321), TADDataSet.Offline (see page 640)

Example

```
ADConnection1.Offline;
lblState.Caption := '<not connected>';
```

1.17.1.1.5.1.73 TADCustomConnection.Online Method

Sets connection to online mode.

Description

The Online method opens the connection to the DBMS by implicitly setting the Connected (see page 321) to True and changing the state to online.

To bring connection and associated datasets to the offline state, call the Offline (see page 352) method.

Calling Online method is the same as setting Offlined (see page 331) to False.

Syntax

```
procedure Online;
```

See Also

Offlining Connection (see page 40), Offline (see page 352), Offlined (see page 331), Connected (see page 321)

Example

```
ADConnection1.Online;
lblState.Caption := '<connected>';
```

1.17.1.1.5.1.74 TADCustomConnection.Open Method (String)

Open connection to the database, using specified connection string.

Parameters

Parameters	Description
const AConnectionString: String	A ';' separated list of the <name>=<value> parameter pairs.

Description

Call Open method to open connection to a DBMS, using specified connection string. The method is shortcut to filling of Params (see page 334) property and setting Connected (see page 321) to True.

AConnectionString string value may be one of two forms:

- it is a ';' separated list of the <name>=<value> connection definition parameter pairs. If connection string contains User_Name and/or Password parameters and AUserName and/or APassword arguments are specified, then they will override corresponding parameters in connection string.
- it is a connection definition name.

Syntax

```
procedure Open(const AConnectionString: String); overload;
```

See Also

Params (see page 334), ConnectionDefName (see page 325), DriverName (see page 327), Connected (see page 321), Open(String, String) (see page 354), Open(String, String, String) (see page 354)

Example

```
ADConnection1.Open('DriverID=Ora;Database=ORA_920_APP;User_Name=addemo;Password=a');
```

1.17.1.1.5.1.75 TADCustomConnection.Open Method (String, String)

Open connection to the database, using specified user name and password.

Parameters

Parameters	Description
const AUserName: String	User name.
const APassword: String	Password.

Description

Call Open method to open connection to a DBMS, using specified user name and password.

Syntax

```
procedure Open(const AUserName: String; const APassword: String); overload;
```

See Also

Params (see page 334), ConnectionDefName (see page 325), DriverName (see page 327), Connected (see page 321), Open(String) (see page 353), Open(String, String, String) (see page 354)

Example

```
ADConnection1.Open('addemo', 'a');
```

1.17.1.1.5.1.76 TADCustomConnection.Open Method (String, String, String)

Open connection to the database, using specified connection parameters, user name and password.

Parameters

Parameters	Description
const AConnectionString: String	A ';' separated list of the <name>=<value> parameter pairs.
const AUserName: String	Optional user name.
const APassword: String	Optional password.

Description

Call Open method to open connection to a DBMS, using specified connection string and optional user name and password. The method is shortcut to filling of Params (see page 334) property and setting Connected (see page 321) to True.

AConnectionString string value may be one of two forms:

- it is a ';' separated list of the <name>=<value> connection definition parameter pairs. If connection string contains User_Name and/or Password parameters and AUserName and/or APassword arguments are specified, then they will override corresponding parameters in connection string.
- it is a connection definition name.

Syntax

```
procedure Open(const AConnectionString: String; const AUserName: String; const APassword:
String); overload;
```

See Also

Params (see page 334), ConnectionDefName (see page 325), DriverName (see page 327), Connected (see page 321), Open(String) (see page 353), Open(String, String) (see page 354)

Example

```
ADConnection1.Open('DriverID=Ora;Database=ORA_920_APP', 'addemo', 'a');
```

1.17.1.1.5.1.77 TADCustomConnection.Ping Method

Pings the DBMS server.

Returns

True, if the connection to the DBMS is working.

Description

The Ping method checks whether a connection to a DBMS server is alive. When the connection is dead and ResourceOptions (see page 335).AutoReconnect (see page 887) is True, then an automatic reconnection is attempted. When a connection is inactive, the Ping method will try to open a connection.

This function can be used by the clients that remain idle for a long while or work in non stable environment (eg bad network), to check whether the server has closed the connection or is unaccessible and reconnect if necessary. Or the server is accessible to open a connection.

Syntax

```
function Ping: Boolean;
```

See Also

TADTopResourceOptions.AutoReconnect (see page 887), Offline (see page 352), Online (see page 353), Connected (see page 321)

Example

```
procedure TMyDataModule.DoPing(ASender: TObject);
begin
  if not ADConnection1.Ping then
    ShowMessage('Connection is losted !');
end;
.......
FTimer := TTimer.Create;
FTimer.OnTimer := DoPing;
FTimer.Interval := 30000; // each 30 secs
FTimer.Enabled := True;
```

1.17.1.1.5.1.78 TADCustomConnection.RefreshMetadataCache Method

Invalidates meta data cache.

Parameters

Parameters	Description
const AObjName: String = ''	An optional object name for which metadata must be discarded. If not specified, then all cached metadata.

Description

The RefreshMetadataCache method discards cached metadata, like a stored procedure definitions and table primary keys.

If AObjName is specified, then will be discarded cached metadata:

- for all tables with AObjName name;
- for all package with AObjName name;
- for all stored procedures with AObjName name.

If AObjName is empty string (default value), then all connection cached metadata will be discarded.

Call this method, if the DB metadata has been changed, so the cached metadata is invalid. The metadata is cached by datasets, if fiMeta is included into FetchOptions.Cache.

Syntax

```
procedure RefreshMetadataCache(const AObjName: String = '');
```

See Also

TADFetchOptions.Cache (see page 852), Working with Metadata (see page 127)

Example

```
ADConnection1.ExecSQL('alter table mytab drop primary key');
ADConnection1.RefreshMetadataCache;
```

1.17.1.1.5.1.79 TADCustomConnection.ReleaseClients Method

Performs resource releasing operation on all associated datasets and commands.

Parameters

Parameters	Description
AMode: TADReleaseClientMode = rmDisconnect	An operation to perform.

Description

Use ReleaseClients method to perform one of few DBMS resources releasing operation on all datasets and commands, associated with this connection object. An operation is defined by AMode:

Mode	Description
rmFetchAll	Performs FetchAll, if dataset is active and not all records are fetched. If dataset is unidirectional, then it will be closed. All active commands will be closed.
rmClose	Close all datasets and commands.
rmOffline	Bring all datasets to offline mode, when all records are fetched. All commands will be closed and unprepared.
rmDisconnect	All datasets and commands will be closed and unprepared.

Syntax

```
procedure ReleaseClients(AMode: TADReleaseClientMode = rmDisconnect);
```

See Also

Offline (see page 352), Online (see page 353), Close, Open (see page 353)

Example

```
// Fetch all records from active cursors, then commit current transaction.
// On MS SQL Server, for example, active cursors are invalidated after finishing of a
transaction.
ADConnection1.ReleaseClients(rmFetchAll);
ADConnection1.Commit;
```

1.17.1.1.5.1.80 TADCustomConnection.Rollback Method

Cancels all modifications to the data made in the current transaction and optionally ends transaction.

Description

Call Rollback to cancel all modifications, like a INSERT's / UPDATE's / DELETE's, made in current transaction to the database.

FireDAC supports nested transactions, so the current transaction is the one started with most recent StartTransaction (see page 358) call. If the database does not support nested transactions, like most of DBMS's, then FireDAC will emulate nested transactions using savepoints.

Before calling Rollback, an application may check the status of the InTransaction (see page 328) property. If an application calls Rollback and there is no current transaction, an exception is raised.

A Rollback call on Interbase / Firebird will close and unprepare all datasets and commands, associated with this transaction object. On some other DBMS's a call will invalidate all active result sets. For example, on MS SQL Server 2005. To force FetchAll (see page 624) on active datasets use ReleaseClients (see page 356) method.

The Rollback call is the shortcut to Transaction (see page 337).Rollback (see page 418), if Transaction property is assigned. Otherwise Rollback will operate on default connection transaction.

Syntax

```
procedure Rollback;
```

See Also

Managing Transactions (see page 41), StartTransaction (see page 358), InTransaction (see page 328), Commit (see page 340), RollbackRetaining (see page 357), ReleaseClients (see page 356)

Example 1

```
procedure TForm1.DoThatButtonClick(Sender: TObject);
begin
  ADConnection1.StartTransaction;
  try
    if CustomerQuery.Locate('ID', [100]) then begin
      CustomerQuery.Edit;
      CustomerQuery.FieldByName('status').AsString := 'top';
      CustomerQuery.Post;
      ADConnection1.ExecSQL('delete from debt where CustID = 100');
      ADConnection1.Commit;
    end;
  except
    ADConnection1.Rollback;
    raise;
  end;
end;
```

Example 2

See FireDAC\Samples\Comp Layer\TADConnection (see page 279)\Transactions sample for details.

1.17.1.1.5.1.81 TADCustomConnection.RollbackRetaining Method

Cancels all modifications to the data made in the current transaction, without ending the current transaction.

Description

Call RollbackRetaining to cancel all modifications, like a INSERT's / UPDATE's / DELETE's, made in current transaction to the database.

FireDAC supports nested transactions, so the current transaction is the one started with most recent StartTransaction (see page 358) call. If the database does not support nested transactions, like most of DBMS's, then FireDAC will emulate nested transactions using savepoints.

The RollbackRetaining call is the shortcut to Transaction (see page 337).RollbackRetaining (see page 419), if Transaction property is assigned. Otherwise RollbackRetaining will operate on default connection transaction.

Before calling RollbackRetaining, an application may check the status of the InTransaction (see page 328) property. If an application calls RollbackRetaining and there is no current transaction, an exception is raised.

RollbackRetaining will use native functionality on IB/FB. On other DBMS's the method will be equal to calls of Rollback (see page 356), then of StartTransaction (see page 358).

Syntax

```
procedure RollbackRetaining;
```

See Also

Managing Transactions (see page 41), StartTransaction (see page 358), InTransaction (see page 328), Commit (see page 340), Rollback (see page 356)

Example

```
procedure TForm1.DoThatButtonClick(Sender: TObject);
begin
  ADConnection1.StartTransaction;
  try
    if CustomerQuery.Locate('ID', [100]) then begin
      CustomerQuery.Edit;
      CustomerQuery.FieldByName('status').AsString := 'top';
      CustomerQuery.Post;
      ADConnection1.ExecSQL('delete from debt where CustID = 100');
      ADConnection1.CommitRetaining;
```

```
    end;
  except
    ADConnection1.RollbackRetaining;
    raise;
  end;
end;
```

1.17.1.1.5.1.82 TADCustomConnection.StartTransaction Method

Starts new transaction at database.

Description

Call StartTransaction to start a new transaction at the DBMS.

The FireDAC supports nested transactions. If DBMS does not support nested transactions, then FireDAC will emulate them using savepoints. If transaction is already active, then FireDAC will put savepoint, otherwise will start new transaction. Use TxOptions (see page 337).EnableNested (see page 894) to enable / disable nested transactions.

Before calling StartTransaction, an application may adjust settings of the TxOptions (see page 337) property as desired. Adjusting options after transaction is started does not affect current transaction.

All data modifications, like a INSERT's / UPDATE's / DELETE's, made after a call of StartTransaction, may be either confirmed by calling Commit (see page 340), either undone by calling Rollback (see page 356).

Some DBMS's will fail to start transaction, if there are active result sets. For example, MS SQL Server 2005.

The StartTransaction call is the shortcut to Transaction (see page 337).StartTransaction (see page 419), if Transaction property is assigned. Otherwise StartTransaction will operate on default connection transaction.

Syntax

```
procedure StartTransaction;
```

See Also

Managing Transactions (see page 41), InTransaction (see page 328), Commit (see page 340), Rollback (see page 356), TxOptions (see page 337)

Example

See FireDAC\Samples\Comp Layer\TADConnection (see page 279)\Transactions sample for details.

1.17.1.1.6 TADCustomEventAlerter Class

The class is responsible for handling the database event notifications.

public

public	Description
Active (see page 359)	Controls the event registration state of the event alerter.
Connection (see page 360)	Specifies the connection component that actually forms the connection to the DBMS.
EventAlerterIntf (see page 360)	Returns IADPhysEventAlerter interface.
Names (see page 360)	Specifies the event names handled by this alerter.
OnAlert (see page 361)	The event fires, when a DBMS event is fired.
OnTimeout (see page 361)	The event fires, when a DBMS event is timed out.
Options (see page 361)	The set of even alerter options.
Register (see page 362)	Registers the events with a DBMS.
Signal (see page 362)	Fires a DBMS event.
Unregister (see page 362)	Unregisters the events with a DBMS.

Class Hierarchy



File

uADCompClient

Description

Use the TADCustomEventAlerter class to handle the database event notifications (see page 75). The kind and general behavior of the events is DBMS specific. But TADCustomEventAlerter provides unified way for handling them.

To start receive the event alerts, fill in Names property with required event names. Set Options.Kind to the event alerter kind or leave it empty to use the default alerter. Specify OnAlert event handler, which will be fired when an event is occurred. And set Active to True or call Register method. To stop receive the event alerts, set Active to False or call Unregister.

For details about what FireDAC drivers are supporting event alerters and how to use them see the Database Alerts (see page 75) chapter.

This class hides some of the properties, allowing to control the properties visibility by the descendant classes. End users should use TADEventAlerter.

Syntax

```
TADCustomEventAlerter = class(TADComponent, IADPhysEventHandler);
```

See Also

Database Alerts (see page 75), TADEventAlerter (see page 422), TADEventAlerterOptions (see page 847)

Example 1

```
ADEventAlerter1.Names.Text := 'Customers';
ADEventAlerter1.Options.Kind := 'DBMS_ALERT';
ADEventAlerter1.Options.Synchronize := True;
ADEventAlerter1.Options.Timeout := 10000;
ADEventAlerter1.OnAlter := DoAlert;
ADEventAlerter1.OnTimeout := DoTimeout;
ADEventAlerter1.Active := True;
........

procedure TForm1.DoAlert(ASender: TADCustomEventAlerter;
  const AEventName: String; const AArgument: Variant);
begin
  if CompareText(AEventName, 'Customers') = 0 then
    qryCustomers.Refresh;
end;

procedure TForm1.DoTimeout(ASender: TObject);
begin
  qryCustomers.Refresh;
end;
```

Example 2

See the demo at: FireDAC\Samples\Comp Layer\TADEventAlerter (see page 422)\Main

1.17.1.1.6.1 public

1.17.1.1.6.1.1 TADCustomEventAlerter.Active Property

Controls the event registration state of the event alerter.

Description

Use Active property to set or check when an event alerter has registered the Names events at a DBMS and waiting for events.

Set the Active property to True to register the events at a DBMS and start processing of incoming events. If an FireDAC DBMS driver supports few event alerters, then the event alerter kind must be explicitly assigned to the Options (see page 361).Kind (see page 848).

Set the property to False to stop processing of the events.

Syntax

```
property Active: Boolean;
```

See Also

Database Alerts (see page 75), Names (see page 360), Register (see page 362), Unregister (see page 362)

1.17.1.1.6.1.2 TADCustomEventAlerter.Connection Property

Specifies the connection component that actually forms the connection to the DBMS.

Description

Set Connection to the connection component that establishes a connection to the DBMS. If to change Connection property value, while an even alerter is active, then it will unregister his events.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

Names (see page 360), Register (see page 362)

1.17.1.1.6.1.3 TADCustomEventAlerter.EventAlerterIntf Property

Returns IADPhysEventAlerter interface.

Description

The EventAlerterIntf property returns the reference to the IADPhysEventAlerter interface. It is not nil, if the connection is active and alerter has registered his events. Otherwise it is nil. It is not recommended to mix usage of TADCustomEventAlerter API and TADCustomEventAlerter.EventAlerterIntf API.

Syntax

```
property EventAlerterIntf: IADPhysEventAlerter;
```

See Also

IADPhysEventAlerter

1.17.1.1.6.1.4 TADCustomEventAlerter.Names Property

Specifies the event names handled by this alerter.

Description

Use the Names property to specify the event names.

The event alerter will be notified only about the listed events. Note, that the Oracle DBMS_PIPE event alerter supports only a single event name. The event names may be case-sensitive or case-insensitive, depending on a DBMS.

Syntax

```
property Names: TStrings;
```

See Also

Database Alerts (see page 75), Register (see page 362), Active (see page 359)

Example

```
with ADEventAlerter1 do begin
  Unregister;
  Names.Clear;
  Names.Add('Customers');
  Names.Add('Orders');
  Register;
end;
```

1.17.1.1.6.1.5 TADCustomEventAlerter.OnAlert Event

The event fires, when a DBMS event is fired.

Description

Use the OnAlert event handler to specify an event handler, which will be fired when one of the specified in the Names DBMS events is fired. To start receive event notifications, the events must be registered using the Register (see page 362) method or the Active (see page 359) property. The event handler may be fired in the own event alerter thread (False) or in the main application thread (True), depending on the Options (see page 361).Synchronize (see page 848) option.

The event handler receives three arguments:

- ASender is the event alerter reference;
- AEventName specifies the name of the event;
- AArgument specifies the additional arguments to the event. The arguments are fully DBMS and application dependent. There must be a Null or Unassigned value, if no arguments are specified. The scalar variant value, when only single argument is specified. Or an variant array of values.

Syntax

```
property OnAlert: TADEventAlerterEvent;
```

See Also

Database Alerts (see page 75), OnTimeout (see page 361), Names (see page 360)

Example

```
procedure TForm1.ADEventAlerter1Alert(ASender: TADCustomEventAlerter;
    const AEventName: String; const AArgument: Variant);
begin
  if CompareText(AEventName, 'Customers') = 0 then
    qryCustomers.Refresh
  else if CompareText(AEventName, 'Orders') = 0 then
    qryOrders.Refresh;
end;
```

1.17.1.1.6.1.6 TADCustomEventAlerter.OnTimeout Event

The event fires, when a DBMS event is timed out.

Description

Use the OnTimeout event handler to specify an event handler, which will be fired after the specified by the Options (see page 361).Timeout (see page 849) time and no one event is fired. The specified timeout is in 1/1000 of sec.The event handler may be fired in the own event alerter thread (False) or in the main application thread (True), depending on the Options (see page 361).Synchronize (see page 848) option.

Syntax

```
property OnTimeout: TNotifyEvent;
```

See Also

Database Alerts (see page 75), OnAlert (see page 361), Options (see page 361)

1.17.1.1.6.1.7 TADCustomEventAlerter.Options Property

The set of even alerter options.

Description

Use the Options property to specify the different aspects of even handling.

Syntax

```
property Options: TADEventAlerterOptions;
```

See Also

Database Alerts (see page 75), TADEventAlerterOptions (see page 847)

1.17.1.1.6.1.8 TADCustomEventAlerter.Register Method

Registers the events with a DBMS.

Description

Use the Register method to register the events at a DBMS and start processing of incoming events. If an FireDAC DBMS driver supports few event alerters, then the event alerter kind must be explicitly assigned to the Options (see page 361).Kind (see page 848). If the event alerter is already registered at a DBMS, then no actions will be taken. Alternatively, you can set the Active (see page 359) property to True.

Syntax

```
procedure Register;
```

See Also

Database Alerts (see page 75), Active (see page 359), Name, Unregister (see page 362)

1.17.1.1.6.1.9 TADCustomEventAlerter.Signal Method

Fires a DBMS event.

Parameters

Parameters	Description
const AEvent: String	The event name to fire.
const AArgument: Variant	The optional arguments for the event. Use Unassigned or Null if no arguments needed. Use Variant array if more than one argument need to be specified.

Description

Use the Signal method to fire a DBMS event. To fire an event, the event alerter must be registered with a DBMS.

Syntax

```
procedure Signal(const AEvent: String; const AArgument: Variant);
```

See Also

Database Alerts (see page 75), Register (see page 362)

Example

```
ADEventAlerter1.Names.Text := 'Customers';
ADEventAlerter1.Register;
ADEventAlerter1.Signal('Customers', 123);
```

1.17.1.1.6.1.10 TADCustomEventAlerter.Unregister Method

Unregisters the events with a DBMS.

Description

Use the Register method to unregister the events at a DBMS and stop processing of incoming events. If the event alerter is already unregistered at a DBMS, then no actions will be taken. Alternatively, you can set the Active (see page 359) property to False.

Syntax

```
procedure Unregister;
```

See Also

Database Alerts (see page 75), Active (see page 359), Name, Register (see page 362)

1.17.1.1.7 TADCustomLocalSQL Class

The base class for the Local SQL engine.

public

public	Description
Active (see page 364)	Activates Local SQL engine.
Connection (see page 364)	Specifies the FireDAC connection component to use.
DataSets (see page 364)	Collection of the registered datasets.
OnGetDataSet (see page 365)	The event firing when Local SQL engine looks for a dataset object.
OnReadDataSet (see page 365)	The event firing when Local SQL is going to read a dataset object.

Class Hierarchy



File

uADCompClient

Description

The TADCustomLocalSQL class is a base class for Local SQL engine (see page 96). The programmer must not use it directly. The engine itself is implemented by TADLocalSQL (see page 825) class.

The Connection (see page 364) property points to a connection object which will provide ability to execute the Local SQL commands. To activate engine set Active to True.

FireDAC datasets may be registered with the engine by setting their TADAdaptedDataSet.LocalSQL (see page 261) properties. FireDAC and Non-FireDAC datasets may be registered with the engine using TADCustomLocalSQL.DataSets (see page 364) property.

Syntax

```
TADCustomLocalSQL = class(TADComponent, IADPhysSQLHandler);
```

See Also

Local SQL (see page 96), TADLocalSQL (see page 825)

Example

```
ADConnection1.DriverName := 'SQLite';

with ADLocalSQL1.DataSets.Add do begin
  DataSet := ADOQuery1;
  Name := 'Orders';
end;
with ADLocalSQL1.DataSets.Add do begin
  DataSet := ADOQuery2;
  Name := 'Order Details';
end;
with ADLocalSQL1.DataSets.Add do begin
  DataSet := ADMemTable1;
  Name := 'Customers';
end;
with ADLocalSQL1.DataSets.Add do begin
  DataSet := ADMemTable2;
  Name := 'Vendors';
end;
ADLocalSQL1.Connection := ADConnection1;
ADLocalSQL1.Active := True;

ADQuery1.Connection := ADConnection1;
ADQuery1.Open('select from Orders o left join "Order Details" od on o.OrderID =
od.OrderID');
```

1.17.1.1.7.1 public

1.17.1.1.7.1.1 TADCustomLocalSQL.Active Property

Activates Local SQL engine.

Description

Set Active property to True to activate Local SQL engine and add registered datasets to Local SQL namespace. Set Active to False to remove datasets from Local SQL namespace and deactivate Local SQL engine. Local SQL queries may be performed only when Active is True and connection is active too.

When application needs to perform massive Local SQL configuration tasks, like to add a set of datasets, remove set of datasets, etc, then it is recommended first set Active to False. And when a task is finished set Active to True.

Active may be set to False only when there is no active / prepared SQL commands using this Local SQL.

Syntax

```
property Active: Boolean;
```

See Also

Local SQL (see page 96), Connection (see page 364)

1.17.1.1.7.1.2 TADCustomLocalSQL.Connection Property

Specifies the FireDAC connection component to use.

Description

Use Connection to specify an FireDAC connection connected to a SQLite database. This connection will be used to publish registered datasets and execute Local SQL commands.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

Local SQL (see page 96), Active (see page 364)

1.17.1.1.7.1.3 TADCustomLocalSQL.DataSets Property

Collection of the registered datasets.

Description

Use DataSets collection property to manage the datasets registered with this Local SQL engine.

FireDAC datasets may be added to the collection implicitly by setting TADAdaptedDataSet.LocalSQL (see page 261) property. Non-FireDAC and FireDAC datasets may be added explicitly by adding new items to DataSets collection. To remove a dataset from the list of registered datasets, remove corresponding item from the DataSets collection. The datasets may be added or removed as at design time, as at run time.

All datasets in collection must have unique names. When item Name (see page 427) property is specified, then it is used as a dataset name. If not specified, then TDataSet.Name value will be used.

Syntax

```
property DataSets: TADLocalSQLDataSets;
```

See Also

Local SQL (see page 96), TADAdaptedDataSet.LocalSQL (see page 261), TADLocalSQLDataSets (see page 428), TADLocalSQLDataSet (see page 425)

Example

```
ADLocalSQL1.DataSets.Add.DataSet := ADOQuery1;
```

```
ADLocalSQL1.DataSets.Add.DataSet := ADOQuery2;
ADQuery1.LocalSQL := ADLocalSQL1;
ADQuery2.LocalSQL := ADLocalSQL1;
```

1.17.1.1.7.1.4 TADCustomLocalSQL.OnGetDataSet Event

The event firing when Local SQL engine looks for a dataset object.

Description

Use OnGetDataSet event handler to submit a dataset object by its name.

This event may be used to provide a dataset dynamically without registering it first with the Local SQL engine. The value of AName argument and ADataSet.Name may be different. After returning a dataset, it will be registered in DataSets (see page 364) collection under AName argument value.

The event handler must return ADataSet, AAdapter or both arguments assigned. For FireDAC datasets the ADataSet may be assigned only. For non-FireDAC datasets is recommended to assign both arguments.

Syntax

```
property OnGetDataSet: TADGetDatasetEvent;
```

See Also

Local SQL (see page 96), DataSets (see page 364), OnReadDataSet (see page 365)

1.17.1.1.7.1.5 TADCustomLocalSQL.OnReadDataSet Event

The event firing when Local SQL is going to read a dataset object.

Description

Use OnReadDataSet event handler to setup a dataset object before it will be read.

This event may be used to set parameter or macro values. Or even to change the SQL command text. If a dataset is inactive, then Local SQL will activate it right after the event handler call.

Syntax

```
property OnReadDataSet: TADReadDatasetEvent;
```

See Also

Local SQL (see page 96), OnGetDataSet (see page 365)

1.17.1.1.8 TADCustomManager Class

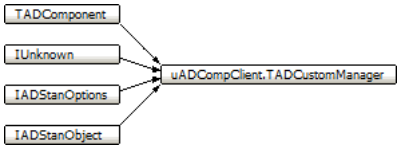
The class is responsible to connection definitions and connections management.

public

public	Description
Active (see page 367)	Gets or sets FireDAC manager active status.
ActiveStoredUsage (see page 368)	Controls how to use Active (see page 367) property value saved to DFM.
R ActualConnectionDefFileName (see page 368)	Returns expanded connection definition file name.
R ActualDriverDefFileName (see page 368)	Returns expanded driver definition file name.
AfterLoadConnectionDefFile (see page 368)	Fires after connection definitions file is loaded.
AfterShutdown (see page 368)	Fires after manager is finished.
AfterStartup (see page 369)	Fires after manager is started.
R AutoCreated (see page 369)	Is manager object auto created.
BeforeLoadConnectionDefFile (see page 369)	Fires before connection definitions file is loaded.
BeforeShutdown (see page 369)	Fires before manager will be finished.

BeforeStartup (see page 370)	Fires before manager will be started.
R CanRefreshConnectionDefFile (see page 370)	Checks if connection definition list may be refreshed.
R ConnectionCount (see page 370)	The number of connection objects.
ConnectionDefFileAutoLoad (see page 370)	Controls how connection definitions file must be loaded.
R ConnectionDefFileLoaded (see page 371)	Checks is loaded the connection definitions file.
ConnectionDefFileName (see page 371)	Gets / sets the name of a connection definitions file.
R ConnectionDefs (see page 372)	The list of connection definitions.
R Connections (see page 372)	The application connection objects.
DriverDefFileAutoLoad (see page 373)	Controls how driver definitions file must be loaded.
DriverDefFileName (see page 373)	Gets / sets the name of a driver definitions file.
R DriverDefs (see page 373)	The list of driver definitions.
FetchOptions (see page 374)	The "root" fetch options.
FormatOptions (see page 374)	The "root" data format options.
ResourceOptions (see page 374)	The "root" resources control options.
SilentMode (see page 375)	Gets or sets FireDAC global "silent" flag.
R State (see page 375)	Returns the current state of FireDAC manager.
UpdateOptions (see page 375)	The "root" data updating options.
WaitCursor (see page 375)	Gets or sets FireDAC wait cursor shape.
AddConnectionDef (see page 376)	Adds private connection definition.
Close (see page 376)	Terminates FireDAC manager.
CloseConnectionDef (see page 377)	Closes all connections for a specified connection definition name.
DeleteConnectionDef (see page 377)	Deletes connection definition.
DropConnections (see page 377)	Closes all unused connections.
FindConnection (see page 378)	Find the TADCustomConnection (see page 318) instance with ConnectionName equal to AConnectionName.
GetBaseDriverID (see page 378)	Returns ID of a base driver for the specified driver ID.
GetCatalogNames (see page 378)	Populates a string list with the names of catalogs in a DB.
GetConnectionDefNames (see page 379)	Fills the list by existing connection definition names.
GetConnectionDefParams (see page 379)	Fills the list by connection definition parameters.
GetConnectionNames (see page 380)	Fills the list by connection names.
GetDriverNames (see page 380)	Fills the list by registered driver ID's.
GetFieldNames (see page 380)	Populates a string list with the names of fields in a table.
GetGeneratorNames (see page 381)	Populates a string list with the names of generators / sequences / <how else> in a database.
GetKeyFieldNames (see page 382)	Populates a string list with the names of key fields in a table.
GetPackageNames (see page 382)	Populates a string list with the names of the packages in a DB.
GetRDBMSKind (see page 383)	Returns a DBMS kind for the specified driver ID.
GetSchemaNames (see page 383)	Populates a string list with the names of schemas in a DB.
GetStoredProcNames (see page 384)	Populates a string list with the names of stored procedures in a DB.
GetTableNames (see page 385)	Populates a string list with the names of tables in a DB.
IsConnectionDef (see page 385)	Check if name is connection definition name.
LoadConnectionDefFile (see page 386)	Loads connection definitions file.
ModifyConnectionDef (see page 386)	Modifies parameters of an existing connection definition.
Open (see page 386)	Activates FireDAC manager.
RefreshConnectionDefFile (see page 387)	Refreshes connection definition list.
RefreshMetadataCache (see page 387)	Invalidates meta data cache for all connections.
SaveConnectionDefFile (see page 387)	Saves connection definitions into a file.

Class Hierarchy



File

uADCompClient

Description

Use TADCustomManager to manage connection definitions and connection objects.

Consider accessing the singleton instance of TADManager via ADManager (see page 570) function instead of its explicit creating.

This class hides some of properties, allowing to control visibility of properties by descendant classes. End users should use TADManager.

Syntax

```
TADCustomManager = class(TADComponent, IUnknown, IADStanOptions, IADStanObject);
```

See Also

TADManager (see page 429), ADManager (see page 570), IADPhysManager

1.17.1.1.8.1 public

1.17.1.1.8.1.1 TADCustomManager.Active Property

Gets or sets FireDAC manager active status.

Description

Set Active to True to activate FireDAC manager. After that the State (see page 375) = dmsActive. At activating, ADPhysManager loads driver definition file. When required set DriverDefFileAutoLoad (see page 373) and DriverDefFileName (see page 373) before activating manager. TADCustomConnection automatically activates manager before a first connection.

Set Active to False to terminate FireDAC manager. After that the State (see page 375) = dmsInactive. At terminating, manager:

- calls TADCustomConnection (see page 318).Close for all connection objects in application;
- ADPhysManager waits until all connections interfaces (IADPhysConnection) will be released;
- unloads DBMS client DLL's and releases internal driver structures.

Note, to change any driver settings, manager may be active or inactive, but there must be no connections through this driver.

We strongly recommend to not close FireDAC manager from within OnLosted (see page 333), OnRestored (see page 334), and OnRecover (see page 333) events of TADConnection component, as that can lead to unexpected problems.

A multi-threading application should activate FireDAC manager before threads, which are connecting to a DB, will start.

Syntax

```
property Active: Boolean;
```

See Also

DriverDefFileAutoLoad (see page 373), DriverDefFileName (see page 373), Open (see page 386), Close (see page 376)

1.17.1.1.8.1.2 TADCustomManager.ActiveStoredUsage Property

Controls how to use Active (see page 367) property value saved to DFM.

Description

The ActiveStoredUsage property controls how to use Active (see page 367) property value saved to DFM. Include:

- auDesignTime, to use property value at design time;
- auRunTime, to use property value at run time.

Syntax

```
property ActiveStoredUsage: TADStoredActivationUsage;
```

1.17.1.1.8.1.3 TADCustomManager.ActualConnectionDefFileName Property

Returns expanded connection definition file name.

Description

If a connection definition file path is a relative one or contains template markers, then it will be expanded into full path. Use ActualConnectionDefFileName to get the expanded value.

To assign connection definition file path use ConnectionDefFileName property.

Syntax

```
property ActualConnectionDefFileName: String;
```

See Also

ConnectionDefFileName (see page 371)

1.17.1.1.8.1.4 TADCustomManager.ActualDriverDefFileName Property

Returns expanded driver definition file name.

Description

If a driver definition file path is a relative one or contains template markers, then it will be expanded into full path. Use ActualDriverDefFileName to get the expanded value.

To assign driver definition file path use DriverDefFileName property.

Syntax

```
property ActualDriverDefFileName: String;
```

See Also

DriverDefFileName (see page 373)

1.17.1.1.8.1.5 TADCustomManager.AfterLoadConnectionDefFile Event

Fires after connection definitions file is loaded.

Description

The AfterLoadConnectionDefFile event is fired after the connection definitions file is loaded.

Syntax

```
property AfterLoadConnectionDefFile: TNotifyEvent;
```

See Also

BeforeLoadConnectionDefFile (see page 369)

1.17.1.1.8.1.6 TADCustomManager.AfterShutdown Event

Fires after manager is finished.

Description

The AfterShutdown event is fired after FireDAC manager is terminated.

Syntax

```
property AfterShutdown: TNotifyEvent;
```

See Also

Active (see page 367), Close (see page 376), BeforeShutdown (see page 369)

1.17.1.1.8.1.7 TADCustomManager.AfterStartup Event

Fires after manager is started.

Description

The AfterStartup event is fired after FireDAC manager is started.

Syntax

```
property AfterStartup: TNotifyEvent;
```

See Also

Active (see page 367), Open (see page 386), BeforeStartup (see page 370)

1.17.1.1.8.1.8 TADCustomManager.AutoCreated Property

Is manager object auto created.

Description

Returns True, if manager object is created internally by FireDAC.

Syntax

```
property AutoCreated: Boolean;
```

1.17.1.1.8.1.9 TADCustomManager.BeforeLoadConnectionDefFile Event

Fires before connection definitions file is loaded.

Description

The BeforeLoadConnectionDefFile event is fired before the connection definitions file is loaded.

Syntax

```
property BeforeLoadConnectionDefFile: TNotifyEvent;
```

See Also

AfterLoadConnectionDefFile (see page 368)

1.17.1.1.8.1.10 TADCustomManager.BeforeShutdown Event

Fires before manager will be finished.

Description

The BeforeShutdown event is fired before FireDAC manager will be terminated.

Note, multi-threaded application may close connections opened in the background threads in a BeforeShutdown event handler to avoid possible dead lock.

Syntax

```
property BeforeShutdown: TNotifyEvent;
```

See Also

Multi Threading (see page 46), Active (see page 367), Close (see page 376), AfterShutdown (see page 368)

1.17.1.1.8.1.11 TADCustomManager.BeforeStartup Event

Fires before manager will be started.

Description

The BeforeStartup event is fired before FireDAC manager will be started.

Note, application cannot open a connection in a BeforeStartup event handler, as that will lead to the endless recursion.

Syntax

```
property BeforeStartup: TNotifyEvent;
```

See Also

Active (see page 367), Open (see page 386), AfterStartup (see page 369)

1.17.1.1.8.1.12 TADCustomManager.CanRefreshConnectionDefFile Property

Checks if connection definition list may be refreshed.

Description

The CanRefreshConnectionDefFile property returns True, if not one active connection is associated with any of connection definitions in the ConnectionDefs (see page 372) list. In this case ConnectionDefs (see page 372) may be erased or refreshed.

Syntax

```
property CanRefreshConnectionDefFile: Boolean;
```

See Also

ConnectionDefs (see page 372)

Example

```
// reload connection definition list
if ADManager.CanRefreshConnectionDefFile then begin
  ADManager.ConnectionDefs.Clear;
  ADManager.ConnectionDefs.Load;
end;
```

1.17.1.1.8.1.13 TADCustomManager.ConnectionCount Property

The number of connection objects.

Description

The ConnectionCount property returns the number of existing TADCustomConnection (see page 318) objects in the application.

Syntax

```
property ConnectionCount: Integer;
```

See Also

TADCustomConnection (see page 318), Connections (see page 372)

Example

```
// perform some task for all application connection objects
for i := 0 to ADManager.ConnectionCount - 1 do
  ADManager.Connections[i].Ping;
```

1.17.1.1.8.1.14 TADCustomManager.ConnectionDefFileAutoLoad Property

Controls how connection definitions file must be loaded.

Description

The ConnectionDefFileAutoLoad property controls, should be a connection definition file loaded automatically or explicitly by

programmer. If True, then FireDAC will load file at first "touch" to the content of the ConnectionDefs (see page 372) list.

If False, then programmer should call ConnectionDefs (see page 372).Load method before first "touch" to the ConnectionDefs (see page 372) list, including:

- opening a connection referencing to a connection definition by its name (see TADCustomConnection (see page 318).ConnectionDefName (see page 325));
- iterating through existing connection definitions (see TADCustomManager.ConnectionDefs (see page 372));
- adding new connection definition, deleting existing connection definition or looking for existing connection definition (see TADCustomManager.ConnectionDefs (see page 372));

Otherwise an exception will be raised.

Syntax

```
property ConnectionDefFileAutoLoad: Boolean;
```

See Also

Defining Connection (see page 27), ConnectionDefs (see page 372), ConnectionDefFileLoaded (see page 371), ConnectionDefFileName (see page 371), LoadConnectionDefFile (see page 386)

1.17.1.1.8.1.15 TADCustomManager.ConnectionDefFileLoaded Property

Checks is loaded the connection definitions file.

Description

The ConnectionDefFileLoaded property returns True, if connection definitions file is loaded and it content is accessible through the ConnectionDefs (see page 372) list.

Syntax

```
property ConnectionDefFileLoaded: Boolean;
```

See Also

Defining Connection (see page 27), ConnectionDefs (see page 372), ConnectionDefFileautoLoad, ConnectionDefFileName (see page 371)

1.17.1.1.8.1.16 TADCustomManager.ConnectionDefFileName Property

Gets / sets the name of a connection definitions file.

Description

The ConnectionDefFileName property allows to get or set the name of a connection definition file.

The application can set the name before connection definitions file will be loaded. For example in BeforeLoadConnectionDefFile event handler.

If a path is a relative one or contains template markers, then it will be expanded into full path. Use ActualConnectionDefFileName to get the expanded value.

A path may include path variables (see page 163).

Syntax

```
property ConnectionDefFileName: String;
```

See Also

Defining Connection (see page 27), ConnectionDefs (see page 372), ConnectionDefFileAutoLoad (see page 370), ConnectionDefFileLoaded (see page 371), ActualConnectionDefFileName (see page 368)

Example 1

```
ADManager.ConnectionDefFileName := ExtractFilePath(Application.ExeName) + 'myconndef.ini';
ADManager.ConnectionDefFileAutoLoad := True;
oConn := TADConnection.Create(nil);
oConn.ConnectionDefName := 'myconn';
```

```
oConn.Connected := True;
```

Example 2

```
procedure TMyDataModule.DoBeforeLoad(ASender: TNotifyEvent);
begin
  ADManager.ConnectionDefFileName := FSettings.MyConnectionDefFile;
end;
...
ADManager.BeforeLoadConnectionDefFile := DoBeforeLoad;
```

1.17.1.1.8.1.17 TADCustomManager.ConnectionDefs Property

The list of connection definitions.

Description

The ConnectionDefs property returns the list of connection definitions loaded from connection definitions file, if it exists. The list may be manipulated directly, including adding, changing or deleting of connection definitions.

Syntax

```
property ConnectionDefs: IADStanConnectionDefs;
```

See Also

Defining Connection (see page 27), ConnectionDefFileName (see page 371), ConnectionDefFileAutoLoad (see page 370), ConnectionDefFileLoaded (see page 371)

Example 1

```
var
  oDef: IADStanConnectionDef;
...
// find the connection definition, change it properties and save it to file
oDef := ADManager.ConnectionDefs.ConnectionDefByName('MyDef');
oDef.Server := '127.0.0.1';
oDef.Apply;
```

Example 2

```
var
  oDef: IADStanConnectionDef;
...
// add new connection definition and save it to file
oDef := ADManager.ConnectionDefs.AddConnectionDef;
oDef.Name := 'MyDef';
oDef.Server := '127.0.0.1';
oDef.UserName := 'dba';
// use new connection definition
ADConnection1.ConnectionDefName := 'MyDef';
```

Example 3

See FireDAC\Samples\Comp Layer\TADConnection (see page 279)\ConnectionDefs sample for details of how to work with ConnectionDefs.

1.17.1.1.8.1.18 TADCustomManager.Connections Property

The application connection objects.

Description

The Connections property gives access by index to the list of application connection objects.

Syntax

```
property Connections [Index: Integer]: TADCustomConnection;
```

See Also

TADCustomConnection (see page 318), ConnectionCount (see page 370)

Example

```
// perform some task for all application connection objects
for i := 0 to ADManager.ConnectionCount - 1 do
  ADManager.Connections[i].Ping;
```

1.17.1.1.8.1.19 TADCustomManager.DriverDefFileAutoLoad Property

Controls how driver definitions file must be loaded.

Description

The DriverDefFileAutoLoad property controls, should the driver definition file be loaded automatically or explicitly by programmer. When True, then FireDAC will load file at manager activation. When False, then programmer should call ADPhysManager.DriverDefs.Load method before manager activation. Otherwise an exception will be raised.

Syntax

```
property DriverDefFileAutoLoad: Boolean;
```

See Also

Configuring Drivers (see page 31), DriverDefFileName (see page 373)

1.17.1.1.8.1.20 TADCustomManager.DriverDefFileName Property

Gets / sets the name of a driver definitions file.

Description

The DriverDefFileName property allows to get or set the name of a driver definition file. The application can set the name before manager activation.

The application can set the name before driver definitions file will be loaded.

If a path is a relative one or contains template markers, then it will be expanded into full path. Use ActualDriverDefFileName to get the expanded value.

A path may include path variables (see page 163).

Syntax

```
property DriverDefFileName: String;
```

See Also

Configuring Drivers (see page 31), DriverDefFileAutoLoad (see page 373), ActualDriverDefFileName (see page 368)

1.17.1.1.8.1.21 TADCustomManager.DriverDefs Property

The list of driver definitions.

Description

The DriverDefs property returns the list of driver definitions loaded from driver definitions file, if it exists. The list may be manipulated directly, including adding, changing or deleting of driver definitions.

Syntax

```
property DriverDefs: IADStanDefinitions;
```

See Also

Configuring Drivers (see page 31), DriverDefFileName (see page 373), DriverDefFileAutoLoad (see page 373)

Example

```
var
  oDef: IADStanDefinition;
...
// add new virtual driver definition
oDef := ADManager.DriverDefs.Add;
oDef.Name := 'Firebird250';
```

```
oDef.AsStirngs['VendorLib'] := 'c:\Firebird250\Bin\fbclient.dll';

ADConnection1.Params.Clear;
ADConnection1.Params.Add('DriverID=Firebird250');
ADConnection1.Params.Add('Server=127.0.0.1');
....
ADConnection1.Connected := True;
```

1.17.1.1.8.1.22 TADCustomManager.FetchOptions Property

The "root" fetch options.

Description

The FetchOptions property returns the reference to the "root" instance of the fetch options. These options control how the data will be fetched and cached in the memory.

The option values will be inherited by all TADCustomConnection (see page 318)'s and, consequently, by all TADCustomCommand (see page 291)'s and all datasets in application.

See Setting Options (see page 34) for review of how to work with options.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), FormatOptions (see page 374), UpdateOptions (see page 375), ResourceOptions (see page 374)

1.17.1.1.8.1.23 TADCustomManager.FormatOptions Property

The "root" data format options.

Description

The FormatOptions property returns the reference to the "root" instance of the data format options. These options control how the DBMS data types will be mapped into dataset data types.

The option values will be inherited by all TADCustomConnection (see page 318)'s and, consequently, by all TADCustomCommand (see page 291)'s and all datasets in application.

See Setting Options (see page 34) for review of how to work with options.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), FetchOptions (see page 374), UpdateOptions (see page 375), ResourceOptions (see page 374)

1.17.1.1.8.1.24 TADCustomManager.ResourceOptions Property

The "root" resources control options.

Description

The ResourceOptions property returns the reference to the "root" instance of the resources control options. These options control how FireDAC should use DBMS and workstation resources.

The option values will be inherited by all TADCustomConnection (see page 318)'s and, consequently, by all TADCustomCommand (see page 291)'s and all datasets in application.

See Setting Options (see page 34) for review of how to work with options.

Syntax

```
property ResourceOptions: TADTopResourceOptions;
```

See Also

Setting Options (see page 34), FetchOptions (see page 374), UpdateOptions (see page 375), FormatOptions (see page 374)

1.17.1.1.8.1.25 TADCustomManager.SilentMode Property

Gets or sets FireDAC global "silent" flag.

Description

Set SilentMode to True to disable wait cursors and dialogs. This flag is application wide. Its value overrides all TADResourceOptions (see page 874).SilentMode (see page 884) values.

Syntax

```
property SilentMode: Boolean;
```

See Also

TADResourceOptions.SilentMode, WaitCursor (see page 375)

1.17.1.1.8.1.26 TADCustomManager.State Property

Returns the current state of FireDAC manager.

Description

The State property returns the current state of FireDAC manager.

Syntax

```
property State: TADPhysManagerState;
```

See Also

Active (see page 367)

1.17.1.1.8.1.27 TADCustomManager.UpdateOptions Property

The "root" data updating options.

Description

The UpdateOptions property returns the reference to the "root" instance of the data updating options. These options control how FireDAC should post data changes back to a DB.

The option values will be inherited by all TADCustomConnection's and, consequently, by all TADCustomTableAdapter's and all datasets in application.

See Setting Options (see page 34) for review of how to work with options.

Syntax

```
property UpdateOptions: TADUpdateOptions;
```

See Also

Setting Options (see page 34), FetchOptions (see page 374), ResourceOptions (see page 374), FormatOptions (see page 374)

1.17.1.1.8.1.28 TADCustomManager.WaitCursor Property

Gets or sets FireDAC wait cursor shape.

Description

The WaitCursor property gets or sets the wait cursor shape.

Syntax

```
property WaitCursor: TADGUIxScreenCursor;
```

See Also

SilentMode (see page 375)

1.17.1.1.8.1.29 TADCustomManager.AddConnectionDef Method

Adds private connection definition.

Parameters

Parameters	Description
const AName: string	A connection definition name.
const ADriver: string	A driver ID.
AList: TStrings	A list of parameter name=value pairs.
APersistent: Boolean = False	A persistence flag. By default is False.

Description

The AddConnectionDef method adds new private or persistent connection definition to the ConnectionDefs (see page 372) list.

The connection definition will get name AName, DriverID ADriver and parameters AList. The name must be unique across other connection definitions in the ConnectionDefs (see page 372) list, otherwise exception will be raised.

Set APersistent to True, to mark connection definition as persistent, otherwise it will be private one. After the call a persistent connection definition is not stored to a file. Call SaveConnectionDefFile (see page 387) to save this and other changes to a file. Not stored connection definitions will be discarded after FireDAC manager or application will be terminated.

After connection definition is added, it may be referenced by TADCustomConnection (see page 318).ConnectionDefName (see page 325).

Syntax

```
procedure AddConnectionDef(const AName: string; const ADriver: string; AList: TStrings;
APersistent: Boolean = False);
```

See Also

Defining Connection (see page 27), ConnectionDefs (see page 372), SaveConnectionDefFile (see page 387), DeleteConnectionDef (see page 377), ModifyConnectionDef (see page 386), IADStanConnectionDefs.AddConnectionDef, TADCustomConnection.ConnectionDefName

Example

```
var
  oList: TStringList;
......
  oList := TStringList.Create;
  oList.Add('Server=127.0.0.1');
  oList.Add('Database=addemo');
  ADManager.AddConnectionDef('myconn', 'MySQL', oList);
......
  ADConnection1.ConnectionDefName := 'myconn';
  ADConnection1.Connected := True;
```

1.17.1.1.8.1.30 TADCustomManager.Close Method

Terminates FireDAC manager.

Description

The Close method terminates manager, by setting Active (see page 367) to False.

Syntax

```
procedure Close;
```

See Also

Open (see page 386), Active (see page 367), CloseConnectionDef (see page 377)

1.17.1.1.8.1.31 TADCustomManager.CloseConnectionDef Method

Closes all connections for a specified connection definition name.

Parameters

Parameters	Description
const AName: string	A connection definition name.

Description

Use CloseConnectionDef method to close all connections opened using specified connection definition and to close all pooled connection for this connection definition. After CloseConnectionDef call the application may modify the connection definition parameters.

CloseConnectionDef forcibly closes all opened connections and waits until all of them will be closed. If a connection is kept by other thread and cannot be closed (eg Abort in BeforeDisconnect), then CloseConnectionDef will hang up.

If none connections were made for the connection definition or even connection definition does not exist, then no actions will be taken.

Syntax

```
procedure CloseConnectionDef(const AName: string);
```

See Also

Close (see page 376)

1.17.1.1.8.1.32 TADCustomManager.DeleteConnectionDef Method

Deletes connection definition.

Parameters

Parameters	Description
const AName: string	A connection definition name.

Description

The DeleteConnectionDef method deletes connection definition by its name from the ConnectionDefs (see page 372) list. If specified name is not found, then exception will be raised.

After the call a persistent connection definition is not deleted from a file. Call SaveConnectionDefFile (see page 387) to delete it and save other changes to a file.

Syntax

```
procedure DeleteConnectionDef(const AName: string);
```

See Also

Defining Connection (see page 27), ConnectionDefs (see page 372), SaveConnectionDefFile (see page 387), AddConnectionDef (see page 376), IADStanDefinition.Delete

Example

```
ADManager.DeleteConnectionDef('myconn');
```

1.17.1.1.8.1.33 TADCustomManager.DropConnections Method

Closes all unused connections.

Description

Call DropConnection method to close all unused connections and delete temporary ones. The connection is not used if it

RefCount is equal to 0.

Syntax

```
procedure DropConnections;
```

See Also

TADCustomConnection.RefCount, TADCustomConnection.Temporary, Connections (see page 372), ConnectionCount (see page 370)

1.17.1.1.8.1.34 TADCustomManager.FindConnection Method

Find the TADCustomConnection (see page 318) instance with ConnectionName equal to AConnectionName.

Parameters

Parameters	Description
const AConnectionName: string	The connection name

Returns

An TADCustomConnection (see page 318) instance, if found. Nil - otherwise.

Description

The FindConnection method finds the TADCustomConnection (see page 318) instance with ConnectionName (see page 326) equal to AConnectionName.

Syntax

```
function FindConnection(const AConnectionName: string): TADCustomConnection;
```

See Also

GetConnectionNames (see page 380), TADCustomConnection.ConnectionName, TADCustomCommand.ConnectionName, TADRdbmsDataSet.ConnectionName

1.17.1.1.8.1.35 TADCustomManager.GetBaseDriverID Method

Returns ID of a base driver for the specified driver ID.

Parameters

Parameters	Description
const ADriverID: String	A driver ID.

Description

Use GetBaseDriverID to get a base driver ID for the specified driver ID. Where the specified driver ID may be base or virtual driver ID's.

Syntax

```
function GetBaseDriverID(const ADriverID: String): String;
```

Example

```
var
  sBaseID: String;
......
  sBaseID := ADManager.GetBaseDriverID('MSSQLExpress');
  if SameText(sBaseID, S_AD_MSSQLId) then
    ShowMessage('It is a MSSQL base driver');
```

1.17.1.1.8.1.36 TADCustomManager.GetCatalogNames Method

Populates a string list with the names of catalogs in a DB.

Parameters

Parameters	Description
const AConnectionName: string	A connection name.
const APattern: string	A LIKE pattern.
AList: TStrings	A list to fill.

Description

Call GetCatalogNames to retrieve a list of catalogs in a DB.

AConnectionName is the connection name to use.

APattern is the LIKE-pattern filtering catalog names.

AList is a TStrings descendant that receives the catalog names. Any existing strings are deleted from the list before GetCatalogNames adds the names of all the catalogs in a DB.

Syntax

```
procedure GetCatalogNames(const AConnectionName: string; const APattern: string; AList:
TStrings);
```

See Also

Querying Metadata (see page 127), TADMetaInfoQuery (see page 461), TADCustomConnection.GetCatalogNames

Example

```
ADManager.GetCatalogNames('myconn', '', ListBox1.Items);
```

1.17.1.1.8.1.37 TADCustomManager.GetConnectionDefNames Method

Fills the list by existing connection definition names.

Parameters

Parameters	Description
AList: TStrings	A list to fill.

Description

The GetConnectionDefNames method fills an AList list by the names of connection definitions in the ConnectionDefs list. A list will be sorted alphabetically.

Syntax

```
procedure GetConnectionDefNames(AList: TStrings);
```

See Also

IADStanDefinition.Name

Example 1

```
ADManager.GetConnectionDefNames(ListBox1.Items);
```

Example 2

See FireDAC\Samples\Comp Layer\TADConnection (see page 279)\ConnectionDefs (see page 372) sample.

1.17.1.1.8.1.38 TADCustomManager.GetConnectionDefParams Method

Fills the list by connection definition parameters.

Parameters

Parameters	Description
const AName: string	A connection definition name.
AList: TStrings	A list to fill.

Description

The GetConnectionDefParams method fills an AList list by the name=value pairs of AName connection definition in the ConnectionDefs list.

Syntax

```
procedure GetConnectionDefParams(const AName: string; AList: TStrings);
```

See Also

IADStanDefinition.Params

Example

```
ADManager.GetConnectionDefParams('myconn', ListBox1.Items);
```

1.17.1.1.8.1.39 TADCustomManager.GetConnectionNames Method

Fills the list by connection names.

Parameters

Parameters	Description
AList: TStrings	A list to fill.

Description

The GetConnectionNames method fills an AList list by the connection names. A list will be sorted alphabetically.

Syntax

```
procedure GetConnectionNames(AList: TStrings);
```

See Also

FindConnection (see page 378), TADCustomConnection.ConnectionName, TADCustomCommand.ConnectionName, TADRdbmsDataSet.ConnectionName

1.17.1.1.8.1.40 TADCustomManager.GetDriverNames Method

Fills the list by registered driver ID's.

Parameters

Parameters	Description
AList: TStrings	A list to fill.
AValidate: Boolean = False	Check that some driver and a DBMS CLI library are really accessible. Default is False.
ABaseOnly: Boolean = False	Include only FireDAC base driver ID's. Default is False.

Description

The GetDriverNames method fills an AList list by the registered driver ID's. A list will be sorted alphabetically.

Syntax

```
procedure GetDriverNames(AList: TStrings; AValidate: Boolean = False; ABaseOnly: Boolean =
False);
```

See Also

Querying Metadata (see page 127), IADStanConnectionDef.DriverID

Example

```
ADManager.GetDriverNames(ListBox1.Items);
```

1.17.1.1.8.1.41 TADCustomManager.GetFieldNames Method

Populates a string list with the names of fields in a table.

Parameters

Parameters	Description
const AConnectionName: string	A connection name.
const ACatalogName: string	A catalog name.
const ASchemaName: string	A schema name.
const ATableName: string	A table name.
const APattern: string	A LIKE patten.
AList: TStrings	A list to fill.

Description

Call GetFieldNames to retrieve a list of fields in a table.

AConnectionName is the connection name to use.

ACatalogName, ASchemaName, ATableName names the table whose field names you want added to the list.

APattern is the LIKE-pattern filtering field names.

AList is a TStrings descendant that receives the field names. Any existing strings are deleted from the list before GetFieldNames adds the names of all the fields in ATableName.

The field names are normalized - enclosed into quotes, if that is required, otherwise converted to default dictionary case.

Syntax

```
procedure GetFieldNames(const AConnectionName: string; const ACatalogName: string; const
ASchemaName: string; const ATableName: string; const APattern: string; AList: TStrings);
```

See Also

Querying Metadata (see page 127), TADMetaInfoQuery (see page 461), TADCustomConnection.GetFieldNames

Example

```
ADManager.GetFieldNames('myconn', '', '', 'MYTAB', '', ListBox1.Items);
```

1.17.1.1.8.1.42 TADCustomManager.GetGeneratorNames Method

Populates a string list with the names of generators / sequences / <how else> in a database.

Parameters

Parameters	Description
const AConnectionName: string	A connection name.
const ACatalogName: string	A catalog name.
const ASchemaName: string	A schema name.
const APattern: string	A LIKE pattern.
AList: TStrings	A list to fill.
AScopes: TADPhysObjectScopes = [osMy]	A object scopes.

Description

Call GetGeneratorNames to retrieve a list of generators / sequences / <how else> in a database.

AConnectionName is the connection name to use.

ACatalogName and ASchemaName restrict generator names to the catalog and schema.

APattern is the LIKE-pattern filtering generator names.

AList is a TStrings descendant that receives the generator names. Any existing strings are deleted from the list before GetFieldNames adds the names of generators.

AScopes restricts generator names to the specified scopes.

The generator names are normalized - enclosed into quotes, if that is required, otherwise converted to default dictionary

case. Specify the MetaDefSchema and/or MetaDefCatalog parameters in connection definition, to avoid these schema and/or catalog names in generator names.

Syntax

```
procedure GetGeneratorNames(const AConnectionName: string; const ACatalogName: string;
const ASchemaName: string; const APattern: string; AList: TStrings; AScopes:
TADPhysObjectScopes = [osMy]; AFullName: Boolean = True);
```

See Also

Querying Metadata (see page 127), TADMetaInfoQuery (see page 461), TADCustomConnection.GetGeneratorNames

Example

```
ADManager.GetGeneratorNames('myconn', '', '', 'MY%', ListBox1.Items, [osMy, osSystem]);
```

1.17.1.1.8.1.43 TADCustomManager.GetKeyFieldNames Method

Populates a string list with the names of key fields in a table.

Parameters

Parameters	Description
const AConnectionName: string	A connection name.
const ACatalogName: string	A catalog name.
const ASchemaName: string	A schema name.
const ATableName: string	A table name.
const APattern: string	A LIKE pattern.
AList: TStrings	A list to fill.

Description

Call GetKeyFieldNames to retrieve a list of primary key fields in a table.

AConnectionName is the connection name to use.

ACatalogName, ASchemaName, ATableName names the table whose primary field names you want added to the list.

APattern is the LIKE-pattern filtering field names.

AList is a TStrings descendant that receives the primary key field names. Any existing strings are deleted from the list before GetKeyFieldNames adds the names of the primary key fields in ATableName.

The field names are normalized - enclosed into quotes, if that is required, otherwise converted to default dictionary case.

Syntax

```
procedure GetKeyFieldNames(const AConnectionName: string; const ACatalogName: string; const
ASchemaName: string; const ATableName: string; const APattern: string; AList: TStrings);
```

See Also

Querying Metadata (see page 127), TADMetaInfoQuery (see page 461), TADCustomConnection.GetKeyFieldNames

Example

```
ADManager.GetKeyFieldNames('myconn', 'NORTHWIND', 'DBO', 'MYTAB', '', ListBox1.Items);
```

1.17.1.1.8.1.44 TADCustomManager.GetPackageNames Method

Populates a string list with the names of the packages in a DB.

Parameters

Parameters	Description
const AConnectionName: string	A connection name.
const ACatalogName: string	A catalog name.
const ASchemaName: string	A schema name.
const APattern: string	A LIKE pattern.

AList: TStrings	A list to fill.
AScopes: TADPhysObjectScopes = [osMy]	A object scopes.

Description

Call GetPackageNames to retrieve a list of packages in a DB.

AConnectionName is the connection name to use.

ACatalogName, ASchemaName restrict package names to the catalog and schema.

APattern is the LIKE-pattern filtering package names.

AList is a TStrings descendant that receives the package names. Any existing strings are deleted from the list before GetPackageNames adds the names of all the packages in a DB.

AScopes restricts package names to the specified scopes.

The generator names are normalized - enclosed into quotes, if that is required, otherwise converted to default dictionary case. Specify the MetaDefSchema and/or MetaDefCatalog parameters in connection definition, to avoid these schema and/or catalog names in package names.

Syntax

```
procedure GetPackageNames(const AConnectionName: string; const ACatalogName: string; const ASchemaName: string; const APattern: string; AList: TStrings; AScopes: TADPhysObjectScopes = [osMy]; AFullName: Boolean = True);
```

See Also

Querying Metadata (see page 127), TADMetaInfoQuery (see page 461), TADCustomConnection.GetPackageNames

Example

```
ADManager.GetPackageNames('myconn', '', 'SYS', 'DBMS%', ListBox1.Items, [osMy, osSystem]);
```

1.17.1.1.8.1.45 TADCustomManager.GetRDBMSKind Method

Returns a DBMS kind for the specified driver ID.

Parameters

Parameters	Description
const ADriverID: String	The driver ID.

Description

Use the GetRDBMSKind method to get a DBMS kind for the specified driver ID. The driver ID may be an id of base or virtual driver. If the driver ID is unknown, then mkUnknown is returned.

Syntax

```
function GetRDBMSKind(const ADriverID: String): TADRDBMSKind;
```

See Also

GetBaseDriverID (see page 378)

1.17.1.1.8.1.46 TADCustomManager.GetSchemaNames Method

Populates a string list with the names of schemas in a DB.

Parameters

Parameters	Description
const AConnectionName: string	A connection name.
const ACatalogName: string	A catalog name.
const APattern: string	A LIKE pattern.
AList: TStrings	A list to fill.

Description

Call GetSchemaNames to retrieve a list of schemas in a DB.

AConnectionName is the connection name to use.

ACatalogName restricts schema names to the specified catalog. If it is empty, then all schemas will be returned.

APattern is the LIKE-pattern filtering schema names.

AList is a TStrings descendant that receives the schema names. Any existing strings are deleted from the list before GetSchemaNames adds the names of all the schemas in a DB.

Syntax

```
procedure GetSchemaNames(const AConnectionName: string; const ACatalogName: string; const
APattern: string; AList: TStrings);
```

See Also

Querying Metadata (see page 127), TADMetaInfoQuery (see page 461), TADCustomConnection.GetSchemaNames

Example

```
ADManager.GetSchemaNames('myconn', '', '', ListBox1.Items);
```

1.17.1.1.8.1.47 TADCustomManager.GetStoredProcNames Method

Populates a string list with the names of stored procedures in a DB.

Parameters

Parameters	Description
const AConnectionName: string	A connection name.
const ACatalogName: string	A catalog name.
const ASchemaName: string	A schema name.
const APackage: string	A package name.
const APattern: string	A LIKE pattern.
AList: TStrings	A list to fill.
AScopes: TADPhysObjectScopes = [osMy]	A object scopes.

Description

Call GetStoredProcNames to retrieve a list of stored procedures and functions in a DB.

AConnectionName is the connection name to use.

ACatalogName, ASchemaName restrict procedure names to the catalog and schema.

APackage restricts procedure names to the package.

APattern is the LIKE-pattern filtering procedure names.

AList is a TStrings descendant that receives the procedure names. Any existing strings are deleted from the list before GetStoredProcNames adds the names of all the procedures in a DB.

AScopes restricts stored procedure names to the specified scopes. Applies only to non-packaged procedures.

The stored procedure names are normalized - enclosed into quotes, if that is required, otherwise converted to default dictionary case. Specify the MetaDefSchema and/or MetaDefCatalog parameters in connection definition, to avoid these schema and/or catalog names in procedure names.

Syntax

```
procedure GetStoredProcNames(const AConnectionName: string; const ACatalogName: string;
const ASchemaName: string; const APackage: string; const APattern: string; AList: TStrings;
AScopes: TADPhysObjectScopes = [osMy]; AFullName: Boolean = True);
```

See Also

Querying Metadata (see page 127), TADMetaInfoQuery (see page 461), TADCustomConnection.GetStoredProcNames

Example

```
ADManager.GetStoredProcNames('myconn', '', '', '', '', ListBox1.Items, [osMy, osSystem]);
```

1.17.1.1.8.1.48 TADCustomManager.GetTableNames Method

Populates a string list with the names of tables in a DB.

Parameters

Parameters	Description
const AConnectionName: string	A connection name.
const ACatalogName: string	A catalog name.
const ASchemaName: string	A schema name.
const APattern: string	A LIKE pattern.
AList: TStrings	A list to fill.
AScopes: TADPhysObjectScopes = [osMy]	A object scopes.

Description

Call GetTableNames to retrieve a list of tables in a DB.

AConnectionName is the connection name to use.

ACatalogName, ASchemaName restrict table names to the catalog and schema.

APattern is the LIKE-pattern filtering table names.

AList is a TStrings descendant that receives the table names. Any existing strings are deleted from the list before GetTableNames adds the names of all the tables in a DB.

AScopes restricts table names to the specified scopes.

AKinds restricts tables to the specified object kinds.

The table names are normalized - enclosed into quotes, if that is required, otherwise converted to default dictionary case. Specify the MetaDefSchema and/or MetaDefCatalog parameters in connection definition, to avoid these schema and/or catalog names in table names.

Syntax

```
procedure GetTableNames(const AConnectionName: string; const ACatalogName: string; const
ASchemaName: string; const APattern: string; AList: TStrings; AScopes: TADPhysObjectScopes
= [osMy]; AFullName: Boolean = True);
```

See Also

Querying Metadata (see page 127), TADMetaInfoQuery (see page 461), TADCustomConnection.GetTableNames

Example

```
ADManager.GetTableNames('myconn', 'NORTHWIND', 'DBO', 'ADQA%', '', ListBox1.Items,
  [osMy, osSystem, osOther], [tkTable, tkView]);
```

1.17.1.1.8.1.49 TADCustomManager.IsConnectionDef Method

Check if name is connection definition name.

Parameters

Parameters	Description
const AName: String	A name to check.

Returns

True, if the AName is connection definition name. False - otherwise.

Description

The IsConnectionDef method checks if AName value is the name of existing connection definition in ConnectionDefs list.

Syntax

```
function IsConnectionDef(const AName: String): Boolean;
```

1.17.1.1.8.1.50 TADCustomManager.LoadConnectionDefFile Method

Loads connection definitions file.

Description

The LoadConnectionDefFile method explicitly loads a connection definition file, specified by ConnectionDefFileName. Use this method, if you set ConnectionDefFileAutoLoad to False.

Syntax

```
procedure LoadConnectionDefFile;
```

See Also

ConnectionDefs (see page 372), ConnectionDefFileAutoLoad (see page 370), ConnectionDefFileLoaded (see page 371), ConnectionDefFileName (see page 371)

1.17.1.1.8.1.51 TADCustomManager.ModifyConnectionDef Method

Modifies parameters of an existing connection definition.

Parameters

Parameters	Description
const AName: string	A connection definition name to modify.
AList: TStrings	A list of name=value pairs.

Description

The ModifyConnectionDef method modifies parameters of an existing connection definition in the ConnectionDefs (see page 372) list using AList parameter values.

After the call a persistent connection definition is not stored to a file. Call SaveConnectionDefFile (see page 387) to save this and other changes to a file. Not stored connection definition changes will be discarded after FireDAC manager or application will be terminated.

If connection definition with AName name is not found, then exception will be raised.

Syntax

```
procedure ModifyConnectionDef(const AName: string; AList: TStrings);
```

See Also

Defining Connection (see page 27), ConnectionDefs (see page 372), SaveConnectionDefFile (see page 387), DeleteConnectionDef (see page 377), AddConnectionDef (see page 376)

Example

```
var
  oList: TStringList;
......
  oList := TStringList.Create;
  oList.Add('Server=myhoster');
  ADManager.ModifyConnectionDef('myconn', oList);
  ADManager.SaveConnectionDefFile;
```

1.17.1.1.8.1.52 TADCustomManager.Open Method

Activates FireDAC manager.

Description

The Open method activates manager, by setting Active (see page 367) to True.

Syntax

```
procedure Open;
```

See Also

Close (see page 376), Active (see page 367)

1.17.1.1.8.1.53 TADCustomManager.RefreshConnectionDefFile Method

Refreshes connection definition list.

Description

The RefreshConnectionDefFile method discards all changes in the ConnectionDefs (see page 372) list and loads connection definition file. The list may be refreshed, if there is no active connections using connection definitions from the list.

Syntax

```
procedure RefreshConnectionDefFile;
```

See Also

CanRefreshConnectionDefFile (see page 370), ConnectionDefs (see page 372)

Example

```
if ADManager.CanRefreshConnectionDefFile then
  ADManager.RefreshConnectionDefFile;
```

1.17.1.1.8.1.54 TADCustomManager.RefreshMetadataCache Method

Invalidates meta data cache for all connections.

Description

The RefreshMetadataCache method discards all cached metadata, like a stored procedure definitions and table primary keys for all TADCustomConnection (see page 318) instances. Call this method, if the DB metadata has been changed, so the cached metadata is invalid. The metadata is cached by datasets, if fiMeta is included into FetchOptions.Cache (see page 852).

Syntax

```
procedure RefreshMetadataCache;
```

See Also

Querying Metadata (see page 127), TADFetchOptions.Cache, TADCustomConnection.RefreshMetadataCache

1.17.1.1.8.1.55 TADCustomManager.SaveConnectionDefFile Method

Saves connection definitions into a file.

Description

The SaveConnectionDefFile method saves all changes in the ConnectionDefs (see page 372) list to the file specified by ConnectionDefFileName (see page 371).

Syntax

```
procedure SaveConnectionDefFile;
```

See Also

ConnectionDefs (see page 372), DeleteConnectionDef (see page 377), AddConnectionDef (see page 376), ModifyConnectionDef (see page 386)

Example

```
ADManager.DeleteConnectionDef('myconn');
ADManager.SaveConnectionDefFile;
```

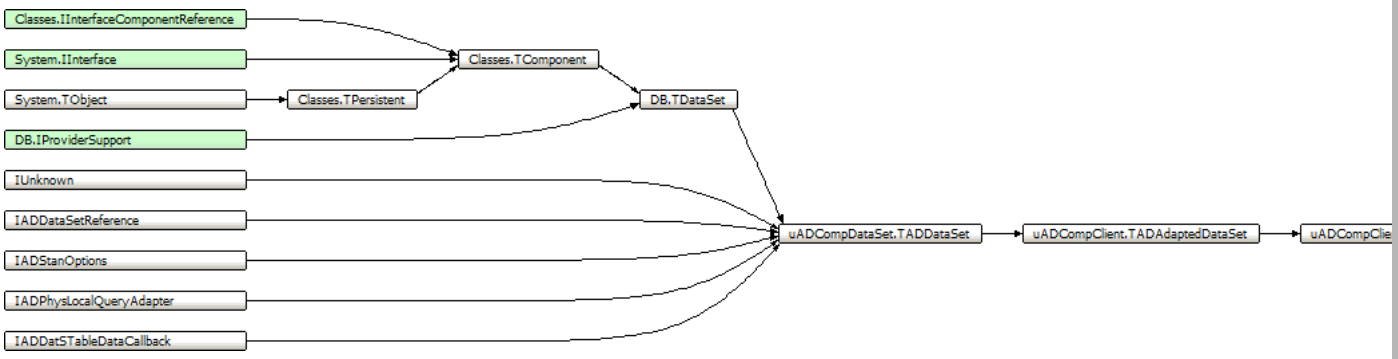
1.17.1.1.9 TADCustomMemTable Class

The class implementing in-memory dataset.

public

public	Description
Adapter (see page 389)	Add a short summary here...
CommandText (see page 389)	Shortcut for Adapter.SelectCommand.CommandText.Text.
DisableStringTrim (see page 389)	Shortcut for FormatOptions.StrsTrim.
FetchOnDemand (see page 390)	Shortcut for FormatOptions.Mode.
FileName (see page 390)	Shortcut for ResourceOptions.PersistentFileName.
IsClone (see page 390)	Returns True, if this dataset is a cloned dataset.
LogChanges (see page 390)	Shortcut for CachedUpdates.
PacketRecords (see page 391)	Shortcut for FetchOptions.RowsetSize.
ProviderEOF (see page 391)	Shortcut for SourceEOF.
ReadOnly (see page 391)	Shortcut for UpdateOptions.ReadOnly.
StatusFilter (see page 391)	Shortcut for FilterChanges.
XMLData (see page 392)	Represents dataset content as an XML.
AppendData (see page 392)	Appends data to this dataset from other dataset.
ConstraintsDisabled (see page 393)	Shortcut for ConstraintsEnabled.
GetOptionalParam (see page 393)	Returns an optional custom parameter, associated with a dataset.
MergeChangeLog (see page 393)	Shortcut for CommitUpdates.
SetOptionalParam (see page 394)	Sets an optional custom parameter, associated with a dataset.

Class Hierarchy



File

uADCompClient

Description

Use TADCustomMemTable to manage data in the client memory and optionally exchange the data with a DBMS.

In general, TADCustomMemTable is an in-memory dataset. It may be populated directly by the code at run time without connection to a database. For example:

```
ADMemTable1.FieldDefs.Add('id', ftInteger);
ADMemTable1.FieldDefs.Add('name', ftString, 20);
ADMemTable1.Open;
ADMemTable1.AppendRecord([1, 'MySQL']);
ADMemTable1.AppendRecord([2, 'SQLite']);
```

And TADCustomMemTable may be populated from a database using TADTableAdapter and TADCommand (see page 267) components. That may be considered as an advanced technique, because TADQuery (see page 476) may be used for most DB data exchange operations.

This class hides some of properties, allowing to control visibility of properties by descendant classes. End users should use TADMemTable (see page 436).

Syntax

```
TADCustomMemTable = class(TADAdaptedDataSet);
```

See Also

TADMemTable (see page 436), TADDatSTable, TADCustomTableAdapter

Example

See FireDAC\Samples\Comp Layer\TADMemTable (see page 436)\Main sample for details of how to setup TADMemTable (see page 436).

See FireDAC\Samples\Comp Layer\TADMemTable (see page 436)\MasterDetail sample for details of how to work with master-detail relation.

1.17.1.1.9.1 public

1.17.1.1.9.1.1 TADCustomMemTable.Adapter Property

Add a short summary here...

Description

Add a description here...

Syntax

```
property Adapter: TADCustomTableAdapter;
```

See Also

List here...

Example

1.17.1.1.9.1.2 TADCustomMemTable.CommandText Property

Shortcut for Adapter.SelectCommand.CommandText.Text.

Description

The CommandText is a shortcut for the Adapter (see page 389).SelectCommand.CommandText (see page 298).Text property. And allows to set / get a command text more quickly.

The property is for the TClientDataSet compatibility.

Syntax

```
property CommandText: TADWideString;
```

See Also

Adapter (see page 389), TADCustomCommand.CommandText

1.17.1.1.9.1.3 TADCustomMemTable.DisableStringTrim Property

Shortcut for FormatOptions.StrsTrim.

Description

The DisableStringTrim is a shortcut for the "not FormatOptions (see page 651).StrsTrim (see page 869)". And allows to set / get a CHAR fields trimming mode.

The property is for the TClientDataSet compatibility.

Syntax

```
property DisableStringTrim: Boolean;
```

See Also

FormatOptions, TADFormatOptions.StrsTrim

1.17.1.1.9.1.4 TADCustomMemTable.FetchOnDemand Property

Shortcut for FormatOptions.Mode.

Description

The FetchOnDemand is a shortcut for the FetchOptions (see page 651).Mode (see page 857) = fmAll / fmOnDemand. And allows to set / get resultset fetching mode.

The property is for the TClientDataSet compatibility.

Syntax

```
property FetchOnDemand: Boolean;
```

See Also

FetchOptions, TADFetchOptions.Mode (see page 857)

1.17.1.1.9.1.5 TADCustomMemTable.FileName Property

Shortcut for ResourceOptions.PersistentFileName.

Description

The FileName is a shortcut for the ResourceOptions (see page 652).PersistentFileName (see page 843) property. And allows to set / get persistent file name to store dataset content.

The property is for the TClientDataSet compatibility.

Syntax

```
property FileName: string;
```

See Also

ResourceOptions, TADBottomResourceOptions.PersistentFileName (see page 843)

1.17.1.1.9.1.6 TADCustomMemTable.IsClone Property

Returns True, if this dataset is a cloned dataset.

Description

Read the IsClone property to check is this dataset a cloned one. Alternatively the application may read the CloneSource (see page 593) property. If it is not nil, then it is referencing to the original dataset.

The property is for the TClientDataSet compatibility.

Syntax

```
property IsClone: Boolean;
```

See Also

CloneSource, TADDataSet.CloneCursor

1.17.1.1.9.1.7 TADCustomMemTable.LogChanges Property

Shortcut for CachedUpdates.

Description

The LogChanges is a shortcut for the CachedUpdates (see page 592) property. And allows to set / get the dataset

updates caching mode.

The property is for the TClientDataSet compatibility.

Syntax

```
property LogChanges: Boolean;
```

See Also

CachedUpdates

1.17.1.1.9.1.8 TADCustomMemTable.PacketRecords Property

Shortcut for FetchOptions.RowsetSize.

Description

The PacketRecords is a shortcut for the FetchOptions (see page 651).RowsetSize (see page 859) property. And allows to set / get the dataset rowset size.

The property is for the TClientDataSet compatibility.

Syntax

```
property PacketRecords: Integer;
```

See Also

FetchOptions, TADFetchOptions.RowsetSize (see page 859)

1.17.1.1.9.1.9 TADCustomMemTable.ProviderEOF Property

Shortcut for SourceEOF.

Description

The ProviderEOF is a shortcut for the TADDataSet.SourceEOF (see page 608) property. And allows to set / get the sign that all rows are fetched from a DB.

The property is for the TClientDataSet compatibility.

Syntax

```
property ProviderEOF: Boolean;
```

See Also

TADDataSet.SourceEOF (see page 608)

1.17.1.1.9.1.10 TADCustomMemTable.ReadOnly Property

Shortcut for UpdateOptions.ReadOnly.

Description

The ReadOnly is a shortcut for the UpdateOptions (see page 653).ReadOnly (see page 903) property. And allows to control the dataset read-write or read-only mode.

The property is for the TClientDataSet compatibility.

Syntax

```
property ReadOnly: Boolean;
```

See Also

UpdateOptions, TADUpdateOptions.ReadOnly (see page 903)

1.17.1.1.9.1.11 TADCustomMemTable.StatusFilter Property

Shortcut for FilterChanges.

Description

The StatusFilter is a shortcut for the FilterChanges (see page 597) property. And allows to set / get the dataset changes filter.

The property is for the TClientDataSet compatibility.

Syntax

```
property StatusFilter: TADUpdateRecordTypes;
```

See Also

FilterChanges

1.17.1.1.9.1.12 TADCustomMemTable.XMLData Property

Represents dataset content as an XML.

Description

The property XMLData allows to exchange data between dataset and external systems. The property is a shortcut for LoadFromStream (see page 633) / SaveToStream (see page 645) methods.

The data is represented as an Unicode string, containing XML encoded data. Read property to get the data. Set property to populate dataset structure and fill in by rows. After setting the property the dataset will remain open.

The property is for the TClientDataSet compatibility.

Syntax

```
property XMLData: string;
```

See Also

LoadFromStream, SaveToStream

Example

```
var
  s: WideString;
....
  s := ADMemTable1.XMLData;
....
  ADMemTable2.XMLData := s;
```

1.17.1.1.9.1.13 TADCustomMemTable.AppendData Method

Appends data to this dataset from other dataset.

Parameters

Parameters	Description
const AData: IADDataSetReference	The FireDAC dataset reference.
AHitEOF: Boolean = True	Set to True to set SourceEOF to True.

Description

Use the AppendData method to append the rows from one FireDAC dataset to this dataset. The property is a shortcut for Data property and CopyDataSet (see page 615) method.

When this dataset is inactive, then it will inherit structure from AData, activated and rows will be appended. When this dataset is active, then rows will be imported and only compatible fields will be filled.

The property is for the TClientDataSet compatibility

Syntax

```
procedure AppendData(const AData: IADDataSetReference; AHitEOF: Boolean = True);
```

See Also

Data, CopyDataSet

Example

```
ADQuery1.Connection := ADConnection1;
ADQuery1.Open('select * from [Orders]');
ADMemTable1.AppendData(ADQuery1);

ADQuery2.Connection := ADConnection2;
ADQuery2.Open('select * from "Orders"');
ADMemTable1.AppendData(ADQuery2);
```

1.17.1.1.9.1.14 TADCustomMemTable.ConstraintsDisabled Method

Shortcut for ConstraintsEnabled.

Description

The ConstraintsDisabled is a shortcut for the "not ConstraintsEnabled (see page 594)".

The property is for the TClientDataSet compatibility.

Syntax

```
function ConstraintsDisabled: Boolean;
```

See Also

ConstraintsEnabled

1.17.1.1.9.1.15 TADCustomMemTable.GetOptionalParam Method

Returns an optional custom parameter, associated with a dataset.

Parameters

Parameters	Description
const AParamName: string	The parameter name.

Description

Use GetOptionalParam method to get a custom parameter value, associated with dataset.

The parameter value may be set using SetOptionalParam. The optional parameters are part of internal dataset storage and are saved / loaded with dataset at SaveToStream (see page 645) / LoadFromStream (see page 633) calls. FireDAC does not interpret these parameters in any way.

The method is for the TClientDataSet compatibility.

Syntax

```
function GetOptionalParam(const AParamName: string): Variant;
```

See Also

SetOptionalParam (see page 394)

Example

```
var
  dt: Variant;
...
dt := ADMemTable1.GetOptionalParam('last_modified_date');
if VarIsNull(dt) then
  ShowMessage('No modifications were done')
else
  ShowMessage('Last modification was done at ' + DateToStr(dt));
```

1.17.1.1.9.1.16 TADCustomMemTable.MergeChangeLog Method

Shortcut for CommitUpdates.

Description

The MergeChangeLog is a shortcut for the CommitUpdates (see page 615) method.

The property is for the TClientDataSet compatibility.

Syntax

```
procedure MergeChangeLog;
```

See Also

CommitUpdates

1.17.1.1.9.1.17 TADCustomMemTable.SetOptionalParam Method

Sets an optional custom parameter, associated with a dataset.

Parameters

Parameters	Description
const AParamName: string	The parameter name.
const AValue: Variant	The paramet value.
AIncludeInDelta: Boolean = False	Has no meaning in FireDAC.

Description

Use SetOptionalParam method to set a custom parameter value, associated with dataset.

The parameter value may be retrieved using GetOptionalParam. The optional parameters are part of internal dataset storage and are saved / loaded with dataset at SaveToStream (see page 645) / LoadFromStream (see page 633) calls. FireDAC does not interpret these parameters in any way.

The method is for the TClientDataSet compatibility.

Syntax

```
procedure SetOptionalParam(const AParamName: string; const AValue: Variant;
AIncludeInDelta: Boolean = False); virtual;
```

See Also

GetOptionalParam (see page 393)

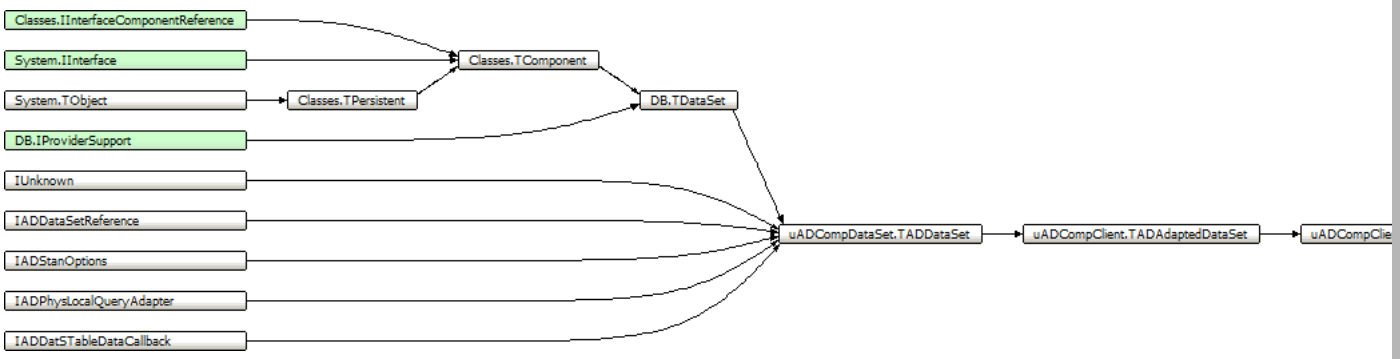
1.17.1.1.10 TADCustomQuery Class

The class implementing dataset, capable to execute SQL (see page 396) queries.

public

public	Description
DataSource (see page 395)	Specifies the data source object from which associated dataset to get matching field values to the details dataset parameters.
ParamCount (see page 396)	Returns the current number of parameters in the query SQL (see page 396) text.
SQL (see page 396)	Contains the text of the SQL statement to execute for the query.
R Text (see page 397)	Points to the actual text of the SQL (see page 396) query passed to the DBMS CLI.
ExecSQL (see page 397)	Executes the SQL (see page 396) statement for the query.
ExecSQL (see page 398)	Executes the specified SQL (see page 396) statement for the query.
ExecSQL (see page 399)	Executes the specified SQL (see page 396) statement and parameter values for the query.
ExecSQL (see page 399)	Executes the specified SQL (see page 396) statement and assigns parameter values for the query.

Class Hierarchy



File

uADCompClient

Description

Use TADCustomQuery to execute SQL queries, browse the result sets and edit the result set records.

This class hides some of properties, allowing to control visibility of properties by descendant classes. End users should use TADQuery (see page 476).

For options description, please, look at uADStanOption (see page 840) unit description.

Syntax

```
TADCustomQuery = class(TADRdbmsDataSet);
```

See Also

Executing Command (see page 66), Update Command Generation (see page 121), TADQuery (see page 476), TADCustomCommand (see page 291), uADStanOption (see page 840)

1.17.1.1.10.1 public

1.17.1.1.10.1.1 TADCustomQuery.DataSource Property

Specifies the data source object from which associated dataset to get matching field values to the details dataset parameters.

Description

Use the DataSource property to automatically fill parameters in a query with fields values from another dataset. Parameters that have the same name as fields in the other dataset are filled with the field values. Parameters with names that are not the same as fields in the other dataset do not automatically get values, and must be programmatically set.

When the master dataset current position is changed or is changed the data in the current record of the master dataset, then this dataset (detail dataset) will reopen query using new master dataset field values. If that is not desired behavior or you must quickly walk through master dataset, then use DisableControls / EnableControls on master dataset.

Syntax

```
property DataSource: TDataSource;
```

See Also

SQL (see page 396), Params, ParamByName, TDataSet.BeforeOpen

Example

```
MasterQuery.Open('select * from {id Orders}');
MasterSource.DataSet := MasterQuery;
DetailQuery.DataSource := MasterSource;
DetailQuery.Open('select * from {id Order Details} where OrderID = :OrderID');
// here :OrderID parameter of DetailQuery will get the current value of OrderID
```

```
// field from MasterQuery
```

1.17.1.1.10.1.2 TADCustomQuery.ParamCount Property

Returns the current number of parameters in the query SQL (see page 396) text.

Description

The ParamCount property value is the shortcut for Params (see page 651).Count property.

If the ResourceOptions (see page 506).ParamCreate (see page 883) property is True, ParamCount always corresponds to the number of actual parameters in the SQL statement for the query.

Syntax

```
property ParamCount: Word;
```

See Also

Params, SQL (see page 396), TADResourceOptions.ParamCreate (see page 883)

1.17.1.1.10.1.3 TADCustomQuery.SQL Property

Contains the text of the SQL statement to execute for the query.

Description

Use SQL property to specify the SQL command that a query will Execute (see page 622) / ExecSQL (see page 397) / Open.

At design time the SQL property can be edited by invoking FireDAC Query Editor dialog (see page 66). For that double click on TADCustomQuery component. The dialog offers syntax hilighting editor, query builder, ability to test query and other.

The SQL property may contain one SQL command or "batch" SQL command, consisting from few SQL command or server side programming language block of code. To execute full featured SQL script use the TADScript (see page 685) component.

The SQL statement in the SQL property may contain also:

- parameter markers, following standard SQL-92 syntax conventions. Parameters are created automatically and stored in the Params (see page 651) property, if ResourceOptions (see page 652).ParamCreate (see page 883) is True. If ParamBindMode (see page 513)is pbByName, then for all occurrences of the same marker will be created single item in Params (see page 651). If pbByNumber, then - one item per each marker.
- substitution variable markers. Macros are created automatically and stored in the Macros (see page 512) property, if ResourceOptions (see page 652).MacroCreate (see page 882) is True.
- escape sequences. They allow writing DBMS independent SQL commands.
- conditional substitutions. They allow expand SQL command conditionally, depending on application defined attributes

For details see Preprocessing Command Text (see page 56).

To improove performance on adding big SQL queries to the property using TStringList methods, surround this code into SQL.BeginUpdate / SQL.EndUpdate.

After filling this property value the Param collection will be filled automatically, if ResourceOptions.ParamCreate is True.

Syntax

```
property SQL: TStrings;
```

See Also

Executing Command (see page 66), Preprocessing Command Text (see page 56), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, ExecSQL (see page 397), TADResourceOptions.ParamCreate (see page 883), TADResourceOptions.MacroCreate (see page 882), TADRdbmsDataSet.ParamBindMode (see page 513)

Example

```
ADQuery1.SQL.BeginUpdate;
try
  ADQuery1.SQL.Add('SELECT ...');
```

```
  ADQuery1.SQL.Add('...');
  ADQuery1.SQL.Add('...');
  ADQuery1.SQL.Add('WHERE ID = :ID');
finally
  ADQuery1.SQL.EndUpdate;
end;
ADQuery1.ParamByName('ID').AsInteger := 100;
ADQuery1.Open;
```

1.17.1.1.10.1.4 TADCustomQuery.Text Property

Points to the actual text of the SQL (see page 396) query passed to the DBMS CLI.

Description

Text is a read-only property that can be examined to determine the actual contents of SQL statement passed to the DBMS. For parameterized queries, Text contains the SQL statement with parameters replaced by the parameter substitution symbol (? or else) in place of actual parameter values, substituted macro values and processed escape sequences.

In general there should be no need to examine the Text property. To access or change the SQL statement for the query, use the SQL (see page 396) property. To examine or modify parameters, use the Params (see page 651) property.

To check what FireDAC sends to the DBMS, consider to use FireDAC monitoring.

Syntax

```
property Text: String;
```

See Also

SQL (see page 396), Params (see page 651)

1.17.1.1.10.1.5 TADCustomQuery.ExecSQL Method ()

Executes the SQL (see page 396) statement for the query.

Description

Call ExecSQL to execute the SQL statement currently assigned to the SQL property.

Use ExecSQL to execute queries that do not return a cursor to data (such as INSERT, UPDATE, DELETE, and CREATE TABLE). For SELECT statements and others, returning cursors, call Open instead of ExecSQL. If FireDAC has recognized a command as returning a cursor, then "[FireDAC][Phys]-310. Cannot execute command returning results set." will be raised on ExecSQL. This may be incorrect, for example on batch commands, like a:

```
SELECT f1 INTO @v1 FROM myTab1 WHERE ...;
INSERT INTO myTab2 VALUES (@v1 + 1, ...)
```

In this case you should explicitly change command kind:

```
ADQuery1.Command.CommandKind := skInsert;
ADQuery1.ExecSQL;
```

For Array DML (see page 82) call Execute (see page 622) instead of ExecSQL.

ExecSQL prepares the statement in SQL (see page 396) property for execution if it has not already been prepared. To speed performance, an application should ordinarily call Prepare (see page 511) before calling ExecSQL for the first time.

Use ResourceOptions (see page 652).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 652).CmdExecTimeout (see page 881) to set maximum command execution time. After that time command execution will be canceled and exception raised.

To cancel command execution use TADAdaptedDataSet.AbortJob (see page 263).

Before command execution the BeforeExecute (see page 591) event is fired. If server will return command execution error, then FireDAC raises an exception. It may be analyzed in OnError (see page 261) event. After command execution is finished, AfterExecute (see page 589) event is fired.

Syntax

```
procedure ExecSQL; overload;
```

See Also

Executing Command (see page 66), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, SQL (see page 396), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261), TADDataSet.Execute (see page 622)

Example 1

```
// standard parameterized SQL execution
ADQuery1.SQL.Text := 'insert into mytab values (:id, :name)';
ADQuery1.Params[0].AsInteger := 100;
ADQuery1.Params[0].AsString := 'qwe';
ADQuery1.ExecSQL;
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery (see page 476)\ExecSQL\Async demo for asynchronous execution mode.

Example 3

```
// avoid [FireDAC][Phys]-310
ADQuery1.SQL.Clear;
ADQuery1.SQL.Add('SELECT f1 INTO @v1 FROM myTab1 WHERE ...;');
ADQuery1.SQL.Add('INSERT INTO myTab2 VALUES (@v1 + 1, ...)');
ADQuery1.Command.CommandKind := skInsert;
ADQuery1.ExecSQL;
```

1.17.1.1.10.1.6 TADCustomQuery.ExecSQL Method (String)

Executes the specified SQL (see page 396) statement for the query.

Parameters

Parameters	Description
const ASQL: String	A SQL (see page 396) command text.

Returns

The number of updated rows, if any.

Description

Call ExecSQL to execute the SQL statement specified in ASQL or is currently assigned to the SQL (see page 396) property, if ASQL is empty. Use ExecSQL to execute queries that do not return a cursor to data (such as INSERT, UPDATE, DELETE, and CREATE TABLE), does not have parameters (such as CREATE TABLE). For SELECT statements and others, returning cursors, call Open instead of ExecSQL. For Array DML (see page 82) call Execute (see page 622) instead of ExecSQL.

ExecSQL prepares the statement in ASQL, then executes it. If an application needs to call the same SQL command many times, then an application may ordinarily set SQL (see page 396) property, call Prepare (see page 511) before calling ExecSQL for the first time, then use any ExecSQL version without specifing ASQL.

Use ResourceOptions (see page 652).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 652).CmdExecTimeout (see page 881) to set maximum command execution time. After that time command execution will be canceled and exception raised.

To cancel command execution use TADAdaptedDataSet.AbortJob (see page 263).

Before command execution the BeforeExecute (see page 591) event is fired. If server will return command execution error, then FireDAC raises an exception. It may be analyzed in OnError (see page 504) event. After command execution is finished, AfterExecute (see page 589) event is fired.

Syntax

```
function ExecSQL(const ASQL: String): LongInt; overload;
```

See Also

Executing Command (see page 66), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, SQL (see page 396), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261),

TADDataSet.Execute (see page 622)

Example

```
ADQuery1.ExecSQL('create table ...');
```

1.17.1.1.10.1.7 TADCustomQuery.ExecSQL Method (String, array of Variant)

Executes the specified SQL (see page 396) statement and parameter values for the query.

Parameters

Parameters	Description
const ASQL: String	A SQL (see page 396) command text.
const AParams: array of Variant	An open array of parameter values.

Returns

The number of updated rows, if any.

Description

Call ExecSQL to execute the SQL statement specified in ASQL or is currently assigned to the SQL (see page 396) property, if ASQL is empty. Use ExecSQL to execute queries that do not return a cursor to data and have parameters (such as INSERT, UPDATE, DELETE). For SELECT statements and others, returning cursors, call Open instead of ExecSQL. For Array DML (see page 82) call Execute (see page 622) instead of ExecSQL.

ExecSQL prepares the statement in ASQL, then executes it. If an application needs to call the same SQL command many times, then an application may ordinarily set SQL (see page 396) property, call Prepare (see page 511) before calling ExecSQL for the first time, then use any ExecSQL version without specifing ASQL.

Use ResourceOptions (see page 652).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 652).CmdExecTimeout (see page 881) to set maximum command execution time. After that time command execution will be canceled and exception raised.

To cancel command execution use TADAdaptedDataSet.AbortJob (see page 263).

Before command execution the BeforeExecute (see page 591) event is fired. If server will return command execution error, then FireDAC raises an exception. It may be analyzed in OnError (see page 504) event. After command execution is finished, AfterExecute (see page 589) event is fired.

Syntax

```
function ExecSQL(const ASQL: String; const AParams: array of Variant): LongInt; overload;
```

See Also

Executing Command (see page 66), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, SQL (see page 396), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261), TADDataSet.Execute (see page 622)

Example 1

```
ADQuery1.ExecSQL('insert into mytab (f1, f2) values (:f1, :f2)', [100, 'qweqwe']);
```

Example 2

```
ADQuery1.SQL.Text := 'insert into mytab (f1, f2) values (:f1, :f2)';
ADQuery1.Params[0].DataType := ftInteger;
ADQuery1.Params[1].DataType := ftWideString;
ADQuery1.ExecSQL('', [100, 'qweqwe']);
ADQuery1.ExecSQL('', [101, 'asdasd']);
ADQuery1.ExecSQL('', [102, 'zxczxc']);
```

1.17.1.1.10.1.8 TADCustomQuery.ExecSQL Method (String, array of Variant, array of TFieldType)

Executes the specified SQL (see page 396) statement and assigns parameter values for the query.

Parameters

Parameters	Description
const ASQL: String	A SQL (see page 396) command text.
const AParams: array of Variant	An open array of parameter values.
const ATypes: array of TFieldType	An open array of parameter data types.

Returns

The number of updated rows, if any.

Description

Call ExecSQL to execute the SQL statement specified in ASQL or is currently assigned to the SQL property, if ASQL is empty. Use ExecSQL to execute queries that do not return a cursor to data, have parameters (such as INSERT, UPDATE, DELETE) and will be executed only once. For SELECT statements and others, returning cursors, call Open instead of ExecSQL. For Array DML call Execute instead of ExecSQL.

This method allows to specify exact parameter data types. If you need to leave parameter data type unchanged, then put ftUnknown into array item corresponding to the parameter.

ExecSQL prepares the statement in ASQL, then executes it. If an application needs to call the same SQL command many times, then an application may ordinarily set SQL property, call Prepare before calling ExecSQL for the first time, then use ExecSQL version without specifing ASQL and parameter types.

Use ResourceOptions.CmdExecMode to control asynchronous execution mode. And ResourceOptions.CmdExecTimeout to set maximum command execution time. After that time command execution will be canceled and exception raised.

To cancel command execution use TADAdaptedDataSet.AbortJob.

Before command execution the BeforeExecute event is fired. If server will return command execution error, then FireDAC raises an exception. It may be analyzed in OnError event. After command execution is finished, AfterExecute event is fired.

Syntax

```
function ExecSQL(const ASQL: String; const AParams: array of Variant; const ATypes: array
of TFieldType): LongInt; overload;
```

See Also

Executing Command (see page 66), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, SQL (see page 396), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261), TADDataSet.Execute (see page 622)

Example

```
ADQuery1.ExecSQL('insert into mytab (f1, f2) values (:f1, :f2)',
  [100, 'qweqwe'], [ftInteger, ftWideString]);
```

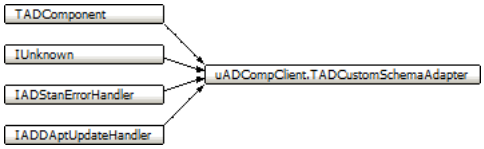
1.17.1.1.11 TADCustomSchemaAdapter Class

The class implementing Centralized Cached Updates (see page 114) support.

public

public	Description
Count (see page 401)	Returns the number of associated table adapters.
TableAdapters (see page 401)	Returns references to the associated table adapters.
ApplyUpdates (see page 402)	Applies changes for all records in the centralized change journal to the database.
Reconcile (see page 402)	Clears successfully updated records from the centralized cache of changes.

Class Hierarchy



File

uADCompClient

Description

Use TADCustomSchemaAdapter to manage Centralized Cached Updates (see page 114).

This class hides some of properties, allowing to control visibility of properties by descendant classes. End users should use TADSchemaAdapter (see page 514).

Syntax

```
TADCustomSchemaAdapter = class(TADComponent, IUnknown, IADStanErrorHandler,
IADDAptUpdateHandler);
```

See Also

Caching Updates (see page 114), Master-Detail Relationship (see page 104), TADSchemaAdapter (see page 514)

Example

See demos:

- FireDAC\Samples\Comp Layer\TADQuery (see page 476)\CachedUpdates\Centralized

- FireDAC\Samples\Comp Layer\TADSchemaAdapter (see page 514)\MemTable

1.17.1.1.11.1 public

1.17.1.1.11.1.1 TADCustomSchemaAdapter.Count Property

Returns the number of associated table adapters.

Description

Use Count property to get the number of TADCustomTableAdapter's associated with this schema adapter.

Syntax

```
property Count: Integer;
```

See Also

TableAdapters (see page 401)

1.17.1.1.11.1.2 TADCustomSchemaAdapter.TableAdapters Property

Returns references to the associated table adapters.

Description

Use TableAdapters indexed property to get a reference to a associated table adapter by its index.

Syntax

```
property TableAdapters [AIndex: Integer]: TADCustomTableAdapter;
```

See Also

Count (see page 401)

1.17.1.1.11.1.3 TADCustomSchemaAdapter.ApplyUpdates Method

Applies changes for all records in the centralized change journal to the database.

Parameters

Parameters	Description
AMaxErrors: Integer = -1	A maximum number of allowed errors. Default value is -1.

Returns

The number of errors is encountered.

Description

Use ApplyUpdates to apply changes for all records in the centralized change journal to the database. The centralized change journal collects changes from:

- FireDAC datasets with SchemaAdapter (see page 506) property set to this TADCustomSchemaAdapter instance;
- table adapters with SchemaAdapter property set to this TADCustomSchemaAdapter instance.

The ApplyUpdates call performs the steps:

1. Fires a BeforeApplyUpdate event.
2. Calls the data adapters to generate / use SQL update commands for each row and post changes to the database.
3. Fires an AfterApplyUpdate event.
4. Calls the Reconcile (see page 402) method to reconcile any records that has errors in step 2.

Note, there is no CommitUpdates (see page 615) method, and it must be called in code for each associated dataset.

AMaxErrors indicates the maximum number of errors that the FireDAC should allow before prematurely stopping the update operation. Set AMaxErrors to –1 to indicate that there is no limit to the number of errors, or to 0 to indicate that no error is allowed.

ApplyUpdates returns the number of errors it encountered. Based on this return value and the setting of AMaxErrors, successfully applied records are removed from the centralized change log. If the update process is aborted before all updates are applied, any unapplied updates remain in the change log.

ApplyUpdates does not raise exception. Instead application should review erroneous records using Reconcile and OnReconcileRow event handler, or FilterChanges (see page 597) and RowError (see page 607) properties for each dataset. Read "Reviewing errors" at Caching Updates (see page 114) for more details.

Syntax

```
function ApplyUpdates(AMaxErrors: Integer = -1): Integer;
```

See Also

SchemaAdapter (see page 506), Reconcile (see page 402), Caching Updates (see page 114)

1.17.1.1.11.1.4 TADCustomSchemaAdapter.Reconcile Method

Clears successfully updated records from the centralized cache of changes.

Description

Reconcile is called by the ApplyUpdates (see page 402) method to update the change log so that it reflects the result of an update. Reconcile is called after the schema adapter has attempted to apply all updates in the centralized change log.

Results of updates applying are recorded in internal data storage, including error information for each record that could not be applied. Reconcile generates an OnReconcileRow event for every record that was not successfully applied. Finally, Reconcile adjusts the change log so that it includes only those records that were not successfully applied. Any changes made in the OnReconcileRow event are reflected in the new log state.

Reconcile returns a value that indicates whether reconciling is successful. If True, Reconcile succeeded without error. If False, one or more errors could not be reconciled.

Alternatively an application may review erroneous records using the datasets FilterChanges (see page 597) and RowError (see page 607) properties. Read "Reviewing errors" at Caching Updates (see page 114) for more details.

Syntax

```
function Reconcile: Boolean;
```

See Also

Caching Updates (see page 114), ApplyUpdates (see page 402)

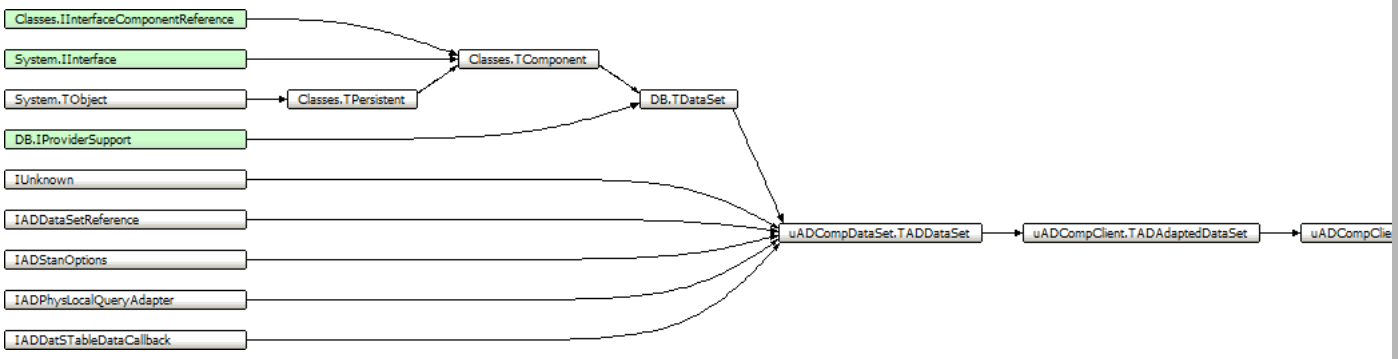
1.17.1.1.12 TADCustomStoredProc Class

The class implementing dataset, capable to execute server side stored procedures.

public

public	Description
CatalogName (see page 404)	Limits the stored procedure names to the specified catalog.
Overload (see page 404)	Specifies which Oracle overloaded packaged stored procedure to execute.
PackageName (see page 404)	Limits the stored procedure names to the specified package name.
ParamCount (see page 405)	Returns the number of parameters for the stored procedure.
RowsAffected (see page 405)	Returns the number of rows operated upon by the latest query execution.
SchemaName (see page 405)	Limits the stored procedure names to the specified schema.
StoredProcName (see page 406)	Identifies the name of the stored procedure on the server to call.
DescriptionsAvailable (see page 406)	Determines if parameter information is available from the stored procedure on the server.
ExecFunc (see page 406)	Executes a stored function in the DBMS session and returns its value.
ExecFunc (see page 407)	Executes the specified stored function in the DBMS session and returns its value.
ExecFunc (see page 408)	Executes the specified stored function, assigns parameter values for the stored function arguments and returns its value.
ExecFunc (see page 409)	Executes the specified stored function, assigns parameter data type and values for the stored proc arguments and returns its value.
ExecProc (see page 410)	Executes the specified stored procedure in the DBMS session.
ExecProc (see page 411)	Executes the specified stored procedure in the DBMS session.
ExecProc (see page 411)	Executes the specified stored procedure and assigns parameter values for the stored proc arguments.
ExecProc (see page 412)	Executes the specified stored procedure and assigns parameter data type and values for the stored proc arguments.

Class Hierarchy



File

uADCompClient

Description

Use TADCustomStoredProc to execute server side stored procedures, browse the result sets and edit the result set records.

This class hides some of properties, allowing to control visibility of properties by descendant classes. End users should use TADStoredProc (see page 514).

Read "Executing Stored Procedure (see page 71)" for more details.

Syntax

```
TADCustomStoredProc = class(TADRdbmsDataSet);
```

See Also

Executing Stored Procedure (see page 71), TADStoredProc (see page 514), TADCustomQuery (see page 394)

1.17.1.1.12.1 public

1.17.1.1.12.1.1 TADCustomStoredProc.CatalogName Property

Limits the stored procedure names to the specified catalog.

Description

Specify the CatalogName property value to limit the set of stored procedures to the specified catalog.

Syntax

```
property CatalogName: String;
```

See Also

Object Names (see page 133), SchemaName (see page 405), PackageName (see page 404), Overload (see page 404), StoredProcName (see page 406)

1.17.1.1.12.1.2 TADCustomStoredProc.Overload Property

Specifies which Oracle overloaded packaged stored procedure to execute.

Description

Use Overload to specify which overloaded stored procedure to execute on an Oracle server. An Oracle overloaded stored procedure is one that shares a name with one or more other stored procedures. Oracle distinguishes among overloaded stored procedures by assigning each procedure a unique numeric identifier. An application can specify this identifier using the Overload property.

By default, Overload is zero, which assumes there is no procedure overloading. For all servers except Oracle, this value will not have the effect. If an application attempts to access an Oracle overloaded procedure without setting Overload, the FireDAC accesses the lowest numbered, or first, overloaded procedure on the server.

If Overload is 1, the FireDAC executes the first overloaded procedure on the server. If Overload is 2, it executes the second, and so on.

While Oracle overloaded procedures share names, their parameter lists are unique. An application must ensure that it passes the correct parameter list to an overloaded procedure.

Syntax

```
property Overload: Word;
```

See Also

Executing Stored Procedure (see page 71), Params (see page 651), StoredProcName (see page 406)

1.17.1.1.12.1.3 TADCustomStoredProc.PackageName Property

Limits the stored procedure names to the specified package name.

Description

Specify the PackageName property value to limit the procedure name to the specified package.

Syntax

```
property PackageName: String;
```

See Also

Object Names (see page 133), CatalogName (see page 404), SchemaName (see page 405), Overload (see page 404), StoredProcName (see page 406)

1.17.1.1.12.1.4 TADCustomStoredProc.ParamCount Property

Returns the number of parameters for the stored procedure.

Description

The ParamCount property is shortcut for the Params (see page 651).Count property. If the fiMeta is in FetchOptions (see page 651).Items (see page 855), then ParamCount corresponds to the number of actual parameters of the stored procedure.

Syntax

```
property ParamCount: Word;
```

See Also

Params, SQL, TADResourceOptions.ParamCreate (see page 883)

1.17.1.1.12.1.5 TADCustomStoredProc.RowsAffected Property

Returns the number of rows operated upon by the latest query execution.

Description

Inspect RowsAffected to determine how many rows were inserted, updated, deleted or fetched by the last dataset operation. If no rows were processed, RowsAffected = 0. If numbers of processed rows is not accessible, RowsAffected = –1.

At MS SQL Server RowsAffected may be unexpectedly equal to -1, if stored procedure or table trigger omits SET NOCOUNT ON. Also, this may be a reason of "Command inserted / updated / deleted [0] records instead of [1] record" error at posting updates to database. Check MS BOL for more details about SET NOCOUNT ON.

Syntax

```
property RowsAffected: TADCounter;
```

Example

```
procedure TForm1.ADQuery1AfterExecute(ASender: TObject);
begin
  if ADQuery1.RowsAffected = -1 then
    StatusBar1.SimpleText := 'Ok'
  else
    case ADQuery1.Command1.CommandKind of
    skDelete: StatusBar1.SimpleText := Format('%d rows deleted', [ADQuery1.RowsAffected]);
    skInsert: StatusBar1.SimpleText := Format('%d rows inserted', [ADQuery1.RowsAffected]);
    skUpdate: StatusBar1.SimpleText := Format('%d rows updated', [ADQuery1.RowsAffected]);
    else      StatusBar1.SimpleText := Format('%d rows affected', [ADQuery1.RowsAffected]);
    end;
end;
```

1.17.1.1.12.1.6 TADCustomStoredProc.SchemaName Property

Limits the stored procedure names to the specified schema.

Description

Specify the SchemaName property value to limit the set of schemas to the specified schema.

Syntax

```
property SchemaName: String;
```

See Also

Object Names (see page 133), CatalogName (see page 404), PackageName (see page 404), Overload (see page 404), StoredProcName (see page 406)

1.17.1.1.12.1.7 TADCustomStoredProc.StoredProcName Property

Identifies the name of the stored procedure on the server to call.

Description

Set StoredProcName to specify the name of the stored in database procedure to call. The full name of the stored procedure will be build from CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), Overload (see page 404) property values. Also a full name may be assigned to the StoredProcName (only for non-packaged procedures). If name is case sensitive or contains special characters, then it must be explicitly quoted.

If database does not have stored procedure with StoredProcName name, then when the application attempts to prepare the stored procedure, an exception will be raised.

Syntax

```
property StoredProcName: string;
```

See Also

Object Names (see page 133), Executing Stored Procedure (see page 71), CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), Overload (see page 404)

Example 1

```
ADStoredProc1.StoredProcName := 'Northwind.dbo.[my proc]';
// the same as following
ADStoredProc1.CatalogName := 'Northwind';
ADStoredProc1.SchemaName := 'dbo';
ADStoredProc1.StoredProcName := '[my proc]';
```

Example 2

```
ADStoredProc1.PackageName := 'sys.dbms_sql';
ADStoredProc1.StoredProcName := 'execute';
```

1.17.1.1.12.1.8 TADCustomStoredProc.DescriptionsAvailable Method

Determines if parameter information is available from the stored procedure on the server.

Returns

True, if info is available.

Description

Call DescriptionsAvailable to determine if a stored procedure on a server can return parameter information to the application. If DescriptionsAvailable returns True, parameter information is available from the server. If DescriptionsAvailable returns False, parameter information is not available.

Syntax

```
function DescriptionsAvailable: Boolean;
```

See Also

ExecProc (see page 410), Params

1.17.1.1.12.1.9 TADCustomStoredProc.ExecFunc Method ()

Executes a stored function in the DBMS session and returns its value.

Returns

The result of stored function.

Description

Call ExecFunc to execute a stored function in the DBMS session, using the currently assigned stored procedure name. Use ExecFunc to execute stored functions, that do not return a cursor to data.

Before calling ExecFunc, assign stored function additional name parts to the CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), Overload (see page 404) properties. As part of ExecFunc call, the SQL statement, calling stored procedure, is built.

Use ResourceOptions (see page 652).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 652).CmdExecTimeout (see page 881) to set maximum stored function execution time. After that time stored function execution will be canceled and exception raised.

To cancel stored function execution use TADAdaptedDataSet.AbortJob (see page 263).

Before stored function execution the BeforeExecute (see page 591) event is fired. If server will return stored function execution error, then FireDAC raises an exception. It may be analyzed in OnError (see page 261) event. After stored function execution is finished, AfterExecute (see page 589) event is fired.

ExecFunc returns the stored function value.

Syntax

```
function ExecFunc: Variant; overload;
```

See Also

Executing Stored Procedure (see page 71), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), StoredProcName (see page 406), Overload (see page 404), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261)

Example

```
var
  v: Variant;
...
ADStoredProc1.StoredProcName := 'myfunc';
v := ADStoredProc1.ExecFunc;
```

1.17.1.1.12.1.10 TADCustomStoredProc.ExecFunc Method (String)

Executes the specified stored function in the DBMS session and returns its value.

Parameters

Parameters	Description
const AProcName: String	A stored function name.

Returns

The result of stored function.

Description

Call ExecFunc to execute a stored function in the DBMS session, specified in AProcName or is currently assigned to the StoredProcName property, if AProcName is empty. Use ExecFunc to execute stored functions, that do not return a cursor to data and does not have parameters.

Before calling ExecFunc, assign stored function additional name parts to the CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404) properties. As part of the ExecFunc call the SQL statement calling the stored procedure is built. AProcName may include overload number specified after ';'.

Use ResourceOptions (see page 652).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 652).CmdExecTimeout (see page 881) to set maximum stored function execution time.

After that time stored function execution will be canceled and exception raised.

To cancel stored function execution use TADAdaptedDataSet.AbortJob (see page 263).

Before stored function execution the BeforeExecute (see page 591) event is fired. If server will return stored function execution error, then FireDAC raises an exception. It may be analyzed in OnError (see page 261) event. After stored function execution is finished, AfterExecute (see page 589) event is fired.

ExecFunc returns the stored function value.

Syntax

```
function ExecFunc(const AProcName: String): Variant; overload;
```

See Also

Executing Stored Procedure (see page 71), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), StoredProcName (see page 406), Overload (see page 404), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261)

Example

```
var
  v: Variant;
...
v := ADStoredProc1.ExecFunc('myfunc');
```

1.17.1.1.12.1.11 TADCustomStoredProc.ExecFunc Method (String, array of Variant)

Executes the specified stored function, assigns parameter values for the stored function arguments and returns its value.

Parameters

Parameters	Description
const AProcName: String	A stored function name.
const AParams: array of Variant	An open array of parameter values.

Returns

The result of stored function.

Description

Call ExecFunc to execute a stored function in the DBMS session, specified in AProcName or is currently assigned to the StoredProcName property, if AProcName is empty. Use ExecFunc to execute stored functions, that do not return a cursor to data.

Before calling ExecFunc, assing stored function additional name parts to the CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404) properties. As part of the ExecFunc call the SQL statement calling the stored procedure is built, parameters are binded, and AParams values are assigned to the input parameters. AProcName may include overload number specified after ';'.

After return from ExecFunc you can read output parameter values.

Use ResourceOptions (see page 652).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 652).CmdExecTimeout (see page 881) to set maximum stored function execution time. After that time stored function execution will be canceled and exception raised.

To cancel stored function execution use TADAdaptedDataSet.AbortJob (see page 263).

Before stored function execution the BeforeExecute (see page 591) event is fired. If server will return stored function execution error, then FireDAC raises an exception. It may be analyzed in OnError (see page 261) event. After stored function execution is finished, AfterExecute (see page 589) event is fired.

ExecFunc returns the stored function value.

Syntax

```
function ExecFunc(const AProcName: String; const AParams: array of Variant): Variant;
```

```
overload;
```

See Also

Executing Stored Procedure (see page 71), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), StoredProcName (see page 406), Overload (see page 404), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261)

Example 1

```
var
  v: Variant;
...
v := ADStoredProc1.ExecFunc('myfunc;2', [100, 'qweqwe']);
```

Example 2

```
var
  v: Variant;
...
ADStoredProc1.StoredProcName := 'myfunc';
ADStoredProc1.Overload := 2;
v := ADStoredProc1.ExecFunc('', [100, 'qweqwe']);
v := ADStoredProc1.ExecFunc('', [101, 'asdasd']);
v := ADStoredProc1.ExecFunc('', [102, 'zxczxc']);
```

1.17.1.1.12.1.12 TADCustomStoredProc.ExecFunc Method (String, array of Variant, array of TFieldType)

Executes the specified stored function, assigns parameter data type and values for the stored proc arguments and returns its value.

Parameters

Parameters	Description
const AProcName: String	A stored function name.
const AParams: array of Variant	An open array of parameter values.
const ATypes: array of TFieldType	An open array of parameter data types.

Returns

The result of stored function.

Description

Call ExecFunc to execute a stored function in the DBMS session, specified in AProcName or is currently assigned to the StoredProcName property, if AProcName is empty. Use ExecFunc to execute stored functiona, that do not return a cursor to data.

Before calling ExecFunc, assing stored function additional name parts to the CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404) properties. As part of the ExecFunc call the SQL statement calling the stored procedure is built, the parameters data types are assigned and binded, and the AParams values are assigned to the input parameters. AProcName may include overload number specified after ';'.

After return from ExecFunc you can read output parameter values.

Use ResourceOptions (see page 652).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 652).CmdExecTimeout (see page 881) to set maximum stored function execution time. After that time stored function execution will be canceled and exception raised.

To cancel stored function execution use TADAdaptedDataSet.AbortJob (see page 263).

Before stored function execution the BeforeExecute (see page 591) event is fired. If server will return stored function execution error, then FireDAC raises an exception. It may be analyzed in OnError (see page 261) event. After stored function execution is finished, AfterExecute (see page 589) event is fired.

ExecFunc returns the stored function value.

Syntax

```
function ExecFunc(const AProcName: String; const AParams: array of Variant; const ATypes:
array of TFieldType): Variant; overload;
```

See Also

Executing Stored Procedure (see page 71), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), StoredProcName (see page 406), Overload (see page 404), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261)

Example

```
var
  v: Variant;
...
v := ADStoredProc1.ExecFunc('myfunc', [100, 'qweqwe'], [ftInteger, ftWideString]);
```

1.17.1.1.12.1.13 TADCustomStoredProc.ExecProc Method ()

Executes the specified stored procedure in the DBMS session.

Description

Call ExecProc to execute a stored procedure in the DBMS session, using the currently assigned stored procedure name. Use ExecProc to execute stored procedures, that do not return a cursor to data.

Before calling ExecProc, assing stored procedure name to the CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), StoredProcName (see page 406), Overload (see page 404) properties. Then if fiMeta in FetchOptions (see page 651).Items (see page 855), then call Prepare (see page 511) method. After that Params (see page 651) property will be filled by parameters. Otherwise, if fiMeta is not in FetchOptions (see page 651).Items (see page 855), provide correct set of parameters in the Params (see page 651) property, then call Prepare (see page 511) method. After that SQL statement, calling stored procedure, is built and parameters are binded.

Then assign values to the input parameters and call ExecProc. After return from ExecProc you can read output parameter values.

Use ResourceOptions (see page 652).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 652).CmdExecTimeout (see page 881) to set maximum stored procedure execution time. After that time stored procedure execution will be canceled and exception raised.

To cancel stored procedure execution use TADAdaptedDataSet.AbortJob (see page 263).

Before stored procedure execution the BeforeExecute (see page 591) event is fired. If server will return stored procedure execution error, then FireDAC raises an exception. It may be analyzed in OnError (see page 261) event. After stored procedure execution is finished, AfterExecute (see page 589) event is fired.

Syntax

```
procedure ExecProc; overload;
```

See Also

Executing Stored Procedure (see page 71), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), StoredProcName (see page 406), Overload (see page 404), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261)

Example

```
ADStoredProc1.StoredProcName := 'myproc';
ADStoredProc1.Prepare;
ADStoredProc1.ParamByName('inval').Value := 100;
ADStoredProc1.ExecProc;
ShowMessage(ADStoredProc1.ParamByName('outval').AsString);
```

1.17.1.1.12.1.14 TADCustomStoredProc.ExecProc Method (String)

Executes the specified stored procedure in the DBMS session.

Parameters

Parameters	Description
const AProcName: String	A stored procedure name.

Returns

The number of updated rows, if any.

Description

Call ExecProc to execute a stored procedure in the DBMS session, specified in AProcName or is currently assigned to the StoredProcName property, if AProcName is empty. Use ExecProc to execute stored procedures, that do not return a cursor to data and does not have parameters.

Before calling ExecProc, assing stored procedure additional name parts to the CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404) properties. As part of the ExecProc call the SQL statement calling the stored procedure is built. AProcName may include overload number specified after ';'.

Use ResourceOptions (see page 652).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 652).CmdExecTimeout (see page 881) to set maximum stored procedure execution time. After that time stored procedure execution will be canceled and exception raised.

To cancel stored procedure execution use TADAdaptedDataSet.AbortJob (see page 263).

Before stored procedure execution the BeforeExecute (see page 591) event is fired. If server will return stored procedure execution error, then FireDAC raises an exception. It may be analyzed in OnError (see page 261) event. After stored procedure execution is finished, AfterExecute (see page 589) event is fired.

Syntax

```
function ExecProc(const AProcName: String): LongInt; overload;
```

See Also

Executing Stored Procedure (see page 71), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), StoredProcName (see page 406), Overload (see page 404), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261)

Example

```
ADStoredProc1.ExecProc('myproc');
```

1.17.1.1.12.1.15 TADCustomStoredProc.ExecProc Method (String, array of Variant)

Executes the specified stored procedure and assigns parameter values for the stored proc arguments.

Parameters

Parameters	Description
const AProcName: String	A stored procedure name.
const AParams: array of Variant	An open array of parameter values.

Returns

The number of updated rows, if any.

Description

Call ExecProc to execute a stored procedure in the DBMS session, specified in AProcName or is currently assigned to the StoredProcName property, if AProcName is empty. Use ExecProc to execute stored procedures, that do not return a cursor to data.

Before calling ExecProc, assing stored procedure additional name parts to the CatalogName (see page 404),

SchemaName (see page 405), PackageName (see page 404) properties. As part of the ExecProc call the SQL statement calling the stored procedure is built, the parameters are binded, and the AParams values are assigned to the input parameters. AProcName may include overload number specified after ';'.

After return from ExecProc you can read output parameter values.

Use ResourceOptions (see page 652).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 652).CmdExecTimeout (see page 881) to set maximum stored procedure execution time. After that time stored procedure execution will be canceled and exception raised.

To cancel stored procedure execution use TADAdaptedDataSet.AbortJob (see page 263).

Before stored procedure execution the BeforeExecute (see page 591) event is fired. If server will return stored procedure execution error, then FireDAC raises an exception. It may be analyzed in OnError (see page 261) event. After stored procedure execution is finished, AfterExecute (see page 589) event is fired.

Syntax

```
function ExecProc(const AProcName: String; const AParams: array of Variant): LongInt;
overload;
```

See Also

Executing Stored Procedure (see page 71), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), StoredProcName (see page 406), Overload (see page 404), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261)

Example 1

```
ADStoredProc1.ExecProc('myproc;2', [100, 'qweqwe']);
```

Example 2

```
ADStoredProc1.StoredProcName := 'myproc';
ADStoredProc1.Overload := 2;
ADStoredProc1.ExecProc('', [100, 'qweqwe']);
ADStoredProc1.ExecProc('', [101, 'asdasd']);
ADStoredProc1.ExecProc('', [102, 'zxczxc']);
```

1.17.1.1.12.1.16 TADCustomStoredProc.ExecProc Method (String, array of Variant, array of TFieldType)

Executes the specified stored procedure and assigns parameter data type and values for the stored proc arguments.

Parameters

Parameters	Description
const AProcName: String	A stored procedure name.
const AParams: array of Variant	An open array of parameter values.
const ATypes: array of TFieldType	An open array of parameter data types.

Returns

The number of updated rows, if any.

Description

Call ExecProc to execute a stored procedure in the DBMS session, specified in AProcName or is currently assigned to the StoredProcName property, if AProcName is empty. Use ExecProc to execute stored procedures, that do not return a cursor to data.

Before calling ExecProc, assing stored procedure additional name parts to the CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404) properties. As part of the ExecProc call the SQL statement calling the stored procedure is built, the parameters data types are assigned and binded, and the AParams values are assigned to the input parameters. AProcName may include overload number specified after ';'.

After return from ExecProc you can read output parameter values.

Use ResourceOptions (see page 652).CmdExecMode (see page 880) to control asynchronous execution mode. And

ResourceOptions (see page 652).CmdExecTimeout (see page 881) to set maximum stored procedure execution time. After that time stored procedure execution will be canceled and exception raised.

To cancel stored procedure execution use TADAdaptedDataSet.AbortJob (see page 263).

Before stored procedure execution the BeforeExecute (see page 591) event is fired. If server will return stored procedure execution error, then FireDAC raises an exception. It may be analyzed in OnError (see page 261) event. After stored procedure execution is finished, AfterExecute (see page 589) event is fired.

Syntax

```
function ExecProc(const AProcName: String; const AParams: array of Variant; const ATypes:
array of TFieldType): LongInt; overload;
```

See Also

Executing Stored Procedure (see page 71), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), StoredProcName (see page 406), Overload (see page 404), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261)

Example

```
ADStoredProc1.ExecProc('myproc', [100, 'qweqwe'], [ftInteger, ftWideString]);
```

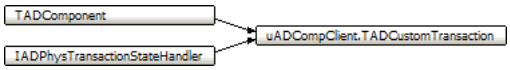
1.17.1.1.13 TADCustomTransaction Class

The class is responsible for connection transaction management.

public

public	Description
Active (see page 414)	Indicates whether a database transaction is in progress or not.
AfterCommit (see page 414)	Fires after transaction is committed. The AfterCommit event fires after the Commit (see page 417) method is executed and the transaction is committed. In case of a nested transaction in AfterCommit event handler the transaction is not yet finished, but nesting level is decremented.
AfterRollback (see page 414)	Fires after transaction is rolled back.
AfterStartTransaction (see page 415)	Fires after transaction is started.
BeforeCommit (see page 415)	Fires before transaction will be committed.
BeforeRollback (see page 415)	Fires before transaction will be rolled back.
BeforeStartTransaction (see page 415)	Fires before transaction will be started.
Connection (see page 416)	Specifies the connection component that actually forms the connection to the DBMS.
DataSetCount (see page 416)	Returns the number of datasets associated with this transaction.
DataSets (see page 416)	Returns the dataset associated with this transaction by it index.
Options (see page 416)	The set of options controlling transactions behaviour.
TransactionIntf (see page 417)	Returns IADPhysTransaction interface.
Commit (see page 417)	Permanently stores modifications to the data made in the current transaction, and optionally ends the current transactions.
CommitRetaining (see page 417)	Permanently stores modifications to the data made in the current transaction, without ending the current transaction.
Rollback (see page 418)	Cancels all modifications to the data made in the current transaction and optionally ends transaction.
RollbackRetaining (see page 419)	Cancels all modifications to the data made in the current transaction, without ending the current transaction.
StartTransaction (see page 419)	Starts new transaction at database.

Class Hierarchy



File

uADCompClient

Description

Use TADCustomTransaction to manage transactions in a connection to a DBMS.

The component, at first, is target to Interbase / Firebird client application developers. Because these DBMS's are supporting multiple active transactions per a connection. If a DBMS does not support this feature, then multiple TADCustomTransaction objects on the same connection will share control under the same single transaction. Also, the application may use TADCustomConnection (see page 318) transaction control methods.

This class hides some of properties, allowing to control visibility of properties by descendant classes. End users should use TADTransaction (see page 561).

Syntax

```
TADCustomTransaction = class(TADComponent, IADPhysTransactionStateHandler);
```

See Also

Managing Transactions (see page 41), Establishing Connection (see page 37), TADTransaction (see page 561), TADCustomConnection (see page 318)

1.17.1.1.13.1 public

1.17.1.1.13.1.1 TADCustomTransaction.Active Property

Indicates whether a database transaction is in progress or not.

Description

Examine Active at run-time to determine if a database transaction is currently in progress. Active is True if a transaction is in progress, False otherwise.

The value of Active cannot be changed directly. Calling StartTransaction (see page 419) sets Active to True. Calling Commit (see page 417) or Rollback (see page 418) sets Active to False.

Syntax

```
property Active: Boolean;
```

See Also

StartTransaction (see page 419), Commit (see page 417), Rollback (see page 418)

1.17.1.1.13.1.2 TADCustomTransaction.AfterCommit Event

Fires after transaction is committed. The AfterCommit event fires after the Commit (see page 417) method is executed and the transaction is committed. In case of a nested transaction in AfterCommit event handler the transaction is not yet finished, but nesting level is decremented.

Syntax

```
property AfterCommit: TNotifyEvent;
```

See Also

BeforeCommit (see page 415)

1.17.1.1.13.1.3 TADCustomTransaction.AfterRollback Event

Fires after transaction is rolled back.

Description

The AfterRollback event fires after the Rollback method is executed and the transaction is rolled back. In case of a nested transaction in AfterRollback event handler the transaction is not yet rolled back in full, but nesting level is decremented.

Syntax

```
property AfterRollback: TNotifyEvent;
```

See Also

BeforeRollback (see page 415)

1.17.1.1.13.1.4 TADCustomTransaction.AfterStartTransaction Event

Fires after transaction is started.

Description

The AfterStartTransaction event fires after the StartTransaction method is executed and the transaction is started. In case of a nested transaction in AfterStartTransaction event handler the transaction is not just started, but nesting level is incremented.

Syntax

```
property AfterStartTransaction: TNotifyEvent;
```

See Also

BeforeStartTransaction (see page 415)

1.17.1.1.13.1.5 TADCustomTransaction.BeforeCommit Event

Fires before transaction will be committed.

Description

The BeforeCommit event fires before the Commit method will be executed and the transaction will be committed.

Syntax

```
property BeforeCommit: TNotifyEvent;
```

See Also

AfterCommit (see page 414)

1.17.1.1.13.1.6 TADCustomTransaction.BeforeRollback Event

Fires before transaction will be rolled back.

Description

The BeforeRollback event fires before the Rollback method will be executed and the transaction will be rolled back.

Syntax

```
property BeforeRollback: TNotifyEvent;
```

See Also

AfterRollback (see page 414)

1.17.1.1.13.1.7 TADCustomTransaction.BeforeStartTransaction Event

Fires before transaction will be started.

Description

The BeforeStartTransaction event fires before the StartTransaction method will be executed and the transaction will be started.

Syntax

```
property BeforeStartTransaction: TNotifyEvent;
```

See Also

AfterStartTransaction (see page 415)

1.17.1.1.13.1.8 TADCustomTransaction.Connection Property

Specifies the connection component that actually forms the connection to the DBMS.

Description

Set Connection to the connection component that establishes a connection to the DBMS. If to change Connection property value, while transaction is active, an exception will be raised.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

Active (see page 414)

1.17.1.1.13.1.9 TADCustomTransaction.DataSetCount Property

Returns the number of datasets associated with this transaction.

Description

Returns the number of datasets whose TADRdbmsDataSet (see page 501).Transaction (see page 507) property points to this transaction.

Syntax

```
property DataSetCount: Integer;
```

See Also

TADRdbmsDataSet.Transaction (see page 507), DataSets (see page 416)

1.17.1.1.13.1.10 TADCustomTransaction.DataSets Property

Returns the dataset associated with this transaction by it index.

Description

Returns the dataset whose TADRdbmsDataSet (see page 501).Transaction (see page 507) property points to this transaction by dataset index.

Syntax

```
property DataSets [AIndex: Integer]: TADDataSet;
```

See Also

DataSetCount (see page 416)

1.17.1.1.13.1.11 TADCustomTransaction.Options Property

The set of options controlling transactions behaviour.

Description

The Options property returns reference to object, consisting of the set of options, controlling transactions behavior. Changing options will not take immediate effect, it is deferred until first transaction control statement.

Syntax

```
property Options: TADTxOptions;
```

See Also

StartTransaction (see page 419), Commit (see page 417), Rollback (see page 418)

Example

```
ADTransaction1.Options.ReadOnly := True;
ADTransaction1.StartTransaction;
```

1.17.1.1.13.1.12 TADCustomTransaction.TransactionIntf Property

Returns IADPhysTransaction interface.

Description

The TransactionIntf property returns the reference to the IADPhysTransaction interface. It is not nil, if the connection is active. Otherwise it is nil. It is not recommended to mix usage of TADCustomTransaction API and TADCustomTransaction.TransactionIntf API.

Syntax

```
property TransactionIntf: IADPhysTransaction;
```

See Also

IADPhysTransaction

1.17.1.1.13.1.13 TADCustomTransaction.Commit Method

Permanently stores modifications to the data made in the current transaction, and optionally ends the current transactions.

Description

Call Commit to permanently store modifications, like a INSERT's / UPDATE's / DELETE's, made in current transaction to the database.

FireDAC supports nested transactions, so the current transaction is the one started with most recent StartTransaction (see page 419) call. If the database does not support nested transactions, like most of DBMS's, then FireDAC will emulate nested transactions using savepoints.

Before calling Commit, an application may check the status of the Active (see page 414) property. If an application calls Commit and there is no current transaction, an exception is raised.

A Commit call on Interbase / Firebird will close and unprepare all datasets and commands, associated with this transaction object. On some other DBMS's a call will invalidate all active result sets. For example, on MS SQL Server 2005.

Syntax

```
procedure Commit;
```

See Also

Managing Transactions (see page 41), StartTransaction (see page 419), Active (see page 414), Rollback (see page 418), CommitRetaining (see page 417)

Example

```
procedure TForm1.ChangeButtonClick(Sender: TObject);
begin
  ADQuery1.Transaction := ADTransaction1;
  ADQuery1.SQL.Text := 'update employees set salary = salary * :k where id = :id';
  ADTransaction1.StartTransaction;
  try
    ADQuery1.ExecSQL('', [1.2, 100]);
    ADQuery1.ExecSQL('', [1.3, 200]);
    ADTransaction1.Commit;
  except
    ADTransaction1.Rollback;
    raise;
  end;
end;
```

1.17.1.1.13.1.14 TADCustomTransaction.CommitRetaining Method

Permanently stores modifications to the data made in the current transaction, without ending the current transaction.

Description

Call CommitRetaining to permanently store modifications, like a INSERT's / UPDATE's / DELETE's, made in current transaction to the database without finishing transaction.

FireDAC supports nested transactions, so the current transaction is the one started with most recent StartTransaction (see page 419) call. If the database does not support nested transactions, like most of DBMS's, then FireDAC will emulate nested transactions using savepoints. CommitRetaining is usefull only for main transaction, not for nested ones.

Before calling CommitRetaining, an application may check the status of the Active (see page 414) property. If an application calls CommitRetaining and there is no current transaction, an exception is raised.

CommitRetaining will use native functionality on IB/FB. On other DBMS's the method will be equal to calls of Commit (see page 417), then of StartTransaction (see page 419).

Syntax

```
procedure CommitRetaining;
```

See Also

Managing Transactions (see page 41), StartTransaction (see page 419), Active (see page 414), Rollback (see page 418), Commit (see page 417)

Example

```
procedure TForm1.ChangeButtonClick(Sender: TObject);
begin
  ADQuery1.Transaction := ADTransaction1;
  ADQuery1.SQL.Text := 'select * from employees';

  ADQuery2.Transaction := ADTransaction1;
  ADQuery2.SQL.Text := 'update employees set salary = salary * :k where id = :id';

  ADTransaction1.StartTransaction;
  try
    ADQuery1.Open;
    while not ADQuery1.Eof do begin
      ADQuery2.ExecSQL('', [1.2, ADQuery1.FieldByName('id').AsInteger]);
      ADTransaction1.CommitRetaining;
      ADQuery1.Next;
    end;
    ADTransaction1.Commit;
  except
    ADTransaction1.Rollback;
    raise;
  end;
end;
```

1.17.1.1.13.1.15 TADCustomTransaction.Rollback Method

Cancels all modifications to the data made in the current transaction and optionally ends transaction.

Description

Call Rollback to cancel all modifications, like a INSERT's / UPDATE's / DELETE's, made in current transaction to the database.

FireDAC supports nested transactions, so the current transaction is the one started with most recent StartTransaction (see page 419) call. If the database does not support nested transactions, like most of DBMS's, then FireDAC will emulate nested transactions using savepoints.

Before calling Rollback, an application may check the status of the Active (see page 414) property. If an application calls Rollback and there is no current transaction, an exception is raised.

Rollback call on Interbase / Firebird will close and unprepare all datasets and commands, associated with this transaction object. On some other DBMS's a call will invalidate all active result sets. For example, on MS SQL Server 2005.

Syntax

```
procedure Rollback;
```

See Also

Managing Transactions (see page 41), StartTransaction (see page 419), Active (see page 414), Commit (see page 417), RollbackRetaining (see page 419)

Example

```
procedure TForm1.ChangeButtonClick(Sender: TObject);
begin
  ADQuery1.Transaction := ADTransaction1;
  ADQuery1.SQL.Text := 'update employees set salary = salary * :k where id = :id';
  ADTransaction1.StartTransaction;
  try
    ADQuery1.ExecSQL('', [1.2, 100]);
    ADQuery1.ExecSQL('', [1.3, 200]);
    ADTransaction1.Commit;
  except
    ADTransaction1.Rollback;
    raise;
  end;
end;
```

1.17.1.1.13.1.16 TADCustomTransaction.RollbackRetaining Method

Cancels all modifications to the data made in the current transaction, without ending the current transaction.

Description

Call RollbackRetaining to cancel all modifications, like a INSERT's / UPDATE's / DELETE's, made in current transaction to the database.

FireDAC supports nested transactions, so the current transaction is the one started with most recent StartTransaction (see page 419) call. If the database does not support nested transactions, like most of DBMS's, then FireDAC will emulate nested transactions using savepoints.

Before calling RollbackRetaining, an application may check the status of the Active (see page 414) property. If an application calls RollbackRetaining and there is no current transaction, an exception is raised.

RollbackRetaining will use native functionality on IB/FB. On other DBMS's the method will be equal to calls of Rollback (see page 418), then of StartTransaction (see page 419).

Syntax

```
procedure RollbackRetaining;
```

See Also

Managing Transactions (see page 41), StartTransaction (see page 419), InTransaction, Commit (see page 417), Rollback (see page 418)

1.17.1.1.13.1.17 TADCustomTransaction.StartTransaction Method

Starts new transaction at database.

Description

Call StartTransaction to start a new transaction at the DBMS.

The FireDAC supports nested transactions. If DBMS does not support nested transactions, then FireDAC will emulate them using savepoints. If transaction is already active, then FireDAC will put savepoint, otherwise will start new transaction. Use Options (see page 416).EnableNested (see page 894) to enable / disable nested transactions.

Before calling StartTransaction, an application may adjust settings of the Options (see page 416) property as desired. Adjusting options after transaction is started does not affect current transaction.

All data modifications, like a INSERT's / UPDATE's / DELETE's, made after a call of StartTransaction, may be either confirmed by calling Commit (see page 417), either undone by calling Rollback (see page 418).

Some DBMS's will fail to start transaction, if there are active result sets. For example, MS SQL Server 2005.

Syntax

```
procedure StartTransaction;
```

See Also

Managing Transactions (see page 41), Active (see page 414), Commit (see page 417), Rollback (see page 418), Options (see page 416)

Example

```
procedure TForm1.ChangeButtonClick(Sender: TObject);
begin
  ADQuery1.Transaction := ADTransaction1;
  ADQuery1.SQL.Text := 'select * from employees';

  ADQuery2.Transaction := ADTransaction1;
  ADQuery2.SQL.Text := 'update employees set salary = salary * :k where id = :id';

  ADTransaction1.Options.Isolation := xiRepeatableRead;
  ADTransaction1.StartTransaction;
  try
    ADQuery1.Open;
    while not ADQuery1.Eof do begin
      ADQuery2.ExecSQL('', [1.2, ADQuery1.FieldByName('id').AsInteger]);
      ADTransaction1.CommitRetaining;
      ADQuery1.Next;
    end;
    ADTransaction1.Commit;
  except
    ADTransaction1.Rollback;
    raise;
  end;
end;
```

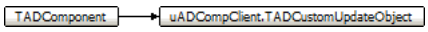
1.17.1.1.14 TADCustomUpdateObject Class

The base class for update objects used to override updates posting.

public

public	Description
DataSet (see page 421)	Identifies the dataset on which change log update object will operate.
Apply (see page 421)	Applies record update request to DB for current record in DataSet (see page 421).

Class Hierarchy



File

uADCompClient

Description

Use TADCustomUpdateObject as a base class when creating customized update objects that applies dataset updates the a database.

Syntax

```
TADCustomUpdateObject = class(TADComponent);
```

See Also

Overriding Posting Updates (see page 123), TADUpdateSQL (see page 563)

1.17.1.1.14.1 public

1.17.1.1.14.1.1 TADCustomUpdateObject.DataSet Property

Identifies the dataset on which change log update object will operate.

Description

TADCustomUpdateObject uses the DataSet property for two purposes:

- It fetches the original and updated field values from this dataset when performing parameter substitution.
- If this dataset is a TADRdbmsDataSet (see page 501) descendant, it uses its Connection (see page 503) and ConnectionName (see page 503) properties to identify the connection that it uses when performing its update queries.

If you are using a single update object, then the DataSet property is set automatically when you set the source dataset's UpdateObject (see page 267) property. If you are using multiple update objects, you must set the DataSet property at runtime in an OnUpdateRecord (see page 605) event handler.

Syntax

```
property DataSet: TADAdaptedDataSet;
```

See Also

TADDataSet.OnUpdateRecord (see page 605), TADRdbmsDataSet.Connection (see page 503), TADRdbmsDataSet.ConnectionName (see page 503)

1.17.1.1.14.1.2 TADCustomUpdateObject.Apply Method

Applies record update request to DB for current record in DataSet (see page 421).

Parameters

Parameters	Description
ARequest: TADUpdateRequest	A record update request.
var AAction: TADErrorAction	A completion status.
AOptions: TADUpdateRowOptions	An additional request options.

Description

Call Apply to apply changes of current DataSet (see page 421) record to database.

Standard implementation - TADUpdateSQL (see page 563) - sets parameters for a SQL statement and executes it to update a record. ARequest indicates the update kind, so for TADUpdateSQL (see page 563) specifies which SQL statement to execute. If appropriate SQL statement is not specified, then FireDAC will generate default one, as it is doing when TADAdaptedDataSet.UpdateObject (see page 267) is not specified.

Apply is primarily intended for manually executing update statements from an OnUpdateRecord (see page 605) event handler. There event handler ARequest, AAction and AOptions argument values must be assigned to Apply method corresponding arguments.

ARequest can be one of the following values:

Value	Meaning	TADUpdateSQL SQL property
arInsert	Insert record to database.	InsertSQL
arUpdate	Update record in database.	ModifySQL
arDelete	Delete record from database.	DeleteSQL
arLock	Lock record in database.	LockSQL
arUnlock	Release record lock in database.	UnlockSQL
arFetchRow	Refetch record from database.	FetchRowSQL
arUpdateHBlobs	Update Oracle BLOB/CLOB values in database.	-
arFetchGenerators	Fetch generator values to put into record.	-

AAction returns Apply method completion status, specifying what action should take the calling code. Normally this status is used by FireDAC code. When Apply method is called from OnUpdateRecord (see page 605) event handler, then assign AAction event handler argument to the AAction Apply method argument. Otherwise use eaDefault as initial value.

AOptions specifies additional options. When Apply method is called from OnUpdateRecord (see page 605) event handler, then assign AOptions event handler argument to the AOptions Apply method argument. Otherwise use [] as a value.

Syntax

```
procedure Apply(ARequest: TADUpdateRequest; var AAction: TADErrorAction; AOptions:
TADUpdateRowOptions); virtual; abstract;
```

See Also

Overriding Posting Updates (see page 123), TADUpdateSQL (see page 563), TADAdaptedDataSet.UpdateObject (see page 267)

Example 1

```
procedure TfrmCachedUpdates.qrProductsUpdateRecord(ASender: TDataSet;
  ARequest: TADUpdateRequest; var AAction: TADErrorAction;
  AOptions: TADUpdateRowOptions);
begin
  usProducts.ConnectionName := qrProducts.ConnectionName;
  usProducts.DataSet := qrProducts;
  usProducts.Apply(ARequest, AAction, AOptions);

  usCategories.ConnectionName := qrProducts.ConnectionName;
  usCategories.DataSet := qrProducts;
  usCategories.Apply(ARequest, AAction, AOptions);

  AAction := eaApplied;
end;
```

Example 2

See FireDAC\Samples\Comp Layer\TADUpdateSQL (see page 563)\Main demo application.

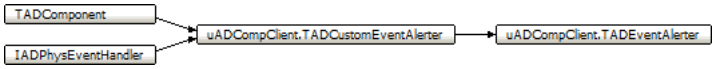
1.17.1.1.15 TADEventAlerter Class

The class is responsible for handling the database event notifications.

published

published	Description
Active (see page 423)	Controls the event registration state of the event alerter.
Connection (see page 423)	Specifies the connection component that actually forms the connection to the DBMS.
Names (see page 424)	Specifies the event names handled by this alerter.
OnAlert (see page 424)	The event fires, when a DBMS event is fired.
OnTimeout (see page 425)	The event fires, when a DBMS event is timed out.
Options (see page 425)	The set of even alerter options.

Class Hierarchy



File

uADCompClient

Description

Use the TADEventAlerter class to handle the database event notifications (see page 75).

The kind and general behavior of the events is DBMS specific. But TADEventAlerter provides unified way for handling them.

Syntax

```
TADEventAlerter = class(TADCustomEventAlerter);
```

See Also

Database Alerts (see page 75), TADCustomEventAlerter (see page 358)

Example 1

```
ADEventAlerter1.Options.Kind := 'DBMS_ALERT';
ADEventAlerter1.Names.Text := 'Customers';
ADEventAlerter1.Options.Synchronize := True;
ADEventAlerter1.Options.Timeout := 10000;
ADEventAlerter1.OnAlter := DoAlert;
ADEventAlerter1.OnTimeout := DoTimeout;
ADEventAlerter1.Active := True;
........

procedure TForm1.DoAlert(ASender: TADCustomEventAlerter;
  const AEventName: String; const AArgument: Variant);
begin
  if CompareText(AEventName, 'Customers') = 0 then
    qryCustomers.Refresh;
end;

procedure TForm1.DoTimeout(ASender: TObject);
begin
  qryCustomers.Refresh;
end;
```

Example 2

See the demo at: FireDAC\Samples\Comp Layer\TADEventAlerter\Main

1.17.1.1.15.1 published

1.17.1.1.15.1.1 TADEventAlerter.Active Property

Controls the event registration state of the event alerter.

Description

Use Active property to set or check when an event alerter has registered the Names events at a DBMS and waiting for events.

Set the Active property to True to register the events at a DBMS and start processing of incoming events. If an FireDAC DBMS driver supports few event alerters, then the event alerter kind must be explicitly assigned to the Options (see page 361).Kind (see page 848).

Set the property to False to stop processing of the events.

Syntax

```
property Active: Boolean;
```

See Also

Database Alerts (see page 75), Names, Register, Unregister

1.17.1.1.15.1.2 TADEventAlerter.Connection Property

Specifies the connection component that actually forms the connection to the DBMS.

Description

Set Connection to the connection component that establishes a connection to the DBMS. If to change Connection property value, while an even alerter is active, then it will unregister his events.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

Names, Register

1.17.1.1.15.1.3 TADEventAlerter.Names Property

Specifies the event names handled by this alerter.

Description

Use the Names property to specify the event names.

The event alerter will be notified only about the listed events. Note, that the Oracle DBMS_PIPE event alerter supports only a single event name. The event names may be case-sensitive or case-insensitive, depending on a DBMS.

Syntax

```
property Names: TStrings;
```

See Also

Database Alerts (see page 75), Register, Active

Example

```
with ADEventAlerter1 do begin
  Unregister;
  Names.Clear;
  Names.Add('Customers');
  Names.Add('Orders');
  Register;
end;
```

1.17.1.1.15.1.4 TADEventAlerter.OnAlert Event

The event fires, when a DBMS event is fired.

Description

Use the OnAlert event handler to specify an event handler, which will be fired when one of the specified in the Names DBMS events is fired. To start receive event notifications, the events must be registered using the Register (see page 362) method or the Active (see page 359) property. The event handler may be fired in the own event alerter thread (False) or in the main application thread (True), depending on the Options (see page 361).Synchronize (see page 848) option.

The event handler receives three arguments:

- ASender is the event alerter reference;
- AEventName specifies the name of the event;
- AArgument specifies the additional arguments to the event. The arguments are fully DBMS and application dependent. There must be a Null or Unassigned value, if no arguments are specified. The scalar variant value, when only single argument is specified. Or an variant array of values.

Syntax

```
property OnAlert: TADEventAlerterEvent;
```

See Also

Database Alerts (see page 75), OnTimeout, Names

Example

```
procedure TForm1.ADEventAlerter1Alert(ASender: TADCustomEventAlerter;
    const AEventName: String; const AArgument: Variant);
begin
  if CompareText(AEventName, 'Customers') = 0 then
    qryCustomers.Refresh
  else if CompareText(AEventName, 'Orders') = 0 then
```

```
    qryOrders.Refresh;
end;
```

1.17.1.1.15.1.5 TADEventAlerter.OnTimeout Event

The event fires, when a DBMS event is timed out.

Description

Use the OnTimeout event handler to specify an event handler, which will be fired after the specified by the Options (see page 361).Timeout (see page 849) time and no one event is fired. The specified timeout is in 1/1000 of sec.The event handler may be fired in the own event alerter thread (False) or in the main application thread (True), depending on the Options (see page 361).Synchronize (see page 848) option.

Syntax

```
property OnTimeout: TNotifyEvent;
```

See Also

Database Alerts (see page 75), OnAlert, Options

1.17.1.1.15.1.6 TADEventAlerter.Options Property

The set of even alerter options.

Description

Use the Options property to specify the different aspects of even handling.

Syntax

```
property Options: TADEventAlerterOptions;
```

See Also

Database Alerts (see page 75), TADEventAlerterOptions

1.17.1.1.16 TADLocalSQLDataSet Class

TADLocalSQLDataSet represents a dataset registered with Local SQL engine.

public

public	Description
ActualName (see page 426)	Returns actual dataset name.
Adapter (see page 426)	Specifies a dataset reference for the Local SQL engine.
IsValid (see page 426)	Returns True, if dataset may be registered.

published

published	Description
DataSet (see page 427)	Specifies a dataset reference for the Local SQL engine.
Name (see page 427)	Specifies alternative dataset name.
Temporary (see page 427)	Designates implicitly registered datasets.

Class Hierarchy



File

uADCompClient

Description

Use TADLocalSQLDataSet to create and maintain information about a dataset registered with Local SQL (see page 363) engine.

TADLocalSQLDataSet are collected into TADLocalSQLDataSets (see page 428) collection. The set of TADLocalSQLDataSet may be created explicitly as at design time, as at run time. Also implicitly by setting TADAdaptedDataSet.LocalSQL (see page 261) property value. Implicitly created items have Temporary (see page 427) property as True and they will not be stored in DFM.

DataSet (see page 427) property specifies the dataset to register. Name (see page 427) property specifies alternative name for the dataset.

Syntax

```
TADLocalSQLDataSet = class(TCollectionItem);
```

See Also

Local SQL (see page 96), TADLocalSQLDataSets (see page 428), TADCustomLocalSQL (see page 363), TADAdaptedDataSet (see page 259).LocalSQL (see page 261)

1.17.1.1.16.1 public

1.17.1.1.16.1.1 TADLocalSQLDataSet.ActualName Property

Returns actual dataset name.

Description

Use ActualName property to get a name, which will be used to register dataset.

It is Name (see page 427) value, when it is specified. Otherwise DataSet (see page 427).Name, when DataSet is specified. Otherwise is empty string.

Syntax

```
property ActualName: String;
```

See Also

Name (see page 427), DataSet (see page 427)

1.17.1.1.16.1.2 TADLocalSQLDataSet.Adapter Property

Specifies a dataset reference for the Local SQL engine.

Description

Use Adapter property to set a reference to a dataset adapter. A adapter is an COM interface providing additional dataset services to Local SQL engine.

FireDAC datasets are implementing this interface and do not require specify adapter. It is recommended to specify adapter for non-FireDAC datasets, although it is optional.

Syntax

```
property Adapter: IADPhysLocalQueryAdapter;
```

See Also

Local SQL (see page 96), DataSet (see page 427)

1.17.1.1.16.1.3 TADLocalSQLDataSet.IsValid Property

Returns True, if dataset may be registered.

Description

Use IsValid property to check that the dataset may be registered. It is possible, when DataSet is not nil, ActualName is not empty.

Syntax

```
property IsValid: Boolean;
```

See Also

DataSet (see page 427), ActualName (see page 426)

1.17.1.1.16.2 published

1.17.1.1.16.2.1 TADLocalSQLDataSet.DataSet Property

Specifies a dataset reference for the Local SQL engine.

Description

Use DataSet property to set a reference to a dataset and register it with the Local SQL engine.

The dataset may be any one TDataSet descendant. For non-FireDAC datasets is recommended also set Adapter property. If Name property value is specified, then it will be used a dataset name. Otherwise, TDataSet.Name will be used.

When dataset is specified and Local SQL engine is activated, then FireDAC will open the dataset, if it is closed. For in-memory datasets this means, that their structure must be defined before activating Local SQL engine or setting DataSet property.

Syntax

```
property DataSet: TDataSet;
```

See Also

Local SQL (see page 96), Name (see page 427), Adapter (see page 426)

1.17.1.1.16.2.2 TADLocalSQLDataSet.Name Property

Specifies alternative dataset name.

Description

Use Name property to specify alternative name for the registered dataset. When Name property is not specified, then DataSet.Name will be used. So, the property value is optional.

When a dataset was created at run-time, then DataSet.Name or Name properties must be specified. When one dataset is registered twice or more, then Name property must be specified to make dataset name unique.

The property value does not have limitations, like DataSet.Name. If the value is a valid SQLite DB identifier, then it may be referenced in Local SQL as is. Otherwise the name must be quoted.

Syntax

```
property Name: String;
```

See Also

Local SQL (see page 96), DataSet (see page 427)

1.17.1.1.16.2.3 TADLocalSQLDataSet.Temporary Property

Designates implicitly registered datasets.

Description

The Temporary property is set to True, when FireDAC dataset was registered using TADAdaptedDataSet.LocalSQL (see page 261) property or a dataset was submitted by TADCustomLocalSQL.OnGetDataSet (see page 365) event handler.

Items with Temporary = True are not saved in DFM file at design-time. Also TADLocalSQLDataSets.Remove method deletes corresponding items from a collection.

Syntax

```
property Temporary: Boolean;
```

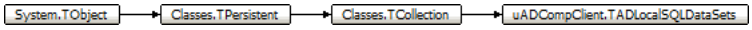
1.17.1.1.17 TADLocalSQLDataSets Class

TADLocalSQLDataSets is a collection of TADLocalSQLDataSet (see page 425) objects.

public

public	Description
Items (see page 428)	Returns TADLocalSQLDataSet (see page 425) object by it index.
Add (see page 428)	Appends new TADLocalSQLDataSet (see page 425) object to a collection.
Add (see page 429)	Appends new TADLocalSQLDataSet (see page 425) object with specified arguments to a collection.
Remove (see page 429)	Removes dataset registration from the collection.

Class Hierarchy



File

uADCompClient

Description

TADLocalSQLDataSets is a list of all datasets, registered with a Local SQL engine. A single TADLocalSQLDataSet (see page 425) object represents a single registered dataset. The datasets are registered and available for querying when TADCustomLocalSQL.Active (see page 364) is True.

Syntax

```
TADLocalSQLDataSets = class(TCollection);
```

See Also

Local SQL (see page 96), TADLocalSQLDataSet (see page 425)

1.17.1.1.17.1 public

1.17.1.1.17.1.1 TADLocalSQLDataSets.Items Property

Returns TADLocalSQLDataSet (see page 425) object by it index.

Description

Use Items indexed property to get TADLocalSQLDataSet (see page 425) object by it index.

The property is default one, so you does not need to write ADLocalSQL1.DataSets.Items[i], just ADLocalSQL1.DataSets[i].

Syntax

```
property Items [AIndex: Integer]: TADLocalSQLDataSet;
```

See Also

TADLocalSQLDataSet (see page 425), TCollection.Count

1.17.1.1.17.1.2 TADLocalSQLDataSets.Add Method ()

Appends new TADLocalSQLDataSet (see page 425) object to a collection.

Description

Call Add method to add new empty TADLocalSQLDataSet object to a collection.

Syntax

```
function Add: TADLocalSQLDataSet; overload;
```

See Also

Items (see page 428), TCollection.Count

Example

```
with ADLocalSQL1.DataSets.Add do begin
  DataSet := ADQuery1;
  Name := 'Orders';
end;
ADLocalSQL1.Active := True;
```

1.17.1.1.17.1.3 TADLocalSQLDataSets.Add Method (TDataSet, String, IADPhysLocalQueryAdapter)

Appends new TADLocalSQLDataSet (see page 425) object with specified arguments to a collection.

Parameters

Parameters	Description
ADataSet: TDataSet	A dataset to add. Mandatory.
const AName: String = ''	A dataset name. If not specified, then ADataSet.Name will be used. Optional.
const AAdapter: IADPhysLocalQueryAdapter = nil	A dataset adapter. Optional.

Description

Call Add method to add new TADLocalSQLDataSet (see page 425) object to a collection and set it corresponding properties to the specified arguments.

Syntax

```
function Add(ADataSet: TDataSet; const AName: String = ''; const AAdapter:
IADPhysLocalQueryAdapter = nil): TADLocalSQLDataSet; overload;
```

See Also

Items (see page 428), TCollection.Count

Example

```
ADLocalSQL1.DataSets.Add(ADQuery1, 'Orders');
ADLocalSQL1.Active := True;
```

1.17.1.1.17.1.4 TADLocalSQLDataSets.Remove Method

Removes dataset registration from the collection.

Parameters

Parameters	Description
ADataSet: TDataSet	A dataset to remove.

Description

Use Remove method to remove dataset registration from the collection.

Syntax

```
procedure Remove(ADataSet: TDataSet);
```

1.17.1.1.18 TADManager Class

The class is responsible to connection definitions and connections management.

published

published	Description
Active (see page 430)	Gets or sets FireDAC manager active status.
ActiveStoredUsage (see page 431)	Controls how to use Active property value saved to DFM.
AfterLoadConnectionDefFile (see page 431)	Fires after connection definitions file is loaded.

AfterShutdown (see page 431)	Fires after manager is finished.
AfterStartup (see page 431)	Fires after manager is started.
BeforeLoadConnectionDefFile (see page 432)	Fires before connection definitions file is loaded.
BeforeShutdown (see page 432)	Fires before manager will be finished.
BeforeStartup (see page 432)	Fires before manager will be started.
ConnectionDefFileAutoLoad (see page 432)	Controls how connection definitions file must be loaded.
ConnectionDefFileName (see page 433)	Gets / sets the name of a connection definitions file.
DriverDefFileAutoLoad (see page 433)	Controls how driver definitions file must be loaded.
DriverDefFileName (see page 434)	Gets / sets the name of a driver definitions file.
FetchOptions (see page 434)	The "root" fetch options.
FormatOptions (see page 434)	The "root" data format options.
ResourceOptions (see page 435)	The "root" resources control options.
SilentMode (see page 435)	Gets or sets FireDAC global "silent" flag.
UpdateOptions (see page 435)	The "root" data updating options.
WaitCursor (see page 435)	Gets or sets FireDAC wait cursor shape.

Class Hierarchy



File

uADCompClient

Description

Use TADCustomManager to manage connection definitions and connection objects.

Consider accessing the singleton instance of TADManager via ADManager (see page 570) function instead of its explicit creating.

Syntax

```
TADManager = class(TADCustomManager);
```

See Also

Defining Connection (see page 27), TADCustomManager (see page 365), ADManager (see page 570)

1.17.1.1.18.1 published

1.17.1.1.18.1.1 TADManager.Active Property

Gets or sets FireDAC manager active status.

Description

Set Active to True to activate FireDAC manager. After that the State (see page 375) = dmsActive. At activating, ADPhysManager loads driver definition file. When required set DriverDefFileAutoLoad (see page 373) and DriverDefFileName (see page 373) before activating manager. TADCustomConnection automatically activates manager before a first connection.

Set Active to False to terminate FireDAC manager. After that the State (see page 375) = dmsInactive. At terminating, manager:

- calls TADCustomConnection (see page 318).Close for all connection objects in application;
- ADPhysManager waits until all connections interfaces (IADPhysConnection) will be released;

- unloads DBMS client DLL's and releases internal driver structures.

Note, to change any driver settings, manager may be active or inactive, but there must be no connections through this driver.

We strongly recommend to not close FireDAC manager from within OnLosted (see page 333), OnRestored (see page 334), and OnRecover (see page 333) events of TADConnection component, as that can lead to unexpected problems.

A multi-threading application should activate FireDAC manager before threads, which are connecting to a DB, will start.

Syntax

```
property Active: Boolean;
```

See Also

DriverDefFileAutoLoad, DriverDefFileName, Open, Close

1.17.1.1.18.1.2 TADManager.ActiveStoredUsage Property

Controls how to use Active property value saved to DFM.

Description

The ActiveStoredUsage property controls how to use Active (see page 367) property value saved to DFM. Include:

- auDesignTime, to use property value at design time;
- auRunTime, to use property value at run time.

Syntax

```
property ActiveStoredUsage: TADStoredActivationUsage;
```

1.17.1.1.18.1.3 TADManager.AfterLoadConnectionDefFile Event

Fires after connection definitions file is loaded.

Description

The AfterLoadConnectionDefFile event is fired after the connection definitions file is loaded.

Syntax

```
property AfterLoadConnectionDefFile: TNotifyEvent;
```

See Also

BeforeLoadConnectionDefFile

1.17.1.1.18.1.4 TADManager.AfterShutdown Event

Fires after manager is finished.

Description

The AfterShutdown event is fired after FireDAC manager is terminated.

Syntax

```
property AfterShutdown: TNotifyEvent;
```

See Also

Active, Close, BeforeShutdown

1.17.1.1.18.1.5 TADManager.AfterStartup Event

Fires after manager is started.

Description

The AfterStartup event is fired after FireDAC manager is started.

Syntax

```
property AfterStartup: TNotifyEvent;
```

See Also

Active, Open, BeforeStartup

1.17.1.1.18.1.6 TADManager.BeforeLoadConnectionDefFile Event

Fires before connection definitions file is loaded.

Description

The BeforeLoadConnectionDefFile event is fired before the connection definitions file is loaded.

Syntax

```
property BeforeLoadConnectionDefFile: TNotifyEvent;
```

See Also

AfterLoadConnectionDefFile

1.17.1.1.18.1.7 TADManager.BeforeShutdown Event

Fires before manager will be finished.

Description

The BeforeShutdown event is fired before FireDAC manager will be terminated.

Note, multi-threaded application may close connections opened in the background threads in a BeforeShutdown event handler to avoid possible dead lock.

Syntax

```
property BeforeShutdown: TNotifyEvent;
```

See Also

Multi Threading (see page 46), Active, Close, AfterShutdown

1.17.1.1.18.1.8 TADManager.BeforeStartup Event

Fires before manager will be started.

Description

The BeforeStartup event is fired before FireDAC manager will be started.

Note, application cannot open a connection in a BeforeStartup event handler, as that will lead to the endless recursion.

Syntax

```
property BeforeStartup: TNotifyEvent;
```

See Also

Active, Open, AfterStartup

1.17.1.1.18.1.9 TADManager.ConnectionDefFileAutoLoad Property

Controls how connection definitions file must be loaded.

Description

The ConnectionDefFileAutoLoad property controls, should be a connection definition file loaded automatically or explicitly by programmer. If True, then FireDAC will load file at first "touch" to the content of the ConnectionDefs (see page 372) list.

If False, then programmer should call ConnectionDefs (see page 372).Load method before first "touch" to the ConnectionDefs (see page 372) list, including:

- opening a connection referencing to a connection definition by its name (see TADCustomConnection (see page 318).ConnectionDefName (see page 325));
- iterating through existing connection definitions (see TADCustomManager.ConnectionDefs (see page 372));
- adding new connection definition, deleting existing connection definition or looking for existing connection definition (see TADCustomManager.ConnectionDefs (see page 372));

Otherwise an exception will be raised.

Syntax

```
property ConnectionDefFileAutoLoad: Boolean;
```

See Also

Defining Connection (see page 27), ConnectionDefs, ConnectionDefFileLoaded, ConnectionDefFileName, LoadConnectionDefFile

1.17.1.1.18.1.10 TADManager.ConnectionDefFileName Property

Gets / sets the name of a connection definitions file.

Description

The ConnectionDefFileName property allows to get or set the name of a connection definition file.

The application can set the name before connection definitions file will be loaded. For example in BeforeLoadConnectionDefFile event handler.

If a path is a relative one or contains template markers, then it will be expanded into full path. Use ActualConnectionDefFileName to get the expanded value.

A path may include path variables (see page 163).

Syntax

```
property ConnectionDefFileName: String;
```

See Also

Defining Connection (see page 27), ConnectionDefs, ConnectionDefFileAutoLoad, ConnectionDefFileLoaded, ActualConnectionDefFileName

Example 1

```
ADManager.ConnectionDefFileName := ExtractFilePath(Application.ExeName) + 'myconndef.ini';
ADManager.ConnectionDefFileAutoLoad := True;
oConn := TADConnection.Create(nil);
oConn.ConnectionDefName := 'myconn';
oConn.Connected := True;
```

Example 2

```
procedure TMyDataModule.DoBeforeLoad(ASender: TNotifyEvent);
begin
  ADManager.ConnectionDefFileName := FSettings.MyConnectionDefFile;
end;
...
ADManager.BeforeLoadConnectionDefFile := DoBeforeLoad;
```

1.17.1.1.18.1.11 TADManager.DriverDefFileAutoLoad Property

Controls how driver definitions file must be loaded.

Description

The DriverDefFileAutoLoad property controls, should the driver definition file be loaded automatically or explicitly by programmer. When True, then FireDAC will load file at manager activation. When False, then programmer should call ADPhysManager.DriverDefs.Load method before manager activation. Otherwise an exception will be raised.

Syntax

```
property DriverDefFileAutoLoad: Boolean;
```

See Also

Configuring Drivers (see page 31), DriverDefFileName

1.17.1.1.18.1.12 TADManager.DriverDefFileName Property

Gets / sets the name of a driver definitions file.

Description

The DriverDefFileName property allows to get or set the name of a driver definition file. The application can set the name before manager activation.

The application can set the name before driver definitions file will be loaded.

If a path is a relative one or contains template markers, then it will be expanded into full path. Use ActualDriverDefFileName to get the expanded value.

A path may include path variables (see page 163).

Syntax

```
property DriverDefFileName: String;
```

See Also

Configuring Drivers (see page 31), DriverDefFileAutoLoad, ActualDriverDefFileName

1.17.1.1.18.1.13 TADManager.FetchOptions Property

The "root" fetch options.

Description

The FetchOptions property returns the reference to the "root" instance of the fetch options. These options control how the data will be fetched and cached in the memory.

The option values will be inherited by all TADCustomConnection (see page 318)'s and, consequently, by all TADCustomCommand (see page 291)'s and all datasets in application.

See Setting Options (see page 34) for review of how to work with options.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), FormatOptions, UpdateOptions, ResourceOptions

1.17.1.1.18.1.14 TADManager.FormatOptions Property

The "root" data format options.

Description

The FormatOptions property returns the reference to the "root" instance of the data format options. These options control how the DBMS data types will be mapped into dataset data types.

The option values will be inherited by all TADCustomConnection (see page 318)'s and, consequently, by all TADCustomCommand (see page 291)'s and all datasets in application.

See Setting Options (see page 34) for review of how to work with options.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), FetchOptions, UpdateOptions, ResourceOptions

1.17.1.1.18.1.15 TADManager.ResourceOptions Property

The "root" resources control options.

Description

The ResourceOptions property returns the reference to the "root" instance of the resources control options. These options control how FireDAC should use DBMS and workstation resources.

The option values will be inherited by all TADCustomConnection (see page 318)'s and, consequently, by all TADCustomCommand (see page 291)'s and all datasets in application.

See Setting Options (see page 34) for review of how to work with options.

Syntax

```
property ResourceOptions: TADTopResourceOptions;
```

See Also

Setting Options (see page 34), FetchOptions, UpdateOptions, FormatOptions

1.17.1.1.18.1.16 TADManager.SilentMode Property

Gets or sets FireDAC global "silent" flag.

Description

Set SilentMode to True to disable wait cursors and dialogs. This flag is application wide. Its value overrides all TADResourceOptions (see page 874).SilentMode (see page 884) values.

Syntax

```
property SilentMode: Boolean;
```

See Also

TADResourceOptions.SilentMode, WaitCursor

1.17.1.1.18.1.17 TADManager.UpdateOptions Property

The "root" data updating options.

Description

The UpdateOptions property returns the reference to the "root" instance of the data updating options. These options control how FireDAC should post data changes back to a DB.

The option values will be inherited by all TADCustomConnection's and, consequently, by all TADCustomTableAdapter's and all datasets in application.

See Setting Options (see page 34) for review of how to work with options.

Syntax

```
property UpdateOptions: TADUpdateOptions;
```

See Also

Setting Options (see page 34), FetchOptions, ResourceOptions, FormatOptions

1.17.1.1.18.1.18 TADManager.WaitCursor Property

Gets or sets FireDAC wait cursor shape.

Description

The WaitCursor property gets or sets the wait cursor shape.

Syntax

```
property WaitCursor: TADGUIxScreenCursor;
```

See Also

SilentMode

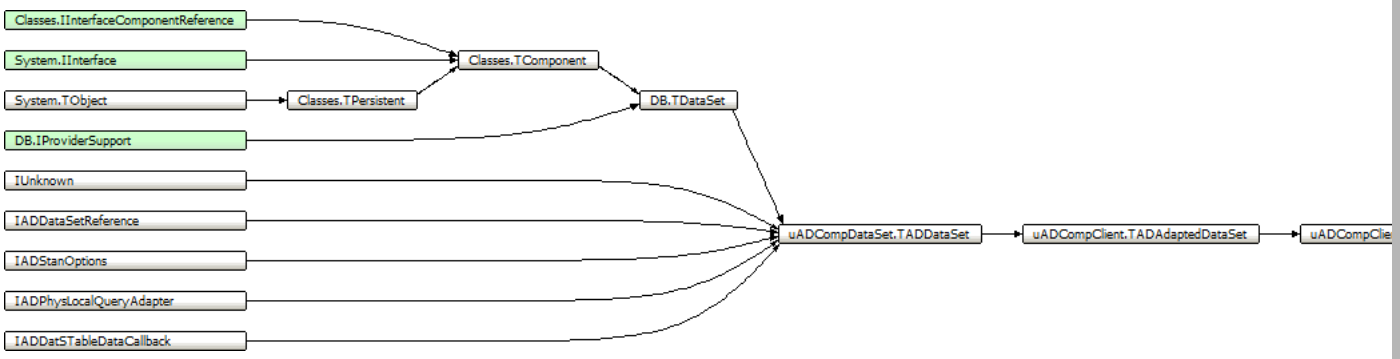
1.17.1.1.19 TADMemTable Class

The class implementing in-memory dataset.

published

published	Description
ActiveStoredUsage (see page 437)	Controls how to use Active property value saved to DFM.
Adapter (see page 437)	Add a short summary here...
AfterApplyUpdates (see page 438)	Fires after cached updates are applied to DB.
AfterExecute (see page 438)	Fires after SQL command execution is finished.
AfterGetRecord (see page 438)	Fires after fetching next rowset one time for each fetched record.
AfterGetRecords (see page 438)	Fires after fetching next rowset.
AfterRowRequest (see page 439)	Fires after fetching record.
Aggregates (see page 439)	The collection of client side aggregates, defined for dataset.
AggregatesActive (see page 440)	Controls automatic calculation of aggregate values.
BeforeApplyUpdates (see page 440)	Fires before cached updates are applied to DB.
BeforeExecute (see page 441)	Fires before SQL command execution is started.
BeforeGetRecords (see page 441)	Fires before fetching next rowset.
BeforeRowRequest (see page 441)	Fires before fetching record.
CachedUpdates (see page 441)	Specifies whether dataset will log changes to the data without immediate applying of them to the database.
Constraints (see page 442)	Specifies record-level client constraints that must be met when editing the data.
ConstraintsEnabled (see page 443)	Specifies whether the dataset perform constraint checking.
DetailFields (see page 443)	Gets / sets fields in detail dataset, used to establish a master-detail relationship with this dataset.
FetchOptions (see page 444)	The set of options to control rows fetching.
FilterChanges (see page 444)	Specifies what kinds of changed records must be "visible" in the dataset.
FormatOptions (see page 445)	The set of options to control data representation.
IndexDefs (see page 445)	Returns indexes definition for the dataset.
Indexes (see page 445)	Lists all client indexes that apply to the dataset.
IndexesActive (see page 446)	Controls automatic maintenance of Index'es.
IndexFieldNames (see page 447)	Lists the field names to use as an index.
IndexName (see page 447)	Gets / sets the current index for the dataset by its name.
LocalSQL (see page 448)	The reference to a Local SQL engine.
MasterFields (see page 448)	Gets / sets fields in master dataset, used to establish a master-detail relationship with this dataset.
MasterSource (see page 448)	Gets / sets a master data source, used to establish a master-detail relationship with this dataset.
OnReconcileError (see page 449)	Fires when a dataset needs to reconcile an update to a record that cannot not be applied.
OnUpdateError (see page 450)	Fires if an exception is generated when updates are applied to a database.
OnUpdateRecord (see page 450)	Occurs when record update is applying to a database.
ResourceOptions (see page 452)	The set of options to control resources usage.
UpdateOptions (see page 452)	The set of options to control updates posting.

Class Hierarchy



File

uADCompClient

Description

Use TADMemTable to manage data in the client memory and optionally exchange the data with a DBMS.

Syntax

```
TADMemTable = class(TADCustomMemTable);
```

See Also

TADDataSet (see page 584), TADCustomMemTable (see page 388)

Example

See FireDAC\Samples\Comp Layer\TADMemTable\Main sample for details of how to setup TADMemTable.

See FireDAC\Samples\Comp Layer\TADMemTable\MasterDetail sample for details of how to work with master-detail relation.

1.17.1.1.19.1 published

1.17.1.1.19.1.1 TADMemTable.ActiveStoredUsage Property

Controls how to use Active property value saved to DFM.

Description

The ActiveStoredUsage property controls how to use Active property value saved to DFM. Include:

- auDesignTime, to use property value at design time;
- auRunTime, to use property value at run time.

Syntax

```
property ActiveStoredUsage: TADStoredActivationUsage;
```

See Also

TDataSet.Active

1.17.1.1.19.1.2 TADMemTable.Adapter Property

Add a short summary here...

Description

Add a description here...

Syntax

```
property Adapter: TADCustomTableAdapter;
```

See Also

List here...

Example

1.17.1.1.19.1.3 TADMemTable.AfterApplyUpdates Event

Fires after cached updates are applied to DB.

Description

The AfterApplyUpdates event fires after cached updates are applied to DB as result of ApplyUpdates (see page 611) method call.

Syntax

```
property AfterApplyUpdates: TADAfterApplyUpdatesEvent;
```

See Also

TADAfterApplyUpdatesEvent, ApplyUpdates, BeforeApplyUpdates

1.17.1.1.19.1.4 TADMemTable.AfterExecute Event

Fires after SQL command execution is finished.

Description

The AfterExecute event fires after SQL command execution is finished as result of Execute (see page 622) method call.

Syntax

```
property AfterExecute: TADDataSetEvent;
```

See Also

TADDataSetEvent, Execute, BeforeExecute

1.17.1.1.19.1.5 TADMemTable.AfterGetRecord Event

Fires after fetching next rowset one time for each fetched record.

Description

The AfterGetRecord event fires after dataset fetched next rowset as result of navigation methods calls. The event fires one time for each fetched record. For example, fetching occurs automatically inside of Next, Last, Locate methods. Inside of an event handler application may read dataset fields to get fetched record field values.

Syntax

```
property AfterGetRecord: TADDataSetEvent;
```

See Also

AfterGetRecords

1.17.1.1.19.1.6 TADMemTable.AfterGetRecords Event

Fires after fetching next rowset.

Description

The AfterGetRecords event fires after dataset fetched next rowset as result of navigation methods calls. For example, fetching occurs automatically inside of Next, Last, Locate methods.

Syntax

```
property AfterGetRecords: TADDataSetEvent;
```

See Also

TADDataSetEvent, BeforeGetRecords, AfterGetRecord

1.17.1.1.19.1.7 TADMemTable.AfterRowRequest Event

Fires after fetching record.

Description

The AfterRowRequest event fires after dataset fetched current record as result of RefreshRecord (see page 642), FetchBlobs (see page 624) or FetchDetails (see page 625) method calls.

Syntax

```
property AfterRowRequest: TADDataSetEvent;
```

See Also

TADDataSetEvent, RefreshRecord, FetchBlobs, FetchDetails, BeforeRowRequest

1.17.1.1.19.1.8 TADMemTable.Aggregates Property

The collection of client side aggregates, defined for dataset.

Description

Use Aggregates to define client side aggregating formulas, that will be automatically maintained and calculated for group of or for all records of dataset.

All records in the group of records has the same field values for defined set of fields. Aggregates performing calculation for group of records must be associated with one of the indexes. These aggregate expressions will be calculated only is associated index is current and active.

Adding aggregate field does not add TADAggregate (see page 572) object to Aggregates collection. That are two alternative ways - to use aggregate fields or to use Aggregates.

DataSet will automatically maintain and calculate Aggregates values when dataset is fetching the data or application edits the data, if AggregatesActive (see page 590) is True. If application needs to perform large updates to dataset and aggregate values are not needed while updating, then set AggregatesActive (see page 590) to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

When aggregates are maintained, the Value method of every active aggregate object returns a value that reflects the current data in the dataset. When users edit the data in the dataset, these values are recalculated to reflect the user's changes.

The expressions in Aggregates must contain aggregating functions, like a SUM, COUNT. FireDAC supports expression syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write constraint expressions.

Syntax

```
property Aggregates: TADAggregates;
```

See Also

Calculated and Aggregated Fields (see page 108), AggregatesActive, BeginBatch, EndBatch

Example 1

```
with ADMemTable1.Aggregates.Add do begin
  Expression := 'sum(sal + bonus)';
```

```
  Active := True;
end;
with ADMemTable1.Aggregates.Add do begin
  Expression := 'sum(sal + bonus)';
  IndexName := 'by_deps';
  Active := True;
end;
ADMemTable1.IndexName := 'by_deps';
ADMemTable1.AggregatesActive := True;

Label1.Caption := 'Total payments : ' + VarToStr(ADMemTable1.Aggregates[0].Value);
Label2.Caption := 'Current department payments : ' +
VarToStr(ADMemTable1.Aggregates[1].Value);
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\Aggregates sample for details.

1.17.1.1.19.1.9 TADMemTable.AggregatesActive Property

Controls automatic calculation of aggregate values.

Description

Use AggregatesActive to get / set the flag controlling automatic maintenance and calculation of dataset aggregate values.

When AggregatesActive is False (the default), the dataset does not maintain aggregates. This allows the dataset to perform large updates to the dataset or fetch big volumes of data without overhead of calculating aggregate values, which happens at fetching or data editing. When AggregatesActive is True, the dataset calculates and maintains all aggregate values specified by the Aggregates (see page 590) property that are compatible with the current index.

To selectively enable and disable aggregates rather than turning them all on or off at once, use the Active (see page 575) property of individual TADAggregate (see page 572) objects. These objects are available through the Aggregates (see page 590) property.

If application needs to perform batch updates to dataset, then set AggregatesActive to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property AggregatesActive: Boolean;
```

See Also

Calculated and Aggregated Fields (see page 108),Aggregates, TADAggregate.Active (see page 575), BeginBatch, EndBatch

Example

```
var
  lPrevAggsActive: Boolean;
...
lPrevAggsActive := ADQuery1.AggregatesActive;
ADQuery1.AggregatesActive := False;
try
  // perform updates here, without calculating and reading client aggregates
finally
  ADQuery1.AggregatesActive := lPrevAggsActive;
end;
```

1.17.1.1.19.1.10 TADMemTable.BeforeApplyUpdates Event

Fires before cached updates are applied to DB.

Description

The BeforeApplyUpdates event fires before cached updates are applied to database as result of ApplyUpdates (see page 611) method call.

Syntax

```
property BeforeApplyUpdates: TADDataSetEvent;
```

See Also

TADDataSetEvent, ApplyUpdates, AfterApplyUpdates

1.17.1.1.19.1.11 TADMemTable.BeforeExecute Event

Fires before SQL command execution is started.

Description

The BeforeExecute event fires before SQL command execution is started as result of Execute (see page 622) method call.

Syntax

```
property BeforeExecute: TADDataSetEvent;
```

See Also

TADDataSetEvent, Execute, AfterExecute

1.17.1.1.19.1.12 TADMemTable.BeforeGetRecords Event

Fires before fetching next rowset.

Description

The BeforeGetRecords event fires before dataset will fetch next rowset as result of navigation methods calls. For example, fetching occurs automatically inside of Next, Last, Locate methods.

Syntax

```
property BeforeGetRecords: TADDataSetEvent;
```

See Also

TADDataSetEvent, AfterGetRecords

1.17.1.1.19.1.13 TADMemTable.BeforeRowRequest Event

Fires before fetching record.

Description

The BeforeRowRequest event fires before dataset will fetch current record as result of RefreshRecord (see page 642), FetchBlobs (see page 624) or FetchDetails (see page 625) method calls.

Syntax

```
property BeforeRowRequest: TADDataSetEvent;
```

See Also

TADDataSetEvent, RefreshRecord, FetchBlobs, FetchDetails, AfterRowRequest

1.17.1.1.19.1.14 TADMemTable.CachedUpdates Property

Specifies whether dataset will log changes to the data without immediate applying of them to the database.

Description

CachedUpdates enables (True) or disables (False) the logging of data changes (Insert/Post, Edit/Post, Delete) without immediate applying of them to the database.

An application must explicitly apply changes from change log to the database, using ApplyUpdates (see page 611) method. All changes will be written in comparably small amount of time in single transaction. The main benefits of enabling cached updates are:

- fewer transactions and shorter transaction times;
- minimization of network traffic;
- simplified implementation of undo / redo functionality for dataset;

- ability to implement application offline mode or briefcase model.

The potential drawbacks of enabling cached updates are:

- other applications can access and change the actual data on the server while users are editing local copies of the data, resulting in an update conflict when cached updates are applied to the database;
- other applications cannot access data changes made by an application until its cached updates are applied to the database.

Note, to change CachedUpdate property value for TADTable, it must be inactive.

Syntax

```
property CachedUpdates: Boolean;
```

See Also

Caching Updates (see page 114), ApplyUpdates, CancelUpdates, CommitUpdates

Example

```
ADQuery1.CachedUpdates := True;
...
ADQuery1.Edit;
...
ADQuery1.Post;
...
ADQuery1.Append;
...
ADQuery1.Post;
...
ADQuery1.ApplyUpdates;
ADQuery1.CommitUpdates;
ADQuery1.CachedUpdates := False;
```

1.17.1.1.19.1.15 TADMemTable.Constraints Property

Specifies record-level client constraints that must be met when editing the data.

Description

Use Constraints to maintain record-level constraints for the dataset.

Record-level constraints force business ruless limiting few fields in a single record. This constraints are checked at end of data editing (Post / AppendRecord / InsertRecord). Constraints, limiting single field are field constraints and must specified at TField.CustomConstraint or TField.ImportedConstraint.

The Constraints checking is performed when ConstraintsEnabled (see page 594) is True. If application needs to perform large updates to the dataset and application can guarantee the data consistancy, then set ConstraintsEnabled (see page 594) to False before updates, and return to original value after updates. The constraints are not checked at data fetching.

The expressions in Constraints must be predicates, evaluating to Boolean value. FireDAC supports expression syntax compatible with:

- BDE data access components;
- Client dataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write constraint expressions.

Syntax

```
property Constraints;
```

See Also

ConstraintsEnabled, DisableConstraints, EnableConstraints, BeginBatch, EndBatch, TDataSet.Post, TDataSet.AppendRecord, TDataSet.InsertRecord, TField.CustomConstraint, TField.ImportedConstraint

Example

```
with ADMemTable1.Constraints.Add do begin
```

```
    CustomConstraint := 'sal + bonus <= 2000';
    ErrorMessage := 'The employee payments must be equal or less than 2000 usd';
  end;

  ADMemTable1.ConstraintsEnabled := True;
  ADMemTable1.Edit;
  try
    ADMemTable1.FieldByName('sal').AsFloat := 1800;
    ADMemTable1.FieldByName('bonus').AsFloat := 300;
    // here exception will be raised
    ADMemTable1.Post;
  except
    ADMemTable1.Cancel;
    Application.HandleException(Self);
  end;

  ADMemTable1.ConstraintsEnabled := False;
  ADMemTable1.Edit;
  ADMemTable1.FieldByName('sal').AsFloat := 1800;
  ADMemTable1.FieldByName('bonus').AsFloat := 300;
  // here exception will be NOT raised, because constraint checking is disabled
  ADMemTable1.Post;
```

1.17.1.1.19.1.16 TADMemTable.ConstraintsEnabled Property

Specifies whether the dataset perform constraint checking.

Description

Use ConstraintsEnabled to get / set the flag controlling automatic record-level constraints enforcement.

When ConstraintsEnabled is False (the default), the dataset does not check Constraints (see page 594). This allows the dataset to perform the large data updates without overhead of checking constraints. When ConstraintsEnabled is True, the dataset checks Constraints (see page 594) at at end of data editing (Post / AppendRecord / InsertRecord).

If application needs to perform batch updates to dataset, then set ConstraintsEnabled to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property ConstraintsEnabled: Boolean;
```

See Also

Constraints, BeginBatch, EndBatch, DisableConstraints, EnableConstraints, TDataSet.Post, TDataSet.AppendRecord, TDataSet.InsertRecord, TADUpdateOptions.CheckRequired (see page 898)

Example

```
var
  lPrevConsEnabled: Boolean;
...
lPrevConsEnabled := ADQuery1.ConstraintEnabled;
ADQuery1.ConstraintEnabled := False;
try
  // perform updates here, without checking client constraints
finally
  ADQuery1.ConstraintEnabled := lPrevConsEnabled;
end;
```

1.17.1.1.19.1.17 TADMemTable.DetailFields Property

Gets / sets fields in detail dataset, used to establish a master-detail relationship with this dataset.

Description

Use DetailFields property value to specify semicolon separated list of the detail dataset field names, used to establish a master-detail relationship with this dataset.

The DetailFields property is useful only for parameter based master-detail setup. For range based setup the indexed fields are used as the detail field list. When DetailFields is not specified explicitly, then field names matching to query parameter names will be used as DetailFields value.

The DetailFields will be used only to assign specified field values when a new record is inserted into detail dataset. The mechanism used to filter the detail dataset records does not use DetailFields.

Syntax

```
property DetailFields: String;
```

See Also

Master-Detail Relationship (see page 104), MasterSource, MasterFields, IndexFieldNames, IndexName

Example

```
ADQuery1.SQL.Text := 'SELECT * FROM "Orders" WHERE CustomerID = :ID';
ADQuery1.MasterSource := CustomersDS;
ADQuery1.MasterFields := 'ID';
ADQuery1.DetailFields := 'CustomerID';
```

1.17.1.1.19.1.18 TADMemTable.FetchOptions Property

The set of options to control rows fetching.

Description

The FetchOptions property is the set of properties, controlling the fetching of cursor rows. These properties will inherit its values from FetchOptions of connection object.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), TADFetchOptions (see page 849)

1.17.1.1.19.1.19 TADMemTable.FilterChanges Property

Specifies what kinds of changed records must be "visible" in the dataset.

Description

The FilterChanges property allows to get / set changed records kinds, which must be accessible through navigation interface of dataset ("visible" in dataset):

Kind	Meaning
rtModified	Records changed after fetching or last CommitUpdates (see page 615) / CancelUpdates (see page 614).
rtInserted	New records, not yet posted to DB and added after dataset open or after last CommitUpdates (see page 615).
rtDeleted	Deleted records, not yet deleted from DB and deleted after dataset open or after last CommitUpdates (see page 615).
rtUnmodified	Unchanged records.
rtHasErrors	Records having associated errors after ApplyUpdates (see page 611). To get an error object use RowError (see page 607) property.

Syntax

```
property FilterChanges: TADUpdateRecordTypes;
```

See Also

Filtering Records (see page 102), Caching Updates (see page 114), RowError (see page 607), UpdateStatus (see page 650)

Example

```
// see only inserted records
ADQuery1.FilterChanges := [rtInserted];
```

1.17.1.1.19.1.20 TADMemTable.FormatOptions Property

The set of options to control data representation.

Description

The FormatOptions property is the set of properties, controlling the data type mapping, and some other aspects of data handling.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), TADFormatOptions (see page 860), Params

1.17.1.1.19.1.21 TADMemTable.IndexDefs Property

Returns indexes definition for the dataset.

Description

Use IndexDefs property to read or maintain indexes definitions.

The dataset also has Indexes collection. Application should use either IndexDefs, either Indexes, but not both in the same time. We suggest to use Indexes, as it is more flexible. IndexDefs property at first is needed for compatibility with other software using TDataSet, where IndexDefs is really defined.

The TADTable will automatically fill IndexDefs by database indexes, if fiMeta in FetchOptions (see page 651).Items (see page 855).

Syntax

```
property IndexDefs: TIndexDefs;
```

See Also

Sorting Records (see page 100), Indexes, TADFetchOptions.Items (see page 855)

1.17.1.1.19.1.22 TADMemTable.Indexes Property

Lists all client indexes that apply to the dataset.

Description

Use Indexes to define client side data views on the dataset records. Indexes are a collection of the TADIndex (see page 653) objects, each of them defines optional:

- Sorting by fields (Fields (see page 657), CaseInsFields (see page 655), DescFields (see page 656), Options (see page 658)).
- Sorting by expression (Expression (see page 656), Options (see page 658)).
- Records filter (Filter (see page 657), FilterOptions (see page 657)).
- Records distinction flag (Distinct (see page 656)).

The individual indexes are not just the sort and filter definitions. FireDAC maintains a consistent view on the data for each active index, while application fetches data or edits data. That creates additional overhead, but allows to switch between indexes without delays.

To make a view current, set it Selected (see page 659) property to True or dataset IndexName (see page 601) property to the view name. Note, the view must have Active = True to be maintained.

Dataset maintains Indexes views, when IndexesActive (see page 600) is True. When an application needs to perform batch updates to dataset, then set IndexesActive (see page 600) to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Setting IndexFieldNames (see page 600) does not add a TADIndex (see page 653) object to Indexes collection.

Defining IndexDefs (see page 598) before dataset opening, will clear Indexes and fill them using IndexDefs (see page 598) information. So, there are two alternative ways - use IndexDefs (see page 598) or use Indexes. We suggest to use Indexes, as it is more flexible.

Some of the navigation methods, like a Locate (see page 634) / Lookup (see page 637) / SetRange (see page 647) are using indexes to optimize their operations.

The sorting and filter expressions in Indexes supports syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

Syntax

```
property Indexes: TADIndexes;
```

See Also

Sorting Records (see page 100), Writing Expressions (see page 110), IndexDefs, IndexesActive, IndexFieldNames, IndexName, BeginBatch, EndBatch

Example 1

```
with ADMemTable1.Indexes.Add do begin
  Name := 'by_name';
  Fields := 'NAME;DT';
  Active := True;
end;
with ADMemTable1.Indexes.Add do begin
  Name := 'by_payments';
  Expression := 'sum(sal + bonus)';
  Filter := 'dep_id is not null;
  Active := True;
end;
ADMemTable1.IndexesActive := True;
ADMemTable1.IndexName := 'by_name';
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\Indices demo for details.

1.17.1.1.19.1.23 TADMemTable.IndexesActive Property

Controls automatic maintenance of Index'es.

Description

Use IndexesActive to indicate whether the dataset should maintain the data views, defined by the Indexes (see page 598) collection.

When IndexesActive is False, the dataset does not maintain the data views. That avoids the overhead of updating views, while data is fetched or is edited. When IndexesActive is True (by default), the dataset maintains the data views.

When indexes are maintained, each of them may be current index, selected by TADIndex (see page 653).Selected (see page 659) or by IndexName (see page 601) properties.

To selectively enable and disable data views rather than turning them all on or off at once, use the Active (see page 655) property of individual TADIndex (see page 653) objects. These objects are available through the Indexes (see page 598) property.

If application needs to perform batch updates to dataset, then set IndexesActive to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property IndexesActive: Boolean;
```

See Also

Sorting Records (see page 100), Indexes, BeginBatch, EndBatch

Example

```
var
  lPrevIndsActive: Boolean;
...
lPrevIndsActive := ADQuery1.IndexesActive;
ADQuery1.IndexesActive := False;
try
  // perform updates here, without maintaining client views
finally
  ADQuery1.IndexesActive := lPrevIndsActive;
end;
```

1.17.1.1.19.1.24 TADMemTable.IndexFieldNames Property

Lists the field names to use as an index.

Description

Use IndexFieldNames as an alternative method of specifying the current index to use for a dataset.

Specify the name of each field on which to index the dataset, separating names with semicolons. Ordering of field names is significant. Optionally specify the postfix for any of fields in form "field[:[D][A][N]]", where:

- 'D' - use descending sorting on this field;

- 'A' - use ascending sorting on this field;

- 'N' - use case-insensitive sorting on this field.

Use IndexFieldNames to create sort orders on the fly. The IndexFieldNames and IndexName (see page 601) properties are mutually exclusive. Setting one clears the other.

Syntax

```
property IndexFieldNames: String;
```

See Also

Sorting Records (see page 100), Indexes, IndexName

Example

```
ADQuery1.IndexFieldNames := 'order_date;customer_name:N';
```

1.17.1.1.19.1.25 TADMemTable.IndexName Property

Gets / sets the current index for the dataset by its name.

Description

Use IndexName to specify the current index for the dataset.

If IndexName is empty, the dataset's sort order is based on the IndexFieldNames (see page 600) property or on its default ordering as it is in dataset. If IndexName contains a name of valid index from Indexes collection, then that index is used to determine data view (including sort order) of records.

IndexFieldNames (see page 600) and IndexName are mutually exclusive. Setting one clears the other.

Syntax

```
property IndexName: String;
```

See Also

Sorting Records (see page 100), Indexes, IndexFieldNames

Example

```
with ADMemTable1.Indexes.Add do begin
  Name := 'by_name';
```

```
  Fields := 'NAME;DT';
  Active := True;
end;
ADMemTable1.IndexName := 'by_name';
```

1.17.1.1.19.1.26 TADMemTable.LocalSQL Property

The reference to a Local SQL engine.

Description

Use this property to get / set reference to a Local SQL engine.

When engine and Local SQL connection are active, then right after setting LocalSQL property, the dataset will be opened (if not yet) and will be registered in Local SQL engine. Otherwise dataset will be registered when engine and connection will be active. The TADMemTable (see page 436) structure must be defined before it will be opened.

The dataset will be registered using its Name property value. So, its Name property value must be not empty and unique across all datasets linked to the same Local SQL engine.

Setting this property to nil unregisters this dataset from Local SQL engine.

Syntax

```
property LocalSQL: TADCustomLocalSQL;
```

See Also

Local SQL (see page 96), TADLocalSQL (see page 825), TADMemTable (see page 436)

1.17.1.1.19.1.27 TADMemTable.MasterFields Property

Gets / sets fields in master dataset, used to establish a master-detail relationship with this dataset.

Description

Use MasterFields property value to specify semicolon separated list of the master dataset field names, used to establish a master-detail relationship with this dataset.

Specify MasterFields property value after assigning MasterSource (see page 603) property value. The mechanism used to filter this detail dataset records depends on the dataset class:

- TADQuery (see page 476) and TADStoredProc (see page 514) will match master fields to this query / procedure parameters by their names. When the master dataset current record is changed, then this dataset parameters will get values from the corresponding MasterFields.
- TADMemTable (see page 436) will match master fields to this dataset indexed fields by their positions. When the master dataset current record is changed, then a range will be applied to this dataset, where starting and ending range values are equal to the corresponding MasterFields field values. The application must have a current active index. The indexed fields will be the detail dataset fields used to establish mater-detail.

Syntax

```
property MasterFields: String;
```

See Also

Master-Detail Relationship (see page 104), MasterSource, DetailFields, IndexFieldNames, IndexName

Example

```
ADMemTable1.IndexFieldNames := 'CustomerID';
ADMemTable1.MasterSource := CustomersDS;
ADMemTable1.MasterFields := 'ID';
```

1.17.1.1.19.1.28 TADMemTable.MasterSource Property

Gets / sets a master data source, used to establish a master-detail relationship with this dataset.

Description

Use MasterSource to specify the data source linked to a dataset, which will be used as master dataset in master-detail relationship between this and specified datasets.

Additionally application must specify MasterFields (see page 602) property value. See MasterFields (see page 602) property description for additional information.

Syntax

```
property MasterSource: TDataSource;
```

See Also

Master-Detail Relationship (see page 104), MasterFields, IndexFieldNames, IndexName

Example 1

```
ADMemTable1.IndexFieldNames := 'CustomerID';
ADMemTable1.MasterSource := CustomersDS;
ADMemTable1.MasterFields := 'ID';
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\MasterDetail demo for details.

1.17.1.1.19.1.29 TADMemTable.OnReconcileError Event

Fires when a dataset needs to reconcile an update to a record that cannot not be applied.

Description

Use the OnReconcileError event handler to respond to error conditions that arise when the ApplyUpdates (see page 611) method was applying changes to the records to the database. The OnReconcileError event handler is called by Reconcile (see page 642) method.

As part of Reconcile call the dataset will loop through all records with associated exception objects. And for each such record will call the OnReconcileError event handler. The event handler gets:

- E - reference to the exception object. See Handling Errors (see page 44) topics for how to work with exception objects.
- UpdateKind - the update kind of the current record. It may be one of the values: rsInserted, rsDeleted, rsModified, rsUnchanged.

After handling error the event handler should set Action argument. The default value is raMerge. The possible values are:

Action	Description
raSkip	Just skip current record.
raAbort	Just quit from Reconcile call.
raMerge	Clear the current record error state, make current record changes the new initial state of this record. IOW, merge changes into dataset records cache.
raCorrect	Clear current record error state. IOW, mark the record as correctly applied.
raCancel	Cancel current record changes.
raRefresh	Cancel current record changes and reread the record values from the database.

The event handler may analyze the original and current field values, by reading TField.OldValue and NewValue properties. Application may also update the current field value, set Action to raCorrect, and later call ApplyUdpates again.

Syntax

```
property OnReconcileError: TADReconcileErrorEvent;
```

See Also

Caching Updates (see page 114), Reconcile, ApplyUpdates, OnUpdateError

Example

```
procedure TForm1.ADMemTable1ReconcileError(DataSet: TADDataSet; E: EADException;
  UpdateKind: TADDatSRowState; var Action: TADDAptReconcileAction);
begin
  if (UpdateKind = rsInserted) and (E is EADDBEngineException) and
(EADDBEngineException(E).Kind = ekUKViolated) then begin
    DataSet.FieldByName('ID').AsInteger := GetNextFreeID;
```

```
    Action := raCorrect;
  end;
end;
```

1.17.1.1.19.1.30 TADMemTable.OnUpdateError Event

Fires if an exception is generated when updates are applied to a database.

Description

Use the OnUpdateError event handler to respond to exceptions raised while applying immediate or cached updates to a database.

ASender is the dataset to which updates are applied.

AException is a pointer to a EADException (see page 836) object from which an application can extract an error message and the actual cause of the error condition. The OnUpdateError handler can use this information to determine how to respond to the error condition. In most cases AException object will be of class EADDBEngineException (see page 833) or even one of it DBMS specific subclasses.

ARow is a DatS row object, repesenting erroneous record in dataset. This record is also current record in ASender.

ARequest indicates whether the error occurred while inserting, deleting, or modifying a record.

AAction indicates the action to take when the OnUpdateError handler exits. On entry into the handler, AAction is always set to eaDefault, which leads to uaFail, if not changed. If OnUpdateError can handle or correct the error, set AAction to uaRetry before exiting the error handler. Or consider other options.

The error handler can use the OldValue and NewValue properties TField to evaluate error conditions and set NewValue to a new value to reapply. In this case, set AAction to uaRetry before exiting.

The code in an OnUpdateError handler must not call any methods that make a different record the current one !

Syntax

```
property OnUpdateError: TADUpdateErrorEvent;
```

See Also

Caching Updates (see page 114), ApplyUpdates, CachedUpdates, Post, Delete, EADDBEngineException, OnUpdateRecord

Example

```
procedure TForm1.ADQuery1UpdateError(ASender: TDataSet; AException: EADException;
    ARow: TADDatSRow; ARequest: TADUpdateRequest; var AAction: TADErrorAction);
begin
  if (E is EADDBEngineException) and (EADDBEngineException(E).Kind = ekUKViolated) then
begin
    DataSet.FieldByName('ID').AsInteger := GetNextFreeID;
    Action := eaRetry;
  end;
end;
```

1.17.1.1.19.1.31 TADMemTable.OnUpdateRecord Event

Occurs when record update is applying to a database.

Description

Use an OnUpdateRecord event handler to process immediate or cached updates that cannot be correctly or easily handled:

- using FireDAC automatic SQL update command generation (see page 121);
- by a single table adapter component;
- by a single update component.

This event is useful for applications that require:

- additional control over parameters and macros substitution in update components;
- multi SQL statement updates, when DBMS does not support batches or blocks;

- non-SQL updates posting;
- etc

ASender is the dataset to which updates are applied.

ARequest indicates whether the current update is the insertion, deletion or modification of a record.

AAction indicates the action taken by the OnUpdateRecord handler before it exits. On entry into the handler, AAction is always set to eaDefault. If OnUpdateRecord is successful, it should set AAction to eaApplied before exiting.

AAction value	Description
eaFail	Mark the update as failed and return an error.
eaSkip	Skip current update and do not mark it as applied.
eaRetry	Retry the current operation.
eaApplied	Mark current update as applied.
eaDefault	Take default action. For successful update it is eaApplied, for failed update it is eaFail.
eaExitSuccess	Stop to process the cached updates, return success.
eaExitFailure	Stop to process the cached updates, return failure.

The OnUpdateRecord event handler code should read the dataset field values, including TField.NewValue, OldValue and CurValue. There:

- NewValue is the new field value before posting updates.
- OldValue is the original field value, as it was after record fetching or after last updates applying.
- CurValue is the current field value, the same as Value property.

Note, the OnUpdateRecord handler code must not call the methods that change the current dataset position.

Syntax

```
property OnUpdateRecord: TADUpdateRecordEvent;
```

See Also

Overriding Posting Updates (see page 123), ApplyUpdates, CachedUpdates, Post, Delete, OnUpdateError

Example 1

```
procedure TForm1.ADQuery1UpdateRecord(ASender: TDataSet; ARequest: TADUpdateRequest;
 var AAction: TADErrorAction; AOptions: TADUpdateRowOptions);
begin
  if ARequest = arInsert then begin
    // set the SQL command to insert new record
    ADQuery2.SQL := 'insert into mytab (id, code, name) values (:id, :code, :name)
returning tmstamp into :ts';

    // set parameter values
    ADQuery2.Params[0].Value := ASender['id'];
    ADQuery2.Params[1].Value := ASender['code'];
    ADQuery2.Params[2].Value := ASender['name'];

    // specially define TS parameter
    ADQuery2.Params[3].DataType := ftDateTime;
    ADQuery2.Params[3].ParamType := ptOutput;

    // insert new record
    ADQuery2.ExecSQL;

    // move TS output parameter value back to dataset
    ASender['tmstamp'] := ADQuery2.Params[3].Value;

    // return 'Ok' status
    AAction := eaApplied;
  end;
end;
```

Example 2

See demos:

- FireDAC\Samples\Comp Layer\TADQuery\CachedUpdates\OnUpdateRecord
- FireDAC\Samples\Comp Layer\TADUpdateSQL\Main

1.17.1.1.19.1.32 TADMemTable.ResourceOptions Property

The set of options to control resources usage.

Description

The ResourceOptions property is the set of properties, controlling the resources usage by the dataset. These properties will inherit its values from ResourceOptions of connection object.

Syntax

```
property ResourceOptions: TADBottomResourceOptions;
```

See Also

Setting Options (see page 34), TADBottomResourceOptions (see page 841)

1.17.1.1.19.1.33 TADMemTable.UpdateOptions Property

The set of options to control updates posting.

Description

The UpdateOptions property is the set of properties, controlling the posting updates from the dataset. These properties will inherit its values from UpdateOptions of connection object.

Syntax

```
property UpdateOptions: TADBottomUpdateOptions;
```

See Also

Setting Options (see page 34), TADBottomUpdateOptions (see page 843)

1.17.1.1.20 TADMetaInfoCommand Class

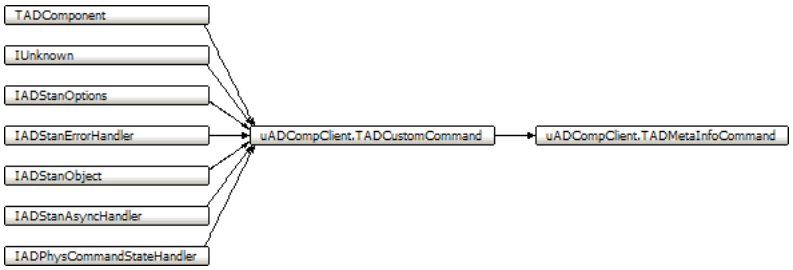
The class is responsible for execution of meta-data commands.

published

published	Description
Active (see page 453)	Gets or sets command active status.
AfterClose (see page 454)	The event fires after command is closed.
AfterOpen (see page 454)	The event fires after command is opened.
BaseObjectName (see page 454)	Gets / sets the name of base object.
BeforeClose (see page 455)	The event fires before command will be closed.
BeforeOpen (see page 455)	The event fires before command will be opened.
CatalogName (see page 455)	Identifies catalog name for some command kinds.
Connection (see page 456)	Specifies the connection object.
ConnectionName (see page 456)	Specifies connection name.
FormatOptions (see page 456)	The set of options to control data representation.
MetaInfoKind (see page 457)	Gets / sets the kind of metadata to retrieve.
ObjectName (see page 457)	Gets / sets the name of object to get detailed information about it.
ObjectScopes (see page 458)	Gets / sets object scopes filter.
OnCommandChanged (see page 458)	The event fires after CommandText is changed.

OnError (see page 459)	The event fires when an error happens, while command is communicating with DBMS.
Overload (see page 459)	Oracle stored procedure overload index.
SchemaName (see page 460)	Identifies schema name for some of command kinds.
TableKinds (see page 460)	Gets / sets table kind filter.
Wildcard (see page 461)	Gets / sets objects filter by their name.

Class Hierarchy



File

uADCompClient

Description

Use the TADMetaInfoCommand class to execute meta-data commands.

The class does not provide TDataSet-aware access to returned resultsets.

Syntax

```
TADMetaInfoCommand = class(TADCustomCommand);
```

See Also

Working with Metadata (see page 127), TADCustomCommand (see page 291), IADPhysCommand, TADMetaInfoQuery (see page 461)

1.17.1.1.20.1 published

1.17.1.1.20.1.1 TADMetaInfoCommand.Active Property

Gets or sets command active status.

Description

Set Active to True to execute SQL command and return cursor. After that the State (see page 306) = csOpen and you can fetch rows from current cursor using Fetch (see page 312) method. Setting Active to True, will call Open (see page 316) method.

Set Active to False to close current command cursor. After that State (see page 306) = csPrepared. Setting Active to False, will call Close (see page 308)method. To close all command cursors (if command returns multiple result sets), use CloseAll (see page 309) method.

Syntax

```
property Active: Boolean;
```

See Also

Fetch, Define, Open, Close, CloseAll, State

Example

```
ADCommand1.CommandText.Text := 'select * from "Orders"';
ADCommand1.Active := True;
oTab := ADCommand1.Define;
try
```

```
    ADCommand1.Fetch(oTab);
    ....
  finally
    oTab.Free;
  end;
```

1.17.1.1.20.1.2 TADMetaInfoCommand.AfterClose Event

The event fires after command is closed.

Description

The AfterClose event fires, after command is closed by calling Close (see page 308) or setting Active (see page 293) = False.

Syntax

```
property AfterClose: TNotifyEvent;
```

See Also

BeforeClose, Close, Active

1.17.1.1.20.1.3 TADMetaInfoCommand.AfterOpen Event

The event fires after command is opened.

Description

The AfterOpen event fires, after command is opened by calling Open (see page 316) or setting Active (see page 293) = True.

Syntax

```
property AfterOpen: TNotifyEvent;
```

See Also

BeforeOpen, Open, Active

1.17.1.1.20.1.4 TADMetaInfoCommand.BaseObjectName Property

Gets / sets the name of base object.

Description

The BaseObjectName property value meaning and optionality depends on MetaInfoKind (see page 457):

MetaInfoKind	Meaning
mkIndexFields	Table name.
mkPrimaryKeyFields	Table name.
mkForeignKeyFields	Table name.
mkProcs	Package name.
mkProcArgs	Package name.

The BaseObjectName property value together with CatalogName (see page 455) and SchemaName (see page 460) constitutes the full base object name. If BaseObjectName is case sensitive or contains special character, then it must be explicitly quoted.

Assignment to this property value closes and unprepares command.

Syntax

```
property BaseObjectName: String;
```

See Also

MetaInfoKind (see page 457), CatalogName (see page 455), SchemaName (see page 460), ObjectName (see

page 457)

Example 1

```
// SQL Server: retrieve foreign key fields for FK_DEPARTMENTS constraint of the
Northwind.dbo.Employees table
var
  oTab: TADDatSTable;
....
  ADMetaInfoCommand1.BaseObjectName := 'Northwind.dbo.Employees';
  ADMetaInfoCommand1.ObjectName := 'FK_DEPARTMENTS';
  ADMetaInfoCommand1.MetaInfoKind := mkForeignKeyFields;
  ADMetaInfoCommand1.Define(oTab);
  ADMetaInfoCommand1.Open(oTab);
  ADMetaInfoCommand1.Fetch(oTab, True);
```

Example 2

```
// Oracle: retrieve procedure list for the SYS.DBMS_SQL package
var
  oTab: TADDatSTable;
....
  ADMetaInfoCommand1.BaseObjectName := 'SYS.DBMS_SQL';
  ADMetaInfoCommand1.MetaInfoKind := mkProcs;
  ADMetaInfoCommand1.Define(oTab);
  ADMetaInfoCommand1.Open(oTab);
  ADMetaInfoCommand1.Fetch(oTab, True);
```

1.17.1.1.20.1.5 TADMetaInfoCommand.BeforeClose Event

The event fires before command will be closed.

Description

The BeforeClose event fires, before command will be closed by calling Close (see page 308) or setting Active (see page 293) = False.

Syntax

```
property BeforeClose: TNotifyEvent;
```

See Also

AfterClose, Close, Active

1.17.1.1.20.1.6 TADMetaInfoCommand.BeforeOpen Event

The event fires before command will be opened.

Description

The BeforeOpen event fires, before command will be opened by calling Open (see page 316) or setting Active (see page 293) = True.

Syntax

```
property BeforeOpen: TNotifyEvent;
```

See Also

AfterOpen, Open, Active

1.17.1.1.20.1.7 TADMetaInfoCommand.CatalogName Property

Identifies catalog name for some command kinds.

Description

The CatalogName identifies the name of the catalog where resides command object. The meaning depends on CommandKind (see page 298) and MetainfoKind (see page 457):

- If CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs], then CatalogName (see page 297) specifies the name of the catalog, where resides the procedure.

- If MetainfoKind (see page 457) is not mkNone, then CatalogName (see page 297) specifies the name of the catalog, where resides the describing object.

Syntax

```
property CatalogName: String;
```

See Also

CommandKind, TADMetaInfoCommand.MetaInfoKind (see page 457), SchemaName

Example 1

```
ADCommand1.CatalogName := 'MY_APP';
ADCommand1.CommandText.Text := 'PROC1';
ADCommand1.CommandKind := skStoredProc;
ADCommand1.Prepare;
ADCommand1.Params[0].AsInteger := 100;
ADCommand1.Params[1].AsString := 'Bill Gates';
ADCommand1.Execute;
```

Example 2

```
ADMetaInfoCommand1.CatalogName := 'MY_APP';
ADMetaInfoCommand1.ObjectName := 'MY_TAB';
// or on SQL Server just ADMetaInfoCommand1.ObjectName := 'MY_APP..MY_TAB';
ADMetaInfoCommand1.MetaInfoKind := mkTableFields;
ADMetaInfoCommand1.Open;
```

1.17.1.1.20.1.8 TADMetaInfoCommand.Connection Property

Specifies the connection object.

Description

The Connection property value points to the connection object. Alternatively may be specified the ConnectionName (see page 300) property value. The Connection property value must be specified before Prepare (see page 317) call.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

ConnectionName, Prepare

1.17.1.1.20.1.9 TADMetaInfoCommand.ConnectionName Property

Specifies connection name.

Description

The ConnectionName property value specifies the name of the connection. It must match to the:

- name of one of connection definitions, either stored in external file or created on fly;
- ConnectionName of one of the TADConnection (see page 279) objects.

Alternatively may be specified the Connection (see page 299) property value. The ConnectionName property value must be specified before Prepare (see page 317)call. If is specified the name of one of connection definitions, then FireDAC will transparently create connection object and link it with command.

Syntax

```
property ConnectionName: String;
```

See Also

Connection, Prepare

1.17.1.1.20.1.10 TADMetaInfoCommand.FormatOptions Property

The set of options to control data representation.

Description

The FormatOptions property is the set of properties, controlling the data type mapping, and some other aspects of data handling.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), TADFormatOptions, Fetch, Params

1.17.1.1.20.1.11 TADMetaInfoCommand.MetaInfoKind Property

Gets / sets the kind of metadata to retrieve.

Description

The MetaInfoKind property determines what meta information will be retrieved. If DBMS or FireDAC driver does not support specified MetaInfoKind (for example, mkGenerators for Sybase SQL Anywhere), then no exception will be raised at Open and no records will be returned.

Assignment to this property value closes and unprepares command.

See Working with Metadata (see page 127) for structure of returned datasets.

Syntax

```
property MetaInfoKind: TADPhysMetaInfoKind;
```

See Also

CatalogName (see page 455), SchemaName (see page 460), BaseObjectName (see page 454), Overload (see page 459), ObjectName (see page 457)

Example

1.17.1.1.20.1.12 TADMetaInfoCommand.ObjectName Property

Gets / sets the name of object to get detailed information about it.

Description

The ObjectName property value defines the database objects, for which will be returned detailed information. The meaning, optionality and usage of catalog and schema names depends on MetaInfoKind (see page 457):

MetaInfoKind	Meaning	Catalog and schema names
mkTableFields	Table name.	Applicable.
mkIndexes	Table name.	Applicable.
mkIndexFields	Index name.	--
mkPrimaryKey	Table name.	Applicable.
mkForeignKeys	Table name.	Applicable.
mkForeignKeyFields	Foreign key constraint name.	--
mkProcArgs	Stored procedure name.	Applicable, for non-packages stored procedures.

The ObjectName together with CatalogName (see page 455) and SchemaName (see page 460) may constitute the full object name. If object name is case sensitive or contains special character, then it must be explicitly quoted.

Assignment to this property value closes dataset.

Syntax

```
property ObjectName: String;
```

See Also

MetaInfoKind (see page 457), CatalogName (see page 455), SchemaName (see page 460), BaseObjectName (see page 454)

Example

```
var
  oTab: TADDatSTable;
....
  ADMetaInfoCommand1.CatalogName := 'Northwind';
  ADMetaInfoCommand1.SchemaName := 'dbo';
  ADMetaInfoCommand1.ObjectName := 'Employees';
  ADMetaInfoCommand1.MetaInfoKind := mkTableFields;
  ADMetaInfoCommand1.Define(oTab);
  ADMetaInfoCommand1.Open(oTab);
  ADMetaInfoCommand1.Fetch(oTab, True);
```

1.17.1.1.20.1.13 TADMetaInfoCommand.ObjectScopes Property

Gets / sets object scopes filter.

Description

Use the ObjectScopes property value to filter database objects by their scope:

Scope	Meaning
osMy	Objects created by the current login user.
osSystem	Objects belonging to the DBMS.
osOther	All other objects.

Assignment to this property value closes and unprepares command.

Syntax

```
property ObjectScopes: TADPhysObjectScopes;
```

See Also

TableKinds (see page 460)

1.17.1.1.20.1.14 TADMetaInfoCommand.OnCommandChanged Event

The event fires after CommandText is changed.

Description

The OnCommandChanged event fires, after CommandText (see page 298) is changed.

Syntax

```
property OnCommandChanged: TNotifyEvent;
```

See Also

CommandText

Example

```
procedure TForm1.ADCommand1CommandChanged(ASender: TObject);
begin
  if ADCommand1.FindParam('ID') <> nil then
    ADCommand1.FindParams('ID').AsInteger := 100;
end;
```

1.17.1.1.20.1.15 TADMetaInfoCommand.OnError Event

The event fires when an error happens, while command is communicating with DBMS.

Parameters

Parameters	Description
ASender	The reference to command.
AInitiator	The reference to interface of object, which initially raised the exception.
AException	The exception object, which may be modified or even replaced by another exception object instance.

Description

The OnError event fires, when command executes one of the following operations and an error happens:

- Prepare (see page 317). The command is preparing DBMS command text for execution.
- Open (see page 316). The command is executing DBMS command to open cursor.
- Execute (see page 311). The command is executing DBMS command text.
- Fetch (see page 312). The command is fetching rows from cursor.

If most cases AException object will of EADDBEngineException (see page 833) class. If AException is of EADDBArrayExecuteError (see page 831) class, then it is Array DML (see page 82) error handling case. You should use code like in the example.

Syntax

```
property OnError: TADErrorEvent;
```

See Also

Handling Errors (see page 44), Array DML (see page 82), Open, Execute, Prepare, Fetch, EADDBArrayExecuteError (see page 831) class, EADDBEngineException class

Example 1

Handling Array DML errors:

```
procedure TForm1.ADCommand1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if AException is EADPhysArrayExecuteError then
    with EADPhysArrayExecuteError(AException) do
      if Errors[0].Kind = ekUKViolated then
        AAction := eaSkip
      else
        AAction := eaFail
  else
    ...
end;
```

Example 2

Error substitution:

```
procedure TForm1.ADCommand1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if EADDBEngineException(AException).Errors[0].Kind = ekPKViolated then begin
    AException.Free;
    AException := Exception.Create('Please, enter unique ID value');
  end;
end;
```

1.17.1.1.20.1.16 TADMetaInfoCommand.Overload Property

Oracle stored procedure overload index.

Description

The Overload property specifies Oracle packaged stored procedure overload index. It has no meaning for other DBMS's.

Syntax

```
property Overload: Word;
```

See Also

CommandText, CommandKind

Example

```
ADCommand1.CommandKind := skStoredProc;
ADCommand1.CommandText.Text := 'DBMS_SQL.PARSE';
ADCommand1.Overload := 1;
```

1.17.1.1.20.1.17 TADMetaInfoCommand.SchemaName Property

Identifies schema name for some of command kinds.

Description

The SchemaName identifies the name of the schema where resides command object. The meaning depends on CommandKind (see page 298) and MetainfoKind (see page 457):

- If CommandKind (see page 298) in [skStoredProc, skStoredProcWithCrs, skStoredProcNoCrs], then SchemaName specifies the name of the schema, where resides the procedure.
- If MetainfoKind (see page 457) is not mkNone, then SchemaName specifies the name of the schema, where resides the describing object.

Syntax

```
property SchemaName: String;
```

See Also

CommandKind, TADMetaInfoCommand.MetaInfoKind (see page 457), SchemaName

Example 1

```
ADCommand1.SchemaName := 'SCOTT';
ADCommand1.CommandText.Text := 'PROC1';
ADCommand1.CommandKind := skStoredProc;
ADCommand1.Prepare;
ADCommand1.Params[0].AsInteger := 100;
ADCommand1.Params[1].AsString := 'Bill Gates';
ADCommand1.Execute;
```

Example 2

```
ADMetaInfoCommand1.SchemaName := 'ADDEMO';
ADMetaInfoCommand1.ObjectName := 'MY_TAB';
// or on Oracle just ADMetaInfoCommand1.ObjectName := 'ADDEMO.MY_TAB';
ADMetaInfoCommand1.MetaInfoKind := mkTableFields;
ADMetaInfoCommand1.Open;
```

1.17.1.1.20.1.18 TADMetaInfoCommand.TableKinds Property

Gets / sets table kind filter.

Description

The TableKinds property value filters database tables, by their kinds:

Kind	Meaning
tkSynonym	Synonyms.
tkTable	Regular tables.
tkView	Views.
tkTempTable	Temporary tables.

tkLocalTable	Local tables.

Syntax

```
property TableKinds: TADPhysTableKinds;
```

See Also

ObjectScopes (see page 458)

1.17.1.1.20.1.19 TADMetaInfoCommand.Wildcard Property

Gets / sets objects filter by their name.

Description

The Wildcard property value is the LIKE mask, which will be applied to the object names.

Syntax

```
property Wildcard: String;
```

See Also

ObjectScopes (see page 458), TableKinds (see page 460)

1.17.1.1.21 TADMetaInfoQuery Class

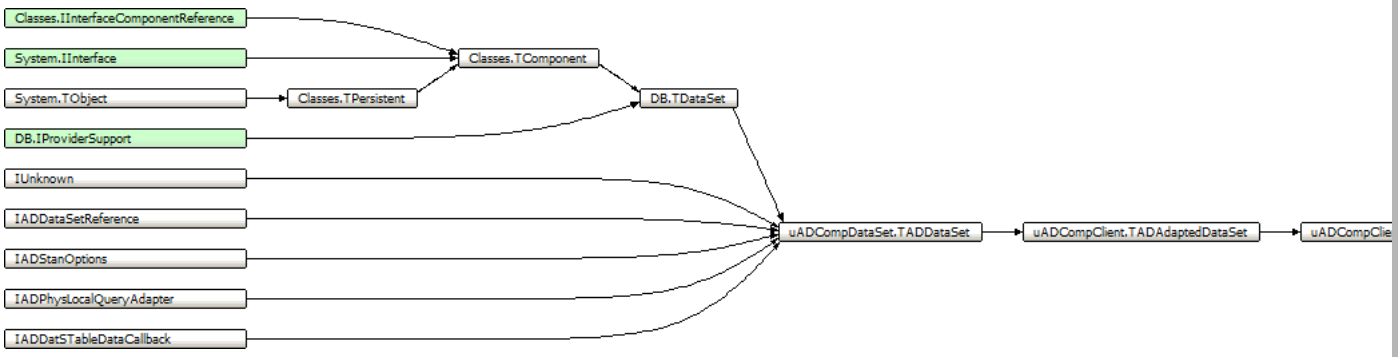
The class implementing dataset, capable to execute meta-info queries.

published

published	Description
ActiveStoredUsage (see page 462)	Controls how to use Active property value saved to DFM.
AfterGetRecord (see page 463)	Fires after fetching next rowset one time for each fetched record.
AfterGetRecords (see page 463)	Fires after fetching next rowset.
AfterRowRequest (see page 463)	Fires after fetching record.
Aggregates (see page 463)	The collection of client side aggregates, defined for dataset.
AggregatesActive (see page 464)	Controls automatic calculation of aggregate values.
BaseObjectName (see page 465)	Gets / sets the name of base object.
BeforeGetRecords (see page 466)	Fires before fetching next rowset.
BeforeRowRequest (see page 466)	Fires before fetching record.
CatalogName (see page 466)	Specified the DBMS catalog name
Connection (see page 466)	Specifies the FireDAC connection component to use.
ConnectionName (see page 467)	Specifies the FireDAC connection to use by its name.
FetchOptions (see page 467)	The set of options to control rows fetching.
FormatOptions (see page 467)	The set of options to control data representation.
Indexes (see page 468)	Lists all client indexes that apply to the dataset.
IndexesActive (see page 469)	Controls automatic maintenance of Index'es.
IndexFieldNames (see page 469)	Lists the field names to use as an index.
IndexName (see page 470)	Gets / sets the current index for the dataset by its name.
LocalSQL (see page 470)	The reference to a Local SQL engine.
MetaInfoKind (see page 471)	Gets / sets the kind of metadata to retrieve.
ObjectName (see page 471)	Gets / sets the name of object to get detailed information about it.
ObjectScopes (see page 472)	Gets / sets object scopes filter.
OnCommandChanged (see page 472)	Fires after the SQL property value is changed.
OnError (see page 472)	The event fires when an error happens, while dataset is communicating with DBMS.

ResourceOptions (see page 473)	The set of options to control resources usage.
SchemaAdapter (see page 474)	Gets or sets a reference to a schema adapter, as a Centralized Cached Updates log.
SchemaName (see page 474)	Specified the DBMS schema name.
TableKinds (see page 475)	Gets / sets tables filter by their kinds.
Transaction (see page 475)	Gets / sets reference to transaction object.
UpdateTransaction (see page 475)	Gets / sets reference to transaction object.
Wildcard (see page 476)	Gets / sets objects filter by their name.

Class Hierarchy



File

uADCompClient

Description

Use TADMetaInfoQuery to execute meta-info queries, browse the meta data resultsets records. To execute a query, specify the following optional property values:

- CatalogName (see page 466);
- SchemaName (see page 474);
- ObjectScopes (see page 472);
- MetaInfoKind (see page 471);
- BaseObjectName (see page 465);
- ObjectName (see page 471);
- TableKinds (see page 475);
- Wildcard (see page 476).

Syntax

```
TADMetaInfoQuery = class(TADRdbmsDataSet);
```

See Also

Working with Metadata (see page 127), TADMetaInfoCommand (see page 452)

Example

See FireDAC\Samples\Comp Layer\TADMetaInfoQuery\Main sample for details.

1.17.1.1.21.1 published

1.17.1.1.21.1.1 TADMetaInfoQuery.ActiveStoredUsage Property

Controls how to use Active property value saved to DFM.

Description

The ActiveStoredUsage property controls how to use Active property value saved to DFM. Include:

- auDesignTime, to use property value at design time;
- auRunTime, to use property value at run time.

Syntax

```
property ActiveStoredUsage: TADStoredActivationUsage;
```

See Also

TDataSet.Active

1.17.1.1.21.1.2 TADMetaInfoQuery.AfterGetRecord Event

Fires after fetching next rowset one time for each fetched record.

Description

The AfterGetRecord event fires after dataset fetched next rowset as result of navigation methods calls. The event fires one time for each fetched record. For example, fetching occurs automatically inside of Next, Last, Locate methods. Inside of an event handler application may read dataset fields to get fetched record field values.

Syntax

```
property AfterGetRecord: TADDataSetEvent;
```

See Also

AfterGetRecords

1.17.1.1.21.1.3 TADMetaInfoQuery.AfterGetRecords Event

Fires after fetching next rowset.

Description

The AfterGetRecords event fires after dataset fetched next rowset as result of navigation methods calls. For example, fetching occurs automatically inside of Next, Last, Locate methods.

Syntax

```
property AfterGetRecords: TADDataSetEvent;
```

See Also

TADDataSetEvent, BeforeGetRecords, AfterGetRecord

1.17.1.1.21.1.4 TADMetaInfoQuery.AfterRowRequest Event

Fires after fetching record.

Description

The AfterRowRequest event fires after dataset fetched current record as result of RefreshRecord (see page 642), FetchBlobs (see page 624) or FetchDetails (see page 625) method calls.

Syntax

```
property AfterRowRequest: TADDataSetEvent;
```

See Also

TADDataSetEvent, RefreshRecord, FetchBlobs, FetchDetails, BeforeRowRequest

1.17.1.1.21.1.5 TADMetaInfoQuery.Aggregates Property

The collection of client side aggregates, defined for dataset.

Description

Use Aggregates to define client side aggregating formulas, that will be automatically maintained and calculated for group of or for all records of dataset.

All records in the group of records has the same field values for defined set of fields. Aggregates performing calculation for group of records must be associated with one of the indexes. These aggregate expressions will be calculated only is associated index is current and active.

Adding aggregate field does not add TADAggregate (see page 572) object to Aggregates collection. That are two alternative ways - to use aggregate fields or to use Aggregates.

DataSet will automatically maintain and calculate Aggregates values when dataset is fetching the data or application edits the data, if AggregatesActive (see page 590) is True. If application needs to perform large updates to dataset and aggregate values are not needed while updating, then set AggregatesActive (see page 590) to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

When aggregates are maintained, the Value method of every active aggregate object returns a value that reflects the current data in the dataset. When users edit the data in the dataset, these values are recalculated to reflect the user's changes.

The expressions in Aggregates must contain aggregating functions, like a SUM, COUNT. FireDAC supports expression syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write constraint expressions.

Syntax

```
property Aggregates: TADAggregates;
```

See Also

Calculated and Aggregated Fields (see page 108), AggregatesActive, BeginBatch, EndBatch

Example 1

```
with ADMemTable1.Aggregates.Add do begin
  Expression := 'sum(sal + bonus)';
  Active := True;
end;
with ADMemTable1.Aggregates.Add do begin
  Expression := 'sum(sal + bonus)';
  IndexName := 'by_deps';
  Active := True;
end;
ADMemTable1.IndexName := 'by_deps';
ADMemTable1.AggregatesActive := True;

Label1.Caption := 'Total payments : ' + VarToStr(ADMemTable1.Aggregates[0].Value);
Label2.Caption := 'Current department payments : ' +
VarToStr(ADMemTable1.Aggregates[1].Value);
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\Aggregates sample for details.

1.17.1.1.21.1.6 TADMetaInfoQuery.AggregatesActive Property

Controls automatic calculation of aggregate values.

Description

Use AggregatesActive to get / set the flag controlling automatic maintenance and calculation of dataset aggregate values.

When AggregatesActive is False (the default), the dataset does not maintain aggregates. This allows the dataset to perform large updates to the dataset or fetch big volumes of data without overhead of calculating aggregate values, which happens

at fetching or data editing. When AggregatesActive is True, the dataset calculates and maintains all aggregate values specified by the Aggregates (see page 590) property that are compatible with the current index.

To selectively enable and disable aggregates rather than turning them all on or off at once, use the Active (see page 575) property of individual TADAggregate (see page 572) objects. These objects are available through the Aggregates (see page 590) property.

If application needs to perform batch updates to dataset, then set AggregatesActive to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property AggregatesActive: Boolean;
```

See Also

Calculated and Aggregated Fields (see page 108),Aggregates, TADAggregate.Active (see page 575), BeginBatch, EndBatch

Example

```
var
  lPrevAggsActive: Boolean;
...
lPrevAggsActive := ADQuery1.AggregatesActive;
ADQuery1.AggregatesActive := False;
try
  // perform updates here, without calculating and reading client aggregates
finally
  ADQuery1.AggregatesActive := lPrevAggsActive;
end;
```

1.17.1.1.21.1.7 TADMetaInfoQuery.BaseObjectName Property

Gets / sets the name of base object.

Description

The BaseObjectName property value meaning and optionality depends on MetaInfoKind:

MetaInfoKind	Meaning
mkIndexFields	Table name.
mkPrimaryKeyFields	Table name.
mkForeignKeyFields	Table name.
mkProcs	Package name.
mkProcArgs	Package name.

The BaseObjectName property value together with CatalogName (see page 466) and SchemaName (see page 474) constitutes the full base object name. If current DBMS session may "see" few objects with the same name, but in different catalogs and/or schemas, then we strongly suggest to specify CatalogName (see page 466) and/or SchemaName (see page 474) property values to explicitly limit the list of objects.

If BaseObjectName is case sensitive or contains special character, then it must be explicitly quoted.

Assignment to this property value closes dataset.

Syntax

```
property BaseObjectName: String;
```

See Also

MetaInfoKind (see page 471), CatalogName (see page 466), SchemaName (see page 474), ObjectName (see page 471)

Example 1

```
  // SQL Server: retrieve foreign key fields for FK_DEPARTMENTS constraint of the
```

```
Northwind.dbo.Employees table
  ADMetaInfoQuery1.BaseObjectName := 'Northwind.dbo.Employees';
  ADMetaInfoQuery1.ObjectName := 'FK_DEPARTMENTS';
  ADMetaInfoQuery1.MetaInfoKind := mkForeignKeyFields;
  ADMetaInfoQuery1.Open;
```

Example 2

```
// Oracle: retrieve procedure list for the SYS.DBMS_SQL package
ADMetaInfoQuery1.BaseObjectName := 'SYS.DBMS_SQL';
ADMetaInfoQuery1.MetaInfoKind := mkProcs;
ADMetaInfoQuery1.Open;
```

1.17.1.1.21.1.8 TADMetaInfoQuery.BeforeGetRecords Event

Fires before fetching next rowset.

Description

The BeforeGetRecords event fires before dataset will fetch next rowset as result of navigation methods calls. For example, fetching occurs automatically inside of Next, Last, Locate methods.

Syntax

```
property BeforeGetRecords: TADDataSetEvent;
```

See Also

TADDataSetEvent, AfterGetRecords

1.17.1.1.21.1.9 TADMetaInfoQuery.BeforeRowRequest Event

Fires before fetching record.

Description

The BeforeRowRequest event fires before dataset will fetch current record as result of RefreshRecord (see page 642), FetchBlobs (see page 624) or FetchDetails (see page 625) method calls.

Syntax

```
property BeforeRowRequest: TADDataSetEvent;
```

See Also

TADDataSetEvent, RefreshRecord, FetchBlobs, FetchDetails, AfterRowRequest

1.17.1.1.21.1.10 TADMetaInfoQuery.CatalogName Property

Specified the DBMS catalog name

Description

Use the CatalogName to restrict the metadata retrieval to the specified DBMS catalog.

When CatalogName or SchemaName are assigned, then ObjectScopes may be not used.

Syntax

```
property CatalogName: String;
```

See Also

SchemaName (see page 474), ObjectScopes (see page 472)

1.17.1.1.21.1.11 TADMetaInfoQuery.Connection Property

Specifies the FireDAC connection component to use.

Description

Use Connection to specify an FireDAC connection object to use to connect to an DBMS.

At design-time, select from the list of available TADCustomConnection (see page 318) objects (if any have been added to

the application) invoked from the Object Inspector. If a form or data module has TADCustomConnection (see page 318) dropped on it, then TADRdbmsDataSet assigns this connection object to Connection property automatically right after dropping on the form or data module.

At runtime, set Connection to reference an existing TADConnection (see page 279) object.

The TADRdbmsDataSet can be bind to TADCustomConnection (see page 318) object using Connection or ConnectionName (see page 503) property. Using ConnectionName (see page 503) property allows to bind to a connection object, if it is even not yet created or is not accessible.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

ConnectionName

1.17.1.1.21.1.12 TADMetaInfoQuery.ConnectionName Property

Specifies the FireDAC connection to use by its name.

Description

Use ConnectionName to specify an FireDAC connection to use to connect to an DBMS. The ConnectionName property value specifies the name of the connection. It must match to the:

- name of one of connection definitions, either stored in external file (persistent) or created on fly (private);
- ConnectionName of one of the TADConnection (see page 279) objects.

The ConnectionName property value must be specified before Prepare (see page 511) call. If it matches the name of one of connection definitions, then FireDAC will transparently create connection object and link the dataset with it.

The TADRdbmsDataSet cab be bind to TADCustomConnection (see page 318) object using Connection or ConnectionName property. Using Connection (see page 503) property allows to bind to a connection object explicitly and reduce the overhead for resolving connection names.

Syntax

```
property ConnectionName: String;
```

See Also

Connection, TADCustomConnection.Temporary (see page 337)

Example

```
ADQuery1.ConnectionName := 'Ora_Demo';
ADQuery1.Open('select * from "Customers"');
```

1.17.1.1.21.1.13 TADMetaInfoQuery.FetchOptions Property

The set of options to control rows fetching.

Description

The FetchOptions property is the set of properties, controlling the fetching of cursor rows. These properties will inherit its values from FetchOptions of connection object.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), TADFetchOptions (see page 849)

1.17.1.1.21.1.14 TADMetaInfoQuery.FormatOptions Property

The set of options to control data representation.

Description

The FormatOptions property is the set of properties, controlling the data type mapping, and some other aspects of data handling.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), TADFormatOptions (see page 860), Params

1.17.1.1.21.1.15 TADMetaInfoQuery.Indexes Property

Lists all client indexes that apply to the dataset.

Description

Use Indexes to define client side data views on the dataset records. Indexes are a collection of the TADIndex (see page 653) objects, each of them defines optional:

- Sorting by fields (Fields (see page 657), CaseInsFields (see page 655), DescFields (see page 656), Options (see page 658)).
- Sorting by expression (Expression (see page 656), Options (see page 658)).
- Records filter (Filter (see page 657), FilterOptions (see page 657)).
- Records distinction flag (Distinct (see page 656)).

The individual indexes are not just the sort and filter definitions. FireDAC maintains a consistent view on the data for each active index, while application fetches data or edits data. That creates additional overhead, but allows to switch between indexes without delays.

To make a view current, set it Selected (see page 659) property to True or dataset IndexName (see page 601) property to the view name. Note, the view must have Active = True to be maintained.

Dataset maintains Indexes views, when IndexesActive (see page 600) is True. When an application needs to perform batch updates to dataset, then set IndexesActive (see page 600) to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Setting IndexFieldNames (see page 600) does not add a TADIndex (see page 653) object to Indexes collection. Defining IndexDefs (see page 598) before dataset opening, will clear Indexes and fill them using IndexDefs (see page 598) information. So, there are two alternative ways - use IndexDefs (see page 598) or use Indexes. We suggest to use Indexes, as it is more flexible.

Some of the navigation methods, like a Locate (see page 634) / Lookup (see page 637) / SetRange (see page 647) are using indexes to optimize their operations.

The sorting and filter expressions in Indexes supports syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

Syntax

```
property Indexes: TADIndexes;
```

See Also

Sorting Records (see page 100), Writing Expressions (see page 110), IndexDefs, IndexesActive, IndexFieldNames, IndexName, BeginBatch, EndBatch

Example 1

```
with ADMemTable1.Indexes.Add do begin
  Name := 'by_name';
  Fields := 'NAME;DT';
```

```
  Active := True;
end;
with ADMemTable1.Indexes.Add do begin
  Name := 'by_payments';
  Expression := 'sum(sal + bonus)';
  Filter := 'dep_id is not null;
  Active := True;
end;
ADMemTable1.IndexesActive := True;
ADMemTable1.IndexName := 'by_name';
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\Indices demo for details.

1.17.1.1.21.1.16 TADMetaInfoQuery.IndexesActive Property

Controls automatic maintenance of Index'es.

Description

Use IndexesActive to indicate whether the dataset should maintain the data views, defined by the Indexes (see page 598) collection.

When IndexesActive is False, the dataset does not maintain the data views. That avoids the overhead of updating views, while data is fetched or is edited. When IndexesActive is True (by default), the dataset maintains the data views.

When indexes are maintained, each of them may be current index, selected by TADIndex (see page 653).Selected (see page 659) or by IndexName (see page 601) properties.

To selectively enable and disable data views rather than turning them all on or off at once, use the Active (see page 655) property of individual TADIndex (see page 653) objects. These objects are available through the Indexes (see page 598) property.

If application needs to perform batch updates to dataset, then set IndexesActive to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property IndexesActive: Boolean;
```

See Also

Sorting Records (see page 100), Indexes, BeginBatch, EndBatch

Example

```
var
  lPrevIndsActive: Boolean;
...
lPrevIndsActive := ADQuery1.IndexesActive;
ADQuery1.IndexesActive := False;
try
  // perform updates here, without maintaining client views
finally
  ADQuery1.IndexesActive := lPrevIndsActive;
end;
```

1.17.1.1.21.1.17 TADMetaInfoQuery.IndexFieldNames Property

Lists the field names to use as an index.

Description

Use IndexFieldNames as an alternative method of specifying the current index to use for a dataset.

Specify the name of each field on which to index the dataset, separating names with semicolons. Ordering of field names is significant. Optionally specify the postfix for any of fields in form "field[:[D][A][N]]", where:

- 'D' - use descending sorting on this field;

- 'A' - use ascending sorting on this field;

- 'N' - use case-insensitive sorting on this field.

Use IndexFieldNames to create sort orders on the fly. The IndexFieldNames and IndexName (see page 601) properties are mutually exclusive. Setting one clears the other.

Syntax

```
property IndexFieldNames: String;
```

See Also

Sorting Records (see page 100), Indexes, IndexName

Example

```
ADQuery1.IndexFieldNames := 'order_date;customer_name:N';
```

1.17.1.1.21.1.18 TADMetaInfoQuery.IndexName Property

Gets / sets the current index for the dataset by its name.

Description

Use IndexName to specify the current index for the dataset.

If IndexName is empty, the dataset's sort order is based on the IndexFieldNames (see page 600) property or on its default ordering as it is in dataset. If IndexName contains a name of valid index from Indexes collection, then that index is used to determine data view (including sort order) of records.

IndexFieldNames (see page 600) and IndexName are mutually exclusive. Setting one clears the other.

Syntax

```
property IndexName: String;
```

See Also

Sorting Records (see page 100), Indexes, IndexFieldNames

Example

```
with ADMemTable1.Indexes.Add do begin
  Name := 'by_name';
  Fields := 'NAME;DT';
  Active := True;
end;
ADMemTable1.IndexName := 'by_name';
```

1.17.1.1.21.1.19 TADMetaInfoQuery.LocalSQL Property

The reference to a Local SQL engine.

Description

Use this property to get / set reference to a Local SQL engine.

When engine and Local SQL connection are active, then right after setting LocalSQL property, the dataset will be opened (if not yet) and will be registered in Local SQL engine. Otherwise dataset will be registered when engine and connection will be active. The TADMemTable (see page 436) structure must be defined before it will be opened.

The dataset will be registered using its Name property value. So, its Name property value must be not empty and unique across all datasets linked to the same Local SQL engine.

Setting this property to nil unregisters this dataset from Local SQL engine.

Syntax

```
property LocalSQL: TADCustomLocalSQL;
```

See Also

Local SQL (see page 96), TADLocalSQL (see page 825), TADMemTable (see page 436)

1.17.1.1.21.1.20 TADMetaInfoQuery.MetaInfoKind Property

Gets / sets the kind of metadata to retrieve.

Description

The MetaInfoKind property determines what meta information will be retrieved. If DBMS or FireDAC driver does not support specified MetaInfoKind (for example, mkGenerators for Sybase SQL Anywhere), then no exception will be raised at Open and no records will be returned.

Assignment to this property value closes dataset.

Syntax

```
property MetaInfoKind: TADPhysMetaInfoKind;
```

See Also

Working with Metadata (see page 127), CatalogName (see page 466), SchemaName (see page 474), BaseObjectName (see page 465), Overload, ObjectName (see page 471)

Example

```
// get the table list for Northwind catalog, dbo schema
ADMetaInfoQuery1.CatalogName := 'Northwind';
ADMetaInfoQuery1.SchemaName := 'dbo';
ADMetaInfoQuery1.MetaInfoKind := mkTables;
ADMetaInfoQuery1.Open;
```

1.17.1.1.21.1.21 TADMetaInfoQuery.ObjectName Property

Gets / sets the name of object to get detailed information about it.

Description

The ObjectName property value defines the database objects, for which will be returned detailed information. The meaning, optionality and usage of the catalog and schema names depends on MetaInfoKind (see page 471):

MetaInfoKind	Meaning	Catalog and schema names
mkTableFields	Table name.	Applicable.
mkIndexes	Table name.	Applicable.
mkIndexFields	Index name.	--
mkPrimaryKey	Table name.	Applicable.
mkForeignKeys	Table name.	Applicable.
mkForeignKeyFields	Foreign key constraint name.	--
mkProcArgs	Stored procedure name.	Applicable, for non-packages stored procedures.

The ObjectName together with CatalogName (see page 466) and SchemaName (see page 474) may constitute the full object name. If current DBMS session may "see" few objects with the same name, but in different catalogs and/or schemas, then we strongly suggest to specify CatalogName (see page 466) and/or SchemaName (see page 474) property values to explicitly limit the list of objects.

If object name is case sensitive or contains special character, then it must be explicitly quoted.

Assignment to this property value closes dataset.

Syntax

```
property ObjectName: String;
```

See Also

Working with Metadata (see page 127), MetaInfoKind (see page 471), CatalogName (see page 466), SchemaName (see page 474), BaseObjectName (see page 465)

Example

```
// get the field list of Northwind.dbo.Employees table
ADMetaInfoQuery1.CatalogName := 'Northwind';
ADMetaInfoQuery1.SchemaName := 'dbo';
ADMetaInfoQuery1.ObjectName := 'Employees';
ADMetaInfoQuery1.MetaInfoKind := mkTableFields;
ADMetaInfoQuery1.Open;
```

1.17.1.1.21.1.22 TADMetaInfoQuery.ObjectScopes Property

Gets / sets object scopes filter.

Description

Use the ObjectScopes property value to filter database objects by their scope:

Scope	Meaning
osMy	Objects created by the current login user.
osSystem	Objects belonging to the DBMS.
osOther	All other objects.

Assignment to this property value closes dataset. When CatalogName or SchemaName properties are assigned then ObjectScopes may be not used.

Syntax

```
property ObjectScopes: TADPhysObjectScopes;
```

See Also

TableKinds (see page 475), CatalogName (see page 466), SchemaName (see page 474)

1.17.1.1.21.1.23 TADMetaInfoQuery.OnCommandChanged Event

Fires after the SQL property value is changed.

Description

The OnCommandChanged event fires after the SQL command text value is changed:

- for TADQuery (see page 476) - after SQL (see page 396) is changed;
- for TADStoredProc (see page 514) - after StoredProcName (see page 406) is changed;
- for TADTable (see page 539) - after TableName (see page 559) is changed.

Syntax

```
property OnCommandChanged: TNotifyEvent;
```

See Also

TADCustomQuery.SQL (see page 396), TADCustomStoredProc.StoredProcName (see page 406), TADTable.TableName (see page 559)

1.17.1.1.21.1.24 TADMetaInfoQuery.OnError Event

The event fires when an error happens, while dataset is communicating with DBMS.

Parameters

Parameters	Description
ASender	The reference to dataset.
AInitiator	The reference to interface of object, which initially raised the exception.
AException	The exception object, which may be modified or even replaced by another exception object instance.

Description

The OnError event fires, when dataset executes one of the following operations and an error happens:

- Prepare (see page 511). The dataset is preparing DBMS command text for execution.
- Open (see page 509). The dataset is executing DBMS command to open cursor.
- Execute (see page 622), ExecSQL (see page 397), ExecProc (see page 410). The dataset is executing DBMS command text.
- Navigation methods. The dataset is fetching rows from cursor.

If most cases AException object will of EADDBEngineException (see page 833) class. If AException is of EADDBArrayExecuteError (see page 831) class, then it is Array DML (see page 82) error handling case. You should use a code like in the first example.

To centralize error handling, you can consider to use TADCustomConnection (see page 318).OnError (see page 332).

Syntax

```
property OnError: TADErrorEvent;
```

See Also

TADCustomQuery.ExecSQL (see page 397), TADCustomStoredProc.ExecProc (see page 410), TDataSet.Open, Prepare, EADDBArrayExecuteError (see page 831), EADDBEngineException (see page 833), TADCustomConnection.OnError (see page 332)

Example 1

Handling Array DML errors:

```
procedure TForm1.ADQuery1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if AException is EADPhysArrayExecuteError then
    with EADPhysArrayExecuteError(AException) do
      if Errors[0].Kind = ekUKViolated then
        AAction := eaSkip
      else
        AAction := eaFail
  else
    ...
end;
```

Example 2

Error substitution:

```
procedure TForm1.ADQuery1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if EADDBEngineException(AException).Errors[0].Kind = ekPKViolated then begin
    AException.Free;
    AException := Exception.Create('Please, enter unique ID value');
  end;
end;
```

1.17.1.1.21.1.25 TADMetaInfoQuery.ResourceOptions Property

The set of options to control resources usage.

Description

The ResourceOptions property is the set of properties, controlling the resources usage by the dataset. These properties will inherit its values from ResourceOptions of connection object.

Syntax

```
property ResourceOptions: TADBottomResourceOptions;
```

See Also

Setting Options (see page 34), TADBottomResourceOptions (see page 841)

1.17.1.1.21.1.26 TADMetaInfoQuery.SchemaAdapter Property

Gets or sets a reference to a schema adapter, as a Centralized Cached Updates log.

Description

Use SchemaAdapter property to set a reference to the TADSchemaAdapter (see page 514) object to enable Centralized Cashed Updates (see page 114) mode. The schema adapter object will be used as a central change log share by few datasets.

To enable propagation of master dataset changes to a detail dataset set FetchOptions.DetailCascade (see page 854) to True for a detail dataset. Also both master and detail datasets must point to the same TADSchemaAdapter (see page 514) object.

Syntax

```
property SchemaAdapter: TADCustomSchemaAdapter;
```

See Also

Caching Updates (see page 114), Master-Detail Relationship (see page 104), TADFetchOptions.DetailCascade (see page 854)

Example

```
dsOrders.DataSet := qOrders;
qOrders.SchemaAdapter := ADSchemaAdapter1;
qOrders.CachedUpdates := True;

dsOrderDetails.DataSet := qOrderDetails;
qOrderDetails.SchemaAdapter := ADSchemaAdapter1;
qOrderDetails.CachedUpdates := True;
qOrderDetails.MasterSource := dsOrders;
qOrderDetails.MasterFields := 'OrderID';
qOrderDetails.IndexFieldNames := 'OrderID';
qOrderDetails.FetchOptions.DetailCascade := True;

qOrders.Open;
qOrderDetails.Open;

qOrders.Append;
....
qOrders.Post;

qOrderDetails.Append;
...
qOrderDetails.Post;

ADSchemaAdapter1.ApplyUpdates;
qOrders.CommitUpdates;
qOrderDetails.CommitUpdates;
```

1.17.1.1.21.1.27 TADMetaInfoQuery.SchemaName Property

Specified the DBMS schema name.

Description

Use the SchemaName to restrict the metadata retrieval to the specified DBMS schema.

When CatalogName or SchemaName are assigned, then ObjectScopes may be not used.

Syntax

```
property SchemaName: String;
```

See Also

CatalogName (see page 466), ObjectScopes (see page 472)

1.17.1.1.21.1.28 TADMetaInfoQuery.TableKinds Property

Gets / sets tables filter by their kinds.

Description

The TableKinds property value filters database tables, by their kinds. Assignment to this property will close dataset.

Kind	Meaning
tkSynonym	Synonyms.
tkTable	Regular tables.
tkView	Views.
tkTempTable	Temporary tables.
tkLocalTable	Local tables.

Syntax

```
property TableKinds: TADPhysTableKinds;
```

See Also

Working with Metadata (see page 127), ObjectScopes (see page 472), CatalogName (see page 466), SchemaName (see page 474)

1.17.1.1.21.1.29 TADMetaInfoQuery.Transaction Property

Gets / sets reference to transaction object.

Description

The Transaction property allows to associate dataset with explicit transaction object. If it is not specified, then dataset will be associated with connection default transaction.

The specified transaction object will be used only for Prepare (see page 511), Open (see page 509), ExecSQL (see page 397), ExecProc (see page 410), Execute (see page 622) methods. For commands, generated to apply changes to database, FireDAC will use UpdateTransaction (see page 507), if it is specified, otherwise - Transaction.

Currently explicit transaction object assignment is supported only for Interbase / Firebird servers.

Syntax

```
property Transaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41), UpdateTransaction, TADCustomConnection.Transaction (see page 337)

1.17.1.1.21.1.30 TADMetaInfoQuery.UpdateTransaction Property

Gets / sets reference to transaction object.

Description

The UpdateTransaction property allows to associate dataset internal update commands with explicit transaction object. If it is not specified, then dataset update commands will be associated with connection default update transaction. If it is not specified, then - with dataset Transaction.

For Prepare (see page 511), Open (see page 509), ExecSQL (see page 397), ExecProc (see page 410), Execute (see page 622) methods will be used Transaction (see page 507) objects.

Currently explicit update transaction object assignment is supported only for Interbase / Firebird servers.

Syntax

```
property UpdateTransaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41),Transaction, TADCustomConnection.UpdateTransaction (see page 338)

Example

```
var
  oSelectTransaction: TADTransaction;
  oUpdateTransaction: TADTransaction;
  oCusts: TADQuery;
.....
  oSelectTransaction := TADTransaction.Create(nil);
  oSelectTransaction.Connection := ADConnection1;
  oSelectTransaction.Options.ReadOnly := True;

  oUpdateTransaction := TADTransaction.Create(nil);
  oUpdateTransaction.Connection := ADConnection1;

  oCusts := TADQuery.Create(nil);
  oCusts.Connection := ADConnection1;
  oCusts.Transaction := oSelectTransaction;
  oCusts.UpdateTransaction := oUpdateTransaction;
  oCusts.Open('select * from Customers'); // uses oSelectTransaction
  ...
  oCusts.Edit; // uses oUpdateTransaction, if needed
  ...
  oCusts.Post; // uses oUpdateTransaction
```

1.17.1.1.21.1.31 TADMetaInfoQuery.Wildcard Property

Gets / sets objects filter by their name.

Description

The Wildcard property value is the LIKE mask, which will be applied to the object names.

Syntax

```
property Wildcard: String;
```

See Also

ObjectScopes (see page 472), TableKinds (see page 475)

1.17.1.1.22 TADQuery Class

The class implementing dataset, capable to execute SQL (see page 499) queries.

public

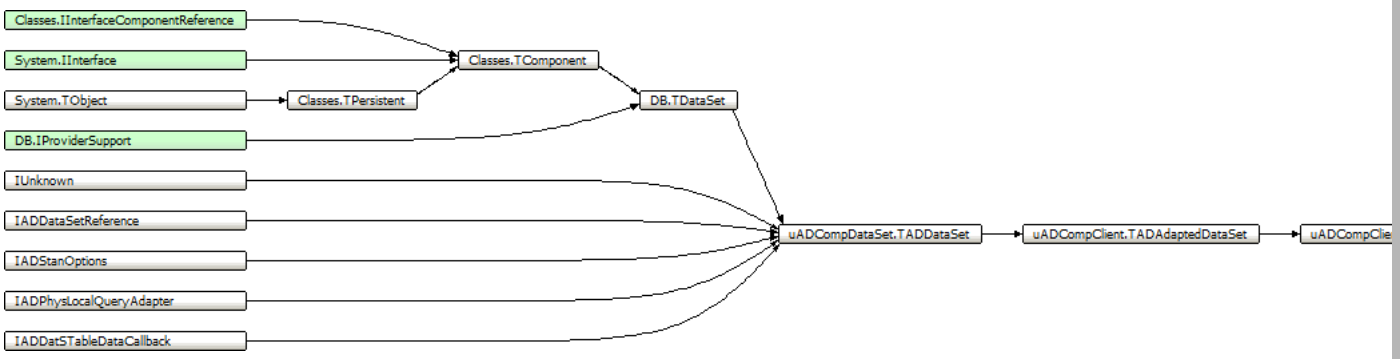
public	Description
MacroCount (see page 478)	Returns the number of macro in Macros collection.
RowsAffected (see page 478)	Returns the number of rows operated upon by the latest query execution.

published

published	Description
ActiveStoredUsage (see page 479)	Controls how to use Active property value saved to DFM.
AfterApplyUpdates (see page 479)	Fires after cached updates are applied to DB.
AfterExecute (see page 479)	Fires after SQL command execution is finished.
AfterGetRecord (see page 480)	Fires after fetching next rowset one time for each fetched record.
AfterGetRecords (see page 480)	Fires after fetching next rowset.
AfterRowRequest (see page 480)	Fires after fetching record.
Aggregates (see page 480)	The collection of client side aggregates, defined for dataset.
AggregatesActive (see page 481)	Controls automatic calculation of aggregate values.
BeforeApplyUpdates (see page 482)	Fires before cached updates are applied to DB.

BeforeExecute (see page 482)	Fires before SQL command execution is started.
BeforeGetRecords (see page 482)	Fires before fetching next rowset.
BeforeRowRequest (see page 482)	Fires before fetching record.
CachedUpdates (see page 483)	Specifies whether dataset will log changes to the data without immediate applying of them to the database.
Connection (see page 483)	Specifies the FireDAC connection component to use.
ConnectionName (see page 484)	Specifies the FireDAC connection to use by its name.
Constraints (see page 484)	Specifies record-level client constraints that must be met when editing the data.
ConstraintsEnabled (see page 485)	Specifies whether the dataset perform constraint checking.
DetailFields (see page 486)	Gets / sets fields in detail dataset, used to establish a master-detail relationship with this dataset.
FetchOptions (see page 486)	The set of options to control rows fetching.
FilterChanges (see page 486)	Specifies what kinds of changed records must be "visible" in the dataset.
FormatOptions (see page 487)	The set of options to control data representation.
Indexes (see page 487)	Lists all client indexes that apply to the dataset.
IndexesActive (see page 488)	Controls automatic maintenance of Index'es.
IndexFieldNames (see page 489)	Lists the field names to use as an index.
IndexName (see page 489)	Gets / sets the current index for the dataset by its name.
LocalSQL (see page 490)	The reference to a Local SQL engine.
Macros (see page 490)	The collection of macro substitution variables.
MasterFields (see page 491)	Gets / sets fields in master dataset, used to establish a master-detail relationship with this dataset.
MasterSource (see page 491)	Gets / sets a master data source, used to establish a master-detail relationship with this dataset.
OnCommandChanged (see page 491)	Fires after the SQL property value is changed.
OnError (see page 492)	The event fires when an error happens, while dataset is communicating with DBMS.
OnExecuteError (see page 493)	The event is triggered when an error happens, while executing a Array DML query.
OnMasterSetValues (see page 493)	Fires after a detailed dataset has set detailed query parameters.
OnReconcileError (see page 494)	Fires when a dataset needs to reconcile an update to a record that cannot not be applied.
OnUpdateError (see page 495)	Fires if an exception is generated when updates are applied to a database.
OnUpdateRecord (see page 495)	Occurs when record update is applying to a database.
Params (see page 497)	Contains the parameters for a SQL statement.
ResourceOptions (see page 498)	The set of options to control resources usage.
SchemaAdapter (see page 498)	Gets or sets a reference to a schema adapter, as a Centralized Cached Updates log.
SQL (see page 499)	Contains the text of the SQL statement to execute for the query.
Transaction (see page 500)	Gets / sets reference to transaction object.
UpdateObject (see page 500)	Specifies the update object component used to post updates by dataset.
UpdateOptions (see page 500)	The set of options to control updates posting.
UpdateTransaction (see page 501)	Gets / sets reference to transaction object.

Class Hierarchy



File

uADCompClient

Description

Use TADQuery to execute SQL queries, browse the result sets and edit the result set records.

See the TADCustomQuery description for more details.

Syntax

```
TADQuery = class(TADCustomQuery);
```

See Also

Executing Command (see page 66), TADCommand (see page 267), TADCustomQuery (see page 394), IADPhysCommand

1.17.1.1.22.1 public

1.17.1.1.22.1.1 TADQuery.MacroCount Property

Returns the number of macro in Macros collection.

Description

The MacroCount property teturns the number of of macro substitution variables in Macros collection.

Syntax

```
property MacroCount: Word;
```

See Also

Macros

1.17.1.1.22.1.2 TADQuery.RowsAffected Property

Returns the number of rows operated upon by the latest query execution.

Description

Inspect RowsAffected to determine how many rows were inserted, updated, deleted or fetched by the last dataset operation. If no rows were processed, RowsAffected = 0. If numbers of processed rows is not accessible, RowsAffected = –1.

At MS SQL Server RowsAffected may be unexpectedly equal to -1, if stored procedure or table trigger omits SET NOCOUNT ON. Also, this may be a reason of "Command inserted / updated / deleted [0] records instead of [1] record" error at posting updates to database. Check MS BOL for more details about SET NOCOUNT ON.

Syntax

```
property RowsAffected: TADCounter;
```

Example

```
procedure TForm1.ADQuery1AfterExecute(ASender: TObject);
begin
  if ADQuery1.RowsAffected = -1 then
    StatusBar1.SimpleText := 'Ok'
  else
    case ADQuery1.Command1.CommandKind of
    skDelete: StatusBar1.SimpleText := Format('%d rows deleted', [ADQuery1.RowsAffected]);
    skInsert: StatusBar1.SimpleText := Format('%d rows inserted', [ADQuery1.RowsAffected]);
    skUpdate: StatusBar1.SimpleText := Format('%d rows updated', [ADQuery1.RowsAffected]);
    else      StatusBar1.SimpleText := Format('%d rows affected', [ADQuery1.RowsAffected]);
    end;
end;
```

1.17.1.1.22.2 published

1.17.1.1.22.2.1 TADQuery.ActiveStoredUsage Property

Controls how to use Active property value saved to DFM.

Description

The ActiveStoredUsage property controls how to use Active property value saved to DFM. Include:

- auDesignTime, to use property value at design time;
- auRunTime, to use property value at run time.

Syntax

```
property ActiveStoredUsage: TADStoredActivationUsage;
```

See Also

TDataSet.Active

1.17.1.1.22.2.2 TADQuery.AfterApplyUpdates Event

Fires after cached updates are applied to DB.

Description

The AfterApplyUpdates event fires after cached updates are applied to DB as result of ApplyUpdates (see page 611) method call.

Syntax

```
property AfterApplyUpdates: TADAfterApplyUpdatesEvent;
```

See Also

TADAfterApplyUpdatesEvent, ApplyUpdates, BeforeApplyUpdates

1.17.1.1.22.2.3 TADQuery.AfterExecute Event

Fires after SQL command execution is finished.

Description

The AfterExecute event fires after SQL command execution is finished as result of Execute (see page 622) method call.

Syntax

```
property AfterExecute: TADDataSetEvent;
```

See Also

TADDataSetEvent, Execute, BeforeExecute

1.17.1.1.22.2.4 TADQuery.AfterGetRecord Event

Fires after fetching next rowset one time for each fetched record.

Description

The AfterGetRecord event fires after dataset fetched next rowset as result of navigation methods calls. The event fires one time for each fetched record. For example, fetching occurs automatically inside of Next, Last, Locate methods. Inside of an event handler application may read dataset fields to get fetched record field values.

Syntax

```
property AfterGetRecord: TADDataSetEvent;
```

See Also

AfterGetRecords

1.17.1.1.22.2.5 TADQuery.AfterGetRecords Event

Fires after fetching next rowset.

Description

The AfterGetRecords event fires after dataset fetched next rowset as result of navigation methods calls. For example, fetching occurs automatically inside of Next, Last, Locate methods.

Syntax

```
property AfterGetRecords: TADDataSetEvent;
```

See Also

TADDataSetEvent, BeforeGetRecords, AfterGetRecord

1.17.1.1.22.2.6 TADQuery.AfterRowRequest Event

Fires after fetching record.

Description

The AfterRowRequest event fires after dataset fetched current record as result of RefreshRecord (see page 642), FetchBlobs (see page 624) or FetchDetails (see page 625) method calls.

Syntax

```
property AfterRowRequest: TADDataSetEvent;
```

See Also

TADDataSetEvent, RefreshRecord, FetchBlobs, FetchDetails, BeforeRowRequest

1.17.1.1.22.2.7 TADQuery.Aggregates Property

The collection of client side aggregates, defined for dataset.

Description

Use Aggregates to define client side aggregating formulas, that will be automatically maintained and calculated for group of or for all records of dataset.

All records in the group of records has the same field values for defined set of fields. Aggregates performing calculation for group of records must be associated with one of the indexes. These aggregate expressions will be calculated only is associated index is current and active.

Adding aggregate field does not add TADAggregate (see page 572) object to Aggregates collection. That are two alternative ways - to use aggregate fields or to use Aggregates.

DataSet will automatically maintain and calculate Aggregates values when dataset is fetching the data or application edits the data, if AggregatesActive (see page 590) is True. If application needs to perform large updates to dataset and

aggregate values are not needed while updating, then set AggregatesActive (see page 590) to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

When aggregates are maintained, the Value method of every active aggregate object returns a value that reflects the current data in the dataset. When users edit the data in the dataset, these values are recalculated to reflect the user's changes.

The expressions in Aggregates must contain aggregating functions, like a SUM, COUNT. FireDAC supports expression syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write constraint expressions.

Syntax

```
property Aggregates: TADAggregates;
```

See Also

Calculated and Aggregated Fields (see page 108), AggregatesActive, BeginBatch, EndBatch

Example 1

```
with ADMemTable1.Aggregates.Add do begin
  Expression := 'sum(sal + bonus)';
  Active := True;
end;
with ADMemTable1.Aggregates.Add do begin
  Expression := 'sum(sal + bonus)';
  IndexName := 'by_deps';
  Active := True;
end;
ADMemTable1.IndexName := 'by_deps';
ADMemTable1.AggregatesActive := True;

Label1.Caption := 'Total payments : ' + VarToStr(ADMemTable1.Aggregates[0].Value);
Label2.Caption := 'Current department payments : ' +
VarToStr(ADMemTable1.Aggregates[1].Value);
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\Aggregates sample for details.

1.17.1.1.22.2.8 TADQuery.AggregatesActive Property

Controls automatic calculation of aggregate values.

Description

Use AggregatesActive to get / set the flag controlling automatic maintenance and calculation of dataset aggregate values.

When AggregatesActive is False (the default), the dataset does not maintain aggregates. This allows the dataset to perform large updates to the dataset or fetch big volumes of data without overhead of calculating aggregate values, which happens at fetching or data editing. When AggregatesActive is True, the dataset calculates and maintains all aggregate values specified by the Aggregates (see page 590) property that are compatible with the current index.

To selectively enable and disable aggregates rather than turning them all on or off at once, use the Active (see page 575) property of individual TADAggregate (see page 572) objects. These objects are available through the Aggregates (see page 590) property.

If application needs to perform batch updates to dataset, then set AggregatesActive to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property AggregatesActive: Boolean;
```

See Also

Calculated and Aggregated Fields (see page 108),Aggregates, TADAggregate.Active (see page 575), BeginBatch, EndBatch

Example

```
var
  lPrevAggsActive: Boolean;
...
lPrevAggsActive := ADQuery1.AggregatesActive;
ADQuery1.AggregatesActive := False;
try
  // perform updates here, without calculating and reading client aggregates
finally
  ADQuery1.AggregatesActive := lPrevAggsActive;
end;
```

1.17.1.1.22.2.9 TADQuery.BeforeApplyUpdates Event

Fires before cached updates are applied to DB.

Description

The BeforeApplyUpdates event fires before cached updates are applied to database as result of ApplyUpdates (see page 611) method call.

Syntax

```
property BeforeApplyUpdates: TADDataSetEvent;
```

See Also

TADDataSetEvent, ApplyUpdates, AfterApplyUpdates

1.17.1.1.22.2.10 TADQuery.BeforeExecute Event

Fires before SQL command execution is started.

Description

The BeforeExecute event fires before SQL command execution is started as result of Execute (see page 622) method call.

Syntax

```
property BeforeExecute: TADDataSetEvent;
```

See Also

TADDataSetEvent, Execute, AfterExecute

1.17.1.1.22.2.11 TADQuery.BeforeGetRecords Event

Fires before fetching next rowset.

Description

The BeforeGetRecords event fires before dataset will fetch next rowset as result of navigation methods calls. For example, fetching occurs automatically inside of Next, Last, Locate methods.

Syntax

```
property BeforeGetRecords: TADDataSetEvent;
```

See Also

TADDataSetEvent, AfterGetRecords

1.17.1.1.22.2.12 TADQuery.BeforeRowRequest Event

Fires before fetching record.

Description

The BeforeRowRequest event fires before dataset will fetch current record as result of RefreshRecord (see page 642), FetchBlobs (see page 624) or FetchDetails (see page 625) method calls.

Syntax

```
property BeforeRowRequest: TADDataSetEvent;
```

See Also

TADDataSetEvent, RefreshRecord, FetchBlobs, FetchDetails, AfterRowRequest

1.17.1.1.22.2.13 TADQuery.CachedUpdates Property

Specifies whether dataset will log changes to the data without immediate applying of them to the database.

Description

CachedUpdates enables (True) or disables (False) the logging of data changes (Insert/Post, Edit/Post, Delete) without immediate applying of them to the database.

An application must explicitly apply changes from change log to the database, using ApplyUpdates (see page 611) method. All changes will be written in comparably small amount of time in single transaction. The main benefits of enabling cached updates are:

- fewer transactions and shorter transaction times;
- minimization of network traffic;
- simplified implementation of undo / redo functionality for dataset;
- ability to implement application offline mode or briefcase model.

The potential drawbacks of enabling cached updates are:

- other applications can access and change the actual data on the server while users are editing local copies of the data, resulting in an update conflict when cached updates are applied to the database;
- other applications cannot access data changes made by an application until its cached updates are applied to the database.

Note, to change CachedUpdate property value for TADTable, it must be inactive.

Syntax

```
property CachedUpdates: Boolean;
```

See Also

Caching Updates (see page 114), ApplyUpdates, CancelUpdates, CommitUpdates

Example

```
ADQuery1.CachedUpdates := True;
...
ADQuery1.Edit;
...
ADQuery1.Post;
...
ADQuery1.Append;
...
ADQuery1.Post;
...
ADQuery1.ApplyUpdates;
ADQuery1.CommitUpdates;
ADQuery1.CachedUpdates := False;
```

1.17.1.1.22.2.14 TADQuery.Connection Property

Specifies the FireDAC connection component to use.

Description

Use Connection to specify an FireDAC connection object to use to connect to an DBMS.

At design-time, select from the list of available TADCustomConnection (see page 318) objects (if any have been added to the application) invoked from the Object Inspector. If a form or data module has TADCustomConnection (see page 318) dropped on it, then TADRdbmsDataSet assigns this connection object to Connection property automatically right after dropping on the form or data module.

At runtime, set Connection to reference an existing TADConnection (see page 279) object.

The TADRdbmsDataSet can be bind to TADCustomConnection (see page 318) object using Connection or ConnectionName (see page 503) property. Using ConnectionName (see page 503) property allows to bind to a connection object, if it is even not yet created or is not accessible.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

ConnectionName

1.17.1.1.22.2.15 TADQuery.ConnectionName Property

Specifies the FireDAC connection to use by its name.

Description

Use ConnectionName to specify an FireDAC connection to use to connect to an DBMS. The ConnectionName property value specifies the name of the connection. It must match to the:

- name of one of connection definitions, either stored in external file (persistent) or created on fly (private);
- ConnectionName of one of the TADConnection (see page 279) objects.

The ConnectionName property value must be specified before Prepare (see page 511) call. If it matches the name of one of connection definitions, then FireDAC will transparently create connection object and link the dataset with it.

The TADRdbmsDataSet cab be bind to TADCustomConnection (see page 318) object using Connection or ConnectionName property. Using Connection (see page 503) property allows to bind to a connection object explicitly and reduce the overhead for resolving connection names.

Syntax

```
property ConnectionName: String;
```

See Also

Connection, TADCustomConnection.Temporary (see page 337)

Example

```
ADQuery1.ConnectionName := 'Ora_Demo';
ADQuery1.Open('select * from "Customers"');
```

1.17.1.1.22.2.16 TADQuery.Constraints Property

Specifies record-level client constraints that must be met when editing the data.

Description

Use Constraints to maintain record-level constraints for the dataset.

Record-level constraints force business ruless limiting few fields in a single record. This constraints are checked at end of data editing (Post / AppendRecord / InsertRecord). Constraints, limiting single field are field constraints and must specified at TField.CustomConstraint or TField.ImportedConstraint.

The Constraints checking is performed when ConstraintsEnabled (see page 594) is True. If application needs to perform large updates to the dataset and application can guarantee the data consistancy, then set ConstraintsEnabled (see page 594) to False before updates, and return to original value after updates. The constraints are not checked at data fetching.

The expressions in Constraints must be predicates, evaluating to Boolean value. FireDAC supports expression syntax compatible with:

- BDE data access components;
- Client dataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write constraint expressions.

Syntax

```
property Constraints;
```

See Also

ConstraintsEnabled, DisableConstraints, EnableConstraints, BeginBatch, EndBatch, TDataSet.Post, TDataSet.AppendRecord, TDataSet.InsertRecord, TField.CustomConstraint, TField.ImportedConstraint

Example

```
with ADMemTable1.Constraints.Add do begin
  CustomConstraint := 'sal + bonus <= 2000';
  ErrorMessage := 'The employee payments must be equal or less than 2000 usd';
end;

ADMemTable1.ConstraintsEnabled := True;
ADMemTable1.Edit;
try
  ADMemTable1.FieldByName('sal').AsFloat := 1800;
  ADMemTable1.FieldByName('bonus').AsFloat := 300;
  // here exception will be raised
  ADMemTable1.Post;
except
  ADMemTable1.Cancel;
  Application.HandleException(Self);
end;

ADMemTable1.ConstraintsEnabled := False;
ADMemTable1.Edit;
ADMemTable1.FieldByName('sal').AsFloat := 1800;
ADMemTable1.FieldByName('bonus').AsFloat := 300;
// here exception will be NOT raised, because constraint checking is disabled
ADMemTable1.Post;
```

1.17.1.1.22.2.17 TADQuery.ConstraintsEnabled Property

Specifies whether the dataset perform constraint checking.

Description

Use ConstraintsEnabled to get / set the flag controlling automatic record-level constraints enforcement.

When ConstraintsEnabled is False (the default), the dataset does not check Constraints (see page 594). This allows the dataset to perform the large data updates without overhead of checking constraints. When ConstraintsEnabled is True, the dataset checks Constraints (see page 594) at at end of data editing (Post / AppendRecord / InsertRecord).

If application needs to perform batch updates to dataset, then set ConstraintsEnabled to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property ConstraintsEnabled: Boolean;
```

See Also

Constraints, BeginBatch, EndBatch, DisableConstraints, EnableConstraints, TDataSet.Post, TDataSet.AppendRecord, TDataSet.InsertRecord, TADUpdateOptions.CheckRequired (see page 898)

Example

```
var
  lPrevConsEnabled: Boolean;
```

```
...
lPrevConsEnabled := ADQuery1.ConstraintEnabled;
ADQuery1.ConstraintEnabled := False;
try
  // perform updates here, without checking client constraints
finally
  ADQuery1.ConstraintEnabled := lPrevConsEnabled;
end;
```

1.17.1.1.22.2.18 TADQuery.DetailFields Property

Gets / sets fields in detail dataset, used to establish a master-detail relationship with this dataset.

Description

Use DetailFields property value to specify semicolon separated list of the detail dataset field names, used to establish a master-detail relationship with this dataset.

The DetailFields property is useful only for parameter based master-detail setup. For range based setup the indexed fields are used as the detail field list. When DetailFields is not specified explicitly, then field names matching to query parameter names will be used as DetailFields value.

The DetailFields will be used only to assign specified field values when a new record is inserted into detail dataset. The mechanism used to filter the detail dataset records does not use DetailFields.

Syntax

```
property DetailFields: String;
```

See Also

Master-Detail Relationship (see page 104), MasterSource, MasterFields, IndexFieldNames, IndexName

Example

```
ADQuery1.SQL.Text := 'SELECT * FROM "Orders" WHERE CustomerID = :ID';
ADQuery1.MasterSource := CustomersDS;
ADQuery1.MasterFields := 'ID';
ADQuery1.DetailFields := 'CustomerID';
```

1.17.1.1.22.2.19 TADQuery.FetchOptions Property

The set of options to control rows fetching.

Description

The FetchOptions property is the set of properties, controlling the fetching of cursor rows. These properties will inherit its values from FetchOptions of connection object.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), TADFetchOptions (see page 849)

1.17.1.1.22.2.20 TADQuery.FilterChanges Property

Specifies what kinds of changed records must be "visible" in the dataset.

Description

The FilterChanges property allows to get / set changed records kinds, which must be accessible through navigation interface of dataset ("visible" in dataset):

Kind	Meaning
rtModified	Records changed after fetching or last CommitUpdates (see page 615) / CancelUpdates (see page 614).
rtInserted	New records, not yet posted to DB and added after dataset open or after last CommitUpdates (see page 615).

rtDeleted	Deleted records, not yet deleted from DB and deleted after dataset open or after last CommitUpdates (see page 615).
rtUnmodified	Unchanged records.
rtHasErrors	Records having associated errors after ApplyUpdates (see page 611). To get an error object use RowError (see page 607) property.

Syntax

```
property FilterChanges: TADUpdateRecordTypes;
```

See Also

Filtering Records (see page 102), Caching Updates (see page 114), RowError (see page 607), UpdateStatus (see page 650)

Example

```
// see only inserted records
ADQuery1.FilterChanges := [rtInserted];
```

1.17.1.1.22.2.21 TADQuery.FormatOptions Property

The set of options to control data representation.

Description

The FormatOptions property is the set of properties, controlling the data type mapping, and some other aspects of data handling.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), TADFormatOptions (see page 860), Params

1.17.1.1.22.2.22 TADQuery.Indexes Property

Lists all client indexes that apply to the dataset.

Description

Use Indexes to define client side data views on the dataset records. Indexes are a collection of the TADIndex (see page 653) objects, each of them defines optional:

- Sorting by fields (Fields (see page 657), CaseInsFields (see page 655), DescFields (see page 656), Options (see page 658)).
- Sorting by expression (Expression (see page 656), Options (see page 658)).
- Records filter (Filter (see page 657), FilterOptions (see page 657)).
- Records distinction flag (Distinct (see page 656)).

The individual indexes are not just the sort and filter definitions. FireDAC maintains a consistent view on the data for each active index, while application fetches data or edits data. That creates additional overhead, but allows to switch between indexes without delays.

To make a view current, set it Selected (see page 659) property to True or dataset IndexName (see page 601) property to the view name. Note, the view must have Active = True to be maintained.

Dataset maintains Indexes views, when IndexesActive (see page 600) is True. When an application needs to perform batch updates to dataset, then set IndexesActive (see page 600) to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Setting IndexFieldNames (see page 600) does not add a TADIndex (see page 653) object to Indexes collection. Defining IndexDefs (see page 598) before dataset opening, will clear Indexes and fill them using IndexDefs (see page 598) information. So, there are two alternative ways - use IndexDefs (see page 598) or use Indexes. We suggest to use

Indexes, as it is more flexible.

Some of the navigation methods, like a Locate (see page 634) / Lookup (see page 637) / SetRange (see page 647) are using indexes to optimize their operations.

The sorting and filter expressions in Indexes supports syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

Syntax

```
property Indexes: TADIndexes;
```

See Also

Sorting Records (see page 100), Writing Expressions (see page 110), IndexDefs, IndexesActive, IndexFieldNames, IndexName, BeginBatch, EndBatch

Example 1

```
with ADMemTable1.Indexes.Add do begin
  Name := 'by_name';
  Fields := 'NAME;DT';
  Active := True;
end;
with ADMemTable1.Indexes.Add do begin
  Name := 'by_payments';
  Expression := 'sum(sal + bonus)';
  Filter := 'dep_id is not null;
  Active := True;
end;
ADMemTable1.IndexesActive := True;
ADMemTable1.IndexName := 'by_name';
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\Indices demo for details.

1.17.1.1.22.2.23 TADQuery.IndexesActive Property

Controls automatic maintenance of Index'es.

Description

Use IndexesActive to indicate whether the dataset should maintain the data views, defined by the Indexes (see page 598) collection.

When IndexesActive is False, the dataset does not maintain the data views. That avoids the overhead of updating views, while data is fetched or is edited. When IndexesActive is True (by default), the dataset maintains the data views.

When indexes are maintained, each of them may be current index, selected by TADIndex (see page 653).Selected (see page 659) or by IndexName (see page 601) properties.

To selectively enable and disable data views rather than turning them all on or off at once, use the Active (see page 655) property of individual TADIndex (see page 653) objects. These objects are available through the Indexes (see page 598) property.

If application needs to perform batch updates to dataset, then set IndexesActive to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property IndexesActive: Boolean;
```

See Also

Sorting Records (see page 100), Indexes, BeginBatch, EndBatch

Example

```
var
  lPrevIndsActive: Boolean;
...
lPrevIndsActive := ADQuery1.IndexesActive;
ADQuery1.IndexesActive := False;
try
  // perform updates here, without maintaining client views
finally
  ADQuery1.IndexesActive := lPrevIndsActive;
end;
```

1.17.1.1.22.2.24 TADQuery.IndexFieldNames Property

Lists the field names to use as an index.

Description

Use IndexFieldNames as an alternative method of specifying the current index to use for a dataset.

Specify the name of each field on which to index the dataset, separating names with semicolons. Ordering of field names is significant. Optionally specify the postfix for any of fields in form "field[:[D][A][N]]", where:

- 'D' - use descending sorting on this field;

- 'A' - use ascending sorting on this field;

- 'N' - use case-insensitive sorting on this field.

Use IndexFieldNames to create sort orders on the fly. The IndexFieldNames and IndexName (see page 601) properties are mutually exclusive. Setting one clears the other.

Syntax

```
property IndexFieldNames: String;
```

See Also

Sorting Records (see page 100), Indexes, IndexName

Example

```
ADQuery1.IndexFieldNames := 'order_date;customer_name:N';
```

1.17.1.1.22.2.25 TADQuery.IndexName Property

Gets / sets the current index for the dataset by its name.

Description

Use IndexName to specify the current index for the dataset.

If IndexName is empty, the dataset's sort order is based on the IndexFieldNames (see page 600) property or on its default ordering as it is in dataset. If IndexName contains a name of valid index from Indexes collection, then that index is used to determine data view (including sort order) of records.

IndexFieldNames (see page 600) and IndexName are mutually exclusive. Setting one clears the other.

Syntax

```
property IndexName: String;
```

See Also

Sorting Records (see page 100), Indexes, IndexFieldNames

Example

```
with ADMemTable1.Indexes.Add do begin
  Name := 'by_name';
  Fields := 'NAME;DT';
  Active := True;
end;
```

```
ADMemTable1.IndexName := 'by_name';
```

1.17.1.1.22.2.26 TADQuery.LocalSQL Property

The reference to a Local SQL engine.

Description

Use this property to get / set reference to a Local SQL engine.

When engine and Local SQL connection are active, then right after setting LocalSQL property, the dataset will be opened (if not yet) and will be registered in Local SQL engine. Otherwise dataset will be registered when engine and connection will be active. The TADMemTable (see page 436) structure must be defined before it will be opened.

The dataset will be registered using its Name property value. So, its Name property value must be not empty and unique across all datasets linked to the same Local SQL engine.

Setting this property to nil unregisters this dataset from Local SQL engine.

Syntax

```
property LocalSQL: TADCustomLocalSQL;
```

See Also

Local SQL (see page 96), TADLocalSQL (see page 825), TADMemTable (see page 436)

1.17.1.1.22.2.27 TADQuery.Macros Property

The collection of macro substitution variables.

Description

The Macros property is the collection of macro substitution variables. It is populated automatically after assigning SQL property, if ResourceOptions.MacroCreate = True. Otherwise it may be populated by hands. At Prepare call, macro values will be substituted into SQL, if ResourceOptions.MacroExpand = True.

Syntax

```
property Macros: TADMacros;
```

See Also

TADCustomQuery.SQL, Params, Prepare, TADMacros, TADResourceOptions.MacroCreate, TADResourceOptions.MacroExpand

Example 1

Substitute table name:

```
ADQuery1.SQL.Text := 'select * from &Tab';
ADQuery1.Macros[0].AsIdentifier := 'Order Details';
ADQuery1.Prepare;
ShowMessage(ADQuery1.SQLText); // select * from "Order Details"
```

Example 2

Substitute WHERE condition:

```
ADQuery1.SQL.Text := 'select * from MyTab {if !cond} where !cond {fi}';

ADQuery1.Macros[0].AsString := 'ID > 100';
ADQuery1.Prepare;
ShowMessage(ADQuery1.SQLText); // select * from MyTab where ID > 100

ADQuery1.Macros[0].Clear;
ADQuery1.Prepare;
ShowMessage(ADQuery1.SQLText); // select * from MyTab
```

Example 3

See FireDAC\Samples\Comp Layer\TADQuery\Macros demo for details.

1.17.1.1.22.2.28 TADQuery.MasterFields Property

Gets / sets fields in master dataset, used to establish a master-detail relationship with this dataset.

Description

Use MasterFields property value to specify semicolon separated list of the master dataset field names, used to establish a master-detail relationship with this dataset.

Specify MasterFields property value after assigning MasterSource (see page 603) property value. The mechanism used to filter this detail dataset records depends on the dataset class:

- TADQuery (see page 476) and TADStoredProc (see page 514) will match master fields to this query / procedure parameters by their names. When the master dataset current record is changed, then this dataset parameters will get values from the corresponding MasterFields.
- TADMemTable (see page 436) will match master fields to this dataset indexed fields by their positions. When the master dataset current record is changed, then a range will be applied to this dataset, where starting and ending range values are equal to the corresponding MasterFields field values. The application must have a current active index. The indexed fields will be the detail dataset fields used to establish mater-detail.

Syntax

```
property MasterFields: String;
```

See Also

Master-Detail Relationship (see page 104), MasterSource, DetailFields, IndexFieldNames, IndexName

Example

```
ADMemTable1.IndexFieldNames := 'CustomerID';
ADMemTable1.MasterSource := CustomersDS;
ADMemTable1.MasterFields := 'ID';
```

1.17.1.1.22.2.29 TADQuery.MasterSource Property

Gets / sets a master data source, used to establish a master-detail relationship with this dataset.

Description

Use MasterSource to specify the data source linked to a dataset, which will be used as master dataset in master-detail relationship between this and specified datasets.

Additionally application must specify MasterFields (see page 602) property value. See MasterFields (see page 602) property description for additional information.

Syntax

```
property MasterSource: TDataSource;
```

See Also

Master-Detail Relationship (see page 104), MasterFields, IndexFieldNames, IndexName

Example 1

```
ADMemTable1.IndexFieldNames := 'CustomerID';
ADMemTable1.MasterSource := CustomersDS;
ADMemTable1.MasterFields := 'ID';
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\MasterDetail demo for details.

1.17.1.1.22.2.30 TADQuery.OnCommandChanged Event

Fires after the SQL property value is changed.

Description

The OnCommandChanged event fires after the SQL command text value is changed:

- for TADQuery (see page 476) - after SQL (see page 396) is changed;

- for TADStoredProc (see page 514) - after StoredProcName (see page 406) is changed;
- for TADTable (see page 539) - after TableName (see page 559) is changed.

Syntax

```
property OnCommandChanged: TNotifyEvent;
```

See Also

TADCustomQuery.SQL (see page 396), TADCustomStoredProc.StoredProcName (see page 406), TADTable.TableName (see page 559)

1.17.1.1.22.2.31 TADQuery.OnError Event

The event fires when an error happens, while dataset is communicating with DBMS.

Parameters

Parameters	Description
ASender	The reference to dataset.
AInitiator	The reference to interface of object, which initially raised the exception.
AException	The exception object, which may be modified or even replaced by another exception object instance.

Description

The OnError event fires, when dataset executes one of the following operations and an error happens:

- Prepare (see page 511). The dataset is preparing DBMS command text for execution.
- Open (see page 509). The dataset is executing DBMS command to open cursor.
- Execute (see page 622), ExecSQL (see page 397), ExecProc (see page 410). The dataset is executing DBMS command text.
- Navigation methods. The dataset is fetching rows from cursor.

If most cases AException object will of EADDBEngineException (see page 833) class. If AException is of EADDBArrayExecuteError (see page 831) class, then it is Array DML (see page 82) error handling case. You should use a code like in the first example.

To centralize error handling, you can consider to use TADCustomConnection (see page 318).OnError (see page 332).

Syntax

```
property OnError: TADErrorEvent;
```

See Also

TADCustomQuery.ExecSQL (see page 397), TADCustomStoredProc.ExecProc (see page 410), TDataSet.Open, Prepare, EADDBArrayExecuteError (see page 831), EADDBEngineException (see page 833), TADCustomConnection.OnError (see page 332)

Example 1

Handling Array DML errors:

```
procedure TForm1.ADQuery1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if AException is EADPhysArrayExecuteError then
    with EADPhysArrayExecuteError(AException) do
      if Errors[0].Kind = ekUKViolated then
        AAction := eaSkip
      else
        AAction := eaFail
  else
    ...
end;
```

Example 2

Error substitution:

```
procedure TForm1.ADQuery1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if EADDBEngineException(AException).Errors[0].Kind = ekPKViolated then begin
    AException.Free;
    AException := Exception.Create('Please, enter unique ID value');
  end;
end;
```

1.17.1.1.22.2.32 TADQuery.OnExecuteError Event

The event is triggered when an error happens, while executing a Array DML query.

Parameters

Parameters	Description
ASender	The reference to dataset.
ATimes	The current Array DML size. It may be different from value of the Execute method ATimes parameter, if full operation is splitted into few more small (on Firebird, for example).
AOffset	The current Array DML offset. It may be different from value of the Execute method AOffset parameter, if there was skipped erroneous rows or full operation is splitted into few more small (on Firebird, for example).
AError	The original DBMS returned error. Check AError.Errors (see page 834)[i].RowIndex (see page 839) for the index of erroneous row in parameters array.
AAction	The action dataset should take after return from event handler.

Description

The OnExecuteError event is triggered, when the dataset failed to execute Array DML using Execute method and ATimes > 1. Using this event handler you can:

- skip erroneous row of the parameters array;
- change values in erroneous row of the parameters array;
- stop full Array DML execution.

Note, OnExecuteError will be not called for the syntax errors or when ATimes = 1.

Syntax

```
property OnExecuteError: TADExecuteErrorEvent;
```

See Also

Array DML (see page 82), EADDBEngineException (see page 833), TADErrorAction, TADDataSet.Execute (see page 622)

Example

```
procedure TForm1.ADQuery1ExecuteError(ASender: TObject; ATimes,
  AOffset: Integer; AError: EADDBEngineException; var AAction: TADErrorAction);
begin
  if AException.Errors[0].Kind = ekUKViolated then
    AAction := eaSkip
  else
    AAction := eaFail;
end;
```

1.17.1.1.22.2.33 TADQuery.OnMasterSetValues Event

Fires after a detailed dataset has set detailed query parameters.

Description

Use the OnMasterSetValues event handler to override parameter values supplied to detail dataset from the master dataset. Also, because BeforeOpen and AfterOpen events do not fire for a detail dataset, the OnMasterSetValues may be used instead of them.

Syntax

```
property OnMasterSetValues: TADDataSetEvent;
```

See Also

Master-Detail Relationship (see page 104), MasterSource, MasterFields

1.17.1.1.22.2.34 TADQuery.OnReconcileError Event

Fires when a dataset needs to reconcile an update to a record that cannot not be applied.

Description

Use the OnReconcileError event handler to respond to error conditions that arise when the ApplyUpdates (see page 611) method was applying changes to the records to the database. The OnReconcileError event handler is called by Reconcile (see page 642) method.

As part of Reconcile call the dataset will loop through all records with associated exception objects. And for each such record will call the OnReconcileError event handler. The event handler gets:

- E - reference to the exception object. See Handling Errors (see page 44) topics for how to work with exception objects.
- UpdateKind - the update kind of the current record. It may be one of the values: rsInserted, rsDeleted, rsModified, rsUnchanged.

After handling error the event handler should set Action argument. The default value is raMerge. The possible values are:

Action	Description
raSkip	Just skip current record.
raAbort	Just quit from Reconcile call.
raMerge	Clear the current record error state, make current record changes the new initial state of this record. IOW, merge changes into dataset records cache.
raCorrect	Clear current record error state. IOW, mark the record as correctly applied.
raCancel	Cancel current record changes.
raRefresh	Cancel current record changes and reread the record values from the database.

The event handler may analyze the original and current field values, by reading TField.OldValue and NewValue properties. Application may also update the current field value, set Action to raCorrect, and later call ApplyUdpates again.

Syntax

```
property OnReconcileError: TADReconcileErrorEvent;
```

See Also

Caching Updates (see page 114), Reconcile, ApplyUpdates, OnUpdateError

Example

```
procedure TForm1.ADMemTable1ReconcileError(DataSet: TADDataSet; E: EADException;
  UpdateKind: TADDatSRowState; var Action: TADDAptReconcileAction);
begin
  if (UpdateKind = rsInserted) and (E is EADDBEngineException) and
(EADDBEngineException(E).Kind = ekUKViolated) then begin
    DataSet.FieldByName('ID').AsInteger := GetNextFreeID;
    Action := raCorrect;
  end;
end;
```

1.17.1.1.22.2.35 TADQuery.OnUpdateError Event

Fires if an exception is generated when updates are applied to a database.

Description

Use the OnUpdateError event handler to respond to exceptions raised while applying immediate or cached updates to a database.

ASender is the dataset to which updates are applied.

AException is a pointer to a EADException (see page 836) object from which an application can extract an error message and the actual cause of the error condition. The OnUpdateError handler can use this information to determine how to respond to the error condition. In most cases AException object will be of class EADDBEngineException (see page 833) or even one of it DBMS specific subclasses.

ARow is a DatS row object, repesenting erroneous record in dataset. This record is also current record in ASender.

ARequest indicates whether the error occurred while inserting, deleting, or modifying a record.

AAction indicates the action to take when the OnUpdateError handler exits. On entry into the handler, AAction is always set to eaDefault, which leads to uaFail, if not changed. If OnUpdateError can handle or correct the error, set AAction to uaRetry before exiting the error handler. Or consider other options.

The error handler can use the OldValue and NewValue properties TField to evaluate error conditions and set NewValue to a new value to reapply. In this case, set AAction to uaRetry before exiting.

The code in an OnUpdateError handler must not call any methods that make a different record the current one !

Syntax

```
property OnUpdateError: TADUpdateErrorEvent;
```

See Also

Caching Updates (see page 114), ApplyUpdates, CachedUpdates, Post, Delete, EADDBEngineException, OnUpdateRecord

Example

```
procedure TForm1.ADQuery1UpdateError(ASender: TDataSet; AException: EADException;
    ARow: TADDatSRow; ARequest: TADUpdateRequest; var AAction: TADErrorAction);
begin
  if (E is EADDBEngineException) and (EADDBEngineException(E).Kind = ekUKViolated) then
begin
    DataSet.FieldByName('ID').AsInteger := GetNextFreeID;
    Action := eaRetry;
  end;
end;
```

1.17.1.1.22.2.36 TADQuery.OnUpdateRecord Event

Occurs when record update is applying to a database.

Description

Use an OnUpdateRecord event handler to process immediate or cached updates that cannot be correctly or easily handled:

- using FireDAC automatic SQL update command generation (see page 121);
- by a single table adapter component;
- by a single update component.

This event is useful for applications that require:

- additional control over parameters and macros substitution in update components;
- multi SQL statement updates, when DBMS does not support batches or blocks;
- non-SQL updates posting;
- etc

ASender is the dataset to which updates are applied.

ARequest indicates whether the current update is the insertion, deletion or modification of a record.

AAction indicates the action taken by the OnUpdateRecord handler before it exits. On entry into the handler, AAction is always set to eaDefault. If OnUpdateRecord is successful, it should set AAction to eaApplied before exiting.

AAction value	Description
eaFail	Mark the update as failed and return an error.
eaSkip	Skip current update and do not mark it as applied.
eaRetry	Retry the current operation.
eaApplied	Mark current update as applied.
eaDefault	Take default action. For successful update it is eaApplied, for failed update it is eaFail.
eaExitSuccess	Stop to process the cached updates, return success.
eaExitFailure	Stop to process the cached updates, return failure.

The OnUpdateRecord event handler code should read the dataset field values, including TField.NewValue, OldValue and CurValue. There:

- NewValue is the new field value before posting updates.
- OldValue is the original field value, as it was after record fetching or after last updates applying.
- CurValue is the current field value, the same as Value property.

Note, the OnUpdateRecord handler code must not call the methods that change the current dataset position.

Syntax

```
property OnUpdateRecord: TADUpdateRecordEvent;
```

See Also

Overriding Posting Updates (see page 123), ApplyUpdates, CachedUpdates, Post, Delete, OnUpdateError

Example 1

```
procedure TForm1.ADQuery1UpdateRecord(ASender: TDataSet; ARequest: TADUpdateRequest;
 var AAction: TADErrorAction; AOptions: TADUpdateRowOptions);
begin
  if ARequest = arInsert then begin
    // set the SQL command to insert new record
    ADQuery2.SQL := 'insert into mytab (id, code, name) values (:id, :code, :name)
returning tmstamp into :ts';

    // set parameter values
    ADQuery2.Params[0].Value := ASender['id'];
    ADQuery2.Params[1].Value := ASender['code'];
    ADQuery2.Params[2].Value := ASender['name'];

    // specially define TS parameter
    ADQuery2.Params[3].DataType := ftDateTime;
    ADQuery2.Params[3].ParamType := ptOutput;

    // insert new record
    ADQuery2.ExecSQL;

    // move TS output parameter value back to dataset
    ASender['tmstamp'] := ADQuery2.Params[3].Value;

    // return 'Ok' status
    AAction := eaApplied;
  end;
end;
```

Example 2

See demos:

- FireDAC\Samples\Comp Layer\TADQuery\CachedUpdates\OnUpdateRecord
- FireDAC\Samples\Comp Layer\TADUpdateSQL\Main

1.17.1.1.22.2.37 TADQuery.Params Property

Contains the parameters for a SQL statement.

Description

Use Params property to view and set SQL statement parameter names, values, and data types at run time. Use the collection editor for the Params property to set parameter information at design time.

If dataset is TADQuery (see page 476), then Params is populated automatically after changing SQL (see page 396) property value, if ResourceOptions (see page 652).ParamCreate (see page 883) = True. The parameters order is the same as they appear in the SQL command. When Params.BindMode = pbByName, then each parameter name will appear in Params only once. When Params.BindMode = pbByNumber, then each parameter marker will appear in the Params, including duplicated names.

If dataset is TADStoredProc (see page 514), then Params is populated automatically after calling Prepare (see page 511) method at run time or changing StoredProcName (see page 406) at design time, if fiMeta is in FetchOption (see page 651).Items (see page 855). The parameters order is the same as in the stored procedure header.

Otherwise Params may be populated by hands. At Prepare (see page 511) call, parameters will be bind to prepared SQL command. After that you cannot change parameter types, otherwise will get an exception.

An easier way to set and retrieve parameter values when the name of each parameter is known is to call ParamByName (see page 641). This method is especially preferred versus accessing parameters by index when they are populated automatically and there is a chance that the SQL property or StoredProcName property will be changed in the code dynamically.

Syntax

```
property Params: TADParams;
```

See Also

Executing Command (see page 66), Executing Stored Procedure (see page 71), TADCustomQuery.SQL (see page 396), TADCustomStoredProc.StoredProcName (see page 406), TADParams, TADResourceOptions.ParamCreate (see page 883), TADFetchOptions.Items (see page 855)

Example 1

```
// TADQuery automatic parameters creation - ResourceOptions.ParamCreate = True
ADQuery1.SQL.Text := 'select * from mytab where id = :id';
ADQuery1.Params[0].AsString := 'qwe';
ADQuery1.Open;
```

Example 2

```
// TADQuery manual parameters creation - ResourceOptions.ParamCreate = False
ADQuery1.ResourceOptions.ParamCreate := False;
ADQuery1.SQL.Text := 'select * from mytab where id = :id';
with ADQuery1.Params.Add do begin
  Name := 'id';
  DataType := ftString;
  Size := 10;
  AsString := 'qwe';
end;
ADQuery1.Open;
```

Example 3

```
// TADStoredProc automatic parameters creation - fiMeta in FetchOptions.Items
ADStoredProc1.StoredProcName := 'my_proc';
ADStoredProc1.Prepare;
ADStoredProc1.Params[0].AsString := 'qwe';
ADStoredProc1.ExecProc;
```

Example 4

```
// TADStoredProc manual parameters creation - fiMeta not in FetchOptions.Items
ADStoredProc1.FetchOptions.Items := ADStoredProc1.FetchOptions.Items - [fiMeta];
ADStoredProc1.StoredProcName := 'my_proc';
with ADStoredProc1.Params.Add do begin
  Name := 'id';
  DataType := ftString;
  Size := 10;
end;
ADStoredProc1.Prepare;
ADStoredProc1.Params[0].AsString := 'qwe';
ADStoredProc1.ExecProc;
```

1.17.1.1.22.2.38 TADQuery.ResourceOptions Property

The set of options to control resources usage.

Description

The ResourceOptions property is the set of properties, controlling the resources usage by the dataset. These properties will inherit its values from ResourceOptions of connection object.

Syntax

```
property ResourceOptions: TADBottomResourceOptions;
```

See Also

Setting Options (see page 34), TADBottomResourceOptions (see page 841)

1.17.1.1.22.2.39 TADQuery.SchemaAdapter Property

Gets or sets a reference to a schema adapter, as a Centralized Cached Updates log.

Description

Use SchemaAdapter property to set a reference to the TADSchemaAdapter (see page 514) object to enable Centralized Cashed Updates (see page 114) mode. The schema adapter object will be used as a central change log share by few datasets.

To enable propagation of master dataset changes to a detail dataset set FetchOptions.DetailCascade (see page 854) to True for a detail dataset. Also both master and detail datasets must point to the same TADSchemaAdapter (see page 514) object.

Syntax

```
property SchemaAdapter: TADCustomSchemaAdapter;
```

See Also

Caching Updates (see page 114), Master-Detail Relationship (see page 104), TADFetchOptions.DetailCascade (see page 854)

Example

```
dsOrders.DataSet := qOrders;
qOrders.SchemaAdapter := ADSchemaAdapter1;
qOrders.CachedUpdates := True;

dsOrderDetails.DataSet := qOrderDetails;
qOrderDetails.SchemaAdapter := ADSchemaAdapter1;
qOrderDetails.CachedUpdates := True;
qOrderDetails.MasterSource := dsOrders;
qOrderDetails.MasterFields := 'OrderID';
qOrderDetails.IndexFieldNames := 'OrderID';
qOrderDetails.FetchOptions.DetailCascade := True;

qOrders.Open;
qOrderDetails.Open;

qOrders.Append;
```

```
....
qOrders.Post;

qOrderDetails.Append;
...
qOrderDetails.Post;

ADSchemaAdapter1.ApplyUpdates;
qOrders.CommitUpdates;
qOrderDetails.CommitUpdates;
```

1.17.1.1.22.2.40 TADQuery.SQL Property

Contains the text of the SQL statement to execute for the query.

Description

Use SQL property to specify the SQL command that a query will Execute (see page 622) / ExecSQL (see page 397) / Open.

At design time the SQL (see page 396) property can be edited by invoking FireDAC Query Editor dialog (see page 66). For that double click on TADCustomQuery component. The dialog offers syntax hilighting editor, query builder, ability to test query and other.

The SQL (see page 396) property may contain one SQL command or "batch" SQL command, consisting from few SQL command or server side programming language block of code. To execute full featured SQL script use the TADScript (see page 685) component.

The SQL statement in the SQL (see page 396) property may contain also:

- parameter markers, following standard SQL-92 syntax conventions. Parameters are created automatically and stored in the Params (see page 651) property, if ResourceOptions (see page 652).ParamCreate (see page 883) is True. If ParamBindMode (see page 513)is pbByName, then for all occurrences of the same marker will be created single item in Params (see page 651). If pbByNumber, then - one item per each marker.
- substitution variable markers. Macros are created automatically and stored in the Macros (see page 512) property, if ResourceOptions (see page 652).MacroCreate (see page 882) is True.
- escape sequences. They allow writing DBMS independent SQL commands.
- conditional substitutions. They allow expand SQL command conditionally, depending on application defined attributes

For details see Preprocessing Command Text (see page 56).

To improove performance on adding big SQL queries to the property using TStringList methods, surround this code into SQL.BeginUpdate / SQL.EndUpdate.

After filling this property value the Param collection will be filled automatically, if ResourceOptions.ParamCreate is True.

Syntax

```
property SQL: TStrings;
```

See Also

Executing Command (see page 66), Preprocessing Command Text (see page 56), TADRdbmsDataSet.Prepare (see page 511), TDataSet.Open, ExecSQL, TADResourceOptions.ParamCreate (see page 883), TADResourceOptions.MacroCreate (see page 882), TADRdbmsDataSet.ParamBindMode (see page 513)

Example

```
ADQuery1.SQL.BeginUpdate;
try
  ADQuery1.SQL.Add('SELECT ...');
  ADQuery1.SQL.Add('...');
  ADQuery1.SQL.Add('...');
  ADQuery1.SQL.Add('WHERE ID = :ID');
finally
  ADQuery1.SQL.EndUpdate;
end;
ADQuery1.ParamByName('ID').AsInteger := 100;
ADQuery1.Open;
```

1.17.1.1.22.2.41 TADQuery.Transaction Property

Gets / sets reference to transaction object.

Description

The Transaction property allows to associate dataset with explicit transaction object. If it is not specified, then dataset will be associated with connection default transaction.

The specified transaction object will be used only for Prepare (see page 511), Open (see page 509), ExecSQL (see page 397), ExecProc (see page 410), Execute (see page 622) methods. For commands, generated to apply changes to database, FireDAC will use UpdateTransaction (see page 507), if it is specified, otherwise - Transaction.

Currently explicit transaction object assignment is supported only for Interbase / Firebird servers.

Syntax

```
property Transaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41), UpdateTransaction, TADCustomConnection.Transaction (see page 337)

1.17.1.1.22.2.42 TADQuery.UpdateObject Property

Specifies the update object component used to post updates by dataset.

Description

Use UpdateObject to specify the TADUpdateSQL (see page 563) component to use in an application that must be able to post updates in a non-standard way.

The FireDAC has build in updates command generator, which using multiple information sources is able to generate SQL update commands for most of scenarios. In some cases, such as a query made against multiple tables, which may require multi-statement update, FireDAC cannot generate SQL update commands. In these cases, UpdateObject can be used to specify a TADUpdateSQL (see page 563) component that allows to specify custom SQL commands for each update posting case, like a insert / update / delete.

As alternative to update object usage, you can consider to use OnUpdateRecord (see page 605) event. It gives more options, but requires manual coding of posting updates. Also, in OnUpdateRecord (see page 605) handler you can use multiple update objects. See demo FireDAC\Samples\Comp Layer\TADUpdateSQL\Main.

Syntax

```
property UpdateObject: TADCustomUpdateObject;
```

See Also

OnUpdateRecord (see page 605), TADUpdateSQL

Example 1

```
ADUpdateSQL1.InsertSQL.Text := 'insert into mytab values (:new_id, :new_name)';
ADQuery1.UpdateObject := ADUpdateSQL1;
ADQuery1.Append;
ADQuery1['id'] := 100;
ADQuery1['name'] := 'qwe';
ADQuery1.Post;
```

Example 2

Demo FireDAC\Samples\Comp Layer\TADUpdateSQL\Main.

1.17.1.1.22.2.43 TADQuery.UpdateOptions Property

The set of options to control updates posting.

Description

The UpdateOptions property is the set of properties, controlling the posting updates from the dataset. These properties will

inherit its values from UpdateOptions of connection object.

Syntax

```
property UpdateOptions: TADBottomUpdateOptions;
```

See Also

Setting Options (see page 34), TADBottomUpdateOptions (see page 843)

1.17.1.1.22.2.44 TADQuery.UpdateTransaction Property

Gets / sets reference to transaction object.

Description

The UpdateTransaction property allows to associate dataset internal update commands with explicit transaction object. If it is not specified, then dataset update commands will be associated with connection default update transaction. If it is not specified, then - with dataset Transaction.

For Prepare (see page 511), Open (see page 509), ExecSQL (see page 397), ExecProc (see page 410), Execute (see page 622) methods will be used Transaction (see page 507) objects.

Currently explicit update transaction object assignment is supported only for Interbase / Firebird servers.

Syntax

```
property UpdateTransaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41),Transaction, TADCustomConnection.UpdateTransaction (see page 338)

Example

```
var
  oSelectTransaction: TADTransaction;
  oUpdateTransaction: TADTransaction;
  oCusts: TADQuery;
.....
  oSelectTransaction := TADTransaction.Create(nil);
  oSelectTransaction.Connection := ADConnection1;
  oSelectTransaction.Options.ReadOnly := True;

  oUpdateTransaction := TADTransaction.Create(nil);
  oUpdateTransaction.Connection := ADConnection1;

  oCusts := TADQuery.Create(nil);
  oCusts.Connection := ADConnection1;
  oCusts.Transaction := oSelectTransaction;
  oCusts.UpdateTransaction := oUpdateTransaction;
  oCusts.Open('select * from Customers'); // uses oSelectTransaction
  ...
  oCusts.Edit; // uses oUpdateTransaction, if needed
  ...
  oCusts.Post; // uses oUpdateTransaction
```

1.17.1.1.23 TADRdbmsDataSet Class

The TADRdbmsDataSet class includes everything, required by descendants to execute SQL commands.

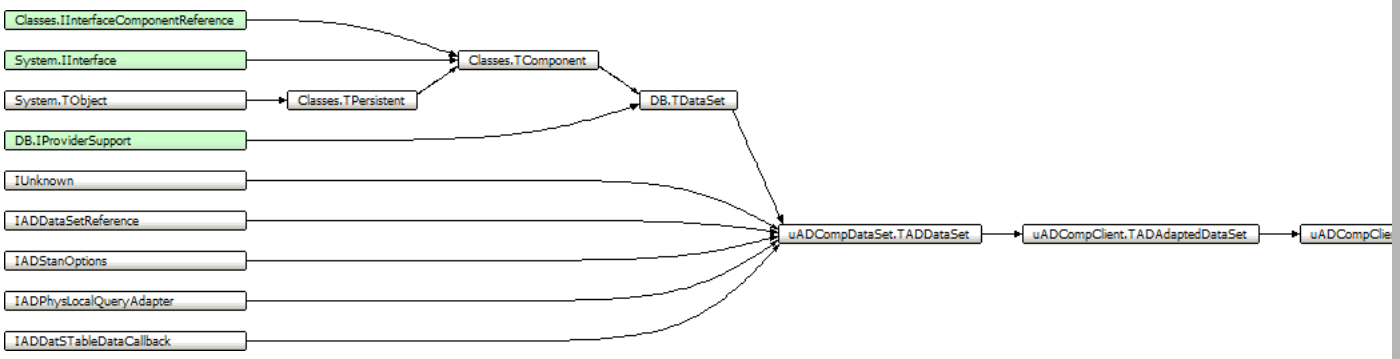
protected

protected	Description
MacroCount (see page 512)	Returns the number of macro in Macros (see page 512) collection.
Macros (see page 512)	The collection of macro substitution variables.
ParamBindMode (see page 513)	Determines the order in which a component's parameters are assigned to the parameter list for SQL command on the DBMS.
RowsAffected (see page 513)	Returns the number of rows operated upon by the latest query execution.

public

public	Description
Connection (see page 503)	Specifies the FireDAC connection component to use.
ConnectionName (see page 503)	Specifies the FireDAC connection to use by its name.
FetchOptions (see page 503)	The set of options to control rows fetching.
FormatOptions (see page 504)	The set of options to control data representation.
OnCommandChanged (see page 504)	Fires after the SQL property value is changed.
OnError (see page 504)	The event fires when an error happens, while dataset is communicating with DBMS.
Prepared (see page 505)	Gets or sets command prepare for execution status.
ResourceOptions (see page 506)	The set of options to control resources usage.
SchemaAdapter (see page 506)	Gets or sets a reference to a schema adapter, as a Centralized Cached Updates log.
Transaction (see page 507)	Gets / sets reference to transaction object.
UpdateOptions (see page 507)	The set of options to control updates posting.
UpdateTransaction (see page 507)	Gets / sets reference to transaction object.
Disconnect (see page 508)	Disconnects the dataset from DBMS.
FindMacro (see page 508)	Returns macro by its name.
MacroByName (see page 509)	Returns macro by its name.
Open (see page 509)	Executes the specified SQL statement and opens dataset.
Open (see page 510)	Executes the specified SQL statement with specified parameter values and opens dataset.
Open (see page 510)	Executes the specified SQL statement with specified parameter values and opens dataset.
Prepare (see page 511)	Prepares SQL command for execution.
Unprepare (see page 512)	Releases SQL command resources.

Class Hierarchy



File

uADCompClient

Description

The TADRdbmsDataSet class is an intermediate class in FireDAC dataset classes hierarchy. The programmer must not use it directly.

Use the TADRdbmsDataSet to subclass new dataset class capable to execute SQL commands. This class creates and maintains everything, required by descendants to execute SQL commands - table adapter and command objects.

For options description, please, look at uADStanOption (see page 840) description.

Syntax

```
TADRdbmsDataSet = class(TADAdaptedDataSet);
```

See Also

TADAdaptedDataSet (see page 259), TADCustomQuery (see page 394), TADCustomStoredProc (see page 403), TADMetaInfoQuery (see page 461), uADStanOption (see page 840)

1.17.1.1.23.1 public

1.17.1.1.23.1.1 TADRdbmsDataSet.Connection Property

Specifies the FireDAC connection component to use.

Description

Use Connection to specify an FireDAC connection object to use to connect to an DBMS.

At design-time, select from the list of available TADCustomConnection (see page 318) objects (if any have been added to the application) invoked from the Object Inspector. If a form or data module has TADCustomConnection (see page 318) dropped on it, then TADRdbmsDataSet assigns this connection object to Connection property automatically right after dropping on the form or data module.

At runtime, set Connection to reference an existing TADConnection (see page 279) object.

The TADRdbmsDataSet can be bind to TADCustomConnection (see page 318) object using Connection or ConnectionName (see page 503) property. Using ConnectionName (see page 503) property allows to bind to a connection object, if it is even not yet created or is not accessible.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

ConnectionName (see page 503)

1.17.1.1.23.1.2 TADRdbmsDataSet.ConnectionName Property

Specifies the FireDAC connection to use by its name.

Description

Use ConnectionName to specify an FireDAC connection to use to connect to an DBMS. The ConnectionName property value specifies the name of the connection. It must match to the:

- name of one of connection definitions, either stored in external file (persistent) or created on fly (private);
- ConnectionName of one of the TADConnection (see page 279) objects.

The ConnectionName property value must be specified before Prepare (see page 511) call. If it matches the name of one of connection definitions, then FireDAC will transparently create connection object and link the dataset with it.

The TADRdbmsDataSet cab be bind to TADCustomConnection (see page 318) object using Connection or ConnectionName property. Using Connection (see page 503) property allows to bind to a connection object explicitly and reduce the overhead for resolving connection names.

Syntax

```
property ConnectionName: String;
```

See Also

Connection (see page 503), TADCustomConnection.Temporary (see page 337)

Example

```
ADQuery1.ConnectionName := 'Ora_Demo';
ADQuery1.Open('select * from "Customers"');
```

1.17.1.1.23.1.3 TADRdbmsDataSet.FetchOptions Property

The set of options to control rows fetching.

Description

The FetchOptions property is the set of properties, controlling the fetching of cursor rows. These properties will inherit its values from FetchOptions of connection object.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), TADFetchOptions (see page 849)

1.17.1.1.23.1.4 TADRdbmsDataSet.FormatOptions Property

The set of options to control data representation.

Description

The FormatOptions property is the set of properties, controlling the data type mapping, and some other aspects of data handling.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), TADFormatOptions (see page 860), Params

1.17.1.1.23.1.5 TADRdbmsDataSet.OnCommandChanged Event

Fires after the SQL property value is changed.

Description

The OnCommandChanged event fires after the SQL command text value is changed:

- for TADQuery (see page 476) - after SQL (see page 396) is changed;
- for TADStoredProc (see page 514) - after StoredProcName (see page 406) is changed;
- for TADTable (see page 539) - after TableName (see page 559) is changed.

Syntax

```
property OnCommandChanged: TNotifyEvent;
```

See Also

TADCustomQuery.SQL (see page 396), TADCustomStoredProc.StoredProcName (see page 406), TADTable.TableName (see page 559)

1.17.1.1.23.1.6 TADRdbmsDataSet.OnError Event

The event fires when an error happens, while dataset is communicating with DBMS.

Parameters

Parameters	Description
ASender	The reference to dataset.
AInitiator	The reference to interface of object, which initially raised the exception.
AException	The exception object, which may be modified or even replaced by another exception object instance.

Description

The OnError event fires, when dataset executes one of the following operations and an error happens:

- Prepare (see page 511). The dataset is preparing DBMS command text for execution.

- Open (see page 509). The dataset is executing DBMS command to open cursor.
- Execute (see page 622), ExecSQL (see page 397), ExecProc (see page 410). The dataset is executing DBMS command text.
- Navigation methods. The dataset is fetching rows from cursor.

If most cases AException object will of EADDBEngineException (see page 833) class. If AException is of EADDBArrayExecuteError (see page 831) class, then it is Array DML (see page 82) error handling case. You should use a code like in the first example.

To centralize error handling, you can consider to use TADCustomConnection (see page 318).OnError (see page 332).

Syntax

```
property OnError: TADErrorEvent;
```

See Also

TADCustomQuery.ExecSQL (see page 397), TADCustomStoredProc.ExecProc (see page 410), TDataSet.Open, Prepare (see page 511), EADDBArrayExecuteError (see page 831), EADDBEngineException (see page 833), TADCustomConnection.OnError (see page 332)

Example 1

Handling Array DML errors:

```
procedure TForm1.ADQuery1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if AException is EADPhysArrayExecuteError then
    with EADPhysArrayExecuteError(AException) do
      if Errors[0].Kind = ekUKViolated then
        AAction := eaSkip
      else
        AAction := eaFail
  else
    ...
end;
```

Example 2

Error substitution:

```
procedure TForm1.ADQuery1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if EADDBEngineException(AException).Errors[0].Kind = ekPKViolated then begin
    AException.Free;
    AException := Exception.Create('Please, enter unique ID value');
  end;
end;
```

1.17.1.1.23.1.7 TADRdbmsDataSet.Prepared Property

Gets or sets command prepare for execution status.

Description

Set Prepared to True to prepare SQL command for execution. Setting Prepared to True, will call Prepare (see page 511) method. It is not required to set Prepared to True for standard SQL command before execution, because first Execute (see page 622) / ExecSQL (see page 397) / Open call will automatically prepare command. But it is required for stored procedures, if you need to automatically populate Params (see page 651) collection. After command is prepared, SQLText (see page 306) will return SQL command text as it is sent to the DBMS.

To prepare SQL command DBMS connection must be active, otherwise an exception will be raised.

After SQL command preparation application cannot change command parameter data types and sizes, otherwise on next Execute or Open call an exception will be raised. So, setup parameters before setting Prepared to True.

Set Prepared to False to unprepare SQL command and release all it resources. Setting Prepared to False, will call Unprepare (see page 512) method.

Syntax

```
property Prepared: Boolean;
```

See Also

Prepare (see page 511), Unprepare (see page 512), Params

1.17.1.1.23.1.8 TADRdbmsDataSet.ResourceOptions Property

The set of options to control resources usage.

Description

The ResourceOptions property is the set of properties, controlling the resources usage by the dataset. These properties will inherit its values from ResourceOptions of connection object.

Syntax

```
property ResourceOptions: TADBottomResourceOptions;
```

See Also

Setting Options (see page 34), TADBottomResourceOptions (see page 841)

1.17.1.1.23.1.9 TADRdbmsDataSet.SchemaAdapter Property

Gets or sets a reference to a schema adapter, as a Centralized Cached Updates log.

Description

Use SchemaAdapter property to set a reference to the TADSchemaAdapter (see page 514) object to enable Centralized Cashed Updates (see page 114) mode. The schema adapter object will be used as a central change log share by few datasets.

To enable propagation of master dataset changes to a detail dataset set FetchOptions.DetailCascade (see page 854) to True for a detail dataset. Also both master and detail datasets must point to the same TADSchemaAdapter (see page 514) object.

Syntax

```
property SchemaAdapter: TADCustomSchemaAdapter;
```

See Also

Caching Updates (see page 114), Master-Detail Relationship (see page 104), TADFetchOptions (see page 849).DetailCascade (see page 854)

Example

```
dsOrders.DataSet := qOrders;
qOrders.SchemaAdapter := ADSchemaAdapter1;
qOrders.CachedUpdates := True;

dsOrderDetails.DataSet := qOrderDetails;
qOrderDetails.SchemaAdapter := ADSchemaAdapter1;
qOrderDetails.CachedUpdates := True;
qOrderDetails.MasterSource := dsOrders;
qOrderDetails.MasterFields := 'OrderID';
qOrderDetails.IndexFieldNames := 'OrderID';
qOrderDetails.FetchOptions.DetailCascade := True;

qOrders.Open;
qOrderDetails.Open;

qOrders.Append;
....
qOrders.Post;

qOrderDetails.Append;
...
qOrderDetails.Post;
```

```
ADSchemaAdapter1.ApplyUpdates;
qOrders.CommitUpdates;
qOrderDetails.CommitUpdates;
```

1.17.1.1.23.1.10 TADRdbmsDataSet.Transaction Property

Gets / sets reference to transaction object.

Description

The Transaction property allows to associate dataset with explicit transaction object. If it is not specified, then dataset will be associated with connection default transaction.

The specified transaction object will be used only for Prepare (see page 511), Open (see page 509), ExecSQL (see page 397), ExecProc (see page 410), Execute (see page 622) methods. For commands, generated to apply changes to database, FireDAC will use UpdateTransaction (see page 507), if it is specified, otherwise - Transaction.

Currently explicit transaction object assignment is supported only for Interbase / Firebird servers.

Syntax

```
property Transaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41), UpdateTransaction (see page 507), TADCustomConnection.Transaction (see page 337)

1.17.1.1.23.1.11 TADRdbmsDataSet.UpdateOptions Property

The set of options to control updates posting.

Description

The UpdateOptions property is the set of properties, controlling the posting updates from the dataset. These properties will inherit its values from UpdateOptions of connection object.

Syntax

```
property UpdateOptions: TADBottomUpdateOptions;
```

See Also

Setting Options (see page 34), TADBottomUpdateOptions (see page 843)

1.17.1.1.23.1.12 TADRdbmsDataSet.UpdateTransaction Property

Gets / sets reference to transaction object.

Description

The UpdateTransaction property allows to associate dataset internal update commands with explicit transaction object. If it is not specified, then dataset update commands will be associated with connection default update transaction. If it is not specified, then - with dataset Transaction.

For Prepare (see page 511), Open (see page 509), ExecSQL (see page 397), ExecProc (see page 410), Execute (see page 622) methods will be used Transaction (see page 507) objects.

Currently explicit update transaction object assignment is supported only for Interbase / Firebird servers.

Syntax

```
property UpdateTransaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41),Transaction (see page 507), TADCustomConnection.UpdateTransaction (see page 338)

Example

```
var
  oSelectTransaction: TADTransaction;
  oUpdateTransaction: TADTransaction;
  oCusts: TADQuery;
.....
  oSelectTransaction := TADTransaction.Create(nil);
  oSelectTransaction.Connection := ADConnection1;
  oSelectTransaction.Options.ReadOnly := True;

  oUpdateTransaction := TADTransaction.Create(nil);
  oUpdateTransaction.Connection := ADConnection1;

  oCusts := TADQuery.Create(nil);
  oCusts.Connection := ADConnection1;
  oCusts.Transaction := oSelectTransaction;
  oCusts.UpdateTransaction := oUpdateTransaction;
  oCusts.Open('select * from Customers'); // uses oSelectTransaction
  ...
  oCusts.Edit; // uses oUpdateTransaction, if needed
  ...
  oCusts.Post; // uses oUpdateTransaction
```

1.17.1.1.23.1.13 TADRdbmsDataSet.Disconnect Method

Disconnects the dataset from DBMS.

Parameters

Parameters	Description
AAbortJob: Boolean = False	True, if current dataset task must be aborted.

Description

The Disconnect method optionally aborts current dataset task, like a Open (see page 509), ExecSQL (see page 397), ExecProc (see page 410), Execute (see page 622) or rows fetching. Then closes dataset and unprepares it.

Syntax

```
procedure Disconnect(AAbortJob: Boolean = False); override;
```

See Also

AbortJob (see page 263)

Example

```
ADQuery1.Disconnect(True);
// is equivalent to:
// ADQuery1.AbortJob(True);
// ADQuery1.Close;
// ADQuery1.Unprepare;
```

1.17.1.1.23.1.14 TADRdbmsDataSet.FindMacro Method

Returns macro by its name.

Parameters

Parameters	Description
const AValue: string	A name of macro to return.

Returns

The macro with Name = AValue or nil if such macro does not exists.

Description

The FindMacro method returns a macro from Macros (see page 512) collection by its name AValue. If there is no such macro, then method will return nil.

Syntax

```
function FindMacro(const AValue: string): TADMacro;
```

See Also

Macros (see page 512), MacroByName (see page 509)

1.17.1.1.23.1.15 TADRdbmsDataSet.MacroByName Method

Returns macro by its name.

Parameters

Parameters	Description
const AValue: string	A name of macro to return.

Returns

The macro with Name = AValue.

Description

The MacroByName method returns a macro from Macros (see page 512) collection by its name AValue. If there is no such macro, then exception will be raised.

Syntax

```
function MacroByName(const AValue: string): TADMacro;
```

See Also

Macros (see page 512), FindMacro (see page 508)

1.17.1.1.23.1.16 TADRdbmsDataSet.Open Method (String)

Executes the specified SQL statement and opens dataset.

Parameters

Parameters	Description
const ASQL: String	A SQL command to execute.

Description

Call Open to execute the SQL statement specified in ASQL or is currently assigned to the internal command object, if ASQL is empty.

The method closes the dataset if it was active, then assigns ASQL to the command text, and sets the Active property to True. When Active is true, the dataset is populated with data from a database.

When an application needs to execute the same SQL command many times, then an application may ordinarily set SQL property, call Prepare before calling Open for the first time, then use any Open version without specifying ASQL.

Use ResourceOptions (see page 506).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 506).CmdExecTimeout (see page 881) to set maximum command execution time. After that time command execution will be canceled and exception raised.

To cancel command execution use TADAdaptedDataSet (see page 259).AbortJob (see page 263).

If DBMS returns error, then FireDAC raises an exception. It may be analyzed in OnError (see page 504) event.

Syntax

```
procedure Open(const ASQL: String); overload;
```

See Also

Prepare (see page 511), TDataSet.Open, TADCustomQuery.SQL (see page 396), TADCustomStoredProc.StoredProcName (see page 406), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261)

Example

```
ADQuery1.Open('select * from customers');
```

1.17.1.1.23.1.17 TADRdbmsDataSet.Open Method (String, array of Variant)

Executes the specified SQL statement with specified parameter values and opens dataset.

Parameters

Parameters	Description
const ASQL: String	A SQL command to execute.
const AParams: array of Variant	And open array of parameter values.

Description

Call Open to execute the SQL statement specified in ASQL or is currently assigned to the internal command object, if ASQL is empty. And a command has parameters.

The method closes the dataset if it was active, then assigns ASQL to the command text, and sets the Active property to True. When Active is true, the dataset is populated with data from a database. This method allows to specify the parameter values using positional parameter binding.

When an application needs to execute the same SQL command many times, then an application may ordinarily set SQL property, call Prepare before calling Open for the first time, then use any Open version without specifying ASQL.

Use ResourceOptions (see page 506).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 506).CmdExecTimeout (see page 881) to set maximum command execution time. After that time command execution will be canceled and exception raised.

To cancel command execution use TADAdaptedDataSet (see page 259).AbortJob (see page 263).

When a DBMS returns an error, then FireDAC raises an exception. See Handling Errors (see page 44) for details. In case of long string values and error like that:

[FireDAC][Phys][ODBC]-345. Data too large for variable [P]. Max len = [8002], actual len = [18329]

you have to use the Open (see page 510) version, which accepts the parameter data types. And to specify ftMemo or ftBlob for corresponding ('P') parameter. Or just use the standard Open call and setup parameter data types and values separately.

Syntax

```
procedure Open(const ASQL: String; const AParams: array of Variant); overload;
```

See Also

Prepare (see page 511), TDataSet.Open, TADCustomQuery.SQL (see page 396), TADCustomStoredProc.StoredProcName (see page 406), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261)

Example

```
ADQuery1.Open('select * from Employees where department_id = :id', [100]);
....
ADQuery1.Open('', [101]); // executes the above SQL command, but for ID = 101
....
ADQuery1.Open('', [102]); // executes the above SQL command, but for ID = 101
....
```

1.17.1.1.23.1.18 TADRdbmsDataSet.Open Method (String, array of Variant, array of TFieldType)

Executes the specified SQL statement with specified parameter values and opens dataset.

Parameters

Parameters	Description
const ASQL: String	A SQL command to execute.
const AParams: array of Variant	And open array of parameter values.

const ATypes: array of TFieldType	And open array of parameter data types.

Description

Call Open to execute the SQL statement specified in ASQL or is currently assigned to the internal command object, if ASQL is empty. And a command has parameters.

The method closes the dataset if it was active, then assigns ASQL to the command text, and sets the Active property to True. When Active is true, the dataset is populated with data from a database. This method allows to specify the parameter values using positional parameter binding, and exact parameter data types. If you need to leave the parameter data type unchanged, then put ftUnknown into array item corresponding to the parameter.

When an application needs to execute the same SQL command many times, then an application may ordinarily set SQL property, call Prepare before calling Open for the first time, then use any Open version without specifing ASQL.

Use ResourceOptions (see page 506).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 506).CmdExecTimeout (see page 881) to set maximum command execution time. After that time command execution will be canceled and exception raised.

To cancel command execution use TADAdaptedDataSet (see page 259).AbortJob (see page 263).

When a DBMS returns an error, then FireDAC raises an exception. See Handling Errors (see page 44) for details. In case of long string values and error like that:

[FireDAC][Phys][ODBC]-345. Data too large for variable [P]. Max len = [8002], actual len = [18329]

you have to specify ftMemo or ftBlob for corresponding ('P') parameter. Or just use the standard Open call and setup parameter data types and values separately.

Syntax

```
procedure Open(const ASQL: String; const AParams: array of Variant; const ATypes: array of
TFieldType); overload;
```

See Also

Prepare (see page 511), TDataSet.Open, TADCustomQuery.SQL (see page 396), TADCustomStoredProc.StoredProcName (see page 406), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261)

Example

```
ADQuery1.Open('select * from Employees where department_id = :id', [100], [ftInteger]);
```

1.17.1.1.23.1.19 TADRdbmsDataSet.Prepare Method

Prepares SQL command for execution.

Description

Call the Prepare method to prepare SQL command for execution. It is not required to call Prepare for standard SQL command before execution, because first Execute (see page 622) / ExecSQL (see page 397) / Open (see page 509) call will automatically prepare command. But it is required for stored procedures, if you need to automatically populate Params collection. After command is prepared, SQLText (see page 306) will return SQL command text as it is sent to the DBMS.

To prepare SQL command DBMS connection must be active, otherwise exception will be raised.

After prepare call application cannot change command parameter data types and sizes, otherwise on next Execute (see page 622) / ExecSQL (see page 397) / ExecProc (see page 410) / Open (see page 509) call an exception will be raised. So, setup parameters before Prepare call.

Syntax

```
procedure Prepare;
```

See Also

Prepared (see page 505), Unprepare (see page 512), Params

Example

```
ADQuery1.SQL.Text := 'select * from MyTab';
ADQuery1.Prepare;
```

1.17.1.1.23.1.20 TADRdbmsDataSet.Unprepare Method

Releases SQL command resources.

Description

Use Unprepare call to release prepared SQL command resources. You can unprepare command by setting Prepared to False.

Syntax

```
procedure Unprepare;
```

See Also

Prepare (see page 511), Prepared (see page 505)

1.17.1.1.23.2 protected

1.17.1.1.23.2.1 TADRdbmsDataSet.MacroCount Property

Returns the number of macro in Macros (see page 512) collection.

Description

The MacroCount property teturns the number of of macro substitution variables in Macros collection.

Syntax

```
property MacroCount: Word;
```

See Also

Macros (see page 512)

1.17.1.1.23.2.2 TADRdbmsDataSet.Macros Property

The collection of macro substitution variables.

Description

The Macros property is the collection of macro substitution variables. It is populated automatically after assigning SQL property, if ResourceOptions.MacroCreate = True. Otherwise it may be populated by hands. At Prepare call, macro values will be substituted into SQL, if ResourceOptions.MacroExpand = True.

Syntax

```
property Macros: TADMacros;
```

See Also

TADCustomQuery.SQL, Params, Prepare (see page 511), TADMacros, TADResourceOptions.MacroCreate, TADResourceOptions.MacroExpand

Example 1

Substitute table name:

```
ADQuery1.SQL.Text := 'select * from &Tab';
ADQuery1.Macros[0].AsIdentifier := 'Order Details';
ADQuery1.Prepare;
ShowMessage(ADQuery1.SQLText); // select * from "Order Details"
```

Example 2

Substitute WHERE condition:

```
ADQuery1.SQL.Text := 'select * from MyTab {if !cond} where !cond {fi}';
```

```
ADQuery1.Macros[0].AsString := 'ID > 100';
ADQuery1.Prepare;
ShowMessage(ADQuery1.SQLText); // select * from MyTab where ID > 100

ADQuery1.Macros[0].Clear;
ADQuery1.Prepare;
ShowMessage(ADQuery1.SQLText); // select * from MyTab
```

Example 3

See FireDAC\Samples\Comp Layer\TADQuery (see page 476)\Macros demo for details.

1.17.1.1.23.2.3 TADRdbmsDataSet.ParamBindMode Property

Determines the order in which a component's parameters are assigned to the parameter list for SQL command on the DBMS.

Description

Get or set ParamBindMode to determine the order in which parameters in the Params property are matched to the parameters used by the SQL command on the server. ParamBindMode can be one of the following:

Value	Ordering
pbByName	Parameters specified in the Params property are matched to identically named parameters on the server. This is the default.
pbByNumber	Parameters in Params are assigned one-by-one to the next available parameter on the server (for example, the first parameter in Params is assigned to the first parameter used by the SQL command, and so on).

Whenever possible, ParamBindMode should be pbByName. This guarantees that parameters are matched to the correct parameters used by the SQL command regardless of physical ordering in Params. At design time, the names of known parameters appear in the Parameters editor.

Syntax

```
property ParamBindMode: TADParamBindMode;
```

See Also

Params

1.17.1.1.23.2.4 TADRdbmsDataSet.RowsAffected Property

Returns the number of rows operated upon by the latest query execution.

Description

Inspect RowsAffected to determine how many rows were inserted, updated, deleted or fetched by the last dataset operation. If no rows were processed, RowsAffected = 0. If numbers of processed rows is not accessible, RowsAffected = –1.

At MS SQL Server RowsAffected may be unexpectedly equal to -1, if stored procedure or table trigger omits SET NOCOUNT ON. Also, this may be a reason of "Command inserted / updated / deleted [0] records instead of [1] record" error at posting updates to database. Check MS BOL for more details about SET NOCOUNT ON.

Syntax

```
property RowsAffected: TADCounter;
```

Example

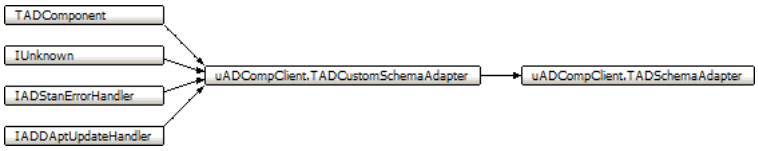
```
procedure TForm1.ADQuery1AfterExecute(ASender: TObject);
begin
  if ADQuery1.RowsAffected = -1 then
    StatusBar1.SimpleText := 'Ok'
  else
    case ADQuery1.Command1.CommandKind of
    skDelete: StatusBar1.SimpleText := Format('%d rows deleted', [ADQuery1.RowsAffected]);
    skInsert: StatusBar1.SimpleText := Format('%d rows inserted', [ADQuery1.RowsAffected]);
    skUpdate: StatusBar1.SimpleText := Format('%d rows updated', [ADQuery1.RowsAffected]);
    else      StatusBar1.SimpleText := Format('%d rows affected', [ADQuery1.RowsAffected]);
```

```
    end;
end;
```

1.17.1.1.24 TADSchemaAdapter Class

The class implementing Centralized Cached Updates (see page 114) support.

Class Hierarchy



File

uADCompClient

Description

Use TADCustomSchemaAdapter to manage Centralized Cached Updates (see page 114).

Syntax

```
TADSchemaAdapter = class(TADCustomSchemaAdapter);
```

See Also

Caching Updates (see page 114), Master-Detail Relationship (see page 104), TADCustomSchemaAdapter (see page 400)

Example

See demos:

- FireDAC\Samples\Comp Layer\TADQuery (see page 476)\CachedUpdates\Centralized

- FireDAC\Samples\Comp Layer\TADSchemaAdapter\MemTable

1.17.1.1.25 TADStoredProc Class

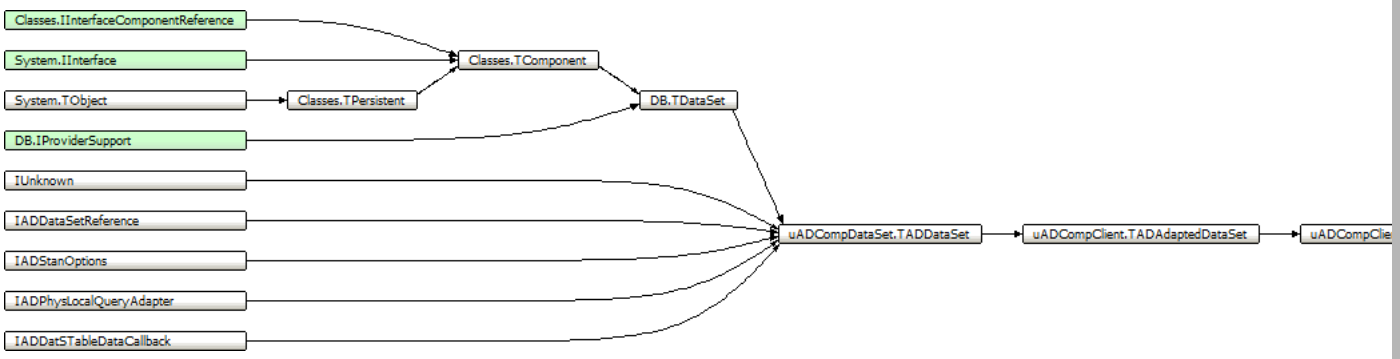
The class implementing dataset, capable to execute server side stored procedures.

published

published	Description
ActiveStoredUsage (see page 516)	Controls how to use Active property value saved to DFM.
AfterApplyUpdates (see page 516)	Fires after cached updates are applied to DB.
AfterExecute (see page 517)	Fires after SQL command execution is finished.
AfterGetRecord (see page 517)	Fires after fetching next rowset one time for each fetched record.
AfterGetRecords (see page 517)	Fires after fetching next rowset.
AfterRowRequest (see page 517)	Fires after fetching record.
Aggregates (see page 517)	The collection of client side aggregates, defined for dataset.
AggregatesActive (see page 518)	Controls automatic calculation of aggregate values.
BeforeApplyUpdates (see page 519)	Fires before cached updates are applied to DB.
BeforeExecute (see page 519)	Fires before SQL command execution is started.
BeforeGetRecords (see page 519)	Fires before fetching next rowset.
BeforeRowRequest (see page 520)	Fires before fetching record.
CachedUpdates (see page 520)	Specifies whether dataset will log changes to the data without immediate applying of them to the database.
CatalogName (see page 521)	Limits the stored procedure names to the specified catalog.
Connection (see page 521)	Specifies the FireDAC connection component to use.

ConnectionName (see page 521)	Specifies the FireDAC connection to use by its name.
Constraints (see page 522)	Specifies record-level client constraints that must be met when editing the data.
ConstraintsEnabled (see page 523)	Specifies whether the dataset perform constraint checking.
DetailFields (see page 523)	Gets / sets fields in detail dataset, used to establish a master-detail relationship with this dataset.
FetchOptions (see page 523)	The set of options to control rows fetching.
FilterChanges (see page 524)	Specifies what kinds of changed records must be "visible" in the dataset.
FormatOptions (see page 524)	The set of options to control data representation.
Indexes (see page 524)	Lists all client indexes that apply to the dataset.
IndexesActive (see page 526)	Controls automatic maintenance of Index'es.
IndexFieldNames (see page 526)	Lists the field names to use as an index.
IndexName (see page 527)	Gets / sets the current index for the dataset by its name.
LocalSQL (see page 527)	The reference to a Local SQL engine.
MasterFields (see page 527)	Gets / sets fields in master dataset, used to establish a master-detail relationship with this dataset.
MasterSource (see page 528)	Gets / sets a master data source, used to establish a master-detail relationship with this dataset.
OnCommandChanged (see page 528)	Fires after the SQL property value is changed.
OnError (see page 529)	The event fires when an error happens, while dataset is communicating with DBMS.
OnExecuteError (see page 530)	The event is triggered when an error happens, while executing a Array DML query.
OnReconcileError (see page 530)	Fires when a dataset needs to reconcile an update to a record that cannot not be applied.
OnUpdateError (see page 531)	Fires if an exception is generated when updates are applied to a database.
OnUpdateRecord (see page 532)	Occurs when record update is applying to a database.
Overload (see page 533)	Specifies which Oracle overloaded packaged stored procedure to execute.
PackageName (see page 534)	Limits the stored procedure names to the specified package name.
ParamBindMode (see page 534)	Determines the order in which a component’s parameters are assigned to the parameter list for SQL command on the DBMS.
Params (see page 534)	Contains the parameters for a SQL statement.
ResourceOptions (see page 535)	The set of options to control resources usage.
SchemaAdapter (see page 536)	Gets or sets a reference to a schema adapter, as a Centralized Cached Updates log.
SchemaName (see page 536)	Limits the stored procedure names to the specified schema.
StoredProcName (see page 536)	Identifies the name of the stored procedure on the server to call.
Transaction (see page 537)	Gets / sets reference to transaction object.
UpdateObject (see page 537)	Specifies the update object component used to post updates by dataset.
UpdateOptions (see page 538)	The set of options to control updates posting.
UpdateTransaction (see page 538)	Gets / sets reference to transaction object.

Class Hierarchy



File

uADCompClient

Description

Use TADStoredProc to execute server side stored procedures, browse the result sets and edit the result set records.

Syntax

```
TADStoredProc = class(TADCustomStoredProc);
```

See Also

Executing Stored Procedure (see page 71), TADCustomStoredProc (see page 403), TADQuery (see page 476)

1.17.1.1.25.1 published

1.17.1.1.25.1.1 TADStoredProc.ActiveStoredUsage Property

Controls how to use Active property value saved to DFM.

Description

The ActiveStoredUsage property controls how to use Active property value saved to DFM. Include:

- auDesignTime, to use property value at design time;
- auRunTime, to use property value at run time.

Syntax

```
property ActiveStoredUsage: TADStoredActivationUsage;
```

See Also

TDataSet.Active

1.17.1.1.25.1.2 TADStoredProc.AfterApplyUpdates Event

Fires after cached updates are applied to DB.

Description

The AfterApplyUpdates event fires after cached updates are applied to DB as result of ApplyUpdates (see page 611) method call.

Syntax

```
property AfterApplyUpdates: TADAfterApplyUpdatesEvent;
```

See Also

TADAfterApplyUpdatesEvent, ApplyUpdates, BeforeApplyUpdates

1.17.1.1.25.1.3 TADStoredProc.AfterExecute Event

Fires after SQL command execution is finished.

Description

The AfterExecute event fires after SQL command execution is finished as result of Execute (see page 622) method call.

Syntax

```
property AfterExecute: TADDataSetEvent;
```

See Also

TADDataSetEvent, Execute, BeforeExecute

1.17.1.1.25.1.4 TADStoredProc.AfterGetRecord Event

Fires after fetching next rowset one time for each fetched record.

Description

The AfterGetRecord event fires after dataset fetched next rowset as result of navigation methods calls. The event fires one time for each fetched record. For example, fetching occurs automatically inside of Next, Last, Locate methods. Inside of an event handler application may read dataset fields to get fetched record field values.

Syntax

```
property AfterGetRecord: TADDataSetEvent;
```

See Also

AfterGetRecords

1.17.1.1.25.1.5 TADStoredProc.AfterGetRecords Event

Fires after fetching next rowset.

Description

The AfterGetRecords event fires after dataset fetched next rowset as result of navigation methods calls. For example, fetching occurs automatically inside of Next, Last, Locate methods.

Syntax

```
property AfterGetRecords: TADDataSetEvent;
```

See Also

TADDataSetEvent, BeforeGetRecords, AfterGetRecord

1.17.1.1.25.1.6 TADStoredProc.AfterRowRequest Event

Fires after fetching record.

Description

The AfterRowRequest event fires after dataset fetched current record as result of RefreshRecord (see page 642), FetchBlobs (see page 624) or FetchDetails (see page 625) method calls.

Syntax

```
property AfterRowRequest: TADDataSetEvent;
```

See Also

TADDataSetEvent, RefreshRecord, FetchBlobs, FetchDetails, BeforeRowRequest

1.17.1.1.25.1.7 TADStoredProc.Aggregates Property

The collection of client side aggregates, defined for dataset.

Description

Use Aggregates to define client side aggregating formulas, that will be automatically maintained and calculated for group of or for all records of dataset.

All records in the group of records has the same field values for defined set of fields. Aggregates performing calculation for group of records must be associated with one of the indexes. These aggregate expressions will be calculated only is associated index is current and active.

Adding aggregate field does not add TADAggregate (see page 572) object to Aggregates collection. That are two alternative ways - to use aggregate fields or to use Aggregates.

DataSet will automatically maintain and calculate Aggregates values when dataset is fetching the data or application edits the data, if AggregatesActive (see page 590) is True. If application needs to perform large updates to dataset and aggregate values are not needed while updating, then set AggregatesActive (see page 590) to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

When aggregates are maintained, the Value method of every active aggregate object returns a value that reflects the current data in the dataset. When users edit the data in the dataset, these values are recalculated to reflect the user's changes.

The expressions in Aggregates must contain aggregating functions, like a SUM, COUNT. FireDAC supports expression syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write constraint expressions.

Syntax

```
property Aggregates: TADAggregates;
```

See Also

Calculated and Aggregated Fields (see page 108), AggregatesActive, BeginBatch, EndBatch

Example 1

```
with ADMemTable1.Aggregates.Add do begin
  Expression := 'sum(sal + bonus)';
  Active := True;
end;
with ADMemTable1.Aggregates.Add do begin
  Expression := 'sum(sal + bonus)';
  IndexName := 'by_deps';
  Active := True;
end;
ADMemTable1.IndexName := 'by_deps';
ADMemTable1.AggregatesActive := True;

Label1.Caption := 'Total payments : ' + VarToStr(ADMemTable1.Aggregates[0].Value);
Label2.Caption := 'Current department payments : ' +
VarToStr(ADMemTable1.Aggregates[1].Value);
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\Aggregates sample for details.

1.17.1.1.25.1.8 TADStoredProc.AggregatesActive Property

Controls automatic calculation of aggregate values.

Description

Use AggregatesActive to get / set the flag controlling automatic maintenance and calculation of dataset aggregate values.

When AggregatesActive is False (the default), the dataset does not maintain aggregates. This allows the dataset to perform large updates to the dataset or fetch big volumes of data without overhead of calculating aggregate values, which happens

at fetching or data editing. When AggregatesActive is True, the dataset calculates and maintains all aggregate values specified by the Aggregates (see page 590) property that are compatible with the current index.

To selectively enable and disable aggregates rather than turning them all on or off at once, use the Active (see page 575) property of individual TADAggregate (see page 572) objects. These objects are available through the Aggregates (see page 590) property.

If application needs to perform batch updates to dataset, then set AggregatesActive to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property AggregatesActive: Boolean;
```

See Also

Calculated and Aggregated Fields (see page 108),Aggregates, TADAggregate.Active (see page 575), BeginBatch, EndBatch

Example

```
var
  lPrevAggsActive: Boolean;
...
lPrevAggsActive := ADQuery1.AggregatesActive;
ADQuery1.AggregatesActive := False;
try
  // perform updates here, without calculating and reading client aggregates
finally
  ADQuery1.AggregatesActive := lPrevAggsActive;
end;
```

1.17.1.1.25.1.9 TADStoredProc.BeforeApplyUpdates Event

Fires before cached updates are applied to DB.

Description

The BeforeApplyUpdates event fires before cached updates are applied to database as result of ApplyUpdates (see page 611) method call.

Syntax

```
property BeforeApplyUpdates: TADDataSetEvent;
```

See Also

TADDataSetEvent, ApplyUpdates, AfterApplyUpdates

1.17.1.1.25.1.10 TADStoredProc.BeforeExecute Event

Fires before SQL command execution is started.

Description

The BeforeExecute event fires before SQL command execution is started as result of Execute (see page 622) method call.

Syntax

```
property BeforeExecute: TADDataSetEvent;
```

See Also

TADDataSetEvent, Execute, AfterExecute

1.17.1.1.25.1.11 TADStoredProc.BeforeGetRecords Event

Fires before fetching next rowset.

Description

The BeforeGetRecords event fires before dataset will fetch next rowset as result of navigation methods calls. For example,

fetching occurs automatically inside of Next, Last, Locate methods.

Syntax

```
property BeforeGetRecords: TADDataSetEvent;
```

See Also

TADDataSetEvent, AfterGetRecords

1.17.1.1.25.1.12 TADStoredProc.BeforeRowRequest Event

Fires before fetching record.

Description

The BeforeRowRequest event fires before dataset will fetch current record as result of RefreshRecord (see page 642), FetchBlobs (see page 624) or FetchDetails (see page 625) method calls.

Syntax

```
property BeforeRowRequest: TADDataSetEvent;
```

See Also

TADDataSetEvent, RefreshRecord, FetchBlobs, FetchDetails, AfterRowRequest

1.17.1.1.25.1.13 TADStoredProc.CachedUpdates Property

Specifies whether dataset will log changes to the data without immediate applying of them to the database.

Description

CachedUpdates enables (True) or disables (False) the logging of data changes (Insert/Post, Edit/Post, Delete) without immediate applying of them to the database.

An application must explicitly apply changes from change log to the database, using ApplyUpdates (see page 611) method. All changes will be written in comparably small amount of time in single transaction. The main benefits of enabling cached updates are:

- fewer transactions and shorter transaction times;
- minimization of network traffic;
- simplified implementation of undo / redo functionality for dataset;
- ability to implement application offline mode or briefcase model.

The potential drawbacks of enabling cached updates are:

- other applications can access and change the actual data on the server while users are editing local copies of the data, resulting in an update conflict when cached updates are applied to the database;
- other applications cannot access data changes made by an application until its cached updates are applied to the database.

Note, to change CachedUpdate property value for TADTable, it must be inactive.

Syntax

```
property CachedUpdates: Boolean;
```

See Also

Caching Updates (see page 114), ApplyUpdates, CancelUpdates, CommitUpdates

Example

```
ADQuery1.CachedUpdates := True;
...
ADQuery1.Edit;
...
ADQuery1.Post;
...
ADQuery1.Append;
```

```
...
ADQuery1.Post;
...
ADQuery1.ApplyUpdates;
ADQuery1.CommitUpdates;
ADQuery1.CachedUpdates := False;
```

1.17.1.1.25.1.14 TADStoredProc.CatalogName Property

Limits the stored procedure names to the specified catalog.

Description

Specify the CatalogName property value to limit the set of stored procedures to the specified catalog.

Syntax

```
property CatalogName: String;
```

See Also

Object Names (see page 133), SchemaName, PackageName, Overload, StoredProcName

1.17.1.1.25.1.15 TADStoredProc.Connection Property

Specifies the FireDAC connection component to use.

Description

Use Connection to specify an FireDAC connection object to use to connect to an DBMS.

At design-time, select from the list of available TADCustomConnection (see page 318) objects (if any have been added to the application) invoked from the Object Inspector. If a form or data module has TADCustomConnection (see page 318) dropped on it, then TADRdbmsDataSet assigns this connection object to Connection property automatically right after dropping on the form or data module.

At runtime, set Connection to reference an existing TADConnection (see page 279) object.

The TADRdbmsDataSet can be bind to TADCustomConnection (see page 318) object using Connection or ConnectionName (see page 503) property. Using ConnectionName (see page 503) property allows to bind to a connection object, if it is even not yet created or is not accessible.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

ConnectionName

1.17.1.1.25.1.16 TADStoredProc.ConnectionName Property

Specifies the FireDAC connection to use by its name.

Description

Use ConnectionName to specify an FireDAC connection to use to connect to an DBMS. The ConnectionName property value specifies the name of the connection. It must match to the:

- name of one of connection definitions, either stored in external file (persistent) or created on fly (private);
- ConnectionName of one of the TADConnection (see page 279) objects.

The ConnectionName property value must be specified before Prepare (see page 511) call. If it matches the name of one of connection definitions, then FireDAC will transparently create connection object and link the dataset with it.

The TADRdbmsDataSet cab be bind to TADCustomConnection (see page 318) object using Connection or ConnectionName property. Using Connection (see page 503) property allows to bind to a connection object explicitly and reduce the overhead for resolving connection names.

Syntax

```
property ConnectionName: String;
```

See Also

Connection, TADCustomConnection.Temporary (see page 337)

Example

```
ADQuery1.ConnectionName := 'Ora_Demo';
ADQuery1.Open('select * from "Customers"');
```

1.17.1.1.25.1.17 TADStoredProc.Constraints Property

Specifies record-level client constraints that must be met when editing the data.

Description

Use Constraints to maintain record-level constraints for the dataset.

Record-level constraints force business ruless limiting few fields in a single record. This constraints are checked at end of data editing (Post / AppendRecord / InsertRecord). Constraints, limiting single field are field constraints and must specified at TField.CustomConstraint or TField.ImportedConstraint.

The Constraints checking is performed when ConstraintsEnabled (see page 594) is True. If application needs to perform large updates to the dataset and application can guarantee the data consistancy, then set ConstraintsEnabled (see page 594) to False before updates, and return to original value after updates. The constraints are not checked at data fetching.

The expressions in Constraints must be predicates, evaluating to Boolean value. FireDAC supports expression syntax compatible with:

- BDE data access components;
- Client dataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write constraint expressions.

Syntax

```
property Constraints;
```

See Also

ConstraintsEnabled, DisableConstraints, EnableConstraints, BeginBatch, EndBatch, TDataSet.Post, TDataSet.AppendRecord, TDataSet.InsertRecord, TField.CustomConstraint, TField.ImportedConstraint

Example

```
with ADMemTable1.Constraints.Add do begin
  CustomConstraint := 'sal + bonus <= 2000';
  ErrorMessage := 'The employee payments must be equal or less than 2000 usd';
end;

ADMemTable1.ConstraintsEnabled := True;
ADMemTable1.Edit;
try
  ADMemTable1.FieldByName('sal').AsFloat := 1800;
  ADMemTable1.FieldByName('bonus').AsFloat := 300;
  // here exception will be raised
  ADMemTable1.Post;
except
  ADMemTable1.Cancel;
  Application.HandleException(Self);
end;

ADMemTable1.ConstraintsEnabled := False;
ADMemTable1.Edit;
ADMemTable1.FieldByName('sal').AsFloat := 1800;
ADMemTable1.FieldByName('bonus').AsFloat := 300;
// here exception will be NOT raised, because constraint checking is disabled
ADMemTable1.Post;
```

1.17.1.1.25.1.18 TADStoredProc.ConstraintsEnabled Property

Specifies whether the dataset perform constraint checking.

Description

Use ConstraintsEnabled to get / set the flag controlling automatic record-level constraints enforcement.

When ConstraintsEnabled is False (the default), the dataset does not check Constraints (see page 594). This allows the dataset to perform the large data updates without overhead of checking constraints. When ConstraintsEnabled is True, the dataset checks Constraints (see page 594) at at end of data editing (Post / AppendRecord / InsertRecord).

If application needs to perform batch updates to dataset, then set ConstraintsEnabled to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property ConstraintsEnabled: Boolean;
```

See Also

Constraints, BeginBatch, EndBatch, DisableConstraints, EnableConstraints, TDataSet.Post, TDataSet.AppendRecord, TDataSet.InsertRecord, TADUpdateOptions.CheckRequired (see page 898)

Example

```
var
  lPrevConsEnabled: Boolean;
...
lPrevConsEnabled := ADQuery1.ConstraintEnabled;
ADQuery1.ConstraintEnabled := False;
try
  // perform updates here, without checking client constraints
finally
  ADQuery1.ConstraintEnabled := lPrevConsEnabled;
end;
```

1.17.1.1.25.1.19 TADStoredProc.DetailFields Property

Gets / sets fields in detail dataset, used to establish a master-detail relationship with this dataset.

Description

Use DetailFields property value to specify semicolon separated list of the detail dataset field names, used to establish a master-detail relationship with this dataset.

The DetailFields property is useful only for parameter based master-detail setup. For range based setup the indexed fields are used as the detail field list. When DetailFields is not specified explicitly, then field names matching to query parameter names will be used as DetailFields value.

The DetailFields will be used only to assign specified field values when a new record is inserted into detail dataset. The mechanism used to filter the detail dataset records does not use DetailFields.

Syntax

```
property DetailFields: String;
```

See Also

Master-Detail Relationship (see page 104), MasterSource, MasterFields, IndexFieldNames, IndexName

Example

```
ADQuery1.SQL.Text := 'SELECT * FROM "Orders" WHERE CustomerID = :ID';
ADQuery1.MasterSource := CustomersDS;
ADQuery1.MasterFields := 'ID';
ADQuery1.DetailFields := 'CustomerID';
```

1.17.1.1.25.1.20 TADStoredProc.FetchOptions Property

The set of options to control rows fetching.

Description

The FetchOptions property is the set of properties, controlling the fetching of cursor rows. These properties will inherit its values from FetchOptions of connection object.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), TADFetchOptions (see page 849)

1.17.1.1.25.1.21 TADStoredProc.FilterChanges Property

Specifies what kinds of changed records must be "visible" in the dataset.

Description

The FilterChanges property allows to get / set changed records kinds, which must be accessible through navigation interface of dataset ("visible" in dataset):

Kind	Meaning
rtModified	Records changed after fetching or last CommitUpdates (see page 615) / CancelUpdates (see page 614).
rtInserted	New records, not yet posted to DB and added after dataset open or after last CommitUpdates (see page 615).
rtDeleted	Deleted records, not yet deleted from DB and deleted after dataset open or after last CommitUpdates (see page 615).
rtUnmodified	Unchanged records.
rtHasErrors	Records having associated errors after ApplyUpdates (see page 611). To get an error object use RowError (see page 607) property.

Syntax

```
property FilterChanges: TADUpdateRecordTypes;
```

See Also

Filtering Records (see page 102), Caching Updates (see page 114), RowError (see page 607), UpdateStatus (see page 650)

Example

```
// see only inserted records
ADQuery1.FilterChanges := [rtInserted];
```

1.17.1.1.25.1.22 TADStoredProc.FormatOptions Property

The set of options to control data representation.

Description

The FormatOptions property is the set of properties, controlling the data type mapping, and some other aspects of data handling.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), TADFormatOptions (see page 860), Params

1.17.1.1.25.1.23 TADStoredProc.Indexes Property

Lists all client indexes that apply to the dataset.

Description

Use Indexes to define client side data views on the dataset records. Indexes are a collection of the TADIndex (see page 653) objects, each of them defines optional:

- Sorting by fields (Fields (see page 657), CaseInsFields (see page 655), DescFields (see page 656), Options (see page 658)).
- Sorting by expression (Expression (see page 656), Options (see page 658)).
- Records filter (Filter (see page 657), FilterOptions (see page 657)).
- Records distinction flag (Distinct (see page 656)).

The individual indexes are not just the sort and filter definitions. FireDAC maintains a consistent view on the data for each active index, while application fetches data or edits data. That creates additional overhead, but allows to switch between indexes without delays.

To make a view current, set it Selected (see page 659) property to True or dataset IndexName (see page 601) property to the view name. Note, the view must have Active = True to be maintained.

Dataset maintains Indexes views, when IndexesActive (see page 600) is True. When an application needs to perform batch updates to dataset, then set IndexesActive (see page 600) to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Setting IndexFieldNames (see page 600) does not add a TADIndex (see page 653) object to Indexes collection. Defining IndexDefs (see page 598) before dataset opening, will clear Indexes and fill them using IndexDefs (see page 598) information. So, there are two alternative ways - use IndexDefs (see page 598) or use Indexes. We suggest to use Indexes, as it is more flexible.

Some of the navigation methods, like a Locate (see page 634) / Lookup (see page 637) / SetRange (see page 647) are using indexes to optimize their operations.

The sorting and filter expressions in Indexes supports syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

Syntax

```
property Indexes: TADIndexes;
```

See Also

Sorting Records (see page 100), Writing Expressions (see page 110), IndexDefs, IndexesActive, IndexFieldNames, IndexName, BeginBatch, EndBatch

Example 1

```
with ADMemTable1.Indexes.Add do begin
  Name := 'by_name';
  Fields := 'NAME;DT';
  Active := True;
end;
with ADMemTable1.Indexes.Add do begin
  Name := 'by_payments';
  Expression := 'sum(sal + bonus)';
  Filter := 'dep_id is not null;
  Active := True;
end;
ADMemTable1.IndexesActive := True;
ADMemTable1.IndexName := 'by_name';
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\Indices demo for details.

1.17.1.1.25.1.24 TADStoredProc.IndexesActive Property

Controls automatic maintenance of Index'es.

Description

Use IndexesActive to indicate whether the dataset should maintain the data views, defined by the Indexes (see page 598) collection.

When IndexesActive is False, the dataset does not maintain the data views. That avoids the overhead of updating views, while data is fetched or is edited. When IndexesActive is True (by default), the dataset maintains the data views.

When indexes are maintained, each of them may be current index, selected by TADIndex (see page 653).Selected (see page 659) or by IndexName (see page 601) properties.

To selectively enable and disable data views rather than turning them all on or off at once, use the Active (see page 655) property of individual TADIndex (see page 653) objects. These objects are available through the Indexes (see page 598) property.

If application needs to perform batch updates to dataset, then set IndexesActive to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property IndexesActive: Boolean;
```

See Also

Sorting Records (see page 100), Indexes, BeginBatch, EndBatch

Example

```
var
  lPrevIndsActive: Boolean;
...
lPrevIndsActive := ADQuery1.IndexesActive;
ADQuery1.IndexesActive := False;
try
  // perform updates here, without maintaining client views
finally
  ADQuery1.IndexesActive := lPrevIndsActive;
end;
```

1.17.1.1.25.1.25 TADStoredProc.IndexFieldNames Property

Lists the field names to use as an index.

Description

Use IndexFieldNames as an alternative method of specifying the current index to use for a dataset.

Specify the name of each field on which to index the dataset, separating names with semicolons. Ordering of field names is significant. Optionally specify the postfix for any of fields in form "field[:[D][A][N]]", where:

- 'D' - use descending sorting on this field;

- 'A' - use ascending sorting on this field;

- 'N' - use case-insensitive sorting on this field.

Use IndexFieldNames to create sort orders on the fly. The IndexFieldNames and IndexName (see page 601) properties are mutually exclusive. Setting one clears the other.

Syntax

```
property IndexFieldNames: String;
```

See Also

Sorting Records (see page 100), Indexes, IndexName

Example

```
ADQuery1.IndexFieldNames := 'order_date;customer_name:N';
```

1.17.1.1.25.1.26 TADStoredProc.IndexName Property

Gets / sets the current index for the dataset by its name.

Description

Use IndexName to specify the current index for the dataset.

If IndexName is empty, the dataset's sort order is based on the IndexFieldNames (see page 600) property or on its default ordering as it is in dataset. If IndexName contains a name of valid index from Indexes collection, then that index is used to determine data view (including sort order) of records.

IndexFieldNames (see page 600) and IndexName are mutually exclusive. Setting one clears the other.

Syntax

```
property IndexName: String;
```

See Also

Sorting Records (see page 100), Indexes, IndexFieldNames

Example

```
with ADMemTable1.Indexes.Add do begin
  Name := 'by_name';
  Fields := 'NAME;DT';
  Active := True;
end;
ADMemTable1.IndexName := 'by_name';
```

1.17.1.1.25.1.27 TADStoredProc.LocalSQL Property

The reference to a Local SQL engine.

Description

Use this property to get / set reference to a Local SQL engine.

When engine and Local SQL connection are active, then right after setting LocalSQL property, the dataset will be opened (if not yet) and will be registered in Local SQL engine. Otherwise dataset will be registered when engine and connection will be active. The TADMemTable (see page 436) structure must be defined before it will be opened.

The dataset will be registered using its Name property value. So, its Name property value must be not empty and unique across all datasets linked to the same Local SQL engine.

Setting this property to nil unregisters this dataset from Local SQL engine.

Syntax

```
property LocalSQL: TADCustomLocalSQL;
```

See Also

Local SQL (see page 96), TADLocalSQL (see page 825), TADMemTable (see page 436)

1.17.1.1.25.1.28 TADStoredProc.MasterFields Property

Gets / sets fields in master dataset, used to establish a master-detail relationship with this dataset.

Description

Use MasterFields property value to specify semicolon separated list of the master dataset field names, used to establish a master-detail relationship with this dataset.

Specify MasterFields property value after assigning MasterSource (see page 603) property value. The mechanism used to filter this detail dataset records depends on the dataset class:

- TADQuery (see page 476) and TADStoredProc (see page 514) will match master fields to this query / procedure parameters by their names. When the master dataset current record is changed, then this dataset parameters will get values from the corresponding MasterFields.
- TADMemTable (see page 436) will match master fields to this dataset indexed fields by their positions. When the master dataset current record is changed, then a range will be applied to this dataset, where starting and ending range values are equal to the corresponding MasterFields field values. The application must have a current active index. The indexed fields will be the detail dataset fields used to establish mater-detail.

Syntax

```
property MasterFields: String;
```

See Also

Master-Detail Relationship (see page 104), MasterSource, DetailFields, IndexFieldNames, IndexName

Example

```
ADMemTable1.IndexFieldNames := 'CustomerID';
ADMemTable1.MasterSource := CustomersDS;
ADMemTable1.MasterFields := 'ID';
```

1.17.1.1.25.1.29 TADStoredProc.MasterSource Property

Gets / sets a master data source, used to establish a master-detail relationship with this dataset.

Description

Use MasterSource to specify the data source linked to a dataset, which will be used as master dataset in master-detail relationship between this and specified datasets.

Additionally application must specify MasterFields (see page 602) property value. See MasterFields (see page 602) property description for additional information.

Syntax

```
property MasterSource: TDataSource;
```

See Also

Master-Detail Relationship (see page 104), MasterFields, IndexFieldNames, IndexName

Example 1

```
ADMemTable1.IndexFieldNames := 'CustomerID';
ADMemTable1.MasterSource := CustomersDS;
ADMemTable1.MasterFields := 'ID';
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\MasterDetail demo for details.

1.17.1.1.25.1.30 TADStoredProc.OnCommandChanged Event

Fires after the SQL property value is changed.

Description

The OnCommandChanged event fires after the SQL command text value is changed:

- for TADQuery (see page 476) - after SQL (see page 396) is changed;
- for TADStoredProc (see page 514) - after StoredProcName (see page 406) is changed;
- for TADTable (see page 539) - after TableName (see page 559) is changed.

Syntax

```
property OnCommandChanged: TNotifyEvent;
```

See Also

TADCustomQuery.SQL (see page 396), TADCustomStoredProc.StoredProcName (see page 406), TADTable.TableName (see page 559)

1.17.1.1.25.1.31 TADStoredProc.OnError Event

The event fires when an error happens, while dataset is communicating with DBMS.

Parameters

Parameters	Description
ASender	The reference to dataset.
AInitiator	The reference to interface of object, which initially raised the exception.
AException	The exception object, which may be modified or even replaced by another exception object instance.

Description

The OnError event fires, when dataset executes one of the following operations and an error happens:

- Prepare (see page 511). The dataset is preparing DBMS command text for execution.
- Open (see page 509). The dataset is executing DBMS command to open cursor.
- Execute (see page 622), ExecSQL (see page 397), ExecProc (see page 410). The dataset is executing DBMS command text.
- Navigation methods. The dataset is fetching rows from cursor.

If most cases AException object will of EADDBEngineException (see page 833) class. If AException is of EADDBArrayExecuteError (see page 831) class, then it is Array DML (see page 82) error handling case. You should use a code like in the first example.

To centralize error handling, you can consider to use TADCustomConnection (see page 318).OnError (see page 332).

Syntax

```
property OnError: TADErrorEvent;
```

See Also

TADCustomQuery.ExecSQL (see page 397), TADCustomStoredProc.ExecProc (see page 410), TDataSet.Open, Prepare, EADDBArrayExecuteError (see page 831), EADDBEngineException (see page 833), TADCustomConnection.OnError (see page 332)

Example 1

Handling Array DML errors:

```
procedure TForm1.ADQuery1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if AException is EADPhysArrayExecuteError then
    with EADPhysArrayExecuteError(AException) do
      if Errors[0].Kind = ekUKViolated then
        AAction := eaSkip
      else
        AAction := eaFail
  else
    ...
end;
```

Example 2

Error substitution:

```
procedure TForm1.ADQuery1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if EADDBEngineException(AException).Errors[0].Kind = ekPKViolated then begin
    AException.Free;
    AException := Exception.Create('Please, enter unique ID value');
  end;
end;
```

1.17.1.1.25.1.32 TADStoredProc.OnExecuteError Event

The event is triggered when an error happens, while executing a Array DML query.

Parameters

Parameters	Description
ASender	The reference to dataset.
ATimes	The current Array DML size. It may be different from value of the Execute method ATimes parameter, if full operation is splitted into few more small (on Firebird, for example).
AOffset	The current Array DML offset. It may be different from value of the Execute method AOffset parameter, if there was skipped erroneous rows or full operation is splitted into few more small (on Firebird, for example).
AError	The original DBMS returned error. Check AError.Errors (see page 834)[i].RowIndex (see page 839) for the index of erroneous row in parameters array.
AAction	The action dataset should take after return from event handler.

Description

The OnExecuteError event is triggered, when the dataset failed to execute Array DML using Execute method and ATimes > 1. Using this event handler you can:

- skip erroneous row of the parameters array;
- change values in erroneous row of the parameters array;
- stop full Array DML execution.

Note, OnExecuteError will be not called for the syntax errors or when ATimes = 1.

Syntax

```
property OnExecuteError: TADExecuteErrorEvent;
```

See Also

Array DML (see page 82), EADDBEngineException (see page 833), TADErrorAction, TADDataSet.Execute (see page 622)

Example

```
procedure TForm1.ADQuery1ExecuteError(ASender: TObject; ATimes,
  AOffset: Integer; AError: EADDBEngineException; var AAction: TADErrorAction);
begin
  if AException.Errors[0].Kind = ekUKViolated then
    AAction := eaSkip
  else
    AAction := eaFail;
end;
```

1.17.1.1.25.1.33 TADStoredProc.OnReconcileError Event

Fires when a dataset needs to reconcile an update to a record that cannot not be applied.

Description

Use the OnReconcileError event handler to respond to error conditions that arise when the ApplyUpdates (see page 611) method was applying changes to the records to the database. The OnReconcileError event handler is called by Reconcile (see page 642) method.

As part of Reconcile call the dataset will loop through all records with associated exception objects. And for each such record will call the OnReconcileError event handler. The event handler gets:

- E - reference to the exception object. See Handling Errors (see page 44) topics for how to work with exception objects.
- UpdateKind - the update kind of the current record. It may be one of the values: rsInserted, rsDeleted, rsModified,

rsUnchanged.

After handling error the event handler should set Action argument. The default value is raMerge. The possible values are:

Action	Description
raSkip	Just skip current record.
raAbort	Just quit from Reconcile call.
raMerge	Clear the current record error state, make current record changes the new initial state of this record. IOW, merge changes into dataset records cache.
raCorrect	Clear current record error state. IOW, mark the record as correctly applied.
raCancel	Cancel current record changes.
raRefresh	Cancel current record changes and reread the record values from the database.

The event handler may analyze the original and current field values, by reading TField.OldValue and NewValue properties. Application may also update the current field value, set Action to raCorrect, and later call ApplyUdpates again.

Syntax

```
property OnReconcileError: TADReconcileErrorEvent;
```

See Also

Caching Updates (see page 114), Reconcile, ApplyUpdates, OnUpdateError

Example

```
procedure TForm1.ADMemTable1ReconcileError(DataSet: TADDataSet; E: EADException;
  UpdateKind: TADDatSRowState; var Action: TADDAptReconcileAction);
begin
  if (UpdateKind = rsInserted) and (E is EADDBEngineException) and
(EADDBEngineException(E).Kind = ekUKViolated) then begin
    DataSet.FieldByName('ID').AsInteger := GetNextFreeID;
    Action := raCorrect;
  end;
end;
```

1.17.1.1.25.1.34 TADStoredProc.OnUpdateError Event

Fires if an exception is generated when updates are applied to a database.

Description

Use the OnUpdateError event handler to respond to exceptions raised while applying immediate or cached updates to a database.

ASender is the dataset to which updates are applied.

AException is a pointer to a EADException (see page 836) object from which an application can extract an error message and the actual cause of the error condition. The OnUpdateError handler can use this information to determine how to respond to the error condition. In most cases AException object will be of class EADDBEngineException (see page 833) or even one of it DBMS specific subclasses.

ARow is a DatS row object, repesenting erroneous record in dataset. This record is also current record in ASender.

ARequest indicates whether the error occurred while inserting, deleting, or modifying a record.

AAction indicates the action to take when the OnUpdateError handler exits. On entry into the handler, AAction is always set to eaDefault, which leads to uaFail, if not changed. If OnUpdateError can handle or correct the error, set AAction to uaRetry before exiting the error handler. Or consider other options.

The error handler can use the OldValue and NewValue properties TField to evaluate error conditions and set NewValue to a new value to reapply. In this case, set AAction to uaRetry before exiting.

The code in an OnUpdateError handler must not call any methods that make a different record the current one !

Syntax

```
property OnUpdateError: TADUpdateErrorEvent;
```

See Also

Caching Updates (see page 114), ApplyUpdates, CachedUpdates, Post, Delete, EADDBEngineException, OnUpdateRecord

Example

```
procedure TForm1.ADQuery1UpdateError(ASender: TDataSet; AException: EADException;
    ARow: TADDatSRow; ARequest: TADUpdateRequest; var AAction: TADErrorAction);
begin
  if (E is EADDBEngineException) and (EADDBEngineException(E).Kind = ekUKViolated) then
begin
    DataSet.FieldByName('ID').AsInteger := GetNextFreeID;
    Action := eaRetry;
  end;
end;
```

1.17.1.1.25.1.35 TADStoredProc.OnUpdateRecord Event

Occurs when record update is applying to a database.

Description

Use an OnUpdateRecord event handler to process immediate or cached updates that cannot be correctly or easily handled:

- using FireDAC automatic SQL update command generation (see page 121);
- by a single table adapter component;
- by a single update component.

This event is useful for applications that require:

- additional control over parameters and macros substitution in update components;
- multi SQL statement updates, when DBMS does not support batches or blocks;
- non-SQL updates posting;
- etc

ASender is the dataset to which updates are applied.

ARequest indicates whether the current update is the insertion, deletion or modification of a record.

AAction indicates the action taken by the OnUpdateRecord handler before it exits. On entry into the handler, AAction is always set to eaDefault. If OnUpdateRecord is successful, it should set AAction to eaApplied before exiting.

AAction value	Description
eaFail	Mark the update as failed and return an error.
eaSkip	Skip current update and do not mark it as applied.
eaRetry	Retry the current operation.
eaApplied	Mark current update as applied.
eaDefault	Take default action. For successful update it is eaApplied, for failed update it is eaFail.
eaExitSuccess	Stop to process the cached updates, return success.
eaExitFailure	Stop to process the cached updates, return failure.

The OnUpdateRecord event handler code should read the dataset field values, including TField.NewValue, OldValue and CurValue. There:

- NewValue is the new field value before posting updates.
- OldValue is the original field value, as it was after record fetching or after last updates applying.
- CurValue is the current field value, the same as Value property.

Note, the OnUpdateRecord handler code must not call the methods that change the current dataset position.

Syntax

```
property OnUpdateRecord: TADUpdateRecordEvent;
```

See Also

Overriding Posting Updates (see page 123), ApplyUpdates, CachedUpdates, Post, Delete, OnUpdateError

Example 1

```
procedure TForm1.ADQuery1UpdateRecord(ASender: TDataSet; ARequest: TADUpdateRequest;
 var AAction: TADErrorAction; AOptions: TADUpdateRowOptions);
begin
  if ARequest = arInsert then begin
    // set the SQL command to insert new record
    ADQuery2.SQL := 'insert into mytab (id, code, name) values (:id, :code, :name)
returning tmstamp into :ts';

    // set parameter values
    ADQuery2.Params[0].Value := ASender['id'];
    ADQuery2.Params[1].Value := ASender['code'];
    ADQuery2.Params[2].Value := ASender['name'];

    // specially define TS parameter
    ADQuery2.Params[3].DataType := ftDateTime;
    ADQuery2.Params[3].ParamType := ptOutput;

    // insert new record
    ADQuery2.ExecSQL;

    // move TS output parameter value back to dataset
    ASender['tmstamp'] := ADQuery2.Params[3].Value;

    // return 'Ok' status
    AAction := eaApplied;
  end;
end;
```

Example 2

See demos:

- FireDAC\Samples\Comp Layer\TADQuery\CachedUpdates\OnUpdateRecord
- FireDAC\Samples\Comp Layer\TADUpdateSQL\Main

1.17.1.1.25.1.36 TADStoredProc.Overload Property

Specifies which Oracle overloaded packaged stored procedure to execute.

Description

Use Overload to specify which overloaded stored procedure to execute on an Oracle server. An Oracle overloaded stored procedure is one that shares a name with one or more other stored procedures. Oracle distinguishes among overloaded stored procedures by assigning each procedure a unique numeric identifier. An application can specify this identifier using the Overload property.

By default, Overload is zero, which assumes there is no procedure overloading. For all servers except Oracle, this value will not have the effect. If an application attempts to access an Oracle overloaded procedure without setting Overload, the FireDAC accesses the lowest numbered, or first, overloaded procedure on the server.

If Overload is 1, the FireDAC executes the first overloaded procedure on the server. If Overload is 2, it executes the second, and so on.

While Oracle overloaded procedures share names, their parameter lists are unique. An application must ensure that it passes the correct parameter list to an overloaded procedure.

Syntax

```
property Overload: Word;
```

See Also

Executing Stored Procedure (see page 71), Params (see page 651), StoredProcName

1.17.1.1.25.1.37 TADStoredProc.PackageName Property

Limits the stored procedure names to the specified package name.

Description

Specify the PackageName property value to limit the procedure name to the specified package.

Syntax

```
property PackageName: String;
```

See Also

Object Names (see page 133), CatalogName, SchemaName, Overload, StoredProcName

1.17.1.1.25.1.38 TADStoredProc.ParamBindMode Property

Determines the order in which a component's parameters are assigned to the parameter list for SQL command on the DBMS.

Description

Get or set ParamBindMode to determine the order in which parameters in the Params property are matched to the parameters used by the SQL command on the server. ParamBindMode can be one of the following:

Value	Ordering
pbByName	Parameters specified in the Params property are matched to identically named parameters on the server. This is the default.
pbByNumber	Parameters in Params are assigned one-by-one to the next available parameter on the server (for example, the first parameter in Params is assigned to the first parameter used by the SQL command, and so on).

Whenever possible, ParamBindMode should be pbByName. This guarantees that parameters are matched to the correct parameters used by the SQL command regardless of physical ordering in Params. At design time, the names of known parameters appear in the Parameters editor.

Syntax

```
property ParamBindMode: TADParamBindMode;
```

See Also

Params

1.17.1.1.25.1.39 TADStoredProc.Params Property

Contains the parameters for a SQL statement.

Description

Use Params property to view and set SQL statement parameter names, values, and data types at run time. Use the collection editor for the Params property to set parameter information at design time.

If dataset is TADQuery (see page 476), then Params is populated automatically after changing SQL (see page 396) property value, if ResourceOptions (see page 652).ParamCreate (see page 883) = True. The parameters order is the same as they appear in the SQL command. When Params.BindMode = pbByName, then each parameter name will appear in Params only once. When Params.BindMode = pbByNumber, then each parameter marker will appear in the Params, including duplicated names.

If dataset is TADStoredProc (see page 514), then Params is populated automatically after calling Prepare (see page 511) method at run time or changing StoredProcName (see page 406) at design time, if fiMeta is in FetchOption (see page 651).Items (see page 855). The parameters order is the same as in the stored procedure header.

Otherwise Params may be populated by hands. At Prepare (see page 511) call, parameters will be bind to prepared SQL

command. After that you cannot change parameter types, otherwise will get an exception.

An easier way to set and retrieve parameter values when the name of each parameter is known is to call ParamByName (see page 641). This method is especially preferred versus accessing parameters by index when they are populated automatically and there is a chance that the SQL property or StoredProcName property will be changed in the code dynamically.

Syntax

```
property Params: TADParams;
```

See Also

Executing Command (see page 66), Executing Stored Procedure (see page 71), TADCustomQuery.SQL (see page 396), TADCustomStoredProc.StoredProcName (see page 406), TADParams, TADResourceOptions.ParamCreate (see page 883), TADFetchOptions.Items (see page 855)

Example 1

```
// TADQuery automatic parameters creation - ResourceOptions.ParamCreate = True
ADQuery1.SQL.Text := 'select * from mytab where id = :id';
ADQuery1.Params[0].AsString := 'qwe';
ADQuery1.Open;
```

Example 2

```
// TADQuery manual parameters creation - ResourceOptions.ParamCreate = False
ADQuery1.ResourceOptions.ParamCreate := False;
ADQuery1.SQL.Text := 'select * from mytab where id = :id';
with ADQuery1.Params.Add do begin
  Name := 'id';
  DataType := ftString;
  Size := 10;
  AsString := 'qwe';
end;
ADQuery1.Open;
```

Example 3

```
// TADStoredProc automatic parameters creation - fiMeta in FetchOptions.Items
ADStoredProc1.StoredProcName := 'my_proc';
ADStoredProc1.Prepare;
ADStoredProc1.Params[0].AsString := 'qwe';
ADStoredProc1.ExecProc;
```

Example 4

```
// TADStoredProc manual parameters creation - fiMeta not in FetchOptions.Items
ADStoredProc1.FetchOptions.Items := ADStoredProc1.FetchOptions.Items - [fiMeta];
ADStoredProc1.StoredProcName := 'my_proc';
with ADStoredProc1.Params.Add do begin
  Name := 'id';
  DataType := ftString;
  Size := 10;
end;
ADStoredProc1.Prepare;
ADStoredProc1.Params[0].AsString := 'qwe';
ADStoredProc1.ExecProc;
```

1.17.1.1.25.1.40 TADStoredProc.ResourceOptions Property

The set of options to control resources usage.

Description

The ResourceOptions property is the set of properties, controlling the resources usage by the dataset. These properties will inherit its values from ResourceOptions of connection object.

Syntax

```
property ResourceOptions: TADBottomResourceOptions;
```

See Also

Setting Options (see page 34), TADBottomResourceOptions (see page 841)

1.17.1.1.25.1.41 TADStoredProc.SchemaAdapter Property

Gets or sets a reference to a schema adapter, as a Centralized Cached Updates log.

Description

Use SchemaAdapter property to set a reference to the TADSchemaAdapter (see page 514) object to enable Centralized Cashed Updates (see page 114) mode. The schema adapter object will be used as a central change log share by few datasets.

To enable propagation of master dataset changes to a detail dataset set FetchOptions.DetailCascade (see page 854) to True for a detail dataset. Also both master and detail datasets must point to the same TADSchemaAdapter (see page 514) object.

Syntax

```
property SchemaAdapter: TADCustomSchemaAdapter;
```

See Also

Caching Updates (see page 114), Master-Detail Relationship (see page 104), TADFetchOptions.DetailCascade (see page 854)

Example

```
dsOrders.DataSet := qOrders;
qOrders.SchemaAdapter := ADSchemaAdapter1;
qOrders.CachedUpdates := True;

dsOrderDetails.DataSet := qOrderDetails;
qOrderDetails.SchemaAdapter := ADSchemaAdapter1;
qOrderDetails.CachedUpdates := True;
qOrderDetails.MasterSource := dsOrders;
qOrderDetails.MasterFields := 'OrderID';
qOrderDetails.IndexFieldNames := 'OrderID';
qOrderDetails.FetchOptions.DetailCascade := True;

qOrders.Open;
qOrderDetails.Open;

qOrders.Append;
....
qOrders.Post;

qOrderDetails.Append;
...
qOrderDetails.Post;

ADSchemaAdapter1.ApplyUpdates;
qOrders.CommitUpdates;
qOrderDetails.CommitUpdates;
```

1.17.1.1.25.1.42 TADStoredProc.SchemaName Property

Limits the stored procedure names to the specified schema.

Description

Specify the SchemaName property value to limit the set of schemas to the specified schema.

Syntax

```
property SchemaName: String;
```

See Also

Object Names (see page 133), CatalogName, PackageName, Overload, StoredProcName

1.17.1.1.25.1.43 TADStoredProc.StoredProcName Property

Identifies the name of the stored procedure on the server to call.

Description

Set StoredProcName to specify the name of the stored in database procedure to call. The full name of the stored procedure will be build from CatalogName (see page 404), SchemaName (see page 405), PackageName (see page 404), Overload (see page 404) property values. Also a full name may be assigned to the StoredProcName (only for non-packaged procedures). If name is case sensitive or contains special characters, then it must be explicitly quoted.

If database does not have stored procedure with StoredProcName name, then when the application attempts to prepare the stored procedure, an exception will be raised.

Syntax

```
property StoredProcName: string;
```

See Also

Object Names (see page 133), Executing Stored Procedure (see page 71), CatalogName, SchemaName, PackageName, Overload

Example 1

```
ADStoredProc1.StoredProcName := 'Northwind.dbo.[my proc]';
// the same as following
ADStoredProc1.CatalogName := 'Northwind';
ADStoredProc1.SchemaName := 'dbo';
ADStoredProc1.StoredProcName := '[my proc]';
```

Example 2

```
ADStoredProc1.PackageName := 'sys.dbms_sql';
ADStoredProc1.StoredProcName := 'execute';
```

1.17.1.1.25.1.44 TADStoredProc.Transaction Property

Gets / sets reference to transaction object.

Description

The Transaction property allows to associate dataset with explicit transaction object. If it is not specified, then dataset will be associated with connection default transaction.

The specified transaction object will be used only for Prepare (see page 511), Open (see page 509), ExecSQL (see page 397), ExecProc (see page 410), Execute (see page 622) methods. For commands, generated to apply changes to database, FireDAC will use UpdateTransaction (see page 507), if it is specified, otherwise - Transaction.

Currently explicit transaction object assignment is supported only for Interbase / Firebird servers.

Syntax

```
property Transaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41), UpdateTransaction, TADCustomConnection.Transaction (see page 337)

1.17.1.1.25.1.45 TADStoredProc.UpdateObject Property

Specifies the update object component used to post updates by dataset.

Description

Use UpdateObject to specify the TADUpdateSQL (see page 563) component to use in an application that must be able to post updates in a non-standard way.

The FireDAC has build in updates command generator, which using multiple information sources is able to generate SQL update commands for most of scenarios. In some cases, such as a query made against multiple tables, which may require multi-statement update, FireDAC cannot generate SQL update commands. In these cases, UpdateObject can be used to specify a TADUpdateSQL (see page 563) component that allows to specify custom SQL commands for each update posting case, like a insert / update / delete.

As alternative to update object usage, you can consider to use OnUpdateRecord (see page 605) event. It gives more

options, but requires manual coding of posting updates. Also, in OnUpdateRecord (see page 605) handler you can use multiple update objects. See demo FireDAC\Samples\Comp Layer\TADUpdateSQL\Main.

Syntax

```
property UpdateObject: TADCustomUpdateObject;
```

See Also

OnUpdateRecord (see page 605), TADUpdateSQL

Example 1

```
ADUpdateSQL1.InsertSQL.Text := 'insert into mytab values (:new_id, :new_name)';
ADQuery1.UpdateObject := ADUpdateSQL1;
ADQuery1.Append;
ADQuery1['id'] := 100;
ADQuery1['name'] := 'qwe';
ADQuery1.Post;
```

Example 2

Demo FireDAC\Samples\Comp Layer\TADUpdateSQL\Main.

1.17.1.1.25.1.46 TADStoredProc.UpdateOptions Property

The set of options to control updates posting.

Description

The UpdateOptions property is the set of properties, controlling the posting updates from the dataset. These properties will inherit its values from UpdateOptions of connection object.

Syntax

```
property UpdateOptions: TADBottomUpdateOptions;
```

See Also

Setting Options (see page 34), TADBottomUpdateOptions (see page 843)

1.17.1.1.25.1.47 TADStoredProc.UpdateTransaction Property

Gets / sets reference to transaction object.

Description

The UpdateTransaction property allows to associate dataset internal update commands with explicit transaction object. If it is not specified, then dataset update commands will be associated with connection default update transaction. If it is not specified, then - with dataset Transaction.

For Prepare (see page 511), Open (see page 509), ExecSQL (see page 397), ExecProc (see page 410), Execute (see page 622) methods will be used Transaction (see page 507) objects.

Currently explicit update transaction object assignment is supported only for Interbase / Firebird servers.

Syntax

```
property UpdateTransaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41),Transaction, TADCustomConnection.UpdateTransaction (see page 338)

Example

```
var
  oSelectTransaction: TADTransaction;
  oUpdateTransaction: TADTransaction;
  oCusts: TADQuery;
.....
  oSelectTransaction := TADTransaction.Create(nil);
  oSelectTransaction.Connection := ADConnection1;
  oSelectTransaction.Options.ReadOnly := True;
```

```
oUpdateTransaction := TADTransaction.Create(nil);
oUpdateTransaction.Connection := ADConnection1;

oCusts := TADQuery.Create(nil);
oCusts.Connection := ADConnection1;
oCusts.Transaction := oSelectTransaction;
oCusts.UpdateTransaction := oUpdateTransaction;
oCusts.Open('select * from Customers'); // uses oSelectTransaction
...
oCusts.Edit; // uses oUpdateTransaction, if needed
...
oCusts.Post; // uses oUpdateTransaction
```

1.17.1.1.26 TADTable Class

The class implementing dataset, working with single database table.

protected

protected	Description
GetCustomWhere (see page 542)	Allows to submit a custom WHERE phrase or additional filtering condition.

public

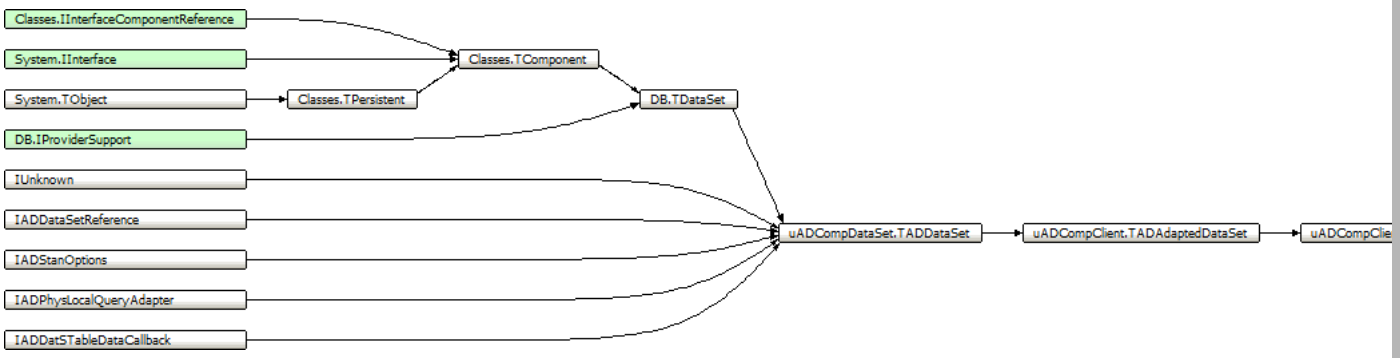
public	Description
Exists (see page 541)	Returns True when the specified table exists.
Disconnect (see page 541)	Disconnects the dataset from DBMS.
Open (see page 541)	Executes the specified SQL statement and opens dataset.
RefireSQL (see page 542)	Allows to rebuild a SELECT statement and reopen the table.

published

published	Description
ActiveStoredUsage (see page 542)	Controls how to use Active property value saved to DFM.
AfterApplyUpdates (see page 543)	Fires after cached updates are applied to DB.
AfterGetRecord (see page 543)	Fires after fetching next rowset one time for each fetched record.
AfterGetRecords (see page 543)	Fires after fetching next rowset.
AfterRowRequest (see page 543)	Fires after fetching record.
Aggregates (see page 544)	The collection of client side aggregates, defined for dataset.
AggregatesActive (see page 544)	Controls automatic calculation of aggregate values.
BeforeApplyUpdates (see page 545)	Fires before cached updates are applied to DB.
BeforeGetRecords (see page 545)	Fires before fetching next rowset.
BeforeRowRequest (see page 546)	Fires before fetching record.
CachedUpdates (see page 546)	Specifies whether dataset will log changes to the data without immediate applying of them to the database.
CatalogName (see page 547)	Limits the table names to the catalog.
Connection (see page 547)	Specifies the FireDAC connection component to use.
ConnectionName (see page 547)	Specifies the FireDAC connection to use by its name.
Constraints (see page 548)	Specifies record-level client constraints that must be met when editing the data.
ConstraintsEnabled (see page 548)	Specifies whether the dataset perform constraint checking.
DetailFields (see page 549)	Gets / sets fields in detail dataset, used to establish a master-detail relationship with this dataset.
FetchOptions (see page 549)	The set of options to control rows fetching.
FilterChanges (see page 550)	Specifies what kinds of changed records must be "visible" in the dataset.
FormatOptions (see page 550)	The set of options to control data representation.
IndexFieldNames (see page 550)	Lists the field names to use as an index.

IndexName (see page 551)	Gets / sets the current index for the dataset by its name.
LocalSQL (see page 551)	The reference to a Local SQL engine.
MasterFields (see page 552)	Gets / sets fields in master dataset, used to establish a master-detail relationship with this dataset.
MasterSource (see page 552)	Gets / sets a master data source, used to establish a master-detail relationship with this dataset.
OnCommandChanged (see page 552)	Fires after the SQL property value is changed.
OnError (see page 553)	The event fires when an error happens, while dataset is communicating with DBMS.
OnExecuteError (see page 554)	The event is triggered when an error happens, while executing a Array DML query.
OnReconcileError (see page 555)	Fires when a dataset needs to reconcile an update to a record that cannot not be applied.
OnUpdateError (see page 555)	Fires if an exception is generated when updates are applied to a database.
OnUpdateRecord (see page 556)	Occurs when record update is applying to a database.
ResourceOptions (see page 557)	The set of options to control resources usage.
SchemaAdapter (see page 558)	Gets or sets a reference to a schema adapter, as a Centralized Cached Updates log.
SchemaName (see page 558)	Limits the table names to the schema.
TableName (see page 559)	Gets / sets table name to open.
Transaction (see page 559)	Gets / sets reference to transaction object.
UpdateObject (see page 559)	Specifies the update object component used to post updates by dataset.
UpdateOptions (see page 560)	The set of options to control updates posting.
UpdateTransaction (see page 560)	Gets / sets reference to transaction object.

Class Hierarchy



File

uADCompClient

Description

Use TADTable to browse database table and edit it records. The TADTable generates a SELECT statement basing on its properties and send it to a DBMS.

Syntax

```
TADTable = class(TADCustomQuery);
```

See Also

Browsing Table (see page 73), TADQuery (see page 476)

1.17.1.1.26.1 public

1.17.1.1.26.1.1 TADTable.Exists Property

Returns True when the specified table exists.

Description

Use the Exists property to verify when a table specified by the CatalogName (see page 547), SchemaName (see page 558), TableName (see page 559) properties is existing.

Syntax

```
property Exists: Boolean;
```

See Also

CatalogName (see page 547), SchemaName (see page 558), TableName (see page 559)

1.17.1.1.26.1.2 TADTable.Disconnect Method

Disconnects the dataset from DBMS.

Parameters

Parameters	Description
AAbortJob: Boolean = False	True, if current dataset task must be aborted.

Description

The Disconnect method optionally aborts current dataset task, like a Open (see page 509), ExecSQL (see page 397), ExecProc (see page 410), Execute (see page 622) or rows fetching. Then closes dataset and unprepares it.

Syntax

```
procedure Disconnect(AAbortJob: Boolean = False); override;
```

See Also

AbortJob

Example

```
ADQuery1.Disconnect(True);
// is equivalent to:
// ADQuery1.AbortJob(True);
// ADQuery1.Close;
// ADQuery1.Unprepare;
```

1.17.1.1.26.1.3 TADTable.Open Method

Executes the specified SQL statement and opens dataset.

Parameters

Parameters	Description
ASQL	A SQL command to execute.

Description

Call Open to execute the SQL statement specified in ASQL or is currently assigned to the internal command object, if ASQL is empty.

The method closes the dataset if it was active, then assigns ASQL to the command text, and sets the Active property to True. When Active is true, the dataset is populated with data from a database.

When an application needs to execute the same SQL command many times, then an application may ordinarily set SQL property, call Prepare before calling Open for the first time, then use any Open version without specifying ASQL.

Use ResourceOptions (see page 506).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 506).CmdExecTimeout (see page 881) to set maximum command execution time. After that time command execution will be canceled and exception raised.

To cancel command execution use TADAdaptedDataSet (see page 259).AbortJob (see page 263).

If DBMS returns error, then FireDAC raises an exception. It may be analyzed in OnError (see page 504) event.

Syntax

```
procedure Open(const ATableName: String); overload;
```

See Also

Prepare, TDataSet.Open, TADCustomQuery.SQL (see page 396), TADCustomStoredProc.StoredProcName (see page 406), TADAdaptedDataSet.AbortJob (see page 263), TADAdaptedDataSet.OnError (see page 261)

Example

```
ADQuery1.Open('select * from customers');
```

1.17.1.1.26.1.4 TADTable.RefireSQL Method

Allows to rebuild a SELECT statement and reopen the table.

Description

Use the RefireSQL method to force FireDAC to rebuild a SELECT statement, which TADTable will send to a DBMS, and to reopen the table. That may be useful, when the GetCustomWhere (see page 542) method value has to be changed.

Syntax

```
procedure RefireSQL;
```

See Also

GetCustomWhere (see page 542)

1.17.1.1.26.2 protected

1.17.1.1.26.2.1 TADTable.GetCustomWhere Method

Allows to submit a custom WHERE phrase or additional filtering condition.

Description

Override the GetCustomWhere method to submit a custom WHERE phrase or additional filtering condition to a SELECT command, which TADTable will send to a DBMS. The returned value must be a SQL predicate.

Syntax

```
function GetCustomWhere: String; virtual;
```

See Also

Filtering Records (see page 102)

1.17.1.1.26.3 published

1.17.1.1.26.3.1 TADTable.ActiveStoredUsage Property

Controls how to use Active property value saved to DFM.

Description

The ActiveStoredUsage property controls how to use Active property value saved to DFM. Include:

- auDesignTime, to use property value at design time;
- auRunTime, to use property value at run time.

Syntax

```
property ActiveStoredUsage: TADStoredActivationUsage;
```

See Also

TDataSet.Active

1.17.1.1.26.3.2 TADTable.AfterApplyUpdates Event

Fires after cached updates are applied to DB.

Description

The AfterApplyUpdates event fires after cached updates are applied to DB as result of ApplyUpdates (see page 611) method call.

Syntax

```
property AfterApplyUpdates: TADAfterApplyUpdatesEvent;
```

See Also

TADAfterApplyUpdatesEvent, ApplyUpdates, BeforeApplyUpdates

1.17.1.1.26.3.3 TADTable.AfterGetRecord Event

Fires after fetching next rowset one time for each fetched record.

Description

The AfterGetRecord event fires after dataset fetched next rowset as result of navigation methods calls. The event fires one time for each fetched record. For example, fetching occurs automatically inside of Next, Last, Locate methods. Inside of an event handler application may read dataset fields to get fetched record field values.

Syntax

```
property AfterGetRecord: TADDataSetEvent;
```

See Also

AfterGetRecords

1.17.1.1.26.3.4 TADTable.AfterGetRecords Event

Fires after fetching next rowset.

Description

The AfterGetRecords event fires after dataset fetched next rowset as result of navigation methods calls. For example, fetching occurs automatically inside of Next, Last, Locate methods.

Syntax

```
property AfterGetRecords: TADDataSetEvent;
```

See Also

TADDataSetEvent, BeforeGetRecords, AfterGetRecord

1.17.1.1.26.3.5 TADTable.AfterRowRequest Event

Fires after fetching record.

Description

The AfterRowRequest event fires after dataset fetched current record as result of RefreshRecord (see page 642), FetchBlobs (see page 624) or FetchDetails (see page 625) method calls.

Syntax

```
property AfterRowRequest: TADDataSetEvent;
```

See Also

TADDataSetEvent, RefreshRecord, FetchBlobs, FetchDetails, BeforeRowRequest

1.17.1.1.26.3.6 TADTable.Aggregates Property

The collection of client side aggregates, defined for dataset.

Description

Use Aggregates to define client side aggregating formulas, that will be automatically maintained and calculated for group of or for all records of dataset.

All records in the group of records has the same field values for defined set of fields. Aggregates performing calculation for group of records must be associated with one of the indexes. These aggregate expressions will be calculated only is associated index is current and active.

Adding aggregate field does not add TADAggregate (see page 572) object to Aggregates collection. That are two alternative ways - to use aggregate fields or to use Aggregates.

DataSet will automatically maintain and calculate Aggregates values when dataset is fetching the data or application edits the data, if AggregatesActive (see page 590) is True. If application needs to perform large updates to dataset and aggregate values are not needed while updating, then set AggregatesActive (see page 590) to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

When aggregates are maintained, the Value method of every active aggregate object returns a value that reflects the current data in the dataset. When users edit the data in the dataset, these values are recalculated to reflect the user's changes.

The expressions in Aggregates must contain aggregating functions, like a SUM, COUNT. FireDAC supports expression syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write constraint expressions.

Syntax

```
property Aggregates: TADAggregates;
```

See Also

Calculated and Aggregated Fields (see page 108), AggregatesActive, BeginBatch, EndBatch

Example 1

```
with ADMemTable1.Aggregates.Add do begin
  Expression := 'sum(sal + bonus)';
  Active := True;
end;
with ADMemTable1.Aggregates.Add do begin
  Expression := 'sum(sal + bonus)';
  IndexName := 'by_deps';
  Active := True;
end;
ADMemTable1.IndexName := 'by_deps';
ADMemTable1.AggregatesActive := True;

Label1.Caption := 'Total payments : ' + VarToStr(ADMemTable1.Aggregates[0].Value);
Label2.Caption := 'Current department payments : ' +
VarToStr(ADMemTable1.Aggregates[1].Value);
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\Aggregates sample for details.

1.17.1.1.26.3.7 TADTable.AggregatesActive Property

Controls automatic calculation of aggregate values.

Description

Use AggregatesActive to get / set the flag controlling automatic maintenance and calculation of dataset aggregate values.

When AggregatesActive is False (the default), the dataset does not maintain aggregates. This allows the dataset to perform large updates to the dataset or fetch big volumes of data without overhead of calculating aggregate values, which happens at fetching or data editing. When AggregatesActive is True, the dataset calculates and maintains all aggregate values specified by the Aggregates (see page 590) property that are compatible with the current index.

To selectively enable and disable aggregates rather than turning them all on or off at once, use the Active (see page 575) property of individual TADAggregate (see page 572) objects. These objects are available through the Aggregates (see page 590) property.

If application needs to perform batch updates to dataset, then set AggregatesActive to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property AggregatesActive: Boolean;
```

See Also

Calculated and Aggregated Fields (see page 108),Aggregates, TADAggregate.Active (see page 575), BeginBatch, EndBatch

Example

```
var
  lPrevAggsActive: Boolean;
...
lPrevAggsActive := ADQuery1.AggregatesActive;
ADQuery1.AggregatesActive := False;
try
  // perform updates here, without calculating and reading client aggregates
finally
  ADQuery1.AggregatesActive := lPrevAggsActive;
end;
```

1.17.1.1.26.3.8 TADTable.BeforeApplyUpdates Event

Fires before cached updates are applied to DB.

Description

The BeforeApplyUpdates event fires before cached updates are applied to database as result of ApplyUpdates (see page 611) method call.

Syntax

```
property BeforeApplyUpdates: TADDataSetEvent;
```

See Also

TADDataSetEvent, ApplyUpdates, AfterApplyUpdates

1.17.1.1.26.3.9 TADTable.BeforeGetRecords Event

Fires before fetching next rowset.

Description

The BeforeGetRecords event fires before dataset will fetch next rowset as result of navigation methods calls. For example, fetching occurs automatically inside of Next, Last, Locate methods.

Syntax

```
property BeforeGetRecords: TADDataSetEvent;
```

See Also

TADDataSetEvent, AfterGetRecords

1.17.1.1.26.3.10 TADTable.BeforeRowRequest Event

Fires before fetching record.

Description

The BeforeRowRequest event fires before dataset will fetch current record as result of RefreshRecord (see page 642), FetchBlobs (see page 624) or FetchDetails (see page 625) method calls.

Syntax

```
property BeforeRowRequest: TADDataSetEvent;
```

See Also

TADDataSetEvent, RefreshRecord, FetchBlobs, FetchDetails, AfterRowRequest

1.17.1.1.26.3.11 TADTable.CachedUpdates Property

Specifies whether dataset will log changes to the data without immediate applying of them to the database.

Description

CachedUpdates enables (True) or disables (False) the logging of data changes (Insert/Post, Edit/Post, Delete) without immediate applying of them to the database.

An application must explicitly apply changes from change log to the database, using ApplyUpdates (see page 611) method. All changes will be written in comparably small amount of time in single transaction. The main benefits of enabling cached updates are:

- fewer transactions and shorter transaction times;
- minimization of network traffic;
- simplified implementation of undo / redo functionality for dataset;
- ability to implement application offline mode or briefcase model.

The potential drawbacks of enabling cached updates are:

- other applications can access and change the actual data on the server while users are editing local copies of the data, resulting in an update conflict when cached updates are applied to the database;
- other applications cannot access data changes made by an application until its cached updates are applied to the database.

Note, to change CachedUpdate property value for TADTable, it must be inactive.

Syntax

```
property CachedUpdates: Boolean;
```

See Also

Caching Updates (see page 114), ApplyUpdates, CancelUpdates, CommitUpdates

Example

```
ADQuery1.CachedUpdates := True;
...
ADQuery1.Edit;
...
ADQuery1.Post;
...
ADQuery1.Append;
...
ADQuery1.Post;
...
ADQuery1.ApplyUpdates;
ADQuery1.CommitUpdates;
ADQuery1.CachedUpdates := False;
```

1.17.1.1.26.3.12 TADTable.CatalogName Property

Limits the table names to the catalog.

Description

Specify the CatalogName property value to limit the set of catalogs to the single value.

Syntax

```
property CatalogName: String;
```

See Also

Object Names (see page 133), SchemaName (see page 558), TableName (see page 559)

1.17.1.1.26.3.13 TADTable.Connection Property

Specifies the FireDAC connection component to use.

Description

Use Connection to specify an FireDAC connection object to use to connect to an DBMS.

At design-time, select from the list of available TADCustomConnection (see page 318) objects (if any have been added to the application) invoked from the Object Inspector. If a form or data module has TADCustomConnection (see page 318) dropped on it, then TADRdbmsDataSet assigns this connection object to Connection property automatically right after dropping on the form or data module.

At runtime, set Connection to reference an existing TADConnection (see page 279) object.

The TADRdbmsDataSet can be bind to TADCustomConnection (see page 318) object using Connection or ConnectionName (see page 503) property. Using ConnectionName (see page 503) property allows to bind to a connection object, if it is even not yet created or is not accessible.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

ConnectionName

1.17.1.1.26.3.14 TADTable.ConnectionName Property

Specifies the FireDAC connection to use by its name.

Description

Use ConnectionName to specify an FireDAC connection to use to connect to an DBMS. The ConnectionName property value specifies the name of the connection. It must match to the:

- name of one of connection definitions, either stored in external file (persistent) or created on fly (private);
- ConnectionName of one of the TADConnection (see page 279) objects.

The ConnectionName property value must be specified before Prepare (see page 511) call. If it matches the name of one of connection definitions, then FireDAC will transparently create connection object and link the dataset with it.

The TADRdbmsDataSet cab be bind to TADCustomConnection (see page 318) object using Connection or ConnectionName property. Using Connection (see page 503) property allows to bind to a connection object explicitly and reduce the overhead for resolving connection names.

Syntax

```
property ConnectionName: String;
```

See Also

Connection, TADCustomConnection.Temporary (see page 337)

Example

```
ADQuery1.ConnectionName := 'Ora_Demo';
ADQuery1.Open('select * from "Customers"');
```

1.17.1.1.26.3.15 TADTable.Constraints Property

Specifies record-level client constraints that must be met when editing the data.

Description

Use Constraints to maintain record-level constraints for the dataset.

Record-level constraints force business ruless limiting few fields in a single record. This constraints are checked at end of data editing (Post / AppendRecord / InsertRecord). Constraints, limiting single field are field constraints and must specified at TField.CustomConstraint or TField.ImportedConstraint.

The Constraints checking is performed when ConstraintsEnabled (see page 594) is True. If application needs to perform large updates to the dataset and application can guarantee the data consistancy, then set ConstraintsEnabled (see page 594) to False before updates, and return to original value after updates. The constraints are not checked at data fetching.

The expressions in Constraints must be predicates, evaluating to Boolean value. FireDAC supports expression syntax compatible with:

- BDE data access components;
- Client dataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write constraint expressions.

Syntax

```
property Constraints;
```

See Also

ConstraintsEnabled, DisableConstraints, EnableConstraints, BeginBatch, EndBatch, TDataSet.Post, TDataSet.AppendRecord, TDataSet.InsertRecord, TField.CustomConstraint, TField.ImportedConstraint

Example

```
with ADMemTable1.Constraints.Add do begin
  CustomConstraint := 'sal + bonus <= 2000';
  ErrorMessage := 'The employee payments must be equal or less than 2000 usd';
end;

ADMemTable1.ConstraintsEnabled := True;
ADMemTable1.Edit;
try
  ADMemTable1.FieldByName('sal').AsFloat := 1800;
  ADMemTable1.FieldByName('bonus').AsFloat := 300;
  // here exception will be raised
  ADMemTable1.Post;
except
  ADMemTable1.Cancel;
  Application.HandleException(Self);
end;

ADMemTable1.ConstraintsEnabled := False;
ADMemTable1.Edit;
ADMemTable1.FieldByName('sal').AsFloat := 1800;
ADMemTable1.FieldByName('bonus').AsFloat := 300;
// here exception will be NOT raised, because constraint checking is disabled
ADMemTable1.Post;
```

1.17.1.1.26.3.16 TADTable.ConstraintsEnabled Property

Specifies whether the dataset perform constraint checking.

Description

Use ConstraintsEnabled to get / set the flag controlling automatic record-level constraints enforcement.

When ConstraintsEnabled is False (the default), the dataset does not check Constraints (see page 594). This allows the dataset to perform the large data updates without overhead of checking constraints. When ConstraintsEnabled is True, the dataset checks Constraints (see page 594) at at end of data editing (Post / AppendRecord / InsertRecord).

If application needs to perform batch updates to dataset, then set ConstraintsEnabled to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property ConstraintsEnabled: Boolean;
```

See Also

Constraints, BeginBatch, EndBatch, DisableConstraints, EnableConstraints, TDataSet.Post, TDataSet.AppendRecord, TDataSet.InsertRecord, TADUpdateOptions.CheckRequired (see page 898)

Example

```
var
  lPrevConsEnabled: Boolean;
...
lPrevConsEnabled := ADQuery1.ConstraintEnabled;
ADQuery1.ConstraintEnabled := False;
try
  // perform updates here, without checking client constraints
finally
  ADQuery1.ConstraintEnabled := lPrevConsEnabled;
end;
```

1.17.1.1.26.3.17 TADTable.DetailFields Property

Gets / sets fields in detail dataset, used to establish a master-detail relationship with this dataset.

Description

Use DetailFields property value to specify semicolon separated list of the detail dataset field names, used to establish a master-detail relationship with this dataset.

The DetailFields property is useful only for parameter based master-detail setup. For range based setup the indexed fields are used as the detail field list. When DetailFields is not specified explicitly, then field names matching to query parameter names will be used as DetailFields value.

The DetailFields will be used only to assign specified field values when a new record is inserted into detail dataset. The mechanism used to filter the detail dataset records does not use DetailFields.

Syntax

```
property DetailFields: String;
```

See Also

Master-Detail Relationship (see page 104), MasterSource, MasterFields, IndexFieldNames, IndexName

Example

```
ADQuery1.SQL.Text := 'SELECT * FROM "Orders" WHERE CustomerID = :ID';
ADQuery1.MasterSource := CustomersDS;
ADQuery1.MasterFields := 'ID';
ADQuery1.DetailFields := 'CustomerID';
```

1.17.1.1.26.3.18 TADTable.FetchOptions Property

The set of options to control rows fetching.

Description

The FetchOptions property is the set of properties, controlling the fetching of cursor rows. These properties will inherit its values from FetchOptions of connection object.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), TADFetchOptions (see page 849)

1.17.1.1.26.3.19 TADTable.FilterChanges Property

Specifies what kinds of changed records must be "visible" in the dataset.

Description

The FilterChanges property allows to get / set changed records kinds, which must be accessible through navigation interface of dataset ("visible" in dataset):

Kind	Meaning
rtModified	Records changed after fetching or last CommitUpdates (see page 615) / CancelUpdates (see page 614).
rtInserted	New records, not yet posted to DB and added after dataset open or after last CommitUpdates (see page 615).
rtDeleted	Deleted records, not yet deleted from DB and deleted after dataset open or after last CommitUpdates (see page 615).
rtUnmodified	Unchanged records.
rtHasErrors	Records having associated errors after ApplyUpdates (see page 611). To get an error object use RowError (see page 607) property.

Syntax

```
property FilterChanges: TADUpdateRecordTypes;
```

See Also

Filtering Records (see page 102), Caching Updates (see page 114), RowError (see page 607), UpdateStatus (see page 650)

Example

```
// see only inserted records
ADQuery1.FilterChanges := [rtInserted];
```

1.17.1.1.26.3.20 TADTable.FormatOptions Property

The set of options to control data representation.

Description

The FormatOptions property is the set of properties, controlling the data type mapping, and some other aspects of data handling.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), TADFormatOptions (see page 860), Params

1.17.1.1.26.3.21 TADTable.IndexFieldNames Property

Lists the field names to use as an index.

Description

Use IndexFieldNames as an alternative method of specifying the current index to use for a dataset.

Specify the name of each field on which to index the dataset, separating names with semicolons. Ordering of field names is significant. Optionally specify the postfix for any of fields in form "field[:[D][A][N]]", where:

- 'D' - use descending sorting on this field;

- 'A' - use ascending sorting on this field;

- 'N' - use case-insensitive sorting on this field.

Use IndexFieldNames to create sort orders on the fly. The IndexFieldNames and IndexName (see page 601) properties are mutually exclusive. Setting one clears the other.

Syntax

```
property IndexFieldNames: String;
```

See Also

Sorting Records (see page 100), Indexes, IndexName

Example

```
ADQuery1.IndexFieldNames := 'order_date;customer_name:N';
```

1.17.1.1.26.3.22 TADTable.IndexName Property

Gets / sets the current index for the dataset by its name.

Description

Use IndexName to specify the current index for the dataset.

If IndexName is empty, the dataset's sort order is based on the IndexFieldNames (see page 600) property or on its default ordering as it is in dataset. If IndexName contains a name of valid index from Indexes collection, then that index is used to determine data view (including sort order) of records.

IndexFieldNames (see page 600) and IndexName are mutually exclusive. Setting one clears the other.

Syntax

```
property IndexName: String;
```

See Also

Sorting Records (see page 100), Indexes, IndexFieldNames

Example

```
with ADMemTable1.Indexes.Add do begin
  Name := 'by_name';
  Fields := 'NAME;DT';
  Active := True;
end;
ADMemTable1.IndexName := 'by_name';
```

1.17.1.1.26.3.23 TADTable.LocalSQL Property

The reference to a Local SQL engine.

Description

Use this property to get / set reference to a Local SQL engine.

When engine and Local SQL connection are active, then right after setting LocalSQL property, the dataset will be opened (if not yet) and will be registered in Local SQL engine. Otherwise dataset will be registered when engine and connection will be active. The TADMemTable (see page 436) structure must be defined before it will be opened.

The dataset will be registered using its Name property value. So, its Name property value must be not empty and unique across all datasets linked to the same Local SQL engine.

Setting this property to nil unregisters this dataset from Local SQL engine.

Syntax

```
property LocalSQL: TADCustomLocalSQL;
```

See Also

Local SQL (see page 96), TADLocalSQL (see page 825), TADMemTable (see page 436)

1.17.1.1.26.3.24 TADTable.MasterFields Property

Gets / sets fields in master dataset, used to establish a master-detail relationship with this dataset.

Description

Use MasterFields property value to specify semicolon separated list of the master dataset field names, used to establish a master-detail relationship with this dataset.

Specify MasterFields property value after assigning MasterSource (see page 603) property value. The mechanism used to filter this detail dataset records depends on the dataset class:

- TADQuery (see page 476) and TADStoredProc (see page 514) will match master fields to this query / procedure parameters by their names. When the master dataset current record is changed, then this dataset parameters will get values from the corresponding MasterFields.
- TADMemTable (see page 436) will match master fields to this dataset indexed fields by their positions. When the master dataset current record is changed, then a range will be applied to this dataset, where starting and ending range values are equal to the corresponding MasterFields field values. The application must have a current active index. The indexed fields will be the detail dataset fields used to establish mater-detail.

Syntax

```
property MasterFields: String;
```

See Also

Master-Detail Relationship (see page 104), MasterSource, DetailFields, IndexFieldNames, IndexName

Example

```
ADMemTable1.IndexFieldNames := 'CustomerID';
ADMemTable1.MasterSource := CustomersDS;
ADMemTable1.MasterFields := 'ID';
```

1.17.1.1.26.3.25 TADTable.MasterSource Property

Gets / sets a master data source, used to establish a master-detail relationship with this dataset.

Description

Use MasterSource to specify the data source linked to a dataset, which will be used as master dataset in master-detail relationship between this and specified datasets.

Additionally application must specify MasterFields (see page 602) property value. See MasterFields (see page 602) property description for additional information.

Syntax

```
property MasterSource: TDataSource;
```

See Also

Master-Detail Relationship (see page 104), MasterFields, IndexFieldNames, IndexName

Example 1

```
ADMemTable1.IndexFieldNames := 'CustomerID';
ADMemTable1.MasterSource := CustomersDS;
ADMemTable1.MasterFields := 'ID';
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery\MasterDetail demo for details.

1.17.1.1.26.3.26 TADTable.OnCommandChanged Event

Fires after the SQL property value is changed.

Description

The OnCommandChanged event fires after the SQL command text value is changed:

- for TADQuery (see page 476) - after SQL (see page 396) is changed;
- for TADStoredProc (see page 514) - after StoredProcName (see page 406) is changed;
- for TADTable (see page 539) - after TableName (see page 559) is changed.

Syntax

```
property OnCommandChanged: TNotifyEvent;
```

See Also

TADCustomQuery.SQL (see page 396), TADCustomStoredProc.StoredProcName (see page 406), TADTable.TableName (see page 559)

1.17.1.1.26.3.27 TADTable.OnError Event

The event fires when an error happens, while dataset is communicating with DBMS.

Parameters

Parameters	Description
ASender	The reference to dataset.
AInitiator	The reference to interface of object, which initially raised the exception.
AException	The exception object, which may be modified or even replaced by another exception object instance.

Description

The OnError event fires, when dataset executes one of the following operations and an error happens:

- Prepare (see page 511). The dataset is preparing DBMS command text for execution.
- Open (see page 509). The dataset is executing DBMS command to open cursor.
- Execute (see page 622), ExecSQL (see page 397), ExecProc (see page 410). The dataset is executing DBMS command text.
- Navigation methods. The dataset is fetching rows from cursor.

If most cases AException object will of EADDBEngineException (see page 833) class. If AException is of EADDBArrayExecuteError (see page 831) class, then it is Array DML (see page 82) error handling case. You should use a code like in the first example.

To centralize error handling, you can consider to use TADCustomConnection (see page 318).OnError (see page 332).

Syntax

```
property OnError: TADErrorEvent;
```

See Also

TADCustomQuery.ExecSQL (see page 397), TADCustomStoredProc.ExecProc (see page 410), TDataSet.Open, Prepare, EADDBArrayExecuteError (see page 831), EADDBEngineException (see page 833), TADCustomConnection.OnError (see page 332)

Example 1

Handling Array DML errors:

```
procedure TForm1.ADQuery1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if AException is EADPhysArrayExecuteError then
    with EADPhysArrayExecuteError(AException) do
      if Errors[0].Kind = ekUKViolated then
        AAction := eaSkip
      else
```

```
        AAction := eaFail
  else
    ...
end;
```

Example 2

Error substitution:

```
procedure TForm1.ADQuery1Error(ASender: TObject; const AInitiator: IADStanObject; var
AException: Exception);
begin
  if EADDBEngineException(AException).Errors[0].Kind = ekPKViolated then begin
    AException.Free;
    AException := Exception.Create('Please, enter unique ID value');
  end;
end;
```

1.17.1.1.26.3.28 TADTable.OnExecuteError Event

The event is triggered when an error happens, while executing a Array DML query.

Parameters

Parameters	Description
ASender	The reference to dataset.
ATimes	The current Array DML size. It may be different from value of the Execute method ATimes parameter, if full operation is splitted into few more small (on Firebird, for example).
AOffset	The current Array DML offset. It may be different from value of the Execute method AOffset parameter, if there was skipped erroneous rows or full operation is splitted into few more small (on Firebird, for example).
AError	The original DBMS returned error. Check AError.Errors (see page 834)[i].RowIndex (see page 839) for the index of erroneous row in parameters array.
AAction	The action dataset should take after return from event handler.

Description

The OnExecuteError event is triggered, when the dataset failed to execute Array DML using Execute method and ATimes > 1. Using this event handler you can:

- skip erroneous row of the parameters array;
- change values in erroneous row of the parameters array;
- stop full Array DML execution.

Note, OnExecuteError will be not called for the syntax errors or when ATimes = 1.

Syntax

```
property OnExecuteError: TADExecuteErrorEvent;
```

See Also

Array DML (see page 82), EADDBEngineException (see page 833), TADErrorAction, TADDataSet.Execute (see page 622)

Example

```
procedure TForm1.ADQuery1ExecuteError(ASender: TObject; ATimes,
  AOffset: Integer; AError: EADDBEngineException; var AAction: TADErrorAction);
begin
  if AException.Errors[0].Kind = ekUKViolated then
    AAction := eaSkip
  else
    AAction := eaFail;
end;
```

1.17.1.1.26.3.29 TADTable.OnReconcileError Event

Fires when a dataset needs to reconcile an update to a record that cannot not be applied.

Description

Use the OnReconcileError event handler to respond to error conditions that arise when the ApplyUpdates (see page 611) method was applying changes to the records to the database. The OnReconcileError event handler is called by Reconcile (see page 642) method.

As part of Reconcile call the dataset will loop through all records with associated exception objects. And for each such record will call the OnReconcileError event handler. The event handler gets:

- E - reference to the exception object. See Handling Errors (see page 44) topics for how to work with exception objects.
- UpdateKind - the update kind of the current record. It may be one of the values: rsInserted, rsDeleted, rsModified, rsUnchanged.

After handling error the event handler should set Action argument. The default value is raMerge. The possible values are:

Action	Description
raSkip	Just skip current record.
raAbort	Just quit from Reconcile call.
raMerge	Clear the current record error state, make current record changes the new initial state of this record. IOW, merge changes into dataset records cache.
raCorrect	Clear current record error state. IOW, mark the record as correctly applied.
raCancel	Cancel current record changes.
raRefresh	Cancel current record changes and reread the record values from the database.

The event handler may analyze the original and current field values, by reading TField.OldValue and NewValue properties. Application may also update the current field value, set Action to raCorrect, and later call ApplyUdpates again.

Syntax

```
property OnReconcileError: TADReconcileErrorEvent;
```

See Also

Caching Updates (see page 114), Reconcile, ApplyUpdates, OnUpdateError

Example

```
procedure TForm1.ADMemTable1ReconcileError(DataSet: TADDataSet; E: EADException;
  UpdateKind: TADDatSRowState; var Action: TADDAptReconcileAction);
begin
  if (UpdateKind = rsInserted) and (E is EADDBEngineException) and
(EADDBEngineException(E).Kind = ekUKViolated) then begin
    DataSet.FieldByName('ID').AsInteger := GetNextFreeID;
    Action := raCorrect;
  end;
end;
```

1.17.1.1.26.3.30 TADTable.OnUpdateError Event

Fires if an exception is generated when updates are applied to a database.

Description

Use the OnUpdateError event handler to respond to exceptions raised while applying immediate or cached updates to a database.

ASender is the dataset to which updates are applied.

AException is a pointer to a EADException (see page 836) object from which an application can extract an error message and the actual cause of the error condition. The OnUpdateError handler can use this information to determine how to respond to the error condition. In most cases AException object will be of class EADDBEngineException (see page 833)

or even one of it DBMS specific subclasses.

ARow is a DatS row object, repesenting erroneous record in dataset. This record is also current record in ASender.

ARequest indicates whether the error occurred while inserting, deleting, or modifying a record.

AAction indicates the action to take when the OnUpdateError handler exits. On entry into the handler, AAction is always set to eaDefault, which leads to uaFail, if not changed. If OnUpdateError can handle or correct the error, set AAction to uaRetry before exiting the error handler. Or consider other options.

The error handler can use the OldValue and NewValue properties TField to evaluate error conditions and set NewValue to a new value to reapply. In this case, set AAction to uaRetry before exiting.

The code in an OnUpdateError handler must not call any methods that make a different record the current one !

Syntax

```
property OnUpdateError: TADUpdateErrorEvent;
```

See Also

Caching Updates (see page 114), ApplyUpdates, CachedUpdates, Post, Delete, EADDBEngineException, OnUpdateRecord

Example

```
procedure TForm1.ADQuery1UpdateError(ASender: TDataSet; AException: EADException;
    ARow: TADDatSRow; ARequest: TADUpdateRequest; var AAction: TADErrorAction);
begin
  if (E is EADDBEngineException) and (EADDBEngineException(E).Kind = ekUKViolated) then
begin
    DataSet.FieldByName('ID').AsInteger := GetNextFreeID;
    Action := eaRetry;
  end;
end;
```

1.17.1.1.26.3.31 TADTable.OnUpdateRecord Event

Occurs when record update is applying to a database.

Description

Use an OnUpdateRecord event handler to process immediate or cached updates that cannot be correctly or easily handled:

- using FireDAC automatic SQL update command generation (see page 121);
- by a single table adapter component;
- by a single update component.

This event is useful for applications that require:

- additional control over parameters and macros substitution in update components;
- multi SQL statement updates, when DBMS does not support batches or blocks;
- non-SQL updates posting;
- etc

ASender is the dataset to which updates are applied.

ARequest indicates whether the current update is the insertion, deletion or modification of a record.

AAction indicates the action taken by the OnUpdateRecord handler before it exits. On entry into the handler, AAction is always set to eaDefault. If OnUpdateRecord is successful, it should set AAction to eaApplied before exiting.

AAction value	Description
eaFail	Mark the update as failed and return an error.
eaSkip	Skip current update and do not mark it as applied.
eaRetry	Retry the current operation.

eaApplied	Mark current update as applied.
eaDefault	Take default action. For successful update it is eaApplied, for failed update it is eaFail.
eaExitSuccess	Stop to process the cached updates, return success.
eaExitFailure	Stop to process the cached updates, return failure.

The OnUpdateRecord event handler code should read the dataset field values, including TField.NewValue, OldValue and CurValue. There:

- NewValue is the new field value before posting updates.
- OldValue is the original field value, as it was after record fetching or after last updates applying.
- CurValue is the current field value, the same as Value property.

Note, the OnUpdateRecord handler code must not call the methods that change the current dataset position.

Syntax

```
property OnUpdateRecord: TADUpdateRecordEvent;
```

See Also

Overriding Posting Updates (see page 123), ApplyUpdates, CachedUpdates, Post, Delete, OnUpdateError

Example 1

```
procedure TForm1.ADQuery1UpdateRecord(ASender: TDataSet; ARequest: TADUpdateRequest;
 var AAction: TADErrorAction; AOptions: TADUpdateRowOptions);
begin
  if ARequest = arInsert then begin
    // set the SQL command to insert new record
    ADQuery2.SQL := 'insert into mytab (id, code, name) values (:id, :code, :name)
returning tmstamp into :ts';

    // set parameter values
    ADQuery2.Params[0].Value := ASender['id'];
    ADQuery2.Params[1].Value := ASender['code'];
    ADQuery2.Params[2].Value := ASender['name'];

    // specially define TS parameter
    ADQuery2.Params[3].DataType := ftDateTime;
    ADQuery2.Params[3].ParamType := ptOutput;

    // insert new record
    ADQuery2.ExecSQL;

    // move TS output parameter value back to dataset
    ASender['tmstamp'] := ADQuery2.Params[3].Value;

    // return 'Ok' status
    AAction := eaApplied;
  end;
end;
```

Example 2

See demos:

- FireDAC\Samples\Comp Layer\TADQuery\CachedUpdates\OnUpdateRecord
- FireDAC\Samples\Comp Layer\TADUpdateSQL\Main

1.17.1.1.26.3.32 TADTable.ResourceOptions Property

The set of options to control resources usage.

Description

The ResourceOptions property is the set of properties, controlling the resources usage by the dataset. These properties will inherit its values from ResourceOptions of connection object.

Syntax

```
property ResourceOptions: TADBottomResourceOptions;
```

See Also

Setting Options (see page 34), TADBottomResourceOptions (see page 841)

1.17.1.1.26.3.33 TADTable.SchemaAdapter Property

Gets or sets a reference to a schema adapter, as a Centralized Cached Updates log.

Description

Use SchemaAdapter property to set a reference to the TADSchemaAdapter (see page 514) object to enable Centralized Cashed Updates (see page 114) mode. The schema adapter object will be used as a central change log share by few datasets.

To enable propagation of master dataset changes to a detail dataset set FetchOptions.DetailCascade (see page 854) to True for a detail dataset. Also both master and detail datasets must point to the same TADSchemaAdapter (see page 514) object.

Syntax

```
property SchemaAdapter: TADCustomSchemaAdapter;
```

See Also

Caching Updates (see page 114), Master-Detail Relationship (see page 104), TADFetchOptions.DetailCascade (see page 854)

Example

```
dsOrders.DataSet := qOrders;
qOrders.SchemaAdapter := ADSchemaAdapter1;
qOrders.CachedUpdates := True;

dsOrderDetails.DataSet := qOrderDetails;
qOrderDetails.SchemaAdapter := ADSchemaAdapter1;
qOrderDetails.CachedUpdates := True;
qOrderDetails.MasterSource := dsOrders;
qOrderDetails.MasterFields := 'OrderID';
qOrderDetails.IndexFieldNames := 'OrderID';
qOrderDetails.FetchOptions.DetailCascade := True;

qOrders.Open;
qOrderDetails.Open;

qOrders.Append;
....
qOrders.Post;

qOrderDetails.Append;
...
qOrderDetails.Post;

ADSchemaAdapter1.ApplyUpdates;
qOrders.CommitUpdates;
qOrderDetails.CommitUpdates;
```

1.17.1.1.26.3.34 TADTable.SchemaName Property

Limits the table names to the schema.

Description

Specify the SchemaName property value to limit the set of schemas to the single value.

Syntax

```
property SchemaName: String;
```

See Also

Object Names (see page 133), CatalogName (see page 547), TableName (see page 559)

1.17.1.1.26.3.35 TADTable.TableName Property

Gets / sets table name to open.

Description

The TableName property value together with CatalogName and SchemaName forms the full table name. FireDAC will build SELECT statement to fetch data from specified table.

Syntax

```
property TableName: String;
```

See Also

Browsing Table (see page 73), Object Names (see page 133), CatalogName (see page 547), SchemaName (see page 558)

1.17.1.1.26.3.36 TADTable.Transaction Property

Gets / sets reference to transaction object.

Description

The Transaction property allows to associate dataset with explicit transaction object. If it is not specified, then dataset will be associated with connection default transaction.

The specified transaction object will be used only for Prepare (see page 511), Open (see page 509), ExecSQL (see page 397), ExecProc (see page 410), Execute (see page 622) methods. For commands, generated to apply changes to database, FireDAC will use UpdateTransaction (see page 507), if it is specified, otherwise - Transaction.

Currently explicit transaction object assignment is supported only for Interbase / Firebird servers.

Syntax

```
property Transaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41), UpdateTransaction, TADCustomConnection.Transaction (see page 337)

1.17.1.1.26.3.37 TADTable.UpdateObject Property

Specifies the update object component used to post updates by dataset.

Description

Use UpdateObject to specify the TADUpdateSQL (see page 563) component to use in an application that must be able to post updates in a non-standard way.

The FireDAC has build in updates command generator, which using multiple information sources is able to generate SQL update commands for most of scenarios. In some cases, such as a query made against multiple tables, which may require multi-statement update, FireDAC cannot generate SQL update commands. In these cases, UpdateObject can be used to specify a TADUpdateSQL (see page 563) component that allows to specify custom SQL commands for each update posting case, like a insert / update / delete.

As alternative to update object usage, you can consider to use OnUpdateRecord (see page 605) event. It gives more options, but requires manual coding of posting updates. Also, in OnUpdateRecord (see page 605) handler you can use multiple update objects. See demo FireDAC\Samples\Comp Layer\TADUpdateSQL\Main.

Syntax

```
property UpdateObject: TADCustomUpdateObject;
```

See Also

OnUpdateRecord (see page 605), TADUpdateSQL

Example 1

```
ADUpdateSQL1.InsertSQL.Text := 'insert into mytab values (:new_id, :new_name)';
ADQuery1.UpdateObject := ADUpdateSQL1;
ADQuery1.Append;
ADQuery1['id'] := 100;
ADQuery1['name'] := 'qwe';
ADQuery1.Post;
```

Example 2

Demo FireDAC\Samples\Comp Layer\TADUpdateSQL\Main.

1.17.1.1.26.3.38 TADTable.UpdateOptions Property

The set of options to control updates posting.

Description

The UpdateOptions property is the set of properties, controlling the posting updates from the dataset. These properties will inherit its values from UpdateOptions of connection object.

Syntax

```
property UpdateOptions: TADBottomUpdateOptions;
```

See Also

Setting Options (see page 34), TADBottomUpdateOptions (see page 843)

1.17.1.1.26.3.39 TADTable.UpdateTransaction Property

Gets / sets reference to transaction object.

Description

The UpdateTransaction property allows to associate dataset internal update commands with explicit transaction object. If it is not specified, then dataset update commands will be associated with connection default update transaction. If it is not specified, then - with dataset Transaction.

For Prepare (see page 511), Open (see page 509), ExecSQL (see page 397), ExecProc (see page 410), Execute (see page 622) methods will be used Transaction (see page 507) objects.

Currently explicit update transaction object assignment is supported only for Interbase / Firebird servers.

Syntax

```
property UpdateTransaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41),Transaction, TADCustomConnection.UpdateTransaction (see page 338)

Example

```
var
  oSelectTransaction: TADTransaction;
  oUpdateTransaction: TADTransaction;
  oCusts: TADQuery;
.....
  oSelectTransaction := TADTransaction.Create(nil);
  oSelectTransaction.Connection := ADConnection1;
  oSelectTransaction.Options.ReadOnly := True;

  oUpdateTransaction := TADTransaction.Create(nil);
  oUpdateTransaction.Connection := ADConnection1;

  oCusts := TADQuery.Create(nil);
  oCusts.Connection := ADConnection1;
```

```
oCusts.Transaction := oSelectTransaction;
oCusts.UpdateTransaction := oUpdateTransaction;
oCusts.Open('select * from Customers'); // uses oSelectTransaction
...
oCusts.Edit; // uses oUpdateTransaction, if needed
...
oCusts.Post; // uses oUpdateTransaction
```

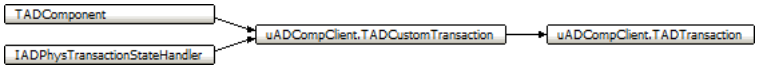
1.17.1.1.27 TADTransaction Class

The class is responsible for connection transaction management.

published

published	Description
AfterCommit (see page 561)	Fires after transaction is committed. The AfterCommit event fires after the Commit method is executed and the transaction is committed. In case of a nested transaction in AfterCommit event handler the transaction is not yet finished, but nesting level is decremented.
AfterRollback (see page 562)	Fires after transaction is rolled back.
AfterStartTransaction (see page 562)	Fires after transaction is started.
BeforeCommit (see page 562)	Fires before transaction will be committed.
BeforeRollback (see page 562)	Fires before transaction will be rolled back.
BeforeStartTransaction (see page 563)	Fires before transaction will be started.
Connection (see page 563)	Specifies the connection component that actually forms the connection to the DBMS.
Options (see page 563)	The set of options controlling transactions behaviour.

Class Hierarchy



File

uADCompClient

Description

Use TADTransaction to manage transactions in a connection to a DBMS.

The component, at first, is target to Interbase / Firebird client application developers.

Syntax

```
TADTransaction = class(TADCustomTransaction);
```

See Also

Managing Transactions (see page 41), TADCustomTransaction (see page 413), TADCustomConnection (see page 318)

1.17.1.1.27.1 published

1.17.1.1.27.1.1 TADTransaction.AfterCommit Event

Fires after transaction is committed. The AfterCommit event fires after the Commit method is executed and the transaction is committed. In case of a nested transaction in AfterCommit event handler the transaction is not yet finished, but nesting level is decremented.

Syntax

```
property AfterCommit: TNotifyEvent;
```

See Also

BeforeCommit

1.17.1.1.27.1.2 TADTransaction.AfterRollback Event

Fires after transaction is rolled back.

Description

The AfterRollback event fires after the Rollback method is executed and the transaction is rolled back. In case of a nested transaction in AfterRollback event handler the transaction is not yet rolled back in full, but nesting level is decremented.

Syntax

```
property AfterRollback: TNotifyEvent;
```

See Also

BeforeRollback

1.17.1.1.27.1.3 TADTransaction.AfterStartTransaction Event

Fires after transaction is started.

Description

The AfterStartTransaction event fires after the StartTransaction method is executed and the transaction is started. In case of a nested transaction in AfterStartTransaction event handler the transaction is not just started, but nesting level is incremented.

Syntax

```
property AfterStartTransaction: TNotifyEvent;
```

See Also

BeforeStartTransaction

1.17.1.1.27.1.4 TADTransaction.BeforeCommit Event

Fires before transaction will be committed.

Description

The BeforeCommit event fires before the Commit method will be executed and the transaction will be committed.

Syntax

```
property BeforeCommit: TNotifyEvent;
```

See Also

AfterCommit

1.17.1.1.27.1.5 TADTransaction.BeforeRollback Event

Fires before transaction will be rolled back.

Description

The BeforeRollback event fires before the Rollback method will be executed and the transaction will be rolled back.

Syntax

```
property BeforeRollback: TNotifyEvent;
```

See Also

AfterRollback

1.17.1.1.27.1.6 TADTransaction.BeforeStartTransaction Event

Fires before transaction will be started.

Description

The BeforeStartTransaction event fires before the StartTransaction method will be executed and the transaction will be started.

Syntax

```
property BeforeStartTransaction: TNotifyEvent;
```

See Also

AfterStartTransaction

1.17.1.1.27.1.7 TADTransaction.Connection Property

Specifies the connection component that actually forms the connection to the DBMS.

Description

Set Connection to the connection component that establishes a connection to the DBMS. If to change Connection property value, while transaction is active, an exception will be raised.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

Active

1.17.1.1.27.1.8 TADTransaction.Options Property

The set of options controlling transactions behaviour.

Description

The Options property returns reference to object, consisting of the set of options, controlling transactions behavior. Changing options will not take immediate effect, it is deferred until first transaction control statement.

Syntax

```
property Options: TADTxOptions;
```

See Also

StartTransaction, Commit, Rollback

Example

```
ADTransaction1.Options.ReadOnly := True;
ADTransaction1.StartTransaction;
```

1.17.1.1.28 TADUpdateSQL Class

TADUpdateSQL applies updates on behalf of queries or stored procedures that can't post updates directly.

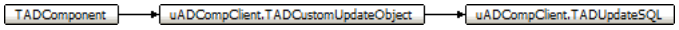
public

public	Description
Commands (see page 564)	Returns reference to internal command object, that will handle update request.
SQL (see page 565)	Gets / sets a specified SQL statement used when applying updates to DB.
Apply (see page 565)	Applies record update request to DB for current record in DataSet.

published

published	Description
Connection (see page 566)	Gets / sets the connection object.
ConnectionName (see page 566)	Gets / sets the connection name.
DeleteSQL (see page 567)	Specifies a SQL (see page 565) statement to use when applying a deletion of a record.
FetchRowSQL (see page 567)	Specifies a SQL (see page 565) statement to use to refetch a single record.
InsertSQL (see page 568)	Specifies a SQL (see page 565) statement to use when applying an insertion of a record.
LockSQL (see page 568)	Specifies the SQL (see page 565) statement to use when locking a record.
ModifySQL (see page 569)	Specifies the SQL (see page 565) statement to use when applying an update of a record.
UnlockSQL (see page 569)	Specifies the SQL (see page 565) statement to use when unlocking a record.

Class Hierarchy



File

uADCompClient

Description

Use a TADUpdateSQL object to provide SQL statements used to post updates from TADQuery (see page 476), TADStoredProc (see page 514) or TADTable (see page 539) components.

The TADUpdateSQL allows to override the SQL update commands automatically generated by the datasets, but not to execute the SQL commands and not to post changes to a DB. The usage of TADUpdateSQL is optional in case of TADQuery (see page 476) and TADTable (see page 539). Because they are capable to automatically generate the SQL commands to post updates from a dataset to a DB. And the usage is mandatory for TADStoredProc (see page 514).

To specify SQL commands at design-time use the TADUpdateSQL design-time editor, by double clicking a component. To specify SQL commands at run-time use the ModifySQL (see page 569), InsertSQL (see page 568), DeleteSQL (see page 567) and other properties. To specify additional parameter or macro values for specific update SQL use the Commands (see page 564) property.

Syntax

```
TADUpdateSQL = class(TADCustomUpdateObject);
```

See Also

Overriding Posting Updates (see page 123), TADQuery (see page 476), TADStoredProc (see page 514), TADTable (see page 539), TADCustomUpdateObject (see page 420), TADAdaptedDataSet.UpdateObject (see page 267)

1.17.1.1.28.1 public

1.17.1.1.28.1.1 TADUpdateSQL.Commands Property

Returns reference to internal command object, that will handle update request.

Description

The Commands indexed property returns a reference to the internal TADCustomCommand (see page 291) objects, that will handle the update requests, specified by the ARequest index. Use this reference to setup the command properties not accessible through the TADUpdateSQL interface. For example, Params (see page 303) or Macros (see page 301).

Syntax

```
property Commands [ARequest: TADUpdateRequest]: TADCustomCommand;
```

See Also

TADCustomCommand (see page 291), TADCustomCommand.Params (see page 303), TADCustomCommand.Macros (see page 301), SQL (see page 565)

Example

```
ADUpdateSQL1.InsertSQL := 'insert into !tab (workstation_time, id, name) values
(:workstation_time, :new_id, :new_name)';
ADUpdateSQL1.Commands[arInsert].ParamByName('workstation_time').AsDateTime := Now();
ADUpdateSQL1.Commands[arInsert].MacroByName('tab').AsRaw := 'mytab';
```

1.17.1.1.28.1.2 TADUpdateSQL.SQL Property

Gets / sets a specified SQL statement used when applying updates to DB.

Description

Returns the SQL statement in the one of XxxSQL properties, depending on the setting of the ARequest index.

Syntax

```
property SQL [ARequest: TADUpdateRequest]: TStrings;
```

See Also

Commands (see page 564), InsertSQL (see page 568), ModifySQL (see page 569), DeleteSQL (see page 567), LockSQL (see page 568), UnlockSQL (see page 569), FetchRowSQL (see page 567)

1.17.1.1.28.1.3 TADUpdateSQL.Apply Method

Applies record update request to DB for current record in DataSet.

Parameters

Parameters	Description
ARequest: TADUpdateRequest	A record update request.
var AAction: TADErrorAction	A completion status.
AOptions: TADUpdateRowOptions	An additional request options.

Description

Call Apply to apply changes of current DataSet (see page 421) record to database.

Standard implementation - TADUpdateSQL (see page 563) - sets parameters for a SQL statement and executes it to update a record. ARequest indicates the update kind, so for TADUpdateSQL (see page 563) specifies which SQL statement to execute. If appropriate SQL statement is not specified, then FireDAC will generate default one, as it is doing when TADAdaptedDataSet.UpdateObject (see page 267) is not specified.

Apply is primarily intended for manually executing update statements from an OnUpdateRecord (see page 605) event handler. There event handler ARequest, AAction and AOptions argument values must be assigned to Apply method corresponding arguments.

ARequest can be one of the following values:

Value	Meaning	TADUpdateSQL SQL property
arInsert	Insert record to database.	InsertSQL
arUpdate	Update record in database.	ModifySQL
arDelete	Delete record from database.	DeleteSQL
arLock	Lock record in database.	LockSQL
arUnlock	Release record lock in database.	UnlockSQL

arFetchRow	Refetch record from database.	FetchRowSQL
arUpdateHBlobs	Update Oracle BLOB/CLOB values in database.	-
arFetchGenerators	Fetch generator values to put into record.	-

AAction returns Apply method completion status, specifying what action should take the calling code. Normally this status is used by FireDAC code. When Apply method is called from OnUpdateRecord (see page 605) event handler, then assign AAction event handler argument to the AAction Apply method argument. Otherwise use eaDefault as initial value.

AOptions specifies additional options. When Apply method is called from OnUpdateRecord (see page 605) event handler, then assign AOptions event handler argument to the AOptions Apply method argument. Otherwise use [] as a value.

Syntax

```
procedure Apply(ARequest: TADUpdateRequest; var AAction: TADErrorAction; AOptions:
TADUpdateRowOptions); override;
```

See Also

Overriding Posting Updates (see page 123), TADUpdateSQL, TADAdaptedDataSet.UpdateObject (see page 267)

Example 1

```
procedure TfrmCachedUpdates.qrProductsUpdateRecord(ASender: TDataSet;
  ARequest: TADUpdateRequest; var AAction: TADErrorAction;
  AOptions: TADUpdateRowOptions);
begin
  usProducts.ConnectionName := qrProducts.ConnectionName;
  usProducts.DataSet := qrProducts;
  usProducts.Apply(ARequest, AAction, AOptions);

  usCategories.ConnectionName := qrProducts.ConnectionName;
  usCategories.DataSet := qrProducts;
  usCategories.Apply(ARequest, AAction, AOptions);

  AAction := eaApplied;
end;
```

Example 2

See FireDAC\Samples\Comp Layer\TADUpdateSQL\Main demo application.

1.17.1.1.28.2 published

1.17.1.1.28.2.1 TADUpdateSQL.Connection Property

Gets / sets the connection object.

Description

The Connection property value points to the connection object, which will be assigned to all internal update commands. If the connection is the same as used by DataSet, then leave Connection property empty.

Alternatively may be specified the ConnectionName (see page 566) property value.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

ConnectionName (see page 566), Commands (see page 564)

1.17.1.1.28.2.2 TADUpdateSQL.ConnectionName Property

Gets / sets the connection name.

Description

The ConnectionName property value is the name of a connection, which will be used by all internal update commands. If the

connection name is the same as used by DataSet, then leave ConnectionName property empty.

Alternatively may be specified the Connection (see page 566) property value.

Syntax

```
property ConnectionName: String;
```

See Also

Connection (see page 566), Commands (see page 564)

1.17.1.1.28.2.3 TADUpdateSQL.DeleteSQL Property

Specifies a SQL (see page 565) statement to use when applying a deletion of a record.

Description

Set DeleteSQL to a SQL statement to use when applying a record deletion to a database.

To submit to a command a dataset original field value, use the :OLD_<field name> parameter name. Statements can use additional parameters and macros. To get access to the parameters or macros collection use Commands (see page 564) property.

To create a DELETE statement at design time, use the UpdateSQL editor to create statements. For that double click on TADUpdateSQL component. Also, you can use stored procedure call or other SQL command to apply deletion.

See the Commands (see page 564) property for details of how to use macros or additional parameters in the command text.

Syntax

```
property DeleteSQL: TStrings;
```

See Also

Commands (see page 564), InsertSQL (see page 568), ModifySQL (see page 569), LockSQL (see page 568), UnlockSQL (see page 569), FetchRowSQL (see page 567)

Example

```
ADUpdateSQL1.DeleteSQL := 'DELETE FROM ADDEMO."Shippers" WHERE SHIPPERID = :OLD_SHIPPERID';
```

1.17.1.1.28.2.4 TADUpdateSQL.FetchRowSQL Property

Specifies a SQL (see page 565) statement to use to refetch a single record.

Description

Set FetchRowSQL to a SQL statement to use when refetching single record from a database.

To submit a dataset original field value, use the :OLD_<field name> parameter name. Statement can use additional parameters and macros. To get access to the parameters or macros collection use Commands (see page 564) property.

If statement returns record field values using parameters, then use the :NEW_<field name> or :<field name> parameter name and set parameter type to ptInputOutput / ptOutput. If statement produces a result set, then it fields must have the same names as the dataset fields.

If UpdateOptions (see page 653).RefreshMode (see page 903) is rmAll, then FetchRowSQL command will be executed automatically after posting a new record or changes to existing one to a database. In this case WHERE phrase must use :NEW_<field name> parameter names.

To create a SELECT statement at design time, use the UpdateSQL editor to create statements. For that double click on TADUpdateSQL component. Also, you can use stored procedure call or other SQL command to apply record fetching.

See the Commands (see page 564) property for details of how to use macros or additional parameters in the command text.

Syntax

```
property FetchRowSQL: TStrings;
```

See Also

Commands (see page 564), InsertSQL (see page 568), ModifySQL (see page 569), LockSQL (see page 568), UnlockSQL (see page 569), DeleteSQL (see page 567), TADUpdateOptions.RefreshMode (see page 903)

Example

```
ADUpdateSQL1.FetchRowSQL := 'SELECT SHIPPERID, COMPANYNAME, PHONE FROM ADDEMO."Shippers" ' +
  'WHERE SHIPPERID = :OLD_SHIPPERID';
```

1.17.1.1.28.2.5 TADUpdateSQL.InsertSQL Property

Specifies a SQL (see page 565) statement to use when applying an insertion of a record.

Description

Set InsertSQL to a SQL statement to use when applying a record insertion to a database.

To submit to a command a new field value, use the :NEW_<field name> or :<field name> parameter name.Statements can use additional parameters and macros. To get access to the parameters or macros collection use Commands (see page 564) property.

If statement fetches new record field values (identity or modified in trigger) using parameters, then to put parameter values into record, use the :NEW_<field name> or :<field name> parameter name and set parameter types to ptOuput. If statement will produce recordset, then it fields must have the same names as in dataset.

To create an INSERT statement at design time, use the UpdateSQL editor to create statements. For that double click on TADUpdateSQL component. Also, you can use stored procedure call or other SQL command to apply insertion.

See the Commands (see page 564) property for details of how to use macros or additional parameters in the command text.

Syntax

```
property InsertSQL: TStrings;
```

See Also

Commands (see page 564), ModifySQL (see page 569), DeleteSQL (see page 567), LockSQL (see page 568), UnlockSQL (see page 569), FetchRowSQL (see page 567)

Example 1

```
// SQL Server sample
ADUpdateSQL1.InsertSQL := 'INSERT INTO [Shippers] (COMPANYNAME, PHONE) ' +
  'VALUES (:NEW_COMPANYNAME, :NEW_PHONE); SELECT SCOPE_IDENTITY() AS SHIPPERID';
```

Example 2

```
// Oracle sample
ADUpdateSQL1.InsertSQL := 'INSERT INTO ADDEMO."Shippers" (COMPANYNAME, PHONE) ' +
  'VALUES (:NEW_COMPANYNAME, :NEW_PHONE) RETURNING SHIPPERID INTO :NEW_SHIPPERID';
ADUpdateSQL1.Commands[arInsert].ParamByName('NEW_SHIPPERID').ParamType := ptOutput;
```

1.17.1.1.28.2.6 TADUpdateSQL.LockSQL Property

Specifies the SQL (see page 565) statement to use when locking a record.

Description

Set LockSQL to the SQL statement to use when locking single record in a database. Library uses the statement if UpdateOptions.LockMode is pessimistic or optimistic.

To submit to a command a dataset original field value, use the :OLD_<field name> parameter name. Statements can use additional parameters and macros. To get access to the parameters or macros collection use Commands (see page 564) property.

To create a SELECT [FOR UPDATE] statement at design time, use the UpdateSQL editor to create statements. For that double click on TADUpdateSQL component. Also, you can use stored procedure call or other SQL command to lock record.

See the Commands (see page 564) property for details of how to use macros or additional parameters in the command

text.

Syntax

```
property LockSQL: TStrings;
```

See Also

Commands (see page 564), ModifySQL (see page 569), DeleteSQL (see page 567), InsertSQL (see page 568), UnlockSQL (see page 569), FetchRowSQL (see page 567)

1.17.1.1.28.2.7 TADUpdateSQL.ModifySQL Property

Specifies the SQL (see page 565) statement to use when applying an update of a record.

Description

Set UpdateSQL to the SQL statement to use when applying a record modification to a database.

To submit to a command a dataset original field value, use the :OLD_<field name> parameter name. To submit to a command a new field value, use the :NEW_<field name> or :<field name> parameter name. Statement can use additional parameters and macros. To get access to the parameters or macros collection use Commands (see page 564) property.

If statement fetches new record field values (identity or modified in trigger) using parameters, then to put parameter values into record, use the :NEW_<field name> or :<field name> parameter name and set parameter types to ptOuput. If statement produces result set, then it fields must have the same names as a dataset fields.

To create an UPDATE statement at design time, use the UpdateSQL editor to create statements. For that double click on TADUpdateSQL component. Also, you can use stored procedure call or other SQL command to apply modification.

See the Commands (see page 564) property for details of how to use macros or additional parameters in the command text.

Syntax

```
property ModifySQL: TStrings;
```

See Also

Commands (see page 564), InsertSQL (see page 568), DeleteSQL (see page 567), LockSQL (see page 568), UnlockSQL (see page 569), FetchRowSQL (see page 567)

Example

```
ADUpdateSQL1.InsertSQL := 'UPDATE ADDEMO."Shippers" SET SHIPPERID = :NEW_SHIPPERID, ' +
  'COMPANYNAME = :NEW_COMPANYNAME, PHONE = :NEW_PHONE WHERE SHIPPERID = :OLD_SHIPPERID';
```

1.17.1.1.28.2.8 TADUpdateSQL.UnlockSQL Property

Specifies the SQL (see page 565) statement to use when unlocking a record.

Description

Set UnlockSQL to the SQL statement to use when unlocking single record in a database. Library uses the statement if UpdateOptions.LockMode is pessimistic or optimistic.

To submit to a command a dataset original field value, use the :OLD_<field name> parameter name. Statement can use additional parameters and macros. To get access to the parameters or macros collection use Commands (see page 564) property.

Most DBMS's unlock record after finishing transaction. So, FireDAC does not generate any SQL statement for record unlocking by default.

See the Commands (see page 564) property for details of how to use macros or additional parameters in the command text.

Syntax

```
property UnlockSQL: TStrings;
```

See Also

Commands (see page 564), ModifySQL (see page 569), DeleteSQL (see page 567), InsertSQL (see page 568), LockSQL (see page 568), FetchRowSQL (see page 567)

1.17.1.2 Functions

The following table lists functions in this documentation.

Functions

Function	Description
ADManager (see page 570)	Function returns reference to global TADCustomManager (see page 365) object.
ADSetConnectionClass (see page 570)	Sets the TADCustomConnection (see page 318) descendant class.
ADSetManagerClass (see page 571)	Set the TADCustomManager (see page 365) descendant class.

1.17.1.2.1 uADCompClient.ADManager Function

Function returns reference to global TADCustomManager (see page 365) object.

File

uADCompClient

Returns

TADCustomManager (see page 365) instance.

Description

The ADManager function returns reference to singleton object, representing default instance of TADCustomManager (see page 365).

Syntax

```
function ADManager: TADCustomManager;
```

See Also

Multi Threading (see page 46), TADCustomManager (see page 365)

Example

```
var
  oList: TStringList;
......
  oList := TStringList.Create;
  oList.Add('Server=127.0.0.1');
  oList.Add('Database=addemo');
  ADManager.AddConnectionDef('myconn', 'MySQL', oList);
```

1.17.1.2.2 uADCompClient.ADSetConnectionClass Function

Sets the TADCustomConnection (see page 318) descendant class.

File

uADCompClient

Parameters

Parameters	Description
AClass: TADCustomConnectionClass	The class reference.

Description

Sets the TADCustomConnection descendant class, that FireDAC will use to create internally managed connection objects.

Syntax

```
procedure ADSetConnectionClass(AClass: TADCustomConnectionClass);
```

See Also

TADCustomConnection (see page 318), TADCustomConnection.ConnectionName

Example

```
ADSetConnectionClass(TMyConnection);
```

1.17.1.2.3 uADCompClient.ADSetManagerClass Function

Set the TADCustomManager (see page 365) descendant class.

File

uADCompClient

Parameters

Parameters	Description
AClass: TADCustomManagerClass	The class reference.

Description

Sets the TADCustomManager descendant class, that FireDAC will use to create internally managed manager object.

Syntax

```
procedure ADSetManagerClass(AClass: TADCustomManagerClass);
```

See Also

TADCustomManager (see page 365), ADManager (see page 570)

Example

```
ADSetManagerClass(TMyManager);
```

1.17.2 uADCompDataSet Namespace

Contains TADDataSet (see page 584) base dataset class and additional utility methods and classes.

Classes

Class	Description
TADAggregate (see page 572)	TADAggregate represents a maintained client aggregate value in an FireDAC dataset.
TADAggregates (see page 577)	TADAggregates is a collection of TADAggregate (see page 572) objects.
TADAutoIncField (see page 579)	TADAutoIncField represents integer auto incremental field in an FireDAC dataset.
TADBlobStream (see page 582)	TADBlobStream is a stream object giving access to BLOB / Memo fields.
TADDataSet (see page 584)	The TADDataSet is a direct TDataSet descendant, introducing base functionality for all FireDAC datasets.
TADIndex (see page 653)	TADIndex represents a maintained client view to records in an FireDAC dataset.
TADIndexes (see page 659)	TADIndexes is a collection of TADIndex (see page 653) objects.
TADMasterDataLink (see page 661)	TADMasterDataLink controlls master-detail link between two datasets.

TADSQLTimeIntervalField (see page 665)	TADSQLTimeIntervalField represents a time interval field in a dataset.
TADWideMemoField (see page 666)	TADWideMemoField represents a Unicode encoded memo field in a dataset.
TADXMLField (see page 667)	TADXMLField represents a Unicode encoded XML data field in a dataset.

Description

The uADCompDataSet unit contains:

- TADDataSet (see page 584) - base class for all FireDAC datasets;
- TADIndex (see page 653), TADIndexes (see page 659) - dataset view classes;
- TADAggregate (see page 572), TADAggregates (see page 577) - aggregate value classes;
- TADWideMemoField (see page 666) - Unicode memo field;
- TADAutoIncField (see page 579) - autoincremental field;
- TADBlobStream (see page 582) - BLOB stream.

See Also

uADCompClient (see page 257), uADCompScript (see page 684)

1.17.2.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADAggregate (see page 572)	TADAggregate represents a maintained client aggregate value in an FireDAC dataset.
TADAggregates (see page 577)	TADAggregates is a collection of TADAggregate (see page 572) objects.
TADAutoIncField (see page 579)	TADAutoIncField represents integer auto incremental field in an FireDAC dataset.
TADBlobStream (see page 582)	TADBlobStream is a stream object giving access to BLOB / Memo fields.
TADDataSet (see page 584)	The TADDataSet is a direct TDataSet descendant, introducing base functionality for all FireDAC datasets.
TADIndex (see page 653)	TADIndex represents a maintained client view to records in an FireDAC dataset.
TADIndexes (see page 659)	TADIndexes is a collection of TADIndex (see page 653) objects.
TADMasterDataLink (see page 661)	TADMasterDataLink controlls master-detail link between two datasets.
TADSQLTimeIntervalField (see page 665)	TADSQLTimeIntervalField represents a time interval field in a dataset.
TADWideMemoField (see page 666)	TADWideMemoField represents a Unicode encoded memo field in a dataset.
TADXMLField (see page 667)	TADXMLField represents a Unicode encoded XML data field in a dataset.

1.17.2.1.1 TADAggregate Class

TADAggregate represents a maintained client aggregate value in an FireDAC dataset.

public

public	Description
R ActualActive (see page 573)	Returns True, if aggregated value is really calculated.
R DataSet (see page 574)	Returns the reference to containing dataset.
R DataType (see page 574)	Returns the data type of aggregated value.
R InUse (see page 574)	Indicates whether the aggregate value is currently available.

Value (see page 575)	Returns the calculated value of the aggregate expression.

published

published	Description
Active (see page 575)	Controls when aggregated value must be calculated.
Expression (see page 575)	Specifies the expression to be calculated.
GroupingLevel (see page 576)	Specifies the set of records in the group.
IndexName (see page 576)	Specifies the index name, for which aggregated value applies.
Name (see page 577)	Specifies the name of an aggregated value.

Class Hierarchy



File

uADCompDataSet

Description

Use TADAggregate to create and maintain calculated client aggreated value.

TADAggregate are collected into TADAggregates (see page 577) collection. The set of TADAggregate may be created as at design time, as at run time.

Expression property specifies the formula. GroupingLevel (see page 576) defines the set of records on which value will be calculated. The value will be calculated if Active (see page 575) is True and TADDataSet (see page 584).AggregatesActive (see page 590) is True.

If you want to display aggregated value using data-aware controls, then use TAggregatedField instead of TADAggregate.

Syntax

```
TADAggregate = class(TCollectionItem);
```

See Also

Calculated and Aggregated Fields (see page 108), TAggregatedField, TADAggregates (see page 577), TADDataSet (see page 584), TADDataSet.Aggregates (see page 590), TADDataSet.AggregatesActive (see page 590)

Example 1

```
ADQuery1.AggregatesActive := True;
ADQuery1.Open;

with ADQuery1.Aggregates.Add do begin
  Name := 'AvgSalary';
  Expression := 'Avg(Salary)';
  Active := True;

  if InUse then
    Label1.Caption := VarToStr(Value)
  else
    Label1.Caption := '<not accessible>';
end;
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery (see page 476)\Aggregates sample for details.

1.17.2.1.1.1 public

1.17.2.1.1.1.1 TADAggregate.ActualActive Property

Returns True, if aggregated value is really calculated.

Description

Use ActualActive to see if FireDAC is maintaining this aggregated value. In general, the value is maintained if:

- Expression (see page 575) is not empty;
- Active (see page 575) is True;
- dataset is active;
- dataset AggregatesActive (see page 590) is True;
- IndexName (see page 576) is empty or current dataset index has IndexName (see page 576) name;

Use InUse (see page 574) property to detect is aggregated value accessable or not.

Syntax

```
property ActualActive: Boolean;
```

See Also

Active (see page 575), Expression (see page 575), InUse (see page 574), TADDataSet.AggregatesActive (see page 590)

1.17.2.1.1.1.2 TADAggregate.DataSet Property

Returns the reference to containing dataset.

Description

Use the DataSet property to get reference to dataset, containing this aggregated value.

Syntax

```
property DataSet: TADDataSet;
```

See Also

TADDataSet (see page 584)

1.17.2.1.1.1.3 TADAggregate.DataType Property

Returns the data type of aggregated value.

Description

Use the DataType property to get data type of value returned by aggregated expression.

Syntax

```
property DataType: TADDataType;
```

See Also

TADDataType

1.17.2.1.1.1.4 TADAggregate.InUse Property

Indicates whether the aggregate value is currently available.

Description

Read InUse to determine whether the Value (see page 575) method returns an aggregated value.

When InUse is False, the value is not accessible and Value returns Null.

When InUse is True, the aggregate is maintained so that it calculates the formula specified by Expression (see page 575) for the current set of records, including any edits to records in the current group. The value is accessible and Value (see page 575) returns meaningful value.

Syntax

```
property InUse: Boolean;
```

See Also

Active (see page 575), Expression (see page 575), Value (see page 575)

Example

```
if ADQuery1.Aggregates[0].InUse then
  Label1.Caption := VarToStr(ADQuery1.Aggregates[0].Value)
else
  Label1.Caption := '<not accessible>';
```

1.17.2.1.1.5 TADAggregate.Value Property

Returns the calculated value of the aggregate expression.

Description

Use the Value property to get the value of the aggregate expression for current dataset record.

The value is maintained if ActualActive (see page 573) is True and is accessible if InUse (see page 574) is True. The value is calculated for all records in the current group specified by GroupingLevel (see page 576) and is the same for all records in the group.

Syntax

```
property Value: Variant;
```

1.17.2.1.1.2 published

1.17.2.1.1.2.1 TADAggregate.Active Property

Controls when aggregated value must be calculated.

Description

Set Active property value to True to activate aggregated value calculation. By default it is False.

All aggregated values will be calculated when TADDataSet (see page 584).AggregatesActive (see page 590) is True.

Really, aggregated value will be calculated only when ActualActive (see page 573) returns True.

Syntax

```
property Active: Boolean;
```

See Also

ActualValue, TADDataSet.AggregatesActive (see page 590)

1.17.2.1.1.2.2 TADAggregate.Expression Property

Specifies the expression to be calculated.

Description

Use the Expression property to specify a formula text. The formula must contain aggregating functions, otherwise exception will be raised. The following aggregating functions are supported:

Function	Description
SUM	Summarize the argument values.
MIN	Returns the minimal argument value.
MAX	Returns the maximal argument value.
AVG	Computes the average value of argument, which is the total of all values, divided by the count of all not-NULL values.
COUNT	Returns the count of all not-NULL argument values, if argument is specified. Otherwise the count of all records.
TFIRST	Returns the first argument value.

TLAST	Returns the last argument value.

The aggregating functions operates on field values or on expressions built from field values, constants and scalar function calls, using the standard FireDAC expression syntax (see page 110). But few restrictions are applied:

- You can create expressions by using operators on aggregated values with other aggregated values or constants.
- You can't combine aggregated values with field values, because such expressions are ambiguous - there is no indication of which record should supply the field value.
- You can't nest aggregating functions.

The expression will be automatically evaluated by FireDAC and the result will be assigned to the Value (see page 575) property. All calculations are performed for current group of the records. Use GroupingLevel (see page 576) to specify grouping.

Syntax

```
property Expression: String;
```

See Also

GroupingLevel (see page 576), Value (see page 575)

Example 1

```
// count number of records with Discount > 0
ADQuery1.Aggregates[0].Expression := 'COUNT(IIF(Discount > 0, 1, NULL))';

// OK
ADQuery1.Aggregates[0].Expression := 'COUNT(*) * 5';
// INVALID - cannot combine aggregating function with field
ADQuery1.Aggregates[0].Expression := 'AVG(Discount) * Amount';
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery (see page 476)\Aggregates sample for details.

1.17.2.1.1.2.3 TADAggregate.GroupingLevel Property

Specifies the set of records in the group.

Description

Use GroupingLevel to define to set of records in group. The aggregated value calculation is performed for each group of records separately.

The GroupingLevel is the number of indexed field. This number is the subset of all indexed fields in the current dataset index. All records in the group has the same value of these first GroupingLevel indexed fields. The 0 means - all dataset records.

Syntax

```
property GroupingLevel: Integer;
```

See Also

Expression (see page 575), IndexName (see page 576), TADDataSet.IndexName (see page 601), TADDataSet.IndexFieldNames (see page 600)

Example

```
ADQuery1.IndexFieldNames : = 'ORDER_NO;PART_NO';
ADQuery1.AgregatesActive := True;
// calculate number of rows in each order
ADQuery1.Aggregates[0].Expression := 'COUNT(*)';
ADQuery1.Aggregates[0].GroupingLevel := 1;
ADQuery1.Aggregates[0].Active := True;
```

1.17.2.1.1.2.4 TADAggregate.IndexName Property

Specifies the index name, for which aggregated value applies.

Description

Use IndexName to specify a name of the index. When this index is active, then aggregated value is calculating.

Syntax

```
property IndexName: String;
```

See Also

TADDataSet.IndexName (see page 601), TADDataSet.IndexFieldNames (see page 600), GroupingLevel (see page 576)

1.17.2.1.1.2.5 TADAggregate.Name Property

Specifies the name of an aggregated value.

Description

Use Name to specify the name of an aggregated value. This name is used to display item at design time and to find item in a collection.

Syntax

```
property Name: String;
```

See Also

TADAggregates.FindAggregate (see page 579), TADAggregates.AggregateByName (see page 578)

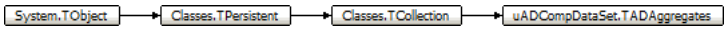
1.17.2.1.2 TADAggregates Class

TADAggregates is a collection of TADAggregate (see page 572) objects.

public

public	Description
Items (see page 578)	Returns TADAggregate (see page 572) object by it index.
Add (see page 578)	Appends new TADAggregate (see page 572) object to a collection.
AggregateByName (see page 578)	Find TADAggregate (see page 572) object by its name and raises exception if it is not found.
FindAggregate (see page 579)	Find TADAggregate (see page 572) object by its name.

Class Hierarchy



File

uADCompDataSet

Description

TADAggregates is a list of all aggregated values, maintained by dataset. A single TADAggregate (see page 572) object represents a single maintained aggregated value. The aggregate values are maintained and calculated by dataset automatically, if:

- TADAggregate (see page 572).Active (see page 575) is True and value is really active (see TADAggregate (see page 572).ActualActive (see page 573));
- TADDataSet (see page 584).AggregatesActive (see page 590) is True.

The list does not include counterparts of TAggregatedField fields.

Syntax

```
TADAggregates = class(TCollection);
```

See Also

Calculated and Aggregated Fields (see page 108), TADAggregate (see page 572), TADDataSet (see page 584), TADDataSet.AggregatesActive

1.17.2.1.2.1 public

1.17.2.1.2.1.1 TADAggregates.Items Property

Returns TADAggregate (see page 572) object by it index.

Description

Use Items indexed property to get TADAggregate (see page 572) object by it index.

The property is default one, so you does not need to write ADQuery1.Aggregates.Items[i], just ADQuery1.Aggregates[i].

Syntax

```
property Items [AIndex: Integer]: TADAggregate;
```

See Also

TADAggregate (see page 572), TCollection.Count

1.17.2.1.2.1.2 TADAggregates.Add Method

Appends new TADAggregate (see page 572) object to a collection.

Description

Call Add method to add new empty TADAggregate (see page 572) object to a collection.

Syntax

```
function Add: TADAggregate;
```

See Also

Items (see page 578), TCollection.Count

Example

```
with ADQuery1.Aggregates.Add do begin
  Name := 'a1';
  Expression := 'count(*)';
  Active := True;
end;
```

1.17.2.1.2.1.3 TADAggregates.AggregateByName Method

Find TADAggregate (see page 572) object by its name and raises exception if it is not found.

Parameters

Parameters	Description
const AName: String	A name of the aggregate object to find.

Description

Use AggregateByName method to find aggregate value by its name. If object is not found, then AggregateByName raises exception.

Syntax

```
function AggregateByName(const AName: String): TADAggregate;
```

See Also

TADAggregate.Name (see page 577), FindAggregate (see page 579), Items (see page 578)

Example

```
Label1.Caption := VarToStr(ADQuery1.Aggregates.AggregateByName('a1').Value);
```

1.17.2.1.2.1.4 TADAggregates.FindAggregate Method

Find TADAggregate (see page 572) object by its name.

Parameters

Parameters	Description
const AName: String	A name of the aggregate object to find.

Description

Use FindAggregate method to find aggregate value by its name. If object is not found, then FindAggregate returns nil.

Syntax

```
function FindAggregate(const AName: String): TADAggregate;
```

See Also

TADAggregate.Name (see page 577), AggregateByName (see page 578), Items (see page 578)

Example

```
var
  oAgg: TADAggregate;
...
oAgg := ADQuery1.Aggregates.FindAggregate('a1');
if oAgg <> nil then
  Label1.Caption := VarToStr(oAgg.Value);
```

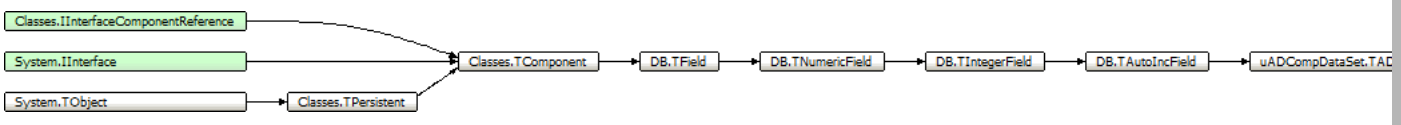
1.17.2.1.3 TADAutoIncField Class

TADAutoIncField represents integer auto incremental field in an FireDAC dataset.

published

published	Description
AutoIncrementSeed (see page 580)	Specifies initial value for client side auto incrementing value.
AutoIncrementStep (see page 580)	Specifies incrementing step for client side auto incrementing value.
ClientAutoIncrement (see page 581)	Specifies when dataset need to generate auto incremental value for new record column.
GeneratorName (see page 581)	Specifies name of a sequence / generator to the field.
IdentityInsert (see page 581)	Enables to explicitly assign values to the auto incremental column.
ServerAutoIncrement (see page 582)	Specifies when DBMS generates auto incremental value for new record column.

Class Hierarchy



File

uADCompDataSet

Description

Use TADAutoIncField to control client and server side auto incremental fields.

If FireDAC recognizes resultset column as representing an auto incremental field, then it will automatically create TADAutoIncField for this column. Otherwise, you can manually create TADAutoIncField for integer column at design time using Dataset Editor.

Set ClientAutoIncrement (see page 581) property to True (default value) and dataset will generate auto incrementing value for each new record. This is at first usefull in CachedUpdates (see page 592) = True mode.

Set ServerAutoIncrement (see page 582) property to True (default value) and dataset will not require value for this column and will not include it into updates. But will refresh after posting new record to a DBMS.

Set GeneratorName (see page 581) property to the name of sequence / generator and dataset will automatically fetch next value and assign it to dataset new record column.

Syntax

```
TADAutoIncField = class(TAutoIncField);
```

See Also

Auto-Incremental Fields (see page 118), TADUpdateOptions.GeneratorName (see page 581), TADUpdateOptions.FetchGeneratorsPoint, TADUpdateOptions.RefreshMode

1.17.2.1.3.1 published

1.17.2.1.3.1.1 TADAutoIncField.AutoIncrementSeed Property

Specifies initial value for client side auto incrementing value.

Description

Set AutoIncrementSeed to value, which will be the first auto generated value for new record column.

FireDAC recognizes negative auto incremental values, as explicitly generated by dataset at client side. Default value is -1.

The value of AutoIncrementSeed property may be changed only if dataset is inactive. After dataset is closed, the internal auto incremental value generator is reset to AutoIncrementSeed value.

Syntax

```
property AutoIncrementSeed: Integer;
```

See Also

ClientAutoIncrement (see page 581), AutoIncrementStep (see page 580)

Example

```
TADAutoIncfield(ADQuery1.FieldByName('ID')).AutoIncrementSeed := -1000;
```

1.17.2.1.3.1.2 TADAutoIncField.AutoIncrementStep Property

Specifies incrementing step for client side auto incrementing value.

Description

Set AutoIncrementStep to value, to which will be the auto generated value incremented for new record column.

FireDAC recognizes negative auto incremental values, as explicitly generated by dataset at client side. Default value is -1.

The value of AutoIncrementStep property may be changed only if dataset is inactive. After dataset is closed, the internal auto incremental value generator is reset to AutoIncrementSeed (see page 580) value.

Syntax

```
property AutoIncrementStep: Integer;
```

See Also

ClientAutoIncrement (see page 581), AutoIncrementSeed (see page 580)

Example

TADAutoIncfield(ADQuery1.FieldByName('ID')).AutoIncrementStep := -1;

1.17.2.1.3.1.3 TADAutoIncField.ClientAutoIncrement Property

Specifies when dataset need to generate auto incremental value for new record column.

Description

Set ClientAutoIncrement property to True (default value) and dataset will generate auto incrementing value for new record column.

The dataset uses internal auto increment value generator. The initial value is equal to AutoIncrementSeed (see page 580). The value is incrementing for new record by AutoIncrementStep (see page 580). After dataset is closed, the internal auto increment value generator is reset to AutoIncrementSeed (see page 580).

This may be usefull in CachedUpdates (see page 592) = True mode.

Syntax

```
property ClientAutoIncrement: Boolean;
```

See Also

Auto-Incremental Fields (see page 118), ServerAutoIncrement (see page 582)

Example

```
ADQuery1.CachedUpdates := True;
ADQuery1.Open;
ADQuery1.Append;
....
ADQuery1.Post;
// TADAutoIncField(ADQuery1.Fields[0]).Value -> -1
ADQuery1.Append;
....
ADQuery1.Post;
// TADAutoIncField(ADQuery1.Fields[0]).Value -> -2
ADQuery1.Append;
....
ADQuery1.Post;
// TADAutoIncField(ADQuery1.Fields[0]).Value -> -3
ADQuery1.ApplyUpdates;
```

1.17.2.1.3.1.4 TADAutoIncField.GeneratorName Property

Specifies name of a sequence / generator to the field.

Description

Set GeneratorName property to a name of a sequence / generator and dataset will automatically fetch a next value and assign it to a dataset new record column.

If GeneratorName is not specified, then dataset takes it value from UpdateOptions.GeneratorName (see page 901). Property UpdateOptions.FetchGeneratorsPoint (see page 901) controls when a value will be assigned. See it description for more details.

Syntax

```
property GeneratorName: String;
```

See Also

Auto-Incremental Fields (see page 118), Object Names (see page 133), TADUpdateOptions.GeneratorName (see page 901), TADUpdateOptions.FetchGeneratorsPoint (see page 901)

Example

```
TADAutoIncField(ADQuery1.Fields[0]).GeneratorName := 'GEN_ID';
```

1.17.2.1.3.1.5 TADAutoIncField.IdentityInsert Property

Enables to explicitly assign values to the auto incremental column.

Description

Set IdentityInsert property to True to enable explicit assignment of a value to the auto incremental column. By default it is False. Note, that not all DBMS's support assignment of a value to the identity column.

When a DBMS supports sequences or generators, and application uses client side field value filling from a sequence, then it is required to set IdentityInsert to True.

Syntax

```
property IdentityInsert: Boolean;
```

See Also

Auto-Incremental Fields (see page 118), TADDataMove

Example

```
TADAutoIncField(ADQuery1.Fields[0]).IdentityInsert := True;
ADQuery1.Append;
ADQuery1.Fields[0].Value := 123;
ADQuery1.Post;
```

1.17.2.1.3.1.6 TADAutoIncField.ServerAutoIncrement Property

Specifies when DBMS generates auto incremental value for new record column.

Description

Set ServerAutoIncrement property to True (default value) and dataset:

- does not require value for column (Required = False);
- does not include column into updates (pfInUpdate not in ProviderFlags);
- refreshes column values after posting new record to a database (AutoGenerateValue = arAutoInc).

Best result will be achieved if to set ClientAutoIncrement (see page 581) to True.

By default it is not possible to explicitly assign value to field with ServerAutoIncrement = True. If you need to explicitly assign value, then set IdentityInsert (see page 581) to True.

Syntax

```
property ServerAutoIncrement: Boolean;
```

See Also

Auto-Incremental Fields (see page 118), IdentityInsert (see page 581), ClientAutoIncrement (see page 581)

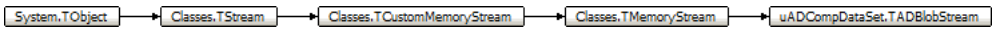
1.17.2.1.4 TADBlobStream Class

TADBlobStream is a stream object giving access to BLOB / Memo fields.

public

public	Description
Create (see page 583)	Creates new TADBlobStream object.
Truncate (see page 584)	Truncates BLOB value to zero length.

Class Hierarchy



File

uADCompDataSet

Description

Use TADBlobStream to read or modify the value of BLOB field. The BLOB field is a field of one of the following data types:

- ftBLOB (LONG RAW, bytea, IMAGE, etc);

- ftMemo (LONG, TEXT, etc);
- ftWideMemo / ftFmtMemo (NCLOB, NTEXT, etc);
- ftOraBLOB (BLOB);
- ftOraClob (CLOB).

To create TADBlobStream use TADDataSet.CreateBlobStream method or just create TADBlobStream object explicitly using Create constructor.

To modify content of BLOB field, dataset must be in dsEdit / dsInsert mode and field must be ReadOnly = False.

Syntax

```
TADBlobStream = class(TMemoryStream);
```

See Also

TADDataSet.CreateBlobStream

Example 1

The demo shows how to load BLOB field value from external file

```
ADQuery1.Append;
// assign other fields
....
// load BLOB field value
oStr := TADBlobStream.Create(ADQuery1.FieldByName('PHOTO'), bmWrite);
try
  oStr.LoadFromFile('c:\aaa.jpg');
finally
  oStr.Free;
end;
ADQuery1.Post;
```

Example 2

The demo shows how to save BLOB field value into external file

```
// save BLOB field value
oStr := TADBlobStream.Create(ADQuery1.FieldByName('PHOTO'), bmRead);
try
  oStr.SaveToFile('c:\aaa.jpg');
finally
  oStr.Free;
end;
```

Example 3

See FireDAC\Samples\Comp Layer\TADQuery (see page 476)\Blobs demo for details.

1.17.2.1.4.1 public

1.17.2.1.4.1.1 TADBlobStream.Create Constructor

Creates new TADBlobStream (see page 582) object.

Parameters

Parameters	Description
AField: TField	A BLOB field to open.
AMode: TBlobStreamMode	A open mode.

Description

Use Create constructor to explicitly create new TADBlobStream object.

Specify AField field to read from / writ to data. AMode controls access mode to BLOB field value:

Mode	Description
bmRead	Opens BLOB field for reading.
bmWrite	Opens BLOB field for writing. After open BLOB field value will be truncated to zero length.
bmReadWrite	Opens BLOB field for reading and writing.

To open BLOB field value in bmWrite or bmReadWrite mode, dataset must be in dsEdit / dsInsert mode and field must be ReadOnly = False.

The stream object must be destroyed using Free method after usage and not later when dataset will be closed. If BLOB field value was opened in bmWrite or bmReadWrite mode, then object must be destroyed before Post method call.

Syntax

```
constructor Create(AField: TField; AMode: TBlobStreamMode);
```

1.17.2.1.4.1.2 TADBlobStream.Truncate Method

Truncates BLOB value to zero length.

Description

Call Truncate method to truncate BLOB field value to zero length.

Syntax

```
procedure Truncate;
```

See Also

Create (see page 583)

1.17.2.1.5 TADDataSet Class

The TADDataSet is a direct TDataSet descendant, introducing base functionality for all FireDAC datasets.

protected

protected	Description
FetchOptions (see page 651)	The set of options to control rows fetching.
FormatOptions (see page 651)	The set of options to control data representation.
R ParamCount (see page 651)	Returns the current number of parameters in the command text.
Params (see page 651)	Contains the parameters for a SQL statement.
ResourceOptions (see page 652)	The set of options to control resources usage.
UpdateOptions (see page 653)	The set of options to control updates posting.
R Updates (see page 653)	The dataset change log.

public

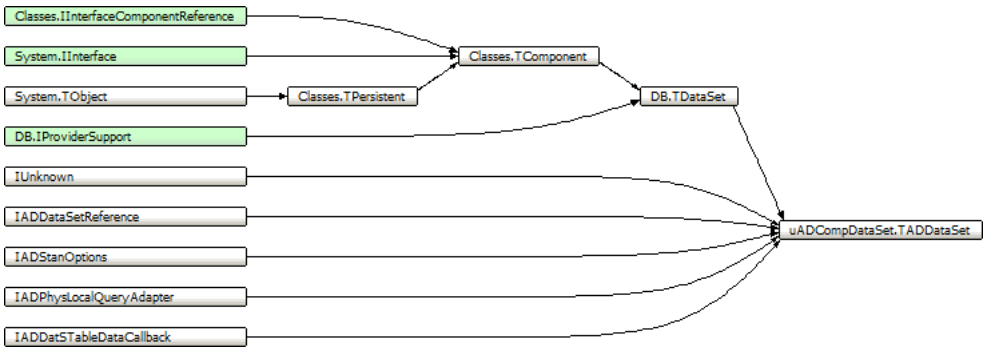
public	Description
ActiveStoredUsage (see page 588)	Controls how to use Active property value saved to DFM.
AfterApplyUpdates (see page 588)	Fires after cached updates are applied to DB.
AfterExecute (see page 589)	Fires after SQL command execution is finished.
AfterGetRecord (see page 589)	Fires after fetching next rowset one time for each fetched record.
AfterGetRecords (see page 589)	Fires after fetching next rowset.
AfterRowRequest (see page 589)	Fires after fetching record.
Aggregates (see page 590)	The collection of client side aggregates, defined for dataset.
AggregatesActive (see page 590)	Controls automatic calculation of aggregate values.
BeforeApplyUpdates (see page 591)	Fires before cached updates are applied to DB.
BeforeExecute (see page 591)	Fires before SQL command execution is started.

BeforeGetRecords (see page 592)	Fires before fetching next rowset.
BeforeRowRequest (see page 592)	Fires before fetching record.
CachedUpdates (see page 592)	Specifies whether dataset will log changes to the data without immediate applying of them to the database.
R ChangeCount (see page 593)	Gets the number of changes in the change log.
R CloneSource (see page 593)	Gets the dataset (if any) with which this dataset shares data.
Constraints (see page 594)	Specifies record-level client constraints that must be met when editing the data.
ConstraintsEnabled (see page 594)	Specifies whether the dataset perform constraint checking.
Data (see page 595)	Represents the data of the dataset, allowing to copy full data of one dataset into another.
R DatSManager (see page 596)	Returns the reference to the internal data storage manager.
R Delta (see page 596)	Represents the changed records of the dataset.
DetailFields (see page 596)	Gets / sets fields in detail dataset, used to establish a master-detail relationship with this dataset.
FilterChanges (see page 597)	Specifies what kinds of changed records must be "visible" in the dataset.
R FilteredData (see page 597)	Represents the currently "visible" data of the dataset.
R GroupingLevel (see page 598)	Returns the depth of grouping support provided by the current index.
IndexDefs (see page 598)	Returns indexes definition for the dataset.
Indexes (see page 598)	Lists all client indexes that apply to the dataset.
IndexesActive (see page 600)	Controls automatic maintenance of Index'es.
R IndexFieldCount (see page 600)	Returns the number of fields that make up the current index.
IndexFieldNames (see page 600)	Lists the field names to use as an index.
R IndexFields (see page 601)	Returns the fields that make up the current index.
IndexName (see page 601)	Gets / sets the current index for the dataset by its name.
KeyExclusive (see page 602)	Controls the exclusion of upper and lower boundaries for a range.
KeyFieldCount (see page 602)	Controls the number of indexed field values to use for key search or range setup.
MasterFields (see page 602)	Gets / sets fields in master dataset, used to establish a master-detail relationship with this dataset.
R MasterLink (see page 603)	Returns data link object servicing connection with master dataset.
MasterSource (see page 603)	Gets / sets a master data source, used to establish a master-detail relationship with this dataset.
OnMasterSetValues (see page 604)	Fires after a detailed dataset has set detailed query parameters.
OnReconcileError (see page 604)	Fires when a dataset needs to reconcile an update to a record that cannot not be applied.
OnUpdateError (see page 605)	Fires if an exception is generated when updates are applied to a database.
OnUpdateRecord (see page 605)	Occurs when record update is applying to a database.
R ParentDataSet (see page 607)	Returns the reference to parent dataset, if this one is nested.
R RowError (see page 607)	Returns an exception object associated with the current dataset record.
SavePoint (see page 607)	Gets / sets the change log version number.
R SourceEOF (see page 608)	Specifies if no more records to fetch from current cursor.
R SourceView (see page 608)	The reference to DatS view object, by which rows the dataset is navigating.
R Table (see page 609)	The reference to DatS table object, actually representing the internal data storage.
R UpdatesPending (see page 609)	Indicates whether the changes log is not empty.
R View (see page 610)	The reference to base DatS view object.
AddIndex (see page 610)	Creates a new client index for the dataset.
ApplyMaster (see page 611)	Applies a master-detail relation to this detail dataset.
ApplyRange (see page 611)	Applies a range to the dataset.

ApplyUpdates (see page 611)	Applies changes for all records in the dataset change log to the database.
AttachTable (see page 612)	Attaches dataset to existing DatS table and optionaly view.
BeginBatch (see page 613)	Optimizes dataset functionality for batch updates.
CancelRange (see page 613)	Cancels range for the dataset.
CancelUpdates (see page 614)	Removes all records from the change log and restores the dataset rows to its prior editing state.
CloneCursor (see page 614)	Shares the data belonging to another dataset.
CommitUpdates (see page 615)	Clears the changes log and marks all records as not modified.
CopyDataSet (see page 615)	Copies records from source dataset to this dataset.
CopyRecord (see page 616)	Copies all record fields from source dataset to the Self dataset.
CreateDataSet (see page 617)	Creates new empty internal data storage for dataset.
CreateExpression (see page 617)	Creates expression evaluator for dataset.
DeleteIndex (see page 618)	Deletes an client index for the dataset.
DeleteIndexes (see page 619)	Deletes all client indexes for the dataset.
DisableConstraints (see page 619)	Disables the enforcement of the client constraints.
Disconnect (see page 619)	Use Disconnect method to release DBMS resources, used by this dataset.
EditKey (see page 620)	Enables modification of the search key buffer.
EditRangeEnd (see page 620)	Enables changing the ending value for an existing range.
EditRangeStart (see page 621)	Enables changing the starting value for an existing range.
EmptyDataSet (see page 621)	Removes all records from the dataset internal data storage.
EnableConstraints (see page 621)	Reenables enforcement of the client constraints when records are posted.
EndBatch (see page 622)	Restores regular dataset functionality after batch updates.
Execute (see page 622)	Executes an SQL command.
FetchAgain (see page 623)	Allows to fetch additional records.
FetchAll (see page 624)	Retrieves all records from the current resultset and stores them into internal DatS table.
FetchBlobs (see page 624)	Retrieves BLOB field values from database for current record.
FetchDetails (see page 625)	Retrieves nested datasets from database for current record.
FetchNext (see page 625)	A shortcut to the GetNextPacket (see page 629) method.
FindKey (see page 625)	Searches for a record containing specified index field values.
FindNearest (see page 626)	Searches for a record containing specified index field values.
FindParam (see page 626)	Finds the parameter in Params (see page 651) property by a given name.
GetColumnField (see page 627)	Returns field corresponding to DatS column.
GetDetailLinkFields (see page 627)	Lists the field objects that link this dataset as a detail of a master dataset.
GetFieldColumn (see page 627)	Returns DatS column corresponding to field.
GetGroupState (see page 628)	Indicates where the current record sits within a specified group of records.
GetIndexNames (see page 628)	Fills the list by client index names.
GetNextPacket (see page 629)	Fetches the next rowset from the DBMS.
GetParentRow (see page 629)	Returns parent row of this nested dataset.
GetRow (see page 629)	Returns DatS row.
GotoCurrent (see page 630)	Sets the current record in this dataset the same as the current record in another, cloned dataset.
GotoKey (see page 630)	Searches for a record, using values in the key buffer.
GotoNearest (see page 631)	Searches for a closest record, using values in the key buffer.
IsIndexed (see page 631)	Returns True, if dataset has current client index.
IsLinked (see page 631)	Returns True, if dataset is linked to another dataset.
IsRanged (see page 632)	Returns True, whether the dataset records are limited by a range.
LoadFromFile (see page 632)	Loads a dataset's data from a file.

LoadFromFileAtOpen (see page 633)	Returns True, if dataset content will be loaded from file at Open.
LoadFromStream (see page 633)	Loads a dataset's data from a stream.
Locate (see page 634)	Searches the dataset for a record with specified field values and makes it current.
LocateEx (see page 635)	Searches the dataset for a specified record by predicate expression and makes that record current or returns its index.
LocateEx (see page 636)	Searches the dataset for a specified record by its fields values and makes that record current or returns its index.
Lookup (see page 637)	Searches the dataset for a record with specified key field values and returns result field values.
LookupEx (see page 638)	Searches the dataset for a record with specified key field values and returns result field values.
LookupEx (see page 639)	Searches the dataset for a record by predicate expression and returns result field values.
Offline (see page 640)	Sets dataset to offline mode.
OpenOrExecute (see page 641)	Executes SQL command and optionally opens dataset.
ParamByName (see page 641)	Searches parameter by its name.
Reconcile (see page 642)	Clears successfully updated records from the dataset's cache of changes.
RefreshRecord (see page 642)	Rereads field values of the current record from a data source.
RefreshUnknownRecord (see page 643)	Registers new record and optionally rereads it field values from a data source.
Release (see page 644)	Releases all dataset cursors.
RevertRecord (see page 644)	Undoes changes to the current record.
SaveToFile (see page 644)	Saves a dataset's data to a file.
SaveToFileAtClose (see page 645)	Returns True, if dataset content will be saved to file at Close.
SaveToStream (see page 645)	Saves a dataset's data to a stream.
SetFieldAttributes (see page 646)	Setups fields ProviderFlags.
SetKey (see page 647)	Starts the enter key values mode.
SetRange (see page 647)	Sets the starting and ending values of a range, and applies it.
SetRangeEnd (see page 648)	Sets the ending values of a range.
SetRangeStart (see page 648)	Sets the starting values of a range.
UndoLastChange (see page 649)	Undoes the last modification to the dataset records.
UpdateAttributes (see page 649)	Actualize changes to the client default expressions and auto-incremental fields.
UpdateConstraints (see page 650)	Actualize changes to the client constraints.
UpdateStatus (see page 650)	Returns the modification status for current record.

Class Hierarchy



File

uADCompDataSet

Description

The TADDataSet is a direct TDataSet descendant that holds data in memory in a table-like internal data storage consisting of rows (records) and columns (fields).

The TADDatSTable is the class representing a table of records. The TADDataSet stores all records in a single table, if it does not have nested datasets, array-type fields and other complex-type fields. You can get reference to table using TADDataSet.Table (see page 609) property.

The TADDatSTable object may have few TADDatSView objects - data views. A data view does not store records, but provides access to TADDatSTable records. For each view may defined records sort order, few record filters and few aggregate values. TADDataSet automatically creates data views, basing on their property values and method calls, like a Indexes (see page 598), Filter, FilterChanges (see page 597), ranges, etc. You can get reference to current view using TADDataSet.DefaultView property.

Also TADDatSTable has a change log. It contains a list of changed, deleted and new record references. The list is ordered by the moment of time, when a change is happened. If record was changed twice or more, log contains information for last change only.

For options description, please, look at uADStanOption (see page 840) description.

Syntax

```
TADDataSet = class(TDataSet, IUnknown, IADDataSetReference, IADStanOptions,
IADPhysLocalQueryAdapter, IADDatSTableDataCallback);
```

See Also

TADAdaptedDataSet (see page 259), TADDatSTable, TADDatSView, TADDatSRow, uADStanOption (see page 840)

1.17.2.1.5.1 public

1.17.2.1.5.1.1 TADDataSet.ActiveStoredUsage Property

Controls how to use Active property value saved to DFM.

Description

The ActiveStoredUsage property controls how to use Active property value saved to DFM. Include:

- auDesignTime, to use property value at design time;
- auRunTime, to use property value at run time.

Syntax

```
property ActiveStoredUsage: TADStoredActivationUsage;
```

See Also

TDataSet.Active

1.17.2.1.5.1.2 TADDataSet.AfterApplyUpdates Event

Fires after cached updates are applied to DB.

Description

The AfterApplyUpdates event fires after cached updates are applied to DB as result of ApplyUpdates (see page 611) method call.

Syntax

```
property AfterApplyUpdates: TADAfterApplyUpdatesEvent;
```

See Also

TADAfterApplyUpdatesEvent, ApplyUpdates (see page 611), BeforeApplyUpdates (see page 591)

1.17.2.1.5.1.3 TADDataSet.AfterExecute Event

Fires after SQL command execution is finished.

Description

The AfterExecute event fires after SQL command execution is finished as result of Execute (see page 622) method call.

Syntax

```
property AfterExecute: TADDataSetEvent;
```

See Also

TADDataSetEvent, Execute (see page 622), BeforeExecute (see page 591)

1.17.2.1.5.1.4 TADDataSet.AfterGetRecord Event

Fires after fetching next rowset one time for each fetched record.

Description

The AfterGetRecord event fires after dataset fetched next rowset as result of navigation methods calls. The event fires one time for each fetched record. For example, fetching occurs automatically inside of Next, Last, Locate methods. Inside of an event handler application may read dataset fields to get fetched record field values.

Syntax

```
property AfterGetRecord: TADDataSetEvent;
```

See Also

AfterGetRecords (see page 589)

1.17.2.1.5.1.5 TADDataSet.AfterGetRecords Event

Fires after fetching next rowset.

Description

The AfterGetRecords event fires after dataset fetched next rowset as result of navigation methods calls. For example, fetching occurs automatically inside of Next, Last, Locate methods.

Syntax

```
property AfterGetRecords: TADDataSetEvent;
```

See Also

TADDataSetEvent, BeforeGetRecords (see page 592), AfterGetRecord (see page 589)

1.17.2.1.5.1.6 TADDataSet.AfterRowRequest Event

Fires after fetching record.

Description

The AfterRowRequest event fires after dataset fetched current record as result of RefreshRecord (see page 642), FetchBlobs (see page 624) or FetchDetails (see page 625) method calls.

Syntax

```
property AfterRowRequest: TADDataSetEvent;
```

See Also

TADDataSetEvent, RefreshRecord (see page 642), FetchBlobs (see page 624), FetchDetails (see page 625), BeforeRowRequest (see page 592)

1.17.2.1.5.1.7 TADDataSet.Aggregates Property

The collection of client side aggregates, defined for dataset.

Description

Use Aggregates to define client side aggregating formulas, that will be automatically maintained and calculated for group of or for all records of dataset.

All records in the group of records has the same field values for defined set of fields. Aggregates performing calculation for group of records must be associated with one of the indexes. These aggregate expressions will be calculated only is associated index is current and active.

Adding aggregate field does not add TADAggregate (see page 572) object to Aggregates collection. That are two alternative ways - to use aggregate fields or to use Aggregates.

DataSet will automatically maintain and calculate Aggregates values when dataset is fetching the data or application edits the data, if AggregatesActive (see page 590) is True. If application needs to perform large updates to dataset and aggregate values are not needed while updating, then set AggregatesActive (see page 590) to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

When aggregates are maintained, the Value method of every active aggregate object returns a value that reflects the current data in the dataset. When users edit the data in the dataset, these values are recalculated to reflect the user's changes.

The expressions in Aggregates must contain aggregating functions, like a SUM, COUNT. FireDAC supports expression syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write constraint expressions.

Syntax

```
property Aggregates: TADAggregates;
```

See Also

Calculated and Aggregated Fields (see page 108), AggregatesActive (see page 590), BeginBatch (see page 613), EndBatch (see page 622)

Example 1

```
with ADMemTable1.Aggregates.Add do begin
  Expression := 'sum(sal + bonus)';
  Active := True;
end;
with ADMemTable1.Aggregates.Add do begin
  Expression := 'sum(sal + bonus)';
  IndexName := 'by_deps';
  Active := True;
end;
ADMemTable1.IndexName := 'by_deps';
ADMemTable1.AggregatesActive := True;

Label1.Caption := 'Total payments : ' + VarToStr(ADMemTable1.Aggregates[0].Value);
Label2.Caption := 'Current department payments : ' +
VarToStr(ADMemTable1.Aggregates[1].Value);
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery (see page 476)\Aggregates sample for details.

1.17.2.1.5.1.8 TADDataSet.AggregatesActive Property

Controls automatic calculation of aggregate values.

Description

Use AggregatesActive to get / set the flag controlling automatic maintenance and calculation of dataset aggregate values.

When AggregatesActive is False (the default), the dataset does not maintain aggregates. This allows the dataset to perform large updates to the dataset or fetch big volumes of data without overhead of calculating aggregate values, which happens at fetching or data editing. When AggregatesActive is True, the dataset calculates and maintains all aggregate values specified by the Aggregates (see page 590) property that are compatible with the current index.

To selectively enable and disable aggregates rather than turning them all on or off at once, use the Active (see page 575) property of individual TADAggregate (see page 572) objects. These objects are available through the Aggregates (see page 590) property.

If application needs to perform batch updates to dataset, then set AggregatesActive to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property AggregatesActive: Boolean;
```

See Also

Calculated and Aggregated Fields (see page 108),Aggregates (see page 590), TADAggregate.Active (see page 575), BeginBatch (see page 613), EndBatch (see page 622)

Example

```
var
  lPrevAggsActive: Boolean;
...
lPrevAggsActive := ADQuery1.AggregatesActive;
ADQuery1.AggregatesActive := False;
try
  // perform updates here, without calculating and reading client aggregates
finally
  ADQuery1.AggregatesActive := lPrevAggsActive;
end;
```

1.17.2.1.5.1.9 TADDataSet.BeforeApplyUpdates Event

Fires before cached updates are applied to DB.

Description

The BeforeApplyUpdates event fires before cached updates are applied to database as result of ApplyUpdates (see page 611) method call.

Syntax

```
property BeforeApplyUpdates: TADDataSetEvent;
```

See Also

TADDataSetEvent, ApplyUpdates (see page 611), AfterApplyUpdates (see page 588)

1.17.2.1.5.1.10 TADDataSet.BeforeExecute Event

Fires before SQL command execution is started.

Description

The BeforeExecute event fires before SQL command execution is started as result of Execute (see page 622) method call.

Syntax

```
property BeforeExecute: TADDataSetEvent;
```

See Also

TADDataSetEvent, Execute (see page 622), AfterExecute (see page 589)

1.17.2.1.5.1.11 TADDataSet.BeforeGetRecords Event

Fires before fetching next rowset.

Description

The BeforeGetRecords event fires before dataset will fetch next rowset as result of navigation methods calls. For example, fetching occurs automatically inside of Next, Last, Locate methods.

Syntax

```
property BeforeGetRecords: TADDataSetEvent;
```

See Also

TADDataSetEvent, AfterGetRecords (see page 589)

1.17.2.1.5.1.12 TADDataSet.BeforeRowRequest Event

Fires before fetching record.

Description

The BeforeRowRequest event fires before dataset will fetch current record as result of RefreshRecord (see page 642), FetchBlobs (see page 624) or FetchDetails (see page 625) method calls.

Syntax

```
property BeforeRowRequest: TADDataSetEvent;
```

See Also

TADDataSetEvent, RefreshRecord (see page 642), FetchBlobs (see page 624), FetchDetails (see page 625), AfterRowRequest (see page 589)

1.17.2.1.5.1.13 TADDataSet.CachedUpdates Property

Specifies whether dataset will log changes to the data without immediate applying of them to the database.

Description

CachedUpdates enables (True) or disables (False) the logging of data changes (Insert/Post, Edit/Post, Delete) without immediate applying of them to the database.

An application must explicitly apply changes from change log to the database, using ApplyUpdates (see page 611) method. All changes will be written in comparably small amount of time in single transaction. The main benefits of enabling cached updates are:

- fewer transactions and shorter transaction times;
- minimization of network traffic;
- simplified implementation of undo / redo functionality for dataset;
- ability to implement application offline mode or briefcase model.

The potential drawbacks of enabling cached updates are:

- other applications can access and change the actual data on the server while users are editing local copies of the data, resulting in an update conflict when cached updates are applied to the database;
- other applications cannot access data changes made by an application until its cached updates are applied to the database.

Note, to change CachedUpdate property value for TADTable, it must be inactive.

Syntax

```
property CachedUpdates: Boolean;
```

See Also

Caching Updates (see page 114), ApplyUpdates (see page 611), CancelUpdates (see page 614), CommitUpdates (

see page 615)

Example

```
ADQuery1.CachedUpdates := True;
...
ADQuery1.Edit;
...
ADQuery1.Post;
...
ADQuery1.Append;
...
ADQuery1.Post;
...
ADQuery1.ApplyUpdates;
ADQuery1.CommitUpdates;
ADQuery1.CachedUpdates := False;
```

1.17.2.1.5.1.14 TADDataSet.ChangeCount Property

Gets the number of changes in the change log.

Description

Use ChangeCount to determine how many modifications are in the datasets change log. ChangeCount:

- increases when the data is edited (eg Edit / Post);
- decreses when changes are undone (eg UndoLastChange (see page 649));
- sets to zero after commiting (CommitUpdates (see page 615)) or canceling all updates (CancelUpdates (see page 614)).

Syntax

```
property ChangeCount: Integer;
```

See Also

Caching Updates (see page 114), CachedUpdates (see page 592), UndoLastChange (see page 649), CancelUpdates (see page 614), CommitUpdates (see page 615), ApplyUpdates (see page 611)

Example 1

```
procedure TForm1.ADQuery1AfterPost(DataSet: TDataSet);
begin
  if TADDataSet(DataSet).ChangeCount = 0 then
    StatusBar1.SimpleText := 'no changes'
  else
    StatusBar1.SimpleText := Format('%d changed rows', [TADDataSet(DataSet).ChangeCount]);
end;
```

Example 2

See FireDAC\Samples\Comp Layer\TADMemTable (see page 436)\CachedUpdates (see page 592) sample for details.

1.17.2.1.5.1.15 TADDataSet.CloneSource Property

Gets the dataset (if any) with which this dataset shares data.

Description

Use CloneSource to access another dataset that shares its internal data storage with this one. CloneSource is set by the CloneCursor (see page 614) method when the datasets are linked. If the dataset has not called CloneCursor (see page 614) to share the data storage of another dataset, CloneSource is nil.

Syntax

```
property CloneSource: TADDataSet;
```

See Also

CloneCursor (see page 614)

1.17.2.1.5.1.16 TADDataSet.Constraints Property

Specifies record-level client constraints that must be met when editing the data.

Description

Use Constraints to maintain record-level constraints for the dataset.

Record-level constraints force business ruless limiting few fields in a single record. This constraints are checked at end of data editing (Post / AppendRecord / InsertRecord). Constraints, limiting single field are field constraints and must specified at TField.CustomConstraint or TField.ImportedConstraint.

The Constraints checking is performed when ConstraintsEnabled (see page 594) is True. If application needs to perform large updates to the dataset and application can guarantee the data consistancy, then set ConstraintsEnabled (see page 594) to False before updates, and return to original value after updates. The constraints are not checked at data fetching.

The expressions in Constraints must be predicates, evaluating to Boolean value. FireDAC supports expression syntax compatible with:

- BDE data access components;
- Client dataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write constraint expressions.

Syntax

```
property Constraints;
```

See Also

ConstraintsEnabled (see page 594), DisableConstraints (see page 619), EnableConstraints (see page 621), BeginBatch (see page 613), EndBatch (see page 622), TDataSet.Post, TDataSet.AppendRecord, TDataSet.InsertRecord, TField.CustomConstraint, TField.ImportedConstraint

Example

```
with ADMemTable1.Constraints.Add do begin
  CustomConstraint := 'sal + bonus <= 2000';
  ErrorMessage := 'The employee payments must be equal or less than 2000 usd';
end;

ADMemTable1.ConstraintsEnabled := True;
ADMemTable1.Edit;
try
  ADMemTable1.FieldByName('sal').AsFloat := 1800;
  ADMemTable1.FieldByName('bonus').AsFloat := 300;
  // here exception will be raised
  ADMemTable1.Post;
except
  ADMemTable1.Cancel;
  Application.HandleException(Self);
end;

ADMemTable1.ConstraintsEnabled := False;
ADMemTable1.Edit;
ADMemTable1.FieldByName('sal').AsFloat := 1800;
ADMemTable1.FieldByName('bonus').AsFloat := 300;
// here exception will be NOT raised, because constraint checking is disabled
ADMemTable1.Post;
```

1.17.2.1.5.1.17 TADDataSet.ConstraintsEnabled Property

Specifies whether the dataset perform constraint checking.

Description

Use ConstraintsEnabled to get / set the flag controlling automatic record-level constraints enforcement.

When ConstraintsEnabled is False (the default), the dataset does not check Constraints (see page 594). This allows the

dataset to perform the large data updates without overhead of checking constraints. When ConstraintsEnabled is True, the dataset checks Constraints (see page 594) at at end of data editing (Post / AppendRecord / InsertRecord).

If application needs to perform batch updates to dataset, then set ConstraintsEnabled to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property ConstraintsEnabled: Boolean;
```

See Also

Constraints (see page 594), BeginBatch (see page 613), EndBatch (see page 622), DisableConstraints (see page 619), EnableConstraints (see page 621), TDataSet.Post, TDataSet.AppendRecord, TDataSet.InsertRecord, TADUpdateOptions.CheckRequired (see page 898)

Example

```
var
  lPrevConsEnabled: Boolean;
...
lPrevConsEnabled := ADQuery1.ConstraintEnabled;
ADQuery1.ConstraintEnabled := False;
try
  // perform updates here, without checking client constraints
finally
  ADQuery1.ConstraintEnabled := lPrevConsEnabled;
end;
```

1.17.2.1.5.1.18 TADDataSet.Data Property

Represents the data of the dataset, allowing to copy full data of one dataset into another.

Description

The Data property represents the dataset's internal in-memory data storage. Using this property application can copy current structure and data of one FireDAC dataset to another FireDAC dataset.

The property value is reference to IADDataSet interface. It is reference counted and application does not need explicitly to free it. If application keeps the interface reference using a variable or a field, then reference must be released before dataset will be closed.

The dataset must be inactive to set this property value, otherwise exception will be raised. After its setting, the Self dataset:

- has a structure of an original dataset, excluding Indexes (see page 598), IndexDefs (see page 598), Filter, etc;
- has a copy of an original dataset data, including all row versions and states (inserted, deleted, updated, unchanged);
- has CachedUpdates (see page 592) = True, if an original dataset has unapplied changed;
- is open.

Syntax

```
property Data: IADDataSetReference;
```

See Also

Delta (see page 596), FilteredData (see page 597), CopyDataSet (see page 615), CachedUpdates (see page 592)

Example

```
ADQuery1.SQL.Text := 'select * from orders; select * from customers';

ADQuery1.Open;
ADQuery1.FetchAll;
// assign orders records to ADMemTable1
ADMemTable1.Data := ADQuery1.Data;

ADQuery1.NextRecordSet;
ADQuery1.FetchAll;
// assign customers records to ADMemTable2
ADMemTable2.Data := ADQuery1.Data;
```

1.17.2.1.5.1.19 TADDataSet.DatSManager Property

Returns the reference to the internal data storage manager.

Description

The DatSManager property value is the reference to the internal data storage manager.

Syntax

```
property DatSManager: TADDatSManager;
```

See Also

Table (see page 609), View (see page 610), SourceView (see page 608), AttachTable (see page 612)

1.17.2.1.5.1.20 TADDataSet.Delta Property

Represents the changed records of the dataset.

Description

The Delta property value represents the set of changed records in the dataset. The set may be assigned to other datasetsData (see page 595) property. The delta may be not empty only when CachedUpdates (see page 592) is True.

The property value is reference to IADDataSet interface. It is reference counted and application does not need explicitly to free it. If application keeps the interface reference using a variable or a field, then reference must be released before dataset will be closed.

In case you are processing the dataset Delta using TADMemTable as:

```
ADMemTable1.Data := ADQuery1.Delta;
```

Make sure the filter changes property reflects the records (modified, inserted or deleted) you want to process:

```
ADMemTable1.FilterChanges := [rtModified, rtInserted, rtDeleted];
```

Syntax

```
property Delta: IADDataSetReference;
```

See Also

Data (see page 595), FilteredData (see page 597), FilterChanges (see page 597)

Example

```
// copy to ADMemTable1 all ADQuery1 changed records and their versions
ADMemTable1.FilterChanges := [rtModified, rtInserted, rtDeleted];
ADMemTable1.Data := ADQuery1.Delta;
```

1.17.2.1.5.1.21 TADDataSet.DetailFields Property

Gets / sets fields in detail dataset, used to establish a master-detail relationship with this dataset.

Description

Use DetailFields property value to specify semicolon separated list of the detail dataset field names, used to establish a master-detail relationship with this dataset.

The DetailFields property is useful only for parameter based master-detail setup. For range based setup the indexed fields are used as the detail field list. When DetailFields is not specified explicitly, then field names matching to query parameter names will be used as DetailFields value.

The DetailFields will be used only to assign specified field values when a new record is inserted into detail dataset. The mechanism used to filter the detail dataset records does not use DetailFields.

Syntax

```
property DetailFields: String;
```

See Also

Master-Detail Relationship (see page 104), MasterSource (see page 603), MasterFields (see page 602), IndexFieldNames (see page 600), IndexName (see page 601)

Example

```
ADQuery1.SQL.Text := 'SELECT * FROM "Orders" WHERE CustomerID = :ID';
ADQuery1.MasterSource := CustomersDS;
ADQuery1.MasterFields := 'ID';
ADQuery1.DetailFields := 'CustomerID';
```

1.17.2.1.5.1.22 TADDataSet.FilterChanges Property

Specifies what kinds of changed records must be "visible" in the dataset.

Description

The FilterChanges property allows to get / set changed records kinds, which must be accessible through navigation interface of dataset ("visible" in dataset):

Kind	Meaning
rtModified	Records changed after fetching or last CommitUpdates (see page 615) / CancelUpdates (see page 614).
rtInserted	New records, not yet posted to DB and added after dataset open or after last CommitUpdates (see page 615).
rtDeleted	Deleted records, not yet deleted from DB and deleted after dataset open or after last CommitUpdates (see page 615).
rtUnmodified	Unchanged records.
rtHasErrors	Records having associated errors after ApplyUpdates (see page 611). To get an error object use RowError (see page 607) property.

Syntax

```
property FilterChanges: TADUpdateRecordTypes;
```

See Also

Filtering Records (see page 102), Caching Updates (see page 114), RowError (see page 607), UpdateStatus (see page 650)

Example

```
// see only inserted records
ADQuery1.FilterChanges := [rtInserted];
```

1.17.2.1.5.1.23 TADDataSet.FilteredData Property

Represents the currently "visible" data of the dataset.

Description

The FilteredData property value represents the set of records accessible through navigation interface of the dataset ("visible" in the dataset). This set is formed by applying of filters, ranges, indexes, etc. The set may be assigned to other datasets Data property.

The property value is reference to IADDataSet interface. It is reference counted and application does not need explicitly to free it. If application keeps the interface reference using a variable or a field, then reference must be released before one of following events occurs:

- close dataset;
- change Filter / FilterChanges (see page 597) / FilterOptions / OnFilterRecord / Filtered property values;
- change IndexName (see page 601) / IndexFieldNames (see page 600) / Indexes (see page 598) property values;
- change current ranges.

Syntax

```
property FilteredData: IADDataSetReference;
```

See Also

Delta (see page 596), Data (see page 595), Filter, OnFilterRecord, IndexName (see page 601), IndexFieldNames (see page 600), Indexes (see page 598)

Example

```
ADQuery1.Filter := 'upper(name) like ''D%''';
ADQuery1.Filtered := True;
// copy to ADMemTable1 all ADQuery1 visible (where name starts from D) records
ADMemTable1.Data := ADQuery1.FilteredData;
```

1.17.2.1.5.1.24 TADDataSet.GroupingLevel Property

Returns the depth of grouping support provided by the current index.

Description

Read GroupingLevel to determine the maximum level group for which grouping support is available. The datasets provide grouping support by GetGroupState (see page 628) method. It indicates where the current record falls within a group.

A group is defined as the set of records that have the same value on each of a set of fields. A level 0 group is the set of all records in the dataset. A level 1 group is the set of records with the same value on the first field of the index. A level 2 group is the set of records with the same values on each of the first two fields of the index. A level n group is the set of records with the same values on each of the first n fields of the index.

By default, GroupingLevel is the default grouping level of the current index or the maximum grouping level of a maintained aggregate on the current index, whichever is greater. GroupingLevel can never be greater than IndexFieldCount (see page 600).

Syntax

```
property GroupingLevel: Integer;
```

See Also

GetGroupState (see page 628), IndexFieldCount (see page 600)

1.17.2.1.5.1.25 TADDataSet.IndexDefs Property

Returns indexes definition for the dataset.

Description

Use IndexDefs property to read or maintain indexes definitions.

The dataset also has Indexes collection. Application should use either IndexDefs, either Indexes, but not both in the same time. We suggest to use Indexes, as it is more flexible. IndexDefs property at first is needed for compatibility with other software using TDataSet, where IndexDefs is really defined.

The TADTable will automatically fill IndexDefs by database indexes, if fiMeta in FetchOptions (see page 651).Items (see page 855).

Syntax

```
property IndexDefs: TIndexDefs;
```

See Also

Sorting Records (see page 100), Indexes (see page 598), TADFetchOptions.Items (see page 855)

1.17.2.1.5.1.26 TADDataSet.Indexes Property

Lists all client indexes that apply to the dataset.

Description

Use Indexes to define client side data views on the dataset records. Indexes are a collection of the TADIndex (see page 653) objects, each of them defines optional:

- Sorting by fields (Fields (see page 657), CaseInsFields (see page 655), DescFields (see page 656), Options (see page 658)).
- Sorting by expression (Expression (see page 656), Options (see page 658)).
- Records filter (Filter (see page 657), FilterOptions (see page 657)).
- Records distinction flag (Distinct (see page 656)).

The individual indexes are not just the sort and filter definitions. FireDAC maintains a consistent view on the data for each active index, while application fetches data or edits data. That creates additional overhead, but allows to switch between indexes without delays.

To make a view current, set it Selected (see page 659) property to True or dataset IndexName (see page 601) property to the view name. Note, the view must have Active = True to be maintained.

Dataset maintains Indexes views, when IndexesActive (see page 600) is True. When an application needs to perform batch updates to dataset, then set IndexesActive (see page 600) to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Setting IndexFieldNames (see page 600) does not add a TADIndex (see page 653) object to Indexes collection. Defining IndexDefs (see page 598) before dataset opening, will clear Indexes and fill them using IndexDefs (see page 598) information. So, there are two alternative ways - use IndexDefs (see page 598) or use Indexes. We suggest to use Indexes, as it is more flexible.

Some of the navigation methods, like a Locate (see page 634) / Lookup (see page 637) / SetRange (see page 647) are using indexes to optimize their operations.

The sorting and filter expressions in Indexes supports syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

Syntax

```
property Indexes: TADIndexes;
```

See Also

Sorting Records (see page 100), Writing Expressions (see page 110), IndexDefs (see page 598), IndexesActive (see page 600), IndexFieldNames (see page 600), IndexName (see page 601), BeginBatch (see page 613), EndBatch (see page 622)

Example 1

```
with ADMemTable1.Indexes.Add do begin
  Name := 'by_name';
  Fields := 'NAME;DT';
  Active := True;
end;
with ADMemTable1.Indexes.Add do begin
  Name := 'by_payments';
  Expression := 'sum(sal + bonus)';
  Filter := 'dep_id is not null;
  Active := True;
end;
ADMemTable1.IndexesActive := True;
ADMemTable1.IndexName := 'by_name';
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery (see page 476)\Indices demo for details.

1.17.2.1.5.1.27 TADDataSet.IndexesActive Property

Controls automatic maintenance of Index'es.

Description

Use IndexesActive to indicate whether the dataset should maintain the data views, defined by the Indexes (see page 598) collection.

When IndexesActive is False, the dataset does not maintain the data views. That avoids the overhead of updating views, while data is fetched or is edited. When IndexesActive is True (by default), the dataset maintains the data views.

When indexes are maintained, each of them may be current index, selected by TADIndex (see page 653).Selected (see page 659) or by IndexName (see page 601) properties.

To selectively enable and disable data views rather than turning them all on or off at once, use the Active (see page 655) property of individual TADIndex (see page 653) objects. These objects are available through the Indexes (see page 598) property.

If application needs to perform batch updates to dataset, then set IndexesActive to False before updates, and return to original value after updates. Also see BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
property IndexesActive: Boolean;
```

See Also

Sorting Records (see page 100), Indexes (see page 598), BeginBatch (see page 613), EndBatch (see page 622)

Example

```
var
  lPrevIndsActive: Boolean;
...
lPrevIndsActive := ADQuery1.IndexesActive;
ADQuery1.IndexesActive := False;
try
  // perform updates here, without maintaining client views
finally
  ADQuery1.IndexesActive := lPrevIndsActive;
end;
```

1.17.2.1.5.1.28 TADDataSet.IndexFieldCount Property

Returns the number of fields that make up the current index.

Description

Use IndexFieldCount to determine how many fields are used by the current index for the dataset.

The property has the meaning only for non-expressional indexes.

Syntax

```
property IndexFieldCount: Integer;
```

See Also

IndexFieldNames (see page 600), IndexName (see page 601)

1.17.2.1.5.1.29 TADDataSet.IndexFieldNames Property

Lists the field names to use as an index.

Description

Use IndexFieldNames as an alternative method of specifying the current index to use for a dataset.

Specify the name of each field on which to index the dataset, separating names with semicolons. Ordering of field names is significant. Optionally specify the postfix for any of fields in form "field[:[D][A][N]]", where:

- 'D' - use descending sorting on this field;

- 'A' - use ascending sorting on this field;

- 'N' - use case-insensitive sorting on this field.

Use IndexFieldNames to create sort orders on the fly. The IndexFieldNames and IndexName (see page 601) properties are mutually exclusive. Setting one clears the other.

Syntax

```
property IndexFieldNames: String;
```

See Also

Sorting Records (see page 100), Indexes (see page 598), IndexName (see page 601)

Example

```
ADQuery1.IndexFieldNames := 'order_date;customer_name:N';
```

1.17.2.1.5.1.30 TADDataSet.IndexFields Property

Returns the fields that make up the current index.

Description

IndexFields is a zero-based array of TField objects, each of which corresponds to a field in the current index.

Index parameter is an ordinal value indicating the position of a field in the index.

To set dataset ordering use the IndexFieldNames (see page 600) or IndexName (see page 601) properties to order datasets on the fly at runtime.

The property has the meaning only for non-expressional indexes.

Syntax

```
property IndexFields [AIndex: Integer]: TField;
```

See Also

IndexFieldNames (see page 600), IndexName (see page 601), IndexFieldCount (see page 600)

1.17.2.1.5.1.31 TADDataSet.IndexName Property

Gets / sets the current index for the dataset by its name.

Description

Use IndexName to specify the current index for the dataset.

If IndexName is empty, the dataset's sort order is based on the IndexFieldNames (see page 600) property or on its default ordering as it is in dataset. If IndexName contains a name of valid index from Indexes collection, then that index is used to determine data view (including sort order) of records.

IndexFieldNames (see page 600) and IndexName are mutually exclusive. Setting one clears the other.

Syntax

```
property IndexName: String;
```

See Also

Sorting Records (see page 100), Indexes (see page 598), IndexFieldNames (see page 600)

Example

```
with ADMemTable1.Indexes.Add do begin
  Name := 'by_name';
  Fields := 'NAME;DT';
  Active := True;
end;
ADMemTable1.IndexName := 'by_name';
```

1.17.2.1.5.1.32 TADDataSet.KeyExclusive Property

Controls the exclusion of upper and lower boundaries for a range.

Description

Use KeyExclusive to get / set the flag controlling should a range include or exclude the records that exactly match starting and ending values range values. By default, KeyExclusive is False, meaning that exact matching values are included.

To restrict a range to those records that are greater than the specified starting value and less than the specified ending value, set KeyExclusive to True. Setting of KeyExclusive is possible only in dsSetKeyMode.

The property has the meaning only for non-expressional indexes.

Syntax

```
property KeyExclusive: Boolean;
```

See Also

Filtering Records (see page 102), FindNearest (see page 626), GotoNearest (see page 631), SetRange (see page 647), SetRangeEnd (see page 648), SetRangeStart (see page 648)

1.17.2.1.5.1.33 TADDataSet.KeyFieldCount Property

Controls the number of indexed field values to use for key search or range setup.

Description

Use KeyFieldCount to get / set the number of indexed field values to use for key searching or ranges setup. The default value - all indexed fields.

The KeyFieldCount must be less than or equal to the number of indexed fields - IndexFieldCount (see page 600). If specified, then FireDAC will use the first KeyFieldCount indexed fields for searching, instead of IndexFieldCount (see page 600) fields. Setting of KeyFieldCount is possible only in dsSetKeyMode.

The property has the meaning only for non-expressional indexes.

Syntax

```
property KeyFieldCount: Integer;
```

See Also

Filtering Records (see page 102), FindKey (see page 625), FindNearest (see page 626), GotoKey (see page 630), GotoNearest (see page 631)

Example 1

```
ADQuery1.IndexFieldNames := 'EmployeeID;CustomerID;SellDate';
ADQuery1.SetKey;
ADQuery1.KeyFieldCount := 2;
ADQuery1.FieldByName('EmployeeID').AsInteger := 100;
ADQuery1.FieldByName('CustomerID').AsInteger := 200;
ADQuery1.GotoKey;
```

Example 2

```
ADQuery1.IndexFieldNames := 'EmployeeID;CustomerID;SellDate';
ADQuery1.SetRangeStart;
ADQuery1.KeyFieldCount := 1;
ADQuery1.FieldByName('EmployeeID').AsInteger := 100;
ADQuery1.SetRangeEnd;
ADQuery1.KeyFieldCount := 1;
ADQuery1.FieldByName('EmployeeID').AsInteger := 100;
ADQuery1.ApplyRange;
```

1.17.2.1.5.1.34 TADDataSet.MasterFields Property

Gets / sets fields in master dataset, used to establish a master-detail relationship with this dataset.

Description

Use MasterFields property value to specify semicolon separated list of the master dataset field names, used to establish a master-detail relationship with this dataset.

Specify MasterFields property value after assigning MasterSource (see page 603) property value. The mechanism used to filter this detail dataset records depends on the dataset class:

- TADQuery (see page 476) and TADStoredProc (see page 514) will match master fields to this query / procedure parameters by their names. When the master dataset current record is changed, then this dataset parameters will get values from the corresponding MasterFields.
- TADMemTable (see page 436) will match master fields to this dataset indexed fields by their positions. When the master dataset current record is changed, then a range will be applied to this dataset, where starting and ending range values are equal to the corresponding MasterFields field values. The application must have a current active index. The indexed fields will be the detail dataset fields used to establish mater-detail.

Syntax

```
property MasterFields: String;
```

See Also

Master-Detail Relationship (see page 104), MasterSource (see page 603), DetailFields (see page 596), IndexFieldNames (see page 600), IndexName (see page 601)

Example

```
ADMemTable1.IndexFieldNames := 'CustomerID';
ADMemTable1.MasterSource := CustomersDS;
ADMemTable1.MasterFields := 'ID';
```

1.17.2.1.5.1.35 TADDataSet.MasterLink Property

Returns data link object servicing connection with master dataset.

Description

The MasterLink property returns a reference to TADMasterLink object, which services master-detail link.

Syntax

```
property MasterLink: TADMasterDataLink;
```

See Also

Master-Detail Relationship (see page 104)

1.17.2.1.5.1.36 TADDataSet.MasterSource Property

Gets / sets a master data source, used to establish a master-detail relationship with this dataset.

Description

Use MasterSource to specify the data source linked to a dataset, which will be used as master dataset in master-detail relationship between this and specified datasets.

Additionally application must specify MasterFields (see page 602) property value. See MasterFields (see page 602) property description for additional information.

Syntax

```
property MasterSource: TDataSource;
```

See Also

Master-Detail Relationship (see page 104), MasterFields (see page 602), IndexFieldNames (see page 600), IndexName (see page 601)

Example 1

```
ADMemTable1.IndexFieldNames := 'CustomerID';
ADMemTable1.MasterSource := CustomersDS;
```

```
ADMemTable1.MasterFields := 'ID';
```

Example 2

See FireDAC\Samples\Comp Layer\TADQuery (see page 476)\MasterDetail demo for details.

1.17.2.1.5.1.37 TADDataSet.OnMasterSetValues Event

Fires after a detailed dataset has set detailed query parameters.

Description

Use the OnMasterSetValues event handler to override parameter values supplied to detail dataset from the master dataset. Also, because BeforeOpen and AfterOpen events do not fire for a detail dataset, the OnMasterSetValues may be used instead of them.

Syntax

```
property OnMasterSetValues: TADDataSetEvent;
```

See Also

Master-Detail Relationship (see page 104), MasterSource (see page 603), MasterFields (see page 602)

1.17.2.1.5.1.38 TADDataSet.OnReconcileError Event

Fires when a dataset needs to reconcile an update to a record that cannot not be applied.

Description

Use the OnReconcileError event handler to respond to error conditions that arise when the ApplyUpdates (see page 611) method was applying changes to the records to the database. The OnReconcileError event handler is called by Reconcile (see page 642) method.

As part of Reconcile call the dataset will loop through all records with associated exception objects. And for each such record will call the OnReconcileError event handler. The event handler gets:

- E - reference to the exception object. See Handling Errors (see page 44) topics for how to work with exception objects.
- UpdateKind - the update kind of the current record. It may be one of the values: rsInserted, rsDeleted, rsModified, rsUnchanged.

After handling error the event handler should set Action argument. The default value is raMerge. The possible values are:

Action	Description
raSkip	Just skip current record.
raAbort	Just quit from Reconcile call.
raMerge	Clear the current record error state, make current record changes the new initial state of this record. IOW, merge changes into dataset records cache.
raCorrect	Clear current record error state. IOW, mark the record as correctly applied.
raCancel	Cancel current record changes.
raRefresh	Cancel current record changes and reread the record values from the database.

The event handler may analyze the original and current field values, by reading TField.OldValue and NewValue properties. Application may also update the current field value, set Action to raCorrect, and later call ApplyUdpates again.

Syntax

```
property OnReconcileError: TADReconcileErrorEvent;
```

See Also

Caching Updates (see page 114), Reconcile (see page 642), ApplyUpdates (see page 611), OnUpdateError (see page 605)

Example

```
procedure TForm1.ADMemTable1ReconcileError(DataSet: TADDataSet; E: EADException;
  UpdateKind: TADDatSRowState; var Action: TADDAptReconcileAction);
begin
  if (UpdateKind = rsInserted) and (E is EADDBEngineException) and
(EADDBEngineException(E).Kind = ekUKViolated) then begin
    DataSet.FieldByName('ID').AsInteger := GetNextFreeID;
    Action := raCorrect;
  end;
end;
```

1.17.2.1.5.1.39 TADDataSet.OnUpdateError Event

Fires if an exception is generated when updates are applied to a database.

Description

Use the OnUpdateError event handler to respond to exceptions raised while applying immediate or cached updates to a database.

ASender is the dataset to which updates are applied.

AException is a pointer to a EADException (see page 836) object from which an application can extract an error message and the actual cause of the error condition. The OnUpdateError handler can use this information to determine how to respond to the error condition. In most cases AException object will be of class EADDBEngineException (see page 833) or even one of it DBMS specific subclasses.

ARow is a DatS row object, repesenting erroneous record in dataset. This record is also current record in ASender.

ARequest indicates whether the error occurred while inserting, deleting, or modifying a record.

AAction indicates the action to take when the OnUpdateError handler exits. On entry into the handler, AAction is always set to eaDefault, which leads to uaFail, if not changed. If OnUpdateError can handle or correct the error, set AAction to uaRetry before exiting the error handler. Or consider other options.

The error handler can use the OldValue and NewValue properties TField to evaluate error conditions and set NewValue to a new value to reapply. In this case, set AAction to uaRetry before exiting.

The code in an OnUpdateError handler must not call any methods that make a different record the current one !

Syntax

```
property OnUpdateError: TADUpdateErrorEvent;
```

See Also

Caching Updates (see page 114), ApplyUpdates (see page 611), CachedUpdates (see page 592), Post, Delete, EADDBEngineException (see page 833), OnUpdateRecord (see page 605)

Example

```
procedure TForm1.ADQuery1UpdateError(ASender: TDataSet; AException: EADException;
    ARow: TADDatSRow; ARequest: TADUpdateRequest; var AAction: TADErrorAction);
begin
  if (E is EADDBEngineException) and (EADDBEngineException(E).Kind = ekUKViolated) then
begin
    DataSet.FieldByName('ID').AsInteger := GetNextFreeID;
    Action := eaRetry;
  end;
end;
```

1.17.2.1.5.1.40 TADDataSet.OnUpdateRecord Event

Occurs when record update is applying to a database.

Description

Use an OnUpdateRecord event handler to process immediate or cached updates that cannot be correctly or easily handled:

- using FireDAC automatic SQL update command generation (see page 121);

- by a single table adapter component;
- by a single update component.

This event is useful for applications that require:

- additional control over parameters and macros substitution in update components;
- multi SQL statement updates, when DBMS does not support batches or blocks;
- non-SQL updates posting;
- etc

ASender is the dataset to which updates are applied.

ARequest indicates whether the current update is the insertion, deletion or modification of a record.

AAction indicates the action taken by the OnUpdateRecord handler before it exits. On entry into the handler, AAction is always set to eaDefault. If OnUpdateRecord is successful, it should set AAction to eaApplied before exiting.

AAction value	Description
eaFail	Mark the update as failed and return an error.
eaSkip	Skip current update and do not mark it as applied.
eaRetry	Retry the current operation.
eaApplied	Mark current update as applied.
eaDefault	Take default action. For successful update it is eaApplied, for failed update it is eaFail.
eaExitSuccess	Stop to process the cached updates, return success.
eaExitFailure	Stop to process the cached updates, return failure.

The OnUpdateRecord event handler code should read the dataset field values, including TField.NewValue, OldValue and CurValue. There:

- NewValue is the new field value before posting updates.
- OldValue is the original field value, as it was after record fetching or after last updates applying.
- CurValue is the current field value, the same as Value property.

Note, the OnUpdateRecord handler code must not call the methods that change the current dataset position.

Syntax

```
property OnUpdateRecord: TADUpdateRecordEvent;
```

See Also

Overriding Posting Updates (see page 123), ApplyUpdates (see page 611), CachedUpdates (see page 592), Post, Delete, OnUpdateError (see page 605)

Example 1

```
procedure TForm1.ADQuery1UpdateRecord(ASender: TDataSet; ARequest: TADUpdateRequest;
 var AAction: TADErrorAction; AOptions: TADUpdateRowOptions);
begin
  if ARequest = arInsert then begin
    // set the SQL command to insert new record
    ADQuery2.SQL := 'insert into mytab (id, code, name) values (:id, :code, :name)
returning tmstamp into :ts';

    // set parameter values
    ADQuery2.Params[0].Value := ASender['id'];
    ADQuery2.Params[1].Value := ASender['code'];
    ADQuery2.Params[2].Value := ASender['name'];

    // specially define TS parameter
    ADQuery2.Params[3].DataType := ftDateTime;
    ADQuery2.Params[3].ParamType := ptOutput;
```

```
      // insert new record
      ADQuery2.ExecSQL;

      // move TS output parameter value back to dataset
      ASender['tmstamp'] := ADQuery2.Params[3].Value;

      // return 'Ok' status
      AAction := eaApplied;
    end;
  end;
```

Example 2

See demos:

- FireDAC\Samples\Comp Layer\TADQuery (see page 476)\CachedUpdates (see page 592)\OnUpdateRecord
- FireDAC\Samples\Comp Layer\TADUpdateSQL (see page 563)\Main

1.17.2.1.5.1.41 TADDataSet.ParentDataSet Property

Returns the reference to parent dataset, if this one is nested.

Description

The ParentDataSet property returns reference to parent dataset, if this dataset is nested.

Syntax

```
property ParentDataSet: TADDataSet;
```

See Also

DB.TDataSetField

1.17.2.1.5.1.42 TADDataSet.RowError Property

Returns an exception object associated with the current dataset record.

Description

Use the RowError property to get an exception object associated with the current dataset record. If record does not have an error, then the value will be nil. The property is useful when CachedUpdates (see page 592)=True. It may be used together with FilterChanges (see page 597) and rtHasErrors.

Syntax

```
property RowError: EADException;
```

See Also

Caching Updates (see page 114), CachedUpdates (see page 592), FilterChanges (see page 597)

Example

```
var
  oErr: EADException;
...
if ADQuery1.ApplyUpdate > 0 then begin
  ADQuery1.FilterChanges := [rtModified, rtInserted, rtDeleted, rtHasErrors];
  ADQuery1.First;
  while not ADQuery1.Eof do begin
    oErr := ADQuery1.RowError;
    // process exception object
    ADQuery1.Next;
  end;
  ADQuery1.FilterChanges := [rtUnmodified, rtModified, rtInserted];
end;
```

1.17.2.1.5.1.43 TADDataSet.SavePoint Property

Gets / sets the change log version number.

Description

Use SavePoint to get the change log current version number. And later revert change log to the specified version number.

SavePoint may be used only in CachedUpdates (see page 592) mode. SavePoint's may be usefull for applications that use:

- undo functionality. Although there is no standard ability to implement redo.
- in-memory transactions.

Datasets can't return to a SavePoint once the change log has been backed out to a prior state. This includes:

- undoing changes by setting SavePoint to a previously saved value;
- backing out too many changes using the RevertRecord (see page 644) method;
- clearing the change log using CancelUpdates (see page 614) or CommitUpdates (see page 615).

Trying to set SavePoint to a state that is no longer available in the change log raises an exception.

Syntax

```
property SavePoint: Integer;
```

See Also

CachedUpdates (see page 592), ChangeCount (see page 593), UndoLastChange (see page 649), RevertRecord (see page 644), CancelUpdates (see page 614)

Example 1

```
var
  iSavePoint: Integer;
....
  // remember changes log state
  iSavePoint := ADMemTable1.SavePoint;
  try
    ADMemTable1.AppendRecord(....);
    ADMemTable1.AppendRecord(....);
    ADMemTable1.AppendRecord(....);
    ...
    ADMemTable1.Delete;
    // mark changes as persistent and clear change log
    ADMemTable1.CommitUpdates;
  except
    // in case of exception, undo changes
    ADMemTable1.SavePoint := iSavePoint;
    raise;
  end;
```

Example 2

See FireDAC\Samples\Comp Layer\TADMemTable (see page 436)\CachedUpdates (see page 592) sample for details.

1.17.2.1.5.1.44 TADDataSet.SourceEOF Property

Specifies if no more records to fetch from current cursor.

Description

The SourceEOF property returns True, if there is no more records to fetch from current cursor.

Syntax

```
property SourceEOF: Boolean;
```

See Also

Next, FetchAll (see page 624)

1.17.2.1.5.1.45 TADDataSet.SourceView Property

The reference to DatS view object, by which rows the dataset is navigating.

Description

The SourceView property returns reference to TADDatSView object. The dataset is navigating on this view rows. The view is build by FireDAC automatically as result of applying of filters, indexes and ranges. The view will be destroyed after dataset closing.

Syntax

```
property SourceView: TADDatSView;
```

See Also

Table (see page 609), View (see page 610), uADDatSManager.TADDatSView

Example

```
var
  i, iVal, iMax: Integer;
....
// searching the maximum value of ID integer field, without navigating through dataset
iMax := 0;
for i := 0 to ADQuery1.SourceView.Rows.Count - 1 do
  if ADQuery1.SourceView.Rows[i].GetData('id', @iVal) and (iVal > iMax) then
    iMax := iVal;
```

1.17.2.1.5.1.46 TADDataSet.Table Property

The reference to DatS table object, actually representing the internal data storage.

Description

The Table property returns reference to TADDatSTable object. It is the internal data storage containing all dataset rows.

FireDAC creates table for each dataset automatically at Open. TADCustomMemTable (see page 388) allows to attach to existing table before Open call, using AttachTable (see page 612) method. If table was automatically created, then it will be destroyed after dataset closing.

As result of applying of filters, indexes and ranges, FireDAC builds views for this table object. All dataset navigation methods does not work with table, but with SourceView (see page 608) view object.

Syntax

```
property Table: TADDatSTable;
```

See Also

View (see page 610), SourceView (see page 608), AttachTable (see page 612), TADCustomMemTable (see page 388)

Example

```
var
  i: Integer;
....
// delete all rows, where ID = 100, without navigating through dataset
for i := ADQuery1.Table.Rows.Count - 1 downto 0 do
  if ADQuery1.Table.Rows[i].GetData('id') = 100 then
    ADQuery1.Table.Rows[i].Delete;
ADQuery1.Resync([]);
```

1.17.2.1.5.1.47 TADDataSet.UpdatesPending Property

Indicates whether the changes log is not empty.

Description

Examine UpdatesPending to check the status of the changes log. If UpdatesPending is true, then there are edited, deleted, or inserted records to apply to the database. If UpdatesPending is false, there are no records in the changes log.

Syntax

```
property UpdatesPending: Boolean;
```

See Also

Caching Updates (see page 114), CachedUpdates (see page 592), ChangeCount (see page 593), ApplyUpdates (see page 611), CommitUpdates (see page 615), CancelUpdates (see page 614)

1.17.2.1.5.1.48 TADDataSet.View Property

The reference to base DatS view object.

Description

The View property returns reference to TADDatSView object. It is the base view object, specified as second argument to AttachTable (see page 612) method call. As result of applying of filters, indexes and ranges, FireDAC builds views above of this view object.

Syntax

```
property View: TADDatSView;
```

See Also

Table (see page 609), SourceView (see page 608), AttachTable (see page 612), TADCustomMemTable (see page 388)

1.17.2.1.5.1.49 TADDataSet.AddIndex Method

Creates a new client index for the dataset.

Parameters

Parameters	Description
const AName: string	A name of the new index.
const AFields: string	A semicolon-delimited list of the fields to include in the index.
const AExpression: string	A sorting expression.
AOptions: TADSortOptions	A set of additional options for index.
const ADescFields: string = ''	A semicolon-delimited list of field names. Use ADescFields instead of an AOptions value that includes soDescending to create an index that is in ascending order on some fields and descending order on others. All fields named in ADescFields are sorted in descending order.
const ACaseInsFields: string = ''	A semicolon-delimited list of field names. Use ACaseInsFields instead of an AOptions value that includes soNoCase to create an index that is case-insensitive on some fields and case-sensitive on others. All fields named in ACaseInsFields are sorted without regard to case.
ADistinct: Boolean = False	A True, if you need to get only records with the distinct AFields / AExpression values. A False, if all records. False is the default value.

Description

Call AddIndex to create a new index for the client dataset.

The method corresponds to manual adding of new index to Indexes (see page 598) property and setting properties of new index. If index with AName name already exists, then exception will be raised. AField and AExpression properties are mutually exclusive. The new index will be active, but not current.

Syntax

```
procedure AddIndex(const AName: string; const AFields: string; const AExpression: string;
AOptions: TADSortOptions; const ADescFields: string = ''; const ACaseInsFields: string =
''; ADistinct: Boolean = False);
```

See Also

Indexes (see page 598)

1.17.2.1.5.1.50 TADDataSet.ApplyMaster Method

Applies a master-detail relation to this detail dataset.

Description

Use the ApplyMaster method to synchronize this detail dataset with the current master record.

This method is useful, when DisableControls was called for the master dataset. Or scrolling is disabled by the MasterLink.DisableScroll (see page 664).

Syntax

```
procedure ApplyMaster;
```

See Also

Master-Detail Relationship (see page 104), TDataSet.DisableControls, TADMasterLink.DisableScroll (see page 664), TADMasterLink.Synchronize (see page 665)

1.17.2.1.5.1.51 TADDataSet.ApplyRange Method

Applies a range to the dataset.

Description

Use ApplyRange to actually filter dataset records using range values specified with SetRangeStart (see page 648) and SetRangeEnd (see page 648), or EditRangeStart (see page 621) and EditRangeEnd (see page 620) method calls.

When a range is in effect, only those records that fall within the range are available to the application for navigating, viewing and editing. After a call to ApplyRange, the cursor is left on the first record in the range.

Syntax

```
procedure ApplyRange;
```

See Also

Filtering Records (see page 102), CancelRange (see page 613), EditRangeEnd (see page 620), EditRangeStart (see page 621), SetRange (see page 647), SetRangeEnd (see page 648), SetRangeStart (see page 648), IsRanged (see page 632)

Example

```
ADQuery1.IndexFieldNames := 'CUST_NO';
ADQuery1.SetRangeStart;
ADQuery1['CUST_NO'] := '100';
ADQuery1.SetRangeEnd;
ADQuery1['CUST_NO'] := '200';
ADQuery1.ApplyRange;
```

1.17.2.1.5.1.52 TADDataSet.ApplyUpdates Method

Applies changes for all records in the dataset change log to the database.

Parameters

Parameters	Description
AMaxErrors: Integer = -1	A maximum number of allowed errors. Default value is -1.

Returns

The number of errors is encountered.

Description

Use ApplyUpdates to apply changes for all records in the dataset change journal to the database.

This method is useful when CachedUpdates (see page 592) = True mode. The ApplyUpdates call performs the steps:

1. Fires a BeforeApplyUpdates (see page 591) event.

2. Calls the data adapter to generate / use SQL update commands for each row and post changes to the database.
3. Fires an AfterApplyUpdates (see page 588) event.
4. Calls the dataset's Reconcile (see page 642) method to reconcile any records that has errors in step 2.
5. If there were no errors, then calls CommitUpdates (see page 615) for the TADCustomMemTable (see page 388) descendants. For the TADRdbmsDataSet (see page 501) descendants (TADQuery (see page 476), TADStoredProc (see page 514), TADTable (see page 539), etc), application should call Reconcile (see page 642) and CommitUpdates (see page 615) explicitly.

AMaxErrors indicates the maximum number of errors that the FireDAC should allow before prematurely stopping the update operation. Set AMaxErrors to –1 to indicate that there is no limit to the number of errors, or to 0 to indicate that no error is allowed.

ApplyUpdates returns the number of errors it encountered. Based on this return value and the setting of AMaxErrors, successfully applied records are removed from the dataset's change log. If the update process is aborted before all updates are applied, any unapplied updates remain in the change log.

ApplyUpdates does not raise exception. Instead application should review erroneous records using Reconcile and OnReconcileError (see page 604) event handler, or FilterChanges (see page 597) and RowError (see page 607) properties. Read "Reviewing errors" at Caching Updates (see page 114) for more details.

All changes are applied in the dataset modifications order. First modification (insert / update / delete) will be applied first, last modification will be applied last.

Syntax

```
function ApplyUpdates(AMaxErrors: Integer = -1): Integer;
```

See Also

Caching Updates (see page 114), CachedUpdates (see page 592), CancelUpdates (see page 614), CommitUpdates (see page 615), RowError (see page 607), FilterChanges (see page 597), Reconcile (see page 642), UpdatesPending (see page 609)

Example 1

```
procedure TForm1.btnApplyClick(ASender: TObject);
begin
  if ADStoredProc1.UpdatesPending then
    ADStoredProc1.ApplyUpdates;
end;
```

Example 2

See FireDAC\Samples\Comp Layer\TADMemTable (see page 436)\CachedUpdates (see page 592) sample for details.

1.17.2.1.5.1.53 TADDataSet.AttachTable Method

Attaches dataset to existing DatS table and optionaly view.

Parameters

Parameters	Description
ATable: TADDatSTable	A DatS table, storing data.
AView: TADDatSView	A DatS view, representing rows.

Description

Use the AttachTable method to attach dataset to existing DatS table and optionally view to navigate, visualize and edit their data using standard TDataSet interface. Call dataset Open method to activate dataset after attaching to DatS objects. Before call theyr structure must be filled in. To detach specified DatS objects from dataset, call AttachTable with nils.

The application is responsible for creation and destruction of specified table and view.

This method has meaning for TADCustomMemTable (see page 388) descendants only. And it is used internally only for other FireDAC datasets.

Syntax

```
procedure AttachTable(ATable: TADDatSTable; AView: TADDatSView); virtual;
```

See Also

Table (see page 609), View (see page 610), SourceView (see page 608)

Example

```
var
  oTab: TADDatSTable;
...
  oTab := TADDatSTable.Create;
  oTab.Columns.Add('id', dtInt32);
  oTab.Columns.Add('name').Size := 13;
  oTab.Columns.Add('cnt', dtInt16);
  oTab.Columns.Add('price', dtCurrency);
  ADMemTable1.Attach(oTab, nil);
  ADMemTable1.Open;
  // working with ADMemTable1
  ADMemTable1.AttachTable(nil, nil);
  oTab.Free;
```

1.17.2.1.5.1.54 TADDataSet.BeginBatch Method

Optimizes dataset functionality for batch updates.

Parameters

Parameters	Description
AWithDelete: Boolean = False	True if after call application will also delete records.

Description

Use BeginBatch method to disable resources consuming operations and setup dataset to perform maximum fast updates. The BeginBatch call disables:

- data-aware controls refreshing;
- constraints updating;
- aggregates maintaining;
- wait cursor showing;
- similar resources consuming processing in DatS objects, which is not controlled through dataset API.

If batch algorithm calls Delete method, then set AWithDelete to True. It is prohibited to call Refresh inside of a batch. The BeginBatch calls cannot be nested, although FireDAC will not raise exception on consequent calls. After finishing updates call EndBatch (see page 622).

Syntax

```
procedure BeginBatch(AWithDelete: Boolean = False);
```

See Also

EndBatch (see page 622)

Example

```
ADQuery1.BeginBatch;
try
  ADQuery1.AppendRecord([...]);
  ADQuery1.AppendRecord([...]);
  ADQuery1.AppendRecord([...]);
  ......
finally
  ADQuery1.EndBatch;
end;
```

1.17.2.1.5.1.55 TADDataSet.CancelRange Method

Cancels range for the dataset.

Description

Use CancelRange to remove a range currently applied to a dataset.

Canceling a range re-enables access to all records in the dataset, excluding ones, that may be:

- filtered out by Filtered / OnFilterRecord / FilterChanges (see page 597);
- filtered out by Indexes (see page 598) with Filter expression specified.

Syntax

```
procedure CancelRange;
```

See Also

Filtering Records (see page 102), ApplyRange (see page 611), EditRangeEnd (see page 620), EditRangeStart (see page 621), SetRange (see page 647), SetRangeEnd (see page 648), SetRangeStart (see page 648), IsRanged (see page 632)

1.17.2.1.5.1.56 TADDataSet.CancelUpdates Method

Removes all records from the change log and restores the dataset rows to its prior editing state.

Description

Use CancelUpdates to remove all records from the change log, while the dataset is in CachedUpdates (see page 592) = True mode, and restore the dataset records to the state they was in when the dataset was opened, cached updates were last enabled, or updates were last successfully applied to the database.

When a dataset is closed, or the CachedUpdates (see page 592) property is set to false, CancelUpdates is called automatically.

Note, to undo changes to a single record, call RevertRecord (see page 644).

Syntax

```
procedure CancelUpdates;
```

See Also

Caching Updates (see page 114), ApplyUpdates (see page 611), CachedUpdates (see page 592), CommitUpdates (see page 615), UndoLastChange (see page 649), SavePoint (see page 607)

Example

See FireDAC\Samples\Comp Layer\TADMemTable (see page 436)\CachedUpdates (see page 592) sample for details.

1.17.2.1.5.1.57 TADDataSet.CloneCursor Method

Shares the data belonging to another dataset.

Parameters

Parameters	Description
ASource: TADDataSet	A source dataset to clone.
AReset: Boolean = False	A flag to reset properties for this dataset.
AKeepSettings: Boolean = False	A flag to keep current setting for this dataset.

Description

Use CloneCursor to share the data belonging to another dataset with this dataset. After calling CloneCursor, the internal data storage is physically the same and shared for this dataset and for a ASource dataset.

ASource references to a dataset whose data will be shared with this one.

AReset and AKeepSettings determine how to set the values of the following properties and events:

- Filter, Filtered, FilterOptions, OnFilterRecord and FilterChanges (see page 597);
- IndexName (see page 601), IndexFieldNames (see page 600);

- MasterSource (see page 603) and MasterFields (see page 602);
- ReadOnly.

If AReset and AKeepSettings are both False, the values of the properties listed above are all set to match the source dataset.

If AReset is True, the properties listed above are all set to the default values.

If AReset is False and AKeepSettings is True, the properties listed above are not changed. In this case, the application must ensure that existing indexes, filters, and so on are compatible with the cloned data.

CloneCursor does not clone:

- persistent fields;
- adapter / update object;
- event handlers, excluding OnFilterRecord.

Syntax

```
procedure CloneCursor(ASource: TADDataSet; AReset: Boolean = False; AKeepSettings: Boolean
= False); virtual;
```

See Also

Data (see page 595)

Example 1

```
ADMemTable1.CloneCursor(ADQuery1, True, False);
ADMemTable1.Locate(...);
ADMemTable1.Edit;
...
```

Example 2

See FireDAC\Samples\Comp Layer\TADMemTable (see page 436)\CloneCursor sample for details.

1.17.2.1.5.1.58 TADDataSet.CommitUpdates Method

Clears the changes log and marks all records as not modified.

Description

Call CommitUpdates to clear the dataset changes log, working in CachedUpdates (see page 592) = True mode, after both a successful call to ApplyUpdates (see page 611) and a database component's Commit method. And marks all dataset record as unchanged.

Applications that use a connection component's ApplyUpdates (see page 611) method to apply and commit pending updates for all datasets associated with the database component do not need to call CommitUpdates.

Syntax

```
procedure CommitUpdates;
```

See Also

CachedUpdates (see page 592), UpdatesPending (see page 609), OnUpdateError (see page 605), OnUpdateRecord (see page 605), ApplyUpdates (see page 611), CancelUpdates (see page 614)

1.17.2.1.5.1.59 TADDataSet.CopyDataSet Method

Copies records from source dataset to this dataset.

Parameters

Parameters	Description
ASource: TDataset	A source dataset.
AOptions: TADCopyDataSetOptions = [coRestart, coAppend]	A copy options. The default value is [coRestart, coAppend].

Description

Use the CopyDataSet method to copy the records from an ASource dataset to this dataset.

The method performs steps:

- copies ASource dataset structure into Self dataset, if **coStructure** is in AOptions, and:
  - copies ASource dataset indexes setup into Self dataset, if **coIndexesCopy** is in AOptions. Or resets Self indexes setup, if **coIndexesReset** is in AOptions;
  - copies ASource dataset aggregates setup into Self dataset, if **coAggregatesCopy** is in AOptions. Or resets Self aggregates setup, if **coAggregatesReset** is in AOptions;
  - copies ASource dataset constraints setup into Self dataset, if **coConstraintsCopy** is in AOptions. Or resets Self constraints setup, if **coConstraintsReset** is in AOptions;
- opens Self dataset, if it is inactive;
- sets a ASource to a beginning, if **soRestart** is in AOptions;
- for each record in ASource do:
  - if **coEdit** is in AOptions, then locate corresponding record in Self dataset, using primary key field values;
  - if **coAppend** is in AOptions, then append new record to Self dataset;
  - copies all fields, which names exists in both datasets and which are modifiable. If ASource field is not found in this dataset, then it will be skipped. It is important that fields with the same name in both datasets must be compatible by data types. The CopyDataSet method does not verify, that fields are assignment compatible.
  - post the record change and select next record in ASource.

Use **coRefresh** instead of coEdit to change records only in a local cache. And do not post changes to a database. This option may be used to refresh this dataset using records from ASource dataset.

This method is similar to assigning to Data property. The differences are:

- CopyDataSet can copy from non-FireDAC dataset. Data property allows to copy data only from an FireDAC dataset.
- CopyDataSet works through TDataSet API, firing appropriate events. Data property works through DatS API, no events are fired.
- CopyDataSet copies only current field values. Data property copies all record field versions and preserves row state (inserted, deleted, updated or unchanged).
- Assigning to Data is much faster, than CopyDataSet.

Syntax

```
procedure CopyDataSet(ASource: TDataset; AOptions: TADCopyDataSetOptions = [coRestart,
coAppend]);
```

See Also

CopyRecord (see page 616), Data (see page 595), TADDataMove

Example

```
// copies dataset structure and all records
ADMemTable1.CopyDataSet(Query1, [coStructure, coRestart, coAppend]);
// refreshed dataset data using records from ADQuery2 dataset. If a record is not found,
then it will be appended
ADQuery1.CopyDataSet(ADQuery2, [coRestart, coRefresh, coAppend]);
```

1.17.2.1.5.1.60 TADDataSet.CopyRecord Method

Copies all record fields from source dataset to the Self dataset.

Parameters

Parameters	Description
ASource: TDataset	A source dataset.

Description

Use CopyRecord method to copy the current record field values from an ASource dataset to the Self dataset. The method copies all fields, for which the following requirements are met:

- the names exists in both datasets;
- a field in the Self dataset is not calculated;
- a field in the Self dataset is not read-only.

It is important that fields with the same name in both datasets must be compatible by data types. The CopyRecord method does not verify, that fields are assignment compatible.

Syntax

```
procedure CopyRecord(ASource: TDataset);
```

See Also

CopyDataSet (see page 615), Data (see page 595), TADDataMove

1.17.2.1.5.1.61 TADDataSet.CreateDataSet Method

Creates new empty internal data storage for dataset.

Description

Use CreateDataSet at runtime to create a empty internal data storage for dataset and activate dataset.

After call application may populate dataset by data and optionally save data to the external file. Optionally dataset may be created setting Active to True.

If the FieldDefs property contains items, then these values are used to create field definitions. Otherwise the Fields property is used. One or both of these properties must contain values in order to create a dataset. If neither property is set, CreateDataSet raises an exception.

The method is not applicable for all FireDAC datasets, but for TADCustomMemTable (see page 388).

Syntax

```
procedure CreateDataSet;
```

See Also

TDataSet.FieldDefs, TDataSet.Fields, TADCustomMemTable (see page 388)

Example

```
with ADMemTable1 do begin
  FieldDefs.Add('ID', ftAutoInc);
  FieldDefs.Add('Code', ftString, 10);
  FieldDefs.Add('Name', ftString, 50);
  CreateDataSet;
  // here dataset is active, has 3 fields and is empty
end;
```

1.17.2.1.5.1.62 TADDataSet.CreateExpression Method

Creates expression evaluator for dataset.

Parameters

Parameters	Description
const AExpr: String	A expression to evaluate.
AOptions: TADExpressionOptions = []	An evaluator options.

Returns

The evaluator interface reference.

Description

Use CreateExpression method to translate string, containing expression, into binary representation and create expression evaluator. The expression may contain field names from this dataset. To create evaluator, dataset must be active.

The method returns evaluator, which may be used few times to effectively evaluate specified expression, by calling its Evaluate method. The evaluator is associated with current record. After position is changed, application must adjust evaluator:

```
oEval.DataSource.Position := ADMemTable1.GetRow;
```

The evaluator is reference counted interface. You does not need to free it explicitly. All references to evaluator must be released before dataset will be closed.

The expressions cannot contain aggregating functions, like a SUM, COUNT. FireDAC supports expression syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write expressions.

Syntax

```
function CreateExpression(const AExpr: String; AOptions: TADExpressionOptions = []):
IADStanExpressionEvaluator;
```

See Also

Writing Expressions (see page 110), GetRow (see page 629), Aggregates (see page 590), Constraints (see page 594), Indexes (see page 598)

Example

```
var
  oEval: IADStanExpressionEvaluator;
...
oEval := ADMemTable1.CreateExpresison('(sal + comm) * tax / 100');
Label1.Caption := oEval.Evaluate;
...
ADMemTable1.Next;
oEval.DataSource.Position := ADMemTable1.GetRow;
Label1.Caption := oEval.Evaluate;
...
oEval := nil;
ADMemTable1.Close;
```

1.17.2.1.5.1.63 TADDataSet.DeleteIndex Method

Deletes an client index for the dataset.

Parameters

Parameters	Description
const AName: string	An index name.

Description

Call DeleteIndex to remove an client index for a dataset from Indexes (see page 598) property.

AName is the name of the index to delete. If index is not found in Indexes (see page 598), then exception will be riased.

Syntax

```
procedure DeleteIndex(const AName: string);
```

See Also

Indexes (see page 598), AddIndex (see page 610), DeleteIndexes (see page 619)

1.17.2.1.5.1.64 TADDataSet.DeleteIndexes Method

Deletes all client indexes for the dataset.

Description

Call DeleteIndexes to remove all client indexes for a dataset from Indexes (see page 598) property.

Syntax

```
procedure DeleteIndexes;
```

See Also

Indexes (see page 598), AddIndex (see page 610), DeleteIndex (see page 618)

1.17.2.1.5.1.65 TADDataSet.DisableConstraints Method

Disables the enforcement of the client constraints.

Description

Use DisableConstraints to turn off an enforcement of the client constraints that are in Constraints (see page 594) property (record constraints) or are assigned to the dataset fields (field constraints).

Calling DisableConstraints increments an internal counter. As long as this reference count is greater than zero, constraints are disabled for the dataset. To prevent accidental disabling of constraints, always group a call to DisableConstraints with a call to EnableConstraints (see page 621).

If application needs to perform batch updates to dataset, then consider to use BeginBatch (see page 613) / EndBatch (see page 622) methods.

Syntax

```
procedure DisableConstraints;
```

See Also

EnableConstraints (see page 621), Constraints (see page 594), ConstraintsEnabled (see page 594), UpdateConstraints (see page 650)

Example

```
ADQuery1.DisableConstraints;
try
  // perform updates here, without checking client constraints
finally
  ADQuery1.EnableConstraints;
end;
```

1.17.2.1.5.1.66 TADDataSet.Disconnect Method

Use Disconnect method to release DBMS resources, used by this dataset.

Parameters

Parameters	Description
AAbortJob: Boolean = False	If True, then method will abort current operation.

Description

The Disconnect method optionally abort current operation and releases DBMS resources, used by this dataset. After the call dataset is in dsInactive state. And descendants will have Prepared (see page 505) = False.

Syntax

```
procedure Disconnect(AAbortJob: Boolean = False); virtual;
```

Example

```
ADQuery1.SQL.Text := 'select * from [Order Details]';
ADQuery1.Prepare := True;
ADQuery1.Active := True;
```

```
ADQuery1.Disconnect;
// Active = False and Prepare = False
```

1.17.2.1.5.1.67 TADDataSet.EditKey Method

Enables modification of the search key buffer.

Description

Use EditKey to put the dataset into dsSetKey state, preserving the current contents of the search key buffer.

After call the application can modify indexed fields. To iterate through them use IndexFieldCount (see page 600) and IndexFields (see page 601) properties. Optionally may be edited value of KeyFieldCount (see page 602) and/or KeyExclusive (see page 602) properties.

Syntax

```
procedure EditKey;
```

See Also

Finding a Record (see page 107), IndexFields (see page 601), FindKey (see page 625), GotoKey (see page 630), GotoNearest (see page 631), SetKey (see page 647), TDataSet.State, KeyFieldCount (see page 602), KeyExclusive (see page 602)

Example

```
ADQuery1.IndexFieldNames := 'CUSTOMER_ID;ORDER_DATE';
...
ADQuery1.EditKey;
ADQuery1.FieldByName('ORDER_DATE').AsDateTime := EncodeDate(2008, 5, 2);
if not ADQuery1.GotoKey then
  ShowMesage('Order is not found');
```

1.17.2.1.5.1.68 TADDataSet.EditRangeEnd Method

Enables changing the ending value for an existing range.

Description

Call EditRangeEnd to bring the dataset to the dsSetKey state and change the ending value for an existing range.

After call the application can modify indexed fields. To iterate through them use IndexFieldCount (see page 600) and IndexFields (see page 601) properties. Optionally may be edited value of KeyFieldCount (see page 602) and/or KeyExclusive (see page 602) properties.

After assigning a new ending value, call ApplyRange (see page 611) to activate the modified range.

Syntax

```
procedure EditRangeEnd;
```

See Also

Filtering Records (see page 102), ApplyRange (see page 611), CancelRange (see page 613), EditRangeStart (see page 621), IndexFieldNames (see page 600), IndexName (see page 601), KeyExclusive (see page 602), SetRange (see page 647), SetRangeEnd (see page 648), SetRangeStart (see page 648)

Example

```
ADQuery1.IndexFieldNames := 'CUST_NO';
ADQuery1.SetRangeStart;
ADQuery1['CUST_NO'] := 100;
ADQuery1.SetRangeEnd;
ADQuery1['CUST_NO'] := 200;
ADQuery1.ApplyRange;
......
ADQuery1.EditRangeEnd;
ADQuery1['CUST_NO'] := ADQuery1['CUST_NO'] - 1;
ADQuery1.ApplyRange;
```

1.17.2.1.5.1.69 TADDataSet.EditRangeStart Method

Enables changing the starting value for an existing range.

Description

Call EditRangeStart to bring dataset to the dsSetKey state and change the starting value for an existing range.

After call the application can modify indexed fields. To iterate through them use IndexFieldCount (see page 600) and IndexFields (see page 601) properties. Optionally may be edited value of KeyFieldCount (see page 602) and/or KeyExclusive (see page 602) properties.

After assigning a new ending value, call ApplyRange (see page 611) to activate the modified range.

Syntax

```
procedure EditRangeStart;
```

See Also

Filtering Records (see page 102), ApplyRange (see page 611), CancelRange (see page 613), EditRangeEnd (see page 620), IndexFieldNames (see page 600), IndexName (see page 601), KeyExclusive (see page 602), SetRange (see page 647), SetRangeEnd (see page 648), SetRangeStart (see page 648)

Example

```
ADQuery1.IndexFieldNames := 'CUST_NO';
ADQuery1.SetRangeStart;
ADQuery1['CUST_NO'] := 100;
ADQuery1.SetRangeEnd;
ADQuery1['CUST_NO'] := 200;
ADQuery1.ApplyRange;
......
ADQuery1.EditRangeStart;
ADQuery1['CUST_NO'] := ADQuery1['CUST_NO'] + 1;
ADQuery1.ApplyRange;
```

1.17.2.1.5.1.70 TADDataSet.EmptyDataSet Method

Removes all records from the dataset internal data storage.

Description

Call EmptyDataSet to empty the dataset by removing all records from internal data storage and changes log.

Syntax

```
procedure EmptyDataSet;
```

See Also

Table (see page 609), TDataSet.Delete

1.17.2.1.5.1.71 TADDataSet.EnableConstraints Method

Reenables enforcement of the client constraints when records are posted.

Description

Call EnableConstraints to turn on the client constraints previously disabled by a call to DisableConstraints (see page 619).

Calling EnableConstraints decrements a reference count. When this reference count is zero constraints are enabled for the dataset. To prevent accidental disabling of constraints, always follow a call to DisableConstraints (see page 619) with a matching call to EnableConstraints.

Syntax

```
procedure EnableConstraints;
```

See Also

DisableConstraints (see page 619), Constraints (see page 594), ConstraintsEnabled (see page 594),

UpdateConstraints (see page 650)

Example

```
ADQuery1.DisableConstraints;
try
  // perform updates here, without checking client constraints
finally
  ADQuery1.EnableConstraints;
end;
```

1.17.2.1.5.1.72 TADDataSet.EndBatch Method

Restores regular dataset functionality after batch updates.

Description

Use EndBatch method to re-enable regular dataset processing as it was before BeginBatch (see page 613) call.

Syntax

```
procedure EndBatch;
```

See Also

BeginBatch (see page 613)

Example

```
ADQuery1.BeginBatch;
try
  ADQuery1.AppendRecord([...]);
  ADQuery1.AppendRecord([...]);
  ADQuery1.AppendRecord([...]);
  ......
finally
  ADQuery1.EndBatch;
end;
```

1.17.2.1.5.1.73 TADDataSet.Execute Method

Executes an SQL command.

Parameters

Parameters	Description
ATimes: Integer = 0	A parameters array size.
AOffset: Integer = 0	A offset in parameters array.

Description

Call Execute to execute a SQL statement currently assigned to descendant class properties:

- TADCustomQuery (see page 394).SQL (see page 396);
- TADCustomStoredProc (see page 403).StoredProcName (see page 406);
- TADCustomMemTable (see page 388).Adapter (see page 389).SelectCommand.CommandText (see page 298). Or command assigned to TADCustomCommand.CommandText of command object. Which is assigned to TADCustomTableAdapter.SelectCommand. And adapter object is assigned to TADAdaptedDataSet.Adapter.

Use Execute to execute queries that do not return a cursor to data (such as INSERT, UPDATE, DELETE, CREATE TABLE, CALL, BEGIN .. END, etc). For SELECT statements and others, returning cursors, call Open instead of Execute.

Execute prepares the SQL statement for execution, if it has not already been prepared. To speed performance, an application should ordinarily call Prepare method before calling Execute for the first time.

Use ResourceOptions (see page 652).CmdExecMode (see page 880) to control asynchronous execution mode. And ResourceOptions (see page 652).CmdExecTimeout (see page 881) to set maximum command execution time. After that time command execution will be canceled and exception raised.

To cancel command execution use AbortJob (see page 263).

Before command execution the BeforeExecute (see page 591) event is fired. If server will return command execution error, then FireDAC raises an exception. It may be analyzed in OnError (see page 261) event. After command execution is finished, any output parameters are put into Params (see page 651) property and AfterExecute (see page 589) event is fired.

If ATimes is equal to zero or one, then command is executing in standard mode, otherwise in Array DML mode. See Array DML (see page 82) article for details. By default ATimes and AOffset are equal to 0. ATimes specifies parameters array size. AOffset specifies from which row index in parameters array Array DML command execution should be started. ATimes must be less or equal to Params (see page 651).ArraySize, otherwise exeception will be raised.

After command execution application may check RowsAffected (see page 513) property value to see how much database records were affected by last command execution.

Syntax

```
procedure Execute(ATimes: Integer = 0; AOffset: Integer = 0); virtual;
```

See Also

Executing Command (see page 66), Array DML (see page 82), TADCustomQuery.SQL (see page 396), TADCustomQuery.ExecSQL (see page 397), TADCustomStoredProc.StoredProcName (see page 406), TADCustomStoredProc.ExecProc (see page 410), TADAdaptedDataSet.AbortJob (see page 263), TADCustomMemTable.Adapter (see page 389), TADRdbmsDataSet.Prepare (see page 511), TADRdbmsDataSet.OnError (see page 504), BeforeExecute (see page 591), AfterExecute (see page 589)

Example 1

```
var
  i: Integer;
....
ADQuery1.SQL.Text := 'insert into mytab values (:p1, :p2)';
ADQuery1.Params.ArraySize := 10;
for i := 0 to ADQuery1.Params.ArraySize - 1 do begin
  ADQuery1.Params[0].AsIntegers[i] := i;
  ADQuery1.Params[1].AsStrings[i] := 'qwe';
end;
ADQuery1.Execute(ADQuery1.Params.ArraySize, 0);
```

Example 2

See demos for details:

- FireDAC\Samples\Comp Layer\TADQuery (see page 476)\ExecSQL\Batch;
- FireDAC\Samples\Comp Layer\TADQuery (see page 476)\ExecSQL\BatchErrorHandling.

1.17.2.1.5.1.74 TADDataSet.FetchAgain Method

Allows to fetch additional records.

Description

Call FetchAgain method to reset "all-data-fetched" flag - SourceEOF (see page 608). This is useful for TADCustomMemTable (see page 388) linked to table adapter and allows to fetch few result sets with the same structure into single dataset. After calling FetchAgain TADCustomTableAdapter.SelectCommand must be re-executed with different parameter values.

The similar thing may be performed for TADCustomQuery (see page 394) or TADCustomStoredProc (see page 403), for example. Just must be reexecuted internal command object TADAdaptedDataSet (see page 259).Command (see page 261).

Syntax

```
procedure FetchAgain;
```

See Also

FetchAll (see page 624), TDataSet.Open, TADCustomCommand (see page 291), TADCustomTableAdapter

Example

```
// set SQL command
ADCommand1.CommandText := 'select * from orders where customerid = :cid';
// link command to table adapter
ADTableAdapter1.SelectCommand := ADCommand1;
// link table adapter to memtable
ADMemTable1.Adapter := ADTableAdapter1;

// set SQL command parameter value, then execute command and fetch all records
ADCommand1.Params[0].AsInteger := 100;
ADMemTable1.Open;
ADMemTable1.FetchAll;

// reset SourceEOF flag, reexecute command by hands and fetch all records
ADMemTable1.FetchAgain;
ADCommand1.Close;
ADCommand1.Params[0].AsInteger := 200;
ADCommand1.Open;
ADMemTable1.FetchAll;

// here we will have in memtable orders for customers with ID=100 and ID=200
```

1.17.2.1.5.1.75 TADDataSet.FetchAll Method

Retrieves all records from the current resultset and stores them into internal DatS table.

Description

Call FetchAll to retrieve all not yet fetched records from current resultset. Calling FetchAll allows:

- to release server resources faster, because FetchAll implicitly closes DBMS cursor. That does not deactivate dataset.
- to release Call Level Interface result set buffer, allowing to execute next command and get result set. That is applicable for some DBMS's, like a SQL Server and MySQL.

If to set FetchOptions (see page 651).Mode (see page 857) to fmAll, then dataset automatically calls FetchAll on Open call.

Syntax

```
procedure FetchAll;
```

See Also

TADFetchOptions.Mode (see page 857), TDataSet.Open

1.17.2.1.5.1.76 TADDataSet.FetchBlobs Method

Retrieves BLOB field values from database for current record.

Description

Call FetchBlobs to retrieve BLOB field values from a database for current dataset record, when these values are not provided automatically. BLOB field values are automatically fetched together with other record fields, if fiBlobs is included into FetchOptions (see page 651).Items (see page 855), otherwise - not.

Syntax

```
procedure FetchBlobs;
```

See Also

TADFetchOptions.Items (see page 855)

Example

```
ADQuery1.FetchOptions.Items := ADQuery1.FetchOptions.Items - [fiBlobs];
ADQuery1.Open;
....
ADQuery1.FetchBlobs;
oStr := ADQuery1.CreateBlobStream(ADQuery1.FieldByName('image'), bmRead);
try
```

```
  // process image
finally
  oStr.Free;
end;
```

1.17.2.1.5.1.77 TADDataSet.FetchDetails Method

Retrieves nested datasets from database for current record.

Parameters

Parameters	Description
ADataSetField: TDataSetField = nil	A field, whos nested dataset to fetch.

Description

Call FetchDetails to retrieve nested datasets from a database for current dataset record, when these dataset are not provided automatically. Nested dataset are automatically fetched together with other record fields, if fiDetails is included into FetchOptions (see page 651).Items (see page 855), otherwise - not.

If ADataSetField is specified, then FireDAC fetches nested dataset only for these field. Otherwise - all nested datasets.

Syntax

```
procedure FetchDetails(ADataSetField: TDataSetField = nil);
```

See Also

TADFetchOptions.Items (see page 855)

1.17.2.1.5.1.78 TADDataSet.FetchNext Method

A shortcut to the GetNextPacket (see page 629) method.

Description

Use the FetchNext method to fetch the next rowset from the DBMS. See GetNextPacket (see page 629) method description for more details.

Syntax

```
function FetchNext: Integer;
```

See Also

GetNextPacket (see page 629)

1.17.2.1.5.1.79 TADDataSet.FindKey Method

Searches for a record containing specified index field values.

Parameters

Parameters	Description
const AKeyValues: array of const	An open array of values to search.

Returns

True, if key is found. Otherwise - False.

Description

Use FindKey to search for the record in an indexed dataset, using specified index field values.

AKeyValues is an open array containing field values, called a key. Each value in the key can be a literal, a variable or nil. If the number of values passed in AKeyValues is less than the number of columns in the index used for the search, the missing values are assumed to be NULL.

If a search is successful, FindKey positions the cursor on the matching record and returns True. Otherwise the cursor is not moved, and FindKey returns False.

Syntax

```
function FindKey(const AKeyValues: array of const): Boolean;
```

See Also

Finding a Record (see page 107), FindNearest (see page 626), GotoKey (see page 630), GotoNearest (see page 631), IndexName (see page 601), IndexFieldNames (see page 600), Locate (see page 634)

Example

```
ADQuery1.IndexFieldNames := 'F1;F2';
if not ADQuery1.FindKey([100, 'qwe']) then
  ShowMessage('100;qwe is not found');
```

1.17.2.1.5.1.80 TADDataSet.FindNearest Method

Searches for a record containing specified index field values.

Parameters

Parameters	Description
const AKeyValues: array of const	An open array of values to search.

Description

Use FindNearest to search for the record in an indexed dataset, using specified index field values.

AKeyValues is an open array containing field values, called a key. Each value in the key can be a literal, a variable or nil. If the number of values passed in AKeyValues is less than the number of columns in the index used for the search, the missing values are assumed to be NULL.

If a search is successful, FindNearest positions the cursor on the matching record and returns True. Otherwise, positions the cursor on the record that is greater than specified key.

Syntax

```
procedure FindNearest(const AKeyValues: array of const);
```

See Also

Finding a Record (see page 107), FindKey (see page 625), GotoKey (see page 630), GotoNearest (see page 631), IndexName (see page 601), IndexFieldNames (see page 600), Locate (see page 634)

Example

```
ADQuery1.IndexFieldNames := 'F1;F2';
ADQuery1.FindNearest([100, 'qwe']);
```

1.17.2.1.5.1.81 TADDataSet.FindParam Method

Finds the parameter in Params (see page 651) property by a given name.

Parameters

Parameters	Description
const AValue: string	A parameter name to find.

Returns

Parameter, if found, Otherwise - nil.

Description

Use FindParam to find a parameter by a AValue name in the Params (see page 651) collection.

If a parameter is not found, FindParam returns nil values.

Syntax

```
function FindParam(const AValue: string): TADParam;
```

See Also

Params (see page 651), ParamByName (see page 641)

1.17.2.1.5.1.82 TADDataSet.GetColumnField Method

Returns field corresponding to DatS column.

Parameters

Parameters	Description
AColumn: TADDatSColumn	A DatS column.

Returns

A field, if column is found. Otherwise - nil.

Description

Use GetColumnField to get field object, corresponding to specified DatS column. A column must belong to dataset internal DatS table. If column is not found, then method returns nil.

Syntax

```
function GetColumnField(AColumn: TADDatSColumn): TField;
```

See Also

GetFieldColumn (see page 627)

1.17.2.1.5.1.83 TADDataSet.GetDetailLinkFields Method

Lists the field objects that link this dataset as a detail of a master dataset.

Parameters

Parameters	Description
AMasterFields: TADFieldList	A list of master fields.
ADetailFields: TADFieldList	A list of detail fields.

Description

GetDetailLinkFields fills the two lists with field objects that define a master-detail relationship between this dataset and master dataset. The AMasterFields list is filled with fields from the master dataset, whose values must be equal to the values of the fields in the ADetailFields list. The ADetailFields list is filled with fields from this dataset.

Syntax

```
procedure GetDetailLinkFields(AMasterFields: TADFieldList; ADetailFields: TADFieldList);
override;
```

See Also

Master-Detail Relationship (see page 104)

1.17.2.1.5.1.84 TADDataSet.GetFieldColumn Method

Returns DatS column corresponding to field.

Parameters

Parameters	Description
AField: TField	A field.

Returns

A column, if field is found. Otherwise - nil.

Description

Use GetFieldColumn to get DatS column object, corresponding to specified field. If field is not found, then method returns nil.

Syntax

```
function GetFieldColumn(AField: TField): TADDatSColumn; overload;
```

See Also

Master-Detail Relationship (see page 104), GetColumnField (see page 627)

1.17.2.1.5.1.85 TADDataSet.GetGroupState Method

Indicates where the current record sits within a specified group of records.

Parameters

Parameters	Description
ALevel: Integer	A grouping level.

Returns

The set of position markers.

Description

Call GetGroupState to determine where the current record falls in the group of records specified by the ALevel parameter.

ALevel identifies a group by its grouping level on the current index. Grouping level 1 is the set of records with the same value on the first field in the index. Grouping level 2 is the set of records with the same values on the first two fields in the index, and so on.

GetGroupState returns one of the following values:

Value	Meaning
[gbMiddle]	The current record is neither the first nor the last in the group.
[gbFirst]	The current record is the first in the group, which contains at least two records.
[gbLast]	The current record is the last in the group, which contains at least two records.
[gbFirst,gbLast]	The current record is the only record in the group.
[]	• ALevel specifies a grouping level greater, than the GroupingLevel (see page 598) property. • dataset is in one of the states [dsInactive, dsOpening, dsInsert, dsSetKey] • AggregatesActive (see page 590) = False • IndexName (see page 601) is empty

Syntax

```
function GetGroupState(ALevel: Integer): TGroupPosInds;
```

See Also

GroupingLevel (see page 598), IndexName (see page 601), AggregatesActive (see page 590)

1.17.2.1.5.1.86 TADDataSet.GetIndexNames Method

Fills the list by client index names.

Parameters

Parameters	Description
AList: TStrings	A list to fill.

Description

Call GetIndexNames to fill AList by client index names.

Syntax

```
procedure GetIndexNames(AList: TStrings);
```

See Also

IndexDefs (see page 598), Indexes (see page 598)

1.17.2.1.5.1.87 TADDataSet.GetNextPacket Method

Fetches the next rowset from the DBMS.

Returns

The number of fetched records.

Description

Call GetNextPacket to fetch the next rowset from a DBMS cursor.

To specify the maximum number of records to return in a rowset, set the FetchOptions (see page 651).RowsetSize (see page 859) property before calling GetNextPacket. A rowset is appended to those records already stored in the internal data storage.

GetNextPacket returns the number of records fetched. If the return value is less than FetchOptions (see page 651).RowsetSize (see page 859), the dataset has already received all available records from the DBMS cursor.

Use GetNextPacket for manual fetching mode, when FetchOptions (see page 651).Mode (see page 857) = fmManual.

Syntax

```
function GetNextPacket: Integer;
```

See Also

TDataSet.Open, TADFetchOptions.Mode (see page 857), TADFetchOptions.RowsetSize (see page 859)

Example

```
ADQuery1.FetchOptions.Mode := dmManual;
ADQuery1.SQL.Text := 'select * from mytab';
ADQuery1.Open;
// here RecordCount = 0
ADQuery1.GetNextPacket;
// here RecordCount <= FetchOptions.RowsetSize
```

1.17.2.1.5.1.88 TADDataSet.GetParentRow Method

Returns parent row of this nested dataset.

Returns

The DatS row or nil.

Description

Call GetParentRow method to get reference to parent row for this nested dataset.

Syntax

```
function GetParentRow: TADDatSRow;
```

See Also

ParentDataSet (see page 607)

1.17.2.1.5.1.89 TADDataSet.GetRow Method

Returns DatS row.

Parameters

Parameters	Description
ABuffer: PADBuffer = C_NIL	A dataset record buffer or nil (default).
AForceBufferRead: Boolean = False	Set to False (default).

Returns

A DatS row.

Description

Call GetRow method to get DatS row object for specified dataset record buffer or for current record.

If ABuffer specified, then FireDAC takes assocciated row object from it. Otherwise it uses TDataSet.ActiveBuffer value, which corresponds to current position in dataset.

Syntax

```
function GetRow(ABuffer: PADBuffer = C_NIL; AForceBufferRead: Boolean = False): TADDatSRow;
```

See Also

TADDatSRow, TDataSet.ActiveBuffer

Example

```
var
  oRow: TADDatSRow;
.....
oRow := ADQuery1.GetRow;
// cancel all changes for this row
oRow.RejectChanges;
```

1.17.2.1.5.1.90 TADDataSet.GotoCurrent Method

Sets the current record in this dataset the same as the current record in another, cloned dataset.

Parameters

Parameters	Description
ADataSet: TADDataSet	A dataset to synchronize with.

Description

Use GotoCurrent to position the cursor on the same record that is current in the specified dataset.

ADataSet is the cloned dataset whose cursor position will be used to position on the record in this dataset. GotoCurrent is only useful for synchronizing two cloned datasets that use separate data sources to view and edit the same data.

Syntax

```
procedure GotoCurrent(ADataSet: TADDataSet);
```

See Also

CloneCursor (see page 614)

1.17.2.1.5.1.91 TADDataSet.GotoKey Method

Searches for a record, using values in the key buffer.

Returns

True, if key is found. Otherwise - False.

Description

Use GotoKey to search for a record, using values assigned to the indexed fields after previous call of SetKey (see page 647) / EditKey (see page 620).

Before call dataset must be in dsSetKey state. If GotoKey finds a record, it makes it current record in the dataset and returns

True. Otherwise the current position remains unchanged, and GotoKey returns False.

Syntax

```
function GotoKey: Boolean;
```

See Also

Finding a Record (see page 107), FindKey (see page 625), FindNearest (see page 626), GotoNearest (see page 631), SetKey (see page 647), Locate (see page 634)

Example

```
ADQuery1.IndexFieldNames := 'CUSTOMER_ID;ORDER_DATE';
ADQuery1.SetKey;
ADQuery1.FieldByName('CUSTOMER_ID').AsInteger := 100;
ADQuery1.FieldByName('ORDER_DATE').AsDateTime := EncodeDate(2008, 5, 1);
if not ADQuery1.GotoKey then
  ShowMesage('Order is not found');
```

1.17.2.1.5.1.92 TADDataSet.GotoNearest Method

Searches for a closest record, using values in the key buffer.

Description

Use GotoNearest to search for a record, using values assigned to the indexed fields after previous call of SetKey (see page 647) / EditKey (see page 620).

Before call dataset must be in dsSetKey state. If GotoNearest finds a record, it makes it current record in the dataset. Otherwise the first record with greater values, than the specified values, becomes current record in the dataset.

Syntax

```
procedure GotoNearest;
```

See Also

Finding a Record (see page 107), FindKey (see page 625), FindNearest (see page 626), GotoKey (see page 630), SetKey (see page 647), Locate (see page 634)

Example

```
ADQuery1.IndexFieldNames := 'CUSTOMER_ID;ORDER_DATE';
ADQuery1.SetKey;
ADQuery1.FieldByName('CUSTOMER_ID').AsInteger := 100;
ADQuery1.FieldByName('ORDER_DATE').AsDateTime := EncodeDate(2008, 5, 1);
ADQuery1.GotoNearest;
```

1.17.2.1.5.1.93 TADDataSet.IsIndexed Method

Returns True, if dataset has current client index.

Returns

True, if indexed.

Description

Use IsIndexed to determine, if this dataset has current client index set by IndexFieldNames (see page 600) or IndexName (see page 601) properties.

Syntax

```
function IsIndexed: Boolean;
```

See Also

IndexFieldNames (see page 600), IndexName (see page 601)

1.17.2.1.5.1.94 TADDataSet.IsLinked Method

Returns True, if dataset is linked to another dataset.

Returns

True, if is linked.

Description

Use IsLinked to determine, if this dataset is nested dataset or detail dataset.

Syntax

```
function IsLinked: Boolean;
```

See Also

Master-Detail Relationship (see page 104), MasterSource (see page 603), ParentDataSet (see page 607)

1.17.2.1.5.1.95 TADDataSet.IsRanged Method

Returns True, whether the dataset records are limited by a range.

Description

The IsRanged property returns True, if dataset has range applied using ApplyRange (see page 611) call. When a range is in effect, only those records that fall within the range are available to the application for viewing and editing.

Syntax

```
function IsRanged: Boolean;
```

See Also

Filtering Records (see page 102), ApplyRange (see page 611), CancelRange (see page 613)

1.17.2.1.5.1.96 TADDataSet.LoadFromFile Method

Loads a dataset's data from a file.

Parameters

Parameters	Description
const AFileName: String = ''	A file name to load.
AFormat: TADStorageFormat = sfAuto	A file format. Default value is sfAuto.

Description

Use LoadFromFile to populate the dataset with data stored in an external file. The data is not moved to a database, but just loaded into a dataset in-memory storage.

AFileName is the name of the file containing the data to load. If file does not exists, then exception is raised.

AFormat is the file format:

- sfXML - file is well formed standalone XML file;
- sfBinary - file is binary file with custom FireDAC format;
- sfAuto - format is determined by AFileName extention: ".XML" - XML file, ".ADB" / ".BIN" / ".DAT" - binary file. If AFileName does not have an extention, then format is determined by ResourceOptions (see page 652).DefaultStoreExt (see page 877) if it is specified, otherwise by ResourceOptions (see page 652).DefaultStoreFormat (see page 877).

As option, you can specify ResourceOptions (see page 652).PersistentFileName (see page 843) and call Open, then dataset data is loaded from specified file. And at Close will be saved back, if data was modified.

AFileName must contain data saved to a file by a previous call to this or another client dataset's SaveToFile (see page 644) method. After call dataset is active.

LoadFromFile tries to load from the file the items specified by the ResourceOptions.StoreItems (see page 884). If the file was written with different StoreItems (see page 884) value, then exception will be raised likes this:

```
[FireDAC][Stan]-712 Cannot read [Relationlist] object
```

Syntax

```
procedure LoadFromFile(const AFileName: String = ''; AFormat: TADStorageFormat = sfAuto);
```

See Also

LoadFromStream (see page 633), SaveToFile (see page 644), TADResourceOptions.DefaultStoreExt (see page 877), TADResourceOptions.DefaultStoreFormat (see page 877), TADBottomResourceOptions.PersistentFileName (see page 843)

Example

```
ADQuery1.Open('select * from customers');
ADQuery1.FetchAll;
ADQuery1.SaveToFile('c:\customers.xml', sfAuto);
....
ADMemTable1.LoadFromFile('c:\customers.xml', sfAuto);
ADMemTable1.Locate('CUSTOMER_ID', [100], []);
```

1.17.2.1.5.1.97 TADDataSet.LoadFromFileAtOpen Method

Returns True, if dataset content will be loaded from file at Open.

Returns

True, if content will be loaded from file at Open.

Description

Use LoadFromFileAtOpen method to see, if dataset will execute command to get cursor and fetch records (False) or will load its content from data file (True). To automatically load content from data file, following must be true:

- dataset must be not nested dataset;
- ResourceOptions (see page 652).Persistent (see page 884) = True;
- File with name resolved from ResourceOptions (see page 652).PersistentFileName (see page 843) must exist.

Syntax

```
function LoadFromFileAtOpen: Boolean;
```

See Also

TADBottomResourceOptions.PersistentFileName (see page 843), TADTopResourceOptions.DefaultStoreFolder (see page 889), TADResourceOptions.Persistent (see page 884)

1.17.2.1.5.1.98 TADDataSet.LoadFromStream Method

Loads a dataset's data from a stream.

Parameters

Parameters	Description
AStream: TStream	A stream to load.
AFormat: TADStorageFormat = sfAuto	A file format. Default value is sfAuto.

Description

Use LoadFromStream to populate the dataset with data stored in an external file. The data is not moved to a database, but just loaded into a dataset in-memory storage.

AStream is the reference to the stream, containing the data to load. The stream position must be at begining of the dataset data, otherwise loading will fail.

AFormat is the file format:

- sfXML - stream contains a well formed standalone XML data;
- sfBinary - stream contains a binary data with custom FireDAC format;
- sfAuto - format is determined by ResourceOptions (see page 652).DefaultStoreExt (see page 877) if it is specified, otherwise by ResourceOptions (see page 652).DefaultStoreFormat (see page 877).

LoadFromStream tries to load from the stream the items specified by the ResourceOptions.StoreItems (see page 884). If the stream was written with different StoreItems (see page 884) value, then exception will be raised likes this:

```
[FireDAC][Stan]-712 Cannot read [Relationlist] object
```

Syntax

```
procedure LoadFromStream(AStream: TStream; AFormat: TADStorageFormat = sfAuto);
```

See Also

LoadFromFile (see page 632), SaveToStream (see page 645), TADResourceOptions.DefaultStoreExt (see page 877), TADResourceOptions.DefaultStoreFormat (see page 877), TADBottomResourceOptions.PersistentFileName (see page 843)

Example

```
var
  oStr: TStream;
....
oStr := ADQuery1.CreateBlobStream(ADQuery1.FieldByName('data'), bmRead);
try
  ADMemTable1.LoadFromStream(oStr, sfBinary);
finally
  oStr.Free;
end;
```

1.17.2.1.5.1.99 TADDataSet.Locate Method

Searches the dataset for a record with specified field values and makes it current.

Parameters

Parameters	Description
const AKeyFields: string	A list of fields to match.
const AKeyValues: Variant	A variant value / variant array of values to search.
AOptions: TLocateOptions = []	A set of options for string values matching.

Returns

True if a matching record is found. Otherwise - False.

Description

Use Locate to search the dataset for a record with specified field values and make it current, if found.

AKeyFields is a semicolon separated list of field names on which to search.

AKeyValues is a Variant containing the values to match to the key fields. If AKeyFields contains a single field name, then AKeyValues is a simple value. If AKeyFields contains few field names, then AKeyValues is a variant array, where items correspond to the key fields.

AOptions is a set of search mode for string fields. If AOptions contains the loCaseInsensitive, then Locate ignores case when matching fields. If AOptions contains the loPartialKey, then the AKeyValues parameter can contain partial strings.

If matching record is found, then this record becomes current and Locate returns True.

To optimize record searching, application can setup client index. The index will be used by the Locate, if following conditions are met:

- The index field list has as prefix AKeyFields fields;
- The index is build with soNoCase option and AOptions includes loCaseInsensitive. Or index is build without soNoCase and AOptions does not include loCaseInsensitive.
- The index is active and is current.

Syntax

```
function Locate(const AKeyFields: string; const AKeyValues: Variant; AOptions:
TLocateOptions = []): Boolean; override;
```

See Also

Finding a Record (see page 107), LocateEx (see page 635), Lookup (see page 637), LookupEx (see page 638), Indexes (see page 598), IndexFieldNames (see page 600), IndexName (see page 601)

Example 1

```
CustTable.IndexFieldNames := 'ID';
if CustTable.Locate('ID', 1001, []) then
  CustTable.Delete
else
  ShowMessage('The customer with ID=1001 is not found');
```

Example 2

```
CustTable.Locate('Company;Contact;Phone', VarArrayOf(['Sight Diver', 'P', '831-431-1000']),
[loPartialKey]);
```

1.17.2.1.5.1.100 TADDataSet.LocateEx Method (string, TADDataSetLocateOptions, PInteger)

Searches the dataset for a specified record by predicate expression and makes that record current or returns its index.

Parameters

Parameters	Description
const AExpression: string	A predicate expression to search.
AOptions: TADDataSetLocateOptions = []	A search modes.
ApRecordIndex: PInteger = nil	An optional pointer to variable, receiving record index. Is nil by default.

Returns

True, if record is found.

Description

Use LocateEx to search the dataset for a record meeting the predicate expression and make it current, if found.

AExpression is a string containing the predicate expression on which to search. The expression cannot contain aggregating functions, like a SUM, COUNT and must be the predicate evaluating to True or to False. FireDAC supports expression syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write expressions.

AOptions is a set that optionally specifies additional search modes:

Option	Description
lxoCaseInsensitive	If included, then the LocateEx ignores case when matching string values.
lxoPartialKey	If included, then the expression can specify incomplete strings for string values comparision.
lxoFromCurrent	If included, then the LocateEx starts searching from the next after current record in dataset, otherwise from beginning.
lxoBackward	If included, then the LocateEx searches for record in backward direction, otherwise in forward direction.
lxoCheckOnly	If included, then LocateEx will not: • change current position; • fire BeforeScroll / AfterScroll events; • finish editing mode, if it is.
lxoNoFetchAll	If included, then FetchAll will be not called. So, the search is performed in already fetched records.

If matching record is found, then this record becomes current if lxoCheckOnly is not specified, ApRecordIndex pointed variable receives index of found record if ApRecordIndex is specified, and LocateEx returns True.

The searching using this version of LocateEx cannot be optimized. For optimized searching use another version of LocateEx method. Note, that for TADTable in Live Data Window mode this LocateEx may be optimized by DBMS optimizer.

Syntax

```
function LocateEx(const AExpression: string; AOptions: TADDataSetLocateOptions = [];
ApRecordIndex: PInteger = nil): Boolean; virtual; overload;
```

See Also

Finding a Record (see page 107), Locate (see page 634), Lookup (see page 637), LookupEx (see page 638)

Example

```
procedure TForm1.BtnFindClick(ASender: TObject);
begin
  if not CustTable.LocateEx('name like ''P%D''', [lxoCaseInsensitive]) then
    ShowMessage('The customer with name starting from P and ending by D is not found');
end;

procedure TForm1.BtnNextClick(ASender: TObject);
begin
  if not CustTable.LocateEx('name like ''P%D''', [lxoCaseInsensitive, lxoFromCurrent]) then
    ShowMessage('The customer with name starting from P and ending by D is not found');
end;
```

1.17.2.1.5.1.101 TADDataSet.LocateEx Method (string, Variant, TADDataSetLocateOptions, PInteger)

Searches the dataset for a specified record by its fields values and makes that record current or returns its index.

Parameters

Parameters	Description
const AKeyFields: string	A list of fields to match.
const AKeyValues: Variant	A variant value / variant array of values to search.
AOptions: TADDataSetLocateOptions = []	A search modes.
ApRecordIndex: PInteger = nil	An optional pointer to variable, receiving record index. Is nil by default.

Returns

True, if record is found.

Description

Use LocateEx to search a dataset for a record with specified field values and make it current, if found.

AKeyFields is a semicolon-separated list of field names on which to search.

AKeyValues is a Variant containing the values to match to the key fields. If AKeyFields contains a single field name, then AKeyValues is a simple value. If AKeyFields contains few field names, then AKeyValues is a variant array, where items correspond to the key fields.

AOptions is a set that optionally specifies additional search modes:

Option	Description
lxoCaseInsensitive	If included, then the LocateEx ignores case when matching string values.
lxoPartialKey	If included, then the expression can specify incomplete strings for string values comparision.
lxoFromCurrent	If included, then the LocateEx starts searching from the next after current record in dataset, otherwise from beginning.
lxoBackward	If included, then the LocateEx searches for record in backward direction, otherwise in forward direction.

lxoCheckOnly	If included, then LocateEx will not: • change current position; • fire BeforeScroll / AfterScroll events; • finish editing mode, if it is.
lxoNoFetchAll	If included, then FetchAll will be not called. So, the search is performed in already fetched records.

If matching record is found, then this record becomes current if lxoCheckOnly is not specified, ApRecordIndex pointed variable receives index of found record if ApRecordIndex is specified, and LocateEx returns True.

To optimize record searching, application can setup client index. The index will be used by the LocateEx, if following conditions are met:

- The index field list has as prefix AKeyFields fields;
- The index is build with soNoCase option and AOptions includes lxoCaseInsensitive. Or index is build without soNoCase and AOptions does not include lxoCaseInsensitive.
- The index is active and is current.

Syntax

```
function LocateEx(const AKeyFields: string; const AKeyValues: Variant; AOptions:
TADDataSetLocateOptions = []; ApRecordIndex: PInteger = nil): Boolean; virtual; overload;
```

See Also

Finding a Record (see page 107), Locate (see page 634), Lookup (see page 637), LookupEx (see page 638), Indexes (see page 598), IndexFieldNames (see page 600), IndexName (see page 601)

Example

```
procedure TForm1.BtnFindClick(ASender: TObject);
begin
  if not CustTable.LocateEx('COMPANY', 'AMCO', [lxoCaseInsensitive]) then
    ShowMessage('The customer from AMCO company is not found');
end;

procedure TForm1.BtnNextClick(ASender: TObject);
begin
  if not CustTable.LocateEx('COMPANY', 'AMCO', [lxoCaseInsensitive, lxoFromCurrent]) then
    ShowMessage('The customer from AMCO company is not found');
end;
```

1.17.2.1.5.1.102 TADDataSet.Lookup Method

Searches the dataset for a record with specified key field values and returns result field values.

Parameters

Parameters	Description
const AKeyFields: string	A list of fields to match.
const AKeyValues: Variant	A variant value / variant array of values to search.
const AResultFields: string	A list of fields which values to return.

Returns

If record is found, the values from the fields specified in AResultFields. Otherwise - Null.

Description

Use Lookup to search the dataset for a record with specified key field values and return result field values without changing the current position in the dataset.

AKeyFields is a semicolon separated list of field names on which to search.

AKeyValues is a Variant containing the values to match to the key fields. If AKeyFields contains a single field name, then AKeyValues is a simple value. If AKeyFields contains few field names, then AKeyValues is a variant array, where items

correspond to the key fields.

AResultFields is a semicolon separated list of field names, which values to return. If record is found, Lookup returns a variant array containing the values from the fields specified in AResultFields. Otherwise - Null.

To optimize record searching, application can setup client index. The index will be used by the Lookup, if following conditions are met:

- The index field list has as prefix AKeyFields fields;
- The index is build without soNoCase option;
- The index is active and is current.

Syntax

```
function Lookup(const AKeyFields: string; const AKeyValues: Variant; const AResultFields:
string): Variant; override;
```

See Also

Finding a Record (see page 107), LookupEx (see page 638), Locate (see page 634), LocateEx (see page 635), Indexes (see page 598), IndexFieldNames (see page 600), IndexName (see page 601)

Example

```
procedure TForm1.Button1Click(Sender: TObject);
var
  V: Variant;
  C: Integer;
  A: String;
begin
  V := ADQuery1.Lookup('Company;State', VarArrayOf(['Blue Sports', 'OR']), 'CustNo;Addr1');
  if not VarIsNull(V) then begin
    C := V[0];
    A := V[1];
    ShowMessage(IntToStr(C) + #13#10 + A);
  end
  else
    ShowMessage('The record is not found !');
end;
```

1.17.2.1.5.1.103 TADDataSet.LookupEx Method (string, Variant, string, TADDataSetLocateOptions, PInteger)

Searches the dataset for a record with specified key field values and returns result field values.

Parameters

Parameters	Description
const AKeyFields: string	A list of fields to match.
const AKeyValues: Variant	A variant value / variant array of values to search.
const AResultFields: string	A list of fields which values to return.
AOptions: TADDataSetLocateOptions = []	A set of search modes.
ApRecordIndex: PInteger = nil	An optional pointer to variable, receiving record index. Is nil by default.

Returns

If record is found, the values from the fields specified in AResultFields. Otherwise - Null.

Description

Use Lookup to search the dataset for a record with specified key field values and return result field values without changing the current position in the dataset.

AKeyFields is a semicolon separated list of field names on which to search.

AKeyValues is a Variant containing the values to match to the key fields. If AKeyFields contains a single field name, then AKeyValues is a simple value. If AKeyFields contains few field names, then AKeyValues is a variant array, where items correspond to the key fields.

AResultFields is a semicolon separated list of field names, which values to return. If record is found, Lookup returns a variant array containing the values from the fields specified in AResultFields. Otherwise - Null.

AOptions is a set that optionally specifies additional search modes:

Option	Description
lxoCaseInsensitive	If included, then the LookupEx ignores case when matching string values.
lxoPartialKey	If included, then the AKeyValues can specify incomplete strings for string values comparision.
lxoFromCurrent	If included, then the LookupEx starts searching from the next after current record in dataset, otherwise from beginning.
lxoBackward	If included, then the LookupEx searches for record in backward direction, otherwise in forward direction.
lxoCheckOnly	If included, then LookupEx will not: • finish editing mode, if it is.
lxoNoFetchAll	If included, then FetchAll will be not called. So, the search is performed in already fetched records.

If matching record is found and ApRecordIndex is not nil, then pointed variable receives index of found record.

To optimize record searching, application can setup client index. The index will be used by the LookupEx, if following conditions are met:

- The index field list has as prefix AKeyFields fields;
- The index is build without soNoCase option;
- The index is active and is current.

Syntax

```
function LookupEx(const AKeyFields: string; const AKeyValues: Variant; const AResultFields:
string; AOptions: TADDataSetLocateOptions = []; ApRecordIndex: PInteger = nil): Variant;
virtual; overload;
```

See Also

Finding a Record (see page 107), Lookup (see page 637), Locate (see page 634), LocateEx (see page 635), Indexes (see page 598), IndexFieldNames (see page 600), IndexName (see page 601)

1.17.2.1.5.1.104 TADDataSet.LookupEx Method (string, string, TADDataSetLocateOptions, PInteger)

Searches the dataset for a record by predicate expression and returns result field values.

Parameters

Parameters	Description
const AExpression: string	A predicate expression to search.
const AResultFields: string	A list of fields which values to return.
AOptions: TADDataSetLocateOptions = []	A set of search modes.
ApRecordIndex: PInteger = nil	An optional pointer to variable, receiving record index. Is nil by default.

Returns

If record is found, the values from the fields specified in AResultFields. Otherwise - Null.

Description

Use Lookup to search the dataset for a record for which specified predicate expression evaluates to True and return other field values from the found record.

AExpression is a string containing the predicate expression on which to search. The expression cannot contain aggregating functions, like a SUM, COUNT and must be the predicate evaluating to True or to False. FireDAC supports expression syntax compatible with:

- BDE data access components;
- TClientDataset;
- Oracle 8 (not for 100%).

See Writing Expressions (see page 110) for details of how to write expressions.

AResultFields is a semicolon separated list of field names, which values to return. If record is found, Lookup returns a variant array containing the values from the fields specified in AResultFields. Otherwise - Null.

AOptions is a set that optionally specifies additional search modes:

Option	Description
lxoCaseInsensitive	If included, then the LookupEx ignores case when matching string values.
lxoPartialKey	If included, then the AKeyValues can specify incomplete strings for string values comparision.
lxoFromCurrent	If included, then the LookupEx starts searching from the next after current record in dataset, otherwise from beginning.
lxoBackward	If included, then the LookupEx searches for record in backward direction, otherwise in forward direction.
lxoCheckOnly	If included, then LookupEx will not: • finish editing mode, if it is.
lxoNoFetchAll	If included, then FetchAll will be not called. So, the search is performed in already fetched records.

If matching record is found and ApRecordIndex is not nil, then pointed variable receives index of found record.

The searching using this version of LookupEx cannot be optimized. For optimized searching use another version of LookupEx method.

Syntax

```
function LookupEx(const AExpression: string; const AResultFields: string; AOptions:
TADDataSetLocateOptions = []; ApRecordIndex: PInteger = nil): Variant; virtual; overload;
```

See Also

Finding a Record (see page 107), Lookup (see page 637), Locate (see page 634), LocateEx (see page 635)

Example

```
procedure TForm1.Button1Click(Sender: TObject);
var
  V: Variant;
  C: Integer;
  A: String;
begin
  V := ADQuery1.LookupEx('Company = ''Blue Sports'' and State = ''OR''', 'CustNo;Addr1',
[lxoCaseInsensitive]);
  if not VarIsNull(V) then begin
    C := V[0];
    A := V[1];
    ShowMessage(IntToStr(C) + #13#10 + A);
  end
  else
    ShowMessage('The record is not found !');
end;
```

1.17.2.1.5.1.105 TADDataSet.Offline Method

Sets dataset to offline mode.

Description

The Offline method call disconnects dataset from DBMS, but does not close it. After this call no more cursors will be accessible, as result the dataset will not fetch any more records from a cursor, SourceEOF (see page 608) will be True and a command will be unprepared.

If ResourceOptions (see page 652).AutoFetchAll (see page 852) is True, then before entering into offline mode, dataset will fetch all remaining records from the current cursor. If it is False and not all records are fetched, then an exception will be raised. Application can fetch all dataset records explicitly by calling FetchAll method.

In most cases there is no need to call Offline method explicitly. It is implicitly used by the connection Offline method to set the connection and the all associated datasets to offline mode.

The Release method is similar to the Offline method, but it does not unprepare command.

Syntax

```
procedure Offline;
```

See Also

TADCustomConnection.Offline (see page 352), FetchAll (see page 624), Release (see page 644)

1.17.2.1.5.1.106 TADDataSet.OpenOrExecute Method

Executes SQL command and optionally opens dataset.

Returns

True, if a command returned result set.

Description

Use OpenOrExecute method, if you does not know - will command return result set or not. The most common case for that is the execution of an ad hoc query.

The OpenOrExecute method executes SQL command. And if command returns any result set, then this method opens dataset. The method supplements Open and Execute (see page 622) / ExecSQL (see page 398) / ExecProc (see page 410) methods:

- the Open method raises an exception "[FireDAC][Phys]-308. Command must return row set", if a command does not return result set, like a INSERT;
- the Execute (see page 622) / ExecSQL (see page 398) / ExecProc (see page 410) methods raise an exception "[FireDAC][Phys]-310. Cannot execute command returning results set", if a command returns result set, like a SELECT.

At OpenOrExecute call a "[FireDAC][Phys]-308" may be raised, but it will be not propagated out of the method.

Syntax

```
function OpenOrExecute: Boolean;
```

See Also

TDataSet.Open, Execute (see page 622), TADCustomQuery.ExecSQL (see page 397), TADCustomStoredProc.ExecProc (see page 410)

Example

```
ADQuery1.Disconnect(True);
ADQuery1.SQL.Assign(Memo1.Lines);
if ADQuery1.OpenOrExecute then begin
  PageControl1.ActivePage := DataGridPage;
  DataGridPage.TabVisible := True;
end
else
  DataGridPage.TabVisible := False;
```

1.17.2.1.5.1.107 TADDataSet.ParamByName Method

Searches parameter by its name.

Parameters

Parameters	Description
const AValue: string	A name of parameter.

Returns

Parameter, if it is found.

Description

Use ParamByName to find a parameter by its name in the Params (see page 651) collection.

AValue is the name of the parameter for which to retrieve information. If parameter with AValue name is not found, then exception is raised.

Syntax

```
function ParamByName(const AValue: string): TADParam;
```

See Also

FindParam (see page 626)

1.17.2.1.5.1.108 TADDataSet.Reconcile Method

Clears successfully updated records from the dataset's cache of changes.

Description

Reconcile is called by the ApplyUpdates (see page 611) method to update the dataset so that it reflects the result of an update. Reconcile is called after the dataset has attempted to apply all updates in the dataset's Delta property.

Results of updates applying are recorded in internal data storage, including error information for each record that could not be applied. Reconcile generates an OnReconcileError (see page 604) event for every record that was not successfully applied. Finally, Reconcile adjusts the Delta property so that it includes only those records that were not successfully applied. Any changes made in the OnReconcileError (see page 604) event are reflected in the new value of Delta.

Reconcile returns a value that indicates whether reconciling is successful. If True, Reconcile succeeded without error. If False, one or more errors could not be reconciled.

Alternatively an application may review erroneous records using FilterChanges (see page 597) and RowError (see page 607) properties. Read "Reviewing errors" at Caching Updates (see page 114) for more details.

Syntax

```
function Reconcile: Boolean;
```

See Also

Caching Updates (see page 114), ApplyUpdates (see page 611), RowError (see page 607), FilterChanges (see page 597), OnReconcileError (see page 604)

1.17.2.1.5.1.109 TADDataSet.RefreshRecord Method

Rereads field values of the current record from a data source.

Parameters

Parameters	Description
AClearRow: Boolean = True	When True, then the record fields will be initially erased.

Returns

True, if a record is refreshed. False, if a record is deleted from a dataset.

Description

Use RefreshRecord to discard all changes of the current record and reread it from a data source. A similar method, TDataSet.Refresh, replaces the entire contents of the dataset by re-executing SQL command.

To reread record the FireDAC will perform the sequence of steps:

- If is assigned OnUpdateRecord (see page 605) event handler, then it will be called with ARequest = arFetchRow.
- If event handler is not assigned or it returns AAction = eaDefault, then:

- If is assigned TADAdaptedDataSet (see page 259).UpdateObject (see page 267) and TADUpdateSQL (see page 563).FetchRowSQL (see page 567) is not empty, then this SQL command will be executed.
- Otherwise FireDAC will generate SELECT command rereading single row from database and will execute command.

If query to a data source will return no rows (probably a record is deleted), then when UpdateOptions (see page 653).RefreshDelete (see page 903) is True a record will be removed from a dataset too, otherwise an exception will be raised. When AClearRow parameters is set to True (default value), then the record will be initially erased. Otherwise the read values will override the corresponding column values.

The method returns True, if a record is refreshed. And False, if it is deleted from a dataset.

Syntax

```
function RefreshRecord(AClearRow: Boolean = True): Boolean;
```

See Also

TDataSet.Refresh, OnUpdateRecord (see page 605), TADAdaptedDataSet.UpdateObject (see page 267), TADUpdateSQL.FetchRowSQL (see page 567), TADUpdateOptions.RefreshDelete (see page 903), Update Command Generation (see page 121)

Example

```
ADQuery1.UpdateObject := ADUpdateSQL;
// PL/SQL block calling packaged procedure returning customer data by its ID
ADUpdateSQL.FetchRowSQL.Text := 'begin cust_pack.read_cust_data(:new_name, :new_company,
:new_state, :old_id); end;';
// always is required to setup output parameters
with ADUpdateSQL.Commands[arFetchRow] do begin
  Params[0].ParamType := ptOutput;
  Params[0].DataType := ftString;
  Params[1].ParamType := ptOutput;
  Params[1].DataType := ftString;
  Params[2].ParamType := ptOutput;
  Params[2].DataType := ftString;
end;
...
ADQuery1.RefreshRecord;
```

1.17.2.1.5.1.110 TADDataSet.RefreshUnknownRecord Method

Registers new record and optionally rereads it field values from a data source.

Parameters

Parameters	Description
const AValues: array of Variant	The values to assign to new record.
AReread: Boolean = True	If True, then new added record will be immediately refreshed.
ASetPos: Boolean = True	If True, then dataset will set current position to the new added record.

Description

Use RefreshUnknownRecord to register new record not yet fetched at internal data storage, assigns AValues to it columns and optionally rereads it field values from data source.

This method is useful, if you need to fetch record, not existing in internal data storage, but existing in database. For example, it was not fetched together with result set, but later it is known that new record is added to the database. And application needs to fetch this record.

FireDAC assigns not-Null values from AValues to corresponding record columns. Only those columns values will be used in WHERE phrase at fetching record. Most convenient is to specify key column values.

Syntax

```
procedure RefreshUnknownRecord(const AValues: array of Variant; AReread: Boolean = True;
ASetPos: Boolean = True);
```

See Also

RefreshRecord (see page 642), TDataSet.Refresh

Example

```
// Assign 101 to ID field, fetch record immediately and set current position to it.
ADQuery1.RefreshUnknownRecord([101], True, True);
```

1.17.2.1.5.1.111 TADDataSet.Release Method

Releases all dataset cursors.

Description

The Release method call Releases all dataset cursors, but does not close it. After this call no more cursors will be accessible, as result the dataset will not fetch any more records from a cursor and SourceEOF (see page 608) will be True.

If ResourceOptions (see page 652).AutoFetchAll (see page 852) is True, then before releasing cursors, dataset will fetch all remaining records from the current cursor. If it is False and not all records are fetched, then an exception will be raised. Application can fetch all dataset records explicitly by calling FetchAll (see page 624) method.

The Disconnect (see page 619) method is similar to the Release method, but it also unprepares a command.

Syntax

```
procedure Release;
```

See Also

FetchAll (see page 624), Offline (see page 640), Disconnect (see page 619)

1.17.2.1.5.1.112 TADDataSet.RevertRecord Method

Undoes changes to the current record.

Description

Use RevertRecord to undo changes to the current record and remove it from the dataset changes log.

To undo the last edit made to a data set, call UndoLastChange (see page 649). To cancel all pending modifications, call CancelUpdates (see page 614). To undo all edits since an application-defined state, use the SavePoint (see page 607) property.

Syntax

```
procedure RevertRecord;
```

See Also

UndoLastChange (see page 649), CancelUpdates (see page 614), SavePoint (see page 607)

Example

See FireDAC\Samples\Comp Layer\TADMemTable (see page 436)\CachedUpdates (see page 592) sample for details.

1.17.2.1.5.1.113 TADDataSet.SaveToFile Method

Saves a dataset's data to a file.

Parameters

Parameters	Description
const AFileName: String = ''	An external file name.
AFormat: TADStorageFormat = sfAuto	A file format.

Description

Use SaveToFile to write a dataset's data to an external file for later use by this or other datasets.

AFileName is the name of the external file to save the data.

AFormat is the file format:

- sfXML - file is well formed standalone XML file;
- sfBinary - file is binary file with custom FireDAC format;
- sfAuto - format is determined by AFileName extension: ".XML" - XML file, ".ADB" / ".BIN" / ".DAT" - binary file. If AFileName does not have an extension, then format is determined by ResourceOptions (see page 652).DefaultStoreExt (see page 877) if it is specified, otherwise by ResourceOptions (see page 652).DefaultStoreFormat (see page 877).

As option, you can specify ResourceOptions (see page 652).PersistentFileName (see page 843) before calling Open. Then data will be saved to file at Close, if data was modified or file does not exists.

If file exists and ResourceOptions (see page 652).Backup (see page 879) = True, then original file extension will be changed to ResourceOptions (see page 652).BackupExt (see page 876) and it will be saved to ResourceOptions (see page 652).BackupFolder (see page 876). And then will be written a new file.

SaveToFile stores into the file the items specified by the ResourceOptions.StoreItems (see page 884).

Syntax

```
procedure SaveToFile(const AFileName: String = ''; AFormat: TADStorageFormat = sfAuto);
```

See Also

SaveToStream (see page 645), TADResourceOptions.DefaultStoreExt (see page 877), TADResourceOptions.DefaultStoreFormat (see page 877), TADBottomResourceOptions.PersistentFileName (see page 843)

Example

```
ADQuery1.Open('select * from customers');
ADQuery1.FetchAll;
ADQuery1.SaveToFile('c:\customers.xml', sfAuto);
....
ADMemTable1.LoadFromFile('c:\customers.xml', sfAuto);
ADMemTable1.Locate('CUSTOMER_ID', [100], []);
```

1.17.2.1.5.1.114 TADDataSet.SaveToFileAtClose Method

Returns True, if dataset content will be saved to file at Close.

Returns

True, if dataset content will be saved.

Description

Call SaveFileAtClose method to see, if dataset will save its data to the external file at Close. To automatically save content to data file, following must be true:

- dataset must be not nested dataset;
- ResourceOptions (see page 652).Persistent (see page 884) = True;
- resolved ResourceOptions (see page 652).PersistentFileName (see page 843) must be not empty;
- file does not exists or dataset was changed science last saving or loading of data from file or opening.

Syntax

```
function SaveToFileAtClose: Boolean;
```

See Also

TADBottomResourceOptions.PersistentFileName (see page 843), TADTopResourceOptions.DefaultStoreFolder (see page 889), TADResourceOptions.Persistent (see page 884)

1.17.2.1.5.1.115 TADDataSet.SaveToStream Method

Saves a dataset's data to a stream.

Parameters

Parameters	Description
AStream: TStream	A stream to write.
AFormat: TADStorageFormat = sfAuto	A file format.

Description

Call SaveToStream to save the dataset data to an external file.

AStream is the reference to the stream, to which the data to write. The dataset will start to write its data at current stream position.

AFormat is the file format:

- sfXML - stream contains a well formed standalone XML data;
- sfBinary - stream contains a binary data with custom FireDAC format;
- sfAuto - format is determined by ResourceOptions (see page 652).DefaultStoreExt (see page 877) if it is specified, otherwise by ResourceOptions (see page 652).DefaultStoreFormat (see page 877).

SaveToStream stores into the stream the items specified by the ResourceOptions.StoreItems (see page 884).

Syntax

```
procedure SaveToStream(AStream: TStream; AFormat: TADStorageFormat = sfAuto);
```

See Also

SaveToFile (see page 644), LoadFromStream (see page 633), TADResourceOptions.DefaultStoreExt (see page 877), TADResourceOptions.DefaultStoreFormat (see page 877), TADBottomResourceOptions.PersistentFileName (see page 843)

Example

```
var
  oStr: TStream;
  ....
  ADQuery1.Edit;
  oStr := ADQuery1.CreateBlobStream(ADQuery1.FieldByName('data'), bmWrite);
  try
    ADMemTable1.SaveToStream(oStr, sfBinary);
  finally
    oStr.Free;
    ADQuery1.Post;
  end;
```

1.17.2.1.5.1.116 TADDataSet.SetFieldAttributes Method

Setups fields ProviderFlags.

Description

Use the SetFieldAttributes method to setup dataset fields ProviderFlags and some others properties. FireDAC datasets are doing that automatically, when:

- dataset uses temporary fields;
- programmer is adding persistent fields at design time.

When application is creating fields manually or using CreateFields method, then above properties will be not setup automatically. To setup them call SetFieldAttributes method after dataset opening.

Syntax

```
procedure SetFieldAttributes; overload;
```

See Also

TField.ProviderFlags, TDataSet.CreateFields

Example

```
ADQuery1.FieldDefs.Updated := False;
ADQuery1.FieldDefs.Update;
ADQuery1.CreateFields;
....
ADQuery1.Open;
ADQuery1.SetFieldAttributes;
```

1.17.2.1.5.1.117 TADDataSet.SetKey Method

Starts the enter key values mode.

Description

Use SetKey to put the dataset into dsSetKey state, clear the current content of the search key buffer and enable to enter key field values.

Use EditKey (see page 620) to modify an existing key field values, without initially erasing them.

Syntax

```
procedure SetKey;
```

See Also

Finding a Record (see page 107), EditKey (see page 620), GotoKey (see page 630), SetRange (see page 647), SetRangeEnd (see page 648), SetRangeStart (see page 648), TDataSet.State, TDataSet.FieldByName, IndexName (see page 601), IndexFieldNames (see page 600), Indexes (see page 598)

Example

```
ADQuery1.IndexFieldNames := 'CUSTOMER_ID;ORDER_DATE';
ADQuery1.SetKey;
ADQuery1.FieldByName('CUSTOMER_ID').AsInteger := 100;
ADQuery1.FieldByName('ORDER_DATE').AsDateTime := EncodeDate(2008, 5, 1);
if not ADQuery1.GotoKey then
  ShowMesage('Order is not found');
```

1.17.2.1.5.1.118 TADDataSet.SetRange Method

Sets the starting and ending values of a range, and applies it.

Parameters

Parameters	Description
const AStartValues: array of const	An open array of starting range values.
const AEndValues: array of const	An open array of ending range values.
AStartExclusive: Boolean = False	Exclude start range value. Default False.
AEndExclusive: Boolean = False	Exclude end range value. Default False.

Description

Use SetRange to specify the beginning and ending range values for the dataset and apply the range.

AStartValues sets starting range the indexed field values. AEndValues sets ending range the indexed fields values. SetRange performs the following steps:

- puts the dataset into dsSetKey state;
- erases any previously specified starting range values and ending range values;
- sets the start and end range values;
- applies the range to the dataset.

If the current dataset record is within new range, then it will be current after a call to SetRange. Otherwise the current position will be set to the first record in the range.

If either AStartValues or AEndValues has fewer elements than the number of fields in the current index, then the remaining entries are set to NULL.

Syntax

```
procedure SetRange(const AStartValues: array of const; const AEndValues: array of const;
AStartExclusive: Boolean = False; AEndExclusive: Boolean = False);
```

See Also

Filtering Records (see page 102), SetRangeEnd (see page 648), SetRangeStart (see page 648), ApplyRange (see page 611), TDataSet.State, TDataSet.FieldByName, IndexName (see page 601), IndexFieldNames (see page 600), Indexes (see page 598)

Example

```
ADQuery1.IndexFieldNames := 'CUST_NO';
ADQuery1.SetRange([100], [200]);
```

1.17.2.1.5.1.119 TADDataSet.SetRangeEnd Method

Sets the ending values of a range.

Description

Use SetRangeEnd to put the dataset into dsSetKey state and erase any previously specified ending range values.

After that application can assign ending range values to the dataset indexed fields.

Syntax

```
procedure SetRangeEnd;
```

See Also

Filtering Records (see page 102), SetRangeStart (see page 648), EditRangeStart (see page 621), ApplyRange (see page 611), IndexName (see page 601), IndexFieldNames (see page 600), Indexes (see page 598), TDataSet.State, TDataSet.FieldByName

Example

```
ADQuery1.IndexFieldNames := 'CUST_NO';
...
ADQuery1.SetRangeEnd;
ADQuery1.FieldByName('CUST_NO').AsInteger := 200;
...
ADQuery1.ApplyRange;
```

1.17.2.1.5.1.120 TADDataSet.SetRangeStart Method

Sets the starting values of a range.

Description

Use SetRangeStart to put the dataset into dsSetKey state and erase any previously specified starting range values.

After that application can assign starting range values to the dataset indexed fields.

Syntax

```
procedure SetRangeStart;
```

See Also

Filtering Records (see page 102), SetRangeEnd (see page 648), EditRangeStart (see page 621), ApplyRange (see page 611), IndexName (see page 601), IndexFieldNames (see page 600), Indexes (see page 598), TDataSet.State, TDataSet.FieldByName

Example

```
ADQuery1.IndexFieldNames := 'CUST_NO';
ADQuery1.SetRangeStart;
ADQuery1.FieldByName('CUST_NO').AsInteger := 100;
...
ADQuery1.ApplyRange;
```

1.17.2.1.5.1.121 TADDataSet.UndoLastChange Method

Undoes the last modification to the dataset records.

Parameters

Parameters	Description
AFollowChange: Boolean	A flag controlling current position in dataset.

Returns

True, if a change was undone.

Description

Use UndoLastChange to undo the last modification to the dataset records and remove the change from the dataset change log.

The method is applicable to the datasets working in CachedUpdates (see page 592) = True mode.

If AFollowChange is True, then the current position in dataset is set to restored record. Otherwise the position remains unchanged.

To check if changed log is not empty read the UpdatesPending (see page 609) property value. If it is True, then there are changes to undo.

Syntax

```
function UndoLastChange(AFollowChange: Boolean): Boolean;
```

See Also

Caching Updates (see page 114), UpdatesPending (see page 609), RevertRecord (see page 644), CancelUpdates (see page 614), SavePoint (see page 607)

Example 1

```
procedure TForm1.btnUndoClick(ASender: TObject);
begin
  ADQuery1.UndoLastChange(True);
end;
```

Example 2

See FireDAC\Samples\Comp Layer\TADMemTable (see page 436)\CachedUpdates (see page 592) sample for details.

1.17.2.1.5.1.122 TADDataSet.UpdateAttributes Method

Actualize changes to the client default expressions and auto-incremental fields.

Description

Use UpdateAttributes method to actualize changes to the default expressions and auto-incremental fields made after dataset was activated.

The fields default expressions that are in TField.DefaultExpression property or auto-incremental fields that are TADAutoIncField (see page 579) properties, are actualized automatically at dataset Open call. The changes to these properties for an active dataset are not actualizing automatically.

Syntax

```
procedure UpdateAttributes;
```

See Also

Specifying Default Values (see page 126), UpdateConstraints (see page 650), TField.DefaultExpression, TADAutoIncField (see page 579)

Example

```
ADQuery1.FieldByName('ObjGUID').DefaultExpression := 'NEWGUID()';
ADQuery1.UpdateAttributes;
```

1.17.2.1.5.1.123 TADDataSet.UpdateConstraints Method

Actualize changes to the client constraints.

Description

Use UpdateConstraints method to actualize changes to the constraints made after dataset was activated.

The client constraints that are in Constraints (see page 594) property (record constraints) or are assigned to the dataset fields (field constraints) are actualized automatically at dataset Open call. The constraint changes for an active dataset are not actualizing automatically.

Syntax

```
procedure UpdateConstraints;
```

See Also

DisableConstraints (see page 619), EnableConstraints (see page 621), Constraints (see page 594), ConstraintsEnabled (see page 594), UpdateAttributes (see page 649)

Example

```
with ADQuery1.FieldByName('age') do begin
  CustomConstraint := 'age >= 18';
  ConstraintErrorMessage := 'The age must be greater or equal to 18 years';
end;
ADQuery1.UpdateConstraints;
```

1.17.2.1.5.1.124 TADDataSet.UpdateStatus Method

Returns the modification status for current record.

Returns

Record modification status.

Description

Use UpdateStatus method to get the modification status for the current dataset record. The function returns one of the following values:

Status	Description
usUnmodified	Record is in its original unmodified state.
usModified	Record is modified. Changes are not posted to database.
usDeleted	Record is deleted. Changes are not posted to database.
usInserted	Record is inserted. Changes are not posted to database.

Syntax

```
function UpdateStatus: TUpdateStatus; override;
```

See Also

Caching Updates (see page 114), UndoLastChange (see page 649), FilterChanges (see page 597)

Example

```
procedure TForm1.ADQuery1AfterScroll(DataSet: TDataSet);
begin
  case TADQuery(DataSet).UpdateStatus of
  usUnmodified: Label1.Caption := '';
  usModified: Label1.Caption := 'Modified';
  usDeleted: Label1.Caption := 'Deleted';
  usInserted: Label1.Caption := 'Inserted';
  end;
end;
```

1.17.2.1.5.2 protected

1.17.2.1.5.2.1 TADDataSet.FetchOptions Property

The set of options to control rows fetching.

Description

The FetchOptions property is the set of properties, controlling the fetching of cursor rows. These properties will inherit its values from FetchOptions of connection object.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

Setting Options (see page 34), TADFetchOptions (see page 849)

1.17.2.1.5.2.2 TADDataSet.FormatOptions Property

The set of options to control data representation.

Description

The FormatOptions property is the set of properties, controlling the data type mapping, and some other aspects of data handling.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

Setting Options (see page 34), TADFormatOptions (see page 860), Params (see page 651)

1.17.2.1.5.2.3 TADDataSet.ParamCount Property

Returns the current number of parameters in the command text.

Description

The ParamCount property value is the shortcut for Params (see page 651).Count property.

If the ResourceOptions (see page 652).ParamCreate (see page 883) property is True, ParamCount always corresponds to the number of actual parameters in the SQL statement.

Syntax

```
property ParamCount: Word;
```

See Also

Params (see page 651), TADResourceOptions.ParamCreate

Example

```
var
  i: Integer;
...
Memo1.Lines.Clear;
for i := 0 to ADQuery1.ParamCount - 1 do
  Memo1.Lines.Add(ADQuery1.Params[i].Name);
```

1.17.2.1.5.2.4 TADDataSet.Params Property

Contains the parameters for a SQL statement.

Description

Use Params property to view and set SQL statement parameter names, values, and data types at run time. Use the collection editor for the Params property to set parameter information at design time.

If dataset is TADQuery (see page 476), then Params is populated automatically after changing SQL (see page 396) property value, if ResourceOptions (see page 652).ParamCreate (see page 883) = True. The parameters order is the

same as they appear in the SQL command. When Params.BindMode = pbByName, then each parameter name will appear in Params only once. When Params.BindMode = pbByNumber, then each parameter marker will appear in the Params, including duplicated names.

If dataset is TADStoredProc (see page 514), then Params is populated automatically after calling Prepare (see page 511) method at run time or changing StoredProcName (see page 406) at design time, if fiMeta is in FetchOption (see page 651).Items (see page 855). The parameters order is the same as in the stored procedure header.

Otherwise Params may be populated by hands. At Prepare (see page 511) call, parameters will be bind to prepared SQL command. After that you cannot change parameter types, otherwise will get an exception.

An easier way to set and retrieve parameter values when the name of each parameter is known is to call ParamByName (see page 641). This method is especially preferred versus accessing parameters by index when they are populated automatically and there is a chance that the SQL property or StoredProcName property will be changed in the code dynamically.

Syntax

```
property Params: TADParams;
```

See Also

Executing Command (see page 66), Executing Stored Procedure (see page 71), TADCustomQuery (see page 394).SQL (see page 396), TADCustomStoredProc (see page 403).StoredProcName (see page 406), TADParams, TADResourceOptions (see page 874).ParamCreate (see page 883), TADFetchOptions (see page 849).Items (see page 855)

Example 1

```
// TADQuery automatic parameters creation - ResourceOptions.ParamCreate = True
ADQuery1.SQL.Text := 'select * from mytab where id = :id';
ADQuery1.Params[0].AsString := 'qwe';
ADQuery1.Open;
```

Example 2

```
// TADQuery manual parameters creation - ResourceOptions.ParamCreate = False
ADQuery1.ResourceOptions.ParamCreate := False;
ADQuery1.SQL.Text := 'select * from mytab where id = :id';
with ADQuery1.Params.Add do begin
  Name := 'id';
  DataType := ftString;
  Size := 10;
  AsString := 'qwe';
end;
ADQuery1.Open;
```

Example 3

```
// TADStoredProc automatic parameters creation - fiMeta in FetchOptions.Items
ADStoredProc1.StoredProcName := 'my_proc';
ADStoredProc1.Prepare;
ADStoredProc1.Params[0].AsString := 'qwe';
ADStoredProc1.ExecProc;
```

Example 4

```
// TADStoredProc manual parameters creation - fiMeta not in FetchOptions.Items
ADStoredProc1.FetchOptions.Items := ADStoredProc1.FetchOptions.Items - [fiMeta];
ADStoredProc1.StoredProcName := 'my_proc';
with ADStoredProc1.Params.Add do begin
  Name := 'id';
  DataType := ftString;
  Size := 10;
end;
ADStoredProc1.Prepare;
ADStoredProc1.Params[0].AsString := 'qwe';
ADStoredProc1.ExecProc;
```

1.17.2.1.5.2.5 TADDataSet.ResourceOptions Property

The set of options to control resources usage.

Description

The ResourceOptions property is the set of properties, controlling the resources usage by the dataset. These properties will inherit its values from ResourceOptions of connection object.

Syntax

```
property ResourceOptions: TADBottomResourceOptions;
```

See Also

Setting Options (see page 34), TADBottomResourceOptions (see page 841)

1.17.2.1.5.2.6 TADDataSet.UpdateOptions Property

The set of options to control updates posting.

Description

The UpdateOptions property is the set of properties, controlling the posting updates from the dataset. These properties will inherit its values from UpdateOptions of connection object.

Syntax

```
property UpdateOptions: TADBottomUpdateOptions;
```

See Also

Setting Options (see page 34), TADBottomUpdateOptions (see page 843)

1.17.2.1.5.2.7 TADDataSet.Updates Property

The dataset change log.

Description

Use Updates property to get reference to the dataset change log (record updates journal).

The change log contains list of changed, deleted and new record references. The list is ordered by the moment of time, when a change is happened. If record was changed twice or more, log contains information for last change only.

Syntax

```
property Updates: TADDatSUpdatesJournal;
```

See Also

TADDatSUpdatesJournal

1.17.2.1.6 TADIndex Class

TADIndex represents a maintained client view to records in an FireDAC dataset.

public

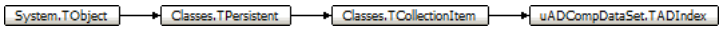
public	Description
ActualActive (see page 654)	Returns True, if dataset view is really maintained.
DataSet (see page 655)	Returns the reference to containing dataset.

published

published	Description
Active (see page 655)	Controls when dataset view must be maintained.
CaseInsFields (see page 655)	Specifies fields for which sorting must be case-insensitive.
DescFields (see page 656)	Specifies fields for which sorting must be done in descending order.
Distinct (see page 656)	Controls returning of distinct view value.
Expression (see page 656)	Specifies expression which must be used to sort records.

Fields (see page 657)	Identifies the fields that comprise the index.
Filter (see page 657)	Specifies the text of a view filter.
FilterOptions (see page 657)	Specifies filtering options.
Name (see page 658)	Specifies the name of dataset view.
Options (see page 658)	Specifies sorting options.
Selected (see page 659)	Specifies, if a dataset view is the current dataset view.

Class Hierarchy



File

uADCompDataSet

Description

Use TADIndex to create and maintain client view to records in an FireDAC dataset.

TADIndex are collected into TADIndexes (see page 659) collection. The set of TADIndex may be created as at design time, as at run time.

Fields (see page 657) property specifies the list of sort fields. Alternatively Expression (see page 656) property specifies sort expression. Filter (see page 657) property specifies filtering expression. The view will be maintained if Active (see page 655) is True and TADataSet.IndexesActive (see page 600) is True.

The view becomes current in dataset, if Selected (see page 659) is True or TADDataSet.IndexName (see page 601) is set to the name of this view.

Notes

The name may confuse you. It talks about "index". Originally TADIndex was representing a maintained index, but later we have added extended features to TADIndex, so it is not just controlling sorting order.

It is possible to define indexes using TADDataSet.IndexDefs. But we recomend to use TADDataSet.Indexes. It offers more options, while IndexDefs is for compatibility with legacy software.

Syntax

```
TADIndex = class(TCollectionItem);
```

See Also

TADIndexes (see page 659), TADDataSet (see page 584), TADDataSet.IndexFieldNames (see page 600), TADDataSet.IndexName (see page 601), TADDataSet.Indexes (see page 598), TADDataSet.IndexesActive (see page 600)

Example

```
ADQuery1.IndexesActive := True;
ADQuery1.Open;

with ADQuery1.Indexes.Add do begin
  Name := 'Orders';
  Fields := 'ORDER_NO;PART_NO';
  Filter := 'OrderDate < ''01.01.2008''';
  Active := True;
  Selected := True;
end;
```

1.17.2.1.6.1 public

1.17.2.1.6.1.1 TADIndex.ActualActive Property

Returns True, if dataset view is really maintained.

Description

Use ActualActive to see if FireDAC is maintaining this dataset view. In general, the view is maintained if:

- Active is True;
- dataset is active;
- dataset IndexesActive is True.

Syntax

```
property ActualActive: Boolean;
```

See Also

Active (see page 655), Selected (see page 659), TADDataSet.IndexesActive

1.17.2.1.6.1.2 TADIndex.DataSet Property

Returns the reference to containing dataset.

Description

Use the DataSet property to get reference to dataset, containing this view.

Syntax

```
property DataSet: TADDataSet;
```

See Also

TADDataSet (see page 584)

1.17.2.1.6.2 published

1.17.2.1.6.2.1 TADIndex.Active Property

Controls when dataset view must be maintained.

Description

Set Active property value to True to activate view maintenance. By default it is False.

All views will be maintained when TADDataSet.IndexesActive (see page 600) is True. Really, view will be maintained only when ActualActive (see page 654) returns True.

To make dataset view current in the dataset, set Selected (see page 659) property to True.

Syntax

```
property Active: Boolean;
```

See Also

ActualValue, Selected (see page 659), TADDataSet.IndexesActive

1.17.2.1.6.2.2 TADIndex.CaseInsFields Property

Specifies fields for which sorting must be case-insensitive.

Description

The CaseInsFields specifies fields for which sorting must be case-insensitive.

This fields must be included into Fields (see page 657) property value too. If index is expressional (Expression (see page 656) is specified), then CaseInsFields has no effect. Alternatively case insensitive fields may be specified right in the Fields (see page 657) property.

Syntax

```
property CaseInsFields: String;
```

See Also

Fields (see page 657), DescFields (see page 656)

Example

```
with ADQuery1.Indexes.Add do begin
  Name := 'i1';
  Fields := 'Name;BirthDate';
  CaseInsFields := 'Name';
  Active := True;
  Selected := True;
end;
```

1.17.2.1.6.2.3 TADIndex.DescFields Property

Specifies fields for which sorting must be done in descending order.

Description

The DescFields specifies fields for which sorting must be done in descending order.

This fields must be included into Fields (see page 657) property value. If index is expressional (Expression (see page 656) is specified), then DescFields has no effect. Alternatively descending fields may be specified right in the Fields (see page 657) property.

Syntax

```
property DescFields: String;
```

See Also

Fields (see page 657), CaseInsFields (see page 655)

Example

```
with ADQuery1.Indexes.Add do begin
  Name := 'i1';
  Fields := 'Name;BirthDate';
  DescFields := 'BirthDate';
  Active := True;
  Selected := True;
end;
```

1.17.2.1.6.2.4 TADIndex.Distinct Property

Controls returning of distinct view value.

Description

Set the Distinct property to True, to include into view only a first record from the set of records with the same index value. The option is similar to the SQL DISTINCT keyword.

Syntax

```
property Distinct: Boolean;
```

1.17.2.1.6.2.5 TADIndex.Expression Property

Specifies expression which must be used to sort records.

Description

Set Expression property value to a formula. The dataset view records will be ordered by these formula values, calculated for each record.

Assignment to the Expression property value erases a Fields (see page 657) property value. The view sort order may be either expressional (Expression is specified), either field list based (Fields (see page 657) is specified).

Also use Options (see page 658) to setup sorting.

Syntax

```
property Expression: String;
```

See Also

Fields (see page 657), Options (see page 658)

Example

```
ADQuery1.Indexes[0].Expression := 'Upper(Left(Name, 1))';
```

1.17.2.1.6.2.6 TADIndex.Fields Property

Identifies the fields that comprise the index.

Description

Set Fields property value to the ';' separated list of field names with optional modifiers. The dataset view records will be ordered by these field values.

The general syntax for property value is: field[:[D][A][N]][;...]. There:

- D - is descending modifier. Also may be used DescFields (see page 656).
- A - is ascending modifier. It is default order.
- N - case-insensitive modifier. Also may be used CaseInsFields (see page 655).

Assignment to Fields property value erases Expression property value. The view sort order may be either expressional (Expression (see page 656) is specified), either field list based (Fields is specified).

Also use Options (see page 658) to setup sorting.

Syntax

```
property Fields: String;
```

See Also

DescFields (see page 656), CaseInsFields (see page 655), Expression (see page 656), Options (see page 658)

Example

```
// sort by Name - case insensitive and by BithDate - descending
ADQuery1.Indexes[0].Fields := 'Name:N;BirthDate:D';
```

1.17.2.1.6.2.7 TADIndex.Filter Property

Specifies the text of a view filter.

Description

Use Filter property to specify a view filter. When Filter is applied to view, only those records that meet a filter's conditions are available to an application. Filter contains the string that describes the filter condition. See Writing Expressions (see page 110) for details of expression syntax.

To specify sort order for view use either Expression (see page 656), either Fields (see page 657) properties. To setup filtering use FilterOptions (see page 657) property.

Syntax

```
property Filter: String;
```

See Also

Fields (see page 657), Expression (see page 656), FilterOptions (see page 657)

Example

```
ADMemTable.Indexes[0].Filter := 'Name LIKE ''A%''';
```

1.17.2.1.6.2.8 TADIndex.FilterOptions Property

Specifies filtering options.

Description

Use FilterOptions to specify filtering options for dataset view:

Option	Description
ekNoCase	String comparisons will be performed case-insensitive.
ekPartial	String comparisons will be performed partially.

All these options may be implemented by programmer using additional constructions in Filter. ekPartial - using LIKE operator, and ekNoCase using UPPER / LOWER functions.

Syntax

```
property FilterOptions: TADExpressionOptions;
```

See Also

Filter (see page 657)

1.17.2.1.6.2.9 TADIndex.Name Property

Specifies the name of dataset view.

Description

Use Name to specify the name of a dataset view. This name is used to display item at design time and to find item in a collection.

Syntax

```
property Name: String;
```

See Also

TADIndexes.FindIndex, TADIndexes.IndexByName

1.17.2.1.6.2.10 TADIndex.Options Property

Specifies sorting options.

Description

Use Options to specify sorting options for dataset view:

Option	Description
soNoCase	Use case-insensitive sorting for all fields or expression result.
soNullFirst	Put NULL values before not-NULL values when sorting in ascending order. Otherwise - after.
soDescNullLast	Put NULL values after not-NULL values when sorting in descending order. Otherwise - before.
soDescending	Use descending order for full sorting.
soUnique	Enforce uniqueness in view. If the application will try to insert a new record with duplicating sorting field values and TADDataSet.ConstraintsEnabled (see page 594) is True, then exception will be raised.
soPrimary	As above, but dataset view with soPrimary may be only one.
soNoSymbols	Ignore symbols when comparing string to build the sort order.

Default value is empty set.

Syntax

```
property Options: TADSortOptions;
```

See Also

Fields (see page 657), Expression (see page 656), CaseInsFields (see page 655), DescFields (see page 656), TADDataSet.ConstraintsEnabled (see page 594)

1.17.2.1.6.2.11 TADIndex.Selected Property

Specifies, if a dataset view is the current dataset view.

Description

Set selected to True to make the dataset view active and the current one in a dataset. An alternative is to set TADDataSet.IndexName (see page 601) to the name of a dataset view. The Selected property returns True, if the dataset view is selected.

Syntax

```
property Selected: Boolean;
```

See Also

TADDataSet.IndexName, TADDataSet.IndexFieldNames, TADIndex.Active (see page 655)

1.17.2.1.7 TADIndexes Class

TADIndexes is a collection of TADIndex (see page 653) objects.

public

public	Description
Items (see page 659)	Returns TADIndex (see page 653) object by it index.
Add (see page 660)	Appends new TADIndex (see page 653) object to a collection.
FindIndex (see page 660)	Find TADIndex (see page 653) object by its name.
FindIndexForFields (see page 660)	Find existing TADIndex (see page 653) object by sort fields and options.
IndexByName (see page 661)	Find TADIndex (see page 653) object by its name and raises exception if it is not found.

Class Hierarchy



File

uADCompDataSet

Description

TADIndexes is a list of all dataset views, maintained by dataset. A single TADIndex object represents a single maintained dataset view. The dataset views are maintained by dataset automatically, if:

- TADIndex.Active (see page 655) is True and view is really active (see TADAggregate.ActualActive (see page 573));
- TADDataSet.IndexesActive (see page 600) is True.

Syntax

```
TADIndexes = class(TCollection);
```

See Also

TADIndex (see page 653), TADDataSet (see page 584), TADDataSet.IndexesActive

1.17.2.1.7.1 public

1.17.2.1.7.1.1 TADIndexes.Items Property

Returns TADIndex (see page 653) object by it index.

Description

Use Items indexed property to get TADIndex (see page 653) object by it index.

The property is default one, so you does not need to write ADQuery1.Indexes.Items[i], just ADQuery1.Indexes[i].

Syntax

```
property Items [AIndex: Integer]: TADIndex;
```

See Also

TADIndex (see page 653), TCollection.Count

1.17.2.1.7.1.2 TADIndexes.Add Method

Appends new TADIndex (see page 653) object to a collection.

Description

Call Add method to add new empty TADIndex (see page 653) object to a collection.

Syntax

```
function Add: TADIndex;
```

See Also

Items (see page 659), TCollection.Count

Example

```
with ADQuery1.Indexes.Add do begin
  Name := 'i1';
  Fields := 'Name';
  Active := True;
end;
```

1.17.2.1.7.1.3 TADIndexes.FindIndex Method

Find TADIndex (see page 653) object by its name.

Parameters

Parameters	Description
const AName: String	A name of the view to find.

Description

Use FindIndex method to find view by its name. If object is not found, then FindIndex returns nil.

Syntax

```
function FindIndex(const AName: String): TADIndex;
```

See Also

TADIndex.Name (see page 658), IndexByName (see page 661), FindIndexForFields (see page 660), Items (see page 659)

Example

```
var
  oInd: TADIndex;
...
oInd := ADQuery1.Indexes.FindIndex('i1');
if oInd = nil then
  with ADQuery1.Indexes.Add do begin
    Name := 'i1';
    Fields := 'Name';
    Active := True;
  end;
```

1.17.2.1.7.1.4 TADIndexes.FindIndexForFields Method

Find existing TADIndex (see page 653) object by sort fields and options.

Parameters

Parameters	Description
const AFields: String	A fields to find.
ARequiredOptions: TADSortOptions = []	A required options.
AProhibitedOptions: TADSortOptions = []	A prohibited options.

Description

Use FindIndex method to find view by sort fields and sort options. If view is not found, then FindIndexForFields returns nil.

The matched view must have:

- Fields starting from AFields. Fields may include more fields, than AFields.
- All ARequiredOptions options must be included into Options (see page 658).
- None of AProhibitedOptions must be included into Options (see page 658).

Syntax

```
function FindIndexForFields(const AFields: String; ARequiredOptions: TADSortOptions = [];
AProhibitedOptions: TADSortOptions = []): TADIndex;
```

See Also

TADIndex.Fields (see page 657), TADIndex.Options (see page 658), FindIndex (see page 660), IndexByName (see page 661), Items (see page 659)

Example

```
var
  oInd: TADIndex;
...
oInd := ADMemTable1.Indexes.FindIndexForFields('ID', [soUnique], []);
if oInd <> nil then
  oInd.Selected := True;
```

1.17.2.1.7.1.5 TADIndexes.IndexByName Method

Find TADIndex (see page 653) object by its name and raises exception if it is not found.

Parameters

Parameters	Description
const AName: String	A name of the view to find.

Description

Use IndexByName method to find dataset view by its name. If view is not found, then IndexByName raises exception.

Syntax

```
function IndexByName(const AName: String): TADIndex;
```

See Also

TADIndex.Name (see page 658), FindIndex (see page 660), Items (see page 659)

Example

```
ADQuery1.Indexes.IndexByName('i1').Selected := True;
```

1.17.2.1.8 TADMasterDataLink Class

TADMasterDataLink controlls master-detail link between two datasets.

public

public	Description
ActualDelayedScrollEnabled (see page 662)	Returns True, if a scrolling will be really delayed.

DelayedScrollEnabled (see page 662)	Returns True, if a delayed scrolling is enabled.
CancelSync (see page 663)	Cancels delayed detail dataset refreshing.
DisableDelayedScroll (see page 663)	Disables delayed refresh of detail dataset.
DisableScroll (see page 664)	Disables refreshing of detail dataset.
EnableDelayedScroll (see page 664)	Reenables delayed refresh of detail dataset.
EnableScroll (see page 664)	Reenables refresh of detail dataset.
Synchronize (see page 665)	Synchronizes detail dataset with master dataset.

Class Hierarchy



File

uADCompDataSet

Description

The TADMasterDataLink is a class controlling master-detail link between two datasets. Each FireDAC dataset has public property MasterLink (see page 603). Use it to control delayed refresh of detail dataset on master dataset scroll or update.

To turn on delayed refresh use TADDataSet.FetchOptions (see page 651).DetailDelay (see page 854) property. Use DisableDelayedScroll / EnableDelayedScroll to temporary disable delayed refresh and use immediate refresh instead.

Use DisableScroll / EnableScroll to temporary disable refresh.

Syntax

```
TADMasterDataLink = class(TMasterDataLink);
```

See Also

Master-Detail Relationship (see page 104), DetailDelay (see page 854)

1.17.2.1.8.1 public

1.17.2.1.8.1.1 TADMasterDataLink.ActualDelayedScrollEnabled Property

Returns True, if a scrolling will be really delayed.

Description

Use the ActualDelayedScrollEnabled property to detect when a scrolling will be really delayed for this dataset.

The property returns True, when DelayedScrollEnabled (see page 662) is True and TADDataSet.FetchOptions (see page 651).DetailDelay (see page 854) is greater than zero.

Syntax

```
property ActualDelayedScrollEnabled: Boolean;
```

See Also

DelayedScrollEnabled (see page 662), DetailDelay (see page 854)

1.17.2.1.8.1.2 TADMasterDataLink.DelayedScrollEnabled Property

Returns True, if a delayed scrolling is enabled.

Description

Use the DelayedScrollEnabled property to detect when delayed scrolling is enabled for this dataset.

The property returns True, when for this and for the all master datasets (if any) were no DisableDelayedScroll (see page 663) calls. Otherwise, the property returns False. The property does not check TADDataSet.FetchOptions (see page 651).DetailDelay (see page 854) for zero value.

Use ActualDelayedScrollEnabled (see page 662) to check, will happens a delayed scrolling or not.

Syntax

```
property DelayedScrollEnabled: Boolean;
```

See Also

ActualDelayedScrollEnabled (see page 662), DisableDelayedScroll (see page 663), EnableDelayedScroll (see page 664), DetailDelay (see page 854)

1.17.2.1.8.1.3 TADMasterDataLink.CancelSync Method

Cancels delayed detail dataset refreshing.

Parameters

Parameters	Description
AWithDetails: Boolean = True	If True, then will be canceled refreshes for all sub-detail datasets too.

Description

Call CancelSync method to cancel delayed detail dataset refreshing. The method has effect only is delayed refreshing is in effect.

Syntax

```
procedure CancelSync(AWithDetails: Boolean = True);
```

See Also

Synchronize (see page 665)

1.17.2.1.8.1.4 TADMasterDataLink.DisableDelayedScroll Method

Disables delayed refresh of detail dataset.

Description

Call DisableDelayedScroll method of the MasterLink property of detail dataset to disable delayed refresh of detail dataset and use immediate refresh instead. The method has effect only is delayed refreshing is in effect.

The method is useful in an application code, which needs to walk through master dataset and read also detail dataset records. And detail dataset has TADDataSet.FetchOptions (see page 651).DetailDelay (see page 854) property value > 0. This allows to combine delayed refreshing of detail dataset, while user scrolls through master dataset in GUI and synchronized refreshing in code.

To re-enable delayed refresh of detail dataset, use EnableDelayedScroll (see page 664) method. The DisableDelayedScroll / EnableDelayedScroll (see page 664) method calls must be in pairs and may be nested. FireDAC uses counter to track DisableDelayedScroll / EnableDelayedScroll (see page 664) nested calls. The topmost call of DisableDelayedScroll method synchronizes detail dataset with master, if they are out of sync.

To synchronize detail dataset with its master single time, use Synchronize (see page 665) method.

Syntax

```
procedure DisableDelayedScroll;
```

See Also

Synchronize (see page 665), EnableDelayedScroll (see page 664), DetailDelay (see page 854)

Example

```
qDetail.MasterLink.DisableDelayedScroll;
try
  qMaster.First;
  while not qMaster.Eof do begin
    if qMaster.Fields[i].AsInteger = 100 then begin
      // read qDetail dataset - it is synchronized with qMaster
    end;
    qMaster.Next;
```

```
  end;
finally
  qDetail.MasterLink.EnableDelayedScroll;
end;
```

1.17.2.1.8.1.5 TADMasterDataLink.DisableScroll Method

Disables refreshing of detail dataset.

Description

Call DisableScroll method of the MasterLink property of detail dataset to disable refresh of a detail dataset.

The DisableControls method of master dataset disables all master dataset "feedback". The DisableScroll method allows to turn off detail dataset refreshing of single detail dataset.

To re-enable refresh of detail dataset, use EnableScroll (see page 664) method. The DisableScroll / EnableScroll (see page 664) method calls must be in pairs and may be nested. FireDAC uses counter to track DisableScroll / EnableScroll (see page 664) nested calls. The topmost call of EnableScroll method synchronizes detail dataset with master, if they are out of sync.

Syntax

```
procedure DisableScroll;
```

See Also

EnableScroll (see page 664)

Example

```
qDetail.MasterLink.DisableScroll;
try
  qMaster.First;
  while not qMaster.Eof do begin
    if qMaster.Fields[i].AsInteger = 100 then begin
      qDetail.ApplyMaster;
      // read qDetail dataset - it is synchronized with qMaster
    end;
    qMaster.Next;
  end;
finally
  qDetail.MasterLink.EnableScroll;
end;
```

1.17.2.1.8.1.6 TADMasterDataLink.EnableDelayedScroll Method

Reenables delayed refresh of detail dataset.

Description

Call EnableDelayedScroll method of the MasterLink property of detail dataset to re-enable delayed refresh of detail dataset and stop to use immediate refresh. The method reverts effect of DisableDelayedScroll (see page 663) method call.

Syntax

```
procedure EnableDelayedScroll;
```

See Also

Synchronize (see page 665), DisableDelayedScroll (see page 663)

1.17.2.1.8.1.7 TADMasterDataLink.EnableScroll Method

Reenables refresh of detail dataset.

Description

Call EnableScroll method of the MasterLink property of detail dataset to re-enable delayed refresh of detail dataset. The method reverts effect of DisableScroll (see page 664) method call.

Syntax

```
procedure EnableScroll;
```

See Also

DisableScroll (see page 664)

1.17.2.1.8.1.8 TADMasterDataLink.Synchronize Method

Synchronizes detail dataset with master dataset.

Parameters

Parameters	Description
AWithDetails: Boolean = True	If True, then will be synchronized all sub-detail datasets too.

Description

Call Synchronize method to synchronize not yet refreshed detail dataset with the master dataset. The method is useful when delayed refreshing is in effect (TADDataSet.FetchOptions (see page 651).DetailDelay (see page 854) > 0) or when refreshing is disabled (DisableScroll (see page 664)).

Optionally, you can use DisableDelayedScroll (see page 663) / EnableDelayedScroll (see page 664) to temporary disable / enable delayed refreshing.

Syntax

```
procedure Synchronize(AWithDetails: Boolean = True);
```

See Also

CancelSync (see page 663), DisableDelayedScroll (see page 663), EnableDelayedScroll (see page 664), DisableScroll (see page 664), EnableScroll (see page 664), DetailDelay (see page 854)

Example

```
qMaster.First;
while not qMaster.Eof do begin
  if qMaster.Fields[i].AsInteger = 100 then begin
    qDetail.MasterLink.Synchronize(True);
    // read qDetail dataset - here it is synchronized with qMaster
  end;
  qMaster.Next;
end;
```

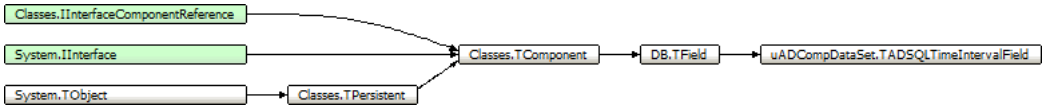
1.17.2.1.9 TADSQLTimeIntervalField Class

TADSQLTimeIntervalField represents a time interval field in a dataset.

public

public	Description
AsSQLTimeInterval (see page 666)	Value of the time interval field.
IntervalKind (see page 666)	Returns time interval kind.

Class Hierarchy



File

uADCompDataSet

Description

TADSQLTimeIntervalField encapsulates the fundamental behavior common to fields that contain SQL time interval data. The

time inverval data in most of the DBMS's is represented by the INTERVAL YEAR TO MONTH or INTERVAL DAY TO SECOND SQL data types.

The TADSQLTimeIntervalField corresponds to the following data type:

- Delphi 2006 and higher - ftOraInterval;
- Delphi 2005 or less - ftParadoxOle.

These two data types are synonyms for FireDAC.

Syntax

```
TADSQLTimeIntervalField = class(TField);
```

See Also

uADStanSQLTimeInt

1.17.2.1.9.1 public

1.17.2.1.9.1.1 TADSQLTimeIntervalField.AsSQLTimeInterval Property

Value of the time interval field.

Description

Use AsSQLTimeInterval property to read / write time interval field value, represented by the TADSQLTimeInterval structure.

Alternatively, you can use TField.AsVariant property to assign time interval, packed into Variant data type, or TField.AsString.

Syntax

```
property AsSQLTimeInterval: TADSQLTimeInterval;
```

See Also

TField.AsVariant, TField.AsString, uADStanSQLTimeInt.TADSQLTimeInterval

Example

```
var
  rInt: TADSQLTimeInterval;
...
  rInt.Sign := 1;
  rInt.Kind := itYear2Month;
  rInt.Years := 5;
  rInt.Months := 11;
  ADQuery1EventInt.AsSQLTimeInterval := rInt;
```

1.17.2.1.9.1.2 TADSQLTimeIntervalField.IntervalKind Property

Returns time interval kind.

Description

Use IntervalKind property to read the time interval kind of this field. The property value may be itDay2Second or itYear2Month.

Syntax

```
property IntervalKind: TADSQLTimeIntervalKind;
```

See Also

uADStanSQLTimeInt

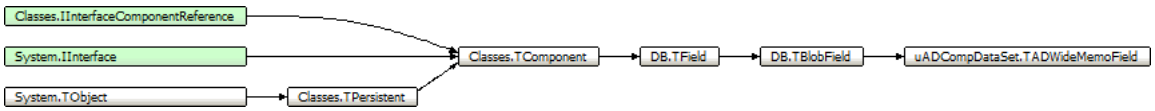
1.17.2.1.10 TADWideMemoField Class

TADWideMemoField represents a Unicode encoded memo field in a dataset.

public

public	Description
Value (see page 667)	Value of long text field.

Class Hierarchy



File

uADCompDataSet

Description

TADWideMemoField encapsulates the fundamental behavior common to fields that contain Unicode encoded (UTF8 / UTF16) long text data of arbitrary length. Memo fields are a form of binary large object (BLOB) field where the data consists of simple text.

TADWideMemoField was introduced at first to support Unicode long text data on early Delphi versions, which does not had TWideMemoField.

The TADWideMemoField corresponds to the following data type:

- Delphi 2006 and higher - ftWideMemo;
- Delphi 2005 or less - ftFmtMemo.

These two data types are synonyms for FireDAC.

Syntax

```
TADWideMemoField = class(TBlobField);
```

See Also

TWideMemoField

1.17.2.1.10.1 public

1.17.2.1.10.1.1 TADWideMemoField.Value Property

Value of long text field.

Description

Use Value property to read / write Unicode encoded long text field value. The value is always UTF16 encoded.

Syntax

```
property Value: TADWideString;
```

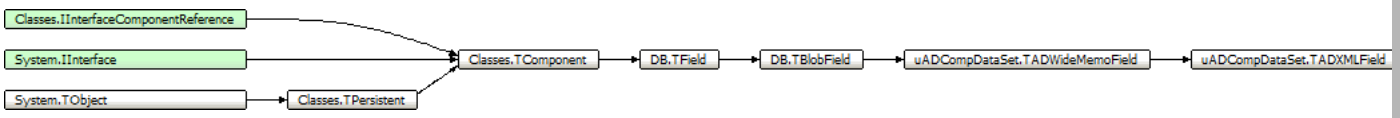
1.17.2.1.11 TADXMLField Class

TADXMLField represents a Unicode encoded XML data field in a dataset.

public

public	Description
SchemaName (see page 668)	Returns optional XML schema name.

Class Hierarchy



File

uADCompDataSet

Description

TADXMLField encapsulates the fundamental behavior common to fields that contain XML text data of arbitrary length. XML fields are a form of binary large object (BLOB) field where the data consists of a XML text.

The TADXMLField corresponds to the ftDBaseOle data type.

Syntax

```
TADXMLField = class(TADWideMemoField);
```

See Also

TADWideMemoField (see page 666)

1.17.2.1.11.1 public

1.17.2.1.11.1.1 TADXMLField.SchemaName Property

Returns optional XML schema name.

Description

Use SchemaName property to read the optional XML schema name for this field. The semantic of this value is DBMS dependent.

Syntax

```
property SchemaName: String;
```

1.17.3 uADCompGUIx Namespace

Contains most of the UI components, including TADGUIxAsyncExecuteDialog (see page 669), TADGUIxErrorDialog (see page 672), TADGUIxLoginDialog (see page 675), TADGUIxScriptDialog (see page 680) and TADGUIxWaitCursor (see page 683) classes.

Classes

Class	Description
TADGUIxAsyncExecuteDialog (see page 669)	The dialog showing a SQL query execution progress.
TADGUIxComponent (see page 671)	The base class for all FireDAC GUIx components.
TADGUIxErrorDialog (see page 672)	The dialog displaying FireDAC exceptions.
TADGUIxLoginDialog (see page 675)	The dialog allows the users to enter their DB credentials.
TADGUIxScriptDialog (see page 680)	The dialog showing a SQL script execution progress.
TADGUIxWaitCursor (see page 683)	The component allowing to control the wait cursor.

Description

The uADCompGUIx unit contains most of the UI components:

- TADGUIxAsyncExecuteDialog (see page 669) - dialog showing FireDAC SQL execute progress.
- TADGUIxErrorDialog (see page 672) - dialog showing FireDAC specific exceptions.

- TADGUIxLoginDialog (see page 675) - dialog allowing the end users to enter their DB credentials.
- TADGUIxScriptDialog (see page 680) - dialog showing FireDAC SQL script execution progress.
- TADGUIxWaitCursor (see page 683) - component implementing and controlling the wait cursor.

See Also

TADResourceOptions.CmdExecMode (see page 880), Handling Errors (see page 44), uADStanError (see page 831), Establishing Connection (see page 37), Executing SQL Scripts (see page 88), uADCompScript Namespace (see page 684)

1.17.3.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADGUIxAsyncExecuteDialog (see page 669)	The dialog showing a SQL query execution progress.
TADGUIxComponent (see page 671)	The base class for all FireDAC GUIx components.
TADGUIxErrorDialog (see page 672)	The dialog displaying FireDAC exceptions.
TADGUIxLoginDialog (see page 675)	The dialog allows the users to enter their DB credentials.
TADGUIxScriptDialog (see page 680)	The dialog showing a SQL script execution progress.
TADGUIxWaitCursor (see page 683)	The component allowing to control the wait cursor.

1.17.3.1.1 TADGUIxAsyncExecuteDialog Class

The dialog showing a SQL query execution progress.

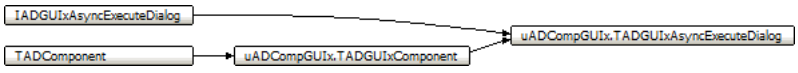
public

public	Description
AsyncDialog (see page 670)	Reference to dialog interface.

published

published	Description
Caption (see page 670)	The dialog caption.
HideDelay (see page 670)	Specifies a delay before dialog hiding.
OnHide (see page 670)	Fires after the dialog will be hidden.
OnShow (see page 671)	Fires before the dialog will be displayed.
Prompt (see page 671)	Specifies the dialog prompt.
ShowDelay (see page 671)	Specifies a delay before dialog showing.

Class Hierarchy



File

uADCompGUIx

Description

Use the TADGUIxAsyncExecuteDialog in FireDAC GUI application to display SQL query execution progress. The dialog allows:

- to see, that a query is executing;
- to cancel query execution.

The TADQuery (see page 476) will show the dialog automatically, when TADResourceOptions.CmdExecMode (see page 880) is set to amCancelDialog. Only single query may be executed in this mode at each moment at a time.

To use the dialog, drop it on a form or a data module. All additional setups are optional.

IADGUIxAsyncExecuteDialog has Forms and FMX implementations. Use Provider (see page 672) property to select the desired implementation.

Syntax

```
TADGUIxAsyncExecuteDialog = class(TADGUIxComponent, IADGUIxAsyncExecuteDialog);
```

See Also

TADResourceOptions.CmdExecMode (see page 880)

1.17.3.1.1.1 public

1.17.3.1.1.1.1 TADGUIxAsyncExecuteDialog.AsyncDialog Property

Reference to dialog interface.

Description

Use the AsyncDialog to get a dialog interface reference.

Syntax

```
property AsyncDialog: IADGUIxAsyncExecuteDialog;
```

1.17.3.1.1.2 published

1.17.3.1.1.2.1 TADGUIxAsyncExecuteDialog.Caption Property

The dialog caption.

Description

Use the Caption property to specify the dialog caption.

Syntax

```
property Caption: String;
```

See Also

Prompt (see page 671)

1.17.3.1.1.2.2 TADGUIxAsyncExecuteDialog.HideDelay Property

Specifies a delay before dialog hiding.

Description

Use the HideDelay to specify the delay in milliseconds after a query execution will be finished and other query execution will not be started before the dialog will be hidden. When an application executes one query by one, then setting HideDelay to a correct value allows to avoid flickering.

Syntax

```
property HideDelay: Integer;
```

See Also

ShowDelay (see page 671).

1.17.3.1.1.2.3 TADGUIxAsyncExecuteDialog.OnHide Event

Fires after the dialog will be hidden.

Description

Use the OnHide event to perform the actions after the dialog was hidden at the end of a query execution.

Syntax

```
property OnHide: TNotifyEvent;
```

See Also

OnShow (see page 671)

1.17.3.1.1.2.4 TADGUIxAsyncExecuteDialog.OnShow Event

Fires before the dialog will be displayed.

Description

Use the OnShow event to perform the actions before the dialog will be displayed.

Syntax

```
property OnShow: TNotifyEvent;
```

See Also

OnHide (see page 670)

1.17.3.1.1.2.5 TADGUIxAsyncExecuteDialog.Prompt Property

Specifies the dialog prompt.

Description

Use the Prompt property to specify the dialog prompt.

Syntax

```
property Prompt: String;
```

See Also

Caption (see page 670)

1.17.3.1.1.2.6 TADGUIxAsyncExecuteDialog.ShowDelay Property

Specifies a delay before dialog showing.

Description

Use the ShowDelay to specify the delay in milliseconds after a query execution is started and before the dialog will be shown. When an application executes fast queries, then setting ShowDelay to a correct value allows to avoid flickering.

Syntax

```
property ShowDelay: Integer;
```

See Also

HideDelay (see page 670)

1.17.3.1.2 TADGUIxComponent Class

The base class for all FireDAC GUIx components.

published

published	Description
Provider (see page 672)	Specifies implementation kind.

Class Hierarchy



File

uADCompGUIx

Description

The TADGUIxComponent class is not used directly and serves as the base class for all FireDAC GUIx components. The main property is Provider (see page 672).

Syntax

```
TADGUIxComponent = class(TADComponent);
```

1.17.3.1.2.1 published

1.17.3.1.2.1.1 TADGUIxComponent.Provider Property

Specifies implementation kind.

Description

Use Provider property to select the GUIx component implementation:

Value	Description
'Forms'	VCL based implementation for Delphi / C++ Builder. LCL based implementation for Lazarus / FPC.
'FMX'	FireMonkey based implementation for Delphi / C++ Builder XE2 and higher.
'Console'	Console based implementation.

After setting the Provider property at design-time, save the form and corresponding implementation unit will be automatically added to the form "uses" clause. At run-time you will need to add the corresponding implementation unit by hands to the "uses" clause. When corresponding implementation is not linked into application, then an exception may be raised. See the corresponding component description for supported implementations.

Note, when Provider property value was switched from one to another value, then developer may need to delete implementation units for old Provider value from the uses clauses. Eg, switching from 'Forms' to 'FMX' requires to delete uADGUIxFormsXxx units.

If Provider property value is not explicitly assigned, then default value will be used. By default it is 'FMX' for FireMonkey based applications, 'Console' for Windows console applications, 'Forms' - otherwise. The TADGUIxWaitCursor (see page 683).Provider or uADGUIxIntf.FADGUIxProvider global variable allow to specify the default GUIx provider.

Provider property value must be assigned before first component usage.

Syntax

```
property Provider: String;
```

Example

```
uses
   uADGUIxFMXfLogin;
....
ADGUIxLoginDialog1.Provider := 'FMX';
ADConnection1.LoginDialog := ADGUIxLoginDialog1;
ADConnection1.Connected := True;
```

1.17.3.1.3 TADGUIxErrorDialog Class

The dialog displaying FireDAC exceptions.

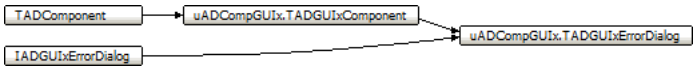
public

public	Description
ErrorDialog (see page 673)	Reference to dialog interface.
Execute (see page 673)	Displays the error dialog.

published

published	Description
Caption (see page 674)	The dialog caption.
Enabled (see page 674)	Controls the dialog usage.
OnHide (see page 674)	Fires after the dialog will be hidden.
OnShow (see page 674)	Fires before the dialog will be displayed.
StayOnTop (see page 675)	Controls the position of dialog.

Class Hierarchy



File

uADCompGUIx

Description

Use the TADGUIxErrorDialog in FireDAC GUI application to display FireDAC exceptions to the user. The dialog allows:

- to browse multiple error items, included into an exception;
- error kind, native error code, erroneous DB object, etc;
- original SQL command and parameter values leaded to an exception.

For that the dialog hooks TApllication.OnException event. To use the dialog, drop it on a form or a data module. All additional setups are optional.

IADGUIxErrorDialog has Forms and FMX implementations. Use Provider (see page 672) property to select the desired implementation.

Syntax

```
TADGUIxErrorDialog = class(TADGUIxComponent, IADGUIxErrorDialog);
```

See Also

Handling Errors (see page 44), EADDBEngineException (see page 833)

1.17.3.1.3.1 public

1.17.3.1.3.1.1 TADGUIxErrorDialog.ErrorDialog Property

Reference to dialog interface.

Description

Use the ErrorDialog to get a dialog interface reference.

Syntax

```
property ErrorDialog: IADGUIxErrorDialog;
```

1.17.3.1.3.1.2 TADGUIxErrorDialog.Execute Method

Displays the error dialog.

Parameters

Parameters	Description
E: EADDBEngineException	An exception object to show.

Description

Use the Execute method to manually display the error dialog and show the content of an exception object.

Syntax

```
procedure Execute(E: EADDBEngineException);
```

1.17.3.1.3.2 published

1.17.3.1.3.2.1 TADGUIxErrorDialog.Caption Property

The dialog caption.

Description

Use the Caption property to specify a custom dialog caption.

Syntax

```
property Caption: String;
```

1.17.3.1.3.2.2 TADGUIxErrorDialog.Enabled Property

Controls the dialog usage.

Description

Use the Enabled to to control the dialog usage. When Enabled is True, then dialog hooks the Application.OnException event and will display FireDAC exceptions. When Enabled is False, the dialog unhooks the event and will not display exceptions. The default value is True.

Syntax

```
property Enabled: Boolean;
```

1.17.3.1.3.2.3 TADGUIxErrorDialog.OnHide Property

Fires after the dialog will be hidden.

Description

Use the OnHide event to perform the actions after the dialog was displayed and user pressed OK button to hide the dialog.

Syntax

```
property OnHide: TADGUIxErrorDialogEvent;
```

See Also

OnShow (see page 674), Enabled (see page 674)

1.17.3.1.3.2.4 TADGUIxErrorDialog.OnShow Property

Fires before the dialog will be displayed.

Description

Use the OnShow event to perform the actions before the dialog will be displayed.

Syntax

```
property OnShow: TADGUIxErrorDialogEvent;
```

See Also

OnHide (see page 674), Enabled (see page 674)

1.17.3.1.3.2.5 TADGUIxErrorDialog.StayOnTop Property

Controls the position of dialog.

Description

Use the StayOnTop property to control the dialog position on the screen. When StayOnTop is True, then the dialog will be displayed on top of all windows on the screen. When False, then it will be displayed on top of the application forms.

Syntax

```
property StayOnTop: Boolean;
```

See Also

Enabled (see page 674)

1.17.3.1.4 TADGUIxLoginDialog Class

The dialog allows the users to enter their DB credentials.

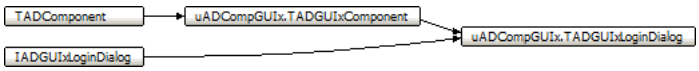
public

public	Description
ConnectionDef (see page 676)	Specifies the connection definition to modify.
LoginDialog (see page 676)	Reference to dialog interface.
Execute (see page 676)	Executes the login dialog.
GetAllLoginParams (see page 677)	Fills the VisibleItems (see page 679).

published

published	Description
Caption (see page 677)	The dialog caption.
ChangeExpiredPassword (see page 677)	Controls the user ability to change an expired password.
HistoryEnabled (see page 677)	Controls the login history storage.
HistoryKey (see page 678)	Specifies where to store login history.
HistoryStorage (see page 678)	Specifies what storage kind to use for the login history.
HistoryWithPassword (see page 678)	Controls should the login history contain the passwords or not.
LoginRetries (see page 678)	Specifies the number of unsuccessful login retries.
OnChangePassword (see page 679)	Fires when user password is expired.
OnHide (see page 679)	Fires after the dialog will be hidden.
OnLogin (see page 679)	Fires when user should provide database credentials.
OnShow (see page 679)	Fires before the dialog will be displayed.
VisibleItems (see page 679)	Specifies the visible login dialog fields.

Class Hierarchy



File

uADCompGUIx

Description

Use the TADGUIxLoginDialog in FireDAC GUI application to allow to the users:

- enter their credentials;
- optionally change expired password.

The dialog may be used globally by all connections in an applications or may be linked directly to TADConnection (see page 279).To use the dialog globally, drop it on a form or a data module. To use dialog privately by some connection, drop

component on a form and set TADConnection.LoginDialog (see page 285) property to this dialog.

The dialog may track the login history. This is useful in environments, which does not require security, for example the developer workstations. In such case, set HistoryEnabled (see page 677) to True. The dialog will store recent credentials in registry or INI-file, as specified by HistoryStorage (see page 678) and HistoryKey (see page 678) properties. And will offer the recent credentials to the user at login.

To limit the number of login retries an application should set LoginRetries (see page 678) property.

IADGUIxLoginDialog has Forms and FMX implementations. Use Provider (see page 672) property to select the desired implementation.

Syntax

```
TADGUIxLoginDialog = class(TADGUIxComponent, IADGUIxLoginDialog);
```

See Also

Establishing Connection (see page 37), TADCustomConnection.LoginDialog (see page 676)

Example

```
ADGUIxLoginDialog1.Caption := 'Your ERP login';
with ADGUIxLoginDialog1.VisibleItems do begin
  Clear;
  Add('User_name=Your name');
  Add('Password=Your key');
end;
ADGUIxLoginDialog1.HistoryEnabled := False;

ADConnection1.ConnectionDefName := 'Oracle_Demo';
ADConnection1.LoginDialog := ADGUIxLoginDialog1;
ADConnection1.Connected := True;
```

1.17.3.1.4.1 public

1.17.3.1.4.1.1 TADGUIxLoginDialog.ConnectionDef Property

Specifies the connection definition to modify.

Description

Use the ConnectionDef property to specify or to get a reference to a connection definition. The dialog uses the connection definition to retrieve specified parameters and to save the entered by user parameter values. When dialog is used with TADConnection, then connection object will automatically specify a connection definition.

Syntax

```
property ConnectionDef: IADStanConnectionDef;
```

1.17.3.1.4.1.2 TADGUIxLoginDialog.LoginDialog Property

Reference to dialog interface.

Description

Use the LoginDialog to get a dialog interface reference.

Syntax

```
property LoginDialog: IADGUIxLoginDialog;
```

1.17.3.1.4.1.3 TADGUIxLoginDialog.Execute Method

Executes the login dialog.

Parameters

Parameters	Description
ALoginAction: TADGUIxLoginAction	An action to perform after the user successfully entered the database credentials.

Description

Use the Execute method to manually execute the login dialog and enter user credentials. When dialog is used with TADConnection, then connection object will automatically execute the login dialog.

Syntax

```
function Execute(ALoginAction: TADGUIxLoginAction): Boolean;
```

1.17.3.1.4.1.4 TADGUIxLoginDialog.GetAllLoginParams Method

Fills the VisibleItems (see page 679).

Description

Use the GetAllLoginParams method to retrieve all driver parameters, eligible for modification by the end user, and put them into VisibleItems property. The connection definition must have DriverID parameter specified.

Syntax

```
procedure GetAllLoginParams;
```

See Also

ConnectionDef (see page 676)

1.17.3.1.4.2 published

1.17.3.1.4.2.1 TADGUIxLoginDialog.Caption Property

The dialog caption.

Description

Use the Caption property to specify a custom dialog caption.

Syntax

```
property Caption: String;
```

1.17.3.1.4.2.2 TADGUIxLoginDialog.ChangeExpiredPassword Property

Controls the user ability to change an expired password.

Description

Use the ChangeExpiredPassword to control the user ability to change an expired password at login to a database. When ChangeExpiredPassword is True and driver returns an exception with Kind = ekUserPwdExpired, then dialog will allow to user to change an expired password. When ChangeExpiredPassword is False, then corresponding exception will be raised. The default value is True.

Syntax

```
property ChangeExpiredPassword: Boolean;
```

1.17.3.1.4.2.3 TADGUIxLoginDialog.HistoryEnabled Property

Controls the login history storage.

Description

Use the HistoryEnabled to control the storage of login history. When HistoryEnabled is True, then login history will be stored as specified by the HistoryStorage and HistoryKey properties. Default value is False.

Syntax

```
property HistoryEnabled: Boolean;
```

See Also

HistoryKey (see page 678), HistoryStorage (see page 678), HistoryWithPassword (see page 678)

1.17.3.1.4.2.4 TADGUIxLoginDialog.HistoryKey Property

Specifies where to store login history.

Description

Depending on the HistoryStorage property the login history will be stored:

- hsRegistry - in specified registry key.
- hsFile - in specified INI-file.

Syntax

```
property HistoryKey: String;
```

See Also

HistoryStorage (see page 678), HistoryEnabled (see page 677), HistoryWithPassword (see page 678)

1.17.3.1.4.2.5 TADGUIxLoginDialog.HistoryStorage Property

Specifies what storage kind to use for the login history.

Description

Use HistoryStorage property to specify the login history storage kind:

- hsRegistry - in registry. HistoryKey property specifies the registry key, where to store history.
- hsFile - in INI-file. HistoryKey property specifies the INI-file name, where to store history.

Syntax

```
property HistoryStorage: TADGUIxLoginHistoryStorage;
```

See Also

HistoryKey (see page 678), HistoryEnabled (see page 677), HistoryWithPassword (see page 678)

1.17.3.1.4.2.6 TADGUIxLoginDialog.HistoryWithPassword Property

Controls should the login history contain the passwords or not.

Description

Use the HistoryWithPassword to control the passwords storage in login history. When the HistoryWithPassword is True, then the passwords will be stored in login history. Default value is True.

Syntax

```
property HistoryWithPassword: Boolean;
```

See Also

HistoryStorage (see page 678), HistoryKey (see page 678), HistoryEnabled (see page 677)

1.17.3.1.4.2.7 TADGUIxLoginDialog.LoginRetries Property

Specifies the number of unsuccessful login retries.

Description

Use the LoginRetries property to specify the number of unsuccessful login retries. When user will fail to enter correct credentials specified number of times, an exception will be raised and login process will be stoped. To disable retries set LoginRetries to -1. The default value is 3.

Syntax

```
property LoginRetries: Integer;
```

See Also

Establishing Connection (see page 37)

1.17.3.1.4.2.8 TADGUIxLoginDialog.OnChangePassword Property

Fires when user password is expired.

Description

Use the OnChangePassword even handler to override the dialog asking user for a new password. In this event handler application may provide the new password by setting NewPassword parameter of the connection definition.

Syntax

```
property OnChangePassword: TADGUIxLoginDialogEvent;
```

See Also

ConnectionDef (see page 676)

1.17.3.1.4.2.9 TADGUIxLoginDialog.OnHide Event

Fires after the dialog will be hidden.

Description

Use the OnHide event to perform the actions after the dialog was closed.

Syntax

```
property OnHide: TNotifyEvent;
```

See Also

OnShow (see page 679)

1.17.3.1.4.2.10 TADGUIxLoginDialog.OnLogin Property

Fires when user should provide database credentials.

Description

Use the OnLogin even handler to override the dialog asking user for the login credentials. In this event handler application may provide the required connection definition parameters.

Syntax

```
property OnLogin: TADGUIxLoginDialogEvent;
```

See Also

ConnectionDef (see page 676)

1.17.3.1.4.2.11 TADGUIxLoginDialog.OnShow Event

Fires before the dialog will be displayed.

Description

Use the OnShow event to perform the actions before the dialog will be displayed.

Syntax

```
property OnShow: TNotifyEvent;
```

See Also

OnHide (see page 679)

1.17.3.1.4.2.12 TADGUIxLoginDialog.VisibleItems Property

Specifies the visible login dialog fields.

Description

Use the VisibleItems to specify the list of visible login dialog fields. The list consist of the connection definition parameters,

where each parameter is on separated line. Optionally application may specify an alternative label for a field after '='.

Syntax

```
property VisibleItems: TStrings;
```

Example

```
with ADGUIxLoginDialog1.VisibleItems do begin
  Clear;
  Add('Database');
  Add('User_name=Your name');
  Add('Password=Your key');
end;
```

1.17.3.1.5 TADGUIxScriptDialog Class

The dialog showing a SQL script execution progress.

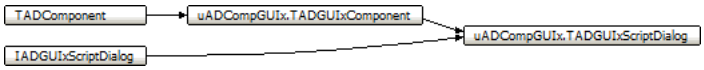
public

public	Description
ScriptDialog (see page 681)	Reference to dialog interface.

published

published	Description
Caption (see page 681)	The dialog caption.
OnHide (see page 681)	Fires after the dialog will be hidden.
OnInput (see page 681)	Fires when a script executes ACCEPT command.
OnOutput (see page 681)	Fires when a script provides some feedback to a user.
OnPause (see page 682)	Fires when a script executes PAUSE command.
OnProgress (see page 682)	Fires when a script updates the job progress.
OnShow (see page 682)	Fires before the dialog will be displayed.
Options (see page 682)	Specifies the dialog options.

Class Hierarchy



File

uADCompGUIx

Description

Use the TADGUIxScriptDialog in FireDAC GUI application to display SQL script execution progress. The dialog allows:

- to see, that a script is executing;
- to see, what part of a script is executing;
- to see, how much job remains to perform;
- to cancel script execution.

The TADScript (see page 685) will show the dialog automatically, when TADScript.ScriptDialog (see page 702) property is set to the dialog instance.

To use the dialog, drop it on a form or a data module. All additional setups are optional.

IADGUIxScriptDialog has Forms, FMX and Console implementations. Use Provider (see page 672) property to select the desired implementation.

Syntax

```
TADGUIxScriptDialog = class(TADGUIxComponent, IADGUIxScriptDialog);
```

See Also

Executing SQL Scripts (see page 88), TADScript Class (see page 685)

1.17.3.1.5.1 public

1.17.3.1.5.1.1 TADGUIxScriptDialog.ScriptDialog Property

Reference to dialog interface.

Description

Use the ScriptDialog to get a dialog interface reference.

Syntax

```
property ScriptDialog: IADGUIxScriptDialog;
```

1.17.3.1.5.2 published

1.17.3.1.5.2.1 TADGUIxScriptDialog.Caption Property

The dialog caption.

Description

Use the Caption property to specify the dialog caption.

Syntax

```
property Caption: String;
```

1.17.3.1.5.2.2 TADGUIxScriptDialog.OnHide Event

Fires after the dialog will be hidden.

Description

Use the OnHide event to perform the actions after the dialog was hidden at end of a script execution.

Syntax

```
property OnHide: TNotifyEvent;
```

See Also

OnShow (see page 682)

1.17.3.1.5.2.3 TADGUIxScriptDialog.OnInput Property

Fires when a script executes ACCEPT command.

Description

Use the OnInput event handler to perform required actions to show APrompt to the user and return AResult value to a script.

Syntax

```
property OnInput: TADGUIxScriptInputEvent;
```

See Also

OnPause (see page 682)

1.17.3.1.5.2.4 TADGUIxScriptDialog.OnOutput Property

Fires when a script provides some feedback to a user.

Description

Use the OnOutput event handler to perform required actions to output a AStr string to some console. A provided string is of

AKind kind.

Syntax

```
property OnOutput: TADGUIxScriptOutputEvent;
```

1.17.3.1.5.2.5 TADGUIxScriptDialog.OnPause Property

Fires when a script executes PAUSE command.

Description

Use the OnPause event handler to perform required actions to show APrompt to the user, pause a script execution and continue on user request.

Syntax

```
property OnPause: TADGUIxScriptPauseEvent;
```

See Also

OnInput (see page 681)

1.17.3.1.5.2.6 TADGUIxScriptDialog.OnProgress Property

Fires when a script updates the job progress.

Description

Use the OnProgress event handler to show the job progress to a user. The event handler receives AInfoProvider parameter, which is a reference to IADGUIxScriptDialogInfoProvider interface. It provides detailed information about job progress.

Syntax

```
property OnProgress: TADGUIxScriptProgressEvent;
```

1.17.3.1.5.2.7 TADGUIxScriptDialog.OnShow Event

Fires before the dialog will be displayed.

Description

Use the OnShow event to perform the actions before the dialog will be displayed.

Syntax

```
property OnShow: TNotifyEvent;
```

See Also

OnHide (see page 681)

1.17.3.1.5.2.8 TADGUIxScriptDialog.Options Property

Specifies the dialog options.

Description

Use the Options property to specify dialog options.

Option	Description
ssCallstack	The dialog will show the current script call stack.
ssConsole	The dialog will show script execution feedback.
ssAutoHide	The dialog will be automatically hidden after a script execution is finished.

The default value is [ssCallstack, ssConsole, ssAutoHide].

Syntax

```
property Options: TADGUIxScriptOptions;
```

1.17.3.1.6 TADGUIxWaitCursor Class

The component allowing to control the wait cursor.

public

public	Description
WaitCursor (see page 683)	Returns reference to wait cursor interface.

published

published	Description
OnHide (see page 683)	Fires before the wait cursor will be hidden.
OnShow (see page 684)	Fires after the wait cursor will be shown.
ScreenCursor (see page 684)	Specifies the cursor to use when FireDAC is busy.

Class Hierarchy



File

uADCompGUIx

Description

Use the TADGUIxWaitCursor to add the IADGUIxWaitCursor interface implementation to your application. This implementation is mandatory for FireDAC applications. And the TADGUIxWaitCursor allows to control the wait cursor.

IADGUIxWaitCursor has Forms, FMX and Console implementations. Use Provider (see page 672) property to select the desired implementation. Note, setting TADGUIxWaitCursor.Provider property value changes also the default application GUIx provider. For example, setting it to 'Console' will set console provider for all GUIx components.

Syntax

```
TADGUIxWaitCursor = class(TADGUIxComponent, IADGUIxWaitCursor);
```

1.17.3.1.6.1 public

1.17.3.1.6.1.1 TADGUIxWaitCursor.WaitCursor Property

Returns reference to wait cursor interface.

Description

The WaitCursor returns a reference to the IADGUIxWaitCursor interface.

Syntax

```
property WaitCursor: IADGUIxWaitCursor;
```

1.17.3.1.6.2 published

1.17.3.1.6.2.1 TADGUIxWaitCursor.OnHide Event

Fires before the wait cursor will be hidden.

Description

Use the OnHide event handler to take some actions before the wait cursor will be hidden.

Syntax

```
property OnHide: TNotifyEvent;
```

See Also

OnShow (see page 684)

1.17.3.1.6.2.2 TADGUIxWaitCursor.OnShow Event

Fires after the wait cursor will be shown.

Description

Use the OnShow event handler to take some actions after the wait cursor will be shown.

Syntax

```
property OnShow: TNotifyEvent;
```

See Also

OnHide (see page 683)

1.17.3.1.6.2.3 TADGUIxWaitCursor.ScreenCursor Property

Specifies the cursor to use when FireDAC is busy.

Description

Use the ScreenCursor property to specify the mouse cursor to use to show that FireDAC is busy. Default value is gcrSQLWait.

Syntax

```
property ScreenCursor: TADGUIxScreenCursor;
```

1.17.4 uADCompScript Namespace

Contains TADScript (see page 685) scripting engine class and additional utility methods and classes.

Classes

Class	Description
TADScript (see page 685)	The class implementing SQL script engine, capable to execute a series of SQL queries.
TADScriptCommand (see page 704)	The base class for custom script commands.
TADScriptCommandRegistry (see page 708)	The list of the registered script commands.
TADScriptOptions (see page 709)	The class implementing SQL script engine options.
TADSQLScript (see page 720)	An in-memory stored SQL (see page 721) script.
TADSQLScripts (see page 721)	TADSQLScripts is a collection of the TADSQLScript (see page 720) objects.

Description

The uADCompScript unit contains:

- TADScript (see page 685) - SQL script execution engine component;
- TADScriptCommand (see page 704) - base class for SQL script engine command implementations;
- TADScriptCommandRegistry (see page 708) - class implementing registry for command implementations;
- TADScriptOptions (see page 709) - class containing the set of SQL script engine options;
- TADSQLScript (see page 720), TADSQLScripts (see page 721) - collection of SQL scripts.

See Also

Executing SQL Scripts (see page 88), uADCompClient (see page 257), uADCompScriptCommands

1.17.4.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADScript (see page 685)	The class implementing SQL script engine, capable to execute a series of SQL queries.
TADScriptCommand (see page 704)	The base class for custom script commands.
TADScriptCommandRegistry (see page 708)	The list of the registered script commands.
TADScriptOptions (see page 709)	The class implementing SQL script engine options.
TADSQLScript (see page 720)	An in-memory stored SQL (see page 721) script.
TADSQLScripts (see page 721)	TADSQLScripts is a collection of the TADSQLScript (see page 720) objects.

1.17.4.1.1 TADScript Class

The class implementing SQL script engine, capable to execute a series of SQL queries.

public

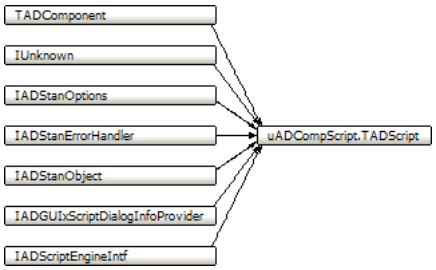
public	Description
AllSpools (see page 687)	A list of the all produced spool files.
CallStack (see page 687)	Returns the current script call stack.
CurrentCommand (see page 687)	A reference to the current SQL script command.
Eof (see page 687)	Returns True, if current position is beyond the last command in SQL script.
Finished (see page 688)	Controls the script execution.
LastSpoolFileName (see page 688)	Returns the name of the current active spool file.
Position (see page 688)	Controls the current SQL script position.
ProcessedAfterCommit (see page 689)	Returns the number of the SQL commands processed after last commiting of a transaction.
Status (see page 689)	Returns the script engine current status.
TotalErrors (see page 689)	The number of exceptions raised.
TotalJobDone (see page 690)	The already processed part of the script.
TotalJobSize (see page 690)	The total size of the script.
TotalPct10Done (see page 690)	The percents of the already processed part of the script.
AbortJob (see page 690)	Aborts the script validation of execution.
ExecuteAll (see page 691)	Executes a SQL script.
ExecuteFile (see page 692)	Executes the specified SQL script file.
ExecuteFile (see page 692)	Executes the specified SQL script file with the specified arguments.
ExecuteScript (see page 693)	Executes a SQL script contained in the string list.
ExecuteScript (see page 693)	Executes a SQL script contained in the string list with the specified arguments.
ExecuteStep (see page 694)	Extracts and executes a next command from the SQL script.
ValidateAll (see page 694)	Validates the SQL script.
ValidateStep (see page 695)	Validates a next command in the SQL script.

published

published	Description
AfterExecute (see page 695)	The event is fired after the execution is finished.

AfterScript (see page 695)	The event is fired after the script processing is finished.
Arguments (see page 696)	The script arguments.
BeforeExecute (see page 696)	The event is fired before the execution is started.
BeforeScript (see page 697)	The event is fired vefore the script processing is started.
Connection (see page 697)	Specifies a connection object.
FetchOptions (see page 697)	Fetch options for the SQL commands.
FormatOptions (see page 697)	Format options for the SQL commands.
Macros (see page 698)	The script macros.
OnConsoleGet (see page 698)	The event allows to an end-user to enter a value.
OnConsoleLockUpdate (see page 698)	The event allows to temporary lock an end-user interface.
OnConsolePut (see page 698)	The event allows to produce an end-user output.
OnError (see page 699)	The event fires, when an error happens, while script engine is communicating with DBMS.
OnGetText (see page 699)	The event fires, when a script engine needs a reference to a stream.
OnHostCommand (see page 700)	The event fires, when a script engine needs to execute external host OS command.
OnPause (see page 700)	The event fires, when a script engine needs to make a pause in script execution.
OnProgress (see page 700)	The event is firing, while a script engine is processing a script.
OnReleaseText (see page 701)	The event fires, when a script engine needs to release a reference to a stream.
OnSpoolPut (see page 701)	The event allows to produce a script execution trace output.
Params (see page 701)	The parameters for the script SQL commands.
ResourceOptions (see page 701)	Resource options for the SQL commands.
ScriptDialog (see page 702)	A reference to a script feedback dialog.
ScriptOptions (see page 702)	The script control options.
SQLScriptFileName (see page 703)	The SQL script file name.
SQLScripts (see page 703)	The collection of the SQL scripts.
Transaction (see page 704)	Specifies a transaction object.

Class Hierarchy



File

uADCompScript

Description

Use TADScript to execute a series of the SQL queries, script control commands, produce a execution log and so on.

Syntax

```
TADScript = class(TADComponent, IUnknown, IADStanOptions, IADStanErrorHandler,
IADStanObject, IADGUIxScriptDialogInfoProvider, IADScriptEngineIntf);
```

See Also

Executing SQL Scripts (see page 88), TADQuery (see page 476), TADConnection (see page 279)

Example

You can find TADScript related demos in the FireDAC\Samples\Comp Layer\TADScript folder:

- Main - the demo application, showing different aspects of TADScript
- GUI - the generic SQL script execution GUI application
- Console - the generic SQL script execution console application

1.17.4.1.1.1 public

1.17.4.1.1.1.1 TADScript.AllSpools Property

A list of the all produced spool files.

Description

Use the AllSpools property to get a list of all spool files produced by the last ExecuteAll (see page 691) call. The list will be updated right after a new spool file is opened.

Syntax

```
property AllSpools: TStrings;
```

See Also

ExecuteAll (see page 691), SpoolOutput (see page 718), SpoolFileName (see page 718), CloseSpool, UpdateSpool

1.17.4.1.1.1.2 TADScript.CallStack Property

Returns the current script call stack.

Description

Use the CallStack property to get the current script call stack. The stack will contain one line for each active script, starting from the root script. Each line consists of the script name and line number in this script.

Syntax

```
property CallStack: TStrings;
```

1.17.4.1.1.1.3 TADScript.CurrentCommand Property

A reference to the current SQL script command.

Description

Use the CurrentCommand property to a reference to the current SQL script control command.

Syntax

```
property CurrentCommand: TADScriptCommand;
```

See Also

CallStack (see page 687)

1.17.4.1.1.1.4 TADScript.Eof Property

Returns True, if current position is beyond the last command in SQL script.

Description

Use the Eof property to check the current position specified by the Position (see page 688) property. If the value is True, then position is beyond the last command in the current script. If the value is False, then there may be few other commands after the current position.

Syntax

```
property Eof: Boolean;
```

See Also

Position (see page 688), ValidateStep (see page 695), ExecuteStep (see page 694), Finished (see page 688), Status (see page 689)

Example

```
procedure TForm1.acRunUpdate(Sender: TObject);
begin
  acStep.Enabled := ADScript1.Connection.Connected and not ADScript1.Eof;
end;
```

1.17.4.1.1.5 TADScript.Finished Property

Controls the script execution.

Description

Use Finished property to get / set the script termination status.

If Finished is True, then script execution is going to be terminated or is already terminated. If to set Finished to True, then the script execution will be terminated after the current command execution will be finished.

To abort the execution of the current command and the whole script use the AbortJob (see page 690) method. We are recommending always use AbortJob (see page 690) method.

Syntax

```
property Finished: Boolean;
```

See Also

AbortJob (see page 690), Status (see page 689), ValidateStep (see page 695), ExecuteStep (see page 694)

1.17.4.1.1.6 TADScript.LastSpoolFileName Property

Returns the name of the current active spool file.

Description

Read the LastSpoolFileName property to get the name of the currently active spool file. If there is no active spooling, then the value will be an empty string.

Syntax

```
property LastSpoolFileName: String;
```

See Also

SpoolOutput (see page 718), SpoolFileName (see page 718)

1.17.4.1.1.7 TADScript.Position Property

Controls the current SQL script position.

Description

Use Position property to get / set the script current position. A first line corresponds to Position.Y = 0 and a first character in line corresponds to Position.X = 0.

The property value will be used only for ValidateStep (see page 695) and ExecuteStep (see page 694) methods, when a script is contained in the SQLScripts (see page 703) collection. The next command extraction and execution will start from the specified position. After the command is extracted and executed, the Position will be updated to the new value.

If there is no more commands after specified Position, then the Eof (see page 687) property will return True, otherwise False.

Syntax

```
property Position: TPoint;
```

See Also

Eof (see page 687), ValidateStep (see page 695), ExecuteStep (see page 694)

1.17.4.1.1.8 TADScript.ProcessedAfterCommit Property

Returns the number of the SQL commands processed after last commiting of a transaction.

Description

Read the ProcessedAfterCommit property to get the number of the SQL commands, processed after last transaction commiting.

When the CommitEachNCommands (see page 712) option is 0 (never commit) or is greater than 1 (commit each N SQL commands), then the ProcessedAfterCommit may be greater than 0, if there were executed SQL commands after last COMMIT.

When the CommitEachNCommands (see page 712) is 0, then ProcessedAfterCommit will be always 0.

Syntax

```
property ProcessedAfterCommit: Integer;
```

See Also

CommitEachNCommands (see page 712)

1.17.4.1.1.9 TADScript.Status Property

Returns the script engine current status.

Description

Read the Status property to get the SQL script engine current status.

Status	Description
ssInactive	Engine is inactive and was not yet running.
ssValidating	Engine is validating the script after calling the ValidateAll (see page 694) or ValidateStep (see page 695) methods.
ssRunning	Engine is executing the script after calling the ExecuteAll (see page 691) or ExecuteStep (see page 694) methods.
ssFinishWithErrors	Engine has finished the run with errors.
ssFinishSuccess	Engine has finished the run without errors.
ssAborted	Engine run has aborted.

Also an application can check the Finished (see page 688) property to see, when the engine has finished.

Syntax

```
property Status: TADScriptStatus;
```

See Also

Finished (see page 688), Eof (see page 687), TotalErrors (see page 689)

1.17.4.1.1.10 TADScript.TotalErrors Property

The number of exceptions raised.

Description

Read the TotalErrors property to get the number of exceptions raised in the last ValidateStep (see page 695), ValidateAll (see page 694), ExecuteStep (see page 694), ExecuteAll (see page 691) method call.

If the IgnoreError (see page 716) option is True, then exceptions will be hidden. Otherwise, the exceptions will be counted

and showed in the user feedback. Also, if the BreakOnError (see page 711) options is True, then the script execution will be stopped after a first exception.

Syntax

```
property TotalErrors: Integer;
```

See Also

IgnoreError (see page 716), BreakOnError (see page 711)

1.17.4.1.1.11 TADScript.TotalJobDone Property

The already processed part of the script.

Description

Read the TotalJobDone property to get the already processed part of the script and produce an end-user feedback. The value is in bytes.

Syntax

```
property TotalJobDone: Integer;
```

See Also

TotalJobSize (see page 690), TotalPct10Done (see page 690)

1.17.4.1.1.12 TADScript.TotalJobSize Property

The total size of the script.

Description

Read the TotalJobSize property to get the total size of the script and produce an end-user feedback. The value is in bytes.

An application may call ValidateAll (see page 694) method to get an estimation of the TotalJobSize.

Syntax

```
property TotalJobSize: Integer;
```

See Also

TotalJobDone (see page 690), TotalPct10Done (see page 690), ValidateAll (see page 694)

1.17.4.1.1.13 TADScript.TotalPct10Done Property

The percents of the already processed part of the script.

Description

Read the TotalPct10Done property to get the percents of the already processed part of the script and produce an end-user feedback. The value is in 0.1 of percents. For example, 100 = 10%, 567 = 56.7%.

Syntax

```
property TotalPct10Done: Integer;
```

See Also

TotalJobDone (see page 690), TotalJobSize (see page 690)

1.17.4.1.1.14 TADScript.AbortJob Method

Aborts the script validation of execution.

Parameters

Parameters	Description
const AWait: Boolean = False	True, if to wait for a DBMS query cancelation.

Description

Call the AbortJob method to abort the script:

- validation, run by the ValidateStep (see page 695) or ValidateAll (see page 694) method calls;
- execution, run by the ExecuteStep (see page 694) or ExecuteAll (see page 691) method calls.

The method also will abort any DBMS query launched by the script execution. If AWait is True, then the AbortJob will wait until a DBMS query will finish its processing. After the method call the Status (see page 689) property will be equal to ssAborted and Finished (see page 688) will be True. The method also aborts all nested script calls.

Syntax

```
procedure AbortJob(const AWait: Boolean = False);
```

See Also

Finished (see page 688), Status (see page 689)

Example

```
// background thread
ADScript1.ExecuteAll;
...
// main thread
ADScript1.AbortJob(True);
```

1.17.4.1.1.15 TADScript.ExecuteAll Method

Executes a SQL script.

Description

Call the ExecuteAll method to execute the SQL script.

The TADScript allows to execute a script from a file pointed by SQLScriptFileName (see page 703), if it specified. Otherwise from a SQL (see page 721) property of the item with a zero index from the SQLScripts (see page 703) collection.

The script will be executed in full, including all nested script calls. The script execution will be stoped, after one of the following will happen:

- all script commands are processed;
- the AbortJob (see page 690) method is called;
- the Finished property is set to True;
- EXIT | QUIT | STOP command is found and processed in the script;
- exception is raised by one of the script commands and IgnoreError (see page 716) option is False and BreakOnError (see page 711) option is True.

The exceptions, raised at the script command executions are not propagated out of ExecuteAll method. To check the script completion status, use:

- Method result value. If it is True, then there was no errors. If it is False, then there was at least one error.
- TotalErrors (see page 689) property. It returns the total number of errors at script execution.
- Status (see page 689) property. It has ssFinishWithErrors value, if there was any error, which stoped the script execution.

Before execution the BeforeExecute (see page 696) event handler will be called. And after the execution is finished the AfterExecute (see page 695) event handler will be called. A script execution may be nested. The BeforeScript (see page 697) event handler will be called before each script start up. And after a script is finished the AfterScript (see page 695) event handler will be called.

Alternatively a script may be executed by the ExecuteFile (see page 692) or ExecuteScript (see page 693) methods.

Syntax

```
function ExecuteAll: Boolean; overload;
```

See Also

ExecuteFile (see page 692), ExecuteScrip (see page 693)t, AbortJob (see page 690), BeforeExecute (see page 696), AfterExecute (see page 695), BeforeScript (see page 697), AfterScript (see page 695)

Example 1

```
ADScript1.SQLScriptFileName := 'c:\temp\createdb.sql';
ADScript1.ExecuteAll;
```

Example 2

The following code shows how to wrap the script execution into transaction:

```
ADConnection1.StartTransaction;
try
  ADScript1.ExecuteAll;
finally
  if ADScript1.TotalErrors > 0 then
    ADConnection1.Rollback
  else
    ADConnection1.Commit;
end;
```

1.17.4.1.1.16 TADScript.ExecuteFile Method (String)

Executes the specified SQL script file.

Parameters

Parameters	Description
const AFileName: String	A SQL script file to execute.

Description

Use the ExecuteFile method to execute the specifed SQL script file. The method is a shortcut for the following pseudocode:

```
  SQLScripts.Clear;
  SQLScriptFileName := AFileName;
  ValidateAll;
  ExecuteAll;
```

Please, see the above methods for details, at first ExecuteAll (see page 691).

Syntax

```
procedure ExecuteFile(const AFileName: String); overload;
```

See Also

SQLScripts (see page 703), SQLScriptFileName (see page 703), ValidateAll (see page 694), ExecuteAll (see page 691)

Example

```
 ADScript1.ExecuteFile('c:\temp\createdb.sql');
```

1.17.4.1.1.17 TADScript.ExecuteFile Method (String, array of String)

Executes the specified SQL script file with the specified arguments.

Parameters

Parameters	Description
const AFileName: String	A SQL script file to execute.
const AArguments: array of String	A script arguments.

Description

Use the ExecuteFile method to execute the specifed SQL script file with the specified arguments. The method is a shortcut for the following pseudocode:

```
Arguments := AArguments;
```

```
SQLScripts.Clear;
SQLScriptFileName := AFileName;
ValidateAll;
ExecuteAll;
```

Please, see the above methods for details, at first ExecuteAll (see page 691).

Syntax

```
procedure ExecuteFile(const AFileName: String; const AArguments: array of String); overload;
```

See Also

SQLScripts (see page 703), SQLScriptFileName (see page 703), ValidateAll (see page 694), ExecuteAll (see page 691), Arguments (see page 696)

Example

```
ADScript1.ExecuteFile('c:\temp\createdb.sql', ['scott/tiger@mydb', 'c:\temp\']);
```

1.17.4.1.1.18 TADScript.ExecuteScript Method (TStrings)

Executes a SQL script contained in the string list.

Parameters

Parameters	Description
const AScript: TStrings	A string list with a SQL script to execute.

Description

Use the ExecuteScript method to execute a SQL script contained in the string list. The method is a shortcut for the following pseudocode:

```
SQLScripts.Clear;
SQLScripts.Add.SQL := AScript;
SQLScriptFileName := '';
ValidateAll;
ExecuteAll;
```

Please, see the above methods for details, at first ExecuteAll (see page 691).

Syntax

```
procedure ExecuteScript(const AScript: TStrings); overload;
```

See Also

SQLScripts (see page 703), SQLScriptFileName (see page 703), ValidateAll (see page 694), ExecuteAll (see page 691)

Example

```
ADScript1.ExecuteScript(Memo1.Lines);
```

1.17.4.1.1.19 TADScript.ExecuteScript Method (TStrings, array of String)

Executes a SQL script contained in the string list with the specified arguments.

Parameters

Parameters	Description
const AScript: TStrings	A string list with a SQL script to execute.
const AArguments: array of String	A script arguments.

Description

Use the ExecuteScript method to execute a SQL script contained in the string list. The method is a shortcut for the following pseudocode:

```
Arguments := AArguments;
SQLScripts.Clear;
SQLScripts.Add.SQL := AScript;
SQLScriptFileName := '';
```

```
ValidateAll;
ExecuteAll;
```

Please, see the above methods for details, at first ExecuteAll (see page 691).

Syntax

```
procedure ExecuteScript(const AScript: TStrings; const AArguments: array of String);
overload;
```

See Also

SQLScripts (see page 703), SQLScriptFileName (see page 703), ValidateAll (see page 694), ExecuteAll (see page 691), Arguments (see page 696)

Example

```
ADScript1.ExecuteScript(Memo1.Lines, ['scott/tiger@mydb', 'c:\temp\']);
```

1.17.4.1.1.20 TADScript.ExecuteStep Method

Extracts and executes a next command from the SQL script.

Description

Call the ExecuteStep to extract and execute a next command from the SQL script.

The extraction will be started from the Position position in the SQL script, when the SQL script is stored in the SQLScript (see page 703) collection. When the SQL script is stored in a file, then extraction will be started from the current position in a file stream.

After execution of a command, the current position will be forwarded to the first symbol after the processed command. When a script is stored in SQLScript (see page 703), then the Position (see page 688) property will be updated. When a script is stored in a file, then file stream position will be updated.

If script command execution raised an exception, and there TADScriptOptions.IgnoreError (see page 716) is False, then this exception will propagated out of the method.

Before execution the BeforeExecute (see page 696) event handler will be called. And after the execution is finished the AfterExecute (see page 695) event handler will be called. When a subscript call is encountered, then the subscript will be executed by the ExecuteStep command.

Syntax

```
function ExecuteStep: Boolean; overload;
```

See Also

Position (see page 688), Eof (see page 687), ValidateStep (see page 695), ExecuteAll (see page 691)

Example

```
ADScript1.Position := Memo1.CaretPos;
ADScript1.ExecuteStep;
Memo1.CaretPos := ADScript1.Position;
```

1.17.4.1.1.21 TADScript.ValidateAll Method

Validates the SQL script.

Description

Call the ValidateAll method to validate the SQL script.

The ValidateAll method does not execute SQL commands and does not execute many of the script control commands. The method checks the syntax of the next control command and executes it, if that is required to continue with checking. Also, the ValidateAll method updates the total job volume indicator - TotalJobSize (see page 690). Which is required to show the correct work progress values.

It is a good practice to call the ValidateAll before the ExecuteAll (see page 691) method.

Syntax

```
function ValidateAll: Boolean; overload;
```

See Also

ExecuteAll (see page 691), TotalJobDone (see page 690)

1.17.4.1.1.22 TADScript.ValidateStep Method

Validates a next command in the SQL script.

Description

Call the ValidateStep command to extract and validate a next command in the SQL script.

The ValidateStep method does not execute a SQL command and does not execute many of the script control commands. The method checks the syntax of the next script control command and executes it, if that is required to continue with checking.

The ValidateStep method may be used to skip a next command in the GUI applicaitons allowing to execute add-hoc commands.

Syntax

```
function ValidateStep: Boolean; overload;
```

See Also

Position (see page 688), Eof (see page 687), ExecuteStep (see page 694), ValidateAll (see page 694)

Example

```
ADScript1.Position := Memo1.CaretPos;
ADScript1.ValidateStep;
Memo1.CaretPos := ADScript1.Position;
```

1.17.4.1.1.2 published

1.17.4.1.1.2.1 TADScript.AfterExecute Event

The event is fired after the execution is finished.

Description

Use the AfterExecute event to handle the event, when the execution is finished.

All the ExecuteXxx and ValidateXxx methods are calling the AfterExecute event handler at their end.

Syntax

```
property AfterExecute: TNotifyEvent;
```

See Also

BeforeExecute (see page 696), AfterScript (see page 695)

1.17.4.1.1.2.2 TADScript.AfterScript Event

The event is fired after the script processing is finished.

Description

Use the AfterScript event to handle the event, when the script processing is finished.

The event is not called by the ExecuteStep (see page 694), ValidateStep (see page 695) events, if they does not call a subscripts.

Syntax

```
property AfterScript: TNotifyEvent;
```

See Also

BeforeScript (see page 697), AfterExecute (see page 695)

1.17.4.1.1.2.3 TADScript.Arguments Property

The script arguments.

Description

Use the Arguments property to specify the script arguments.

An argument may be referenced as a substitution variable with the &<argument number> name. The arguments may be used in any place of the script. They are similar to the Macros (see page 698), but in contrast to the Macros (see page 698) are using positional syntax, while macros are using named syntax.

The @ / @@ commands allow to specify arguments as a comma-separated list after the script name. When arguments are not specified, then they will be inherited from the parent script.

Syntax

```
property Arguments: TStrings;
```

See Also

Macros (see page 698), Params (see page 701)

Example 1

```
with ADScript1.Arguments do begin
  Add('scott/tiger@orasrv');
  Add('c:\temp\');
end;
with ADScript1.SQLScripts.Add do begin
  SQL.Add('....');
  SQL.Add('connect &1');
  SQL.Add('....');
  SQL.Add('spool &2');
end;
```

Example 2

```
with ADScript1.SQLScripts.Add do begin
  SQL.Add('@@DropTab Orders');
  SQL.Add('@@DropTab Tickets');
  SQL.Add('@@DropTab Posts');
end;
with ADScript1.SQLScripts.Add do begin
  Name := 'DropTab';
  SQL.Add('drop table &1;');
  SQL.Add('drop table &1_details;');
end;
```

1.17.4.1.1.2.4 TADScript.BeforeExecute Event

The event is fired before the execution is started.

Description

Use the BeforeExecute event to handle the event, when the execution is started.

All the ExecuteXxx and ValidateXxx methods are calling the BeforeExecute event handler at their start.

Syntax

```
property BeforeExecute: TNotifyEvent;
```

See Also

AfterExecute (see page 695), BeforeScript (see page 697)

1.17.4.1.1.2.5 TADScript.BeforeScript Event

The event is fired vefore the script processing is started.

Description

Use the AfterScript event to handle the event, when the script processing is started.

The event is not called by the ExecuteStep (see page 694), ValidateStep (see page 695) events, if they does not call a subscripts.

Syntax

```
property BeforeScript: TNotifyEvent;
```

See Also

AfterScript (see page 695), BeforeExecute (see page 696)

1.17.4.1.1.2.6 TADScript.Connection Property

Specifies a connection object.

Description

The Connection property value points to a connection object.

The Connection property value must be specified before ExecuteXxx or ValidateXxx calls. When it will be required the connection object will be activated. To control transaction, the Transaction (see page 704) property may be used additionally.

The connection may be changed from the script using CONNECT <connection string command>.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

TADScriptOptions.DriverID (see page 713), Transaction (see page 704)

1.17.4.1.1.2.7 TADScript.FetchOptions Property

Fetch options for the SQL commands.

Description

Use the FetchOptions to specify the fetch options, which will be used for the SQL commands, returning the cursors.

Syntax

```
property FetchOptions: TADFetchOptions;
```

See Also

FormatOptions (see page 697), ResourceOptions (see page 701)

1.17.4.1.1.2.8 TADScript.FormatOptions Property

Format options for the SQL commands.

Description

Use the FormatOptions to specify the data format options, which will be used for the SQL command parameters and cursors.

Syntax

```
property FormatOptions: TADFormatOptions;
```

See Also

FetchOptions (see page 697), ResourceOptions (see page 701)

1.17.4.1.1.2.9 TADScript.Macros Property

The script macros.

Description

Use the Macros property to specify the script macros.

The macros may be used in any place of the script.

Syntax

```
property Macros: TADMacros;
```

See Also

Arguments, Params (see page 701)

1.17.4.1.1.2.10 TADScript.OnConsoleGet Event

The event allows to an end-user to enter a value.

Description

Use the OnConsoleGet event to get an end-user input. At moment this is required for the ACCEPT control command.

APrompt - a text used as a prompt to an end-user.

AResult - a text provided by am end-user.

If the ScriptDialog (see page 702) is specified and OnConsoleGet is not specified, then the script dialog will ask an end-user for the input.

Syntax

```
property OnConsoleGet: TADConsoleGetEvent;
```

See Also

ScriptDialog (see page 702), OnConsolePut (see page 698), OnPause (see page 700)

Example

```
procedure TForm1.ADScript1ConsoleGet(AEngine: TADScript; const APrompt: String; var
AResult: String);
begin
  InputQuery('Enter value', APrompt, AResult);
end;
```

1.17.4.1.1.2.11 TADScript.OnConsoleLockUpdate Event

The event allows to temporary lock an end-user interface.

Description

Use the OnConsoleLockUpdate event to temporary lock and unlock an end-user interface. That is required to optimize the output of large data volumes. At moment OnConsoleLockUpdate event is fired with a ALock = True before a cursor data will be output. After that it is fired with ALock = False.

Syntax

```
property OnConsoleLockUpdate: TADConsoleLockUpdate;
```

See Also

OnConsolePut (see page 698)

1.17.4.1.1.2.12 TADScript.OnConsolePut Event

The event allows to produce an end-user output.

Description

Use the OnConsolePut event to produce an end-user output.

AMessage - a message output to an end-user.

AKind - the output message kind.

If the ScriptDialog (see page 702) is specified, then it will show an end-user output, additionally to the OnConsolePut event handler.

Syntax

```
property OnConsolePut: TADConsolePutEvent;
```

See Also

ScriptDialog (see page 702), OnConsoleGet (see page 698), OnConsoleLockUpdate (see page 698)

Example

```
procedure TForm1.ADScrip1ConsolePut(AEngine: TADScript; const AMessage: String; AKind:
TADScriptOuputKind);
begin
  mmLog.Lines.AddObject(AMessage, nil);
  Application.ProcessMessages;
end;
```

1.17.4.1.1.2.13 TADScript.OnError Event

The event fires, when an error happens, while script engine is communicating with DBMS.

Description

Use the OnError event to handle an exception raised, while script engine is communicating with a DBMS.

This event should not be used to produce an end-user output regarding the exceptions.

Syntax

```
property OnError: TADErrorEvent;
```

1.17.4.1.1.2.14 TADScript.OnGetText Event

The event fires, when a script engine needs a reference to a stream.

Description

Use the OnGetText event handler to provide a reference to a text stream to the script engine. The text stream is represented by the TADTextFile class, which may work with a string or a TStream reference. If event handler returns nil as the AText parameter value, then default action will be took.

When the text stream will be no more required, the OnReleaseText (see page 701) event will be called.

AFileName - the name of a file to open.

AMode - the file open mode.

AText - returns a reference to the TADTextFile object.

Syntax

```
property OnGetText: TADGetTextEvent;
```

See Also

OnReleaseText (see page 701)

Example

```
procedure TForm1.ADScript1GetText(AEngine: TADScript; const AFileName: string; AMode:
TADScriptTextMode; out AText: TADTextFile);
begin
   AText := TADTextFile.Create(...);
end;
```

1.17.4.1.1.2.15 TADScript.OnHostCommand Event

The event fires, when a script engine needs to execute external host OS command.

Description

Use the OnHostCommand event handler, to execute external host OS commands, launched by the HOST | !! | SHELL script commands. The event handler may do whatever an application is needing - as launch an OS command, as, for example, show some dialog.

ACommand - a command to execute.

ADoDefault - if True, then default action will be performed. The default action is ShellExecute('open', ACommand) call.

Syntax

```
property OnHostCommand: TADHostCommandEvent;
```

1.17.4.1.1.2.16 TADScript.OnPause Event

The event fires, when a script engine needs to make a pause in script execution.

Description

Use the OnPause event handler, to make a pause in the script execution, as result of PAUSE <prompt> script command call.

For a GUI application OnPause event handler may show a message box using the ShowMessage procedure call. For a console application OnPause event handler may use WRITELN / READLN RTL procedures.

If the ScriptDialog (see page 702) is specified and OnPause is not specified, then the script dialog will ask an end-user to wait.

Syntax

```
property OnPause: TADPauseEvent;
```

See Also

ScriptDialog (see page 702), OnConsoleGet (see page 698)

Example

```
procedure TForm1.ADScript1Pause(AEngine: TADScript; const AText: String);
begin
  ShowMessage(AText);
end;
```

1.17.4.1.1.2.17 TADScript.OnProgress Event

The event is firing, while a script engine is processing a script.

Description

Use the OnProgress event handler, to show a script processing progress to an end-user.

In the event handler an application may read the TotalJobDone (see page 690), TotalJobSize (see page 690), TotalPct10Done (see page 690) properties to get information about how large is the full job and what it part is already done.

If the ScriptDialog (see page 702) is specified, then it will show a script processing progress, additionally to the OnProgress event handler.

Syntax

```
property OnProgress: TNotifyEvent;
```

See Also

TotalJobDone (see page 690), TotalJobSize (see page 690), TotalPct10Done (see page 690), ScriptDialog (see page 702)

Example

```
procedure TForm1.ADScript1Progress(ASender: TObject);
begin
  ProgressBar1.Position := ADScript1.TotalPct10Done;
end;
```

1.17.4.1.1.2.18 TADScript.OnReleaseText Event

The event fires, when a script engine needs to release a reference to a stream.

Description

Use the OnReleaseText event handler to release a reference to a text stream, after it was acquired using OnGetText (see page 699) event handler and is no more used by the script engine.

Syntax

```
property OnReleaseText: TADGetTextEvent;
```

See Also

OnGetText (see page 699)

1.17.4.1.1.2.19 TADScript.OnSpoolPut Event

The event allows to produce a script execution trace output.

Description

Use the OnSpoolPut event handler to produce a script execution trace output or spool output.

The spool output is enabled:

- by the SPOOL | OUTPUT script control commands and disabled by them with OFF parameter;
- by the SpoolOutput (see page 718) and SpoolFileName (see page 718) options.

Syntax

```
property OnSpoolPut: TADConsolePutEvent;
```

See Also

SpoolOutput (see page 718), SpoolFileName (see page 718)

Example

```
procedure TForm1.ADScript1SpoolPut(AEngine: TADScript; const AMessage: String; AKind:
TADScriptOuputKind);
begin
  Memo1.Lines.Add(AMessage);
end;
```

1.17.4.1.1.2.20 TADScript.Params Property

The parameters for the script SQL commands.

Description

Use the Params property to specify the script SQL command parameters.

The parameters may be used only in the SQL commands.

Syntax

```
property Params: TADParams;
```

See Also

Arguments, Macros (see page 698)

1.17.4.1.1.2.21 TADScript.ResourceOptions Property

Resource options for the SQL commands.

Description

Use the ResourceOptions to specify the resource usage options, which will be used for the SQL commands.

Syntax

```
property ResourceOptions: TADResourceOptions;
```

See Also

FetchOptions (see page 697), FormatOptions (see page 697)

1.17.4.1.1.2.22 TADScript.ScriptDialog Property

A reference to a script feedback dialog.

Description

Use the ScriptDialog property to set a reference to the TADGUIxScriptDialog (see page 680) instance. The dialog allows to the script engine to communicate with the end-user. The interface implementation provides a standard implementation for the following events:

- OnConsoleGet (see page 698)
- OnConsolePut (see page 698)
- OnProgress (see page 700)
- OnPause (see page 700)

GUI applications should use TADGUIxScriptDialog (see page 680) and set it Provider (see page 672) property to 'Forms' or 'FMX'. Console applications should set it to 'Console'.

Note, if a dialog is created not at design-time, but at run-time, then the application should include uADGUIxFormsfScript unit for VCL applications, and uADGUIxFMXfScript for FireMonkey applications. The console applications should include uADGUIxConsoleScript unit.

The dialog will be shown when global silent flag is not set to True (ADGUIxSilent() = False).

Syntax

```
property ScriptDialog: TADGUIxScriptDialog;
```

See Also

TADGUIxScriptDialog (see page 680), OnProgress (see page 700), OnConsoleGet (see page 698), OnConsolePut (see page 698), OnPause (see page 700)

Example 1

GUI application:

```
uses
  uADGUIxFormsfScript;
...
ADGUIxScriptDialog1.Provider := 'Forms';
ADScript1.ScriptDialog := ADGUIxScriptDialog1;
```

Example 2

Console application:

```
uses
  uADGUIxConsoleScript;
...
ADGUIxScriptDialog1.Provider := 'Console';
ADScript1.ScriptDialog := ADGUIxScriptDialog1;
```

1.17.4.1.1.2.23 TADScript.ScriptOptions Property

The script control options.

Description

Use the ScriptOptions to set the script control options.

These options may be changed by the script control commands too. The property descriptions are showing the corresponding commands.

Syntax

```
property ScriptOptions: TADScriptOptions;
```

See Also

TADScriptOptions (see page 709)

1.17.4.1.1.2.24 TADScript.SQLScriptFileName Property

The SQL script file name.

Description

Use the SQLScriptFileName property to set a SQL script file name to run.

The name may use macros from the Macros (see page 698) collection or arguments from the Arguments (see page 696) list. If the OS file name does not have an extension, then the ".SQL" will be appended. If there is no path, then DefaultScriptPath (see page 713) option will be used.

A SQL script loading mechanism may be overridden using the OnGetText (see page 699) event handler. Including, for example, loading a SQL script not from a file, but from a EXE resource. Alternatively a SQL script may be submitted from a memory using the SQLScript (see page 703) collection. The SQLScriptFileName has high priority than SQLScript (see page 703) collection.

Note, that ExecuteFile (see page 692) and ExecuteScript (see page 693) methods clear the existing SQLScriptFileName value.

A path may include path variables (see page 163).

Syntax

```
property SQLScriptFileName: String;
```

See Also

ExecuteAll (see page 691), DefaultScriptPath (see page 713), OnGetText (see page 699), SQLScripts (see page 703), Macros (see page 698), Arguments (see page 696)

Example

```
ADScript1.Arguments.Clear;
ADScript1.Arguments.Add('financial');
ADScript1.ScriptOptions.DefaultScriptPath := 'c:\temp';
ADScript1.SQLScriptFileName := 'create_&1_db';
// a file name will be resolved into c:\temp\create_financial_db.sql
ADScript1.ExecuteAll;
```

1.17.4.1.1.2.25 TADScript.SQLScripts Property

The collection of the SQL scripts.

Description

Use the SQLScripts collection to store and manage SQL scripts in application memory, instead of the external OS files.

Each collection item has the Name property, identifying a script, and the SQL property, actually containing a script. A script then may be referenced by its name. The item with zero index is the main (root) script and the execution will start from it.

A SQL script loading mechanism may be overridden using the OnGetText (see page 699) event handler. Alternatively a SQL script may be submitted from an external OS file using the SQLScriptFileName property. The SQLScriptFileName has high priority than SQLScript collection.

Note, that ExecuteFile (see page 692) and ExecuteScript (see page 693) methods clear the existing SQLScripts content.

Syntax

```
property SQLScripts: TADSQLScripts;
```

See Also

ExecuteAll (see page 691), OnGetText (see page 699), SQLScriptFileName (see page 703)

Example

```
with ADScript1.SQLScripts do begin
  Clear;
  with Add do begin
    Name := 'root';
    SQL.Add('@first');  // explicitly call 'first' script
    SQL.Add('@second'); // explicitly call 'second' script
  end;
  with Add do begin
    Name := 'first';
    SQL.Add('create table t1 (...);');
    SQL.Add('create table t2 (...);');
  end;
  with Add do begin
    Name := 'second';
    SQL.Add('create procedure p1 (...);');
    SQL.Add('create procedure p2 (...);');
  end;
end;
ADScript1.ValidateAll;
ADScript1.ExecuteAll;
```

1.17.4.1.1.2.26 TADScript.Transaction Property

Specifies a transaction object.

Description

The Transaction property value points to a transaction object. The SQL commands, executed by the script engine, will be prepared and executed in the context of this transaction. The Transaction property value must be specified before ExecuteAll (see page 691) or ExecuteStep (see page 694) calls. If a value is not specified, then the commands will use a default transaction object, specified at TADCustomConnection.Transaction (see page 337) or internal connection transaction object.

Syntax

```
property Transaction: TADCustomTransaction;
```

See Also

Managing Transactions (see page 41), Connection (see page 697)

1.17.4.1.2 TADScriptCommand Class

The base class for custom script commands.

public

public	Description
Engine (see page 705)	Returns a reference to SQL script engine.
EngineIntf (see page 705)	Returns reference to SQL script engine private API.
Parser (see page 706)	Returns reference to a SQL Script parser object.
Position (see page 706)	Returns the current script command position.
AbortJob (see page 706)	Implements SQL script command abortion.
Execute (see page 706)	Implements the script control command execution.
Help (see page 707)	Implements the command help output.
Keywords (see page 707)	Implements command keywords providing.
Parse (see page 707)	Implements script command parsing.

Validate (see page 708)	Implements the script control command validation.

Class Hierarchy



File

uADCompScript

Description

The FireDAC SQL Script engine is fully extendable and allows to create custom script control commands.

Use the TADScriptCommand class as a base class for custom script commands. The command developer must implement the following virtual methods:

- Keywords (see page 707) - returns the list of the keywords, which run this command;
- Help (see page 707) - returns the command help;
- Parse (see page 707) - parses the script and prepares the command for execution;
- Validate (see page 708) - run the command in script validation mode;
- Execute (see page 706) - run the command in actual execution mode.

The command class must be registered with SQL Script engine, using ADScriptCommandRegistry().AddCommand (see page 709)(...) call.

Syntax

```
TADScriptCommand = class(TObject);
```

See Also

Executing SQL Scripts (see page 88), SQL Script Control Commands (see page 92), Developing Custom Commands (see page 95), AddCommand (see page 709)

Example

Please, check the uADCompScriptCommands unit for the command examples.

1.17.4.1.2.1 public

1.17.4.1.2.1.1 TADScriptCommand.Engine Property

Returns a reference to SQL script engine.

Description

Use the Engine property to get the reference to SQL script engine.

Syntax

```
property Engine: TADScript;
```

See Also

Developing Custom Commands (see page 95), EngineIntf (see page 705)

1.17.4.1.2.1.2 TADScriptCommand.EngineIntf Property

Returns reference to SQL script engine private API.

Description

Use the EngineIntf to get a reference to SQL script engine private API, accessible only to script command classes.

Syntax

```
property EngineIntf: IADScriptEngineIntf;
```

See Also

Developing Custom Commands (see page 95), Engine (see page 705)

1.17.4.1.2.1.3 TADScriptCommand.Parser Property

Returns reference to a SQL Script parser object.

Description

Use the Parser property to get a reference to a SQL Script parser object. It is useful only inside of Parse method implementation.

Syntax

```
property Parser: TADScriptParser;
```

See Also

Developing Custom Commands (see page 95), Parse (see page 707)

1.17.4.1.2.1.4 TADScriptCommand.Position Property

Returns the current script command position.

Description

Use the Position property to get / set the current SQL script position. This property is useful only in Parse method implementation.

Syntax

```
property Position: TPoint;
```

See Also

Developing Custom Commands (see page 95), Parse (see page 707)

1.17.4.1.2.1.5 TADScriptCommand.AbortJob Method

Implements SQL script command abortion.

Parameters

Parameters	Description
const AWait: Boolean = False	True, if the AbortJob must wait, until the execution will be finished.

Description

The AbortJob method implementation allows to abort the SQL script execution in full, by terminating the current control command execution. The method may be calling from other thread, than Execute (see page 706) method.

Syntax

```
procedure AbortJob(const AWait: Boolean = False); virtual;
```

See Also

Developing Custom Commands (see page 95), Execute (see page 706)

1.17.4.1.2.1.6 TADScriptCommand.Execute Method

Implements the script control command execution.

Description

The Execute method implementation is responsible for the script control command actual execution. The method should use the internal data prepared by the Parse (see page 707) method call.

Syntax

```
procedure Execute; virtual;
```

See Also

Developing Custom Commands (see page 95), Parse (see page 707)

1.17.4.1.2.1.7 TADScriptCommand.Help Method

Implements the command help output.

Description

The Help method provides the command readable description, which may be output to a script console using HELP script command.

Syntax

```
class function Help: String; virtual;
```

See Also

Developing Custom Commands (see page 95)

1.17.4.1.2.1.8 TADScriptCommand.Keywords Method

Implements command keywords providing.

Parameters

Parameters	Description
AKwds: TStrings	The keyword list.

Description

The Keywords method implementation returns the keyword list, which SQL Script engine will use to recognize this command in script.

Each keyword consist from mandatory part (upper case characters) and optional part (lower case characters). The mandatory part must be unique across other commands. The optional part may be avoided in script text.

Syntax

```
class procedure Keywords(AKwds: TStrings); virtual;
```

See Also

Developing Custom Commands (see page 95), Parse (see page 707)

1.17.4.1.2.1.9 TADScriptCommand.Parse Method

Implements script command parsing.

Parameters

Parameters	Description
const AKwd: String	A keyword, which triggered this command execution.

Description

The parse method implementation parses the script text and builds an command internal state, which will be used by the Execute (see page 706) and Validate (see page 708)methods.

Syntax

```
function Parse(const AKwd: String): Boolean; virtual;
```

See Also

Developing Custom Commands (see page 95), Execute (see page 706), Keywords (see page 707)

1.17.4.1.2.1.10 TADScriptCommand.Validate Method

Implements the script control command validation.

Description

The Validate method implementation is responsible for the script control command validation mode. The method should use the internal data prepared by the Parse (see page 707) method call.

Syntax

```
procedure Validate; virtual;
```

See Also

Developing Custom Commands (see page 95), Execute (see page 706), Parse (see page 707)

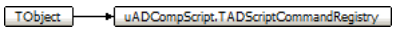
1.17.4.1.3 TADScriptCommandRegistry Class

The list of the registered script commands.

public

public	Description
Commands (see page 708)	Returns reference to a command class by the index.
Count (see page 709)	Returns the number of registered command classes.
AddCommand (see page 709)	Registers new command class.
LookupCommand (see page 709)	Finds a command class by the command name.

Class Hierarchy



File

uADCompScript

Description

The TADScriptCommandRegistry class implement the registry of the script commands.

Syntax

```
TADScriptCommandRegistry = class(TObject);
```

See Also

Executing SQL Scripts (see page 88), Developing Custom Commands (see page 95), TADScriptCommand (see page 704)

1.17.4.1.3.1 public

1.17.4.1.3.1.1 TADScriptCommandRegistry.Commands Property

Returns reference to a command class by the index.

Description

Use Commands property to get a registered script command class by its index.

Syntax

```
property Commands [AIndex: Integer]: TADScriptCommandClass;
```

See Also

Count (see page 709)

1.17.4.1.3.1.2 TADScriptCommandRegistry.Count Property

Returns the number of registered command classes.

Description

Use Count property to get the number of registered script command classes.

Syntax

```
property Count: Integer;
```

See Also

Commands (see page 708)

1.17.4.1.3.1.3 TADScriptCommandRegistry.AddCommand Method

Registers new command class.

Parameters

Parameters	Description
ACommand: TADScriptCommandClass	A command class.

Description

Use AddCommand method to register new script command class.

Syntax

```
procedure AddCommand(ACommand: TADScriptCommandClass);
```

See Also

Developing Custom Commands (see page 95), Commands (see page 708), Count (see page 709)

1.17.4.1.3.1.4 TADScriptCommandRegistry.LookupCommand Method

Finds a command class by the command name.

Parameters

Parameters	Description
const AStr: String	A keyword.
out ACommand: TADScriptCommandClass	A command class.

Description

The LookupCommand method is used by the ADScript to find the registered command class by its keyword.

Syntax

```
function LookupCommand(const AStr: String; out ACommand: TADScriptCommandClass):
TADScriptCommandLookupRes;
```

See Also

AddCommand (see page 709)

1.17.4.1.4 TADScriptOptions Class

The class implementing SQL script engine options.

published

published	Description
AutoPrintParams (see page 711)	Controls the output of the parameter values after a SQL command execution.

BreakOnError (see page 711)	Controls an action the script engine should take if an error has happened.
CharacterSet (see page 711)	Specifies the default connection character set.
ClientLib (see page 711)	Specifies the default connection client library.
ColumnHeadings (see page 712)	Controls output of the result set column headers.
CommandSeparator (see page 712)	Specifies the SQL command separator.
CommitEachNCommands (see page 712)	Specifies the number of SQL commands, after which a COMMIT must be performed.
ConsoleOutput (see page 713)	Controls the script engine output to a console.
DefaultDataPath (see page 713)	Specifies default path to the data files for the COPY command.
DefaultScriptPath (see page 713)	Specifies default path to the script files.
DriverID (see page 713)	Specifies the default connection driver ID.
DropNonexistObj (see page 714)	Controls the dropping of the nonexistent database objects.
EchoCommands (see page 714)	Specifies what parts of a SQL script must be echoed to console.
EchoCommandTrim (see page 714)	Controls the command echo output trimming.
FeedbackCommands (see page 715)	Controls the feedback output after each SQL command execution.
FeedbackScript (see page 715)	Controls the feedback output after each SQL script execution.
FileEncoding (see page 715)	Specifies the SQL script file encoding.
FileEndOfLine (see page 716)	Specifies the SQL script file end-of-line character sequence.
IgnoreError (see page 716)	Controls the script engine action on the errors.
LineSize (see page 716)	Specifies the maximum line width.
MacroExpand (see page 716)	Controls macro substitution for script engine.
MaxStringWidth (see page 717)	Specifies the maximum string field value size.
PageSize (see page 717)	Specifies the rowset page size.
ParamArraySize (see page 717)	Specifies the Array DML size.
RaisePLSQLErrors (see page 717)	Controls raising of exceptions after the PL/SQL compilation errors.
SpoolFileName (see page 718)	Specifies the spool file name.
SpoolOutput (see page 718)	Controls the spooling output mode.
SQLDialect (see page 718)	Specifies the SQL dialect for new Firebird or Interbase connections.
Timing (see page 719)	Controls the command performance feedback.
TrimConsole (see page 719)	Controls the right spaces trimming from the console output.
TrimSpool (see page 719)	Controls the right spaces trimming from the spool output.
Verify (see page 719)	Enables to show a complete SQL command text before execution.
Assign (see page 720)	Assigns a TADScriptOptions object to this object.
Reset (see page 720)	Reset this object property values to their default state.

Class Hierarchy



File

uADCompScript

Description

The TADScriptOptions class contains a set of the SQL script engine options.

Syntax

```
TADScriptOptions = class(TPersistent);
```

See Also

Executing SQL Scripts (see page 88), TADScript (see page 685)

1.17.4.1.4.1 published

1.17.4.1.4.1.1 TADScriptOptions.AutoPrintParams Property

Controls the output of the parameter values after a SQL command execution.

Description

Set AutoPrintParams property to True to enable the SQL command parameter output after the command execution. The default value is False.

The property may be set from a script using SET AUTOPRINT ON|OFF command.

Syntax

```
property AutoPrintParams: Boolean;
```

1.17.4.1.4.1.2 TADScriptOptions.BreakOnError Property

Controls an action the script engine should take if an error has happened.

Description

Set BreakOnError to True to stop a script execution on first error. The default value is False. The property has effect only when IgnoreError (see page 716) is False.

The property may be set from a script using SET BREAK ON|OFF command.

Syntax

```
property BreakOnError: Boolean;
```

See Also

IgnoreError (see page 716)

1.17.4.1.4.1.3 TADScriptOptions.CharacterSet Property

Specifies the default connection character set.

Description

Use the CharacterSet property to specify the character set, which will be used for the script CONNECT <connection string> command.

The property may be set from a script using SET (CHARACTERset | NAMES) <value> command.

Syntax

```
property CharacterSet: String;
```

See Also

DriverID (see page 713), ClientLib (see page 711)

1.17.4.1.4.1.4 TADScriptOptions.ClientLib Property

Specifies the default connection client library.

Description

Use the ClientLib property to specify the DBMS client library, which will be used for the script CONNECT <connection string> command. Note, that setting ClientLib has effect only before a first connection to a DB will be established.

The property may be set from a script using SET CLIENTlib <value> command.

Syntax

```
property ClientLib: String;
```

See Also

DriverID (see page 713), TADScript.Connection

Example 1

```
ADScript1.ScriptOptions.DriverID := 'IB';
ADScript1.ScriptOptions.ClientLib := 'c:\fbclient.dll';
with ADScript1.SQLScripts.Add do begin
  SQL.Add(...);
  SQL.Add('connect sysdba/masterkey');
end;
```

Example 2

```
with ADScript1.SQLScripts.Add do begin
  SQL.Add('SET DRIVERID IB');
  SQL.Add('SET CLIENTLIB ''c:\fbclient.dll''');
  SQL.Add('connect sysdba/masterkey');
end;
```

1.17.4.1.4.1.5 TADScriptOptions.ColumnHeadings Property

Controls output of the result set column headers.

Description

Set property ColumnHeadings to False to suppress output of the result set column headings at beginning of result set and at beginning of each next page with the result set rows. The default value is True. The headings will be output before each PageSize (see page 717) rows.

The property may be set from a script using SET HEADING ON|OFF command.

Syntax

```
property ColumnHeadings: Boolean;
```

See Also

PageSize (see page 717), ConsoleOutput (see page 713), SpoolOutput (see page 718)

1.17.4.1.4.1.6 TADScriptOptions.CommandSeparator Property

Specifies the SQL command separator.

Description

Use the CommandSeparator property to specify the SQL command separator. The default value is empty string, that means, that actual separator value will be choosed depending on a DBMS. For example, ';' for Oracle and 'go' for SQL Server. If a DBMS is not known to FireDAC, then ';' will be used as actual command separator value.

The property may be set from a script using SET (CMDSep | TERMinator) <value> command.

Syntax

```
property CommandSeparator: String;
```

See Also

ActualCommandSeparator

1.17.4.1.4.1.7 TADScriptOptions.CommitEachNCommands Property

Specifies the number of SQL commands, after which a COMMIT must be performed.

Description

Set CommitEachNCommands to the number of SQL commands, after which the script engine will perform COMMIT. Set this property to 0 to enable auto-commit mode, where each SQL command will be committed. The default value is 0.

The property may be set from a script using SET AUTOCOMMIT N command.

Syntax

```
property CommitEachNCommands: Integer;
```

See Also

TADTxOptions.AutoCommit

1.17.4.1.4.1.8 TADScriptOptions.ConsoleOutput Property

Controls the script engine output to a console.

Description

Set ConsoleOutput to True to enable scrupt engine output to a console. The default value is True. The output will be provided through the script dialog, if it is attached to the TADScript.ScriptDialog (see page 702), and/or TADScript.OnConsolePut (see page 698) event handler.

The property may be set from a script using SET (TERMOUT | CONSOLE) ON|OFF command.

Syntax

```
property ConsoleOutput: Boolean;
```

See Also

TrimConsole (see page 719), TADScript.OnConsolePut, TADScript.ScriptDialog

1.17.4.1.4.1.9 TADScriptOptions.DefaultDataPath Property

Specifies default path to the data files for the COPY command.

Description

Set DefaultDataPath property to a folder with the text data files, which will be used by the COPY SQL script command. If COPY command has text data file without a path specified, then DefaultDataPath will be used as a path.

Syntax

```
property DefaultDataPath: String;
```

1.17.4.1.4.1.10 TADScriptOptions.DefaultScriptPath Property

Specifies default path to the script files.

Description

Set DefaultScriptPath property value to a folder with the SQL script files. If a script file name is specified without a path, then DefaultScriptPath will be used as a path.

Syntax

```
property DefaultScriptPath: String;
```

See Also

TADScript.SQLScriptFileName

1.17.4.1.4.1.11 TADScriptOptions.DriverID Property

Specifies the default connection driver ID.

Description

Use the DriverID property to specify the ID of default driver, which will be used for the script CONNECT <connection string> command. The property may be set from a script using SET DRIVERid <value> command.

Syntax

```
property DriverID: String;
```

See Also

TADScript.Connection

Example

```
ADScript1.ScriptOptions.DriverID := 'Ora';
with ADScript1.SQLScripts.Add do begin
  SQL.Add(...);
  SQL.Add('connect scott/tiger@orasrv');
end;
```

1.17.4.1.4.1.12 TADScriptOptions.DropNonexistObj Property

Controls the dropping of the nonexistent database objects.

Description

Set DropNonexistObj to True to ignore the "table / view / what else not exists" error at the DROP or ALTER TABLE DROP CONSTRAINT SQL command execution. This is useful to avoid the checking for the dropping object existence. The default value is True.

The property may be set from a script using SET DROPnonexistent ON|OFF command.

Syntax

```
property DropNonexistObj: Boolean;
```

See Also

BreakOnError (see page 711)

1.17.4.1.4.1.13 TADScriptOptions.EchoCommands Property

Specifies what parts of a SQL script must be echoed to console.

Description

Set EchoCommands to one of the values, to enable SQL script commands echo output to a script console:

Value	Description
ecNone	No commands will be output.
ecSQL	Only SQL and PL/SQL commands will be output.
ecAll	SQL, PL/SQL and the script control commands will be output.

The default value is ecSQL. The output to the console is performed only when ConsoleOutput (see page 713) is True. When a command text is greater than EchoCommandTrim (see page 714), then the command text will be trimmed to the specified by EchoCommandTrim (see page 714) length.

The property may be set from a script using SET ECHO OFF | ON [SQL|ALL] [TRIM <size>] command.

Syntax

```
property EchoCommands: TADScriptEchoCommands;
```

See Also

ConsoleOutput (see page 713), EchoCommandTrim (see page 714), TrimConsole (see page 719)

1.17.4.1.4.1.14 TADScriptOptions.EchoCommandTrim Property

Controls the command echo output trimming.

Description

Set EchoCommandTrim to an integer value, which is the maximum command text length to output. When the text is greater than the specified value, then the text will be trimmed and '...' appended. When the value is <= 0, then no trimming is performed. The default value is 0. The EchoCommandTrim is used together with EchoCommands (see page 714).

The property may be set from a script using SET ECHO OFF | ON [SQL|ALL] [TRIM <size>] command.

Syntax

```
property EchoCommandTrim: Integer;
```

See Also

ConsoleOutput (see page 713), EchoCommands (see page 714), TrimConsole (see page 719)

1.17.4.1.4.1.15 TADScriptOptions.FeedbackCommands Property

Controls the feedback output after each SQL command execution.

Description

Set FeedbackCommands to True, to enable the feedback output after each SQL command execution. The feedback may include:

- the number of rows fetched for the commands returning a result set;
- the number of rows updated by the INSERT/ UPDATE / DELETE SQL commands;
- the command execution time, when Timing is True;
- DB properties after connecting to a DB using CONNECT command;
- the number of processed rows by the COPY command.

The default value is True.

The property may be set from a script using SET (FEEDback | COUNT) (OFF | ON) command.

Syntax

```
property FeedbackCommands: Boolean;
```

See Also

ConsoleOutput (see page 713), FeedbackScript (see page 715), Timing (see page 719)

1.17.4.1.4.1.16 TADScriptOptions.FeedbackScript Property

Controls the feedback output after each SQL script execution.

Description

Set FeedbackScript to True, to enable the feedback output after each SQL script execution. The feedback may include:

- the script name to execute;
- the number of errors returned, while the script was processing;
- the script execution time, when Timing is True.

The default value is True.

Syntax

```
property FeedbackScript: Boolean;
```

See Also

ConsoleOutput (see page 713), FeedbackCommands (see page 715), Timing (see page 719)

1.17.4.1.4.1.17 TADScriptOptions.FileEncoding Property

Specifies the SQL script file encoding.

Description

Set FileEncoding property value to the encoding of the SQL script file. Default value is enANSI.

The property may be set from a script using SET ENCoding (UTF8 | UTF16 | ANSI) command.

Syntax

```
property FileEncoding: TADEncoding;
```

See Also

FileEndOfLine (see page 716)

1.17.4.1.4.1.18 TADScriptOptions.FileEndOfLine Property

Specifies the SQL script file end-of-line character sequence.

Description

Set FileEndOfLine property value to the end-of-line character sequence kind. The default value is elDefault.

Syntax

```
property FileEndOfLine: TADTextEndOfLine;
```

See Also

FileEncoding (see page 715)

1.17.4.1.4.1.19 TADScriptOptions.IgnoreError Property

Controls the script engine action on the errors.

Description

Set IgnoreError to True, to stop the error message output to the console. When it is False the exception error message will be output to the console. And when BreakOnError (see page 711) is True, then the script execution will be terminated after the exception. Default value is False.

The property may be set from a script using SET IGNORE (OFF | ON) command.

Syntax

```
property IgnoreError: Boolean;
```

See Also

BreakOnError (see page 711)

1.17.4.1.4.1.20 TADScriptOptions.LineSize Property

Specifies the maximum line width.

Description

Set LineSize to the maximum number of characters per console line. When script engine will output more long line, then it will be splitted into few lines, each of LineSize characters long. The default value is 0 (do not split).

The property may be set from a script using SET LINEsize <value> command.

Syntax

```
property LineSize: Integer;
```

1.17.4.1.4.1.21 TADScriptOptions.MacroExpand Property

Controls macro substitution for script engine.

Description

Set MacroExpand to True to enable macro expansion for the script engine.

The macro substitution variables has the form &name or !name. They may appear in practically all script places, including script names. The macro variables and their values are listed in TADScript.Macros (see page 698) property and may be set from script using DEFINE name=value command.

Also, the script arguments may be referenced using &index or !index form. There the index is a value index from

TADScript.Arguments (see page 696).

The default value is True.

The property may be set from a script using SET (DEFine | SCAN) (ON | OFF) command.

Syntax

```
property MacroExpand: Boolean;
```

See Also

TADScript.Macros, TADScript.Arguments

1.17.4.1.4.1.22 TADScriptOptions.MaxStringWidth Property

Specifies the maximum string field value size.

Description

Set MaxStringWidth to the maximum number of characters per string field value. When script engine will output more long field value, then it will be trimmed to MaxStringWidth characters. The default value is 80 characters.

The property may be set from a script using SET (LONG | WIDTH) <value> command.

Syntax

```
property MaxStringWidth: Integer;
```

See Also

LineSize (see page 716)

1.17.4.1.4.1.23 TADScriptOptions.PageSize Property

Specifies the rowset page size.

Description

Set PageSize to the number of rows, before which script engine will put the column headings, when ColumnHeadings (see page 712) is True. The default value is 24.

The property may be set from a script using SET PAGESize <value> command.

Syntax

```
property PageSize: Integer;
```

See Also

ColumnHeadings (see page 712), ConsoleOutput (see page 713), SpoolOutput (see page 718)

1.17.4.1.4.1.24 TADScriptOptions.ParamArraySize Property

Specifies the Array DML size.

Description

Set the ParamArraySize to the Array DML size.

Syntax

```
property ParamArraySize: Integer;
```

See Also

TADScript.Params

1.17.4.1.4.1.25 TADScriptOptions.RaisePLSQLErrors Property

Controls raising of exceptions after the PL/SQL compilation errors.

Description

Set RaisePLSQLErrors to True to raise an exception after PL/SQL compilation, if there were any errors. That will allow to stop the script execution, if some PL/SQL related command, like a CREATE PROCEDURE or LATER PACKAGE finished with the errors.

The default value is False.

Syntax

```
property RaisePLSQLErrors: Boolean;
```

See Also

BreakOnError (see page 711), IgnoreError (see page 716)

1.17.4.1.4.1.26 TADScriptOptions.SpoolFileName Property

Specifies the spool file name.

Description

Set the SpoolFileName property value to the name of a file, where the spooling output must be recorded. The spooling output duplicates output to the console. The output to the spool file will be performed only when SpoolFileName is specified and SpoolOutput (see page 718) is no smNone.

The property may be set from a script using (SPOol | OUTput) [OFF|[APPend] <spool name>] command.

Syntax

```
property SpoolFileName: String;
```

See Also

SpoolOutput (see page 718), TrimSpool (see page 719), TADScript.OnSpoolPut

1.17.4.1.4.1.27 TADScriptOptions.SpoolOutput Property

Controls the spooling output mode.

Description

Set SpoolOutput to smOnAppend or smOnReset to enable output to the spool file with SpoolFileName (see page 718) name. smOnReset - overwrite the file, amOnAppend - append output to the existing file. Set SpoolOutput to smNone to disable output. The default value is smNone.

The property may be set from a script using (SPOol | OUTput) [OFF|[APPend] <spool name>] command.

Syntax

```
property SpoolOutput: TADScriptSpoolOutputMode;
```

See Also

SpoolFileName (see page 718), TrimSpool (see page 719), TADScript.OnSpoolPut

1.17.4.1.4.1.28 TADScriptOptions.SQLDialect Property

Specifies the SQL dialect for new Firebird or Interbase connections.

Description

Set SQLDialect property value to the Firebird / Interbase SQL dialect, which will be used at new connections opened with CONNECT script command or with CREATE DATABASE command.

The property may be set from a script using SET SQL DIALECT value command.

Syntax

```
property SQLDialect: Integer;
```

1.17.4.1.4.1.29 TADScriptOptions.Timing Property

Controls the command performance feedback.

Description

Set Timing to True to enable performance output after each SQL command and after each script. The output includes the command and script execution times. The Timing has only effect, when FeedbackCommands (see page 715) is True. The default value is True.

The property may be set from a script using SET TIMing OFF|ON command.

Syntax

```
property Timing: Boolean;
```

See Also

FeedbackCommands (see page 715)

1.17.4.1.4.1.30 TADScriptOptions.TrimConsole Property

Controls the right spaces trimming from the console output.

Description

Set TrimConsole to True to trim right spaces from the console output text. The default value is False.

The property may be set from a script using SET TRIMout OFF|ON command.

Syntax

```
property TrimConsole: Boolean;
```

See Also

ConsoleOutput (see page 713), TrimSpool (see page 719)

1.17.4.1.4.1.31 TADScriptOptions.TrimSpool Property

Controls the right spaces trimming from the spool output.

Description

Set TrimSpool to True to trim right spaces from the spool output text. The default value is True.

The property may be set from a script using SET TRIMSpool OFF|ON command.

Syntax

```
property TrimSpool: Boolean;
```

See Also

SpoolOutput (see page 718)

1.17.4.1.4.1.32 TADScriptOptions.Verify Property

Enables to show a complete SQL command text before execution.

Description

Set Verify to True to enable output of a SQL command text with substituted macro variables, if any. The default value is False.

The property may be set from a script using SET VERify OFF|ON command.

Syntax

```
property Verify: Boolean;
```

1.17.4.1.4.1.33 TADScriptOptions.Assign Method

Assigns a TADScriptOptions (see page 709) object to this object.

Parameters

Parameters	Description
Source: TPersistent	The source object.

Description

Use the Assign method to assign TADScriptOptions object property values to corresponding properties of this object.

Syntax

```
procedure Assign(Source: TPersistent); override;
```

1.17.4.1.4.1.34 TADScriptOptions.Reset Method

Reset this object property values to their default state.

Description

Call Reset method to reset all properties of this objects to their default values.

Syntax

```
procedure Reset;
```

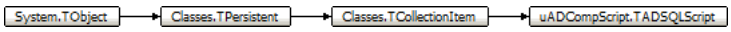
1.17.4.1.5 TADSQLScript Class

An in-memory stored SQL (see page 721) script.

published

published	Description
Name (see page 720)	The name of a script.
SQL (see page 721)	The SQL script.

Class Hierarchy



File

uADCompScript

Description

The TADSQLScript class represents a SQL script, stored in program memory. Each script is identified by its name by the Name property and contains script commands in its SQL (see page 721) property.

Syntax

```
TADSQLScript = class(TCollectionItem);
```

See Also

Executing SQL Scripts (see page 88)

1.17.4.1.5.1 published

1.17.4.1.5.1.1 TADSQLScript.Name Property

The name of a script.

Description

Set Name property to a name of a SQL script. Engine will use this name to find a script, when it is launched by @<name> or similar command.

Syntax

```
property Name: String;
```

See Also

SQL (see page 721), TADSQLScripts.FindScript (see page 722)

1.17.4.1.5.1.2 TADSQLScript.SQL Property

The SQL script.

Description

Use SQL property to store the list of SQL and script control commands. The commands may span few lines and must be separated by the command separator.

Syntax

```
property SQL: TStrings;
```

See Also

Name (see page 720), TADScriptOptions.CommandSeparator (see page 712)

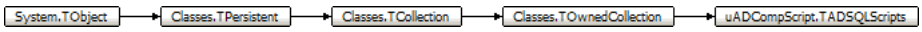
1.17.4.1.6 TADSQLScripts Class

TADSQLScripts is a collection of the TADSQLScript (see page 720) objects.

public

public	Description
Items (see page 721)	Returns SQL script object by its index.
Add (see page 722)	Add new SQL script object to a collection.
FindScript (see page 722)	Returns a SQL script by its name.

Class Hierarchy



File

uADCompScript

Description

The TADSQLScripts class is a collection of the TADSQLScript (see page 720) objects. All scripts in a collection must have unique names. Use:

- Add (see page 722) method to add new script object.
- FindScript (see page 722) to find a script by its name.

Syntax

```
TADSQLScripts = class(TOwnedCollection);
```

See Also

Executing SQL Scripts (see page 88), TADScript.SQLScripts

1.17.4.1.6.1 public

1.17.4.1.6.1.1 TADSQLScripts.Items Property

Returns SQL script object by its index.

Description

Use Items property to get access by an index to the SQL script objects.

Syntax

```
property Items [AIndex: Integer]: TADSQLScript;
```

See Also

FindScript (see page 722)

1.17.4.1.6.1.2 TADSQLScripts.Add Method

Add new SQL script object to a collection.

Description

Call the Add method to add new empty SQL script object to this collection.

Syntax

```
function Add: TADSQLScript;
```

1.17.4.1.6.1.3 TADSQLScripts.FindScript Method

Returns a SQL script by its name.

Parameters

Parameters	Description
const AName: String	A name.

Description

Use the FindScript method to find a SQL script by its name.

Syntax

```
function FindScript(const AName: String): TADSQLScript;
```

1.17.5 uADMoniRemoteClient Namespace

Contains TADMoniRemoteClientLink (see page 723) class.

Classes

Class	Description
TADMoniRemoteClientLink (see page 723)	Use the TADMoniRemoteClientLink component to link ADMonitor (see page 185) tracing capabilities to an application and setup it.

Description

The uADMoniRemoteClient unit contains:

- TADMoniRemoteClientLink (see page 723) class - component implementing and controlling the trace output to ADMonitor (see page 185) utility.

See Also

Tracing and Monitoring (see page 174)

1.17.5.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADMoniRemoteClientLink (see page 723)	Use the TADMoniRemoteClientLink component to link ADMonitor (see page 185) tracing capabilities to an application and setup it.

1.17.5.1.1 TADMoniRemoteClientLink Class

Use the TADMoniRemoteClientLink component to link ADMonitor (see page 185) tracing capabilities to an application and setup it.

published

published	Description
Host (see page 723)	The IP host address, where ADMonitor (see page 185) is running.
Port (see page 723)	The IP port, where ADMonitor (see page 185) is listening.
Timeout (see page 724)	ADMonitor (see page 185) connection timeout.
Tracing (see page 724)	Add a short summary here...

Class Hierarchy



File

uADMoniRemoteClient

Description

Use the TADMoniRemoteClientLink component to link the ADMonitor tracing capabilities to an application.

To enable ADMonitor trace output, add MonitorBy=Remote parameter (see page 188) to your connection definition (see page 27). And set Tracing (see page 724) property to True.

ADMonitor and TADMoniRemoteClientLink are using TPC/IP protocol to communicate. Use the Host (see page 723) property to specify the ADMonitor host IP address, and Port (see page 723) to specify the IP port where ADMonitor is listening. EventKinds property specifies the information set to send to ADMonitor.

Syntax

```
TADMoniRemoteClientLink = class(TADMoniClientLinkBase);
```

See Also

Tracing and Monitoring (see page 174)

1.17.5.1.1.1 published

1.17.5.1.1.1.1 TADMoniRemoteClientLink.Host Property

The IP host address, where ADMonitor (see page 185) is running.

Description

Use the Host property to specify the IP address of the host where ADMonitor utility is running. Default value is localhost.

Syntax

```
property Host: String;
```

See Also

Port (see page 723)

1.17.5.1.1.1.2 TADMoniRemoteClientLink.Port Property

The IP port, where ADMonitor (see page 185) is listening.

Description

Use the Port property to specify the IP port where ADMonitor utility is listening. Default value is 8050.

Syntax

```
property Port: Integer;
```

See Also

Host (see page 723)

1.17.5.1.1.1.3 TADMoniRemoteClientLink.Timeout Property

ADMonitor (see page 185) connection timeout.

Description

Use Timeout property to specify the connection establishment timeout in msecs. Default value is 3000 (3 secs).

Syntax

```
property Timeout: Integer;
```

1.17.5.1.1.1.4 TADMoniRemoteClientLink.Tracing Property

Add a short summary here...

Description

Add a description here...

Syntax

```
property Tracing;
```

See Also

List here...

Example

1.17.6 uADPhysADS Namespace

Contains Advantage Database Server driver and service components.

Classes

Class	Description
TADADSBackup (see page 725)	The class implementing Advantage backup database service.
TADADSBackupRestore (see page 727)	The base class for backup and restored Advantage service classes.
TADADSRestore (see page 729)	The class implementing Advantage service restoring a database from a backup.
TADADSService (see page 731)	The base class for all Advantage service classes.
TADADSUtility (see page 733)	The class implementing Advantage table utilities.
TADPhysADSDriverLink (see page 737)	Use the TADPhysADSDriverLink component to link the Advantage Database Server driver to an application and setup it.

Description

The uADPhysADS unit contains:

- Advantage Database Server driver implementation;
- TADPhysADSDriverLink (see page 737) - component to link the driver and setup CLI library;

- TADADSBackup (see page 725) - backup DB service component;
- TADADSRestore (see page 729) - restore DB service component.

See Also

Configuring Drivers (see page 31), Connect to Advantage Database Server (see page 189)

1.17.6.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADADSBackup (see page 725)	The class implementing Advantage backup database service.
TADADSBackupRestore (see page 727)	The base class for backup and restored Advantage service classes.
TADADSRestore (see page 729)	The class implementing Advantage service restoring a database from a backup.
TADADSService (see page 731)	The base class for all Advantage service classes.
TADADSUtility (see page 733)	The class implementing Advantage table utilities.
TADPhysADSDriverLink (see page 737)	Use the TADPhysADSDriverLink component to link the Advantage Database Server driver to an application and setup it.

1.17.6.1.1 TADADSBackup Class

The class implementing Advantage backup database service.

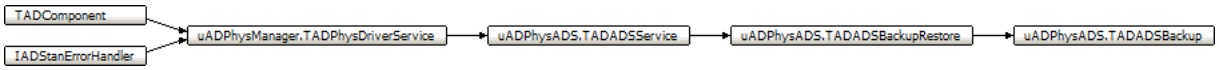
public

public	Description
Backup (see page 726)	Performs the backup task.

published

published	Description
BackupPath (see page 726)	Specifies the path to place the backup image in.
Options (see page 727)	Specifies the backup options.

Class Hierarchy



File

uADPhysADS

Description

Use the TADADSBackup component to add a backup Advantage database capability to an application. This is a programmatic method to invoke the **adsbackup** tool.

To produce a DB backup an application should:

- specify DriverLink (see page 732), UserName (see page 733), Password (see page 732), Database (see page 732);
- specify BackupPath (see page 726) - the path to put the backup files;
- call Backup (see page 726) method.

Other properties and methods are optional.

Paths of backup files are relative to the server. Because the TADADSBackup executes backup on the DB server host and

writes backup files on the server host. You must be logged in as the ADSSYS user or a user in the *DB:Backup* group. Note, the TADADSBackup works only with Advantage network server and not with local server.

To restore an backup performed by TADADSBackup use the TADADSRestore (see page 729) component.

Syntax

```
TADADSBackup = class(TADADSBackupRestore);
```

See Also

TADADSRestore (see page 729)

Example

```
ADADSBackup1.DriverLink := ADPhysADSDriverLink1;

ADADSBackup1.Database := '\\DA\ADS_DB\addemo.add';
ADADSBackup1.UserName := 'adssys';
ADADSBackup1.Password := 'a';
ADADSBackup1.BackupPath := 'C:\Temp';

ADADSBackup1.Backup;

ADMemTable1.AttachTable(ADADSBackup1.Results, nil);
ADMemTable1.Open;
```

1.17.6.1.1.1 public

1.17.6.1.1.1.1 TADADSBackup.Backup Method

Performs the backup task.

Description

Use the Backup method to start the backup task on the Advantage server.

To perform a backup task an application must specify the connection parameters (see page 731) and BackupPath (see page 726). This is a programmatic method to invoke the **adsbackup** tool or the **sp_BackupDatabase** stored procedure.

Syntax

```
procedure Backup;
```

See Also

TADADSService.Database (see page 732), TADADSService.UserName (see page 733), TADADSService.Password (see page 732), BackupPath (see page 726), Options (see page 727)

1.17.6.1.1.2 published

1.17.6.1.1.2.1 TADADSBackup.BackupPath Property

Specifies the path to place the backup image in.

Description

Path to place the backup image in. This path can be any form recognized by the server operating system. For example, in Microsoft Windows this path can be UNC (\\myserver\myshare\mydir) or a local drive letter (c:\mydir). If using drive letters keep in mind the drive letter must be recognized by the server machine, any drives mapped on the client executing the backup command are irrelevant. Mapped network drive letters are not supported, and should not be used.

Syntax

```
property BackupPath: String;
```

See Also

TADADSService.Database (see page 732)

1.17.6.1.1.2.2 TADADSBackup.Options Property

Specifies the backup options.

Description

Use the Options property to specify additional options controlling the backup task. The default value is empty set.

Option	Description
boDontOverwrite	Specify if you do not want the backup operation to overwrite existing tables. Instead a warning will be logged to the Results (see page 727).
boMetaOnly	Specify if you only want to backup the data dictionary file.
boPrepareDiff	Specify if you want to initialize a differential backup.
boDiff	Specify if you want to perform a differential backup on a database that you have previously initialized using the boPrepareDiff.

Syntax

```
property Options: TADADSBackupOptions;
```

1.17.6.1.2 TADADSBackupRestore Class

The base class for backup and restored Advantage service classes.

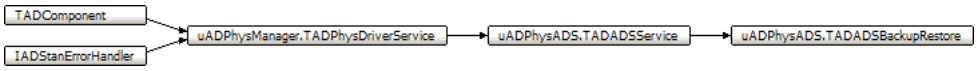
public

public	Description
Results (see page 727)	The reference to the backup / restore errors list.

published

published	Description
Exclude (see page 728)	Specifies the tables to exclude from backup or restore.
Include (see page 728)	Specifies the tables to include into backup or restore.
TableTypeMap (see page 728)	Specifies the table type to file extension mapping.

Class Hierarchy



File

uADPhysADS

Description

The TADADSBackupRestoreService class is used as a base class for TADADSBackup (see page 725) and TADADSRestore (see page 729) Advantage classes and should not be used directly. The class contains the common properties, like the Exclude, Include and TableTypeMap.

Syntax

```
TADADSBackupRestore = class(TADADSService);
```

1.17.6.1.2.1 public

1.17.6.1.2.1.1 TADADSBackupRestore.Results Property

The reference to the backup / restore errors list.

Description

Use the Results property to read the all last backup / restore operation error list. The property is filled after the call to Backup

(see page 726) or Restore (see page 730) methods.

Syntax

```
property Results: TADDatSTable;
```

See Also

Backup (see page 726), Restore (see page 730)

Example

```
ADADSBackup.Backup;
ADMemTable1.AttachTable(ADADSBackup.Results, nil);
ADMemTable1.Open;
```

1.17.6.1.2.2 published

1.17.6.1.2.2.1 TADADSBackupRestore.Exclude Property

Specifies the tables to exclude from backup or restore.

Description

Use the Exclude property to specify a list of tables to exclude from the backup or restore. The tables in this list will not be processed. Each table name must be located on a single line. When backing up or restoring a data dictionary, use the table object names. When backing up or restoring free tables, use the base filename and the file extension.

Syntax

```
property Exclude: TStrings;
```

See Also

Include (see page 728), TableTypeMap (see page 728)

1.17.6.1.2.2.2 TADADSBackupRestore.Include Property

Specifies the tables to include into backup or restore.

Description

Use this option to specify a list of tables to include in the backup or restore. Only the tables in this list will be processed. Each table name must be located on a single line. When backing up or restoring a data dictionary, use the table object names. When backing up or restoring free tables, use the base filename and the file extension.

Syntax

```
property Include: TStrings;
```

See Also

Exclude (see page 728), TableTypeMap (see page 728)

1.17.6.1.2.2.3 TADADSBackupRestore.TableTypeMap Property

Specifies the table type to file extension mapping.

Description

Use the TableTypeMap property to specify what table type should be used for a specific table extension. Only necessary for free table backup / restore operations. This option can be used to backup and/or restore a directory of free tables that consists of both ADT and DBF tables. Each mapping must be located on a separated line and should include a three character extension, followed by an equal sign, followed by the Advantage table type constant.

Syntax

```
property TableTypeMap: TStrings;
```

See Also

Exclude (see page 728), Include (see page 728)

Example

```
with ADADSBackup1.TableTypeMap do begin
  Clear;
  Add('adt=ADS_ADT');
  Add('dbf=ADS_CDX');
end;
```

1.17.6.1.3 TADADSRestore Class

The class implementing Advantage service restoring a database from a backup.

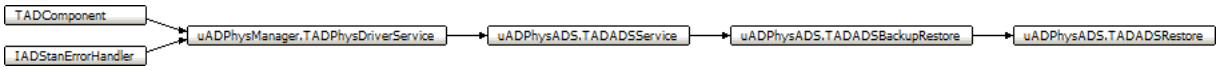
public

public	Description
Restore (see page 730)	Performs the restore task.

published

published	Description
DDPassword (see page 730)	Specify the data dictionary password.
DestinationPath (see page 730)	Path to restore the database to.
Options (see page 731)	Specifies the restore options.
SourcePassword (see page 731)	Source dictionary password.
SourcePath (see page 731)	Path to the existing backup image.

Class Hierarchy



File

uADPhysADS

Description

Use the TADADSRestore component to add a restoring database from backup capability to an application. This is a programmatic method to invoke the **adsbackup** tool.

To restore a database from a DB backup an application should:

- specify DriverLink (see page 732), UserName (see page 733), Password (see page 732), Database (see page 732);
- specify SourcePath (see page 731) - the path to the backup files;
- specify DestinationPath (see page 730) - the path to restore a database;
- call Restore (see page 730) method.

Other properties and methods are optional.

SourcePath is relative to the server. Because the TADADSRestore restores backup on the DB server host. The restoration can be performed using a free connection or a dictionary connection. You must be logged in as the ADSSYS user or a user in the *DB:Backup* group.

Syntax

```
TADADSRestore = class(TADADSBackupRestore);
```

See Also

TADADSBackup (see page 725)

Example

```
ADADSRestore1.DriverLink := ADPhysADSDriverLink1;

ADADSRestore1.Database := '\\DA\ADS_DB\addemo.add';
```

```
ADADSRestore1.UserName := 'adssys';
ADADSRestore1.Password := 'a';
ADADSRestore1.SourcePath := 'C:\Temp';
ADADSRestore1.DestinationPath := '\\DA\ADS_DB\addemo.add';

ADADSRestore1.Restore;

ADMemTable1.AttachTable(ADADSRestore1.Results, nil);
ADMemTable1.Open;
```

1.17.6.1.3.1 public

1.17.6.1.3.1.1 TADADSRestore.Restore Method

Performs the restore task.

Description

Use the Restore method to start the restore task on the Advantage server.

To perform a restore task an application must specify the connection parameters (see page 731), SourcePath (see page 731) and DestinationPath (see page 730). This is a programmatic method to invoke the **adsbackup** tool or the **sp_RestoreDatabase** stored procedure.

Syntax

```
procedure Restore;
```

See Also

TADADSService.Database (see page 732), TADADSService.UserName (see page 733), TADADSService.Password (see page 732), SourcePath (see page 731), DestinationPath (see page 730), Options (see page 731)

1.17.6.1.3.2 published

1.17.6.1.3.2.1 TADADSRestore.DDPassword Property

Specify the data dictionary password.

Description

Use the DDPassword property to specify the data dictionary password to use on a restore operation for a dictionary that uses AES encryption.

Syntax

```
property DDPassword: String;
```

1.17.6.1.3.2.2 TADADSRestore.DestinationPath Property

Path to restore the database to.

Description

Use the DestinationPath to specify path to restore the database to, including the name of the new .add file to create. An example DestinationPath would be \\myserver\myshare\myrestoredir\mydd.add. This path can be any form recognized by the server operating system. For example, in Microsoft Windows this path can be UNC (\\myserver\myshare\mydir) or a local drive letter (c:\mydir). If using drive letters keep in mind the drive letter must be recognized by the server machine, any drives mapped on the client executing the restore command are irrelevant. Mapped network drive letters are not supported, and should not be used.

Syntax

```
property DestinationPath: String;
```

See Also

SourcePath (see page 731)

1.17.6.1.3.2.3 TADADSRestore.Options Property

Specifies the restore options.

Description

Use the Options property to specify additional options controlling the restore task. The default value is empty set.

Option	Description
roDontOverwrite	Specify if you do not want the restore operation to overwrite existing tables. Instead a warning will be logged to the Results (see page 727).
roMetaOnly	Specify if you only want to restore the data dictionary file.

Syntax

```
property Options: TADADSRestoreOptions;
```

1.17.6.1.3.2.4 TADADSRestore.SourcePassword Property

Source dictionary password.

Description

Use the SourcePassword to specify a password to the dictionary specified in the SourcePath parameter.

Syntax

```
property SourcePassword: String;
```

See Also

SourcePath (see page 731)

1.17.6.1.3.2.5 TADADSRestore.SourcePath Property

Path to the existing backup image.

Description

Use the SourcePath to specify path to the existing backup image, including the name of the .add file. An example SourcePath would be \\myserver\myshare\mybackupdir\mydd.add. This path can be any form recognized by the server operating system. For example, in Microsoft Windows this path can be UNC (\\myserver\myshare\mydir) or a local drive letter (c:\mydir). If using drive letters, keep in mind the drive letter must be recognized by the server machine, any drives mapped on the client executing the restore command are irrelevant. If using a drive letter for the source path, the drive must be a "local" drive, it cannot be a mapped drive.

Syntax

```
property SourcePath: String;
```

See Also

SourcePassword (see page 731), DestinationPath (see page 730)

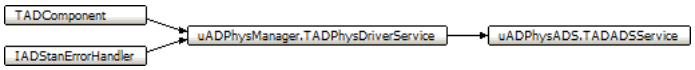
1.17.6.1.4 TADADSService Class

The base class for all Advantage service classes.

published

published	Description
Database (see page 732)	Specifies the connection path to the Advantage database.
DriverLink (see page 732)	Specifies the DriverLink.
Password (see page 732)	Specifies the SQL user password to connect to the DB server.
UserName (see page 733)	Specifies the SQL user name to connect to the DB server.

Class Hierarchy



File

uADPhysADS

Description

The TADADSService class is used as a base class for all Advantage service classes and should not be used directly. The class contains the common properties, like the DriverLink (see page 732), Database (see page 732), UserName (see page 733) and Password (see page 732).

Syntax

```
TADADSService = class(TADPhysDriverService);
```

1.17.6.1.4.1 published

1.17.6.1.4.1.1 TADADSService.Database Property

Specifies the connection path to the Advantage database.

Description

Use the Database property to specify the connection path to the database. This property must be specified for any service operation.

The fully qualified path to the computer where the data files exist and the default location of the data files. This fully qualified path must contain a drive letter or use UNC. If a free connection is desired, the directory location must be specified. For a database connection, the path and data dictionary file must be specified.

Additionally may be specified other Advantage connection definition parameters (see page 189), separated by ';'. For example, 'C:\DB;ServerTypes=Local'.

Syntax

```
property Database: String;
```

See Also

Connect to Advantage Database Server (see page 189)

Example

```
ADADSUtility1.Database := 'C:\DB;ServerTypes=Local';
```

1.17.6.1.4.1.2 TADADSService.DriverLink Property

Specifies the DriverLink.

Description

Use the DriverLink property to specify the Advantage driver link. This property must be specified for any service operation.

Syntax

```
property DriverLink: TADPhysADSDriverLink;
```

1.17.6.1.4.1.3 TADADSService.Password Property

Specifies the SQL user password to connect to the DB server.

Description

Use the Password property to specify the user password to connect to the Advantage database. This property must be specified for any service operation.

Syntax

```
property Password: String;
```

See Also

UserName (see page 733)

1.17.6.1.4.1.4 TADADSService.UserName Property

Specifies the SQL user name to connect to the DB server.

Description

Use the UserName property to specify the SQL user name to connect to the Advantage database. This property must be specified for any service operation.

Syntax

```
property UserName: String;
```

See Also

Password (see page 732)

1.17.6.1.5 TADADSUtility Class

The class implementing Advantage table utilities.

public

public	Description
Decrypt (see page 734)	Decrypts encrypted table and removes the password.
Encrypt (see page 734)	Encrypts a table and assigns a password.
Pack (see page 735)	Packs a table.
Recall (see page 735)	Recalls all deleted records in a table.
Reindex (see page 735)	Rebuilds all open indexes associated with the given table.
Zap (see page 736)	Removes all records from the table and re-indexes it.

published

published	Description
OnProgress (see page 736)	Fires after a table is processed.
TablePassword (see page 736)	Specifies encryption password for free tables.
Tables (see page 737)	The list of tables to process.
TableType (see page 737)	Specifies a table type.

Class Hierarchy



File

uADPhysADS

Description

Use the TADADSUtility component to add the database table utilities to an application, like zap, pack or reindex a table.

To use an utility an application should:

- specify DriverLink (see page 732), UserName (see page 733), Password (see page 732), Database (see page 732);
- specify Tables - the list of table names;

- specify TableType - the table type for free connection;
- specify TablePassword - the encrypted tables password for free connection;
- call Pack, Zap or other method.

Syntax

```
TADADSUtility = class(TADADSService);
```

Example

```
ADADSUtility1.DriverLink := ADPhysADSDriverLink1;
ADADSUtility1.Database := 'C:\DB;ServerTypes=Local';
ADADSUtility1.Tables.Add('EVENT.DBF');
ADADSUtility1.TableType := ttVFP;
ADADSUtility1.Pack;
```

1.17.6.1.5.1 public

1.17.6.1.5.1.1 TADADSUtility.Decrypt Method

Decrypts encrypted table and removes the password.

Description

Use Decrypt method to decrypt all rows of an encrypted table, disable encryption for the table and remove the password. The tables must not be used by any of the processes. The Decrypt method applicable for free connections only.

The Tables (see page 737) property specifies the list of the tables to process. TableType (see page 737) specifies type of the tables. The TablePassword (see page 736) property specifies the table password.

Syntax

```
procedure Decrypt;
```

See Also

Tables (see page 737), TableType (see page 737), TablePassword (see page 736), Encrypt (see page 734)

Example

```
ADADSUtility1.DriverLink := ADPhysADSDriverLink1;
ADADSUtility1.Database := 'C:\DB;ServerTypes=Local';
ADADSUtility1.Tables.Add('EVENT.DBF');
ADADSUtility1.TableType := ttVFP;
ADADSUtility1.TablePassword := '12345';
ADADSUtility1.Decrypt;
```

1.17.6.1.5.1.2 TADADSUtility.Encrypt Method

Encrypts a table and assigns a password.

Description

Use Encrypt method to encrypt all rows of a table, enable encryption for the table and assign a password. The tables must not be used by any of the processes. The Decrypt method applicable for free connections only.

The Tables (see page 737) property specifies the list of the tables to process. TableType (see page 737) specifies type of the tables. The TablePassword (see page 736) property specifies the table password.

Syntax

```
procedure Encrypt;
```

See Also

Tables (see page 737), TableType (see page 737), TablePassword (see page 736), Decrypt (see page 734)

Example

```
ADADSUtility1.DriverLink := ADPhysADSDriverLink1;
ADADSUtility1.Database := 'C:\DB;ServerTypes=Local';
ADADSUtility1.Tables.Add('ORDERS.DBF');
ADADSUtility1.TableType := ttVFP;
```

```
ADADSUtility1.TablePassword := '12345';
ADADSUtility1.Encrypt;
```

1.17.6.1.5.1.3 TADADSUtility.Pack Method

Packs a table.

Description

Use Pack method to remove deleted records from a table and re-index the table. The tables must not be used by any of the processes.

The Tables (see page 737) property specifies the list of tables to process. TableType (see page 737) specifies type of the tables. The TablePassword (see page 736) property optionally specifies the table password.

Syntax

```
procedure Pack;
```

See Also

Tables (see page 737), TableType (see page 737), TablePassword (see page 736), Recall (see page 735), Reindex (see page 735)

Example

```
ADADSUtility1.DriverLink := ADPhysADSDriverLink1;
ADADSUtility1.Database := 'C:\DB;ServerTypes=Local';
ADADSUtility1.Tables.Add('ORDERS.DBF');
ADADSUtility1.TableType := ttVFP;
ADADSUtility1.Pack;
```

1.17.6.1.5.1.4 TADADSUtility.Recall Method

Recalls all deleted records in a table.

Description

Use Recall method to recall all deleted records in a table, pack the table and re-index the table. The tables must not be used by any of the processes.

The Tables (see page 737) property specifies the list of tables to process. TableType (see page 737) specifies type of the tables. The TablePassword (see page 736) property optionally specifies the table password.

Syntax

```
procedure Recall;
```

See Also

Tables (see page 737), TableType (see page 737), TablePassword (see page 736), Pack (see page 735), Reindex (see page 735)

Example

```
ADADSUtility1.DriverLink := ADPhysADSDriverLink1;
ADADSUtility1.Database := '\\DA\ADS_DB';
ADADSUtility1.Tables.Add('ORDERS');
ADADSUtility1.Tables.Add('ORDER DETAILS');
ADADSUtility1.Recall;
```

1.17.6.1.5.1.5 TADADSUtility.Reindex Method

Rebuilds all open indexes associated with the given table.

Description

Use Reindex method to rebuild all open indexes associated with the given table. The tables must not be used by any of the processes.

The Tables (see page 737) property specifies the list of tables to process. TableType (see page 737) specifies type of the tables. The TablePassword (see page 736) property optionally specifies the table password.

Syntax

```
procedure Reindex;
```

See Also

Tables (see page 737), TableType (see page 737), TablePassword (see page 736), Pack (see page 735)

Example

```
ADADSUtility1.DriverLink := ADPhysADSDriverLink1;
ADADSUtility1.Database := '\\DA\ADS_DB';
ADADSUtility1.Tables.Add('ORDERS');
ADADSUtility1.Tables.Add('ORDER DETAILS');
ADADSUtility1.Reindex;
```

1.17.6.1.5.1.6 TADADSUtility.Zap Method

Removes all records from the table and re-indexes it.

Description

Use Zap method to remove all records from the table and re-index it. The tables must not be used by any of the processes.

The Tables (see page 737) property specifies the list of tables to process. TableType (see page 737) specifies type of the tables. The TablePassword (see page 736) property optionally specifies the table password.

Syntax

```
procedure Zap;
```

See Also

Tables (see page 737), TableType (see page 737), TablePassword (see page 736), Pack (see page 735)

Example

```
ADADSUtility1.DriverLink := ADPhysADSDriverLink1;
ADADSUtility1.Database := '\\DA\ADS_DB';
ADADSUtility1.Tables.Add('ORDERS');
ADADSUtility1.Tables.Add('ORDER DETAILS');
ADADSUtility1.Reindex;
```

1.17.6.1.5.2 published

1.17.6.1.5.2.1 TADADSUtility.OnProgress Event

Fires after a table is processed.

Description

The OnProgress event fires after each table specified in Tables (see page 737) is processed. Use the event handler to provide a feedback to the end users.

Syntax

```
property OnProgress: TADPhysServiceProgressEvent;
```

See Also

Tables (see page 737)

1.17.6.1.5.2.2 TADADSUtility.TablePassword Property

Specifies encryption password for free tables.

Description

Use the TablePassword property to specify a table password for tables in a free connection.

Syntax

```
property TablePassword: String;
```

See Also

Tables (see page 737)

1.17.6.1.5.2.3 TADADSUtility.Tables Property

The list of tables to process.

Description

Use the Tables property to specify a list of tables to process. When list is empty, then FireDAC will process all tables in a connection.

Syntax

```
property Tables: TStrings;
```

See Also

TableType (see page 737)

1.17.6.1.5.2.4 TADADSUtility.TableType Property

Specifies a table type.

Description

Use the TableType property to specify a table type for tables specified in Tables (see page 737) property. At first is useful for a free connection.

Syntax

```
property TableType: TADSTableType;
```

See Also

Tables (see page 737)

1.17.6.1.6 TADPhysADSDriverLink Class

Use the TADPhysADSDriverLink component to link the Advantage Database Server driver to an application and setup it.

published

published	Description
DateFormat (see page 738)	Specifies the default driver date format.
Decimals (see page 738)	Specifies the default number of digits after decimal point.
DefaultPath (see page 739)	Specifies the default path for the free tables.
Epoch (see page 739)	Specifies the default epoch for dates with two year digits.
Exact (see page 739)	Specifies the default string comparison mode.
SearchPath (see page 740)	Specifies the search path for the free tables.
ShowDeleted (see page 740)	Specifies the deleted records visibility mode.

Class Hierarchy



File

uADPhysADS

Description

Use the TADPhysADSDriverLink component to link the Advantage Database Server driver to an application. In general it is enough to only include uADPhysADS unit into your application uses clause.

The TADPhysADSDriverLink component may be used to specify:

- VendorHome (see page 782) - the Advantage installation root folder;
- VendorLib (see page 782) - the name and optional path to the ACE32.DLL / ACE64.DLL (Advantage Client Engine) client library.
- DateFormat (see page 738), Decimals (see page 738), Epoch (see page 739) - default Advantage data format settings;
- DefaultPath (see page 739), SearchPath (see page 740) - default Advantage table paths;
- Exact (see page 739), ShowDeleted (see page 740) - default Advantage operation modes.

All properties have default values appropriate for most case. To have an effect all property changes must be performed before first connection through this driver or use the Release (see page 780) method.

Syntax

```
TADPhysADSDriverLink = class(TADPhysDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to Advantage Database Server (see page 189)

Example

```
ADPhysADSDriverLink1.VendorLib := 'c:\ads\ace32.dll';
ADPhysADSDriverLink1.DefaultPath := 'c:\temp';
```

1.17.6.1.6.1 published

1.17.6.1.6.1.1 TADPhysADSDriverLink.DateFormat Property

Specifies the default driver date format.

Description

Use the DateFormat property to specify the default date format for FireDAC Advantage driver.

The format is used for dates represented as character strings. This allows control of date formatting for applications being used internationally. This setting affects how the Advantage interprets all date strings passed to the Advantage from the calling application. It also defines how the Advantage formats all dates passed back from the Advantage to the calling application. DateFormat can also be used to specify separators in the date format. For example, 'DD/MM/YYYY' and 'DD-MM-YYYY' are both valid date formats. The default value is "MM/DD/YYYY".

Syntax

```
property DateFormat: String;
```

See Also

Epoch (see page 739), Decimals (see page 738)

1.17.6.1.6.1.2 TADPhysADSDriverLink.Decimals Property

Specifies the default number of digits after decimal point.

Description

Use the Decimals property to specify the default number of digits after decimal point for FireDAC Advantage driver.

The property controls the resulting number of decimal places in a division, modulus, or exponentiation operation in an index or filter expression evaluated in the expression engine. The default value is 2.

Syntax

```
property Decimals: Word;
```

See Also

DateFormat (see page 738), Epoch (see page 739)

1.17.6.1.6.1.3 TADPhysADSDriverLink.DefaultPath Property

Specifies the default path for the free tables.

Description

Use the DefaultPath property to specify the default path for the opening and creating tables for FireDAC Advantage driver. If a path is given, then subsequent CREATE TABLE or Open calls will look first in the given path (if no path is provided with the table name). This setting affects open/creation of indexes in the same manner. DefaultPath does not result in a connection to the server, nor verifies the existence of the server of the given path.

The default path is used for path resolution when opening and creating tables. If an application creates or opens a table, and it does not supply a path with the table name, then the default path is used as the path for opening the table. If the table name has no path, no default path, or no search path (see SearchPath), then the current working directory of the application is used. Only a single path can be used.

The default value is the application current folder.

Syntax

```
property DefaultPath: String;
```

See Also

SearchPath (see page 740)

1.17.6.1.6.1.4 TADPhysADSDriverLink.Epoch Property

Specifies the default epoch for dates with two year digits.

Description

Use the Epoch to specify the default epoch for the textual dates with two year digits for FireDAC Advantage driver.

Epoch determines how date strings are interpreted that contain only two year digits. For example, if a date is given as "5/15/98", then the Epoch value will be used to determine the full four digit year. If a date is given as "5/15/1998", then the Epoch setting is not used. The Epoch setting will be interpreted as the 100-year range for the two digit year. For example, if the Epoch value is 1970, a two digit year of "96" will be interpreted as 1996, while a two digit year of "12" will be interpreted as 2012.

The default epoch value is 1900.

Syntax

```
property Epoch: Word;
```

See Also

DateFormat (see page 738), Decimals (see page 738)

1.17.6.1.6.1.5 TADPhysADSDriverLink.Exact Property

Specifies the default string comparison mode.

Description

Use the Exact to specify how string comparisons are performed with the relational operators =, >, <, >=, and <=.

The exact equal, ==, operator is not affected by this setting. In general, if the Exact setting is True, trailing spaces are ignored during the comparison. The Exact setting is not that simple, however. It is somewhat complex and affects the various relational operators differently. See Expression Engine Operators for detailed information on how the Exact setting affects relational operators.

Exact will affect string comparisons for all currently opened tables and all tables opened in the future for all connections. Changing the set exact setting via Exact also affects all string comparisons in index expressions.

The default value is False.

Syntax

```
property Exact: Boolean;
```

1.17.6.1.6.1.6 TADPhysADSDriverLink.SearchPath Property

Specifies the search path for the free tables.

Description

Use the SearchPath property to specify the search path for free tables for FireDAC Advantage driver.

If the opening table does not have a fully qualified path, and the table is not found in the default path (see DefaultPath), then each directory in the search path is searched for the table. For CREATE TABLE, the default path is used if supplied, otherwise the first search path will be used. For example, if there is no default path and the search path is "f:\data; g:\data", then CREATE TABLE will create tables in f:\data.

When using a search path, the existence check for the file is done on the client so that it does not have to attempt to open the table on each server in the search path. If there is no search path, then no existence check is made on the client and the table is assumed to exist in the default path.

When using the Advantage Internet Server, the use of a search path can cause the Advantage to connect to each server in the search path because a connection is required before the existence check can be made. For example, if the search path is "\\server1\vol1\data; \\server2\vol1\data", then the Advantage will connect to both server1 and server2 if a table exists on server2 and not on server1. If this is not desired, then an application should avoid using search paths and either use DefaultPath or supply a full path with each open or create request.

Syntax

```
property SearchPath: String;
```

See Also

DefaultPath (see page 739)

1.17.6.1.6.1.7 TADPhysADSDriverLink.ShowDeleted Property

Specifies the deleted records visibility mode.

Description

Use the ShowDeleted to specify the deleted records visibility mode for FireDAC Advantage driver.

In DBF tables, deleted records still physically exist inside tables so they can be returned to client applications. ShowDeleted controls whether the Advantage filters out records that are marked for deletion in DBF tables. If ShowDeleted is False, then the server will filter the deleted records, and the client will never "see" those records. This call also affects all currently connected servers and all server connections that are made after the call.

ShowDeleted has no effect upon Advantage proprietary ADT tables. Records that are deleted in ADT tables can never be retrieved by a client application.

The default value is True.

Syntax

```
property ShowDeleted: Boolean;
```

1.17.7 uADPhysASA Namespace

Contains Sybase SQL Anywhere driver and services components.

Classes

Class	Description
TADASABackup (see page 741)	The class implementing SQL Anywhere backup database service.

TADASAService (see page 744)	The base class for all Sybase SQL Anywhere service classes.
TADASAValidate (see page 745)	The class implementing SQL Anywhere database validate service.
TADPhysASADriverLink (see page 748)	Use the TADPhysASADriverLink component to link the Sybase SQL Anywhere driver to an application and setup it.

Description

The uADPhysASA unit contains:

- Sybase SQL Anywhere driver implementation;
- TADPhysASADriverLink (see page 748) - component to link the driver and setup CLI library.
- TADASABackup (see page 741) - backup DB service component;
- TADASAValidate (see page 745) - DB validation service component.

See Also

Configuring Drivers (see page 31), Connect to Sybase SQL Anywhere (see page 224)

1.17.7.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADASABackup (see page 741)	The class implementing SQL Anywhere backup database service.
TADASAService (see page 744)	The base class for all Sybase SQL Anywhere service classes.
TADASAValidate (see page 745)	The class implementing SQL Anywhere database validate service.
TADPhysASADriverLink (see page 748)	Use the TADPhysASADriverLink component to link the Sybase SQL Anywhere driver to an application and setup it.

1.17.7.1.1 TADASABackup Class

The class implementing SQL Anywhere backup database service.

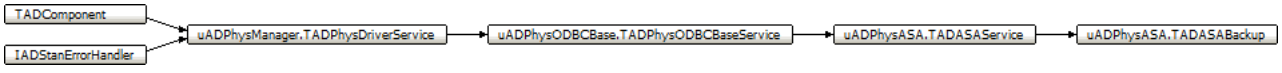
public

public	Description
Backup (see page 742)	Starts the backup task.

published

published	Description
CheckpointLogType (see page 742)	Control copying of checkpoint log.
ConnectParams (see page 743)	Specify connection parameters.
Flags (see page 743)	Specifies additional backup flags.
HotlogFilename (see page 743)	Specifies the file name for the live backup file.
OutputDir (see page 743)	Specifies the directory the backup files are copied to.
PageBlocksize (see page 744)	Specifies the maximum block size to be used to transfer pages from the database server to dbbackup.
StartLine (see page 744)	Specifies the start line.

Class Hierarchy



File

uADPhysASA

Description

Use the TADASABackup component to add a backup database capability to an application. This is a programmatic method to invoke the **dbbackup** tool as a thread in the DB server process.

To produce a DB backup an application should:

- specify DriverLink (see page 745);
- specify ConnectParams (see page 743);
- specify OutputDir (see page 743);
- call Backup (see page 742) method.

Other properties and methods are optional.

Using the TADASABackup component on a running database is equivalent to copying the database files when the database is not running. You can use the Backup utility to back up the database while other applications or users are using it.

To communicate with an user or application the SQL Anywhere DB tool is using OnProgress (see page 745) event. To ask for confirmation the event will be fired with AKind = tmConfirm. The OnProgress (see page 745) event also may be used to produce a DB backup log.

Syntax

```
TADASABackup = class(TADASAService);
```

See Also

TADASAService.OnProgress (see page 745)

Example

```
ADASABackup1.DriverLink := ADPhysASADriverLink1;
ADASABackup1.ConnectParams := 'ENG=addemo_asa11;DBN=addemo_asa11;UID=DBA;PWD=sql';
ADASABackup1.OutputDir := 'c:\temp\db';
ADASABackup1.Flags := [bfBackupDB, bfBackupLog];
ADASABackup1.OnProgress := ADASABackup1Progress;
ADASABackup1.Backup;
```

1.17.7.1.1.1 public

1.17.7.1.1.1.1 TADASABackup.Backup Method

Starts the backup task.

Description

Use the Backup method to start the backup task on the DB server host using DB tools.

To perform a backup task an application must specify the ConnectParams (see page 743) and OutputDir (see page 743). This is a programmatic method to invoke the **dbbackup** tool as a thread in the DB server process.

Syntax

```
procedure Backup;
```

See Also

ConnectParams (see page 743), StartLine (see page 744), OutputDir (see page 743)

1.17.7.1.1.2 published

1.17.7.1.1.2.1 TADASABackup.CheckpointLogType Property

Control copying of checkpoint log.

Description

Use the CheckpointLogType to control copying of checkpoint log. Corresponds to **-k checkpoint-log-copy-option**.

Syntax

```
property CheckpointLogType: TASABackupCheckpointLogType;
```

1.17.7.1.1.2.2 TADASABackup.ConnectParams Property

Specify connection parameters.

Description

Use the ConnectParams to specify connection parameters. The user ID must have DBA authority or REMOTE DBA authority to connect to the database. Corresponds to **-c "keyword=value; ..."**.

Syntax

```
property ConnectParams: String;
```

See Also

StartLine (see page 744)

1.17.7.1.1.2.3 TADASABackup.Flags Property

Specifies additional backup flags.

Description

Use the Flags property to specify additional backup flags.

Flag	Description
bfBackupDB	Back up the database file or not. Corresponds to **-d**.
bfBackupLog	Back up the transaction log file or not. Corresponds to **-t**.
bfNoConfirm	Operate with or without confirmation. Corresponds to **-y**.
bfQuiet	Operate without printing messages, or print messages. Corresponds to **-q**.
bfRenameLog	Rename the transaction log. Corresponds to **-r**.
bfTruncateLog	Delete the transaction log. Corresponds to **-x**.
bfRenameLocalLog	Rename the local backup of the transaction log. Corresponds to **-n**.
bfServerBackup	Rename the local backup of the transaction log. Corresponds to **-n**.

Syntax

```
property Flags: TASABackupFlags;
```

See Also

HotlogFilename (see page 743)

1.17.7.1.1.2.4 TADASABackup.HotlogFilename Property

Specifies the file name for the live backup file.

Description

Use the HotlogFilename to specify the file name for the live backup file. Corresponds to **-l filename**.

Syntax

```
property HotlogFilename: String;
```

See Also

Flags (see page 743)

1.17.7.1.1.2.5 TADASABackup.OutputDir Property

Specifies the directory the backup files are copied to.

Description

Use the OutputDir to specify the directory the backup files are copied to. If the directory does not exist, it is created. However, the parent directory must exist. Corresponds to **target-directory**.

Syntax

```
property OutputDir: String;
```

1.17.7.1.1.2.6 TADASABackup.PageBlocksize Property

Specifies the maximum block size to be used to transfer pages from the database server to dbbackup.

Description

Use the PageBlockSize property to specify the maximum block size (in number of pages) to be used to transfer pages from the database server to dbbackup. Corresponds to **-b block-size**.

Syntax

```
property PageBlocksize: Longword;
```

See Also

Flags (see page 743)

1.17.7.1.1.2.7 TADASABackup.StartLine Property

Specifies the start line.

Description

Use the StartLine to specify SQL Anywhere start line. May be used for SQL Anywhere v 9 or less only.

Syntax

```
property StartLine: String;
```

See Also

ConnectParams (see page 743)

1.17.7.1.2 TADASAService Class

The base class for all Sybase SQL Anywhere service classes.

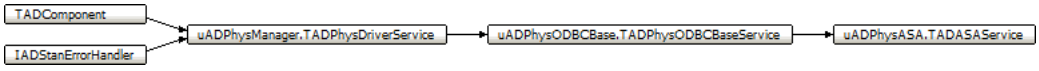
public

public	Description
ToolLib (see page 745)	Reference to the tool library.

published

published	Description
DriverLink (see page 745)	Specifies the DriverLink.
OnProgress (see page 745)	Specifies the event handler allowing to communicate with the DB server.

Class Hierarchy



File

uADPhysASA

Description

The TADASAService class is used as a base class for all Sybase SQL Anywhere service classes and should not be used directly. The class contains the common properties, like the DriverLink (see page 745), OnProgress (see page 745).

Syntax

```
TADASAService = class(TADPhysODBCBaseService);
```

1.17.7.1.2.1 public

1.17.7.1.2.1.1 TADASAService.ToolLib Property

Reference to the tool library.

Description

Returns a reference to the tool library object.

Syntax

```
property ToolLib: TASAToolLib;
```

1.17.7.1.2.2 published

1.17.7.1.2.2.1 TADASAService.DriverLink Property

Specifies the DriverLink.

Description

Use the DriverLink property to specify the driver link. This property must be specified for any service operation.

Syntax

```
property DriverLink: TADPhysASADriverLink;
```

1.17.7.1.2.2.2 TADASAService.OnProgress Property

Specifies the event handler allowing to communicate with the DB server.

Description

Use the OnProgress event handler to communicate with the DB server. The event handler receives arguments:

- AMessage - message from a DB services;
- AKind - message kind;
- AResult - respond from a handler.

Syntax

```
property OnProgress: TADASAProgressEvent;
```

Example

```
procedure TForm1.ADASABackup1Progress(ASender: TObject; AMessage: string;
  AKind: TASAToolMessageKind; var AResult: Integer);
begin
  if Memo1 <> nil then
    Memo1.Lines.Add(AMessage);
  if AKind = tmConfirm then
    if MessageDlg(AMessage, mtConfirmation, mbYesNo, -1) = mrYes then
      AResult := 1
    else
      AResult := 0;
end;
```

1.17.7.1.3 TADASAValidate Class

The class implementing SQL Anywhere database validate service.

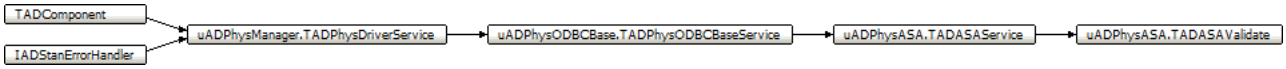
public

public	Description
Validate (see page 746)	Starts the validation task.

published

published	Description
ConnectParams (see page 747)	Specify connection parameters.
Flags (see page 747)	Specifies additional validate flags.
StartLine (see page 747)	Specifies the start line.
Tables (see page 747)	Specifies the table or materialized view list to validate.
ValidateType (see page 748)	Specifies the database validation mode.

Class Hierarchy



File

uADPhysASA

Description

Use the TADASAValidate component to add a database validate capability to an application. This is a programmatic method to invoke the **dbvalid** tool as a thread in the DB server process.

To validate and or repair a database an application should:

- specify DriverLink (see page 745);
- specify ConnectParams (see page 747);
- call Validate (see page 746) method.

To produce a DB validate log an application should use OnProgress (see page 745) event.

Syntax

```
TADASAValidate = class(TADASAService);
```

See Also

TADASAService.OnProgress (see page 745)

Example

```
ADASAValidate1.DriverLink := ADPhysASADriverLink1;
ADASAValidate1.ConnectParams := 'ENG=addemo_asa11;DBN=addemo_asa11;UID=DBA;PWD=sql';
ADASAValidate1.ValidateType := vtNormal;
ADASAValidate1.Flags := [];
ADASAValidate1.OnProgress := ADASABackup1Progress;
ADASAValidate1.Validate;
```

1.17.7.1.3.1 public

1.17.7.1.3.1.1 TADASAValidate.Validate Method

Starts the validation task.

Description

Use the Validate method to start the validation task on the DB server host using DB tools.

To perform a validation task an application must specify the ConnectParams (see page 747). This is a programmatic method to invoke the **dbvalid** tool as a thread in the DB server process.

Syntax

```
procedure Validate;
```

See Also

ConnectParams (see page 747), StartLine (see page 747)

1.17.7.1.3.2 published

1.17.7.1.3.2.1 TADASAValidate.ConnectParams Property

Specify connection parameters.

Description

Use the ConnectParams to specify connection parameters. The user ID must have DBA authority or VALIDATE authority. Corresponds to **-c "keyword=value; ..."**.

Syntax

```
property ConnectParams: String;
```

See Also

StartLine (see page 747)

1.17.7.1.3.2.2 TADASAValidate.Flags Property

Specifies additional validate flags.

Description

Use the Flags property to specify additional validate flags.

Flag	Description
vfQuiet	Operate without printing messages, or print messages. Corresponds to **-q**.
vfIndex	Validate indexes. Corresponds to **-i** when included, to **-t** when excluded.

Syntax

```
property Flags: TASAValidateFlags;
```

See Also

ValidateType (see page 748), Tables (see page 747)

1.17.7.1.3.2.3 TADASAValidate.StartLine Property

Specifies the start line.

Description

Use the StartLine to specify SQL Anywhere start line. May be used for SQL Anywhere v 9 or less only.

Syntax

```
property StartLine: String;
```

See Also

ConnectParams (see page 747)

1.17.7.1.3.2.4 TADASAValidate.Tables Property

Specifies the table or materialized view list to validate.

Description

Use the Tables property to specify the table or materialized view list to validate. When vfIndex is included into Flags, then Tables specifies index names to validate. Corresponds to **object-name**.

Syntax

```
property Tables: TStrings;
```

See Also

Flags (see page 747)

1.17.7.1.3.2.5 TADASAValidate.ValidateType Property

Specifies the database validation mode.

Description

Use the ValidateType to specifiy the database validation mode. Corresponds to **-d**, **-fx**, **-s**.

Syntax

```
property ValidateType: TASAValidateType;
```

See Also

Flags (see page 747), Tables (see page 747)

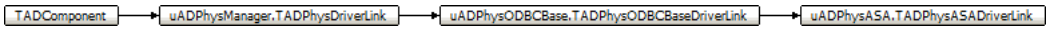
1.17.7.1.4 TADPhysASADriverLink Class

Use the TADPhysASADriverLink component to link the Sybase SQL Anywhere driver to an application and setup it.

published

published	Description
ToolLib (see page 749)	Specifies the path to the DBTOOLn.DLL.

Class Hierarchy



File

uADPhysASA

Description

Use the TADPhysASADriverLink component to link the Sybase SQL Anywhere driver to an application. In general it is enough to only include uADPhysASA (see page 740) unit into your application uses clause.

The TADPhysASADriverLink component may be used to specify:

- ODBCDriver (see page 799) - Sybase SQL Anywhere ODBC driver name;
- ODBCAdvanced (see page 799) - Sybase SQL Anywhere ODBC driver connection parameter common for all connections;
- VendorLib (see page 782) - ODBC driver manager. By default it is odbc32.dll and for most cases should remain unchanged.

All properties have default values appropriate for most case. To have an effect all property changes must be performed when FireDAC manager (see page 365) is inactive.

Syntax

```
TADPhysASADriverLink = class(TADPhysODBCBaseDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to Sybase SQL Anywhere (see page 224)

Example

```
ADPhysASADriverLink1.ODBCAdvanced := 'AutoStart=Yes;CharSet=Windows-1251';
ADPhysASADriverLink1.ODBCDriver := 'Sybase SQL Anywhere 5.0';
```

1.17.7.1.4.1 published

1.17.7.1.4.1.1 TADPhysASADriverLink.ToolLib Property

Specifies the path to the DBTOOLn.DLL.

Description

Use the ToolLib property to specify the path and name for the DBTOOLn.DLL library. This library is used by all SQL Anywhere service classes.

Syntax

```
property ToolLib: String;
```

1.17.8 uADPhysDataSnap Namespace

Contains Multi-Tier DataSnap driver for RAD Studio XE2 Enterprise and higher.

Classes

Class	Description
TADPhysDataSnapDriverLink (see page 749)	Use the TADPhysDataSnapDriverLink component to link the Multi-Tier DataSnap driver to an application.

Description

The uADPhysDataSnap unit contains:

- Multi-Tier DataSnap driver implementation;
- TADPhysDataSnapDriverLink (see page 749) - component to link the driver.

See Also

Configuring Drivers (see page 31), Connect to DataSnap server (see page 193)

1.17.8.1 Classes

The following table lists classes in this documentation.

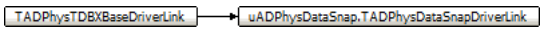
Classes

Class	Description
TADPhysDataSnapDriverLink (see page 749)	Use the TADPhysDataSnapDriverLink component to link the Multi-Tier DataSnap driver to an application.

1.17.8.1.1 TADPhysDataSnapDriverLink Class

Use the TADPhysDataSnapDriverLink component to link the Multi-Tier DataSnap driver to an application.

Class Hierarchy



File

uADPhysDataSnap

Description

Use the TADPhysDataSnapDriverLink component to link the Multi-Tier DataSnap driver to an application. In general it is enough to only include uADPhysDataSnap unit into your application uses clause.

TADPhysDataSnapDriverLink does not use VendorLib (see page 782) and VendorHome (see page 782) properties in the current FireDAC version.

DataSnap driver is supported only on RAD Studio XE2 and higher.

Syntax

```
TADPhysDataSnapDriverLink = class(TADPhysTDBXBaseDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to DataSnap server (see page 193)

1.17.9 uADPhysDB2 Namespace

Contains IBM DB2 driver and services components.

Classes

Class	Description
EDB2NativeException (see page 750)	Class representing a IBM DB2 driver exception.
TADPhysDB2DriverLink (see page 751)	Use the TADPhysDB2DriverLink component to link the IBM DB2 driver to an application and setup it.

Description

The uADPhysDB2 unit contains:

- IBM DB2 driver implementation;
- TADPhysDB2DriverLink (see page 751) - component to link the driver and setup CLI library.

See Also

Configuring Drivers (see page 31), Connect to IBM DB2 Server (see page 196)

1.17.9.1 Classes

The following table lists classes in this documentation.

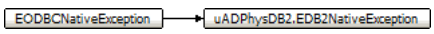
Classes

Class	Description
EDB2NativeException (see page 750)	Class representing a IBM DB2 driver exception.
TADPhysDB2DriverLink (see page 751)	Use the TADPhysDB2DriverLink component to link the IBM DB2 driver to an application and setup it.

1.17.9.1.1 EDB2NativeException Class

Class representing a IBM DB2 driver exception.

Class Hierarchy



File

uADPhysDB2

Description

The FireDAC DB2 driver raises exceptions of EDB2NativeException class. Use this class to handle driver specific errors.

Syntax

```
EDB2NativeException = class(EODBCNativeException);
```

See Also

EODBCNativeException, TADPhysDB2DriverLink (see page 751), Connect to IBM DB2 Server (see page 196)

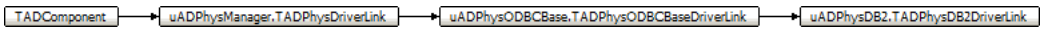
Example

```
try
  ADConnection1.Connected := True;
except
  on E: EDB2NativeException do
    // perform DB2 error specific actions
end;
```

1.17.9.1.2 TADPhysDB2DriverLink Class

Use the TADPhysDB2DriverLink component to link the IBM DB2 driver to an application and setup it.

Class Hierarchy



File

uADPhysDB2

Description

Use the TADPhysDB2DriverLink component to link the IBM DB2 driver to an application. In general it is enough to only include uADPhysDB2 unit into your application uses clause.

The TADPhysDB2DriverLink component may be used to specify:

- ODBCDriver (see page 799) - DB2 ODBC driver name;
- ODBCAdvanced (see page 799) - DB2 ODBC driver connection parameter common for all connections;
- VendorLib (see page 782) - ODBC driver manager. By default it is odbc32.dll and for most cases should remain unchanged.

All properties have default values appropriate for most case. To have an effect all property changes must be performed before first connection through this driver or use the Release (see page 780) method.

Syntax

```
TADPhysDB2DriverLink = class(TADPhysODBCBaseDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to IBM DB2 Server (see page 196)

Example

```
ADPhysDB2DriverLink1.ODBCAdvanced := 'IGNOREWARNINGS=1';
ADPhysDB2DriverLink1.ODBCDriver := 'IBM DB2 ODBC DRIVER - DB2COPY1';
```

1.17.10 uADPhysDBExp Namespace

Contains dbExpress v 1-3 bridge driver.

Classes

Class	Description
TADPhysDBXDriverLink (see page 752)	Use the TADPhysDBXDriverLink component to link the dbExpress v 1-3 bridge driver to an application.

Description

The uADPhysDBExp unit contains:

- dbExpress v 1-3 bridge driver implementation;

- TADPhysDBXDriverLink (see page 752) - component to link the driver.

See Also

Configuring Drivers (see page 31), Connect to dbExpress data source (see page 194)

1.17.10.1 Classes

The following table lists classes in this documentation.

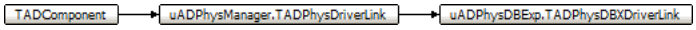
Classes

Class	Description
TADPhysDBXDriverLink (see page 752)	Use the TADPhysDBXDriverLink component to link the dbExpress v 1-3 bridge driver to an application.

1.17.10.1.1 TADPhysDBXDriverLink Class

Use the TADPhysDBXDriverLink component to link the dbExpress v 1-3 bridge driver to an application.

Class Hierarchy



File

uADPhysDBExp

Description

Use the TADPhysDBXDriverLink component to link the dbExpress v 1-3 bridge driver to an application. In general it is enough to only include uADPhysDBExp unit into your application uses clause.

TADPhysDBXDriverLink does not use VendorLib and VendorHome properties in current FireDAC version.

Syntax

```
TADPhysDBXDriverLink = class(TADPhysDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to dbExpress data source (see page 194)

1.17.11 uADPhysIB Namespace

Contains Firebird and Interbase driver and services components.

Classes

Class	Description
TADIBBackup (see page 753)	The class implementing FB/IB backup database service.
TADIBNBackup (see page 756)	The class implementing Firebird-only backup database service.
TADIBNRestore (see page 759)	The class implementing Firebird-only restored database service.
TADIBRestore (see page 761)	The class implementing FB/IB service restoring a database from a backup.
TADIBSecurity (see page 764)	The class implementing FB/IB database security management service.
TADIBService (see page 769)	The base class for all FB/IB service classes.
TADIBTrace (see page 771)	The class implementing Firebird database trace service.
TADIBValidate (see page 774)	The class implementing FB/IB database validate and repair service.

TADPhysIBDriverLink (see page 777)	Use the TADPhysIBDriverLink component to link the FB/IB driver to an application and setup it.

Description

The uADPhysIB unit contains:

- Firebird and Interbase driver implementation;
- TADPhysIBDriverLink (see page 777) - component to link the driver and setup CLI library;
- TADIBBackup (see page 753) - backup DB service component;
- TADIBNBackup (see page 756) - backup DB service component, using NBackup;
- TADIBRestore (see page 761) - restore DB service component;
- TADIBNRestore (see page 759) - restore DB service component, using NBackup;
- TADIBValidate (see page 774) - DB validation and repair service component;
- TADIBTrace (see page 771) - Firebird trace service component;
- TADIBSecurity (see page 764) - security service component.

See Also

Configuring Drivers (see page 31), Connect to Interbase or Firebird (see page 198)

1.17.11.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADIBBackup (see page 753)	The class implementing FB/IB backup database service.
TADIBNBackup (see page 756)	The class implementing Firebird-only backup database service.
TADIBNRestore (see page 759)	The class implementing Firebird-only restored database service.
TADIBRestore (see page 761)	The class implementing FB/IB service restoring a database from a backup.
TADIBSecurity (see page 764)	The class implementing FB/IB database security management service.
TADIBService (see page 769)	The base class for all FB/IB service classes.
TADIBTrace (see page 771)	The class implementing Firebird database trace service.
TADIBValidate (see page 774)	The class implementing FB/IB database validate and repair service.
TADPhysIBDriverLink (see page 777)	Use the TADPhysIBDriverLink component to link the FB/IB driver to an application and setup it.

1.17.11.1.1 TADIBBackup Class

The class implementing FB/IB backup database service.

public

public	Description
Backup (see page 754)	Starts the backup task.

published

published	Description
BackupFiles (see page 755)	Specifies the backup files.
Database (see page 755)	Specifies a database file to backup.
Options (see page 755)	Specifies backup options.
Verbose (see page 756)	Controls the DB Service Manager feedback.

Class Hierarchy



File

uADPhysIB

Description

Use the TADIBBackup component to add a backup FB/IB database capability to an application. This is a programmatic method to invoke the **gbak** tool as a thread in the DB server process.

To produce a DB backup an application should:

- specify DriverLink (see page 769), UserName (see page 771), Password (see page 770), Host (see page 769);
- specify Database (see page 755) - a primary database file to backup;
- specify BackupFiles (see page 755) - the file names to backup a database into;
- call Backup (see page 754) method.

Other properties and methods are optional.

Paths of backup files are relative to the server. Because the TADIBBackup executes backup on the DB server host, the DB Service Manager writes backup files on the server host. The DB Services Manager also creates files in the context of the server.

To produce a DB backup log an application should set Verbose (see page 756) to True and use OnProgress (see page 770) event.

The TADIBNBackup (see page 756) component may be used for an incremental backup. To restore an backup performed by TADIBBackup use the TADIBRestore (see page 761) component.

Syntax

```
TADIBBackup = class(TADIBService);
```

See Also

TADIBNBackup (see page 756), TADIBRestore (see page 761)

Example

```
ADIBBackup1.DriverLink := ADPhysIBDriverLink1;

ADIBBackup1.UserName := 'sysdba';
ADIBBackup1.Password := 'masterkey';
ADIBBackup1.Host := 'db_srv_host';
ADIBBackup1.Protocol := ipTCPIP;

ADIBBackup1.Database := 'e:\ib\addemo.fdb';
ADIBBackup1.BackupFiles.Add('e:\ib\addemo.backup');

ADIBBackup1.Backup;
```

1.17.11.1.1.1 public

1.17.11.1.1.1.1 TADIBBackup.Backup Method

Starts the backup task.

Description

Use the Backup method to start the backup task on the DB server host using Service Manager.

To perform a backup task an application must specify the connection parameters (see page 769), Database (see page 755) and BackupFiles (see page 755). This is a programmatic method to invoke the **gbak** tool as a thread in the DB server process.

Syntax

```
procedure Backup;
```

See Also

TADIBService.Host (see page 769), TADIBService.UserName (see page 771), TADIBService.Password (see page 770), Database (see page 755), BackupFiles (see page 755)

1.17.11.1.1.2 published

1.17.11.1.1.2.1 TADIBBackup.BackupFiles Property

Specifies the backup files.

Description

Use the BackupFiles property to specify one or more backup output files.

The paths are relative to the DB server host. When few files are specified, that corresponds to **gsplit** functionality. Each file must be specified on a separated line.

Optionally BackupFiles allows to specify length in bytes of the backup output file. You may specify one length value for each output file except the last. The length may be specified after a backup file name, separated by '='.

Syntax

```
property BackupFiles: TStrings;
```

See Also

Database (see page 755), TADIBRestore (see page 761)

Example 1

Specifying multiple files with length:

```
with ADIBBackup1.BackupFiles do begin
  Clear;
  Add('/tmp/addemo.backup1=10000000');
  Add('/tmp/addemo.backup2=10000000');
  Add('/tmp/addemo.backup3');
end;
```

Example 2

Specifying single file:

```
ADIBBackup1.BackupFiles.Text := 'e:\temp\addemo.backup';
```

1.17.11.1.1.2.2 TADIBBackup.Database Property

Specifies a database file to backup.

Description

Use the Database property to specify a path of the primary file of the database to backup. The path is relative to a DB server.

Syntax

```
property Database: String;
```

See Also

BackupFiles (see page 755)

1.17.11.1.1.2.3 TADIBBackup.Options Property

Specifies backup options.

Description

Use the Options property to specify additional options controlling the backup task. The default value is empty set.

Option	Description
boIgnoreChecksum	Ignore checksums during backup. Corresponds to **gbak -ignore**.
boIgnoreLimbo	Ignore limbo transactions during backup. Corresponds to **gbak -limbo**.
boMetadataOnly	Output backup file for metadata only with empty tables. Corresponds to **gbak -metadata**.
boNoGarbageCollect	Suppress normal garbage collection during backup. Improves performance on some databases. Corresponds to **gbak -garbage_collect**.
boOldDescriptions	Output metadata in pre-4.0 format. Corresponds to **gbak -old_descriptions**.
boNonTransportable	Output backup file format with non-XDR data format. Improves space and performance by a negligible amount. Corresponds to **gbak -nt**.
boConvert	Convert external table data to internal tables. Corresponds to **gbak -convert**.
boExpand	Undocumented and unimplemented in Firebird.

Syntax

```
property Options: TIBBackupOptions;
```

1.17.11.1.1.2.4 TADIBBackup.Verbose Property

Controls the DB Service Manager feedback.

Description

Use the Verbose property to control the DB Service Manager output. When is True, the TADIBBackup will fire OnProgress (see page 770) event and submit a backup task status. The default value is False. Corresponds to **gbak -verbose**.

Syntax

```
property Verbose: Boolean;
```

See Also

TADIBService.OnProgress (see page 770)

1.17.11.1.2 TADIBNBackup Class

The class implementing Firebird-only backup database service.

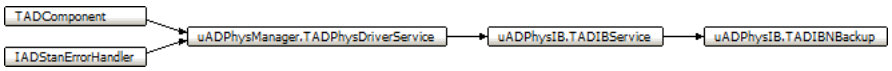
public

public	Description
Backup (see page 757)	Starts the backup task.

published

published	Description
BackupFile (see page 758)	Specifies a backup file.
Database (see page 758)	Specifies a database file to backup.
Level (see page 758)	Specifies the backup level.
Options (see page 758)	Specifies backup options.

Class Hierarchy



File

uADPhysIB

Description

Use the TADIBNBackup component to add a Firebird-only backup database capability to an application. This is a

programmatic method to invoke the **nbackup** tool as a thread in the DB server process to backup a database.

To produce a DB backup an application should:

- specify DriverLink (see page 769), UserName (see page 771), Password (see page 770), Host (see page 769);
- specify Database (see page 755) - a primary database file to backup;
- specify BackupFiles (see page 755) - the file name to backup a database into;
- specify Level (see page 758) - the incremental level or full backup mode;
- call Backup (see page 757) method.

Other properties and methods are optional.

Paths of backup files are relative to the server. Because the TADIBNBackup executes backup on the DB server host, the DB Service Manager writes backup files on the server host. The DB Services Manager also creates files in the context of the server.

To restore an backup performed by TADIBNBackup use the TADIBNRestore (see page 759) component.

Read http://www.firebirdsql.org/manual/nbackup.html for more details.

Syntax

```
TADIBNBackup = class(TADIBService);
```

See Also

TADIBBackup (see page 753), TADIBNRestore (see page 759)

Example

```
ADIBNBackup1.DriverLink := ADPhysIBDriverLink1;

ADIBNBackup1.UserName := 'sysdba';
ADIBNBackup1.Password := 'masterkey';
ADIBNBackup1.Host := 'db_srv_host';
ADIBNBackup1.Protocol := ipTCPIP;

ADIBNBackup1.Database := 'e:\ib\addemo.fdb';
ADIBNBackup1.BackupFile := 'e:\ib\addemo.backup';
ADIBNBackup1.Level := 0; // full backup

ADIBNBackup1.Backup;
```

1.17.11.1.2.1 public

1.17.11.1.2.1.1 TADIBNBackup.Backup Method

Starts the backup task.

Description

Use the Backup method to start the backup task on the DB server host using Service Manager.

To perform a backup task an application must specify the connection parameters (see page 769), Database (see page 758) and BackupFile (see page 758). This is a programmatic method to invoke the **nbackup** tool as a thread in the DB server process.

Syntax

```
procedure Backup;
```

See Also

TADIBService.Host (see page 769), TADIBService.UserName (see page 771), TADIBService.Password (see page 770), Database (see page 758), BackupFile (see page 758)

1.17.11.1.2.2 published

1.17.11.1.2.2.1 TADIBNBackup.BackupFile Property

Specifies a backup file.

Description

Use the BackupFiles property to specify a backup output file. The path is relative to the DB server host.

Syntax

```
property BackupFile: String;
```

See Also

Database (see page 758), TADIBNRestore (see page 759)

Example

```
ADIBNBackup1.BackupFile := 'e:\temp\addemo.backup';
```

1.17.11.1.2.2.2 TADIBNBackup.Database Property

Specifies a database file to backup.

Description

Use the Database property to specify a path of the primary file of the database to backup. The path is relative to a DB server.

Syntax

```
property Database: String;
```

See Also

BackupFile (see page 758)

1.17.11.1.2.2.3 TADIBNBackup.Level Property

Specifies the backup level.

Description

Use the Level property to specify the backup task level.

The 0 means the full backup.

To make an incremental (“differential”) backup specify a backup level greater than 0. An incremental backup of level N always contains the database mutations since the most recent level N-1 backup.

Syntax

```
property Level: Integer;
```

See Also

BackupFile (see page 758), Database (see page 758)

1.17.11.1.2.2.4 TADIBNBackup.Options Property

Specifies backup options.

Description

Use the Options property to specify additional options controlling the backup task. The default value is empty set.

Option	Description
noNoTriggers	Firebird 2.1 introduced the concept of database triggers. To keep these database triggers from firing during an nbackup run, use noNoTriggers option. Corresponds to **nbackup -T** or **gbak -nodbtriggers**.

Syntax

```
property Options: TIBNBackupOptions;
```

1.17.11.1.3 TADIBNRestore Class

The class implementing Firebird-only restored database service.

public

public	Description
Restore (see page 760)	Starts the restore task.

published

published	Description
BackupFiles (see page 760)	Specifies the backup files.
Database (see page 760)	Specifies a database file to restore.
Options (see page 760)	Specifies restore options.

Class Hierarchy



File

uADPhysIB

Description

Use the TADIBNRestore component to add a Firebird-only a restoring database from backup capability to an application. This is a programmatic method to invoke the **nbackup** tool as a thread in the DB server process to restore a database.

To restore a database from a DB backup an application should:

- specify DriverLink (see page 769), UserName (see page 771), Password (see page 770), Host (see page 769);
- specify Database (see page 760) - a primary database file to restore;
- specify BackupFiles (see page 760) - the file names to restored a database from;
- call Restore (see page 760) method.

Other properties and methods are optional.

Paths of backup files are relative to the server. Because the TADIBNBackup restores backup on the DB server host, the DB Service Manager reads backup files on the server host.

To produce a DB restore log an application should set Verbose to True and use OnProgress (see page 770) event.

Read http://www.firebirdsql.org/manual/nbackup.html for more details.

Syntax

```
TADIBNRestore = class(TADIBService);
```

See Also

TADIBNBackup (see page 756), TADIBBackup (see page 753)

Example

```
ADIBNRestore1.DriverLink := ADPhysIBDriverLink1;

ADIBNRestore1.UserName := 'sysdba';
ADIBNRestore1.Password := 'masterkey';
ADIBNRestore1.Host := 'db_srv_host';
ADIBNRestore1.Protocol := ipTCPIP;

ADIBNRestore1.Database := 'e:\ib\addemo.fdb';
ADIBNRestore1.BackupFile := 'e:\ib\addemo.backup';

ADIBNRestore1.Restore;
```

1.17.11.1.3.1 public

1.17.11.1.3.1.1 TADIBNRestore.Restore Method

Starts the restore task.

Description

Use the Restore method to start the restore task on the DB server host using Service Manager.

To perform a restore task an application must specify the connection parameters (see page 769), Database (see page 760) and BackupFiles (see page 760). This is a programmatic method to invoke the **nbackup** tool as a thread in the DB server process.

Syntax

```
procedure Restore;
```

See Also

TADIBService.Host (see page 769), TADIBService.UserName (see page 771), TADIBService.Password (see page 770), Database (see page 760), BackupFiles (see page 760)

1.17.11.1.3.2 published

1.17.11.1.3.2.1 TADIBNRestore.BackupFiles Property

Specifies the backup files.

Description

Use the BackupFiles property to specify one or more backup input files.

The paths are relative to the DB server host. When few files are specified, each file must be specified on a separate line.

Syntax

```
property BackupFiles: TStrings;
```

See Also

List here...

Example

1.17.11.1.3.2.2 TADIBNRestore.Database Property

Specifies a database file to restore.

Description

Use the Database property to specify a path of the primary file of the database to restore. The path is relative to a DB server.

Syntax

```
property Database: String;
```

See Also

BackupFiles (see page 760)

1.17.11.1.3.2.3 TADIBNRestore.Options Property

Specifies restore options.

Description

Use the Options property to specify additional options controlling the restore task. The default value is empty set.

Option	Description
noNoTriggers	Firebird 2.1 introduced the concept of database triggers. To keep these database triggers from firing during an nbackup run, use noNoTriggers option. Corresponds to **nbackup -T** or **gbak -nodbtriggers**.

Syntax

```
property Options: TIBNBackupOptions;
```

1.17.11.1.4 TADIBRestore Class

The class implementing FB/IB service restoring a database from a backup.

public

public	Description
Restore (see page 762)	Starts the restore task.

published

published	Description
BackupFiles (see page 762)	Specifies the backup files.
Database (see page 762)	Specifies a database file to restore.
Options (see page 763)	Specifies restore options.
PageBuffers (see page 763)	Specifies the number of default cache buffer for a restored database.
PageSize (see page 763)	Specifies the page size for a restored database.
Verbose (see page 764)	Controls the DB Service Manager feedback.

Class Hierarchy



File

uADPhysIB

Description

Use the TADIBRestore component to add a restoring database from backup capability to an application. This is a programmatic method to invoke the **gbak** tool as a thread in the DB server process.

To restore a database from a DB backup an application should:

- specify DriverLink (see page 769), UserName (see page 771), Password (see page 770), Host (see page 769);
- specify Database (see page 762) - a primary database file to restore;
- specify BackupFiles (see page 762) - the file names to restore a database from;
- call Restore (see page 762) method.

Other properties and methods are optional.

Paths of backup files are relative to the server. Because the TADIBRestore restores backup on the DB server host, the DB Service Manager reads backup files on the server host.

To produce a DB restore log an application should set Verbose (see page 764) to True and use OnProgress (see page 770) event.

The TADIBNRestore (see page 759) component should be used to restored an incremental backup.

Syntax

```
TADIBRestore = class(TADIBService);
```

See Also

TADIBNRestore (see page 759), TADIBBackup (see page 753)

Example

```
ADIBRestore1.DriverLink := ADPhysIBDriverLink1;

ADIBRestore1.UserName := 'sysdba';
ADIBRestore1.Password := 'masterkey';
ADIBRestore1.Host := 'db_srv_host';
ADIBRestore1.Protocol := ipTCPIP;

ADIBRestore1.Database := 'e:\ib\addemo.fdb';
ADIBRestore1.BackupFiles.Add('e:\ib\addemo.backup');

ADIBRestore1.Restore;
```

1.17.11.1.4.1 public

1.17.11.1.4.1.1 TADIBRestore.Restore Method

Starts the restore task.

Description

Use the Restore method to start the restore task on the DB server host using Service Manager.

To perform a restore task an application must specify the connection parameters (see page 769), Database (see page 762) and BackupFiles (see page 762). This is a programmatic method to invoke the **gbak** tool as a thread in the DB server process.

Syntax

```
procedure Restore;
```

See Also

TADIBService.Host (see page 769), TADIBService.UserName (see page 771), TADIBService.Password (see page 770), Database (see page 762), BackupFiles (see page 762)

1.17.11.1.4.2 published

1.17.11.1.4.2.1 TADIBRestore.BackupFiles Property

Specifies the backup files.

Description

Use the BackupFiles property to specify one or more backup input files.

The paths are relative to the DB server host. When few files are specified, each file must be specified on a separate line.

Syntax

```
property BackupFiles: TStrings;
```

See Also

Database (see page 762)

1.17.11.1.4.2.2 TADIBRestore.Database Property

Specifies a database file to restore.

Description

Use the Database property to specify a path of the primary file of the database to restore. The path is relative to a DB server.

Syntax

```
property Database: String;
```

See Also

BackupFiles (see page 762)

1.17.11.1.4.2.3 TADIBRestore.Options Property

Specifies restore options.

Description

Use the Options property to specify additional options controlling the restore task. The default value is empty set.

Option	Description
roDeactivateIdx	Do not build user indexes during restore. Corresponds to **gbak -inactive**.
roNoShadow	Do not recreate shadow files during restore. Corresponds to **gbak -kill**.
roNoValidity	Do not enforce validity conditions (for example, NOT NULL) during restore. Corresponds to **gbak -no_validity**.
roOneAtATime	Commit after completing restore of each table. Corresponds to **gbak -one_at_a_time**.
roReplace	Replace database, if one exists. Corresponds to **gbak -replace**. For a restore you must supply either roReplace or roCreate.
roCreate	Restore but do not overwrite an existing database. Corresponds to **gbak -create**. For a restore you must supply either roReplace or roCreate.
roUseAllSpace	Do not reserve 20 percent of each data page for future record versions. Useful for read-only databases. Corresponds to **gbak -use_all_space**.
roValidate	Enables validation during a restore. Corresponds to **gbak -validate**.

Syntax

```
property Options: TIBRestoreOptions;
```

See Also

PageSize (see page 763)

1.17.11.1.4.2.4 TADIBRestore.PageBuffers Property

Specifies the number of default cache buffer for a restored database.

Description

Use the PageBuffer property to specify the number of default cache buffers to configure for attachments to the restored database. The default value is 0, means a value is not specified. Corresponds to **gbak -buffers**.

Syntax

```
property PageBuffers: LongWord;
```

See Also

PageSize (see page 763), Options (see page 763)

1.17.11.1.4.2.5 TADIBRestore.PageSize Property

Specifies the page size for a restored database.

Description

Use the PageSize property to specify the page size for a restored database. The default value is 0, means a value is not specified. Corresponds to **gbak -page_size**.

Syntax

```
property PageSize: LongWord;
```

See Also

PageBuffers (see page 763), Options (see page 763)

1.17.11.1.4.2.6 TADIBRestore.Verbose Property

Controls the DB Service Manager feedback.

Description

Use the Verbose property to control the DB Service Manager output. When is True, the TADIBRestore will fire OnProgress (see page 770) event and submit a restore task status. The default value is False. Corresponds to **gbak -verbose**.

Syntax

```
property Verbose: Boolean;
```

See Also

TADIBService.OnProgress (see page 770)

1.17.11.1.5 TADIBSecurity Class

The class implementing FB/IB database security management service.

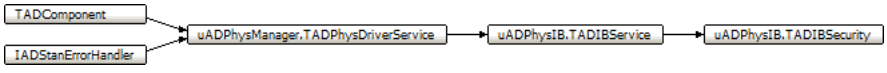
public

public	Description
Users (see page 765)	The reference to the all users list.
AddUser (see page 766)	Adds a new user to FB/IB security database.
DeleteUser (see page 766)	Deletes an existing user from FB/IB security database.
DisplayUser (see page 766)	Fetches an existing user properties from FB/IB security database.
DisplayUsers (see page 766)	Fetches all user list from FB/IB security database.
ModifyUser (see page 767)	Modifies an existing user in FB/IB security database.

published

published	Description
AFirstName (see page 767)	A person first name.
AGroupID (see page 767)	A person group ID.
AGroupName (see page 767)	A person group name.
ALastName (see page 767)	A person last name.
AMiddleName (see page 768)	A person middle name.
APassword (see page 768)	A person password.
ARoleName (see page 768)	A person role name.
AUserID (see page 768)	A person ID.
AUserName (see page 768)	A person SQL user name.
Modified (see page 768)	A list of modified user attributes.

Class Hierarchy



File

uADPhysIB

Description

Use the TADIBSecurity component to add a database security management capabilities to an application.

To manage a security an application should:

- specify DriverLink (see page 769), UserName (see page 771), Password (see page 770), Host (see page 769);
- optionally specify AUserName (see page 768);
- call AddUser (see page 766) / DeleteUser (see page 766) / ModifyUser (see page 767) / DisplayUser (see page 766) / DisplayUsers (see page 766) method.

Syntax

```
TADIBSecurity = class(TADIBService);
```

Example 1

Adding a user:

```
ADIBSecurity1.DriverLink := ADPhysIBDriverLink1;

ADIBSecurity1.UserName := 'sysdba';
ADIBSecurity1.Password := 'masterkey';
ADIBSecurity1.Host := 'db_srv_host';
ADIBSecurity1.Protocol := ipTCPIP;

ADIBSecurity1.AUserName := 'user1';
ADIBSecurity1.APassword := '12345';
ADIBSecurity1.AFirstName := 'Bill';
ADIBSecurity1.ALastName := 'Scott';

ADIBSecurity1.AddUser;
```

Example 2

Change a user password:

```
ADIBSecurity1.DriverLink := ADPhysIBDriverLink1;

ADIBSecurity1.UserName := 'sysdba';
ADIBSecurity1.Password := 'masterkey';
ADIBSecurity1.Host := 'db_srv_host';
ADIBSecurity1.Protocol := ipTCPIP;

ADIBSecurity1.AUserName := 'user1';
ADIBSecurity1.APassword := '12345';

ADIBSecurity1.ModifyUser;
```

Example 3

List all users using TADMemTable (see page 436):

```
ADIBSecurity1.DriverLink := ADPhysIBDriverLink1;

ADIBSecurity1.UserName := 'sysdba';
ADIBSecurity1.Password := 'masterkey';
ADIBSecurity1.Host := 'db_srv_host';
ADIBSecurity1.Protocol := ipTCPIP;

ADIBSecurity1.DisplayUsers;
ADMemTable1.AttachTable(ADIBSecurity1.Users, nil);
ADMemTable1.Open;
```

1.17.11.1.5.1 public

1.17.11.1.5.1.1 TADIBSecurity.Users Property

The reference to the all users list.

Description

Use the Users property to read the all existing user list. The property is filled after the call to DisplayUsers (see page 766) method.

Syntax

```
property Users: TADDatSTable;
```

See Also

DisplayUsers (see page 766)

Example

```
ADIBSecurity.DisplayUsers;
ADMemTable1.AttachTable(ADIBSecurity.Users, nil);
ADMemTable1.Open;
```

1.17.11.1.5.1.2 TADIBSecurity.AddUser Method

Adds a new user to FB/IB security database.

Description

Use the AddUser method to add a new user to the FB/IB security database. To add a user an application must specifiy AUserName (see page 768) and APassword (see page 768) properties. Other AXxxx properties are optional.

Syntax

```
procedure AddUser;
```

See Also

ModifyUser (see page 767), DeleteUser (see page 766)

1.17.11.1.5.1.3 TADIBSecurity.DeleteUser Method

Deletes an existing user from FB/IB security database.

Description

Use the DeleteUser method to delete an existing user from FB/IB security database. To delete a user an application must specifiy AUserName (see page 768).

Syntax

```
procedure DeleteUser;
```

See Also

AddUser (see page 766), ModifyUser (see page 767)

1.17.11.1.5.1.4 TADIBSecurity.DisplayUser Method

Fetches an existing user properties from FB/IB security database.

Description

Use the DisplayUser method to fetch an existing user properties from FB/IB security database. To describe a user an application must specifiy AUserName (see page 768). After the DisplayUser call the application may check the AXxxx properties for the user properties.

Syntax

```
procedure DisplayUser;
```

See Also

DisplayUsers (see page 766)

1.17.11.1.5.1.5 TADIBSecurity.DisplayUsers Method

Fetches all user list from FB/IB security database.

Description

Use the DisplayUsers method to fetch all existing users list from the FB/IB security database. No additional properties should be specified.

The list of the existing properties will be stored in the Users (see page 765) table.

Syntax

```
procedure DisplayUsers;
```

See Also

DisplayUser (see page 766)

1.17.11.1.5.1.6 TADIBSecurity.ModifyUser Method

Modifies an existing user in FB/IB security database.

Description

Use the ModifyUser method to modify an existing user in the FB/IB security database. To modify a user an application must specify AUserName (see page 768) property. Other AXxxx properties are optional. The user name cannot be changed.

Syntax

```
procedure ModifyUser;
```

See Also

List here...

Example

1.17.11.1.5.2 published

1.17.11.1.5.2.1 TADIBSecurity.AFirstName Property

A person first name.

Description

Use the AFirstName property to specify an optional first name of the person identified by AUserName.

Syntax

```
property AFirstName: String;
```

1.17.11.1.5.2.2 TADIBSecurity.AGroupID Property

A person group ID.

Description

Use the AGroupID property to specify an optional grop ID of the person identified by AUserName.

Syntax

```
property AGroupID: Integer;
```

1.17.11.1.5.2.3 TADIBSecurity.AGroupName Property

A person group name.

Description

Use the AGroupName property to specify an optional group name of the person identified by AUserName.

Syntax

```
property AGroupName: String;
```

1.17.11.1.5.2.4 TADIBSecurity.ALastName Property

A person last name.

Description

Use the ALastName property to specify an optional last name of the person identified by AUserName.

Syntax

```
property ALastName: String;
```

1.17.11.1.5.2.5 TADIBSecurity.AMiddleName Property

A person middle name.

Description

Use the AMiddleName property to specify an optional middle name of the person identified by AUserName.

Syntax

```
property AMiddleName: String;
```

1.17.11.1.5.2.6 TADIBSecurity.APassword Property

A person password.

Description

Use the APassword property to specify an password of the person identified by AUserName.

Syntax

```
property APassword: String;
```

1.17.11.1.5.2.7 TADIBSecurity.ARoleName Property

A person role name.

Description

Use the ARoleName property to specify an optional SQL role name of the person identified by AUserName.

Syntax

```
property ARoleName: String;
```

1.17.11.1.5.2.8 TADIBSecurity.AUserID Property

A person ID.

Description

Use the AUserID property to specify an optional user ID of the person identified by AUserName.

Syntax

```
property AUserID: Integer;
```

1.17.11.1.5.2.9 TADIBSecurity.AUserName Property

A person SQL user name.

Description

Use the AUserName property to specify a SQL user name to create / modify / delete / retrieve information.

Syntax

```
property AUserName: String;
```

1.17.11.1.5.2.10 TADIBSecurity.Modified Property

A list of modified user attributes.

Description

The Modified property is automatically updated, when an application is updating the AXxxx properties. The AddUser (see page 766) and ModifyUser (see page 767)methods will use just AXxxx property values, which have corresponding flags in

the Modified property value.

Syntax

```
property Modified: TIBSecurityValues;
```

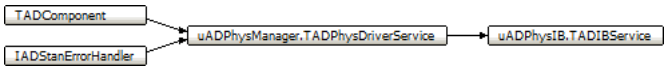
1.17.11.1.6 TADIBService Class

The base class for all FB/IB service classes.

published

published	Description
DriverLink (see page 769)	Specifies the DriverLink.
Host (see page 769)	Specifies the host name with the DB server.
OnProgress (see page 770)	Specifies the event handler allowing to handle the DB server feedback.
Password (see page 770)	Specifies the SQL user password to connect to the DB server.
Protocol (see page 770)	Specifies the protocol used to connect to the DB server.
QueryTimeout (see page 770)	Add a short summary here...
SqlRoleName (see page 771)	Specifies the SQL role name to connect to the DB server.
UserName (see page 771)	Specifies the SQL user name to connect to the DB server.

Class Hierarchy



File

uADPhysIB

Description

The TADIBService class is used as a base class for all FB/IB service classes and should not be used directly. The class contains the common properties, like the DriverLink (see page 769), Host (see page 769), Protocol (see page 770), UserName (see page 771) and Password (see page 770).

Syntax

```
TADIBService = class(TADPhysDriverService);
```

1.17.11.1.6.1 published

1.17.11.1.6.1.1 TADIBService.DriverLink Property

Specifies the DriverLink.

Description

Use the DriverLink property to specify the FB/IB driver link. This property must be specified for any service operation.

Syntax

```
property DriverLink: TADPhysIBDriverLink;
```

1.17.11.1.6.1.2 TADIBService.Host Property

Specifies the host name with the DB server.

Description

Use the Host property to specify the host name where the DB server is running. This property must be specified for any service operation.

Syntax

```
property Host: String;
```

See Also

Protocol (see page 770)

1.17.11.1.6.1.3 TADIBService.OnProgress Event

Specifies the event handler allowing to handle the DB server feedback.

Description

Use the OnProgress event handler to handle the DB server feedback, produced by the different services.

Additionally is required to set TADIBBackup.Verbose (see page 756) or TADIBRestore.Verbose (see page 764) to True.

Syntax

```
property OnProgress: TADPhysServiceProgressEvent;
```

Example

```
ADIBBackup1.DriverLink := ADPhysIBDriverLink1;
// set host, database, etc properties
ADIBBackup1.Verbose := True;
ADIBBackup1.OnProgress := Form1Progress;

procedure TForm1.Form1Progress(ASender: TADPhysDriverService; const AMessage: String);
begin
  Memo1.Lines.Add(AMessage);
  Application.ProcessMessages;
end;
```

1.17.11.1.6.1.4 TADIBService.Password Property

Specifies the SQL user password to connect to the DB server.

Description

Use the Password property to specify the SQL user password to connect to the DB server. This property must be specified for any service operation.

Syntax

```
property Password: String;
```

See Also

UserName (see page 771), SqlRoleName (see page 771)

1.17.11.1.6.1.5 TADIBService.Protocol Property

Specifies the protocol used to connect to the DB server.

Description

Use the Protocol property to specify the network protocol used to connect to the DB server.

Syntax

```
property Protocol: TADIBProtocol;
```

See Also

Host (see page 769)

1.17.11.1.6.1.6 TADIBService.QueryTimeout Property

Add a short summary here...

Description

Add a description here...

Syntax

```
property QueryTimeout: LongWord;
```

See Also

List here...

Example

1.17.11.1.6.1.7 TADIBService.SqlRoleName Property

Specifies the SQL role name to connect to the DB server.

Description

Use the SqlRoleName property to specify the SQL role name to connect to the DB server.

Syntax

```
property SqlRoleName: String;
```

See Also

UserName (see page 771), Password (see page 770)

1.17.11.1.6.1.8 TADIBService.UserName Property

Specifies the SQL user name to connect to the DB server.

Description

Use the UserName property to specify the SQL user name to connect to the DB server. This property must be specified for any service operation.

Syntax

```
property UserName: String;
```

See Also

Password (see page 770), SqlRoleName (see page 771)

1.17.11.1.7 TADIBTrace Class

The class implementing Firebird database trace service.

public

public	Description
List (see page 772)	Produces a list of trace sessions.
Resume (see page 772)	Resumes a trace session.
Start (see page 773)	Starts new trace session.
Stop (see page 773)	Stops a trace session.
Suspend (see page 773)	Suspends a trace session.

published

published	Description
Config (see page 773)	Specifies the trace session configuration.
SessionID (see page 774)	The trace session ID.
SessionName (see page 774)	The trace session name.

Class Hierarchy



File

uADPhysIB

Description

Use the TADIBTrace component to add a DBMS trace capabilities to an application.

To manage a trace an application should:

- specify DriverLink (see page 769), UserName (see page 771), Password (see page 770), Host (see page 769);
- optionally specify SessionName (see page 774) / SessionID (see page 774);
- call Start (see page 773) / Stop (see page 773) / etc method.

For more details check this article.

Syntax

```
TADIBTrace = class(TADIBService);
```

Example

Start (see page 773) trace:

```
ADIBTrace1.DriverLink := ADPhysIBDriverLink1;

ADIBTrace1.UserName := 'sysdba';
ADIBTrace1.Password := 'masterkey';
ADIBTrace1.Host := 'db_srv_host';
ADIBTrace1.Protocol := ipTCPIP;

ADIBTrace1.SessionName := 'trace1';

ADIBTrace1.Start;
```

1.17.11.1.7.1 public

1.17.11.1.7.1.1 TADIBTrace.List Method

Produces a list of trace sessions.

Description

The the List method to get a list of all active trace session on Firebird server.

To receive the list an application should use an OnProgress (see page 770) event handler.

Syntax

```
procedure List;
```

See Also

OnProgress (see page 770)

1.17.11.1.7.1.2 TADIBTrace.Resume Method

Resumes a trace session.

Description

Use the Resume method to resume a trace session specified by SessionID (see page 774) property and previously suspended by the Suspend (see page 773)method.

Syntax

```
procedure Resume;
```

See Also

Suspend (see page 773), SessionID (see page 774)

1.17.11.1.7.1.3 TADIBTrace.Start Method

Starts new trace session.

Description

Use the Start method to start a new trace session on Firebird server, identified by SessionName (see page 774).

After successful call of this method the SessionID (see page 774) property will get a new session ID value. To process trace output an application should use an OnProgress (see page 770) event handler.

To stop the session, use the Stop (see page 773) method. To suspend it temporarily, use the Suspend (see page 773) method.

Syntax

```
procedure Start;
```

See Also

OnProgress (see page 770), SessionName (see page 774), SessionID (see page 774), Stop (see page 773), Suspend (see page 773)

1.17.11.1.7.1.4 TADIBTrace.Stop Method

Stops a trace session.

Description

Use the Stop method to permanently stop a trace session specified by SessionID (see page 774) property.

Syntax

```
procedure Stop;
```

See Also

Start (see page 773), SessionID (see page 774)

1.17.11.1.7.1.5 TADIBTrace.Suspend Method

Suspends a trace session.

Description

Use the Suspend method to temporarily suspend a trace session, specified by SessionID (see page 774) property.

Syntax

```
procedure Suspend;
```

See Also

SessionID (see page 774), Resume (see page 772)

1.17.11.1.7.2 published

1.17.11.1.7.2.1 TADIBTrace.Config Property

Specifies the trace session configuration.

Description

Use the Config property to specify the trace session configuration. A template file named **fbtrace.conf** is provided in Firebird's root directory as a guide to the contents of this string.

Syntax

```
property Config: TStrings;
```

1.17.11.1.7.2.2 TADIBTrace.SessionID Property

The trace session ID.

Description

Use the SessionID to specify a trace session ID, which will be used by the Resume (see page 772), Stop (see page 773), Suspend (see page 773) methods to identify a trace session. After calling the Start (see page 773) method the SessionID property will be filled automatically by the new trace session ID value.

Syntax

```
property SessionID: Integer;
```

See Also

SessionName (see page 774), Resume (see page 772), Stop (see page 773), Suspend (see page 773)

1.17.11.1.7.2.3 TADIBTrace.SessionName Property

The trace session name.

Description

Use the SessionName property to specify the trace session name used by the Start (see page 773) method call only. After calling the Start (see page 773)method, the SessionID (see page 774) method will receive corresponding value.

Syntax

```
property SessionName: String;
```

See Also

Start (see page 773), SessionID (see page 774)

1.17.11.1.8 TADIBValidate Class

The class implementing FB/IB database validate and repair service.

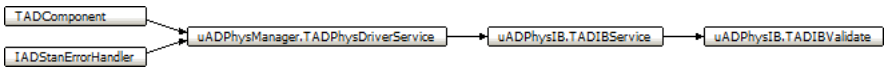
public

public	Description
Analyze (see page 775)	Starts the statistics gathering task.
CheckOnly (see page 776)	Starts the read-only validate database task.
Repair (see page 776)	Starts the repair task.
Sweep (see page 776)	Starts the sweep task.

published

published	Description
Database (see page 776)	Specifies a database file to validate / repair.
Options (see page 777)	Specifies validate / repair options.

Class Hierarchy



File

uADPhysIB

Description

Use the TADIBValidate component to add a database validate and repair capability to an application. This is a programmatic method to invoke the **gfix** tool as a thread in the DB server process.

To validate and or repair a database an application should:

- specify DriverLink (see page 769), UserName (see page 771), Password (see page 770), Host (see page 769);
- specify Database (see page 776) - a database file to validate;
- call Analyze (see page 775), CheckOnly (see page 776), Repair (see page 776) or Sweep (see page 776) method.

To produce a DB validate or repair log an application should use OnProgress (see page 770) event.

Syntax

```
TADIBValidate = class(TADIBService);
```

Example

```
ADIBValidate1.DriverLink := ADPhysIBDriverLink1;

ADIBValidate1.UserName := 'sysdba';
ADIBValidate1.Password := 'masterkey';
ADIBValidate1.Host := 'db_srv_host';
ADIBValidate1.Protocol := ipTCPIP;

ADIBValidate1.Database := 'e:\ib\addemo.fdb';

ADIBValidate1.Repair;
```

1.17.11.1.8.1 public

1.17.11.1.8.1.1 TADIBValidate.Analyze Method

Starts the statistics gathering task.

Parameters

Parameters	Description
const ATable: String = ''	A database table name.
const AIndex: String = ''	A database index name.

Description

Use the Analyze method to start the database statistics gathering task.

To perform a statistics gathering task an application must specify the connection parameters (see page 769) and Database (see page 776). This is a programmatic method to invoke the SET STATISTICS INDEX command. Depending on the specified arguments the method modes are:

- AIndex only - analyzes the specified index.
- ATable only - analyzes all indexes for the specified table.
- None specified - analyzes all indexes for all tables in the database.

Syntax

```
procedure Analyze(const ATable: String = ''; const AIndex: String = '');
```

See Also

CheckOnly (see page 776), Sweep (see page 776)

1.17.11.1.8.1.2 TADIBValidate.CheckOnly Method

Starts the read-only validate database task.

Description

Use the CheckOnly method to start the read-only validate database task on the DB server host using Service Manager.

To perform a validation task an application must specify the connection parameters (see page 769) and Database (see page 776). This is a programmatic method to invoke the **gfix -no_update** tool as a thread in the DB server process.

Syntax

```
procedure CheckOnly;
```

See Also

Repair (see page 776), Sweep (see page 776)

1.17.11.1.8.1.3 TADIBValidate.Repair Method

Starts the repair task.

Description

Use the Repair method to start the validate and repair database task on the DB server host using Service Manager.

To perform a repair task an application must specify the connection parameters (see page 769) and Database (see page 776). This is a programmatic method to invoke the **gfix** tool as a thread in the DB server process.

Syntax

```
procedure Repair;
```

See Also

CheckOnly (see page 776), Sweep (see page 776)

1.17.11.1.8.1.4 TADIBValidate.Sweep Method

Starts the sweep task.

Description

Use the Sweep method to start the database sweep task on the DB server host using Service Manager.

To perform a sweep task an application must specify the connection parameters (see page 769) and Database (see page 776). This is a programmatic method to invoke the **gfix -sweep** tool as a thread in the DB server process.

Syntax

```
procedure Sweep;
```

See Also

CheckOnly (see page 776), Repair (see page 776)

1.17.11.1.8.2 published

1.17.11.1.8.2.1 TADIBValidate.Database Property

Specifies a database file to validate / repair.

Description

Use the Database property to specify a path of the primary file of the database to validate or repair. The path is relative to a DB server.

Syntax

```
property Database: String;
```

1.17.11.1.8.2.2 TADIBValidate.Options Property

Specifies validate / repair options.

Description

Use the Options property to specify additional options controlling the validate or repair task. The default value is empty set. The Options are used only by CheckOnly (see page 776) and Repair (see page 776) methods.

Option	Description
roValidateFull	Check record and page structures, releasing unassigned record fragments. Corresponds to **gfix -full**.
roValidatePages	Locate and release pages that are allocated but unassigned to any data structures. Corresponds to **gfix -validate**.
roMendRecords	Mark corrupted records as unavailable, so subsequent operations skip them. Corresponds to **gfix -mend**.
roIgnoreChecksum	Ignore all checksum errors. Corresponds to **gfix -ignore**.
roKillShadows	Remove references to unavailable shadow files. Corresponds to **gfix -kill**.

Syntax

```
property Options: TIBRepairOptions;
```

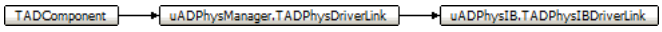
See Also

CheckOnly (see page 776), Repair (see page 776)

1.17.11.1.9 TADPhysIBDriverLink Class

Use the TADPhysIBDriverLink component to link the FB/IB driver to an application and setup it.

Class Hierarchy



File

uADPhysIB

Description

Use the TADPhysIBDriverLink component to link the FB/IB driver to an application. In general it is enough to only include uADPhysIB unit into your application uses clause.

The TADPhysIBDriverLink component may be used to specify:

- VendorHome (see page 782) - the Firebird / Interbase installation root folder;
- VendorLib (see page 782) - the name and optional path to the Firebird / Interbase client library.

All properties have default values appropriate for most case. To have an effect all property changes must be performed before first connection through this driver or use the Release (see page 780) method.

Syntax

```
TADPhysIBDriverLink = class(TADPhysDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to Interbase or Firebird (see page 198)

Example

```
ADPhysIBDriverLink1.VendorLib := 'c:\fb\fb25\bin\fbclient.dll';
```

1.17.12 uADPhysManager Namespace

Contains FireDAC driver and services base classes.

Classes

Class	Description
TADPhysDriverLink (see page 778)	The TADPhysDriverLink component is a base class for all driver link components.
TADPhysDriverService (see page 783)	The TADPhysDriverService component is a base class for all driver service components.

Description

The uADPhysManager unit contains:

- FireDAC driver base implementation classes;
- TADPhysDriverLink (see page 778) - the base class for FireDAC driver link components;
- TADPhysDriverService (see page 783) - the base class for FireDAC driver service components.

1.17.12.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADPhysDriverLink (see page 778)	The TADPhysDriverLink component is a base class for all driver link components.
TADPhysDriverService (see page 783)	The TADPhysDriverService component is a base class for all driver service components.

1.17.12.1.1 TADPhysDriverLink Class

The TADPhysDriverLink component is a base class for all driver link components.

public

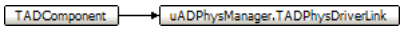
public	Description
ActualDriverID (see page 779)	The driver link actual driver ID.
DriverIntf (see page 779)	The driver Phys interface reference.
DriverState (see page 780)	The driver state.
Services (see page 780)	The services attached to this driver link.
ServicesCount (see page 780)	The number of services attached to this driver link.
Release (see page 780)	Releases the driver resources, allowing to initialize the driver with different parameters.

published

published	Description
BaseDriverID (see page 781)	The base driver ID.
DriverID (see page 781)	Specifies the virtual driver ID.
OnDriverCreated (see page 782)	The event firing after the driver has been created and the CLI library was loaded.
OnDriverDestroying (see page 782)	The event firing before the driver will be destroyed and the CLI library will be unloaded.

VendorHome (see page 782)	The CLI library installation folder.
VendorLib (see page 782)	The CLI library path and name.

Class Hierarchy



File

uADPhysManager

Description

The TADPhysDriverLink component is a base class for all driver link components. It implements the ability:

- to link an FireDAC DBMS driver into an application.
- to configure an FireDAC DBMS driver at runtime, using the VendorHome (see page 782) and VendorLib (see page 782) properties and Release (see page 780) method.
- to create an FireDAC virtual driver at runtime, using the DriverID (see page 781), BaseDriverID (see page 781) and ActualDriverID (see page 779) properties.
- to give access to an FireDAC driver API and live cycle, using DriverIntf (see page 779), DriverState (see page 780), OnDriverCreated (see page 782) and OnDriverDestroying (see page 782) properties.
- to manage the FireDAC DBMS specific services, using Services (see page 780) and ServicesCount (see page 780) properties.

An application should not use the TADPhysDriverLink component directly. But it may use one of the many descendants, for example, TADPhysOracleDriverLink (see page 800) or TADPhysSQLiteDriverLink (see page 805).

To have an effect all property changes must be performed before first connection through this driver or use the Release (see page 780) method.

Syntax

```
TADPhysDriverLink = class(TADComponent);
```

See Also

Configuring Drivers (see page 31), Database Connectivity (see page 188)

1.17.12.1.1.1 public

1.17.12.1.1.1.1 TADPhysDriverLink.ActualDriverID Property

The driver link actual driver ID.

Description

Use the ActualDriverID property to get an ID derived from current BaseDriverID (see page 781) and DriverID (see page 781) property values.

Syntax

```
property ActualDriverID: String;
```

See Also

BaseDriverID (see page 781), DriverID (see page 781)

1.17.12.1.1.1.2 TADPhysDriverLink.DriverIntf Property

The driver Phys interface reference.

Description

Use the DriverIntf property to get the driver IADPhysDriver interface reference.

Syntax

```
property DriverIntf: IADPhysDriver;
```

See Also

DriverState (see page 780)

1.17.12.1.1.1.3 TADPhysDriverLink.DriverState Property

The driver state.

Description

Use the DriverState property to get the current driver state.

Syntax

```
property DriverState: TADPhysDriverState;
```

See Also

DriverIntf (see page 779)

1.17.12.1.1.1.4 TADPhysDriverLink.Services Property

The services attached to this driver link.

Description

Use the Services property to get access to the service components attached to this driver link.

Syntax

```
property Services [AIndex: Integer]: TADPhysDriverService;
```

See Also

TADPhysDriverService (see page 783)

1.17.12.1.1.1.5 TADPhysDriverLink.ServicesCount Property

The number of services attached to this driver link.

Description

Use the ServicesCount property to get the number of service components attached to this driver link.

Syntax

```
property ServicesCount: Integer;
```

See Also

Services (see page 780)

1.17.12.1.1.1.6 TADPhysDriverLink.Release Method

Releases the driver resources, allowing to initialize the driver with different parameters.

Description

Use the Release method to release all driver resources and unload the DBMS client library.

The next connection established after the Release call will initialize driver and load the DBMS client library using the current driver parameters. So, after changing VendorHome and VendorLib property values, call Release method to force DBMS client reloading.

Before calling Release method an application must close all connections made through this driver. Otherwise an exception will be raised. When there were no connections made through this driver, then application may avoid Release call, because driver is not initialized and DBMS client library is not loaded.

Syntax

```
procedure Release;
```

See Also

VendorHome (see page 782), VendorLib (see page 782)

Example

```
ADConnection1.Close;
...
ADConnectionN.Close;

ADPhysIBDriverLink.VendorLib := 'c:\fbclient.dll';
ADPhysIBDriverLink.Release;

ADConnection1.Open;
...
ADConnectionN.Open;
```

1.17.12.1.1.2 published

1.17.12.1.1.2.1 TADPhysDriverLink.BaseDriverID Property

The base driver ID.

Description

Use the BaseDriverID property to get a base driver ID. The base driver ID is constant and represents the FireDAC DBMS driver implementation.

Syntax

```
property BaseDriverID: String;
```

See Also

Configuring Drivers (see page 31), DriverID (see page 781)

1.17.12.1.1.2.2 TADPhysDriverLink.DriverID Property

Specifies the virtual driver ID.

Description

Use the DriverID to specify a virtual driver ID. If DriverID is not specified, then any changes to a driver link properties will be applied to a base driver. Otherwise the virtual driver will be created and configured.

The DriverID must be unique across an application.

Syntax

```
property DriverID: String;
```

See Also

Configuring Drivers (see page 31), BaseDriverID (see page 781)

Example

```
ADPhysIBDriverLink1.DriverID := 'FB25';
ADPhysIBDriverLink1.VendorLib := 'c:\fb25_64\fbclient.dll';

ADConnection1.Params.Clear;
ADConnection1.Params.Add('DriverID=FB25');
ADConnection1.Params.Add('Database=c:\addemo.fdb');
ADConnection1.Params.Add('Server=127.0.0.1');
ADConnection1.Params.Add('Protocol=TCPIP');
ADConnection1.Connected := True;
```

1.17.12.1.1.2.3 TADPhysDriverLink.OnDriverCreated Event

The event firing after the driver has been created and the CLI library was loaded.

Description

Use the OnDriverCreate event handler to perform some actions after an FireDAC DBMS driver has been created and a DBMS CLI library was loaded. In this event the DriverIntf (see page 779) property value is accessible.

Syntax

```
property OnDriverCreated: TNotifyEvent;
```

See Also

OnDriverDestroying (see page 782)

1.17.12.1.1.2.4 TADPhysDriverLink.OnDriverDestroying Event

The event firing before the driver will be destroyed and the CLI library will be unloaded.

Description

Use the OnDriverDestroying event handler to perform some actions before an FireDAC DBMS driver will be destroyed and a DBMS CLI library will be unloaded. In this event the DriverIntf (see page 779) property value is still accessible.

Syntax

```
property OnDriverDestroying: TNotifyEvent;
```

See Also

OnDriverCreated (see page 782)

1.17.12.1.1.2.5 TADPhysDriverLink.VendorHome Property

The CLI library installation folder.

Description

Use the VendorHome to specify the CLI library installation folder. Depending on a DBMS, that may be:

- Installation Home Name for Oracle;
- Installation root folder for the other DBMS's. The CLI library will be loaded from Bin sub-folder. For example for MySQL the VendorHome may be c:\mysql.

A VendorHome change will have an effect only before first connection through this driver or use the Release (see page 780) method.

The property value is platform independent and will be used for all platforms, including Win32, Win64, MacOS32, MacOS64, Linux32 and Linux64. To specify multiple platform specific values use driver configuration file.

Syntax

```
property VendorHome: String;
```

See Also

Configuring Drivers (see page 31), VendorLib (see page 782), Release (see page 780)

Example

```
ADConnection1.Close;
ADPhysOracleDriverLink1.Release;
ADPhysOracleDriverLink1.VendorHome := 'OracleXE';
ADConnection1.Open;
```

1.17.12.1.1.2.6 TADPhysDriverLink.VendorLib Property

The CLI library path and name.

Description

Use the VendorLib to specify the CLI library name and optionally path. If VendorLib and VendorHome (see page 782) are both specified, then VendorLib will be appended to VendorHome (see page 782).

A VendorLib change will have an effect only before first connection through this driver or use the Release (see page 780) method.

The property value is platform independent and will be used for all platforms, including Win32, Win64, MacOS32, MacOS64, Linux32 and Linux64. To specify multiple platform specific values use driver configuration file.

Syntax

```
property VendorLib: String;
```

See Also

Configuring Drivers (see page 31), VendorHome (see page 782), Release (see page 780)

Example

```
ADConnection1.Close;
ADPhysIBDriverLink1.Release;
ADPhysIBDriverLink1.VendorLib := 'c:\fb25_64\fbclient.dll';
ADConnection1.Open;
```

1.17.12.1.2 TADPhysDriverService Class

The TADPhysDriverService component is a base class for all driver service components.

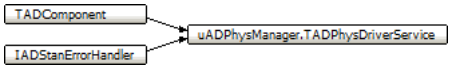
public

public	Description
CliObj (see page 783)	Returns a reference to the driver CLI object.

published

published	Description
AfterExecute (see page 784)	The event firing after the service action will be executed.
BeforeExecute (see page 784)	The event firing before the service action will be executed.
OnError (see page 784)	The event firing in case an exception will be raised by a service.

Class Hierarchy



File

uADPhysManager

Description

The TADPhysDriverService component is a base class for all driver service components. It implements the ability to link a service to a driver link.

An application should not use the TADPhysDriverService component directly.

Syntax

```
TADPhysDriverService = class(TADComponent, IADStanErrorHandler);
```

1.17.12.1.2.1 public

1.17.12.1.2.1.1 TADPhysDriverService.CliObj Property

Returns a reference to the driver CLI object.

Description

Use CliObj property to get the reference to the driver CLI object.

Syntax

```
property CliObj: TObject;
```

See Also

DriverLink

1.17.12.1.2.2 published

1.17.12.1.2.2.1 TADPhysDriverService.AfterExecute Event

The event firing after the service action will be executed.

Description

Use the AfterExecute event handler to perform the actions after the service action is finished.

Syntax

```
property AfterExecute: TNotifyEvent;
```

See Also

BeforeExecute (see page 784)

1.17.12.1.2.2.2 TADPhysDriverService.BeforeExecute Event

The event firing before the service action will be executed.

Description

Use the BeforeExecute event handler to perform the actions before the service action will be executed.

Syntax

```
property BeforeExecute: TNotifyEvent;
```

See Also

AfterExecute (see page 784)

1.17.12.1.2.2.3 TADPhysDriverService.OnError Event

The event firing in case an exception will be raised by a service.

Description

Use the OnError event handler to perform the actions when a service execution raising an exception

Syntax

```
property OnError: TADErrorEvent;
```

1.17.13 uADPhysMSAcc Namespace

Contains Microsoft Access driver and services components.

Classes

Class	Description
EMSAccessNativeException (see page 785)	Class representing a Microsoft Access driver exception.

TADMSAccessService (see page 786)	The class implementing Microsoft Access database create, drop, compact and repair services.
TADPhysMSAccessDriverLink (see page 791)	Use the TADPhysMSAccessDriverLink component to link the Microsoft Access driver to an application and setup it.

Description

The uADPhysMSAcc unit contains:

- Microsoft Access driver implementation;
- TADPhysMSAccessDriverLink (see page 791) - component to link the driver and setup CLI library;
- TADMSAccessService (see page 786) - database service component.

See Also

Configuring Drivers (see page 31), Connect to Microsoft Access database (see page 207)

1.17.13.1 Classes

The following table lists classes in this documentation.

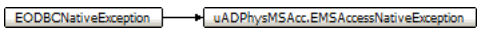
Classes

Class	Description
EMSAccessNativeException (see page 785)	Class representing a Microsoft Access driver exception.
TADMSAccessService (see page 786)	The class implementing Microsoft Access database create, drop, compact and repair services.
TADPhysMSAccessDriverLink (see page 791)	Use the TADPhysMSAccessDriverLink component to link the Microsoft Access driver to an application and setup it.

1.17.13.1.1 EMSAccessNativeException Class

Class representing a Microsoft Access driver exception.

Class Hierarchy



File

uADPhysMSAcc

Description

The FireDAC Microsoft Access driver raises exceptions of EMSAccessNativeException class. Use this class to handle driver specific errors.

Syntax

```
EMSAccessNativeException = class(EODBCNativeException);
```

See Also

EODBCNativeException, TADMSAccessService (see page 786), Connect to Microsoft Access database (see page 207)

Example

```
try
  ADConnection1.Connected := True;
except
  on E: EMSAccessNativeException do
    // perform Access error specific actions
end;
```

1.17.13.1.2 TADMSAccessService Class

The class implementing Microsoft Access database create, drop, compact and repair services.

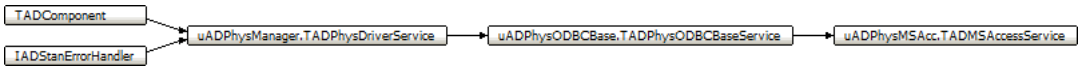
public

public	Description
Compact (see page 786)	Compacts an Access database.
CreateDB (see page 787)	Creates a new Access database.
Drop (see page 788)	Drops an existing Access database.
Repair (see page 788)	Repairs an existing Access database.

published

published	Description
Database (see page 788)	A database file name.
DBVersion (see page 789)	The database format version.
DestDatabase (see page 789)	A destination database file name.
DriverLink (see page 789)	Specifies the DriverLink.
Encrypted (see page 790)	The database encryption mode.
Password (see page 790)	The database password.
ResetPassword (see page 790)	Whether or not to reset the database password.
SortOrder (see page 791)	The database sort order.

Class Hierarchy



File

uADPhysMSAcc

Description

Use the TADMSAccessService component to add a database create, drop, repair and compact capability to an application.

To validate and or repair a database an application should:

- specify DriverLink (see page 789);
- specify Database (see page 788) - a database file to handle;
- specify Password (see page 790);
- call CreateDB (see page 787), Drop (see page 788), Repair (see page 788) or Compact (see page 786) methods.

Read more about these features (there) and (here).

Syntax

```
TADMSAccessService = class(TADPhysODBCBaseService);
```

Example

```
ADMSAccessService1.Database := 'c:\test.mdb';
ADMSAccessService1.Repair;
```

1.17.13.1.2.1 public

1.17.13.1.2.1.1 TADMSAccessService.Compact Method

Compacts an Access database.

Description

Use the Compact method to compact a database. To run Compact, an application must specify:

- Database (see page 788) - the source database file name;
- DestDatabase (see page 789) - the destination database file name. If it is not specified, then DestDatabase (see page 789) is equal to Database (see page 788).

Optionally Encrypted (see page 790), Password (see page 790) and SortOrder (see page 791) may be specified.

The file pointed by Database (see page 788) must be a valid .MDB file. DestDatabase (see page 789) can point to the same file as Database (see page 788), in which case the file will be compacted into the same location. If DestDatabase (see page 789) names a different file than Database (see page 788), the file named as the DestDatabase (see page 789) will be deleted at the time Compact is called. When the operation fails, then EMSAccessNativeException (see page 785) exception is raised.

Syntax

```
procedure Compact;
```

See Also

Database (see page 788), DestDatabase (see page 789), Encrypted (see page 790), Password (see page 790), SortOrder (see page 791)

Example 1

```
// Compact non-secured database
ADMSAccessService1.Database := 'c:\test.mdb';
ADMSAccessService1.DestDatabase := 'c:\test2.mdb';
ADMSAccessService1.Compact;
```

Example 2

```
// Compact secured database
ADMSAccessService1.Database := 'c:\test.mdb';
ADMSAccessService1.Password := 'J@^^1234pw';
ADMSAccessService1.Compact;
```

Example 3

```
// Reset password of secured database
ADMSAccessService1.Database := 'c:\test.mdb';
ADMSAccessService1.Password := 'J@^^1234pw';
ADMSAccessService1.ResetPassword := True;
ADMSAccessService1.Compact;
```

1.17.13.1.2.1.2 TADMSAccessService.CreateDB Method

Creates a new Access database.

Description

Use the CreateDB method to create a new empty database. To run CreateDB, an application must specify Database (see page 788) - the source database file name. And optionally may be specified:

- DBVersion (see page 789) - the database format version;
- Password (see page 790) - the password to protect the database;
- Encrypted (see page 790) - whether to encrypt the database;
- SortOrder (see page 791) - the default database sort order.

The file pointed by Database (see page 788) must not exist at the time CreateDB is called. When the operation fails, then "Driver's ConfigDSN, ConfigDriver, or ConfigTranslator failed" exception is raised.

Syntax

```
procedure CreateDB;
```

See Also

Database (see page 788), DBVersion (see page 789), SortOrder (see page 791)

Example 1

```
ADMSAccessService1.Database := 'c:\test.mdb';
ADMSAccessService1.DBVersion := avDefault;
ADMSAccessService1.CreateDB;
```

Example 2

```
// Create encrypted database
ADMSAccessService1.Database := 'c:\test.accdb';
ADMSAccessService1.DBVersion := avAccess2007;
ADMSAccessService1.Encrypted := True;
ADMSAccessService1.CreateDB;
```

Example 3

```
// Specify default sort order of database
ADMSAccessService1.Database := 'c:\test.accdb';
ADMSAccessService1.DBVersion := avAccess2007;
ADMSAccessService1.SortOrder := '0x00000409';
ADMSAccessService1.CreateDB;
```

1.17.13.1.2.1.3 TADMSAccessService.Drop Method

Drops an existing Access database.

Description

Use the Drop method to drop an existing database. To run Drop, an application must specify Database (see page 788) - the database file name to drop. The database file must be not in use by an application or Microsoft Access.

Syntax

```
procedure Drop;
```

See Also

Database (see page 788)

1.17.13.1.2.1.4 TADMSAccessService.Repair Method

Repairs an existing Access database.

Description

Use the Repair method to repair an existing database. To run Repair, an application must specify Database (see page 788) - the database file name to repair.

The file pointed by Database (see page 788) must be a valid .MDB file. For more information see Compact (see page 786) method.

Syntax

```
procedure Repair;
```

See Also

Database (see page 788), Compact (see page 786)

1.17.13.1.2.2 published

1.17.13.1.2.2.1 TADMSAccessService.Database Property

A database file name.

Description

Use Database property to specify the Microsoft Access database file name. The requested operation will be performed on

this file. The file name may include $(<variable>) markers, where *variable* is a name of environment variable.

Syntax

```
property Database: String;
```

See Also

CreateDB (see page 787), Drop (see page 788), Compact (see page 786), Repair (see page 788)

1.17.13.1.2.2.2 TADMSAccessService.DBVersion Property

The database format version.

Description

Use DBVersion property to specify the database format version for the CreateDB (see page 787), Compact (see page 786) and Repair (see page 788) methods:

Value	Description
avDefault	Use a default format for a Microsoft Access ODBC driver.
avAccess2	Use Microsoft Access v 2 compatible format.
avAccess95	Use Microsoft Access 95 compatible format.
avAccess97	Use Microsoft Access 97 compatible format.
avAccess2000	Use Microsoft Access 2000 compatible format.
avAccess2003	Use Microsoft Access 2003 compatible format.
avAccess2007	Use Microsoft Access 2003 compatible format*.

* Because of the current limitation the latest Microsoft.ACE.OLEDB.12.0 provider will be used for all operations but still the database of Microsoft Access 2003 format will be created.

Syntax

```
property DBVersion: TADMSAccessDBVersion;
```

See Also

CreateDB (see page 787), Compact (see page 786), Repair (see page 788)

1.17.13.1.2.2.3 TADMSAccessService.DestDatabase Property

A destination database file name.

Description

Use DestDatabase property to specify the destination database file name for Compact (see page 786) and Repair (see page 788) methods. The file name may include $(<variable>) markers, where *variable* is a name of environment variable.

Syntax

```
property DestDatabase: String;
```

See Also

Compact (see page 786), Repair (see page 788)

1.17.13.1.2.2.4 TADMSAccessService.DriverLink Property

Specifies the DriverLink.

Description

Use DriverLink property to specify the driver link. This property must be specified for any service operation.

Syntax

```
property DriverLink: TADPhysMSAccessDriverLink;
```

1.17.13.1.2.2.5 TADMSAccessService.Encrypted Property

The database encryption mode.

Description

Use Encrypted property to protect the database from prying eyes for CreateDB (see page 787), Compact (see page 786) or Repair (see page 788) methods.

Syntax

```
property Encrypted: Boolean;
```

See Also

CreateDB (see page 787), Compact (see page 786), Repair (see page 788)

1.17.13.1.2.2.6 TADMSAccessService.Password Property

The database password.

Description

Use Password property to work with protected databases for CreateDB (see page 787), Compact (see page 786) or Repair (see page 788) methods. This property indicates the password if the database is already secured or sets the password for the non-secured database after calling these methods.

Syntax

```
property Password: String;
```

See Also

CreateDB (see page 787), Compact (see page 786), Repair (see page 788), ResetPassword (see page 790)

Example 1

```
// Create database and set password
ADMSAccessService1.Database := 'c:\test.mdb';
ADMSAccessService1.Password := 'J@^^1234pw';
ADMSAccessService1.CreateDB;
```

Example 2

```
// Set password for non-secured database
ADMSAccessService1.Database := 'c:\test.mdb';
ADMSAccessService1.Password := 'J@^^1234pw';
ADMSAccessService1.Compact;
```

1.17.13.1.2.2.7 TADMSAccessService.ResetPassword Property

Whether or not to reset the database password.

Description

Use ResetPassword property to reset the database password for Compact (see page 786) and Repair (see page 788) methods. The Password property value should be set to the current database password.

Syntax

```
property ResetPassword: Boolean;
```

See Also

Password (see page 790), Compact (see page 786), Repair (see page 788)

Example

```
// Reset password for secured database
ADMSAccessService1.Database := 'c:\test.mdb';
ADMSAccessService1.Password := 'J@^^1234pw';
ADMSAccessService1.ResetPassword := True;
ADMSAccessService1.Compact;
```

1.17.13.1.2.2.8 TADMSAccessService.SortOrder Property

The database sort order.

Description

Use SortOrder property to specify the database sort order for CreateDB (see page 787), Compact (see page 786) or Repair (see page 788) methods. The SortOrder value should be the one out of Locale Identifier values available (here).

Syntax

```
property SortOrder: String;
```

See Also

CreateDB (see page 787), Compact (see page 786), Repair (see page 788)

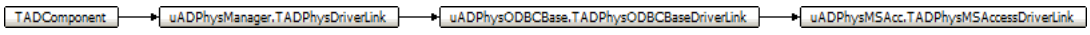
Example

```
// Set SortOrder to German Phone Book
ADMSAccessService1.Database := 'c:\test.mdb';
ADMSAccessService1.SortOrder := '0x00010407';
ADMSAccessService1.CreateDB;
```

1.17.13.1.3 TADPhysMSAccessDriverLink Class

Use the TADPhysMSAccessDriverLink component to link the Microsoft Access driver to an application and setup it.

Class Hierarchy



File

uADPhysMSAcc

Description

Use the TADPhysMSAccessDriverLink component to link the Microsoft Access driver to an application. In general it is enough to only include uADPhysMSAcc unit into your application uses clause.

The TADPhysMSAccessDriverLink component may be used to specify:

- ODBCDriver (see page 799) - Access ODBC driver name;
- ODBCAdvanced (see page 799) - Access ODBC driver connection parameter common for all connections;
- VendorLib (see page 782) - ODBC driver manager. By default it is odbc32.dll and for most cases should remain unchanged.

All properties have default values appropriate for most case. To have an effect all property changes must be performed before first connection through this driver or use the Release (see page 780) method.

Syntax

```
TADPhysMSAccessDriverLink = class(TADPhysODBCBaseDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to Microsoft Access database (see page 207)

Example

```
ADPhysMSAccessDriverLink1.ODBCAdvanced := 'IMPLICITCOMMITSYNC=NO';
ADPhysMSAccessDriverLink1.ODBCDriver := 'Microsoft Access Driver (*.mdb)';
```

1.17.14 uADPhysMSSQL Namespace

Contains Microsoft SQL Server driver and services components.

Classes

Class	Description
EMSSQLNativeException (see page 792)	Class representing a Microsoft SQL Server driver exception.
TADMSSQLError (see page 793)	TADMSSQLError represents FireDAC SQL Server errors, warnings and messages for the EMSSQLNAtiveException exception class.
TADPhysMSSQLDriverLink (see page 793)	Use the TADPhysMSSQLDriverLink component to link the Microsoft SQL Server driver to an application and setup it.

Description

The uADPhysMSSQL unit contains:

- Microsoft SQL Server driver implementation;
- TADPhysMSSQLDriverLink (see page 793) - component to link the driver and setup CLI library.

See Also

Configuring Drivers (see page 31), Connect to Microsoft SQL Server (see page 202)

1.17.14.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
EMSSQLNativeException (see page 792)	Class representing a Microsoft SQL Server driver exception.
TADMSSQLError (see page 793)	TADMSSQLError represents FireDAC SQL Server errors, warnings and messages for the EMSSQLNAtiveException exception class.
TADPhysMSSQLDriverLink (see page 793)	Use the TADPhysMSSQLDriverLink component to link the Microsoft SQL Server driver to an application and setup it.

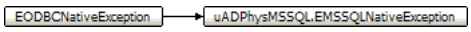
1.17.14.1.1 EMSSQLNativeException Class

Class representing a Microsoft SQL Server driver exception.

public

public	Description
Errors (see page 793)	Returns an error item.

Class Hierarchy



File

uADPhysMSSQL

Description

The FireDAC Microsoft SQL Server driver raises exceptions of EMSSQLNativeException class. Use this class to handle driver specific errors.

Syntax

```
EMSSQLNativeException = class(EODBCNativeException);
```

See Also

EODBCNativeException, TADPhysMSSQLDriverLink (see page 793), Connect to Microsoft SQL Server (see page 202)

Example

```
try
```

```
  ADConnection1.Connected := True;
except
  on E: EMSSQLNativeException do
    // perform SQL Server error specific actions
end;
```

1.17.14.1.1.1 public

1.17.14.1.1.1.1 EMSSQLNativeException.Errors Property

Returns an error item.

Description

Use Errors property to get access to an error item from the collection of errors, associated with this exception object.

Syntax

```
property Errors [Index: Integer]: TADMSSQLError;
```

See Also

TADMSSQLError (see page 793), TADDBError (see page 837)

1.17.14.1.2 TADMSSQLError Class

TADMSSQLError represents FireDAC SQL Server errors, warnings and messages for the EMSSQLNAtiveException exception class.

Class Hierarchy



File

uADPhysMSSQL

Description

The TADMSSQLError object is a container for the information pertaining to one database error, warning or message. One or more TADMSSQLErrorobjects are contained in the Errors (see page 793) property of the EMSSQLNativeException (see page 792) object. TADMSSQLError class represents SQL Server specific error attributes additionally to TADDBError (see page 837).

Syntax

```
TADMSSQLError = class(TADODBCNativeError);
```

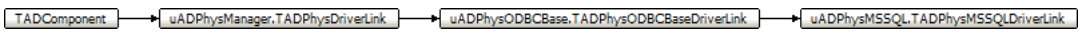
See Also

TADDBError (see page 837)

1.17.14.1.3 TADPhysMSSQLDriverLink Class

Use the TADPhysMSSQLDriverLink component to link the Microsoft SQL Server driver to an application and setup it.

Class Hierarchy



File

uADPhysMSSQL

Description

Use the TADPhysMSSQLDriverLink component to link the Microsoft SQL Server driver to an application. In general it is enough to only include uADPhysMSSQL unit into your application uses clause.

The TADPhysMSSQLDriverLink component may be used to specify:

- ODBCDriver (see page 799) - SQL Server ODBC driver name;
- ODBCAdvanced (see page 799) - SQL Server ODBC driver connection parameter common for all connections;
- VendorLib (see page 782) - ODBC driver manager. By default it is odbc32.dll and for most cases should remain unchanged.

All properties have default values appropriate for most case. To have an effect all property changes must be performed before first connection through this driver or use the Release (see page 780) method.

Syntax

```
TADPhysMSSQLDriverLink = class(TADPhysODBCBaseDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to Microsoft SQL Server (see page 202)

Example

```
ADPhysMSSQLDriverLink1.ODBCAdvanced := 'MARS_Connection=no;Regional=yes';
ADPhysMSSQLDriverLink1.ODBCDriver := 'SQL Native Client';
```

1.17.15 uADPhysMySQL Namespace

Contains MySQL Server driver and services components.

Classes

Class	Description
TADPhysMySQLDriverLink (see page 794)	Use the TADPhysMySQLDriverLink component to link the MySQL Server driver to an application and setup it.

Description

The uADPhysMySQL unit contains:

- MySQL Server driver implementation;
- TADPhysMySQLDriverLink (see page 794) - component to link the driver and setup CLI library.

See Also

Configuring Drivers (see page 31), Connect to MySQL Server (see page 209)

1.17.15.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADPhysMySQLDriverLink (see page 794)	Use the TADPhysMySQLDriverLink component to link the MySQL Server driver to an application and setup it.

1.17.15.1.1 TADPhysMySQLDriverLink Class

Use the TADPhysMySQLDriverLink component to link the MySQL Server driver to an application and setup it.

published

published	Description
EmbeddedArgs (see page 795)	Specifies the MySQL Embedded server arguments.
EmbeddedGroups (see page 795)	Specifies the MySQL Embedded server argument groups.

Class Hierarchy



File

uADPhysMySQL

Description

Use the TADPhysMySQLDriverLink component to link the MySQL Server driver to an application. In general it is enough to only include uADPhysMySQL unit into your application uses clause.

The TADPhysMySQLDriverLink component may be used to specify:

- VendorHome (see page 782) - the MySQL installation root folder;
- VendorLib (see page 782) - the name and optional path to the MySQL client library.

All properties have default values appropriate for most case. To have an effect all property changes must be performed before first connection through this driver or use the Release (see page 780) method.

Syntax

```
TADPhysMySQLDriverLink = class(TADPhysDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to MySQL Server (see page 209)

Example

```
ADPhysMySQLDriverLink1.VendorLib := 'E:\mysql\5-0-27\bin\libmySQL.dll';
```

1.17.15.1.1.1 published

1.17.15.1.1.1.1 TADPhysMySQLDriverLink.EmbeddedArgs Property

Specifies the MySQL Embedded server arguments.

Description

Use the EmbeddedArgs property to specify the MySQL Embedded server arguments, as they are specified in Connect to MySQL Server (see page 209).

Syntax

```
property EmbeddedArgs: TStrings;
```

See Also

EmbeddedGroups (see page 795)

1.17.15.1.1.1.2 TADPhysMySQLDriverLink.EmbeddedGroups Property

Specifies the MySQL Embedded server argument groups.

Description

Use the EmbeddedGroups property to specify the MySQL Embedded server argument groups, as they are specified in Connect to MySQL Server (see page 209).

Syntax

```
property EmbeddedGroups: TStrings;
```

See Also

EmbeddedArgs (see page 795)

1.17.16 uADPhysODBC Namespace

Contains ODBC bridge driver.

Classes

Class	Description
TADPhysODBCDriverLink (see page 796)	Use the TADPhysODBCDriverLink component to link the ODBC bridge driver to an application.

Description

The uADPhysODBC unit contains:

- ODBC bridge driver implementation;
- TADPhysODBCDriverLink (see page 796) - component to link the driver.

See Also

Configuring Drivers (see page 31), Connect to ODBC data source (see page 213)

1.17.16.1 Classes

The following table lists classes in this documentation.

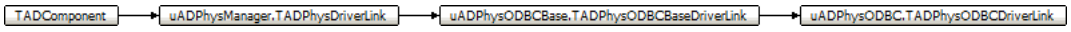
Classes

Class	Description
TADPhysODBCDriverLink (see page 796)	Use the TADPhysODBCDriverLink component to link the ODBC bridge driver to an application.

1.17.16.1.1 TADPhysODBCDriverLink Class

Use the TADPhysODBCDriverLink component to link the ODBC bridge driver to an application.

Class Hierarchy



File

uADPhysODBC

Description

Use the TADPhysODBCDriverLink component to link the ODBC bridge driver to an application. In general it is enough to only include uADPhysODBC unit into your application uses clause.

The TADPhysODBCDriverLink component may be used to specify:

- ODBCDriver (see page 799) - ODBC driver name;
- ODBCAdvanced (see page 799) - ODBC driver connection parameter common for all connections;
- VendorLib (see page 782) - ODBC driver manager. By default it is odbc32.dll and for most cases should remain unchanged.

All properties have default values appropriate for most case. To have an effect all property changes must be performed before first connection through this driver or use the Release (see page 780) method.

Syntax

```
TADPhysODBCDriverLink = class(TADPhysODBCBaseDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to ODBC data source (see page 213)

Example

```
ADPhysODBCDriverLink1.DriverID := 'Informix';
ADPhysODBCDriverLink1.ODBCAdvanced := 'CLOC=en_US.CP1252;DLOC=en_US.819';
ADPhysODBCDriverLink1.ODBCDriver := 'IBM INFORMIX ODBC DRIVER';
```

1.17.17 uADPhysODBCBase Namespace

Contains FireDAC driver and services base classes for all ODBC based drivers.

Classes

Class	Description
TADPhysODBCBaseDriverLink (see page 797)	The TADPhysODBCBaseDriverLink component is a base class for all ODBC based driver link components.
TADPhysODBCBaseService (see page 799)	The TADPhysODBCBaseService component is a base class for all ODBC based driver service components.

Description

The uADPhysODBCBase unit contains:

- FireDAC driver base implementation classes for all ODBC based drivers;
- TADPhysODBCBaseDriverLink (see page 797) - the base class for FireDAC ODBC based driver link components;
- TADPhysODBCBaseService (see page 799) - the base class for FireDAC ODBC based driver service components.

See Also

uADPhysODBC Namespace (see page 796)

1.17.17.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADPhysODBCBaseDriverLink (see page 797)	The TADPhysODBCBaseDriverLink component is a base class for all ODBC based driver link components.
TADPhysODBCBaseService (see page 799)	The TADPhysODBCBaseService component is a base class for all ODBC based driver service components.

1.17.17.1.1 TADPhysODBCBaseDriverLink Class

The TADPhysODBCBaseDriverLink component is a base class for all ODBC based driver link components.

public

public	Description
GetDrivers (see page 798)	Returns the list of ODBC driver names.
GetDSNs (see page 798)	Returns the list of ODBC data source names.

published

published	Description
ODBCAdvanced (see page 799)	The ODBC driver common parameters.

ODBCDriver (see page 799)	The ODBC driver name.

Class Hierarchy



File

uADPhysODBCBase

Description

The TADPhysODBCBaseDriverLink component is a base class for all ODBC based driver link components. It allows to configure a DBMS driver at runtime, using the ODBCAdvanced (see page 799) and ODBCDriver (see page 799) properties.

An application should not use the TADPhysODBCBaseDriverLink component directly. But it may use one of the many descendants, for example, TADPhysMSSQLDriverLink (see page 793) or TADPhysMSAccessDriverLink (see page 791).

Syntax

```
TADPhysODBCBaseDriverLink = class(TADPhysDriverLink);
```

1.17.17.1.1.1 public

1.17.17.1.1.1.1 TADPhysODBCBaseDriverLink.GetDrivers Method

Returns the list of ODBC driver names.

Parameters

Parameters	Description
AList: TStrings	A list to fill by the driver names.

Description

Use GetDrivers method to get the list of installed ODBC driver names.

Syntax

```
procedure GetDrivers(AList: TStrings);
```

See Also

GetDSNs (see page 798)

Example

```
ADPhysODBCDriverLink1.GetDrivers(Memo1.Lines);
```

1.17.17.1.1.1.2 TADPhysODBCBaseDriverLink.GetDSNs Method

Returns the list of ODBC data source names.

Parameters

Parameters	Description
AList: TStrings	A list to fill by the DSN names.
AWithDescription: Boolean = False	Set to True to return the list with descriptions.

Description

Use GetDSNs method to get the list of known ODBC data source names. Set AWithDescription to True to return the list with descriptions in format <DSN name>=<description>.

Syntax

```
procedure GetDSNs(AList: TStrings; AWithDescription: Boolean = False);
```

See Also

GetDrivers (see page 798)

Example

```
ADPhysODBCDriverLink1.GetDSNs(Memo1.Lines, False);
```

1.17.17.1.1.2 published

1.17.17.1.1.2.1 TADPhysODBCBaseDriverLink.ODBCAdvanced Property

The ODBC driver common parameters.

Description

Use the ODBCAdvanced to specify the ODBC driver parameters. The specified ODBC driver parameters will be used by all connections through this FireDAC driver. The parameters may be overridden at the connection level.

Syntax

```
property ODBCAdvanced: String;
```

See Also

Configuring Drivers (see page 31), ODBCDriver (see page 799)

1.17.17.1.1.2.2 TADPhysODBCBaseDriverLink.ODBCDriver Property

The ODBC driver name.

Description

Use the ODBCDriver to specify the ODBC driver name. Although application may assign any value to this property, only drivers of the same family, compatible with the DBMS driver must be assigned. Eg only SQL Server driver names must be assigned to TADPhysMSSQLDriverLink.ODBCDriver.

A ODBCDriver change will have an effect only before first connection through this driver. Also you can use the Release (see page 780) method.

The specified ODBC driver will be used by all connections through this FireDAC driver. It is impossible to specify a ODBC driver for a connection.

Syntax

```
property ODBCDriver: String;
```

See Also

Configuring Drivers (see page 31), ODBCAdvanced (see page 799), Release (see page 780)

1.17.17.1.2 TADPhysODBCBaseService Class

The TADPhysODBCBaseService component is a base class for all ODBC based driver service components.

Class Hierarchy



File

uADPhysODBCBase

Description

The TADPhysODBCBaseService component is a base class for all ODBC based driver link components.

An application should not use the TADPhysODBCBaseService component directly.

Syntax

```
TADPhysODBCBaseService = class(TADPhysDriverService);
```

1.17.18 uADPhysOracle Namespace

Contains Oracle Database driver and services components.

Classes

Class	Description
TADPhysOracleDriverLink (see page 800)	Use the TADPhysOracleDriverLink component to link the Oracle Database driver to an application and setup it.

Description

The uADPhysOracle unit contains:

- Oracle Database driver implementation;
- TADPhysOracleDriverLink (see page 800) - component to link the driver and setup CLI library.

See Also

Connect to Oracle Server (see page 215), Using Oracle with FireDAC (see page 149), Configuring Drivers (see page 31)

1.17.18.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADPhysOracleDriverLink (see page 800)	Use the TADPhysOracleDriverLink component to link the Oracle Database driver to an application and setup it.

1.17.18.1.1 TADPhysOracleDriverLink Class

Use the TADPhysOracleDriverLink component to link the Oracle Database driver to an application and setup it.

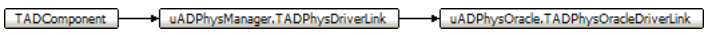
public

public	Description
GetOracleHomes (see page 801)	Returns the list of installed Oracle homes.
GetTNSServices (see page 801)	Returns the list of TNS aliases.

published

published	Description
NLSLang (see page 802)	Allows to specify the NLS_LANG environment variable value.
Threaded (see page 802)	Allows to specify OCI multi-threaded initialization mode.
TNSAdmin (see page 802)	Allows to specify the TNS_ADMIN environment variable value.

Class Hierarchy



File

uADPhysOracle

Description

Use the TADPhysOracleDriverLink component to link the Oracle Database driver to an application. In general it is enough to only include uADPhysOracle unit into your application uses clause.

The TADPhysOracleDriverLink component may be used to specify:

- VendorHome (see page 782) - the Oracle Home name;
- VendorLib (see page 782) - the name and optional path to the Oracle client library;
- NLSLang (see page 802) - the Oracle client NLS_LANG environment value;
- TNSAdmin (see page 802) - the Oracle client TNS_ADMIN environment value;
- Threaded (see page 802) - the OCI threading mode.

All properties have default values appropriate for most case. To have an effect all property changes must be performed before first connection through this driver or use the Release (see page 780) method.

Syntax

```
TADPhysOracleDriverLink = class(TADPhysDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to Oracle Server (see page 215), Using Oracle with FireDAC (see page 149)

Example

```
ADPhysOracleDriverLink1.VendorHome := 'Oracle11_Home';
```

1.17.18.1.1.1 public

1.17.18.1.1.1.1 TADPhysOracleDriverLink.GetOracleHomes Method

Returns the list of installed Oracle homes.

Parameters

Parameters	Description
AList: TStrings	A list of names.

Description

Use the GetOracleHomes method to get a list of installed Oracle Home names.

Syntax

```
class procedure GetOracleHomes(AList: TStrings);
```

See Also

GetTNSServices (see page 801)

Example

```
ADPhysOracleDriverLink1.GetOracleHomes(Memo1.Lines);
```

1.17.18.1.1.1.2 TADPhysOracleDriverLink.GetTNSServices Method

Returns the list of TNS aliases.

Parameters

Parameters	Description
AList: TStrings	A list of aliases.

Description

Use the GetTNSServices method to get a list of TNS aliases in current or specified Oracle Home.

Syntax

```
procedure GetTNSServices(AList: TStrings);
```

See Also

GetOracleHomes (see page 801)

Example

```
ADPhysOracleDriverLink1.GetTNSServices(Memo1.Lines);
```

1.17.18.1.1.2 published

1.17.18.1.1.2.1 TADPhysOracleDriverLink.NLSLang Property

Allows to specify the NLS_LANG environment variable value.

Description

Use the NLSLang property to specify the Oracle client NLS_LANG environment variable value.

When you have the Oracle full client installed, the best way to specify the NLS_LANG is to set the NLS_LANG registry value in HKLM\Software\Oracle\<Oracle home>.

When you are using Oracle instant client, the best way is to use NLSLang property. If you will leave it as is the NLS_LANG registry value in HKLM\Software\Oracle will be used. If it is empty, then AMERICAN_AMERICA.US7ASCII will be used.

Syntax

```
property NLSLang: String;
```

See Also

TNSAdmin (see page 802)

Example

```
ADPhysOracleDriverLink1.NLSLang := 'AMERICAN_AMERICA.UTF8';
```

1.17.18.1.1.2.2 TADPhysOracleDriverLink.Threaded Property

Allows to specify OCI multi-threaded initialization mode.

Description

Use Threaded property to specify should FireDAC initialize OCI in multi-threaded (True) or single-threaded (False) initialization mode. The default value is True.

Setting Threaded to False, when your application does not use connection pooling and has only single thread working with an Oracle database, will improve general data access performance.

Setting Threaded to True is required for when multiple threads will access to an Oracle database. This is equivalent to setting OCI_THREADED or OCI_NO_MUTEX in OCI environment initialization call.

Syntax

```
property Threaded: Boolean;
```

1.17.18.1.1.2.3 TADPhysOracleDriverLink.TNSAdmin Property

Allows to specify the TNS_ADMIN environment variable value.

Description

Use the TNSAdmin property to specify the Oracle client TNS_ADMIN environment variable value.

When you have the Oracle full client installed, the best way to specify the TNS_ADMIN is to set the TNS_ADMIN registry value in HKLM\Software\Oracle\<Oracle home>. Or just leave it as is, then the default path <Oracle home folder>\NETWORK\Admin is used.

When you are using Oracle instant client, the best way is to use TNSAdmin property. Or just leave it as is, because the default path is application EXE folder.

Syntax

```
property TNSAdmin: String;
```

See Also

NLSLang (see page 802)

Example

```
ADPhysOracleDriverLink1.TNSAdmin := 'c:\Program Files\MyApp\Config';
```

1.17.19 uADPhysPG Namespace

Contains PostgreSQL driver and services components.

Classes

Class	Description
TADPhysPgDriverLink (see page 803)	Use the TADPhysPgDriverLink component to link the PostgreSQL driver to an application and setup it.

Description

The uADPhysPG unit contains:

- PostgreSQL driver implementation;
- TADPhysPgDriverLink (see page 803) - component to link the driver and setup CLI library.

See Also

Configuring Drivers (see page 31), Connect to PostgreSQL (see page 218)

1.17.19.1 Classes

The following table lists classes in this documentation.

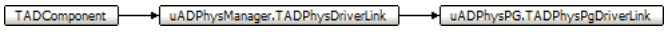
Classes

Class	Description
TADPhysPgDriverLink (see page 803)	Use the TADPhysPgDriverLink component to link the PostgreSQL driver to an application and setup it.

1.17.19.1.1 TADPhysPgDriverLink Class

Use the TADPhysPgDriverLink component to link the PostgreSQL driver to an application and setup it.

Class Hierarchy



File

uADPhysPG

Description

Use the TADPhysPgDriverLink component to link the PostgreSQL driver to an application. In general it is enough to only include uADPhysPG unit into your application uses clause.

The TADPhysPgDriverLink component may be used to specify:

- VendorHome (see page 782) - the PostgreSQL installation root folder;
- VendorLib (see page 782) - the name and optional path to the PostgreSQL client library.

All properties have default values appropriate for most case. To have an effect all property changes must be performed before first connection through this driver or use the Release (see page 780) method.

Syntax

```
TADPhysPgDriverLink = class(TADPhysDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to PostgreSQL (see page 218)

Example

```
ADPhysPgDriverLink1.VendorLib := 'E:\PostgreSQL\9.0\bin\libpq.dll';
```

1.17.20 uADPhysSQLite Namespace

Contains SQLite driver and services components.

Classes

Class	Description
TADPhysSQLiteDriverLink (see page 805)	Use the TADPhysSQLiteDriverLink component to link the SQLite driver to an application and setup it.
TADSQLiteBackup (see page 805)	The class implementing SQLite backup, restore, copy database functionality.
TADSQLiteCollation (see page 811)	The class implementing custom SQLite collation.
TADSQLiteFunction (see page 814)	The class implementing custom SQLite function.
TADSQLiteRTree (see page 816)	The class implementing custom SQLite RTree module.
TADSQLiteSecurity (see page 817)	The class allowing to manage SQLite database encryption.
TADSQLiteService (see page 821)	The base class for all SQLite service classes.
TADSQLiteValidate (see page 821)	The class implementing SQLite database validate service.

Description

The uADPhysSQLite unit contains:

- SQLite driver implementation;
- TADPhysSQLiteDriverLink (see page 805) - component to link the driver and setup CLI library for dynamic linking;
- TADSQLiteBackup (see page 805) - backup database component;
- TADSQLiteCollation (see page 811) - database custom collation setup component;
- TADSQLiteFunction (see page 814) - database custom function setup component;
- TADSQLiteRTree (see page 816) - database custom RTree module setup component;
- TADSQLiteValidate (see page 821) - DB validation service component;
- TADSQLiteSecurity (see page 817) - database encryption control component.

See Also

Connect to SQLite database (see page 221), Using SQLite with FireDAC (see page 135), Configuring Drivers (see page 31)

1.17.20.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADPhysSQLiteDriverLink (see page 805)	Use the TADPhysSQLiteDriverLink component to link the SQLite driver to an application and setup it.

TADSQLiteBackup (see page 805)	The class implementing SQLite backup, restore, copy database functionality.
TADSQLiteCollation (see page 811)	The class implementing custom SQLite collation.
TADSQLiteFunction (see page 814)	The class implementing custom SQLite function.
TADSQLiteRTree (see page 816)	The class implementing custom SQLite RTree module.
TADSQLiteSecurity (see page 817)	The class allowing to manage SQLite database encryption.
TADSQLiteService (see page 821)	The base class for all SQLite service classes.
TADSQLiteValidate (see page 821)	The class implementing SQLite database validate service.

1.17.20.1.1 TADPhysSQLiteDriverLink Class

Use the TADPhysSQLiteDriverLink component to link the SQLite driver to an application and setup it.

Class Hierarchy



File

uADPhysSQLite

Description

Use the TADPhysSQLiteDriverLink component to link the SQLite driver to an application. In general it is enough to only include uADPhysSQLite unit into your application uses clause.

The TADPhysSQLiteDriverLink component may be used to specify SQLite client library to the driver, when the dynamic linking is used. To have an effect all property changes must be performed before first connection through this driver or use the Release (see page 780) method.

Syntax

```
TADPhysSQLiteDriverLink = class(TADPhysDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to SQLite database (see page 221), Using SQLite with FireDAC (see page 135)

1.17.20.1.2 TADSQLiteBackup Class

The class implementing SQLite backup, restore, copy database functionality.

public

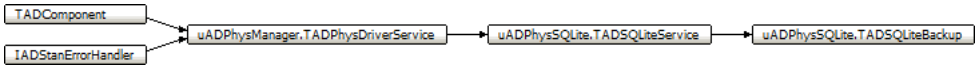
public	Description
DatabaseObj (see page 807)	A source database object.
DestDatabaseObj (see page 807)	A destination database object.
R PageCount (see page 807)	The total number of pages to copy.
R Remaining (see page 808)	The remaining number of pages to copy.
Backup (see page 808)	Executes the backup or copy operation.

published

published	Description
BusyTimeout (see page 808)	Controls the timeout of waiting for a locks to be released.
Catalog (see page 808)	The name of source attached database.
Database (see page 809)	A source database file name.
DestCatalog (see page 809)	The name of destination attached database.
DestDatabase (see page 809)	A destination database file name.
DestMode (see page 809)	The destination database open mode.

DestPassword (see page 809)	The password for destination database.
OnProgress (see page 810)	The event firing after each copying step.
PagesPerStep (see page 810)	Number of pages per single copy step.
Password (see page 810)	The password for source database.
WaitForLocks (see page 811)	Controls the waiting for a locks to be released.

Class Hierarchy



File

uADPhysSQLite

Description

Use the TADSQLiteBackup component to add a backup, restore, copy database capability to an application.

To produce a DB backup an application should:

- specify DriverLink (see page 821);
- specify Database (see page 809) or DatabaseObj (see page 807) - a source database file name or object to backup;
- specify DestDatabase (see page 809) or DestDatabaseObj (see page 807) - a destination database file name or object to put a database backup file;
- optionally specify WaitForLocks (see page 811) and BusyTimeout (see page 808) to avoid locking conflicts;
- call Backup (see page 808) method.

Other properties and methods are optional.

The SQLite backup actually copies the specified database to a new file. The copying is consistent and does not depend on the currently connected users.

Also the copying may be performed not only between files, but also between memory and file or backward. In this case use DatabaseObj or DestDatabaseObj to specify connected memory database.

Syntax

```
TADSQLiteBackup = class(TADSQLiteService);
```

Example 1

```
ADSQLiteBackup1.Database := 'c:\addemo.sdb';
ADSQLiteBackup1.DestDatabase := 'c:\addemo.backup';
ADSQLiteBackup1.Backup;
```

Example 2

```
// create SQLite in-memory database
with ADConnection1.Params do begin
  Clear;
  Add('DriverID=SQLite');
  Add('Database=:memory:');
end;
ADConnection1.Connected := True;

// populate the in-memory database
ADConnection1.ExecSQL('create table aaa (f1 integer, f2 text)');
ADConnection1.ExecSQL('insert into aaa values (1, ''qqq'')');
ADConnection1.ExecSQL('insert into aaa values (2, ''www'')');
ADConnection1.ExecSQL('insert into aaa values (3, ''eee'')');

// specify the in-memory database objects as a backup source
ADSQLiteBackup1.DatabaseObj := ADConnection1.CliObj;
// specify database file name as a backup destination
ADSQLiteBackup1.DestDatabase := 'c:\db.sdb';
// copy the in-memory database to the file
ADSQLiteBackup1.Backup;
```

1.17.20.1.2.1 public

1.17.20.1.2.1.1 TADSQLiteBackup.DatabaseObj Property

A source database object.

Description

Use the DatabaseObj property to specify the source SQLite database object of an API wrapping class. The object may be obtained from the TADConnection.CliObj (see page 323) property. There ADConnection must be connected to a SQLite database.

Note, when a source database is an in-memory database then DatabaseObj must be used instead of Database property.

Syntax

```
property DatabaseObj: TSQLiteDatabase;
```

See Also

Database (see page 809), DestDatabaseObj (see page 807)

Example

```
ADSQLiteBackup1.DatabaseObj := ADConnection1.CliObj;
ADSQLiteBackup1.DestDatabase := 'c:\backup.sdb';
ADSQLiteBackup1.Backup;
```

1.17.20.1.2.1.2 TADSQLiteBackup.DestDatabaseObj Property

A destination database object.

Description

Use the DestDatabaseObj property to specify the destination SQLite database object of an API wrapping class. The object may be obtained from the TADConnection.CliObj (see page 323) property. There ADConnection must be connected to a SQLite database.

Note, when a destination database is an in-memory database then DestDatabaseObj must be used instead of DestDatabase property.

Syntax

```
property DestDatabaseObj: TSQLiteDatabase;
```

See Also

DestDatabase (see page 809), Database (see page 809), BusyTimeout (see page 808), WaitForLocks (see page 811)

Example

```
ADSQLiteBackup1.Database := 'c:\db.sdb';
ADSQLiteBackup1.DestDatabaseObj := ADConnection1.CliObj;
ADSQLiteBackup1.Backup;
```

1.17.20.1.2.1.3 TADSQLiteBackup.PageCount Property

The total number of pages to copy.

Description

Use the PageCount property to get the total number of pages to copy.

Syntax

```
property PageCount: Integer;
```

See Also

Remaining (see page 808), OnProgress (see page 810)

1.17.20.1.2.1.4 TADSQLiteBackup.Remaining Property

The remaining number of pages to copy.

Description

Use the Remaining property to get the remaining number of pages to copy.

Syntax

```
property Remaining: Integer;
```

See Also

PageCount (see page 807), OnProgress (see page 810)

1.17.20.1.2.1.5 TADSQLiteBackup.Backup Method

Executes the backup or copy operation.

Description

Call the Backup method to run the copy operation from Database (see page 809) source database to the DestDatabase (see page 809) destination database. For feedback an application may use the OnProgress (see page 810) event.

Syntax

```
procedure Backup;
```

See Also

Database (see page 809), DestDatabase (see page 809), OnProgress (see page 810)

1.17.20.1.2.2 published

1.17.20.1.2.2.1 TADSQLiteBackup.BusyTimeout Property

Controls the timeout of waiting for a locks to be released.

Description

Use BusyTimeout property to specify the time in msecs that FireDAC should wait for a lock on source or destination database will be released. The property will be used together with WaitForLocks.

The default value is 10000.

Syntax

```
property BusyTimeout: Integer;
```

See Also

WaitForLocks (see page 811)

1.17.20.1.2.2.2 TADSQLiteBackup.Catalog Property

The name of source attached database.

Description

Use the Catalog to specify a name of source attached database.

Syntax

```
property Catalog: String;
```

See Also

DestCatalog (see page 809)

1.17.20.1.2.2.3 TADSQLiteBackup.Database Property

A source database file name.

Description

Use the Database property to specify the source SQLite database file name. The file name may include $(<variable>) markers, where *variable* is a name of environment variable.

Syntax

```
property Database: String;
```

See Also

DatabaseObj (see page 807), DestDatabase (see page 809)

1.17.20.1.2.2.4 TADSQLiteBackup.DestCatalog Property

The name of destination attached database.

Description

Use the DestCatalog to specify a name of destination attached database.

Syntax

```
property DestCatalog: String;
```

See Also

Catalog (see page 808), DestDatabase (see page 809)

1.17.20.1.2.2.5 TADSQLiteBackup.DestDatabase Property

A destination database file name.

Description

Use the DestDatabase property to specify the destination SQLite database file name. The file name may include $(<variable>) markers, where *variable* is a name of environment variable.

Syntax

```
property DestDatabase: String;
```

See Also

DestDatabaseObj (see page 807), Database (see page 809), BusyTimeout (see page 808), WaitForLocks (see page 811)

1.17.20.1.2.2.6 TADSQLiteBackup.DestMode Property

The destination database open mode.

Description

Use the DestMode to specify the destination database open mode.

The default value is smCreate, what means - create a new database if it does not exists, otherwise use an existing database.

Syntax

```
property DestMode: TSQLiteDatabaseMode;
```

See Also

DestDatabase (see page 809)

1.17.20.1.2.2.7 TADSQLiteBackup.DestPassword Property

The password for destination database.

Description

Use the DestPassword to specify an password for the destination database. The password may be different from the source database password.

You can specify the cipher algorithm prefix in the DestPassword property value using format: <prefix>:<password>.

Syntax

```
property DestPassword: String;
```

See Also

Connect to SQLite database (see page 221), Password (see page 810), DestDatabase (see page 809)

1.17.20.1.2.2.8 TADSQLiteBackup.OnProgress Event

The event firing after each copying step.

Description

Use the OnProgress event handler to provide the copy operation feedback.

For calculating the amount of work done, the event handler may use PageCount (see page 807) and Remaining (see page 808) properties.

Syntax

```
property OnProgress: TADPhysServiceProgressEvent;
```

See Also

PageCount (see page 807), Remaining (see page 808)

1.17.20.1.2.2.9 TADSQLiteBackup.PagesPerStep Property

Number of pages per single copy step.

Description

Use the PagesPerStep property to specify the number of pages, which will be copied per single copying operation step. After each step the OnProgress (see page 810) event is fired.

The default value is -1, what means all database pages.

Syntax

```
property PagesPerStep: Integer;
```

See Also

OnProgress (see page 810)

1.17.20.1.2.2.10 TADSQLiteBackup.Password Property

The password for source database.

Description

Use the Password to specify a password for the source database. The destination password may be different from the source database password.

You can specify the cipher algorithm prefix in the Password property value using format: <prefix>:<password>. See example.

Note, if a source database file was created with a non-default cipher algorithm (aes-256) by specifying:

- the Encrypt connection definition parameter,
- or a cipher algorithm prefix for the Password connection definition parameter,

then prefix must be specified, otherwise "Out of memory" exception will be raised.

Syntax

```
property Password: String;
```

See Also

Connect to SQLite database (see page 221), Database (see page 809), DestPassword (see page 809)

Example

```
// DB is encrypted using aes-ecb-128 cipher algorithm
ADConnection1.Params.Add('Password=aes-ecb-128:12345');
ADConnection1.Connected  := True;
....
// specify aes-ecb-128 cipher algorithm for source database
ADSQLiteBackup1.Password := 'aes-ecb-128:12345';
```

1.17.20.1.2.2.11 TADSQLiteBackup.WaitForLocks Property

Controls the waiting for a locks to be released.

Description

Use WaitForLocks property to specify should copying operation wait for a lock on source or destination database will be released. When True, then Backup (see page 808) method will wait. Use the BusyTimeout (see page 808) property to specify the timeout. When False, then an exception will be raised immediately after a lock will be found.

The default value is True.

Syntax

```
property WaitForLocks: Boolean;
```

See Also

BusyTimeout (see page 808)

1.17.20.1.3 TADSQLiteCollation Class

The class implementing custom SQLite collation.

published

published	Description
Active (see page 812)	Activates and registers a collation.
CollationKind (see page 812)	The collation implementation kind.
CollationName (see page 813)	The unique collation name.
Flags (see page 813)	The scCompareString collation kind collation flags.
Locale (see page 813)	The scCompareString collation kind collation.
OnCompare (see page 813)	The scCustomXxx compare strings event.

Class Hierarchy



File

uADPhysSQLite

Description

Use the TADSQLiteCollation to register a custom SQLite collation. A registered collation then may be used in any place in SQL command, where a collation may be specified. For example:

```
SELECT * FROM "Employees" ORDER BY LastName COLLATE UTF16NoCase
```

Each collation must have an unique name specified by CollationName (see page 813). And one of the three collation kinds specified as the additional properties:

- CollationKind (see page 812) = scCompareString. The collation will use Win32 / Win64 API CompareStringW. To setup it properties use the Locale and Flags properties.
- CollationKind (see page 812) = scCustomUTF8. To compare strings application should use OnCompare (see page 813) event, which will receive strings in UTF8 encoding.
- CollationKind (see page 812) = scCustomUTF16. Similar to scCustomUTF8, but strings will be in UTF16 encoding.

After setting up the collation it must be activated using Active (see page 812) property. To use a collation in a SQLite connection, application must activate the collation before opening a database connection.

Syntax

```
TADSQLiteCollation = class(TADSQLiteService);
```

See Also

Using SQLite with FireDAC (see page 135)

Example 1

```
ADSQLiteCollation1.DriverLink := ADPhysSQLiteDriverLink1;
ADSQLiteCollation1.CollationName := 'UTF16NoCase';
ADSQLiteCollation1.CollationKind := scCompareString;
ADSQLiteCollation1.Flags := [sfIgnoreCase];
ADSQLiteCollation1.Active := True;
```

Example 2

See <FireDAC>\Samples\DBMS Specific\SQLite\UserCollation

1.17.20.1.3.1 published

1.17.20.1.3.1.1 TADSQLiteCollation.Active Property

Activates and registers a collation.

Description

Use the Active property to activate the collation and register it at with SQLite engine.

All active connections may use this collation. Setting Active to False will unregister the collation. Note, that SQLite actually has problems with collation unregistration. So, in general, leave the collation registered until application finish.

Changing any of the collation properties will set Active to False.

Syntax

```
property Active;
```

1.17.20.1.3.1.2 TADSQLiteCollation.CollationKind Property

The collation implementation kind.

Description

Use the CollationKind property to specify the collation implementation kind.

In general, the collation may use the Win32 / Win64 API CompareStringW or the application OnCompare (see page 813) event handler. The OnCompare (see page 813) event handler may receive as strings UTF8 encoded, as UTF16 encoded. The default value is scCompareString.

For scCompareString collations, the application should specify Flags (see page 813) and Locale (see page 813) properties. For scCustomUTF8 and scCustomUTF16, the application should implement the OnCompare (see page 813) event handler.

Kind	Description
scCompareString	The collation will use Win32 / Win64 API CompareStringW. The application should specify Flags (see page 813) and Locale (see page 813) properties.

scCustomUTF8	The collation will use OnCompare (see page 813) event handler. The strings in the event handler will be UTF8 encoded.
scCustomUTF16	The collation will use OnCompare (see page 813) event handler. The strings in the event handler will be UTF16 encoded.

Syntax

```
property CollationKind: TADSQLiteCollationKind;
```

See Also

Flags (see page 813), Locale (see page 813), OnCompare (see page 813)

1.17.20.1.3.1.3 TADSQLiteCollation.CollationName Property

The unique collation name.

Description

Use the CollationName to specify the collation name. It must be unique across the application. The collation name must follow to the SQLite identifier rules.

Syntax

```
property CollationName: String;
```

See Also

CollationKind (see page 812)

1.17.20.1.3.1.4 TADSQLiteCollation.Flags Property

The scCompareString collation kind collation flags.

Description

Use the Flags property to specify the flags for the CollationKind (see page 812) = scCompareString. The meaning of each flag may be derived from the CompareStringW MSDN documentation, dwCmpFlags parameter description.

Syntax

```
property Flags: TADSQLiteCollationFlags;
```

See Also

Locale (see page 813)

1.17.20.1.3.1.5 TADSQLiteCollation.Locale Property

The scCompareString collation kind collation.

Description

Use the Locale property to specify the locale for the CollationKind (see page 812) = scCompareString. The meaning of this property may be derived from the CompareStringW MSDN documentation, Collate parameter description.

Syntax

```
property Locale: TADLocalID;
```

See Also

Flags (see page 813)

1.17.20.1.3.1.6 TADSQLiteCollation.OnCompare Property

The scCustomXxx compare strings event.

Description

Use the OnCompare event handler to compare strings for the CollationKind (see page 812) = scCompareUTF8 or UTF16. The event handler will receive the strings in the specified by CollationKind (see page 812) encodings, should compare them, and return a comparison result in AResult.

Syntax

```
property OnCompare: TSQLiteCollationEvent;
```

See Also

CollationKind (see page 812)

1.17.20.1.4 TADSQLiteFunction Class

The class implementing custom SQLite function.

published

published	Description
Active (see page 815)	Activates and registers a function.
Aggregated (see page 815)	Controls the function mode.
ArgumentsCount (see page 815)	The number of formal arguments the function is awaiting.
FunctionName (see page 815)	The unique function name.
OnCalculate (see page 816)	The calculate function event.

Class Hierarchy



File

uADPhysSQLite

Description

Use the TADSQLiteFunction to register a custom SQLite function. A registered function then may be used in any place in SQL command, where an expression may be used.

Syntax

```
TADSQLiteFunction = class(TADSQLiteService);
```

See Also

Using SQLite with FireDAC (see page 135)

Example 1

```
ADSQLiteFunction1.DriverLink := ADPhysSQLiteDriverLink1;
ADSQLiteFunction1.FunctionName := 'MyFunc';
ADSQLiteFunction1.ArgumentsCount := 2;
ADSQLiteFunction1.Active := True;
ADSQLiteFunction1.OnCalculate := ADSQLiteFunction1Calculate;

procedure TForm1.ADSQLiteFunction1Calculate(AFunc: TSQLiteFunction;
  AInputs: TSQLiteInputs; AOutput: TSQLiteOutput; var AUserData: TObject);
begin
  AOutput.AsInteger := AInputs[0].AsInteger * AInputs[1].AsInteger;
end;

procedure TForm1.Button1Click(Sender: TObject);
begin
  // demo query
  ADQuery1.Open('select RegionID, 'MyFunc'(RegionID, 10) from "Region"');
end;
```

Example 2

See <FireDAC>\Samples\DBMS Specific\SQLite\UserFunction

1.17.20.1.4.1 published

1.17.20.1.4.1.1 TADSQLiteFunction.Active Property

Activates and registers a function.

Description

Use the Active property to activate the function and register it with SQLite engine.

All active connections may use this function. Setting Active to False will unregister the function. Note, that SQLite actually has problems with function un-registration. So, in general, leave the function registered until application finish.

Changing any of the function properties will set Active to False.

Syntax

```
property Active;
```

1.17.20.1.4.1.2 TADSQLiteFunction.Aggregated Property

Controls the function mode.

Description

Use the Aggregated property to specify is the function scalar or aggregated one. The default value is False.

Syntax

```
property Aggregated: Boolean;
```

1.17.20.1.4.1.3 TADSQLiteFunction.ArgumentsCount Property

The number of formal arguments the function is awaiting.

Description

Use the ArgumentsCount to specify the number of arguments, which the function will receive. The arguments are unnamed and untyped. Important is the number of arguments only.

To create a custom function with the default or different number of arguments you will need to specify the same FunctionName and different number of arguments. That will register overloaded functions inside the SQLite engine.

Syntax

```
property ArgumentsCount: Integer;
```

See Also

FunctionaName, OnCalculate (see page 816)

1.17.20.1.4.1.4 TADSQLiteFunction.FunctionName Property

The unique function name.

Description

Use the FunctionName to specify the function name. The function name must follow to the SQLite identifier rules.

To create a custom function with the default or different number of arguments you will need to specify the same FunctionName and different number of arguments. That will register overloaded functions inside the SQLite engine.

Syntax

```
property FunctionName: String;
```

See Also

ArgumentCount, OnCalculate (see page 816)

1.17.20.1.4.1.5 TADSQLiteFunction.OnCalculate Property

The calculate function event.

Description

Use the OnCalculate event handler to calculate the function result. The event handler will receive an array of arguments in AInputs parameter. The event handler should return a result using the AOutput parameter.

Syntax

```
property OnCalculate: TSQLiteFunctionCalculateEvent;
```

See Also

ArgumentsCount (see page 815), OnFinalize

1.17.20.1.5 TADSQLiteRTree Class

The class implementing custom SQLite RTree module.

published

published	Description
Active (see page 816)	Activates and registers a RTree.
OnCalculate (see page 817)	The RTree match event.
RTreeName (see page 817)	The unique RTree module name.

Class Hierarchy



File

uADPhysSQLite

Description

Use the TADSQLiteRTree to register a custom SQLite RTree module. A registered RTree then may be used MATCH SQL operator (more).

Syntax

```
TADSQLiteRTree = class(TADSQLiteService);
```

See Also

Using SQLite with FireDAC (see page 135)

1.17.20.1.5.1 published

1.17.20.1.5.1.1 TADSQLiteRTree.Active Property

Activates and registers a RTree.

Description

Use the Active property to activate the RTree and register it with SQLite engine.

All active connections may use this RTree. Setting Active to False will unregister the RTree.

Changing any of the RTree properties will set Active to False.

Syntax

```
property Active;
```

1.17.20.1.5.1.2 TADSQLiteRTree.OnCalculate Property

The RTree match event.

Description

Use the OnCalculate event handler to match the coordinates to the RTree MATCH operator. The event handler will receive an array of parameters in AParams parameter and record coordinates in AColumns parameter. The event handler should return a result using the AResult parameter.

Syntax

```
property OnCalculate: TSQLiteRTreeCalculateEvent;
```

See Also

OnFinalize

1.17.20.1.5.1.3 TADSQLiteRTree.RTreeName Property

The unique RTree module name.

Description

Use the RTreeName to specify the RTree module name. The RTree name must follow to the SQLite identifier rules.

Syntax

```
property RTreeName: String;
```

See Also

OnCalculate (see page 817)

1.17.20.1.6 TADSQLiteSecurity Class

The class allowing to manage SQLite database encryption.

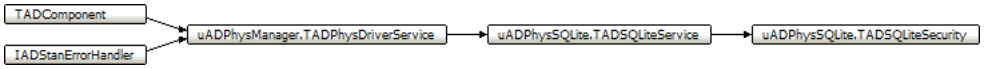
public

public	Description
ChangePassword (see page 818)	Changes the password for encrypted database.
CheckEncryption (see page 819)	Returns the database encryption status.
RemovePassword (see page 819)	Removes the password and decrypts the database.
SetPassword (see page 819)	Adds the password and encrypts the database.

published

published	Description
Database (see page 820)	A database file name.
Options (see page 820)	The encryption options.
Password (see page 820)	The database password.
ToPassword (see page 821)	The database password change to.

Class Hierarchy



File

uADPhysSQLite

Description

Use the TADSQLiteSecurity component to add a SQLite database security management capabilities to an application.

To manage a security an application should:

- specify DriverLink (see page 821);
- specify Database (see page 820) and Password (see page 820);
- optionally specify ToPassword (see page 821);
- optionally include soSetLargeCache into Options (see page 820);
- call SetPassword (see page 819) / RemovePassword (see page 819) / ChangePassword (see page 818) / CheckEncryption (see page 819) method.

Syntax

```
TADSQLiteSecurity = class(TADSQLiteService);
```

See Also

Using SQLite with FireDAC (see page 135)

Example 1

Sets password and encrypts the unencrypted database:

```
ADSQLiteSecurity1.DriverLink := ADPhysSQLiteDriverLink1;
ADSQLiteSecurity1.Database := 'c:\addemo.sdb';
ADSQLiteSecurity1.Password := '12345';
ADSQLiteSecurity1.SetPassword;
```

Example 2

Decrypts the encrypted database:

```
ADSQLiteSecurity1.DriverLink := ADPhysSQLiteDriverLink1;
ADSQLiteSecurity1.Database := 'c:\addemo.sdb';
ADSQLiteSecurity1.Password := '12345';
ADSQLiteSecurity1.RemovePassword;
```

Example 3

Changes the encrypted database password:

```
ADSQLiteSecurity1.DriverLink := ADPhysSQLiteDriverLink1;
ADSQLiteSecurity1.Database := 'c:\addemo.sdb';
ADSQLiteSecurity1.Password := '12345';
ADSQLiteSecurity1.ToPassword := 'qwerty';
ADSQLiteSecurity1.ChangePassword;
```

1.17.20.1.6.1 public

1.17.20.1.6.1.1 TADSQLiteSecurity.ChangePassword Method

Changes the password for encrypted database.

Description

Use the ChangePassword method to change a password for an encrypted database from Password (see page 820) to ToPassword (see page 821). To perform the operation both properties must be specified.

See SetPassword (see page 819) article for additional considerations.

Syntax

```
procedure ChangePassword;
```

See Also

Database (see page 820), Password (see page 820), ToPassword (see page 821), Options (see page 820), SetPassword (see page 819)

1.17.20.1.6.1.2 TADSQLiteSecurity.CheckEncryption Method

Returns the database encryption status.

Description

Use the CheckEncryption method to get a database status. To perform the operation Database (see page 820) and optionally Password (see page 820) must be specified. The Password (see page 820) must be specified if a database is encrypted. Function returns:

Value	Description
<unencrypted>	A database is not encrypted.
One of the encryption modes. For example: aes-256.	A database is encrypted and correct password is specified.
<encrypted>	• A database is probably encrypted, but the specified password is not correct. • A file is not a SQLite database.
empty string	Some other error happened at encryption checking.

Syntax

```
function CheckEncryption: String;
```

See Also

Database (see page 820), Password (see page 820)

1.17.20.1.6.1.3 TADSQLiteSecurity.RemovePassword Method

Removes the password and decrypts the database.

Description

Use the RemovePassword method to remove a password and decrypt an encrypted database. To perform operation the Password (see page 820) must be specified.

See SetPassword (see page 819) article for additional considerations.

Syntax

```
procedure RemovePassword;
```

See Also

Database (see page 820), Password (see page 820), Options (see page 820), SetPassword (see page 819)

1.17.20.1.6.1.4 TADSQLiteSecurity.SetPassword Method

Adds the password and encrypts the database.

Description

Use the SetPassword method to add a password and encrypt an unencrypted database. To perform operation the Password (see page 820) must be specified.

That is good practice before the SetPassword call to perform:

- backup the original database file, using TADSQLiteBackup (see page 805) or "by hands";
- verify the original database file, using TADSQLiteValidate.CheckOnly (see page 823). Do not encrypt database, if there are errors.

Note, due to current SQLite encryption limitation SetPassword call will fail, when database has blob fields with value size greater than 1 DB page, and database does not fit into SQLite cache. In this case, further access to encrypted database will return "database disk image is malformed" error. In this case:

- restore the original database from backup;

- include soSetLargeCache into Options (see page 820);
- perform SetPassword.

Note, the same considerations apply to ChangePassword and RemovePassword calls.

Syntax

```
procedure SetPassword;
```

See Also

Database (see page 820), Password (see page 820), Options (see page 820)

1.17.20.1.6.2 published

1.17.20.1.6.2.1 TADSQLiteSecurity.Database Property

A database file name.

Description

Use the Database property to specify the SQLite database file name. The file name may include $(<variable>) markers, where *variable* is a name of environment variable.

Syntax

```
property Database: String;
```

See Also

List here...

1.17.20.1.6.2.2 TADSQLiteSecurity.Options Property

The encryption options.

Description

Use the Options property to specify the database encryption options.

Option	Descryption
soSetLargeCache	Due to current SQLite encryption limitation SetPassword / ChangePassword / RemovePassword calls will fail, when database has blob fields with value size greater than 1 DB page, and database does not fit into SQLite cache. When soSetLargeCache is set, then SetPassword / ChangePassword / RemovePassword will automatically set cache size greater than DB size, to fit database into memory in full. When DB size is greater than accessible system memory, then corresponding call will fail.

Syntax

```
property Options: TADSQLiteSecurityOptions;
```

See Also

SetPassword (see page 819), ChangePassword (see page 818), RemovePassword (see page 819)

1.17.20.1.6.2.3 TADSQLiteSecurity.Password Property

The database password.

Description

Use the Password to specify a current or initial password for the encrypt / decrypt operations.

Syntax

```
property Password: String;
```

See Also

ToPassword (see page 821)

1.17.20.1.6.2.4 TADSQLiteSecurity.ToPassword Property

The database password change to.

Description

Use the ToPassword to specify a new password for the ChangePassword (see page 818) operation.

Syntax

```
property ToPassword: String;
```

See Also

ChangePassword (see page 818)

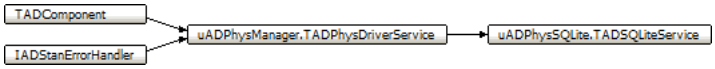
1.17.20.1.7 TADSQLiteService Class

The base class for all SQLite service classes.

published

published	Description
DriverLink (see page 821)	Specifies the DriverLink.

Class Hierarchy



File

uADPhysSQLite

Description

The TADSQLiteService class is used as a base class for all SQLite service classes and should not be used directly. The class contains the common properties, like the DriverLink (see page 821).

Syntax

```
TADSQLiteService = class(TADPhysDriverService);
```

1.17.20.1.7.1 published

1.17.20.1.7.1.1 TADSQLiteService.DriverLink Property

Specifies the DriverLink.

Description

Use the DriverLink property to specify the driver link. This property must be specified for any service operation.

Syntax

```
property DriverLink: TADPhysSQLiteDriverLink;
```

1.17.20.1.8 TADSQLiteValidate Class

The class implementing SQLite database validate service.

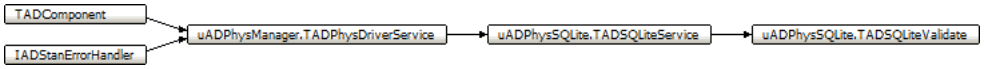
public

public	Description
Analyze (see page 823)	Starts the statistics gathering task.
CheckOnly (see page 823)	Starts the read-only validate database task.
Sweep (see page 823)	Starts the sweep task.

published

published	Description
Database (see page 824)	Specifies a database file to validate.
MaxErrors (see page 824)	Specifies the maximum number of errors.
OnProgress (see page 824)	Specifies the event handler allowing to handle the SQLite feedback.
Options (see page 824)	Specifies validate options.
Password (see page 824)	Specifies a password to database to validate.

Class Hierarchy



File

uADPhysSQLite

Description

Use the TADSQLiteValidate component to add a database validate capability to an application. This is a programmatic method to invoke the specific SQLite PRAGMA and other commands.

To validate a database an application should:

- specify DriverLink (see page 821);
- specify Database (see page 824) - a database file to validate;
- call Analyze (see page 823), CheckOnly (see page 823) or Sweep (see page 823) method.

To produce a DB validate log an application should use OnProgress (see page 824) event. We recommend to use TADSQLiteValidate, when there is no active connections to a database file.

Syntax

```
TADSQLiteValidate = class(TADSQLiteService);
```

Example

Validate database:

```
procedure TForm1.ADSQLiteValidate1Progress(ASender: TADPhysDriverService; const AMessage:
String);
begin
  Memo1.Lines.Add(AMessage);
end;

procedure TForm1.Button1Click(Sender: TObject);
begin
  ADSQLiteValidate1.DriverLink := ADPhysSQLiteDriverLink1;
  ADSQLiteValidate1.Database := 'c:\addemo.sq3';
  ADSQLiteValidate1.OnProgress := Form1Progress;
  Memo1.Lines.Clear;
  if not ADSQLiteValidate1.CheckOnly then
    ShowMessage('Database has problems ! See log for details.')
  else
    ShowMessage('Database is valid');
end;
```

1.17.20.1.8.1 public

1.17.20.1.8.1.1 TADSQLiteValidate.Analyze Method

Starts the statistics gathering task.

Parameters

Parameters	Description
const ATable: String = ''	A database table name.
const AIndex: String = ''	A database index name.

Description

Use the Analyze method to start the database statistics gathering task.

To perform a statistics gathering task an application must specify the connection parameters (see page 821) and Database (see page 824). This is a programmatic method to invoke the ANALYZE command. Depending on the specified arguments the method modes are:

- AIndex only - analyzes the specified index.
- ATable only - analyzes all indexes for the specified table.
- None specified - analyzes all indexes for all tables in the database.

Syntax

```
procedure Analyze(const ATable: String = ''; const AIndex: String = '');
```

See Also

CheckOnly (see page 823), Sweep (see page 823)

1.17.20.1.8.1.2 TADSQLiteValidate.CheckOnly Method

Starts the read-only validate database task.

Description

Use the CheckOnly method to start the read-only validate database task.

To perform a validation task an application must specify the connection parameters (see page 821) and Database (see page 824). This is a programmatic method to invoke the PRAGMA integrity_check command. CheckOnly method returns True, when database has no errors or warning, otherwise - False. To receive the found issues use the OnProgress (see page 824) event handler.

Syntax

```
function CheckOnly: Boolean;
```

See Also

Analyze (see page 823), Sweep (see page 823), OnProgress (see page 824)

1.17.20.1.8.1.3 TADSQLiteValidate.Sweep Method

Starts the sweep task.

Description

Use the Sweep method to start the database sweep task.

To perform a sweep task an application must specify the connection parameters (see page 821) and Database (see page 824). This is a programmatic method to invoke the VACUUM command.

Syntax

```
procedure Sweep;
```

See Also

Analyze (see page 823), CheckOnly (see page 823)

1.17.20.1.8.2 published

1.17.20.1.8.2.1 TADSQLiteValidate.Database Property

Specifies a database file to validate.

Description

Use the Database property to specify a path of the database file to validate.

Syntax

```
property Database: String;
```

1.17.20.1.8.2.2 TADSQLiteValidate.MaxErrors Property

Specifies the maximum number of errors.

Description

Use the MaxErrors property to specify maximum number of errors for CheckOnly (see page 823) method. If database validation task will discover more errors, then validation will stop. The default value is -1, what means unlimited number of errors.

Syntax

```
property MaxErrors: Integer;
```

See Also

CheckOnly (see page 823)

1.17.20.1.8.2.3 TADSQLiteValidate.OnProgress Event

Specifies the event handler allowing to handle the SQLite feedback.

Description

Use the OnProgress event handler to handle the SQLite feedback, produced by the CheckOnly (see page 823) method.

Syntax

```
property OnProgress: TADPhysServiceProgressEvent;
```

1.17.20.1.8.2.4 TADSQLiteValidate.Options Property

Specifies validate options.

Description

Use the Options property to specify additional options controlling the validate task. The default value is [voCheckIndexes]. The Options are used only by CheckOnly (see page 823) method.

Option	Description
voCheckIndexes	When included, then CheckOnly (see page 823) performs also validation of indexes for matching to the table datas. That corresponds to PRAGMA integrity_check. When excluded that corresponds to PRAGMA quick_check.

Syntax

```
property Options: TADSQLiteValidateOptions;
```

See Also

CheckOnly (see page 823)

1.17.20.1.8.2.5 TADSQLiteValidate.Password Property

Specifies a password to database to validate.

Description

Use the Password property to specify a database password. The password will be used for all operations with encrypted database.

Syntax

```
property Password: String;
```

1.17.21 uADPhysSQLiteVDataSet Namespace

Contains SQLite based Local SQL engine.

Classes

Class	Description
TADLocalSQL (see page 825)	The class implementing the SQLite based Local SQL engine.

Description

The uADPhysSQLiteVDataSet unit contains:

- SQLite based Local SQL (see page 96) engine implementation;
- TADLocalSQL (see page 825) - component representing Local SQL API.

See Also

Local SQL (see page 96)

1.17.21.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADLocalSQL (see page 825)	The class implementing the SQLite based Local SQL engine.

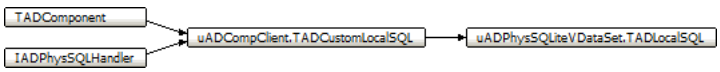
1.17.21.1.1 TADLocalSQL Class

The class implementing the SQLite based Local SQL engine.

published

published	Description
Active (see page 826)	Activates Local SQL engine.
Connection (see page 826)	Specifies the FireDAC connection component to use.
DataSets (see page 827)	Collection of the registered datasets.
DisableControls (see page 827)	Controls dataset DisableControls / EnableControls calls.
IncludeHiddenFields (see page 827)	Controls inclusion of invisible and hidden fields into Local SQL result sets.
MultipleCursors (see page 828)	Controls dataset cloning and copying.
OnGetDataSet (see page 828)	The event firing when Local SQL engine looks for a dataset object.
OnReadDataSet (see page 828)	The event firing when Local SQL is going to read a dataset object.
SchemaName (see page 829)	Set schema name for datasets.

Class Hierarchy



File

uADPhysSQLiteVDataSet

Description

The TADLocalSQL class implements SQLite based Local SQL engine (see page 96).

The Connection (see page 364) property points to a SQLite connection object which will provide ability to execute the Local SQL commands. To activate engine set Active to True.

FireDAC datasets may be registered with the engine by setting their LocalSQL (see page 261) properties. FireDAC and Non-FireDAC datasets may be registered with the engine using TADLocalSQL.DataSets (see page 364) collection property. To remove dataset delete corresponding collection item. To dynamically discover and register datasets use OnGetDataSet (see page 365) event handler.

To group dataset objects into a schema set SchemaName (see page 829) property. To disable dataset cloning or copying set MultipleCursors (see page 828) property to False. To enable events of a TDataSource associated with a base dataset set DisableControls (see page 827) to False. To exclude invisible fields from Local SQL result sets set IncludeHiddenFields (see page 827) to False.

Syntax

```
TADLocalSQL = class(TADCustomLocalSQL);
```

See Also

Local SQL (see page 96), Executing Command (see page 66)

Example

```
ADConnection1.DriverName := 'SQLite';
ADLocalSQL1.Connection := ADConnection1;
ADLocalSQL1.SchemaName := 'orders';
ADLocalSQL1.DataSets.Add.DataSet := adoQOrders;
ADLocalSQL1.DataSets.Add.DataSet := qDetails;

ADQuery1.Connection := ADConnection1;
ADQuery1.Open('select * from orders.adoQOrders o left join orders.qDetails d on o.OrderID =
d.OrderID');
```

1.17.21.1.1.1 published

1.17.21.1.1.1.1 TADLocalSQL.Active Property

Activates Local SQL engine.

Description

Set Active property to True to activate Local SQL engine and add registered datasets to Local SQL namespace. Set Active to False to remove datasets from Local SQL namespace and deactivate Local SQL engine. Local SQL queries may be performed only when Active is True and connection is active too.

When application needs to perform massive Local SQL configuration tasks, like to add a set of datasets, remove set of datasets, etc, then it is recommended first set Active to False. And when a task is finished set Active to True.

Active may be set to False only when there is no active / prepared SQL commands using this Local SQL.

Syntax

```
property Active: Boolean;
```

See Also

Local SQL (see page 96), Connection

1.17.21.1.1.1.2 TADLocalSQL.Connection Property

Specifies the FireDAC connection component to use.

Description

Use Connection to specify an FireDAC connection connected to a SQLite database. This connection will be used to publish registered datasets and execute Local SQL commands.

Syntax

```
property Connection: TADCustomConnection;
```

See Also

Local SQL (see page 96), Active

1.17.21.1.1.1.3 TADLocalSQL.DataSets Property

Collection of the registered datasets.

Description

Use DataSets collection property to manage the datasets registered with this Local SQL engine.

FireDAC datasets may be added to the collection implicitly by setting TADAdaptedDataSet.LocalSQL (see page 261) property. Non-FireDAC and FireDAC datasets may be added explicitly by adding new items to DataSets collection. To remove a dataset from the list of registered datasets, remove corresponding item from the DataSets collection. The datasets may be added or removed as at design time, as at run time.

All datasets in collection must have unique names. When item Name (see page 427) property is specified, then it is used as a dataset name. If not specified, then TDataSet.Name value will be used.

Syntax

```
property DataSets: TADLocalSQLDataSets;
```

See Also

Local SQL (see page 96), TADAdaptedDataSet.LocalSQL (see page 261), TADLocalSQLDataSets, TADLocalSQLDataSet

Example

```
ADLocalSQL1.DataSets.Add.DataSet := ADOQuery1;
ADLocalSQL1.DataSets.Add.DataSet := ADOQuery2;
ADQuery1.LocalSQL := ADLocalSQL1;
ADQuery2.LocalSQL := ADLocalSQL1;
```

1.17.21.1.1.1.4 TADLocalSQL.DisableControls Property

Controls dataset DisableControls / EnableControls calls.

Description

Use DisableControls property to control the dataset DisableControls / EnableControls calls.

When DisableControls is True, then FireDAC will call DisableControls before creating a cursor for the dataset. And will call EnableControls, when the cursor will be released.

Syntax

```
property DisableControls: Boolean;
```

1.17.21.1.1.1.5 TADLocalSQL.IncludeHiddenFields Property

Controls inclusion of invisible and hidden fields into Local SQL result sets.

Description

Use IncludeHiddenFields property to control the inclusion of the base dataset fields with TField.Visible = False or pfHidden in TField.ProviderFlags into the Local SQL result sets.

When it is True, then such fields will be included into the result sets in similar way to other fields. When it is False, then the invisible fields will be:

- not listed in the dataset returned by "PRAGMA table_info";
- not included in the expansion of a "*" expression in the result set of a SELECT;
- not included in the implicit column-list used by an INSERT statement that lacks an explicit column-list.

Syntax

```
property IncludeHiddenFields: Boolean;
```

1.17.21.1.1.1.6 TADLocalSQL.MultipleCursors Property

Controls dataset cloning and copying.

Description

Use MultipleCursors property to control the dataset cloning and copying.

SQLite creates a cursor to handle a request to a specific dataset for each reference in a Local SQL command. Because TDataSet API does not allow to navigate through TDataSet simultaneously in different cursors, FireDAC clones an FireDAC dataset using TADMemTable, and copies a non-FireDAC dataset into TADMemTable. Then this TADMemTable is used to implement the cursor. The copying is time consuming operation.

When developer is sure, that each dataset registered with Local SQL engine will be used not more than by a single cursor in each moment of time, then to avoid copying / cloning set MultipleCursors to False. When MultipleCursors is False and Local SQL engine needs additional cursors to the same dataset, then exception will be raised.

Note, due to SQLite engine limitations, an join operation with an unidirectional dataset, may lead to bad performance when MultipleCursors is False.

Syntax

```
property MultipleCursors: Boolean;
```

1.17.21.1.1.1.7 TADLocalSQL.OnGetDataSet Event

The event firing when Local SQL engine looks for a dataset object.

Description

Use OnGetDataSet event handler to submit a dataset object by its name.

This event may be used to provide a dataset dynamically without registering it first with the Local SQL engine. The value of AName argument and ADataSet.Name may be different. After returning a dataset, it will be registered in DataSets (see page 364) collection under AName argument value.

The event handler must return ADataSet, AAdapter or both arguments assigned. For FireDAC datasets the ADataSet may be assigned only. For non-FireDAC datasets is recommended to assign both arguments.

Syntax

```
property OnGetDataSet: TADGetDatasetEvent;
```

See Also

Local SQL (see page 96), DataSets, OnReadDataSet

1.17.21.1.1.1.8 TADLocalSQL.OnReadDataSet Event

The event firing when Local SQL is going to read a dataset object.

Description

Use OnReadDataSet event handler to setup a dataset object before it will be read.

This event may be used to set parameter or macro values. Or even to change the SQL command text. If a dataset is inactive, then Local SQL will activate it right after the event handler call.

Syntax

```
property OnReadDataSet: TADReadDatasetEvent;
```

See Also

Local SQL (see page 96), OnGetDataSet

1.17.21.1.1.9 TADLocalSQL.SchemaName Property

Set schema name for datasets.

Description

Use SchemaName property to set a schema name to logically group datasets registered with this engine into a namespace.

The schema names of all TADLocalSQL's connected to the same connection object and all attached database names in the same connection must be unique. FireDAC does not control the uniqueness.

When SchemaName is specified, then the dataset may be referenced in a SQL as <dataset> or <schema>.<dataset>. If the specified <schema> does not exist, then exception will be raised.

Syntax

```
property SchemaName: String;
```

1.17.22 uADPhysTDBX Namespace

Contains dbExpress v 4 bridge driver.

Classes

Class	Description
TADPhysTDBXDriverLink (see page 829)	Use the TADPhysTDBXDriverLink component to link the dbExpress v 4 bridge driver to an application.

Description

The uADPhysTDBX unit contains:

- dbExpress v 4 bridge driver implementation;
- TADPhysTDBXDriverLink (see page 829) - component to link the driver.

See Also

Configuring Drivers (see page 31), Connect to dbExpress data source (see page 194)

1.17.22.1 Classes

The following table lists classes in this documentation.

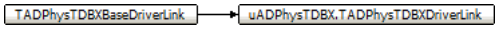
Classes

Class	Description
TADPhysTDBXDriverLink (see page 829)	Use the TADPhysTDBXDriverLink component to link the dbExpress v 4 bridge driver to an application.

1.17.22.1.1 TADPhysTDBXDriverLink Class

Use the TADPhysTDBXDriverLink component to link the dbExpress v 4 bridge driver to an application.

Class Hierarchy



File

uADPhysTDBX

Description

Use the TADPhysTDBXDriverLink component to link the dbExpress v 4 bridge driver to an application. In general it is enough to only include uADPhysTDBX unit into your application uses clause.

TADPhysTDBXDriverLink does not use VendorLib (see page 782) and VendorHome (see page 782) properties in the current FireDAC version.

Syntax

```
TADPhysTDBXDriverLink = class(TADPhysTDBXBaseDriverLink);
```

See Also

Configuring Drivers (see page 31), Connect to dbExpress data source (see page 194)

1.17.23 uADPhysTDBXBase Namespace

Contains FireDAC driver base classes for all TDBX based drivers.

Classes

Class	Description
ETDBXNativeException (see page 830)	Represents exception class for all TDBX based drivers.

Description

The uADPhysODBCBase unit contains FireDAC driver base implementation classes for all TDBX based drivers.

See Also

uADPhysTDBX Namespace (see page 829), uADPhysDataSnap Namespace (see page 749)

1.17.23.1 Classes

The following table lists classes in this documentation.

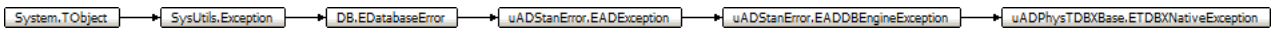
Classes

Class	Description
ETDBXNativeException (see page 830)	Represents exception class for all TDBX based drivers.

1.17.23.1.1 ETDBXNativeException Class

Represents exception class for all TDBX based drivers.

Class Hierarchy



File

uADPhysTDBXBase

Description

The ETDBXNativeException class represents the exception class for all TDBX based drivers, including TDBX and DataSnap.

Syntax

```
ETDBXNativeException = class(EADDBEngineException);
```

See Also

EADDBEngineException (see page 833)

1.17.24 uADStanError Namespace

Contains error classes - EADException (see page 836), EADDBEngineException (see page 833), etc

Classes

Class	Description
EADDBArrayExecuteError (see page 831)	EADDBArrayExecuteError is used as an Array DML error description item.
EADDBEngineException (see page 833)	EADDBEngineException is the base exception class for all FireDAC DBMS related errors.
EADException (see page 836)	EADException is the base exception class for all FireDAC exceptions.
TADDBError (see page 837)	TADDBError represents FireDAC / DBMS errors, warnings and messages for the EADDBEngineException (see page 833) exception class.

Description

The uADStanError contains error describing base classes:

- EADException (see page 836) - base class for all FireDAC exceptions;
- EADDBEngineException (see page 833) - base class for all FireDAC DBMS exceptions;
- TADDBError (see page 837) - base class for a DBMS exception error item

See Also

Handling Errors (see page 44)

1.17.24.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
EADDBArrayExecuteError (see page 831)	EADDBArrayExecuteError is used as an Array DML error description item.
EADDBEngineException (see page 833)	EADDBEngineException is the base exception class for all FireDAC DBMS related errors.
EADException (see page 836)	EADException is the base exception class for all FireDAC exceptions.
TADDBError (see page 837)	TADDBError represents FireDAC / DBMS errors, warnings and messages for the EADDBEngineException (see page 833) exception class.

1.17.24.1.1 EADDBArrayExecuteError Class

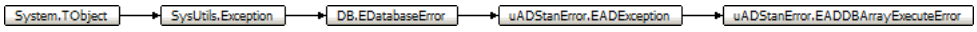
EADDBArrayExecuteError is used as an Array DML error description item.

public

public	Description
Action (see page 832)	Specifies an error processor action.
R Exception (see page 832)	Returns a reference to the original exception object.
R Offset (see page 832)	Returns an offset of a failed array item from the array beginning.
R RetryLevel (see page 832)	Returns a number of a retry operation.

Times (see page 833)	Returns a number of the array items remained to process in the array after the error has happened.

Class Hierarchy



File

uADStanError

Description

An EADDBArrayExecuteError object is used as an Array DML (see page 82) error description argument of the appropriate event handlers.

Syntax

```
EADDBArrayExecuteError = class(EADException);
```

1.17.24.1.1.1 public

1.17.24.1.1.1.1 EADDBArrayExecuteError.Action Property

Specifies an error processor action.

Description

Use Action to specify an action, that error processor should take after return from event handler.

Syntax

```
property Action: TADErrorAction;
```

1.17.24.1.1.1.2 EADDBArrayExecuteError.Exception Property

Returns a reference to the original exception object.

Description

Use Exception property to get a reference to the original exception object, which was raised after erroneous DB operation.

Syntax

```
property Exception: EADDBEngineException;
```

1.17.24.1.1.1.3 EADDBArrayExecuteError.Offset Property

Returns an offset of a failed array item from the array beginning.

Description

Use the Offset property to get an offset of a failed array item from the array beginning.

Syntax

```
property Offset: LongInt;
```

1.17.24.1.1.1.4 EADDBArrayExecuteError.RetryLevel Property

Returns a number of a retry operation.

Description

Use a RetryLevel to get a number of a retry operation. The number will be incremented after each eaSkip Action value.

Syntax

```
property RetryLevel: Integer;
```

1.17.24.1.1.1.5 EADDBArrayExecuteError.Times Property

Returns a number of the array items remained to process in the array after the error has happened.

Description

Use the Time property value to get a number of the array items remained to process in the array after the error has happened.

Syntax

```
property Times: LongInt;
```

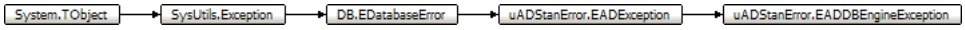
1.17.24.1.2 EADDBEngineException Class

EADDBEngineException is the base exception class for all FireDAC DBMS related errors.

public

public	Description
ErrorCode (see page 834)	Returns a DBMS specific error code.
ErrorCount (see page 834)	Indicates the total number of errors contained in the Errors (see page 834) property.
Errors (see page 834)	Lists the entire FireDAC / DBMS error, warning or message collection.
Kind (see page 835)	Returns a DBMS-independent error kind.
Params (see page 835)	Return a failed SQL (see page 835) command parameter values.
SQL (see page 835)	Return a failed SQL command text.

Class Hierarchy



File

uADStanError

Description

An EDBEngineException object is used as:

- an exception object, raised when a DBMS operation produces an error condition;
- an object describing a warning, stored at connection when a DBMS operation produces an warning condition;
- and object representing a message, stored at connection when a DBMS operation sends a message to a client.

Its Errors (see page 834) property contains TADDBError (see page 837) objects. Each of them corresponds to a single error, warning or message item, produced inside of a single DBMS operation. The Kind (see page 835) property returns DBMS-independent error kind. The SQL (see page 835) and Params (see page 835) properties allows to get a failed SQL command.

All FireDAC drivers are implementing own EADDBEngineException descendant classes (see page 44). For example, MySQL driver implements EMySQLNativeException exception class.

Use a TADGUIxErrorDialog (see page 672) dialog to display the complete information about FireDAC / DBMS error.

Syntax

```
EADDBEngineException = class(EADException);
```

See Also

Handling Errors (see page 44), TADDBError (see page 837), EADException (see page 836)

Example

```
uses
  uADStanError;
......
try
  ADQuery1.ExecSQL;
except
  on E: EADDBEngineException do
    if E.Kind = ekUKViolated then begin
```

```
      ShowMessage('The record must be unique');
      Abort;
    end;
end;
```

1.17.24.1.2.1 public

1.17.24.1.2.1.1 EADDBEngineException.ErrorCode Property

Returns a DBMS specific error code.

Description

Use the ErrorCode property to read a DBMS specific error code.

This property return value of the ErrorCode (see page 838) property of the first item in Errors (see page 834) collection.

Syntax

```
property ErrorCode: Integer;
```

See Also

TADDBError.Kind (see page 835), TADDBError.ErrorCode

1.17.24.1.2.1.2 EADDBEngineException.ErrorCount Property

Indicates the total number of errors contained in the Errors (see page 834) property.

Description

Use ErrorCount as an upper bound when iterating through the errors in the error collection.

Syntax

```
property ErrorCount: Integer;
```

See Also

Errors (see page 834)

1.17.24.1.2.1.3 EADDBEngineException.Errors Property

Lists the entire FireDAC / DBMS error, warning or message collection.

Description

Use Errors to read TADDBError (see page 837) objects.

Each object corresponds to an error, warning or message returned by a DBMS. In many cases there will be only single object. although some DBMS's, like a Microsoft SQL Server, may return few messages.

The first error has an index value of 0. The last error has an index value of ErrorCount (see page 834)-1. The property is default array property. So, you can avoid it.

Syntax

```
property Errors [Index: Integer]: TADDBError;
```

See Also

TADDBError (see page 837), ErrorCount (see page 834)

Example

```
try
   ADQuery1.ExecSQL;
except
  on E: EADDBEngineException do
    for i := 0 to E.ErrorCount - 1 do
      Memo1.Lines.Add(E[i].Message);
end;
```

1.17.24.1.2.1.4 EADDBEngineException.Kind Property

Returns a DBMS-independent error kind.

Description

Use the Kind property to read a DBMS-independent error kind.

This property return value of the Kind (see page 838) property of the first item in Errors (see page 834) collection.

Syntax

```
property Kind: TADCommandExceptionKind;
```

See Also

TADDBError.Kind, TADDBError.ErrorCode (see page 834)

Example

```
try
  ADConnection1.Connected := True;
except
  on E: EADDBEngineException do
    if E.Kind = ekUserPwdInvalid then begin
      ShowMessage('A user name or a password are invalid');
      Abort;
    end;
end;
```

1.17.24.1.2.1.5 EADDBEngineException.Params Property

Return a failed SQL (see page 835) command parameter values.

Description

Use Params property value to get a failed SQL command parameter values.

This property together with the SQL (see page 835) property may be used for the debugging purposes.

Syntax

```
property Params: TStrings;
```

See Also

SQL (see page 835)

1.17.24.1.2.1.6 EADDBEngineException.SQL Property

Return a failed SQL command text.

Description

Use SQL property value to get a SQL command text, for which DBMS returned an error.

This property together with the Params (see page 835) property may be used for the debugging purposes.

Syntax

```
property SQL: String;
```

See Also

Params (see page 835)

Example

```
try
  Log('Removing records');
  ADQuery1.ExecSQL('delete from tab');
  ....
  Log('Inserting records');
  ADQuery1.ExecSQL('insert into tab ...');
except
```

```
    on E: EADDBEngineException do begin
      Log(E.Message);
      Log('Failed SQL: ' + E.SQL);
      Log('Failed params: ' + E.Params.Text);
      raise;
    end;
  end;
```

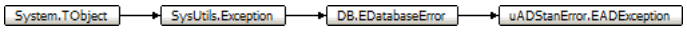
1.17.24.1.3 EADException Class

EADException is the base exception class for all FireDAC exceptions.

public

public	Description
ADCode (see page 836)	Returns an FireDAC specific error code.
ADObjName (see page 837)	Returns object name raised the exception.

Class Hierarchy



File

uADStanError

Description

The EADException class is used as the base class for the EADDBEngineException (see page 833) class (DBMS specific errors) and as a class for all DBMS unspecific errors.

Syntax

```
EADException = class(EDatabaseError);
```

See Also

EADDBEngineException (see page 833)

1.17.24.1.3.1 public

1.17.24.1.3.1.1 EADException.ADCode Property

Returns an FireDAC specific error code.

Description

Use ADCode property to get an FireDAC specific error code. This code allows to identify an error condition discovered by the FireDAC code.

Syntax

```
property ADCode: Integer;
```

1.17.24.1.3.1.2 EADException.ADObjName Property

Returns object name raised the exception.

Description

Use ADObjName property to get the name of an object raised the exception. The name is like <form name>.<query name>, and is useful for debugging purposes, allowing to identify an object raised the exception.

Syntax

```
property ADObjName: String;
```

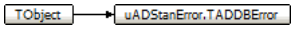
1.17.24.1.4 TADDBError Class

TADDBError represents FireDAC / DBMS errors, warnings and messages for the EADDBEngineException (see page 833) exception class.

public

public	Description
CommandTextOffset (see page 838)	Returns offset from a SQL command text beginning.
ErrorCode (see page 838)	Returns a DBMS specific error code.
Kind (see page 838)	Returns a DBMS independent error kind.
Level (see page 838)	Number of a TADDBError item.
Message (see page 839)	Returns a error, warning or message text.
ObjName (see page 839)	Returns a DBMS object name, participating in an error or warning.
RowIndex (see page 839)	Returns a row index, where an error has happened.

Class Hierarchy



File

uADStanError

Description

The TADDBError object is a container for the information pertaining to one database error, warning or message. One or more TADDBError objects are contained in the Errors (see page 834) property of the EADDBEngineException (see page 833) object.

Each item is represented by the:

- Message (see page 839) - actually error message. For the DBMS messages only Message property is used;
- ErrorCode (see page 838) - DBMS specific error code;
- Kind (see page 838) - DBMS independent error code and other properties;
- RowIndex (see page 839) - index of failed row in an Array DML operation, etc.

Some of the FireDAC drivers are implementing own TADDBError descendant classes. For example, Microsoft SQL Server driver implements TADMSSQLError class with four additional properties.

Syntax

```
TADDBError = class(TObject);
```

See Also

EADDBEngineException (see page 833)

1.17.24.1.4.1 public

1.17.24.1.4.1.1 TADDBError.CommandTextOffset Property

Returns offset from a SQL command text beginning.

Description

Use the CommandTextOffset to get an offset from a SQL command text beginning, where a DBMS failed to parse a SQL command.

This property is greater than or equal to zero for syntax errors and is equal to -1 for other errors. Note, that it is not applied to an original SQL command text, like a TADQuery.SQL (see page 396). But to a SQL command text sent to a DBMS, like a TADCustomQuery.Text (see page 397).

Syntax

```
property CommandTextOffset: Integer;
```

See Also

TADCustomQuery.Text (see page 397), TADCustomCommand.SQLText (see page 306)

1.17.24.1.4.1.2 TADDBError.ErrorCode Property

Returns a DBMS specific error code.

Description

Use ErrorCode to read a DBMS specific error code. For example, for Oracle ORA-00942 "Table or view does not exists" error, the ErrorCode will be equal to 942.

Also, you can use the Kind property value to use DBMS independent error kind. Note, that Kind is much more limited in the error condition description, than ErrorCode.

Syntax

```
property ErrorCode: Integer;
```

See Also

Kind (see page 838), Message (see page 839)

1.17.24.1.4.1.3 TADDBError.Kind Property

Returns a DBMS independent error kind.

Description

Use the Kind property to get a DBMS independent error kind. For example, for Oracle ORA-00942 "Table or view does not exists" error, the Kind will be equal to ekObjNotExists.

Also, you can use the ErrorCode (see page 838) property value to use DBMS specific error code. Note, that Kind is much more limited in the error condition description, than ErrorCode (see page 838).

Syntax

```
property Kind: TADCommandExceptionKind;
```

See Also

ErrorCode (see page 838), Message (see page 839)

1.17.24.1.4.1.4 TADDBError.Level Property

Number of a TADDBError (see page 837) item.

Description

Use Level property to get a number of a TADDBError item among other items in the EADDBEngineException (see page 833).Errors (see page 834) collection.

Syntax

```
property Level: Integer;
```

1.17.24.1.4.1.5 TADDBError.Message Property

Returns a error, warning or message text.

Description

Use the Message property to get a error, warning or message text, returned by a DBMS.

This text does not have the FireDAC prefix '[FireDAC]....'.

Syntax

```
property Message: String;
```

See Also

ObjName (see page 839), ErrorCode (see page 838)

1.17.24.1.4.1.6 TADDBError.ObjName Property

Returns a DBMS object name, participating in an error or warning.

Description

Use the ObjName property value to get a DBMS object name participating in an error or a warning.

The ObjName property value is extracted from the error message text. Depending on the DBMS and the error, the value may be a table name in the "object is not found" error, a constraint name in the "contraint violated" error, etc. Note, that some DBMS's does not return any object name as part of an error message. Then ObjName will be empty.

Syntax

```
property ObjName: String;
```

See Also

Message (see page 839), ErrorCode (see page 838), Kind (see page 838)

1.17.24.1.4.1.7 TADDBError.RowIndex Property

Returns a row index, where an error has happened.

Description

Use the RowIndex property to get a row index in an Array DML (see page 82) operation, where a DBMS has encountered an error condition.

If an error or a warning belongs to the full operation or an operation was not an Array DML operation, then RowIndex property value be equal to -1. Otherwise it will be greater than or equal to an AOffset property value of an Execute method call.

Note, that some DBMS for some Array DML operations cannot produce correct RowIndex value, then it will be equal to -1.

Syntax

```
property RowIndex: Integer;
```

See Also

Array DML (see page 82)

1.17.25 uADStanOption Namespace

Contains option classes - TADFetchOptions (see page 849), TADUpdateOptions (see page 896), etc.

Classes

Class	Description
TADBottomResourceOptions (see page 841)	The set of options controlling dataset persistence.
TADBottomUpdateOptions (see page 843)	The set of options controlling dataset editing and updates posting.
TADCustomOptions (see page 846)	The base class for all option classes.
TADEventAlerterOptions (see page 847)	The set of options controlling event alerter behaviour.
TADFetchOptions (see page 849)	The set of options controlling data and metadata fetching.

TADFormatOptions (see page 860)	The set of options controlling representation of fetched or sending data.
TADMapRule (see page 870)	TADMapRule represents a maintained client view to records in an FireDAC dataset...
TADMapRules (see page 873)	TADMapRules is a collection of TADMapRule (see page 870) objects.
TADResourceOptions (see page 874)	The set of options controlling SQL command preprocessor, command execution and dataset persistence.
TADTopResourceOptions (see page 885)	The set of options controlling SQL automatic connection recover, dataset persistence and server output.
TADTxOptions (see page 891)	The set of options controlling transaction behaviour.
TADUpdateOptions (see page 896)	The set of options controlling dataset editing and updates posting.

Structs, Records, Enums

Struct, Record, Enum	Description
TADActionRequest (see page 905)	The enumerated type lists all possible FireDAC requests to a database.
TADAutoFetchAll (see page 906)	Add a short summary here...

Description

The uADStanOption unit contains option classes:

- TADFetchOptions (see page 849) - result set fetching options;
- TADUpdateOptions (see page 896) - options controlling data editing and updates posting;
- TADFormatOptions (see page 860) - options controlling database data representation;
- TADResourceOptions (see page 874) - command / dataset general behavior, persistence and resource usage options;
- TADTxOptions (see page 891) - transaction control options;
- etc

See Also

Setting Options (see page 34)

1.17.25.1 Classes

The following table lists classes in this documentation.

Classes

Class	Description
TADBottomResourceOptions (see page 841)	The set of options controlling dataset persistence.
TADBottomUpdateOptions (see page 843)	The set of options controlling dataset editing and updates posting.
TADCustomOptions (see page 846)	The base class for all option classes.
TADEventAlerterOptions (see page 847)	The set of options controlling event alerter behaviour.
TADFetchOptions (see page 849)	The set of options controlling data and metadata fetching.
TADFormatOptions (see page 860)	The set of options controlling representation of fetched or sending data.
TADMapRule (see page 870)	TADMapRule represents a maintained client view to records in an FireDAC dataset...
TADMapRules (see page 873)	TADMapRules is a collection of TADMapRule (see page 870) objects.
TADResourceOptions (see page 874)	The set of options controlling SQL command preprocessor, command execution and dataset persistence.
TADTopResourceOptions (see page 885)	The set of options controlling SQL automatic connection recover, dataset persistence and server output.
TADTxOptions (see page 891)	The set of options controlling transaction behaviour.
TADUpdateOptions (see page 896)	The set of options controlling dataset editing and updates posting.

1.17.25.1.1 TADBottomResourceOptions Class

The set of options controlling dataset persistence.

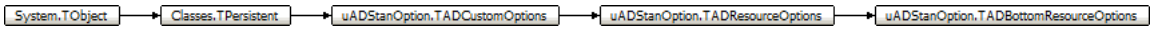
public

public	Description
Assign (see page 842)	Assigns all options of one option object to another.
ResolveFileName (see page 842)	Returns actual persistent dataset file name.
RestoreDefaults (see page 843)	Restore option default values.

published

published	Description
PersistentFileName (see page 843)	Specifies persistent dataset file name.

Class Hierarchy



File

uADStanOption

Description

The TADBottomResourceOptions class represents the set of properties controlling persistence of datasets:

- PersistentFileName (see page 843).

The TADBottomResourceOptions is terminal class in resource options classes hierarchy. The TADCustomCommand (see page 291) and TADDataSet (see page 584) use TADBottomResourceOptions.

Syntax

```
TADBottomResourceOptions = class(TADResourceOptions);
```

See Also

Setting Options (see page 34), TADCustomCommand.ResourceOptions (see page 304), TADQuery.ResourceOptions (see page 498)

1.17.25.1.1.1 public

1.17.25.1.1.1.1 TADBottomResourceOptions.Assign Method

Assigns all options of one option object to another.

Parameters

Parameters	Description
ASource: TPersistent	A source options object.

Description

Use the Assign method to assign all changed option values of ASource option object to this object. After that corresponding options will stop inheriting the values from more high level option object.

Syntax

```
procedure Assign(ASource: TPersistent); override;
```

See Also

RestoreDefaults

Example

```
ADQuery2.UpdateOptions.Assign(ADQuery1.UpdateOptions);
```

1.17.25.1.1.2 TADBottomResourceOptions.ResolveFileName Method

Returns actual persistent dataset file name.

Parameters

Parameters	Description
const AFileName: String = ''	A file name to resolve.

Description

Use ResolveFileName method to resolve AFileName into actual persistent dataset file name.

if AFileName is empty, then will be used TADBottomResourceOptions.PersistentFileName (see page 843).

If AFileName does not include extension, then will be used TADTopResourceOptions (see page 885).DefaultStoreExt (see page 888) as an extension. If TADTopResourceOptions (see page 885).DefaultStoreExt (see page 888) is empty, then an extension will be evaluated from TADTopResourceOptions (see page 885).DefaultStoreFormat (see page 889).

If AFileName does not include extension, then will be used TADTopResourceOptions (see page 885).DefaultStoreFolder (see page 889) as a folder.

At end file name will be expanded by replacing $(NAME) placeholders with corresponding registry or environment variable values.

Syntax

```
function ResolveFileName(const AFileName: String = ''): String;
```

See Also

PersistentFileName (see page 843), TADTopResourceOptions.DefaultStoreExt (see page 877), TADTopResourceOptions.DefaultStoreFormat (see page 877), TADTopResourceOptions.DefaultStoreFolder (see page 877)

1.17.25.1.1.3 TADBottomResourceOptions.RestoreDefaults Method

Restore option default values.

Description

Call RestoreDefaults to reset all changed options to their default values.

Syntax

```
procedure RestoreDefaults; override;
```

See Also

AssignedValues

1.17.25.1.2 published

1.17.25.1.2.1 TADBottomResourceOptions.PersistentFileName Property

Specifies persistent dataset file name.

Description

Use PersistentFileName to specify a persistent dataset file name. The dataset will save / load its content if TADResourceOptions (see page 874).Persistent (see page 884) is True.

If PersistentFileName does not include extension, then will be used TADTopResourceOptions (see page 885).DefaultStoreExt (see page 888) as an extension. If TADTopResourceOptions (see page 885).DefaultStoreExt (see page 888) is empty, then an extension will be evaluated from TADTopResourceOptions (see page 885).DefaultStoreFormat (see page 889).

If PersistentFileName does not include extension, then will be used TADTopResourceOptions (see page

885).DefaultStoreFolder (see page 889) as a folder.

Syntax

```
property PersistentFileName: string;
```

See Also

TADResourceOptions.Persistent (see page 884), TADTopResourceOptions.DefaultStoreExt (see page 877), TADTopResourceOptions.DefaultStoreFormat (see page 877), TADTopResourceOptions.DefaultStoreFolder (see page 877)

1.17.25.1.2 TADBottomUpdateOptions Class

The set of options controlling dataset editing and updates posting.

public

public	Description
Assign (see page 844)	Assigns all options of one option object to another.
RestoreDefaults (see page 844)	Restore option default values.

published

published	Description
AutoIncFields (see page 845)	Specifies dataset fields, which are auto-incrementing by DBMS.
KeyFields (see page 845)	Specifies dataset primary key fields.
UpdateTableName (see page 846)	Specifies table name to use in SQL commands posting updates.

Class Hierarchy



File

uADStanOption

Description

The TADBottomUpdateOptions class represents the set of properties controlling how FireDAC datasets and DApt Layer adapter interfaces:

- use database generators and sequences for auto-incrementing fields (AutoIncFields (see page 845));
- generate updates posting commands (KeyFields (see page 845), UpdateTableName (see page 846)).

The TADBottomUpdateOptions is terminal class in update options classes hierarchy. The TADDataSet (see page 584) and DApt Layer adapters use TADBottomUpdateOptions.

Syntax

```
TADBottomUpdateOptions = class(TADUpdateOptions);
```

See Also

Setting Options (see page 34), TADDataSet.UpdateOptions (see page 653)

1.17.25.1.2.1 public

1.17.25.1.2.1.1 TADBottomUpdateOptions.Assign Method

Assigns all options of one option object to another.

Parameters

Parameters	Description
ASource: TPersistent	A source options object.

Description

Use the Assign method to assign all changed option values of ASource option object to this object. After that corresponding options will stop inheriting the values from more high level option object.

Syntax

```
procedure Assign(ASource: TPersistent); override;
```

See Also

RestoreDefaults

Example

```
ADQuery2.UpdateOptions.Assign(ADQuery1.UpdateOptions);
```

1.17.25.1.2.1.2 TADBottomUpdateOptions.RestoreDefaults Method

Restore option default values.

Description

Call RestoreDefaults to reset all changed options to their default values.

Syntax

```
procedure RestoreDefaults; override;
```

See Also

AssignedValues

1.17.25.1.2.2 published

1.17.25.1.2.2.1 TADBottomUpdateOptions.AutoIncFields Property

Specifies dataset fields, which are auto-incrementing by DBMS.

Description

Use AutoIncFields to specify ';' separated list of fields, whose values are filled from the generators / sequences. The names must be without quotes and must match to the original result set field names. AutoIncFields must be specified before preparing / opening a dataset.

Specifying AutoIncFields is required when FireDAC cannot determine auto incrementing fields, when:

- DBMS does not support auto-incremental / identity fields, but supports generators / sequences - Oracle, Firebird and Interbase;
- DBMS does not return information about auto-incrementing fields correctly.

Setting AutoIncFields sets specified fields attributes to [caAutoInc, caAllowNull]. For dtInt32 and dtUInt32 columns will be created TADAutoIncField (see page 579) fields. TField.Required property will be set to False.

The FetchGeneratorsPoint (see page 901) property controls, how FireDAC fills these field values at client side. The GeneratorName (see page 901) / TADAutoIncField (see page 579).GeneratorName (see page 581) properties control, from which database generator / sequence FireDAC fills these field values.

Syntax

```
property AutoIncFields: String;
```

See Also

Auto-Incremental Fields (see page 118), FetchGeneratorsPoint (see page 901), GeneratorName (see page 901), TADAutoIncField.GeneratorName (see page 901), TField.AutoGenerateValue

1.17.25.1.2.2.2 TADBottomUpdateOptions.KeyFields Property

Specifies dataset primary key fields.

Description

Use KeyFields to specify ';' separated list of fields, which compromise primary key for dataset records. The names must be without quotes and must match to the original result set field names. KeyFields must be specified before preparing / opening a dataset.

Specifying KeyFields is required when FireDAC cannot determine primary key fields, when:

- fiMeta is not included into TADUpdateOptions (see page 896).Items (see page 855), for example to raise performance;
- application needs to edit a result set obtained from a view, tables join or a table without defined primary key;
- application needs to edit a result set obtained from a stored procedure or from other non-SELECT SQL construction.

Setting KeyFields removes pfInKey from all fields TField.ProviderFlags and includes pfInKey for specifed fields ProviderFlags. Alternatively application can use only pfInKey and TField.ProviderFlags. But not both ways in the same time.

These fields will be used to build record search condition:

- at posting a record update using UPDATE ... WHERE <record search condition>, when UpdateMode (see page 904) = upWhereKeyOnly or upWhereChanged;
- at refreshing a record using SELECT ... WHERE <record search condition>.

Syntax

```
property KeyFields: String;
```

See Also

Unique Identifying Fields (see page 117), TField.ProviderFlags

1.17.25.1.2.2.3 TADBottomUpdateOptions.UpdateTableName Property

Specifies table name to use in SQL commands posting updates.

Description

Use UpdateTableName property to specify database table name to use in automatically generated SQL commands. The name may be quoted. UpdateTableName must be specified before preparing / opening a dataset.

Specifying UpdateTableName is required when FireDAC cannot correctly determine a table to post updates to or when application needs:

- to edit a result set obtained from a view or tables join;
- to edit a result set obtained from a stored procedure or from other non-SELECT SQL construction;
- to redirect updates posting to another table, than one specified in the SQL query.

Syntax

```
property UpdateTableName: String;
```

See Also

Overriding Posting Updates (see page 123), Object Names (see page 133), Update Command Generation (see page 121)

1.17.25.1.3 TADCustomOptions Class

The base class for all option classes.

public

public	Description
Version (see page 846)	The current version of options.
Assign (see page 847)	Assigns all options of one option object to another.
RestoreDefaults (see page 847)	Restores option default values.

Class Hierarchy



File

uADStanOption

Description

The TADCustomOptions class is a base class for other TADXxxxOptions classes.

A programmer should not use it directly.

Syntax

```
TADCustomOptions = class(TPersistent);
```

See Also

TADFetchOptions (see page 849), TADFormatOptions (see page 860), TADResourceOptions (see page 874), TADUpdateOptions (see page 896)

1.17.25.1.3.1 public

1.17.25.1.3.1.1 TADCustomOptions.Version Property

The current version of options.

Description

Use the Version property to determine were there an option value change or not. If some option was changed, when a code may decide to reprepare a query object or re-execute it or how else. The property value is growing sequentionally, but the step may be greater than one.

Syntax

```
property Version: Longword;
```

1.17.25.1.3.1.2 TADCustomOptions.Assign Method

Assigns all options of one option object to another.

Parameters

Parameters	Description
ASource: TPersistent	A source options object.

Description

Use the Assign method to assign all changed option values of ASource option object to this object. After that corresponding options will stop inheriting the values from more high level option object.

Syntax

```
procedure Assign(ASource: TPersistent); override;
```

See Also

RestoreDefaults (see page 847)

Example

```
ADQuery2.UpdateOptions.Assign(ADQuery1.UpdateOptions);
```

1.17.25.1.3.1.3 TADCustomOptions.RestoreDefaults Method

Restores option default values.

Description

Use the RestoreDefaults method to restore the default values for all options and make them inheritable from more high level

options.

Syntax

```
procedure RestoreDefaults; virtual;
```

Example

```
ADQuery1.FetchOptions.RowsetSize := 1000;
ADQuery1.FetchOptions.Mode := fmAll;
ADQuery1.Open('select * from big_table');
....
ADQuery1.FetchOptions.RestoreDefaults;
```

1.17.25.1.4 TADEventAlerterOptions Class

The set of options controlling event alerter behaviour.

published

published	Description
AutoRegister (see page 848)	Controls the automatic registration of the events at a DBMS.
Kind (see page 848)	Specifies the event alerter kind.
Synchronize (see page 848)	Controls the thread, where the alerter event handlers will be executed.
Timeout (see page 849)	Specifies the event alerter timeout.

Class Hierarchy



File

uADStanOption

Description

The TADEventAlerterOptions class represents the set of properties controlling behavior of an event alerter, represented as the FireDAC component or as the Phys Layer command interface:

- event alerter kind (Kind (see page 848));
- timeout;
- the event handler thread (Synchronize (see page 848));
- events registration mode (AutoRegister (see page 848)).

The event alerter option applying is deferred until events registration.

Syntax

```
TADEventAlerterOptions = class(TPersistent);
```

See Also

Database Alerts (see page 75), TADEventAlerter (see page 422)

1.17.25.1.4.1 published

1.17.25.1.4.1.1 TADEventAlerterOptions.AutoRegister Property

Controls the automatic registration of the events at a DBMS.

Description

Set the AutoRegister property to True to enable automatic registration of the event alerter at a DBMS.

Syntax

```
property AutoRegister: Boolean;
```

1.17.25.1.4.1.2 TADEventAlerterOptions.Kind Property

Specifies the event alerter kind.

Description

Use the Kind property to specify the event alerter implementation kind.

The property value is FireDAC driver dependent. One driver may implement few event alerter's. All of them are identified by the different kinds. To use the driver default event alerter, set Kind to an empty string.

Syntax

```
property Kind: String;
```

See Also

Database Alerts (see page 75), Synchronize (see page 848)

1.17.25.1.4.1.3 TADEventAlerterOptions.Synchronize Property

Controls the thread, where the alerter event handlers will be executed.

Description

Set the Synchronize property to True to execute the event alerter handlers in the main thread context. Otherwise the event handlers may be executed in the context of the background thread, servicing the event alerter.

Syntax

```
property Synchronize: Boolean;
```

See Also

Database Alerts (see page 75), Kind (see page 848), Timeout (see page 849)

1.17.25.1.4.1.4 TADEventAlerterOptions.Timeout Property

Specifies the event alerter timeout.

Description

Use the Timeout to specify the event alerter timeout.

After the specified timeout, if there are no events occurred, the OnTimeout (see page 361) event will be fired. The Timeout property value is in msec's. A value <= 0 means - no timeout.

Syntax

```
property Timeout: Integer;
```

1.17.25.1.5 TADFetchOptions Class

The set of options controlling data and metadata fetching.

public

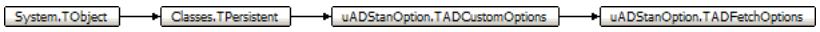
public	Description
ActualRowsetSize (see page 850)	Returns actual rowset size a command will use.
Assign (see page 850)	Assigns all options of one option object to another.
RestoreDefaults (see page 851)	Restore option default values.

published

published	Description
AssignedValues (see page 851)	Specifies the set of changed options.
AutoClose (see page 851)	Controls automatic closing of a dataset cursor after fetching the last record.

AutoFetchAll (see page 852)	Controls automatic fetching of all records from a dataset cursor before disconnecting a command.
Cache (see page 852)	Controls what kind of data to cache in memory.
CursorKind (see page 853)	Specifies type of cursor an FireDAC command or dataset uses.
DetailCascade (see page 854)	Controls propagation of a master dataset changes to a detail dataset in Centralized Cached Updates mode.
DetailDelay (see page 854)	Controls the delay between master dataset scroll and detail dataset refresh.
DetailOptimize (see page 855)	Specifies to refresh detail dataset only when master record is changed.
Items (see page 855)	Controls what kind of data to fetch.
LiveWindowFastFirst (see page 856)	Controls the TADTable (see page 539) live data window fetch optimization mode.
LiveWindowParanoic (see page 856)	Controls the TADTable (see page 539) live data window mode records search mode.
Mode (see page 857)	Controls how to fetch data.
RecordCountMode (see page 858)	Controls how to count records in a result set.
RecsMax (see page 858)	Controls the maximum number of records to be fetched.
RecsSkip (see page 859)	Controls the offset of the first record to be fetched.
RowsetSize (see page 859)	Controls the number of records per single fetch.
Unidirectional (see page 860)	Specifies possible directions of navigation by dataset records.

Class Hierarchy



File

uADStanOption

Description

The TADFetchOptions class represents the set of properties controlling how FireDAC components and Phys Layer command interfaces fetch the data and metadata:

- how to fetch (CursorKind (see page 853), Mode (see page 857), RowsetSize (see page 859)). For example, it is possible to fetch all result set records at once, or fetch records on demand.
- how to preserve resources (Unidirectional (see page 860), AutoClose (see page 851), Cache (see page 852), AutoFetchAll (see page 852));
- how BLOB fields, nested recordsets and metadata to fetch and cache (Items (see page 855), Cache (see page 852));
- master-detail dataset relationship handling (DetailDelay (see page 854), DetailOptimize (see page 855), DetailCascade (see page 854));
- a rowset paging (RecsMax (see page 858), RecsSkip (see page 859));
- TADTable operation mode (CursorKind (see page 853), LiveWindowParanoic (see page 856), LiveWindowFastFirst (see page 856));
- how to count records in a dataset (RecordCountMode (see page 858)).

Syntax

```
TADFetchOptions = class(TADCustomOptions);
```

See Also

Fetching Rows (see page 78), Setting Options (see page 34), Unique Identifying Fields (see page 117), Master-Detail Relationship (see page 104), TADCustomManager.FetchOptions (see page 374), TADCustomConnection.FetchOptions (see page 328), TADCustomCommand.FetchOptions (see page 300), TADQuery.FetchOptions (see page 486), IADPhysCommand.Options

1.17.25.1.5.1 public

1.17.25.1.5.1.1 TADFetchOptions.ActualRowsetSize Property

Returns actual rowset size a command will use.

Description

Use ActualRowsetSize to get actual rowset size a command will use.

Actual rowset size depends on Mode (see page 857). If it is fmExactRecsMax, then actual rowset size will be equal to RecsMax (see page 858).

Syntax

```
property ActualRowsetSize: Integer;
```

See Also

Mode (see page 857), RecsMax (see page 858)

1.17.25.1.5.1.2 TADFetchOptions.Assign Method

Assigns all options of one option object to another.

Parameters

Parameters	Description
ASource: TPersistent	A source options object.

Description

Use the Assign method to assign all changed option values of ASource option object to this object. After that corresponding options will stop inheriting the values from more high level option object.

Syntax

```
procedure Assign(ASource: TPersistent); override;
```

See Also

RestoreDefaults

Example

```
ADQuery2.UpdateOptions.Assign(ADQuery1.UpdateOptions);
```

1.17.25.1.5.1.3 TADFetchOptions.RestoreDefaults Method

Restore option default values.

Description

Call RestoreDefaults to reset all changed options to their default values.

Syntax

```
procedure RestoreDefaults; override;
```

See Also

AssignedValues (see page 851)

1.17.25.1.5.2 published

1.17.25.1.5.2.1 TADFetchOptions.AssignedValues Property

Specifies the set of changed options.

Description

Use AssignedValues to get / set the set of modified option values.

If some option is changed at this level, then corresponding flag is included into AssignedValues. If flag is not included, then

an option value is inherited from more high level. Excluding flag from AssignedValues makes option inheriting its value from more high level.

See Setting Options (see page 34) for more details.

Syntax

```
property AssignedValues: TADFetchOptionValues;
```

See Also

RestoreDefaults (see page 851)

1.17.25.1.5.2.2 TADFetchOptions.AutoClose Property

Controls automatic closing of a dataset cursor after fetching the last record.

Description

Use AutoClose property value to control the automatic closing of a dataset cursor.

If AutoClose is True and is fetched the last record from the current result set, then the underlying cursor will be closed, all additional not yet processed result sets will be discarded and resources associated with the cursors will be released to the DBMS. The dataset (TADDataSet (see page 584) descendant) itself will be not closed, but will be closed its internal command object. That allows to release cursor related DBMS resource as fast as it is possible.

If your SQL command returns the few result sets, and you need to process all of them, then you must set AutoClose to False. Otherwise after processing of the first result set, all other result sets will be discarded.

The default value is True.

Syntax

```
property AutoClose: Boolean;
```

See Also

Mode (see page 857), TADAdaptedDataSet.NextRecordSet (see page 265), IADPhysCommand.NextRecordSet

1.17.25.1.5.2.3 TADFetchOptions.AutoFetchAll Property

Controls automatic fetching of all records from a dataset cursor before disconnecting a command.

Description

Use AutoFetchAll property to control automatic fetching of all records from a dataset cursor before disconnecting an associated command. The default value is afAll. A command may be disconnected, when one of the following occurs:

- A connection object will be set to offline mode.
- When amount of active commands on the same connection object is greater than TADTopResourceOptions.MaxCursors (see page 890), then less used commands will be disconnected.
- On some DBMS's before a transaction, where a cursor was opened, will be committed.

The property may have values:

Value	Description
afAll	FetchAll (see page 624) method will be called.
afTruncate	Fetching from the cursor will be finished. Previously fetched records will be preserved.
afDisable	Raise an exception, if not all records are fetched.

Syntax

```
property AutoFetchAll: TADAutoFetchAll;
```

See Also

Managing Transactions (see page 41), Mode (see page 857), AutoClose (see page 851)

1.17.25.1.5.2.4 TADFetchOptions.Cache Property

Controls what kind of data to cache in memory.

Description

Use Cache property to control what kind of data and metadata to cache in internal data storage.

Option	Description
fiBlobs	When included, then the BLOB field values are not discarded from memory after usage. When excluded, then after changing the current dataset record, BLOB field values in the current record will be discarded.
fiDetails	When included: • for a nested dataset an associated nested cursor will be not discarded from memory after changing a parent dataset record; • for a detail dataset an associated detail records will be not discarded from memory after changing a master dataset record.
fiMeta	When included, then the command metadata is not discarded from memory after usage. When excluded, then after fetching and using a required metadata (eg, primary key fields) it will be discarder from the cache.

The default value is [fiBlobs, fiDetails, fiMeta], meaning all data and metadata will be cached after usage.

Note, that excluding items from Cache does not affect already fetched data or metadata.

Syntax

```
property Cache: TADFetchItems;
```

See Also

Unique Identifying Fields (see page 117), Fetching Rows (see page 78), Master-Detail Relationship (see page 104), Items (see page 855)

1.17.25.1.5.2.5 TADFetchOptions.CursorKind Property

Specifies type of cursor an FireDAC command or dataset uses.

Description

Set CursorKind to indicate the type of cursor the FireDAC command or dataset uses for the result set when it is opening. CursorKind must be set prior to preparing command or dataset. To change CursorKind for already used dataset, application may call Disconnect (see page 619) method first.

CursorKind affects on the:

- time to return the first record or open the result set;
- time to return all result set records;
- ability to use multiple open cursors;
- cursor stability;
- used DBMS resources;
- TADTable mode.

Option	Description
ckAutomatic	A driver will automatically choose cursor kind, depending on the other fetch options and result set structure.
ckDefault	A driver will use a default client side cursor. A default cursor produces the records snapshot, as they was at query execution time. May give maximum fetch performance. But requires more time to return first record, because all result set records will be sent to the client at Open call. Some DBMS support only single active default cursor per connection.

ckDynamic	A driver will use a dynamic server side cursor. Dynamic cursor rows may be affected by the updates to the query tables, while the cursor is active. May require less time to return first record. And more time to return all records.
ckStatic	A driver will use a static server side cursor. Static cursor produces the records snapshot, as they was at query execution time. The fetch performance may be similar to dynamic cursor.
ckForwardOnly	A driver will use a forward only server side cursor. For FireDAC forward only cursors are similar to dynamic or static cursors. To control FireDAC dataset scrolling abilities use the Unidirectional property.

The default value is ckAutomatic.

Most of FireDAC drivers support only some cursor kinds or even does not use this option at all. Please, consult to the DBMS documentation for details. FireDAC chooses actual cursor kind, when CursorKind = ckAutomatic, as the following:

DBMS	Conditions	Actual cursor kind
Microsoft SQL Server	• if server version is 2005 or higher; • or FetchOptions.Mode (see page 857) is fmExactRecsMax or fmAll; • or ResourceOptions.DirectExecute (see page 881) is True.	Default cursor.
	Otherwise.	Static cursor.
IBM DB2	--	Forward only cursor.
Sybase SQL Anywhere, Microsoft Access	If result set has BLOB fields.	Dynamic cursor.
	Otherwise.	Default cursor.
PostgreSQL	If FetchOptions.Mode (see page 857) is fmExactRecsMax or fmAll.	Default cursor.
	Otherwise	Static cursor.
Others	--	Default cursor.

For TADTable (see page 539) ckAutomatic enables live data window (see page 73), when a table has unique identifying fields. ckDynamic enables live data window unconditionally, and an exception will be raised if a DB table does not have an unique key. Other cursor kinds disable live data window mode and sets TADTable (see page 539) to standard mode, when it acts like the TADQuery (see page 476).

Syntax

```
property CursorKind: TADCursorKind;
```

See Also

Fetching Rows (see page 78), Mode (see page 857), Unidirectional (see page 860), RowsetSize (see page 859)

1.17.25.1.5.2.6 TADFetchOptions.DetailCascade Property

Controls propagation of a master dataset changes to a detail dataset in Centralized Cached Updates mode.

Description

Set the DetailCascade property to True to enable propagation of a master dataset changes to a detail dataset in Centralized Cached Updates (see page 114) mode.

The DetailCascade = True has an effect, when master and detail datasets SchemaAdapter (see page 506) property points to the same TADCustomSchemaAdapter. Setting DetailCascade to True enables an in-memory referential constraint for a detail dataset.

Syntax

```
property DetailCascade: Boolean;
```

See Also

Caching Updates (see page 114), Master-Detail Relationship (see page 104), TADRdbmsDataSet (see page 501).SchemaAdapter (see page 506)

1.17.25.1.5.2.7 TADFetchOptions.DetailDelay Property

Controls the delay between master dataset scroll and detail dataset refresh.

Description

Set DetailDelay property value to the time in msecs. After the master dataset is scrolled, the detail (this) dataset will be refreshed after DetailDelay msecs. If another master dataset scrolling happens at that delay time, then the delay will be restarted.

This is at first useful at master-detail relation, where a detail dataset must execute a SQL command against database to query details records. And master dataset is attached to GUI, where the user can scroll through records. If a user will press and hold a navigation button, then master dataset will scroll fast, but this property allows to postpone detail dataset refreshing, until the master dataset scrolling will be finished.

FireDAC cannot distinguish - is it a user scrolling through master dataset or an application code. So, programmer should take care in code that detail dataset contains records corresponding to the current master dataset record. That may be done:

- persistently disabling delayed refreshing - by setting DetailDelay to 0 and calling TADDataSet.MasterLink.Synchronize (see page 665) method;
- one time synchronization - by calling TADDataSet.MasterLink.Synchronize (see page 665) method.

Syntax

```
property DetailDelay: Integer;
```

See Also

Master-Detail Relationship (see page 104), MasterSource (see page 603), MasterLink (see page 603), TADMasterDataLink (see page 661), DetailOptimize (see page 855)

Example

```
// allow to user to scroll through master dataset, using GUI
qDetail.FetchOptions.DetailDelay := 200;
...
// there detail dataset is in sync with master dataset
qDetail.FetchOptions.DetailDelay := 0;
qDetail.MasterLink.Synchronize(True);
while not qMaster.Eof do begin
  ...
  qMaster.Next;
end;
// again allow to user to scroll through master dataset, using GUI
qDetail.FetchOptions.DetailDelay := 200;
```

1.17.25.1.5.2.8 TADFetchOptions.DetailOptimize Property

Specifies to refresh detail dataset only when master record is changed.

Description

Set DetailOptimize property to True (default), to refresh detail dataset only when current detail dataset key field values are different from master dataset key field values. That allows to avoid extra refreshing's, when master dataset state is changed, non-key field values are changed, etc.

Set DetailOptimize property to False to always refresh detail dataset. That may be useful, when an application expects that detail dataset will be refreshed after calling Refresh method for master dataset, and in some other cases.

Syntax

```
property DetailOptimize: Boolean;
```

See Also

Master-Detail Relationship (see page 104), DetailDelay (see page 854)

1.17.25.1.5.2.9 TADFetchOptions.Items Property

Controls what kind of data to fetch.

Description

Use Items property to control which kinds of data and metadata to fetch.

Option	Description
fiBlobs	When included, then the BLOB field values will be fetched together with other record fields. Otherwise, the fetching will be deferred until a BLOB value will be really read. Note, that excluding fiBlobs from Items do not modify the SELECT list of a SQL command. If a DBMS transfers BLOB values by-value (eg, Oracle LONG, MySQL, SQL Server, SQLite), then in any case the BLOB values will be delivered to the client other the network. But they will be not stored in the client records cache. But if a DBMS transfers BLOB values by-reference (eg, Oracle CLOB / BLOB, Interbase, Firebird), then the BLOB values will be not delivered and not stored.
fiDetails	When included, then the nested cursor fields will be fetched together with other record fields. Otherwise, the fetching will be deferred until a nested cursor value will be really read.
fiMeta	When included, then the command metadata will be fetched. Otherwise, programmer must supply metadata on your own. The kind of metadata and the point then it will be fetched depends on the command kind: • TADCustomStoredProc (see page 403), TADCustomCommand (see page 291).CommandKind (see page 298) = ckStoredProc, etc. The metadata is the description of the stored procedure parameters. It will be fetched at Prepare (see page 511) call. If fiMeta is not in Items, then programmer must fill Params (see page 534) collection by his own. • TADCustomQuery (see page 394), TADCustomCommand (see page 291).CommandKind (see page 298) = ckSelect, etc. The metadata is the set of the base table columns, forming the table primary key. It is used to automatically generate updating SQL commands. The metadata will be fetched at Open / Define calls. If fiMeta is not in Items, then programmer must do one of following to specify PK fields (see page 117): • include pfInKey into TField.ProviderFlags for PK fields; • set UpdateOptions.KeyFields (see page 845) to the colon separated list of PK field names; • include coInKey in TADDatSColumn.Options for PK columns.

The default value is [fiBlobs, fiDetails, fiMeta], meaning all data and metadata will be fetched automatically at proper moments.

Syntax

```
property Items: TADFetchItems;
```

See Also

Unique Identifying Fields (see page 117), Fetching Rows (see page 78), Cache (see page 852), TField.ProviderFlags, TADBottomUpdateOptions.KeyFields (see page 845), TADDatSColumn.Options

1.17.25.1.5.2.10 TADFetchOptions.LiveWindowFastFirst Property

Controls the TADTable (see page 539) live data window fetch optimization mode.

Description

Use the LiveWindowFastFirst property to setup the TADTable Live Data Window (see page 73) fetch optimization mode.

When it is True, then FireDAC will build a query to return the first results as fast as it possible. When it is False (the default value), then FireDAC will minimize the time to return all table rows. Note, True may be useful with SQL Server and similar DB servers, which are blocking result set records, until the whole result set will be fetched.

Syntax

```
property LiveWindowFastFirst: Boolean;
```

See Also

Browsing Table (see page 73), LiveWindowParanoic (see page 856)

1.17.25.1.5.2.11 TADFetchOptions.LiveWindowParanoic Property

Controls the TADTable (see page 539) live data window mode records search mode.

Description

Use the LiveWindowParanoic property to setup the TADTable Live Data Window (see page 73) records search mode.

In general, FireDAC to maintain live data window is performing different SQL queries. That may affect the application performance. To improve it, search operations may avoid extra SQL queries for the price of the less exact result. When LiveWindowParanoic = False (default value), then TADTable will avoid:

- SELECT COUNT(*) SQL query to calculate the RecNo property value. Instead -1 will be returned.
- SELECT SQL query for Locate and Lookup methods, when a locating record is found in local client cache.

Note, that LiveWindowParanoic property is used only when TADTable is operating in the live data window mode. It is not used in standard mode.

Syntax

```
property LiveWindowParanoic: Boolean;
```

See Also

Browsing Table (see page 73), CursorKind (see page 853), LiveWindowFastFirst (see page 856)

1.17.25.1.5.2.12 TADFetchOptions.Mode Property

Controls how to fetch data.

Description

Use Mode property to control how the result set records must be fetched into FireDAC internal data storage.

Mode	Description
fmManual	The records are not fetched automatically by FireDAC. A programmer must write code to fetch records at appropriate moments, using following methods: • TADDataSet.FetchAll (see page 624) to fetch all records; • TADDataSet.GetNextPacket (see page 629) to fetch next record set (packet), containing RowsetSize records; • IADPhysCommand.Fetch to fetch all records or next record set;
fmOnDemand	The records are fetched automatically by FireDAC, when they are demanded by the navigation or other dataset methods. At each demand FireDAC will fetch one or more record sets, containing RowsetSize (see page 859) records. This mode allows to open big result set without a significant delay, like a fmAll mode. But total time of fetching of the all records will be more than in fmAll mode. For example, on Microsoft SQL Server will be used server side static cursor.

fmAll	All result set records are fetched automatically by FireDAC at the dataset Open call or at the first IADPhysCommand.Fetch call. On a big resultset it may take a time, but then navigation through a dataset will be fast, because all records will be stored in an FireDAC internal data storage. For many DBMS's, like a Microsoft SQL Server, MySQL Server, Sybase SQL Anywhere: • it releases client result set buffer and allows to execute next command returning resultsets; • gives maximum fetch performance. For example, on Microsoft SQL Server will be used fast forward-only cursor.
fmExactRecsMax	Similar to fmAll, but if the amount of the fetched records differs from specified in RecsMax (see page 858), then exception is raised.

The default value is fmOnDemand, for TADCustomMemTable it is fmAll.

Syntax

```
property Mode: TADFetchMode;
```

See Also

Fetching Rows (see page 78), TADDataSet.FetchAll (see page 624), TADDataSet.GetNextPacket (see page 629), RecsMax (see page 858), RowsetSize (see page 859), Unidirectional (see page 860)

Example

To check for fmExactRecsMax exception use code like that:

```
try
  ...
except
  on E: EADException do
    if E.ADCode = er_AD_AccExactFetchMismatch then
      ; // fetching failed
end;
```

1.17.25.1.5.2.13 TADFetchOptions.RecordCountMode Property

Controls how to count records in a result set.

Description

Use RecordCountMode property to control how the TADDataSet will count number of records in the result set.

Mode	Description
cmVisible	The TDataSet.RecordCount returns the number of records currently accessible through TDataSet navigation interface. For example, all these records will be accessible to a user using associated TDBGrid. • RecordCount tracks records deleted / appended to dataset. • Is not applicable to TADTable in LDV mode (see page 73). • Has no overhead. • Gives a convenient value for the end-user users.
cmFetched	The TDataSet.RecordCount returns the number of fetched records to the current moment of time. • RecordCount tracks records deleted / appended to dataset. • Is not applicable to TADTable in LDV mode (see page 73). • Has no overhead. • Gives rather a current dataset "weight", than a convenient value for the end-user users.

cmTotal	The TDataSet.RecordCount returns number of records, which will be returned by command. For that FireDAC issues SELECT COUNT(*) FROM (<original SQL command text>). • RecordCount does not track records deleted / appended to dataset. To refresh RecordCount call TDataSet.Refresh method. • Applicable to TADTable in LDV mode (see page 73). • Has an overhead at dataset opening. • Gives nearly exact number of records until dataset records will be deleted / inserted.

Default value is cmVisible.

Note, MySQL may return "Commands out of sync; you can't run this command now" error when RecordCountMode = cmTotal. In this case either set Mode to fmFetchAll or do not use cmTotal.

Syntax

```
property RecordCountMode: TADRecordCountMode;
```

See Also

TDataSet.RecordCount, Mode (see page 857)

1.17.25.1.5.2.14 TADFetchOptions.RecsMax Property

Controls the maximum number of records to be fetched.

Description

Use RecsMax to limit number of records to be fetched from a single result set.

FireDAC returns only first RecsMax records, other records are discarded. The -1 value means - no limit. If combined with Mode = fmExactRecsMax, then FireDAC raises an exception, if result set has number of records not equal to RecsMax.

Combining the RecsSkip (see page 859) and the RecsMax properties allows to implement a result set paging, useful for the web applications. Also an application may use the LIMIT (see page 64) escape function. If both are specified, then the LIMIT (see page 64) has a more high priority.

Depending on a DBMS, the RecsMax and RecsSkip (see page 859) may be translated to a SELECT statement clause (TOP, LIMIT, etc) limiting a result set on a server side. If a DBMS does not support such clauses, then limiting will be performed on a client.

The default value is -1.

Syntax

```
property RecsMax: Integer;
```

See Also

Fetching Rows (see page 78), RecsSkip (see page 859), Mode (see page 857), RowsetSize (see page 859), LIMIT (see page 64)

Example

```
ADQuery1.FetchOptions.RecsMax := 20;
ADQuery1.FetchOptions.RecsSkip := 60;
ADQuery1.Open;
ADQuery1.FetchAll; // query will return records from 61'th to 80'th
```

1.17.25.1.5.2.15 TADFetchOptions.RecsSkip Property

Controls the offset of the first record to be fetched.

Description

Use RecsSkip to specify the number of records to skip from the beginning of a single result set.

FireDAC discards RecsSkip first records and returns records, starting from RecsSkip+1 record number. The <= 0 value

means - do not skip.

Combining the RecsSkip and the RecsMax (see page 858) properties allows to implement a result set paging, useful for the web applications. Also an application may use the LIMIT (see page 64) escape function. If both are specified, then the LIMIT (see page 64) has a more high priority.

Depending on a DBMS, the RecsMax (see page 858) and RecsSkip may be translated to a SELECT statement clause (TOP, LIMIT, etc) limiting a result set on a server side. If a DBMS does not support such clauses, then limiting will be performed on a client.

The default value is -1.

Syntax

```
property RecsSkip: Integer;
```

See Also

Fetching Rows (see page 78), RecsMax (see page 858), Mode (see page 857), LIMIT (see page 64)

Example

```
ADQuery1.FetchOptions.RecsMax := 20;
ADQuery1.FetchOptions.RecsSkip := 60;
ADQuery1.Open;
ADQuery1.FetchAll; // query will return records from 61'th to 80'th
```

1.17.25.1.5.2.16 TADFetchOptions.RowsetSize Property

Controls the number of records per single fetch.

Description

Use RowsetSize property to set the size of a single rows set, that will be fetched at a single fetch operation. To change RowsetSize for already used dataset, application may call Disconnect (see page 619) method first.

FireDAC uses underlying DBMS CLI features to optimize the result set fetching. Most of the supported DBMS's offers row set fetching feature, while MySQL Server, Microsoft Access database does not. The row set fetching allows to transfer few records from a DBMS server to a DBMS client in a single packet at single round trip. That raises the fetching performance few times, comparing to fetching by one record per fetch operation and transmitting single record per round trip.

If DBMS does not support row set fetching, then FireDAC will emulate it, unifying fetching behavior across all drivers.

There is significant performance difference between RowsetSize <= 5 and RowsetSize >= 50. But there may be no significant performance difference between RowsetSize ~= 100 and RowsetSize ~= 500, for example. Anyway, you should check what value will be more suitable for your environment.

Syntax

```
property RowsetSize: Integer;
```

See Also

Fetching Rows (see page 78), Mode (see page 857), Unidirectional (see page 860)

1.17.25.1.5.2.17 TADFetchOptions.Unidirectional Property

Specifies possible directions of navigation by dataset records.

Description

Use Unidirectional property to specify will application navigate by dataset records in forward-only direction or in forward and backward directions. To change Unidirectional for already used dataset, application may call Disconnect (see page 619) method first.

Setting Unidirectional to True enables application to navigate in forward-only direction. FireDAC will automatically discard prior rows from internal data storage after moving current position in dataset, because they are not needed anymore. That dramatically reduces memory consumption on big result sets.

Setting Unidirectional to True for TADTable (see page 539) disables the live data window mode.

Notes

Do not use Unidirectional datasets to display data in grid control. It requires bi-directional datasets.

Syntax

```
property Unidirectional: Boolean;
```

See Also

Fetching Rows (see page 78), RowsetSize (see page 859), Mode (see page 857)

1.17.25.1.6 TADFormatOptions Class

The set of options controlling representation of fetched or sending data.

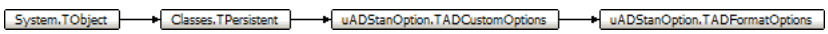
public

public	Description
StrsDivLen2 (see page 861)	Allows to overcome bugs in early dbExpress driver versions.
Assign (see page 862)	Assigns all options of one option object to another.
RestoreDefaults (see page 862)	Restore option default values.

published

published	Description
AssignedValues (see page 862)	Specifies the set of changed options.
DataSnapCompatibility (see page 863)	Controls the compatibility with the DataSnap / MIDAS.
DefaultParamDataType (see page 863)	Specifies default data type for parameters with ftUnknown data type.
FmtDisplayDate (see page 863)	Specifies display format for ftDate fields.
FmtDisplayDateTime (see page 863)	Specifies display format for ftDateDate, ftTimeStamp fields.
FmtDisplayNumeric (see page 864)	Specifies display format for numeric fields.
FmtDisplayTime (see page 864)	Specifies display format for ftTime fields.
FmtEditNumeric (see page 864)	Specifies edit format for numeric fields.
InlineDataSize (see page 865)	Controls how character or byte string values are stored in record buffer.
MapRules (see page 865)	Controls data type mapping.
MaxBcdPrecision (see page 865)	Controls recognition of numeric values as FmtBCD ones, depending on precision.
MaxBcdScale (see page 866)	Controls recognition of numeric values as FmtBCD ones, depending on scale.
MaxStringSize (see page 866)	Controls recognition of string values as BLOB ones.
OwnMapRules (see page 866)	Controls usage of MapRules (see page 865) property.
QuoteIdentifiers (see page 867)	Specifies whether or not to explicitly surround identifiers with quotes.
Round2Scale (see page 867)	Controls values rounding.
SortLocale (see page 867)	Specifies a locale ID for the local sorting operations.
SortOptions (see page 868)	Specifies the default sort options for the local sorting operations.
StrsEmpty2Null (see page 869)	Controls conversion of zero-length string values to NULL value.
StrsTrim (see page 869)	Controls removing of trailing spaces from string values.
StrsTrim2Len (see page 870)	Controls trimming string values to the declared length.

Class Hierarchy



File

uADStanOption

Description

The TADFormatOptions class represents the set of properties controlling how FireDAC components and Phys Layer command interfaces represent and process the data returned by DBMS or sent to DBMS:

- string data type (StrsEmpty2Null (see page 869), StrsTrim (see page 869), StrsTrim2Len (see page 870), InlineDataSize (see page 865), MaxStringSize (see page 866));
- numeric data type (MaxBcdPrecision (see page 865), MaxBcdScale (see page 866), Round2Scale (see page 867));
- time data type (Round2Scale (see page 867));
- general data type mapping (MapRules (see page 865), OwnMapRules (see page 866), DefaultParamDataType (see page 863)). For example, it is possible to map all NUMERIC(9, 0) columns to ftInteger columns, etc;
- field display formats (FmtDisplayDateTime (see page 863), FmtDisplayDate (see page 863), FmtDisplayTime (see page 864), FmtDisplayNumeric (see page 864), FmtEditNumeric (see page 864));
- dataset sorting (SortLocale (see page 867), SortOptions (see page 868));
- identifier quotations (QuoteIdentifiers (see page 867)).

Syntax

```
TADFormatOptions = class(TADCustomOptions);
```

See Also

Setting Options (see page 34), TADCustomManager.FormatOptions (see page 374), TADCustomConnection.FormatOptions (see page 328), TADCustomCommand.FormatOptions (see page 301), TADQuery.FormatOptions (see page 487), IADPhysCommand.Options

1.17.25.1.6.1 public

1.17.25.1.6.1.1 TADFormatOptions.StrsDivLen2 Property

Allows to overcome bugs in early dbExpress driver versions.

Description

Use StrsDivLen2 property with old dbExpress driver versions, which may work incorrectly with Unicode string data. If property is True, then FireDAC will divide the column length returned by driver by 2.

Syntax

```
property StrsDivLen2: Boolean;
```

1.17.25.1.6.1.2 TADFormatOptions.Assign Method

Assigns all options of one option object to another.

Parameters

Parameters	Description
ASource: TPersistent	A source options object.

Description

Use the Assign method to assign all changed option values of ASource option object to this object. After that corresponding options will stop inheriting the values from more high level option object.

Syntax

```
procedure Assign(ASource: TPersistent); override;
```

See Also

RestoreDefaults

Example

```
ADQuery2.UpdateOptions.Assign(ADQuery1.UpdateOptions);
```

1.17.25.1.6.1.3 TADFormatOptions.RestoreDefaults Method

Restore option default values.

Description

Call RestoreDefaults to reset all changed options to their default values.

Syntax

```
procedure RestoreDefaults; override;
```

See Also

AssignedValues (see page 862)

1.17.25.1.6.2 published

1.17.25.1.6.2.1 TADFormatOptions.AssignedValues Property

Specifies the set of changed options.

Description

Use AssignedValues to get / set the set of modified option values.

If some option is changed at this level, then corresponding flag is included into AssignedValues. If flag is not included, then an option value is inherited from more high level. Excluding flag from AssignedValues makes option inheriting its value from more high level.

See Setting Options (see page 34) for more details.

Syntax

```
property AssignedValues: TADFormatOptionValues;
```

See Also

RestoreDefaults (see page 862)

1.17.25.1.6.2.2 TADFormatOptions.DataSnapCompatibility Property

Controls the compatibility with the DataSnap / MIDAS.

Description

Set the DataSnapCompatibility property to True, when you are using FireDAC as a DataSnap back end data access components. The default value is False.

DataSnapCompatibility = True forces FireDAC to use compatible with DataSnap the numeric types precision and scale.

Syntax

```
property DataSnapCompatibility: Boolean;
```

1.17.25.1.6.2.3 TADFormatOptions.DefaultParamDataType Property

Specifies default data type for parameters with ftUnknown data type.

Description

Use DefaultParamDataType property to specify default data type for all parameters with ftUnknown data type. The default value is ftUnknown.

If parameter data type is unknown at Prepare call, then FireDAC sets it type to DefaultParamDataType. If DefaultParamDataType is ftUnknown, then FireDAC raises an exception that parameter data type is unknown. Although DefaultParamDataType allows to avoid this exception, it affects application performance, because a query with inappropriate parameter data type may have bad execution plan.

Syntax

```
property DefaultParamDataType: TFieldType;
```

See Also

TADParam.DataType

1.17.25.1.6.2.4 TADFormatOptions.FmtDisplayDate Property

Specifies display format for ftDate fields.

Description

Use the FmtDisplayDate property to specify DisplayFormat property for TDateField / ftDate dataset fields.

The option will be applied only to default fields or to persistent fields at the create time only. For format details see TDateField.DisplayFormat.

Syntax

```
property FmtDisplayDate: String;
```

See Also

FmtDisplayDateTime (see page 863), FmtDisplayTime (see page 864), FmtDisplayNumeric (see page 864), FmtEditNumeric (see page 864)

Example

```
ADQuery1.FormatOptions.FmtDisplayDate := 'yyyy.mm.dd';
```

1.17.25.1.6.2.5 TADFormatOptions.FmtDisplayDateTime Property

Specifies display format for ftDateDate, ftTimeStamp fields.

Description

Use the FmtDisplayDateTime property to specify DisplayFormat property for TDateTimeField / ftDate, TSQLTimeStampField / ftTimeStamp dataset fields.

The option will be applied only to default fields or to persistent fields at the create time only. For format details see TDateTimeField.DisplayFormat.

Syntax

```
property FmtDisplayDateTime: String;
```

See Also

FmtDisplayDate (see page 863), FmtDisplayTime (see page 864), FmtDisplayNumeric (see page 864), FmtEditNumeric (see page 864)

Example

```
ADQuery1.FormatOptions.FmtDisplayDateTime := 'yyyy.mm.dd hh:mm';
```

1.17.25.1.6.2.6 TADFormatOptions.FmtDisplayNumeric Property

Specifies display format for numeric fields.

Description

Use the FmtDisplayNumeric property to specify DisplayFormat property for TNumericField descendants (ftInteger, ftLongWord, ftSmallint, ftShortint, ftByte, ftLargeint, ftWord, ftAutoInc, ftFloat, ftSingle, ftCurrency, ftExtended, ftBCD, ftFmtBCD) dataset fields.

The option will be applied only to default fields or to persistent fields at the create time only. For format details see TNumericField.DisplayFormat.

Syntax

```
property FmtDisplayNumeric: String;
```

See Also

FmtDisplayDateTime (see page 863), FmtDisplayDate (see page 863), FmtDisplayTime (see page 864), FmtEditNumeric (see page 864)

Example

```
ADQuery1.FormatOptions.FmtDisplayNumeric := '#,##0.';
```

1.17.25.1.6.2.7 TADFormatOptions.FmtDisplayTime Property

Specifies display format for ftTime fields.

Description

Use the FmtDisplayTime property to specify DisplayFormat property for TTimeField / ftTime dataset fields.

The option will be applied only to default fields or to persistent fields at the create time only. For format details see TTimeField.DisplayFormat.

Syntax

```
property FmtDisplayTime: String;
```

See Also

FmtDisplayDateTime (see page 863), FmtDisplayDate (see page 863), FmtDisplayNumeric (see page 864), FmtEditNumeric (see page 864)

Example

```
ADQuery1.FormatOptions.FmtDisplayTime := 'hh:mm';
```

1.17.25.1.6.2.8 TADFormatOptions.FmtEditNumeric Property

Specifies edit format for numeric fields.

Description

Use the FmtEditNumeric property to specify EditFormat property for TNumericField descendants (ftInteger, ftLongWord, ftSmallint, ftShortint, ftByte, ftLargeint, ftWord, ftAutoInc, ftFloat, ftSingle, ftCurrency, ftExtended, ftBCD, ftFmtBCD) dataset fields.

The option will be applied only to default fields or to persistent fields at the create time only. For format details see TNumericField.EditFormat.

Syntax

```
property FmtEditNumeric: String;
```

See Also

FmtDisplayDateTime (see page 863), FmtDisplayDate (see page 863), FmtDisplayTime (see page 864), FmtDisplayNumeric (see page 864)

Example

```
ADQuery1.FormatOptions.FmtEditNumeric := '0.';
```

1.17.25.1.6.2.9 TADFormatOptions.InlineDataSize Property

Controls how character or byte string values are stored in record buffer.

Description

Use InlineDataSize to control how character or byte string values are stored in record buffer. The default value is 1000 bytes.

If the dtAnsiString, dtWideString, dtByteString field values are less than or InlineDataSize, then FireDAC stores field values inside of record buffer (inline storage). If equal or greater, then record buffer will keep pointer to the exactly allocated memory space for field value (exact storage).

An inline storage offers good performance at data fetching, but may use much more memory, than an exact storage. A good performance, because no need to allocate a memory for each string value and allows to streamline a rowset fetching. Uses

more memory, because following. For example, a column in a table is declared as VARCHAR(2000). But some record field has 50 characters long value. An inline storage uses 2000 character long space for a value storage, while only 50 characters is required. While exact storage allocates 50 characters long space and stores pointer to this space inside of record buffer.

Also, long string values (BLOB's) always are stored using exact storage.

Syntax

```
property InlineDataSize: Integer;
```

See Also

MaxStringSize (see page 866)

1.17.25.1.6.2.10 TADFormatOptions.MapRules Property

Controls data type mapping.

Description

Use MapRules property to setup data type mapping schema. By default MapRules is empty.

Read Data Type Mapping (see page 35) for more details. The MapRules are used by FireDAC when OwnMapRules is True.

Syntax

```
property MapRules: TADMapRules;
```

See Also

Data Type Mapping (see page 35), TADMapRules (see page 873), TADMapRule (see page 870), OwnMapRules (see page 866), MaxStringSize (see page 866), MaxBcdPrecision (see page 865), MaxBcdScale (see page 866)

1.17.25.1.6.2.11 TADFormatOptions.MaxBcdPrecision Property

Controls recognition of numeric values as FmtBCD ones, depending on precision.

Description

Use the MaxBcdPrecision property together with the MaxBcdScale property to control, represent numeric column data type as dtFmtBCD or as dtBCD. The default value is 18.

If numeric column precision is less than or equal to MaxBcdPrecision, then numeric column data type is defined as dtBCD, otherwise as dtFmtBCD.

Syntax

```
property MaxBcdPrecision: Integer;
```

See Also

Data Type Mapping (see page 35), MaxBcdScale (see page 866)

1.17.25.1.6.2.12 TADFormatOptions.MaxBcdScale Property

Controls recognition of numeric values as FmtBCD ones, depending on scale.

Description

Use the MaxBcdScale property together with the MaxBcdPrecision property to control, represent numeric column data type as dtFmtBCD or as dtBCD. The default value is 4.

If numeric column scale is less than or equal to MaxBcdScale, then numeric column data type is defined as dtBCD, otherwise as dtFmtBCD.

Syntax

```
property MaxBcdScale: Integer;
```

See Also

Data Type Mapping (see page 35), MaxBcdPrecision (see page 865)

1.17.25.1.6.2.13 TADFormatOptions.MaxStringSize Property

Controls recognition of string values as BLOB ones.

Description

Use MaxStringSize property to specify the maximum string column size in characters. If a column is longer than MaxStringSize, then FireDAC represents its data type as one from BLOB family. The default value is 32767 bytes.

MaxStringSize has a special meaning for Oracle connection. When specified, then it will be a maximum size for the string typed parameters of PL/SQL commands, like a stored procedure call or explicit PL/SQL anonymous block call. The 255 characters is a special reserved value and does not have an effect.

Syntax

```
property MaxStringSize: LongWord;
```

See Also

Data Type Mapping (see page 35), InlineDataSize (see page 865)

1.17.25.1.6.2.14 TADFormatOptions.OwnMapRules Property

Controls usage of MapRules (see page 865) property.

Description

Set OwnMapRules to True to tell FireDAC to use MapRules property of this TADFormatOptions instance. If OwnMapRules is False, then MapRules will be inherited from parent object (TADConnection or TADManager). By default OwnMapRules is False.

Syntax

```
property OwnMapRules: Boolean;
```

See Also

Data Type Mapping (see page 35), MapRules (see page 865)

1.17.25.1.6.2.15 TADFormatOptions.QuoteIdentifiers Property

Specifies whether or not to explicitly surround identifiers with quotes.

Description

Use the QuoteIdentifiers property to explicitly quote identifiers, such as column and table names. That helps if identifiers in a database are quoted and must not be normalized but pointed in all queries as they were defined.

Normally FireDAC will automatically decide should it or not to quote an identifier. But in several cases an application has specify to FireDAC to quote identifiers:

- when a DB table has quoted column names and this table is specified in TADTable.TableName (see page 559).

Syntax

```
property QuoteIdentifiers: Boolean;
```

Example

```
// Firebird table:
// create table "Records"(
//   "Id" int,
//   "RecId" int,
//   "RecName" varchar(15)
// )

ADTable1.FormatOptions.QuoteIdentfiers := True;
```

```
ADTable1.TableName := '"Records"';
ADTable1.IndexFieldNames := 'RecId';
ADTable1.Open;
```

1.17.25.1.6.2.16 TADFormatOptions.Round2Scale Property

Controls values rounding.

Description

Use Round2Scale property to control, should FireDAC round the numeric and time values to the scale specified in a database or not. The default value is False.

If it is True, then rounding will be performed. The rounding happens at the client side, only when an application assigns a value to a field of numeric / time / date & time type. The rounding uses a corresponding TADDatSColumn.Scale value. For numbers it is amount of digits after decimal point. For times it is maximal supported precision in msecs.

Setting Round2Scale property to True, allows to an application to avoid potential rounding errors. Lets consider the scenario. The application has UpdateOptions.LockMode = lmOptimistic and ADQuery with simple SELECT to an Oracle table with a DATE field. The DATE data type in Oracle has 1 sec precision. Now add a new record, assign Now() result to the DATE field and post the change to the database. The client record buffer will store a value with some msecs, while the database will store a value WITHOUT msecs. Lets try to edit the record. Before that the lock will be performed. But FireDAC will fail to compare the field values from the client buffer and the one just fetched from the database. As result "DApt-400" exception will be raised. But if to set Round2Scale to True, then FireDAC will round a client value to seconds and a comparison will succeed.

Notes

At moment (v 2.0.11) only time rounding is supported.

Syntax

```
property Round2Scale: Boolean;
```

1.17.25.1.6.2.17 TADFormatOptions.SortLocale Property

Specifies a locale ID for the local sorting operations.

Description

Use the SortLocale property to specify a locale ID for the local sorting operations. The default value is LOCALE_USER_DEFAULT.

The local sorting is performed by the TADDataSet.IndexFieldNames (see page 600) and TADDataSet.Indexes (see page 598) properties. When the sorting is performed on a string column, the Windows API function CompareString is used. The string comparison is performed using the SortLocale locale.

For TADMemTable, TADQuery and TADStoredProc the specifying of SortLocale is always optional. For TADTable it may be required, when the database collation differs from the default locale. Additionally the SortOptions may be used.

To specify a locale ID the MAKELCID, MAKELANGID and DownlevelLocaleNameToLCID Windows API functions may be used. To enable binary string comparision specify 0.

Syntax

```
property SortLocale: TADLocalID;
```

See Also

TADIndex (see page 653).Options (see page 658), TADDataSet (see page 584).IndexFieldNames (see page 600), TADDataSet (see page 584).Indexes (see page 598), SortOptions (see page 868)

Example

Setup TADTable (see page 539) to query the Firebird database with ISO8859_1 character set containing German language string data:

```
uses
  Windows;
...
```

```
// Set locale ID to German phone book collation
ADTable1.FormatOptions.SortLocale := MAKELCID(MAKELANGID (LANG_GERMAN, SUBLANG_GERMAN),
SORT_DEFAULT);
// Use the the punctuation and other symbols insensitive sorting
ADTable1.FormatOptions.SortOptions := [soNoSymbols];
ADTable1.IndexFieldNames := 'NAME';
ADTable1.TableName := 'CUSTOMERS';
ADTable1.Open;
```

1.17.25.1.6.2.18 TADFormatOptions.SortOptions Property

Specifies the default sort options for the local sorting operations.

Description

Use the SortOptions property to specify the default sorting options for the local sorting operations. See TADIndex.Options (see page 658) for details. The default value is [].

The local sorting is performed by the TADDataSet.IndexFieldNames (see page 600) and TADDataSet.Indexes (see page 598) properties. The SortOptions is the default value for an index in the Indexes (see page 598) collection and may be overridden by the TADIndex.Options (see page 658) property. The SortOptions is the only way to specify the options for IndexFieldNames (see page 600).

For TADMemTable, TADQuery and TADStoredProc the specifying of SortOptions is always optional. For TADTable it may be required, when the database collation differs from the default locale. Then consider to specify soNoCase, soNullLess, soNoSymbols.

Additionally the SortLocale may be used.

Syntax

```
property SortOptions: TADSortOptions;
```

See Also

TADIndex (see page 653).Options (see page 658), TADDataSet (see page 584).IndexFieldNames (see page 600), TADDataSet (see page 584).Indexes (see page 598), SortLocale (see page 867)

Example 1

Setup TADTable (see page 539) to query the Firebird database with ISO8859_1 character set containing German language string data:

```
uses
  Windows;
...
// Set locale ID to German phone book collation
ADTable1.FormatOptions.SortLocale := MAKELCID(MAKELANGID (LANG_GERMAN, SUBLANG_GERMAN),
SORT_DEFAULT);
// Use the the punctuation and other symbols insensitive sorting
ADTable1.FormatOptions.SortOptions := [soNoSymbols];
ADTable1.IndexFieldNames := 'NAME';
ADTable1.TableName := 'CUSTOMERS';
ADTable1.Open;
```

Example 2

Setup IndexFieldNames sort order properties for TADMemTable (see page 436):

```
ADMemTable1.FormatOptions.SortOptions := [soNullFirst, soNoCase];
ADMemTable1.IndexFieldNames := 'SECOND_NAME;FIRST_NAME';
```

1.17.25.1.6.2.19 TADFormatOptions.StrsEmpty2Null Property

Controls conversion of zero-length string values to NULL value.

Description

Use StrsEmpty2Null property to control, should FireDAC convert string values with zero length to NULL value (True) or not (False). The default value is False.

The property applies to column values at rows fetching and to parameter values at command execution. The property will be

applied after StrsTrim is applied. Note, when an application is executing a query:

```
ADQuery1.SQL.Text := 'SELECT * FROM MyTab WHERE fld = :p';
ADQuery1.Params[0].AsString := '';
ADQuery1.Open;
```

And the query returns no records, although the MyTab has some records with empty string in FLD field, then you have to set StrsEmpty2Null to False to get those records.

Syntax

```
property StrsEmpty2Null: Boolean;
```

See Also

StrsTrim (see page 869), StrsTrim2Len (see page 870)

1.17.25.1.6.2.20 TADFormatOptions.StrsTrim Property

Controls removing of trailing spaces from string values.

Description

Use StrsTrim property to specify, should FireDAC remove trailing spaces from fixed length string values (True) or not (False). The default value is True.

The property applies to fixed length column values at rows fetching and to parameter values at command execution. The property will be applied before StrsEmpty2Null is applied. The property is used only for trimming of the fixed length string values, like a CHAR / NCHAR. It does not affect the variable length string values, like a VARCHAR / NVARCHAR, and the BLOB values.

Note, for SQLite this property is applied to all string columns, including ftMemo, ftWideMemo, ftString, ftWideString, ftFixedChar and ftFixedWideChar.

Syntax

```
property StrsTrim: Boolean;
```

See Also

StrsEmpty2Null (see page 869), StrsTrim2Len (see page 870), Using SQLite with FireDAC (see page 135)

1.17.25.1.6.2.21 TADFormatOptions.StrsTrim2Len Property

Controls trimming string values to the declared length.

Description

Use StrsTrim2Len property to specify, should FireDAC trim too long string values (True) to the maximum declared length or not (False). The default value is False.

For example, when application is trying to assign '12345' to the field declared in database as VARCHAR(3), then FireDAC will raise an exception 'value too long'. With StrsTrim2Len = True, FireDAC will trim the value to '123'.

Syntax

```
property StrsTrim2Len: Boolean;
```

See Also

StrsTrim (see page 869), StrsEmpty2Null (see page 869)

1.17.25.1.7 TADMapRule Class

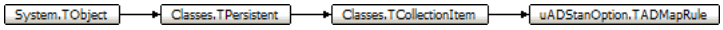
TADMapRule represents a maintained client view to records in an FireDAC dataset...

published

published	Description
NameMask (see page 871)	Specifies column name mask.

PrecMax (see page 871)	Specifies maximal numeric precision.
PrecMin (see page 871)	Specifies minimal numeric precision.
ScaleMax (see page 872)	Specifies maximal numeric scale.
ScaleMin (see page 872)	Specifies minimal numeric scale.
SizeMax (see page 872)	Specifies maximal string size.
SizeMin (see page 873)	Specifies minimal string size.
SourceDataType (see page 873)	Specifies a source data type.
TargetDataType (see page 873)	Specifies a target data type.

Class Hierarchy



File

uADStanOption

Description

Use TADMapRule to create and maintain data type mapping rule.

TADMapRule are collected into TADMapRules (see page 873) collection. The set of TADMapRule may be created as at design time, as at run time.

A map rules collection is used by TADFormatOptions.MapRules (see page 865).

Syntax

```
TADMapRule = class(TCollectionItem);
```

See Also

TADFormatOptions.MapRules (see page 865), TADMapRules (see page 873)

1.17.25.1.7.1 published

1.17.25.1.7.1.1 TADMapRule.NameMask Property

Specifies column name mask.

Description

Use NameMask property to specify a mask to match the column names. The mask uses LIKE operator format, where:

- '%' - many symbols;
- '_' - one any symbol;
- '\' - escape character.

If a column name matches a mask, then exactly this column matches to this map rule. And the column will get TargetDataType type. Other properties may be used to strict the rule.

Syntax

```
property NameMask: String;
```

See Also

TADFormatOptions.MapRules (see page 865), SourceDataType (see page 873), TargetDataType (see page 873)

Example 1

```
// maps all columns ending with ID to dtInt32
with ADQuery1.FormatOptions.MapRules.Add do begin
  NameMask := '%ID';
  TargetDataType := dtInt32;
end;
```

Example 2

```
// maps all ID columns of dtFmtBCD data type to dtInt32
with ADQuery1.FormatOptions.MapRules.Add do begin
  NameMask := 'ID';
  SourceDataType := dtFmtBCD;
  TargetDataType := dtInt32;
end;
```

1.17.25.1.7.1.2 TADMapRule.PrecMax Property

Specifies maximal numeric precision.

Description

Use PrecMax property to specify maximal numeric precision for source data type. If precision will be greater than PrecMax, then a data type does not match to this map rule. If PrecMax is equal to -1, then PrecMax will be not used in rule matching. The default value is -1.

Maximal numeric precision is the maximum number of digits in the number data types.

Syntax

```
property PrecMax: Integer;
```

See Also

TADFormatOptions.MapRules (see page 865), PrecMin (see page 871)

1.17.25.1.7.1.3 TADMapRule.PrecMin Property

Specifies minimal numeric precision.

Description

Use PrecMin property to specify minimal numeric precision for source data type. If precision will be less than PrecMin, then a data type does not match to this map rule. If PrecMin is equal to -1, then PrecMin will be not used in rule matching. The default value is -1.

Minimal numeric precision is the minimal number of digits in the number data types.

Syntax

```
property PrecMin: Integer;
```

See Also

TADFormatOptions.MapRules (see page 865), PrecMax (see page 871)

1.17.25.1.7.1.4 TADMapRule.ScaleMax Property

Specifies maximal numeric scale.

Description

Use ScaleMax property to specify maximal numeric scale for source data type. If scale will be greater than ScaleMax, then a data type does not match to this map rule. If ScaleMax is equal to -1, then ScaleMax will be not used in rule matching. The default value is -1.

Maximal numeric scale is the maximum number of digits after decimal point in the number data types. Zero specifies integer data type.

Syntax

```
property ScaleMax: Integer;
```

See Also

TADFormatOptions.MapRules (see page 865), ScaleMin (see page 872)

1.17.25.1.7.1.5 TADMapRule.ScaleMin Property

Specifies minimal numeric scale.

Description

Use ScaleMin property to specify minimal numeric scale for source data type. If scale will be less than ScaleMin, then a data type does not match to this map rule. If ScaleMin is equal to -1, then ScaleMin will be not used in rule matching. The default value is -1.

Minimal numeric scale is the minimum number of digits after decimal point in the number data types.

Syntax

```
property ScaleMin: Integer;
```

See Also

TADFormatOptions.MapRules (see page 865), ScaleMax (see page 872)

1.17.25.1.7.1.6 TADMapRule.SizeMax Property

Specifies maximal string size.

Description

Use SizeMax property to specify maximal string length for source data type. If length will be greater than SizeMax, then a data type does not match to this map rule. If SizeMax is equal to -1, then SizeMax will be not used in rule matching. The default value is -1.

The length is in characters for Ansi and Unicode string data types and in bytes for byte string data types.

Syntax

```
property SizeMax: LongWord;
```

See Also

TADFormatOptions.MapRules (see page 865), SizeMin (see page 873)

1.17.25.1.7.1.7 TADMapRule.SizeMin Property

Specifies minimal string size.

Description

Use SizeMin property to specify minimal string length for source data type. If length will be less than SizeMin, then a data type does not match to this map rule. If SizeMin is equal to -1, then SizeMin will be not used in rule matching. The default value is -1.

The length is in characters for Ansi and Unicode string data types and in bytes for byte string data types.

Syntax

```
property SizeMin: LongWord;
```

See Also

TADFormatOptions.MapRules (see page 865), SizeMax (see page 872)

1.17.25.1.7.1.8 TADMapRule.SourceDataType Property

Specifies a source data type.

Description

Use SourceDataType to specify source data type, returned by driver for result set column.

Syntax

```
property SourceDataType: TADDataType;
```

See Also

TADFormatOptions.MapRules (see page 865), TargetDataType (see page 873)

1.17.25.1.7.1.9 TADMapRule.TargetDataType Property

Specifies a target data type.

Description

Use TargetDataType to specify target data type, preferred by application.

Syntax

```
property TargetDataType: TADDataType;
```

See Also

TADFormatOptions.MapRules (see page 865), SourceDataType (see page 873)

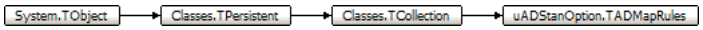
1.17.25.1.8 TADMapRules Class

TADMapRules is a collection of TADMapRule (see page 870) objects.

public

public	Description
Items (see page 874)	Returns TADMapRule (see page 870) object by it index.
Add (see page 874)	Appends new TADMapRule (see page 870) object to a collection.

Class Hierarchy



File

uADStanOption

Description

TADMapRules is a list of data type mapping rules. A single TADMapRule (see page 870) object represents single mapping rule.

A map rules collection is used by TADFormatOptions.MapRules (see page 865).

Syntax

```
TADMapRules = class(TCollection);
```

See Also

TADFormatOptions.MapRules (see page 865), TADMapRule (see page 870)

1.17.25.1.8.1 public

1.17.25.1.8.1.1 TADMapRules.Items Property

Returns TADMapRule (see page 870) object by it index.

Description

Use Items indexed property to get TADMapRule (see page 870) object by it index.

The property is default one, so you does not need to write ADQuery1.FormatOptions.MapRules.Items[i], just ADQuery1.FormatOptions.MapRules[i].

Syntax

```
property Items [AIndex: Integer]: TADMapRule;
```

See Also

TADMapRule (see page 870), TCollection.Count

1.17.25.1.8.1.2 TADMapRules.Add Method

Appends new TADMapRule (see page 870) object to a collection.

Description

Call Add method to add new empty TADMapRule (see page 870) object to a collection.

Syntax

```
function Add: TADMapRule; overload;
```

See Also

Items (see page 874), TCollection.Count

Example

```
with ADConnection1.FormatOptions.MapRules.Add do begin
  SourceDataType := dtWideString;
  TargetDataType := dtAnsiString;
end;
```

1.17.25.1.9 TADResourceOptions Class

The set of options controlling SQL command preprocessor, command execution and dataset persistence.

public

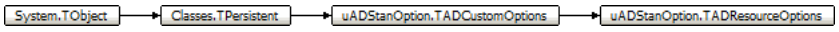
public	Description
ActualSilentMode (see page 876)	Returns actual silent mode a dataset will use.
BackupExt (see page 876)	Specifies persistent dataset backup copy extension.
BackupFolder (see page 876)	Specifies persistent dataset backup copy folder.
DefaultStoreExt (see page 877)	Specifies default extension for persistent dataset data files.
DefaultStoreFolder (see page 877)	Specifies the default folder for the persistent dataset data files.
DefaultStoreFormat (see page 877)	Specifies the default format for the persistent dataset data files.
PreprocessCmdText (see page 878)	Controls SQL command text preprocessor.
Assign (see page 878)	Assigns all options of one option object to another.
RestoreDefaults (see page 878)	Restore option default values.

published

published	Description
ArrayDMLSize (see page 879)	Limits the maximum possible Array DML array size.
AssignedValues (see page 879)	Specifies the set of changed options.
Backup (see page 879)	Controls backup of data file for persistent datasets.
CmdExecMode (see page 880)	Controls DBMS action execution mode.
CmdExecTimeout (see page 881)	Controls DBMS action execution timeout.
DefaultParamType (see page 881)	Specifies default parameter direction.
DirectExecute (see page 881)	Controls preparation of SQL statements.
EscapeExpand (see page 882)	Controls processing of escape sequences.
MacroCreate (see page 882)	Controls automatic filling of Macros collection.
MacroExpand (see page 882)	Controls expansion of macros.
ParamCreate (see page 883)	Controls automatic filling of Params collection.
ParamExpand (see page 883)	Controls expansion of parameters.
Persistent (see page 884)	Controls dataset persistence.
SilentMode (see page 884)	Controls dataset GUI feedback.

StoreItems (see page 884)	Specifies the data to write into the persistent dataset data files or streams.
StoreVersion (see page 885)	Specifies the format version for the persistent dataset data files or streams.
UnifyParams (see page 885)	Controls unification of a stored procedure parameters.

Class Hierarchy



File

uADStanOption

Description

The TADResourceOptions class represents the set of properties controlling how FireDAC components and Phys Layer command interfaces:

- preprocess SQL command text (EscapeExpand (see page 882), MacroCreate (see page 882), MacroExpand (see page 882), ParamCreate (see page 883), ParamExpand (see page 883), PreprocessCmdText (see page 878), UnifyParams (see page 885));
- prepare command (DirectExecute (see page 881), DefaultParamType (see page 881));
- execute command (CmdExecMode (see page 880), CmdExecTimeout (see page 881), ArrayDMLSize (see page 879));
- manage resources (SilentMode (see page 884));
- persist datasets (Persistent (see page 884), Backup (see page 879), BackupExt (see page 888), BackupFolder (see page 876), DefaultStoreExt (see page 877), DefaultStoreFolder (see page 877), DefaultStoreFormat (see page 877), StoreVersion (see page 885), StoreItems (see page 884)).

The TADResourceOptions is intermediate class in resource options classes hierarchy. The TADCustomManager (see page 365) and TADCustomConnection (see page 318) classes use TADTopResourceOptions (see page 885). The TADCustomCommand (see page 291) and TADDataSet (see page 584) use TADBottomResourceOptions (see page 841).

Syntax

```
TADResourceOptions = class(TADCustomOptions);
```

See Also

Setting Options (see page 34), TADCustomManager.ResourceOptions (see page 374), TADCustomConnection.ResourceOptions (see page 335), TADCustomCommand.ResourceOptions (see page 304), TADQuery.ResourceOptions (see page 498), IADPhysCommand.Options

1.17.25.1.9.1 public

1.17.25.1.9.1.1 TADResourceOptions.ActualSilentMode Property

Returns actual silent mode a dataset will use.

Description

Use ActuaSilentMode to get actual silent mode a dataset will use. Actual silent mode is conjunction of SilentMode (see page 884) and uADGUIxIntf.ADGUIxSilent.

Syntax

```
property ActualSilentMode: Boolean;
```

See Also

SilentMode (see page 884), uADGUIxIntf.ADGUIxSilent

1.17.25.1.9.1.2 TADResourceOptions.BackupExt Property

Specifies persistent dataset backup copy extension.

Description

Use BackupExt property to specify extension, which will get backup copy of persistent dataset file. Default value is '.bak'.

The backup copy file gets the same base file name as it is specified by TADBottomResourceOptions.PersistentFileName (see page 843), BackupExt extension and will be put to BackupFolder (see page 876) folder.

Syntax

```
property BackupExt: String;
```

See Also

Backup (see page 879), BackupFolder (see page 876), TADBottomResourceOptions.PersistentFileName (see page 843)

1.17.25.1.9.1.3 TADResourceOptions.BackupFolder Property

Specifies persistent dataset backup copy folder.

Description

Use BackupExt property to specify extension, which will get backup copy of persistent dataset file. The default value is current folder.

The backup copy file gets the same base file name as it is specified by TADBottomResourceOptions.PersistentFileName (see page 843), BackupExt (see page 876) extension and will be put to BackupFolder folder.

Syntax

```
property BackupFolder: String;
```

See Also

Backup (see page 879), BackupExt (see page 876), TADBottomResourceOptions.PersistentFileName (see page 843)

1.17.25.1.9.1.4 TADResourceOptions.DefaultStoreExt Property

Specifies default extension for persistent dataset data files.

Description

Use DefaultStoreExt property to specify default extension for data file, where persistent dataset stores its data. The default value is empty string.

FireDAC uses DefaultStoreExt, when PersistentFileName (see page 843) property value or AFileName argument of the SaveToFile (see page 644) method does not have extension specified.

Syntax

```
property DefaultStoreExt: String;
```

See Also

DefaultStoreFormat (see page 877), DefaultStoreFolder (see page 877), TADBottomResourceOptions.PersistentFileName (see page 843)

1.17.25.1.9.1.5 TADResourceOptions.DefaultStoreFolder Property

Specifies the default folder for the persistent dataset data files.

Description

Use DefaultStoreFolder property to specify the default folder for the data file, where persistent dataset stores its data. The default value is empty string.

FireDAC uses DefaultStoreFolder, when PersistentFileName (see page 843) property value or AFileName argument of the

SaveToFile (see page 644) method does not have a folder specified.

Syntax

```
property DefaultStoreFolder: String;
```

See Also

DefaultStoreFormat (see page 877), DefaultStoreExt (see page 877), TADBottomResourceOptions.PersistentFileName (see page 843)

1.17.25.1.9.1.6 TADResourceOptions.DefaultStoreFormat Property

Specifies the default format for the persistent dataset data files.

Description

Use DefaultStoreFormat property to specify the default format for the data file, where dataset stores its persistent data. The default value is sfBinary.

FireDAC uses DefaultStoreFormat, when PersistentFileName (see page 843) property value or AFileName argument of the SaveToFile (see page 644) method does not have an extension specified, and DefaultStoreExt (see page 877) property value is empty.

Syntax

```
property DefaultStoreFormat: TADStorageFormat;
```

See Also

DefaultStoreExt (see page 877), DefaultStoreFolder (see page 877), TADBottomResourceOptions.PersistentFileName (see page 843)

1.17.25.1.9.1.7 TADResourceOptions.PreprocessCmdText Property

Controls SQL command text preprocessor.

Description

Use PreprocessCmdText property to enable / disable SQL command text preprocessor. The default value is True.

The property is shortcut to 5 properties, allowing to control all aspects of SQL command text preprocessor:

- EscapeExpand (see page 882);
- MacroCreate (see page 882);
- MacroExpand (see page 882);
- ParamCreate (see page 883);
- ParamExpand (see page 883).

Setting PreprocessCmdText to some value, sets all these 5 properties to the same value. By default it is True. Setting it to False, allows to speed up processing of long SQL command texts (tens of KB).

Syntax

```
property PreprocessCmdText: Boolean;
```

See Also

EscapeExpand (see page 882), MacroCreate (see page 882), MacroExpand (see page 882), ParamCreate (see page 883), ParamExpand (see page 883)

1.17.25.1.9.1.8 TADResourceOptions.Assign Method

Assigns all options of one option object to another.

Parameters

Parameters	Description
ASource: TPersistent	A source options object.

Description

Use the Assign method to assign all changed option values of ASource option object to this object. After that corresponding options will stop inheriting the values from more high level option object.

Syntax

```
procedure Assign(ASource: TPersistent); override;
```

See Also

RestoreDefaults

Example

```
ADQuery2.UpdateOptions.Assign(ADQuery1.UpdateOptions);
```

1.17.25.1.9.1.9 TADResourceOptions.RestoreDefaults Method

Restore option default values.

Description

Call RestoreDefaults to reset all changed options to their default values.

Syntax

```
procedure RestoreDefaults; override;
```

See Also

AssignedValues (see page 879)

1.17.25.1.9.2 published

1.17.25.1.9.2.1 TADResourceOptions.ArrayDMLSize Property

Limits the maximum possible Array DML array size.

Description

Set ArrayDMLSize to the maximum possible Array DML (see page 82) size, which is supported by the target DBMS. By default array DML size is unlimited (MAXINT).

If actual array DML size is greater than ArrayDMLSize, then FireDAC will transparently split one big batch into few smaller, where each will be of ArrayDMLSize size or less. Some DBMS (Firebird, MS SQL Server) have problems with large array DML size. If you experience some "strange" problems with array DML, then try to set ArrayDMLSize.

Syntax

```
property ArrayDMLSize: Integer;
```

See Also

Array DML (see page 82), TADDataSet.Execute

1.17.25.1.9.2.2 TADResourceOptions.AssignedValues Property

Specifies the set of changed options.

Description

Use AssignedValues to get / set the set of modified option values.

If some option is changed at this level, then corresponding flag is included into AssignedValues. If flag is not included, then an option value is inherited from more high level. Excluding flag from AssignedValues makes option inheriting its value from

more high level.

See Setting Options (see page 34) for more details.

Syntax

```
property AssignedValues: TADResourceOptionValues;
```

See Also

RestoreDefaults (see page 878)

1.17.25.1.9.2.3 TADResourceOptions.Backup Property

Controls backup of data file for persistent datasets.

Description

Use Backup property to control, should persistent dataset backup data file, where persistent dataset stores its data, before it will be overridden. If it is True, then backup copy of data file will be created. By default it is False.

The backup copy file gets the same base file name as it is specified by TADBottomResourceOptions.PersistentFileName (see page 843), BackupExt (see page 876) extension and will be put to BackupFolder (see page 876) folder.

Syntax

```
property Backup: Boolean;
```

See Also

Persistent (see page 884), Backup, BackupExt (see page 876), BackupFolder (see page 876)

1.17.25.1.9.2.4 TADResourceOptions.CmdExecMode Property

Controls DBMS action execution mode.

Description

Use CmdExecMode property to control DBMS action execution mode, at first it synchronism:

Mode	Description
amBlocking	The calling thread and GUI are blocked until an action will be finished.
amNonBlocking	The calling thread is blocked until an action will be finished. The GUI is not blocked.
amCancelDialog	The calling thread and GUI are blocked until an action will be finished. FireDAC shows dialog, allowing to cancel an action.
amAsync	The calling thread and GUI are not blocked. The called method will return immediately.

The default value is amBlocking. This property controls the following methods synchronism:

- TADDataSet.Open (see page 509), TADCustomCommand.Open (see page 316), IADPhysCommand.Open;
- TADDataSet.Execute (see page 622), TADCustomCommand.Execute (see page 310), IADPhysCommand.Execute;
- fetching in TADDataSet, TADCustomCommand.Fetch (see page 312), IADPhysCommand.Fetch.

If mode is amNonBlocking, then application continues to process GUI messages, but ignores mouse and keyboard events.

If mode is amCancelDialog, then application should include TADGUIxAsyncExecuteDialog (see page 669) component into any form or data module. If it is not included, then exception will be raised.

The dialog has the "Cancel" button, allowing to terminate current action. If DBMS does not support action canceling, then button will be hidden.

If mode is amAsync, an application can get notification about action finish using appropriate event handler or callback:

Class	Method	Event
TADDataSet	Open	AfterOpen event

	Execute	AfterExecute event
TADCustomCommand	Open	AfterOpen event
	Execute	AfterExecute event
	Fetch	AfterFetch event
IADPhysCommand	Open	StateHandler.HandleFinished callback
	Execute	StateHandler.HandleFinished callback
	Fetch	StateHandler.HandleFinished callback

Application can control action timeout by setting CmdExecTimeout (see page 881) property.

Syntax

```
property CmdExecMode: TADStanAsyncMode;
```

See Also

Asynchronous Execution (see page 86), CmdExecTimeout (see page 881), TADDataSet.Open, TADCustomCommand.Open, IADPhysCommand.Open, TADDataSet.Execute, TADCustomCommand.Execute, IADPhysCommand.Execute, TADCustomCommand.Fetch, IADPhysCommand.Fetch

Example

See demos:

- FireDAC\Samples\Comp Layer\TADQuery (see page 476)\ExecSQL\Async;
- FireDAC\Samples\Phys Layer\IADPhysCommand\Async.

1.17.25.1.9.2.5 TADResourceOptions.CmdExecTimeout Property

Controls DBMS action execution timeout.

Description

Use CmdExecTimeout property to control the DBMS action execution timeout. The value <= 0 means - no timeout. The default value is -1. The time is in milliseconds (0,001 sec).

This property controls the following methods timeout:

- TADDataSet.Open (see page 509), TADCustomCommand.Open (see page 316), IADPhysCommand.Open;
- TADDataSet.Execute (see page 622), TADCustomCommand.Execute (see page 310), IADPhysCommand.Execute;
- fetching in TADDataSet, TADCustomCommand.Fetch (see page 312), IADPhysCommand.Fetch.

Syntax

```
property CmdExecTimeout: LongWord;
```

See Also

Asynchronous Execution (see page 86), CmdExecMode (see page 880), TADDataSet.Open, TADCustomCommand.Open, IADPhysCommand.Open, TADDataSet.Execute, TADCustomCommand.Execute, IADPhysCommand.Execute, TADCustomCommand.Fetch, IADPhysCommand.Fetch

Example 1

See demos:

- FireDAC\Samples\Comp Layer\TADQuery (see page 476)\ExecSQL\Async;
- FireDAC\Samples\Phys Layer\IADPhysCommand\Async.

Example 2

```
// set timeout to 5 seconds
ADQuery1.ResourceOptions.CmdExecTimeout := 5000;
ADQuery1.ExecSQL;
```

1.17.25.1.9.2.6 TADResourceOptions.DefaultParamType Property

Specifies default parameter direction.

Description

Use DefaultParamType to specify default parameter direction. The default value is ptInput.

If TADParam.ParamType is equal to ptUnknown, then FireDAC will use DefaultParamType. If TADParam.ParamType remains ptUnknown, then exception will be raised.

Syntax

```
property DefaultParamType: TParamType;
```

See Also

TADParam.ParamType

1.17.25.1.9.2.7 TADResourceOptions.DirectExecute Property

Controls preparation of SQL statements.

Description

Use DirectExecute property to specify should FireDAC prepare SQL statement before execution (False) or execute it directly (True). The default value is False.

This property was introduced at first for MS SQL Server support. Where prepared and direct execution may have different effects. Or even prepared execution may fail. If you will get "strange" errors, then try to set DirectExecute to True.

Syntax

```
property DirectExecute: Boolean;
```

1.17.25.1.9.2.8 TADResourceOptions.EscapeExpand Property

Controls processing of escape sequences.

Description

Use EscapeExpand property to control escape sequences expansion. If it is True, then they will be processed and expanded into target DBMS syntax constructions. If it is False, then FireDAC will ignore escape sequences. The default value is True.

Setting EscapeExpand to False, may be usefull, if:

- target DBMS uses '{', '}' symbols in its own SQL dialect and FireDAC cannot recognize them as a SQL construction. Then SQL commands with these symbols may confuse FireDAC SQL preprocessor. See Preprocessing Command Text (see page 56) for escape sequences description.
- application does not use escape sequences at all.

Setting PreprocessCmdText (see page 878) to False, sets EscapeExpand to False.

Syntax

```
property EscapeExpand: Boolean;
```

See Also

Preprocessing Command Text (see page 56), PreprocessCmdText (see page 878), MacroCreate (see page 882), MacroExpand (see page 882), ParamCreate (see page 883), ParamExpand (see page 883)

1.17.25.1.9.2.9 TADResourceOptions.MacroCreate Property

Controls automatic filling of Macros collection.

Description

Use MacroCreate property to control automatic filling of Macros collection. If it is True, SQL preprocessor scans SQL command text for '!' or '&' symbols, extracts macro names and puts them into Macros collection. The scanning will be

performed at each command text change. The default value is True.

Setting MacroCreate to False, may be usefull, if:

- target DBMS uses '!', '&' symbols in its own SQL dialect and FireDAC cannot recognize them as a SQL construction. Then SQL commands with these symbols may confuse FireDAC SQL preprocessor. See Preprocessing Command Text (see page 56) for macros description.
- application fills Macros collection "by hands".
- application does not use macros at all.

Setting PreprocessCmdText (see page 878) to False, sets MacroCreate to False.

Syntax

```
property MacroCreate: Boolean;
```

See Also

Preprocessing Command Text (see page 56), PreprocessCmdText (see page 878), EscapeExpand (see page 882), MacroExpand (see page 882), ParamCreate (see page 883), ParamExpand (see page 883)

1.17.25.1.9.2.10 TADResourceOptions.MacroExpand Property

Controls expansion of macros.

Description

Use MacroExpand property to control macros expansion. If it is True, then their values will be substituted into SQL command text instead of corresponding macro markers. If it is False, then FireDAC does not substitute macro values into command text. The default value is True.

Setting MacroExpand to False, may be usefull if:

- target DBMS uses '!', '&' symbols in its own SQL dialect and FireDAC cannot recognize them as a SQL construction. Then SQL commands with these symbols may confuse FireDAC SQL preprocessor. See Preprocessing Command Text (see page 56) for macros description.
- application does not use macros at all.

Setting PreprocessCmdText (see page 878) to False, sets MacroExpand to False.

Syntax

```
property MacroExpand: Boolean;
```

See Also

Preprocessing Command Text (see page 56), PreprocessCmdText (see page 878), EscapeExpand (see page 882), MacroCreate (see page 882), ParamCreate (see page 883), ParamExpand (see page 883)

1.17.25.1.9.2.11 TADResourceOptions.ParamCreate Property

Controls automatic filling of Params collection.

Description

Use ParamCreate property to control automatic filling of Params collection. If it is True, SQL preprocessor scans SQL command text for ':' symbols, extracts macro names and puts into a Params collection. The scanning will be performed at each command text change. The default value is True.

Setting ParamCreate to False, may be useful if:

- target DBMS uses ':' symbols in its own SQL dialect and FireDAC cannot recognize them as a SQL construction. Then SQL commands with these symbols may confuse FireDAC SQL preprocessor.
- SQL command cannot contain parameters, like in a DDL SQL.
- application does not use parameters at all.

Setting PreprocessCmdText (see page 878) to False, sets ParamCreate to False.

Syntax

```
property ParamCreate: Boolean;
```

See Also

Preprocessing Command Text (see page 56), PreprocessCmdText (see page 878), EscapeExpand (see page 882), MacroExpand (see page 882), ParamCreate, ParamExpand (see page 883)

1.17.25.1.9.2.12 TADResourceOptions.ParamExpand Property

Controls expansion of parameters.

Description

Use ParamExpand property to control parameters expansion. If it is True, then the FireDAC styled markers (:name) will be replaced with the DBMS SQL dialect markers. If it is False, then FireDAC does not replace the parameter markers. The default value is True.

Setting ParamExpand to False, may be usefull if:

- SQL command cannot contain parameters, like a DDL SQL. But a command may contain FireDAC-like markers, which actually are not the parameters. In some cases such markers may confuse FireDAC.
- application does not use parameters at all.

Setting PreprocessCmdText (see page 878) to False, sets ParamsExpand to False.

Syntax

```
property ParamExpand: Boolean;
```

See Also

Preprocessing Command Text (see page 56), PreprocessCmdText (see page 878), EscapeExpand (see page 882), MacroCreate (see page 882), MacroExpand (see page 882), ParamCreate (see page 883)

1.17.25.1.9.2.13 TADResourceOptions.Persistent Property

Controls dataset persistence.

Description

Use Persistent property to control dataset persistence. By default it is False.

If it is True, then dataset will load it content from data file at Open call and will save data to data file at Close call. At dataset will load data, if TADBottomResourceOptions.PersistentFileName (see page 843) is not empty and file exists. A dataset will save data, TADBottomResourceOptions.PersistentFileName (see page 843) is not empty, file does not exists or data was changed after Open call.

Special case is TADCustomMemTable (see page 388). If TADBottomResourceOptions.PersistentFileName (see page 843) is empty, then dataset will save / load data from DFM. So, to save in-memory table content at design time to DFM file, set Persistent to True and clear PersistentFileName (see page 843).

Syntax

```
property Persistent: Boolean;
```

See Also

TADBottomResourceOptions.PersistentFileName

1.17.25.1.9.2.14 TADResourceOptions.SilentMode Property

Controls dataset GUI feedback.

Description

Use SilentMode property to control should the dataset show the SQL-hourglass mouse cursor, when the dataset performs potentially long running actions. When it is False, then the dataset will show the SQL-hourglass. When True, then not. By

default it is False, for TADCustomMemTable (see page 388) it is True.

Setting to True allows to speedup the dataset processing algorithms.

Application can disable all GUI feedback, by setting the uADGUIxIntf.FADGUIxSilentMode to True.

Syntax

```
property SilentMode: Boolean;
```

See Also

uADGUIxIntf.FADGUIxSilentMode

1.17.25.1.9.2.15 TADResourceOptions.StoreItems Property

Specifies the data to write into the persistent dataset data files or streams.

Description

Use StoreItems property to specify the data to write into a data file or a stream, where persistent dataset stores its data. The default value is [siData, siDelta, siMeta].

Value	Description
siData	Store data.
siDelta	Store changes log together with the data.
siMeta	Store dataset metadata, including column definitions, constraints, etc.

Syntax

```
property StoreItems: TADStoreItems;
```

See Also

StoreVersion (see page 885), TADDataSet (see page 584).SaveToStream (see page 645), TADDataSet (see page 584).SaveToFile (see page 644), TADDataSet (see page 584).LoadFromStream (see page 633), TADDataSet (see page 584).LoadFromFile (see page 632)

1.17.25.1.9.2.16 TADResourceOptions.StoreVersion Property

Specifies the format version for the persistent dataset data files or streams.

Description

Use StoreVersion property to specify the format version for a data file or a stream, where dataset stores its persistent data. The default value is -1.

The StoreVersion property is used only to write a data. For reading is used the version stored in a persistent data, so setting StoreVersion for reading does not have an effect.

Syntax

```
property StoreVersion: Integer;
```

See Also

DefaultStoreFormat (see page 877), StoreItems (see page 884), TADDataSet (see page 584).SaveToStream (see page 645), TADDataSet (see page 584).SaveToFile (see page 644), TADDataSet (see page 584).LoadFromStream (see page 633), TADDataSet (see page 584).LoadFromFile (see page 632)

1.17.25.1.9.2.17 TADResourceOptions.UnifyParams Property

Controls unification of a stored procedure parameters.

Description

Use UnifyParams property to specify, should FireDAC unify the stored procedure parameters (True) or not. By default it is False.

The unification includes:

- removing of the special symbols from the stored procedure parameter names. At moment, FireDAC deletes '@' prefix from Microsoft SQL Server and Sybase SQL Anywhere parameter names. Other DBMS's supported by FireDAC do not use a special symbols.
- removing of the special parameters. At moment, FireDAC deletes 'RESULT' parameter from the Microsoft SQL Server and Sybase SQL Anywhere stored procedure parameters set. Other DBMS's supported by FireDAC do not use a special parameters.

Syntax

```
property UnifyParams: Boolean;
```

1.17.25.1.10 TADTopResourceOptions Class

The set of options controlling SQL automatic connection recover, dataset persistence and server output.

public

public	Description
Assign (see page 886)	Assigns all options of one option object to another.
RestoreDefaults (see page 887)	Restore option default values.

published

published	Description
AutoConnect (see page 887)	Controls automatic management of connection Active state.
AutoReconnect (see page 887)	Controls automatic connection recovery.
BackupExt (see page 888)	Specifies persistent dataset backup copy extension.
BackupFolder (see page 888)	Specifies persistent dataset backup copy folder.
DefaultStoreExt (see page 888)	Specifies default extension for persistent dataset data files.
DefaultStoreFolder (see page 889)	Specifies the default folder for the persistent dataset data files.
DefaultStoreFormat (see page 889)	Specifies the default format for the persistent dataset data files.
KeepConnection (see page 889)	Specifies whether an application remains connected to a database even if no datasets are open.
MaxCursors (see page 890)	Controls maximum allowed number of active commands.
ServerOutput (see page 890)	Controls accessibility of the server side messages.
ServerOutputSize (see page 891)	Controls the size of Oracle message output buffer.

Class Hierarchy



File

uADStanOption

Description

The TADTopResourceOptions class represents the set of properties controlling how FireDAC components and Phys Layer command interfaces:

- automatically activate connection (AutoConnect (see page 887));
- recover broken connection (AutoReconnect (see page 887));
- bring server output to a client (ServerOutput (see page 890), ServerOutputSize (see page 891));
- manage connection resources(KeepConnection (see page 889), MaxCursors (see page 890)).

And also persistence of datasets.

The TADTopResourceOptions is terminal class in resource options classes hierarchy. The TADCustomManager (see page 365) and TADCustomConnection (see page 318) classes are using it.

Syntax

```
TADTopResourceOptions = class(TADResourceOptions);
```

See Also

Setting Options (see page 34), TADCustomManager.ResourceOptions (see page 374), TADCustomConnection.ResourceOptions (see page 335)

1.17.25.1.10.1 public

1.17.25.1.10.1.1 TADTopResourceOptions.Assign Method

Assigns all options of one option object to another.

Parameters

Parameters	Description
ASource: TPersistent	A source options object.

Description

Use the Assign method to assign all changed option values of ASource option object to this object. After that corresponding options will stop inheriting the values from more high level option object.

Syntax

```
procedure Assign(ASource: TPersistent); override;
```

See Also

RestoreDefaults

Example

```
ADQuery2.UpdateOptions.Assign(ADQuery1.UpdateOptions);
```

1.17.25.1.10.1.2 TADTopResourceOptions.RestoreDefaults Method

Restore option default values.

Description

Call RestoreDefaults to reset all changed options to their default values.

Syntax

```
procedure RestoreDefaults; override;
```

See Also

AssignedValues

1.17.25.1.10.2 published

1.17.25.1.10.2.1 TADTopResourceOptions.AutoConnect Property

Controls automatic management of connection Active state.

Description

Use AutoConnect property to control automatic management of a connection object Active state. The default value is True. Setting it to True has the following effect:

- Opens connection object, if it is closed and command or dataset, linked to this connection, requires active connection. Otherwise - exception will be raised.
- Closes connection object, if it is open and current call requires inactive connection. Otherwise - exception will be raised.
- Brings connection object to online state, if it is offline and command or dataset, linked to this connection, requires active online connection. Otherwise - exception will be raised.

Syntax

```
property AutoConnect: Boolean;
```

See Also

Establishing Connection (see page 37), AutoReconnect (see page 887), KeepConnection (see page 889), MaxCursors (see page 890)

1.17.25.1.10.2.2 TADTopResourceOptions.AutoReconnect Property

Controls automatic connection recovery.

Description

Use AutoReconnect property to control automatic connection recovery (see page 38) feature.

Set it to True, so when FireDAC will discover that connection is broken, it will try to transparently reestablish it. If it is False, then FireDAC will raise exception and will not try to reconnect.

Application can control connection recovery, using TADCustomConnection (see page 318) events. See TADCustomConnection (see page 318).OnRecover (see page 333) for details. It is important to understand, that 100% transparent recovery is not possible. After connection is recovered:

- all datasets has the same Active state as before connection recovery;
- all datasets has the same Prepared state as before connection recovery;
- all not yet fetched records are discarded and corresponding cursors are closed;
- all transactions are inactive;
- all event alerters are unregistered.

An application will need to take similar actions, as after logging to the DBMS server.

Syntax

```
property AutoReconnect: Boolean;
```

See Also

Recovering Connection (see page 38), TADCustomConnection.OnLosted (see page 333), TADCustomConnection.OnRecover (see page 333), TADCustomConnection.OnRestored (see page 334)

1.17.25.1.10.2.3 TADTopResourceOptions.BackupExt Property

Specifies persistent dataset backup copy extension.

Description

Use BackupExt property to specify extension, which will get backup copy of persistent dataset file. Default value is '.bak'.

The backup copy file gets the same base file name as it is specified by TADBottomResourceOptions.PersistentFileName (see page 843), BackupExt (see page 876) extension and will be put to BackupFolder (see page 876) folder.

Syntax

```
property BackupExt: String;
```

See Also

Backup, BackupFolder, TADBottomResourceOptions.PersistentFileName (see page 843)

1.17.25.1.10.2.4 TADTopResourceOptions.BackupFolder Property

Specifies persistent dataset backup copy folder.

Description

Use BackupExt property to specify extension, which will get backup copy of persistent dataset file. The default value is current folder.

The backup copy file gets the same base file name as it is specified by TADBottomResourceOptions.PersistentFileName (see page 843), BackupExt (see page 876) extension and will be put to BackupFolder (see page 876) folder.

Syntax

```
property BackupFolder: String;
```

See Also

Backup, BackupExt, TADBottomResourceOptions.PersistentFileName (see page 843)

1.17.25.1.10.2.5 TADTopResourceOptions.DefaultStoreExt Property

Specifies default extension for persistent dataset data files.

Description

Use DefaultStoreExt property to specify default extension for data file, where persistent dataset stores its data. The default value is empty string.

FireDAC uses DefaultStoreExt, when PersistentFileName (see page 843) property value or AFileName argument of the SaveToFile (see page 644) method does not have extension specified.

Syntax

```
property DefaultStoreExt: String;
```

See Also

DefaultStoreFormat, DefaultStoreFolder, TADBottomResourceOptions.PersistentFileName (see page 843)

1.17.25.1.10.2.6 TADTopResourceOptions.DefaultStoreFolder Property

Specifies the default folder for the persistent dataset data files.

Description

Use DefaultStoreFolder property to specify the default folder for the data file, where persistent dataset stores its data. The default value is empty string.

FireDAC uses DefaultStoreFolder, when PersistentFileName (see page 843) property value or AFileName argument of the SaveToFile (see page 644) method does not have a folder specified.

Syntax

```
property DefaultStoreFolder: String;
```

See Also

DefaultStoreFormat, DefaultStoreExt, TADBottomResourceOptions.PersistentFileName (see page 843)

1.17.25.1.10.2.7 TADTopResourceOptions.DefaultStoreFormat Property

Specifies the default format for the persistent dataset data files.

Description

Use DefaultStoreFormat property to specify the default format for the data file, where dataset stores its persistent data. The default value is sfBinary.

FireDAC uses DefaultStoreFormat, when PersistentFileName (see page 843) property value or AFileName argument of the SaveToFile (see page 644) method does not have an extension specified, and DefaultStoreExt (see page 877) property value is empty.

Syntax

```
property DefaultStoreFormat: TADStorageFormat;
```

See Also

DefaultStoreExt, DefaultStoreFolder, TADBottomResourceOptions.PersistentFileName (see page 843)

1.17.25.1.10.2.8 TADTopResourceOptions.KeepConnection Property

Specifies whether an application remains connected to a database even if no datasets are open.

Description

Use KeepConnection to specify whether an application remains connected to a database even if no associated dataset components are currently active. When KeepConnection is true (the default) the connection is maintained. For connections to remote database servers, or for applications that frequently open and close datasets, set KeepConnection to true to reduce network traffic, speed up applications, and avoid logging in to the server each time the connection is reestablished.

When KeepConnection is false a connection is dropped when there are no open datasets. Dropping a connection releases system resources allocated to the connection, but if a dataset is later opened that uses the database, the connection must be reestablished and initialized.

Syntax

```
property KeepConnection: Boolean;
```

See Also

Establishing Connection (see page 37)

TADCustomConnection (see page 318)

1.17.25.1.10.2.9 TADTopResourceOptions.MaxCursors Property

Controls maximum allowed number of active commands.

Description

Use MaxCursors property to limit maximum number of active commands in application.

If MaxCursors is > 0, then FireDAC will track the number of:

- prepared or active commands;
- prepared or active datasets.

And if next call (Prepare, Open or Execute) will make this number greater than MaxCursors, then FireDAC will automatically disconnect least often used commands and datasets. It is not guaranteed, that the exact total number of the active commands will be less or equal to MaxCursors.

For Oracle, that may help to avoid ORA-01000 error. Note, there is no direct relation between number of active FireDAC commands and the number of session active cursors. Read Monitoring Open and Cached Cursors for more details.

Syntax

```
property MaxCursors: Integer;
```

1.17.25.1.10.2.10 TADTopResourceOptions.ServerOutput Property

Controls accessibility of the server side messages.

Description

Use ServerOutput to control accessibility of the server side non-error messages (warning, user messages, etc) to a client application. Set it to True to guarantee that messages will be delivered. Set it to False, then messages may be delivered, may be not - depending on a DBMS.

Not all supported DBMS's are returning non-error messages automatically. For example, for Oracle and MySQL FireDAC need to call additional DBMS API to bring the messages to a client. That requires additional server round trips. But for SQL Server and Sybase SQL Anywhere the messages are returned always automatically. Setting ServerOutput to True guarantees, that for any DBMS the messages will be returned to client automatically, but that leads to additional SQL execution overhead.

The returned messages will be accessible through properties:

- TADCustomConnection (see page 318).Messages (see page 330);

- IADPhysConnectionIntf.Messages.

The messages will be stored there until next DBMS call (Open, ExecSQL (see page 397), StartTransaction (see page 358), etc).

Syntax

```
property ServerOutput: Boolean;
```

See Also

TADCustomConnection.Messages (see page 330), IADPhysConnectionIntf.Messages

Example 1

```
// SQL Server
procedure TForm1.FormCreate(Sender: TObject);
var
  i: Integer;
begin
  ADConnection1.ResourceOptions.ServerOutput := True;
  ADQuery1.ExecSQL('EXEC sp_configure ''show advanced options'', 1');
  for i := 0 to ADConnection1.ConnectionIntf.Messages.ErrorCount - 1 do
    Memo1.Lines.Add(ADConnection1.ConnectionIntf.Messages[i].Message);
end;
```

Example 2

```
// Oracle
procedure TForm1.FormCreate(Sender: TObject);
var
  i: Integer;
begin
  ADConnection1.ResourceOptions.ServerOutputSize := 256 * 1024;
  ADConnection1.ResourceOptions.ServerOutput := True;
  ADQuery1.ExecSQL('begin dbms_output.put_line(''trace message''); end;');
  for i := 0 to ADConnection1.ConnectionIntf.Messages.ErrorCount - 1 do
    Memo1.Lines.Add(ADConnection1.ConnectionIntf.Messages[i].Message);
end;
```

1.17.25.1.10.2.11 TADTopResourceOptions.ServerOutputSize Property

Controls the size of Oracle message output buffer.

Description

Use property ServerOutputSize to specify size of a Oracle DBMS_OUTPUT buffer.

Syntax

```
property ServerOutputSize: Integer;
```

See Also

ServerOutput (see page 890)

1.17.25.1.11 TADTxOptions Class

The set of options controlling transaction behaviour.

public

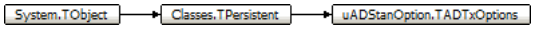
public	Description
Changed (see page 892)	Returns the set of changed transaction options.
ClearChanged (see page 892)	Clears Changed (see page 892) flags.

published

published	Description
AutoCommit (see page 893)	Controls the automatic completion of transaction.
AutoStart (see page 893)	Controls automatic start of a transaction.

AutoStop (see page 893)	Controls automatic finishing of a transaction.
DisconnectAction (see page 894)	Specifies an action FireDAC should take on closing connection.
EnableNested (see page 894)	Controls nested transactions.
Isolation (see page 894)	Specifies the transaction isolation level for the transactions managed by FireDAC.
Params (see page 895)	Specifies custom DBMS transaction parameters.
ReadOnly (see page 895)	Specifies whether the transaction is read-only.
StopOptions (see page 895)	Specifies automatic transaction stoping options.

Class Hierarchy



File

uADStanOption

Description

The TADTxOptions class represents the set of properties controlling behavior of a transactions, initiated by the FireDAC components or by the Phys Layer command interfaces:

- isolation level (Isolation (see page 894));
- update ability (ReadOnly (see page 895));
- automatic committing (AutoCommit (see page 893), AutoStart (see page 893), AutoStop (see page 893), StopOptions (see page 895));
- DBMS specific parameters (Params (see page 895));
- disconnection action (DisconnectAction (see page 894));
- nesting (EnableNested (see page 894)).

The transaction option applying is deferred until next SQL command execution. Changed options will be applied right before execution of a next SQL command.

Syntax

```
TADTxOptions = class(TPersistent);
```

See Also

Managing Transactions (see page 41), TADCustomConnection.TxOptions (see page 337), TADCustomTransaction.Options (see page 416), IADPhysTransaction.Options

1.17.25.1.11.1 public

1.17.25.1.11.1.1 TADTxOptions.Changed Property

Returns the set of changed transaction options.

Description

Use Changed property to read the set of changed transaction options, which are not yet applied to a DBMS. After changed options will be applied to a DBMS, Changed is cleared. In most cases, application does not need to use this property.

Syntax

```
property Changed: TADTxValues;
```

See Also

ClearChanged (see page 892)

1.17.25.1.11.1.2 TADTxOptions.ClearChanged Method

Clears Changed (see page 892) flags.

Description

Use ClearChanged method to clear changed options flag (Changed (see page 892)). In most cases, application does not need to use this method.

Syntax

```
procedure ClearChanged;
```

See Also

Changed (see page 892)

1.17.25.1.11.2 published

1.17.25.1.11.2.1 TADTxOptions.AutoCommit Property

Controls the automatic completion of transaction.

Description

Use AutoCommit property to control automatic transaction management. The default value is True.

If AutoCommit is True, then FireDAC will:

- start transaction (if required) before each SQL command;
- finish started transaction after SQL command execution. If command is successful, then FireDAC will issue COMMIT, otherwise - ROLLBACK.

If application has called StartTransaction (see page 358) method, then automatic transaction management will be disabled until corresponding Commit (see page 340) or Rollback (see page 356) method call. In general, there is no need to set AutoCommit property value to False, just use explicit transaction control (StartTransaction (see page 358), Commit (see page 340), Rollback (see page 356)) when you need it.

Depending on DBMS, FireDAC uses either native DBMS API for automatic transaction management, either emulates automatic transaction management. At moment only Interbase of Firebird are requiring emulation. In case of emulation setting AutoCommit to True is shortcut to setting AutoStart (see page 893) and AutoStop (see page 893) to True and including xoIfAutoStarted and xoIfCmdsInactive into StopOptions (see page 895).

Syntax

```
property AutoCommit: Boolean;
```

See Also

TADCustomTransaction.StartTransaction (see page 419), TADCustomTransaction.Commit (see page 417), TADCustomTransaction.Rollback (see page 418), AutoStart (see page 893), AutoStop (see page 893), StopOptions (see page 895)

1.17.25.1.11.2.2 TADTxOptions.AutoStart Property

Controls automatic start of a transaction.

Description

Use AutoStart property to control automatic start of a transaction, if DBMS does not support automatic transaction management, like Interbase or Firebird. The default value is True.

If AutoStart is True, then transaction will be automatically started, if there is no active transactions, DBMS itself cannot automatically initiate transaction, but transaction is required to perform SQL command.

Syntax

```
property AutoStart: Boolean;
```

See Also

AutoCommit (see page 893), AutoStop (see page 893)

1.17.25.1.11.2.3 TADTxOptions.AutoStop Property

Controls automatic finishing of a transaction.

Description

Use AutoStop property to control automatic finishing of a transaction, if DBMS does not support automatic transaction management, like Interbase or Firebird. The default value is True.

If AutoStop is True, then transaction will be automatically finished, if there is active transactions automatically started by FireDAC.

Syntax

```
property AutoStop: Boolean;
```

See Also

AutoCommit (see page 893), AutoStart (see page 893), StopOptions (see page 895)

1.17.25.1.11.2.4 TADTxOptions.DisconnectAction Property

Specifies an action FireDAC should take on closing connection.

Description

Use DisconnectAction to specify an action, which FireDAC should take on closing connection, if there is active transaction. The default value is xdCommit.

Action	Description
xdNone	Nothing to do. DBMS will take its default action.
xdCommit	Commit transaction.
xdRollback	Rollback transaction.

Syntax

```
property DisconnectAction: TADTxAction;
```

See Also

TADCustomConnection.Connected (see page 321)

1.17.25.1.11.2.5 TADTxOptions.EnableNested Property

Controls nested transactions.

Description

Set EnableNested property to True to enable nested transactions. And to False to disable them. The default value is True.

When a transaction is active then next StartTransaction (see page 358) call will create a nested transaction. FireDAC emulates nested transactions using savepoints if a DBMS does not support nested transactions explicitly. To disable nested transactions set EnableNested to False, then next StartTransaction (see page 358) call will raise an exception.

Syntax

```
property EnableNested: Boolean;
```

See Also

StartTransaction (see page 358)

1.17.25.1.11.2.6 TADTxOptions.Isolation Property

Specifies the transaction isolation level for the transactions managed by FireDAC.

Description

Use Isolation property to specify transaction isolation level for next transactions in current session. Changing of an isolation level does not affect a currently active transaction. The default value is xiReadCommitted.

The Isolation property value is DBMS independent and is translated into most close by meaning native DBMS isolation level.

Level	Description
xiUnspecified	Use default DBMS isolation level.
xiDirtyRead	Permits reading of uncommitted changes made to the database by other simultaneous transactions. Uncommitted changes are not permanent, and might be rolled back (undone) at any time. At this level a transaction is least isolated from the effects of other transactions.
xiReadCommitted	Permits reading of committed (permanent) changes made to the database by other simultaneous transactions. This is the default Isolation property value.
xiRepeatableRead	Permits a single, one-time reading of the database. The transaction cannot see any subsequent changes made by other simultaneous transactions. This isolation level guarantees that once a transaction reads a record, its view of that record does not change unless it makes a modification to the record itself. At this level, a transaction is most isolated from other transactions.
xiSnapshot	
xiSerializible	

Syntax

```
property Isolation: TADTxIsolation;
```

1.17.25.1.11.2.7 TADTxOptions.Params Property

Specifies custom DBMS transaction parameters.

Description

Use Params to specify transaction parameters additional to the other options.

At moment it is used only by Interbase / Firebird driver.

Syntax

```
property Params: TStrings;
```

See Also

Isolation (see page 894), ReadOnly (see page 895), TADUpdateOptions.LockWait (see page 902)

1.17.25.1.11.2.8 TADTxOptions.ReadOnly Property

Specifies whether the transaction is read-only.

Description

Use the ReadOnly property values to tell to a DBMS, that a transaction will only read data, but do not modify it. The default value is False.

Setting ReadOnly to True allows to DBMS optimize resource usage.

Syntax

```
property ReadOnly: Boolean;
```

See Also

Isolation (see page 894), Params (see page 895)

1.17.25.1.11.2.9 TADTxOptions.StopOptions Property

Specifies automatic transaction stoping options.

Description

Use StopOptions property to specify conditions, when FireDAC should finish automatically started transaction, if DBMS does not support automatic transaction management, like Interbase or Firebird. The default value is [xoIfAutoStarted, xoIfCmdsInactive].

If AutoStop is True, then transaction will be automatically finished, if also specified options are valid:

Option	Description
xoIfCmdsInactive	Stop transaction only if all commands, associated with transaction, are inactive.
xoIfAutoStarted	Stop transaction only if it was automatically started by FireDAC, as result of AutoStart = True.
xoFinishRetaining	Use CommitRetaining instead of Commit.

Syntax

```
property StopOptions: TADTxStopOptions;
```

See Also

AutoStart (see page 893), AutoStop (see page 893)

1.17.25.1.12 TADUpdateOptions Class

The set of options controlling dataset editing and updates posting.

public

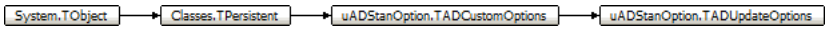
public	Description
Assign (see page 897)	Assigns all options of one option object to another.
RestoreDefaults (see page 897)	Restore option default values.

published

published	Description
AssignedValues (see page 898)	Specifies the set of changed options.
CheckReadOnly (see page 898)	Controls checking of TField.ReadOnly property.
CheckRequired (see page 898)	Controls checking of TField.Required property.
CheckUpdatable (see page 899)	Controls ability to change a TField value.
CountUpdatedRecords (see page 899)	Controls checking of an updated record count.
EnableDelete (see page 899)	Allows to delete records from dataset.
EnableInsert (see page 900)	Allows to insert records into dataset.
EnableUpdate (see page 900)	Allows to edit records of dataset.
FastUpdates (see page 900)	Controls fast vs secure updates.
FetchGeneratorsPoint (see page 901)	Controls the moment, when next generator value will be fetched.
GeneratorName (see page 901)	Specifies the generator name from which to fetch next value for auto-increment fields.
LockMode (see page 902)	Controls how the FireDAC locks database record while editing it.
LockPoint (see page 902)	Controls the moment, when database record will be locked.
LockWait (see page 902)	Controls waiting for pessimistic lock to be acquired.
ReadOnly (see page 903)	Specifies whether the dataset is read-only.
RefreshDelete (see page 903)	Specifies should a dataset delete a record locally when a database record is not found at refreshing.
RefreshMode (see page 903)	Specifies how dataset should refresh record after posting update to it.
RequestLive (see page 904)	Provided for BDE compatibility.
UpdateChangedFields (see page 904)	Controls which fields to include into UPDATE or INSERT command.
UpdateMode (see page 904)	Controls how the FireDAC locates records when posting updates.

UpdateNonBaseFields (see page 905)	Controls should fields from the joined non-base table to be included into UPDATE or INSERT command.

Class Hierarchy



File

uADStanOption

Description

The TADUpdateOptions class represents the set of properties controlling how FireDAC datasets and DApt Layer adapter interfaces:

- allow to edit data (EnableDelete (see page 899), EnableInsert (see page 900), EnableUpdate (see page 900), ReadOnly (see page 903), CheckRequired (see page 898), CheckReadOnly (see page 898), CheckUpdatable (see page 899)). Also TField.ReadOnly, Required affects that.
- lock database records (LockMode (see page 902), LockPoint (see page 902), LockWait (see page 902));
- use database generators and sequences for auto-incrementing fields (FetchGeneratorsPoint (see page 901), GeneratorName (see page 901)). Also TField.AutoGenerateValue affects that;
- generate updates posting commands (UpdateChangedFields (see page 904), UpdateNonBaseFields (see page 905), UpdateMode (see page 904), FastUpdates (see page 900)). Also TField.ProviderFlags, Origin affects that;
- refresh data (RefreshMode (see page 903), RefreshDelete (see page 903));
- checking the updates posting result (CountUpdatedRecords (see page 899)).

The TADUpdateOptions is an intermediate class in update options classes hierarchy. The TADCustomManager (see page 365) and TADCustomConnection (see page 318) classes use TADUpdateOptions. The TADCustomCommand (see page 291), TADDataSet (see page 584) and adapters use TADBottomUpdateOptions (see page 843).

Syntax

```
TADUpdateOptions = class(TADCustomOptions);
```

See Also

Setting Options (see page 34), TADCustomManager.UpdateOptions (see page 375), TADCustomConnection.UpdateOptions (see page 338), TADDataSet.UpdateOptions (see page 653)

1.17.25.1.12.1 public

1.17.25.1.12.1.1 TADUpdateOptions.Assign Method

Assigns all options of one option object to another.

Parameters

Parameters	Description
ASource: TPersistent	A source options object.

Description

Use the Assign method to assign all changed option values of ASource option object to this object. After that corresponding options will stop inheriting the values from more high level option object.

Syntax

```
procedure Assign(ASource: TPersistent); override;
```

See Also

RestoreDefaults

Example

```
ADQuery2.UpdateOptions.Assign(ADQuery1.UpdateOptions);
```

1.17.25.1.12.1.2 TADUpdateOptions.RestoreDefaults Method

Restore option default values.

Description

Call RestoreDefaults to reset all changed options to their default values.

Syntax

```
procedure RestoreDefaults; override;
```

See Also

AssignedValues (see page 898)

1.17.25.1.12.2 published

1.17.25.1.12.2.1 TADUpdateOptions.AssignedValues Property

Specifies the set of changed options.

Description

Use AssignedValues to get / set the set of modified option values.

If some option is changed at this level, then corresponding flag is included into AssignedValues. If flag is not included, then an option value is inherited from more high level. Excluding flag from AssignedValues makes option inheriting its value from more high level.

See Setting Options (see page 34) for more details.

Syntax

```
property AssignedValues: TADUpdateOptionValues;
```

See Also

RestoreDefaults (see page 897)

1.17.25.1.12.2.2 TADUpdateOptions.CheckReadOnly Property

Controls checking of TField.ReadOnly property.

Description

Use the CheckReadOnly property value to control, should FireDAC or not to use the TField.ReadOnly property to control field update ability. The default value is True. CheckReadOnly must be specified before preparing / opening a dataset.

The TField.ReadOnly will be used when CheckReadOnly is True. If at dataset Open call the CheckReadOnly is False, then the fields ReadOnly will be False. When a field ReadOnly is True and CheckReadOnly is False, the field is still not editable using data aware controls, but it may be assigned directly in code.

Additionally CheckReadOnly may be used in combination with UpdateNonBaseFields to enable editing of non-base fields.

Syntax

```
property CheckReadOnly: Boolean;
```

See Also

Editing the Data (see page 113), TField.ReadOnly, CheckRequired (see page 898), CheckUpdatable (see page 899), UpdateNonBaseFields (see page 905)

1.17.25.1.12.2.3 TADUpdateOptions.CheckRequired Property

Controls checking of TField.Required property.

Description

Use the CheckRequired property value to control, should FireDAC or not to use the TField.Required property to control field value requirement. The default value is True, for TADCustomMemTable it is False.

The TField.Required will be used when CheckRequired is True. If at dataset Post call a field Required is True and a field value is Null, then exception will be raised.

Syntax

```
property CheckRequired: Boolean;
```

See Also

Editing the Data (see page 113), TField.Required, CheckUpdatable (see page 899), CheckReadOnly (see page 898)

1.17.25.1.12.2.4 TADUpdateOptions.CheckUpdatable Property

Controls ability to change a TField value.

Description

Use the CheckUpdatable property value to control, should FireDAC or not to raise an exception, when application tries to modify field value in an inappropriate state. The default value is True.

This is normally prohibited when:

- dataset is not in dsEdit, dsInsert, dsSetKey, dsCalcFields, dsFilter, dsNewValue, dsInternalCalc state;
- dataset state is dsEdit, dsInsert and TField has ReadOnly = True and CheckReadOnly = True;
- dataset state is dsSetKey and field is not indexed.

The exception will be raised when CheckUpdatable is True and one of the above conditions is not met.

Syntax

```
property CheckUpdatable: Boolean;
```

See Also

Editing the Data (see page 113), TField.ReadOnly, CheckReadOnly (see page 898), CheckRequired (see page 898)

1.17.25.1.12.2.5 TADUpdateOptions.CountUpdatedRecords Property

Controls checking of an updated record count.

Description

Use CountUpdatedRecords to control should FireDAC check the updated records number (True) or should not (False). The default value is True.

If CountUpdatedRecords = True and number of updated records after posting a single update is not equal to 1, then exception will be raised and an update posting command will be rolled back.

If, at some conditions the record count is not equal to 1, but programmer is sure, that update was posted successfully, then he / she can set CountUpdatedRecords to False.

Syntax

```
property CountUpdatedRecords: Boolean;
```

See Also

Update Command Generation (see page 121)

1.17.25.1.12.2.6 TADUpdateOptions.EnableDelete Property

Allows to delete records from dataset.

Description

Use EnableDelete property to enable records deletion from dataset (True). If it is disallowed (False), then TDataSet.Delete method will raise exception. The default value is True.

If you need to make dataset read-only, then use ReadOnly property.

Syntax

```
property EnableDelete: Boolean;
```

See Also

Editing the Data (see page 113), EnableInsert (see page 900), EnableUpdate (see page 900), ReadOnly (see page 903)

1.17.25.1.12.2.7 TADUpdateOptions.EnableInsert Property

Allows to insert records into dataset.

Description

Use EnableInsert property to enable addition of new records to dataset (True). If it is disallowed (False), then TDataSet.Insert / Append methods will raise exception. The default value is True.

If you need to make dataset read-only, then use ReadOnly property.

Syntax

```
property EnableInsert: Boolean;
```

See Also

Editing the Data (see page 113), EnableDelete (see page 899), EnableUpdate (see page 900), ReadOnly (see page 903)

1.17.25.1.12.2.8 TADUpdateOptions.EnableUpdate Property

Allows to edit records of dataset.

Description

Use EnableUpdate property to enable edition of dataset records (True). If it is disallowed (False), then TDataSet.Edit method will raise an exception. The default value is True.

If you need to make dataset read-only, then use ReadOnly property.

Syntax

```
property EnableUpdate: Boolean;
```

See Also

Editing the Data (see page 113), EnableDelete (see page 899), EnableInsert (see page 900), ReadOnly (see page 903)

1.17.25.1.12.2.9 TADUpdateOptions.FastUpdates Property

Controls fast vs secure updates.

Description

Use FastUpdates property to choose the high performance updates (True) or the secure updates (False).

To perform update commands faster, FireDAC will cache update commands, use primary key fields to locate updated record, does not lock records before editing, does not refresh records after posting updates and avoid some checks inside of a dataset. Setting FastUpdates to True is shortcut for setting:

- UpdateChangedFields (see page 904) = False;
- LockMode (see page 902) = lmNone;

- UpdateMode (see page 904) = upWhereKeyOnly;
- RefreshMode (see page 903) = rmManual;
- CheckRequired (see page 898) = False;
- CheckReadOnly (see page 898) = False;
- CheckUpdatable (see page 899) = False.

Syntax

```
property FastUpdates: Boolean;
```

See Also

Update Command Generation (see page 121), UpdateChangedFields (see page 904), LockMode (see page 902), UpdateMode (see page 904), RefreshMode (see page 903)

1.17.25.1.12.2.10 TADUpdateOptions.FetchGeneratorsPoint Property

Controls the moment, when next generator value will be fetched.

Description

Use FetchGeneratorsPoint to specify a moment, when a next generator value will be fetched from a database and assigned to an auto-incrementing field. The default value is gpDeferred.

A next generator value will be fetched from an generator, specified by GeneratorName (see page 901) / TADAutoIncField (see page 579).GeneratorName (see page 581) properties and assigned to the auto-incrementing fields, which have pfInUpdate in ProviderFlags. An auto-incrementing field is a field:

- of the TADAutoIncField (see page 579) class;
- or whose name is specified in the AutoIncFields (see page 845);
- or with TField.AutoGenerateValue = arAutoInc.

Generators are supported by Interbase and Firebird. Sequences are supported by Oracle and PostgreSQL DBMS's. For other DBMS's this property has no meaning.

The value may be one of the following values:

Mode	Description
gpNone	Do not fetch generator values.
gpImmediate	Fetch values right after beginning to append a new record. Normally this is TDataSet.Insert or Append methods.
gpDeferred	Fetch values right before posting new record to to a database. Normally this is TDataSet.Post or ApplyUpdates.

Syntax

```
property FetchGeneratorsPoint: TADFetchGeneratorsPoint;
```

See Also

Auto-Incremental Fields (see page 118), GeneratorName (see page 901), TADAutoIncField.GeneratorName (see page 581), AutoIncFields (see page 845)

1.17.25.1.12.2.11 TADUpdateOptions.GeneratorName Property

Specifies the generator name from which to fetch next value for auto-increment fields.

Description

Use GeneratorName property to specify a generator name, which next value will be fetched from a database and assigned to the auto-incrementing fields. See FetchGeneratorsPoint (see page 901) property description for details.

Syntax

```
property GeneratorName: String;
```

See Also

Auto-Incremental Fields (see page 118), Object Names (see page 133), FetchGeneratorsPoint (see page 901), TADAutoIncField.GeneratorName (see page 581)

1.17.25.1.12.2.12 TADUpdateOptions.LockMode Property

Controls how the FireDAC locks database record while editing it.

Description

Use LockMode property to specify database record locking mode. You can use locks to resolve conflict of updates to the same record from different DBMS sessions. The default value is lmNone.

Mode	Description
lmNone	No locking at all.
lmPessimistic	Pessimistic locking, if DBMS supports SELECT FOR UPDATE, otherwise optimistic locking is used. If record is locked, then FireDAC will wait LockWait (see page 902) time, if DBMS supports lock timeout, otherwise - depends on DBMS settings.
lmOptimistic	Optimistic locking.

Syntax

```
property LockMode: TADLockMode;
```

See Also

LockPoint (see page 902), LockWait (see page 902)

1.17.25.1.12.2.13 TADUpdateOptions.LockPoint Property

Controls the moment, when database record will be locked.

Description

Use LockPoint to specify moment, when changing database record will be locked in database and marked in internal data storage as locked. The default value is lpDeferred.

The value may be one of the following values:

Mode	Description
lpImmediate	Lock record right after starting record editing or deleting.
lpDeferred	Lock record right before posting updates to database.

FireDAC locks record if LockMode is not a lmNone.

Syntax

```
property LockPoint: TADLockPoint;
```

See Also

LockMode (see page 902), LockWait (see page 902)

1.17.25.1.12.2.14 TADUpdateOptions.LockWait Property

Controls waiting for pessimistic lock to be acquired.

Description

Use LockWait property to control, should FireDAC wait while pessimistic lock will be acquired (True), or return error immediately (False) if record is already locked. The default value is False.

The LockWait property is used only if LockMode (see page 902) = lmPessimistic.

Note, not all DBMS are support LockWait=False or True. So, LockWait declares rather the desired behaviour and is not a must.

Syntax

```
property LockWait: Boolean;
```

See Also

LockMode (see page 902), LockPoint (see page 902)

1.17.25.1.12.2.15 TADUpdateOptions.ReadOnly Property

Specifies whether the dataset is read-only.

Description

Use the ReadOnly property to prevent users from updating, inserting, or deleting data in the dataset. By default ReadOnly is False.

To guarantee that users cannot modify dataset data, set ReadOnly to True. The property is shortcut for EnableDelete (see page 899), EnableInsert (see page 900), EnableUpdate (see page 900) properties.

Syntax

```
property ReadOnly: Boolean;
```

See Also

Editing the Data (see page 113), EnableDelete (see page 899), EnableInsert (see page 900), EnableUpdate (see page 900)

1.17.25.1.12.2.16 TADUpdateOptions.RefreshDelete Property

Specifies should a dataset delete a record locally when a database record is not found at refreshing.

Description

Use the RefreshDelete property to allow a dataset to delete a record from a local storage, when a corresponding database record is not found at TADDataSet.RefreshRecord (see page 642) call. The default value is True.

The most probable reason that the record is not found - it is deleted, while other options are possible too. To delete the record, set this property to True. To leave the record as is, set this property to False. For additional details see the TADDataSet.RefreshRecord (see page 642) method description.

Syntax

```
property RefreshDelete: Boolean;
```

See Also

RefreshMode (see page 903), TADDataSet.RefreshRecord (see page 642)

1.17.25.1.12.2.17 TADUpdateOptions.RefreshMode Property

Specifies how dataset should refresh record after posting update to it.

Description

Use RefreshMode property to control how dataset should refresh record after posting update (insert / update) to it. The default value is rmOnDemand.

Following table lists modes and their meaning in case of automatic command generation:

Mode	Description
rmManual	Do not automatically refresh record. Application should call TADDataSet.RefreshRecord (see page 642) to refresh record.

rmOnDemand	Automatically refresh record fields: • after inserting, if they are auto-incremental, calculated, have default values, contain ROWID, contain row version. Or TField.AutoGenerateValue <> arNone. • after updating, if they are calculated or contain row version.
rmAll	Automatically refresh all record fields.

If dataset is connected to a TADUpdateSQL (see page 563) and FetchRowSQL (see page 567) is specified, then FireDAC will execute FetchRowSQL (see page 567) command to refresh row, only if RefreshMode is rmAll.

Syntax

```
property RefreshMode: TADRefreshMode;
```

See Also

Update Command Generation (see page 121), TADDataSet.RefreshRecord (see page 642), TADUpdateSQL.FetchRowSQL (see page 567)

1.17.25.1.12.2.18 TADUpdateOptions.RequestLive Property

Provided for BDE compatibility.

Description

Use RequestLive property for backward compatibility with BDE. Setting RequestLive to True:

- set ReadOnly (see page 903) to False;
- includes fiMeta into TADFetchOptions.Items (see page 855).

Syntax

```
property RequestLive: Boolean;
```

See Also

Editing the Data (see page 113), ReadOnly (see page 903), TADFetchOptions.Items (see page 855)

1.17.25.1.12.2.19 TADUpdateOptions.UpdateChangedFields Property

Controls which fields to include into UPDATE or INSERT command.

Description

Use the UpdateChangedFields property to specify should FireDAC include into UPDATE SET <field list> or INSERT INTO (<field list>) commands all fields allowed for updating (False) or include only changed fields (True). The default value is True.

A field is allowed to update, if it has ReadOnly (see page 903) = False, pfInUpdate in TField.ProviderFlags. There are possible other options for auto incremental fields.

Setting UpdateChangedFields to True guarantees, that only:

- not NULL fields will be included into INSERT command;
- changed by user / application fields will be included into UPDATE command.

That allows to minimize database recover / transaction rollback information generation. Also fire only table triggers for really changed fields.

Setting UpdateChangedFields to False allows to optimize posting updates for multiple records, as it does not require to regenerate updates posting command for each record. Consider to set FastUpdates to True.

Syntax

```
property UpdateChangedFields: Boolean;
```

See Also

Editing the Data (see page 113), CacheUpdatesCommands, UpdateMode (see page 904), FastUpdates (see page

900)

1.17.25.1.12.2.20 TADUpdateOptions.UpdateMode Property

Controls how the FireDAC locates records when posting updates.

Description

Use UpdateMode property to specify the criteria for locating a record in the database when posting updates. The default value is upWhereKeyOnly.

UpdateMode specifies whether records are located using all columns (fields), on only the key fields, or on the key fields plus the original values of fields that have been modified. To control how FireDAC locates records by specifying individual fields, use the field components' ProviderFlags property.

Syntax

```
property UpdateMode: TUpdateMode;
```

See Also

Unique Identifying Fields (see page 117), Update Command Generation (see page 121), TField.ProviderFlags, TADBottomUpdateOptions.KeyFields (see page 845)

1.17.25.1.12.2.21 TADUpdateOptions.UpdateNonBaseFields Property

Controls should fields from the joined non-base table to be included into UPDATE or INSERT command.

Description

Use UpdateNonBaseFields to enable (True) or disable (False) non-base table fields inclusion into UPDATE or INSERT command. The default value is False. UpdateNonBaseFields must be specified before preparing / opening a dataset.

The options is useful when the original SQL command is a SELECT with joins of few tables. The first leftmost table is a base table. Which columns are base and which are non-base is determined using SELECT list metadata - column original table name, column original name. Most DBMS return this infromation, excluding Oracle and Advantage. See Overriding Posting Updates (see page 123) for details how to setup a DBMS to return this info. Note, in the following SELECT cases all fields may be non-base fields:

- with set operators - UNION, INTERSECT, MINUS, etc;
- with grouping.

In most cases updates must be allowed to the base table columns only. And prohibited for other "lookup" tables. To do so set UpdateNonBaseFields to False. If application needs to exclude non-base columns from updating SQL commands, but enable editing for them, then set CheckReadOnly (see page 898) to False.

Syntax

```
property UpdateNonBaseFields: Boolean;
```

See Also

Overriding Posting Updates (see page 123), UpdateChangedFields (see page 904), CheckReadOnly (see page 898)

1.17.25.2 Structs, Records, Enums

The following table lists structs, records, enums in this documentation.

Enumerations

Enumeration	Description
TADActionRequest (see page 905)	The enumerated type lists all possible FireDAC requests to a database.
TADAutoFetchAll (see page 906)	Add a short summary here...

1.17.25.2.1 uADStanOption.TADActionRequest Enumeration

The enumerated type lists all possible FireDAC requests to a database.

File

uADStanOption

Description

The TADActionRequest enumerated type lists all possible FireDAC database actions, performed as part of the dataset editing operations.

Value	Description
arNone	No action. Used internally.
arFromRow	The action will be determined from the current row status (inserted, modified, deleted). Used internally.
arSelect	Execute the dataset query to select the DB rows. Used internally.
arInsert	Insert a row to a DB.
arUpdate	Update a row in a DB.
arDelete	Delete a row from a DB.
arLock	Lock a row in a DB.
arUnlock	Unlock a row in a DB.
arFetchRow	ReFetch a row from a DB.
arUpdateHBlobs	Update Oracle BLOB's in a DB.
arDeleteAll	Delete all dataset rows from a DB. Used internally.
arFetchGenerators	Fetch generator / sequence values for a row columns.

Syntax

```
TADActionRequest = (
);
```

1.17.25.2.2 uADStanOption.TADAutoFetchAll Enumeration

Add a short summary here...

File

uADStanOption

Description

Add a description here...

Syntax

```
TADAutoFetchAll = (
);
```

See Also

List here...

Example

Index

A

ADAdministrator 178
ADDFMChanger 178
Additional migration hints 168
ADExecutor 180
ADExplorer 181
ADMonitor 185
Architecture 17
Array DML 82
Asynchronous Execution 86
Auto-Incremental Fields 118

B

BDE aliases migration 165
BDE application migration 165
BDE name counterparts 164
Browsing Table 73

C

Caching Updates 114
Calculated and Aggregated Fields 108
Changing DataSet Data 113
Character macro functions 60
Class Hierarchy 27
Command Batches 81
Common connection parameters 188
Compile (tool name).bat 186
Components 20
Configuring Drivers 31
Connect to Advantage Database Server 189
Connect to Blackfish SQL Server 192
Connect to DataSnap server 193
Connect to dbExpress data source 194
Connect to IBM DB2 Server 196
Connect to Interbase or Firebird 198
Connect to Microsoft Access database 207
Connect to Microsoft SQL Server 202
Connect to Microsoft SQL Server Compact Edition 206
Connect to MySQL Server 209
Connect to ODBC data source 213
Connect to Oracle Server 215
Connect to PostgreSQL 218
Connect to SQLite database 221
Connect to Sybase SQL Anywhere 224
Convert macro function 65
Create (DB name).bat 188
Creating Reports with FastReport 10

D

Data Type Mapping 35
Database Alerts 75
Database Connectivity 188
Databases 24
Date and time macro functions 62
DBMS Environment Reports 170
Debugging 170
Debugging and Reporting Environment Questions 247
Defining Connection 27
Delpoying on Mac OS X or Linux 178
Delpoying on Windows 177
Demo Applications 16
Demo Databases 14
Deploying 177
Developing Custom Commands 95
DLL Development 49

E

EADDBArrayExecuteError class 831
    Action 832
    Exception 832
    Offset 832
    RetryLevel 832
    Times 833
EADDBArrayExecuteError.Action property 832
EADDBArrayExecuteError.Exception property 832
EADDBArrayExecuteError.Offset property 832
EADDBArrayExecuteError.RetryLevel property 832
EADDBArrayExecuteError.Times property 833
EADDBEngineException class 833
    ErrorCode 834
    ErrorCount 834

Errors 834
Kind 835
Params 835
SQL 835
EADDBEngineException.ErrorCode property 834
EADDBEngineException.ErrorCount property 834
EADDBEngineException.Errors property 834
EADDBEngineException.Kind property 835
EADDBEngineException.Params property 835
EADDBEngineException.SQL property 835
EADException class 836
ADCode 836
ADObjName 837
EADException.ADCode property 836
EADException.ADObjName property 837
EDB2NativeException class 750
Editing Questions 239
Editing the Data 113
EMSAccessNativeException class 785
EMSSQLNativeException class 792
Errors 793
EMSSQLNativeException.Errors property 793
Establishing Connection 37
ETDBXNativeException class 830
Executing Command 66
Executing SQL Scripts 88
Executing Stored Procedure 71
Extended Metadata 129

F

FAQ 227
Fetching and Populating Questions 236
Fetching Rows 78
Filtering Records 102
Finding a Record 107
Firebird and Interbase Servers Questions 249
FireDAC 1
First Steps to use FireDAC 2

G

General 17
General Questions 229
Getting Started 2
GUI Questions 242

H

Handling Errors 44

I

Installation 227
Installing FireDAC 159
Installing on Linux 162
Installing on Windows 161
Integration with 3d Party Products Questions 248

K

Known Limitations 160

L

Lazarus / FPC 160
Local SQL 96

M

Managing Transactions 41
Master-Detail Relationship 104
Metadata Questions 246
Metadata Structure 130
Migrating BDE applications 163
MS Access Questions 256
MS SQL Server Questions 251
Multi Threading 46
MySQL Server Questions 256

N

Numeric macro functions 61

O

Object Names 133
Offlining Connection 40
Oracle Server Questions 253
Overriding Posting Updates 123
Overview 2

P

Path Variables 163
Preprocessing Command Text 56
Programming Tools 26

Q

Querying Metadata 127

R

Recovering Connection 38
RETURNING unified support 66

S

Setting Options 34
Setting up Connections 4
Sorting Records 100
Sorting, Searching, Locating, Filtering Questions 237
Specifying Default Values 126
SQL Script Control Commands 92
SQL Scripts Questions 244
SQLite Database Questions 254
System macro functions 64

T

TADAdaptedDataSet class 259
- AbortJob 263
- Adapter 260
- AttachTable 264
- Command 261
- DatSManager 261
- Disconnect 264
- GetResults 265
- LocalSQL 261
- NextRecordSet 265
- OnError 261
- OnExecuteError 262
- PointedConnection 263
- ServerAppend 265
- ServerCancel 266
- ServerDelete 266
- ServerDeleteAll 266
- ServerEdit 266
- ServerEditRequest 263
- ServerGotoKey 266
- ServerPerform 267
- ServerSetKey 267
- UpdateObject 267

TADAdaptedDataSet.AbortJob method 263
TADAdaptedDataSet.Adapter property 260
TADAdaptedDataSet.AttachTable method 264
TADAdaptedDataSet.Command property 261
TADAdaptedDataSet.DatSManager property 261
TADAdaptedDataSet.Disconnect method 264
TADAdaptedDataSet.GetResults method 265
TADAdaptedDataSet.LocalSQL property 261
TADAdaptedDataSet.NextRecordSet method 265
TADAdaptedDataSet.OnError event 261
TADAdaptedDataSet.OnExecuteError event 262
TADAdaptedDataSet.PointedConnection property 263
TADAdaptedDataSet.ServerAppend method 265
TADAdaptedDataSet.ServerCancel method 266
TADAdaptedDataSet.ServerDelete method 266
TADAdaptedDataSet.ServerDeleteAll method 266
TADAdaptedDataSet.ServerEdit method 266
TADAdaptedDataSet.ServerEditRequest property 263
TADAdaptedDataSet.ServerGotoKey method 266
TADAdaptedDataSet.ServerPerform method 267
TADAdaptedDataSet.ServerSetKey method 267
TADAdaptedDataSet.UpdateObject property 267
TADADSBackup class 725
- Backup 726
- BackupPath 726
- Options 727

TADADSBackup.Backup method 726
TADADSBackup.BackupPath property 726
TADADSBackup.Options property 727
TADADSBackupRestore class 727
- Exclude 728
- Include 728
- Results 727
- TableTypeMap 728

TADADSBackupRestore.Exclude property 728

TADADSBackupRestore.Include property 728
TADADSBackupRestore.Results property 727
TADADSBackupRestore.TableTypeMap property 728
TADADSRestore class 729
DDPassword 730
DestinationPath 730
Options 731
Restore 730
SourcePassword 731
SourcePath 731
TADADSRestore.DDPassword property 730
TADADSRestore.DestinationPath property 730
TADADSRestore.Options property 731
TADADSRestore.Restore method 730
TADADSRestore.SourcePassword property 731
TADADSRestore.SourcePath property 731
TADADSService class 731
Database 732
DriverLink 732
Password 732
UserName 733
TADADSService.Database property 732
TADADSService.DriverLink property 732
TADADSService.Password property 732
TADADSService.UserName property 733
TADADSUtility class 733
Decrypt 734
Encrypt 734
OnProgress 736
Pack 735
Recall 735
Reindex 735
TablePassword 736
Tables 737
TableType 737
Zap 736
TADADSUtility.Decrypt method 734
TADADSUtility.Encrypt method 734
TADADSUtility.OnProgress event 736
TADADSUtility.Pack method 735
TADADSUtility.Recall method 735
TADADSUtility.Reindex method 735
TADADSUtility.TablePassword property 736
TADADSUtility.Tables property 737
TADADSUtility.TableType property 737
TADADSUtility.Zap method 736
TADAggregate class 572
Active 575
ActualActive 573
DataSet 574
DataType 574
Expression 575
GroupingLevel 576
IndexName 576
InUse 574
Name 577
Value 575
TADAggregate.Active property 575
TADAggregate.ActualActive property 573
TADAggregate.DataSet property 574
TADAggregate.DataType property 574
TADAggregate.Expression property 575
TADAggregate.GroupingLevel property 576
TADAggregate.IndexName property 576
TADAggregate.InUse property 574
TADAggregate.Name property 577
TADAggregate.Value property 575
TADAggregates class 577
Add 578
AggregateByName 578
FindAggregate 579
Items 578
TADAggregates.Add method 578
TADAggregates.AggregateByName method 578
TADAggregates.FindAggregate method 579
TADAggregates.Items property 578
TADASABackup class 741
Backup 742
CheckpointLogType 742
ConnectParams 743
Flags 743
HotlogFilename 743
OutputDir 743
PageBlocksize 744

StartLine 744
TADASABackup.Backup method 742
TADASABackup.CheckpointLogType property 742
TADASABackup.ConnectParams property 743
TADASABackup.Flags property 743
TADASABackup.HotlogFilename property 743
TADASABackup.OutputDir property 743
TADASABackup.PageBlocksize property 744
TADASABackup.StartLine property 744
TADASAService class 744
DriverLink 745
OnProgress 745
ToolLib 745
TADASAService.DriverLink property 745
TADASAService.OnProgress property 745
TADASAService.ToolLib property 745
TADASAValidate class 745
ConnectParams 747
Flags 747
StartLine 747
Tables 747
Validate 746
ValidateType 748
TADASAValidate.ConnectParams property 747
TADASAValidate.Flags property 747
TADASAValidate.StartLine property 747
TADASAValidate.Tables property 747
TADASAValidate.Validate method 746
TADASAValidate.ValidateType property 748
TADAutoIncField class 579
AutoIncrementSeed 580
AutoIncrementStep 580
ClientAutoIncrement 581
GeneratorName 581
IdentityInsert 581
ServerAutoIncrement 582
TADAutoIncField.AutoIncrementSeed property 580
TADAutoIncField.AutoIncrementStep property 580
TADAutoIncField.ClientAutoIncrement property 581
TADAutoIncField.GeneratorName property 581
TADAutoIncField.IdentityInsert property 581
TADAutoIncField.ServerAutoIncrement property 582
TADBlobStream class 582
Create 583
Truncate 584
TADBlobStream.Create constructor 583
TADBlobStream.Truncate method 584
TADBottomResourceOptions class 841
Assign 842
PersistentFileName 843
ResolveFileName 842
RestoreDefaults 843
TADBottomResourceOptions.Assign method 842
TADBottomResourceOptions.PersistentFileName property 843
TADBottomResourceOptions.ResolveFileName method 842
TADBottomResourceOptions.RestoreDefaults method 843
TADBottomUpdateOptions class 843
Assign 844
AutoIncFields 845
KeyFields 845
RestoreDefaults 844
UpdateTableName 846
TADBottomUpdateOptions.Assign method 844
TADBottomUpdateOptions.AutoIncFields property 845
TADBottomUpdateOptions.KeyFields property 845
TADBottomUpdateOptions.RestoreDefaults method 844
TADBottomUpdateOptions.UpdateTableName property 846
TADCommand class 267
Active 269
ActiveStoredUsage 269
AfterClose 269
AfterExecute 270
AfterOpen 270
AfterPrepare 270
AfterUnprepare 270
BaseObjectName 271
BeforeClose 271
BeforeExecute 271
BeforeOpen 272
BeforePrepare 272
BeforeUnprepare 272
CatalogName 272
CommandKind 273
CommandText 273

Connection 274
ConnectionName 274
FetchOptions 275
FormatOptions 275
Macros 275
OnCommandChanged 276
OnError 276
Overload 277
Params 277
ResourceOptions 278
SchemaName 278
Transaction 279
UpdateOptions 279
TADCommand.Active property 269
TADCommand.ActiveStoredUsage property 269
TADCommand.AfterClose event 269
TADCommand.AfterExecute event 270
TADCommand.AfterOpen event 270
TADCommand.AfterPrepare event 270
TADCommand.AfterUnprepare event 270
TADCommand.BaseObjectName property 271
TADCommand.BeforeClose event 271
TADCommand.BeforeExecute event 271
TADCommand.BeforeOpen event 272
TADCommand.BeforePrepare event 272
TADCommand.BeforeUnprepare event 272
TADCommand.CatalogName property 272
TADCommand.CommandKind property 273
TADCommand.CommandText property 273
TADCommand.Connection property 274
TADCommand.ConnectionName property 274
TADCommand.FetchOptions property 275
TADCommand.FormatOptions property 275
TADCommand.Macros property 275
TADCommand.OnCommandChanged event 276
TADCommand.OnError event 276
TADCommand.Overload property 277
TADCommand.Params property 277
TADCommand.ResourceOptions property 278
TADCommand.SchemaName property 278
TADCommand.Transaction property 279
TADCommand.UpdateOptions property 279
TADConnection class 279
AfterCommit 280
AfterRollback 281
AfterStartTransaction 281
BeforeCommit 281
BeforeRollback 281
BeforeStartTransaction 282
Connected 282
ConnectedStoredUsage 282
ConnectionDefName 283
ConnectionName 283
DriverName 284
FetchOptions 284
FormatOptions 285
LoginDialog 285
LoginPrompt 286
OnError 286
OnLogin 287
OnLosted 288
OnRecover 288
OnRestored 288
Params 289
ResourceOptions 289
Transaction 289
TxOptions 290
UpdateOptions 290
UpdateTransaction 290
TADConnection.AfterCommit event 280
TADConnection.AfterRollback event 281
TADConnection.AfterStartTransaction event 281
TADConnection.BeforeCommit event 281
TADConnection.BeforeRollback event 281
TADConnection.BeforeStartTransaction event 282
TADConnection.Connected property 282
TADConnection.ConnectedStoredUsage property 282
TADConnection.ConnectionDefName property 283
TADConnection.ConnectionName property 283
TADConnection.DriverName property 284
TADConnection.FetchOptions property 284
TADConnection.FormatOptions property 285
TADConnection.LoginDialog property 285
TADConnection.LoginPrompt property 286

TADConnection.OnError event 286
TADConnection.OnLogin event 287
TADConnection.OnLosted event 288
TADConnection.OnRecover event 288
TADConnection.OnRestored event 288
TADConnection.Params property 289
TADConnection.ResourceOptions property 289
TADConnection.Transaction property 289
TADConnection.TxOptions property 290
TADConnection.UpdateOptions property 290
TADConnection.UpdateTransaction property 290
TADCustomCommand class 291
AbortJob 307
AcquireConnection 308
Active 293
ActiveStoredUsage 293
AfterClose 293
AfterExecute 294
AfterFetch 294
AfterOpen 294
AfterPrepare 295
AfterUnprepare 295
BaseObjectName 295
BeforeClose 296
BeforeExecute 296
BeforeFetch 296
BeforeOpen 296
BeforePrepare 296
BeforeUnprepare 297
BindedBy 297
CatalogName 297
Close 308
CloseAll 309
CommandIntf 298
CommandKind 298
CommandText 298
Connection 299
ConnectionName 300
DataSet 300
Define 309
Disconnect 310
Execute 310, 311, 312
Fetch 312
FetchOptions 300
FillParams 313
FindMacro 314
FindParam 314
FixedCommandKind 300
FormatOptions 301
GetConnection 314
MacroByName 315
Macros 301
NextRecordSet 315
OnCommandChanged 301
OnError 302
Open 316
OpenOrExecute 316
Overload 303
ParamBindMode 303
ParamByName 316
Params 303
Prepare 317
Prepared 304
ReleaseConnection 317
ResourceOptions 304
RowsAffected 304
SchemaName 305
SQLText 306
State 306
Transaction 306
Unprepare 318
UpdateOptions 307
Values 307
TADCustomCommand.AbortJob method 307
TADCustomCommand.AcquireConnection method 308
TADCustomCommand.Active property 293
TADCustomCommand.ActiveStoredUsage property 293
TADCustomCommand.AfterClose event 293
TADCustomCommand.AfterExecute event 294
TADCustomCommand.AfterFetch event 294
TADCustomCommand.AfterOpen event 294
TADCustomCommand.AfterPrepare event 295
TADCustomCommand.AfterUnprepare event 295
TADCustomCommand.BaseObjectName property 295

TADCustomCommand.BeforeClose event 296
TADCustomCommand.BeforeExecute event 296
TADCustomCommand.BeforeFetch event 296
TADCustomCommand.BeforeOpen event 296
TADCustomCommand.BeforePrepare event 296
TADCustomCommand.BeforeUnprepare event 297
TADCustomCommand.BindedBy property 297
TADCustomCommand.CatalogName property 297
TADCustomCommand.Close method 308
TADCustomCommand.CloseAll method 309
TADCustomCommand.CommandIntf property 298
TADCustomCommand.CommandKind property 298
TADCustomCommand.CommandText property 298
TADCustomCommand.Connection property 299
TADCustomCommand.ConnectionName property 300
TADCustomCommand.DataSet property 300
TADCustomCommand.Define method 309
TADCustomCommand.Disconnect method 310
TADCustomCommand.Execute method 310, 311, 312
TADCustomCommand.Fetch method 312
TADCustomCommand.FetchOptions property 300
TADCustomCommand.FillParams method 313
TADCustomCommand.FindMacro method 314
TADCustomCommand.FindParam method 314
TADCustomCommand.FixedCommandKind property 300
TADCustomCommand.FormatOptions property 301
TADCustomCommand.GetConnection method 314
TADCustomCommand.MacroByName method 315
TADCustomCommand.Macros property 301
TADCustomCommand.NextRecordSet method 315
TADCustomCommand.OnCommandChanged event 301
TADCustomCommand.OnError event 302
TADCustomCommand.Open method 316
TADCustomCommand.OpenOrExecute method 316
TADCustomCommand.Overload property 303
TADCustomCommand.ParamBindMode property 303
TADCustomCommand.ParamByName method 316
TADCustomCommand.Params property 303
TADCustomCommand.Prepare method 317
TADCustomCommand.Prepared property 304
TADCustomCommand.ReleaseConnection method 317
TADCustomCommand.ResourceOptions property 304
TADCustomCommand.RowsAffected property 304
TADCustomCommand.SchemaName property 305
TADCustomCommand.SQLText property 306
TADCustomCommand.State property 306
TADCustomCommand.Transaction property 306
TADCustomCommand.Unprepare method 318
TADCustomCommand.UpdateOptions property 307
TADCustomCommand.Values property 307
TADCustomConnection class 318
AbortJob 338
AfterCommit 321
AfterRollback 321
AfterStartTransaction 322
ApplyUpdates 339
Assign 339
BeforeCommit 322
BeforeRollback 322
BeforeStartTransaction 322
CheckActive 340
CheckConnectionDef 340
CheckInactive 340
CheckOnline 340
CliHandle 323
CliObj 323
CommandCount 324
Commands 324
Commit 340
CommitRetaining 341
Connected 321
ConnectedStoredUsage 325
ConnectionDefName 325
ConnectionIntf 326
ConnectionMetaDataIntf 326
ConnectionName 326
DataSets 327
DecodeObjectName 342
DriverName 327
EncodeObjectName 343
ExecSQL 343, 344
ExecSQLScalar 344, 345
FetchOptions 328
FormatOptions 328

GetCatalogNames 346
GetFieldNames 346
GetGeneratorNames 347
GetInfoReport 347
GetKeyFieldNames 349
GetLastAutoGenValue 349
GetPackageNames 350
GetSchemaNames 350
GetStoredProcNames 351
GetTableNames 351
InTransaction 328
IsSQLBased 329
LastUsed 329
LoginDialog 329
LoginPrompt 329
Messages 330
Offline 352
Offlined 331
OnError 332
Online 353
OnLogin 332
OnLosted 333
OnRecover 333
OnRestored 334
Open 353, 354
OptionsIntf 334
Params 334
Ping 354
RDBMSKind 335
RefCount 335
RefreshMetadataCache 355
ReleaseClients 356
ResourceOptions 335
ResultConnectionDef 336
Rollback 356
RollbackRetaining 357
SharedCliHandle 336
StartTransaction 358
Temporary 337
Transaction 337
TxOptions 337
UpdateOptions 338
UpdateTransaction 338
TADCustomConnection.AbortJob method 338
TADCustomConnection.AfterCommit event 321
TADCustomConnection.AfterRollback event 321
TADCustomConnection.AfterStartTransaction event 322
TADCustomConnection.ApplyUpdates method 339
TADCustomConnection.Assign method 339
TADCustomConnection.BeforeCommit event 322
TADCustomConnection.BeforeRollback event 322
TADCustomConnection.BeforeStartTransaction event 322
TADCustomConnection.CheckActive method 340
TADCustomConnection.CheckConnectionDef method 340
TADCustomConnection.CheckInactive method 340
TADCustomConnection.CheckOnline method 340
TADCustomConnection.CliHandle property 323
TADCustomConnection.CliObj property 323
TADCustomConnection.CommandCount property 324
TADCustomConnection.Commands property 324
TADCustomConnection.Commit method 340
TADCustomConnection.CommitRetaining method 341
TADCustomConnection.Connected property 321
TADCustomConnection.ConnectedStoredUsage property 325
TADCustomConnection.ConnectionDefName property 325
TADCustomConnection.ConnectionIntf property 326
TADCustomConnection.ConnectionMetaDataIntf property 326
TADCustomConnection.ConnectionName property 326
TADCustomConnection.DataSets property 327
TADCustomConnection.DecodeObjectName method 342
TADCustomConnection.DriverName property 327
TADCustomConnection.EncodeObjectName method 343
TADCustomConnection.ExecSQL method 343, 344
TADCustomConnection.ExecSQLScalar method 344, 345
TADCustomConnection.FetchOptions property 328
TADCustomConnection.FormatOptions property 328
TADCustomConnection.GetCatalogNames method 346
TADCustomConnection.GetFieldNames method 346
TADCustomConnection.GetGeneratorNames method 347
TADCustomConnection.GetInfoReport method 347
TADCustomConnection.GetKeyFieldNames method 349
TADCustomConnection.GetLastAutoGenValue method 349
TADCustomConnection.GetPackageNames method 350
TADCustomConnection.GetSchemaNames method 350

TADCustomConnection.GetStoredProcNames method 351
TADCustomConnection.GetTableNames method 351
TADCustomConnection.InTransaction property 328
TADCustomConnection.IsSQLBased property 329
TADCustomConnection.LastUsed property 329
TADCustomConnection.LoginDialog property 329
TADCustomConnection.LoginPrompt property 329
TADCustomConnection.Messages property 330
TADCustomConnection.Offline method 352
TADCustomConnection.Offlined property 331
TADCustomConnection.OnError event 332
TADCustomConnection.Online method 353
TADCustomConnection.OnLogin event 332
TADCustomConnection.OnLosted event 333
TADCustomConnection.OnRecover event 333
TADCustomConnection.OnRestored event 334
TADCustomConnection.Open method 353, 354
TADCustomConnection.OptionsIntf property 334
TADCustomConnection.Params property 334
TADCustomConnection.Ping method 354
TADCustomConnection.RDBMSKind property 335
TADCustomConnection.RefCount property 335
TADCustomConnection.RefreshMetadataCache method 355
TADCustomConnection.ReleaseClients method 356
TADCustomConnection.ResourceOptions property 335
TADCustomConnection.ResultConnectionDef property 336
TADCustomConnection.Rollback method 356
TADCustomConnection.RollbackRetaining method 357
TADCustomConnection.SharedCliHandle property 336
TADCustomConnection.StartTransaction method 358
TADCustomConnection.Temporary property 337
TADCustomConnection.Transaction property 337
TADCustomConnection.TxOptions property 337
TADCustomConnection.UpdateOptions property 338
TADCustomConnection.UpdateTransaction property 338
TADCustomEventAlerter class 358
Active 359
Connection 360
EventAlerterIntf 360
Names 360
OnAlert 361
OnTimeout 361
Options 361
Register 362
Signal 362
Unregister 362
TADCustomEventAlerter.Active property 359
TADCustomEventAlerter.Connection property 360
TADCustomEventAlerter.EventAlerterIntf property 360
TADCustomEventAlerter.Names property 360
TADCustomEventAlerter.OnAlert event 361
TADCustomEventAlerter.OnTimeout event 361
TADCustomEventAlerter.Options property 361
TADCustomEventAlerter.Register method 362
TADCustomEventAlerter.Signal method 362
TADCustomEventAlerter.Unregister method 362
TADCustomLocalSQL class 363
Active 364
Connection 364
DataSets 364
OnGetDataSet 365
OnReadDataSet 365
TADCustomLocalSQL.Active property 364
TADCustomLocalSQL.Connection property 364
TADCustomLocalSQL.DataSets property 364
TADCustomLocalSQL.OnGetDataSet event 365
TADCustomLocalSQL.OnReadDataSet event 365
TADCustomManager class 365
Active 367
ActiveStoredUsage 368
ActualConnectionDefFileName 368
ActualDriverDefFileName 368
AddConnectionDef 376
AfterLoadConnectionDefFile 368
AfterShutdown 368
AfterStartup 369
AutoCreated 369
BeforeLoadConnectionDefFile 369
BeforeShutdown 369
BeforeStartup 370
CanRefreshConnectionDefFile 370
Close 376
CloseConnectionDef 377
ConnectionCount 370

ConnectionDefFileAutoLoad 370
ConnectionDefFileLoaded 371
ConnectionDefFileName 371
ConnectionDefs 372
Connections 372
DeleteConnectionDef 377
DriverDefFileAutoLoad 373
DriverDefFileName 373
DriverDefs 373
DropConnections 377
FetchOptions 374
FindConnection 378
FormatOptions 374
GetBaseDriverID 378
GetCatalogNames 378
GetConnectionDefNames 379
GetConnectionDefParams 379
GetConnectionNames 380
GetDriverNames 380
GetFieldNames 380
GetGeneratorNames 381
GetKeyFieldNames 382
GetPackageNames 382
GetRDBMSKind 383
GetSchemaNames 383
GetStoredProcNames 384
GetTableNames 385
IsConnectionDef 385
LoadConnectionDefFile 386
ModifyConnectionDef 386
Open 386
RefreshConnectionDefFile 387
RefreshMetadataCache 387
ResourceOptions 374
SaveConnectionDefFile 387
SilentMode 375
State 375
UpdateOptions 375
WaitCursor 375
TADCustomManager.Active property 367
TADCustomManager.ActiveStoredUsage property 368
TADCustomManager.ActualConnectionDefFileName property 368
TADCustomManager.ActualDriverDefFileName property 368
TADCustomManager.AddConnectionDef method 376
TADCustomManager.AfterLoadConnectionDefFile event 368
TADCustomManager.AfterShutdown event 368
TADCustomManager.AfterStartup event 369
TADCustomManager.AutoCreated property 369
TADCustomManager.BeforeLoadConnectionDefFile event 369
TADCustomManager.BeforeShutdown event 369
TADCustomManager.BeforeStartup event 370
TADCustomManager.CanRefreshConnectionDefFile property 370
TADCustomManager.Close method 376
TADCustomManager.CloseConnectionDef method 377
TADCustomManager.ConnectionCount property 370
TADCustomManager.ConnectionDefFileAutoLoad property 370
TADCustomManager.ConnectionDefFileLoaded property 371
TADCustomManager.ConnectionDefFileName property 371
TADCustomManager.ConnectionDefs property 372
TADCustomManager.Connections property 372
TADCustomManager.DeleteConnectionDef method 377
TADCustomManager.DriverDefFileAutoLoad property 373
TADCustomManager.DriverDefFileName property 373
TADCustomManager.DriverDefs property 373
TADCustomManager.DropConnections method 377
TADCustomManager.FetchOptions property 374
TADCustomManager.FindConnection method 378
TADCustomManager.FormatOptions property 374
TADCustomManager.GetBaseDriverID method 378
TADCustomManager.GetCatalogNames method 378
TADCustomManager.GetConnectionDefNames method 379
TADCustomManager.GetConnectionDefParams method 379
TADCustomManager.GetConnectionNames method 380
TADCustomManager.GetDriverNames method 380
TADCustomManager.GetFieldNames method 380
TADCustomManager.GetGeneratorNames method 381
TADCustomManager.GetKeyFieldNames method 382
TADCustomManager.GetPackageNames method 382
TADCustomManager.GetRDBMSKind method 383
TADCustomManager.GetSchemaNames method 383
TADCustomManager.GetStoredProcNames method 384
TADCustomManager.GetTableNames method 385

TADCustomManager.IsConnectionDef method 385
TADCustomManager.LoadConnectionDefFile method 386
TADCustomManager.ModifyConnectionDef method 386
TADCustomManager.Open method 386
TADCustomManager.RefreshConnectionDefFile method 387
TADCustomManager.RefreshMetadataCache method 387
TADCustomManager.ResourceOptions property 374
TADCustomManager.SaveConnectionDefFile method 387
TADCustomManager.SilentMode property 375
TADCustomManager.State property 375
TADCustomManager.UpdateOptions property 375
TADCustomManager.WaitCursor property 375
TADCustomMemTable class 388
Adapter 389
AppendData 392
CommandText 389
ConstraintsDisabled 393
DisableStringTrim 389
FetchOnDemand 390
FileName 390
GetOptionalParam 393
IsClone 390
LogChanges 390
MergeChangeLog 393
PacketRecords 391
ProviderEOF 391
ReadOnly 391
SetOptionalParam 394
StatusFilter 391
XMLData 392
TADCustomMemTable.Adapter property 389
TADCustomMemTable.AppendData method 392
TADCustomMemTable.CommandText property 389
TADCustomMemTable.ConstraintsDisabled method 393
TADCustomMemTable.DisableStringTrim property 389
TADCustomMemTable.FetchOnDemand property 390
TADCustomMemTable.FileName property 390
TADCustomMemTable.GetOptionalParam method 393
TADCustomMemTable.IsClone property 390
TADCustomMemTable.LogChanges property 390
TADCustomMemTable.MergeChangeLog method 393
TADCustomMemTable.PacketRecords property 391
TADCustomMemTable.ProviderEOF property 391
TADCustomMemTable.ReadOnly property 391
TADCustomMemTable.SetOptionalParam method 394
TADCustomMemTable.StatusFilter property 391
TADCustomMemTable.XMLData property 392
TADCustomOptions class 846
Assign 847
RestoreDefaults 847
Version 846
TADCustomOptions.Assign method 847
TADCustomOptions.RestoreDefaults method 847
TADCustomOptions.Version property 846
TADCustomQuery class 394
DataSource 395
ExecSQL 397, 398, 399
ParamCount 396
SQL 396
Text 397
TADCustomQuery.DataSource property 395
TADCustomQuery.ExecSQL method 397, 398, 399
TADCustomQuery.ParamCount property 396
TADCustomQuery.SQL property 396
TADCustomQuery.Text property 397
TADCustomSchemaAdapter class 400
ApplyUpdates 402
Count 401
Reconcile 402
TableAdapters 401
TADCustomSchemaAdapter.ApplyUpdates method 402
TADCustomSchemaAdapter.Count property 401
TADCustomSchemaAdapter.Reconcile method 402
TADCustomSchemaAdapter.TableAdapters property 401
TADCustomStoredProc class 403
CatalogName 404
DescriptionsAvailable 406
ExecFunc 406, 407, 408, 409
ExecProc 410, 411, 412
Overload 404
PackageName 404
ParamCount 405
RowsAffected 405
SchemaName 405

StoredProcName 406
TADCustomStoredProc.CatalogName property 404
TADCustomStoredProc.DescriptionsAvailable method 406
TADCustomStoredProc.ExecFunc method 406, 407, 408, 409
TADCustomStoredProc.ExecProc method 410, 411, 412
TADCustomStoredProc.Overload property 404
TADCustomStoredProc.PackageName property 404
TADCustomStoredProc.ParamCount property 405
TADCustomStoredProc.RowsAffected property 405
TADCustomStoredProc.SchemaName property 405
TADCustomStoredProc.StoredProcName property 406
TADCustomTransaction class 413
Active 414
AfterCommit 414
AfterRollback 414
AfterStartTransaction 415
BeforeCommit 415
BeforeRollback 415
BeforeStartTransaction 415
Commit 417
CommitRetaining 417
Connection 416
DataSetCount 416
DataSets 416
Options 416
Rollback 418
RollbackRetaining 419
StartTransaction 419
TransactionIntf 417
TADCustomTransaction.Active property 414
TADCustomTransaction.AfterCommit event 414
TADCustomTransaction.AfterRollback event 414
TADCustomTransaction.AfterStartTransaction event 415
TADCustomTransaction.BeforeCommit event 415
TADCustomTransaction.BeforeRollback event 415
TADCustomTransaction.BeforeStartTransaction event 415
TADCustomTransaction.Commit method 417
TADCustomTransaction.CommitRetaining method 417
TADCustomTransaction.Connection property 416
TADCustomTransaction.DataSetCount property 416
TADCustomTransaction.DataSets property 416
TADCustomTransaction.Options property 416
TADCustomTransaction.Rollback method 418
TADCustomTransaction.RollbackRetaining method 419
TADCustomTransaction.StartTransaction method 419
TADCustomTransaction.TransactionIntf property 417
TADCustomUpdateObject class 420
Apply 421
DataSet 421
TADCustomUpdateObject.Apply method 421
TADCustomUpdateObject.DataSet property 421
TADDataSet class 584
ActiveStoredUsage 588
AddIndex 610
AfterApplyUpdates 588
AfterExecute 589
AfterGetRecord 589
AfterGetRecords 589
AfterRowRequest 589
Aggregates 590
AggregatesActive 590
ApplyMaster 611
ApplyRange 611
ApplyUpdates 611
AttachTable 612
BeforeApplyUpdates 591
BeforeExecute 591
BeforeGetRecords 592
BeforeRowRequest 592
BeginBatch 613
CachedUpdates 592
CancelRange 613
CancelUpdates 614
ChangeCount 593
CloneCursor 614
CloneSource 593
CommitUpdates 615
Constraints 594
ConstraintsEnabled 594
CopyDataSet 615
CopyRecord 616
CreateDataSet 617
CreateExpression 617
Data 595

DatSManager 596
DeleteIndex 618
DeleteIndexes 619
Delta 596
DetailFields 596
DisableConstraints 619
Disconnect 619
EditKey 620
EditRangeEnd 620
EditRangeStart 621
EmptyDataSet 621
EnableConstraints 621
EndBatch 622
Execute 622
FetchAgain 623
FetchAll 624
FetchBlobs 624
FetchDetails 625
FetchNext 625
FetchOptions 651
FilterChanges 597
FilteredData 597
FindKey 625
FindNearest 626
FindParam 626
FormatOptions 651
GetColumnField 627
GetDetailLinkFields 627
GetFieldColumn 627
GetGroupState 628
GetIndexNames 628
GetNextPacket 629
GetParentRow 629
GetRow 629
GotoCurrent 630
GotoKey 630
GotoNearest 631
GroupingLevel 598
IndexDefs 598
Indexes 598
IndexesActive 600
IndexFieldCount 600
IndexFieldNames 600
IndexFields 601
IndexName 601
IsIndexed 631
IsLinked 631
IsRanged 632
KeyExclusive 602
KeyFieldCount 602
LoadFromFile 632
LoadFromFileAtOpen 633
LoadFromStream 633
Locate 634
LocateEx 635, 636
Lookup 637
LookupEx 638, 639
MasterFields 602
MasterLink 603
MasterSource 603
Offline 640
OnMasterSetValues 604
OnReconcileError 604
OnUpdateError 605
OnUpdateRecord 605
OpenOrExecute 641
ParamByName 641
ParamCount 651
Params 651
ParentDataSet 607
Reconcile 642
RefreshRecord 642
RefreshUnknownRecord 643
Release 644
ResourceOptions 652
RevertRecord 644
RowError 607
SavePoint 607
SaveToFile 644
SaveToFileAtClose 645
SaveToStream 645
SetFieldAttributes 646
SetKey 647
SetRange 647

SetRangeEnd 648
SetRangeStart 648
SourceEOF 608
SourceView 608
Table 609
UndoLastChange 649
UpdateAttributes 649
UpdateConstraints 650
UpdateOptions 653
Updates 653
UpdatesPending 609
UpdateStatus 650
View 610
TADDataSet.ActiveStoredUsage property 588
TADDataSet.AddIndex method 610
TADDataSet.AfterApplyUpdates event 588
TADDataSet.AfterExecute event 589
TADDataSet.AfterGetRecord event 589
TADDataSet.AfterGetRecords event 589
TADDataSet.AfterRowRequest event 589
TADDataSet.Aggregates property 590
TADDataSet.AggregatesActive property 590
TADDataSet.ApplyMaster method 611
TADDataSet.ApplyRange method 611
TADDataSet.ApplyUpdates method 611
TADDataSet.AttachTable method 612
TADDataSet.BeforeApplyUpdates event 591
TADDataSet.BeforeExecute event 591
TADDataSet.BeforeGetRecords event 592
TADDataSet.BeforeRowRequest event 592
TADDataSet.BeginBatch method 613
TADDataSet.CachedUpdates property 592
TADDataSet.CancelRange method 613
TADDataSet.CancelUpdates method 614
TADDataSet.ChangeCount property 593
TADDataSet.CloneCursor method 614
TADDataSet.CloneSource property 593
TADDataSet.CommitUpdates method 615
TADDataSet.Constraints property 594
TADDataSet.ConstraintsEnabled property 594
TADDataSet.CopyDataSet method 615
TADDataSet.CopyRecord method 616
TADDataSet.CreateDataSet method 617
TADDataSet.CreateExpression method 617
TADDataSet.Data property 595
TADDataSet.DatSManager property 596
TADDataSet.DeleteIndex method 618
TADDataSet.DeleteIndexes method 619
TADDataSet.Delta property 596
TADDataSet.DetailFields property 596
TADDataSet.DisableConstraints method 619
TADDataSet.Disconnect method 619
TADDataSet.EditKey method 620
TADDataSet.EditRangeEnd method 620
TADDataSet.EditRangeStart method 621
TADDataSet.EmptyDataSet method 621
TADDataSet.EnableConstraints method 621
TADDataSet.EndBatch method 622
TADDataSet.Execute method 622
TADDataSet.FetchAgain method 623
TADDataSet.FetchAll method 624
TADDataSet.FetchBlobs method 624
TADDataSet.FetchDetails method 625
TADDataSet.FetchNext method 625
TADDataSet.FetchOptions property 651
TADDataSet.FilterChanges property 597
TADDataSet.FilteredData property 597
TADDataSet.FindKey method 625
TADDataSet.FindNearest method 626
TADDataSet.FindParam method 626
TADDataSet.FormatOptions property 651
TADDataSet.GetColumnField method 627
TADDataSet.GetDetailLinkFields method 627
TADDataSet.GetFieldColumn method 627
TADDataSet.GetGroupState method 628
TADDataSet.GetIndexNames method 628
TADDataSet.GetNextPacket method 629
TADDataSet.GetParentRow method 629
TADDataSet.GetRow method 629
TADDataSet.GotoCurrent method 630
TADDataSet.GotoKey method 630
TADDataSet.GotoNearest method 631
TADDataSet.GroupingLevel property 598
TADDataSet.IndexDefs property 598

TADDataSet.Indexes property 598
TADDataSet.IndexesActive property 600
TADDataSet.IndexFieldCount property 600
TADDataSet.IndexFieldNames property 600
TADDataSet.IndexFields property 601
TADDataSet.IndexName property 601
TADDataSet.IsIndexed method 631
TADDataSet.IsLinked method 631
TADDataSet.IsRanged method 632
TADDataSet.KeyExclusive property 602
TADDataSet.KeyFieldCount property 602
TADDataSet.LoadFromFile method 632
TADDataSet.LoadFromFileAtOpen method 633
TADDataSet.LoadFromStream method 633
TADDataSet.Locate method 634
TADDataSet.LocateEx method 635, 636
TADDataSet.Lookup method 637
TADDataSet.LookupEx method 638, 639
TADDataSet.MasterFields property 602
TADDataSet.MasterLink property 603
TADDataSet.MasterSource property 603
TADDataSet.Offline method 640
TADDataSet.OnMasterSetValues event 604
TADDataSet.OnReconcileError event 604
TADDataSet.OnUpdateError event 605
TADDataSet.OnUpdateRecord event 605
TADDataSet.OpenOrExecute method 641
TADDataSet.ParamByName method 641
TADDataSet.ParamCount property 651
TADDataSet.Params property 651
TADDataSet.ParentDataSet property 607
TADDataSet.Reconcile method 642
TADDataSet.RefreshRecord method 642
TADDataSet.RefreshUnknownRecord method 643
TADDataSet.Release method 644
TADDataSet.ResourceOptions property 652
TADDataSet.RevertRecord method 644
TADDataSet.RowError property 607
TADDataSet.SavePoint property 607
TADDataSet.SaveToFile method 644
TADDataSet.SaveToFileAtClose method 645
TADDataSet.SaveToStream method 645
TADDataSet.SetFieldAttributes method 646
TADDataSet.SetKey method 647
TADDataSet.SetRange method 647
TADDataSet.SetRangeEnd method 648
TADDataSet.SetRangeStart method 648
TADDataSet.SourceEOF property 608
TADDataSet.SourceView property 608
TADDataSet.Table property 609
TADDataSet.UndoLastChange method 649
TADDataSet.UpdateAttributes method 649
TADDataSet.UpdateConstraints method 650
TADDataSet.UpdateOptions property 653
TADDataSet.Updates property 653
TADDataSet.UpdatesPending property 609
TADDataSet.UpdateStatus method 650
TADDataSet.View property 610
TADDBError class 837
  CommandTextOffset 838
  ErrorCode 838
  Kind 838
  Level 838
  Message 839
  ObjName 839
  RowIndex 839
TADDBError.CommandTextOffset property 838
TADDBError.ErrorCode property 838
TADDBError.Kind property 838
TADDBError.Level property 838
TADDBError.Message property 839
TADDBError.ObjName property 839
TADDBError.RowIndex property 839
TADEventAlerter class 422
  Active 423
  Connection 423
  Names 424
  OnAlert 424
  OnTimeout 425
  Options 425
TADEventAlerter.Active property 423
TADEventAlerter.Connection property 423
TADEventAlerter.Names property 424
TADEventAlerter.OnAlert event 424

TADEventAlerter.OnTimeout event 425
TADEventAlerter.Options property 425
TADEventAlerterOptions class 847
AutoRegister 848
Kind 848
Synchronize 848
Timeout 849
TADEventAlerterOptions.AutoRegister property 848
TADEventAlerterOptions.Kind property 848
TADEventAlerterOptions.Synchronize property 848
TADEventAlerterOptions.Timeout property 849
TADFetchOptions class 849
ActualRowsetSize 850
Assign 850
AssignedValues 851
AutoClose 851
AutoFetchAll 852
Cache 852
CursorKind 853
DetailCascade 854
DetailDelay 854
DetailOptimize 855
Items 855
LiveWindowFastFirst 856
LiveWindowParanoic 856
Mode 857
RecordCountMode 858
RecsMax 858
RecsSkip 859
RestoreDefaults 851
RowsetSize 859
Unidirectional 860
TADFetchOptions.ActualRowsetSize property 850
TADFetchOptions.Assign method 850
TADFetchOptions.AssignedValues property 851
TADFetchOptions.AutoClose property 851
TADFetchOptions.AutoFetchAll property 852
TADFetchOptions.Cache property 852
TADFetchOptions.CursorKind property 853
TADFetchOptions.DetailCascade property 854
TADFetchOptions.DetailDelay property 854
TADFetchOptions.DetailOptimize property 855
TADFetchOptions.Items property 855
TADFetchOptions.LiveWindowFastFirst property 856
TADFetchOptions.LiveWindowParanoic property 856
TADFetchOptions.Mode property 857
TADFetchOptions.RecordCountMode property 858
TADFetchOptions.RecsMax property 858
TADFetchOptions.RecsSkip property 859
TADFetchOptions.RestoreDefaults method 851
TADFetchOptions.RowsetSize property 859
TADFetchOptions.Unidirectional property 860
TADFormatOptions class 860
Assign 862
AssignedValues 862
DataSnapCompatibility 863
DefaultParamDataType 863
FmtDisplayDate 863
FmtDisplayDateTime 863
FmtDisplayNumeric 864
FmtDisplayTime 864
FmtEditNumeric 864
InlineDataSize 865
MapRules 865
MaxBcdPrecision 865
MaxBcdScale 866
MaxStringSize 866
OwnMapRules 866
QuoteIdentifiers 867
RestoreDefaults 862
Round2Scale 867
SortLocale 867
SortOptions 868
StrsDivLen2 861
StrsEmpty2Null 869
StrsTrim 869
StrsTrim2Len 870
TADFormatOptions.Assign method 862
TADFormatOptions.AssignedValues property 862
TADFormatOptions.DataSnapCompatibility property 863
TADFormatOptions.DefaultParamDataType property 863
TADFormatOptions.FmtDisplayDate property 863
TADFormatOptions.FmtDisplayDateTime property 863
TADFormatOptions.FmtDisplayNumeric property 864

TADFormatOptions.FmtDisplayTime property 864
TADFormatOptions.FmtEditNumeric property 864
TADFormatOptions.InlineDataSize property 865
TADFormatOptions.MapRules property 865
TADFormatOptions.MaxBcdPrecision property 865
TADFormatOptions.MaxBcdScale property 866
TADFormatOptions.MaxStringSize property 866
TADFormatOptions.OwnMapRules property 866
TADFormatOptions.QuoteIdentifiers property 867
TADFormatOptions.RestoreDefaults method 862
TADFormatOptions.Round2Scale property 867
TADFormatOptions.SortLocale property 867
TADFormatOptions.SortOptions property 868
TADFormatOptions.StrsDivLen2 property 861
TADFormatOptions.StrsEmpty2Null property 869
TADFormatOptions.StrsTrim property 869
TADFormatOptions.StrsTrim2Len property 870
TADGUIxAsyncExecuteDialog class 669
AsyncDialog 670
Caption 670
HideDelay 670
OnHide 670
OnShow 671
Prompt 671
ShowDelay 671
TADGUIxAsyncExecuteDialog.AsyncDialog property 670
TADGUIxAsyncExecuteDialog.Caption property 670
TADGUIxAsyncExecuteDialog.HideDelay property 670
TADGUIxAsyncExecuteDialog.OnHide event 670
TADGUIxAsyncExecuteDialog.OnShow event 671
TADGUIxAsyncExecuteDialog.Prompt property 671
TADGUIxAsyncExecuteDialog.ShowDelay property 671
TADGUIxComponent class 671
Provider 672
TADGUIxComponent.Provider property 672
TADGUIxErrorDialog class 672
Caption 674
Enabled 674
ErrorDialog 673
Execute 673
OnHide 674
OnShow 674
StayOnTop 675
TADGUIxErrorDialog.Caption property 674
TADGUIxErrorDialog.Enabled property 674
TADGUIxErrorDialog.ErrorDialog property 673
TADGUIxErrorDialog.Execute method 673
TADGUIxErrorDialog.OnHide property 674
TADGUIxErrorDialog.OnShow property 674
TADGUIxErrorDialog.StayOnTop property 675
TADGUIxLoginDialog class 675
Caption 677
ChangeExpiredPassword 677
ConnectionDef 676
Execute 676
GetAllLoginParams 677
HistoryEnabled 677
HistoryKey 678
HistoryStorage 678
HistoryWithPassword 678
LoginDialog 676
LoginRetries 678
OnChangePassword 679
OnHide 679
OnLogin 679
OnShow 679
VisibleItems 679
TADGUIxLoginDialog.Caption property 677
TADGUIxLoginDialog.ChangeExpiredPassword property 677
TADGUIxLoginDialog.ConnectionDef property 676
TADGUIxLoginDialog.Execute method 676
TADGUIxLoginDialog.GetAllLoginParams method 677
TADGUIxLoginDialog.HistoryEnabled property 677
TADGUIxLoginDialog.HistoryKey property 678
TADGUIxLoginDialog.HistoryStorage property 678
TADGUIxLoginDialog.HistoryWithPassword property 678
TADGUIxLoginDialog.LoginDialog property 676
TADGUIxLoginDialog.LoginRetries property 678
TADGUIxLoginDialog.OnChangePassword property 679
TADGUIxLoginDialog.OnHide event 679
TADGUIxLoginDialog.OnLogin property 679
TADGUIxLoginDialog.OnShow event 679
TADGUIxLoginDialog.VisibleItems property 679
TADGUIxScriptDialog class 680

Caption 681
OnHide 681
OnInput 681
OnOutput 681
OnPause 682
OnProgress 682
OnShow 682
Options 682
ScriptDialog 681
TADGUIxScriptDialog.Caption property 681
TADGUIxScriptDialog.OnHide event 681
TADGUIxScriptDialog.OnInput property 681
TADGUIxScriptDialog.OnOutput property 681
TADGUIxScriptDialog.OnPause property 682
TADGUIxScriptDialog.OnProgress property 682
TADGUIxScriptDialog.OnShow event 682
TADGUIxScriptDialog.Options property 682
TADGUIxScriptDialog.ScriptDialog property 681
TADGUIxWaitCursor class 683
OnHide 683
OnShow 684
ScreenCursor 684
WaitCursor 683
TADGUIxWaitCursor.OnHide event 683
TADGUIxWaitCursor.OnShow event 684
TADGUIxWaitCursor.ScreenCursor property 684
TADGUIxWaitCursor.WaitCursor property 683
TADIBBackup class 753
Backup 754
BackupFiles 755
Database 755
Options 755
Verbose 756
TADIBBackup.Backup method 754
TADIBBackup.BackupFiles property 755
TADIBBackup.Database property 755
TADIBBackup.Options property 755
TADIBBackup.Verbose property 756
TADIBNBackup class 756
Backup 757
BackupFile 758
Database 758
Level 758
Options 758
TADIBNBackup.Backup method 757
TADIBNBackup.BackupFile property 758
TADIBNBackup.Database property 758
TADIBNBackup.Level property 758
TADIBNBackup.Options property 758
TADIBNRestore class 759
BackupFiles 760
Database 760
Options 760
Restore 760
TADIBNRestore.BackupFiles property 760
TADIBNRestore.Database property 760
TADIBNRestore.Options property 760
TADIBNRestore.Restore method 760
TADIBRestore class 761
BackupFiles 762
Database 762
Options 763
PageBuffers 763
PageSize 763
Restore 762
Verbose 764
TADIBRestore.BackupFiles property 762
TADIBRestore.Database property 762
TADIBRestore.Options property 763
TADIBRestore.PageBuffers property 763
TADIBRestore.PageSize property 763
TADIBRestore.Restore method 762
TADIBRestore.Verbose property 764
TADIBSecurity class 764
AddUser 766
AFirstName 767
AGroupID 767
AGroupName 767
ALastName 767
AMiddleName 768
APassword 768
ARoleName 768
AUserID 768
AUserName 768

DeleteUser 766
DisplayUser 766
DisplayUsers 766
Modified 768
ModifyUser 767
Users 765
TADIBSecurity.AddUser method 766
TADIBSecurity.AFirstName property 767
TADIBSecurity.AGroupID property 767
TADIBSecurity.AGroupName property 767
TADIBSecurity.ALastName property 767
TADIBSecurity.AMiddleName property 768
TADIBSecurity.APassword property 768
TADIBSecurity.ARoleName property 768
TADIBSecurity.AUserID property 768
TADIBSecurity.AUserName property 768
TADIBSecurity.DeleteUser method 766
TADIBSecurity.DisplayUser method 766
TADIBSecurity.DisplayUsers method 766
TADIBSecurity.Modified property 768
TADIBSecurity.ModifyUser method 767
TADIBSecurity.Users property 765
TADIBService class 769
DriverLink 769
Host 769
OnProgress 770
Password 770
Protocol 770
QueryTimeout 770
SqlRoleName 771
UserName 771
TADIBService.DriverLink property 769
TADIBService.Host property 769
TADIBService.OnProgress event 770
TADIBService.Password property 770
TADIBService.Protocol property 770
TADIBService.QueryTimeout property 770
TADIBService.SqlRoleName property 771
TADIBService.UserName property 771
TADIBTrace class 771
Config 773
List 772
Resume 772
SessionID 774
SessionName 774
Start 773
Stop 773
Suspend 773
TADIBTrace.Config property 773
TADIBTrace.List method 772
TADIBTrace.Resume method 772
TADIBTrace.SessionID property 774
TADIBTrace.SessionName property 774
TADIBTrace.Start method 773
TADIBTrace.Stop method 773
TADIBTrace.Suspend method 773
TADIBValidate class 774
Analyze 775
CheckOnly 776
Database 776
Options 777
Repair 776
Sweep 776
TADIBValidate.Analyze method 775
TADIBValidate.CheckOnly method 776
TADIBValidate.Database property 776
TADIBValidate.Options property 777
TADIBValidate.Repair method 776
TADIBValidate.Sweep method 776
TADIndex class 653
Active 655
ActualActive 654
CaseInsFields 655
DataSet 655
DescFields 656
Distinct 656
Expression 656
Fields 657
Filter 657
FilterOptions 657
Name 658
Options 658
Selected 659
TADIndex.Active property 655

TADIndex.ActualActive property 654
TADIndex.CaseInsFields property 655
TADIndex.DataSet property 655
TADIndex.DescFields property 656
TADIndex.Distinct property 656
TADIndex.Expression property 656
TADIndex.Fields property 657
TADIndex.Filter property 657
TADIndex.FilterOptions property 657
TADIndex.Name property 658
TADIndex.Options property 658
TADIndex.Selected property 659
TADIndexes class 659
Add 660
FindIndex 660
FindIndexForFields 660
IndexByName 661
Items 659
TADIndexes.Add method 660
TADIndexes.FindIndex method 660
TADIndexes.FindIndexForFields method 660
TADIndexes.IndexByName method 661
TADIndexes.Items property 659
TADLocalSQL class 825
Active 826
Connection 826
DataSets 827
DisableControls 827
IncludeHiddenFields 827
MultipleCursors 828
OnGetDataSet 828
OnReadDataSet 828
SchemaName 829
TADLocalSQL.Active property 826
TADLocalSQL.Connection property 826
TADLocalSQL.DataSets property 827
TADLocalSQL.DisableControls property 827
TADLocalSQL.IncludeHiddenFields property 827
TADLocalSQL.MultipleCursors property 828
TADLocalSQL.OnGetDataSet event 828
TADLocalSQL.OnReadDataSet event 828
TADLocalSQL.SchemaName property 829
TADLocalSQLDataSet class 425
ActualName 426
Adapter 426
DataSet 427
IsValid 426
Name 427
Temporary 427
TADLocalSQLDataSet.ActualName property 426
TADLocalSQLDataSet.Adapter property 426
TADLocalSQLDataSet.DataSet property 427
TADLocalSQLDataSet.IsValid property 426
TADLocalSQLDataSet.Name property 427
TADLocalSQLDataSet.Temporary property 427
TADLocalSQLDataSets class 428
Add 428, 429
Items 428
Remove 429
TADLocalSQLDataSets.Add method 428, 429
TADLocalSQLDataSets.Items property 428
TADLocalSQLDataSets.Remove method 429
TADManager and TADConnection Questions 230
TADManager class 429
Active 430
ActiveStoredUsage 431
AfterLoadConnectionDefFile 431
AfterShutdown 431
AfterStartup 431
BeforeLoadConnectionDefFile 432
BeforeShutdown 432
BeforeStartup 432
ConnectionDefFileAutoLoad 432
ConnectionDefFileName 433
DriverDefFileAutoLoad 433
DriverDefFileName 434
FetchOptions 434
FormatOptions 434
ResourceOptions 435
SilentMode 435
UpdateOptions 435
WaitCursor 435
TADManager.Active property 430
TADManager.ActiveStoredUsage property 431

TADManager.AfterLoadConnectionDefFile event 431
TADManager.AfterShutdown event 431
TADManager.AfterStartup event 431
TADManager.BeforeLoadConnectionDefFile event 432
TADManager.BeforeShutdown event 432
TADManager.BeforeStartup event 432
TADManager.ConnectionDefFileAutoLoad property 432
TADManager.ConnectionDefFileName property 433
TADManager.DriverDefFileAutoLoad property 433
TADManager.DriverDefFileName property 434
TADManager.FetchOptions property 434
TADManager.FormatOptions property 434
TADManager.ResourceOptions property 435
TADManager.SilentMode property 435
TADManager.UpdateOptions property 435
TADManager.WaitCursor property 435
TADMapRule class 870
NameMask 871
PrecMax 871
PrecMin 871
ScaleMax 872
ScaleMin 872
SizeMax 872
SizeMin 873
SourceDataType 873
TargetDataType 873
TADMapRule.NameMask property 871
TADMapRule.PrecMax property 871
TADMapRule.PrecMin property 871
TADMapRule.ScaleMax property 872
TADMapRule.ScaleMin property 872
TADMapRule.SizeMax property 872
TADMapRule.SizeMin property 873
TADMapRule.SourceDataType property 873
TADMapRule.TargetDataType property 873
TADMapRules class 873
Add 874
Items 874
TADMapRules.Add method 874
TADMapRules.Items property 874
TADMasterDataLink class 661
ActualDelayedScrollEnabled 662
CancelSync 663
DelayedScrollEnabled 662
DisableDelayedScroll 663
DisableScroll 664
EnableDelayedScroll 664
EnableScroll 664
Synchronize 665
TADMasterDataLink.ActualDelayedScrollEnabled property 662
TADMasterDataLink.CancelSync method 663
TADMasterDataLink.DelayedScrollEnabled property 662
TADMasterDataLink.DisableDelayedScroll method 663
TADMasterDataLink.DisableScroll method 664
TADMasterDataLink.EnableDelayedScroll method 664
TADMasterDataLink.EnableScroll method 664
TADMasterDataLink.Synchronize method 665
TADMemTable class 436
ActiveStoredUsage 437
Adapter 437
AfterApplyUpdates 438
AfterExecute 438
AfterGetRecord 438
AfterGetRecords 438
AfterRowRequest 439
Aggregates 439
AggregatesActive 440
BeforeApplyUpdates 440
BeforeExecute 441
BeforeGetRecords 441
BeforeRowRequest 441
CachedUpdates 441
Constraints 442
ConstraintsEnabled 443
DetailFields 443
FetchOptions 444
FilterChanges 444
FormatOptions 445
IndexDefs 445
Indexes 445
IndexesActive 446
IndexFieldNames 447
IndexName 447
LocalSQL 448

MasterFields 448
MasterSource 448
OnReconcileError 449
OnUpdateError 450
OnUpdateRecord 450
ResourceOptions 452
UpdateOptions 452
TADMemTable Questions 234
TADMemTable.ActiveStoredUsage property 437
TADMemTable.Adapter property 437
TADMemTable.AfterApplyUpdates event 438
TADMemTable.AfterExecute event 438
TADMemTable.AfterGetRecord event 438
TADMemTable.AfterGetRecords event 438
TADMemTable.AfterRowRequest event 439
TADMemTable.Aggregates property 439
TADMemTable.AggregatesActive property 440
TADMemTable.BeforeApplyUpdates event 440
TADMemTable.BeforeExecute event 441
TADMemTable.BeforeGetRecords event 441
TADMemTable.BeforeRowRequest event 441
TADMemTable.CachedUpdates property 441
TADMemTable.Constraints property 442
TADMemTable.ConstraintsEnabled property 443
TADMemTable.DetailFields property 443
TADMemTable.FetchOptions property 444
TADMemTable.FilterChanges property 444
TADMemTable.FormatOptions property 445
TADMemTable.IndexDefs property 445
TADMemTable.Indexes property 445
TADMemTable.IndexesActive property 446
TADMemTable.IndexFieldNames property 447
TADMemTable.IndexName property 447
TADMemTable.LocalSQL property 448
TADMemTable.MasterFields property 448
TADMemTable.MasterSource property 448
TADMemTable.OnReconcileError event 449
TADMemTable.OnUpdateError event 450
TADMemTable.OnUpdateRecord event 450
TADMemTable.ResourceOptions property 452
TADMemTable.UpdateOptions property 452
TADMetaInfoCommand class 452
Active 453
AfterClose 454
AfterOpen 454
BaseObjectName 454
BeforeClose 455
BeforeOpen 455
CatalogName 455
Connection 456
ConnectionName 456
FormatOptions 456
MetaInfoKind 457
ObjectName 457
ObjectScopes 458
OnCommandChanged 458
OnError 459
Overload 459
SchemaName 460
TableKinds 460
Wildcard 461
TADMetaInfoCommand.Active property 453
TADMetaInfoCommand.AfterClose event 454
TADMetaInfoCommand.AfterOpen event 454
TADMetaInfoCommand.BaseObjectName property 454
TADMetaInfoCommand.BeforeClose event 455
TADMetaInfoCommand.BeforeOpen event 455
TADMetaInfoCommand.CatalogName property 455
TADMetaInfoCommand.Connection property 456
TADMetaInfoCommand.ConnectionName property 456
TADMetaInfoCommand.FormatOptions property 456
TADMetaInfoCommand.MetaInfoKind property 457
TADMetaInfoCommand.ObjectName property 457
TADMetaInfoCommand.ObjectScopes property 458
TADMetaInfoCommand.OnCommandChanged event 458
TADMetaInfoCommand.OnError event 459
TADMetaInfoCommand.Overload property 459
TADMetaInfoCommand.SchemaName property 460
TADMetaInfoCommand.TableKinds property 460
TADMetaInfoCommand.Wildcard property 461
TADMetaInfoQuery class 461
ActiveStoredUsage 462
AfterGetRecord 463
AfterGetRecords 463

AfterRowRequest 463
Aggregates 463
AggregatesActive 464
BaseObjectName 465
BeforeGetRecords 466
BeforeRowRequest 466
CatalogName 466
Connection 466
ConnectionName 467
FetchOptions 467
FormatOptions 467
Indexes 468
IndexesActive 469
IndexFieldNames 469
IndexName 470
LocalSQL 470
MetaInfoKind 471
ObjectName 471
ObjectScopes 472
OnCommandChanged 472
OnError 472
ResourceOptions 473
SchemaAdapter 474
SchemaName 474
TableKinds 475
Transaction 475
UpdateTransaction 475
Wildcard 476
TADMetaInfoQuery.ActiveStoredUsage property 462
TADMetaInfoQuery.AfterGetRecord event 463
TADMetaInfoQuery.AfterGetRecords event 463
TADMetaInfoQuery.AfterRowRequest event 463
TADMetaInfoQuery.Aggregates property 463
TADMetaInfoQuery.AggregatesActive property 464
TADMetaInfoQuery.BaseObjectName property 465
TADMetaInfoQuery.BeforeGetRecords event 466
TADMetaInfoQuery.BeforeRowRequest event 466
TADMetaInfoQuery.CatalogName property 466
TADMetaInfoQuery.Connection property 466
TADMetaInfoQuery.ConnectionName property 467
TADMetaInfoQuery.FetchOptions property 467
TADMetaInfoQuery.FormatOptions property 467
TADMetaInfoQuery.Indexes property 468
TADMetaInfoQuery.IndexesActive property 469
TADMetaInfoQuery.IndexFieldNames property 469
TADMetaInfoQuery.IndexName property 470
TADMetaInfoQuery.LocalSQL property 470
TADMetaInfoQuery.MetaInfoKind property 471
TADMetaInfoQuery.ObjectName property 471
TADMetaInfoQuery.ObjectScopes property 472
TADMetaInfoQuery.OnCommandChanged event 472
TADMetaInfoQuery.OnError event 472
TADMetaInfoQuery.ResourceOptions property 473
TADMetaInfoQuery.SchemaAdapter property 474
TADMetaInfoQuery.SchemaName property 474
TADMetaInfoQuery.TableKinds property 475
TADMetaInfoQuery.Transaction property 475
TADMetaInfoQuery.UpdateTransaction property 475
TADMetaInfoQuery.Wildcard property 476
TADMoniRemoteClientLink class 723
Host 723
Port 723
Timeout 724
Tracing 724
TADMoniRemoteClientLink.Host property 723
TADMoniRemoteClientLink.Port property 723
TADMoniRemoteClientLink.Timeout property 724
TADMoniRemoteClientLink.Tracing property 724
TADMSAccessService class 786
Compact 786
CreateDB 787
Database 788
DBVersion 789
DestDatabase 789
DriverLink 789
Drop 788
Encrypted 790
Password 790
Repair 788
ResetPassword 790
SortOrder 791
TADMSAccessService.Compact method 786
TADMSAccessService.CreateDB method 787
TADMSAccessService.Database property 788

TADMSAccessService.DBVersion property 789
TADMSAccessService.DestDatabase property 789
TADMSAccessService.DriverLink property 789
TADMSAccessService.Drop method 788
TADMSAccessService.Encrypted property 790
TADMSAccessService.Password property 790
TADMSAccessService.Repair method 788
TADMSAccessService.ResetPassword property 790
TADMSAccessService.SortOrder property 791
TADMSSQLError class 793
TADPhysADSDriverLink class 737
DateFormat 738
Decimals 738
DefaultPath 739
Epoch 739
Exact 739
SearchPath 740
ShowDeleted 740
TADPhysADSDriverLink.DateFormat property 738
TADPhysADSDriverLink.Decimals property 738
TADPhysADSDriverLink.DefaultPath property 739
TADPhysADSDriverLink.Epoch property 739
TADPhysADSDriverLink.Exact property 739
TADPhysADSDriverLink.SearchPath property 740
TADPhysADSDriverLink.ShowDeleted property 740
TADPhysASADriverLink class 748
ToolLib 749
TADPhysASADriverLink.ToolLib property 749
TADPhysDataSnapDriverLink class 749
TADPhysDB2DriverLink class 751
TADPhysDBXDriverLink class 752
TADPhysDriverLink class 778
ActualDriverID 779
BaseDriverID 781
DriverID 781
DriverIntf 779
DriverState 780
OnDriverCreated 782
OnDriverDestroying 782
Release 780
Services 780
ServicesCount 780
VendorHome 782
VendorLib 782
TADPhysDriverLink.ActualDriverID property 779
TADPhysDriverLink.BaseDriverID property 781
TADPhysDriverLink.DriverID property 781
TADPhysDriverLink.DriverIntf property 779
TADPhysDriverLink.DriverState property 780
TADPhysDriverLink.OnDriverCreated event 782
TADPhysDriverLink.OnDriverDestroying event 782
TADPhysDriverLink.Release method 780
TADPhysDriverLink.Services property 780
TADPhysDriverLink.ServicesCount property 780
TADPhysDriverLink.VendorHome property 782
TADPhysDriverLink.VendorLib property 782
TADPhysDriverService class 783
AfterExecute 784
BeforeExecute 784
CliObj 783
OnError 784
TADPhysDriverService.AfterExecute event 784
TADPhysDriverService.BeforeExecute event 784
TADPhysDriverService.CliObj property 783
TADPhysDriverService.OnError event 784
TADPhysIBDriverLink class 777
TADPhysMSAccessDriverLink class 791
TADPhysMSSQLDriverLink class 793
TADPhysMySQLDriverLink class 794
EmbeddedArgs 795
EmbeddedGroups 795
TADPhysMySQLDriverLink.EmbeddedArgs property 795
TADPhysMySQLDriverLink.EmbeddedGroups property 795
TADPhysODBCBaseDriverLink class 797
GetDrivers 798
GetDSNs 798
ODBCAdvanced 799
ODBCDriver 799
TADPhysODBCBaseDriverLink.GetDrivers method 798
TADPhysODBCBaseDriverLink.GetDSNs method 798
TADPhysODBCBaseDriverLink.ODBCAdvanced property 799
TADPhysODBCBaseDriverLink.ODBCDriver property 799
TADPhysODBCBaseService class 799
TADPhysODBCDriverLink class 796

TADPhysOracleDriverLink class 800
GetOracleHomes 801
GetTNSServices 801
NLSLang 802
Threaded 802
TNSAdmin 802
TADPhysOracleDriverLink.GetOracleHomes method 801
TADPhysOracleDriverLink.GetTNSServices method 801
TADPhysOracleDriverLink.NLSLang property 802
TADPhysOracleDriverLink.Threaded property 802
TADPhysOracleDriverLink.TNSAdmin property 802
TADPhysPgDriverLink class 803
TADPhysSQLiteDriverLink class 805
TADPhysTDBXDriverLink class 829
TADQuery class 476
ActiveStoredUsage 479
AfterApplyUpdates 479
AfterExecute 479
AfterGetRecord 480
AfterGetRecords 480
AfterRowRequest 480
Aggregates 480
AggregatesActive 481
BeforeApplyUpdates 482
BeforeExecute 482
BeforeGetRecords 482
BeforeRowRequest 482
CachedUpdates 483
Connection 483
ConnectionName 484
Constraints 484
ConstraintsEnabled 485
DetailFields 486
FetchOptions 486
FilterChanges 486
FormatOptions 487
Indexes 487
IndexesActive 488
IndexFieldNames 489
IndexName 489
LocalSQL 490
MacroCount 478
Macros 490
MasterFields 491
MasterSource 491
OnCommandChanged 491
OnError 492
OnExecuteError 493
OnMasterSetValues 493
OnReconcileError 494
OnUpdateError 495
OnUpdateRecord 495
Params 497
ResourceOptions 498
RowsAffected 478
SchemaAdapter 498
SQL 499
Transaction 500
UpdateObject 500
UpdateOptions 500
UpdateTransaction 501
TADQuery, TADStoredProc and TADUpdateSQL Questions 231
TADQuery.ActiveStoredUsage property 479
TADQuery.AfterApplyUpdates event 479
TADQuery.AfterExecute event 479
TADQuery.AfterGetRecord event 480
TADQuery.AfterGetRecords event 480
TADQuery.AfterRowRequest event 480
TADQuery.Aggregates property 480
TADQuery.AggregatesActive property 481
TADQuery.BeforeApplyUpdates event 482
TADQuery.BeforeExecute event 482
TADQuery.BeforeGetRecords event 482
TADQuery.BeforeRowRequest event 482
TADQuery.CachedUpdates property 483
TADQuery.Connection property 483
TADQuery.ConnectionName property 484
TADQuery.Constraints property 484
TADQuery.ConstraintsEnabled property 485
TADQuery.DetailFields property 486
TADQuery.FetchOptions property 486
TADQuery.FilterChanges property 486
TADQuery.FormatOptions property 487
TADQuery.Indexes property 487

TADQuery.IndexesActive property 488
TADQuery.IndexFieldNames property 489
TADQuery.IndexName property 489
TADQuery.LocalSQL property 490
TADQuery.MacroCount property 478
TADQuery.Macros property 490
TADQuery.MasterFields property 491
TADQuery.MasterSource property 491
TADQuery.OnCommandChanged event 491
TADQuery.OnError event 492
TADQuery.OnExecuteError event 493
TADQuery.OnMasterSetValues event 493
TADQuery.OnReconcileError event 494
TADQuery.OnUpdateError event 495
TADQuery.OnUpdateRecord event 495
TADQuery.Params property 497
TADQuery.ResourceOptions property 498
TADQuery.RowsAffected property 478
TADQuery.SchemaAdapter property 498
TADQuery.SQL property 499
TADQuery.Transaction property 500
TADQuery.UpdateObject property 500
TADQuery.UpdateOptions property 500
TADQuery.UpdateTransaction property 501
TADRdbmsDataSet class 501
Connection 503
ConnectionName 503
Disconnect 508
FetchOptions 503
FindMacro 508
FormatOptions 504
MacroByName 509
MacroCount 512
Macros 512
OnCommandChanged 504
OnError 504
Open 509, 510
ParamBindMode 513
Prepare 511
Prepared 505
ResourceOptions 506
RowsAffected 513
SchemaAdapter 506
Transaction 507
Unprepare 512
UpdateOptions 507
UpdateTransaction 507
TADRdbmsDataSet.Connection property 503
TADRdbmsDataSet.ConnectionName property 503
TADRdbmsDataSet.Disconnect method 508
TADRdbmsDataSet.FetchOptions property 503
TADRdbmsDataSet.FindMacro method 508
TADRdbmsDataSet.FormatOptions property 504
TADRdbmsDataSet.MacroByName method 509
TADRdbmsDataSet.MacroCount property 512
TADRdbmsDataSet.Macros property 512
TADRdbmsDataSet.OnCommandChanged event 504
TADRdbmsDataSet.OnError event 504
TADRdbmsDataSet.Open method 509, 510
TADRdbmsDataSet.ParamBindMode property 513
TADRdbmsDataSet.Prepare method 511
TADRdbmsDataSet.Prepared property 505
TADRdbmsDataSet.ResourceOptions property 506
TADRdbmsDataSet.RowsAffected property 513
TADRdbmsDataSet.SchemaAdapter property 506
TADRdbmsDataSet.Transaction property 507
TADRdbmsDataSet.Unprepare method 512
TADRdbmsDataSet.UpdateOptions property 507
TADRdbmsDataSet.UpdateTransaction property 507
TADResourceOptions class 874
ActualSilentMode 876
ArrayDMLSize 879
Assign 878
AssignedValues 879
Backup 879
BackupExt 876
BackupFolder 876
CmdExecMode 880
CmdExecTimeout 881
DefaultParamType 881
DefaultStoreExt 877
DefaultStoreFolder 877
DefaultStoreFormat 877
DirectExecute 881

EscapeExpand 882
MacroCreate 882
MacroExpand 882
ParamCreate 883
ParamExpand 883
Persistent 884
PreprocessCmdText 878
RestoreDefaults 878
SilentMode 884
StoreItems 884
StoreVersion 885
UnifyParams 885
TADResourceOptions.ActualSilentMode property 876
TADResourceOptions.ArrayDMLSize property 879
TADResourceOptions.Assign method 878
TADResourceOptions.AssignedValues property 879
TADResourceOptions.Backup property 879
TADResourceOptions.BackupExt property 876
TADResourceOptions.BackupFolder property 876
TADResourceOptions.CmdExecMode property 880
TADResourceOptions.CmdExecTimeout property 881
TADResourceOptions.DefaultParamType property 881
TADResourceOptions.DefaultStoreExt property 877
TADResourceOptions.DefaultStoreFolder property 877
TADResourceOptions.DefaultStoreFormat property 877
TADResourceOptions.DirectExecute property 881
TADResourceOptions.EscapeExpand property 882
TADResourceOptions.MacroCreate property 882
TADResourceOptions.MacroExpand property 882
TADResourceOptions.ParamCreate property 883
TADResourceOptions.ParamExpand property 883
TADResourceOptions.Persistent property 884
TADResourceOptions.PreprocessCmdText property 878
TADResourceOptions.RestoreDefaults method 878
TADResourceOptions.SilentMode property 884
TADResourceOptions.StoreItems property 884
TADResourceOptions.StoreVersion property 885
TADResourceOptions.UnifyParams property 885
TADSchemaAdapter class 514
TADScript class 685
AbortJob 690
AfterExecute 695
AfterScript 695
AllSpools 687
Arguments 696
BeforeExecute 696
BeforeScript 697
CallStack 687
Connection 697
CurrentCommand 687
Eof 687
ExecuteAll 691
ExecuteFile 692
ExecuteScript 693
ExecuteStep 694
FetchOptions 697
Finished 688
FormatOptions 697
LastSpoolFileName 688
Macros 698
OnConsoleGet 698
OnConsoleLockUpdate 698
OnConsolePut 698
OnError 699
OnGetText 699
OnHostCommand 700
OnPause 700
OnProgress 700
OnReleaseText 701
OnSpoolPut 701
Params 701
Position 688
ProcessedAfterCommit 689
ResourceOptions 701
ScriptDialog 702
ScriptOptions 702
SQLScriptFileName 703
SQLScripts 703
Status 689
TotalErrors 689
TotalJobDone 690
TotalJobSize 690
TotalPct10Done 690
Transaction 704

ValidateAll 694
ValidateStep 695
TADScript.AbortJob method 690
TADScript.AfterExecute event 695
TADScript.AfterScript event 695
TADScript.AllSpools property 687
TADScript.Arguments property 696
TADScript.BeforeExecute event 696
TADScript.BeforeScript event 697
TADScript.CallStack property 687
TADScript.Connection property 697
TADScript.CurrentCommand property 687
TADScript.Eof property 687
TADScript.ExecuteAll method 691
TADScript.ExecuteFile method 692
TADScript.ExecuteScript method 693
TADScript.ExecuteStep method 694
TADScript.FetchOptions property 697
TADScript.Finished property 688
TADScript.FormatOptions property 697
TADScript.LastSpoolFileName property 688
TADScript.Macros property 698
TADScript.OnConsoleGet event 698
TADScript.OnConsoleLockUpdate event 698
TADScript.OnConsolePut event 698
TADScript.OnError event 699
TADScript.OnGetText event 699
TADScript.OnHostCommand event 700
TADScript.OnPause event 700
TADScript.OnProgress event 700
TADScript.OnReleaseText event 701
TADScript.OnSpoolPut event 701
TADScript.Params property 701
TADScript.Position property 688
TADScript.ProcessedAfterCommit property 689
TADScript.ResourceOptions property 701
TADScript.ScriptDialog property 702
TADScript.ScriptOptions property 702
TADScript.SQLScriptFileName property 703
TADScript.SQLScripts property 703
TADScript.Status property 689
TADScript.TotalErrors property 689
TADScript.TotalJobDone property 690
TADScript.TotalJobSize property 690
TADScript.TotalPct10Done property 690
TADScript.Transaction property 704
TADScript.ValidateAll method 694
TADScript.ValidateStep method 695
TADScriptCommand class 704
AbortJob 706
Engine 705
EngineIntf 705
Execute 706
Help 707
Keywords 707
Parse 707
Parser 706
Position 706
Validate 708
TADScriptCommand.AbortJob method 706
TADScriptCommand.Engine property 705
TADScriptCommand.EngineIntf property 705
TADScriptCommand.Execute method 706
TADScriptCommand.Help method 707
TADScriptCommand.Keywords method 707
TADScriptCommand.Parse method 707
TADScriptCommand.Parser property 706
TADScriptCommand.Position property 706
TADScriptCommand.Validate method 708
TADScriptCommandRegistry class 708
AddCommand 709
Commands 708
Count 709
LookupCommand 709
TADScriptCommandRegistry.AddCommand method 709
TADScriptCommandRegistry.Commands property 708
TADScriptCommandRegistry.Count property 709
TADScriptCommandRegistry.LookupCommand method 709
TADScriptOptions class 709
Assign 720
AutoPrintParams 711
BreakOnError 711
CharacterSet 711
ClientLib 711

ColumnHeadings 712
CommandSeparator 712
CommitEachNCommands 712
ConsoleOutput 713
DefaultDataPath 713
DefaultScriptPath 713
DriverID 713
DropNonexistObj 714
EchoCommands 714
EchoCommandTrim 714
FeedbackCommands 715
FeedbackScript 715
FileEncoding 715
FileEndOfLine 716
IgnoreError 716
LineSize 716
MacroExpand 716
MaxStringWidth 717
PageSize 717
ParamArraySize 717
RaisePLSQLErrors 717
Reset 720
SpoolFileName 718
SpoolOutput 718
SQLDialect 718
Timing 719
TrimConsole 719
TrimSpool 719
Verify 719
TADScriptOptions.Assign method 720
TADScriptOptions.AutoPrintParams property 711
TADScriptOptions.BreakOnError property 711
TADScriptOptions.CharacterSet property 711
TADScriptOptions.ClientLib property 711
TADScriptOptions.ColumnHeadings property 712
TADScriptOptions.CommandSeparator property 712
TADScriptOptions.CommitEachNCommands property 712
TADScriptOptions.ConsoleOutput property 713
TADScriptOptions.DefaultDataPath property 713
TADScriptOptions.DefaultScriptPath property 713
TADScriptOptions.DriverID property 713
TADScriptOptions.DropNonexistObj property 714
TADScriptOptions.EchoCommands property 714
TADScriptOptions.EchoCommandTrim property 714
TADScriptOptions.FeedbackCommands property 715
TADScriptOptions.FeedbackScript property 715
TADScriptOptions.FileEncoding property 715
TADScriptOptions.FileEndOfLine property 716
TADScriptOptions.IgnoreError property 716
TADScriptOptions.LineSize property 716
TADScriptOptions.MacroExpand property 716
TADScriptOptions.MaxStringWidth property 717
TADScriptOptions.PageSize property 717
TADScriptOptions.ParamArraySize property 717
TADScriptOptions.RaisePLSQLErrors property 717
TADScriptOptions.Reset method 720
TADScriptOptions.SpoolFileName property 718
TADScriptOptions.SpoolOutput property 718
TADScriptOptions.SQLDialect property 718
TADScriptOptions.Timing property 719
TADScriptOptions.TrimConsole property 719
TADScriptOptions.TrimSpool property 719
TADScriptOptions.Verify property 719
TADSQLiteBackup class 805
Backup 808
BusyTimeout 808
Catalog 808
Database 809
DatabaseObj 807
DestCatalog 809
DestDatabase 809
DestDatabaseObj 807
DestMode 809
DestPassword 809
OnProgress 810
PageCount 807
PagesPerStep 810
Password 810
Remaining 808
WaitForLocks 811
TADSQLiteBackup.Backup method 808
TADSQLiteBackup.BusyTimeout property 808
TADSQLiteBackup.Catalog property 808
TADSQLiteBackup.Database property 809

TADSQLiteBackup.DatabaseObj property 807
TADSQLiteBackup.DestCatalog property 809
TADSQLiteBackup.DestDatabase property 809
TADSQLiteBackup.DestDatabaseObj property 807
TADSQLiteBackup.DestMode property 809
TADSQLiteBackup.DestPassword property 809
TADSQLiteBackup.OnProgress event 810
TADSQLiteBackup.PageCount property 807
TADSQLiteBackup.PagesPerStep property 810
TADSQLiteBackup.Password property 810
TADSQLiteBackup.Remaining property 808
TADSQLiteBackup.WaitForLocks property 811
TADSQLiteCollation class 811
Active 812
CollationKind 812
CollationName 813
Flags 813
Locale 813
OnCompare 813
TADSQLiteCollation.Active property 812
TADSQLiteCollation.CollationKind property 812
TADSQLiteCollation.CollationName property 813
TADSQLiteCollation.Flags property 813
TADSQLiteCollation.Locale property 813
TADSQLiteCollation.OnCompare property 813
TADSQLiteFunction class 814
Active 815
Aggregated 815
ArgumentsCount 815
FunctionName 815
OnCalculate 816
TADSQLiteFunction.Active property 815
TADSQLiteFunction.Aggregated property 815
TADSQLiteFunction.ArgumentsCount property 815
TADSQLiteFunction.FunctionName property 815
TADSQLiteFunction.OnCalculate property 816
TADSQLiteRTree class 816
Active 816
OnCalculate 817
RTreeName 817
TADSQLiteRTree.Active property 816
TADSQLiteRTree.OnCalculate property 817
TADSQLiteRTree.RTreeName property 817
TADSQLiteSecurity class 817
ChangePassword 818
CheckEncryption 819
Database 820
Options 820
Password 820
RemovePassword 819
SetPassword 819
ToPassword 821
TADSQLiteSecurity.ChangePassword method 818
TADSQLiteSecurity.CheckEncryption method 819
TADSQLiteSecurity.Database property 820
TADSQLiteSecurity.Options property 820
TADSQLiteSecurity.Password property 820
TADSQLiteSecurity.RemovePassword method 819
TADSQLiteSecurity.SetPassword method 819
TADSQLiteSecurity.ToPassword property 821
TADSQLiteService class 821
DriverLink 821
TADSQLiteService.DriverLink property 821
TADSQLiteValidate class 821
Analyze 823
CheckOnly 823
Database 824
MaxErrors 824
OnProgress 824
Options 824
Password 824
Sweep 823
TADSQLiteValidate.Analyze method 823
TADSQLiteValidate.CheckOnly method 823
TADSQLiteValidate.Database property 824
TADSQLiteValidate.MaxErrors property 824
TADSQLiteValidate.OnProgress event 824
TADSQLiteValidate.Options property 824
TADSQLiteValidate.Password property 824
TADSQLiteValidate.Sweep method 823
TADSQLScript class 720
Name 720
SQL 721
TADSQLScript.Name property 720

TADSQLScript.SQL property 721
TADSQLScripts class 721
Add 722
FindScript 722
Items 721
TADSQLScripts.Add method 722
TADSQLScripts.FindScript method 722
TADSQLScripts.Items property 721
TADSQLTimeIntervalField class 665
AsSQLTimeInterval 666
IntervalKind 666
TADSQLTimeIntervalField.AsSQLTimeInterval property 666
TADSQLTimeIntervalField.IntervalKind property 666
TADStoredProc class 514
ActiveStoredUsage 516
AfterApplyUpdates 516
AfterExecute 517
AfterGetRecord 517
AfterGetRecords 517
AfterRowRequest 517
Aggregates 517
AggregatesActive 518
BeforeApplyUpdates 519
BeforeExecute 519
BeforeGetRecords 519
BeforeRowRequest 520
CachedUpdates 520
CatalogName 521
Connection 521
ConnectionName 521
Constraints 522
ConstraintsEnabled 523
DetailFields 523
FetchOptions 523
FilterChanges 524
FormatOptions 524
Indexes 524
IndexesActive 526
IndexFieldNames 526
IndexName 527
LocalSQL 527
MasterFields 527
MasterSource 528
OnCommandChanged 528
OnError 529
OnExecuteError 530
OnReconcileError 530
OnUpdateError 531
OnUpdateRecord 532
Overload 533
PackageName 534
ParamBindMode 534
Params 534
ResourceOptions 535
SchemaAdapter 536
SchemaName 536
StoredProcName 536
Transaction 537
UpdateObject 537
UpdateOptions 538
UpdateTransaction 538
TADStoredProc.ActiveStoredUsage property 516
TADStoredProc.AfterApplyUpdates event 516
TADStoredProc.AfterExecute event 517
TADStoredProc.AfterGetRecord event 517
TADStoredProc.AfterGetRecords event 517
TADStoredProc.AfterRowRequest event 517
TADStoredProc.Aggregates property 517
TADStoredProc.AggregatesActive property 518
TADStoredProc.BeforeApplyUpdates event 519
TADStoredProc.BeforeExecute event 519
TADStoredProc.BeforeGetRecords event 519
TADStoredProc.BeforeRowRequest event 520
TADStoredProc.CachedUpdates property 520
TADStoredProc.CatalogName property 521
TADStoredProc.Connection property 521
TADStoredProc.ConnectionName property 521
TADStoredProc.Constraints property 522
TADStoredProc.ConstraintsEnabled property 523
TADStoredProc.DetailFields property 523
TADStoredProc.FetchOptions property 523
TADStoredProc.FilterChanges property 524
TADStoredProc.FormatOptions property 524
TADStoredProc.Indexes property 524

TADStoredProc.IndexesActive property 526
TADStoredProc.IndexFieldNames property 526
TADStoredProc.IndexName property 527
TADStoredProc.LocalSQL property 527
TADStoredProc.MasterFields property 527
TADStoredProc.MasterSource property 528
TADStoredProc.OnCommandChanged event 528
TADStoredProc.OnError event 529
TADStoredProc.OnExecuteError event 530
TADStoredProc.OnReconcileError event 530
TADStoredProc.OnUpdateError event 531
TADStoredProc.OnUpdateRecord event 532
TADStoredProc.Overload property 533
TADStoredProc.PackageName property 534
TADStoredProc.ParamBindMode property 534
TADStoredProc.Params property 534
TADStoredProc.ResourceOptions property 535
TADStoredProc.SchemaAdapter property 536
TADStoredProc.SchemaName property 536
TADStoredProc.StoredProcName property 536
TADStoredProc.Transaction property 537
TADStoredProc.UpdateObject property 537
TADStoredProc.UpdateOptions property 538
TADStoredProc.UpdateTransaction property 538
TADTable class 539
ActiveStoredUsage 542
AfterApplyUpdates 543
AfterGetRecord 543
AfterGetRecords 543
AfterRowRequest 543
Aggregates 544
AggregatesActive 544
BeforeApplyUpdates 545
BeforeGetRecords 545
BeforeRowRequest 546
CachedUpdates 546
CatalogName 547
Connection 547
ConnectionName 547
Constraints 548
ConstraintsEnabled 548
DetailFields 549
Disconnect 541
Exists 541
FetchOptions 549
FilterChanges 550
FormatOptions 550
GetCustomWhere 542
IndexFieldNames 550
IndexName 551
LocalSQL 551
MasterFields 552
MasterSource 552
OnCommandChanged 552
OnError 553
OnExecuteError 554
OnReconcileError 555
OnUpdateError 555
OnUpdateRecord 556
Open 541
RefireSQL 542
ResourceOptions 557
SchemaAdapter 558
SchemaName 558
TableName 559
Transaction 559
UpdateObject 559
UpdateOptions 560
UpdateTransaction 560
TADTable Questions 234
TADTable.ActiveStoredUsage property 542
TADTable.AfterApplyUpdates event 543
TADTable.AfterGetRecord event 543
TADTable.AfterGetRecords event 543
TADTable.AfterRowRequest event 543
TADTable.Aggregates property 544
TADTable.AggregatesActive property 544
TADTable.BeforeApplyUpdates event 545
TADTable.BeforeGetRecords event 545
TADTable.BeforeRowRequest event 546
TADTable.CachedUpdates property 546
TADTable.CatalogName property 547
TADTable.Connection property 547
TADTable.ConnectionName property 547

TADTable.Constraints property 548
TADTable.ConstraintsEnabled property 548
TADTable.DetailFields property 549
TADTable.Disconnect method 541
TADTable.Exists property 541
TADTable.FetchOptions property 549
TADTable.FilterChanges property 550
TADTable.FormatOptions property 550
TADTable.GetCustomWhere method 542
TADTable.IndexFieldNames property 550
TADTable.IndexName property 551
TADTable.LocalSQL property 551
TADTable.MasterFields property 552
TADTable.MasterSource property 552
TADTable.OnCommandChanged event 552
TADTable.OnError event 553
TADTable.OnExecuteError event 554
TADTable.OnReconcileError event 555
TADTable.OnUpdateError event 555
TADTable.OnUpdateRecord event 556
TADTable.Open method 541
TADTable.RefireSQL method 542
TADTable.ResourceOptions property 557
TADTable.SchemaAdapter property 558
TADTable.SchemaName property 558
TADTable.TableName property 559
TADTable.Transaction property 559
TADTable.UpdateObject property 559
TADTable.UpdateOptions property 560
TADTable.UpdateTransaction property 560
TADTopResourceOptions class 885
Assign 886
AutoConnect 887
AutoReconnect 887
BackupExt 888
BackupFolder 888
DefaultStoreExt 888
DefaultStoreFolder 889
DefaultStoreFormat 889
KeepConnection 889
MaxCursors 890
RestoreDefaults 887
ServerOutput 890
ServerOutputSize 891
TADTopResourceOptions.Assign method 886
TADTopResourceOptions.AutoConnect property 887
TADTopResourceOptions.AutoReconnect property 887
TADTopResourceOptions.BackupExt property 888
TADTopResourceOptions.BackupFolder property 888
TADTopResourceOptions.DefaultStoreExt property 888
TADTopResourceOptions.DefaultStoreFolder property 889
TADTopResourceOptions.DefaultStoreFormat property 889
TADTopResourceOptions.KeepConnection property 889
TADTopResourceOptions.MaxCursors property 890
TADTopResourceOptions.RestoreDefaults method 887
TADTopResourceOptions.ServerOutput property 890
TADTopResourceOptions.ServerOutputSize property 891
TADTransaction class 561
AfterCommit 561
AfterRollback 562
AfterStartTransaction 562
BeforeCommit 562
BeforeRollback 562
BeforeStartTransaction 563
Connection 563
Options 563
TADTransaction.AfterCommit event 561
TADTransaction.AfterRollback event 562
TADTransaction.AfterStartTransaction event 562
TADTransaction.BeforeCommit event 562
TADTransaction.BeforeRollback event 562
TADTransaction.BeforeStartTransaction event 563
TADTransaction.Connection property 563
TADTransaction.Options property 563
TADTxOptions class 891
AutoCommit 893
AutoStart 893
AutoStop 893
Changed 892
ClearChanged 892
DisconnectAction 894
EnableNested 894
Isolation 894
Params 895

ReadOnly 895
StopOptions 895
TADTxOptions.AutoCommit property 893
TADTxOptions.AutoStart property 893
TADTxOptions.AutoStop property 893
TADTxOptions.Changed property 892
TADTxOptions.ClearChanged method 892
TADTxOptions.DisconnectAction property 894
TADTxOptions.EnableNested property 894
TADTxOptions.Isolation property 894
TADTxOptions.Params property 895
TADTxOptions.ReadOnly property 895
TADTxOptions.StopOptions property 895
TADUpdateOptions class 896
Assign 897
AssignedValues 898
CheckReadOnly 898
CheckRequired 898
CheckUpdatable 899
CountUpdatedRecords 899
EnableDelete 899
EnableInsert 900
EnableUpdate 900
FastUpdates 900
FetchGeneratorsPoint 901
GeneratorName 901
LockMode 902
LockPoint 902
LockWait 902
ReadOnly 903
RefreshDelete 903
RefreshMode 903
RequestLive 904
RestoreDefaults 897
UpdateChangedFields 904
UpdateMode 904
UpdateNonBaseFields 905
TADUpdateOptions.Assign method 897
TADUpdateOptions.AssignedValues property 898
TADUpdateOptions.CheckReadOnly property 898
TADUpdateOptions.CheckRequired property 898
TADUpdateOptions.CheckUpdatable property 899
TADUpdateOptions.CountUpdatedRecords property 899
TADUpdateOptions.EnableDelete property 899
TADUpdateOptions.EnableInsert property 900
TADUpdateOptions.EnableUpdate property 900
TADUpdateOptions.FastUpdates property 900
TADUpdateOptions.FetchGeneratorsPoint property 901
TADUpdateOptions.GeneratorName property 901
TADUpdateOptions.LockMode property 902
TADUpdateOptions.LockPoint property 902
TADUpdateOptions.LockWait property 902
TADUpdateOptions.ReadOnly property 903
TADUpdateOptions.RefreshDelete property 903
TADUpdateOptions.RefreshMode property 903
TADUpdateOptions.RequestLive property 904
TADUpdateOptions.RestoreDefaults method 897
TADUpdateOptions.UpdateChangedFields property 904
TADUpdateOptions.UpdateMode property 904
TADUpdateOptions.UpdateNonBaseFields property 905
TADUpdateSQL class 563
Apply 565
Commands 564
Connection 566
ConnectionName 566
DeleteSQL 567
FetchRowSQL 567
InsertSQL 568
LockSQL 568
ModifySQL 569
SQL 565
UnlockSQL 569
TADUpdateSQL.Apply method 565
TADUpdateSQL.Commands property 564
TADUpdateSQL.Connection property 566
TADUpdateSQL.ConnectionName property 566
TADUpdateSQL.DeleteSQL property 567
TADUpdateSQL.FetchRowSQL property 567
TADUpdateSQL.InsertSQL property 568
TADUpdateSQL.LockSQL property 568
TADUpdateSQL.ModifySQL property 569
TADUpdateSQL.SQL property 565
TADUpdateSQL.UnlockSQL property 569
TADWideMemoField class 666

Value 667
TADWideMemoField.Value property 667
TADXMLField class 667
SchemaName 668
TADXMLField.SchemaName property 668
Tracing and Monitoring 174

U

uADCompClient namespace 257
uADCompClient.ADManager function 570
uADCompClient.ADSetConnectionClass function 570
uADCompClient.ADSetManagerClass function 571
uADCompDataSet namespace 571
uADCompGUIx namespace 668
uADCompScript namespace 684
uADMoniRemoteClient namespace 722
uADPhysADS namespace 724
uADPhysASA namespace 740
uADPhysDataSnap namespace 749
uADPhysDB2 namespace 750
uADPhysDBExp namespace 751
uADPhysIB namespace 752
uADPhysManager namespace 778
uADPhysMSAcc namespace 784
uADPhysMSSQL namespace 791
uADPhysMySQL namespace 794
uADPhysODBC namespace 796
uADPhysODBCBase namespace 797
uADPhysOracle namespace 800
uADPhysPG namespace 803
uADPhysSQLite namespace 804
uADPhysSQLiteVDataSet namespace 825
uADPhysTDBX namespace 829
uADPhysTDBXBase namespace 830
uADStanError namespace 831
uADStanOption namespace 840
uADStanOption.TADActionRequest enumeration 905
uADStanOption.TADAutoFetchAll enumeration 906
Unicode Support 51
Unique Identifying Fields 117
UnixODBC 160
Update Command Generation 121
Using Data Abstract with FireDAC 153
Using FireDAC 159
Using Oracle with FireDAC 149
Using SQLite with FireDAC 135
Utilities 178

V

Very High Performance using the Array DML 8

W

Working with Commands 56
Working with Connections 27
Working with DataSets 100
Working with DBMS 135
Working with Metadata 127
Writing Expressions 110

X

x Platform Development 158